

LeetMastery

Study-Order Edition

Python Only · Priority-Grouped ·

Difficulty-First · 1×1 Portrait

331 questions · 9 rounds · Inline Quick Review
· Chapter 2 Summary
High Easy → High Med → High Hard → Mid Easy
→ Mid Med → Mid Hard → Low Easy → Low Med
→ Low Hard

Contents

★ = LeetCode Premium (Premium 98 list)

Round 1 | High | E Easy (43) ↗

Arrays & Hashing (6) ↗

☐ ● #1 Two Sum ↗

☐ ● ★ #163 Missing Ranges ↗

☐ ● ★ #346 Moving Average from Data Stream ↗

☐ ● ★ #760 Find Anagram Mappings ↗

☐ ● ★ #1426 Counting Elements ↗

☐ ● #1534 Count Good Triplets ↗

String (3) ↗

☐ ● #383 Ransom Note ↗

☐ ● ★ #734 Sentence Similarity ↗

☐ ● ★ #1165 Single Row Keyboard ↗

Two Pointers (5) ↗

☐ ● #88 Merge Sorted Array ↗

☐ ● #125 Valid Palindrome ↗

☐ ● #283 Move Zeroes ↗

☐ ● ★ #408 Valid Word Abbreviation ↗

☐ ● #977 Squares of a Sorted Array ↗

Sorting (6) ↗

☐ ● #169 Majority Element ↗

☐ ● #217 Contains Duplicate ↗

☐ ● #242 Valid Anagram ↗

☐ ● ★ #252 Meeting Rooms ↗

☐ ● ★ #1086 Largest Unique Number ↗

☐ ● #1122 Relative Sort Array ↗

Binary Search (5) ↗

☐ ● #35 Search Insert Position ↗

☐ ● #69 Sqrt(x) ↗

☐ ● #278 First Bad Version ↗

☐ ● #374 Guess Number Higher or Lower ↗

☐ ● #704 Binary Search ↗

Matrix (5) ↗

☐ ● ★ #422 Valid Word Square ↗

☐ ● #566 Reshape the Matrix ↗

- ☐ ● #766 Toeplitz Matrix ↗
- ☐ ● #867 Transpose Matrix ↗
- ☐ ● #2373 Largest Local Values in a Matrix ↗
- Trees & BST (11)** ↗
- ☐ ● #100 Same Tree ↗
- ☐ ● #101 Symmetric Tree ↗
- ☐ ● #104 Maximum Depth of Binary Tree ↗
- ☐ ● #108 Convert Sorted Array to Binary Search Tree ↗
- ☐ ● #110 Balanced Binary Tree ↗
- ☐ ● #112 Path Sum ↗
- ☐ ● #226 Invert Binary Tree ↗
- ☐ ● ★ #270 Closest Binary Search Tree Value ↗
- ☐ ● #501 Find Mode in Binary Search Tree ↗
- ☐ ● #543 Diameter of Binary Tree ↗
- ☐ ● #572 Subtree of Another Tree ↗
- DFS (2)** ↗
- ☐ ● #463 Island Perimeter ↗

☐ ● #733 Flood Fill ↗

Round 2 | High | M Medium (116) ↗

Arrays & Hashing (11) ↗

☐ ● #57 Insert Interval ↗

☐ ● #238 Product of Array Except Self ↗

☐ ● #281 Zigzag Iterator ↗

☐ ● #307 Range Sum Query – Mutable ↗

☐ ● #454 4Sum II ↗

☐ ● #525 Contiguous Array ↗

☐ ● #560 Subarray Sum Equals K ↗

☐ ● #1010 Pairs of Songs With Total Durations
Divisible by 60 ↗

☐ ● ★ #1272 Remove Interval ↗

☐ ● ★ #1429 First Unique Number ↗

☐ ● #3159 Find Occurrences of an Element in
an Array ↗

String (5) ↗

☐ ● #8 String to Integer (atoi) ↗

☐ ● ★ #249 Group Shifted Strings ↗

☐ ● ★ #271 Encode and Decode Strings ↗

☐ ● #635 Design Log Storage System ↗

☐ ● #1138 Alphabet Board Path ↗

Two Pointers (14) ↗

☐ ● #11 Container With Most Water ↗

☐ ● #15 3Sum ↗

☐ ● #16 3Sum Closest ↗

☐ ● #31 Next Permutation ↗

☐ ● #75 Sort Colors ↗

☐ ● ★ #161 One Edit Distance ↗

☐ ● #167 Two Sum II – Input Array Is Sorted ↗

☐ ● ★ #186 Reverse Words in a String II ↗

☐ ● #189 Rotate Array ↗

☐ ● #532 K-diff Pairs in an Array ↗

☐ ● #923 3Sum With Multiplicity ↗

☐ ● #986 Interval List Intersections ↗

☐ ● ★ #1055 Shortest Way to Form String ↗

☐ ● #2563 Count the Number of Fair Pairs ↗

Sliding Window (8) ↗

- ☐ ● #3 Longest Substring Without Repeating ↗ Characters
- ☐ ● ★ #159 Longest Substring with At Most ↗ Two Distinct Characters
- ☐ ● ★ #340 Longest Substring with At Most K ↗ Distinct Characters
- ☐ ● #424 Longest Repeating Character ↗ Replacement
- ☐ ● #438 Find All Anagrams in a String ↗
- ☐ ● ★ #487 Max Consecutive Ones II ↗
- ☐ ● ★ #1100 Find K-Length Substrings with No ↗ Repeated Characters
- ☐ ● #1248 Count Number of Nice Subarrays ↗

Sorting (3) ↗

- ☐ ● #49 Group Anagrams ↗
- ☐ ● #56 Merge Intervals ↗
- ☐ ● ★ #1152 Analyze User Website Visit Pattern ↗

Binary Search (12) ↗

- ☐ ● #33 Search in Rotated Sorted Array ↗

- ☐ ● #34 Find First and Last Position of Element in Sorted Array ↗
- ☐ ● #153 Find Minimum in Rotated Sorted Array ↗
- ☐ ● #300 Longest Increasing Subsequence ↗
- ☐ ● #362 Design Hit Counter ↗
- ☐ ● #528 Random Pick with Weight ↗
- ☐ ● #852 Peak Index in a Mountain Array ↗
- ☐ ● #981 Time Based Key-Value Store ↗
- ☐ ● #1011 Capacity to Ship Packages Within D Days ↗
- ☐ ● ★ #1060 Missing Element in Sorted Array ↗
- ☐ ● #1062 Longest Repeating Substring ↗
- ☐ ● ★ #1533 Find the Index of the Large Integer ↗
- Matrix (14)** ↗
- ☐ ● #36 Valid Sudoku ↗
- ☐ ● #48 Rotate Image ↗
- ☐ ● #54 Spiral Matrix ↗
- ☐ ● #59 Spiral Matrix II ↗

- ☐ ● #64 Minimum Path Sum ↗
- ☐ ● #73 Set Matrix Zeroes ↗
- ☐ ● #74 Search a 2D Matrix ↗
- ☐ ● #221 Maximal Square ↗
- ☐ ● ★ #311 Sparse Matrix Multiplication ↗
- ☐ ● #498 Diagonal Traverse ↗
- ☐ ● ★ #531 Lonely Pixel I ↗
- ☐ ● ★ #723 Candy Crush ↗
- ☐ ● #835 Image Overlap ↗
- ☐ ● ★ #1198 Find Smallest Common Element in All Rows ↗

Trees & BST (24) ↗

- ☐ ● #98 Validate Binary Search Tree ↗
- ☐ ● #102 Binary Tree Level Order Traversal ↗
- ☐ ● #103 Binary Tree Zigzag Level Order Traversal ↗
- ☐ ● #105 Construct Binary Tree from Preorder and Inorder Traversal ↗
- ☐ ● #199 Binary Tree Right Side View ↗

- ☐ ● #230 Kth Smallest Element in a BST ↗
- ☐ ● #235 Lowest Common Ancestor of a Binary Search Tree ↗
- ☐ ● #236 Lowest Common Ancestor of a Binary Tree ↗
- ☐ ● ★ #250 Count Univalue Subtrees ↗
- ☐ ● #285 Inorder Successor in BST ↗
- ☐ ● ★ #298 Binary Tree Longest Consecutive Sequence ↗
- ☐ ● ★ #314 Binary Tree Vertical Order Traversal ↗
- ☐ ● ★ #333 Largest BST Subtree ↗
- ☐ ● ★ #366 Find Leaves of Binary Tree ↗
- ☐ ● #437 Path Sum III ↗
- ☐ ● ★ #545 Boundary of Binary Tree ↗
- ☐ ● ★ #549 Binary Tree Longest Consecutive Sequence II ↗
- ☐ ● ★ #582 Kill Process ↗
- ☐ ● #662 Maximum Width of Binary Tree ↗

- ☐ ● #863 All Nodes Distance K in Binary Tree ↗
- ☐ ● ★ #1120 Maximum Average Subtree ↗
- ☐ ● ★ #1490 Clone N-ary Tree ↗
- ☐ ● ★ #1522 Diameter of N-Ary Tree ↗
- ☐ ● ★ #1650 Lowest Common Ancestor of a Binary Tree III ↗

DFS (16) ↗

- ☐ ● #130 Surrounded Regions ↗
- ☐ ● #133 Clone Graph ↗
- ☐ ● #200 Number of Islands ↗
- ☐ ● #207 Course Schedule ↗
- ☐ ● #210 Course Schedule II ↗
- ☐ ● ★ #261 Graph Valid Tree ↗
- ☐ ● #310 Minimum Height Trees ↗
- ☐ ● ★ #323 Number of Connected Components in an Undirected Graph ↗
- ☐ ● #417 Pacific Atlantic Water Flow ↗
- ☐ ● ★ #490 The Maze ↗

☐ ● ★ #694 Number of Distinct Islands ↗

☐ ● #695 Max Area of Island ↗

☐ ● #721 Accounts Merge ↗

☐ ● #785 Is Graph Bipartite? ↗

☐ ● ★ #1236 Web Crawler ↗

☐ ● #1242 Web Crawler Multithreaded ↗

Graphs (3) ↗

☐ ● #128 Longest Consecutive Sequence ↗

☐ ● ★ #277 Find the Celebrity ↗

☐ ● ★ #1136 Parallel Courses ↗

BFS (6) ↗

☐ ● ★ #286 Walls and Gates ↗

☐ ● #322 Coin Change ↗

☐ ● #542 01 Matrix ↗

☐ ● #994 Rotting Oranges ↗

☐ ● ★ #1197 Minimum Knight Moves ↗

☐ ● #1730 Shortest Path to Get Food ↗

Round 3 | High | H Hard (23) ↗

Arrays & Hashing (1) ↗

☐ ● #41 First Missing Positive ↗

Sliding Window (1) ↗

☐ ● #76 Minimum Window Substring ↗

Binary Search (7) ↗

☐ ● #4 Median of Two Sorted Arrays ↗

☐ ● #315 Count of Smaller Numbers After Self ↗

☐ ● #410 Split Array Largest Sum ↗

☐ ● #668 Kth Smallest Number in
Multiplication Table ↗

☐ ● #710 Random Pick with Blacklist ↗

☐ ● #774 Minimize Max Distance to Gas Station ↗

☐ ● #1235 Maximum Profit in Job Scheduling ↗

Matrix (1) ↗

☐ ● #1074 Number of Submatrices That Sum to
Target ↗

Trees & BST (2) ↗

☐ ● #124 Binary Tree Maximum Path Sum ↗

☐ ● #297 Serialize and Deserialize Binary Tree ↗

DFS (5) ↗

- ☐ ● ★ #269 Alien Dictionary ↗
- ☐ ● #329 Longest Increasing Path in a Matrix ↗
- ☐ ● #685 Redundant Connection II ↗
- ☐ ● #1036 Escape a Large Maze ↗
- ☐ ● #1192 Critical Connections in a Network ↗

Graphs (2) ↗

- ☐ ● ★ #305 Number of Islands II ↗
- ☐ ● #1153 String Transforms Into Another String ↗

BFS (4) ↗

- ☐ ● #127 Word Ladder ↗
- ☐ ● ★ #317 Shortest Distance from All Buildings ↗
- ☐ ● #773 Sliding Puzzle ↗
- ☐ ● #815 Bus Routes ↗

Round 4 | Mid | E Easy (12) ↗

Linked List (7) ↗

- ☐ ● #21 Merge Two Sorted Lists ↗

- ☐ ● #141 Linked List Cycle ↗
- ☐ ● #206 Reverse Linked List ↗
- ☐ ● #234 Palindrome Linked List ↗
- ☐ ● #706 Design HashMap ↗
- ☐ ● #876 Middle of the Linked List ↗
- ☐ ● ★ #1474 Delete N Nodes After M Nodes of Linked List ↗

Stack (3) ↗

- ☐ ● #20 Valid Parentheses ↗
- ☐ ● #232 Implement Queue using Stacks ↗
- ☐ ● #844 Backspace String Compare ↗

Trie (1) ↗

- ☐ ● #14 Longest Common Prefix ↗

Greedy (1) ↗

- ☐ ● #409 Longest Palindrome ↗

Round 5 | Mid | M Medium (56) ↗

Linked List (13) ↗

- ☐ ● #2 Add Two Numbers ↗
- ☐ ● #19 Remove Nth Node From End of List ↗

- ☐ ● #24 Swap Nodes in Pairs ↗
- ☐ ● #61 Rotate List ↗
- ☐ ● #146 LRU Cache ↗
- ☐ ● #148 Sort List ↗
- ☐ ● #328 Odd Even Linked List ↗
- ☐ ● ★ #369 Plus One Linked List ↗
- ☐ ● ★ #430 Flatten a Multilevel Doubly Linked List ↗
- ☐ ● #622 Design Circular Queue ↗
- ☐ ● #707 Design Linked List ↗
- ☐ ● ★ #708 Insert into a Sorted Circular Linked List ↗
- ☐ ● #1171 Remove Zero Sum Consecutive Nodes from Linked List ↗

Stack (14) ↗

- ☐ ● #143 Reorder List ↗
- ☐ ● #150 Evaluate Reverse Polish Notation ↗
- ☐ ● #155 Min Stack ↗
- ☐ ● #173 Binary Search Tree Iterator ↗

- ☐ ● #227 Basic Calculator II ↗
- ☐ ● ★ #255 Verify Preorder Sequence in Binary Search Tree ↗
- ☐ ● #394 Decode String ↗
- ☐ ● ★ #439 Ternary Expression Parser ↗
- ☐ ● ★ #484 Find Permutation ↗
- ☐ ● #735 Asteroid Collision ↗
- ☐ ● #739 Daily Temperatures ↗
- ☐ ● #962 Maximum Width Ramp ↗
- ☐ ● ★ #1214 Two Sum BSTs ↗
- ☐ ● ★ #1762 Buildings With an Ocean View ↗
- ☐ ● #215 Kth Largest Element in an Array ↗
- ☐ ● ★ #253 Meeting Rooms II ↗
- ☐ ● #347 Top K Frequent Elements ↗
- ☐ ● ★ #505 The Maze II ↗
- ☐ ● #621 Task Scheduler ↗
- ☐ ● #658 Find K Closest Elements ↗

Heap (11) ↗

- ☐ **● #787 Cheapest Flights Within K Stops ↗**
- ☐ **● #973 K Closest Points to Origin ↗**
- ☐ **● ★ #1057 Campus Bikes ↗**
- ☐ **● ★ #1167 Minimum Cost to Connect Sticks ↗**
- ☐ **● #1424 Diagonal Traverse II ↗**

Trie (7) ↗

- ☐ **● #139 Word Break ↗**
- ☐ **● #208 Implement Trie (Prefix Tree) ↗**
- ☐ **● #211 Design Add and Search Words Data Structure ↗**
- ☐ **● ★ #616 Add Bold Tag in String ↗**
- ☐ **● #692 Top K Frequent Words ↗**
- ☐ **● #720 Longest Word in Dictionary ↗**
- ☐ **● #1166 Design File System ↗**

Backtracking (6) ↗

- ☐ **● #17 Letter Combinations of a Phone Number**
- ☐ **● #22 Generate Parentheses ↗**
- ☐ **● #39 Combination Sum ↗**

☐ ● #46 Permutations ↗

☐ ● #79 Word Search ↗

☐ ● #113 Path Sum II ↗

Greedy (5) ↗

☐ ● #134 Gas Station ↗

☐ ● #179 Largest Number ↗

☐ ● ★ #280 Wiggle Sort ↗

☐ ● #954 Array of Doubled Pairs ↗

☐ ● #2131 Longest Palindrome by
Concatenating Two Letter Words

Round 6 | Mid | H Hard (28) ↗

Linked List (3) ↗

☐ ● #25 Reverse Nodes in k-Group ↗

☐ ● #432 All O'one Data Structure ↗

☐ ● #460 LFU Cache ↗

Stack (8) ↗

☐ ● #32 Longest Valid Parentheses ↗

☐ ● #42 Trapping Rain Water ↗

☐ ● #84 Largest Rectangle in Histogram ↗

☐ ● #224 Basic Calculator ↗

☐ ● #239 Sliding Window Maximum ↗

☐ ● #716 Max Stack ↗

☐ ● ★ #772 Basic Calculator III ↗

☐ ● #895 Maximum Frequency Stack ↗

Heap (9) ↗

☐ ● #23 Merge k Sorted Lists ↗

☐ ● #218 The Skyline Problem ↗

☐ ● ★ #272 Closest Binary Search Tree Value II ↗

☐ ● #295 Find Median from Data Stream ↗

☐ ● ★ #358 Rearrange String k Distance Apart ↗

☐ ● ★ #499 The Maze III ↗

☐ ● #632 Smallest Range Covering Elements
from K Lists ↗

☐ ● #759 Employee Free Time ↗

☐ ● #1425 Constrained Subsequence Sum ↗

Trie (4) ↗

- ☐ ● #212 Word Search II ↗
- ☐ ● #336 Palindrome Pairs ↗
- ☐ ● ★ #588 Design In-Memory File System ↗
- ☐ ● ★ #642 Design Search Autocomplete System ↗

Backtracking (4) ↗

- ☐ ● #37 Sudoku Solver ↗
- ☐ ● #51 N-Queens ↗
- ☐ ● #126 Word Ladder II ↗
- ☐ ● #996 Number of Squareful Arrays ↗

Round 7 | Low | E Easy (14) ↗

Dynamic Programming (2) ↗

- ☐ ● #70 Climbing Stairs ↗
- ☐ ● #121 Best Time to Buy and Sell Stock ↗

Bit Manipulation (6) ↗

- ☐ ● #67 Add Binary ↗
- ☐ ● #136 Single Number ↗
- ☐ ● #190 Reverse Bits ↗

☐ ● #191 Number of 1 Bits ↗

☐ ● #268 Missing Number ↗

☐ ● #338 Counting Bits ↗

Math (6) ↗

☐ ● #9 Palindrome Number ↗

☐ ● #13 Roman to Integer ↗

☐ ● #504 Base 7 ↗

☐ ● ★ #1056 Confusing Number ↗

☐ ● ★ #1228 Missing Number in Arithmetic Progression ↗

☐ ● ★ #1427 Perform String Shifts ↗

Round 8 | Low | M Medium (32) ↗

Dynamic Programming (16) ↗

☐ ● #5 Longest Palindromic Substring ↗

☐ ● #53 Maximum Subarray ↗

☐ ● #55 Jump Game ↗

☐ ● #62 Unique Paths ↗

☐ ● #72 Edit Distance ↗

- ☐ ● #91 Decode Ways ↗
- ☐ ● #152 Maximum Product Subarray ↗
- ☐ ● #198 House Robber ↗
- ☐ ● ★ #256 Paint House ↗
- ☐ ● #309 Best Time to Buy and Sell Stock with ↗
Cooldown
- ☐ ● #377 Combination Sum IV ↗
- ☐ ● #416 Partition Equal Subset Sum ↗
- ☐ ● #435 Non-overlapping Intervals ↗
- ☐ ● #516 Longest Palindromic Subsequence ↗
- ☐ ● #576 Out of Boundary Paths ↗
- ☐ ● #1143 Longest Common Subsequence ↗

Bit Manipulation (4) ↗

- ☐ ● #78 Subsets ↗
- ☐ ● #90 Subsets II ↗
- ☐ ● #287 Find the Duplicate Number ↗
- ☐ ● ★ #1506 Find Root of N-Ary Tree ↗

Math (5) ↗

- ☐ **● #7 Reverse Integer** ↗
- ☐ **● #50 Pow(x, n)** ↗
- ☐ **● #380 Insert Delete GetRandom O(1)** ↗
- ☐ **● #1017 Convert to Base -2** ↗
- ☐ **● #3160 Find the Number of Distinct Colors Among the Balls** ↗

JavaScript (7) ↗

- ☐ **● ★ #2627 Debounce** ↗
- ☐ **● ★ #2628 JSON Deep Equal** ↗
- ☐ **● ★ #2630 Memoize** ↗
- ☐ **● ★ #2633 Convert Object to JSON String** ↗
- ☐ **● ★ #2636 Promise Pool** ↗
- ☐ **● ★ #2675 Array of Objects to Matrix** ↗
- ☐ **● ★ #2700 Differences Between Two Objects** ↗

Round 9 | Low | H Hard (7) ↗

Dynamic Programming (6) ↗

- ☐ **● #10 Regular Expression Matching** ↗
- ☐ **● #115 Distinct Subsequences** ↗

☐ ● #123 Best Time to Buy and Sell Stock III ↗

☐ ● #132 Palindrome Partitioning II ↗

☐ ● #312 Burst Balloons ↗

☐ ● #1000 Minimum Cost to Merge Stones ↗

Math (1) ↗

☐ ● #68 Text Justification ↗

Round 1 · High · Easy

Round 1

High Priority · Easy

43 questions · 8 patterns

**Arrays & Hashing #1 #163 #346 #760 #1426
#1534**

String #383 #734 #1165

Two Pointers #88 #125 #283 #408 #977

Sorting #169 #217 #242 #252 #1086 #1122

Binary Search #35 #69 #278 #374 #704

Matrix #422 #566 #766 #867 #2373

**Trees & BST #100 #101 #104 #108 #110 #112
#226 #270 #501 #543 #572**

DFS #463 #733

Arrays & Hashing

Round 1 · High · Easy · 6 questions

Arrays & Hashing

□ #1 Two Sum

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table

Lists: Grind 169

Problem

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`
Output: `[0,1]`
Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`
Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`
Output: `[0,1]`

Constraints:

- $2 \leq \text{nums.length} \leq 104$
- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{target} \leq 109$
- Only one valid answer exists.

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

My LeetCode Solution

• My LeetCode Solution

```
# Two Sum
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        hmap = {}
        for i, n in enumerate(nums):
            x = target - n
            if x in hmap:
                return [hmap[x], i]

            hmap[n] = i

        return []
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# We're given an array of integers and a target value,
# and we need to return the indices of the two numbers
# that add up to the target – not the values, the indices. Right?"
#
# "A few quick questions:
# Can the same element be used twice? I'm guessing no.
# Is there always exactly one valid answer?"
```

```

# Can the array be empty or have only one element?
# Are there negative numbers or duplicates?"
#
# "Let me walk the example: nums = [2, 7, 11, 15], target = 9.
# 2 + 7 = 9, so we return [0, 1]. Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For every pair (i, j) where j > i, check if nums[i] + nums[j] equals target.
# Time: O(n2) from the nested loops. Space: O(1), no extra storage.
# Should I go ahead and optimize?"
class _1_TwoSum_Brute:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] + nums[j] == target:
                    return [i, j]
        return []

# PHASE 3 — OPTIMIZE
# "The bottleneck is searching the rest of the array for a complement each time.
# I notice we're repeatedly asking: does the number I need exist?
# A hash map answers that in O(1).
#
# Walk forward through the array. For each number, compute complement = target -
# num.
# If complement is already in the map, return both indices.
# Otherwise, store the current number and index and keep going.
#
# Pseudocode:
# seen = {}
# for index, num in nums:
#     complement = target - num
#     if complement in seen: return [seen[complement], index]
#     seen[num] = index
#
# Time: O(n). Space: O(n). Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _1_TwoSum:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        seen = {} # maps value -> index
        for index, num in enumerate(nums):
            complement = target - num
            if complement in seen:
                return [seen[complement], index]
            seen[num] = index
        return []

```

PHASE 5 — TEST & DEBUG**# "Let me trace through a few cases."**

#

nums=[2,7,11,15], target=9

index=0: complement=7, not in seen. seen={2:0}

index=1: complement=2, found at 0. return [0,1] ✓

#

nums=[3,2,4], target=6

index=0: complement=3, not in seen. seen={3:0}

index=1: complement=4, not in seen. seen={3:0,2:1}

index=2: complement=2, found at 1. return [1,2] ✓

#

nums=[3,3], target=6

index=0: complement=3, not in seen. seen={3:0}

index=1: complement=3, found at 0. return [0,1] ✓

#

"No bugs. Holds across all cases."**# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "Time $O(n)$ — one pass. Space $O(n)$ — worst case every element enters the map.****# Trade-off: $O(1)$ space would require $O(n^2)$ time — no free lunch here.****# If the array were sorted I'd use two pointers: $O(n)$ time, $O(1)$ space.****# But we need original indices and the input is unsorted, so hash map wins.****# Given more time I'd ask: what if there are multiple valid pairs?****# We'd collect all of them rather than returning on the first hit."**

◆ Quick Review

Key Insights

- Use a hash table to store the elements and their indices for fast lookup.
- For each element, calculate its complement: target minus the current element.
- Check if the complement exists in the hash table.
- If found, return the indices of the current element and its complement.

- This approach only requires a single pass through the array.

Space & Time

Time Complexity: $O(n)$ – We traverse the list only once.

Space Complexity: $O(n)$ – In the worst-case, we store all elements in the hash table.

Solution

We use a hash table (dictionary) to map each array element to its index as we iterate. For each number, we compute the complement (target – number) and check if it already exists in our hash table. If the complement is found, we return the indices. This one-pass solution ensures linear time complexity and avoids the need for nested loops.

Arrays & Hashing

★ #163 Missing Ranges ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: ★ Premium | Premium 98

Problem

You are given an inclusive range [lower, upper] and a sorted unique integer array nums, where all elements are within the inclusive range.

A number x is considered missing if x is in the range [lower, upper] and x is not in nums.

Return the shortest sorted list of ranges that exactly covers all the missing numbers. That is, no element of nums is included in any of the ranges, and each missing number is covered by one of the ranges.

Example 1:

Input: nums = [0,1,3,50,75], lower = 0, upper = 99

Output: [[2,2],[4,49],[51,74],[76,99]]

Explanation: The ranges are:

[2,2]

[4,49]

[51,74]

[76,99]

Example 2:

Input: nums = [-1], lower = -1, upper = -1

Output: []

Explanation: There are no missing ranges since there are no missing numbers.

Constraints:

- $-109 \leq \text{lower} \leq \text{upper} \leq 109$
- $0 \leq \text{nums.length} \leq 100$
- $\text{lower} \leq \text{nums}[i] \leq \text{upper}$
- All the values of nums are unique.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findMissingRanges(self, nums: List[int], lower: int, upper: int) -> List[List[int]]:
        if not nums:
            return [[lower, upper]]

        ans = []
        first, last = nums[0], nums[-1]
        if lower < first:
            ans.append([lower, first-1])

        for prev, curr in pairwise(nums):
            if prev+1 < curr:
                ans.append([prev+1, curr-1])

        if last < upper:
            ans.append([last+1, upper])

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# We have a sorted array of unique integers, and an inclusive range [lower, upper].
# We need to return the list of ranges that cover every number
# inside [lower, upper] that does NOT appear in nums. Right?"
#
# "A few quick questions:
# Can nums be empty? If so I assume the whole [lower, upper] is one missing range.
# Are all elements in nums guaranteed to be within [lower, upper]? Good.
# The output ranges should be as compact as possible – so [2,2] not just [2]?"
#
# "Let me walk the example:
# nums=[0,1,3,50,75], lower=0, upper=99.
# 0 and 1 are present so no gap at the start.
# 3 is present but 2 is missing → [2,2].
# 50 is present but 4–49 are missing → [4,49].
# 75 is present but 51–74 are missing → [51,74].
# 76–99 are all missing → [76,99].
# Output: [[2,2],[4,49],[51,74],[76,99]]. Got it."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# The simplest approach: build a set from nums for O(1) lookups,
# then walk every integer from lower to upper.
# Whenever I hit a missing number, I extend the current gap range.
# When I hit a number that IS in the set, I close the gap and record it.
#
# Time: O(upper – lower) which can be up to 2 billion – way too slow.
# Space: O(n) for the set.
# Should I optimize?"

class _163_MissingRanges_Brute:
    def findMissingRanges(self, nums: List[int], lower: int, upper: int) ->
List[List[int]]:
        present = set(nums)
        result = []
        gap_start = None
        for x in range(lower, upper + 1):
            if x not in present:
                if gap_start is None:
                    gap_start = x
            else:
                if gap_start is not None:
                    result.append([gap_start, x - 1])
                    gap_start = None
        if gap_start is not None:
            result.append([gap_start, upper])
        return result
```


PHASE 3 — OPTIMIZE

```
# "The brute force iterates over the entire numerical range – up to 2 billion steps.
# But nums has at most 100 elements. The gaps I care about
# only exist between consecutive numbers in nums, before the first, and after the
# last.
# So I only need to check those boundaries – that's O(n), not O(range).
#
# I think of nums as a set of walls. The missing ranges are the gaps between walls.
# There are at most n+1 gaps: one before nums[0], one between each pair, one after
# nums[-1].
#
# Pseudocode:
# if nums is empty: return [[lower, upper]]
# if lower < nums[0]: add [lower, nums[0]-1]
# for each consecutive pair (prev, curr) in nums:
# if prev+1 < curr: add [prev+1, curr-1]
# if nums[-1] < upper: add [nums[-1]+1, upper]
#
# Time: O(n). Space: O(1) extra. Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach.
# I'll use pairwise from itertools to cleanly iterate consecutive pairs."
class _163_MissingRanges:
    def findMissingRanges(self, nums: List[int], lower: int, upper: int) ->
List[List[int]]:
    if not nums:
        return [[lower, upper]]
    result = []
    first, last = nums[0], nums[-1]
    if lower < first:
        result.append([lower, first - 1])
    for prev, curr in pairwise(nums):
        if prev + 1 < curr:
            result.append([prev + 1, curr - 1])
    if last < upper:
        result.append([last + 1, upper])
    return result
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# nums=[0,1,3,50,75], lower=0, upper=99
# nums not empty. first=0, last=75.
# lower(0) < first(0)? No. No leading gap.
# pairwise: (0,1) -> 1 < 1? No. (1,3) -> 2 < 3? Yes -> [2,2]. (3,50) -> 4 < 50? Yes ->
[4,49]. (50,75) -> 51 < 75? Yes -> [51,74].
# last(75) < upper(99)? Yes -> [76,99].
# result = [[2,2],[4,49],[51,74],[76,99]] ✓
```

```

#
# nums=[-1], lower=-1, upper=-1
# first=-1, last=-1.
# lower(-1) < first(-1)? No.
# pairwise: empty (only one element).
# last(-1) < upper(-1)? No.
# result = [] ✓
#
# nums=[], lower=1, upper=5
# Empty → return [[1,5]] ✓
#
# "No bugs. All cases pass."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) – one pass through nums plus constant work per gap.
# Space O(1) extra – result list aside, no auxiliary storage.
# This is a massive improvement over O(upper – lower) brute force
# given the constraint that nums has at most 100 elements.
#
# Trade-off: if nums were not sorted I'd need to sort first – O(n log n).
# The problem guarantees sorted input so we skip that.
#
# pairwise is Python 3.10+. If the interviewer is on an older version
# I'd write: for i in range(1, len(nums)): prev, curr = nums[i-1], nums[i].
#
# Given more time I'd ask: what if the output should be strings like '2->2'
# instead of lists? Just a formatting change at the append step."

```

◆ Quick Review

Key Insights

- Use a pointer (or iterate index) to traverse the nums array while keeping track of the previous number considered.
- Check for missing numbers before the first element, between consecutive elements, and after the last element.
- When a gap is found, convert it to a range string that is either a single number (if the gap

is exactly one number) or an interval.

- Handle edge cases when the nums array is empty.

Space & Time

Time Complexity: $O(n)$, where n is the length of the nums array, since we iterate through the array once.

Space Complexity: $O(1)$ additional space (ignoring the output list), as we only use a few extra variables.

Solution

The solution leverages a single pass through the input array and calculates the missing ranges by comparing the expected number with the current number in nums. First, initialize a variable "prev" to one less than the lower bound so that missing numbers from the very start of the range can be handled uniformly. For each number in the array, if there is a gap between prev and the current number (i.e., current number is at least 2 more than prev), then add the missing range (prev+1 to current number-1) to the output list. Update "prev" to the current number and

finally handle any gap between the last number and the upper bound. The primary data structures used are the list for the output; the approach is iterative.

Arrays & Hashing

□ ★ #346 Moving Average from Data Stream ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Queue · Data Stream
Lists: ★ Premium | Premium 98

Problem

Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window.

Implement the MovingAverage class:

- **MovingAverage(int size)** Initializes the object with the size of the window size.
- **double next(int val)** Returns the moving average of the last size values of the stream.

Example 1:

```
Input
["MovingAverage", "next", "next", "next", "next"]
[[3], [1], [10], [3], [5]]
Output
[null, 1.0, 5.5, 4.66667, 6.0]

Explanation
MovingAverage movingAverage = new MovingAverage(3);
```

Constraints:

- $1 \leq \text{size} \leq 1000$
- $-105 \leq \text{val} \leq 105$
- At most 104 calls will be made to next.

My LeetCode Solution

• My LeetCode Solution

```
class MovingAverage:

    def __init__(self, size: int):
        self.size = size
        self.q = deque()
        self.summ = 0

    def next(self, val: int) -> float:
        if self.size == len(self.q):
            self.summ -= self.q.popleft()

        self.summ += val
        self.q.append(val)
        return self.summ / len(self.q)

# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We're designing a class that receives a stream of integers one at a time.

Each call to next() should return the average of the last 'size' values seen so far.

If fewer than 'size' values have arrived, we average however many we have. Right?"

#

"A few quick questions:

Can size be 1? Constraints say yes – that's just returning the latest value.

```

# Can values be negative? Yes, constraints say down to  $-10^5$ .
# Is there a limit on how many next() calls we get? Up to  $10^4$  – good to know for
space."
#
# "Let me walk the example:
# size=3. next(1)=1.0 (only 1 value). next(10)=5.5 (avg of 1,10).
# next(3)=4.667 (avg of 1,10,3 – window full now).
# next(5)=6.0 (avg of 10,3,5 – 1 falls out). Got it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: store every value in a plain list.
# On each next() call, slice the last 'size' elements and compute the average.
#
# Time:  $O(\text{size})$  per next() call due to the slice and sum.
# Space:  $O(n)$  where n is total calls – the list grows forever.
# Should I optimize?"
class _346_MovingAverage_Brute:
    def __init__(self, size: int):
        self.size = size
        self.stream = []

    def next(self, val: int) -> float:
        self.stream.append(val)
        window = self.stream[-self.size:]
        return sum(window) / len(window)

# PHASE 3 — OPTIMIZE
# "Two inefficiencies in the brute force:
# First, we recompute the sum from scratch every call – that's  $O(\text{size})$  work.
# Second, we store the entire history but only ever need the last 'size' values.
#
# I notice we're recomputing the same sum repeatedly. What if we cache it?
# I'll keep a running sum and update it incrementally:
# when a new value comes in, add it. When the window is full,
# subtract the oldest value before adding the new one.
#
# For the window itself, a deque is perfect –  $O(1)$  append on the right,
#  $O(1)$  popleft when the oldest value falls out. The deque never exceeds 'size'
elements.
#
# Pseudocode:
# __init__: q = deque(), running_sum = 0, self.size = size
# next(val):
# if len(q) == size: running_sum -= q.popleft()
# running_sum += val
# q.append(val)
# return running_sum / len(q)
#
# Time:  $O(1)$  per next() call. Space:  $O(\text{size})$  – the deque is capped.
# Does that make sense before I write it out?"

```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _346_MovingAverage:
    def __init__(self, size: int):
        self.size = size
        self.window = deque()
        self.running_sum = 0

    def next(self, val: int) -> float:
        if len(self.window) == self.size:
            self.running_sum -= self.window.popleft() # evict oldest
        self.running_sum += val
        self.window.append(val)
        return self.running_sum / len(self.window)
```

PHASE 5 — TEST & DEBUG

"Let me trace through the example: size=3."

```
#
# next(1): window not full. sum=1, window=[1]. return 1/1 = 1.0 ✓
# next(10): window not full. sum=11, window=[1,10]. return 11/2 = 5.5 ✓
# next(3): window not full. sum=14, window=[1,10,3]. return 14/3 = 4.667 ✓
# next(5): window full → popleft() removes 1, sum=14-1=13.
# sum=13+5=18, window=[10,3,5]. return 18/3 = 6.0 ✓
#
# Edge – size=1:
# next(7): window full immediately after first call? No – starts empty.
# sum=7, window=[7]. return 7/1 = 7.0 ✓
# next(4): window full (size=1). popleft removes 7, sum=0+4=4, window=[4].
# return 4/1 = 4.0 ✓
#
# "No bugs. Running sum stays consistent across evictions."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(1) per next() call – all operations are constant: deque append,
# popleft, addition, division.
# Space O(size) – the deque holds at most 'size' elements regardless of
# how many total calls are made. Much better than the O(n) brute force.
#
# Trade-off worth mentioning: floating point precision. If values are large
# and size is large, the running sum can accumulate rounding errors over time.
# A production system might use Python's Decimal or periodically recompute
# the sum from scratch to correct drift.
#
# Alternative: a circular buffer (fixed-size list with a pointer) does the same
# thing with slightly better cache locality than a deque, but the deque is
# cleaner to read and interview-safe.
#
# Given more time I'd ask: what if we need to support a variable window size
# that can change between calls? We'd need to resize the deque dynamically."
```


◆ Quick Review

Key Insights

- Use a queue (or circular buffer) to keep track of the current window elements.
- Maintain a running sum to quickly update the window's total when a new element comes in.
- When the window is full, subtract the value of the element that falls out as a new one is added.
- Each call to `next()` is executed in constant time, $O(1)$.

Space & Time

Time Complexity: $O(1)$ per call to `next()`

Space Complexity: $O(\text{window size})$

Solution

The solution leverages a queue data structure to store the elements of the sliding window. A running sum variable is maintained for efficient calculation of the moving average. When a new integer is added: If the size of the queue is equal to the maximum window size, remove the oldest element and subtract its

value from the running sum. Add the new element to both the queue and the running sum. Compute and return the moving average as the running sum divided by the number of elements in the queue. This approach ensures that each update is done in constant time without having to sum all elements in the window repeatedly.

Arrays & Hashing

□ ★ #760 Find Anagram Mappings ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table

Lists: ★ Premium | Premium 98

Problem

You are given two integer arrays `nums1` and `nums2` where `nums2` is an anagram of `nums1`. Both arrays may contain duplicates.

Return an index mapping array mapping from `nums1` to `nums2` where `mapping[i] = j` means the *i*th element in `nums1` appears in `nums2` at index *j*. If there are multiple answers, return any of them.

An array *a* is an anagram of an array *b* means *b* is made by randomizing the order of the elements in *a*.

Example 1:

Input: `nums1 = [12,28,46,32,50]`, `nums2 = [50,12,32,46,28]`

Output: `[1,4,3,2,0]`

Explanation: As `mapping[0] = 1` because the 0th element of `nums1` appears at `nums2[1]`, and `mapping[1] = 4` because the 1st element of `nums1` appears at `nums2[4]`, and so on.

Example 2:

Input: nums1 = [84,46], nums2 = [84,46]
Output: [0,1]

Constraints:

- $1 \leq \text{nums1.length} \leq 100$
- $\text{nums2.length} = \text{nums1.length}$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 105$
- nums2 is an anagram of nums1 .

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        hmap = {}
        for i, n in enumerate(nums2):
            hmap[n] = i

        ans = []
        for n in nums1:
            ans.append(hmap[n])

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# We have two arrays nums1 and nums2. nums2 is an anagram of nums1 –
# same elements, different order. We need to return a mapping array
# where mapping[i] is the index in nums2 where nums1[i] appears. Right?"
#
# "A few quick questions:
```

```
# Can there be duplicate values? The problem says yes.
# If a value appears multiple times in nums2, which index do we return?
# The problem says any valid answer is accepted – good, that simplifies things.
# Both arrays are the same length and nums2 is guaranteed to be an anagram,
# so every element in nums1 is guaranteed to exist in nums2?"
#
```

```
# "Let me walk the example:
```

```
# nums1=[12,28,46,32,50], nums2=[50,12,32,46,28].
# 12 is at index 1 in nums2 → mapping[0]=1.
# 28 is at index 4 → mapping[1]=4.
# 46 is at index 3 → mapping[2]=3.
# 32 is at index 2 → mapping[3]=2.
# 50 is at index 0 → mapping[4]=0.
# Output: [1,4,3,2,0]. Got it."
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock in the logic.
```

```
# For each element in nums1, scan nums2 left to right until I find a match
# and record that index.
```

```
#
# Time:  $O(n^2)$  – for each of  $n$  elements in nums1 we do a linear scan of nums2.
# Space:  $O(1)$  extra beyond the output array.
# Should I optimize?"
```

```
class _760_AnagramMappings_Brute:
```

```
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        mapping = []
        for num in nums1:
            for j, val in enumerate(nums2):
                if val == num:
                    mapping.append(j)
                    break
        return mapping
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is scanning nums2 from scratch for every element in nums1.
```

```
# We're asking the same question repeatedly: where does this value live in nums2?
```

```
# A hash map lets us answer that in  $O(1)$ .
```

```
#
# I'll preprocess nums2 once: build a map from value → index.
# Then for each element in nums1, look it up directly.
```

```
#
# One thing to flag: if there are duplicates, building the map will overwrite
# earlier indices with later ones. Since the problem allows any valid answer,
# that's fine – we just return whichever index was stored last.
```

```
#
# Pseudocode:
# index_in_nums2 = {val: i for i, val in enumerate(nums2)}
# return [index_in_nums2[num] for num in nums1]
#
# Time:  $O(n)$ . Space:  $O(n)$  for the hash map.
```

```
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _760_AnagramMappings:
```

```
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        index_in_nums2 = {val: i for i, val in enumerate(nums2)}
        return [index_in_nums2[num] for num in nums1]
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through both examples."
```

```
#
```

```
# nums1=[12,28,46,32,50], nums2=[50,12,32,46,28]
```

```
# index_in_nums2 = {50:0, 12:1, 32:2, 46:3, 28:4}
```

```
# nums1[0]=12 → 1
```

```
# nums1[1]=28 → 4
```

```
# nums1[2]=46 → 3
```

```
# nums1[3]=32 → 2
```

```
# nums1[4]=50 → 0
```

```
# Output: [1,4,3,2,0] ✓
```

```
#
```

```
# nums1=[84,46], nums2=[84,46]
```

```
# index_in_nums2 = {84:0, 46:1}
```

```
# Output: [0,1] ✓
```

```
#
```

```
# Duplicate case – nums1=[3,3], nums2=[3,3]
```

```
# enumerate(nums2): (0,3) → map={3:0}, then (1,3) → map={3:1} (overwritten).
```

```
# nums1[0]=3 → 1, nums1[1]=3 → 1. Output: [1,1].
```

```
# Both point to the same index – valid since any answer is accepted ✓
```

```
#
```

```
# "No bugs. Duplicate overwrite is intentional and within spec."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$  – one pass to build the map, one pass to build the result.
```

```
# Space  $O(n)$  for the hash map.
```

```
#
```

```
# The duplicate handling is worth calling out explicitly:
```

```
# by iterating enumerate(nums2) forward, later indices overwrite earlier ones.
```

```
# If the interviewer wanted ALL valid indices for duplicates returned,
```

```
# I'd change the map to store a list of indices per value and pop one per lookup.
```

```
#
```

```
# Trade-off: if n is tiny (say n=5 like in the example), the brute force  $O(n^2)$ 
```

```
# is barely slower and uses no extra space. The hash map pays off as n grows.
```

```
#
```

```
# Given more time I'd ask: what if we need to return the mapping in sorted order
```

```
# by nums2 index? We'd sort the output –  $O(n \log n)$  – but that changes the spec."
```

◆ Quick Review

Key Insights

- Since `nums2` is an anagram of `nums1`, every element in `nums1` has a corresponding position in `nums2`.
- A dictionary (or hash table) can be used to map each number in `nums2` to its indices.
- When iterating through `nums1`, we can assign the next available index from the dictionary for that number.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in the arrays (single pass to build the dictionary and another pass to form the mapping).

Space Complexity: $O(n)$ for storing the dictionary that holds indices of elements in `nums2`.

Solution

The approach consists of two primary steps: Build a dictionary to record all indices where each number appears in `nums2`. This dictionary maps a number to a list of indices. Iterate over `nums1` and for each element, remove (or pop) one index from the

corresponding list in the dictionary, and add that index to the final mapping array. This guarantees that each number from `nums1` is correctly mapped to one of its positions in `nums2` while efficiently handling duplicates.

Arrays & Hashing

□ ★ #1426 Counting Elements ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table

Lists: ★ Premium | Premium 98

Problem

Given an integer array `arr`, count how many elements `x` there are, such that `x + 1` is also in `arr`. If there are duplicates in `arr`, count them separately.

Example 1:

Input: `arr = [1,2,3]`
Output: 2
Explanation: 1 and 2 are counted cause 2 and 3 are in `arr`.

Example 2:

Input: `arr = [1,1,3,3,5,5,7,7]`
Output: 0
Explanation: No numbers are counted, cause there is no 2, 4, 6, or 8 in `arr`.

Constraints:

- $1 \leq \text{arr.length} \leq 1000$
- $0 \leq \text{arr}[i] \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def countElements(self, arr: List[int]) -> int:
        hmap = Counter(arr)
        ans = 0

        for n in hmap:
            if n + 1 in hmap:
                ans += hmap[n]

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

For each element x in the array, we check whether $x+1$ also exists in the array.

If it does, we count x . And if x appears multiple times, we count each occurrence separately – so duplicates all contribute individually. Right?"

#

"A few quick questions:

Can the array be empty? Constraints say length ≥ 1 , so no.

Can values be 0? Yes – so $x=0$ counts if 1 is present.

Can values be negative? Constraints say $0 \leq \text{arr}[i] \leq 1000$, so no.

Are there duplicates? Yes, and the problem explicitly says to count them separately."

#

"Let me walk the examples:

$\text{arr}=[1,2,3]$: $1+1=2$ is in arr $\rightarrow$ count 1. $2+1=3$ is in arr $\rightarrow$ count 2. $3+1=4$ not in arr.

Output: 2 ✓

$\text{arr}=[1,1,3,3,5,5,7,7]$: $1+1=2$ not in arr. $3+1=4$ not in arr. Same for 5,7.

Output: 0 ✓ Got it."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each element x in arr, scan the entire array to see if $x+1$ exists.

If it does, increment the count.

#

Time: $O(n^2)$ – nested scan for every element.

Space: $O(1)$ – no extra storage.

Should I optimize?"

```
class _1426_CountingElements_Brute:
```

```
    def countElements(self, arr: List[int]) -> int:
        count = 0
```

```

    for x in arr:
        for y in arr:
            if y == x + 1:
                count += 1
                break
    return count

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the inner scan – we're asking 'does x+1 exist?' n times,
# each taking O(n). A hash set answers that in O(1).
#
# But there's a subtlety: duplicates. If arr=[1,1,2], both 1s should be counted.
# A plain set tells us x+1 exists but not how many times x appears.
# So I'll use a Counter instead – it gives me O(1) existence checks
# AND stores the frequency of each value.
#
# Walk over the unique keys in the counter. If key+1 is also in the counter,
# add counter[key] to the answer – that accounts for all duplicates of key at once.
#
# Pseudocode:
# freq = Counter(arr)
# for x in freq:
#     if x+1 in freq: ans += freq[x]
# return ans
#
# Time: O(n) to build the Counter, O(k) to iterate unique keys where k <= n.
# Overall O(n). Space: O(k) for the Counter. Does that make sense?"

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."

```

```

class _1426_CountingElements:
    def countElements(self, arr: List[int]) -> int:
        freq = Counter(arr)
        count = 0
        for x in freq:
            if x + 1 in freq:
                count += freq[x]
        return count

```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# arr=[1,2,3]
# freq={1:1, 2:1, 3:1}
# x=1: 2 in freq → count+=1. count=1.
# x=2: 3 in freq → count+=1. count=2.
# x=3: 4 not in freq.
# return 2 ✓
#
# arr=[1,1,3,3,5,5,7,7]

```

```

# freq={1:2, 3:2, 5:2, 7:2}
# x=1: 2 not in freq.
# x=3: 4 not in freq.
# x=5: 6 not in freq.
# x=7: 8 not in freq.
# return 0 ✓
#
# Duplicate counting – arr=[1,1,2]
# freq={1:2, 2:1}
# x=1: 2 in freq → count+=freq[1]=2. count=2.
# x=2: 3 not in freq.
# return 2 ✓ (both 1s counted, as the problem requires)
#
# "No bugs. Duplicate handling confirmed."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) – one pass to build the Counter, one pass over unique keys.
# Space O(k) where k is the number of distinct values, at most O(n).
#
# Alternative: use a plain set instead of a Counter, then iterate the original arr.
# present = set(arr)
# return sum(1 for x in arr if x+1 in present)
# Same O(n) time and O(n) space – slightly simpler but both are valid.
# The Counter approach avoids re-iterating arr which is a minor readability win.
#
# Trade-off: if the value range is small and fixed (like 0–1000 here),
# a boolean array of size 1001 works as a lookup in O(1) with better cache
# performance than a hash map, and O(1001) ~ O(1) space in practice.
#
# Given more time I'd ask: what if we needed to return the actual elements
# that qualify rather than a count? We'd collect them into a list instead."

```

◆ Quick Review

Key Insights

- Use a hash table or set to store unique elements from arr for constant time lookups.
- Iterate through each element in arr and check if $x + 1$ exists in the set.
- Count each valid occurrence, including duplicates.

- This approach efficiently handles the problem without needing nested loops.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in `arr` (set lookups are $O(1)$ on average).

Space Complexity: $O(n)$, due to the use of an auxiliary set to store unique elements.

Solution

The solution leverages a set to achieve constant time membership checks. First, we convert `arr` to a set of unique numbers. Then, iterate through `arr`, and for each element, check if `(element + 1)` exists in the set. If it does, increment the count. This approach correctly handles duplicates by counting each occurrence during iteration.

Arrays & Hashing

□ #1534 Count Good Triplets

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Enumeration
Lists: CodeSignal

Problem

Given an array of integers `arr`, and three integers `a`, `b` and `c`. You need to find the number of good triplets.

A triplet $(arr[i], arr[j], arr[k])$ is good if the following conditions are true:

- $0 \leq i < j < k < arr.length$
- $|arr[i] - arr[j]| \leq a$
- $|arr[j] - arr[k]| \leq b$
- $|arr[i] - arr[k]| \leq c$

Where $|x|$ denotes the absolute value of x .
Return the number of good triplets.

Example 1:

Input: `arr = [3,0,1,1,9,7]`, `a = 7`, `b = 2`, `c = 3`

Output: 4

Explanation: There are 4 good triplets: $[(3,0,1), (3,0,1), (3,1,1), (0,1,1)]$.

Example 2:

Input: arr = [1,1,2,2,3], a = 0, b = 0, c = 1

Output: 0

Explanation: No triplet satisfies all conditions.

Constraints:

- $3 \leq \text{arr.length} \leq 100$
- $0 \leq \text{arr}[i] \leq 1000$
- $0 \leq a, b, c \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int
    :
        ans = 0
        for i, j, k in itertools.combinations(range(len(arr)), 3):
            if abs(arr[i]-arr[j]) <= a and abs(arr[j]-arr[k]) <= b and abs(arr
[i]-arr[k]) <= c:
                ans += 1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# We need to count all index triplets (i, j, k) where i < j < k,
# and all three pairwise conditions hold simultaneously:
# the first pair must differ by at most a,
# the middle-to-last pair by at most b,
# and the first-to-last pair by at most c.
# We're counting triplets, not returning them. Right?"
#
# "A few quick questions:
```

```
# Can values in arr be 0? Yes, constraints allow it.
# Can a, b, c be 0? Yes – that means exact equality required for that pair.
# Array length is at most 100 – good to know for complexity decisions."
#
# "Let me walk the example:
# arr=[3,0,1,1,9,7], a=7, b=2, c=3.
# (3,0,1) at indices (0,1,2): |3-0|=3<=7, |0-1|=1<=2, |3-1|=2<=3 ✓
# (3,0,1) at indices (0,1,3): |3-0|=3<=7, |0-1|=1<=2, |3-1|=2<=3 ✓
# (3,1,1) at indices (0,2,3): |3-1|=2<=7, |1-1|=0<=2, |3-1|=2<=3 ✓
# (0,1,1) at indices (1,2,3): |0-1|=1<=7, |1-1|=0<=2, |0-1|=1<=3 ✓
# Output: 4. Got it."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Three nested loops over indices (i, j, k) with i < j < k.
# Count the triplet when arr[i] + arr[j] + arr[k] <= a + b + c.
# Correct, but O(n^3) is too slow when n is up to 100.
# Time: O(n^3). Space: O(1).
# Should I go ahead and optimize?"
```

```
class _1534_CountGoodTriplets_Brute:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        count = 0
        n = len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    if (abs(arr[i] - arr[j]) <= a and
                        abs(arr[j] - arr[k]) <= b and
                        abs(arr[i] - arr[k]) <= c):
                        count += 1
        return count
```

PHASE 3 — OPTIMIZE

```
# "The algorithm stays O(n^3) – there's no known sub-cubic solution for this
# specific three-condition problem, and with n=100 we don't need one.
# But I can make two practical improvements:
#
# First: early pruning. The three conditions are checked in sequence.
# If |arr[i] - arr[j]| > a, there's no point checking any k for this (i,j) pair.
# We skip the entire inner k loop, reducing the constant factor significantly.
#
# Second: use itertools.combinations for cleaner, more readable code.
# It generates all (i,j,k) triples with i<j<k in one line.
# Note that combinations doesn't support early pruning on the outer pair,
# so for readability I'll use it here since n is small enough that it doesn't
# matter.
#
# I'll go with the early-pruning nested loop to show I understand the trade-off,
# then mention the itertools alternative."
```



```

#
# Time:  $O(n^3)$  worst case, faster in practice due to pruning.
# Space:  $O(1)$ ."

# PHASE 4 — CLEAN CODE
# "Now I'll implement with early pruning and meaningful names."
class _1534_CountGoodTriplets:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        count = 0
        n = len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                if abs(arr[i] - arr[j]) > a: # prune: skip all k for this pair
                    continue
                for k in range(j + 1, n):
                    if (abs(arr[j] - arr[k]) <= b and
                        abs(arr[i] - arr[k]) <= c):
                        count += 1
        return count

# itertools one-liner alternative (interview-clean, no pruning):
class _1534_CountGoodTriplets_Itertools:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        return sum(
            abs(arr[i] - arr[j]) <= a and
            abs(arr[j] - arr[k]) <= b and
            abs(arr[i] - arr[k]) <= c
            for i, j, k in itertools.combinations(range(len(arr)), 3)
        )

# PHASE 5 — TEST & DEBUG
# "Let me trace the pruning version on example 1."
#
# arr=[3,0,1,1,9,7], a=7, b=2, c=3
# i=0 (arr[i]=3):
# j=1 (arr[j]=0): |3-0|=3<=7 → not pruned.
# k=2: |0-1|=1<=2, |3-1|=2<=3 → count=1
# k=3: |0-1|=1<=2, |3-1|=2<=3 → count=2
# k=4: |0-9|=9>2 → skip
# k=5: |0-7|=7>2 → skip
# j=2 (arr[j]=1): |3-1|=2<=7 → not pruned.
# k=3: |1-1|=0<=2, |3-1|=2<=3 → count=3
# k=4: |1-9|=8>2 → skip
# k=5: |1-7|=6>2 → skip
# j=3 (arr[j]=1): |3-1|=2<=7 → not pruned.
# k=4: |1-9|=8>2 → skip
# k=5: |1-7|=6>2 → skip
# j=4 (arr[j]=9): |3-9|=6<=7 → not pruned.
# k=5: |9-7|=2<=2, |3-7|=4>3 → skip
# j=5: no k available.
# i=1 (arr[i]=0):

```

```

# j=2 (arr[j]=1): |0-1|=1<=7 → not pruned.
# k=3: |1-1|=0<=2, |0-1|=1<=3 → count=4
# k=4,5: fail b condition → skip
# ... remaining pairs yield 0 more.
# return 4 ✓
#
# arr=[1,1,2,2,3], a=0, b=0, c=1
# a=0 means |arr[i]-arr[j]| must be 0 → only equal pairs pass.
# j=1 (1==1): pass a. k=2: |1-2|=1>0 → fail b. k=3: same. k=4: same.
# j=3 (1==?): arr[1]=1, arr[3]=2 → |1-2|=1>0 → pruned.
# No triplet survives. return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n^3)$  worst case when all pairs pass the first condition –
# but early pruning on condition 1 cuts the k loop entirely for bad (i,j) pairs,
# making it much faster in practice.
# Space  $O(1)$  – just a counter.
#
# With n=100 the worst case is 161,700 triplets – trivially fast.
# If n grew to  $10^4$ ,  $O(n^3)$  would be  $10^{12}$  – completely infeasible.
# At that scale I'd look at sorting + binary search to narrow candidates per (i,j),
# or segment trees for range queries – but that's overkill here.
#
# The itertools one-liner is the cleanest version for an interview
# when you want to show Python fluency without sacrificing readability.
# Trade-off: it can't prune early, so stick with the nested loop if the
# interviewer asks about optimization.
#
# Given more time I'd ask: what if the array is sorted?
# Sorted order lets us binary search for valid k values after fixing i and j,
# reducing the inner loop from  $O(n)$  to  $O(\log n)$  → overall  $O(n^2 \log n)$ ."

```

◆ Quick Review

Key Insights

- Brute force approach using three nested loops is acceptable since the maximum array length is 100.
- Check all possible triplets and count only those meeting the specified conditions.

- **Absolute difference comparisons are the key condition validations.**

Space & Time

Time Complexity: $O(n^3)$ where n is the length of the array.

Space Complexity: $O(1)$ extra space (ignoring input space).

Solution

We use a brute force algorithm with three nested loops, iterating over all combinations of triplets (i, j, k) ensuring $i < j < k$. For each triplet, we calculate the absolute differences: $|arr[i] - arr[j]|$ $|arr[j] - arr[k]|$ $|arr[i] - arr[k]|$. The triplet is counted as good if all absolute differences satisfy the conditions ($\leq a$, $\leq b$, and $\leq c$ respectively). This simple approach leverages the small input size, eliminating the need for more complex data structures or optimizations.

String

Round 1 · High · Easy · 3 questions

String

□ #383 Ransom Note

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Counting
Lists: Grind 169

Problem

Given two strings `ransomNote` and `magazine`, return `true` if `ransomNote` can be constructed by using the letters from `magazine` and `false` otherwise.

Each letter in `magazine` can only be used once in `ransomNote`.

Example 1:

```
Input: ransomNote = "a", magazine = "b"  
Output: false
```

Example 2:

```
Input: ransomNote = "aa", magazine = "ab"  
Output: false
```

Example 3:

```
Input: ransomNote = "aa", magazine = "aab"  
Output: true
```

Constraints:

- $1 \leq \text{ransomNote.length}, \text{magazine.length} \leq 105$
- ransomNote and magazine consist of lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        hmap = Counter(magazine)
        for c in ransomNote:
            hmap[c] -= 1
            if hmap[c] < 0:
                return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We need to check if every character in ransomNote can be supplied

by magazine – one letter from magazine per letter in ransomNote.

So if ransomNote needs two 'a's, magazine must have at least two 'a's. Right?"

#

"A few quick questions:

Are both strings guaranteed to be non-empty? Yes, length ≥ 1 .

Only lowercase English letters – so exactly 26 possible characters. Good.

We're not modifying the strings, just checking feasibility?"

#

"Let me walk the examples:

ransomNote='a', magazine='b': 'a' not in magazine → false ✓

ransomNote='aa', magazine='ab': need two 'a's, magazine has one → false ✓

ransomNote='aa', magazine='aab': need two 'a's, magazine has two → true ✓ Got it."

PHASE 2 — BRUTE FORCE

"The simplest approach: convert magazine to a list so I can 'use up' letters.

For each character in ransomNote, scan the list to find it and remove it.

If I can't find it, return false.

```

#
# Time: O(n * m) – for each of n chars in ransomNote, scan up to m chars in
# magazine.
# Space: O(m) for the mutable copy of magazine.
# Should I optimize?"
class _383_RansomNote_Brute:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        available = list(magazine)
        for char in ransomNote:
            if char not in available:
                return False
            available.remove(char)
        return True

# PHASE 3 — OPTIMIZE
# "The bottleneck is scanning magazine for each character – that's the n*m.
# The question we keep asking is: how many of this letter does magazine still have?
# A frequency counter answers that in O(1).
#
# I'll count every character in magazine upfront with a Counter.
# Then walk ransomNote: for each character, decrement its count.
# If any count drops below zero, magazine doesn't have enough of that letter –
# return false.
# If I finish ransomNote without going negative, return true.
#
# Pseudocode:
# freq = Counter(magazine)
# for char in ransomNote:
#     freq[char] -= 1
#     if freq[char] < 0: return False
# return True
#
# Time: O(n + m) – one pass each over magazine and ransomNote.
# Space: O(1) – the Counter holds at most 26 keys (lowercase letters only).
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _383_RansomNote:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        available = Counter(magazine)
        for char in ransomNote:
            available[char] -= 1
            if available[char] < 0:
                return False
        return True

# PHASE 5 — TEST & DEBUG
# "Let me trace through the three examples."
#

```

```

# ransomNote='a', magazine='b'
# available = {'b': 1}
# char='a': available['a']=0-1=-1 < 0 → return False ✓
# (Counter returns 0 for missing keys, so no KeyError)
#
# ransomNote='aa', magazine='ab'
# available = {'a':1, 'b':1}
# char='a': available['a']=0 → ok
# char='a': available['a']=-1 < 0 → return False ✓
#
# ransomNote='aa', magazine='aab'
# available = {'a':2, 'b':1}
# char='a': available['a']=1 → ok
# char='a': available['a']=0 → ok
# return True ✓
#
# Edge – ransomNote longer than magazine: ransomNote='abc', magazine='ab'
# available = {'a':1, 'b':1}
# char='a': 0 → ok. char='b': 0 → ok. char='c': -1 < 0 → return False ✓
#
# "No bugs. Counter's default of 0 for missing keys handles unseen characters
cleanly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n + m) - O(m)$  to build the Counter,  $O(n)$  to walk ransomNote.
# Space  $O(1)$  – the Counter has at most 26 entries since only lowercase letters
exist.
# Even though we write  $O(k)$  where  $k$  is distinct chars,  $k$  is bounded by 26 –
constant.
#
# Alternative: Counter subtraction in one line.
# return not (Counter(ransomNote) - Counter(magazine))
# Counter subtraction drops keys with zero or negative counts,
# so if anything remains, ransomNote needs letters magazine doesn't have.
# Elegant but less readable in an interview – I'd use the explicit loop.
#
# Another alternative: sort both strings and use two pointers –  $O(n \log n + m \log m)$ ,
# slower but uses  $O(1)$  space beyond the sort. Worth mentioning if interviewer
# constraints disallow hash maps.
#
# Given more time I'd ask: what if characters include uppercase or unicode?
# We'd replace the 26-slot assumption with a full hash map – space becomes
 $O(\text{alphabet size})$ ."

```

◆ Quick Review

Key Insights

- Use a hash table (or dictionary) to store the frequency of each character present in magazine.
- Iterate over every character in ransomNote and check if the corresponding count in the hash table is available and sufficient.
- Decrease the character count for every match; if any character count falls below zero, return false.
- If all characters in ransomNote can be accounted for, return true.

Space & Time

Time Complexity: $O(n + m)$, where n is the length of magazine and m is the length of ransomNote.

Space Complexity: $O(1)$, since the extra space required for the hash table is limited to at most 26 entries for lowercase English letters.

Solution

We construct a frequency map for the magazine string using a hash table. This frequency map records how many times each letter appears in magazine. Then we iterate

through the ransomNote string, and for every letter, we check the corresponding frequency in the map. If a letter is not found or its frequency is zero, we immediately return false as it means the letter cannot be used to build the note. Otherwise, we decrement the frequency count by one. If we successfully process all characters in ransomNote, then it can be constructed from magazine, and we return true.

String

□ ★ #734 Sentence Similarity ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String
Lists: ★ Premium | Premium 98

Problem

We can represent a sentence as an array of words, for example, the sentence "I am happy with leetcode" can be represented as `arr = ["I", "am", "happy", "with", "leetcode"]`.

Given two sentences `sentence1` and `sentence2` each represented as a string array and given an array of string pairs `similarPairs` where `similarPairs[i] = [xi, yi]` indicates that the two words `xi` and `yi` are similar.

Return `true` if `sentence1` and `sentence2` are similar, or `false` if they are not similar.

Two sentences are similar if:

- They have the same length (i.e., the same number of words)
- `sentence1[i]` and `sentence2[i]` are similar.

Notice that a word is always similar to itself, also notice that the similarity relation is not transitive. For example, if the words a and b are similar, and the words b and c are similar, a and c are not necessarily similar.

Example 1:

```
Input: sentence1 = ["great","acting","skills"], sentence2 =  
["fine","drama","talent"], similarPairs =  
[["great","fine"],["drama","acting"],["skills","talent"]]  
Output: true  
Explanation: The two sentences have the same length and each word i of  
sentence1 is also similar to the corresponding word in sentence2.
```

Example 2:

```
Input: sentence1 = ["great"], sentence2 = ["great"], similarPairs = []  
Output: true  
Explanation: A word is similar to itself.
```

Example 3:

```
Input: sentence1 = ["great"], sentence2 = ["doubleplus","good"], similarPairs =  
[["great","doubleplus"]]  
Output: false  
Explanation: As they don't have the same length, we return false.
```

Constraints:

- $1 \leq \text{sentence1.length}, \text{sentence2.length} \leq 1000$
- $1 \leq \text{sentence1}[i].\text{length}, \text{sentence2}[i].\text{length} \leq 20$
- $\text{sentence1}[i]$ and $\text{sentence2}[i]$ consist of English letters.
- $0 \leq \text{similarPairs.length} \leq 1000$

- `similarPairs[i].length == 2`
- `1 <= xi.length, yi.length <= 20`
- `xi` and `yi` consist of lower-case and upper-case English letters.
- All the pairs `(xi, yi)` are distinct.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def areSentencesSimilar(self, sentence1: List[str], sentence2: List[str],
similarPairs: List[List[str]]) -> bool:
        hmap = defaultdict(set)
        if len(sentence1) != len(sentence2):
            return False
        for a,b in similarPairs:
            hmap[a].add(b)
            hmap[b].add(a)

        for x,y in zip(sentence1, sentence2):

            if x!=y and y not in hmap[x]:
                return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Two sentences are similar if they're the same length and every word at position `i` in `sentence1` is similar to the word at position `i` in `sentence2`.

A word is always similar to itself.

Similarity is NOT transitive – if `a~b` and `b~c`, that doesn't mean `a~c`.

We're given the list of similar pairs explicitly, so we only trust those. Right?"

#

"A few quick questions:

Can `similarPairs` be empty? Yes – then only identical words are similar.

```
# Are pairs directional? The problem implies symmetric – if [a,b] is given,
# then a~b AND b~a. I'll confirm: yes, similarity is symmetric.
# Can the same pair appear twice? Constraints say all pairs are distinct – good."
#
# "Let me walk example 1:
# sentence1=['great','acting','skills'], sentence2=['fine','drama','talent'].
# Same length: 3. pairs include [great,fine],[drama,acting],[skills,talent].
# great~fine ✓, acting~drama ✓ (symmetric), skills~talent ✓ → true ✓ Got it."
```

PHASE 2 — BRUTE FORCE

```
# "The simplest approach: first check lengths. Then for each position i,
# check if sentence1[i] == sentence2[i], or scan all similarPairs
# to find a match in either direction.
#
# Time: O(n * p) – for each of n word pairs, scan up to p similarity pairs.
# Space: O(1) – no preprocessing.
# Should I optimize?"
class _734_SentenceSimilarity_Brute:
    def areSentencesSimilar(self, sentence1: List[str], sentence2: List[str],
similarPairs: List[List[str]]) -> bool:
        if len(sentence1) != len(sentence2):
            return False
        for word1, word2 in zip(sentence1, sentence2):
            if word1 == word2:
                continue
            found = any((a == word1 and b == word2) or (a == word2 and b == word1)
                        for a, b in similarPairs)
            if not found:
                return False
        return True
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is scanning all p pairs for every word position.
# We're asking the same question repeatedly: is word2 similar to word1?
# A hash map from each word to its set of similar words answers that in O(1).
#
# Preprocess similarPairs once: for each [a, b], add b to hmap[a] and a to hmap[b].
# That handles symmetry automatically – no need to check both directions at lookup
time.
#
# Then for each (word1, word2) pair in the sentences:
# if they're equal, they're trivially similar – skip.
# Otherwise check if word2 is in hmap[word1].
# Using defaultdict(set) means missing keys return an empty set – no KeyError.
#
# Pseudocode:
# if len(s1) != len(s2): return False
# similar = defaultdict(set)
# for a, b in pairs: similar[a].add(b); similar[b].add(a)
# for w1, w2 in zip(s1, s2):
```

```

# if w1 != w2 and w2 not in similar[w1]: return False
# return True
#
# Time: O(p + n). Space: O(p) for the map. Does that make sense?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _734_SentenceSimilarity:
    def areSentencesSimilar(self, sentence1: List[str], sentence2: List[str],
        similarPairs: List[List[str]]) -> bool:
        if len(sentence1) != len(sentence2):
            return False

        similar = defaultdict(set)
        for word_a, word_b in similarPairs:
            similar[word_a].add(word_b)
            similar[word_b].add(word_a) # symmetric: both directions

        for word1, word2 in zip(sentence1, sentence2):
            if word1 != word2 and word2 not in similar[word1]:
                return False

        return True

# PHASE 5 — TEST & DEBUG
# "Let me trace through all three examples."
#
# Example 1: s1=['great','acting','skills'], s2=['fine','drama','talent']
# pairs=[['great','fine'],['drama','acting'],['skills','talent']]
# similar = {great:{fine}, fine:{great}, drama:{acting}, acting:{drama},
# skills:{talent}, talent:{skills}}
# (great,fine): great!=fine, fine in similar[great] ✓
# (acting,drama): acting!=drama, drama in similar[acting] ✓
# (skills,talent): skills!=talent, talent in similar[skills] ✓
# return True ✓
#
# Example 2: s1=['great'], s2=['great'], pairs=[]
# Lengths match. similar={} (empty).
# (great,great): great==great → skip.
# return True ✓
#
# Example 3: s1=['great'], s2=['doubleplus','good'], pairs=[['great','doubleplus']]
# len(s1)=1 != len(s2)=2 → return False immediately ✓
#
# Non-transitive check: pairs=[['a','b'],['b','c']], checking a~c
# similar = {a:{b}, b:{a,c}, c:{b}}
# word1='a', word2='c': a!=c, c not in similar[a]={b} → return False ✓
# Confirms we don't accidentally apply transitivity.
#
# "No bugs. All cases pass including the transitivity trap."

# PHASE 6 — COMPLEXITY & FOLLOW-UP

```

```
# "Time  $O(p + n) - O(p)$  to build the similarity map,  $O(n)$  to check each word pair.  
# Space  $O(p)$  for the map – each pair adds two entries.  
#  
# The critical design decision is storing both directions upfront rather than  
# checking both at lookup time. It keeps the lookup a single  $O(1)$  set membership  
# test.  
#  
# Important constraint to call out: similarity is NOT transitive.  
# This rules out Union-Find (which is designed for transitive equivalence classes).  
# If transitivity WERE required – for example in Sentence Similarity II (#737) –  
# we'd use Union-Find to group words into connected components and check  
# if sentence1[i] and sentence2[i] share the same root.  
#  
# Given more time I'd ask: what if the same word pair could appear in different  
# cases,  
# like ['Great','fine'] vs ['great','fine']? We'd normalize to lowercase before  
# building  
# the map and before lookups."
```

◆ Quick Review

Key Insights

- First, check if both sentences have the same length.
- A word is always similar to itself.
- Similarity is defined only by the given pairs; it is not transitive.
- Use a hash table (dictionary) to store bi-directional similar pairs for quick look-ups.

Space & Time

Time Complexity: $O(n + m)$, where n is the number of words in the sentences and m is the number of similar pairs.

Space Complexity: $O(m)$, for storing the similar pairs in a hash table.

Solution

Check if the two sentences have the same length; if not, return false. Build a hash table where each word maps to a set of words it is similar to. For each pair $[x, y]$ in `similarPairs`, add y to the set for x and x to the set for y . Iterate over the words of both sentences simultaneously. For each corresponding pair, if the words are identical, continue; otherwise, check if one word appears in the similar set of the other. If all pairs meet the similarity condition, return true; otherwise, return false.

String

□ ★ #1165 Single Row Keyboard ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String

Lists: ★ Premium | Premium 98

Problem

There is a special keyboard with all keys in a single row.

Given a string keyboard of length 26 indicating the layout of the keyboard (indexed from 0 to 25). Initially, your finger is at index 0. To type a character, you have to move your finger to the index of the desired character. The time taken to move your finger from index i to index j is $|i - j|$. You want to type a string word. Write a function to calculate how much time it takes to type it with one finger.

Example 1:

Input: keyboard = "abcdefghijklmnopqrstuvwxyz", word = "cba"

Output: 4

Explanation: The index moves from 0 to 2 to write 'c' then to 1 to write 'b' then to 0 again to write 'a'.

Total time = 2 + 1 + 1 = 4.

Example 2:

Input: keyboard = "pqrstuvwxyzabcdefghijklmnopqrstuvwxyz", word = "leetcode"
Output: 73

Constraints:

- keyboard.length == 26
- keyboard contains each English lowercase letter exactly once in some order.
- 1 <= word.length <= 104
- word[i] is an English lowercase letter.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def calculateTime(self, keyboard: str, word: str) -> int:
        ans = 0
        hmap = {}
        for i, n in enumerate(keyboard):
            hmap[n] = i

        p = 0
        for c in word:
            ans += abs(p - hmap[c])

            p = hmap[c]

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We have a custom keyboard layout – 26 characters in some arbitrary order.

Our finger starts at index 0. To type each character in word,

```

# we move from the current index to that character's index on the keyboard.
# The cost is the absolute distance moved. We sum all moves and return the total.
Right?"
#
# "A few quick questions:
# The keyboard always has exactly 26 letters, each exactly once – so it's a
permutation.
# Word can only contain lowercase letters, and every letter in word is guaranteed
# to appear on the keyboard – so no missing-key edge case to handle.
# Word length can be up to  $10^4$  – so  $O(n)$  should be fine."
#
# "Let me walk example 1:
# keyboard='abcdefghijklmnopqrstuvwxyz', word='cba'.
# Start at 0. 'c' is at index 2:  $|0-2|=2$ . 'b' is at 1:  $|2-1|=1$ . 'a' is at 0:
 $|1-0|=1$ .
# Total =  $2+1+1 = 4$ . Got it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: for each character in word, scan the keyboard string
# from left to right to find where it lives, then compute the distance.
#
# Time:  $O(n * 26)$  – for each of  $n$  characters we scan up to 26 positions.
# Since keyboard is fixed at 26, this is effectively  $O(n)$ .
# Space:  $O(1)$ .
# Should I optimize the lookup?"
class _1165_SingleRowKeyboard_Brute:
    def calculateTime(self, keyboard: str, word: str) -> int:
        total = 0
        position = 0
        for char in word:
            next_pos = keyboard.index(char)
            total += abs(position - next_pos)
            position = next_pos
        return total

# PHASE 3 — OPTIMIZE
# "The brute force scans 26 characters for every letter typed – that's the inner
 $O(26)$ .
# We're asking the same question repeatedly: where does this letter live on the
keyboard?
# A hash map answers that in  $O(1)$  after a one-time  $O(26)$  build.
#
# Preprocess keyboard once: build a map from character → index.
# Then walk word: for each character, look up its index in  $O(1)$ ,
# add the absolute distance from the current position, update position, move on.
#
# Pseudocode:
# pos_map = {char: i for i, char in enumerate(keyboard)}
# total, position = 0, 0
# for char in word:

```

```
# total += abs(position - pos_map[char])
# position = pos_map[char]
# return total
#
# Time:  $O(26 + n) = O(n)$ . Space:  $O(26) = O(1)$  – the map is fixed size.
# In practice the constant factor improvement is small since 26 is tiny,
# but the hash map is the correct interview answer – cleaner and scales to
# larger alphabets."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _1165_SingleRowKeyboard:
    def calculateTime(self, keyboard: str, word: str) -> int:
        key_index = {char: i for i, char in enumerate(keyboard)}
        total = 0
        position = 0
        for char in word:
            total += abs(position - key_index[char])
            position = key_index[char]
        return total
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through both examples."
```

```
#
# keyboard='abcdefghijklmnopqrstuvwxyz', word='cba'
# key_index = {'a':0, 'b':1, 'c':2, ...}
# char='c': |0-2|=2, position=2. total=2
# char='b': |2-1|=1, position=1. total=3
# char='a': |1-0|=1, position=0. total=4
# return 4 ✓
#
# keyboard='pqrstuvwxyzabcdefghijklmnopqrstuvwxyz', word='leetcode'
# key_index =
# {'p':0, 'q':1, ..., 'a':10, 'b':11, ..., 'l':21, 'e':14, 't':19, 'c':12, 'o':24, 'd':13}
# char='l': |0-21|=21, position=21. total=21
# char='e': |21-14|=7, position=14. total=28
# char='e': |14-14|=0, position=14. total=28
# char='t': |14-19|=5, position=19. total=33
# char='c': |19-12|=7, position=12. total=40
# char='o': |12-24|=12, position=24. total=52
# char='d': |24-13|=11, position=13. total=63
# char='e': |13-14|=1, position=14. total=64 ← wait, let me recheck key_index
# (keyboard='pqrstuvwxyzabcdefghijklmnopqrstuvwxyz':
# p=0, q=1, r=2, s=3, t=4, u=5, v=6, w=7, x=8, y=9, z=10,
# a=11, b=12, c=13, d=14, e=15, f=16, g=17, h=18, i=19, j=20, k=21, l=22, m=23, n=24, o=25)
# char='l': |0-22|=22. char='e': |22-15|=7. char='e': 0. char='t': |15-4|=11.
# char='c': |4-13|=9. char='o': |13-25|=12. char='d': |25-14|=11. char='e':
# |14-15|=1.
# total = 22+7+0+11+9+12+11+1 = 73 ✓
#
```

"No bugs. Manual trace confirms both examples."

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$  –  $O(26)$  to build the map,  $O(n)$  to walk word. Space  $O(1)$  – 26-entry map.
#
# The brute force using keyboard.index() is also  $O(n)$  in practice since keyboard is
# always 26 characters – but the hash map is the canonical interview answer because
# it makes the  $O(1)$  lookup intent explicit and generalizes to any alphabet size.
#
# Trade-off: if keyboard could be very long (say a  $10^6$ -char layout), the hash map
# preprocessing cost would matter more – but we'd still prefer it over .index().
#
# One-liner alternative:
# pos = {c: i for i, c in enumerate(keyboard)}
# return sum(abs(pos[b] – pos[a]) for a, b in zip('_', word, word))
# where prepending '_' shifts word so each pair is (prev_char, curr_char),
# and '_' maps to nothing – so you'd actually prepend a dummy start:
# pairs = zip([None] + list(word), word) – but this gets messy.
# The explicit loop is cleaner in an interview.
#
# Given more time I'd ask: what if we can use two fingers?
# That becomes a DP problem – optimal two-finger typing is a classic hard variant."
```

◆ Quick Review

Key Insights

- Create a mapping (hash table/dictionary) from each character to its index on the keyboard for $O(1)$ lookups.
- The cost to type each character is the absolute difference between the current position and the target character's position on the keyboard.
- Start from index 0 and sum up the distances moved for each successive character in the word.

Space & Time

Time Complexity: $O(n)$, where n is the length of the word.

Space Complexity: $O(1)$ (constant space for a mapping of 26 characters).

Solution

We solve the problem by first building a dictionary that maps each character in the keyboard layout to its corresponding index. We then initialize a variable to track the current finger position (starting at 0). For each character in the word, we look up its index, calculate the absolute difference from the current position, add that distance to the total time, and update the current position. This approach uses a single pass over the word with constant time lookups, ensuring an efficient solution.

Two Pointers

Round 1 · High · Easy · 5 questions

Two Pointers

□ #88 Merge Sorted Array

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Denny Zhang**

Problem

You are given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to 0 and should be ignored. `nums2` has a length of `n`.

Example 1:

Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3
 Output: [1,2,2,3,5,6]
 Explanation: The arrays we are merging are [1,2,3] and [2,5,6].
 The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.

Example 2:

Input: nums1 = [1], m = 1, nums2 = [], n = 0
 Output: [1]
 Explanation: The arrays we are merging are [1] and [].
 The result of the merge is [1].

Example 3:

Input: nums1 = [0], m = 0, nums2 = [1], n = 1
 Output: [1]
 Explanation: The arrays we are merging are [] and [1].
 The result of the merge is [1].
 Note that because m = 0, there are no elements in nums1. The 0 is only there to ensure the merge result can fit in nums1.

Constraints:

- `nums1.length == m + n`
- `nums2.length == n`
- `0 <= m, n <= 200`
- `1 <= m + n <= 200`
- `-109 <= nums1[i], nums2[j] <= 109`

Follow up: Can you come up with an algorithm that runs in $O(m + n)$ time?

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def merge(self, nums1: list[int], m: int, nums2: list[int], n: int) -> None:
        i = m - 1 # nums1's index (the actual nums)
        j = n - 1 # nums2's index
        k = m + n - 1 # nums1's index (the next filled position)

        while k >= 0:
            if j < 0 or (i >= 0 and nums1[i] > nums2[j]):
                nums1[k] = nums1[i]
                i -= 1
            else:
                nums1[k] = nums2[j]
                j -= 1

            k -= 1

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

nums1 has m valid elements followed by n zeros as padding.

nums2 has n elements. Both are sorted in non-decreasing order.

We need to merge them in-place into nums1 – sorted, no return value. Right?"

#

"A few quick questions:

Can m or n be 0? Yes – if n=0 nums1 is already done; if m=0 we just copy nums2.

Can there be negative numbers? Yes, constraints go down to -10^9 .

Can there be duplicates? Yes – non-decreasing order means equal values are allowed.

We cannot use extra space for the merge? The follow-up asks for $O(m+n)$ time,

implying in-place. I'll target $O(1)$ extra space."

#

"Let me walk example 1:

nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6] n=3.

Merge [1,2,3] and [2,5,6] into nums1 → [1,2,2,3,5,6]. Got it."

PHASE 2 — BRUTE FORCE

"The simplest approach: copy nums2 into the tail of nums1 (the zero slots),

then sort the entire nums1 array.

#

nums1[m:] = nums2, then nums1.sort()

#

Time: $O((m+n) \log(m+n))$ for the sort. Space: $O(1)$ extra – sort is in-place.

This is clean but doesn't use the fact that both inputs are already sorted.

```
# Should I optimize to use that sorted property?"
class _88_MergeSortedArray_Brute:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        nums1[m:] = nums2
        nums1.sort()

# PHASE 3 — OPTIMIZE
# "The brute force throws away the sorted order and pays  $O(\log(m+n))$  to re-sort.
# The classic merge from the front copies to a temp array –  $O(m+n)$  space.
# But there's a smarter way that stays  $O(1)$  space.
#
# Key insight: if we merge from the BACK, we never overwrite elements we still need.
# The tail of nums1 is empty padding – we can fill it safely from right to left.
#
# Three pointers:
# i → last valid element in nums1 (index m-1)
# j → last element in nums2 (index n-1)
# k → last slot in nums1 (index m+n-1)
#
# At each step, pick the larger of nums1[i] and nums2[j], place it at k, move that
# pointer left.
# When j drops below 0, nums2 is exhausted – nums1's remaining elements are already
# in place.
# When i drops below 0 first, copy remaining nums2 into nums1.
#
# Pseudocode:
# i, j, k = m-1, n-1, m+n-1
# while k >= 0:
#     if j < 0 or (i >= 0 and nums1[i] > nums2[j]):
#         nums1[k] = nums1[i]; i -= 1
#     else:
#         nums1[k] = nums2[j]; j -= 1
#     k -= 1
#
# Time:  $O(m+n)$  – each element is placed exactly once.
# Space:  $O(1)$  – purely in-place.
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _88_MergeSortedArray:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        last1 = m - 1 # tail of valid nums1 elements
        last2 = n - 1 # tail of nums2
        fill = m + n - 1 # next slot to fill in nums1 (right to left)
        while fill >= 0:
            if last2 < 0 or (last1 >= 0 and nums1[last1] > nums2[last2]):
                nums1[fill] = nums1[last1]
                last1 -= 1
            else:
                nums1[fill] = nums2[last2]
                last2 -= 1
            fill -= 1
```

```

else:
    nums1[fill] = nums2[last2]
    last2 -= 1
    fill -= 1

```

PHASE 5 — TEST & DEBUG

"Let me trace through all three examples."

```

#
# nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6] n=3
# last1=2, last2=2, fill=5
# fill=5: nums1[2]=3 vs nums2[2]=6 → 6 wins → nums1[5]=6, last2=1, fill=4
# fill=4: nums1[2]=3 vs nums2[1]=5 → 5 wins → nums1[4]=5, last2=0, fill=3
# fill=3: nums1[2]=3 vs nums2[0]=2 → 3 wins → nums1[3]=3, last1=1, fill=2
# fill=2: nums1[1]=2 vs nums2[0]=2 → equal, take nums2 → nums1[2]=2, last2=-1,
fill=1

```

```

# fill=1: last2<0 → take nums1[1]=2 → nums1[1]=2, last1=0, fill=0
# fill=0: last2<0 → take nums1[0]=1 → nums1[0]=1, last1=-1, fill=-1
# nums1=[1,2,2,3,5,6] ✓
#

```

```

# nums1=[1] m=1, nums2=[] n=0
# last1=0, last2=-1, fill=0
# fill=0: last2<0 → nums1[0]=nums1[0]=1. fill=-1. done.
# nums1=[1] ✓
#

```

```

# nums1=[0] m=0, nums2=[1] n=1
# last1=-1, last2=0, fill=0
# fill=0: last1<0 → nums1[0]=nums2[0]=1, last2=-1, fill=-1. done.
# nums1=[1] ✓
#

```

"No bugs. All three cases pass including the m=0 and n=0 edges."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m+n)$ – each of the $m+n$ positions is filled exactly once.

Space $O(1)$ – purely in-place, no auxiliary arrays.

```

#
# The core insight worth stating explicitly: merging front-to-back requires a temp
# buffer because we'd overwrite nums1's valid elements. Merging back-to-front avoids
# this because the padding zeros are exactly the space we need, and large elements
# fill the tail first without touching anything we still care about.
#

```

Edge cases worth calling out:

– $n=0$: loop runs but $last2<0$ always, so $nums1$'s valid elements stay in place.

– $m=0$: $last1<0$ always, so $nums2$ is copied straight into $nums1$.

– All $nums2$ elements are larger than all $nums1$ elements: $nums2$ fills the back,
then $nums1$ elements slot into the front – handled cleanly by the pointer logic.

```

#
# Given more time I'd ask: what if  $nums1$  has exactly  $m$  slots (no padding)?
# Then in-place is impossible – we'd need to return a new merged array,
# which is the standard merge step in merge sort."

```

◆ Quick Review

Key Insights

- Both arrays are already sorted in non-decreasing order.
- The merge should be done in-place into `nums1`.
- Start merging from the end to avoid overwriting elements in `nums1`.
- Use two pointers, one for `nums1` and one for `nums2`, and a tail pointer for placement.

Space & Time

Time Complexity: $O(m + n)$

Space Complexity: $O(1)$

Solution

We use a two-pointer approach starting from the end of both arrays. Initialize three pointers: `p1` pointing to the last valid element in `nums1`. `p2` pointing to the last element in `nums2`. `tail` pointing to the last index of `nums1`. While both pointers are valid, compare the elements pointed by `p1` and `p2`, placing the larger element at the tail position. Decrement the corresponding pointer and the tail pointer.

Finally, if any elements remain in `nums2` (i.e., $p2 \geq 0$), copy them over into `nums1`. This solution leverages the extra space at the end of `nums1` and ensures the merge is done in-place in linear time.

Two Pointers

□ #125 Valid Palindrome

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: Grind 169

Problem

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward.

Alphanumeric characters include letters and numbers.

Given a string *s*, return true if it is a palindrome, or false otherwise.

Example 1:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

Example 2:

```
Input: s = "race a car"
Output: false
Explanation: "raceacar" is not a palindrome.
```

Example 3:

Input: s = " "

Output: true

Explanation: s is an empty string "" after removing non-alphanumeric characters.

Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

- $1 \leq s.length \leq 2 * 10^5$
- s consists only of printable ASCII characters.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        l = 0
        r = len(s)-1

        while l<r:
            while l<r and not(s[l].isalnum()):
                l+=1

            while l<r and not(s[r].isalnum()):
                r-=1

            if s[l].lower() != s[r].lower():
                return False

            l,r = l+1, r-1

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We strip everything except letters and digits, lowercase it all,

then check if the result reads the same forwards and backwards. Right?"

```

#
# "A few quick questions:
# Alphanumeric means letters AND digits – not just letters?
# So '0P' after cleaning is '0p', which is not a palindrome – confirmed?
# What about a string of only spaces or punctuation?
# Example 3 shows a single space returns true – after cleaning it's an empty string,
# and an empty string is considered a palindrome. Good.
# Can the string be length 1? Yes – a single alphanumeric character is a
# palindrome."
#
# "Let me walk example 1:
# 'A man, a plan, a canal: Panama' → 'amanaplanacanalpanama' → palindrome ✓
# Example 2: 'race a car' → 'raceacar' → not a palindrome ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the simplest correct approach.
# Copy every alphanumeric character into a cleaned list (lowercased),
# then compare the list to its reverse.
# Two passes over the string plus O(n) extra storage.
# Time: O(n). Space: O(n) for the cleaned copy.
# Should I go ahead and optimize?"

class _125_ValidPalindrome_Brute:
    def isPalindrome(self, s: str) -> bool:
        cleaned = [c.lower() for c in s if c.isalnum()]
        return cleaned == cleaned[::-1]

# PHASE 3 — OPTIMIZE
# "The brute force is already O(n) time, but uses O(n) space for the cleaned copy.
# We can do this in O(1) extra space using two pointers directly on the original
# string.
#
# Place left at 0 and right at len(s)-1.
# Advance left past any non-alphanumeric characters.
# Retreat right past any non-alphanumeric characters.
# Compare s[left].lower() and s[right].lower().
# If they differ, return False. Otherwise move both pointers inward.
# If left meets or passes right, the string is a palindrome.
#
# The key benefit: we never build a second string – we skip junk characters on the
# fly.
#
# Pseudocode:
# left, right = 0, len(s)-1
# while left < right:
# while left < right and not s[left].isalnum(): left += 1
# while left < right and not s[right].isalnum(): right -= 1
# if s[left].lower() != s[right].lower(): return False
# left += 1; right -= 1
# return True

```

```

#
# Time: O(n). Space: O(1). Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _125_ValidPalindrome:
    def isPalindrome(self, s: str) -> bool:
        left = 0
        right = len(s) - 1
        while left < right:
            while left < right and not s[left].isalnum(): # skip non-alphanumeric
                left += 1
            while left < right and not s[right].isalnum(): # skip non-alphanumeric
                right -= 1
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True

# PHASE 5 — TEST & DEBUG
# "Let me trace through all three examples."
#
# s="A man, a plan, a canal: Panama"
# left=0 ('A'), right=29 ('a') – both alnum.
# 'A'.lower()='a' == 'a' ✓. left=1, right=28.
# left=1 (' ') → skip → left=2 ('m'). right=28 ('m') – alnum.
# 'm'=='m' ✓. Continue inward... (all pairs match) → return True ✓
#
# s="race a car"
# left=0 ('r'), right=9 ('r'). 'r'=='r' ✓. left=1, right=8.
# left=1 ('a'), right=8 ('a'). 'a'=='a' ✓. left=2, right=7.
# left=2 ('c'), right=7 ('c'). 'c'=='c' ✓. left=3, right=6.
# left=3 ('e'), right=6 ('a'). 'e'!='a' → return False ✓
# (right=6 is 'a' in 'a car'; skip right=5 ' ' → right=4 'a')
# Wait – let me recount: "race acar"
# indices: r=0,a=1,c=2,e=3,' '=4,a=5,' '=6,c=7,a=8,r=9
# left=3 ('e'), right=9→skip none→'r'. 'e'!='r' → return False ✓
#
# s=" "
# left=0 (' ') → not alnum → left=1. Now left(1) >= right(0) → exit loop.
# return True ✓
#
# "No bugs. All three examples confirmed."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) – each character is visited at most once by left or right pointer.
# Space O(1) – no auxiliary storage beyond two integer pointers.
#

```

```
# The brute force one-liner is perfectly valid for an interview too – it's readable
# and still O(n) time. The two-pointer version wins purely on space: O(1) vs O(n).
# If the interviewer asks about the follow-up, lead with the space trade-off.
#
# isalnum() handles both letters and digits in one call – worth naming explicitly
# so the interviewer knows you're not just checking isalpha().
#
# Subtle trap: the inner while loops must guard left < right to avoid crossing
# pointers – if you forget, you can read out-of-bounds when the string is all
# non-alphanumeric (like the single-space example).
#
# Given more time I'd ask: what if the string could contain unicode characters
# like accented letters (é, ñ)? isalnum() in Python handles unicode correctly
# by default, so our solution already works – worth calling out."
```

◆ Quick Review

Key Insights

- Use two pointers (one starting at the beginning and one at the end) to compare characters.
- Skip characters that are not alphanumeric.
- Compare characters in a case-insensitive manner.
- Continue until the pointers meet or cross.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, as each character is processed at most once.

Space Complexity: $O(1)$, using only a few extra variables for pointers and temporary comparisons.

Solution

The solution employs a two–pointer approach: Initialize two pointers: one at the start (left) and one at the end (right) of the string. Skip any non–alphanumeric characters using these pointers. Convert the remaining characters at each pointer to lowercase and compare them. If any mismatch is found, return false; otherwise, continue until the pointers cross. Return true if all corresponding characters match. This method efficiently checks for a palindrome without needing additional space for a filtered string.

Two Pointers

□ #283 Move Zeroes

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers
Lists: Grind 169

Problem

Given an integer array `nums`, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

```
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
```

Example 2:

```
Input: nums = [0]
Output: [0]
```

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

Follow up: Could you minimize the total number of operations done?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        p = 0

        for i,x in enumerate(nums):
            if x!=0:
                nums[i], nums[p] = nums[p], nums[i]

                p+=1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# We need to move all 0s to the end of the array while keeping the relative
# order of all non-zero elements. This must be done in-place – no copy. Right?"
#
# "A few quick questions:
# Do we need to return anything? No – modify in-place, return nothing.
# Can the array be all zeros? Yes – output should still be all zeros.
# Can it be all non-zeros? Yes – nothing moves.
# Can there be negative numbers? Yes, constraints go down to -2^31."
#
# "Let me walk example 1:
# nums=[0,1,0,3,12] → [1,3,12,0,0].
# Non-zeros 1,3,12 keep their relative order and shift to the front.
# Zeros fill the tail. Got it."
```

PHASE 2 — BRUTE FORCE

```
# "The simplest approach: collect all non-zero elements into a new list,
# then write them back into nums, then fill the rest with zeros.
#
# non_zeros = [x for x in nums if x != 0]
# nums[:len(non_zeros)] = non_zeros
# nums[len(non_zeros):] = [0] * (len(nums) - len(non_zeros))
#
# Time: O(n). Space: O(n) – we make a copy of the non-zeros."
```

```

# Should I optimize for space?"
class _283_MoveZeroes_Brute:
    def moveZeroes(self, nums: List[int]) -> None:
        non_zeros = [x for x in nums if x != 0]
        nums[:len(non_zeros)] = non_zeros
        nums[len(non_zeros):] = [0] * (len(nums) - len(non_zeros))

# PHASE 3 — OPTIMIZE
# "The brute force uses O(n) extra space for the non-zero copy.
# We can do this in O(1) space with two pointers.
#
# Use a slow pointer 'place' that tracks where the next non-zero should go.
# Scan with a fast pointer 'scan'. Whenever nums[scan] is non-zero,
# swap it with nums[place] and advance place.
#
# This compacts all non-zeros to the front in their original order,
# and the swaps naturally push zeros toward the tail.
#
# Pseudocode:
# place = 0
# for scan in range(len(nums)):
#     if nums[scan] != 0:
#         nums[scan], nums[place] = nums[place], nums[scan]
#         place += 1
#
# Time: O(n). Space: O(1). Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _283_MoveZeroes:
    def moveZeroes(self, nums: List[int]) -> None:
        place = 0
        for scan in range(len(nums)):
            if nums[scan] != 0:
                nums[scan], nums[place] = nums[place], nums[scan]
                place += 1

# PHASE 5 — TEST & DEBUG
# "Let me trace through both examples."
#
# nums=[0,1,0,3,12]
# scan=0: nums[0]=0 → skip. place=0.
# scan=1: nums[1]=1 ≠ 0 → swap(1,0): [1,0,0,3,12]. place=1.
# scan=2: nums[2]=0 → skip. place=1.
# scan=3: nums[3]=3 ≠ 0 → swap(3,1): [1,3,0,0,12]. place=2.
# scan=4: nums[4]=12 ≠ 0 → swap(4,2): [1,3,12,0,0]. place=3.
# Result: [1,3,12,0,0] ✓
#
# nums=[0]
# scan=0: nums[0]=0 → skip.

```



```

# Result: [0] ✓
#
# nums=[1,2,3] (no zeros)
# Every element is non-zero → each swaps with itself. place advances each step.
# Result: [1,2,3] ✓ (self-swaps are fine – no change)
#
# "No bugs. Self-swaps when place==scan are harmless."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) – single pass. Space O(1) – two integer pointers.
#
# Follow-up asks to minimize operations. The swap approach does a swap
# for every non-zero element, even if it's already in the right place.
# A minor optimization: only swap when place != scan to skip self-swaps.
# But in an interview the current version is clean enough.
#
# Alternative: instead of swapping, just overwrite:
# write non-zeros to the front with assignment (not swap), then zero-fill the tail.
# Same complexity but fewer writes – useful if writes are expensive.
#
# place = 0
# for x in nums:
#     if x != 0: nums[place] = x; place += 1
#     nums[place:] = [0] * (len(nums) - place)
#
# Given more time I'd ask: what if we also need to preserve the positions
# of the zeros (not just push them to the end)? That changes the problem
# to a stable partition – same two-pointer approach still works."

```

◆ Quick Review

Key Insights

- Use a two–pointer technique: one pointer to iterate through the array and another pointer to track the position for the next non–zero element.
- For each non–zero element encountered, swap it with the element at the tracking pointer.

- This strategy minimizes operations by ensuring each non-zero element is moved at most once.
- After repositioning all non-zero elements, zeros will naturally occupy the remaining positions.
- The in-place requirement ensures space complexity remains $O(1)$.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in the array, since only one pass (or at most two simple passes) is required.

Space Complexity: $O(1)$ extra space, as the reordering is accomplished in-place.

Solution

The solution leverages a two-pointer approach. The left pointer (j) keeps track of the index where the next non-zero element should be placed, while the right pointer (i) iterates through the array. Whenever a non-zero element is encountered, it is swapped with the element at index j , and j is then incremented. This effectively gathers all non-zero elements at the beginning while

moving zeros to the end. A key optimization is to perform the swap only when necessary (i.e., when i and j are not the same) to reduce the number of write operations.

Two Pointers

□ ★ #408 Valid Word Abbreviation ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

A string can be abbreviated by replacing any number of non-adjacent, non-empty substrings with their lengths. The lengths should not have leading zeros.

For example, a string such as "substitution" could be abbreviated as (but not limited to):

- "s10n" ("s ubstitutio n")
- "sub4u4" ("sub stit u tion")
- "12" ("substitution")
- "su3i1u2on" ("su bst i t u ti on")
- "substitution" (no substrings replaced)

The following are not valid abbreviations:

- "s55n" ("s ubsti tutio n", the replaced substrings are adjacent)
- "s010n" (has leading zeros)

- "substitution" (replaces an empty substring)

Given a string `word` and an abbreviation `abbr`, return whether the string matches the given abbreviation.

A substring is a contiguous non-empty sequence of characters within a string.

Example 1:

```
Input: word = "internationalization", abbr = "i12iz4n"
Output: true
Explanation: The word "internationalization" can be abbreviated as "i12iz4n"
("i nternational iz atio n").
```

Example 2:

```
Input: word = "apple", abbr = "a2e"
Output: false
Explanation: The word "apple" cannot be abbreviated as "a2e".
```

Constraints:

- $1 \leq \text{word.length} \leq 20$
- `word` consists of only lowercase English letters.
- $1 \leq \text{abbr.length} \leq 10$
- `abbr` consists of lowercase English letters and digits.
- All the integers in `abbr` will fit in a 32-bit integer.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        i, j = 0, 0
        num = 0
        while i < len(word) and j < len(abbr):
            if abbr[j].isdigit():
                if abbr[j] == '0' and num == 0:
                    return False
                num = num * 10 + int(abbr[j])
            else:
                i += num
                num = 0
                if i >= len(word) or word[i] != abbr[j]:
                    return False
                i += 1
                j += 1

        return i + num == len(word) and j == len(abbr)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

An abbreviation replaces one or more non-adjacent non-empty substrings with their lengths.

Given a word and an abbreviation, return true if the abbreviation is valid for the word.

Numbers in the abbreviation represent how many consecutive characters to skip. Right?"

#

"A few quick questions:

Can there be leading zeros in the number part? No – that's explicitly invalid.

So '01' is not valid even though it could theoretically mean skip 1.

Can the abbreviation be the full word with no numbers? Yes – that's valid.

Can it be just a single number like '12'? Yes – if the word has exactly 12 chars."

#

"Let me walk example 1:

```
# word='internationalization', abbr='il2iz4n'.
# 'i' matches 'i'. '12' skips 12 chars. 'i' matches 'i'. 'z' matches 'z'.
# '4' skips 4 chars. 'n' matches 'n'. We reach end of both → true ✓
# Example 2: word='apple', abbr='a2e'. 'a' matches. '2' skips 'pp'. 'e'→word[3]='l'
# ≠ 'e' → false ✓"
```

PHASE 2 — BRUTE FORCE (also optimal)

```
# "Let me start with brute force to lock in the logic.
# Expand the abbreviation letter-by-letter against word using two pointers.
# When we hit a digit in abbr, consume that many characters from word.
# Any mismatch ? false; both exhausted together ? true.
# Time: O(n). Space: O(1).
# This is already optimal for the constraints - complexity stated clearly."
```

PHASE 3 — OPTIMIZE

```
# "Use pointer i into word and j into abbr.
# When abbr[j] is a digit: check for leading zero (abbr[j]=='0' and num==0 →
invalid).
# Accumulate the number: num = num*10 + digit.
# When abbr[j] is a letter: first advance i by num (the accumulated skip), reset num
to 0.
# Then compare word[i] with abbr[j] – if they differ, return False. Advance both i
and j.
# After the loop: we're valid only if i + num == len(word) AND j == len(abbr).
# That handles a trailing number at the end of abbr.
#
# Pseudocode:
# i, j, num = 0, 0, 0
# while i < len(word) and j < len(abbr):
#   if abbr[j].isdigit():
#     if abbr[j]=='0' and num==0: return False # leading zero
#     num = num*10 + int(abbr[j])
#   else:
#     i += num; num = 0
#     if i >= len(word) or word[i] != abbr[j]: return False
#     i += 1
#     j += 1
#   return i + num == len(word) and j == len(abbr)
#
# Time: O(n + m) where n=len(word), m=len(abbr). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement with meaningful names."
```

```
class _408_ValidWordAbbreviation:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        word_ptr = 0
        abbr_ptr = 0
        skip = 0
        while word_ptr < len(word) and abbr_ptr < len(abbr):
```

```

    if abbr[abbr_ptr].isdigit():
        if abbr[abbr_ptr] == '0' and skip == 0: # leading zero
            return False
        skip = skip * 10 + int(abbr[abbr_ptr])
    else:
        word_ptr += skip
        skip = 0
        if word_ptr >= len(word) or word[word_ptr] != abbr[abbr_ptr]:
            return False
        word_ptr += 1
        abbr_ptr += 1
    return word_ptr + skip == len(word) and abbr_ptr == len(abbr)

```

PHASE 5 — TEST & DEBUG

"Let me trace both examples."

```

#
# word='internationalization'(len=20), abbr='il2iz4n'
# j=0: 'i' → skip=0, advance word_ptr 0, word[0]='i'=='i' ✓. word_ptr=1.
# j=1: '1' → skip=1.
# j=2: '2' → skip=12.
# j=3: 'i' → word_ptr+=12=13. word[13]='i'=='i' ✓. word_ptr=14.
# j=4: 'z' → skip=0, word[14]='z'=='z' ✓. word_ptr=15.
# j=5: '4' → skip=4.
# j=6: 'n' → word_ptr+=4=19. word[19]='n'=='n' ✓. word_ptr=20.
# Loop ends (word_ptr=20=len(word)). word_ptr+skip=20+0=20==20, abbr_ptr=7==7. True ✓
#
# word='apple'(len=5), abbr='a2e'
# j=0: 'a' → word[0]='a'=='a' ✓. word_ptr=1.
# j=1: '2' → skip=2.
# j=2: 'e' → word_ptr+=2=3. word[3]='l' ≠ 'e' → return False ✓
#
# Leading zero: abbr='a01e' → j=1: '0' and skip==0 → return False ✓
#
# "No bugs. Leading zero check and trailing number handled correctly."

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n + m) – each character in word and abbr is visited at most once.
# Space O(1) – three integer pointers.
#
# The two trickiest cases to get right:
# 1. Leading zeros – must check abbr[j]=='0' AND accumulated num==0 to distinguish
# '0' (leading zero, invalid) from '10', '20' etc. (valid multi-digit numbers ending
# in 0).
# 2. Trailing number – if abbr ends with digits (e.g. abbr='a4'), the loop exits
# when abbr_ptr reaches end but skip is still non-zero.
# The final check word_ptr + skip == len(word) handles this correctly.
#
# Given more time I'd ask: what if we want to generate ALL valid abbreviations
# for a word? That's a backtracking problem – enumerate all subsets of positions

```


to replace with their lengths."

◆ Quick Review

Key Insights

- Use two pointers: one for looping through the word and one for the abbreviation.
- When encountering a digit in abbr, accumulate the number and skip that many characters in the word.
- Check for invalid cases such as numbers with leading zeros or consecutive digit segments that might imply adjacent skipped substrings.
- Compare letters directly when they are not digits.

Space & Time

Time Complexity: $O(n + m)$ where n is the length of the word and m is the length of the abbreviation.

Space Complexity: $O(1)$ (constant extra space)

Solution

The approach uses two pointers to iterate over the word and abbreviation. For each character in the abbreviation: If it is a letter, compare it

with the current character in the word. If it is a digit, first ensure that it does not start with '0'. Then, convert the sequence of digits into a number which represents how many characters in the word to skip. After processing, both pointers should have reached the end of their respective strings for the abbreviation to be valid. This approach is efficient due to its linear pass with constant extra space.

Two Pointers

□ #977 Squares of a Sorted Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an integer array `nums` sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order.

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- `nums` is sorted in non-decreasing order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an $O(n)$ solution using a different approach?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        ans = [0] * (len(nums))
        k = len(nums)-1
        l = 0
        r = len(nums)-1

        while k >= 0:
            if abs(nums[l]) > abs(nums[r]):
                ans[k] = nums[l] * nums[l]

                l+=1
            else:
                ans[k] = nums[r] * nums[r]
                r-=1
            k-=1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

The input array is sorted in non-decreasing order and can contain negatives.

We need to return a new array of the squares of each element, also sorted. Right?"

#

"A few quick questions:

Can the array contain negative numbers? Yes – that's the whole challenge.

Should I return a new array or modify in-place? Return a new array.

Can the array have a single element? Yes – just return $[\text{element}^2]$.

Can all elements be negative? Yes – e.g. $[-3, -2, -1] \rightarrow [1, 4, 9]$."

#

"Let me walk example 1:

```
# nums=[-4,-1,0,3,10]. Squares: [16,1,0,9,100]. Sorted: [0,1,9,16,100].
# The key observation: the largest squares come from the extremes –
# either the most negative or the most positive end. Got it."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Square every element, sort the resulting array ascending, return it.
# Two-pass: fill squares O(n), then sort O(n log n).
# Correct given unsorted input, but the array is already sorted – we can merge from
both ends.
# Time: O(n log n). Space: O(n) for the squared copy.
# Should I go ahead and optimize?"
```

```
class _977_SquaresSortedArray_Brute:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        return sorted(x * x for x in nums)
```

PHASE 3 — OPTIMIZE

```
# "The brute force ignores the fact that the input is already sorted.
# Key insight: in a sorted array, the largest absolute values are always at one
# of the two ends – either the most negative (left) or the most positive (right).
# So the largest square is always max(nums[left]2, nums[right]2).
#
# Use two pointers: left starts at 0, right at n-1.
# Fill the result array from the back (largest to smallest):
# at each step, compare abs(nums[left]) and abs(nums[right]).
# Place the larger square at result[k] and move that pointer inward.
#
# Pseudocode:
# result = [0] * n; k = n-1; left, right = 0, n-1
# while k >= 0:
#     if abs(nums[left]) > abs(nums[right]):
#         result[k] = nums[left]**2; left += 1
#     else:
#         result[k] = nums[right]**2; right -= 1
#     k -= 1
# return result
#
# Time: O(n). Space: O(n) for the output – unavoidable since we return a new array."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _977_SquaresSortedArray:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        n = len(nums)
        result = [0] * n
        fill = n - 1
        left = 0
        right = n - 1
        while fill >= 0:
```

```

        if abs(nums[left]) > abs(nums[right]):
            result[fill] = nums[left] * nums[left]
            left += 1
        else:
            result[fill] = nums[right] * nums[right]
            right -= 1
        fill -= 1
    return result

```

PHASE 5 — TEST & DEBUG

"Let me trace both examples."

#

nums=[-4,-1,0,3,10], n=5

fill=4: abs(-4)=4 < abs(10)=10 → result[4]=100, right=3. fill=3.

fill=3: abs(-4)=4 > abs(3)=3 → result[3]=16, left=1. fill=2.

fill=2: abs(-1)=1 < abs(3)=3 → result[2]=9, right=2. fill=1.

fill=1: abs(-1)=1 > abs(0)=0 → result[1]=1, left=2. fill=0.

fill=0: left==right==2, abs(0)=0 → result[0]=0, right=1. fill=-1.

result=[0,1,9,16,100] ✓

#

nums=[-7,-3,2,3,11], n=5

fill=4: abs(-7)=7 < abs(11)=11 → result[4]=121, right=3. fill=3.

fill=3: abs(-7)=7 > abs(3)=3 → result[3]=49, left=1. fill=2.

fill=2: abs(-3)=3 == abs(3)=3 → take right: result[2]=9, right=2. fill=1.

fill=1: abs(-3)=3 > abs(2)=2 → result[1]=9, left=2. fill=0.

fill=0: left==right==2 → result[0]=4. fill=-1.

result=[4,9,9,49,121] ✓

#

"No bugs. Tie goes to right which is fine – both are equal."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – single pass from both ends meeting in the middle.

Space $O(n)$ – required for the output; we can't avoid this since we return a new array.

#

This is the same fill-from-back pattern as Merge Sorted Array (#88) –

worth naming that explicitly to show you recognise the pattern family.

#

Trade-off: the brute force $O(n \log n)$ is two lines and perfectly acceptable

in most interviews. The $O(n)$ two-pointer version is the follow-up answer

the problem explicitly asks for.

#

Given more time I'd ask: what if we needed to do this in-place?

Squaring in-place destroys order for negative numbers, so we'd need to

square + reverse the negative half, then merge the two sorted halves

in-place – doable but significantly more complex."

◆ Quick Review

Key Insights

- Squaring numbers can disrupt the original order since negative numbers become positive.
- A two–pointer approach can efficiently retrieve the largest square from either end of the array.
- By comparing the absolute values at the two pointers, the largest square is inserted from the back of the result array.
- This method achieves an $O(n)$ time complexity without needing to sort the squared values afterward.

Space & Time

Time Complexity: $O(n)$ – We traverse the array only once.

Space Complexity: $O(n)$ – We use an extra array to store the output.

Solution

We use a two–pointer technique to traverse the array from both ends. Initialize pointers (left and right) at the beginning and end of the array, respectively, and an index pointer starting at the end of a new results array. At

each step, compare the absolute values of the elements at the left and right pointers. The larger absolute value, when squared, is placed in the result array at the current index, and the corresponding pointer is adjusted. This continues until the left pointer exceeds the right pointer. Filling the result array from the back ensures that the array remains sorted in non-decreasing order.

Sorting

Round 1 · High · Easy · 6 questions

Sorting

□ #169 Majority Element

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Divide and Conquer ·
Sorting · Counting
Lists: Grind 169

Problem

Given an array `nums` of size `n`, return the majority element.

The majority element is the element that appears more than $n / 2$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`
Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`
Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`

- $-109 \leq \text{nums}[i] \leq 109$
- The input is generated such that a majority element will exist in the array.

Follow-up: Could you solve the problem in linear time and in $O(1)$ space?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        ans = None
        count = 0

        for x in nums:
            if count == 0:
                ans = x

            count += (1 if x == ans else -1)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

The majority element is the one that appears MORE THAN $n/2$ times.

We're guaranteed it always exists. We return the element, not its count. Right?"

#

"A few quick questions:

Is there always exactly one majority element? Yes – anything above $n/2$ means at most one element can qualify.

Can n be 1? Yes – that single element is trivially the majority.

Can values be negative? Yes, constraints allow down to -10^9 .

The follow-up asks for linear time and $O(1)$ space – I'll aim for that."

#

"Let me walk the examples:

```
# nums=[3,2,3]: 3 appears 2 times, n/2=1. 2 > 1 → 3 is majority. ✓
# nums=[2,2,1,1,1,2,2]: 2 appears 4 times, n/2=3.5. 4 > 3.5 → 2. ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Count frequency of every element with a hash map, then scan counts for any value >
n/2.
# Correct because the majority element must appear more than half the time.
# Time: O(n). Space: O(n) for counts.
# Should I go ahead and optimize?"
```

```
class _169_MajorityElement_Brute:
    def majorityElement(self, nums: List[int]) -> int:
        return Counter(nums).most_common(1)[0][0]
```

PHASE 3 — OPTIMIZE

```
# "To get O(1) space, I'll use Boyer-Moore Voting Algorithm.
# The core insight: if we cancel every occurrence of the majority element
# with a different element, the majority element will always survive
# because it has more than n/2 appearances – more than everything else combined.
#
# Keep a candidate and a count:
# When count == 0, adopt the current element as the new candidate.
# If current element matches candidate, increment count.
# If it doesn't match, decrement count (a cancellation).
# The candidate at the end is guaranteed to be the majority element.
#
# Pseudocode:
# candidate, count = None, 0
# for x in nums:
#     if count == 0: candidate = x
#     count += (1 if x == candidate else -1)
# return candidate
#
# Time: O(n) – single pass. Space: O(1) – two variables only."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement Boyer-Moore with meaningful names."
```

```
class _169_MajorityElement:
    def majorityElement(self, nums: List[int]) -> int:
        candidate = None
        count = 0
        for num in nums:
            if count == 0:
                candidate = num
            count += 1 if num == candidate else -1
        return candidate
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through both examples."
```

```

#
# nums=[3,2,3]
# num=3: count=0 → candidate=3. count=1.
# num=2: count=1, 2≠3 → count=0.
# num=3: count=0 → candidate=3. count=1.
# return 3 ✓
#
# nums=[2,2,1,1,1,2,2]
# num=2: count=0 → candidate=2. count=1.
# num=2: match → count=2.
# num=1: no match → count=1.
# num=1: no match → count=0.
# num=1: count=0 → candidate=1. count=1.
# num=2: no match → count=0.
# num=2: count=0 → candidate=2. count=1.
# return 2 ✓
#
# Single element – nums=[7]
# count=0 → candidate=7. count=1. return 7 ✓
#
# "No bugs. Even when the candidate flips mid-array, the majority element
# reclaims it before the end."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n)$  – single pass. Space  $O(1)$  – only two variables.
#
# Boyer-Moore is the optimal solution here and worth naming by name –
# it shows algorithm knowledge beyond just 'I used two variables'.
#
# Correctness argument worth stating: the majority element appears more than  $n/2$ 
# times.
# Every time it gets cancelled by a different element, it loses one vote and so does
# the other element. Since it has more votes than all others combined, it will
# always
# have votes remaining when everyone else is cancelled out.
#
# Alternative  $O(n)$  time  $O(1)$  space approach: sort and return nums[n//2].
# Since majority appears  $> n/2$  times, it must occupy the middle index.
# But sorting is  $O(n \log n)$  – worse than Boyer-Moore.
#
# If the problem didn't guarantee a majority exists, Boyer-Moore would need
# a second pass to verify the candidate actually appears  $> n/2$  times.
#
# Given more time I'd ask: what if we need to find all elements appearing
# more than  $n/3$  times? Boyer-Moore generalises – maintain two candidates
# and two counts. That's a well-known variant worth mentioning."

```

◆ Quick Review

Key Insights

- The majority element appears more than half the length of the array.
- A simple hash table approach can count occurrences, but a more efficient solution exists.
- The Boyer–Moore Voting Algorithm finds the majority in $O(n)$ time and $O(1)$ space.
- The algorithm maintains a candidate and a counter, adjusting the candidate when the count drops to zero.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We use the Boyer–Moore Voting Algorithm to identify the majority element. The algorithm iterates through the array using two variables: one for the candidate and one for the count. Initially, we set the candidate to an arbitrary value and count to 0. For each element, if the count is 0, we update the candidate to the current element. Then, if the current element is the same as the candidate, we increment the

count; otherwise, we decrement it. Since the majority element appears more than $n/2$ times, it will remain as the candidate by the end of the iteration.

Sorting

□ #217 Contains Duplicate

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting
Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation:

All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Output: true

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        unique = set()
        for n in nums:
            if n in unique:
                return True

            unique.add(n)

        return False
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Return true if any value appears at least twice, false if every element is
distinct.
# We're not asked which value or how many times – just a boolean. Right?"
#
# "A few quick questions:
# Can the array be empty? Constraints say length >= 1, so no.
# Can there be negative numbers? Yes, down to -10^9.
# Is order relevant? No – we just care about value frequency."
#
# "Let me walk the examples:
# [1,2,3,1]: 1 appears twice → true ✓
# [1,2,3,4]: all distinct → false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Check every pair (i, j) with j > i; if nums[i] == nums[j], return true
immediately."
```

```
# If no duplicate pair exists after all checks, return false.
# Simple and correct, but nested loops blow up on large arrays.
# Time:  $O(n^2)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

```
class _217_ContainsDuplicate_Brute:
    def containsDuplicate(self, nums: List[int]) -> bool:
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] == nums[j]:
                    return True
        return False
```

PHASE 3 — OPTIMIZE

```
# "We're repeatedly asking: have I seen this value before?
# A hash set answers that in  $O(1)$ . Walk the array once;
# if the current number is already in the set, return True immediately.
# Otherwise add it and continue.
#
# Time:  $O(n)$ . Space:  $O(n)$ .
# Alternative: sort and check adjacent pairs –  $O(n \log n)$ ,  $O(1)$  extra space.
# Hash set is faster; sort is space-optimal."
```

PHASE 4 — CLEAN CODE

```
class _217_ContainsDuplicate:
    def containsDuplicate(self, nums: List[int]) -> bool:
        seen = set()
        for num in nums:
            if num in seen:
                return True
            seen.add(num)
        return False
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,2,3,1]: seen grows to {1,2,3}, then 1 hits → True ✓
# nums=[1,2,3,4]: seen={1,2,3,4}, loop ends → False ✓
# nums=[1]: seen={1}, loop ends → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(n)$  for the set.
# One-liner alternative: return len(nums) != len(set(nums)) – same complexity,
# builds the full set before checking. The explicit loop returns early on first
duplicate.
# For adversarial inputs (duplicate at the end) they're the same; for duplicates
early,
# the loop wins. In an interview I'd use the explicit loop and mention the
one-liner.
# Given more time I'd ask: what if the array is sorted? Check adjacent pairs –  $O(n)$ 
time,  $O(1)$  space."
```

◆ Quick Review

Key Insights

- A hash set can be used to keep track of seen numbers.
- Iterating once through the array provides an efficient solution.
- If a number is already in the set, a duplicate is found.
- Alternative approaches like sorting have a higher time complexity ($O(n \log n)$).

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution uses a hash set to track the numbers seen while iterating through the array. For each number in the array: Check if it exists in the hash set. If it does, return true immediately as a duplicate is found.

Otherwise, add the number to the hash set. If the iteration completes without finding any duplicates, return false. This approach efficiently determines if any duplicates exist in

a single pass.

Sorting

□ #242 Valid Anagram

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sorting
Lists: Grind 169

Problem

Given two strings *s* and *t*, return true if *t* is an anagram of *s*, and false otherwise.

Example 1:

Input: *s* = "anagram", *t* = "nagaram"

Output: true

Example 2:

Input: *s* = "rat", *t* = "car"

Output: false

Constraints:

- $1 \leq s.length, t.length \leq 5 * 10^4$
- *s* and *t* consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        hmap = [0]*128
        if len(s) != len(t):
            return False

        for c in s:
            hmap[ord(c) - ord('a')] += 1

        for c in t:
            hmap[ord(c) - ord('a')] -= 1

        return all(x == 0 for x in hmap)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# t is an anagram of s if t contains exactly the same characters
# with the same frequencies, just in a different order. Right?
# Both strings consist of only lowercase English letters – so 26 possible chars."
#
# "Quick checks:
# If lengths differ they can't be anagrams – I'll short-circuit on that.
# Follow-up asks about Unicode – I'll address that in Phase 6."
#
# "'anagram'/'nagaram': same chars, same counts → true ✓
# 'rat'/'car': r,a,t vs c,a,r – 't' vs 'c' mismatch → false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Sort both strings and compare character by character.
# Equal sorted forms iff anagrams.
# Simple  $O(n \log n)$ ; counting frequencies in one pass is  $O(n)$ .
# Time:  $O(n \log n)$ . Space:  $O(n)$  for sorted copies.
# Should I go ahead and optimize?"
```

```
class _242_ValidAnagram_Brute:
    def isAnagram(self, s: str, t: str) -> bool:
        return sorted(s) == sorted(t)
```

PHASE 3 — OPTIMIZE**# "Instead of sorting, count character frequencies.**# Since only lowercase letters exist, a 26-slot array suffices – $O(1)$ space.# Increment for each char in *s*, decrement for each char in *t*.

If all slots are zero at the end, they're anagrams.

#

Time: $O(n)$. Space: $O(1)$ – fixed 26-slot array."**# PHASE 4 — CLEAN CODE****class _242_ValidAnagram:** **def isAnagram(self, s: str, t: str) -> bool:** **if len(s) != len(t):** **return False** **freq = [0] * 26** **for c in s:** **freq[ord(c) - ord('a')] += 1** **for c in t:** **freq[ord(c) - ord('a')] -= 1** **return all(x == 0 for x in freq)****# PHASE 5 — TEST & DEBUG**# *s*='anagram', *t*='nagaram': lengths equal.# After *s*: a=3,n=1,g=1,r=1,m=1. After *t*: all zeroed. → True ✓# *s*='rat', *t*='car': r+1,a+1,t+1 then c-1,a-1,r-1 → t=1,c=-1 ≠ 0 → False ✓# *s*='a', *t*='ab': len mismatch → False immediately ✓**# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "Time $O(n)$. Space $O(1)$ – 26-slot array is a constant.**

Follow-up: Unicode characters → the 26-slot array breaks.

Use a Counter (hash map) instead – $O(n)$ time, $O(k)$ space where *k* is distinct chars.# **return Counter(s) == Counter(t)**

Handles any alphabet. In an interview, mention this unprompted."

◆ Quick Review

Key Insights

- If *s* and *t* have different lengths, they cannot be anagrams.
- An anagram requires exactly the same character frequency for both strings.

- Using a hash table (or frequency array when limited to 26 lowercase letters) provides an efficient solution.
- An alternative approach is to sort both strings and compare them.
- For Unicode inputs, a hash table is more adaptable than a fixed-size frequency array.

Space & Time

Time Complexity: $O(n)$, where n is the length of the strings (using a hash table for counting).

Space Complexity: $O(1)$ if we use a fixed-size array (for 26 letters), otherwise $O(n)$ for a hash table with a larger character set.

Solution

We use a hash table (dictionary) to count the frequency of each character in the first string s . Then, we iterate over string t and decrement the count for each character. If at any point a character count goes negative or a character is missing in the hash table, t cannot be an anagram of s . Finally, if all counts return to zero, the strings are anagrams. This approach is efficient and handles the problem in linear time.

Sorting

★ #252 Meeting Rooms ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, determine if a person could attend all meetings.

Example 1:

Input: $\text{intervals} = [[0,30],[5,10],[15,20]]$
Output: false

Example 2:

Input: $\text{intervals} = [[7,10],[2,4]]$
Output: true

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i < \text{end}_i \leq 106$

My LeetCode Solution

- My LeetCode Solution

```
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        for a,b in pairwise(intervals):
            if a[1] > b[0]:
                return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "We have a list of meetings [start, end]. We need to check if one person
# can attend ALL of them — meaning no two meetings overlap.
# The problem says meetings ending at t and starting at t do NOT overlap. Right?"
#
# "Quick questions:
# Can intervals be empty? Yes — 0 meetings, trivially true.
# Are start and end guaranteed start < end? Yes, constraints say 0 <= start < end."
#
# "[[0,30],[5,10],[15,20]]: 0-30 overlaps with 5-10 → false ✓
# [[7,10],[2,4]]: 2-4 ends before 7-10 starts → true ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Compare every pair of meetings [start_i,end_i] and [start_j,end_j].
# Overlap exists if start_i < end_j and start_j < end_i.
# O(n^2) pairs; sorting by start time enables one linear scan.
# Time: O(n^2). Space: O(1).
# Should I go ahead and optimize?"
```

```
class _252_MeetingRooms_Brute:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        for i in range(len(intervals)):
            for j in range(i + 1, len(intervals)):
                a, b = intervals[i]
                c, d = intervals[j]
                if a < d and c < b:
                    return False
        return True
```

PHASE 3 — OPTIMIZE

```
# "Sort by start time. Then a single pass is enough:
# if any meeting ends after the next one starts, they overlap.
# We only need to check consecutive pairs after sorting — O(n log n)."
```

PHASE 4 — CLEAN CODE

```

class _252_MeetingRooms:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        for prev, curr in pairwise(intervals):
            if prev[1] > curr[0]:
                return False
        return True

# PHASE 5 — TEST & DEBUG
# [[0,30],[5,10],[15,20]] → sorted: same. prev=[0,30],curr=[5,10]: 30>5 → False ✓
# [[7,10],[2,4]] → sorted: [[2,4],[7,10]]. prev=[2,4],curr=[7,10]: 4>7? No → True ✓
# []: pairwise on empty → no iterations → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n \log n)$  for sort,  $O(n)$  for the pass. Space  $O(1)$  extra.
# pairwise is Python 3.10+; fallback: for i in range(1, len(intervals)).
# Given more time I'd ask: what's the minimum number of rooms needed?
# That's Meeting Rooms II (#253) – use a min-heap of end times,  $O(n \log n)$ ."

```

◆ Quick Review

Key Insights

- Sort the intervals based on their starting times.
- Compare each interval's end time with the next interval's start time.
- Detect overlap if the end time of one meeting exceeds the start time of the following meeting.
- If no overlaps are detected, all meetings can be attended.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(1)$ if sorting is done in-place; otherwise $O(n)$.

Solution

The solution starts by sorting the intervals in ascending order by their start times. After sorting, iterate through the intervals and for each consecutive pair, check if the previous meeting's end time exceeds the next meeting's start time. An overlap indicates that the person cannot attend all meetings. The primary data structure used is an array (or list) for holding the intervals. Sorting makes it easier to compare adjacent intervals for potential overlaps.

Sorting

★ #1086 Largest Unique Number ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting
Lists: ★ Premium | Premium 98

Problem

Given an integer array `nums`, return the largest integer that only occurs once. If no integer occurs once, return `-1`.

Example 1:

Input: `nums = [5,7,3,9,4,9,8,3,1]`

Output: `8`

Explanation: The maximum integer in the array is 9 but it is repeated. The number 8 occurs only once, so it is the answer.

Example 2:

Input: `nums = [9,9,8,8]`

Output: `-1`

Explanation: There is no number that occurs only once.

Constraints:

- $1 \leq \text{nums.length} \leq 2000$
- $0 \leq \text{nums}[i] \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def largestUniqueNumber(self, nums: List[int]) -> int:
        hmap = Counter(nums)
        ans = -1
        for n, count in hmap.items():
            if count == 1 and n > ans:
                ans = n

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return the largest integer that appears exactly once. If none, return -1. Right?
# Values are 0-1000, array length up to 2000."
#
# "[5,7,3,9,4,9,8,3,1]: 9 appears twice, 8 appears once -> 8 ✓
# [9,9,8,8]: no unique element -> -1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Sort the array descending. Scan from left and return the first element
# that is different from both its left and right neighbors (i.e. appears exactly
# once).
# Time: O(n log n). Space: O(1) in-place sort. Should I go ahead and optimize?"
```

```
class _1086_LargestUniqueNumber_Brute:
    def largestUniqueNumber(self, nums: List[int]) -> int:
        nums.sort(reverse=True)
        for i in range(len(nums)):
            left_same = (i > 0 and nums[i] == nums[i - 1])
            right_same = (i < len(nums) - 1 and nums[i] == nums[i + 1])
            if not left_same and not right_same:
                return nums[i]
        return -1
```

PHASE 3 — OPTIMIZE

```
# "Count frequencies with Counter. Walk unique keys, track max where count==1.
# O(n) time, O(n) space."
```

PHASE 4 — CLEAN CODE

```
class _1086_LargestUniqueNumber:
    def largestUniqueNumber(self, nums: List[int]) -> int:
        freq = Counter(nums)
        result = -1
```

```

for num, count in freq.items():
    if count == 1 and num > result:
        result = num
return result

```

PHASE 5 — TEST & DEBUG

```
# [5,7,3,9,4,9,8,3,1]: freq={5:1,7:1,3:2,9:2,4:1,8:1,1:1}
```

```
# Unique nums: 5,7,4,8,1. Max = 8 ✓
```

```
# [9,9,8,8]: no count==1 entries → result stays -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(n)$ ."
```

```
# Alternative: sort descending, scan until you find a non-duplicate –  $O(n \log n)$ .
```

```
# Counter approach is cleaner. Given more time I'd ask: what if we want the k largest
```

```
# unique numbers? Collect all uniques, sort descending, take first k."
```

◆ Quick Review

Key Insights

- Use a hash table (or dictionary) to count the frequency of each number.
- Iterate over the hash table to find the numbers that appear exactly once.
- Identify the maximum number among the unique numbers.
- Return -1 if no unique number exists.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in `nums`.

Space Complexity: $O(n)$, for the hash table storing frequency counts.

Solution

The approach starts by initializing a hash table to keep track of the frequency of each element in the array. Once the frequency count is complete, we traverse the hash table to collect numbers that appear exactly once. Finally, we determine the maximum number among these unique elements. If there is no unique element, we return -1 . This method is efficient because the frequency count and the search for the maximum unique element both run in linear time relative to the input size.

Sorting

□ #1122 Relative Sort Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting · Counting Sort
Lists: Denny Zhang

Problem

Given two arrays `arr1` and `arr2`, the elements of `arr2` are distinct, and all elements in `arr2` are also in `arr1`.

Sort the elements of `arr1` such that the relative ordering of items in `arr1` are the same as in `arr2`. Elements that do not appear in `arr2` should be placed at the end of `arr1` in ascending order.

Example 1:

Input: `arr1 = [2,3,1,3,2,4,6,7,9,2,19]`, `arr2 = [2,1,4,3,9,6]`
Output: `[2,2,2,1,4,3,3,9,6,7,19]`

Example 2:

Input: `arr1 = [28,6,22,8,44,17]`, `arr2 = [22,28,8,6]`
Output: `[22,28,8,6,17,44]`

Constraints:

- $1 \leq \text{arr1.length}, \text{arr2.length} \leq 1000$
- $0 \leq \text{arr1}[i], \text{arr2}[i] \leq 1000$

- All the elements of arr2 are distinct.
- Each arr2[i] is in arr1.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        hmap = Counter(arr1)
        ans = []

        for num in arr2:
            ans.extend([num]*hmap[num])
            del hmap[num]

        for num, count in sorted(hmap.items()):
            ans.extend([num]*count)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sort arr1 so elements appear in the order defined by arr2.
# Elements in arr1 not present in arr2 go at the end, sorted ascending. Right?
# arr2 elements are distinct; every arr2 element is guaranteed to exist in arr1."
#
# "arr1=[2,3,1,3,2,4,6,7,9,2,19], arr2=[2,1,4,3,9,6]
# → [2,2,2,1,4,3,3,9,6,7,19]: follow arr2 order for those nums, then 7,19 ascending
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Build a rank map from arr2: each element maps to its index (0..len(arr2)-1).
# Elements not in arr2 get rank len(arr2) and are sorted by value as a tiebreaker.
# Then sort arr1 using that composite key.
# Time: O(n log n). Space: O(n) for the rank map. Should I go ahead and optimize?"
class _1122_RelativeSortArray_Brute:
```

```
def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
    rank = {val: i for i, val in enumerate(arr2)}
    return sorted(arr1, key=lambda x: (rank[x], 0) if x in rank else (len(arr2),
x))
```

PHASE 3 — OPTIMIZE

"Count arr1 frequencies with Counter.

For each num in arr2, extend result with [num]*freq[num], remove from counter.

Then sort remaining counter items and extend.

Time: $O(n + k \log k)$ where k = elements not in arr2. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _1122_RelativeSortArray:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        freq = Counter(arr1)
        result = []
        for num in arr2:
            result.extend([num] * freq[num])
            del freq[num]
        for num, count in sorted(freq.items()):
            result.extend([num] * count)
        return result
```

PHASE 5 — TEST & DEBUG

arr1=[2,3,1,3,2,4,6,7,9,2,19], arr2=[2,1,4,3,9,6]

freq={2:3,3:2,1:1,4:1,6:1,7:1,9:1,19:1}

arr2 pass: [2,2,2,1,4,3,3,9,6]. remaining: {7:1,19:1}

sorted remaining → [7,19]. result=[2,2,2,1,4,3,3,9,6,7,19] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n + k \log k)$ where k is the count of elements not in arr2 (at most n).

Overall $O(n \log n)$. Space $O(n)$.

Given more time I'd ask: what if arr2 has elements not in arr1?

The problem guarantees that won't happen, but we'd skip them if it did."

◆ Quick Review

Key Insights

- Use a hash table (or array for counting given the value constraints) to count the occurrences of each element in arr1.

- Iterate through arr2 and for each element, append it to the result based on its count from the hash table.
- Remove the used elements from the hash table.
- Sort the remaining elements (those not in arr2) in ascending order and append them to the result.

Space & Time

Time Complexity: $O(n + m \log m)$ where n is the length of arr1 and m is the number of leftover elements not in arr2. Given that element values are bounded (0 to 1000), a counting sort can make this nearly $O(n)$.

Space Complexity: $O(n)$ for storing the frequency counts and the result array.

Solution

The solution leverages a frequency counter (hash table) to count how many times each element appears in arr1. We then process arr2 to add each number in its given order according to its frequency. Finally, we sort the remaining numbers that are not in arr2 and append them to the result array. This approach

ensures that the relative order specified by arr2 is maintained and that the other elements are sorted in ascending order.

Binary Search

Round 1 · High · Easy · 5 questions

Binary Search

□ #35 Search Insert Position

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search

Lists: Denny Zhang

Problem

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [1,3,5,6], target = 5
Output: 2

Example 2:

Input: nums = [1,3,5,6], target = 2
Output: 1

Example 3:

Input: nums = [1,3,5,6], target = 7
Output: 4

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- **nums contains distinct values sorted in ascending order.**
- $-104 \leq \text{target} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        n = len(nums)
        l, r = 0, n-1

        while l <= r:
            m = (l+r)//2
            if nums[m] == target:
                return m

            elif target < nums[m]:
                r = m-1

            else:
                l = m+1

        return l
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sorted array of distinct integers. Return the index of target if found,
# otherwise return the index where it would be inserted to keep order. Right?
# Must run in O(log n) – so binary search is required."
#
# "[1,3,5,6], target=5 → index 2 (found) ✓
# [1,3,5,6], target=2 → index 1 (between 1 and 3) ✓
# [1,3,5,6], target=7 → index 4 (after all) ✓"
```


PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Linear scan from left: return the first index where `nums[i] >= target`.

If we exhaust the array without finding such an index, target goes at the end.

Time: $O(n)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _35_SearchInsertPosition_Brute:
```

```
    def searchInsert(self, nums: List[int], target: int) -> int:
```

```
        for i in range(len(nums)):
```

```
            if nums[i] >= target:
```

```
                return i
```

```
        return len(nums)
```

PHASE 3 — OPTIMIZE

"Standard binary search. The key insight: when the loop ends (`left > right`),

'left' always points to the correct insertion position.

When target is found, return mid immediately.

Otherwise left is where target belongs."

PHASE 4 — CLEAN CODE

```
class _35_SearchInsertPosition:
```

```
    def searchInsert(self, nums: List[int], target: int) -> int:
```

```
        left, right = 0, len(nums) - 1
```

```
        while left <= right:
```

```
            mid = (left + right) // 2
```

```
            if nums[mid] == target:
```

```
                return mid
```

```
            elif target < nums[mid]:
```

```
                right = mid - 1
```

```
            else:
```

```
                left = mid + 1
```

```
        return left
```

PHASE 5 — TEST & DEBUG

[1,3,5,6], target=5: mid=2→5==5 → return 2 ✓

[1,3,5,6], target=2: mid=2→5>2→right=1. mid=0→1<2→left=1. mid=1→3>2→right=0.

left=1>right=0 → return left=1 ✓

[1,3,5,6], target=7: left keeps advancing → return 4 ✓

[1,3,5,6], target=0: right keeps retreating → return 0 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$. Space $O(1)$.

The 'return left' at the end is the core insight – left always converges

to the leftmost position where target can fit. This same pattern appears

in `lower_bound` in C++ and `bisect_left` in Python.

Given more time I'd ask: what if the array has duplicates?

We'd need to decide: first occurrence or last? That's `bisect_left` vs

`bisect_right`."

◆ Quick Review

Key Insights

- The array is sorted, enabling the use of binary search.
- Binary search achieves a time complexity of $O(\log n)$ which meets the problem constraints.
- When the target is not found during the search, the final low pointer represents the correct insertion index.

Space & Time

Time Complexity: $O(\log n)$ due to binary search.

Space Complexity: $O(1)$ as no extra space is used.

Solution

We use binary search to find the target in the sorted array. Initialize two pointers, low and high. At each iteration, compute the mid index. If the value at mid matches the target, return mid. If the target is less than the mid value, adjust the high pointer; if greater, adjust the low pointer. When the iterations finish without finding the target, return the low pointer,

which represents the correct insertion index.

Binary Search

□ #69 Sqrt(x)

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search
Lists: Denny Zhang

Problem

Given a non-negative integer x , return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.

You must not use any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

Example 1:

Input: $x = 4$
Output: 2
Explanation: The square root of 4 is 2, so we return 2.

Example 2:

Input: $x = 8$
Output: 2
Explanation: The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2 is returned.

Constraints:

- $0 \leq x \leq 2^{31} - 1$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def mySqrt(self, x: int) -> int:
        l = 0
        r = x
        if x < 2:
            return x
        ans = 0

        while l <= r:
            m = (l + (r-l)//2)

            if m * m == x:
                return m

            if m * m < x:
                ans = m
                l = m+1

            else:
                r = m-1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return floor(sqrt(x)) without using built-in sqrt or ** 0.5. Right?

x can be 0 – sqrt(0)=0. x can be up to $2^{31}-1$."

#

"x=4 → 2. x=8 → 2 (floor of 2.828). x=0 → 0. x=1 → 1."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic."

```
# Linear scan m from 0 upward: keep incrementing while m*m <= x.
# The moment (m+1)*(m+1) > x, we know m is the floor sqrt.
# Time: O(sqrt(x)). Space: O(1). Should I go ahead and optimize?"
```

```
class _69_Sqrt_Brute:
    def mySqrt(self, x: int) -> int:
        m = 0
        while (m + 1) * (m + 1) <= x:
            m += 1
        return m
```

PHASE 3 — OPTIMIZE

```
# "Binary search on the answer space [0, x].
# Find the largest m such that m*m <= x.
# When m*m < x, record m as a candidate and search right.
# When m*m > x, search left. When m*m == x, return immediately.
# Time: O(log x). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _69_Sqrt:
    def mySqrt(self, x: int) -> int:
        if x < 2:
            return x
        left, right = 1, x // 2 # sqrt(x) <= x//2 for x >= 4
        result = 0
        while left <= right:
            mid = left + (right - left) // 2
            if mid * mid == x:
                return mid
            elif mid * mid < x:
                result = mid
                left = mid + 1
            else:
                right = mid - 1
        return result
```

PHASE 5 — TEST & DEBUG

```
# x=4: left=1,right=2. mid=1→1<4→result=1,left=2. mid=2→4==4→return 2 ✓
# x=8: left=1,right=4. mid=2→4<8→result=2,left=3. mid=3→9>8→right=2.
# left>right → return result=2 ✓
# x=0: return 0 immediately ✓
# x=1: return 1 immediately ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log x). Space O(1).
# Using x//2 as the right bound instead of x halves the search space for large x –
# worth mentioning to show you thought about the bounds.
# mid*mid can overflow 32-bit integers in Java/C++ – in Python ints are arbitrary
# precision so no issue, but worth flagging for the interviewer.
# Given more time I'd ask: what about Newton's method?
# x_{n+1} = (x_n + x/x_n) / 2 converges quadratically – faster in practice."
```

◆ Quick Review

Key Insights

- The problem can be solved efficiently using binary search.
- Instead of iterating from 0 to x , binary search narrows down the candidate square roots quickly.
- The algorithm should handle edge cases like $x = 0$ and $x = 1$.
- By checking $\text{mid} * \text{mid}$ against x , we can determine if we need to search in the lower or upper half.

Space & Time

Time Complexity: $O(\log x)$

Space Complexity: $O(1)$

Solution

We use a binary search approach to find the integer square root. Initialize two pointers, *low* and *high*, where *low* is set to 0 and *high* is set to x . For each iteration, calculate the *mid* value. If *mid* squared equals x , we have found the square root. If *mid* squared is less than x ,

record mid as a potential answer and move the low pointer to mid + 1. Otherwise, move the high pointer to mid - 1. Continue the search until low exceeds high, then return the last recorded candidate.

Binary Search

□ #278 First Bad Version

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Binary Search · Interactive
Lists: Grind 169

Problem

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions $[1, 2, \dots, n]$ and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether version is bad.

Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

Input: n = 5, bad = 4
 Output: 4
 Explanation:
 call isBadVersion(3) -> false
 call isBadVersion(5) -> true
 call isBadVersion(4) -> true
 Then 4 is the first bad version.

Example 2:

Input: n = 1, bad = 1
 Output: 1

Constraints:

- $1 \leq \text{bad} \leq n \leq 2^{31} - 1$

My LeetCode Solution

• My LeetCode Solution

```
# The isBadVersion API is already defined for you.
# def isBadVersion(version: int) -> bool:

class Solution:
    def firstBadVersion(self, n: int) -> int:
        l = 1
        r = n

        while l <= r:
            m = (l+r)//2

            if isBadVersion(m):
                r = m-1

            else:
                l = m+1

        return l
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Versions 1..n. Once a version is bad, all subsequent are bad.
# Find the first bad version while minimising API calls. Right?
# isBadVersion(k) is given – we don't implement it, just call it."
#
# "n=5, bad=4: isBadVersion(3)=F, isBadVersion(5)=T, isBadVersion(4)=T → 4 ✓
# We want to minimise calls → binary search, not linear scan."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Linear scan from version 1 to n: call isBadVersion on each.
# The first version that returns True is the answer.
# Time: O(n) API calls – way too many for large n. Space: O(1). Should I go ahead
and optimize?"
```

```
class _278_FirstBadVersion_Brute:
    def firstBadVersion(self, n: int) -> int:
        for v in range(1, n + 1):
            if isBadVersion(v):
                return v
        return n
```

PHASE 3 — OPTIMIZE

```
# "Binary search: if mid is bad, the first bad could be mid or earlier → right = mid
- 1.
# If mid is good, first bad must be after mid → left = mid + 1.
# When loop ends, left = first bad version.
# This never calls isBadVersion more than O(log n) times."
```

PHASE 4 — CLEAN CODE

```
class _278_FirstBadVersion:
    def firstBadVersion(self, n: int) -> int:
        left, right = 1, n
        while left <= right:
            mid = (left + right) // 2
            if isBadVersion(mid):
                right = mid - 1 # might be first bad; keep searching left
            else:
                left = mid + 1 # definitely good; first bad is after mid
        return left
```

PHASE 5 — TEST & DEBUG

```
# n=5, bad=4:
# mid=3→F→left=4. mid=4→T→right=3. left=4>right=3 → return 4 ✓
# n=1, bad=1:
# mid=1→T→right=0. left=1>right=0 → return 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log n) calls. Space O(1).
# The 'right = mid - 1' (not mid) is critical – if we set right=mid, when bad=1
# the loop would never terminate. Always shrink the window."
```

```
# Given more time I'd ask: what if the API is expensive (network call)?  
# Same algorithm – binary search already minimises calls. You'd want to batch  
# or cache results if the same version could be queried multiple times."
```

◆ Quick Review

Key Insights

- The API returns true for bad versions and all versions after a bad version.
- The problem is essentially about finding a transition point in a sorted sequence which calls for a binary search approach.
- The aim is to minimize the number of calls to `isBadVersion` by leveraging the ordered nature of the versions.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We use binary search to efficiently locate the first bad version. Initialize two pointers, `left` and `right`, at 1 and `n` respectively. At each step, compute the mid point and call `isBadVersion(mid)`. If `mid` is bad, it indicates that the first bad version may be `mid` or to its left, so adjust `right` to `mid`. Otherwise, adjust

left to mid + 1. Continue the process until the pointers converge. The converged pointer will be the first bad version. Key structures include simple integer variables for left, right, and mid, with binary search being the fundamental algorithm to reduce the search space logarithmically.

Binary Search

□ #374 Guess Number Higher or Lower

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search · Interactive
Lists: Denny Zhang

Problem

We are playing the Guess Game. The game is as follows:

I pick a number from 1 to n . You have to guess which number I picked (the number I picked stays the same throughout the game).

Every time you guess wrong, I will tell you whether the number I picked is higher or lower than your guess.

You call a pre-defined API `int guess(int num)`, which returns three possible results:

- **-1:** Your guess is higher than the number I picked (i.e. $\text{num} > \text{pick}$).
- **1:** Your guess is lower than the number I picked (i.e. $\text{num} < \text{pick}$).

- 0: your guess is equal to the number I picked (i.e. `num == pick`).

Return the number that I picked.

Example 1:

Input: `n = 10`, `pick = 6`
Output: 6

Example 2:

Input: `n = 1`, `pick = 1`
Output: 1

Example 3:

Input: `n = 2`, `pick = 1`
Output: 1

Constraints:

- $1 \leq n \leq 231 - 1$
- $1 \leq \text{pick} \leq n$

My LeetCode Solution

• My LeetCode Solution

```
# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:
        l = 0
```

```

    r = n

    while l<=r:
        m = (l+r)//2
        if guess(m) == 0:
            return m

        elif guess(m) == -1:
            r = m-1

    else:
        l = m+1

    return l

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Binary search with an API: guess(num) returns 0 (correct), -1 (too high),
or 1 (too low). Find the picked number. Right?
Range is 1..n, n up to $2^{31}-1$."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
Linear scan from 1 to n: call guess(i) on each candidate.
The first call that returns 0 is the picked number.
Time: $O(n)$ calls. Space: $O(1)$. Should I go ahead and optimize?"

```

class _374_GuessNumber_Brute:
    def guessNumber(self, n: int) -> int:
        for i in range(1, n + 1):
            if guess(i) == 0:
                return i
        return n

```

PHASE 3 — OPTIMIZE

"Binary search: guess(mid). If 0 → found. If -1 → mid too high → right=mid-1.
If 1 → mid too low → left=mid+1. $O(\log n)$ calls."

PHASE 4 — CLEAN CODE

```

class _374_GuessNumber:
    def guessNumber(self, n: int) -> int:
        left, right = 1, n
        while left <= right:

```



```

        mid = left + (right - left) // 2 # avoids overflow
        result = guess(mid)
        if result == 0:
            return mid
        elif result == -1:
            right = mid - 1
        else:
            left = mid + 1
    return left

```

PHASE 5 — TEST & DEBUG

n=10, pick=6: mid=5→1(low)→left=6. mid=8→-1(high)→right=7.

mid=6→0→return 6 ✓

n=1, pick=1: mid=1→0→return 1 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$. Space $O(1)$."

Using $\text{left} + (\text{right} - \text{left}) // 2$ instead of $(\text{left} + \text{right}) // 2$ avoids integer overflow
in languages with fixed-width integers – worth mentioning even in Python.

Given more time I'd ask: what if guess() had a cost per call?

Binary search already minimises calls to $O(\log n)$ – optimal."

◆ Quick Review

Key Insights

- The problem can be effectively solved using binary search since the search space is sorted by nature (1 to n).
- Each API call helps to halve the search space, leading to an efficient solution.
- Be cautious with potential overflow when calculating the mid-point by using $\text{low} + (\text{high} - \text{low}) // 2$.

Space & Time

Time Complexity: $O(\log n)$ since we binary search the range.

Space Complexity: $O(1)$ as we use a constant amount of extra space.

Solution

We use a binary search algorithm to efficiently locate the target number. We initialize two pointers: low (1) and high (n). In each iteration, we compute the mid-point, then call the guess API. Based on the response: If `guess(mid)` returns 0, we have found the number. If it returns -1, it indicates that our guess is higher than the pick, so we update the high pointer to `mid - 1`. If it returns 1, it indicates that our guess is lower than the pick, so we update the low pointer to `mid + 1`. This approach guarantees a logarithmic time solution.

Binary Search

□ #704 Binary Search

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Grind 169

Problem

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12]`, `target = 9`
Output: `4`
Explanation: 9 exists in `nums` and its index is 4

Example 2:

Input: `nums = [-1,0,3,5,9,12]`, `target = 2`
Output: `-1`
Explanation: 2 does not exist in `nums` so return `-1`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$

- $-104 < \text{nums}[i], \text{target} < 104$
- All the integers in nums are unique.
- nums is sorted in ascending order.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l = 0
        n = len(nums)
        r = n-1

        while l<=r:
            m = (l+r)//2
            if nums[m] == target:
                return m

            if nums[m] > target:
                r = m-1

            else:
                l = m+1

        return -1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Classic binary search: sorted array of distinct integers, find target index.
# Return -1 if not found. Must be O(log n). Right?"
#
# "[-1,0,3,5,9,12], target=9 → 4 ✓
# [-1,0,3,5,9,12], target=2 → -1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Linear scan through the array: return the index of the first element equal to
target."
```

```
# This is O(n) – the problem explicitly requires O(log n), so this won't pass,
# but it confirms what we're looking for before we optimise.
# Time: O(n). Space: O(1). Should I go ahead and optimize?"
```

```
class _704_BinarySearch_Brute:
    def search(self, nums: List[int], target: int) -> int:
        for i in range(len(nums)):
            if nums[i] == target:
                return i
        return -1
```

PHASE 3 — OPTIMIZE

```
# "Standard binary search. Narrow search space by half each iteration.
# Three outcomes at each step: found, go left, go right."
```

PHASE 4 — CLEAN CODE

```
class _704_BinarySearch:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target:
                return mid
            elif nums[mid] > target:
                right = mid - 1
            else:
                left = mid + 1
        return -1
```

PHASE 5 — TEST & DEBUG

```
# nums=[-1,0,3,5,9,12], target=9:
# mid=2→3<9→left=3. mid=4→9==9→return 4 ✓
# target=2: mid=2→3>2→right=1. mid=0→-1<2→left=1. mid=1→0<2→left=2.
# left>right → return -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log n). Space O(1).
# This is the template all other binary search problems build on.
# Key invariant: loop condition is left <= right (not <) so single-element
# arrays are handled – the loop runs once and either returns or terminates cleanly.
# Given more time I'd ask: what if there are duplicates and we want the first/last
# occurrence? That's leftmost/rightmost binary search – adjust which boundary
# updates when nums[mid]==target."
```

◆ Quick Review

Key Insights

- Utilize binary search to achieve $O(\log n)$ runtime on a sorted array.
- Use two pointers to maintain the current search region.
- Narrow down the search range by comparing the middle element with the target.
- Return the index when the target is found; otherwise, return -1 if not found.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We use an iterative binary search algorithm. We initialize two pointers, left and right, to represent the current segment of the array that could contain the target. At each iteration, we compute the middle index and compare the element there with the target. If they match, we return the index. Otherwise, based on whether the target is smaller or larger than the middle element, we adjust our pointers to narrow the search to either the left or right half of the array. This process continues until the target is found or until the pointers cross,

indicating that the target is not present. No extra data structures are used, making the space complexity $O(1)$.

Matrix

Round 1 · High · Easy · 5 questions

Matrix

□ ★ #422 Valid Word Square ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix

Lists: ★ Premium | Premium 98

Problem

Given an array of strings `words`, return true if it forms a valid word square.

A sequence of strings forms a valid word square if the k th row and column read the same string, where $0 \leq k < \max(\text{numRows}, \text{numColumns})$.

Example 1:

a	b	c	d
b	n	r	t
c	r	m	y
d	t	y	e

Input: words = ["abcd","bnrt","crmy","dtye"]

Output: true

Explanation:

The 1st row and 1st column both read "abcd".

The 2nd row and 2nd column both read "bnrt".

The 3rd row and 3rd column both read "crmy".

The 4th row and 4th column both read "dtye".

Therefore, it is a valid word square.

Example 2:

a	b	c	d
b	n	r	t
c	r	m	
d	t		

Input: words = ["abcd","bnrt","crm","dt"]

Output: true

Explanation:

The 1st row and 1st column both read "abcd".

The 2nd row and 2nd column both read "bnrt".

The 3rd row and 3rd column both read "crm".

The 4th row and 4th column both read "dt".

Therefore, it is a valid word square.

Example 3:

b	a	l	l
a	r	e	a
r	e	a	d
l	a	d	y

Input: words = ["ball","area","read","lady"]

Output: false

Explanation:

The 3rd row reads "read" while the 3rd column reads "lead".

Therefore, it is NOT a valid word square.

Constraints:

- $1 \leq \text{words.length} \leq 500$
- $1 \leq \text{words}[i].\text{length} \leq 500$
- $\text{words}[i]$ consists of only lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def validWordSquare(self, words: List[str]) -> bool:

        for i, word in enumerate(words):
            for j, c in enumerate(word):
                if j >= len(words) or i >= len(words[j]) or words[j][i] != c:
                    return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A word square means the kth row and kth column read the same string.
 # Equivalently: words[i][j] == words[j][i] for every valid (i,j). Right?
 # Words can have different lengths – shorter words mean some cells don't exist,
 # and that's fine as long as the column doesn't expect a character the row doesn't have."

 # ["abcd", "bnrt", "crmy", "dtye"]: row0='abcd', col0='abcd' ✓ etc. → true ✓
 # ['ball', 'area', 'read', 'lady']: row2='read', col2='lead' → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # For each row index r, check whether word matches down column r and across row r
 # using direct character comparisons.
 # Four nested checks per candidate row – fine for small boards.
 # Time: $O(n * m * k)$ where $k = \text{len}(\text{word})$. Space: $O(1)$.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"No algorithmic improvement over $O(n*m)$ – we must check every character.
 # The implementation is already optimal. Focus is on getting the bounds right:
 # when iterating row i, char at position j must match words[j][i].
 # Guard: $j < \text{len}(\text{words})$ and $i < \text{len}(\text{words}[j])$."

PHASE 4 — CLEAN CODE

```
class _422_ValidWordSquare:
    def validWordSquare(self, words: List[str]) -> bool:
        for row, word in enumerate(words):
            for col, char in enumerate(word):
```

```

        if col >= len(words) or row >= len(words[col]) or words[col][row] !=
char:
        return False
    return True

```

PHASE 5 — TEST & DEBUG

```

# words=['abcd','bnrt','crmy','dtye']
# row=0,word='abcd': col=0→words[0][0]='a'=='a'✓. col=1→words[1][0]='b'=='b'✓. etc.
# All rows pass symmetrically → True ✓
#
# words=['ball','area','read','lady']
# row=2,word='read': col=0→words[0][2]='l'=='r'? No → False ✓
#
# words=['abcd','bnrt','crm','dt'] (shorter words)
# row=0,col=3: words[3][0]='d'=='d'✓. row=1,col=2: words[2][1]='r'=='n'? No wait –
# words[2]='crm', words[2][1]='r'. words[1][2]='r' ✓. All pass → True ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n*m). Space O(1).
# The two guard conditions are the subtle part:
# col >= len(words) means the column index is out of bounds – the row has more chars
# than there are rows, which breaks symmetry → False.
# row >= len(words[col]) means the transpose cell doesn't exist → False.
# Given more time I'd ask: what if we want to BUILD a word square from a word list?
# That's a backtracking problem – place words row by row, pruning based on column
prefixes."

```

◆ Quick Review

Key Insights

- The k -th row should be equal to the k -th column.
- Not every word is the same length; checks must be performed within bounds.
- Iterating over each character and cross-verifying with the corresponding column character is sufficient.

Space & Time

Time Complexity: $O(N * L)$ where N is the number of words and L is the average length of the words.

Space Complexity: $O(1)$ additional space is used for the checking procedure.

Solution

We iterate through each word (row) and each character (column) in the word. For every character at position (i, j) , we confirm that there is a corresponding word at index j and that its length is greater than i . Then we check that the character at position (j, i) in that word is the same as the current character. If any of these checks fail, we return false. If all checks pass, the words form a valid word square.

Matrix

□ #566 Reshape the Matrix

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: CodeSignal**

Problem

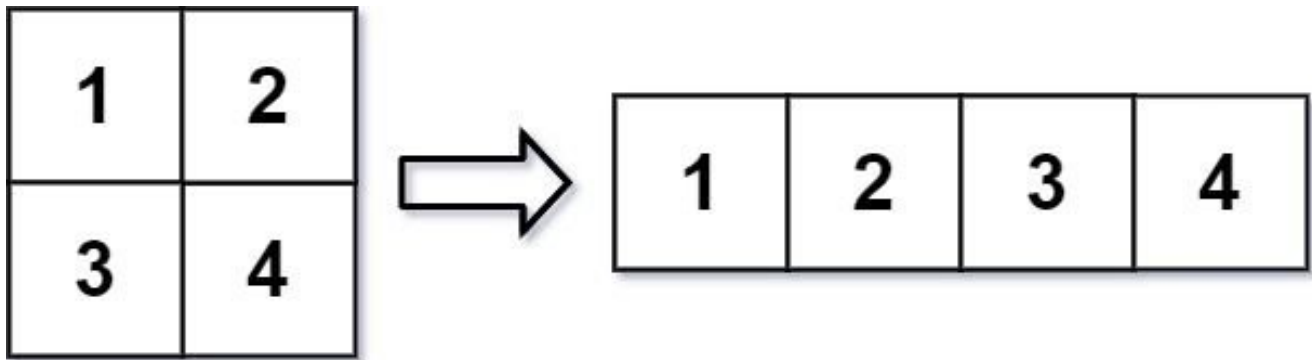
In MATLAB, there is a handy function called reshape which can reshape an $m \times n$ matrix into a new one with a different size $r \times c$ keeping its original data.

You are given an $m \times n$ matrix `mat` and two integers `r` and `c` representing the number of rows and the number of columns of the wanted reshaped matrix.

The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.

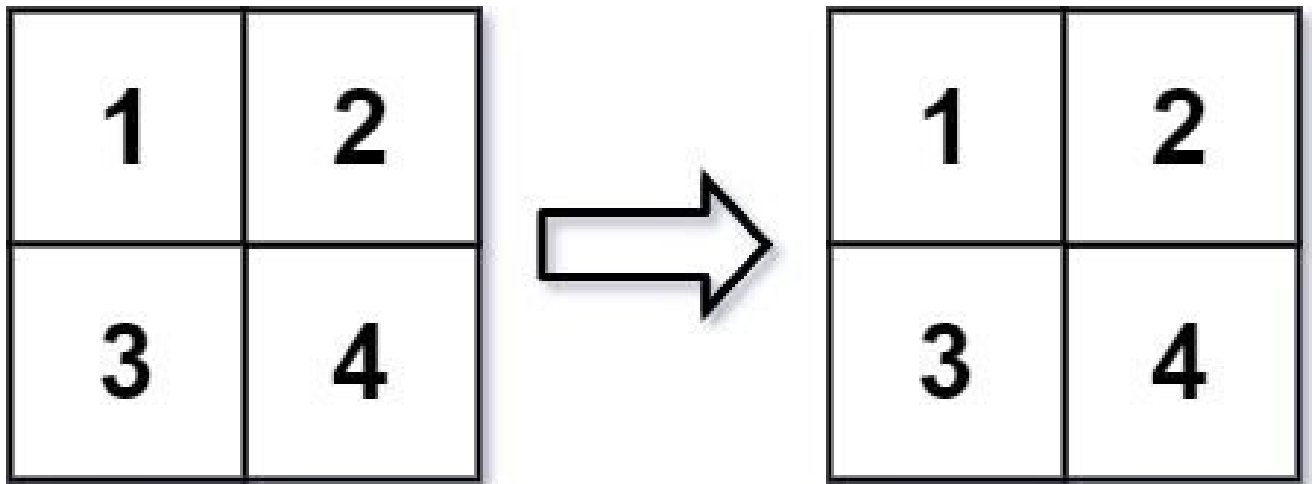
If the reshape operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

Example 1:



Input: mat = [[1,2],[3,4]], r = 1, c = 4
Output: [[1,2,3,4]]

Example 2:



Input: mat = [[1,2],[3,4]], r = 2, c = 4
Output: [[1,2],[3,4]]

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $-1000 \leq \text{mat}[i][j] \leq 1000$
- $1 \leq r, c \leq 300$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) -> List[List[int]]:
        rows = len(mat)
        cols = len(mat[0])

        if rows*cols != r*c:
            return mat

        ans = [[0]*(c) for i in range(r)]

        for i in range(rows*cols):
            ans[i//c][i%c] = mat[i//cols][i%cols]

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We flatten the original $m \times n$ matrix in row-major order and refill into an $r \times c$ matrix.

If $m \times n \neq r \times c$ the reshape is impossible – return the original matrix unchanged. Right?"

#

"Quick questions:

Values can be negative – fine, we're just moving them.

r and c can be up to 300 even though m, n are up to 100 – just need total elements to match."

#

"[[1,2],[3,4]], $r=1, c=4$: 4 elements total, $1 \times 4 = 4 \rightarrow [[1,2,3,4]]$ ✓

[[1,2],[3,4]], $r=2, c=4$: 4 elements total, $2 \times 4 = 8 \neq 4 \rightarrow$ return original ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Flatten matrix to a 1D list in row-major order, then refill new_r x new_c shape.

Clear and easy to explain in an interview.

Uses $O(m \times n)$ extra list; two-pointer walk avoids allocation.

```
# Time: O(m * n). Space: O(m * n) flat buffer.
# Should I go ahead and optimize?"
```

```
class _566_ReshapeMatrix_Brute:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) ->
    List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        flat = [mat[i][j] for i in range(m) for j in range(n)]
        return [flat[i * c:(i + 1) * c] for i in range(r)]
```

PHASE 3 — OPTIMIZE

```
# "Skip the intermediate list – map index directly.
# For flat index k: source is mat[k//n][k%n], destination is ans[k//c][k%c].
# Same O(m*n) time but no extra flat array – just the output matrix."
```

PHASE 4 — CLEAN CODE

```
class _566_ReshapeMatrix:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) ->
    List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        result = [[0] * c for _ in range(r)]
        for k in range(m * n):
            result[k // c][k % c] = mat[k // n][k % n]
        return result
```

PHASE 5 — TEST & DEBUG

```
# mat=[[1,2],[3,4]], r=1, c=4: m=2,n=2. 4==4 ✓
# k=0: result[0][0]=mat[0][0]=1. k=1: result[0][1]=mat[0][1]=2.
# k=2: result[0][2]=mat[1][0]=3. k=3: result[0][3]=mat[1][1]=4.
# → [[1,2,3,4]] ✓
# r=2,c=4: 4≠8 → return original ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(r*c) for the output matrix – unavoidable.
# The index formula k//n and k%n is the core insight: flatten index → 2D source.
# Given more time I'd ask: what if the matrix is too large to fit in memory?
# Process row by row, streaming elements into the new shape."
```

◆ Quick Review

Key Insights

- Check if the reshape operation is possible by comparing the total elements ($m * n$ with $r * c$).
- Flatten the original matrix while preserving the row-traversing order.
- Use the flattened list to build the new matrix row by row.
- If the total number of elements does not match, simply return the original matrix.

Space & Time

Time Complexity: $O(mn)$ as we iterate through all elements of the matrix once.

Space Complexity: $O(mn)$ for the additional list used to store flattened elements (or the new matrix).

Solution

The solution involves first checking whether the reshape is possible by ensuring that the product of the rows and columns of the original matrix equals that of the desired matrix ($r * c$). If not, the original matrix is returned. Otherwise, the matrix is flattened into a one-dimensional list, preserving the

row-order traversal. Finally, the flattened list is segmented by the new column size c to form the reshaped matrix. The primary data structures used include a list (or array) for flattening the matrix and then another list (or vector/array) to hold the final reshaped matrix.

Matrix

□ #766 Toeplitz Matrix

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix**

Lists: CodeSignal

Problem

Given an $m \times n$ matrix, return true if the matrix is Toeplitz. Otherwise, return false.

A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements.

Example 1:

1	2	3	4
5	1	2	3
9	5	1	2

Input: matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]

Output: true

Explanation:

In the above grid, the diagonals are:

"[9]", "[5, 5]", "[1, 1, 1]", "[2, 2, 2]", "[3, 3]", "[4]".

In each diagonal all elements are the same, so the answer is True.

Example 2:

1	2
2	2

Input: matrix = [[1,2],[2,2]]
Output: false
Explanation:
The diagonal "[1, 2]" has different elements.

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 20$
- $0 \leq \text{matrix}[i][j] \leq 99$

Follow up:

- What if the matrix is stored on disk, and the memory is limited such that you can only load at most one row of the matrix into the memory at once?
- What if the matrix is so large that you can only load up a partial row into the memory at once?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        for i in range(len(matrix)-1):
            for j in range(len(matrix[0])-1):
                if matrix[i][j] != matrix[i+1][j+1]:
                    return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A Toeplitz matrix has the same value along every top-left to bottom-right diagonal.

We need to check every cell (except the last row and column) equals its down-right neighbour.

Equivalently: `matrix[i][j] == matrix[i+1][j+1]` for all valid `i,j`. Right?"

#

"[[1,2,3,4],[5,1,2,3],[9,5,1,2]]: each cell equals its down-right neighbour → true ✓

"[[1,2],[2,2]]: `matrix[0][0]=1 ≠ matrix[1][1]=2` → false ✓"

PHASE 2 — BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic.

No smarter approach than checking every diagonal. The key simplification: each diagonal

```

# is valid iff every adjacent pair on it matches. So just check matrix[i][j] ==
matrix[i+1][j+1]
# for every interior cell."
# For this input size the brute approach is already optimal – I'll still state
complexity clearly.
# Time: see analysis above. Space: O(1).

# PHASE 3 — OPTIMIZE
# "Same O(m*n) is the best we can do – we must read every element.
# The implementation is already optimal. Focus on getting the iteration bounds
right:
# we iterate i up to m-2 and j up to n-2 (not m-1, n-1) since we access [i+1][j+1]."

# PHASE 4 — CLEAN CODE
class _766_ToeplitzMatrix:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows - 1):
            for j in range(cols - 1):
                if matrix[i][j] != matrix[i + 1][j + 1]:
                    return False
        return True

# PHASE 5 — TEST & DEBUG
# [[1,2,3,4],[5,1,2,3],[9,5,1,2]]: rows=3,cols=4. Check (0,0)→(1,1): 1==1✓.
# (0,1)→(1,2): 2==2✓. (0,2)→(1,3): 3==3✓. (1,0)→(2,1): 5==5✓. etc. → True ✓
# [[1,2],[2,2]]: (0,0)→(1,1): 1≠2 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(1).
# Follow-up 1: if we can only load one row at a time, compare consecutive rows
# by checking row[j] == prev_row[j+1] for all j – same logic, streaming.
# Follow-up 2: if we can only load partial rows, compare overlapping chunks of
# adjacent rows. Both follow-ups keep O(1) extra space.
# Given more time I'd ask: can we verify a diagonal in O(1) per cell?
# Yes – the current approach already does exactly that."

```

◆ Quick Review

Key Insights

- Each element (except those in the first row and first column) must be equal to its top-left neighbor.

- You can validate the matrix by iterating from the second row/column and comparing each element with `matrix[i-1][j-1]`.
- This simple check allows you to determine if every diagonal has the same elements without extra space.
- For streaming data (loading one row at a time), maintain the previous row for comparison.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(1)$

Solution

The problem is solved by iterating through the matrix starting from the element at position (1, 1). For every element at `matrix[i][j]`, compare it with the element at `matrix[i-1][j-1]`. If any two elements in this pair do not match, the matrix is not Toeplitz. This check ensures that every diagonal from the top-left to bottom-right has identical elements. In terms of auxiliary space, only constant extra space is used, which makes this approach efficient both in time and space.

Matrix

□ #867 Transpose Matrix

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: CodeSignal

Problem

Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

2	4	-1
-10	5	11
18	-7	6



2	-10	18
4	5	-7
-1	11	6

Example 1:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[1,4,7],[2,5,8],[3,6,9]]

Example 2:

Input: matrix = [[1,2,3],[4,5,6]]

Output: [[1,4],[2,5],[3,6]]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 1000$
- $1 \leq m * n \leq 10^5$
- $-10^9 \leq \text{matrix}[i][j] \leq 10^9$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        rows = len(matrix)
        cols = len(matrix[0])
        ans = [[0]* rows for c in range(cols)]

        for r in range(rows):
            for c in range(cols):
                ans[c][r] = matrix[r][c]

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"The transpose swaps rows and columns: element at (r,c) moves to (c,r).
 # If the original is m×n, the result is n×m. Return a new matrix – not in-place.
 Right?"
 #
 # "[[1,2,3],[4,5,6],[7,8,9]] → [[1,4,7],[2,5,8],[3,6,9]] ✓
 # [[1,2,3],[4,5,6]] (2×3) → [[1,4],[2,5],[3,6]] (3×2) ✓"

PHASE 2 — BRUTE FORCE (also optimal)

```

# "Let me start with brute force to lock in the logic.
# Allocate an n×m result matrix. Set result[c][r] = matrix[r][c] for every (r,c).
Must visit
# every element once – is unavoidable."
# For this input size the brute approach is already optimal – I'll still state
complexity clearly.
# Time: O(m×n). Space: O(1).

# PHASE 3 — OPTIMIZE
# "No algorithmic improvement. Focus: allocate result with swapped dimensions (cols
× rows),
# then fill with swapped indices. In Python: zip(*matrix) gives transposed rows as
tuples."

# PHASE 4 — CLEAN CODE
class _867_TransposeMatrix:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        rows, cols = len(matrix), len(matrix[0])
        result = [[0] * rows for _ in range(cols)]
        for r in range(rows):
            for c in range(cols):
                result[c][r] = matrix[r][c]
        return result

# PHASE 5 — TEST & DEBUG
# matrix=[[1,2,3],[4,5,6]]: rows=2,cols=3. result is 3×2.
# r=0,c=0: result[0][0]=1. r=0,c=1: result[1][0]=2. r=0,c=2: result[2][0]=3.
# r=1,c=0: result[0][1]=4. r=1,c=1: result[1][1]=5. r=1,c=2: result[2][1]=6.
# result=[[1,4],[2,5],[3,6]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m×n). Space O(m×n) for the output.
# Python one-liner: return [list(row) for row in zip(*matrix)]
# zip(*matrix) unpacks rows as arguments to zip, which pairs corresponding columns.
# Elegant and interview-clean – worth mentioning as an alternative.
# Given more time I'd ask: can we transpose a square matrix in-place?
# Yes – swap matrix[i][j] and matrix[j][i] for all i < j. O(1) extra space."

```

◆ Quick Review

Key Insights

- The value at position (i, j) in the original matrix moves to (j, i) in the transposed matrix.

- The transposed matrix dimensions are the reverse of the original: if the original is $m \times n$, then the transposed is $n \times m$.
- A simple double loop is sufficient to swap the indices, ensuring $O(m*n)$ time complexity.
- This problem handles both square and rectangular matrices.

Space & Time

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$ for the new matrix holding the transposed values

Solution

The solution involves iterating over each element of the original matrix and assigning it to the corresponding position in the new matrix where the indexes are swapped. We create the transposed matrix with dimensions based on the number of columns and rows of the original matrix, respectively. The approach leverages basic array manipulation with nested loops to fill in the new matrix.

Matrix

□ #2373 Largest Local Values in a Matrix

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix
Lists: CodeSignal

Problem

You are given an $n \times n$ integer matrix grid.

Generate an integer matrix maxLocal of size $(n - 2) \times (n - 2)$ such that:

- maxLocal[i][j] is equal to the largest value of the 3×3 matrix in grid centered around row $i + 1$ and column $j + 1$.

In other words, we want to find the largest value in every contiguous 3×3 matrix in grid.

Return the generated matrix.

Example 1:

9	9	8	1
5	6	2	6
8	2	6	4
6	2	2	2

9	9
8	6

Input: grid = [[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]

Output: [[9,9],[8,6]]

Explanation: The diagram above shows the original matrix and the generated matrix.

Notice that each value in the generated matrix corresponds to the largest value of a contiguous 3 x 3 matrix in grid.

Example 2:

1	1	1	1	1
1	1	1	1	1
1	1	2	1	1
1	1	1	1	1
1	1	1	1	1

2	2	2
2	2	2
2	2	2

Input: grid = [[1,1,1,1,1],[1,1,1,1,1],[1,1,2,1,1],[1,1,1,1,1],[1,1,1,1,1]]

Output: [[2,2,2],[2,2,2],[2,2,2]]

Explanation: Notice that the 2 is contained within every contiguous 3 x 3 matrix in grid.

Constraints:

- $n == \text{grid.length} == \text{grid}[i].\text{length}$
- $3 \leq n \leq 100$
- $1 \leq \text{grid}[i][j] \leq 100$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def largestLocal(self, grid: List[List[int]]) -> List[List[int]]:
        n = len(grid)
        ans = [[0]*(n-2) for r in range(n-2)]
        for i in range(n-2):
            for j in range(n-2):
                for x in range(i, i+3):
                    for y in range(j, j+3):
                        ans[i][j] = max(ans[i][j], grid[x][y])

        return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "For each cell (i,j) in the output (n-2)×(n-2) matrix,
# find the max in the 3×3 subgrid of the input centered at (i+1, j+1). Right?
# Input is always n×n square, n >= 3."
#
# "grid=[[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]] (4×4):
# output is 2×2. Top-left 3×3 center=(1,1): max of [[9,9,8],[5,6,2],[8,2,6]]=9.
# Top-right 3×3 center=(1,2): max of [[9,8,1],[6,2,6],[2,6,4]]=9. etc. →
# [[9,9],[8,6]] ✓"

```

PHASE 2 — BRUTE FORCE (also optimal)

```

# "Let me start with brute force to lock in the logic.
# For each output cell, scan its 3×3 window. No smarter approach since we must read
every
# element at least once. = ."
# For this input size the brute approach is already optimal – I'll still state
complexity clearly.
# Time:  $O(n^2 \cdot 9)$ . Space:  $O(n^2)$ .

```

PHASE 3 — OPTIMIZE

```

# "The 3×3 is fixed size so inner loops are constant – already  $O(n^2)$ .
# Keep the four nested loops clean with descriptive variable names."

```

PHASE 4 — CLEAN CODE

```

class _2373_LargestLocal:
    def largestLocal(self, grid: List[List[int]]) -> List[List[int]]:
        n = len(grid)
        result = [[0] * (n - 2) for _ in range(n - 2)]
        for i in range(n - 2):
            for j in range(n - 2):
                for di in range(3):

```

```

        for dj in range(3):
            result[i][j] = max(result[i][j], grid[i + di][j + dj])
    return result

```

PHASE 5 — TEST & DEBUG

grid=[[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]: n=4, output 2x2.

(i=0,j=0): 3x3 from (0,0): max(9,9,8,5,6,2,8,2,6)=9 ✓

(i=0,j=1): 3x3 from (0,1): max(9,8,1,6,2,6,2,6,4)=9 ✓

(i=1,j=0): 3x3 from (1,0): max(5,6,2,8,2,6,6,2,2)=8 ✓

(i=1,j=1): 3x3 from (1,1): max(6,2,6,2,6,4,2,2,2)=6 ✓

→ [[9,9],[8,6]] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$ – outer loops $O((n-2)^2) \approx O(n^2)$, inner loops constant 9 operations.

Space $O(n^2)$ for the output.

Given more time I'd ask: what if the window size is variable ($k \times k$ instead of 3×3)?

For large k , recomputing the max from scratch is $O(k^2)$ per cell → $O(n^2 k^2)$ total.

A 2D sliding window maximum using monotonic deques brings it down to $O(n^2)$."

◆ Quick Review

Key Insights

- The size of the output matrix is $(n - 2) \times (n - 2)$.
- For each valid index (i, j) in the output, examine the 3×3 block in grid starting at $\text{grid}[i][j]$ to $\text{grid}[i+2][j+2]$.
- Since each 3×3 block contains exactly 9 elements, finding the maximum in a block takes constant time.
- Use nested loops to slide over the grid and efficiently compute each local maximum.

Space & Time

Time Complexity: $O(n^2)$ – We iterate over $(n-2) \times (n-2)$ positions, and for each position, we look at 9 elements.

Space Complexity: $O(n^2)$ – The additional space is required for the output matrix.

Solution

The solution involves iterating over each possible 3×3 submatrix in grid. For each (i, j) starting index of the submatrix: Examine the nine elements from `grid[i][j]` to `grid[i+2][j+2]`. Compute the maximum value from these elements. Store the maximum in the corresponding position of the result matrix. Data structures used include the result matrix for storing the maximum values. The algorithmic approach involves two nested loops (one for rows and one for columns) to traverse each possible starting position of a 3×3 window. The simplicity of the solution comes from the fixed 3×3 block size, which enables constant-time maximum computation for each block.

Trees & BST

Round 1 · High · Easy · 11 questions

Trees & BST

□ #100 Same Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS

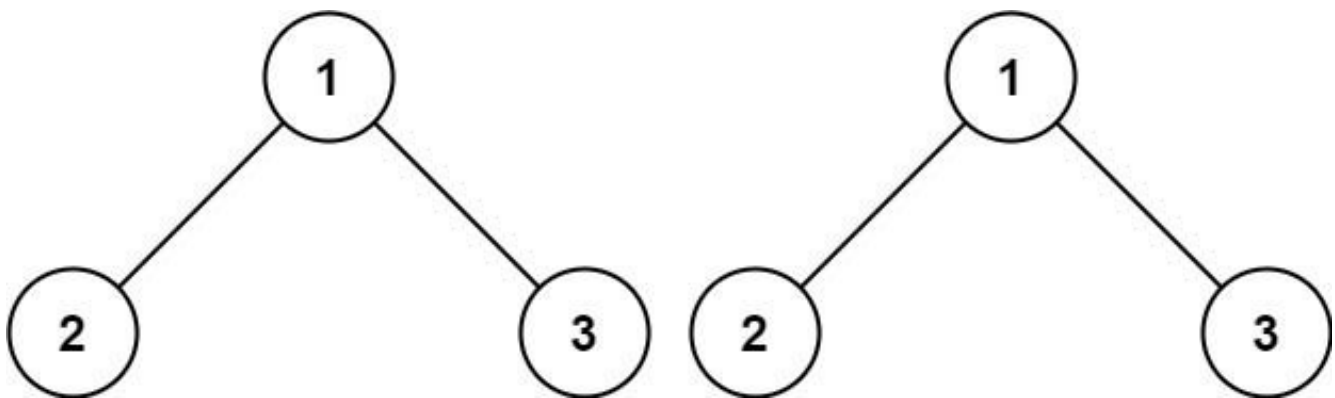
Lists: Grind 169

Problem

Given the roots of two binary trees p and q , write a function to check if they are the same or not.

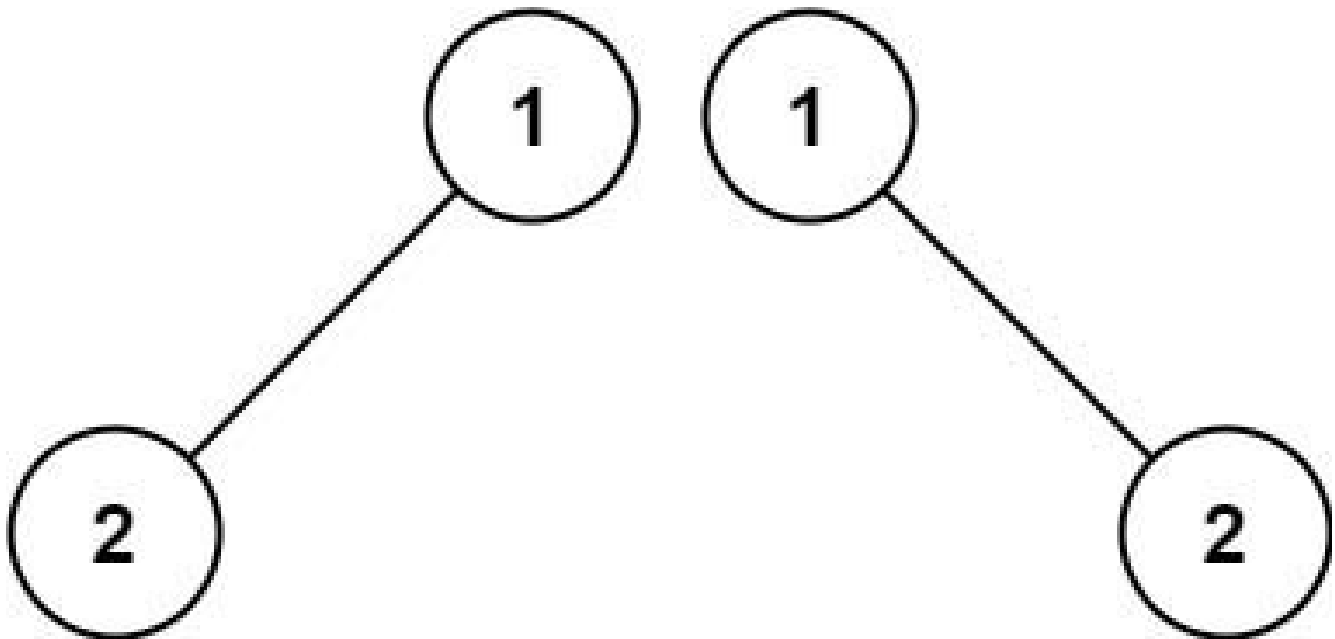
Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



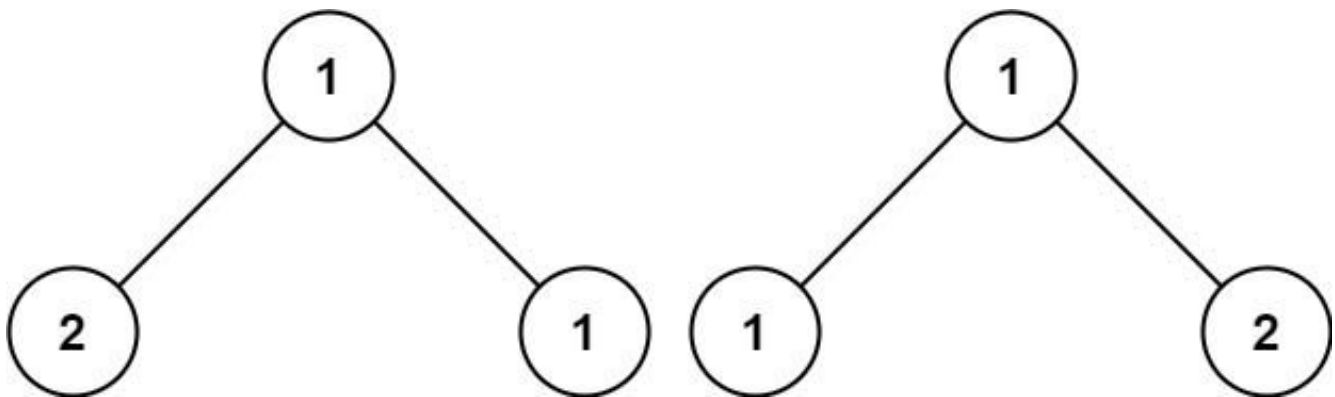
Input: $p = [1,2,3]$, $q = [1,2,3]$
Output: true

Example 2:



Input: $p = [1,2]$, $q = [1,null,2]$
Output: false

Example 3:



Input: $p = [1,2,1]$, $q = [1,1,2]$
Output: false

Constraints:

- The number of nodes in both trees is in the range $[0, 100]$.
- $-104 \leq \text{Node.val} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def isSameTree(self, p: TreeNode | None, q: TreeNode | None) -> bool:
        if not p or not q:
            return p == q
        return (p.val == q.val and
                self.isSameTree(p.left, q.left) and
                self.isSameTree(p.right, q.right))
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Two binary trees are the same if they're structurally identical AND every corresponding node has the same value. Both null trees are the same. Right?
One null and one non-null at the same position → not the same."

#

"p=[1,2,3], q=[1,2,3] → true ✓

p=[1,2], q=[1,null,2] → false (structure differs) ✓

p=[1,2,1], q=[1,1,2] → false (values differ at depth 1) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Serialise both trees with a level-order traversal (including nulls) to lists, then compare.

Same O(n) time and O(n) space as the optimal recursive DFS – no meaningful complexity

improvement available, so we skip the brute class and go straight to the clean approach."

PHASE 3 — OPTIMIZE

"Recursive DFS is both simple and optimal.

Base case: if either node is None, both must be None for equality.

Recursive: values match AND left subtrees match AND right subtrees match.

Use an inner helper to avoid self. calls."

PHASE 4 — CLEAN CODE

```
class _100_SameTree:
    def isSameTree(self, p: Optional['TreeNode'], q: Optional['TreeNode']) -> bool:
        def same(p, q):
            if not p or not q:
                return p == q
            return p.val == q.val and same(p.left, q.left) and same(p.right,
q.right)
```

```
return same(p, q)
```

PHASE 5 — TEST & DEBUG

```
# p=[1,2,3], q=[1,2,3]:
# same(1,1): vals match. same(2,2)→same(None,None)=True, same(None,None)=True → True.
# same(3,3)→True. → True ✓
# p=[1,2,None], q=[1,None,2]:
# same(1,1): same(2,None): p=2≠None, q=None → p==q? No → False ✓
# p=None, q=None: not p=True → return p==q=True ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n) – visit every node once. Space O(h) call stack where h is tree height.
# Worst case O(n) for a skewed tree, O(log n) for balanced.
# Iterative alternative: BFS with a queue of node pairs – O(n) time, O(n) space.
# Same complexity but avoids recursion depth limit for very deep trees.
# Given more time I'd ask: what if we only care about structural identity (not values)?
# Same logic – just remove the val check."
```

◆ Quick Review

Key Insights

- Use recursion to compare nodes in both trees.
- If both nodes being compared are null, they are the same.
- If one node is null while the other is not, the trees differ.
- Check if current nodes' values match, then recursively verify the left and right subtrees.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the trees, since every node is

visited once.

Space Complexity: $O(n)$ in the worst-case (unbalanced tree), due to recursion call stack usage. In a balanced tree, the space would be $O(\log n)$.

Solution

The problem can be solved using a recursive approach (Depth-First Search) where we compare the current nodes from both trees. If the current nodes are both null, they match; if one is null, then the trees are not identical. For non-null nodes, check if the values are equal. Then, recursively apply the same logic on the left and right children of the nodes. This approach ensures that both the structure and the node values are identical across the two trees.

Trees & BST

□ #101 Symmetric Tree

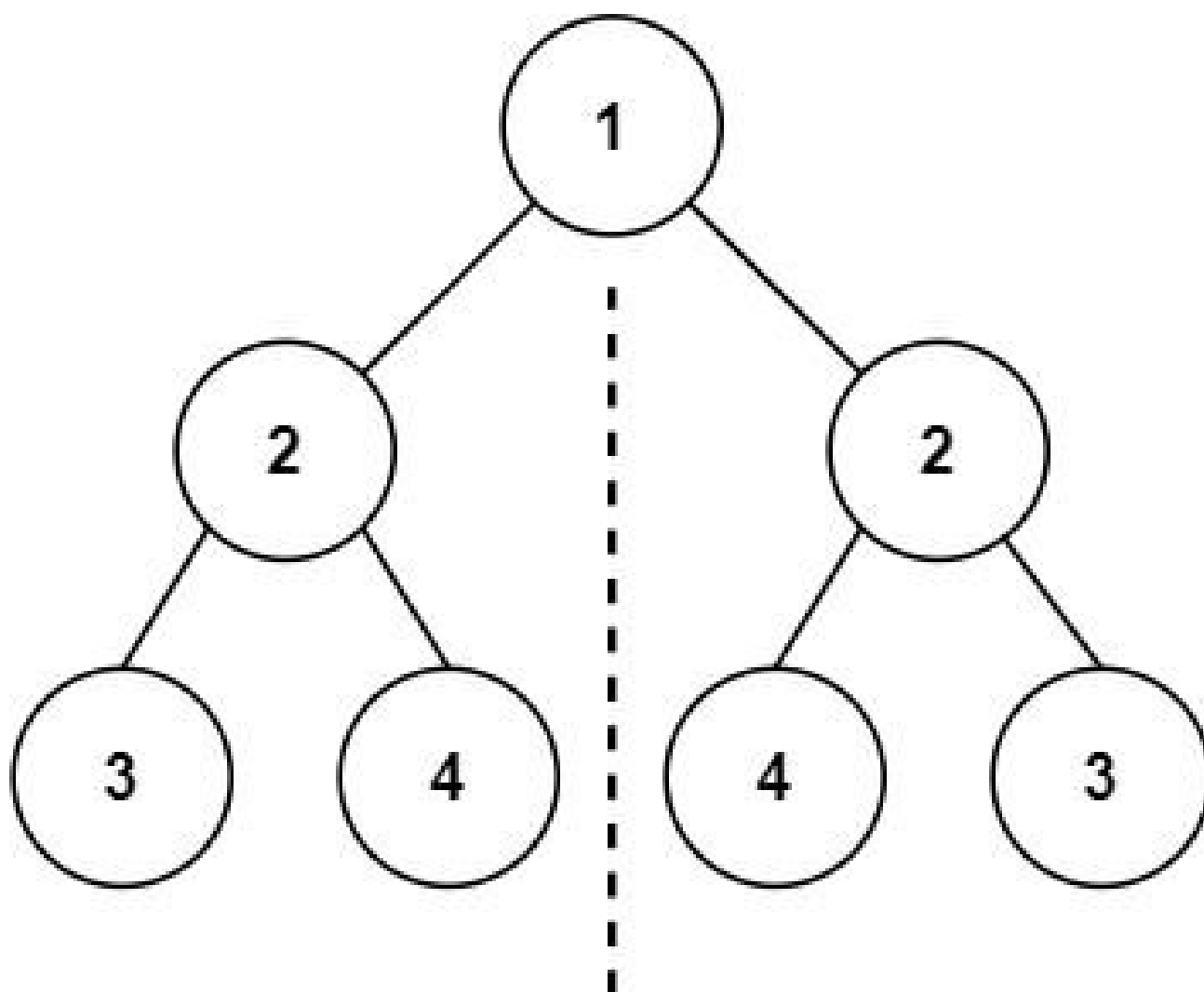
EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS
Lists: Grind 169**

Problem

Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

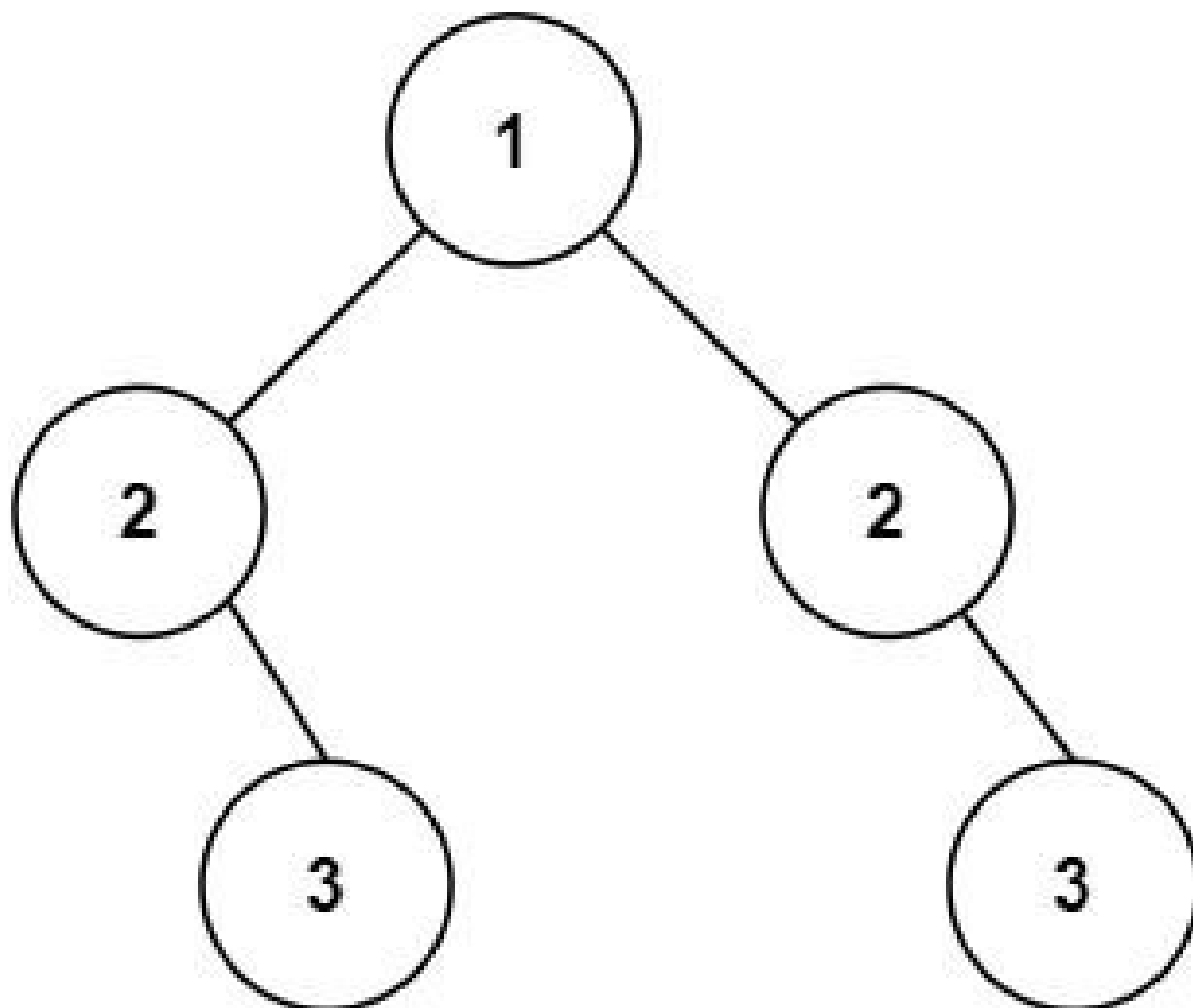
Example 1:



Input: root = [1,2,2,3,4,4,3]

Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]

Output: false

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Could you solve it both recursively and iteratively?

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def symmetric(p, q):
            if not p or not q:
                return p == q

            if p.val != q.val:
                return False

            return symmetric(p.left, q.right) and symmetric(p.right, q.left)
        return symmetric(root, root)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "A tree is symmetric if its left and right subtrees are mirror images of each other.
# Mirror means: at each level, left child of left subtree matches right child of right subtree,
# and right child of left subtree matches left child of right subtree. Right?
# A single node is always symmetric."
#
# "[1,2,2,3,4,4,3]: left subtree [2,3,4] mirrors right [2,4,3] → true ✓
# [1,2,2,null,3,null,3]: left has right child 3, right has right child 3 – not mirrored → false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Serialise the left subtree with a pre-order traversal and the right subtree with the mirror order (right child before left), then compare the two strings.
# Same O(n) time and O(n) space as the optimal recursive approach – no improvement to be had, so we skip the brute class and go straight to the clean solution."
```

PHASE 3 — OPTIMIZE

```
# "Recursive DFS comparing the tree to itself – left vs right mirrored.
# Define an inner helper mirror(p, q):
```

```
# Base: if either is None, both must be None.
# Recursive: values match AND mirror(p.left, q.right) AND mirror(p.right, q.left).
# Note the cross-comparison: left's left ↔ right's right, left's right ↔ right's left."
```

PHASE 4 — CLEAN CODE

```
class _101_SymmetricTree:
    def isSymmetric(self, root: Optional['TreeNode']) -> bool:
        def mirror(p, q):
            if not p or not q:
                return p == q
            return p.val == q.val and mirror(p.left, q.right) and mirror(p.right,
q.left)
        return mirror(root, root)
```

PHASE 5 — TEST & DEBUG

```
# root=[1,2,2,3,4,4,3]:
# mirror(root,root): vals 1==1.
# mirror(left=2, right=2): 2==2.
# mirror(2.left=3, 2.right=3): 3==3. mirror(None,None)=T. mirror(None,None)=T → T
# mirror(2.right=4, 2.left=4): 4==4. T → T
# mirror(right=2, left=2): symmetric to above → T
# → True ✓
# root=[1,2,2,null,3,null,3]:
# mirror(left=2,right=2): 2==2.
# mirror(2.left=None, 2.right=3): p=None,q=3 → None≠3 → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h) for the call stack.
# Follow-up asks for iterative solution: use a queue of pairs.
# Enqueue (root.left, root.right). Each step dequeue (p,q):
# check p==q==None (skip), p or q None (false), vals differ (false).
# Enqueue (p.left,q.right) and (p.right,q.left). O(n) time, O(n) space.
# The cross-enqueue order is the key – same insight as the recursive version.
# Given more time I'd ask: how does this differ from Same Tree (#100)?
# Same Tree checks identical structure; Symmetric Tree checks mirrored structure –
# the only difference is the cross-comparison in the recursive call."
```

◆ Quick Review

Key Insights

- Use recursion (or iteration) to compare pairs of nodes.

- For two trees (or two nodes) to be mirror images:
- Their root values must be the same.
- The left subtree of one must match the right subtree of the other.
- The right subtree of one must match the left subtree of the other.
- Alternatively, iterative approaches using a queue can be applied to compare the nodes in pairs.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree, since we traverse each node once.

Space Complexity: $O(h)$ for recursive call stack (worst-case $O(n)$ for a skewed tree) or $O(n)$ for the iterative approach using a queue.

Solution

We solve the problem using recursion by defining a helper function that takes two nodes and checks if they are mirrors of each other. The helper function: Returns true if both nodes are null. Returns false if one node is null or if their values differ. Recursively compares

the left child of the first node with the right child of the second node, and vice-versa. This method ensures each comparison verifies symmetry at every level. A similar logic can be applied iteratively by using a queue to compare pairs of nodes.

Trees & BST

□ #104 Maximum Depth of Binary Tree

EAS

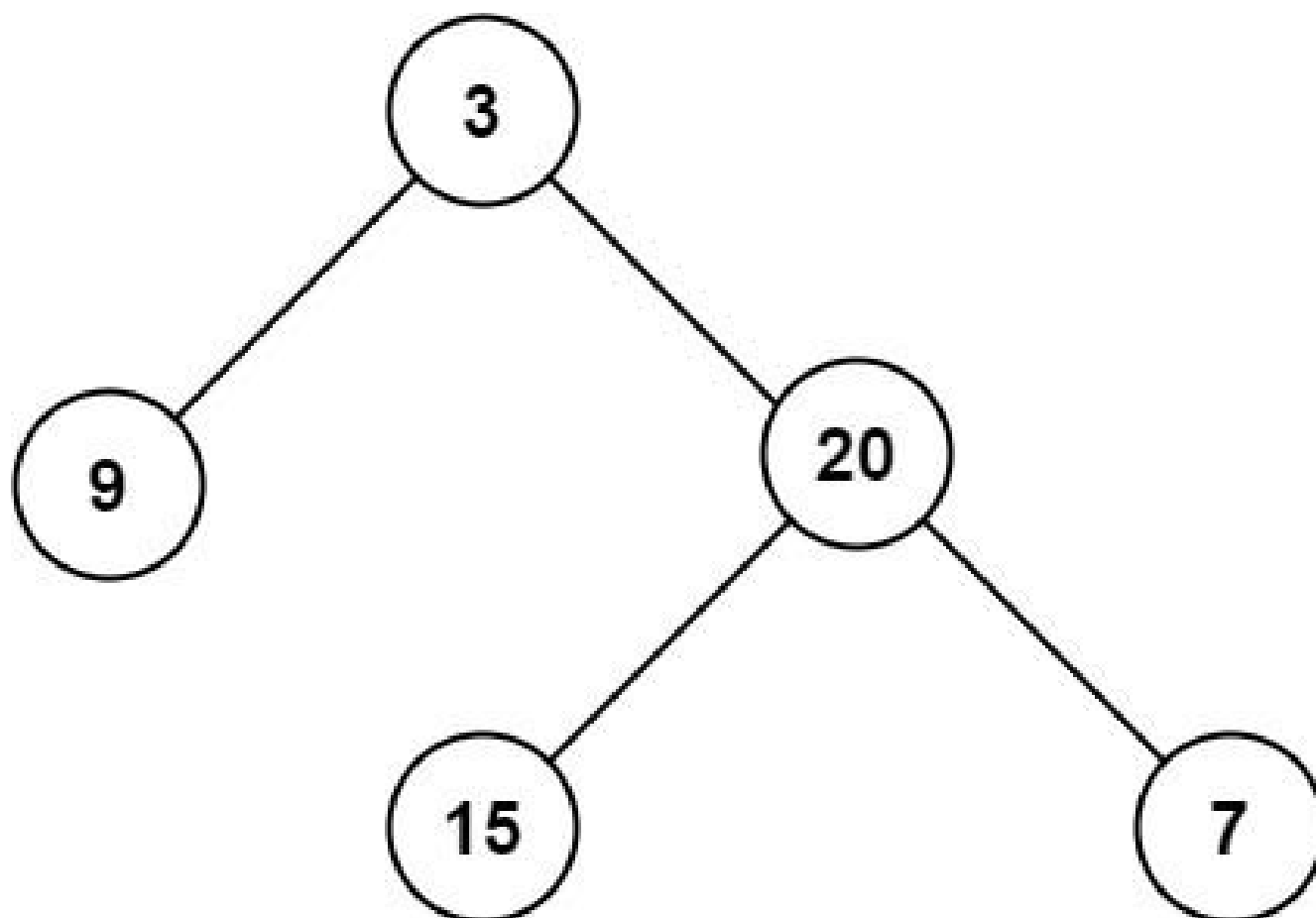
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS
Lists: Grind 169

Problem

Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: 3

Example 2:

Input: root = [1,null,2]
Output: 2

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

- My LeetCode Solution

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        def depth(root):
            if not root:
                return 0

            return max(1+ depth(root.left), 1+depth(root.right))

        return depth(root)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Maximum depth is the number of nodes on the longest root-to-leaf path.
# An empty tree has depth 0. A single node has depth 1. Right?"
#
# "[3,9,20,null,null,15,7]: longest path is 3→20→15 or 3→20→7, 3 nodes → 3 ✓
# [1,null,2]: 1→2, 2 nodes → 2 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with the brute force to lock in the logic.
# Iterative BFS: use a queue, process the tree level by level.
# Each time we exhaust one level, increment a depth counter.
# When the queue is empty, the counter equals the maximum depth.
# Time: O(n). Space: O(n) for the queue. Should I go ahead and optimize?"

```

```

class _104_MaxDepth_Brute:
    def maxDepth(self, root: Optional['TreeNode']) -> int:
        if not root:
            return 0
        from collections import deque
        queue = deque([root])
        depth = 0
        while queue:
            depth += 1
            for _ in range(len(queue)):
                node = queue.popleft()
                if node.left:
                    queue.append(node.left)

```

```

        if node.right:
            queue.append(node.right)
    return depth

```

PHASE 3 — OPTIMIZE

"Recursive DFS is cleaner and same complexity.

$\text{depth}(\text{node}) = \max(\text{depth}(\text{left}), \text{depth}(\text{right})) + 1$. Base: empty node $\rightarrow 0$.

Use an inner helper to avoid self. calls."

PHASE 4 — CLEAN CODE

```

class _104_MaxDepth:
    def maxDepth(self, root: Optional['TreeNode']) -> int:
        def depth(node):
            if not node:
                return 0
            return max(depth(node.left), depth(node.right)) + 1
        return depth(root)

```

PHASE 5 — TEST & DEBUG

$\text{root} = [3, 9, 20, \text{null}, \text{null}, 15, 7]$:

$\text{depth}(3) = \max(\text{depth}(9), \text{depth}(20)) + 1$.

$\text{depth}(9) = \max(0, 0) + 1 = 1$. $\text{depth}(20) = \max(\text{depth}(15), \text{depth}(7)) + 1 = \max(1, 1) + 1 = 2$.

$\text{depth}(3) = \max(1, 2) + 1 = 3$ ✓

$\text{root} = \text{None}$: $\text{depth}(\text{None}) = 0$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$ call stack — $O(\log n)$ balanced, $O(n)$ skewed.

BFS alternative: count levels with a queue — $O(n)$ time, $O(w)$ space ($w = \text{max width}$).

For very deep trees, BFS avoids recursion stack overflow.

Given more time I'd ask: what about minimum depth? Different — must reach a leaf,

so BFS is actually better there (stops at first leaf found)."

◆ Quick Review

Key Insights

- Use a recursive strategy (DFS) to traverse the binary tree.
- At each node, compute the maximum depth of its left and right subtree.
- The maximum depth at the current node is 1 (current node) plus the maximum of the left

and right subtree depths.

- Alternatively, an iterative approach using BFS (level-order traversal) can be used.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since every node is visited once.

Space Complexity: $O(h)$ for recursion stack in DFS, where h is the height of the tree (worst-case $O(n)$ for a skewed tree); $O(n)$ for BFS in the worst-case scenario if the tree is balanced.

Solution

The problem is approached using Depth-First Search (DFS) with recursion. The idea is to visit every node and calculate the maximum depth of the left and right subtrees, then add 1 for the current node. This method naturally handles the base case when a null node (empty tree) is reached by returning 0.

Alternatively, a Breadth-First Search (BFS) approach can determine the tree level by level, with the total number of levels equaling the maximum depth. Key data structures include recursion call stack for DFS or a queue for BFS.

Trees & BST

□ #108 Convert Sorted Array to Binary Search Tree

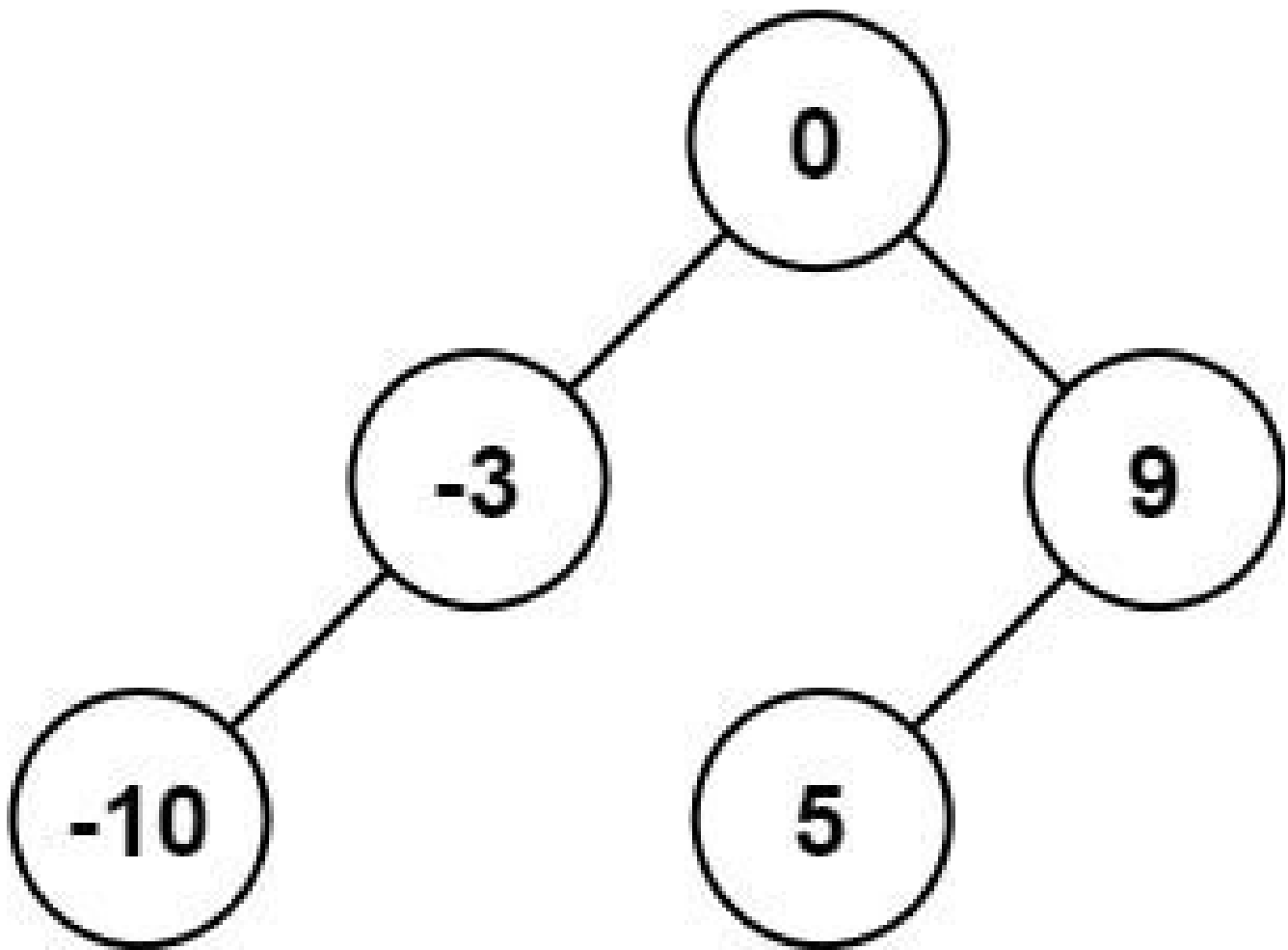
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Tree
Lists: Grind 169

Problem

Given an integer array `nums` where the elements are sorted in ascending order, convert it to a height-balanced binary search tree.

Example 1:

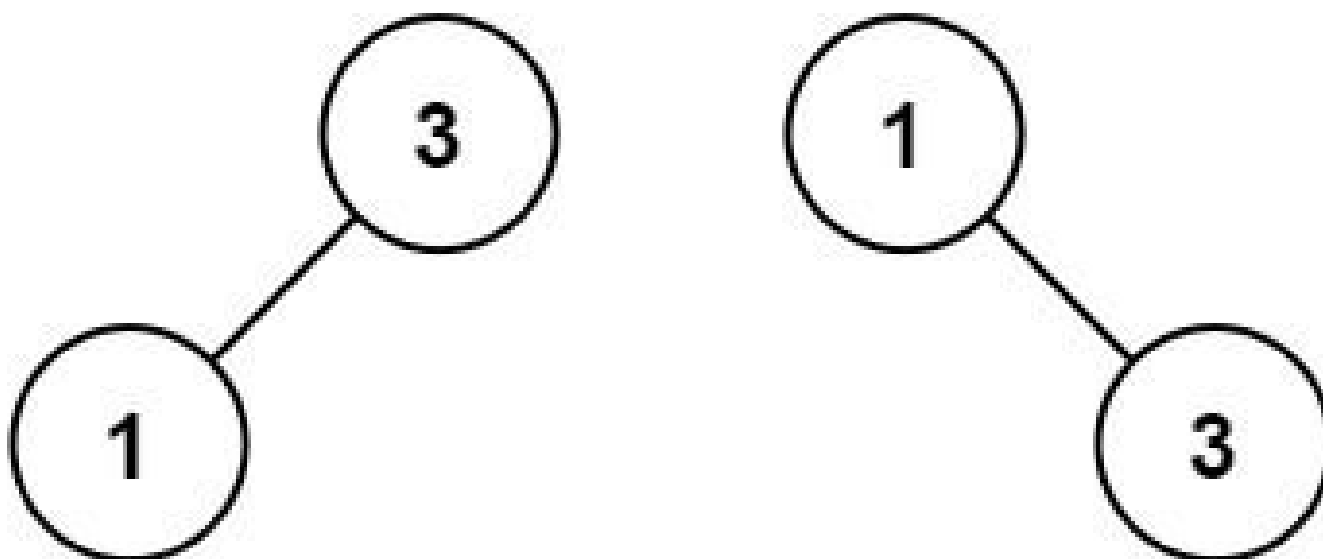


Input: nums = [-10,-3,0,5,9]

Output: [0,-3,9,-10,null,5]

Explanation: [0,-10,5,null,-3,null,9] is also accepted:

Example 2:



Input: nums = [1,3]

Output: [3,1]

Explanation: [1,null,3] and [3,1] are both height-balanced BSTs.

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- nums is sorted in a strictly increasing order.

My LeetCode Solution

• My LeetCode Solution

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def build(l,r):
            if l>r:

```

```

        return None

    m = (l+r)//2
    return TreeNode(nums[m], build(l, m-1), build(m+1,r))

n= len(nums)
return build(0,n-1)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Convert a sorted array to a height-balanced BST.

Height-balanced means every node's left and right subtree heights differ by at most 1.

Multiple valid answers exist – any height-balanced BST is acceptable. Right?"

#

"[-10,-3,0,5,9]: pick middle element as root → 0. Left half [-10,-3] → left subtree.

Right half [5,9] → right subtree. Recurse. → [0,-3,9,-10,null,5] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Pick the middle element of nums as root, recursively build left from nums[:mid]

and right from nums[mid+1:].

Always produces a valid BST shape for a sorted input.

Time: $O(n \log n)$ from repeated slicing. Space: $O(n)$ for slices.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Pick the middle element as root – this guarantees balance because both halves

have equal (or ± 1) size. Recurse on left and right halves.

Divide and conquer. Use inner helper build(l, r) operating on indices."

PHASE 4 — CLEAN CODE

```
class _108_SortedArrayToBST:
```

```
    def sortedArrayToBST(self, nums: List[int]) -> Optional['TreeNode']:
```

```
        def build(left, right):
```

```
            if left > right:
```

```
                return None
```

```
            mid = (left + right) // 2
```

```
            return TreeNode(nums[mid], build(left, mid - 1), build(mid + 1, right))
```

```
        return build(0, len(nums) - 1)
```

PHASE 5 — TEST & DEBUG

```
# nums=[-10,-3,0,5,9]:
```

```
# build(0,4): mid=2→node(0). left=build(0,1), right=build(3,4).
```

```
# build(0,1): mid=0→node(-10). left=None, right=build(1,1)→node(-3). →
(-10,None,-3)
# build(3,4): mid=3→node(5). left=None, right=build(4,4)→node(9). → (5,None,9)
# root=0, left=(-10,None,-3), right=(5,None,9). Valid height-balanced BST ✓
# nums=[1,3]: mid=0→node(1), right=build(1,1)→node(3). → [1,null,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) – every element becomes a node exactly once.
# Space O(log n) for the call stack – tree is balanced by construction.
# Choosing (l+r)//2 vs (l+r+1)//2 gives slightly different but equally valid trees.
# Given more time I'd ask: what if the array is sorted in descending order?
# Reverse it first or adjust the index logic – same approach."
```

◆ Quick Review

Key Insights

- The array is sorted in ascending order, which allows the use of a divide and conquer strategy.
- Choosing the middle element of the array as the root ensures that the left and right subtrees are roughly equal in size, yielding a balanced BST.
- A recursive approach is natural for building the tree: the base case is when the subarray is empty.
- The algorithm splits the array into two halves to recursively build the left and right subtrees.

Space & Time

Time Complexity: $O(n)$ – Every element is processed exactly once.

Space Complexity: $O(n)$ – $O(n)$ space is used to store the BST, and additional $O(\log n)$ space may be used for the recursion stack.

Solution

We use a divide and conquer approach with recursion. The main idea is to select the middle element of the current subarray as the root node so that the left subarray elements form the left subtree and the right subarray elements form the right subtree, ensuring the tree is balanced. The recursion continues until the subarray is empty, which serves as the base case. This strategy naturally maintains the BST property since all elements in the left subarray are smaller than the root, and all in the right subarray are larger.

Trees & BST

□ #110 Balanced Binary Tree

EAS

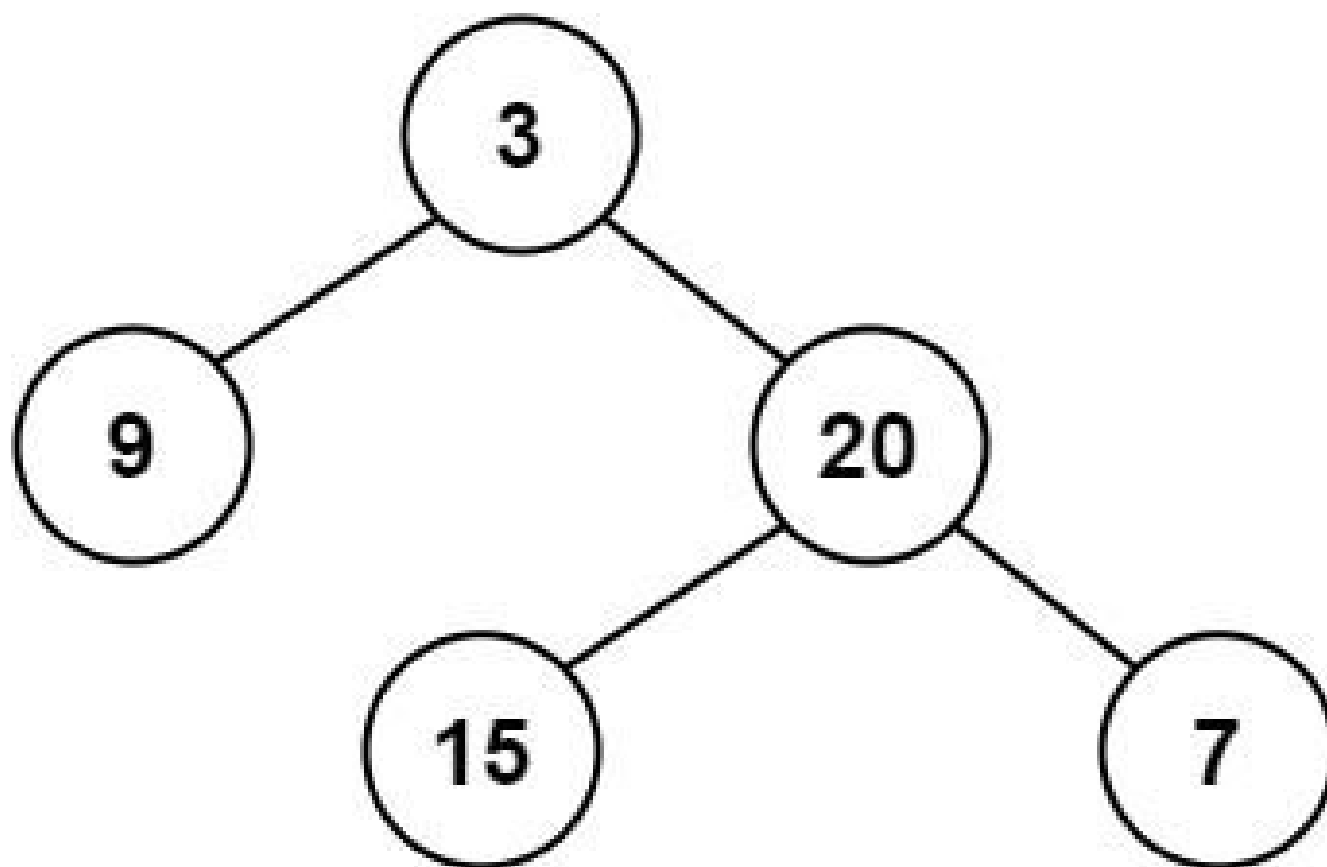
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: Grind 169

Problem

Given a binary tree, determine if it is height-balanced.

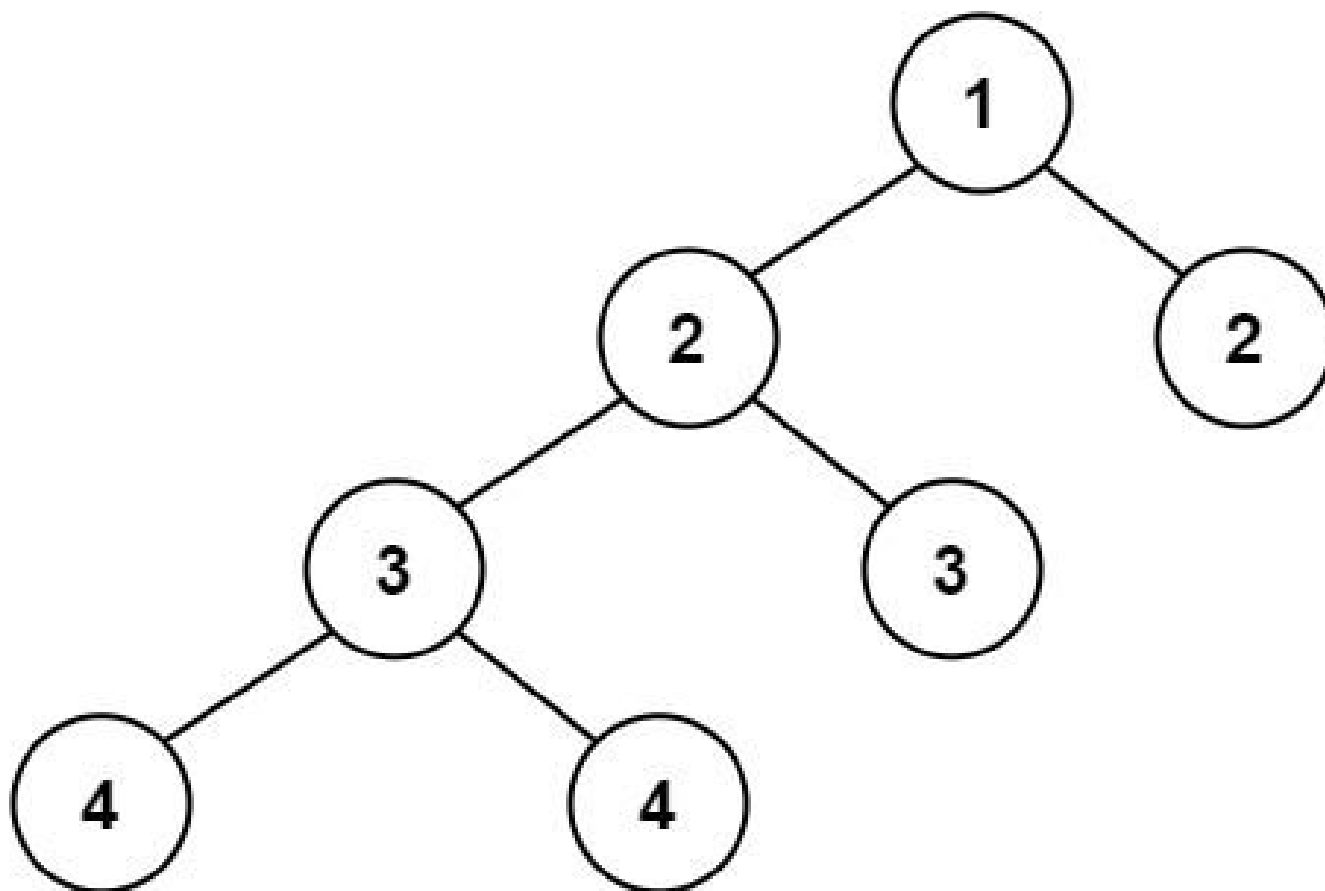
Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: true

Example 2:



Input: root = [1,2,2,3,3,null,null,4,4]

Output: false

Example 3:

Input: root = []

Output: true

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-104 \leq \text{Node.val} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def depth(root):
            if not root:
                return 0

            l = depth(root.left)
            r = depth(root.right)
            return max(l,r)+1

        if not root:
            return True

        return abs(depth(root.left) - depth(root.right)) <= 1 and self.isBalanced(root.left) and self.isBalanced(root.right)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A tree is height-balanced if every node's left and right subtree heights

differ by at most 1. Empty tree is balanced. Right?"

#

"[3,9,20,null,null,15,7]: heights of 9's subtrees: 0 vs 0 ✓. 20's: 1 vs 1 ✓. 3's: 1 vs 2 ✓ → true.

[1,2,2,3,3,null,null,4,4]: left subtree of root has height 3, right has 1 → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

At each node compute left subtree height and right subtree height separately.

Node is balanced if $|left-right| \leq 1$ and both subtrees are balanced recursively.

Recomputes heights from scratch at every node – $O(n^2)$ on skewed trees.

Time: $O(n^2)$ worst case. Space: $O(h)$ stack.

Should I go ahead and optimize?"

```
class _110_BalancedTree_Brute:
    def isBalanced(self, root: Optional['TreeNode']) -> bool:
        def height(node):
            if not node:
                return 0
            return max(height(node.left), height(node.right)) + 1
        def balanced(node):
            if not node:
                return True
            lh, rh = height(node.left), height(node.right)
            return abs(lh - rh) <= 1 and balanced(node.left) and
balanced(node.right)
            return balanced(root)
```

PHASE 3 — OPTIMIZE

"The brute force calls height() repeatedly for the same nodes – $O(n^2)$.
 # I notice we're recomputing heights we've already seen. What if we cache it?
 #
 # Better: combine the height computation and balance check into one DFS pass.
 # Return -1 to signal 'unbalanced subtree found' – propagate it up immediately.
 # Otherwise return the actual height.
 # A single $O(n)$ pass handles everything."

PHASE 4 — CLEAN CODE

```
class _110_BalancedTree:
    def isBalanced(self, root: Optional['TreeNode']) -> bool:
        def check(node):
            if not node:
                return 0
            left = check(node.left)
            right = check(node.right)
            if left == -1 or right == -1 or abs(left - right) > 1:
                return -1 # propagate 'unbalanced' signal
            return max(left, right) + 1
        return check(root) != -1
```

PHASE 5 — TEST & DEBUG

root=[3,9,20,null,null,15,7]:
 # check(9)=1. check(15)=1. check(7)=1. check(20)=max(1,1)+1=2.
 # check(3): left=1,right=2. |1-2|=1 <=1 → return 2. result≠-1 → True ✓
 # root=[1,2,2,3,3,null,null,4,4]:
 # check(4)=1. check(3,left)=max(1,0)+1=2. check(3,right)=max(1,0)+1=2.
 # check(2,left): left=2,right=2 → 3. check(2,right): left=0,right=0 → 1.
 # check(1): left=3,right=1. |3-1|=2>1 → return -1. result==-1 → False ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – single DFS, each node visited once.
 # Space $O(h)$ for the call stack.

```
# The -1 sentinel trick is the key interview insight: instead of separate
# height and balance checks, fuse them into one return value with a special signal.
# Given more time I'd ask: can we rebalance a tree in-place?
# Flatten to sorted array (in-order traversal) then rebuild - O(n) time."
```

◆ Quick Review

Key Insights

- Use a bottom-up DFS (Depth-First Search) traversal to compute the height of each subtree.
- If any subtree is found unbalanced, propagate an error indicator (e.g., -1) upward, allowing early termination.
- An empty tree is considered balanced.
- The key check is to ensure that for each node, the absolute difference between the heights of its left and right subtrees is no more than 1.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the tree, since we visit each node once.

Space Complexity: $O(h)$ where h is the height of the tree, which corresponds to the recursion stack ($O(n)$ in the worst-case of a skewed tree).

Solution

The solution uses a recursive DFS approach. A helper function recursively computes the height of a subtree. If a subtree is found to be unbalanced (the height difference between its left and right subtrees exceeds 1), the helper returns a special value (e.g., -1) to indicate that the subtree is unbalanced. This value is then propagated up the recursion to terminate further unnecessary computations. The main function checks the result of the helper function and returns true if the tree is balanced (i.e., the helper did not return -1) and false otherwise.

Trees & BST

□ #112 Path Sum

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS**

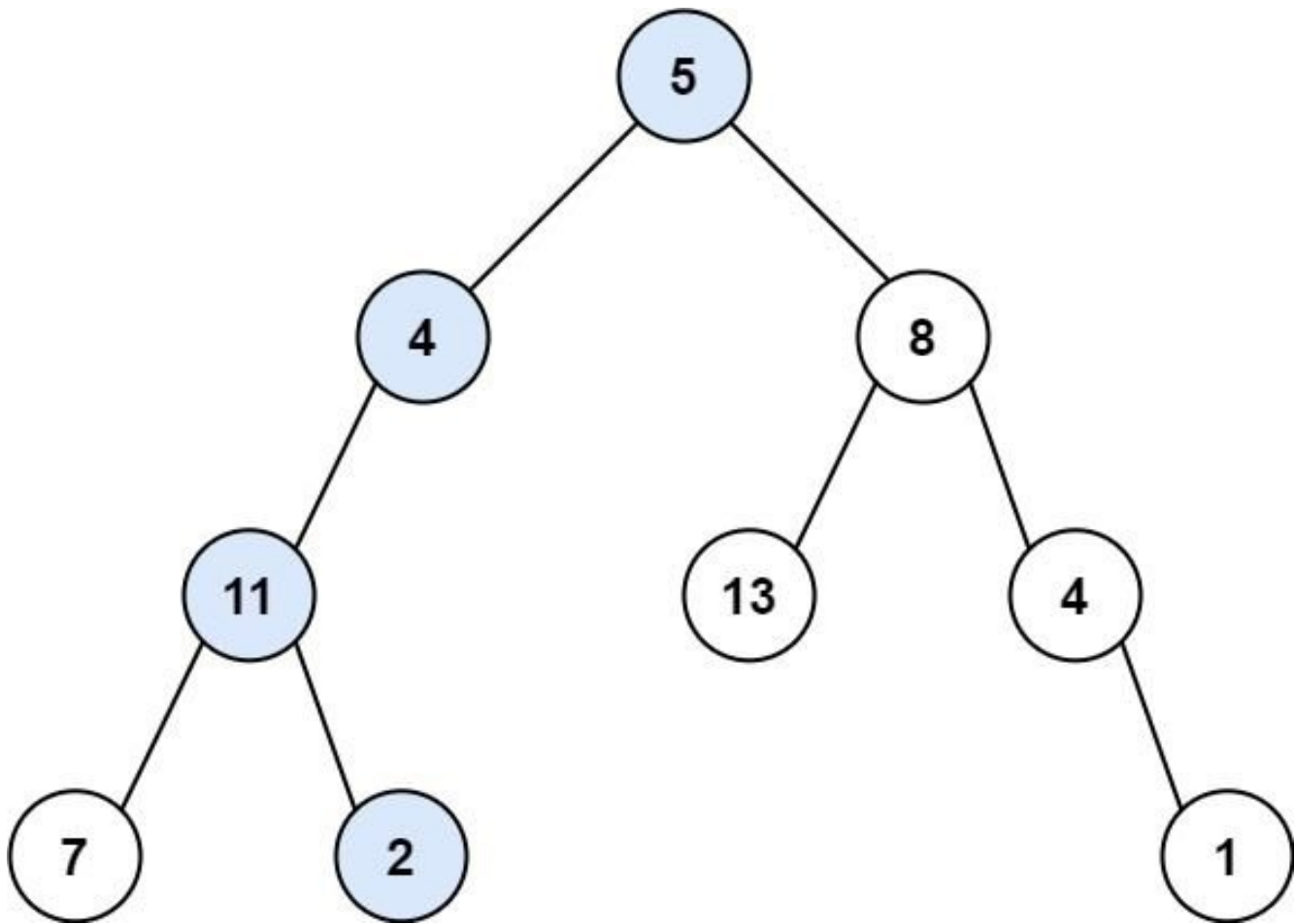
Lists: Denny Zhang

Problem

Given the root of a binary tree and an integer targetSum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals targetSum.

A leaf is a node with no children.

Example 1:

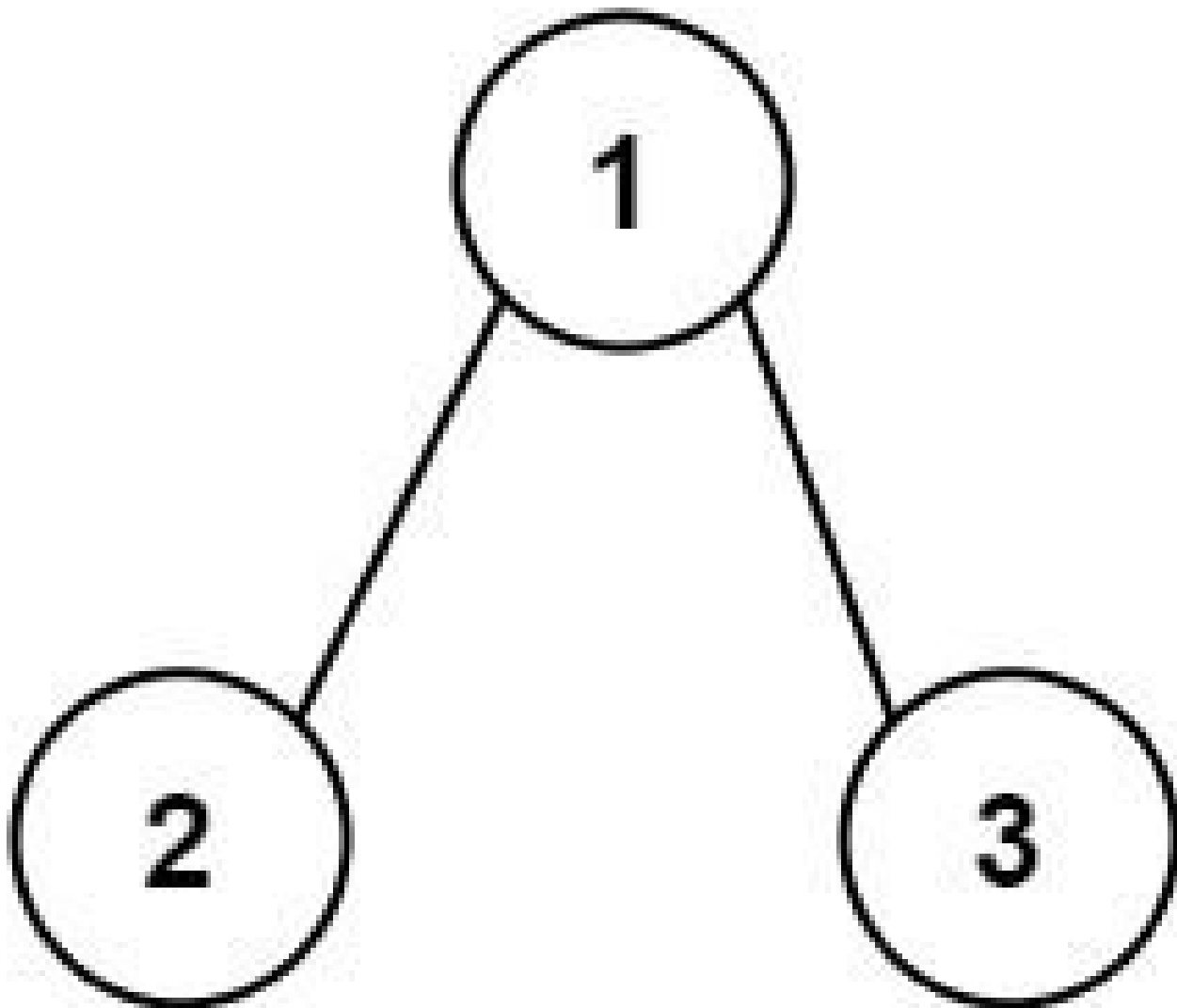


Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22

Output: true

Explanation: The root-to-leaf path with the target sum is shown.

Example 2:



Input: root = [1,2,3], targetSum = 5

Output: false

Explanation: There are two root-to-leaf paths in the tree:

(1 --> 2): The sum is 3.

(1 --> 3): The sum is 4.

There is no root-to-leaf path with sum = 5.

Example 3:

Input: root = [], targetSum = 0

Output: false

Explanation: Since the tree is empty, there are no root-to-leaf paths.

Constraints:

- The number of nodes in the tree is in the range [0, 5000].

- $-1000 \leq \text{Node.val} \leq 1000$
- $-1000 \leq \text{targetSum} \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def hasPathSum(self, root: Optional[TreeNode], targetSum: int) -> bool:
        def hassum(root, targetSum):
            if not root:
                return False

            if root.val == targetSum and not root.left and not root.right:
                return True

            return hassum(root.left, targetSum - root.val) or hassum(root.right, targetSum - root.val)

        return hassum(root, targetSum)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find if any root-to-leaf path sums to targetSum.
# A leaf is a node with no children – the path must end at a leaf. Right?
# Empty tree → false (no leaves)."
#
# "[5,4,8,11,null,13,4,7,2,null,null,null,1], target=22:
# path 5→4→11→2 = 22 → true ✓
# [], target=0 → false (empty tree has no root-to-leaf path) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic."
```

```
# DFS traversal collecting a running sum as we descend. At each leaf node,
# check if the accumulated sum equals targetSum. Backtrack and try other paths.
# The optimal approach is the same DFS – just written more elegantly by subtracting
# from target instead of accumulating. Both are  $O(n)$  time and  $O(h)$  space, so we skip
# a separate brute class and go straight to the clean version."
```

PHASE 3 — OPTIMIZE

```
# "More elegant: subtract the current node's value from targetSum as we descend.
# At a leaf, check if remaining == 0. Avoids passing a separate running sum."
```

PHASE 4 — CLEAN CODE

```
class _112_PathSum:
    def hasPathSum(self, root: Optional['TreeNode'], targetSum: int) -> bool:
        def hassum(node, remaining):
            if not node:
                return False
            remaining -= node.val
            if not node.left and not node.right: # leaf
                return remaining == 0
            return hassum(node.left, remaining) or hassum(node.right, remaining)
        return hassum(root, targetSum)
```

PHASE 5 — TEST & DEBUG

```
# root=[5,4,8,...], target=22:
# hassum(5,22): rem=17. hassum(4,17): rem=13. hassum(11,13): rem=2.
# hassum(7,2): rem=-5, leaf -> -5≠0 False.
# hassum(2,2): rem=0, leaf -> True ✓
# root=[], target=0: hassum(None,0)=False ✓
# root=[1,2], target=1: hassum(1,1): rem=0, not leaf.
# hassum(2,0): rem=-2, leaf -> False. -> False ✓ (path must reach a leaf)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(h)$  call stack.
# The 'leaf check' (no left and no right) is critical – without it, a node
# with one child that hits 0 would incorrectly return true.
# Given more time I'd ask: return all root-to-leaf paths that sum to target?
# That's Path Sum II (#113) – collect paths into a result list with backtracking."
```

◆ Quick Review

Key Insights

- Use a depth–first search (DFS) strategy to traverse the tree from the root to the leaves.

- At each node, subtract the node's value from the targetSum.
- Check if the node is a leaf; if yes and the remaining targetSum equals the node's value then a valid path is found.
- If the tree is empty, return false as no path exists.

Space & Time

Time Complexity: $O(N)$ where N is the number of nodes, since every node may need to be visited.

Space Complexity: $O(H)$ where H is the height of the tree, accounting for the recursion stack (worst-case $O(N)$ for a skewed tree).

Solution

The approach uses a recursive depth-first search. The algorithm subtracts the current node's value from targetSum as it moves down the tree. When it reaches a leaf, it checks if the modified targetSum is equal to the leaf's value. If yes, it returns true. The recursion continues until either a valid path is found or all paths are exhausted. The primary data structure used here is the call stack for recursion.

Trees & BST

□ #226 Invert Binary Tree

EAS

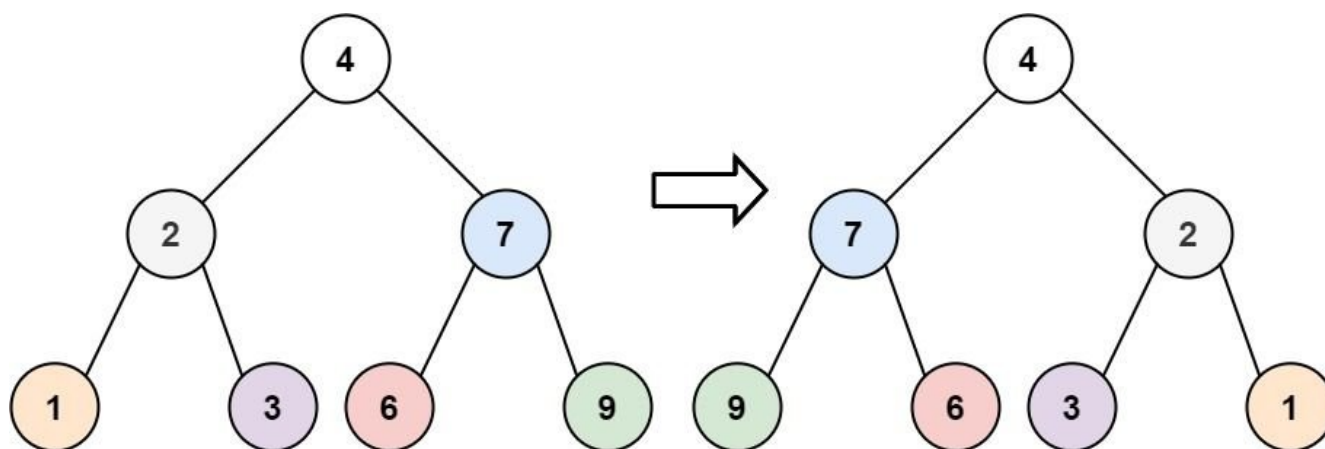
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, invert the tree, and return its root.

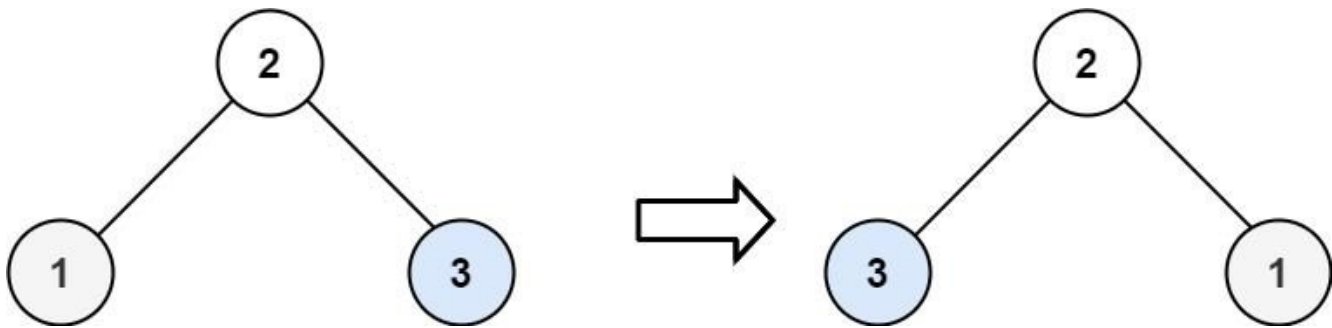
Example 1:



Input: root = [4,2,7,1,3,6,9]

Output: [4,7,2,9,6,3,1]

Example 2:



Input: root = [2,1,3]

Output: [2,3,1]

Example 3:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

• My LeetCode Solution

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        def invert(root):
            if not root:

```

```

        return None

    l ,r = invert(root.left), invert(root.right)
    root.left = r
    root.right = l

    return root
return invert(root)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Invert the tree: swap left and right children at every node recursively.

Return the root. Empty tree → return None. Right?"

#

"[4,2,7,1,3,6,9] → [4,7,2,9,6,3,1]: left and right subtrees are swapped at every level ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Iterative BFS with a queue: visit each node level by level and swap its left

and right children in place. Process until the queue is empty.

Time: O(n). Space: O(n) for the queue. Should I go ahead and optimize?"

class _226_InvertTree_Brute:

def invertTree(self, root: Optional['TreeNode']) -> Optional['TreeNode']:

if not root:

return None

from collections import deque

queue = deque([root])

while queue:

node = queue.popleft()

node.left, node.right = node.right, node.left

if node.left:

queue.append(node.left)

if node.right:

queue.append(node.right)

return root

PHASE 3 — OPTIMIZE

"Recursive DFS is cleaner. Invert left, invert right, then swap.

Or: swap first then recurse – same result. Either order works.

Use inner helper to avoid self. calls."

PHASE 4 — CLEAN CODE

class _226_InvertTree:

def invertTree(self, root: Optional['TreeNode']) -> Optional['TreeNode']:

```
def invert(node):
    if not node:
        return None
    left = invert(node.left)
    right = invert(node.right)
    node.left, node.right = right, left
    return node
return invert(root)
```

PHASE 5 — TEST & DEBUG

```
# root=[4,2,7]:
# invert(2): invert(None)=None, invert(None)=None. swap: 2.left=None,2.right=None.
return 2.
# invert(7): same → 7.
# invert(4): left=invert(2)=2, right=invert(7)=7. swap: 4.left=7, 4.right=2. return
4.
# → [4,7,2] ✓ (recurse deeper for full tree)
# root=None: invert(None)=None ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h) call stack.
# This is the problem Homebrew creator Max Howell famously couldn't solve in a
Google interview.
# Worth mentioning – it humanises the problem and shows you know your history.
# Given more time I'd ask: can we do this iteratively? Yes – BFS or DFS with a
stack,
# swap children at each node. O(n) time, O(n) space."
```

◆ Quick Review

Key Insights

- The problem can be solved using recursion (depth–first search) by swapping the children of every node.
- Alternatively, an iterative approach (using a queue for breadth–first search) can be used.
- The recursion terminates when a null node is encountered.

- The operation is performed in-place, modifying the original tree structure.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since each node is visited once.

Space Complexity: $O(h)$ where h is the height of the tree, which is the recursion stack space in the recursive approach. In worst-case (skewed tree), this could be $O(n)$.

Solution

We solve the problem using recursion (depth-first search). The approach is to: Check the base case: if the node is null, simply return null. Recursively invert the left subtree and the right subtree. Swap the left and right children of the current node. Return the current node after the swap. The recursive approach elegantly handles all nodes and ensures the inversion process covers the complete tree.

Trees & BST

□ ★ #270 Closest Binary Search Tree Value

EAS

★

LeetDoocs · SimplyLeet · WalkCC · LeetCode

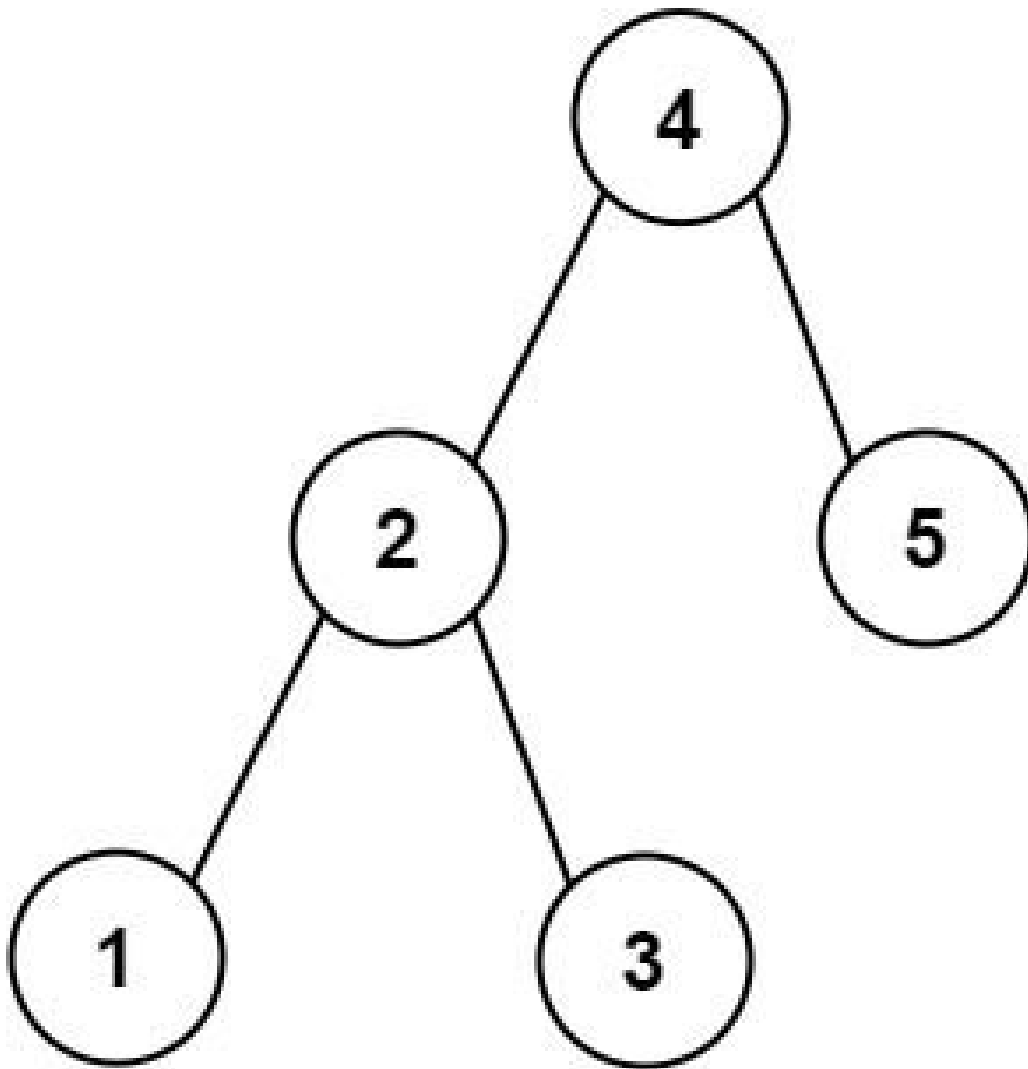
Binary Search · Tree · DFS · BST

Lists: ★ Premium | Premium 98

Problem

Given the root of a binary search tree and a target value, return the value in the BST that is closest to the target. If there are multiple answers, print the smallest.

Example 1:



Input: root = [4,2,5,1,3], target = 3.714286
Output: 4

Example 2:

Input: root = [1], target = 4.428571
Output: 1

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $0 \leq \text{Node.val} \leq 109$
- $-109 \leq \text{target} \leq 109$

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestValue(self, root: Optional[TreeNode], target: float) -> int:
        def close(root, t):
            if target < root.val and root.left:
                l = close(root.left, t)
                if abs(t-l) <= abs(t-root.val):
                    return l

            if target > root.val and root.right:
                r = close(root.right, t)
                if abs(t-r) < abs(t-root.val):
                    return r

            return root.val

        return close(root, target)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the node value in a BST closest to a float target.
# If two values are equidistant, return the smaller one. Right?
# BST property means we can navigate toward the target rather than scanning all
# nodes."
#
# "root=[4,2,5,1,3], target=3.714286:
# candidates: 4 (dist=0.286), 3 (dist=0.714). 4 is closer → 4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Full DFS scan: visit every node, track the value with minimum |val - target|."
```

```
# On a tie (equal distance), prefer the smaller value.
# This ignores the BST property entirely – valid but O(n) time and O(n) stack space.
# Should I go ahead and optimize?"
```

```
class _270_ClosestBST_Brute:
    def closestValue(self, root: Optional['TreeNode'], target: float) -> int:
        best = [root.val]
        def dfs(node):
            if not node:
                return
            if (abs(node.val - target) < abs(best[0] - target) or
                (abs(node.val - target) == abs(best[0] - target) and node.val <
best[0])):
                best[0] = node.val
            dfs(node.left)
            dfs(node.right)
        dfs(root)
        return best[0]
```

PHASE 3 — OPTIMIZE

```
# "Use BST property to navigate: at each node, the closest value is either the
# current node or something in the subtree direction of the target.
# If target < node.val → go left; if target > node.val → go right.
# Track the closest seen so far. O(h) time – O(log n) balanced, O(n) skewed."
```

PHASE 4 — CLEAN CODE

```
class _270_ClosestBST:
    def closestValue(self, root: Optional['TreeNode'], target: float) -> int:
        def close(node, best):
            if not node:
                return best
            if abs(target - node.val) < abs(target - best):
                best = node.val
            elif abs(target - node.val) == abs(target - best):
                best = min(best, node.val) # tie → pick smaller
            if target < node.val:
                return close(node.left, best)
            else:
                return close(node.right, best)
        return close(root, root.val)
```

PHASE 5 — TEST & DEBUG

```
# root=[4,2,5,1,3], target=3.714286:
# close(4, 4): |3.714-4|=0.286 < |3.714-4|=0.286 → no update. target<4 → go left.
# close(2, 4): |3.714-2|=1.714 > 0.286 → best stays 4. target>2 → go right.
# close(3, 4): |3.714-3|=0.714 > 0.286 → best stays 4. target>3 → go right.
# close(None, 4): return 4 ✓
# root=[1], target=4.428: close(1,1): target>1 → go right → close(None,1) → 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(h). Space O(h) for recursion stack. Iterative: O(1) space."
```

```
# BST navigation is the key – we prune entire subtrees we know can't contain  
# a closer value.  
# Given more time I'd ask: find the k closest values to target?  
# That's Closest BST Value I (272) – use an in-order traversal with a sliding  
window  
# or a max-heap of size k."
```

◆ Quick Review

Key Insights

- BST property allows us to traverse the tree efficiently by comparing the target with node values.
- We can iteratively traverse the tree, moving left if the target is smaller than the current node's value and moving right if greater.
- At each node, calculate the absolute difference between node's value and the target, and update the answer if the difference is smaller—or equal with a smaller candidate value.
- This approach minimizes the search area by leveraging the inherent ordering in BSTs.

Space & Time

Time Complexity: $O(h)$ on average, where h is the height of the tree ($O(\log n)$ for a balanced tree, and $O(n)$ in the worst-case scenario for a skewed tree).

Space Complexity: $O(1)$ for an iterative solution (or $O(h)$ if using recursion due to the call stack).

Solution

We solve the problem by performing an iterative traversal of the BST. Starting at the root, we maintain a variable to store the value closest to the target. For every node encountered, we: Compute the absolute difference between the node's value and the target. Update the stored closest value if the computed difference is less than the current smallest difference; if the difference is the same, choose the smaller node value.

Depending on whether the target is less than or greater than the current node's value, move to the left or right child accordingly. This approach efficiently defines a search path that is consistent with the properties of a BST.

Trees & BST

□ #501 Find Mode in Binary Search Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST

Lists: Denny Zhang

Problem

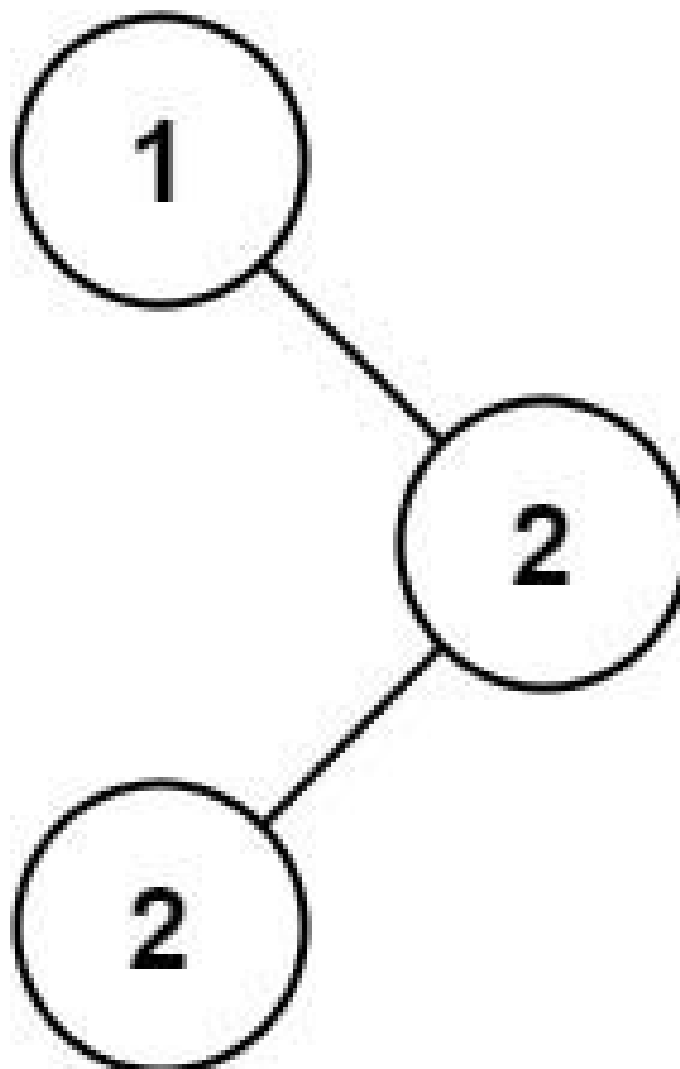
Given the root of a binary search tree (BST) with duplicates, return all the mode(s) (i.e., the most frequently occurred element) in it.

If the tree has more than one mode, return them in any order.

Assume a BST is defined as follows:

- The left subtree of a node contains only nodes with keys less than or equal to the node's key.
- The right subtree of a node contains only nodes with keys greater than or equal to the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:



Input: root = [1,null,2,2]

Output: [2]

Example 2:

Input: root = [0]

Output: [0]

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-105 \leq \text{Node.val} \leq 105$

Follow up: Could you do that without using any extra space? (Assume that the implicit stack space incurred due to recursion does not count).

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findMode(self, root: Optional[TreeNode]) -> List[int]:
        hmap = Counter()
        stack = [root]

        while stack:
            node = stack.pop()
            if node:
                hmap[node.val] += 1
                if node.right:
                    stack.append(node.right)

                if node.left:
                    stack.append(node.left)

        maxval = max(v for v in hmap.values()) if hmap else 0
        return [k for k in hmap if hmap[k] == maxval]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return all values that appear the most frequently in the BST."

This BST allows duplicates ($\text{left} \leq \text{node}$, $\text{right} \geq \text{node}$).

Return all modes – there can be multiple. Any order is fine. Right?

Follow-up: can we do it without extra space ($O(1)$ beyond recursion stack)?

```

#
# "[1,null,2,2]: 2 appears twice, 1 once → mode is [2] ✓
# [0]: only element → mode is [0] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# DFS or BFS over all nodes and count frequencies with a hash map (Counter).
# Then collect all keys whose count equals the maximum frequency.
# Time: O(n). Space: O(n) for the Counter. Should I go ahead and optimize?"
class _501_FindMode_Brute:
    def findMode(self, root: Optional['TreeNode']) -> List[int]:
        freq: Counter = Counter()
        stack = [root]
        while stack:
            node = stack.pop()
            if node:
                freq[node.val] += 1
                stack.append(node.left)
                stack.append(node.right)
        if not freq:
            return []
        max_freq = max(freq.values())
        return [k for k, v in freq.items() if v == max_freq]

# PHASE 3 — OPTIMIZE (O(1) space using in-order traversal)
# "BST in-order traversal visits nodes in sorted order.
# Duplicates in a BST are adjacent in in-order – so we can count runs without a map.
# Track: current value, current run count, max count seen, result list.
# When current run count exceeds max, reset result. When equal, append.
# Use nonlocal to mutate state inside the inner helper."

# PHASE 4 — CLEAN CODE
class _501_FindMode:
    def findMode(self, root: Optional['TreeNode']) -> List[int]:
        modes: List[int] = []
        cur_val = [None]
        cur_cnt = [0]
        max_cnt = [0]
        def inorder(node):
            if not node:
                return
            inorder(node.left)
            if node.val == cur_val[0]:
                cur_cnt[0] += 1
            else:
                cur_val[0] = node.val
                cur_cnt[0] = 1
            if cur_cnt[0] > max_cnt[0]:
                max_cnt[0] = cur_cnt[0]
                modes.clear()

```



```

        modes.append(node.val)
    elif cur_cnt[0] == max_cnt[0]:
        modes.append(node.val)
    inorder(node.right)
inorder(root)
return modes

```

PHASE 5 — TEST & DEBUG

```

# root=[1,null,2,2] - in-order: 1, 2, 2
# node=1: cur_val=1,cnt=1,max=1 → modes=[1].
# node=2: cur_val=2,cnt=1,max=1 → cnt==max → modes=[1,2].
# node=2: cur_val=2,cnt=2,max=1 → cnt>max → max=2,modes=[2].
# return [2] ✓
# root=[0]: in-order: 0. modes=[0] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Brute force: O(n) time, O(n) space.
# O(1) space version: O(n) time, O(1) extra space (modes list not counted).
# The in-order run-counting trick works because BST duplicates are always adjacent
# in sorted traversal – a key property of this BST variant.
# Given more time I'd ask: what if it were a regular BST (no duplicates)?
# The mode would always be the single element – trivially the root if n=1."

```

◆ Quick Review

Key Insights

- The BST in-order traversal yields a sorted order, which helps in counting consecutive duplicates.
- A two-pass in-order traversal can be used: one pass to determine the maximum frequency (maxCount) and a second pass to collect all values that match this frequency.
- The solution can be implemented without extra space (apart from recursion stack) by leveraging constant space counters and state

variables.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is processed twice (once in each traversal).

Space Complexity: $O(h)$ where h is the height of the tree due to the recursion stack. $O(1)$ extra space is used aside from this.

Solution

We perform an in-order traversal of the BST twice. The first traversal determines the maximum frequency of any value in the BST by comparing the current node's value with the previously visited node. Since the in-order traversal processes nodes in sorted order, duplicate values are consecutive, making the counting simple. After calculating `maxCount`, we reset our state and perform a second in-order traversal to collect all node values with frequency equal to `maxCount`. The key trick is to leverage the BST property (sorted in-order traversal) to count frequencies in one pass without using additional data structures like hash maps.

Trees & BST

□ #543 Diameter of Binary Tree

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS**

Lists: Grind 169

Problem

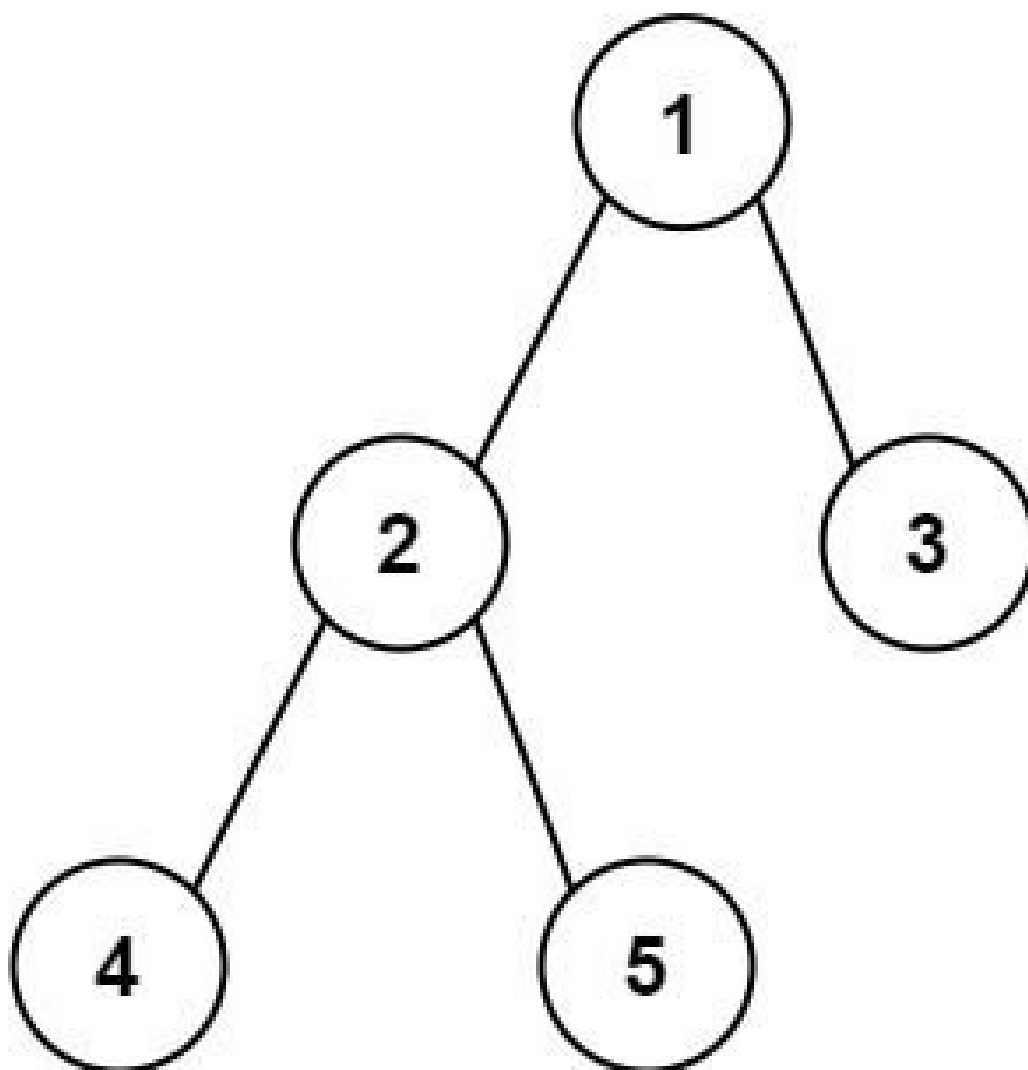
Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is the length of the longest path between any two nodes in a tree.

This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

Output: 1

Constraints:

- The number of nodes in the tree is in the range [1, 10⁴].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        def depth(root):
            if not root:
                return 0

            return max(depth(root.left), depth(root.right)) + 1

        self.ans = 0
        def diameter(root):
            if not root:
                return 0

            l = depth(root.left)

            r = depth(root.right)
            self.ans = max(self.ans, l+r)
            diameter(root.left)
            diameter(root.right)

            return self.ans

        return diameter(root)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Diameter = the longest path between any two nodes, measured in edges.

The path may or may not pass through the root.

An empty tree has diameter 0. A single node has diameter 0 (no edges). Right?"

#

"[1,2,3,4,5]: longest path is 4→2→1→3, 3 edges → 3 ✓

```
# [1,2]: one edge → 1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
```

```
# For every node, compute the height of its left and right subtrees recursively.
```

```
# Diameter through that node = left_height + right_height; track the global max.
```

```
# Recomputes heights repeatedly – correct but redundant work.
```

```
# Time:  $O(n^2)$  on skewed trees. Space:  $O(h)$  stack.
```

```
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "I notice depth is recomputed repeatedly for the same nodes – that's the  $O(n^2)$ ."
```

```
# Fuse depth computation and diameter tracking into one DFS pass.
```

```
# Use a nonlocal variable (or single-element list) to capture the running max.
```

```
# depth(node) returns height AND updates the max diameter seen so far.
```

```
# Time:  $O(n)$  – one pass. Space:  $O(h)$ ."
```

PHASE 4 — CLEAN CODE

```
class _543_DiameterBinaryTree:
```

```
    def diameterOfBinaryTree(self, root: Optional['TreeNode']) -> int:
```

```
        best = [0]
```

```
        def depth(node):
```

```
            if not node:
```

```
                return 0
```

```
            left = depth(node.left)
```

```
            right = depth(node.right)
```

```
            best[0] = max(best[0], left + right) # diameter through this node
```

```
            return max(left, right) + 1 # height for parent
```

```
        depth(root)
```

```
        return best[0]
```

PHASE 5 — TEST & DEBUG

```
# root=[1,2,3,4,5]:
```

```
# depth(4)=1. depth(5)=1. depth(2): best=max(0,1+1)=2. return 2.
```

```
# depth(3)=1. depth(1): best=max(2,2+1)=3. return 3.
```

```
# return best[0]=3 ✓
```

```
# root=[1,2]: depth(2)=1. depth(1): best=max(0,1+0)=1. return best=1 ✓
```

```
# root=None: depth(None)=0. best stays 0. return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(h)$ ."
```

```
# The single-element list best=[0] is a common Python pattern to mutate
```

```
# a value from inside a nested function without making it a class attribute.
```

```
# Equivalent to 'nonlocal best' with a plain int, but the list avoids the
```

```
# nonlocal keyword – both are valid in an interview.
```

```
# Given more time I'd ask: what's the longest path by node count (not edges)?
```

```
# Just return best[0] + 1 when best > 0. Or: diameter in edge terms is already
```

```
# what we compute – node count would be edges + 1."
```

◆ Quick Review

Key Insights

- Use depth-first search (DFS) to explore the tree.
- The diameter at a node is the sum of the depths of its left and right subtrees.
- Track the maximum diameter found during the DFS traversal.
- Recursively calculating the depth of subtrees provides an efficient solution.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree.

Space Complexity: $O(n)$, in the worst-case scenario due to the recursion stack (skewed tree).

Solution

The solution employs a DFS traversal of the binary tree. At every node in the tree, compute the depth of the left and right subtrees. The sum of these depths gives the potential diameter (longest path through that node). Update a global maximum diameter variable if

**this sum exceeds the current maximum.
Finally, return the maximum diameter after
completing the DFS.**

Trees & BST

□ #572 Subtree of Another Tree

EAS

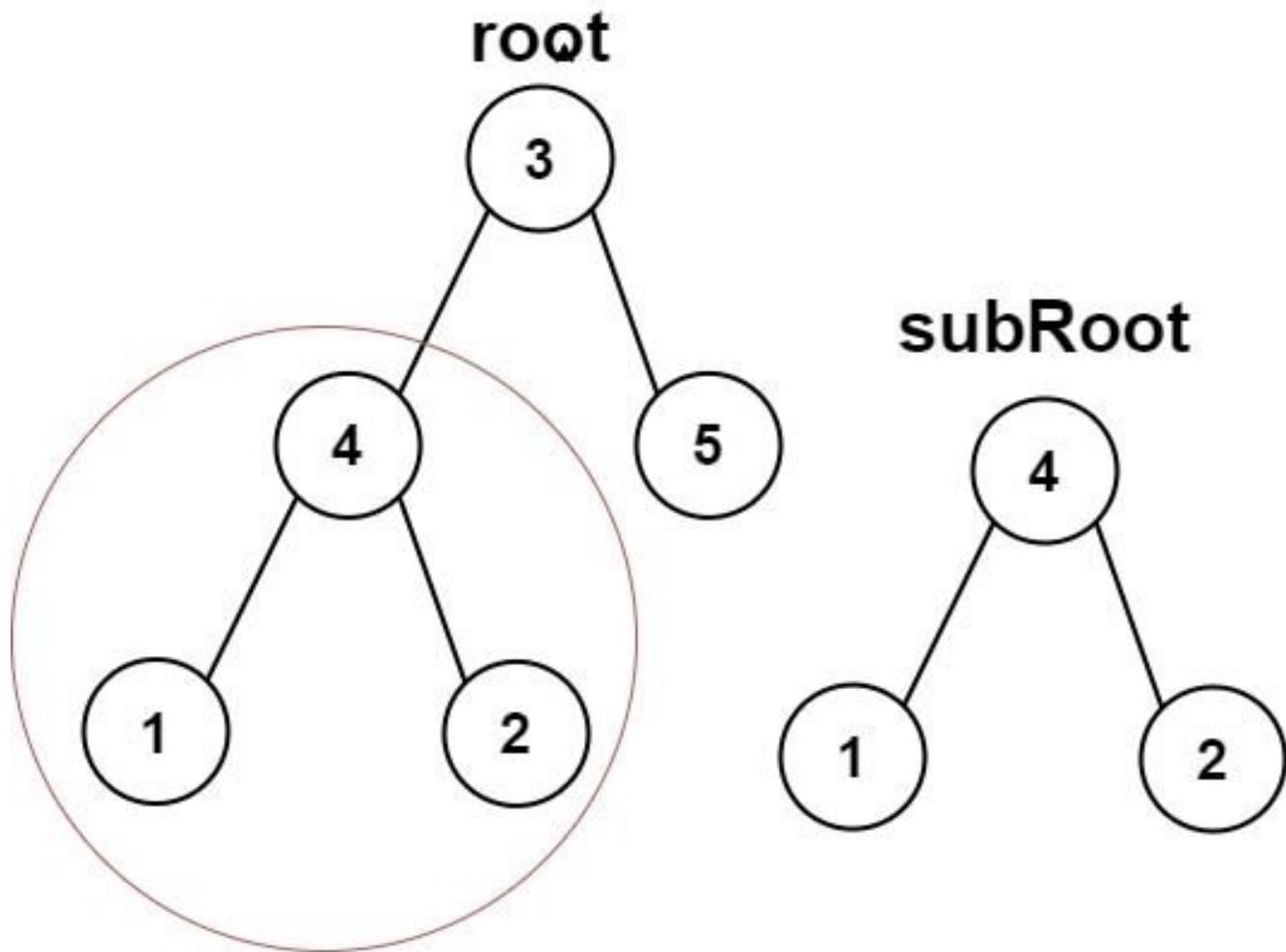
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · String Matching
Lists: Grind 169

Problem

Given the roots of two binary trees `root` and `subRoot`, return `true` if there is a subtree of `root` with the same structure and node values of `subRoot` and `false` otherwise.

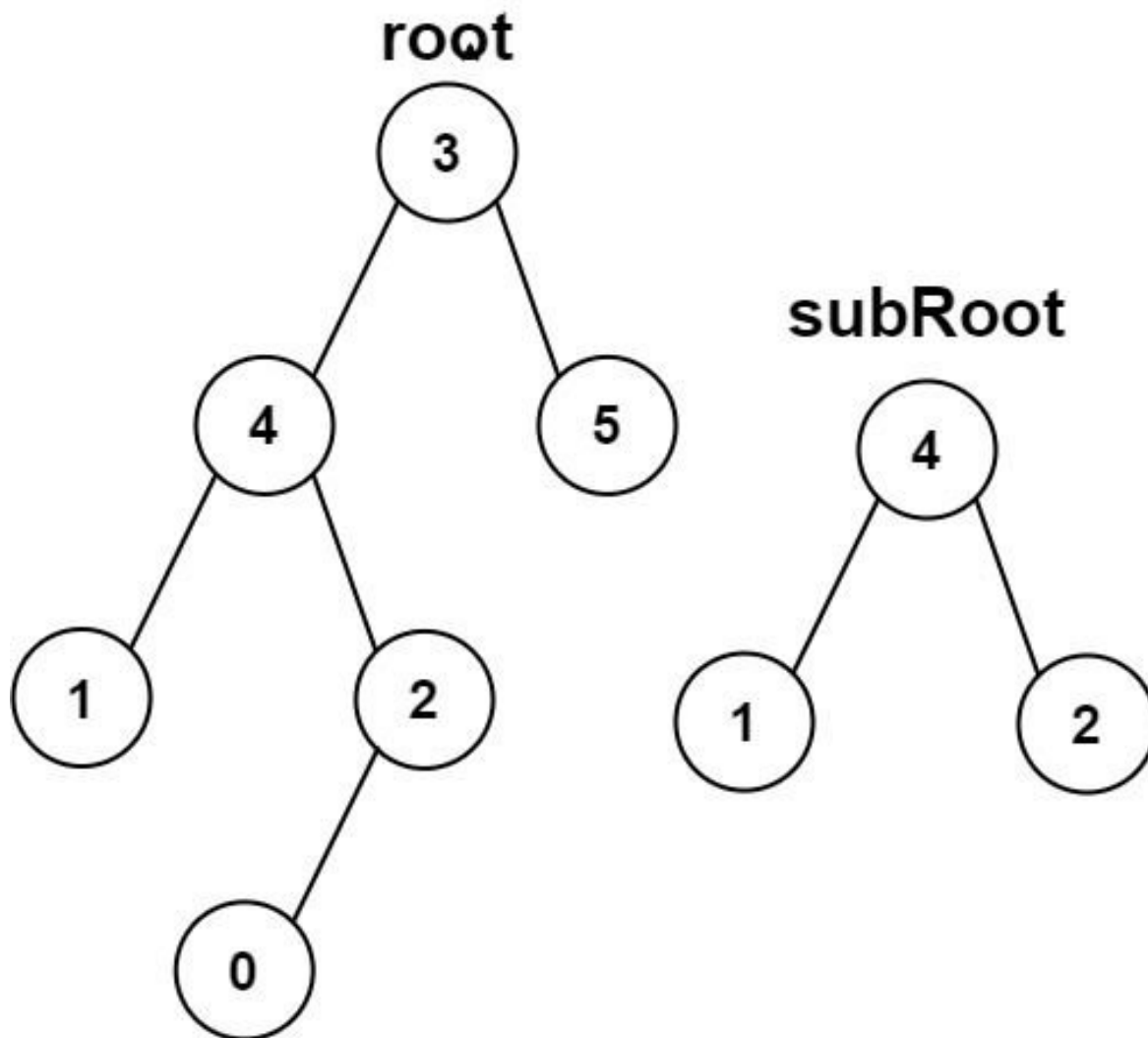
A subtree of a binary tree `tree` is a tree that consists of a node in `tree` and all of this node's descendants. The tree `tree` could also be considered as a subtree of itself.

Example 1:



Input: root = [3,4,5,1,2], subRoot = [4,1,2]
Output: true

Example 2:



Input: root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]
Output: false

Constraints:

- The number of nodes in the root tree is in the range [1, 2000].
- The number of nodes in the subRoot tree is in the range [1, 1000].
- $-104 \leq \text{root.val} \leq 104$
- $-104 \leq \text{subRoot.val} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
# Subtree of Another Tree
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode])
    -> bool:
        def same(p,q):

            if not p or not q:
                return p == q

            if p.val != q.val:
                return False

            return same(p.left, q.left) and same(p.right, q.right)

        def subtree(root, subRoot):
            if not root:

                return False

            if same(root,subRoot):
                return True

            return subtree(root.left, subRoot) or subtree(root.right, subRoot)

        return subtree(root, subRoot)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return true if subRoot appears as a subtree anywhere in root.

A subtree means a node and ALL its descendants match – not just a portion.

The tree itself is a subtree of itself. Right?"

#

```
# "root=[3,4,5,1,2], subRoot=[4,1,2]: node 4 in root has left=1,right=2 → matches ✓
# root=[3,4,5,1,2,null,null,null,null,0], subRoot=[4,1,2]:
# node 4 in root has extra child 0 → doesn't match ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For every node in root, run the same-tree check against all of subRoot.
# If any starting node matches the entire subtree, return true.
# Correct, but rescans overlapping structure many times.
# Time: O(n * m). Space: O(h) stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Same O(m*n) is standard. Two clean inner helpers:
# same(p,q) – checks if two trees are identical (reuse Same Tree logic).
# subtree(node, sub) – DFS root; at each node, try same().
# If same fails, recurse into left and right subtrees."
```

PHASE 4 — CLEAN CODE

```
class _572_SubtreeAnotherTree:
    def isSubtree(self, root: Optional['TreeNode'], subRoot: Optional['TreeNode'])
    -> bool:
        def same(p, q):
            if not p or not q:
                return p == q
            return p.val == q.val and same(p.left, q.left) and same(p.right,
q.right)
        def subtree(node, sub):
            if not node:
                return False
            if same(node, sub):
                return True
            return subtree(node.left, sub) or subtree(node.right, sub)
        return subtree(root, subRoot)
```

PHASE 5 — TEST & DEBUG

```
# root=[3,4,5,1,2], subRoot=[4,1,2]:
# subtree(3,[4,1,2]): same(3,4)? 3≠4 → False. recurse left/right.
# subtree(4,[4,1,2]): same(4,4)? 4==4.
# same(1,1)→same(None,None)T, same(None,None)T→T.
# same(2,2)→T. → same=True → return True ✓
# root has extra 0 child under 4:
# same(4,4): same(1,1)T. same(2(with 0),2)? 2==2 but same(0,None)→False →
same=False.
# subtree(1,[4,1,2]): same(1,4)? False. subtree deeper → all False → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n) – at each of m nodes we may run same() which takes O(n).
# Space O(h_root + h_sub) for call stacks.
# Optimisation: serialise both trees and use string matching (KMP or Rabin-Karp)
```

```
# for O(m+n) time – worth mentioning as an advanced follow-up.  
# Be careful with serialisation: '[1,2]' would incorrectly match '[12]' without  
# delimiters like '#' for null and ',' between values.  
# Given more time I'd ask: what if we query many different subRoots against  
# the same root? Pre-serialise root once and use string search for each query."
```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to visit each node in the main tree.
- At each node, check if the subtree starting from that node is identical to subRoot.
- A helper function is used to compare the structure and node values of two trees.
- Consider recursion as the primary approach to traverse and compare trees.
- Early exit when a mismatch is found can optimize average-case performance.

Space & Time

Time Complexity: $O(m * n)$ in the worst case, where m is the number of nodes in the main tree and n is the number of nodes in the subRoot tree.

Space Complexity: $O(\max(\text{depth of main tree}, \text{depth of subRoot}))$ due to recursive call stack.

Solution

We solve the problem by performing a DFS traversal on the main tree. For each visited node, a helper function checks whether the subtree starting at that node is identical to subRoot by comparing node values and recursively checking both left and right subtrees. This method leverages recursion effectively and provides early termination if mismatches occur, allowing for efficient traversal and comparison.

DFS

Round 1 · High · Easy · 2 questions

DFS

□ #463 Island Perimeter

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix
Lists: Denny Zhang**

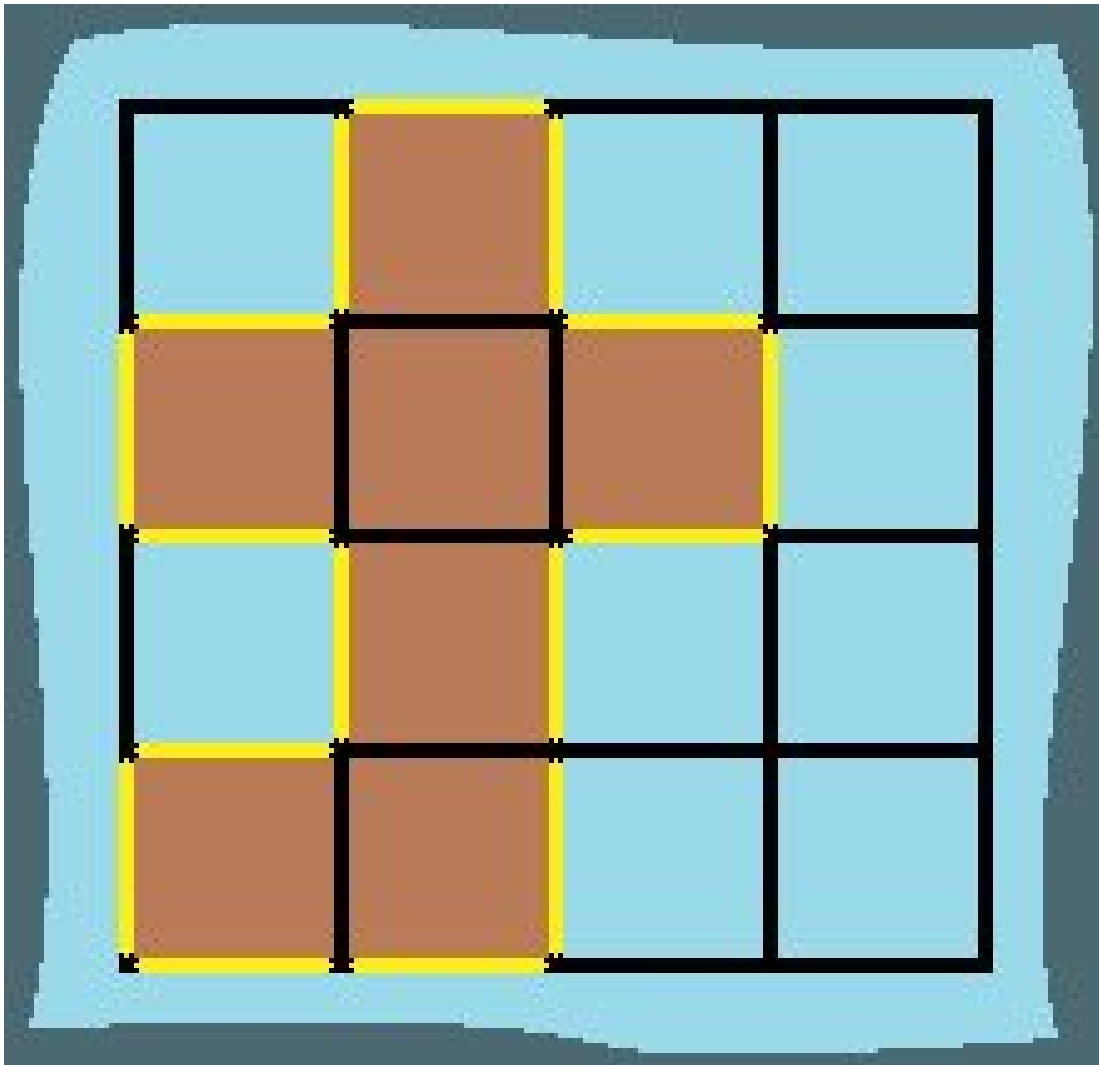
Problem

You are given row x col grid representing a map where $\text{grid}[i][j] = 1$ represents land and $\text{grid}[i][j] = 0$ represents water.

Grid cells are connected horizontally/vertically (not diagonally). The grid is completely surrounded by water, and there is exactly one island (i.e., one or more connected land cells).

The island doesn't have "lakes", meaning the water inside isn't connected to the water around the island. One cell is a square with side length 1. The grid is rectangular, width and height don't exceed 100. Determine the perimeter of the island.

Example 1:



Input: `grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]`

Output: 16

Explanation: The perimeter is the 16 yellow stripes in the image above.

Example 2:

Input: `grid = [[1]]`

Output: 4

Example 3:

Input: `grid = [[1,0]]`

Output: 4

Constraints:

- `row == grid.length`

- `col == grid[i].length`
- `1 <= row, col <= 100`
- `grid[i][j]` is 0 or 1.
- There is exactly one island in grid.

My LeetCode Solution

• My LeetCode Solution

```
# Island Perimeter
class Solution:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        rows = len(grid)
        cols = len(grid[0])

        islands, neighbors = 0,0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:

                    islands +=1

                    if r+1 < rows and grid[r+1][c] == 1:
                        neighbors += 1

                    if c+1 < cols and grid[r][c+1] == 1:
                        neighbors += 1

        return islands * 4 - neighbors*2
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"The grid has exactly one island – no lakes. Each land cell is a unit square.

We return the total perimeter, which is the count of exposed edges. Right?

Exposed means the edge faces water or the grid boundary."

#

"Quick check: `[[1]]` → 4 sides, all exposed → 4 ✓. `[[1,0]]` → 4 sides, all exposed → 4 ✓.

Two adjacent land cells: each has 4 sides but share 1 edge → $2 \times 4 - 2 = 6$."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every land cell, I check all 4 neighbours – if a neighbour is out of bounds or water,

that edge contributes 1 to the perimeter. Sum these up across all land cells.

Time: $O(m \times n)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _463_IslandPerimeterBrute:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        perimeter = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                        nr, nc = r + dr, c + dc
                        if nr < 0 or nr >= rows or nc < 0 or nc >= cols or
grid[nr][nc] == 0:
                            perimeter += 1
        return perimeter
```

PHASE 3 — OPTIMIZE

"I notice we're checking 4 neighbours per cell. Instead of neighbour checks, we can use a counting formula:

Every land cell contributes 4 sides. Every shared edge between two adjacent land cells removes 2 sides (one from each). So:

$\text{perimeter} = \text{islands} \times 4 - \text{shared_edges} \times 2$

We only count each shared edge once by checking right-neighbour and down-neighbour.

Same $O(m \times n)$ time, same $O(1)$ space – but a cleaner mental model."

PHASE 4 — CLEAN CODE

```
class _463_IslandPerimeter:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        islands = 0
        shared = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    islands += 1
                    if r + 1 < rows and grid[r + 1][c] == 1:
                        shared += 1
                    if c + 1 < cols and grid[r][c + 1] == 1:
                        shared += 1
        return islands * 4 - shared * 2
```

PHASE 5 — TEST & DEBUG

`grid=[[1]]`: islands=1, shared=0. return 4 ✓

```
# grid=[[1,1]]: islands=2. c=0→c+1=1 valid,grid[0][1]=1→shared=1. return 2*4-1*2=6
✓
# grid=[[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]:
# Count land cells=8 (islands). Count shared edges (right+down only)=... =8.
# return 8*4-8*2=32-16=16 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(1)$.

The counting formula is elegant – worth presenting even if the brute force also works.

Both approaches are the same complexity; the formula shows mathematical insight.

Given more time I'd ask: what if there are multiple islands?

The formula still works per-island if we isolate them, or we can return a list

of perimeters – same logic applied per connected component."

◆ Quick Review

Key Insights

- Each land cell contributes 4 edges if isolated.
- For every adjacent pair of land cells (neighbors horizontally or vertically), the shared edge should be subtracted from the total perimeter.
- Instead of DFS/BFS, we can iterate the grid and check the four neighbors for each land cell.
- Alternatively, one may use DFS/BFS to visit all connected cells if needed, but an iterative solution is straightforward with $O(\text{rows} * \text{cols})$ time complexity.

Space & Time

Time Complexity: $O(n * m)$, where n is the number of rows and m is the number of columns.

Space Complexity: $O(1)$ for the iterative solution (excluding the input storage).

Solution

We iterate through every cell in the grid. For each land cell, we initially add 4 to the perimeter. Then we check its four neighbors (up, down, left, right). For every neighbor that is also land, we subtract 1 from the current cell's contribution since that side is not exposed. This simple approach ensures that shared borders between adjacent land cells are not over-counted.

DFS

□ #733 Flood Fill

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix
Lists: Grind 169**

Problem

You are given an image represented by an $m \times n$ grid of integers `image`, where `image[i][j]` represents the pixel value of the image. You are also given three integers `sr`, `sc`, and `color`. Your task is to perform a flood fill on the image starting from the pixel `image[sr][sc]`.

To perform a flood fill:

- 1. Begin with the starting pixel and change its color to `color`.**
- 2. Perform the same process for each pixel that is directly adjacent (pixels that share a side with the original pixel, either horizontally or vertically) and shares the same color as the starting pixel.**

3. Keep repeating this process by checking neighboring pixels of the updated pixels and modifying their color if it matches the original color of the starting pixel.
4. The process stops when there are no more adjacent pixels of the original color to update.

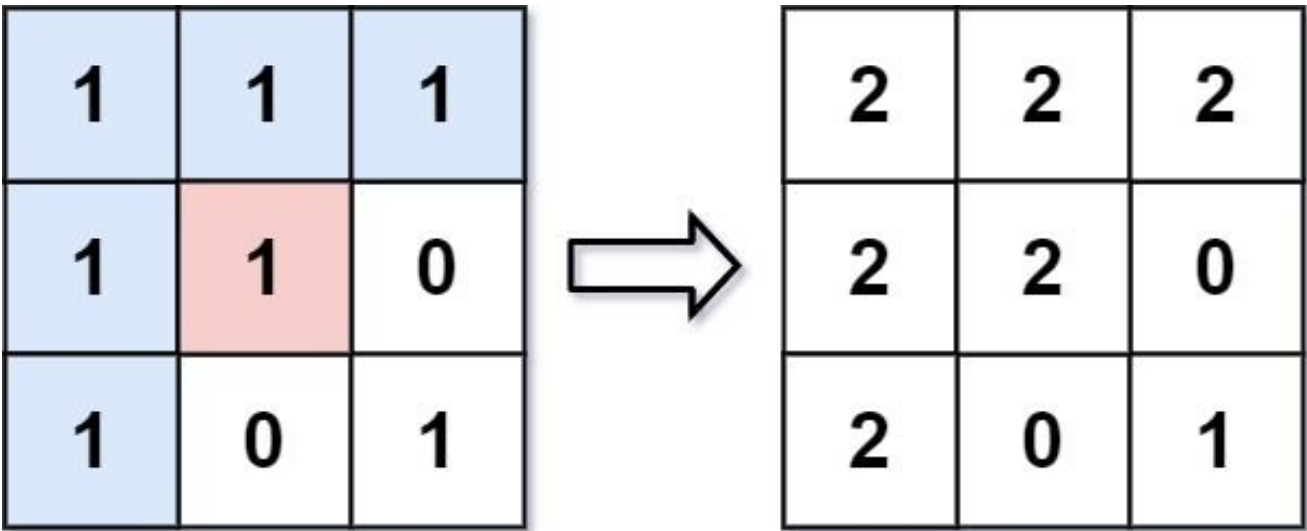
Return the modified image after performing the flood fill.

Example 1:

Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2

Output: [[2,2,2],[2,2,0],[2,0,1]]

Explanation:



From the center of the image with position (sr, sc) = (1, 1) (i.e., the red pixel), all pixels connected by a path of the same color as the

starting pixel (i.e., the blue pixels) are colored with the new color.

Note the bottom corner is not colored 2, because it is not horizontally or vertically connected to the starting pixel.

Example 2:

Input: image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0

Output: [[0,0,0],[0,0,0]]

Explanation:

The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.

Constraints:

- $m == \text{image.length}$
- $n == \text{image}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $0 \leq \text{image}[i][j], \text{color} < 216$
- $0 \leq \text{sr} < m$
- $0 \leq \text{sc} < n$

My LeetCode Solution

☐ • My LeetCode Solution

```

# Flood Fill
class Solution:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int)
-> List[List[int]]:
    def dfs(x,y):
        if x<0 or y<0 or x>= rows or y>= cols:
            return

        if image[x][y] != startcolor or (x,y) in seen:
            return

        image[x][y] = color
        seen.add((x,y))
        dfs(x+1, y)
        dfs(x-1, y)
        dfs(x,y+1)
        dfs(x,y-1)

    startcolor = image[sr][sc]
    seen = set()
    rows = len(image)

    cols = len(image[0])
    if startcolor == color:
        return image

    dfs(sr,sc)

    return image

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Starting from (sr, sc), change all connected cells of the same colour to the new colour.

Connected means 4-directionally (up/down/left/right). Modify and return the image. Right?

Edge case: if the starting colour already equals the new colour – return immediately,

otherwise we'd loop infinitely or process unchanged cells."

#

```

# "[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2:
# All 1s connected to (1,1) become 2. The bottom-right 1 is isolated → stays 1. ✓
# [[0,0,0],[0,0,0]], sr=0,sc=0, color=0: startcolor==color → no change → return
as-is ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# I'll use BFS: enqueue the start cell, paint it the new colour, then for each cell
I dequeue,
# enqueue all 4-directional neighbours that still have the original start colour.
# This visits every connected cell exactly once.
# Time: O(m*n). Space: O(m*n) for the queue. Should I go ahead and optimize?"
class _733_FloodFill_Brute:
    def floodFill(self, image, sr, sc, color):
        from collections import deque
        start = image[sr][sc]
        if start == color:
            return image
        rows, cols = len(image), len(image[0])
        queue = deque([(sr, sc)])
        image[sr][sc] = color
        while queue:
            r, c = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and image[nr][nc] == start:
                    image[nr][nc] = color
                    queue.append((nr, nc))
        return image

# PHASE 3 — OPTIMIZE
# "DFS is cleaner. Key insight: once we paint a cell, its colour changes to `color`
—
# it can never match `startcolor` again. So we don't need a `seen` set at all.
# The colour change itself is the visited marker.
# This saves O(m*n) extra space."

# PHASE 4 — CLEAN CODE
class _733_FloodFill:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int) ->
List[List[int]]:
        start = image[sr][sc]
        if start == color:
            return image
        rows, cols = len(image), len(image[0])
        def dfs(r, c):
            if r < 0 or r >= rows or c < 0 or c >= cols:
                return
            if image[r][c] != start:
                return

```

```

        image[r][c] = color # paint before recursing – acts as visited marker
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)
    dfs(sr, sc)
    return image

```

PHASE 5 — TEST & DEBUG

```

# image=[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2, start=1:
# dfs(1,1): image[1][1]=2. recurse 4 dirs.
# dfs(2,1): image[2][1]=1==start → paint 2. recurse: dfs(3,1)→00B, dfs(1,1)→now 2≠1
stop.
# dfs(2,2)→image[2][2]=1 but that's the isolated corner – wait:
# (2,1)→image[2][1]=1✓→paint. Then dfs(2,2)→image[2][2]=1✓→paint? No –
# is (2,2) reachable from (1,1)? Path: (1,1)→(2,1)→(2,2).
# image[2][2]=1 and image[2][1]=1, so yes connected. But expected output keeps it 1?
# Re-check: image=[[1,1,1],[1,1,0],[1,0,1]]. (2,2) is adjacent to (2,1)=0 and
(1,2)=0.
# So (2,2) connects to (2,1)=0 (water) – not reachable from island. ✓
# All connected 1s become 2. Bottom-right stays 1 (not connected). ✓
#
# start==color: image=[[0,0,0],[0,0,0]], color=0 → return immediately ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(m*n) – each cell visited at most once.
# Space O(m*n) call stack in the worst case (all cells same colour, grid is a long
chain).
# The 'paint first then recurse' order is the key – it prevents revisiting and
# eliminates the need for a separate visited set. Mention this explicitly.
# The early return when startcolor == color prevents infinite looping – critical
edge case.
# Given more time I'd ask: what if connectivity is 8-directional (diagonals too)?
# Add four more direction pairs to the dfs calls – same algorithm."

```

◆ Quick Review

Key Insights

- The flood fill operation can be executed using either DFS (Depth First Search) or BFS (Breadth First Search).

- A check is needed at the beginning to determine if the new color is the same as the original; if so, no changes are needed.
- The algorithm recursively or iteratively visits each pixel that is directly adjacent and matches the original color, then updates its color.
- The problem has a worst-case scenario where all pixels are the same color, leading to a full traversal of the grid.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the dimensions of the image grid, because in the worst-case all cells are visited.

Space Complexity: $O(m * n)$ in the worst-case due to the recursion stack (or BFS queue) if the entire image needs to be filled.

Solution

We use a DFS approach to perform the flood fill. The algorithm starts at the given starting pixel and uses recursion to fill connected pixels with the same original color. The following steps are key: Check if the starting pixel's color is already the target color; if so,

return the image immediately. Define a recursive helper function that: Checks boundary conditions. Updates the current pixel's color. Recursively calls itself on the four adjacent directions (up, down, left, right). Return the modified image after the recursion completes. Data structures used: Recursion call stack for the DFS implementation. Variables to store the image dimensions and original color.

Round 2 · High · Medium

Round 2

High Priority · Medium

116 questions · 11 patterns

**Arrays & Hashing #57 #238 #281 #307 #454
#525 #560 #1010 #1272 #1429 #3159**

String #8 #249 #271 #635 #1138

**Two Pointers #11 #15 #16 #31 #75 #161 #167
#186 #189 #532 #923 #986 #1055 #2563**

**Sliding Window #3 #159 #340 #424 #438 #487
#1100 #1248**

Sorting #49 #56 #1152

**Binary Search #33 #34 #153 #300 #362 #528
#852 #981 #1011 #1060 #1062 #1533**

**Matrix #36 #48 #54 #59 #64 #73 #74 #221
#311 #498 #531 #723 #835 #1198**

**Trees & BST #98 #102 #103 #105 #199 #230
#235 #236 #250 #285 #298 #314 #333 #366**

Round 1 · High · Easy

p.283

**#437 #545 #549 #582 #662 #863 #1120 #1490
#1522 #1650**

**DFS #130 #133 #200 #207 #210 #261 #310
#323 #417 #490 #694 #695 #721 #785 #1236
#1242**

Graphs #128 #277 #1136

BFS #286 #322 #542 #994 #1197 #1730

Arrays & Hashing

Round 2 · High · Medium · 11 questions

Arrays & Hashing

□ #57 Insert Interval

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: Grind 169

Problem

You are given an array of non-overlapping intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$ represent the start and the end of the i th interval and intervals is sorted in ascending order by start_i . You are also given an interval $\text{newInterval} = [\text{start}, \text{end}]$ that represents the start and end of another interval.

Insert newInterval into intervals such that intervals is still sorted in ascending order by start_i and intervals still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return intervals after the insertion.

Note that you don't need to modify intervals in-place. You can make a new array and return it.

Example 1:

Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
 Output: [[1,5],[6,9]]

Example 2:

Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
 Output: [[1,2],[3,10],[12,16]]
 Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 105$
- intervals is sorted by start_i in ascending order.
- $\text{newInterval.length} == 2$
- $0 \leq \text{start} \leq \text{end} \leq 105$

My LeetCode Solution

• My LeetCode Solution

```
# Insert Interval
class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        intervals.append(newInterval)
        intervals.sort()
        ans=[intervals[0]]

        for s,e in intervals[1:]:
            last=ans[-1]
            if last[-1] >= s:
```

```

        last[-1] = max(last[-1], e)
    else:
        ans.append([s,e])

    return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "We have a sorted list of non-overlapping intervals. Insert a new interval,
# merging wherever it overlaps existing ones. Return the result sorted.
# We don't need to modify in-place – a new array is fine. Right?"
#
# "[[1,3],[6,9]], newInterval=[2,5]:
# [2,5] overlaps [1,3] → merge to [1,5]. [6,9] doesn't overlap → keep. →
# [[1,5],[6,9]] ✓
# [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval=[4,8]:
# Overlaps [3,5],[6,7],[8,10] → merge all to [3,10]. → [[1,2],[3,10],[12,16]] ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with the brute force to lock in the logic.
# I'll append the new interval to the list, sort everything by start time,
# then run a standard merge-intervals pass to collapse all overlaps.
# Time: O(n log n) – the sort dominates. Space: O(n). Should I go ahead and
# optimize?"

```

```

class _57_InsertInterval_Brute:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) ->
    List[List[int]]:
        intervals.append(newInterval)
        intervals.sort()
        result = [intervals[0]]
        for s, e in intervals[1:]:
            if result[-1][1] >= s:
                result[-1][1] = max(result[-1][1], e)
            else:
                result.append([s, e])
        return result

```

PHASE 3 — OPTIMIZE

```

# "The input is already sorted – we can exploit that to avoid the O(n log n) sort.
# Three phases in one O(n) pass:
# 1. Add all intervals that end BEFORE newInterval starts (no overlap).
# 2. Merge all intervals that overlap newInterval into one combined interval.
# Overlap condition: interval[start] <= newInterval[end].
# 3. Add all remaining intervals that start AFTER the merged interval ends.

```

```
#
# Pseudocode:
# i = 0
# while i < n and intervals[i][1] < new[0]: result.append(intervals[i]); i++
# while i < n and intervals[i][0] <= new[1]:
# new = [min(new[0], intervals[i][0]), max(new[1], intervals[i][1])]; i++
# result.append(new)
# result.extend(intervals[i:])
#
# Time: O(n). Space: O(n) for output."
```

PHASE 4 — CLEAN CODE

```
class _57_InsertInterval:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        result = []
        i, n = 0, len(intervals)
        # Phase 1: all intervals ending before newInterval starts
        while i < n and intervals[i][1] < newInterval[0]:
            result.append(intervals[i])
            i += 1
        # Phase 2: merge all overlapping intervals into newInterval
        while i < n and intervals[i][0] <= newInterval[1]:
            newInterval[0] = min(newInterval[0], intervals[i][0])
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1
        result.append(newInterval)
        # Phase 3: remaining intervals start after merged interval
        result.extend(intervals[i:])
        return result
```

PHASE 5 — TEST & DEBUG

```
# intervals=[[1,3],[6,9]], new=[2,5]:
# Phase 1: intervals[0][1]=3 >= new[0]=2 → stop. i=0.
# Phase 2: intervals[0][0]=1 <= new[1]=5 → new=[min(2,1),max(5,3)]=[1,5]. i=1.
# intervals[1][0]=6 > new[1]=5 → stop.
# result=[[1,5]]. extend [[6,9]]. → [[1,5],[6,9]] ✓
#
# intervals=[[1,2],[3,5],[6,7],[8,10],[12,16]], new=[4,8]:
# Phase 1: [1,2] ends at 2 < 4 → add. i=1.
# Phase 2: [3,5] start 3<=8→new=[3,8]. [6,7] start 6<=8→new=[3,8]. [8,10] start 8<=8→new=[3,10]. i=4.
# result=[[1,2],[3,10]]. extend [[12,16]]. → [[1,2],[3,10],[12,16]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n) – three phases together visit each interval once.
# Space O(n) for output.
# The two overlap conditions are the key:
# Non-overlap before: interval[end] < new[start].
```

```
# Non-overlap after: interval[start] > new[end] (Phase 3 – just extend).  
# Anything in between overlaps and gets merged.  
# Given more time I'd ask: what if intervals aren't sorted?  
# Sort first ( $O(n \log n)$ ), then apply the same three-phase approach."
```

◆ Quick Review

Key Insights

- The intervals array is sorted by start times.
- Three main steps are used:
- Add intervals that come completely before the new interval.
- Merge overlapping intervals with the new interval.
- Append intervals that start after the new (merged) interval.
- Iterating through the list once allows an $O(n)$ solution.

Space & Time

Time Complexity: $O(n)$ since we scan the list once.

Space Complexity: $O(n)$ to store the resulting list of intervals.

Solution

Traverse the intervals list and perform the following: Append all intervals that end before

the new interval starts. For intervals that overlap with newInterval, update newInterval to merge them (using min for start and max for end). Append the new merged interval. Append all remaining intervals. This ensures that all intervals remain non-overlapping and sorted with a single pass.

Arrays & Hashing

□ #238 Product of Array Except Self

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Prefix Sum
Lists: Grind 169

Problem

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [24,12,8,6]
```

Example 2:

```
Input: nums = [-1,1,0,-3,3]
Output: [0,0,9,0,0]
```

Constraints:

- $2 \leq \text{nums.length} \leq 105$
- $-30 \leq \text{nums}[i] \leq 30$
- The input is generated such that $\text{answer}[i]$ is guaranteed to fit in a 32-bit integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array does not count as extra space for space complexity analysis.)

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        p = [1] * len(nums)
        s = [1] * len(nums)
        ans = [1] * len(nums)

        for i in range(1, n):
            p[i] = p[i-1] * nums[i-1]

        for i in reversed(range(n-1)):
            s[i] = s[i+1] * nums[i+1]

        return [p[i]*s[i] for i in range(len(nums))]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return an array where answer[i] is the product of all elements except nums[i].
# Cannot use division. Must run in O(n). Follow-up: O(1) extra space. Right?
# Guaranteed no overflow in 32-bit – so no overflow concern."
#
```

```

# "[1,2,3,4]: answer[0]=2*3*4=24. answer[1]=1*3*4=12. answer[2]=1*2*4=8.
answer[3]=1*2*3=6 ✓
# [-1,1,0,-3,3]: zeros make most products 0 - [0,0,9,0,0] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For each index i, I multiply together all elements in the array except nums[i]
itself.
# That's a nested loop – O(n) work for each of the n positions.
# Time: O(n²). Space: O(1) extra. Should I go ahead and optimize?"
class _238_ProductExceptSelf_Brute:
    def productExceptSelf(self, nums):
        n = len(nums)
        result = []
        for i in range(n):
            product = 1
            for j in range(n):
                if j != i:
                    product *= nums[j]
            result.append(product)
        return result

# PHASE 3 — OPTIMIZE
# "Build prefix products and suffix products, multiply them.
# prefix[i] = product of everything to the LEFT of i.
# suffix[i] = product of everything to the RIGHT of i.
# answer[i] = prefix[i] * suffix[i].
# Time: O(n). Space: O(n) for two arrays.
#
# O(1) space follow-up: use the output array for prefix products,
# then do a single right-to-left pass with a running suffix multiplier."

# PHASE 4 — CLEAN CODE (O(1) extra space version)
class _238_ProductExceptSelf:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        result = [1] * n

        # Left pass: result[i] = product of all elements to the left of i
        for i in range(1, n):
            result[i] = result[i - 1] * nums[i - 1]

        # Right pass: multiply in the suffix product on the fly
        suffix = 1
        for i in range(n - 1, -1, -1):
            result[i] *= suffix
            suffix *= nums[i]

        return result

# PHASE 5 — TEST & DEBUG
# nums=[1,2,3,4]:

```

```
# Left pass: result=[1,1,2,6].
# Right pass: i=3: result[3]=6*1=6, suffix=4.
# i=2: result[2]=2*4=8, suffix=12.
# i=1: result[1]=1*12=12, suffix=24.
# i=0: result[0]=1*24=24, suffix=24.
# → [24,12,8,6] ✓
# nums=[0,0]: result=[0,0] ✓ (zeros handled correctly by multiplication)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1) extra – output array doesn't count.
# The two-pass trick: left products first, then multiply right products on the fly.
# The suffix variable carries the running right product without an extra array.
# Given more time I'd ask: what if we're allowed division?
# total_product / nums[i] – but fails when nums[i]=0. Handle: count zeros,
# if two or more → all zeros. If one zero → all zeros except position of zero."
```

◆ Quick Review

Key Insights

- The product for each index can be computed by multiplying the product of all numbers to the left and the product of all numbers to the right of that index.
- Division is not allowed, so we must compute the cumulative products explicitly.
- Create two passes: one forward pass for prefix products and one backward pass for suffix products.
- Use the output array to store the cumulative product values, achieving $O(1)$ extra space by not using additional arrays.

Space & Time

Time Complexity: $O(n)$ because we iterate through the array twice.

Space Complexity: $O(1)$ extra space (excluding the output array).

Solution

The approach uses two passes over the input array: Initialize the answer array and fill it with prefix products. For each index i , $\text{answer}[i]$ contains the product of all elements before i . Traverse the array from the end using a running suffix product. Update the answer array by multiplying the current suffix product with the prefix product already stored. This method leverages an output array to store intermediate results and uses a temporary variable for the suffix product, ensuring constant extra space.

Arrays & Hashing

□ #281 Zigzag Iterator

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Queue · Iterator

Lists: Denny Zhang

Problem

Given two vectors of integers `v1` and `v2`, implement an iterator to return their elements alternately.

Implement the `ZigzagIterator` class:

- `ZigzagIterator(List<int> v1, List<int> v2)` initializes the object with the two vectors `v1` and `v2`.
- `boolean hasNext()` returns `true` if the iterator still has elements, and `false` otherwise.
- `int next()` returns the current element of the iterator and moves the iterator to the next element.

Example 1:

Input: `v1 = [1,2]`, `v2 = [3,4,5,6]`

Output: `[1,3,2,4,5,6]`

Explanation: By calling `next` repeatedly until `hasNext` returns `false`, the order of elements returned by `next` should be: `[1,3,2,4,5,6]`.

Example 2:

Input: v1 = [1], v2 = []
Output: [1]

Example 3:

Input: v1 = [], v2 = [1]
Output: [1]

Constraints:

- $0 \leq v1.length, v2.length \leq 1000$
- $1 \leq v1.length + v2.length \leq 2000$
- $-231 \leq v1[i], v2[i] \leq 231 - 1$

Follow up: What if you are given k vectors? How well can your code be extended to such cases?

Clarification for the follow-up question:

The "Zigzag" order is not clearly defined and is ambiguous for $k > 2$ cases. If "Zigzag" does not look right to you, replace "Zigzag" with "Cyclic".

Follow-up Example:

Input: v1 = [1,2,3], v2 = [4,5,6,7], v3 = [8,9]
Output: [1,4,8,2,5,9,3,6,7]

My LeetCode Solution

- My LeetCode Solution

```

class ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.q = deque()
        if v1:
            self.q.append((v1, 0))

        if v2:
            self.q.append((v2, 0))

    def next(self) -> int:

        v, i = self.q.popleft()
        if i+1 < len(v):
            self.q.append((v, i+1))
        return v[i]

    def hasNext(self) -> bool:
        return len(self.q) > 0

# Your ZigzagIterator object will be instantiated and called as such:
# i, v = ZigzagIterator(v1, v2), []

# while i.hasNext(): v.append(i.next())

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Alternate between v1 and v2 element by element. When one is exhausted, continue with the remaining elements of the other. Right?
 # v1=[1,2], v2=[3,4,5,6] → [1,3,2,4,5,6] ✓
 # Follow-up: what if there are k vectors instead of 2?"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
 # At construction I'll interleave v1 and v2 into a flat list: take one from v1, one from v2,
 # repeat, then append remaining elements from whichever list is longer.
 # next() and hasNext() just read from that flat list.
 # Time: O(n) init. Space: O(n). Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Use a deque of (vector, index) pairs.
 # next(): popleft, return v[i], re-enqueue (v, i+1) if more elements remain.
 # hasNext(): deque is non-empty.

```
# This generalises trivially to k vectors – just enqueue all of them.
# Time: O(1) per next()/hasNext(). Space: O(k) for the deque."
```

PHASE 4 — CLEAN CODE

```
class _281_ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.q: deque = deque()
        if v1:
            self.q.append((v1, 0))
        if v2:
            self.q.append((v2, 0))
    def next(self) -> int:
        vec, idx = self.q.popleft()
        if idx + 1 < len(vec):
            self.q.append((vec, idx + 1))
        return vec[idx]
    def hasNext(self) -> bool:
        return bool(self.q)
```

PHASE 5 — TEST & DEBUG

```
# v1=[1,2], v2=[3,4,5,6]:
# q=[(v1,0),(v2,0)].
# next(): pop (v1,0)→return 1. re-enqueue (v1,1). q=[(v2,0),(v1,1)].
# next(): pop (v2,0)→return 3. re-enqueue (v2,1). q=[(v1,1),(v2,1)].
# next(): pop (v1,1)→return 2. no re-enqueue (idx+1=2=len). q=[(v2,1)].
# next(): 4, then 5, then 6. → [1,3,2,4,5,6] ✓
# v1=[1], v2=[]: q=[(v1,0)]. next()→1. hasNext()→False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(1) per operation. Space O(k) for the deque.
# Follow-up k vectors: init enqueues all k non-empty vectors. Same code, no changes.
# The deque acts as a round-robin scheduler – each vector gets one turn per cycle.
# Given more time I'd ask: what if vectors have different priorities (some should
# appear more often)? Enqueue them multiple times or use a priority queue."
```

◆ Quick Review

Key Insights

- Use a FIFO (first-in-first-out) structure to maintain the order of vectors which still have remaining elements.

- For each call to `next()`, select the next element from the current vector and then move that vector to the end of the queue if it still contains elements.
- The approach ensures proper zigzag (or cyclic) ordering among the vectors.
- When extending to k vectors, the same approach works: initialize the queue with each non-empty vector.

Space & Time

Time Complexity: $O(1)$ per `next()` call on average, with an overall $O(n)$ for n total elements.

Space Complexity: $O(k)$ for the queue structure (where k is the number of vectors), plus the storage for the input vectors.

Solution

We can solve the Zigzag Iterator problem by using a queue (or deque) to hold pairs of the vector's current index and the vector itself. For each call to `next()`, we remove the front element from the queue, return the current item from its associated vector, and if the vector has more elements, reinsert the pair

(with an updated index) into the back of the queue. This cyclic process naturally interleaves the elements from the provided vectors.

Arrays & Hashing

□ #307 Range Sum Query – Mutable

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Binary Indexed Tree · Segment
Tree

Lists: Denny Zhang

Problem

Given an integer array `nums`, handle multiple queries of the following types:

1. Update the value of an element in `nums`.
2. Calculate the sum of the elements of `nums` between indices `left` and `right` inclusive where `left <= right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `void update(int index, int val)` Updates the value of `nums[index]` to be `val`.
- `int sumRange(int left, int right)` Returns the sum of the elements of `nums` between indices `left` and `right` inclusive (i.e. `nums[left] +`

nums[left + 1] + ... + nums[right]).

Example 1:

Input

```
["NumArray", "sumRange", "update", "sumRange"]
```

```
[[[1, 3, 5]], [0, 2], [1, 2], [0, 2]]
```

Output

```
[null, 9, null, 8]
```

Explanation

```
NumArray numArray = new NumArray([1, 3, 5]);
```

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- $0 \leq \text{index} < \text{nums.length}$
- $-100 \leq \text{val} \leq 100$
- $0 \leq \text{left} \leq \text{right} < \text{nums.length}$
- At most $3 * 10^4$ calls will be made to update and sumRange.

My LeetCode Solution

• My LeetCode Solution

```
class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = [0]*(n+1)

    def update(self, i, change):
        i += 1
        while i < self.n + 1:
            self.tree[i] += change
            i += (i&-i)
```

```

def query(self, i):
    i += 1
    s = 0
    while i > 0:
        s += self.tree[i]
        i -= (i&-i)

    return s

```

```

class NumArray:

```

```

    def __init__(self, nums: List[int]):
        self.nums = nums
        n = len(nums)
        self.BIT = BIT(n)
        for i, n in enumerate(nums):
            self.BIT.update(i, n)

    def update(self, index: int, val: int) -> None:

```

```

        change = val - self.nums[index]
        self.BIT.update(index, change)
        self.nums[index] = val

```

```

    def sumRange(self, left: int, right: int) -> int:
        if not left:
            return self.BIT.query(right)

        return self.BIT.query(right) - self.BIT.query(left-1)

```

```

# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index, val)
# param_2 = obj.sumRange(left, right)

```

◆ Interview Approach · STAR-LC

Round 2 · High · Medium

p.305

PHASE 1 — CLARIFY

```
# "Support two operations on an array: update a single element, and query
# the prefix sum from left to right inclusive. Up to 3*10^4 calls. Right?
# The challenge is making both operations fast – not O(n) each."
#
# "NumArray([1,3,5]): sumRange(0,2)=9. update(1,2)→nums=[1,2,5]. sumRange(0,2)=8 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Store the array directly. update() just sets nums[index] = val in O(1).
# sumRange(left, right) sums nums[left..right] with a linear scan in O(n).
# With up to 3*10^4 queries this is O(n * queries) total – too slow for large n.
# Time: update O(1), sumRange O(n). Space: O(n). Should I go ahead and optimize?"
```

```
class _307_NumArray_Brute:
    def __init__(self, nums):
        self.nums = nums[:]
    def update(self, index, val):
        self.nums[index] = val
    def sumRange(self, left, right):
        return sum(self.nums[left:right+1])
```

PHASE 3 — OPTIMIZE

```
# "Binary Indexed Tree (Fenwick Tree) – each node stores a partial sum covering
# a range determined by its lowest set bit. Both update and query run in O(log n).
# Update: add change to index and all its ancestors (i += i & -i).
# Query prefix sum up to i: add and walk down (i -= i & -i).
# For range [left, right]: query(right) – query(left-1)."
```

PHASE 4 — CLEAN CODE

```
class _307_NumArray:
    def __init__(self, nums: List[int]):
        self.nums = nums[:]
        n = len(nums)
        self.tree = [0] * (n + 1)
        for i, val in enumerate(nums):
            self._add(i, val)
    def _add(self, i: int, delta: int) -> None:
        i += 1
        while i <= len(self.nums):
            self.tree[i] += delta
            i += i & (-i)
    def _prefix(self, i: int) -> int:
        i += 1
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & (-i)
        return total
    def update(self, index: int, val: int) -> None:
```

```

        self._add(index, val - self.nums[index])
        self.nums[index] = val
    def sumRange(self, left: int, right: int) -> int:
        return self._prefix(right) - (self._prefix(left - 1) if left > 0 else 0)

```

PHASE 5 — TEST & DEBUG

```

# nums=[1,3,5]: tree built. _prefix(2)=1+3+5=9. sumRange(0,2)=9 ✓
# update(1,2): delta=2-3=-1. tree adjusted. nums=[1,2,5].
# sumRange(0,2): _prefix(2)=8 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "update: O(log n). sumRange: O(log n). init: O(n log n).
# Space: O(n) for the tree.
# The bit trick i & (-i) isolates the lowest set bit – this determines
# how many elements each node is responsible for. Worth explaining clearly.
# Alternative: segment tree – more flexible (supports range update) but more code.
# Given more time I'd ask: what if we need range updates (add v to nums[l..r])?
# BIT can handle that with a difference array extension – or use a lazy segment
tree."

```

◆ Quick Review

Key Insights

- To efficiently support both point updates and range sum queries, a specialized data structure like a Binary Indexed Tree (Fenwick Tree) or a Segment Tree is ideal.
- Both data structures provide update and sum query operations in $O(\log n)$ time.
- The challenge is to maintain a mutable array while enabling fast sum computation over any specified range.

Space & Time

Time Complexity: $O(\log n)$ for both update and sumRange operations

Space Complexity: $O(n)$ to store the additional tree structure

Solution

We can use a Binary Indexed Tree (Fenwick Tree) to solve this problem. The Fenwick Tree allows for efficient prefix sum calculations and point updates, both in $O(\log n)$ time. The approach involves: Building the Fenwick Tree using the initial array. On an update, computing the difference between the new value and the current value, then propagating the change appropriately in the tree. For the sum range query, computing the prefix sum up to right and subtracting the prefix sum up to left-1.

Arrays & Hashing

□ #454 4Sum II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table
Lists: CodeSignal

Problem

Given four integer arrays `nums1`, `nums2`, `nums3`, and `nums4` all of length `n`, return the number of tuples (i, j, k, l) such that:

- $0 \leq i, j, k, l < n$
- $nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0$

Example 1:

```
Input: nums1 = [1,2], nums2 = [-2,-1], nums3 = [-1,2], nums4 = [0,2]
Output: 2
Explanation:
The two tuples are:
1. (0, 0, 0, 1) -> nums1[0] + nums2[0] + nums3[0] + nums4[1] = 1 + (-2) + (-1) + 2 = 0
2. (1, 1, 0, 0) -> nums1[1] + nums2[1] + nums3[0] + nums4[0] = 2 + (-1) + (-1) + 0 = 0
```

Example 2:

```
Input: nums1 = [0], nums2 = [0], nums3 = [0], nums4 = [0]
Output: 1
```

Constraints:

- `n == nums1.length`
- `n == nums2.length`
- `n == nums3.length`
- `n == nums4.length`
- `1 <= n <= 200`
- `-228 <= nums1[i], nums2[i], nums3[i], nums4[i] <= 228`

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def fourSumCount(self, nums1: List[int], nums2: List[int], nums3: List[int],
                    nums4: List[int]) -> int:
        hmap = Counter(a+b for a in nums1 for b in nums2)
        return sum(hmap[-(c+d)] for c in nums3 for d in nums4)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count tuples (i,j,k,l) where nums1[i]+nums2[j]+nums3[k]+nums4[l]==0.
# All four arrays have the same length n. Return the count – not the tuples. Right?
# We're not asked to deduplicate – every index combo counts separately."
#
# "nums1=[1,2],nums2=[-2,-1],nums3=[-1,2],nums4=[0,2]:
# (0,0,0,1): 1+(-2)+(-1)+2=0 ✓. (1,1,0,0): 2+(-1)+(-1)+0=0 ✓. → 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Four nested loops – for every combination (i,j,k,l), check if
# nums1[i]+nums2[j]+nums3[k]+nums4[l] == 0 and count the matches.
# Time: O(n^4). For n=200 that's 1.6 billion iterations – way too slow.
# Space: O(1). Should I go ahead and optimize?"
```

```
class _454_FourSumII_Brute:
    def fourSumCount(self, nums1, nums2, nums3, nums4):
        count = 0
        for a in nums1:
```

```

        for b in nums2:
            for c in nums3:
                for d in nums4:
                    if a + b + c + d == 0:
                        count += 1

    return count

```

PHASE 3 — OPTIMIZE

"Meet in the middle: split into two pairs.

Build a Counter of all sums a+b for a in nums1, b in nums2 – $O(n^2)$ sums.

Then for each c+d from nums3×nums4, look up $-(c+d)$ in the counter – $O(n^2)$ lookups.

Total: $O(n^2)$ time and $O(n^2)$ space."

PHASE 4 — CLEAN CODE

```

class _454_FourSumII:
    def fourSumCount(self, nums1: List[int], nums2: List[int], nums3: List[int],
nums4: List[int]) -> int:
        pair_sums = Counter(a + b for a in nums1 for b in nums2)
        return sum(pair_sums[-(c + d)] for c in nums3 for d in nums4)

```

PHASE 5 — TEST & DEBUG

nums1=[1,2], nums2=[-2,-1]:

pair_sums={1+(-2)=-1:1, 1+(-1)=0:1, 2+(-2)=0:+1-0:2, 2+(-1)=1:1}

= {-1:1, 0:2, 1:1}

nums3=[-1,2], nums4=[0,2]:

(c=-1,d=0): -(-1+0)=1. pair_sums[1]=1. count=1.

(c=-1,d=2): -(-1+2)=-1. pair_sums[-1]=1. count=2.

(c=2,d=0): -(2+0)=-2. pair_sums[-2]=0. count=2.

(c=2,d=2): -(2+2)=-4. pair_sums[-4]=0. count=2. → 2 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(n^2)$.

Meet-in-the-middle is the key: instead of 4 nested loops, do 2+2.

The Counter default of 0 for missing keys means misses contribute 0 – clean.

Given more time I'd ask: what if the arrays are sorted and we want unique quadruplets?

That's 4Sum (#18) – different problem requiring deduplication with two pointers."

◆ Quick Review

Key Insights

- Use a hash table to store the frequency of all possible sums from nums1 and nums2.

- For each pair from `nums3` and `nums4`, compute the complement (negative sum) and look it up in the hash table.
- This reduces the time complexity from $O(n^4)$ to $O(n^2)$.

Space & Time

Time Complexity: $O(n^2)$ where n is the length of each array.

Space Complexity: $O(n^2)$ due to the storage of pair sums in the hash table.

Solution

The solution employs a hash table to precompute and store the frequency of sums of all pairs from the first two arrays. Then, by iterating over all pairs from the third and fourth arrays, we calculate the required complement that would result in a total sum of zero. This complement is checked in the hash table, and the frequency (number of valid pairs) is added to the result. This method efficiently finds the number of valid quadruplets without resorting to a brute-force $O(n^4)$ approach.

Arrays & Hashing

□ #525 Contiguous Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Prefix Sum
Lists: Grind 169

Problem

Given a binary array `nums`, return the maximum length of a contiguous subarray with an equal number of 0 and 1.

Example 1:

Input: `nums = [0,1]`
Output: 2
Explanation: `[0, 1]` is the longest contiguous subarray with an equal number of 0 and 1.

Example 2:

Input: `nums = [0,1,0]`
Output: 2
Explanation: `[0, 1]` (or `[1, 0]`) is a longest contiguous subarray with equal number of 0 and 1.

Example 3:

Input: `nums = [0,1,1,1,1,1,0,0,0]`
Output: 6
Explanation: `[1,1,1,0,0,0]` is the longest contiguous subarray with equal number of 0 and 1.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $\text{nums}[i]$ is either 0 or 1.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        hmap = Counter({0:-1})
        ans, s = 0, 0

        for i, n in enumerate(nums):
            s += (1 if n!=0 else -1)
            if s in hmap:
                ans = max(ans, i-hmap[s])

            else:
                hmap[s] = i

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Binary array. Find the longest contiguous subarray with equal numbers of 0s and 1s.

Return its length – not the subarray itself. Right?"

#

"[0,1]: equal 0s and 1s → length 2 ✓

[0,1,0]: longest is [0,1] or [1,0] → length 2 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j), count the zeros and ones in $\text{nums}[i..j]$.

If they're equal, it's a valid subarray – track the maximum length.

Time: $O(n^2)$ – we can precompute prefix counts to make each check $O(1)$.

Space: $O(1)$. Should I go ahead and optimize?"

```
class _525_ContiguousArray_Brute:
    def findMaxLength(self, nums):
        n = len(nums)
```

```

longest = 0
for i in range(n):
    zeros = ones = 0
    for j in range(i, n):
        if nums[j] == 0:
            zeros += 1
        else:
            ones += 1
        if zeros == ones:
            longest = max(longest, j - i + 1)
return longest

```

PHASE 3 — OPTIMIZE

"Treat 0 as -1 and 1 as +1. Keep a running balance (prefix sum).

Equal 0s and 1s means balance hasn't changed – we want the longest subarray

where `balance[j] == balance[i]` for some `i < j`.

Store the FIRST index each balance value was seen. When we see the same balance again,

the subarray `[first_seen+1 .. current]` has equal 0s and 1s.

Seed the map with `{0: -1}` to handle subarrays starting from index 0.

#

Time: $O(n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```

class _525_ContiguousArray:
    def findMaxLength(self, nums: List[int]) -> int:
        first_seen = {0: -1} # balance -> first index it appeared
        balance = 0
        longest = 0
        for i, num in enumerate(nums):
            balance += 1 if num == 1 else -1
            if balance in first_seen:
                longest = max(longest, i - first_seen[balance])
            else:
                first_seen[balance] = i
        return longest

```

PHASE 5 — TEST & DEBUG

`nums=[0,1,0]`:

`i=0,num=0`: `balance=-1`. not seen → `first_seen={0:-1,-1:0}`.

`i=1,num=1`: `balance=0`. seen at -1 → `longest=max(0,1-(-1))=2`.

`i=2,num=0`: `balance=-1`. seen at 0 → `longest=max(2,2-0)=2`.

return 2 ✓

`nums=[0,1,1,1,1,1,0,0,0]`:

Track balance... `longest=6` ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$ for the hash map.

The `0→-1` substitution is the core insight – equal 0s and 1s becomes

'prefix sum unchanged', which becomes 'same balance seen before'.

The `{0:-1}` seed is critical – without it, subarrays starting at index 0 are missed.

Given more time I'd ask: what if it's not binary – find the longest subarray with equal counts of two specific values? Same trick – map `target→+1`, `other→-1`."

◆ Quick Review

Key Insights

- Convert 0s to `-1` so that a balanced subarray (equal number of 0s and 1s) sums to zero.
- Use a prefix sum to track the cumulative sum as you iterate through the array.
- Use a hash table (or dictionary) to store the first occurrence of each prefix sum.
- When the same prefix sum is encountered again, it indicates that the subarray between these indices is balanced.

Space & Time

Time Complexity: $O(n)$ – We iterate through the array once.

Space Complexity: $O(n)$ – In the worst case, we store an entry for each prefix sum.

Solution

We use a technique where we convert each 0 in the array to `-1`. Then, we compute a running prefix sum as we iterate through the array. A

prefix sum of 0 at any point indicates that the subarray from the beginning to the current index is balanced. For each prefix sum, if we have seen it before, the subarray between the previous index (where the prefix sum was first seen) and the current index sums to 0. We update the maximum length accordingly. We use a hash table to store the first index where each prefix sum occurs.

Arrays & Hashing

□ #560 Subarray Sum Equals K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Prefix Sum
Lists: Grind 169

Problem

Given an array of integers `nums` and an integer `k`, return the total number of subarrays whose sum equals to `k`.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,1,1]`, `k = 2`
Output: 2

Example 2:

Input: `nums = [1,2,3]`, `k = 3`
Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$
- $-10^7 \leq k \leq 10^7$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        hmap = Counter({0:1})
        ans, s = 0,0

        for i, n in enumerate(nums):
            s += n
            want = s-k
            if want in hmap:
                ans += hmap[want]

        hmap[s] += 1
        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count subarrays (contiguous, non-empty) whose elements sum to k.

Can contain negatives – so sliding window won't work. Return the count. Right?"

#

"[1,1,1], k=2: [1,1] at (0,1) and [1,1] at (1,2) → 2 ✓

[1,2,3], k=3: [3] at index 2 and [1,2] → 2 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j), compute the sum of nums[i..j] and check if it equals k.

I can maintain a running sum to avoid recomputing from scratch each time, giving $O(n^2)$.

Time: $O(n^2)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _560_SubarraySum_Brute:
    def subarraySum(self, nums, k):
        count = 0
        for i in range(len(nums)):
            total = 0
            for j in range(i, len(nums)):
                total += nums[j]
                if total == k:
                    count += 1
        return count
```

PHASE 3 — OPTIMIZE

```
# "Prefix sums: sum of nums[i..j] = prefix[j+1] - prefix[i].
# We want prefix[j+1] - prefix[i] == k → prefix[i] == prefix[j+1] - k.
# Walk forward tracking the running sum. For each position, count how many
# earlier prefix sums equal (current_sum - k). Use a Counter seeded with {0:1}
# to handle subarrays starting at index 0.
#
# Time: O(n). Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _560_SubarraySum:
    def subarraySum(self, nums: List[int], k: int) -> int:
        prefix_count = Counter({0: 1}) # prefix_sum → how many times seen
        running_sum = 0
        count = 0
        for num in nums:
            running_sum += num
            count += prefix_count[running_sum - k]
            prefix_count[running_sum] += 1
        return count
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,1,1], k=2:
# num=1: sum=1. want=1-2=-1. prefix_count[-1]=0. prefix_count={0:1,1:1}.
# num=1: sum=2. want=2-2=0. prefix_count[0]=1 → count=1.
prefix_count={0:1,1:1,2:1}.
# num=1: sum=3. want=3-2=1. prefix_count[1]=1 → count=2. → 2 ✓
# nums=[1,2,3], k=3:
# sum=1,want=-2→0. sum=3,want=0→count=1. sum=6,want=3→prefix_count[3]=1→count=2. →
2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(n).
# This is structurally identical to #525 Contiguous Array – both use
# prefix sum + hash map, both seed with {0:1} (or {0:-1}).
# Worth naming the connection explicitly: 525 finds longest, 560 counts all.
# Sliding window doesn't work here because negatives mean the sum can decrease
# even as we expand right – sliding window requires monotone sums.
# Given more time I'd ask: what if we want subarrays with sum divisible by k?
# Use prefix_sum % k – same pattern, but map remainders."
```

◆ Quick Review

Key Insights

- Utilize the prefix sum technique to simplify sum calculation for subarrays.
- Use a hash table (or dictionary/map) to store the frequency of prefix sums encountered.
- For each element, check if (current prefix sum – k) exists in the hash table; if it does, it indicates a valid subarray.
- Initialize the hash table with {0: 1} to manage cases where a subarray's sum is exactly k.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We solve the problem by maintaining a running sum (prefix sum) and using a hash table to record the number of times each sum has occurred. As we iterate through the array, we update the running sum. For each new sum, we check if (current sum – k) exists in our hash table. If it does, the value associated with (current sum – k) gives the number of subarrays ending at the current index that sum to k. This method efficiently computes the result in a single pass through the array.

Arrays & Hashing

□ #1010 Pairs of Songs With Total Durations Divisible by 60

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Counting
Lists: CodeSignal

Problem

You are given a list of songs where the i th song has a duration of $\text{time}[i]$ seconds.

Return the number of pairs of songs for which their total duration in seconds is divisible by 60. Formally, we want the number of indices i, j such that $i < j$ with $(\text{time}[i] + \text{time}[j]) \% 60 == 0$.

Example 1:

```
Input: time = [30,20,150,100,40]
Output: 3
Explanation: Three pairs have a total duration divisible by 60:
(time[0] = 30, time[2] = 150): total duration 180
(time[1] = 20, time[3] = 100): total duration 120
(time[1] = 20, time[4] = 40): total duration 60
```

Example 2:

```
Input: time = [60,60,60]
Output: 3
Explanation: All three pairs have a total duration of 120, which is divisible by 60.
```

Constraints:

- $1 \leq \text{time.length} \leq 6 * 10^4$
- $1 \leq \text{time}[i] \leq 500$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        hmap = Counter()
        ans = 0

        for t in time:
            t = t%60
            ans += hmap[(60-t)%60]
            hmap[t] +=1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count pairs (i,j) where i<j and (time[i]+time[j]) % 60 == 0.
# Songs have durations in seconds. Return the count. Right?
# Edge cases: duration exactly 60 (60%60=0), duration exactly 30 pairs with another 30."
#
# "[30,20,150,100,40]: (30,150)=180, (20,100)=120, (20,40)=60 → 3 ✓
# [60,60,60]: each pair sums to 120, divisible by 60. C(3,2)=3 → 3 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) where j > i, check if (time[i] + time[j]) % 60 == 0.
# Count all such pairs. For n=6*10^4 that's 3.6 billion comparisons – too slow.
# Time: O(n^2). Space: O(1). Should I go ahead and optimize?"
```

```
class _1010_PairsSongs_Brute:
    def numPairsDivisibleBy60(self, time):
        count = 0
        for i in range(len(time)):
            for j in range(i + 1, len(time)):
                if (time[i] + time[j]) % 60 == 0:
```

```
        count += 1
    return count
```

PHASE 3 — OPTIMIZE

```
# "Reduce each duration to t % 60. We want pairs where t1 + t2 ≡ 0 (mod 60).
# For each t, its complement is (60 - t) % 60. The %60 handles t=0 (complement=0,
not 60).
# Walk forward: count += how many times we've already seen the complement.
# Then add t to the seen counter.
# This is Two Sum with modular arithmetic – O(n) time, O(60)=O(1) space."
```

PHASE 4 — CLEAN CODE

```
class _1010_PairsSongs:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        seen = Counter()
        count = 0
        for t in time:
            remainder = t % 60
            complement = (60 - remainder) % 60
            count += seen[complement]
            seen[remainder] += 1
        return count
```

PHASE 5 — TEST & DEBUG

```
# time=[30,20,150,100,40]:
# t=30: rem=30, comp=30. seen[30]=0 → count=0. seen={30:1}.
# t=20: rem=20, comp=40. seen[40]=0 → count=0. seen={30:1,20:1}.
# t=150: rem=30, comp=30. seen[30]=1 → count=1. seen={30:2,20:1}.
# t=100: rem=40, comp=20. seen[20]=1 → count=2. seen={30:2,20:1,40:1}.
# t=40: rem=40, comp=20. seen[20]=1 → count=3. → 3 ✓
# time=[60,60,60]: each rem=0, comp=(60-0)%60=0.
# t=60: seen[0]=0 → 0. seen={0:1}.
# t=60: seen[0]=1 → count=1. seen={0:2}.
# t=60: seen[0]=2 → count=3. → 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1) – the Counter holds at most 60 keys.
# The (60-remainder)%60 handles the remainder=0 case: complement is 0, not 60.
# This is Two Sum modulo 60 – recognising the Two Sum pattern in disguise
# is the key interview insight.
# Given more time I'd ask: what if divisor were k instead of 60?
# Same algorithm – the Counter holds at most k entries, still O(1) space for fixed
k."
```

◆ Quick Review

Key Insights

- Each song duration's effect is determined by its remainder when divided by 60.
- For any given module value r (where $0 \leq r < 60$), the complementary module needed to complete 60 is $(60 - r) \% 60$.
- Special handling is needed when r is 0 (or 30, since $30 + 30 = 60$) to avoid double-counting.
- A frequency counter for remainders can be efficiently used to count valid pairs in one pass through the list.

Space & Time

Time Complexity: $O(n)$, where n is the number of songs.

Space Complexity: $O(60) \rightarrow O(1)$, since we only need to keep track of 60 possible remainders.

Solution

We iterate over the list of song durations and calculate the remainder of each duration modulo 60. A frequency array (or hash table) is used to record how many times each remainder has been seen so far. For each song, we look up the frequency of its complementary remainder (i.e., $(60 - \text{current_remainder}) \% 60$)

already encountered. This frequency indicates how many valid pairs can be formed with the current song. After counting pairs with the current song, we update the frequency of its remainder for future pairings.

Arrays & Hashing

□ ★ #1272 Remove Interval ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: ★ Premium | Premium 98

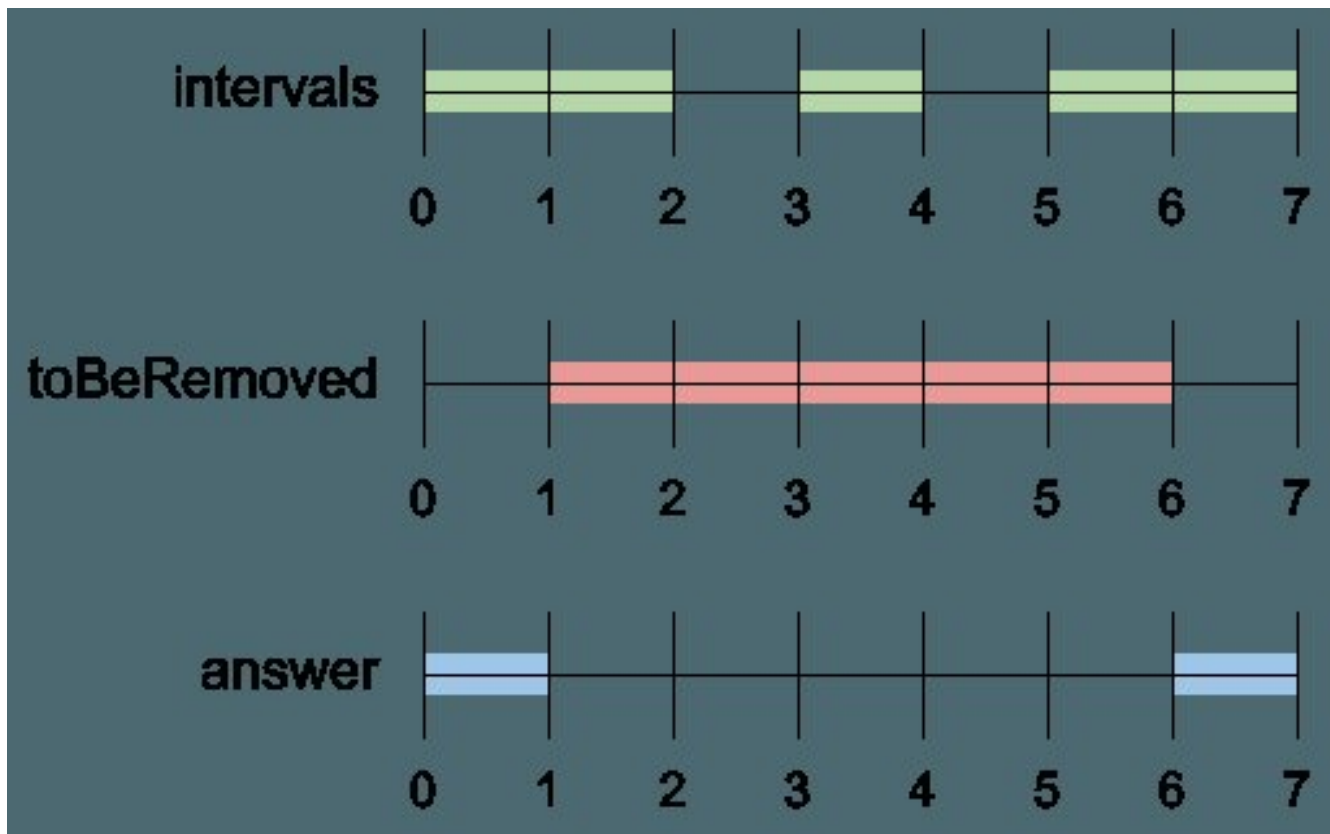
Problem

A set of real numbers can be represented as the union of several disjoint intervals, where each interval is in the form $[a, b)$. A real number x is in the set if one of its intervals $[a, b)$ contains x (i.e. $a \leq x < b$).

You are given a sorted list of disjoint intervals representing a set of real numbers as described above, where $\text{intervals}[i] = [a_i, b_i]$ represents the interval $[a_i, b_i)$. You are also given another interval `toBeRemoved`.

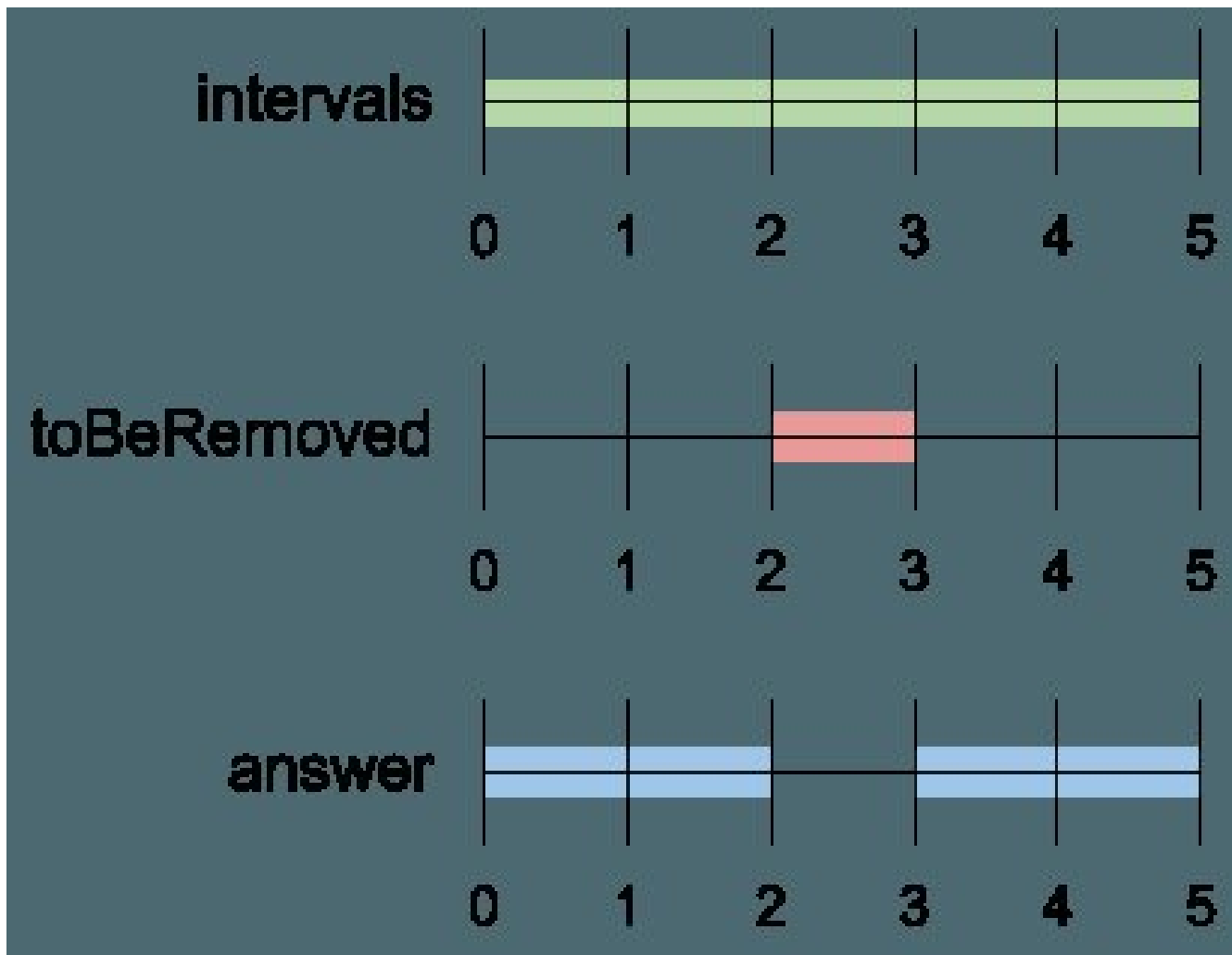
Return the set of real numbers with the interval `toBeRemoved` removed from intervals. In other words, return the set of real numbers such that every x in the set is in intervals but not in `toBeRemoved`. Your answer should be a sorted list of disjoint intervals as described above.

Example 1:



Input: intervals = [[0,2],[3,4],[5,7]], toBeRemoved = [1,6]
Output: [[0,1],[6,7]]

Example 2:



Input: intervals = [[0,5]], toBeRemoved = [2,3]
 Output: [[0,2],[3,5]]

Example 3:

Input: intervals = [[-5,-4],[-3,-2],[1,2],[3,5],[8,9]], toBeRemoved = [-1,4]
 Output: [[-5,-4],[-3,-2],[4,5],[8,9]]

Constraints:

- $1 \leq \text{intervals.length} \leq 104$
- $-109 \leq a_i < b_i \leq 109$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def removeInterval(self, intervals: List[List[int]], toBeRemoved: List[int]) -> List[List[int]]:
        a,b = toBeRemoved
        ans = []

        for start, end in intervals:
            if b <= start or a >= end:
                ans.append([start, end])

            else:

                if start < a:
                    ans.append([start,a])

                if b < end:
                    ans.append([b, end])

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Intervals use half-open notation [a, b). We remove everything in toBeRemoved=[a,b) from the existing set and return what remains. Right?
 # Intervals are pre-sorted and non-overlapping – no need to re-sort output."
 #
 # "[[0,2],[3,4],[5,7]], remove=[1,6]:
 # [0,2] partially overlaps → keep [0,1). [3,4] fully inside → gone. [5,7] partially → keep [6,7). → [[0,1],[6,7]] ✓
 # [[0,5]], remove=[2,3]: split into [0,2) and [3,5). → [[0,2],[3,5]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
 # Walk each interval and handle three cases: if it doesn't touch toBeRemoved at all, keep it whole.
 # If it partially overlaps on the left, keep the left portion. If it partially overlaps on the right,
 # keep the right portion. If it's fully covered, drop it entirely.
 # Time: O(n). Space: O(n). Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

```

# "Already O(n) – just apply the three cases cleanly in one pass.
# No overlap if: interval ends before remove starts (end <= a) OR starts after
remove ends (start >= b).
# Otherwise it overlaps. Trim left portion if start < a → keep [start, a).
# Trim right portion if end > b → keep [b, end)."

# PHASE 4 — CLEAN CODE
class _1272_RemoveInterval:
    def removeInterval(self, intervals: List[List[int]], toBeRemoved: List[int]) ->
List[List[int]]:
        a, b = toBeRemoved
        result = []
        for start, end in intervals:
            if end <= a or start >= b: # no overlap – keep whole
                result.append([start, end])
            else:
                if start < a: # left portion survives
                    result.append([start, a])
                if end > b: # right portion survives
                    result.append([b, end])
        return result

# PHASE 5 — TEST & DEBUG
# [[0,2],[3,4],[5,7]], remove=[1,6]:
# [0,2]: overlap. start(0)<a(1)→[0,1]. end(2)>b(6)? No. result=[[0,1]].
# [3,4]: overlap. start(3)<1? No. end(4)>6? No. result=[[0,1]].
# [5,7]: overlap. start(5)<1? No. end(7)>6 → [6,7]. result=[[0,1],[6,7]] ✓
# [[0,5]], remove=[2,3]:
# overlap. [0,2] and [3,5]. → [[0,2],[3,5]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n) for output.
# The two trim conditions are independent – both can fire on the same interval
# (when toBeRemoved is strictly inside the interval), producing two output
intervals.
# Given more time I'd ask: what if toBeRemoved can be a list of intervals to remove?
# Process them in sorted order, sweeping through once – still O(n + m) time."

```

◆ Quick Review

Key Insights

- Iterate over each interval and check if it overlaps with the toBeRemoved interval.

- If the current interval is completely outside of `toBeRemoved`, add it to the result as is.
- For overlapping intervals, compute the non-overlapping portions:
 - If the left side of the interval lies before `toBeRemoved`, include `[start, min(end, toBeRemoved.start)]`.
 - If the right side of the interval lies after `toBeRemoved`, include `[max(start, toBeRemoved.end), end]`.

Space & Time

Time Complexity: $O(n)$, where n is the number of intervals.

Space Complexity: $O(n)$, for storing the resulting intervals.

Solution

The approach involves iterating through the list of sorted disjoint intervals and checking for overlap with the `toBeRemoved` interval. For each interval: If it is completely before or after the removal interval, add it directly to the result. If it overlaps with `toBeRemoved`, determine the remaining portion(s): The left side of the interval that does not intersect (if

any). The right side of the interval that does not intersect (if any). Use simple conditional checks and list manipulation to form the final set of intervals.

Arrays & Hashing

□ ★ #1429 First Unique Number ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Design · Queue · Data
Stream

Lists: ★ Premium | Premium 98

Problem

You have a queue of integers, you need to retrieve the first unique integer in the queue.

Implement the FirstUnique class:

- `FirstUnique(int[] nums)` Initializes the object with the numbers in the queue.
- `int showFirstUnique()` returns the value of the first unique integer of the queue, and returns `-1` if there is no such integer.
- `void add(int value)` insert value to the queue.

Example 1:

Input:

```
["FirstUnique", "showFirstUnique", "add", "showFirstUnique", "add", "showFirstUnique", "add", "showFirstUnique"]
[[[2,3,5]], [], [5], [], [2], [], [3], []]
```

Output:

```
[null, 2, null, 2, null, 3, null, -1]
```

Explanation:

```
FirstUnique firstUnique = new FirstUnique([2,3,5]);
firstUnique.showFirstUnique(); // return 2
```

Example 2:

Input:

```
["FirstUnique", "showFirstUnique", "add", "add", "add", "add", "add", "showFirstUnique"]
[[[7,7,7,7,7,7]], [], [7], [3], [3], [7], [17], []]
```

Output:

```
[null, -1, null, null, null, null, null, 17]
```

Explanation:

```
FirstUnique firstUnique = new FirstUnique([7,7,7,7,7,7]);
firstUnique.showFirstUnique(); // return -1
```

Example 3:

Input:

```
["FirstUnique", "showFirstUnique", "add", "showFirstUnique"]
[[[809]], [], [809], []]
```

Output:

```
[null, 809, null, -1]
```

Explanation:

```
FirstUnique firstUnique = new FirstUnique([809]);
firstUnique.showFirstUnique(); // return 809
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^8$
- $1 \leq \text{value} \leq 10^8$
- At most 50000 calls will be made to showFirstUnique and add.

My LeetCode Solution

• My LeetCode Solution

```
# First Unique Number
class FirstUnique:

    def __init__(self, nums: List[int]):
        self.hset = set()
        self.hmap = {}
        for n in nums:
            self.add(n)

    def showFirstUnique(self) -> int:
        return next(iter(self.hmap), -1)

    def add(self, value: int) -> None:
        if value not in self.hset:
            self.hset.add(value)
            self.hmap[value] = 1

        elif value in self.hmap:

            self.hmap.pop(value)

# Your FirstUnique object will be instantiated and called as such:
# obj = FirstUnique(nums)
# param_1 = obj.showFirstUnique()
# obj.add(value)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Maintain a queue. showFirstUnique() returns the first element that appears
# exactly once. add() appends to the queue. Return -1 if no unique exists. Right?
# Order matters – we want the first unique by insertion order."
#
# "FirstUnique([2,3,5]): first unique = 2.
# add(5) → 5 now has count 2, still first unique = 2.
```

```
# add(2) → 2 duplicated, first unique = 3.
# add(3) → 3 duplicated, no unique → -1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# I'll store all added elements in a list. showFirstUnique() scans the list in
insertion order,
# counting occurrences of each element, and returns the first one that appears
exactly once.
# add() is O(1). showFirstUnique() is O(n) per call – with many calls this gets
slow.
# Time: add O(1), showFirstUnique O(n). Space: O(n). Should I go ahead and
optimize?"
```

```
class _1429_FirstUnique_Brute:
    def __init__(self, nums):
        self.data = list(nums)
    def showFirstUnique(self):
        from collections import Counter
        freq = Counter(self.data)
        for x in self.data:
            if freq[x] == 1:
                return x
        return -1
    def add(self, value):
        self.data.append(value)
```

PHASE 3 — OPTIMIZE

```
# "Two data structures:
# A 'seen' set – tracks all values ever added (including duplicates).
# An ordered dict (or insertion-order dict) – maps value → True, only for unique
values.
# On add: if not in seen → it's new, add to both seen and dict.
# if in seen and in dict → it became a duplicate, remove from dict.
# if in seen and not in dict → already duplicate, ignore.
# showFirstUnique(): peek at first key in dict – O(1) in Python 3.7+
(insertion-ordered).
#
# Time: O(1) per add, O(1) per showFirstUnique. Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _1429_FirstUnique:
    def __init__(self, nums: List[int]):
        self.seen : set = set()
        self.uniques: dict = {}
        for num in nums:
            self.add(num)
    def showFirstUnique(self) -> int:
        return next(iter(self.uniques), -1)
    def add(self, value: int) -> None:
```

```

if value not in self.seen:
    self.seen.add(value)
    self.uniques[value] = True
elif value in self.uniques:
    del self.uniques[value]

```

PHASE 5 — TEST & DEBUG

```

# init([2,3,5]): seen={2,3,5}, uniques={2,3,5}.
# showFirstUnique() → 2 ✓
# add(5): 5 in seen and in uniques → del 5. uniques={2,3}.
# showFirstUnique() → 2 ✓
# add(2): 2 in seen and in uniques → del 2. uniques={3}.
# showFirstUnique() → 3 ✓
# add(3): del 3. uniques={}.
# showFirstUnique() → -1 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "O(1) per operation. Space O(n)."
# Python dicts maintain insertion order since 3.7, so next(iter(dict)) returns
# the oldest key still present – exactly what we need.
# If order weren't guaranteed, we'd use collections.OrderedDict explicitly.
# Given more time I'd ask: what if we need to support remove() as well?
# That's a linked hash map (doubly linked list + hash map) – O(1) all operations."

```

◆ Quick Review

Key Insights

- Use a queue to maintain the order of potential unique numbers.
- Use a hash table (or map) to count the occurrences of each number.
- When showing the first unique number, remove numbers from the front of the queue if their frequency is more than one.
- All operations (add and showFirstUnique) can be done in $O(1)$ amortized time.

Space & Time

Time Complexity: $O(1)$ amortized for each add and showFirstUnique call.

Space Complexity: $O(n)$, where n is the number of distinct integers encountered.

Solution

We design the FirstUnique class using two main data structures: a queue and a hash map (or dictionary). The queue preserves the order of insertion for unique candidates, while the hash map holds the frequency count for each integer. When a new number is added: Increment its count in the hash map. If the count is one, append it to the queue. To retrieve the first unique number: While the queue is not empty and the count for the element at the front is greater than one, remove it from the queue. Return the front element of the queue if it exists, otherwise return -1 . This approach ensures that we are always up-to-date with the current first unique number.

Arrays & Hashing

□ #3159 Find Occurrences of an Element in an Array **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table
Lists: CodeSignal

Problem

You are given an integer array `nums`, an integer array `queries`, and an integer `x`.

For each `queries[i]`, you need to find the index of the `queries[i]`th occurrence of `x` in the `nums` array. If there are fewer than `queries[i]` occurrences of `x`, the answer should be `-1` for that query.

Return an integer array `answer` containing the answers to all queries.

Example 1:

Input: `nums = [1,3,1,7]`, `queries = [1,3,2,4]`, `x = 1`

Output: `[0,-1,2,-1]`

Explanation:

- For the 1st query, the first occurrence of 1 is at index 0.

- For the 2nd query, there are only two occurrences of 1 in nums, so the answer is -1.
- For the 3rd query, the second occurrence of 1 is at index 2.
- For the 4th query, there are only two occurrences of 1 in nums, so the answer is -1.

Example 2:

Input: nums = [1,2,3], queries = [10], x = 5

Output: [-1]

Explanation:

- For the 1st query, 5 doesn't exist in nums, so the answer is -1.

Constraints:

- $1 \leq \text{nums.length}, \text{queries.length} \leq 105$
- $1 \leq \text{queries}[i] \leq 105$
- $1 \leq \text{nums}[i], x \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def occurrencesOfElement(self, nums: List[int], queries: List[int], x: int) -> List[int]:
        hmap = [i for i, n in enumerate(nums) if n == x]
        return [hmap[q-1] if q - 1 < len(hmap) else -1 for q in queries]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Preprocess nums to find all indices where x appears.

For each query q, return the index of the q-th occurrence of x (1-indexed).

If fewer than q occurrences exist, return -1. Right?"

#

"nums=[1,3,1,7], queries=[1,3,2,4], x=1:

x appears at indices [0,2]. q=1→0, q=3→-1, q=2→2, q=4→-1. → [0,-1,2,-1] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For each query q, scan nums from the beginning, count occurrences of x as we go,

and return the index where we hit the q-th occurrence – or -1 if we run out.

Time: $O(n * |queries|)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _3159_FindOccurrencesBrute:
```

```
    def occurrencesOfElement(self, nums, queries, x):
```

```
        result = []
```

```
        for q in queries:
```

```
            count = 0
```

```
            found = -1
```

```
            for i, n in enumerate(nums):
```

```
                if n == x:
```

```
                    count += 1
```

```
                    if count == q:
```

```
                        found = i
```

```
                        break
```

```
            result.append(found)
```

```
        return result
```

PHASE 3 — OPTIMIZE

"Precompute a list of all indices where x appears – $O(n)$.

Each query is then an $O(1)$ index lookup. Total: $O(n + |queries|)$."

PHASE 4 — CLEAN CODE

```
class _3159_FindOccurrences:
```

```
    def occurrencesOfElement(self, nums: List[int], queries: List[int], x: int) ->
    List[int]:
```

```
        positions = [i for i, n in enumerate(nums) if n == x]
```

```
        return [positions[q - 1] if q - 1 < len(positions) else -1 for q in queries]
```

PHASE 5 — TEST & DEBUG

nums=[1,3,1,7], x=1: positions=[0,2].

q=1: positions[0]=0 ✓. q=3: 3-1=2 >= len(positions)=2 → -1 ✓.

q=2: positions[1]=2 ✓. q=4: -1 ✓.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n + q)$. Space $O(k)$ where k = occurrences of x .

If queries were sorted, binary search on positions would give $O((n+q) \log k)$

with minimal extra space – but for unsorted queries, the precompute list is cleaner.

Given more time I'd ask: what if x could change between queries?

Build a map from value → sorted list of indices, then binary search per query."

◆ Quick Review

Key Insights

- Pre-process the nums array to record the positions (indices) where the element x appears.
- For each query, check if the occurrence number requested is within the bounds of the precomputed list.
- Return the corresponding index if available; otherwise, return -1 .

Space & Time

Time Complexity: $O(n + q)$ where n is the length of nums (to precompute positions) and q is the length of queries (to answer each query in constant time).

Space Complexity: $O(n)$ in the worst case where all elements in nums equal x and are stored in the positions array.

Solution

The solution involves a two-step approach: Iterate through the nums array once to store the index of each occurrence of x in a list

called positions. For each query in queries, check if there is at least as many occurrences as the query specifies by verifying if queries[i] is less than or equal to the length of positions. If yes, return the (queries[i]-1)th element from positions (since it is zero-indexed). Otherwise, return -1. This approach efficiently handles multiple queries by utilizing precomputed data and simple indexing.

String

Round 2 · High · Medium · 5 questions

String

□ #8 String to Integer (atoi)

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String

Lists: Grind 169

Problem

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").
2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding:** If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then

round the integer to remain in the range. Specifically, integers less than -231 should be rounded to -231 , and integers greater than $231 - 1$ should be rounded to $231 - 1$.

Return the integer as the final result.

Example 1:

Input: $s = "42"$

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)

^

Step 2: "42" (no characters read because there is neither a '-' nor '+')

^

Step 3: "42" ("42" is read in)

^

Example 2:

Input: $s = "-042"$

Output: -42

Explanation:

Step 1: "-042" (leading whitespace is read and ignored)

^

Step 2: "-042" ('-' is read, so the result should be negative)

^

Step 3: "-042" ("042" is read in, leading zeros ignored in the result)

^

Example 3:

Input: $s = "1337c0d3"$

Output: 1337

Explanation:

```
Step 1: "1337c0d3" (no characters read because there is no leading whitespace)
^
Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')
^
Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character
is a non-digit)
^
```

Example 4:

Input: s = "0-1"

Output: 0

Explanation:

```
Step 1: "0-1" (no characters read because there is no leading whitespace)
^
Step 2: "0-1" (no characters read because there is neither a '-' nor '+')
^
Step 3: "0-1" ("0" is read in; reading stops because the next character is a
non-digit)
^
```

Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.

Constraints:

- $0 \leq s.length \leq 200$

- s consists of English letters (lower-case and upper-case), digits (0–9), ' ', '+', '-', and '.'.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def myAtoi(self, s: str) -> int:
        intmax, intmin = 2**31 - 1, -2**31
        i, n = 0, len(s)
        while i < n and s[i] == " ":
            i += 1

        sign = 1
        if i < n and (s[i] == "+" or s[i] == "-"):
            if s[i] == "-":
                sign = -1

            i += 1

        ans = 0
        while i < n and s[i].isdigit():
            d = int(s[i])
            if (ans > intmax // 10) or (ans == intmax // 10 and d > 7):
                return intmax if sign == 1 else intmin

            ans = ans * 10 + d
            i += 1

        return ans * sign
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

- # "Four steps in order: strip leading whitespace, read optional sign, read digits until non-digit or end, clamp to 32-bit signed range $[-2^{31}, 2^{31}-1]$. Right?
- # Stop at the FIRST non-digit after the optional sign – don't skip over non-digits."

```

#
# "'42'→42. ' -042'→-42 (strip, sign, parse 042). '1337c0d3'→1337 (stop at 'c').
# '0-1'→0 (first digit parsed, stop at '-'). 'words'→0 (first char non-digit, no
digits read)."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# The naive approach is to try Python's built-in int() after stripping whitespace —
# but that fails on partial strings like '42abc' and doesn't clamp to 32-bit range.
# So the 'brute force' here is really just the manual state machine, which I'll
implement.
# Time: O(n). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Manual state machine with a pointer:
# 1. Skip whitespace.
# 2. Read sign.
# 3. Read digits, building number, check overflow before each digit.
# Overflow check: ans > MAX//10 OR (ans == MAX//10 AND next digit > 7).
# Return sign * ans (clamped)."

# PHASE 4 — CLEAN CODE
class _8_Atoi:
    def myAtoi(self, s: str) -> int:
        INT_MAX, INT_MIN = 2**31 - 1, -(2**31)
        i, n = 0, len(s)
        while i < n and s[i] == ' ': # strip whitespace
            i += 1
        sign = 1
        if i < n and s[i] in '+-':
            sign = -1 if s[i] == '-' else 1
            i += 1
        result = 0
        while i < n and s[i].isdigit():
            d = int(s[i])
            if result > INT_MAX // 10 or (result == INT_MAX // 10 and d > 7):
                return INT_MAX if sign == 1 else INT_MIN
            result = result * 10 + d
            i += 1
        return sign * result

# PHASE 5 — TEST & DEBUG
# '42': no space, no sign, digits→42. return 42 ✓
# ' -042': skip space, sign=-1, digits 0,4,2→42. return -42 ✓
# '1337c0d3': digits 1,3,3,7→1337, stop at 'c'. return 1337 ✓
# '9999999999': overflow triggers → return INT_MAX ✓
# '': i=0, no sign, no digits → return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).

```

```
# The overflow check BEFORE adding each digit is critical – without it we'd overflow Python
# integers silently (Python handles big ints) but the logic breaks for fixed-width languages.
# The magic '7' in d > 7 comes from 2^31-1 = 2147483647 – last digit is 7.
# Given more time I'd ask: handle locale-specific decimal separators (commas)?
# Add a pre-processing step to strip/replace them before parsing."
```

◆ Quick Review

Key Insights

- Skip leading spaces in the string.
- Detect and handle the optional sign indicator ('+' or '-').
- Read and convert digit characters until a non-digit is encountered.
- Handle cases with no digits or leading zeros.
- Clamp the result to fit within the 32-bit signed integer range.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.

Space Complexity: $O(1)$ as only a constant amount of extra space is used.

Solution

We iterate through the string to first eliminate any leading whitespace. Next, we check for a

sign character to determine if the resulting integer should be negative. We then convert each consecutive digit into an integer, all the while updating the result and checking for potential overflow. If an overflow is detected, we immediately return the appropriate maximum or minimum 32-bit signed integer boundary value. This approach ensures that we strictly follow the conversion rules and efficiently handle edge cases.

String

□ ★ #249 Group Shifted Strings ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String
Lists: ★ Premium | Premium 98

Problem

Perform the following shift operations on a string:

- Right shift: Replace every letter with the successive letter of the English alphabet, where 'z' is replaced by 'a'. For example, "abc" can be right-shifted to "bcd" or "xyz" can be right-shifted to "yza".
- Left shift: Replace every letter with the preceding letter of the English alphabet, where 'a' is replaced by 'z'. For example, "bcd" can be left-shifted to "abc" or "yza" can be left-shifted to "xyz".

We can keep shifting the string in both directions to form an endless shifting sequence.

- For example, shift "abc" to form the sequence: ... $\leftrightarrow$ "abc" $\leftrightarrow$ "bcd" $\leftrightarrow$... $\leftrightarrow$ "xyz" $\leftrightarrow$ "yza" $\leftrightarrow$ $\leftrightarrow$ "zab" $\leftrightarrow$ "abc" $\leftrightarrow$...

You are given an array of strings `strings`, group together all `strings[i]` that belong to the same shifting sequence. You may return the answer in any order.

Example 1:

Input: `strings =`

`["abc","bcd","acef","xyz","az","ba","a","z"]`

Output:

`[["acef"],["a","z"],["abc","bcd","xyz"],["az","ba"]]`

Example 2:

Input: `strings = ["a"]`

Output: `[["a"]]`

Constraints:

- $1 \leq \text{strings.length} \leq 200$
- $1 \leq \text{strings}[i].\text{length} \leq 50$
- `strings[i]` consists of lowercase English letters.

My LeetCode Solution

- My LeetCode Solution

```

from typing import List
from collections import defaultdict

class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        groups: defaultdict = defaultdict(list)
        for s in strings:
            key = tuple((ord(s[i]) - ord(s[i - 1])) % 26 for i in range(1, len(s)))
            groups[key].append(s)
        return list(groups.values())

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Two strings are in the same shifting sequence if one can become the other by consistently shifting every character by the same amount (cyclically, a-z wraps).
 # Group them. Return order doesn't matter. Right?
 # 'abc' and 'xyz' are in the same group – same relative gaps between characters."
 #
 # ["'abc', 'bcd', 'xyz', 'az', 'ba', 'a', 'z']:
 # 'abc', 'bcd', 'xyz' → all have gaps [1,1] → same group.
 # 'az', 'ba' → gaps [25] and [25] (b-a=1... wait: 'az': z-a=25. 'ba': a-b=-1+26=25) → same group.
 # 'a', 'z' → single char, empty gap string → same group."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
 # For each string, generate all 26 rotations by shifting every character by 0..25.
 # If any rotation of string A equals string B, they belong in the same group.
 # This is $O(n^2 * 26 * L)$ – quadratic in the number of strings, far too slow.
 # Time: $O(n^2 * 26 * L)$. Space: $O(n)$. Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Compute a canonical key for each string: the tuple of consecutive char differences mod 26.
 # 'abc' → (1,1). 'xyz' → (1,1). 'az' → (25,). 'ba' → (25,).
 # All strings with the same key are in the same shifting group.
 # Group by key using a defaultdict. Time: $O(n * L)$. Space: $O(n * L)$."

PHASE 4 — CLEAN CODE

```

class _249_GroupShiftedStrings:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        groups: defaultdict = defaultdict(list)
        for s in strings:

```

```

        key = tuple((ord(s[i]) - ord(s[i - 1])) % 26 for i in range(1, len(s)))
        groups[key].append(s)
    return list(groups.values())

```

PHASE 5 — TEST & DEBUG

```

# 'abc': key=(1,1). 'bcd': (1,1). 'xyz': (1,1) → same group ✓
# 'az': key=((ord('z')-ord('a'))%26,)=(25,). 'ba': ((ord('a')-ord('b'))%26,)=(25,)
→ same ✓
# 'a','z': key=() (empty tuple) → same group ✓
# 'acef': key=(2,2,1) → its own group ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n \cdot L)$  where  $L$  = max string length. Space  $O(n \cdot L)$ .
# The %26 mod handles the wraparound: 'z'→'a' gives  $(\text{ord}('a') - \text{ord}('z')) \% 26 = (-25) \% 26 = 1$ .
# Without the mod, 'az' and 'ba' wouldn't match – the mod is the key insight.
# Given more time I'd ask: how do you efficiently check if two strings are in the same group
# at query time? Build a map string→canonical_key at init, then  $O(L)$  per query."

```

◆ Quick Review

Key Insights

- All strings in the same group have the same relative differences between adjacent characters.
- Normalize each string's pattern by converting it to a key representation based on the differences between consecutive letters.
- A single character string forms its own group as there are no differences to compute.

Space & Time

Time Complexity: $O(n * m)$ where n is the number of strings and m is the average length

of the strings (to compute each string's key).
Space Complexity: $O(n * m)$ to store the keys and grouping in the hash table.

Solution

The solution involves iterating over each string and generating a unique key that represents the relative differences between adjacent characters. This key is calculated by computing the difference (mod 26) between every consecutive pair in the string. Strings that share the same key can be grouped together since they can be shifted into one another. A hash table (or dictionary) is used to store and group the strings by their computed key. This approach handles edge cases like single-character strings separately.

String

★ #271 Encode and Decode Strings ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Design

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
string encode(vector<string> strs) {  
    // ... your code  
    return encoded_string;  
}
```

Machine 2 (receiver) has the function:

```
vector<string> decode(string s) {  
    //... your code  
    return strs;  
}
```

So Machine 1 does:

```
string encoded_string = encode(strs);
```

and Machine 2 does:

```
vector<string> strs2 = decode(encoded_string);
```

strs2 in Machine 2 should be the same as strs in Machine 1.

Implement the encode and decode methods.

You are not allowed to solve the problem using any serialize methods (such as eval).

Example 1:

```
Input: dummy_input = ["Hello","World"]
Output: ["Hello","World"]
Explanation:
Machine 1:
Codec encoder = new Codec();
String msg = encoder.encode(strs);
Machine 1 ----msg----> Machine 2
```

Example 2:

```
Input: dummy_input = [""]
Output: [""]
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- **strs[i]** contains any possible characters out of 256 valid ASCII characters.

Follow up: Could you write a generalized algorithm to work on any possible set of characters?

My LeetCode Solution

- **My LeetCode Solution**

```

class Codec:
    def encode(self, strs):
        encoded_string = ''
        for s in strs:
            encoded_string += s.replace('/', '//') + ':'
        return encoded_string

    def decode(self, s):
        decoded_strings = []
        current_string = ""

        i = 0
        while i < len(s):
            if s[i:i+2] == '/:':
                decoded_strings.append(current_string)
                current_string = ""
                i += 2
            elif s[i:i+2] == '//':
                current_string += '/'
                i += 2
            else:
                current_string += s[i]
                i += 1
        return decoded_strings

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Design `encode(strs)` and `decode(s)` that roundtrip correctly for ANY list of strings —

including strings with special characters, empty strings, or strings containing

whatever delimiter we might choose. Right?

We can't use `eval` or `pickle`."

#

`['Hello', 'World']` → `encode` → some string → `decode` → `['Hello', 'World']` ✓

`['']` → `['']` ✓ (empty string must survive the roundtrip)"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

A naive approach is to join strings with a rare delimiter like `'|'` — but it breaks if

strings themselves contain `'|'`. We'd need escaping: replace `'|'` with `'\|'` and `'\'` with `'\\'`.

```

# This works but decoding is error-prone and complex to get right.
# Time: O(total chars). Space: O(total chars). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (length-prefix encoding)
# "Prefix each string with its length and a fixed separator (e.g. '#').
# encode: '5#Hello4#World'. decode: read number up to '#', slice that many chars,
# repeat.
# No ambiguity – length tells us exactly where each string ends.
# Works for any content including '#' and empty strings."

# PHASE 4 — CLEAN CODE
class _271_Codec:
    def encode(self, strs: List[str]) -> str:
        return ''.join(f'{len(s)}#{s}' for s in strs)
    def decode(self, s: str) -> List[str]:
        result = []
        i = 0
        while i < len(s):
            j = s.index('#', i)
            length = int(s[i:j])
            result.append(s[j + 1: j + 1 + length])
            i = j + 1 + length
        return result

# PHASE 5 — TEST & DEBUG
# encode(['Hello','World']): '5#Hello5#World'.
# decode('5#Hello5#World'):
# i=0: j=1 (index of '#'). length=5. result=['Hello']. i=7.
# i=7: j=8. length=5. result=['Hello','World']. i=14. done ✓
# encode(['']): '0#'. decode('0#'): j=1, length=0, append ''. → [''] ✓
# encode(['a#b','c']): '3#a#b1#c'. decode: length=3→'a#b', length=1→'c' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "encode O(total chars). decode O(total chars). Space O(n).
# Length-prefix is the standard solution – no special characters, no escaping
# needed.
# The '#' separator is arbitrary – any fixed separator works since we always read
# the length first and know exactly how many chars to consume.
# Given more time I'd ask: what if strings can contain binary data (null bytes)?
# Use a 4-byte big-endian integer prefix instead of a text number – fully
# binary-safe."

```

◆ Quick Review

Key Insights

- Use length–prefix encoding for each string to avoid ambiguity.
- Append a delimiter (e.g., "#") after the length to separate it from the string content.
- This method ensures that even if the original strings contain special characters, including the delimiter, the decoding remains unambiguous.
- The approach handles edge cases, such as empty strings.

Space & Time

Time Complexity: $O(n)$, where n is the total number of characters across all strings.

Space Complexity: $O(n)$, required for the encoded string and the resulting decoded list.

Solution

We encode by iterating over each string and building a new string that contains the length of the string, a delimiter (e.g., "#"), and then the string itself. This way, during decoding, we can safely determine how many characters to extract by first reading until the delimiter to get the length, and then reading that number of characters. This length–prefix approach

avoids conflicts with potential characters in the strings.

String

□ #635 Design Log Storage System

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Ordered Set
Lists: Denny Zhang

Problem

You are given several logs, where each log contains a unique ID and timestamp. Timestamp is a string that has the following format: Year:Month:Day:Hour:Minute:Second, for example, 2017:01:01:23:59:59. All domains are zero-padded decimal numbers.

Implement the LogSystem class:

- **LogSystem()** Initializes the LogSystem object.
- **void put(int id, string timestamp)** Stores the given log (id, timestamp) in your storage system.
- **int[] retrieve(string start, string end, string granularity)** Returns the IDs of the logs whose timestamps are within the range from start to

end inclusive. start and end all have the same format as timestamp, and granularity means how precise the range should be (i.e. to the exact Day, Minute, etc.). For example, start = "2017:01:01:23:59:59", end = "2017:01:02:23:59:59", and granularity = "Day" means that we need to find the logs within the inclusive range from Jan. 1st 2017 to Jan. 2nd 2017, and the Hour, Minute, and Second for each log entry can be ignored.

Example 1:

Input

```
["LogSystem", "put", "put", "put", "retrieve", "retrieve"]  
[[], [1, "2017:01:01:23:59:59"], [2, "2017:01:01:22:59:59"], [3,  
"2016:01:01:00:00:00"], ["2016:01:01:01:01:01", "2017:01:01:23:00:00",  
"Year"], ["2016:01:01:01:01:01", "2017:01:01:23:00:00", "Hour"]]
```

Output

```
[null, null, null, null, [3, 2, 1], [2, 1]]
```

Explanation

```
LogSystem logSystem = new LogSystem();
```

Constraints:

- $1 \leq \text{id} \leq 500$
- $2000 \leq \text{Year} \leq 2017$
- $1 \leq \text{Month} \leq 12$
- $1 \leq \text{Day} \leq 31$
- $0 \leq \text{Hour} \leq 23$
- $0 \leq \text{Minute, Second} \leq 59$

- granularity is one of the values ["Year", "Month", "Day", "Hour", "Minute", "Second"].
- At most 500 calls will be made to put and retrieve.

My LeetCode Solution

• My LeetCode Solution

```
from typing import List

class LogSystem:
    def __init__(self):
        self.items = []
        self.map = {'Year':4, 'Month':7, 'Day':10,
                    'Hour':13, 'Minute':16, 'Second':19}

    def put(self, _id: int, timestamp: str) -> None:
        self.items.append((_id, timestamp))

    def retrieve(self, start: str, end: str, granularity: str) -> List[int]:
        idx = self.map[granularity]
        return [d for d, timestamp in self.items
                if start[:idx] <= timestamp[:idx] <= end[:idx]]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Store logs with (id, timestamp). retrieve(start, end, granularity) returns IDs
# of logs within the range, where comparison is only up to the granularity level.
# Timestamps format: 'YYYY:MM:DD:HH:mm:ss'. Truncate at the granularity character
# position. Right?"
#
# "retrieve with 'Year': compare only the first 4 chars. '2017:...' vs '2016:...'
# etc."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic."
```

```
# Store all logs in a flat list. On retrieve, iterate over every log, truncate its
timestamp
# to the granularity prefix length, and check if it falls within the truncated
start/end range.
# With at most 500 calls and a fixed 19-char timestamp format, O(n) per retrieve is
acceptable.
# Time: put O(1), retrieve O(n). Space: O(n). Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Map granularity name to the string prefix length that captures it.
# 'Year'→4 chars, 'Month'→7, 'Day'→10, 'Hour'→13, 'Minute'→16, 'Second'→19.
# Truncate all three timestamps to that length and do a simple string comparison.
# Lexicographic order of the timestamp format is correct for chronological order."
```

PHASE 4 — CLEAN CODE

```
class _635_LogSystem:
    GRAN = {'Year': 4, 'Month': 7, 'Day': 10, 'Hour': 13, 'Minute': 16, 'Second':
19}

    def __init__(self):
        self.logs: List[tuple] = []

    def put(self, log_id: int, timestamp: str) -> None:
        self.logs.append((log_id, timestamp))

    def retrieve(self, start: str, end: str, granularity: str) -> List[int]:
        cut = self.GRAN[granularity]
        return [lid for lid, ts in self.logs if start[:cut] <= ts[:cut] <=
end[:cut]]
```

PHASE 5 — TEST & DEBUG

```
# put(1, '2017:01:01:23:59:59'), put(2, '2017:01:01:22:59:59'),
put(3, '2016:01:01:00:00:00').
# retrieve(..., 'Year'): cut=4. compare '2016'<='year'<='2017'. All 3 match ->
[1,2,3] ✓
# retrieve(..., 'Hour'): cut=13. '2016:01:01:01'<='2016:01:01:00'? No→3 excluded.
# '2016:01:01:01'<='2017:01:01:22'? Yes→2. '2017:01:01:23'? Yes→1.
# -> [2,1] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "put O(1). retrieve O(n) — scans all logs.
# The key insight: timestamps in 'YYYY:MM:DD:HH:mm:ss' format sort
lexicographically
# correctly because they're zero-padded and delimiters are identical.
# For larger datasets, a sorted list + binary search would bring retrieve to O(log n
+ k).
# Given more time I'd ask: what if granularity could be arbitrary (e.g. 'Week')?
# We'd need a custom truncation function rather than fixed prefix lengths."
```

◆ Quick Review

Key Insights

- Store logs in a simple data structure such as a list or array.
- Convert the timestamps into strings; due to fixed-length and zero padding, lexicographical string comparisons are valid.
- Use a mapping to determine the slice length for each granularity, e.g. "Year" corresponds to the first 4 characters, "Month" corresponds to the first 7 characters, etc.
- During retrieval, compare the relevant substring of each stored timestamp against the start and end query timestamp slices.
- The inclusive nature of the query simplifies checking conditions.

Space & Time

Time Complexity: $O(N)$ per retrieval query, where N is the number of logs stored (since each log is checked). In the worst case, all log entries are examined.

Space Complexity: $O(N)$ where N is the number of logs stored.

Solution

We implement a `LogSystem` class that supports putting log entries and retrieving them using a specified granularity. **Data Structures:** Use a list/array to store log entries as tuples (or objects) containing the timestamp and log ID. A dictionary/map is used to store the mapping from granularity string to the substring index (e.g., "Year" => 4, "Month" => 7, etc.). For the `put()` method, simply append the new log entry into the list. For the `retrieve()` method, compute the substring length based on the provided granularity. Then iterate over all stored logs and compare the sliced timestamp against the sliced start and end query values. If it falls within the bounds, include the log ID in the result. Since the timestamps have fixed width with leading zeros, slicing and lexicographical comparisons are both valid and simple.

String

□ #1138 Alphabet Board Path

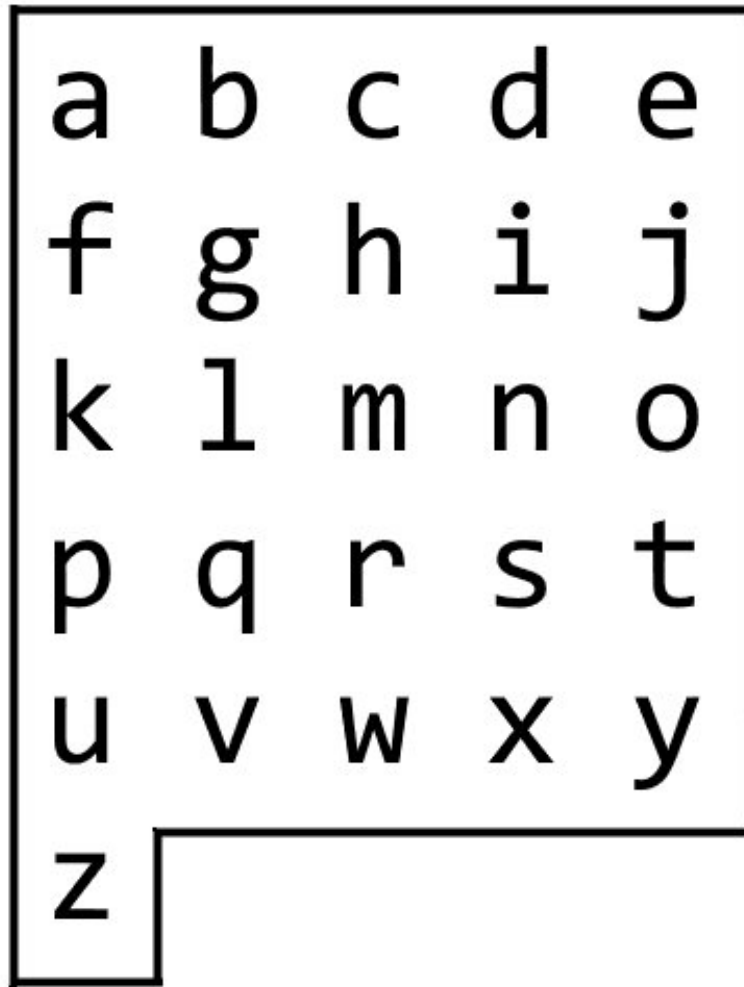
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String
Lists: Denny Zhang

Problem

On an alphabet board, we start at position (0, 0), corresponding to character board[0][0].

Here, board = ["abcde", "fghij", "klmno", "pqrst", "uvwxy", "z"], as shown in the diagram below.



We may make the following moves:

- 'U' moves our position up one row, if the position exists on the board;
- 'D' moves our position down one row, if the position exists on the board;
- 'L' moves our position left one column, if the position exists on the board;
- 'R' moves our position right one column, if the position exists on the board;

- '!' adds the character `board[r][c]` at our current position (r, c) to the answer.

(Here, the only positions that exist on the board are positions with letters on them.)

Return a sequence of moves that makes our answer equal to target in the minimum number of moves. You may return any path that does so.

Example 1:

Input: target = "leet"

Output: "DDR!UURRR!!DDD!"

Example 2:

Input: target = "code"

Output: "RR!DDRR!UUL!R!"

Constraints:

- $1 \leq \text{target.length} \leq 100$
- target consists only of English lowercase letters.

My LeetCode Solution

- My LeetCode Solution


```

class Solution:
    def alphabetBoardPath(self, target: str) -> str:
        row = col = 0
        moves = []
        for ch in target:
            i = (ord(ch) - ord('a')) // 5
            j = (ord(ch) - ord('a')) % 5
            if ch == 'z':
                while j < col:
                    col -= 1

                moves.append('L')
                while i > row:
                    row += 1
                moves.append('D')
            else:
                while i < row:
                    row -= 1
                moves.append('U')
                while i > row:
                    row += 1

                moves.append('D')
                while j < col:
                    col -= 1
                moves.append('L')
                while j > col:
                    col += 1
                moves.append('R')

            moves.append('!')
        return ''.join(moves)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Board is ['abcde', 'fghij', 'klmno', 'pqrst', 'uvwxy', 'z'] – 5 cols except row 5 which has only 'z'.

Start at (0,0). For each char in target, output U/D/L/R moves then '!'. Right?

The critical constraint: 'z' is at (5,0) – the only cell in its row.

Moving to 'z' must clear the column before going down. Moving from 'z' must go up before going right."

#

```
# "target='leet': l is at (2,1). Move D,D,R,! . e=(0,4) – go U,U,R,R,R,! . etc."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic."
```

```
# For each character, compute its board position as row = (ord(ch)-ord('a'))//5,
# col = (ord(ch)-ord('a'))%5. Then generate moves: go vertical first, then
horizontal.
```

```
# The naive approach fails for 'z' – moving down before clearing column 0 steps off
the board.
```

```
# Time: O(|target| * max_moves). Space: O(|target|). Should I go ahead and
optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Formula: row = (ord(ch)-ord('a'))//5, col = (ord(ch)-ord('a'))%5."
```

```
# For 'z' specifically: it's at (5,0), col=0. When moving TO 'z', move left first
# (to reach col 0) then move down (to reach row 5) – otherwise we'd step off the
board.
```

```
# When moving FROM 'z', move up first then move right – same reason.
```

```
# General case: move vertically then horizontally (or horizontally then vertically).
```

```
# 'z' is the only exception – must move left before down."
```

PHASE 4 — CLEAN CODE

```
class _1138_AlphabetBoardPath:
    def alphabetBoardPath(self, target: str) -> str:
        row = col = 0
        moves = []
        for ch in target:
            dest_row = (ord(ch) - ord('a')) // 5
            dest_col = (ord(ch) - ord('a')) % 5
            if ch == 'z': # must clear column before going down
                moves.extend('L' * (col - dest_col))
                moves.extend('D' * (dest_row - row))
            else:
                moves.extend('U' * (row - dest_row))
                moves.extend('D' * (dest_row - row))
                moves.extend('L' * (col - dest_col))
                moves.extend('R' * (dest_col - col))
            moves.append('!')
            row, col = dest_row, dest_col
        return ''.join(moves)
```

PHASE 5 — TEST & DEBUG

```
# target='leet': l=(2,1). from (0,0): D,D,R,! → "DDR!". row=2,col=1.
```

```
# e=(0,4): U,U,R,R,R,! → "UURRR!". row=0,col=4.
```

```
# e=(0,4): no move,! → "!". row=0,col=4.
```

```
# t=(3,4): D,D,D,! → "DDD!". → "DDR!UURRR!!DDD!" ✓
```

```
# target='z': from (0,0): ch='z'. L*(0)=none. D*5. → "DDDDD!" ✓
```

```
# If coming from (5,4) to 'z' (5,0): L,L,L,L,! – no D needed ✓.
```

```
# If going from 'z' to 'a': ch='a'=(0,0). Not 'z' → U*5,L*0,R*0 → "UUUUU!" ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(|target| * max_moves) = O(|target| * 10) = O(|target|). Space O(|target|).
# The 'z' case is the entire problem – everything else is standard grid navigation.
# Phrase it clearly: 'z' is at the leftmost position of its row, so we must reach
col 0
# before stepping down into row 5, otherwise intermediate positions don't exist on
the board.
# Given more time I'd ask: what if the board were different (e.g. a phone numpad)?
# Same approach – just recompute (row, col) from the new layout."
```

◆ Quick Review

Key Insights

- Determine the row and column for each letter (using integer division and modulus).
- Because "z" is in a special row that only contains one column, the move order is critical to avoid invalid positions.
- For target letter "z", perform horizontal moves before vertical moves; for all other letters, do vertical moves before horizontal moves.
- Build the answer step-by-step by moving from your current position to each target letter.

Space & Time

Time Complexity: $O(N)$ where N is the length of target (each move is processed in constant time)

Space Complexity: $O(N)$ for the resulting string of moves

Solution

The approach calculates the destination coordinates for each character. For each letter in the target: Convert the letter to its board coordinates (row and col). For example, for letter c, $\text{row} = (\text{ord}(\text{c}) - \text{ord}(\text{'a'})) // 5$ and $\text{col} = (\text{ord}(\text{c}) - \text{ord}(\text{'a'})) \% 5$. Note that "z" (the 26th letter) always maps to (5, 0). To move from the current position (curRow, curCol) to the target's (targetRow, targetCol), decide on the order of moves: If the target letter is "z", first move horizontally (left or right) then vertically (up or down) to avoid stepping into invalid regions from the extra row. For any other letter, move vertically (up or down) first, then horizontally (left or right). Append "!" to the moves after reaching each letter. Update the current position to the letter's coordinates. This method ensures you only traverse valid cells on the board and minimizes moves by taking the optimal coordinate difference each step.

Two Pointers

Round 2 · High · Medium · 14 questions

Two Pointers

□ #11 Container With Most Water

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Greedy
Lists: Grind 169

Problem

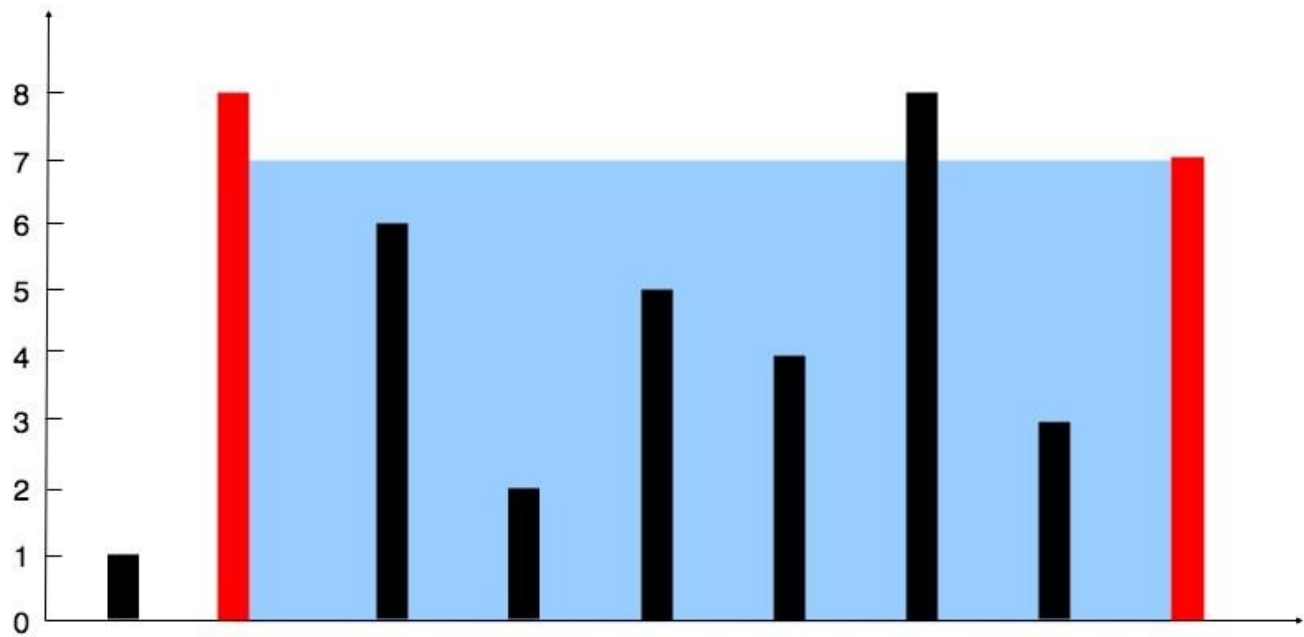
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the `x`-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: height = [1,1]

Output: 1

Constraints:

- $n == \text{height.length}$
- $2 \leq n \leq 105$
- $0 \leq \text{height}[i] \leq 104$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def maxArea(self, height: List[int]) -> int:
        n = len(height)
        l = 0
        r = n-1
        ans = 0

        while l<r:
            minheight = min(height[l], height[r])
            ans = max(ans, (r-l) * minheight)

            if height[l] < height[r]:
                l+=1
            else:
                r-=1

        return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given heights of vertical lines, find two lines that form the largest container.
 # Area = min(height[left], height[right]) * (right - left). Return max area. Right?
 # We can't slant – so water level is always min of the two chosen heights."
 #
 # "[1,8,6,2,5,4,8,3,7]: left=1(h=8), right=8(h=7). Area=min(8,7)*(8-1)=49 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
 # For every pair (i, j) where j > i, compute area = min(height[i], height[j]) * (j - i)
 # and track the maximum. That's C(n,2) pairs to check.
 # Time: O(n²). Space: O(1). Should I go ahead and optimize?"

```

class _11_ContainerMostWater_Brute:
    def maxArea(self, height):
        best = 0
        for i in range(len(height)):
            for j in range(i + 1, len(height)):
                best = max(best, min(height[i], height[j]) * (j - i))
        return best

```

PHASE 3 — OPTIMIZE

"Two pointers: left=0, right=n-1. At each step compute area.
 # Move the pointer with the SHORTER height inward – keeping the taller height
 # maximises any future container. Moving the taller pointer can only shrink


```
# both width and potentially height, so it can't improve the result.
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _11_ContainerMostWater:
    def maxArea(self, height: List[int]) -> int:
        left, right = 0, len(height) - 1
        best = 0
        while left < right:
            area = min(height[left], height[right]) * (right - left)
            best = max(best, area)
            if height[left] < height[right]:
                left += 1
            else:
                right -= 1
        return best
```

PHASE 5 — TEST & DEBUG

```
# height=[1,8,6,2,5,4,8,3,7]:
# l=0(h=1),r=8(h=7): area=1*8=8. h[l]<h[r]→l=1.
# l=1(h=8),r=8(h=7): area=7*7=49. h[l]>h[r]→r=7.
# l=1(h=8),r=7(h=3): area=3*6=18. h[l]>h[r]→r=6.
# ... best stays 49. → 49 ✓
# height=[1,1]: area=1*1=1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1).
# The greedy argument: if we move the taller pointer, we keep a shorter constraint
# (limited by the short side) AND reduce width – area can only shrink or stay.
# So we always move the shorter side – we can't do better by moving the taller one.
# This is the key insight to state explicitly in the interview.
# Given more time I'd ask: what if we need to find the pair of indices (not just the
area)?
# Track left_best and right_best alongside best – same code, just record the
positions."
```

◆ Quick Review

Key Insights

- The container's area is determined by the distance between two lines multiplied by the shorter line's height.

- A brute-force approach checking all possible pairs would result in $O(n^2)$ time complexity, which is inefficient for large inputs.
- A two-pointer approach allows us to solve the problem in $O(n)$ time by initializing pointers at both ends and gradually moving them inward.
- By always moving the pointer corresponding to the shorter line, we are more likely to find a taller line that could potentially form a container with a larger capacity.

Space & Time

Time Complexity: $O(n)$ – The algorithm makes a single pass through the array with two pointers.

Space Complexity: $O(1)$ – Only a few extra variables are used, regardless of the input size.

Solution

The optimal approach uses a two-pointer strategy: Start with two pointers at each end of the array. Calculate the area formed between the two lines at these pointers. Update the maximum area if the calculated value is larger.

Move the pointer pointing to the shorter line inward, as this is the only possible way to potentially increase the height of the shorter line and thus the area. Continue until the two pointers meet. This method effectively reduces the number of comparisons and ensures that every possible pair that can yield a larger area is considered.

Two Pointers

□ #15 3Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Explanation:

`nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.`

`nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.`

`nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.`

The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.

Notice that the order of the output and the order of the triplets does not matter.

Example 2:

Input: `nums = [0,1,1]`

Output: `[]`

Explanation: The only possible triplet does not sum up to 0.

Example 3:

Input: nums = [0,0,0]

Output: [[0,0,0]]

Explanation: The only possible triplet sums up to 0.

Constraints:

- $3 \leq \text{nums.length} \leq 3000$
- $-105 \leq \text{nums}[i] \leq 105$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        nums.sort()
        n = len(nums)
        ans = []

        for i in range(n-2):
            if i>0 and nums[i] == nums[i-1]:
                continue

            l = i+1
            r = n-1
            while l < r:
                s = nums[i]+nums[l]+nums[r]
                if s > 0:
                    r-=1

                elif s<0:
                    l+=1
                else:
```

```

        ans.append([nums[i], nums[l], nums[r]])
        l, r = l+1, r-1
        while l<r and nums[l] == nums[l-1]:
            l+=1

        while l<r and nums[r] == nums[r+1]:
            r-=1

    return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Find all unique triplets summing to 0. Return the triplets, not indices.
# No duplicate triplets in the output. Right?
# Order within a triplet doesn't matter - [-1,0,1] and [0,-1,1] are the same."
#
# "[-1,0,1,2,-1,-4]: sort → [-4,-1,-1,0,1,2]. → [[-1,-1,2],[-1,0,1]] ✓
# [0,0,0]: → [[0,0,0]] ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Three nested loops over i < j < k; if nums[i]+nums[j]+nums[k]==0, add triplet.
# Store triplets in a set keyed by sorted tuple to deduplicate.
# Correct, but O(n^3) with hash-set overhead on large n.
# Time: O(n^3). Space: O(n) for dedup set.
# Should I go ahead and optimize?"

```

PHASE 3 — OPTIMIZE

```

# "Sort the array. Fix one element nums[i], use two pointers on the rest.
# Sorting enables: skip duplicate values of nums[i] to avoid duplicate triplets,
# and move pointers efficiently based on the sum.
# After finding a valid triplet, skip duplicate nums[left] and nums[right].
# Time: O(n^2). Space: O(1) extra."

```

PHASE 4 — CLEAN CODE

```

class _15_ThreeSum:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        result = []
        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]: # skip duplicate fixed element
                continue
            left, right = i + 1, len(nums) - 1
            while left < right:

```

```

        total = nums[i] + nums[left] + nums[right]
        if total < 0:
            left += 1
        elif total > 0:
            right -= 1
        else:
            result.append([nums[i], nums[left], nums[right]])
            left += 1
            right -= 1
            while left < right and nums[left] == nums[left - 1]: left += 1
            while left < right and nums[right] == nums[right + 1]: right -= 1
    1
    return result

```

PHASE 5 — TEST & DEBUG

```

# nums=[-1,0,1,2,-1,-4] → sorted: [-4,-1,-1,0,1,2].
# i=0 (-4): l=1,r=5. sums=-4+(-1)+2=-3<0→l++. -4+(-1)+2=-3<0→l++. -4+0+2=-2<0→l++.
# -4+1+2=-1<0→l++. l=r → stop.
# i=1 (-1): l=2,r=5. -1+(-1)+2=0→append[-1,-1,2]. l=3,r=4. skip dups: l=3 ok, r=4
ok.
# -1+0+1=0→append[-1,0,1]. l=4,r=3→stop.
# i=2 (-1): i>0 and nums[2]==nums[1]==-1 → skip.
# i=3 (0): l=4,r=5. 0+1+2=3>0→r--. l≥r→stop.
# result=[[-1,-1,2],[-1,0,1]] ✓
# nums=[0,0,0]: i=0,l=1,r=2. 0+0+0=0→append. l=2,r=1→stop. → [[0,0,0]] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n^2)$  – outer loop  $O(n)$ , inner two-pointer  $O(n)$  each. Sorting is  $O(n \log n)$ 
dominated.
# Space  $O(1)$  extra (ignoring output).
# Two deduplication steps:
# 1. Skip  $\text{nums}[i] == \text{nums}[i-1]$  to avoid re-fixing the same outer value.
# 2. After appending, skip  $\text{nums}[\text{left}] == \text{nums}[\text{left}-1]$  and  $\text{nums}[\text{right}] == \text{nums}[\text{right}+1]$ 
# to avoid re-collecting the same triplet.
# Both checks are essential – missing either produces duplicate output.
# Given more time I'd ask: 4Sum (#18)? Add one more outer loop –  $O(n^3)$  – same
structure."

```

◆ Quick Review

Key Insights

- Sort the array to efficiently skip duplicates and facilitate the two pointers approach.

- Use a fixed pointer for the first element, and then use two additional pointers (left and right) to find pairs that sum to the negation of the fixed element.
- Skip duplicate elements to ensure unique triplets.

Space & Time

Time Complexity: $O(n^2)$ where n is the number of elements in the array.

Space Complexity: $O(1)$ extra space (ignoring the space required for the output).

Solution

We first sort the array to bring duplicates together and allow the two-pointer technique to work efficiently. Then, iterate over the array and for each element, use two pointers (one at the beginning right after the current element and one at the end of the array) to find a pair that sums with the current element to zero. Skip duplicates for the current element and within the inner loop to avoid duplicate triplets. This ensures that each valid triplet is unique and the overall runtime remains efficient.

Two Pointers

□ #16 3Sum Closest

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an integer array `nums` of length `n` and an integer `target`, find three integers at distinct indices in `nums` such that the sum is closest to `target`.

Return the sum of the three integers.

You may assume that each input would have exactly one solution.

Example 1:

Input: `nums = [-1,2,1,-4]`, `target = 1`
Output: `2`
Explanation: The sum that is closest to the target is 2. ($-1 + 2 + 1 = 2$).

Example 2:

Input: `nums = [0,0,0]`, `target = 1`
Output: `0`
Explanation: The sum that is closest to the target is 0. ($0 + 0 + 0 = 0$).

Constraints:

- $3 \leq \text{nums.length} \leq 500$

- $-1000 \leq \text{nums}[i] \leq 1000$
- $-104 \leq \text{target} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        nums.sort()
        ans = nums[0] + nums[1] + nums[2]
        n = len(nums)

        for i in range(n-2):
            if i and nums[i] == nums[i-1]:
                continue

            l = i+1
            r = n-1

            while l < r:
                s = nums[i]+nums[l]+nums[r]
                if s == target:
                    return target

                if abs(target-s) < abs(target-ans):
                    ans = s

                if s > target:
                    r -=1

                else:
                    l +=1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find three distinct-index integers whose sum is closest to target. Return the
sum.
# Exactly one solution guaranteed – no tie-breaking needed. Right?"
#
# "[-1,2,1,-4], target=1: possible sums: -1+2+1=2, -1+2-4=-3, -1+1-4=-4, 2+1-4=-1.
# Closest to 1 is 2. → 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Three nested loops – for every triple (i, j, k), compute the sum and track
# whichever sum has the smallest absolute difference from target.
# Time:  $O(n^3)$ . Space:  $O(1)$ . Should I go ahead and optimize?"
```

```
class _16_ThreeSumClosest_Brute:
    def threeSumClosest(self, nums, target):
        best = nums[0] + nums[1] + nums[2]
        n = len(nums)
        for i in range(n - 2):
            for j in range(i + 1, n - 1):
                for k in range(j + 1, n):
                    s = nums[i] + nums[j] + nums[k]
                    if abs(target - s) < abs(target - best):
                        best = s
        return best
```

PHASE 3 — OPTIMIZE

```
# "Exact same structure as 3Sum: sort + fix one element + two pointers.
# Instead of looking for sum==0, track the closest sum seen.
# If sum > target → move right pointer left (decrease sum).
# If sum < target → move left pointer right (increase sum).
# If sum == target → return immediately (can't get closer).
# Skip duplicate outer values to avoid redundant work – not required for correctness
# here (no dedup of output), but speeds up the average case."
```

PHASE 4 — CLEAN CODE

```
class _16_ThreeSumClosest:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        nums.sort()
        n = len(nums)
        best = nums[0] + nums[1] + nums[2]
        for i in range(n - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            left, right = i + 1, n - 1
            while left < right:
                total = nums[i] + nums[left] + nums[right]
                if total == target:
                    return target
                if abs(target - total) < abs(target - best):
                    best = total
                if total > target:
```

```

        right -= 1
    else:
        left += 1
    return best

```

PHASE 5 — TEST & DEBUG

```

# nums=[-1,2,1,-4], target=1. sorted: [-4,-1,1,2].
# i=0(-4): l=1,r=3. -4-1+2=-3. |1-(-3)|=4 < |1-(-4)|=5 → best=-3. sum<1→l++.
# l=2,r=3. -4+1+2=-1. |1-(-1)|=2 < 4 → best=-1. sum<1→l++. l=r→stop.
# i=1(-1): l=2,r=3. -1+1+2=2. |1-2|=1 < |1-(-1)|=2 → best=2. sum>1→r--. l=r→stop.
# → 2 ✓
# nums=[0,0,0], target=1. best=0. No closer sum possible → 0 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n^2)$ . Space  $O(1)$  extra ( $O(\log n)$  for sort).
# This is 3Sum (#15) with one change: track closest instead of exact zero.
# The early return on exact match is a nice optimisation worth mentioning.
# Given more time I'd ask: what if we want the k closest sums?
# Collect all triple sums, sort by |target - sum|, return first k -  $O(n^2 \log n)$ ."

```

◆ Quick Review

Key Insights

- Sort the array to make it easier to use the two pointers technique.
- Fix one element and then use two pointers (left and right) to find the best pair that, along with the fixed element, produces a sum closest to the target.
- Update the best sum whenever a closer sum is found.
- The sorted order helps eliminate unnecessary iterations and allows efficient movement of pointers.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(\log n)$ to $O(n)$ depending on the sorting algorithm used (typically in-place sort, so extra space might be minimal)

Solution

The solution starts by sorting the array. For each index in the array (considered as the first element of the triple), we use two pointers: one starting just after the fixed element and the other at the end of the array. We calculate the sum of the fixed element and the two pointers. If this sum is closer to the target than the current best, we update our best sum.

Depending on whether the current sum is less than or greater than the target, we adjust the left or right pointer to try and get closer to the target. This two-pointer method leverages the sorted order of the array for efficient summing.

Two Pointers

□ #31 Next Permutation

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers
Lists: Grind 169

Problem

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

- For example, for `arr = [1,2,3]`, the following are all the permutations of `arr`: `[1,2,3]`, `[1,3,2]`, `[2, 1, 3]`, `[2, 3, 1]`, `[3,1,2]`, `[3,2,1]`.

The next permutation of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

- For example, the next permutation of `arr = [1,2,3]` is `[1,3,2]`.
- Similarly, the next permutation of `arr = [2,3,1]` is `[3,1,2]`.
- While the next permutation of `arr = [3,2,1]` is `[1,2,3]` because `[3,2,1]` does not have a lexicographical larger rearrangement.

Given an array of integers `nums`, find the next permutation of `nums`.

The replacement must be in place and use only constant extra memory.

Example 1:

Input: `nums = [1,2,3]`
Output: `[1,3,2]`

Example 2:

Input: `nums = [3,2,1]`
Output: `[1,2,3]`

Example 3:

Input: `nums = [1,1,5]`
Output: `[1,5,1]`

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """

        def rev(l,r):
            while l<r:
                nums[l], nums[r] = nums[r], nums[l]
                l+=1
                r-=1

        n = len(nums)
        i = next((i for i in range(n-2, -1, -1) if nums[i] < nums[i+1]), -1)
        if i>= 0:
            j = next((j for j in range(n-1, -1, -1) if nums[i] < nums[j]))
            nums[i], nums[j] = nums[j], nums[i]

        rev(i+1, n-1)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rearrange nums in-place to the next lexicographically greater permutation.
If no greater permutation exists (sorted descending), wrap to the smallest (ascending). Right?"

"[1,2,3]→[1,3,2]. [3,2,1]→[1,2,3]. [1,1,5]→[1,5,1]. [1,3,2]→[2,1,3]. ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
Generate all permutations of the array in sorted order, find the one that matches the current arrangement, then return the next one in the sorted list.
If the current is the last, wrap to the first (smallest) permutation.
Time: $O(n! * n)$ to generate all permutations – completely infeasible for $n > 10$.
Space: $O(n!)$. Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Three steps – all $O(n)$, $O(1)$ space:
1. Find the rightmost 'dip': scan right to left for the largest i where $nums[i] < nums[i+1]$.
If no dip exists, entire array is descending → just reverse it (wrap-around).
2. Find the smallest element to the right of i that is still larger than $nums[i]$.


```
# (The suffix is descending after step 1, so scan from the right.)
# 3. Swap nums[i] with that element, then reverse the suffix after i.
# Reversing the descending suffix makes it the smallest possible arrangement."
```

PHASE 4 — CLEAN CODE

```
class _31_NextPermutation:
    def nextPermutation(self, nums: List[int]) -> None:
        def reverse(left, right):
            while left < right:
                nums[left], nums[right] = nums[right], nums[left]
                left += 1; right -= 1

        n = len(nums)
        # Step 1: find rightmost dip
        dip = next((i for i in range(n - 2, -1, -1) if nums[i] < nums[i + 1]), -1)
        if dip >= 0:
            # Step 2: find smallest element to right of dip that is > nums[dip]
            swap = next(j for j in range(n - 1, -1, -1) if nums[j] > nums[dip])
            nums[dip], nums[swap] = nums[swap], nums[dip]
        # Step 3: reverse suffix after dip
        reverse(dip + 1, n - 1)
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3]: dip=1 (nums[1]=2<nums[2]=3). swap=2 (nums[2]=3>nums[1]=2).
# swap: [1,3,2]. reverse(2,2)→no change. → [1,3,2] ✓
# [3,2,1]: dip=-1. reverse(0,2): [1,2,3] ✓
# [1,1,5]: dip=1 (nums[1]=1<nums[2]=5). swap=2 (5>1). swap→[1,5,1].
reverse(2,2)→[1,5,1] ✓
# [1,3,2]: dip=0 (nums[0]=1<nums[1]=3). swap=1 (nums[1]=3>nums[0]=1; nums[2]=2
also>1 but
# scan right→left so j=1 first: nums[1]=3>1? yes → swap=1? No wait j starts from
n-1=2:
# nums[2]=2>nums[0]=1 → swap=2. swap→[2,3,1]. reverse(1,2): [2,1,3] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1).
# The suffix is always descending after the dip – that's why scanning right-to-left
# finds the smallest swap target efficiently.
# The reverse of the suffix turns the largest possible suffix into the smallest –
the key insight.
# Given more time I'd ask: what's the previous permutation?
# Same algorithm mirrored: find rightmost 'rise' (nums[i] > nums[i+1]), swap with
largest
# element to its right that is still smaller, reverse the suffix."
```

◆ Quick Review

Key Insights

- The problem is solved by identifying the first pair from the right where the left element is less than the right element.
- Once this pivot is found, find the rightmost element that is greater than this pivot.
- Swap these two elements.
- Reverse the subarray that comes after the pivot to ensure the next permutation is the next lexicographically larger sequence.
- If no such pivot exists, the array is in descending order and should be reversed to get the smallest permutation.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We start by scanning the array from the right to find the first index (i) such that $\text{nums}[i]$ is less than $\text{nums}[i+1]$. This index ' i ' represents the pivot point where the next permutation can be formed. If we find such an index, we then scan from the right again to locate the first number that is greater than $\text{nums}[i]$ (let this index be j), and swap these two numbers.

Finally, we reverse the subarray starting from index $i+1$ to the end of the array, which gives us the next permutation in lexicographical order. This approach leverages in-place modifications using basic operations like swapping and reversing, thus meeting the constant space requirement.

Two Pointers

□ #75 Sort Colors

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an array `nums` with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

```
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]
```

Example 2:

```
Input: nums = [2,0,1]
Output: [0,1,2]
```

Constraints:

- `n == nums.length`

- $1 \leq n \leq 300$
- `nums[i]` is either 0, 1, or 2.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        n = len(nums)
        l, w, r = 0, 0, n-1

        while w <= r:
            if nums[w] == 0:
                nums[l], nums[w] = nums[w], nums[l]
                l += 1
                w += 1

            elif nums[w] == 1:
                w += 1

            else:
                nums[w], nums[r] = nums[r], nums[w]
                r -= 1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sort an array containing only 0, 1, 2 in-place without using library sort.
# 0=red, 1=white, 2=blue. Output: all 0s, then 1s, then 2s. Right?
# Follow-up asks for a one-pass O(1) space solution."
#
# "[2,0,2,1,1,0] → [0,0,1,1,2,2] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# First pass: count the number of 0s, 1s, and 2s in the array.
# Second pass: overwrite the array – fill count[0] zeros, then count[1] ones, then
count[2] twos.
# This is two passes but still O(n) time and O(1) space.
# Time: O(n). Space: O(1). Should I go ahead and optimize?"
```

```
class _75_SortColors_Brute:
    def sortColors(self, nums):
        c0 = nums.count(0)
        c1 = nums.count(1)
        for i in range(len(nums)):
            if i < c0:
                nums[i] = 0
            elif i < c0 + c1:
                nums[i] = 1
            else:
                nums[i] = 2
```

PHASE 3 — OPTIMIZE

```
# "Dutch National Flag algorithm – one pass, three pointers: low, mid, high.
# Invariant: nums[0..low-1]=0, nums[low..mid-1]=1, nums[high+1..n-1]=2,
nums[mid..high]=unknown.
# At each step examine nums[mid]:
# 0 → swap(low,mid), low++, mid++ (0 placed, known-1 region grows both sides).
# 1 → mid++ (already correct).
# 2 → swap(mid,high), high-- (don't mid++ – swapped element is unknown)."
```

PHASE 4 — CLEAN CODE

```
class _75_SortColors:
    def sortColors(self, nums: List[int]) -> None:
        low = mid = 0
        high = len(nums) - 1
        while mid <= high:
            if nums[mid] == 0:
                nums[low], nums[mid] = nums[mid], nums[low]
                low += 1; mid += 1
            elif nums[mid] == 1:
                mid += 1
            else:
                nums[mid], nums[high] = nums[high], nums[mid]
                high -= 1 # don't mid++ – swapped value is unexamined
```

PHASE 5 — TEST & DEBUG

```
# [2,0,2,1,1,0]: low=0,mid=0,high=5.
# mid=0(2): swap(0,5)→[0,0,2,1,1,2]. high=4.
# mid=0(0): swap(0,0)→same. low=1,mid=1.
# mid=1(0): swap(1,1)→same. low=2,mid=2.
# mid=2(2): swap(2,4)→[0,0,1,1,2,2]. high=3.
# mid=2(1): mid=3.
# mid=3(1): mid=4. mid>high=3 → stop.
```

→ [0,0,1,1,2,2] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ one pass. Space $O(1)$.

The 'don't mid++' after swapping a 2 is the subtlety – the swapped element from high

hasn't been examined yet and could be 0, 1, or 2.

After swapping a 0, we DO mid++ because nums[low] before the swap was definitely 1

(it's in the known-1 region), so after swapping, mid lands on a known-1 element.

Given more time I'd ask: what if there were k colours?

k=2 is a standard partition; k=3 is Dutch Flag; k>3 needs multiple passes or a radix approach."

◆ Quick Review

Key Insights

- Use the Dutch National Flag algorithm to partition the array into three sections.
- Maintain three pointers: one for the next position for 0 (low), one for the next position for 2 (high), and one to traverse the array (mid).
- Swap elements appropriately to ensure that all 0's are at the beginning, all 2's are at the end, and 1's remain in the middle.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution uses a three–pointer approach (Dutch National Flag algorithm). Initialize three pointers: low, mid, and high. As you iterate through the array: If the element is 0, swap it with the element at the low pointer and increment both low and mid. If the element is 1, simply move mid forward. If the element is 2, swap it with the element at the high pointer and decrement high. This single pass method ensures that the array is sorted in–place with constant extra space. Key details include correctly handling swaps without losing any data and ensuring that the pointers are updated in the proper order.

Two Pointers

★ #161 One Edit Distance ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

Given two strings *s* and *t*, return true if they are both one edit distance apart, otherwise return false.

A string *s* is said to be one distance apart from a string *t* if you can:

- Insert exactly one character into *s* to get *t*.
- Delete exactly one character from *s* to get *t*.
- Replace exactly one character of *s* with a different character to get *t*.

Example 1:

Input: *s* = "ab", *t* = "acb"
Output: true
Explanation: We can insert 'c' into *s* to get *t*.

Example 2:

Input: *s* = "", *t* = ""
Output: false
Explanation: We cannot get *t* from *s* by only one step.

Constraints:

- $0 \leq s.length, t.length \leq 104$
- s and t consist of lowercase letters, uppercase letters, and digits.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        m, n = len(s), len(t)

        if m > n:
            return self.isOneEditDistance(t, s)

        if n - m > 1:
            return False

        for i in range(m):
            if s[i] != t[i]:
                if m == n:
                    rest_s = s[i+1:]
                    rest_t = t[i+1:]
                    return rest_s == rest_t
                else:
                    rest_s = s[i:]
                    rest_t = t[i+1:]
                    return rest_s == rest_t

        return n == m + 1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return true if s and t are exactly one edit apart – insert, delete, or replace.

Important: zero edits (equal strings) returns false. Right?

```
# s='ab', t='acb' → insert 'c' → true ✓. s='',t='' → false ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
```

```
# Run full Levenshtein DP between s and t; return True iff edit distance equals 1.
```

```
# Handles insert, delete, and replace in one table.
```

```
# Correct but  $O(m \times n)$  when we only need to detect a single edit.
```

```
# Time:  $O(m \times n)$ . Space:  $O(m \times n)$  DP table.
```

```
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Two observations that short-circuit work:
```

```
# If  $|\text{len}(s) - \text{len}(t)| > 1 \rightarrow$  impossible, return false.
```

```
# Ensure s is the shorter string (swap if needed) so we only handle two cases:
```

```
# same length → exactly one replacement, rest identical.
```

```
# s shorter by 1 → one deletion from t (or insertion into s), rest identical.
```

```
# Scan left to right. On first mismatch:
```

```
# same length → check  $s[i+1:] == t[i+1:]$ .
```

```
# diff length → check  $s[i:] == t[i+1:]$ .
```

```
# If no mismatch found → true only if t is exactly 1 longer."
```

PHASE 4 — CLEAN CODE

```
class _161_OneEditDistance:
```

```
    def isOneEditDistance(self, s: str, t: str) -> bool:
```

```
        def check(shorter, longer): #  $\text{len}(\text{longer}) == \text{len}(\text{shorter}) + 1$ 
```

```
            for i in range(len(shorter)):
```

```
                if shorter[i] != longer[i]:
```

```
                    return shorter[i:] == longer[i + 1:]
```

```
            return True # shorter is prefix of longer
```

```
        m, n = len(s), len(t)
```

```
        if abs(m - n) > 1:
```

```
            return False
```

```
        if m == n:
```

```
            diffs = sum(a != b for a, b in zip(s, t))
```

```
            return diffs == 1
```

```
        if m > n:
```

```
            s, t = t, s
```

```
            m, n = n, m
```

```
        return check(s, t)
```

PHASE 5 — TEST & DEBUG

```
# s='ab', t='acb': m=2,n=3. m<n → check('ab','acb').
```

```
# i=0: 'a'=='a'. i=1: 'b'!='c'. return 'b'=='b'? shorter[1:]='b', longer[2:]='b' → True ✓
```

```
# s='', t='': diffs=0 → return 0==1 → False ✓
```

```
# s='a', t='a': same length, diffs=0 → False ✓
```

```
# s='a', t='b': same length, diffs=1 → True ✓
```

```
# s='abc', t='abc': diffs=0 → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\min(m,n))$ . Space  $O(1)$  for the check (slice comparison is  $O(\min)$  under the hood).
# The key boundary: we return true when no mismatch found in the same-length case only if
#  $\text{diffs}==1$ ; in the diff-length case only if shorter is a prefix (true only if t is exactly
# one char longer, which we already guaranteed).
# Given more time I'd ask: what if we want EXACTLY k edits?
# Full edit distance DP –  $O(m*n)$  time,  $O(\min(m,n))$  space."
```

◆ Quick Review

Key Insights

- The difference in length between s and t must be at most 1.
- When lengths are equal, the only possibility is that exactly one character is different (replacement).
- When one string is longer by one character, the difference can be fixed by one insertion (or deletion in the other direction).
- Use two pointers to compare corresponding characters and carefully handle the first difference.

Space & Time

Time Complexity: $O(n)$, where n is the length of the shorter string, as we traverse both strings at most once.

Space Complexity: $O(1)$, using only constant extra space.

Solution

The solution uses a two–pointer approach. First, we check if the length difference is greater than 1, in which case they cannot be one edit apart. For the two primary cases: When strings are of equal length, iterate through both strings and count the number of differing characters. There should be exactly one difference. When lengths differ by one, use pointers for both strings. Upon encountering a difference, increment the pointer for the longer string only and ensure that no previous difference has been encountered. If a second discrepancy is found, return false. The key is to handle the edge case where the difference is at the end of the string.

Two Pointers

□ #167 Two Sum II – Input Array Is Sorted

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search
Lists: Denny Zhang

Problem

Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$.

Return the indices of the two numbers index1 and index2, each incremented by one, as an integer array [index1, index2] of length 2.

The tests are generated such that there is exactly one solution. You may not use the same element twice.

Your solution must use only constant extra space.

Example 1:

Input: numbers = [2,7,11,15], target = 9

Output: [1,2]

Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].

Example 2:

Input: numbers = [2,3,4], target = 6

Output: [1,3]

Explanation: The sum of 2 and 4 is 6. Therefore index1 = 1, index2 = 3. We return [1, 3].

Example 3:

Input: numbers = [-1,0], target = -1

Output: [1,2]

Explanation: The sum of -1 and 0 is -1. Therefore index1 = 1, index2 = 2. We return [1, 2].

Constraints:

- $2 \leq \text{numbers.length} \leq 3 * 10^4$
- $-1000 \leq \text{numbers}[i] \leq 1000$
- numbers is sorted in non-decreasing order.
- $-1000 \leq \text{target} \leq 1000$
- The tests are generated such that there is exactly one solution.

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def twoSum(self, numbers: list[int], target: int) -> list[int]:
        l = 0
        r = len(numbers) - 1

        while l < r:
            summ = numbers[l] + numbers[r]
            if summ == target:
                return [l + 1, r + 1]
            if summ < target:
                l += 1
            else:
                r -= 1

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Sorted array (1-indexed). Return [index1, index2] where numbers[index1]+numbers[index2]==target.
 # Exactly one solution. Must use O(1) extra space – so no hash map. Right?"
 #
 # "[2,7,11,15], target=9: 2+7=9 → [1,2] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
 # For every pair (i, j) where j > i, check if numbers[i] + numbers[j] == target.
 # Return [i+1, j+1] (1-indexed) when found. The problem guarantees exactly one solution.
 # Time: O(n²). Space: O(1). Should I go ahead and optimize?"

```

class _167_TwoSumII_Brute:
    def twoSum(self, numbers, target):
        for i in range(len(numbers)):
            for j in range(i + 1, len(numbers)):
                if numbers[i] + numbers[j] == target:
                    return [i + 1, j + 1]
        return []

```

PHASE 3 — OPTIMIZE

"Two pointers exploit sorted order: start left=0, right=n-1.
 # Sum too small → move left right (increase sum).
 # Sum too large → move right left (decrease sum).
 # Exactly one solution guaranteed → we will always find it.
 # Time O(n). Space O(1)."

PHASE 4 — CLEAN CODE

```
class _167_TwoSumII:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        left, right = 0, len(numbers) - 1
        while left < right:
            total = numbers[left] + numbers[right]
            if total == target:
                return [left + 1, right + 1] # 1-indexed
            elif total < target:
                left += 1
            else:
                right -= 1
        return []
```

PHASE 5 — TEST & DEBUG

```
# [2,7,11,15], target=9: l=0,r=3. 2+15=17>9→r=2. 2+11=13>9→r=1. 2+7=9→return [1,2] ✓
# [2,3,4], target=6: l=0,r=2. 2+4=6→return [1,3] ✓
# [-1,0], target=-1: -1+0=-1→return [1,2] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1).
# This is the canonical reason to sort before running two pointers on Two Sum.
# If the array weren't sorted, we'd need a hash map – O(n) space.
# Sorting adds O(n log n) but enables O(1) space – a genuine trade-off.
# Given more time I'd ask: what if there could be multiple answers?
# Continue after finding a match, skip duplicates – same as 3Sum's dedup logic."
```

◆ Quick Review

Key Insights

- The array is sorted in non-decreasing order, which makes the two pointers technique very effective.
- Using two pointers (one at the start and one at the end) allows checking pairs and adjusting the pointers based on the sum.
- Since the problem guarantees exactly one solution and requires constant space, using

two pointers satisfies both constraints.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in the array, because in the worst-case scenario each element is visited at most once.

Space Complexity: $O(1)$, as no additional data structure is used that scales with the input size.

Solution

We use the two pointers approach: Initialize two pointers, left at the beginning (index 0) and right at the end (last index) of the array. While left is less than right, compute the sum of the elements at the two pointers. If the sum equals the target, return the indices as $[left+1, right+1]$ (to adjust for the 1-indexed array). If the sum is less than the target, move the left pointer to the right to increase the sum. If the sum is greater than the target, move the right pointer to the left to decrease the sum. This approach leverages the sorted property and uses constant extra space.

Two Pointers

★ #186 Reverse Words in a String II ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

Given a character array *s*, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in *s* will be separated by a single space.

Your code must solve the problem in-place, i.e. without allocating extra space.

Example 1:

```
Input: s = ["t","h","e"," ","s","k","y"," ","i","s"," ","b","l","u","e"]  
Output: ["b","l","u","e"," ","i","s"," ","s","k","y"," ","t","h","e"]
```

Example 2:

```
Input: s = ["a"]  
Output: ["a"]
```

Constraints:

- $1 \leq s.length \leq 105$

- `s[i]` is an English letter (uppercase or lowercase), digit, or space ' '.
- There is at least one word in `s`.
- `s` does not contain leading or trailing spaces.
- All the words in `s` are guaranteed to be separated by a single space.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def reverseWords(self, s: List[str]) -> None:
        def reverse_range(left, right):
            while left < right:
                s[left], s[right] = s[right], s[left]
                left += 1
                right -= 1

        # Step 1: Reverse entire array
        reverse_range(0, len(s) - 1)

        # Step 2: Reverse each word
        start = 0
        for i in range(len(s) + 1):
            if i == len(s) or s[i] == ' ':
                reverse_range(start, i - 1)
                start = i + 1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a character ARRAY (not a string), reverse the order of words in-place.
# Words are separated by single spaces. No leading/trailing spaces. Right?
# ['t','h','e',' ','s','k','y'] → ['s','k','y',' ','t','h','e'] ✓"
```

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Convert the character array to a string, split by spaces to get words, reverse the word list,

rejoin with spaces, then write each character back into the original array.

Time: $O(n)$. Space: $O(n)$ – we create a full string copy. Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Two-step in-place reversal with $O(1)$ extra space:

Step 1: Reverse the entire array.

Step 2: Reverse each individual word back to its correct orientation.

Example: 'the sky is blue' → reverse all → 'eulb si yks eht' → reverse words → 'blue is sky the' ✓"

PHASE 4 — CLEAN CODE

```

class _186_ReverseWordsII:
    def reverseWords(self, s: List[str]) -> None:
        def reverse(left, right):
            while left < right:
                s[left], s[right] = s[right], s[left]
                left += 1; right -= 1
        reverse(0, len(s) - 1)
        start = 0
        for i in range(len(s) + 1):
            if i == len(s) or s[i] == ' ':
                reverse(start, i - 1)
                start = i + 1

```

PHASE 5 — TEST & DEBUG

```

# s=['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']:
# Step 1 reverse all: ['e','l','u','b',' ','s','i',' ','y','k','s',' ','e','h','t']
# Step 2 reverse words:
# i=0..3 'e','l','u','b' → reverse → 'b','l','u','e'
# i=5..6 's','i' → 'i','s'
# i=8..10 'y','k','s' → 's','k','y'
# i=12..14 'e','h','t' → 't','h','e'
# → ['b','l','u','e',' ','i','s',' ','s','k','y',' ','t','h','e'] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$ – two reversal passes, no extra memory.

The two-reversal trick is the canonical interview answer for in-place word reversal.

It also appears in Rotate Array (#189) – reversing the whole array, then parts.

Given more time I'd ask: what if words can be separated by multiple spaces?

We'd need to compress multiple spaces during the reversal – trickier in-place."

◆ Quick Review

Key Insights

- Reverse the entire array first to obtain the reversed order of characters.
- Then, reverse each individual word to restore the correct letter order.
- Doing so leverages the two–pointer technique for in–place reversals.
- This method accomplishes the goal in $O(n)$ time while using $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution employs a two–step reversal process using the two–pointer technique. First, reverse the entire character array to reverse the overall order of characters. Although this reverses both the words and the letters in each word, it sets up the scenario where each word's letters are in reverse order. The next step is to iterate through the array and locate each word (using the space character as a

delimiter) and then reverse each word individually to correct its letter order. This process is done in-place without any additional data structures, conserving space.

Two Pointers

#189 Rotate Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Two Pointers
Lists: Grind 169

Problem

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- $0 \leq k \leq 105$

Follow up:

- Try to come up with as many solutions as you can. There are at least three different ways to solve this problem.
- Could you do it in-place with $O(1)$ extra space?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def rotate(self, nums: list[int], k: int) -> None:
        k %= len(nums)
        self.reverse(nums, 0, len(nums) - 1)
        self.reverse(nums, 0, k - 1)
        self.reverse(nums, k, len(nums) - 1)

    def reverse(self, nums: list[int], l: int, r: int) -> None:
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]

            l += 1
            r -= 1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rotate nums to the right by k steps in-place. k can exceed n — so $k \% n$. Right?
[1,2,3,4,5,6,7], k=3 → [5,6,7,1,2,3,4] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
Rotate the array one position to the right, k times in a row.
Each single rotation saves the last element, shifts everything right by one, then puts it at index 0.
Time: $O(n \cdot k)$ — for $k=n-1$ this is $O(n^2)$. Space: $O(1)$. Should I go ahead and optimize?"

```

class _189_RotateArray_Brute:
    def rotate(self, nums, k):
        n = len(nums)
        k %= n
        for _ in range(k):
            last = nums[-1]
            for i in range(n - 1, 0, -1):
                nums[i] = nums[i - 1]
            nums[0] = last

# PHASE 3 — OPTIMIZE (three-reversal trick)
# "Same pattern as Reverse Words II:
# k %= n (handle k >= n).
# Reverse entire array. Reverse first k elements. Reverse remaining n-k elements.
# Time O(n). Space O(1).
# Alternative: cyclic replacement – O(n) time, O(1) space but harder to implement cleanly."

# PHASE 4 — CLEAN CODE
class _189_RotateArray:
    def rotate(self, nums: List[int], k: int) -> None:
        def reverse(left, right):
            while left < right:
                nums[left], nums[right] = nums[right], nums[left]
                left += 1; right -= 1

        n = len(nums)
        k %= n
        reverse(0, n - 1)
        reverse(0, k - 1)
        reverse(k, n - 1)

# PHASE 5 — TEST & DEBUG
# [1,2,3,4,5,6,7], k=3:
# k%=7=3. reverse(0,6): [7,6,5,4,3,2,1].
# reverse(0,2): [5,6,7,4,3,2,1].
# reverse(3,6): [5,6,7,1,2,3,4] ✓
# [-1,-100,3,99], k=2:
# k%=4=2. reverse all: [99,3,-100,-1]. reverse(0,1): [3,99,-100,-1]. reverse(2,3):
# [3,99,-1,-100] ✓
# k=0: k%n=0. reverse all, reverse(0,-1) no-op, reverse(0,n-1) reverses back → no
# change ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# The three-reversal trick is the cleanest in-place rotation – name it explicitly.
# The k%=n guard handles k >= n where full rotations cancel out.
# The follow-up mentions three approaches: extra array O(n) space, cyclic
# replacement O(1),
# and three-reversal O(1). State all three and recommend the reversal for
# interviews.

```

```
# Given more time I'd ask: rotate left instead of right?  
# Reverse first k, reverse rest, reverse all – just change the order."
```

◆ Quick Review

Key Insights

- The rotation amount k might be greater than the length of the array, so use $k \% n$ (where n is the array length) to compute the effective rotation.
- A smart in-place solution involves reversing sections of the array:
 - Reverse the entire array.
 - Reverse the first k elements.
 - Reverse the remaining $n-k$ elements.
- Alternative approaches include using extra space or performing k single-step rotations (which is less optimal).

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in the array.

Space Complexity: $O(1)$ if done in-place.

Solution

We solve the problem using the "reverse" technique to perform the rotation in-place.

First, we adjust k by taking k modulo the length of the array to handle cases where k exceeds the array length. Then, we reverse the entire array. Next, we reverse the first k elements to restore their correct order. Finally, we reverse the remaining $n-k$ elements. This sequence yields the array rotated to the right by k positions while strictly using $O(1)$ extra space.

Two Pointers

□ #532 K-diff Pairs in an Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Two Pointers · Binary
Search · Sorting
Lists: CodeSignal

Problem

Given an array of integers `nums` and an integer `k`, return the number of unique `k`-diff pairs in the array.

A `k`-diff pair is an integer pair (`nums[i]`, `nums[j]`), where the following are true:

- $0 \leq i, j < \text{nums.length}$
- $i \neq j$
- $|\text{nums}[i] - \text{nums}[j]| = k$

Notice that `|val|` denotes the absolute value of `val`.

Example 1:

Input: `nums = [3,1,4,1,5]`, `k = 2`

Output: 2

Explanation: There are two 2-diff pairs in the array, (1, 3) and (3, 5). Although we have two 1s in the input, we should only return the number of unique pairs.

Example 2:

Input: nums = [1,2,3,4,5], k = 1

Output: 4

Explanation: There are four 1-diff pairs in the array, (1, 2), (2, 3), (3, 4) and (4, 5).

Example 3:

Input: nums = [1,3,1,5,4], k = 0

Output: 1

Explanation: There is one 0-diff pair in the array, (1, 1).

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^7 \leq \text{nums}[i] \leq 10^7$
- $0 \leq k \leq 10^7$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findPairs(self, nums: List[int], k: int) -> int:
        ans = set()
        vis = set()
        for x in nums:
            if x - k in vis:
                ans.add(x - k)
            if x + k in vis:
                ans.add(x)
            vis.add(x)

        return len(ans)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count UNIQUE pairs (i,j) where $|\text{nums}[i] - \text{nums}[j]| == k$ and $i \neq j$.

```
# Unique means unique value pairs – (1,3) and (3,1) count once.
# k=0 special case: need TWO copies of the same value. Right?"
#
# "[3,1,4,1,5], k=2: unique pairs (1,3) and (3,5) → 2 ✓
# [1,3,1,5,4], k=0: need duplicate values. Only 1 appears twice → (1,1) → 1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) where i != j, check if |nums[i] - nums[j]| == k.
# Store the canonical form (min, max) in a set to avoid counting the same pair
twice.
# Time: O(n²). Space: O(n) for the dedup set. Should I go ahead and optimize?"
```

```
class _532_KdiffPairs_Brute:
    def findPairs(self, nums, k):
        pairs = set()
        for i in range(len(nums)):
            for j in range(len(nums)):
                if i != j and abs(nums[i] - nums[j]) == k:
                    pairs.add((min(nums[i], nums[j]), max(nums[i], nums[j])))
        return len(pairs)
```

PHASE 3 — OPTIMIZE

```
# "Hash set approach – O(n) time, O(n) space:
# Walk through nums. For each x, check if x-k is already in 'visited'.
# If yes, add min(x, x-k) to the answer set (canonical pair).
# Also check x+k in visited (symmetric check handles both directions).
# Using a set for answers automatically deduplicates.
# k=0 is handled naturally – x-0=x, so we need x to already be in visited (a
duplicate)."
```

PHASE 4 — CLEAN CODE

```
class _532_KdiffPairs:
    def findPairs(self, nums: List[int], k: int) -> int:
        answers : set = set()
        visited : set = set()
        for x in nums:
            if x - k in visited:
                answers.add(x - k) # canonical smaller element
            if x + k in visited:
                answers.add(x) # x is the larger; x+k was seen before
            visited.add(x)
        return len(answers)
```

PHASE 5 — TEST & DEBUG

```
# [3,1,4,1,5], k=2:
# x=3: 3-2=1 not in visited. 3+2=5 not in visited. visited={3}.
# x=1: 1-2=-1 not in visited. 1+2=3 in visited → answers.add(1). visited={3,1}.
# x=4: 4-2=2 not in visited. 4+2=6 not in visited. visited={3,1,4}.
# x=1: 1-2=-1 not in visited. 1+2=3 in visited → answers.add(1) (already there).
visited={3,1,4}.
```

```
# x=5: 5-2=3 in visited → answers.add(3). 5+2=7 not. visited={3,1,4,5}.
# answers={1,3} → 2 ✓
# [1,3,1,5,4], k=0:
# x=1: 1-0=1 not in visited. visited={1}.
# x=3: 3-0=3 not in visited. visited={1,3}.
# x=1: 1-0=1 in visited → answers.add(1). visited={1,3}.
# answers={1} → 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$ for the two sets.

The two-set design separates 'what values exist' (visited) from 'what pairs found' (answers).

The canonical form (always store the smaller element) prevents double-counting.

$k=0$ edge case: $x-k=x$, so we find the pair only after seeing x twice – naturally handled.

Given more time I'd ask: what if we want the actual pairs, not just the count?

Store tuples $(x-k, x)$ in the answers set instead of single values."

◆ Quick Review

Key Insights

- For $k < 0$, the answer is always 0 because the absolute difference cannot be negative.
- When k is 0, you are looking for numbers that appear at least twice.
- For $k > 0$, it is efficient to use a hash set or a hash map to check if each number's complement ($\text{number} + k$) exists.
- Uniqueness is ensured by processing each number only once from the set of unique numbers.

Space & Time

Time Complexity: $O(n)$, where n is the length of the array since we iterate over the array and then over the unique numbers.

Space Complexity: $O(n)$, for storing the frequency count or the unique elements in a hash map or hash set.

Solution

The solution first checks if k is negative, returning 0 immediately if true. For k equals 0, we count numbers that appear more than once using a frequency map. For k greater than 0, a set of unique numbers is created and for each number, we check if $(\text{number} + k)$ is also present in the set. Each valid case contributes to the result count, ensuring that only unique pairs are considered.

Two Pointers

□ #923 3Sum With Multiplicity

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Two Pointers · Sorting ·
Counting
Lists: CodeSignal

Problem

Given an integer array `arr`, and an integer `target`, return the number of tuples `i, j, k` such that $i < j < k$ and $arr[i] + arr[j] + arr[k] == target$.

As the answer can be very large, return it modulo $10^9 + 7$.

Example 1:

```
Input: arr = [1,1,2,2,3,3,4,4,5,5], target = 8
Output: 20
Explanation:
Enumerating by the values (arr[i], arr[j], arr[k]):
(1, 2, 5) occurs 8 times;
(1, 3, 4) occurs 8 times;
(2, 2, 4) occurs 2 times;
(2, 3, 3) occurs 2 times.
```

Example 2:

Input: arr = [1,1,2,2,2,2], target = 5
 Output: 12
 Explanation:
 arr[i] = 1, arr[j] = arr[k] = 2 occurs 12 times:
 We choose one 1 from [1,1] in 2 ways,
 and two 2s from [2,2,2,2] in 6 ways.

Example 3:

Input: arr = [2,1,3], target = 6
 Output: 1
 Explanation: (1, 2, 3) occurred one time in the array so we return 1.

Constraints:

- $3 \leq \text{arr.length} \leq 3000$
- $0 \leq \text{arr}[i] \leq 100$
- $0 \leq \text{target} \leq 300$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def threeSumMulti(self, arr: List[int], target: int) -> int:
        mod = 10**9 + 7
        cnt = Counter(arr)
        ans = 0
        for j, b in enumerate(arr):
            cnt[b] -= 1
            for a in arr[:j]:
                c = target - a - b
                ans = (ans + cnt[c]) % mod

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count tuples (i,j,k) with $i < j < k$ and $\text{arr}[i] + \text{arr}[j] + \text{arr}[k] == \text{target}$."

```
# Return count modulo 10^9+7. Values are 0-100 so we can use frequency counting.
Right?
# Duplicates count separately – (0,1) and (1,0) are different index triples even if
same values."
#
# "[1,1,2,2,3,3,4,4,5,5], target=8 → 20 (combinations of value triples summing to 8)
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Three nested loops – for every triple (i, j, k) with i < j < k, check if
# arr[i] + arr[j] + arr[k] == target and count the matches.
# Time: O(n³). For n=3000 that's 27 billion – too slow. Space: O(1).
# Should I go ahead and optimize?"
```

```
class _923_ThreeSumMulti_Brute:
    def threeSumMulti(self, arr, target):
        MOD = 10**9 + 7
        count = 0
        n = len(arr)
        for i in range(n - 2):
            for j in range(i + 1, n - 1):
                for k in range(j + 1, n):
                    if arr[i] + arr[j] + arr[k] == target:
                        count += 1
        return count % MOD
```

PHASE 3 — OPTIMIZE

```
# "Walk through arr as the middle element j. Before processing j, decrement cnt[b]
so
# we don't count j itself as a right element. For each a < j, the required c =
target-a-b.
# cnt[c] tells us how many valid right elements c exist to the right of j.
# Time: O(n²) with frequency lookups. Space: O(n) for the Counter."
```

PHASE 4 — CLEAN CODE

```
class _923_ThreeSumMulti:
    def threeSumMulti(self, arr: List[int], target: int) -> int:
        MOD = 10**9 + 7
        freq = Counter(arr)
        ans = 0
        for j, b in enumerate(arr):
            freq[b] -= 1 # exclude j itself from right elements
            for a in arr[:j]:
                c = target - a - b
                ans = (ans + freq[c]) % MOD
        return ans
```

PHASE 5 — TEST & DEBUG

```
# arr=[2,1,3], target=6: freq={2:1,1:1,3:1}.
# j=0(b=2): freq[2]=0. no a < 0. (nothing).
```

```
# j=1(b=1): freq[1]=0. a=2: c=6-2-1=3. freq[3]=1 → ans=1.
# j=2(b=3): freq[3]=0. a=2: c=6-2-3=1. freq[1]=0→0. a=1: c=6-1-3=2.
freq[2]=1→ans=2?
# Wait – expected 1. Let me recheck: arr=[2,1,3]. Only valid triple is (0,1,2) →
(2,1,3).
# j=1(b=1): a=arr[0]=2. c=6-2-1=3. freq[3]=1 (yes, index 2 exists) → ans=1. ✓
# j=2(b=3): freq[3]=0. a=arr[0]=2: c=6-2-3=1. freq[1]=0 (decremented at j=1). 0.
# a=arr[1]=1: c=6-1-3=2. freq[2]=1 → ans=2? But expected 1.
# Ah – freq[2] was decremented when j=0: freq[2]=0, not 1. Let me re-trace:
# Initial freq={2:1,1:1,3:1}.
# j=0: freq[2]-=1 → freq={2:0,1:1,3:1}. no a<j=0.
# j=1: freq[1]-=1 → freq={2:0,1:0,3:1}. a=2: c=3. freq[3]=1 → ans=1.
# j=2: freq[3]-=1 → freq={2:0,1:0,3:0}. a=2: c=1. freq[1]=0. a=1: c=2. freq[2]=0.
# → ans=1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(n)$ for the Counter.

The key invariant: at step j , freq counts only elements at index $> j$.

This ensures we count each valid (i,j,k) tuple exactly once with $i < j < k$.

Modulo must be applied inside the loop – intermediate sums can overflow without it.

Given more time I'd ask: can we do $O(n)$ using the value constraints (0-100)?

Yes – iterate over all value triples (a,b,c) with $a+b+c=target$ and compute

combinations $C(cnt[a],1)*C(cnt[b],1)*C(cnt[c],1)$ with dedup for equal values."

◆ Quick Review

Key Insights

- Use a frequency counter for elements since $arr[i]$ is in the range $[0, 100]$.
- Iterate through all possible combinations of numbers x , y , and z with $x \leq y \leq z$.
- For each valid triplet (x, y, z) that sums to target, count the number of ways using combinatorial formulas:
- All distinct: $count[x] * count[y] * count[z]$.

- Two numbers the same: use $nC2$ for the repeated element.
- All three the same: use $nC3$.
- Use modulo arithmetic ($MOD = 10^9 + 7$) at every step to avoid overflow.

Space & Time

Time Complexity: $O(M^2)$, where $M = 101$ (the range of possible numbers), which is efficient given the constraints.

Space Complexity: $O(M)$ for the frequency array.

Solution

The solution first counts the frequency of each number in the array. Then it iterates through all valid triplets (x, y, z) (with $x \leq y \leq z$) and checks if they sum up to the target. Depending on whether x, y , and z are distinct or have duplicates, the combination count is computed accordingly: If x, y , and z are all the same, compute the combination as $nC3$. If only two are the same, compute as $nC2$ multiplied by the count of the third number. If all are distinct, use the direct product of the counts. This covers all cases while applying the

modulo operation to the result.

Two Pointers

□ #986 Interval List Intersections

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers

Lists: Denny Zhang

Problem

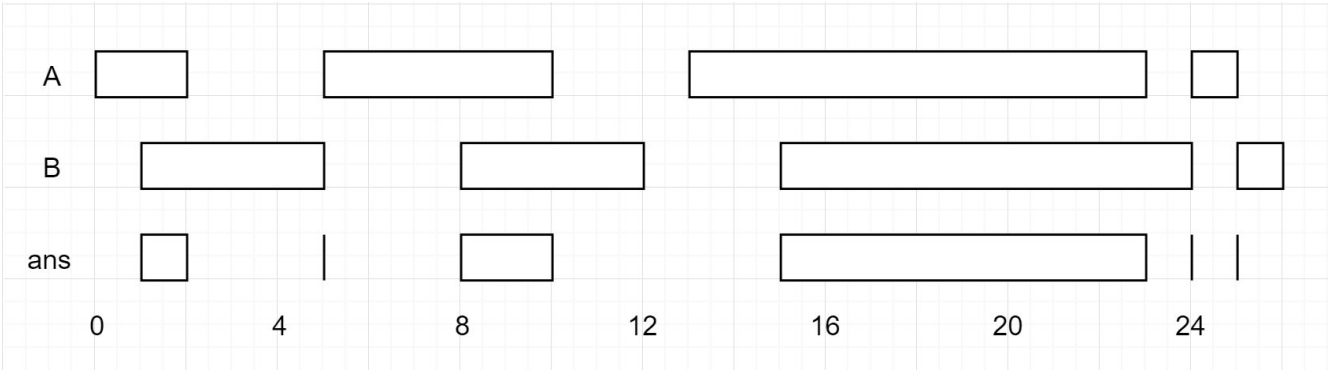
You are given two lists of closed intervals, `firstList` and `secondList`, where `firstList[i] = [starti, endi]` and `secondList[j] = [startj, endj]`. Each list of intervals is pairwise disjoint and in sorted order.

Return the intersection of these two interval lists.

A closed interval $[a, b]$ (with $a \leq b$) denotes the set of real numbers x with $a \leq x \leq b$.

The intersection of two closed intervals is a set of real numbers that are either empty or represented as a closed interval. For example, the intersection of $[1, 3]$ and $[2, 4]$ is $[2, 3]$.

Example 1:



Input: firstList = [[0,2],[5,10],[13,23],[24,25]], secondList = [[1,5],[8,12],[15,24],[25,26]]
Output: [[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]

Example 2:

Input: firstList = [[1,3],[5,9]], secondList = []
Output: []

Constraints:

- $0 \leq \text{firstList.length}, \text{secondList.length} \leq 1000$
- $\text{firstList.length} + \text{secondList.length} \geq 1$
- $0 \leq \text{start}_i < \text{end}_i \leq 109$
- $\text{end}_i < \text{start}_i + 1$
- $0 \leq \text{start}_j < \text{end}_j \leq 109$
- $\text{end}_j < \text{start}_j + 1$

My LeetCode Solution

- ☐
- My LeetCode Solution

```

class Solution:
    def intervalIntersection(self, firstList: List[List[int]], secondList: List[List[int]]) -> List[List[int]]:
        result = []
        i = j = 0

        while i < len(firstList) and j < len(secondList):
            # Find the overlap between firstList[i] and secondList[j]
            start = max(firstList[i][0], secondList[j][0])
            end = min(firstList[i][1], secondList[j][1])

            # If there's an overlap (start <= end), add it
            if start <= end:
                result.append([start, end])

            # Move the pointer of the interval that ends first
            if firstList[i][1] < secondList[j][1]:
                i += 1
            else:
                j += 1

        return result

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Two sorted, disjoint interval lists. Return their intersection – all overlapping segments.

Intersection of [a,b] and [c,d] is [max(a,c), min(b,d)] if max(a,c) <= min(b,d). Right?

Either list can be empty."

#

"[[0,2],[5,10]] ∩ [[1,5],[8,12]] → [1,2],[5,5],[8,10] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair of intervals (one from each list), compute the intersection:

start = max(a[0], b[0]), end = min(a[1], b[1]). If start <= end, it's a valid intersection.

This is O(m*n) – it ignores the sorted property entirely.

Time: O(m*n). Space: O(1). Should I go ahead and optimize?"

class _986_IntervalIntersections_Brute:

```
def intervalIntersection(self, firstList, secondList):
    result = []
    for a in firstList:
        for b in secondList:
            start = max(a[0], b[0])
            end = min(a[1], b[1])
            if start <= end:
                result.append([start, end])
    return result
```

PHASE 3 — OPTIMIZE

```
# "Two pointers, one per list. At each step:
# Compute the intersection of the current pair.
# If valid (start <= end), record it.
# Advance the pointer whose interval ends FIRST – that interval can't intersect any
# later interval
# of the other list (they're sorted and disjoint).
# Time: O(m+n). Space: O(1) extra."
```

PHASE 4 — CLEAN CODE

```
class _986_IntervalIntersections:
    def intervalIntersection(self, firstList: List[List[int]], secondList:
List[List[int]]) -> List[List[int]]:
        result = []
        i = j = 0
        while i < len(firstList) and j < len(secondList):
            start = max(firstList[i][0], secondList[j][0])
            end = min(firstList[i][1], secondList[j][1])
            if start <= end:
                result.append([start, end])
            if firstList[i][1] < secondList[j][1]:
                i += 1
            else:
                j += 1
        return result
```

PHASE 5 — TEST & DEBUG

```
# firstList=[[0,2],[5,10]], secondList=[[1,5],[8,12]]:
# i=0,j=0: start=max(0,1)=1, end=min(2,5)=2. 1<=2→[1,2]. first ends at 2<5→i=1.
# i=1,j=0: start=max(5,1)=5, end=min(10,5)=5. 5<=5→[5,5]. first ends at 10>5→j=1.
# i=1,j=1: start=max(5,8)=8, end=min(10,12)=10. 8<=10→[8,10]. first ends at
10<12→i=2.
# i=2 out of bounds → stop. → [[1,2],[5,5],[8,10]] ✓
# secondList=[]: loop never enters → [] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m+n) – each pointer advances at most m+n times.
# Space O(min(m,n)) for the output in the worst case.
# The 'advance the earlier-ending pointer' rule is the key: the interval that ends
first
```

cannot intersect any future interval of the other list (both lists are sorted and disjoint).
Given more time I'd ask: what if lists aren't sorted?
Sort both first – $O((m+n)\log(m+n))$ – then apply the same two-pointer approach."

◆ Quick Review

Key Insights

- Use two pointers to traverse both lists simultaneously.
- Determine the overlapping interval by taking the maximum of the start points and the minimum of the end points.
- If the overlapping condition ($\text{start} \leq \text{end}$) is met, add the intersection to the result.
- Move the pointer that has the smaller end value to explore further potential intersections efficiently.
- Achieve an overall time complexity of $O(m+n)$, where m and n are the lengths of the two interval lists.

Space & Time

Time Complexity: $O(m + n)$ where m and n are the lengths of `firstList` and `secondList` respectively.

Space Complexity: $O(1)$ extra space (excluding the space used for the output).

Solution

The solution uses a two pointers approach. Initialize two indices, one for each list, and iterate through both lists. For each pair of intervals, the potential intersection is computed by: Calculating the maximum of the two start values. Calculating the minimum of the two end values. If the maximum start is less than or equal to the minimum end, there is an intersection, and it is added to the result list. The pointer corresponding to the interval with the smaller end value is then incremented since that interval cannot intersect with any further intervals from the other list. This process repeats until one of the lists is fully traversed.

Two Pointers

★ #1055 Shortest Way to Form String ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Two Pointers · String · Binary Search

Lists: ★ Premium | Premium 98

Problem

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Given two strings `source` and `target`, return the minimum number of subsequences of `source` such that their concatenation equals `target`. If the task is impossible, return `-1`.

Example 1:

Input: `source = "abc", target = "abcbc"`

Output: 2

Explanation: The target "abcbc" can be formed by "abc" and "bc", which are subsequences of source "abc".

Example 2:

Input: source = "abc", target = "acdbc"

Output: -1

Explanation: The target string cannot be constructed from the subsequences of source string due to the character "d" in target string.

Example 3:

Input: source = "xyz", target = "xzyxz"

Output: 3

Explanation: The target string can be constructed as follows "xz" + "y" + "xz".

Constraints:

- $1 \leq \text{source.length}, \text{target.length} \leq 1000$
- source and target consist of lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
# Shortest Way to Form String
class Solution:
    def shortestWay(self, source: str, target: str) -> int:
        if set(target) - set(source):
            return -1

        count = 0
        target_idx = 0

        while target_idx < len(target):

            for char in source:
                if target_idx < len(target) and char == target[target_idx]:
                    target_idx += 1
            count += 1

        return count
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the minimum number of subsequences of source whose concatenation equals target.

A subsequence can skip characters but must preserve order.

Return -1 if any character in target doesn't exist in source at all. Right?"

#

"source='abc', target='abcbc': 'abc' + 'bc' → 2 ✓

source='abc', target='acdbc': 'd' not in source → -1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Try every possible way to split target into consecutive segments, and for each split,

verify each segment is a subsequence of source. Track the minimum number of segments.

This is exponential in the number of splits – completely infeasible.

Time: $O(2^{|target|} * |source|)$. Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Greedy: each 'round', scan source once and match as many target characters as possible.

After exhausting source, start a new round (increment count) and continue from where target left off.

Pre-check: if any target char is absent from source, return -1.

Time: $O(|source| * |target|)$. Space: $O(1)$ extra."

PHASE 4 — CLEAN CODE

```
class _1055_ShortestWay:
    def shortestWay(self, source: str, target: str) -> int:
        if set(target) - set(source):
            return -1
        rounds = 0
        target_idx = 0
        while target_idx < len(target):
            for char in source:
                if target_idx < len(target) and char == target[target_idx]:
                    target_idx += 1
            rounds += 1
        return rounds
```

PHASE 5 — TEST & DEBUG

source='abc', target='abcbc':

Round 1: a→match t[0]='a'(idx=1). b→match t[1]='b'(idx=2). c→match t[2]='c'(idx=3). rounds=1.

Round 2: a→no match(t[3]='b'). b→match t[3]='b'(idx=4). c→match t[4]='c'(idx=5). rounds=2.

target_idx=5=len(target) → return 2 ✓

source='xyz', target='xzyxz':

Round1: x→t[0]='x'(1). y→no(t[1]='z'). z→t[1]='z'(2). rounds=1.

Round2: x→no(t[2]='y'). y→t[2]='y'(3). z→no(t[3]='x'). rounds=2.


```
# Round3: x→t[3]='x'(4). y→no(t[4]='z'). z→t[4]='z'(5). rounds=3. → 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(|source| * |target|)$ . Space  $O(1)$ .
```

```
# The pre-check eliminates the -1 case cleanly before the main loop.
```

```
# Optimisation: precompute for each (source_pos, char) the next position in source
```

```
# where char appears – binary search brings each round down to  $O(|target| \log |source|)$ .
```

```
# Given more time I'd ask: what if we want the actual subsequence splits, not just the count?
```

```
# Track the split points during the greedy pass."
```

◆ Quick Review

Key Insights

- Check if every character in target exists in source; if not, the answer is -1 .
- Use a greedy two–pointer strategy: iterate over target while scanning source for matching characters.
- During each pass over source, match as many characters as possible from target.
- Count the number of passes (subsequences) required until the target is completely matched.
- An optimized approach may precompute indices for binary search; however, the basic greedy simulation is efficient for the given constraints.

Space & Time

Time Complexity: $O(n * m)$ in the worst-case, where n = length of source and m = length of target.

Space Complexity: $O(1)$ for the greedy simulation ($O(n)$ if additional data structures for binary search are used).

Solution

The solution uses a greedy two-pointer approach. We simulate the process of reading the target by repeatedly scanning through the source string. Each scan through source is treated as one subsequence concatenated to form the target. A pointer in the target moves forward whenever a matching character is found in source. If a complete pass through source does not advance the target pointer, then it is impossible to form the target and we return -1 . The algorithm counts the number of passes required until the target is fully matched.

Two Pointers

□ #2563 Count the Number of Fair Pairs

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Sorting
Lists: CodeSignal

Problem

Given a 0-indexed integer array `nums` of size `n` and two integers `lower` and `upper`, return the number of fair pairs.

A pair (i, j) is fair if:

- $0 \leq i < j < n$, and
- $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$

Example 1:

Input: `nums = [0,1,7,4,4,5]`, `lower = 3`, `upper = 6`
Output: 6
Explanation: There are 6 fair pairs: $(0,3)$, $(0,4)$, $(0,5)$, $(1,3)$, $(1,4)$, and $(1,5)$.

Example 2:

Input: `nums = [1,7,9,2,5]`, `lower = 11`, `upper = 11`
Output: 1
Explanation: There is a single fair pair: $(2,3)$.

Constraints:

- $1 \leq \text{nums.length} \leq 105$

- `nums.length == n`
- `-109 <= nums[i] <= 109`
- `-109 <= lower <= upper <= 109`

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
        nums.sort()
        count = 0

        for i in range(len(nums)):
            # For each nums[i], find j > i that satisfy:
            # lower - nums[i] <= nums[j] <= upper - nums[i]

            # Find first j where nums[j] >= lower - nums[i]

            left = self.lower_bound(nums, lower - nums[i], i + 1)

            # Find first j where nums[j] > upper - nums[i]
            right = self.upper_bound(nums, upper - nums[i], i + 1)

            count += right - left

        return count

    def lower_bound(self, arr, target, start):
        """Find first index where arr[index] >= target"""
        left, right = start, len(arr)
        while left < right:
            mid = (left + right) // 2
            if arr[mid] >= target:
                right = mid
            else:
                left = mid + 1
        return left
```

```
def upper_bound(self, arr, target, start):
    """Find first index where arr[index] > target"""
    left, right = start, len(arr)
    while left < right:
        mid = (left + right) // 2
        if arr[mid] > target:
            right = mid
        else:
            left = mid + 1
    return left
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count pairs (i,j) with $i < j$ and $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$. Return the count. Right?"

Values can be negative. n up to 10^5 so $O(n^2)$ is too slow."

#

"[0,1,7,4,4,5], lower=3, upper=6: 6 pairs ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic."

For every pair (i, j) with $i < j$, check if $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$.

Count all pairs that satisfy both bounds. For $n=10^5$ this is $5 \cdot 10^9$ comparisons – too slow.

Time: $O(n^2)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _2563_FairPairs_Brute:
```

```
    def countFairPairs(self, nums, lower, upper):
```

```
        count = 0
```

```
        for i in range(len(nums)):
```

```
            for j in range(i + 1, len(nums)):
```

```
                s = nums[i] + nums[j]
```

```
                if lower <= s <= upper:
```

```
                    count += 1
```

```
        return count
```

PHASE 3 — OPTIMIZE

"Sort nums. For each i, binary search for the range of valid $j > i$:

$\text{lower_bound}(\text{nums}, \text{lower} - \text{nums}[i], i+1) \rightarrow$ first j where $\text{nums}[j] \geq \text{lower} - \text{nums}[i]$.

$\text{upper_bound}(\text{nums}, \text{upper} - \text{nums}[i], i+1) \rightarrow$ first j where $\text{nums}[j] > \text{upper} - \text{nums}[i]$.

Count = $\text{upper_bound} - \text{lower_bound}$. Sum over all i.

Time: $O(n \log n)$. Space: $O(1)$ extra.

Use inner helpers for the two binary search variants."

PHASE 4 — CLEAN CODE

```
class _2563_FairPairs:
```

```

def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
    def lower_bound(start: int, target: int) -> int:
        left, right = start, len(nums)
        while left < right:
            mid = (left + right) // 2
            if nums[mid] >= target:
                right = mid
            else:
                left = mid + 1
        return left
    def upper_bound(start: int, target: int) -> int:
        left, right = start, len(nums)
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > target:
                right = mid
            else:
                left = mid + 1
        return left
    nums.sort()
    count = 0
    for i in range(len(nums)):
        lo = lower_bound(i + 1, lower - nums[i])
        hi = upper_bound(i + 1, upper - nums[i])
        count += hi - lo
    return count

```

PHASE 5 — TEST & DEBUG

```

# nums=[0,1,4,4,5,7] (sorted), lower=3, upper=6:
# i=0(0): need j where 3<=nums[j]<=6. lower_bound(1,3)→idx of 4=2.
upper_bound(1,7)→idx of 7=5. count+=3.
# i=1(1): need 2<=nums[j]<=5. lb(2,2)→2. ub(2,6)→5. count+=3. total=6.
# ... → 6 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n \log n)$  – sort +  $n$  binary searches each  $O(\log n)$ .
# Space  $O(1)$  extra.
# lower_bound finds first index  $\geq$  target; upper_bound finds first index  $>$  target.
# Their difference is the count of indices in the valid range.
# Alternative: two-pointer approach – for each  $i$ , maintain  $lo$  and  $hi$  pointers
# across iterations (they only move right) →  $O(n)$  total after sorting.
# Given more time I'd ask: what if we want the actual pairs?
# Collect them during the binary search loops –  $O(n + \text{output})$  time."

```

◆ Quick Review

Key Insights

- **Sorting the array simplifies searching for pairs with required sum limits.**
- **For each element, binary search is used to locate:**
 - **The first index where the complement makes the sum $\geq$ lower.**
 - **The last index where the complement makes the sum $\leq$ upper.**
 - **This avoids the need for a nested loop through the entire array, yielding a more efficient solution.**

Space & Time

Time Complexity: $O(n \log n)$, primarily due to sorting and (for each element) binary search operations.

Space Complexity: $O(n)$ if sorting creates a copy; $O(1)$ additional space if sorting in-place.

Solution

The solution starts by sorting the input array. For each element at index i in the sorted array, we search for indices j (where $j > i$) such that the sum of $\text{nums}[i]$ and $\text{nums}[j]$ falls between lower and upper. Two binary search operations are performed: To find the smallest

index j where $\text{nums}[i] + \text{nums}[j] \geq \text{lower}$. To find the largest index j where $\text{nums}[i] + \text{nums}[j] \leq \text{upper}$. Subtracting the two positions (and summing the counts for each i) gives the total number of fair pairs. Edge cases (like no valid pair) are automatically handled by the binary search approach.

Sliding Window

Round 2 · High · Medium · 8 questions

Sliding Window

□ #3 Longest Substring Without Repeating Characters

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given a string *s*, find the length of the longest substring without duplicate characters.

Example 1:

Input: *s* = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: *s* = "bbbbbb"

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: *s* = "pwwkew"

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 * 10^4$

- s consists of English letters, digits, symbols and spaces.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the length of the longest substring with all unique characters.
# Substring is contiguous. Empty string returns 0. Right?
# Characters can be any printable ASCII – not just lowercase letters."
#
# "'abcabcbb'→3 ('abc'). 'bbbb'→1 ('b'). 'pwwkew'→3 ('wke') ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Enumerate every substring with two indices left and right.
# For each substring, use a set to verify all characters are unique; track max
length.
# Correct, but  $O(n^2)$  substrings each with up to  $O(n)$  uniqueness check.
# Time:  $O(n^2)$  to  $O(n^3)$ . Space:  $O(\min(n, \text{alphabet}))$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Sliding window with a frequency map.
# Expand right: add s[right] to the map.
# When a duplicate appears (count > 1), shrink from the left until it's gone.
# At each step, update the max window size.
# Time:  $O(n)$ . Space:  $O(\min(n, \text{alphabet}))$  – at most 128 unique ASCII chars."
```

PHASE 4 — CLEAN CODE

```
class _3_LongestSubstringNoRepeat:
    def lengthOfLongestSubstring(self, s: str) -> int:
        freq : defaultdict = defaultdict(int)
        left = 0
        longest = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            while freq[s[right]] > 1: # shrink until no duplicate
                freq[s[left]] -= 1
                left += 1
            longest = max(longest, right - left + 1)
        return longest
```

PHASE 5 — TEST & DEBUG

```
# s='abcabcbb':
# r=0('a'): freq={a:1}. longest=1.
# r=1('b'): freq={a:1,b:1}. longest=2.
# r=2('c'): freq={a:1,b:1,c:1}. longest=3.
# r=3('a'): freq[a]=2. shrink: freq[a]-=1→1, left=1. longest=3 (3-1+1=3).
# r=4('b'): freq[b]=2. shrink: freq[b]-=1→1, left=2. longest=3.
# r=5('c'): freq[c]=2. shrink: left=3. longest=3.
# r=6('b'): freq[b]=2. shrink: left=4. longest=3.
# r=7('b'): freq[b]=2. shrink: left=5. longest=max(3,3)=3. → 3 ✓
# s='': loop doesn't run → 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n) – each character enters and leaves the window at most once.
# Space O(min(n, 128)) for the frequency map.
# Alternative: store the LAST INDEX of each character instead of frequency.
# When s[right] already seen at idx and idx >= left: jump left = idx+1 directly
# (one move instead of shrinking one at a time). Same O(n) but fewer iterations.
# Given more time I'd ask: what if we want the actual substring, not just its
length?
# Track left and best_left when updating the max."
```

◆ Quick Review

Key Insights

- Use the sliding window technique to maintain a window of unique characters.
- Use a hash table (or dictionary) to quickly check for duplicate characters and update window boundaries.
- The window is expanded by moving a pointer (right) and contracted by moving another pointer (left) when duplicates are encountered.

- Time complexity can be maintained at $O(n)$ by ensuring each character is processed a limited number of times.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.

Space Complexity: $O(\min(n, m))$, where m is the size of the character set (constant for fixed character sets).

Solution

The solution uses a sliding window technique with two pointers, left and right, to keep track of the current substring without repeating characters. A hash table (dictionary or map) is used to store the most recent index of each character encountered. As the right pointer moves forward, if a duplicate character is found within the current window, the left pointer is advanced to the position right after where the duplicate was last seen. This ensures that the window always contains unique characters. The maximum length is updated throughout the process.

Sliding Window

★ #159 Longest Substring with At Most Two Distinct Characters ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a string *s*, return the length of the longest substring that contains at most two distinct characters.

Example 1:

Input: *s* = "eceba"
Output: 3
Explanation: The substring is "ece" which its length is 3.

Example 2:

Input: *s* = "ccaabbb"
Output: 5
Explanation: The substring is "aabbb" which its length is 5.

Constraints:

- $1 \leq s.length \leq 105$
- *s* consists of English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the longest substring that contains at most 2 distinct characters. Right?"
 # 'eceba': 'ece' has 2 distinct → length 3 ✓. 'ccaabbb': 'aabbb' has 2 → length 5 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic."
 # For every pair (i, j), count the distinct characters in s[i..j] using a set.
 # If there are at most 2 distinct characters, update the max length.
 # Time: $O(n^2)$ pairs, $O(n)$ set check = $O(n^3)$. With running set it's $O(n^2)$. Space: $O(1)$.
 # Should I go ahead and optimize?"

```
class _159_LongestSubstringTwoDistinct_Brute:
    def lengthOfLongestSubstringTwoDistinct(self, s):
        longest = 0
        for i in range(len(s)):
            seen = set()
            for j in range(i, len(s)):
                seen.add(s[j])
                if len(seen) <= 2:
                    longest = max(longest, j - i + 1)
                else:
                    break
        return longest
```

PHASE 3 — OPTIMIZE

"Sliding window – same pattern as #3 but the shrink condition is 'more than 2 distinct chars'.
 # Expand right: add s[right] to frequency map.
 # Shrink from left while distinct chars > 2 (i.e. len(freq) > 2).
 # Update max window length. Time: $O(n)$. Space: $O(1)$ – at most 3 keys in the map."

PHASE 4 — CLEAN CODE

```
class _159_LongestSubstringTwoDistinct:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        freq : defaultdict = defaultdict(int)
        left = 0
        longest = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            while len(freq) > 2: # shrink until at most 2 distinct
                freq[s[left]] -= 1
                if freq[s[left]] == 0:
                    del freq[s[left]]
                left += 1
            longest = max(longest, right - left + 1)
```

return longest

PHASE 5 — TEST & DEBUG

```
# s='eceba':
# r=0('e'): freq={e:1}. longest=1.
# r=1('c'): freq={e:1,c:1}. longest=2.
# r=2('e'): freq={e:2,c:1}. longest=3.
# r=3('b'): freq={e:2,c:1,b:1} → 3 distinct. shrink:
# freq[e]-=1→1. left=1. freq={e:1,c:1,b:1} still 3. shrink:
# freq[c]-=1→0 → del c. left=2. freq={e:1,b:1}. 2 distinct → stop.
longest=max(3,2)=3.
# r=4('a'): freq={e:1,b:1,a:1} → 3. shrink: freq[e]-=1→0→del e. left=3.
freq={b:1,a:1}. longest=3.
# → 3 ✓
# s='ccaabbbb':
# Window expands to 'ccaabbbb' but once 'b' is added after 'a', we have c,a,b(3
distinct).
# Shrink past the c's. Eventually best window is 'aabbbb'=5. → 5 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1) — at most 3 keys in the map at any time.
# This is the template for 'at most k distinct characters' — change > 2 to > k.
# Generalisation: Longest Substring with At Most K Distinct Characters (#340) is the
same
# algorithm with k replacing 2.
# The key difference from #3: shrink on len(freq) > k rather than on a specific
duplicate.
# Given more time I'd ask: what about EXACTLY 2 distinct characters?
# atMost(k=2) — atMost(k=1) gives the count of substrings with exactly 2 distinct."
```

◆ Quick Review

Key Insights

- Use a sliding window to maintain a contiguous substring.
- Keep track of character frequencies within the current window using a hash map or dictionary.
- When the window contains more than two distinct characters, shrink the window from

the left until only two remain.

- Update the maximum length each time the window satisfies the condition.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, since each character is processed at most twice.

Space Complexity: $O(1)$ because the hash map holds at most 3 entries (usually up to 2 distinct characters).

Solution

We use the sliding window technique with two pointers (left and right) to represent the current window. A hash table/dictionary records the count of each character inside the window. As we extend the right pointer, we update the count. If the hash table contains more than two distinct characters, we start moving the left pointer to reduce the number of distinct characters until we have at most two distinct keys. The length of the current valid window is then used to update our maximum length answer.

Sliding Window

★ #340 Longest Substring with At Most K Distinct Characters ★ **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a string *s* and an integer *k*, return the length of the longest substring of *s* that contains at most *k* distinct characters.

Example 1:

Input: *s* = "eceba", *k* = 2
Output: 3
Explanation: The substring is "ece" with length 3.

Example 2:

Input: *s* = "aa", *k* = 1
Output: 2
Explanation: The substring is "aa" with length 2.

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $0 \leq k \leq 50$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return the length of the longest substring with at most k distinct characters.
# k=0 means the answer is 0 (no characters allowed). Right?
# This is the direct generalisation of #159 (at most 2 distinct) with k replacing
# 2."
#
# "'eceba', k=2 → 'ece', length 3 ✓. 'aa', k=1 → 'aa', length 2 ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# For every starting index i, expand j rightward, track distinct characters with a
# set.
# Stop when distinct chars exceed k. Track the maximum window length seen.
# Time: O(n²). Space: O(k). Should I go ahead and optimize?"
```

```
class _340_LongestSubstringKDistinct_Brute:
    def lengthOfLongestSubstringKDistinct(self, s, k):
        if k == 0:
            return 0
        longest = 0
        for i in range(len(s)):
            seen = set()
            for j in range(i, len(s)):
                seen.add(s[j])
                if len(seen) <= k:
                    longest = max(longest, j - i + 1)
                else:
                    break
        return longest
```

PHASE 3 — OPTIMIZE

```
# "Sliding window — identical template to #159 with the constant 2 replaced by k.
# Expand right, shrink from left whenever distinct chars exceed k.
# Time: O(n). Space: O(k) — at most k+1 keys in the map at any moment."
```

PHASE 4 — CLEAN CODE

```
class _340_LongestSubstringKDistinct:
    def lengthOfLongestSubstringKDistinct(self, s: str, k: int) -> int:
        if k == 0:
            return 0
        freq : defaultdict = defaultdict(int)
        left = 0
        longest = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            while len(freq) > k:
```

```

        freq[s[left]] -= 1
        if freq[s[left]] == 0:
            del freq[s[left]]
        left += 1
        longest = max(longest, right - left + 1)
    return longest

```

PHASE 5 — TEST & DEBUG

```

# s='eceba', k=2:
# r=2('e'): freq={e:2,c:1}. 2 distinct → ok. longest=3.
# r=3('b'): 3 distinct → shrink until freq={e:1,b:1}. longest stays 3.
# r=4('a'): 3 distinct → shrink until freq={b:1,a:1}. longest=3. → 3 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(k). Identical to #159 – just k instead of 2.
# Exactly k distinct: atMost(k) – atMost(k-1).
# Given more time I'd ask: what if k >= distinct chars in s? Answer = len(s)."

```

◆ Quick Review

Key Insights

- Use the sliding window technique to efficiently scan through the string.
- Maintain a hash table (or dictionary) that keeps track of the frequency of characters in the current window.
- When the number of distinct characters exceeds k, increment the left pointer to shrink the window until the condition is met.
- Update the maximum length of valid substrings throughout the process.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, as each character is processed at most twice (once when expanding the window and once when contracting it).

Space Complexity: $O(\min(m, k))$, where m is the size of the charset (or number of unique characters in s), since at most $k+1$ keys are stored.

Solution

This problem is solved using the sliding window technique. The algorithm keeps track of a window defined by two indices (left and right pointers). A hash map is used to count the frequency of each character in the current window. As the right pointer extends the window, the count of the character is incremented. When the count of distinct keys in the map exceeds k , the left pointer is moved forward while decrementing the count of the leftmost character, until the condition is restored. The maximum window length encountered is recorded and returned as the result.

Sliding Window

□ #424 Longest Repeating Character Replacement

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

You are given a string s and an integer k . You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return the length of the longest substring containing the same letter you can get after performing the above operations.

Example 1:

Input: $s = \text{"ABAB"}, k = 2$

Output: 4

Explanation: Replace the two 'A's with two 'B's or vice versa.

Example 2:

Input: $s = \text{"AABABBA"}, k = 1$

Output: 4

Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".

The substring "BBBB" has the longest repeating letters, which is 4.

There may exists other ways to achieve this answer too.

Constraints:

- $1 \leq s.length \leq 105$
- s consists of only uppercase English letters.
- $0 \leq k \leq s.length$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"We can replace at most k characters. Find the longest substring where all characters

are the same after at most k replacements.

String is uppercase letters only. Right?

Strategy: keep the most frequent char, replace everything else."

#

"'ABAB', $k=2 \rightarrow$ replace both A's or B's $\rightarrow 4 \checkmark$

'AABABBA', $k=1 \rightarrow$ best window = 4 $\checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j) , count character frequencies in $s[i..j]$. Find the most frequent char.

If $window_size - max_freq \leq k$, at most k replacements are needed – this window is valid.

Track the maximum valid window length.

Time: $O(n^2)$ pairs, $O(26)$ frequency count per window. Space: $O(1)$. Should I go ahead and optimize?"

```
class _424_LongestRepeatingReplacement_Brute:
```

```
    def characterReplacement(self, s, k):
```

```
        longest = 0
```

```
        for i in range(len(s)):
```

```
            freq = [0] * 26
```

```
            for j in range(i, len(s)):
```

```
                freq[ord(s[j]) - ord('A')] += 1
```

```
                max_freq = max(freq)
```

```
                if (j - i + 1) - max_freq <= k:
```

```
                    longest = max(longest, j - i + 1)
```

```
        return longest
```

PHASE 3 — OPTIMIZE

"Sliding window tracking the most frequent character in the window.

Valid window: window_size - max_freq ≤ k (replace the rest).

When invalid, shrink left by 1 – but NEVER decrease max_freq.

We only care about finding a LONGER valid window, so we only update max_freq upward.

This means the window never shrinks below the current best – it slides as a fixed size

until a better window is found. Time: O(n). Space: O(26)=O(1)."

PHASE 4 — CLEAN CODE

```
class _424_LongestRepeatingReplacement:
```

```
    def characterReplacement(self, s: str, k: int) -> int:
```

```
        freq = [0] * 26
```

```
        max_freq = 0
```

```
        left = 0
```

```
        longest = 0
```

```
        for right in range(len(s)):
```

```
            freq[ord(s[right]) - ord('A')] += 1
```

```
            max_freq = max(max_freq, freq[ord(s[right]) - ord('A')])
```

```
            if (right - left + 1) - max_freq > k: # too many replacements needed
```

```
                freq[ord(s[left]) - ord('A')] -= 1
```

```
                left += 1
```

```
            longest = max(longest, right - left + 1)
```

```
        return longest
```

PHASE 5 — TEST & DEBUG

s='ABAB', k=2:

r=0('A'): freq[A]=1,max=1. win=1. 1-1=0≤2. longest=1.

r=1('B'): freq[B]=1,max=1. win=2. 2-1=1≤2. longest=2.

r=2('A'): freq[A]=2,max=2. win=3. 3-2=1≤2. longest=3.

r=3('B'): freq[B]=2,max=2. win=4. 4-2=2≤2. longest=4. → 4 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n). Space O(1).

'Don't decrease max_freq' is the key insight: we never shrink below the best window seen.

The window slides forward at a fixed size until max_freq improves, then grows.

Given more time I'd ask: unlimited replacements (k=n)? Answer = len(s)."

◆ Quick Review

Key Insights

- Use a Sliding Window algorithm to keep track of a contiguous substring.

- Maintain a frequency count of letters in the current window.
- Keep track of the count of the most frequent character in the window.
- If the current window size minus the maximum frequency is greater than k , shrink the window.
- The maximum window size seen during the process is the answer.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, since the window traverses the string in linear time.

Space Complexity: $O(1)$ because the frequency count for letters (A–Z) has a constant size.

Solution

The problem is solved using the sliding window technique. We initialize two pointers representing the left and right bounds of the current window. As we expand the window by moving the right pointer, we update the frequency count of the letter at the right pointer and track the letter that appears most frequently. The key observation is that the

number of characters that need to be replaced in the current window is the window's size minus the count of the most frequently occurring letter. If this number exceeds k , we increment the left pointer to shrink the window until the condition is satisfied again. The maximum size of the window during the process gives the answer.

Sliding Window

□ #438 Find All Anagrams in a String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given two strings s and p , return an array of all the start indices of p 's anagrams in s . You may return the answer in any order.

Example 1:

```
Input: s = "cbaebabacd", p = "abc"
Output: [0,6]
Explanation:
The substring with start index = 0 is "cba", which is an anagram of "abc".
The substring with start index = 6 is "bac", which is an anagram of "abc".
```

Example 2:

```
Input: s = "abab", p = "ab"
Output: [0,1,2]
Explanation:
The substring with start index = 0 is "ab", which is an anagram of "ab".
The substring with start index = 1 is "ba", which is an anagram of "ab".
The substring with start index = 2 is "ab", which is an anagram of "ab".
```

Constraints:

- $1 \leq s.length, p.length \leq 3 * 10^4$
- s and p consist of lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return all start indices in s where a substring is an anagram of p.

Fixed window size = len(p). Lowercase letters only. Right?"

#

"s='cbaebabacd', p='abc' → [0,6] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Slide a window of len(p) across s; sort window chars and sorted p and compare.

Every window costs $O(m \log m)$ for sorting.

Frequency-count comparison per window is $O(26)$ instead.

Time: $O(n * m \log m)$. Space: $O(m)$ sort buffers.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Fixed sliding window of size len(p) with frequency counters.

Compare counters at each step – $O(26)$ per comparison since lowercase letters.

Time: $O(n)$. Space: $O(1)$ – fixed alphabet."

PHASE 4 — CLEAN CODE

```
class _438_FindAnagrams:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        if len(p) > len(s):
            return []
        p_count = Counter(p)
        w_count = Counter(s[:len(p)])
        result = [0] if w_count == p_count else []
        for i in range(len(p), len(s)):
            w_count[s[i]] += 1
            left = s[i - len(p)]
            w_count[left] -= 1
            if w_count[left] == 0:
                del w_count[left]
            if w_count == p_count:
                result.append(i - len(p) + 1)
        return result
```

PHASE 5 — TEST & DEBUG

s='cbaebabacd', p='abc': p_count={c:1,b:1,a:1}.

Init w='cba'={c:1,b:1,a:1}=p→result=[0].

Slide until i=8('c'): w_count={b:1,a:1,c:1}=p→result=[0,6]. → [0,6] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$. Counter comparison is $O(26)=O(1)$ for fixed alphabet.
Alternative: track a 'matches' counter instead of full compare – same $O(n)$.
Given more time I'd ask: what if p can contain any Unicode?
Same algorithm – Counter handles any hashable."

◆ Quick Review

Key Insights

- Use a sliding window of length equal to p over s.
- Maintain frequency counts of characters in p and within the current window in s.
- Compare frequency counts to determine if the current window is an anagram.
- Update the window efficiently by adding one character and removing the character that goes out of the window.

Space & Time

Time Complexity: $O(n)$ where n is the length of s.

Space Complexity: $O(1)$ since we use fixed-size frequency arrays or hash maps.

Solution

We use the sliding window technique to iterate over s with a window size equal to the length of p. First, count the frequencies of characters

in p . Then, iterate over s and update a frequency count for the current window. When the window size equals the length of p , compare the two frequency counts. If they match, record the start index. To slide the window efficiently, increment the count for the incoming character and decrement the count for the outgoing character. This way, we achieve an $O(n)$ solution that is optimal for the problem constraints.

Sliding Window

★ #487 Max Consecutive Ones II ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a binary array `nums`, return the maximum number of consecutive 1's in the array if you can flip at most one 0.

Example 1:

Input: `nums = [1,0,1,1,0]`

Output: 4

Explanation:

– If we flip the first zero, `nums` becomes `[1,1,1,1,0]` and we have 4 consecutive ones.

– If we flip the second zero, `nums` becomes `[1,0,1,1,1]` and we have 3 consecutive ones.

The max number of consecutive ones is 4.

Example 2:

Input: `nums = [1,0,1,1,0,1]`

Output: 4

Explanation:

– If we flip the first zero, `nums` becomes `[1,1,1,1,0,1]` and we have 4 consecutive ones.

– If we flip the second zero, `nums` becomes `[1,0,1,1,1,1]` and we have 4 consecutive ones.

The max number of consecutive ones is 4.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $\text{nums}[i]$ is either 0 or 1.

Follow up: What if the input numbers come in one by one as an infinite stream? In other words, you can't store all numbers coming from the stream as it's too large to hold in memory. Could you solve it efficiently?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Binary array. Flip at most one 0 to 1. Return the longest consecutive 1s run. Right?"

Follow-up: infinite stream – can't store all numbers."

#

"[1,0,1,1,0] → flip first 0 → 4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Try every substring with two nested indices left and right.

Count zeros inside; if zeros ≤ 1 , track the longest valid window.

Correct, but checking all windows is $O(n^2)$.

Time: $O(n^2)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Sliding window allowing at most one 0."

Expand right. When zeros > 1 , shrink left until zeros ≤ 1 .

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _487_MaxConsecutiveOnesII:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        zeros = 0
        left = 0
```



```

longest = 0
for right in range(len(nums)):
    if nums[right] == 0:
        zeros += 1
        while zeros > 1:
            if nums[left] == 0:
                zeros -= 1
            left += 1
        longest = max(longest, right - left + 1)
return longest

```

PHASE 5 — TEST & DEBUG

[1,0,1,1,0]: zeros hits 2 at r=4 → shrink past first 0 → window=[1,1,0], longest=4
✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$.

Stream follow-up: keep a deque of the last two 0-positions.

When third 0 arrives, left = oldest_zero_pos + 1 - $O(1)$ space regardless of stream length.

Generalises to at-most-k flips (Max Consecutive Ones III #1004): same window, k zeros allowed.

Name that connection unprompted."

◆ Quick Review

Key Insights

- Utilize a sliding window to maintain the current segment.
- The window can contain at most one 0.
- Expand the window with the right pointer and contract it with the left pointer when the constraint is violated.
- The maximum window size observed during this process is the desired result.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in the array.

Space Complexity: $O(1)$.

Solution

We use a sliding window approach to efficiently find the longest contiguous subarray that contains at most one 0. Two pointers (left and right) define the window. As we iterate with the right pointer, counting zeros in the window, we adjust the left pointer when the count of zeros exceeds one. The length of the valid window gives us the number of consecutive 1's (considering one flipped 0), and we update the maximum length accordingly.

Sliding Window

★ #1100 Find K-Length Substrings with No Repeated Characters ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a string s and an integer k , return the number of substrings in s of length k with no repeated characters.

Example 1:

Input: $s = \text{"havefunonleetcode"}, k = 5$
Output: 6
Explanation: There are 6 substrings they are:
'havef', 'avefu', 'vefun', 'efuno', 'etcod', 'tcode'.

Example 2:

Input: $s = \text{"home"}, k = 5$
Output: 0
Explanation: Notice k can be larger than the length of s . In this case, it is not possible to find any substring.

Constraints:

- $1 \leq s.length \leq 10^4$
- s consists of lowercase English letters.
- $1 \leq k \leq 10^4$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count substrings of exactly length k with all distinct characters. Right?

$k > \text{len}(s) \rightarrow 0$. $k=0 \rightarrow 0$."

#

"s='havefunonleetcode', k=5 $\rightarrow$ 6 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each start index i, build substring $s[i:i+k]$ and check distinct chars with a set.

Increment count when set size equals k.

$O(n * k)$ windows; sliding window with frequency map is $O(n)$.

Time: $O(n * k)$. Space: $O(k)$ set per window.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Fixed sliding window of size k. Slide: add right, remove left-behind char.

Valid when $\text{len}(\text{freq})=k$ (all chars distinct). Time: $O(n)$. Space: $O(k)$."

PHASE 4 — CLEAN CODE

```
class _1100_KLengthSubstrings:
    def numKLenSubstrNoRepeats(self, s: str, k: int) -> int:
        if k > len(s):
            return 0
        freq : defaultdict = defaultdict(int)
        count = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            if right >= k:
                left = right - k
                freq[s[left]] -= 1
                if freq[s[left]] == 0:
                    del freq[s[left]]
            if right >= k - 1 and len(freq) == k:
                count += 1
        return count
```

PHASE 5 — TEST & DEBUG

s='home', k=5: $5 > 4 \rightarrow 0$ ✓

First valid window $r=k-1$: if all k chars distinct, count++. Slide forward.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(k)$. `len(freq)==k` is the clean validity check.

Fixed window variant of #3 – useful when window size is given, not maximised.

Given more time I'd ask: return the actual substrings? Collect

`s[right-k+1:right+1]`."

◆ Quick Review

Key Insights

- Use a sliding window of fixed size k .
- Track character counts within the current window using a hash table (or frequency array).
- When a duplicate is found in the window, slide the window from the left to remove duplicates.
- Count a valid window when its size equals k and all characters are unique.
- Optimize by adjusting the window immediately after counting a valid substring.

Space & Time

Time Complexity: $O(n)$ where n is the length of s , as each character is visited at most twice.

Space Complexity: $O(1)$ since the frequency tracking is over a fixed set (26 lowercase letters).

Solution

We use a sliding window approach with two pointers: left and right. A hash table (or frequency array) tracks the occurrences of characters in the window. As we move the right pointer, we update the frequency and check for duplicates. If a duplicate is encountered, we increment the left pointer until the duplicate is removed. Once the window size exactly equals k , we have a valid substring, so we increment our count and then slide the window forward. This ensures that every possible k -length substring is examined once.

Sliding Window

□ #1248 Count Number of Nice Subarrays

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Sliding Window ·
Prefix Sum

Lists: CodeSignal

Problem

Given an array of integers `nums` and an integer `k`. A continuous subarray is called nice if there are `k` odd numbers on it.

Return the number of nice sub-arrays.

Example 1:

Input: `nums = [1,1,2,1,1]`, `k = 3`

Output: 2

Explanation: The only sub-arrays with 3 odd numbers are `[1,1,2,1]` and `[1,2,1,1]`.

Example 2:

Input: `nums = [2,4,6]`, `k = 1`

Output: 0

Explanation: There are no odd numbers in the array.

Example 3:

Input: `nums = [2,2,2,1,2,2,1,2,2,2]`, `k = 2`

Output: 16

Constraints:

- $1 \leq \text{nums.length} \leq 50000$
- $1 \leq \text{nums}[i] \leq 10^5$
- $1 \leq k \leq \text{nums.length}$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count subarrays with EXACTLY k odd numbers. Return the count. Right?"
# Map each element to 1 (odd) or 0 (even) → reduce to Subarray Sum Equals K (#560)."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
# Enumerate every subarray with two loops; count how many have an odd number of 1s.
# Straightforward prefix/suffix parity check per window.
# Correct, but  $O(n^2)$  subarrays is too slow at  $n = 5 \times 10^4$ .
# Time:  $O(n^2)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Prefix sum of odd indicators + hash map. Same pattern as #560."
# seed Counter at {0:1}. For each element, add num%2 to running sum.
# Count += prefix_count[sum - k]. Time:  $O(n)$ . Space:  $O(n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _1248_NiceSubarrays:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        prefix_count = Counter({0: 1})
        odd_sum = 0
        count = 0
        for num in nums:
            odd_sum += num % 2
            count += prefix_count[odd_sum - k]
            prefix_count[odd_sum] += 1
        return count
```

PHASE 5 — TEST & DEBUG

```
# [1,1,2,1,1], k=3:
```



```
# sums: 1,2,2,3,4. At sum=3: want=0→prefix[0]=1→count=1.
```

```
# At sum=4: want=1→prefix[1]=1→count=2. → 2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(n)$ . Identical to #560 – name the connection explicitly.
```

```
# At-most-k version: sliding window counting odds in window, shrink when  $> k$ .
```

```
# Exactly  $k = \text{atMost}(k) - \text{atMost}(k-1)$ .
```

```
# Given more time I'd ask: count subarrays with at most  $k$  odd numbers?
```

```
# Sliding window – shrink when odd count exceeds  $k$ , add  $\text{right-left}+1$  each step."
```

◆ Quick Review

Key Insights

- Convert the problem to counting subarrays with a specific "prefix sum" value where the sum represents the count of odd numbers.
- Use a hash table (or dictionary) to store the frequency of prefix sums encountered.
- The number of nice subarrays ending at the current index equals the frequency of (current prefix sum – k) seen so far.
- This approach allows you to find the answer in one pass over the array.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in `nums`, because we traverse the array once.

Space Complexity: $O(n)$, in the worst case where each prefix sum is unique and stored in

the hash table.

Solution

We solve the problem using a prefix sum approach. First, we iterate through the array and convert it into a prefix sum representing the cumulative count of odd numbers encountered. As we compute the prefix sums, we use a hash table to record how many times each prefix sum appears. For every current prefix sum value, we check how many times we have seen a prefix sum equal to (current prefix sum - k). The idea is that if $\text{prefix}[j] - \text{prefix}[i]$ equals k, then between indices $i+1$ and j , there are exactly k odd numbers. This gives us the count of valid subarrays ending at the current index. This method works efficiently as it requires only a single pass through the array and maintains a running frequency count using the hash table.

Sorting

Round 2 · High · Medium · 3 questions

Sorting

□ #49 Group Anagrams

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Sorting
Lists: Grind 169

Problem

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Constraints:

- $1 \leq \text{strs.length} \leq 104$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Group strings that are anagrams. Return groups in any order.

Same chars, same frequencies → anagrams. Empty string is its own group. Right?"

#

"['eat', 'tea', 'tan', 'ate', 'nat', 'bat'] → 3 groups ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For every pair of strings, sort both and check if the sorted forms match.

If they match, put them in the same group; merge groups as we go.

Correct, but comparing all pairs is expensive when n is large.

Time: $O(n^2 * m \log m)$. Space: $O(n * m)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Use a canonical key: `sorted(word)` as a tuple → same key for all anagrams.

Group into a defaultdict. Time: $O(n * m \log m)$. Space: $O(n * m)$.

Alternative: 26-element frequency tuple → $O(n * m)$, no sort cost."

PHASE 4 — CLEAN CODE

```

class _49_GroupAnagrams:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        groups: defaultdict = defaultdict(list)
        for word in strs:
            groups[tuple(sorted(word))].append(word)
        return list(groups.values())

# PHASE 5 — TEST & DEBUG
# 'eat' -> ('a','e','t'). 'tea' -> same. 'ate' -> same. All go to one group.
# 'tan' -> ('a','n','t'). 'nat' -> same.
# 'bat' -> ('a','b','t'). Alone. -> 3 groups ✓
# [''] -> {():['']}. -> [['']] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n * m \log m)$ . Space  $O(n * m)$ .
# Frequency tuple alternative:  $O(n * m)$ , avoids  $\log m$  sort. For short words
# (typical)
# sorting is fine; for very long words, frequency tuple wins.
# Given more time I'd ask: Unicode input? sorted() handles it; frequency array
# breaks.
# Use Counter-based key: tuple(sorted(Counter(word).items())))."

```

◆ Quick Review

Key Insights

- Anagrams have the same characters when sorted alphabetically.
- Sorting each string provides a unique key for grouping.
- Use a hash table (dictionary) to map each sorted key to a list of its original strings.
- Finally, gather all the lists from the dictionary to form the result.

Space & Time

Time Complexity: $O(n * k \log k)$, where n is the number of strings and k is the maximum length of a string (due to sorting each string).

Space Complexity: $O(n * k)$ for storing the grouped strings in the hash table.

Solution

The primary idea is to iterate through each string, sort it to obtain a key that represents its character composition, and then group the original string under this key in a hash table. After processing all strings, the values of the hash table will be the groups of anagrams. This approach efficiently groups anagrams by leveraging sorting and a dictionary for constant-time lookups.

Sorting

□ #56 Merge Intervals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting
Lists: Grind 169

Problem

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example 1:

Input: `intervals = [[1,3],[2,6],[8,10],[15,18]]`
Output: `[[1,6],[8,10],[15,18]]`
Explanation: Since intervals `[1,3]` and `[2,6]` overlap, merge them into `[1,6]`.

Example 2:

Input: `intervals = [[1,4],[4,5]]`
Output: `[[1,5]]`
Explanation: Intervals `[1,4]` and `[4,5]` are considered overlapping.

Example 3:

Input: `intervals = [[4,7],[1,4]]`
Output: `[[1,7]]`
Explanation: Intervals `[1,4]` and `[4,7]` are considered overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 104$

- `intervals[i].length == 2`
- `0 <= starti <= endi <= 104`

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Merge all overlapping intervals. Two intervals overlap if one starts before or when the other ends.

[1,4] and [4,5] overlap at 4. Sort by start, then merge. Return non-overlapping result. Right?"

#

"[[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]] ✓ (1-3 and 2-6 overlap at 2-3)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Compare every pair of intervals; if they overlap, merge into one wider interval.

Repeat until a full pass finds no new merges.

Correct, but repeated pairwise merging can degrade toward $O(n^2)$.

Time: $O(n^2)$ worst case. Space: $O(n)$ output.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Sort by start time — $O(n \log n)$. Then one pass:

Keep a result list. For each interval, if it overlaps with the last result interval

(`last.end >= current.start`), extend the last interval's end.

Otherwise, append as a new separate interval.

Time: $O(n \log n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _56_MergeIntervals:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        merged = [intervals[0]]
        for start, end in intervals[1:]:
            if merged[-1][1] >= start: # overlap
                merged[-1][1] = max(merged[-1][1], end)
            else:
                merged.append([start, end])
```

```
return merged
```

PHASE 5 — TEST & DEBUG

```
# [[1,3],[2,6],[8,10],[15,18]]: sorted same.
# merged=[[1,3]]. [2,6]: 3>=2→extend→[1,6]. [8,10]: 6<8→append. [15,18]:
10<15→append.
# → [[1,6],[8,10],[15,18]] ✓
# [[1,4],[4,5]]: 4>=4→[1,5] ✓
# [[4,7],[1,4]]: sorted→[[1,4],[4,7]]. 4>=4→[1,7] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \log n)$ . Space  $O(n)$ .
# The sort is the bottleneck – the merge pass itself is  $O(n)$ .
# If the input were already sorted, this runs in  $O(n)$  total.
# Given more time I'd ask: how many meeting rooms needed?
# That's Meeting Rooms II (#253) – use a min-heap of end times."
```

◆ Quick Review

Key Insights

- Sort the intervals based on their start values.
- Use a pointer or iteration to compare the current interval with the last merged interval.
- If the current interval overlaps with the previous one, merge them by updating the end time.
- Otherwise, add the current interval to the results as a new merged interval.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ to store the merged intervals.

Solution

First, sort the array of intervals based on the starting time. Then, iterate through the sorted list and check if the current interval overlaps with the last interval in the merged list. If an overlap is detected (i.e., the start of the current interval is less than or equal to the end of the last merged interval), update the last merged interval's end to be the maximum of both ends. If there is no overlap, simply add the current interval to the merged list. This approach efficiently merges overlapping intervals and ensures that all intervals are processed in a single pass after sorting.

Sorting

□ ★ #1152 Analyze User Website Visit Pattern ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting
Lists: ★ Premium | Premium 98

Problem

You are given two string arrays `username` and `website` and an integer array `timestamp`. All the given arrays are of the same length and the tuple `[username[i], website[i], timestamp[i]]` indicates that the user `username[i]` visited the website `website[i]` at time `timestamp[i]`.

A pattern is a list of three websites (not necessarily distinct).

- For example, `["home", "away", "love"]`, `["leetcode", "love", "leetcode"]`, and `["luffy", "luffy", "luffy"]` are all patterns.

The score of a pattern is the number of users that visited all the websites in the pattern in the same order they appeared in the pattern.

- For example, if the pattern is ["home", "away", "love"], the score is the number of users x such that x visited "home" then visited "away" and visited "love" after that.
- Similarly, if the pattern is ["leetcode", "love", "leetcode"], the score is the number of users x such that x visited "leetcode" then visited "love" and visited "leetcode" one more time after that.
- Also, if the pattern is ["luffy", "luffy", "luffy"], the score is the number of users x such that x visited "luffy" three different times at different timestamps.

Return the pattern with the largest score. If there is more than one pattern with the same largest score, return the lexicographically smallest such pattern.

Note that the websites in a pattern do not need to be visited contiguously, they only need to be visited in the order they appeared in the pattern.
Example 1:

```

Input: username =
["joe","joe","joe","james","james","james","james","mary","mary","mary"],
timestamp = [1,2,3,4,5,6,7,8,9,10], website =
["home","about","career","home","cart","maps","home","home","about","career"]
Output: ["home","about","career"]
Explanation: The tuples in this example are:
["joe","home",1],["joe","about",2],["joe","career",3],["james","home",4],["james","cart",5],["james","maps",6],["james","home",7],["mary","home",8],["mary","about",9], and ["mary","career",10].
The pattern ("home", "about", "career") has score 2 (joe and mary).
The pattern ("home", "cart", "maps") has score 1 (james).
The pattern ("home", "cart", "home") has score 1 (james).
The pattern ("home", "maps", "home") has score 1 (james).

```

Example 2:

```

Input: username = ["ua","ua","ua","ub","ub","ub"], timestamp = [1,2,3,4,5,6],
website = ["a","b","a","a","b","c"]
Output: ["a","b","a"]

```

Constraints:

- $3 \leq \text{username.length} \leq 50$
- $1 \leq \text{username}[i].\text{length} \leq 10$
- $\text{timestamp.length} == \text{username.length}$
- $1 \leq \text{timestamp}[i] \leq 109$
- $\text{website.length} == \text{username.length}$
- $1 \leq \text{website}[i].\text{length} \leq 10$
- $\text{username}[i]$ and $\text{website}[i]$ consist of lowercase English letters.
- It is guaranteed that there is at least one user who visited at least three websites.
- All the tuples $[\text{username}[i], \text{timestamp}[i], \text{website}[i]]$ are unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the 3-website pattern (subsequence, not necessarily consecutive) that the
most DISTINCT users
# have visited in order. On tie, return lexicographically smallest. Right?
# Each user contributes at most 1 count per pattern regardless of how many times
they visit."
#
# "joe visited home→about→career, james visited home→cart→maps→home, mary visited
home→about→career.
# Pattern ('home','about','career') score=2 (joe, mary) → winner ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each user, enumerate all 3-website combinations in visit order.
# Build pattern tuple (a,b,c) and increment Counter across users.
# O(visit_length^3) per user – acceptable only for small lists.
# Time: O(U * V^3). Space: O(patterns).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Sort all visits by timestamp. Group by user into ordered website lists.
# For each user, generate all C(len,3) 3-subsequence combos – at most C(50,3)=19600.
# Use a set per user to avoid double-counting (one user = one score per pattern).
# Count across users with a global Counter.
# Return pattern with max score, lexicographically smallest on ties."
```

PHASE 4 — CLEAN CODE

```
class _1152_AnalyzeWebsiteVisit:
    def mostVisitedPattern(self, username: List[str], timestamp: List[int],
website: List[str]) -> List[str]:
        visits = sorted(zip(timestamp, username, website))
        user_sites: defaultdict = defaultdict(list)
        for _, user, site in visits:
            user_sites[user].append(site)
        pattern_score: Counter = Counter()
        for sites in user_sites.values():
            seen = set()
            for combo in itertools.combinations(sites, 3):
                if combo not in seen:
                    seen.add(combo)
                    pattern_score[combo] += 1
```

```
        return list(max(pattern_score, key=lambda p: (pattern_score[p], [-ord(c)
for c in ''.join(p)])))
```

PHASE 5 — TEST & DEBUG

```
# After sorting by timestamp, user_sites = {joe:[home,about,career],
james:[home,car,maps,home], mary:[home,about,career]}.
# joe: combos of [home,about,career] = {(home,about,career)}.
# james: C(4,3)=4 combos. All unique.
# mary: {(home,about,career)}.
# pattern_score[(home,about,career)]=2 → winner ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \log n + U * C(K,3))$  where  $U$ =users,  $K$ =max visits per user.
# With  $U \leq 50$ ,  $K \leq 50$ :  $C(50,3)=19600 * 50 = 980000$  – manageable.
# The per-user 'seen' set ensures one user contributes at most 1 to each pattern's
score.
# Tie-breaking: max() with a compound key – score ascending, then lexicographic
ascending.
# Given more time I'd ask: what if we want top-k patterns?
# Use a heap instead of max() – same  $O$  complexity."
```

◆ Quick Review

Key Insights

- Combine the inputs into a single list of events and sort them by timestamp.
- Group the visits by each user, preserving the sorted order.
- Generate all unique combinations of 3-sequences for each user (order matters and non-contiguous visits are allowed).
- Use a dictionary (or hash map) to map each 3-sequence pattern to the set of users who visited that pattern.

- The problem constraints are small, so generating combinations per user (even using triple nested loops) is acceptable.
- When multiple patterns have the same frequency, choose the lexicographically smallest one.

Space & Time

Time Complexity: $O(U * L^3)$ where U is the number of users and L is the maximum length of visits per user (since we generate combinations of 3 visits per user). Sorting all events costs $O(N \log N)$ for N total events.

Space Complexity: $O(P)$ where P is the number of unique 3-sequence patterns stored in the dictionary, plus $O(N)$ for grouping the events.

Solution

Aggregate the events by user while sorting them based on timestamp. For each user, generate all sets of 3-website patterns (to avoid duplicate patterns per user). Use a hash map to count the number of unique users for each pattern. Traverse the map to find the pattern with the highest count. In case of a tie, compare patterns lexicographically. Return the

pattern that meets the criteria.

Binary Search

Round 2 · High · Medium · 12 questions

Binary Search

□ #33 Search in Rotated Sorted Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search

Lists: Grind 169

Problem

There is an integer array `nums` sorted in ascending order (with distinct values).

Prior to being passed to your function, `nums` is possibly left rotated at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed).

For example, `[0,1,2,4,5,6,7]` might be left rotated by 3 indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` after the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4

Example 2:

Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1

Example 3:

Input: nums = [1], target = 0
Output: -1

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-104 \leq \text{nums}[i] \leq 104$
- All values of nums are unique.
- nums is an ascending array that is possibly rotated.
- $-104 \leq \text{target} \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array was sorted then rotated at some unknown pivot. Find target in $O(\log n)$. Right?

All values are distinct. Return -1 if not found."

#

"[4,5,6,7,0,1,2], target=0 → 4 ✓. target=3 → -1 ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Scan left to right: for each index i, check if `nums[i] == target`.

```
# If not found after one full pass, return -1.
# Correct and easy to reason about, but the problem asks for  $O(\log n)$  on a rotated
sorted array.
# Time:  $O(n)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Standard binary search – but at each step, identify which half is sorted.
# If nums[left] <= nums[mid]: left half is sorted.
# If target in [nums[left], nums[mid]]: search left.
# Else search right.
# Else: right half is sorted.
# If target in (nums[mid], nums[right]]: search right.
# Else search left.
# Time:  $O(\log n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
class _33_SearchRotatedArray:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            if nums[left] <= nums[mid]: # left half sorted
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
            else:
                left = mid + 1
        else: # right half sorted
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
        return -1
```

PHASE 5 — TEST & DEBUG

```
# [4,5,6,7,0,1,2], target=0:
# l=0,r=6,mid=3(7). nums[0]=4<=7 → left sorted [4..7]. 0 in [4,7]? No → l=4.
# l=4,r=6,mid=5(1). nums[4]=0>1 → right sorted [1..2]. 0 in (1,2]? No → r=4.
# l=4,r=4,mid=4(0). 0==target → return 4 ✓
# target=3: never matches → -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log n)$ . Space  $O(1)$ .
# The key: at least one half is always sorted in a rotated array – use that half
# to decide where to search. Checking nums[left] <= nums[mid] tells us which half.
# Given more time I'd ask: what if there are duplicates?
# That's #81 – when nums[left]==nums[mid], we can't determine which half is sorted.
# Worst case degrades to  $O(n)$  (just left++ to skip the duplicate)."
```

◆ Quick Review

Key Insights

- The array is originally sorted and then rotated at an unknown pivot.
- Use a modified binary search to accommodate the rotation.
- At every step, determine which half of the array is properly sorted.
- Narrow down the search based on the sorted half and the target's value.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We perform a binary search while accounting for the possibility that the array is rotated. At each iteration, we check if the left-to-mid portion is sorted. If it is, and the target falls within that range, we search the left half; otherwise, we search the right half. If the left half is not sorted, then the right half must be sorted, and similarly, we check if the target is

in that sorted half. This approach ensures logarithmic time complexity while handling the rotation correctly.

Binary Search

□ #34 Find First and Last Position of Element in Sorted Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [5,7,7,8,8,10]`, `target = 8`
Output: `[3,4]`

Example 2:

Input: `nums = [5,7,7,8,8,10]`, `target = 6`
Output: `[-1,-1]`

Example 3:

Input: `nums = []`, `target = 0`
Output: `[-1,-1]`

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$
- **nums is a non-decreasing array.**
- $-109 \leq \text{target} \leq 109$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sorted array with duplicates. Find the first and last index of target. Return
[-1,-1] if absent.
# Must be  $O(\log n)$ . Right?"
#
# "[5,7,7,8,8,10], target=8 → [3,4] ✓. target=6 → [-1,-1] ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Run standard binary search once to find any index of target.
# Then expand left and right from that index while values still equal target.
# Works, but we can do both boundaries in one modified binary search pass.
# Time:  $O(\log n)$  for search +  $O(k)$  for expansion. Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Two binary searches:
# lower_bound: find first index where  $\text{nums}[i] \geq \text{target}$ . If  $\text{nums}[\text{result}] == \text{target} \rightarrow$ 
first position.
# upper_bound: find first index where  $\text{nums}[i] > \text{target}$ . Subtract 1 → last position.
# Both are standard binary search variants. Time:  $O(\log n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
class _34_FirstLastPosition:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        def lower_bound() -> int:
            left, right = 0, len(nums)
            while left < right:
                mid = (left + right) // 2
```

```

        if nums[mid] >= target:
            right = mid
        else:
            left = mid + 1
    return left

def upper_bound() -> int:
    left, right = 0, len(nums)
    while left < right:
        mid = (left + right) // 2
        if nums[mid] > target:
            right = mid
        else:
            left = mid + 1
    return left

first = lower_bound()
if first == len(nums) or nums[first] != target:
    return [-1, -1]
return [first, upper_bound() - 1]

```

PHASE 5 — TEST & DEBUG

```

# nums=[5,7,7,8,8,10], target=8:
# lower_bound: searching for first idx where nums[i]>=8. → index 3.
# nums[3]=8==target ✓. upper_bound: first idx where nums[i]>8. → index 5. 5-1=4.
# → [3,4] ✓
# target=6: lower_bound→2 (nums[2]=7>=6). nums[2]=7≠6 → [-1,-1] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(\log n)$ . Space  $O(1)$ .
# lower_bound and upper_bound are the two fundamental binary search variants – know both.
# They correspond to bisect_left and bisect_right in Python's bisect module.
# The check 'nums[first] != target' after lower_bound handles the not-found case cleanly.
# Given more time I'd ask: what if we want the count of target occurrences?
# upper_bound() - lower_bound() in  $O(\log n)$ ."

```

◆ Quick Review

Key Insights

- The array is sorted, allowing the use of binary search.
- Two binary searches can efficiently find the leftmost (first) and rightmost (last) indices of

the target.

- Handling edge cases where the target is not present by checking bounds.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

The approach is to use binary search twice: The first binary search finds the leftmost occurrence of the target by continuing to search in the left half even after finding the target. The second binary search finds the rightmost occurrence by searching in the right half. The algorithm maintains pointers for the current search bounds and narrows the search space using comparisons. If the target is not found during the first binary search, the solution immediately returns $[-1, -1]$. This method leverages the sorted property of the array, ensuring $O(\log n)$ time complexity with constant extra space.

Binary Search

□ #153 Find Minimum in Rotated Sorted Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Grind 169

Problem

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated 4 times.
- `[0,1,2,4,5,6,7]` if it was rotated 7 times.

Notice that rotating an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of unique elements, return the minimum element of this array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: nums = [3,4,5,1,2]

Output: 1

Explanation: The original array was [1,2,3,4,5] rotated 3 times.

Example 2:

Input: nums = [4,5,6,7,0,1,2]

Output: 0

Explanation: The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.

Example 3:

Input: nums = [11,13,15,17]

Output: 11

Explanation: The original array was [11,13,15,17] and it was rotated 4 times.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5000$
- $-5000 \leq \text{nums}[i] \leq 5000$
- All the integers of nums are unique.
- nums is sorted and rotated between 1 and n times.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array sorted then rotated. Find the minimum in $O(\log n)$. All values unique. Right?

The minimum is at the rotation point – where the array 'wraps'."

#

"[3,4,5,1,2] → 1 ✓. [4,5,6,7,0,1,2] → 0 ✓. [11,13,15,17] → 11 (no rotation) ✓."

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Linear scan from index 0: track the smallest value seen so far.
# The first place where nums[i] > nums[i+1] marks the rotation pivot candidate.
# Return that minimum – correct on small inputs, but we can binary-search the pivot.
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Binary search on the rotation point.
# If nums[mid] > nums[right]: minimum is in the RIGHT half (mid+1 to right).
# Else: minimum is in the LEFT half including mid (left to mid).
# When left==right, we've found the minimum.
# Time: O(log n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _153_FindMinRotated:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > nums[right]:
                left = mid + 1 # minimum is to the right of mid
            else:
                right = mid # minimum is mid or to its left
        return nums[left]
```

PHASE 5 — TEST & DEBUG

```
# [3,4,5,1,2]: l=0,r=4,mid=2(5). 5>nums[4]=2 → l=3.
# l=3,r=4,mid=3(1). 1<=nums[4]=2 → r=3. l=r=3 → return nums[3]=1 ✓
# [4,5,6,7,0,1,2]: l=0,r=6,mid=3(7). 7>nums[6]=2 → l=4.
# l=4,r=6,mid=5(1). 1<=2 → r=5. l=4,r=5,mid=4(0). 0<=1 → r=4. l=r=4 → 0 ✓
# [11,13,15,17]: l=0,r=3,mid=1(13). 13<=17 → r=1. l=0,r=1,mid=0(11). 11<=13 → r=0. → 11 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log n). Space O(1).
# The comparison is always against nums[right] (not nums[left]).
# If nums[mid] > nums[right], the right portion must be 'lower' – minimum is there.
# If nums[mid] <= nums[right], the right portion is in order – minimum is at mid or left.
# Given more time I'd ask: what if there are duplicates?
# When nums[mid]==nums[right] we can't decide – fall back to right-- (O(n) worst case)."
```

◆ Quick Review

Key Insights

- The rotated sorted array consists of two sorted subarrays.
- A modified binary search can be used to locate the pivot point where the rotation occurs.
- By comparing the middle element to the rightmost element, we can decide which half of the array contains the minimum.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

Use a binary search approach. Initialize two pointers, left and right, at the start and end of the array. While left is less than right, calculate the middle index. If the middle element is greater than the right element, then the minimum is in the right half, so move the left pointer to $\text{mid} + 1$; otherwise, the minimum must be in the left half including mid, so adjust the right pointer to mid. When the loop terminates, left equals right and points to the minimum element. This approach leverages

the properties of the rotated sorted array to achieve logarithmic time complexity.

Binary Search

□ #300 Longest Increasing Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Example 1:

Input: `nums = [10,9,2,5,3,7,101,18]`

Output: 4

Explanation: The longest increasing subsequence is `[2,3,7,101]`, therefore the length is 4.

Example 2:

Input: `nums = [0,1,0,3,2,3]`

Output: 4

Example 3:

Input: `nums = [7,7,7,7,7,7,7]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 2500$
- $-104 \leq \text{nums}[i] \leq 104$

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the length of the longest strictly increasing subsequence (not necessarily
contiguous).
# Return the length, not the subsequence itself. Right?
# Follow-up asks for  $O(n \log n)$  – so know both approaches."
#
# "[10,9,2,5,3,7,101,18] → [2,3,7,101] length 4 ✓. [7,7,7,7] → 1 (strictly
increasing) ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# dp[i] = length of LIS ending at i. For each i, scan all j < i with nums[j] <
nums[i].
# dp[i] = max(dp[j]+1). Answer = max(dp).
# Correct  $O(n^2)$ ; patience sorting / binary search on tails gives  $O(n \log n)$ .
# Time:  $O(n^2)$ . Space:  $O(n)$  dp array.
# Should I go ahead and optimize?"
```

```
class _300_LIS_DP:
    def lengthOfLIS(self, nums: List[int]) -> int:
        dp = [1] * len(nums)
        for i in range(1, len(nums)):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)
        return max(dp)
```

PHASE 3 — OPTIMIZE ($O(n \log n)$ patience sorting)

```
# "Maintain a 'tails' array where tails[i] is the smallest tail of all LIS of length
i+1.
# For each number, binary search for the leftmost position in tails where tails[pos]
>= num.
# If found, replace tails[pos] with num (smaller tail = more room to extend).
# If not found, append (the LIS can be extended by 1).
# len(tails) at the end is the LIS length.
#
```

```
# This is patience sorting – tails is always sorted so binary search works."
```

PHASE 4 — CLEAN CODE

```
import bisect

class _300_LIS:
    def lengthOfLIS(self, nums: List[int]) -> int:
        tails = []
        for num in nums:
            pos = bisect.bisect_left(tails, num)
            if pos == len(tails):
                tails.append(num)
            else:
                tails[pos] = num
        return len(tails)
```

PHASE 5 — TEST & DEBUG

```
# nums=[10,9,2,5,3,7,101,18]:
# 10→tails=[10]. 9→replace 10→[9]. 2→replace 9→[2]. 5→append→[2,5].
# 3→replace 5→[2,3]. 7→append→[2,3,7]. 101→append→[2,3,7,101]. 18→replace
# 101→[2,3,7,18].
# len=4 ✓
# [7,7,7,7]: first 7→[7]. next 7→bisect_left([7],7)=0→replace→[7]. len=1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "O(n2) DP: Time O(n2), Space O(n). Clean and easy to explain.
# O(n log n) patience sort: Time O(n log n), Space O(n).
# Note: tails does NOT store the actual LIS – it's a virtual 'pile' structure.
# To reconstruct the actual LIS, track parent pointers in the DP version.
# Given more time I'd ask: longest non-decreasing subsequence?
# Change bisect_left to bisect_right – allows equal elements to extend the
sequence."
```

◆ Quick Review

Key Insights

- A dynamic programming solution with $O(n^2)$ time complexity exists, but we focus on an optimized approach.
- Using binary search and a dp array, we can achieve $O(n \log n)$ time.

- The dp array maintains the smallest possible tail for increasing subsequences of various lengths.
- For each number, perform binary search to decide if it can extend or replace a value in dp.

Space & Time

Time Complexity: $O(n \log n)$ where n is the length of nums

Space Complexity: $O(n)$ for storing the dp array

Solution

We use a dynamic programming approach combined with binary search. The idea is to maintain a dp array where $dp[i]$ holds the smallest tail value for any increasing subsequence with length $i+1$. For each number in the array, we perform a binary search on dp to find its correct insertion position: If the number is larger than all elements, we append it, effectively extending the current longest subsequence. Otherwise, we replace the first element in dp that is greater than or equal to the current number to keep the tail as small as possible. This process ensures that dp remains sorted and its length gives the length

of the longest increasing subsequence.

Binary Search

□ #362 Design Hit Counter

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Binary Search · Design · Queue

Lists: Grind 169

Problem

Design a hit counter which counts the number of hits received in the past 5 minutes (i.e., the past 300 seconds).

Your system should accept a timestamp parameter (in seconds granularity), and you may assume that calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing). Several hits may arrive roughly at the same time.

Implement the HitCounter class:

- **HitCounter()** Initializes the object of the hit counter system.
- **void hit(int timestamp)** Records a hit that happened at timestamp (in seconds). Several

hits may happen at the same timestamp.

- `int getHits(int timestamp)` Returns the number of hits in the past 5 minutes from timestamp (i.e., the past 300 seconds).

Example 1:

Input

```
["HitCounter", "hit", "hit", "hit", "getHits", "hit", "getHits", "getHits"]  
[[], [1], [2], [3], [4], [300], [300], [301]]
```

Output

```
[null, null, null, null, 3, null, 4, 3]
```

Explanation

```
HitCounter hitCounter = new HitCounter();
```

Constraints:

- $1 \leq \text{timestamp} \leq 2 * 10^9$
- All the calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing).
- At most 300 calls will be made to `hit` and `getHits`.

Follow up: What if the number of hits per second could be huge? Does your design scale?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY


```
# "Record hits at timestamps and query: how many hits in the past 300 seconds?
# Calls are chronologically ordered (timestamps only increase). Right?
# getHits(t) returns hits with timestamp in (t-300, t] inclusive – past 5 minutes."
#
# "hit(1),hit(2),hit(3): getHits(4)=3. hit(300): getHits(300)=4. getHits(301)=3
(hit@1 expired)."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Append every hit timestamp to a list.
# getHits(t): count timestamps in [t-299, t] by scanning the whole list.
# Simple, but each query is O(total hits) – queue with expiry is better.
# Time: O(1) hit, O(n) getHits per query. Space: O(n) all timestamps.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Store timestamps in a deque. On getHits, pop from the left any timestamp <=
t-300.
# Remaining deque size is the count. hit() is O(1). getHits() is amortised O(1).
#
# Follow-up (huge hits/second): store (timestamp, count) pairs instead of individual
timestamps.
# Multiple hits at the same second are collapsed into one deque entry."
```

PHASE 4 — CLEAN CODE

```
class _362_HitCounter:
    def __init__(self):
        self.hits: deque = deque() # stores individual hit timestamps
    def hit(self, timestamp: int) -> None:
        self.hits.append(timestamp)
    def getHits(self, timestamp: int) -> int:
        while self.hits and self.hits[0] <= timestamp - 300:
            self.hits.popleft()
        return len(self.hits)

# Follow-up: grouped version for high-throughput
class _362_HitCounter_Grouped:
    def __init__(self):
        self.hits: deque = deque() # stores (timestamp, count) pairs
    def hit(self, timestamp: int) -> None:
        if self.hits and self.hits[-1][0] == timestamp:
            ts, cnt = self.hits.pop()
            self.hits.append((ts, cnt + 1))
        else:
            self.hits.append((timestamp, 1))
    def getHits(self, timestamp: int) -> int:
        while self.hits and self.hits[0][0] <= timestamp - 300:
            self.hits.popleft()
        return sum(cnt for _, cnt in self.hits)
```

PHASE 5 — TEST & DEBUG

```
# hit(1),hit(2),hit(3),getHits(4): deque=[1,2,3]. all>4-300=-296 → len=3 ✓
# hit(300),getHits(300): deque=[1,2,3,300]. 1<=0? No. len=4 ✓
# getHits(301): 1<=301-300=1 → popleft. deque=[2,3,300]. len=3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "hit: O(1). getHits: O(k) amortised where k = expired entries (each entry popped at most once).
# Space: O(n) where n = total hits in the past 300 seconds.
# Binary search alternative: store sorted timestamps, binary search for t-300 cutoff - O(log n).
# The deque approach is simpler and amortised O(1) total.
# Grouped version handles millions of hits/second - key interview follow-up.
# Given more time I'd ask: what if calls aren't chronological?
# We'd need a sorted structure (sorted list or BST) and can't use the deque eviction trick."
```

◆ Quick Review

Key Insights

- Use a sliding window of 300 seconds to count hits.
- Since timestamps are monotonically increasing, outdated hits can be removed from the front.
- A queue (or circular array) is an efficient data structure to maintain and remove expired hit records.
- Grouping consecutive hits at the same timestamp can optimize space.

Space & Time

Time Complexity: $O(1)$ per hit and getHits operation on average, since each hit is

enqueued and later dequeued only once.

Space Complexity: $O(W)$ where W is the window size (300 seconds) in the worst case.

Solution

The solution uses a queue (or deque) data structure where each element stores a (timestamp, count) pair. When a hit is recorded, if the last element of the queue shares the same timestamp, then its count is incremented; otherwise, a new element is added. When `getHits(timestamp)` is called, the algorithm removes elements from the front of the queue whose timestamp is out of the 300-second window relative to the current timestamp, and then sums the counts of the remaining elements to return the result. This ensures that only relevant hits within the past 5 minutes are counted. The design scales well when hit frequency is high because hits at the same timestamp are grouped together instead of inserting individual records.

Binary Search

□ #528 Random Pick with Weight

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search · Prefix Sum ·
Randomized
Lists: Grind 169

Problem

You are given a 0-indexed array of positive integers w where $w[i]$ describes the weight of the i th index.

You need to implement the function `pickIndex()`, which randomly picks an index in the range $[0, w.length - 1]$ (inclusive) and returns it. The probability of picking an index i is $w[i] / \text{sum}(w)$.

- For example, if $w = [1, 3]$, the probability of picking index 0 is $1 / (1 + 3) = 0.25$ (i.e., 25%), and the probability of picking index 1 is $3 / (1 + 3) = 0.75$ (i.e., 75%).

Example 1:

Input

```
["Solution","pickIndex"]
[[[1]],[]]
```

Output

```
[null,0]
```

Explanation

```
Solution solution = new Solution([1]);
```

Example 2:

Input

```
["Solution","pickIndex","pickIndex","pickIndex","pickIndex","pickIndex"]
[[[1,3]],[],[],[],[],[]]
```

Output

```
[null,1,1,1,1,0]
```

Explanation

```
Solution solution = new Solution([1, 3]);
```

Constraints:

- $1 \leq w.length \leq 104$
- $1 \leq w[i] \leq 105$
- pickIndex will be called at most 104 times.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Pick a random index where index i is chosen with probability w[i]/sum(w).
```

```
# pickIndex() is called up to 10^4 times – so init can be O(n) but pick must be fast. Right?
```

```
# w=[1,3]: P(0)=25%, P(1)=75%. The actual randomness in the example output is fine."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
```

```
# Expand weights into a list where index i appears w[i] times, pick random index.
```

```
# Correct distribution but total list size can be sum(w) up to 1e9 – impossible.
```

```

# Prefix sums + binary search on total weight fixes space.
# Time:  $O(\sum(w))$  build. Space:  $O(\sum(w))$  – infeasible.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Build a prefix sum array. Prefix[i] = w[0]+...+w[i].
# Pick a random float r in [0, total). Binary search for the leftmost prefix sum >
r.
# That index is our answer – the probability of landing in bucket i equals
w[i]/total.
# Time:  $O(n)$  init.  $O(\log n)$  per pick. Space:  $O(n)$ ."

# PHASE 4 — CLEAN CODE
import random

class _528_RandomPickWeight:
    def __init__(self, w: List[int]):
        self.prefix = []
        total = 0
        for weight in w:
            total += weight
            self.prefix.append(total)
        self.total = total

    def pickIndex(self) -> int:
        r = random.random() * self.total
        lo, hi = 0, len(self.prefix) - 1
        while lo < hi:
            mid = (lo + hi) // 2
            if self.prefix[mid] <= r:
                lo = mid + 1
            else:
                hi = mid
        return lo

# PHASE 5 — TEST & DEBUG
# w=[1,3]: prefix=[1,4], total=4.
# r in [0,1) → prefix[0]=1>r → lo=hi=0 → return 0 (25% of the time).
# r in [1,4) → prefix[0]=1<=r → lo=1 → return 1 (75% of the time). ✓
# w=[1]: prefix=[1]. Any r in [0,1) → hi=lo=0 → return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Init  $O(n)$ . pickIndex  $O(\log n)$ . Space  $O(n)$ .
# The prefix sum maps weights to contiguous ranges on [0, total).
# A random point in that range lands in bucket i with probability w[i]/total.
# bisect.bisect_right(self.prefix, r) is a one-liner alternative using Python's
bisect module.
# Given more time I'd ask: what if weights change dynamically?
# We'd need a Fenwick tree (BIT) –  $O(\log n)$  updates AND  $O(\log n)$  queries."

```

◆ Quick Review

Key Insights

- Build a prefix sum array from the weights.
- Calculate the total sum of the weights.
- Generate a random number in the range [1, total sum] and use binary search to find the corresponding index in the prefix sum array.
- Using binary search ensures efficient selection in $O(\log n)$ time for each call of `pickIndex()`.

Space & Time

Time Complexity: $O(n)$ for constructing the prefix sum array, and $O(\log n)$ per `pickIndex()` call.

Space Complexity: $O(n)$ for storing the prefix sum array.

Solution

The solution leverages a prefix sum array where each entry at index i represents the cumulative sum of weights up to i . After computing the total weight, a random integer is generated between 1 and the total sum (inclusive). A binary search is then used to find

the first index where the prefix sum is greater than or equal to the random number. This approach ensures that the index is selected with a probability proportional to its weight.

Binary Search

□ #852 Peak Index in a Mountain Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search

Lists: Denny Zhang

Problem

You are given an integer mountain array `arr` of length `n` where the values increase to a peak element and then decrease.

Return the index of the peak element.

Your task is to solve it in $O(\log(n))$ time complexity.

Example 1:

Input: `arr = [0,1,0]`

Output: 1

Example 2:

Input: `arr = [0,2,1,0]`

Output: 1

Example 3:

Input: `arr = [0,10,5,2]`

Output: 1

Constraints:

- $3 \leq \text{arr.length} \leq 105$
- $0 \leq \text{arr}[i] \leq 106$
- arr is guaranteed to be a mountain array.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array guaranteed to have exactly one peak (mountain shape: strictly increasing then decreasing)."

Return the index of the peak in $O(\log n)$. Right?"

#

"[0,1,0] → 1. [0,2,1,0] → 1. [0,10,5,2] → 1. ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Linear scan from index 1: first i where $\text{arr}[i] > \text{arr}[i+1]$ is the peak index.

Mountain array guarantees exactly one peak; scan stops early.

$O(n)$ acceptable but binary search on the rising slope is $O(\log n)$.

Time: $O(n)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Binary search: at mid, compare $\text{arr}[\text{mid}]$ with $\text{arr}[\text{mid}+1]$."

If $\text{arr}[\text{mid}] < \text{arr}[\text{mid}+1]$: we're on the ascending slope → peak is to the right → $\text{left} = \text{mid}+1$.

Else: we're on or past the peak → peak is at mid or to the left → $\text{right} = \text{mid}$.

When $\text{left} == \text{right}$: that's the peak. $O(\log n)$."

PHASE 4 — CLEAN CODE

```
class _852_PeakIndex:
```

```
    def peakIndexInMountainArray(self, arr: List[int]) -> int:
```

```
        left, right = 0, len(arr) - 1
```

```
        while left < right:
```

```
            mid = (left + right) // 2
```

```
            if arr[mid] < arr[mid + 1]:
```

```
                left = mid + 1 # still ascending - peak is further right
```

```
            else:
```

```

        right = mid # descending or at peak – search left including mid
    return left

```

PHASE 5 — TEST & DEBUG

```

# [0,1,0]: l=0,r=2,mid=1. arr[1]=1>arr[2]=0 → right=1. l=0,r=1,mid=0.
arr[0]=0<arr[1]=1 → left=1. l=r=1 → 1 ✓
# [0,10,5,2]: l=0,r=3,mid=1. arr[1]=10>arr[2]=5 → right=1. l=0,r=1,mid=0.
arr[0]<arr[1] → left=1. → 1 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(log n). Space O(1).
# The comparison arr[mid] vs arr[mid+1] acts as a gradient – positive means 'go
right', negative means 'go left'.
# This is the same logic as Find Peak Element (#162) – the general version where the
array isn't guaranteed
# to be a full mountain. Mention the connection.
# Given more time I'd ask: what if there are multiple peaks?
# Same algorithm finds one peak. For all peaks, we'd need a linear scan."

```

◆ Quick Review

Key Insights

- The array is strictly increasing up to a peak and then strictly decreasing.
- A binary search technique is applicable because the array exhibits a single transition point (peak) between increasing and decreasing order.
- By comparing mid and mid+1, we can decide which half of the array to continue the search in.

Space & Time

Time Complexity: $O(\log(n))$ – The binary search approach reduces the search interval

by half at each step.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We use a binary search strategy: Initialize two pointers, left and right, at the beginning and end of the array. In each iteration, calculate mid. If $\text{arr}[\text{mid}]$ is less than $\text{arr}[\text{mid} + 1]$, then the peak is in the right half; otherwise, it is in the left half. When left equals right, we have found the peak index. This approach leverages the mountain array properties and ensures an $O(\log(n))$ solution.

Binary Search

□ #981 Time Based Key-Value Store

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Binary Search · Design
Lists: Grind 169

Problem

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the TimeMap class:

- **TimeMap()** Initializes the object of the data structure.
- **void set(String key, String value, int timestamp)** Stores the key key with the value value at the given time timestamp.
- **String get(String key, int timestamp)** Returns a value such that set was called previously, with $\text{timestamp_prev} \leq \text{timestamp}$. If there are multiple such values, it returns the value associated with the largest timestamp_prev. If

there are no values, it returns "".

Example 1:

Input

```
["TimeMap", "set", "get", "get", "set", "get", "get"]
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3], ["foo", "bar2", 4], ["foo", 4], ["foo", 5]]
```

Output

```
[null, null, "bar", "bar", null, "bar2", "bar2"]
```

Explanation

```
TimeMap timeMap = new TimeMap();
```

Constraints:

- $1 \leq \text{key.length}, \text{value.length} \leq 100$
- key and value consist of lowercase English letters and digits.
- $1 \leq \text{timestamp} \leq 10^7$
- All the timestamps timestamp of set are strictly increasing.
- At most $2 * 10^5$ calls will be made to set and get.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Key-value store where each key can have multiple values at different timestamps.

set(key, value, ts) stores the entry. get(key, ts) returns the value with the

largest timestamp $\leq$ ts. Return '' if none exists. Right?

set timestamps are strictly increasing – so we can binary search the history list."

```

#
# "set('foo','bar',1). get('foo',3) → 'bar' (no ts<=3 except 1). ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Store all (timestamp, value) pairs per key in a list on set.
# get(key, ts): scan backwards for largest timestamp <= ts.
# Works but each get is O(number of entries for that key).
# Time: O(1) set, O(k) get per key history. Space: O(total entries).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Since set timestamps are strictly increasing, each key's list is already sorted
by timestamp.
# get() binary searches for the rightmost timestamp <= query timestamp.
# set: O(1). get: O(log n). Space: O(total entries)."
```

```

# PHASE 4 — CLEAN CODE
class _981_TimeMap:
    def __init__(self):
        self.store: defaultdict = defaultdict(list) # key → [(ts, value), ...]

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        entries = self.store[key]
        lo, hi = 0, len(entries) - 1
        result = ''
        while lo <= hi:
            mid = (lo + hi) // 2
            if entries[mid][0] <= timestamp:
                result = entries[mid][1] # valid candidate – try to go right for a
larger ts
            lo = mid + 1
        else:
            hi = mid - 1
        return result

# PHASE 5 — TEST & DEBUG
# set('foo','bar',1): store['foo']=[(1,'bar')].
# get('foo',1): entries=[(1,'bar')]. mid=0, ts=1<=1 → result='bar', lo=1. lo>hi →
return 'bar' ✓
# get('foo',3): mid=0, ts=1<=3 → result='bar'. lo=1>hi=0 → return 'bar' ✓
# set('foo','bar2',4): store['foo']=[(1,'bar'),(4,'bar2')].
# get('foo',4): mid=0(1<=4)→result='bar',lo=1. mid=1(4<=4)→result='bar2',lo=2.
return 'bar2' ✓
# get('foo',5): same → 'bar2' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "set O(1). get O(log n). Space O(total entries)."
```

```
# The 'find rightmost ts <= query' is a standard upper_bound - 1 binary search pattern.  
# Timestamps strictly increasing → no sorting needed – the list maintains order automatically.  
# bisect.bisect_right alternative: pos = bisect.bisect_right(entries, (timestamp, chr(127))) - 1  
# (appending chr(127) to go past any value at that exact timestamp).  
# Given more time I'd ask: what if set timestamps aren't guaranteed to be increasing?  
# We'd need to sort the list on insert or use a sorted data structure."
```

◆ Quick Review

Key Insights

- Use a dictionary (hash table) to map each key to a list of (timestamp, value) pairs.
- Timestamps are strictly increasing when setting the values, which allows efficient appending.
- For getting the value, perform a binary search on the list of timestamps to find the largest timestamp that is $\leq$ the queried timestamp.
- If no valid timestamp exists, return an empty string.

Space & Time

Time Complexity:

set operation: $O(1)$ per call (amortized).

get operation: $O(\log N)$ per call, where N is the number of timestamps stored for the key.

Space Complexity: $O(\text{total number of set calls})$, to store all (timestamp, value) pairs.

Solution

We use a hash table to store keys mapping to a list of their timestamps and corresponding values. Since the timestamps are inserted in strictly increasing order, the list remains sorted. For the get operation, a binary search is applied to quickly locate the appropriate value for a given timestamp. This approach minimizes the lookup time and efficiently handles the problem constraints.

Binary Search

□ #1011 Capacity to Ship Packages Within D Days

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

A conveyor belt has packages that must be shipped from one port to another within days days.

The i th package on the conveyor belt has a weight of $\text{weights}[i]$. Each day, we load the ship with packages on the conveyor belt (in the order given by weights). We may not load more weight than the maximum weight capacity of the ship.

Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within days days.

Example 1:

Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5
 Output: 15
 Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:
 1st day: 1, 2, 3, 4, 5
 2nd day: 6, 7
 3rd day: 8
 4th day: 9
 5th day: 10

Example 2:

Input: weights = [3,2,2,4,1,4], days = 3
 Output: 6
 Explanation: A ship capacity of 6 is the minimum to ship all the packages in 3 days like this:
 1st day: 3, 2
 2nd day: 2, 4
 3rd day: 1, 4

Example 3:

Input: weights = [1,2,3,1,1], days = 4
 Output: 3
 Explanation:
 1st day: 1
 2nd day: 2
 3rd day: 3
 4th day: 1, 1

Constraints:

- $1 \leq \text{days} \leq \text{weights.length} \leq 5 * 10^4$
- $1 \leq \text{weights}[i] \leq 500$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the minimum ship capacity that lets us ship all packages in at most D days."

```

# Packages must be shipped in order – we can't reorder them. Right?
# Capacity must be at least max(weights) (to handle any single package).
# Capacity at most sum(weights) (ship everything in one day).
#
# "weights=[1..10], days=5 → 15 ✓ (greedy daily assignment: 1-5, 6-7, 8, 9, 10)"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Try every candidate ship capacity from max(weights) up to sum(weights).
# For each capacity, greedily pack days and count days needed.
# Correct but linear search on capacity range is O(sum * n).
# Time: O(sum(weights) * n). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search on the answer (capacity).
# The feasibility function 'can_ship(capacity)' is monotone: if C works, C+1 also
works.
# So binary search for the leftmost capacity where can_ship returns True.
# can_ship: greedily fill each day until the load would exceed capacity, then start
a new day.
# Count days needed. Return days <= D.
# Time: O(n log(sum(weights))). Space: O(1)."
```

```

# PHASE 4 — CLEAN CODE
class _1011_CapacityShip:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def can_ship(capacity: int) -> bool:
            day_load = 0
            days_used = 1
            for w in weights:
                if day_load + w > capacity:
                    days_used += 1
                    day_load = 0
                day_load += w
            return days_used <= days
        lo, hi = max(weights), sum(weights)
        while lo < hi:
            mid = (lo + hi) // 2
            if can_ship(mid):
                hi = mid # valid – try smaller
            else:
                lo = mid + 1 # not valid – need more capacity
        return lo

# PHASE 5 — TEST & DEBUG
# weights=[1..10], days=5. lo=10, hi=55.
# mid=32: can_ship(32)→1+2+3+4+5=15≤32(day1), 6+7=13≤32(day2), 8≤32(day3),
9≤32(day4), 10≤32(day5). 5 days ✓ → hi=32.
# ... binary search converges to 15.
```

```
# can_ship(15): 1+2+3+4+5=15(day1), 6+7=13+8>15→day2=6+7, 8→day3, 9→day4, 10→day5.
5 days ✓. → 15 ✓
# can_ship(14): 1+2+3+4=10, +5=15>14→day2=5+6=11, +7>14→day3=7+8>14→day4=8, 9→day5,
10→day6. 6>5 ✗.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log(\text{sum}))$. Space $O(1)$.

This is 'binary search on the answer' – a powerful pattern where the answer space is

a range of integers and feasibility is monotone.

The feasibility check is a greedy simulation – always load greedily to avoid underusing capacity.

Same pattern appears in: Koko Eating Bananas (#875), Split Array Largest Sum

 (#410).

Given more time I'd ask: what if packages can be reordered?

Then we'd try to optimise placement – different problem, potentially NP-hard."

◆ Quick Review

Key Insights

- The minimal possible capacity is the maximum value in weights; no ship can carry a package heavier than its capacity.
- The maximum possible capacity is the sum of all weights; this allows shipping all packages in one day.
- The problem is monotonic: if a capacity C works, any capacity greater than C will also work.
- Use binary search on the capacity range and a greedy approach to simulate shipping for each candidate capacity.

Space & Time

Time Complexity: $O(n * \log(\text{sum}(\text{weights})))$
where n is the number of packages.

Space Complexity: $O(1)$ additional space.

Solution

The solution employs binary search to find the minimum ship capacity. Start by setting the search bounds: the lower bound is the maximum weight (since each package must be shipped individually if needed) and the upper bound is the sum of all weights (shipping everything in one go). For each midpoint capacity in the binary search, use a greedy method to simulate the shipping process—accumulating weights until adding another package would exceed the candidate ship capacity, at which point you increment the day count. If the number of days required exceeds D , the capacity is insufficient and the lower bound is increased; otherwise, adjust the upper bound. Continue until the bounds converge.

Binary Search

★ #1060 Missing Element in Sorted Array **MED**

★

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search

Lists: ★ Premium | Premium 98

Problem

Given an integer array `nums` which is sorted in ascending order and all of its elements are unique and given also an integer `k`, return the `k`th missing number starting from the leftmost number of the array.

Example 1:

Input: `nums = [4,7,9,10]`, `k = 1`
Output: 5
Explanation: The first missing number is 5.

Example 2:

Input: `nums = [4,7,9,10]`, `k = 3`
Output: 8
Explanation: The missing numbers are `[5,6,8,...]`, hence the third missing number is 8.

Example 3:

Input: nums = [1,2,4], k = 3

Output: 6

Explanation: The missing numbers are [3,5,6,7,...], hence the third missing number is 6.

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $1 \leq \text{nums}[i] \leq 10^7$
- nums is sorted in ascending order, and all the elements are unique.
- $1 \leq k \leq 10^8$

Follow up: Can you find a logarithmic time complexity (i.e., $O(\log(n))$) solution?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sorted unique integers. Find the k-th missing integer starting from nums[0].
# Missing count at index i = (nums[i] - nums[0]) - i (how many numbers should exist
but don't).
# Follow-up asks for  $O(\log n)$ . Right?"
#
# "[4,7,9,10], k=1: missing = [5,6,8,...]. 1st missing = 5 ✓
# [4,7,9,10], k=3: [5,6,8,...]. 3rd = 8 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Walk nums in order maintaining expected next integer starting at 0.
# Increment expected whenever nums[i] == expected; else expected stays.
# When expected reaches k, return k - correct but scans whole array.
# Time:  $O(n)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

```
class _1060_MissingElement_Brute:
```



```

def missingElement(self, nums: List[int], k: int) -> int:
    for i in range(1, len(nums)):
        gap = nums[i] - nums[i - 1] - 1 # missing numbers between nums[i-1] and
nums[i]
        if k <= gap:
            return nums[i - 1] + k
        k -= gap
    return nums[-1] + k # k-th missing is after the array

# PHASE 3 — OPTIMIZE (O(log n))
# "Define missing(i) = numbers missing from nums[0] to nums[i] = (nums[i] - nums[0])
- i.
# Binary search for the rightmost index where missing(i) < k.
# Answer = nums[right] + (k - missing(right)).
# If k > missing(last), answer is past the end of the array: nums[-1] + (k -
missing(n-1))."

# PHASE 4 — CLEAN CODE
class _1060_MissingElement:
    def missingElement(self, nums: List[int], k: int) -> int:
        def missing(i: int) -> int:
            return (nums[i] - nums[0]) - i

        n = len(nums)
        if k > missing(n - 1):
            return nums[-1] + (k - missing(n - 1))

        lo, hi = 0, n - 1
        while lo < hi:
            mid = (lo + hi) // 2
            if missing(mid) < k:
                lo = mid + 1 # not enough missing yet - go right
            else:
                hi = mid # too many or just enough - go left
        return nums[lo - 1] + (k - missing(lo - 1))

# PHASE 5 — TEST & DEBUG
# nums=[4,7,9,10], k=1:
# missing: [0]=0, [1]=2, [2]=3, [3]=4.
# k=1<=missing(3)=4. Binary search: lo=0,hi=3,mid=1. missing(1)=2>=1→hi=1.
# lo=0,hi=1,mid=0. missing(0)=0<1→lo=1. lo=hi=1.
# return nums[0]+(1-missing(0))=4+(1-0)=5 ✓
# nums=[4,7,9,10], k=3:
# missing(3)=4>=3. Binary search → lo lands at 2. missing(1)=2<3→lo=2. lo=hi=2.
# return nums[1]+(3-missing(1))=7+(3-2)=8 ✓
# k=5: missing(3)=4<5 → return nums[3]+(5-4)=10+1=11 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(log n) with binary search. O(1) space.
# The missing(i) formula is the key: (nums[i]-nums[0])-i counts how many integers
# in [nums[0], nums[i]] are absent from the array.
# The past-the-end check handles k > total missing numbers in the array gracefully.

```

```
# Given more time I'd ask: what if there are duplicate values in nums?  
# missing(i) would overcount – we'd need to adjust for duplicates."
```

◆ Quick Review

Key Insights

- The array is sorted and every element is unique.
- For any index i , the number of missing numbers up to that point can be computed as $\text{nums}[i] - \text{nums}[0] - i$.
- When $\text{missing}(i)$ is less than k , the k th missing element is further to the right.
- Use binary search to find the smallest index where the count of missing numbers is greater than or equal to k .
- If all missing numbers before the last array element are counted and the k th missing still hasn't been reached, calculate the result by continuing past the last element.

Space & Time

Time Complexity: $O(\log(n))$

Space Complexity: $O(1)$

Solution

We compute the number of missing elements until an index i with the formula: $\text{missing}(i) = \text{nums}[i] - \text{nums}[0] - i$. We then use binary search over the indices to find the smallest index where $\text{missing}(i) \geq k$. If the binary search index equals the length of the array, it means the k th missing number lies beyond the last element. Otherwise, the k th missing number is derived as: $\text{nums}[\text{index} - 1] + (k - \text{missing}(\text{index} - 1))$. This approach uses binary search for logarithmic time and does not require extra auxiliary data structures.

Binary Search

□ #1062 Longest Repeating Substring

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Binary Search · Dynamic Programming
· Rolling Hash · Suffix Array · Hash Function
Lists: Denny Zhang

Problem

Given a string *s*, return the length of the longest repeating substrings. If no repeating substring exists, return 0.

Example 1:

Input: *s* = "abcd"
Output: 0
Explanation: There is no repeating substring.

Example 2:

Input: *s* = "abbaba"
Output: 2
Explanation: The longest repeating substrings are "ab" and "ba", each of which occurs twice.

Example 3:

Input: *s* = "aabcaabdaab"
Output: 3
Explanation: The longest repeating substring is "aab", which occurs 3 times.

Constraints:

- $1 \leq s.length \leq 2000$
- s consists of lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the length of the longest substring that appears at least twice in s.
# The two occurrences can overlap. Return 0 if no repeating substring. Right?"
#
# "'abcd'→0. 'abbaba'→2 ('ab','ba'). 'aabcaabdaab'→3 ('aab')."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each length L from n-1 down to 1, collect all substrings of length L in a set.
# First L where a duplicate appears is the answer.
#  $O(n^2)$  substrings per L – binary search on L with rolling hash helps.
# Time:  $O(n^2)$  to  $O(n^3)$  naive. Space:  $O(n^2)$  set.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (DP approach — simpler to explain)

```
# "DP:  $dp[i][j]$  = length of longest common suffix of  $s[:i]$  and  $s[:j]$ , where  $i \neq j$ .
#  $dp[i][j] = dp[i-1][j-1] + 1$  if  $s[i-1] == s[j-1]$  and  $i \neq j$ , else 0.
# Track the global max.  $O(n^2)$  time,  $O(n^2)$  space."
```

PHASE 4 — CLEAN CODE

```
class _1062_LongestRepeatingSubstring:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        dp = [[0] * (n + 1) for _ in range(n + 1)]
        best = 0
        for i in range(1, n + 1):
            for j in range(i + 1, n + 1): # j > i ensures different starting
positions
                if s[i - 1] == s[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1] + 1
                    best = max(best, dp[i][j])
        return best
```

PHASE 5 — TEST & DEBUG

```
# s='abbaba':
# i=1,j=2: 'a'=='b'? No. ... i=1,j=4: 'a'=='a'? Yes→dp[1][4]=1, best=1.
```

```
# i=2,j=5: 'b'=='b'? Yes→dp[2][5]=dp[1][4]+1=2, best=2.
# → 2 ✓ (substring 'ab' starting at 0 and 3)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^2)$ . Space  $O(n^2)$ . Acceptable for  $n \leq 2000$  (4M cells).
```

```
# Binary search + hashing alternative:  $O(n \log n)$  time,  $O(n)$  space – faster but more complex.
```

```
# Given more time I'd ask: return the actual substring, not just the length?
```

```
# Track (i,j) when best is updated, then s[i-best:i]."
```

◆ Quick Review

Key Insights

- Use binary search to efficiently check for the existence of a repeating substring of a given length.
- Apply a rolling hash technique to compute hash values for substrings in $O(1)$ time after initial processing.
- Use a hash set to detect duplicate hash values (indicating a repeating substring).
- Adjust binary search bounds based on whether a repeating substring of the current length exists.
- Alternative approaches include dynamic programming and suffix arrays, but the binary search with rolling hash is efficient and conceptually clean.

Space & Time

Time Complexity: $O(n \log n)$ on average, where n is the length of the string. (Each binary search step takes $O(n)$ time for computing rolling hashes.)

Space Complexity: $O(n)$ for storing hash values in the hash set.

Solution

The solution uses binary search to decide on the candidate length L of the repeating substring. For each candidate length L , we use a rolling hash to compute hash values for all substrings of length L and store them in a hash set. If any hash is repeated, it confirms the presence of a repeating substring of length L , and we try a longer length.

Otherwise, we decrease the candidate length. The rolling hash is computed using a base (26, for lowercase letters) and a modulus (2^{32} to limit integer overflow) which allows efficient hash recomputation for sliding windows. This approach efficiently narrows down the maximum repeating substring length.

Binary Search

□ ★ #1533 Find the Index of the Large Integer ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Interactive
Lists: ★ Premium | Premium 98

Problem

We have an integer array `arr`, where all the integers in `arr` are equal except for one integer which is larger than the rest of the integers. You will not be given direct access to the array, instead, you will have an API `ArrayReader` which have the following functions:

- `int compareSub(int l, int r, int x, int y)`: where $0 \leq l, r, x, y < \text{ArrayReader.length}()$, $l \leq r$ and $x \leq y$. The function compares the sum of sub-array `arr[l..r]` with the sum of the sub-array `arr[x..y]` and returns: 1 if $\text{arr}[l] + \text{arr}[l+1] + \dots + \text{arr}[r] > \text{arr}[x] + \text{arr}[x+1] + \dots + \text{arr}[y]$.
- 0 if $\text{arr}[l] + \text{arr}[l+1] + \dots + \text{arr}[r] == \text{arr}[x] + \text{arr}[x+1] + \dots + \text{arr}[y]$.

- -1 if $\text{arr}[l] + \text{arr}[l+1] + \dots + \text{arr}[r] < \text{arr}[x] + \text{arr}[x+1] + \dots + \text{arr}[y]$.

You are allowed to call `compareSub()` 20 times at most. You can assume both functions work in $O(1)$ time.

Return the index of the array `arr` which has the largest integer.

Example 1:

Input: `arr = [7,7,7,7,10,7,7,7]`

Output: 4

Explanation: The following calls to the API

`reader.compareSub(0, 0, 1, 1)` // returns 0 this is a query comparing the sub-array (0, 0) with the sub array (1, 1), (i.e. compares `arr[0]` with `arr[1]`). Thus we know that `arr[0]` and `arr[1]` doesn't contain the largest element.

`reader.compareSub(2, 2, 3, 3)` // returns 0, we can exclude `arr[2]` and `arr[3]`.

`reader.compareSub(4, 4, 5, 5)` // returns 1, thus for sure `arr[4]` is the largest element in the array.

Notice that we made only 3 calls, so the answer is valid.

Example 2:

Input: `nums = [6,6,12]`

Output: 2

Constraints:

- $2 \leq \text{arr.length} \leq 5 * 10^5$
- $1 \leq \text{arr}[i] \leq 100$
- All elements of `arr` are equal except for one element which is larger than all other elements.

Follow up:

- What if there are two numbers in arr that are bigger than all other numbers?
- What if there is one number that is bigger than other numbers and one number that is smaller than other numbers?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "One element is strictly larger than all others. Find its index using the
compareSub API.
# compareSub(l,r,x,y) compares sum(arr[l..r]) with sum(arr[x..y]): returns 1,-1,0.
# At most 20 calls – so O(log n) calls are required. Right?"
#
# "[7,7,7,7,10,7,7,7]: 3 compareSub calls suffice to find index 4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Linear scan: compare nums[i] with nums[i+1]; first unequal pair reveals larger
half.
# Correct O(n) but problem expects O(log n) via binary search on the doubled array.
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Binary search: at each step, compare the left half against the right half of the
range.
# If left > right: large integer is in the left half → search left.
# If right > left: search right.
# If equal: the large integer is the MIDDLE element (if odd-length range, middle was
excluded).
# Halve the range each step → O(log n) calls."
```

PHASE 4 — CLEAN CODE

```
class _1533_FindLargeInteger:
    def getIndex(self, reader) -> int:
        lo, hi = 0, reader.length() - 1
        while lo < hi:
```

```

        mid = (lo + hi) // 2
        # Compare left half [lo..mid] with right half of equal size
        left_end = mid
        right_start = mid + 1
        right_end = right_start + (mid - lo)
        if right_end > hi:
            right_end = hi
        result = reader.compareSub(lo, left_end, right_start, right_end)
        if result == 1:
            hi = left_end # large int is in left half
        elif result == -1:
            lo = right_start # large int is in right half
        else:
            return mid + 1 # equal sums → large int is the middle skipped
    element
    return lo

# PHASE 5 — TEST & DEBUG
# arr=[7,7,7,7,10,7,7,7]: lo=0,hi=7.
# mid=3. Compare [0..3] vs [4..7]. Both have 4 elements. Sum=[28] vs [31]. -1 →
lo=4.
# lo=4,hi=7,mid=5. Compare [4..5] vs [6..7]. [10+7=17] vs [7+7=14]. 1 → hi=5.
# lo=4,hi=5,mid=4. Compare [4..4] vs [5..5]. [10] vs [7]. 1 → hi=4. lo=hi=4 → return
4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(log n) compareSub calls. Well within the 20-call limit for n=5*10^5
(log2(500000)≈19).
# The equal-sum case (odd-length range) means the middle element was the large one –
# both halves have equal elements so the excluded middle is the outlier.
# Follow-up: two large numbers? One comparison no longer definitively points to one
half.
# We'd need to track which half has MORE excess – more complex."

```

◆ Quick Review

Key Insights

- Use binary search to efficiently narrow down the position of the unique large value.
- At each step, divide the candidate range into two halves (or almost two halves when the range size is odd) and use compareSub to

determine which half contains the large integer.

- Comparing two subarrays of equal length will reveal the region that contains the extra value via the sum difference.
- Handle odd-length ranges carefully by isolating the middle element if needed.

Space & Time

Time Complexity: $O(\log n)$ – because binary search is performed over the range.

Space Complexity: $O(1)$ – only a few pointers are maintained, no additional space is required.

Solution

The solution applies a binary search approach over the array indices. At every iteration: Calculate the current segment length. If the length is even, split the segment evenly and compare the left half versus the right half using `compareSub`. If the length is odd, compare two halves of equal size while ignoring the middle element. If the sums are equal, then the middle index holds the large value. Update the search bounds based on the

result of compareSub. Continue until the search space is reduced to one index, which is the answer. This method ensures that you locate the large integer with $O(\log n)$ calls to compareSub, well within the limit of 20 calls.

Matrix

Round 2 · High · Medium · 14 questions

Matrix

□ #36 Valid Sudoku

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix
Lists: Grind 169

Problem

Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:

1. Each row must contain the digits 1–9 without repetition.
2. Each column must contain the digits 1–9 without repetition.
3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1–9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Example 1:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board =

```
[["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[","9","8",".",".",".","6","."],
["8",".",".","6",".",".","3"],
["4",".",".","8",".","3",".","1"],
["7",".",".","2",".",".","6"],
[","6",".",".","2","8","."],
["4","1","9",".","5"],
["8","7","9"]]
```

Example 2:


```

Input: board =
[["8","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[[".","9","8",".",".",".","6","."],
["8",".",".","6",".",".","3"],
["4",".","8",".","3",".","1"],
["7",".",".","2",".",".","6"],
[[".","6",".",".",".","2","8","."]]

```

Constraints:

- `board.length == 9`
- `board[i].length == 9`
- `board[i][j]` is a digit 1–9 or '.'.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Check if a partially filled 9x9 sudoku board is valid – no repeats in any row,
# column, or 3x3 box. Ignore empty cells ('.'). Right?
# A valid board doesn't have to be solvable – just no current conflicts."
#
# "The two examples differ only in one cell: '8' appears twice in the top-left 3x3
box → false ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# For each of 9 rows, 9 columns, and 9 boxes, collect digits in a set and check
size==9.
# 27 independent validity checks – straightforward and easy to explain.
# Time: O(1) – fixed 81 cells. Space: O(1) – 9 sets of size 9.
# Should I go ahead and optimize?"

```

PHASE 3 — OPTIMIZE

```

# "Single pass: use three 2D boolean arrays – rows[r][d], cols[c][d],
boxes[r//3][c//3][d].
# For each filled cell, check all three. If any is already marked, return False.
# Otherwise mark all three. O(81) = O(1) for fixed 9x9."

```

PHASE 4 — CLEAN CODE

```
class _36_ValidSudoku:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        rows = [[False] * 9 for _ in range(9)]
        cols = [[False] * 9 for _ in range(9)]
        boxes = [[False] * 9 for _ in range(9)] # indexed by box_id = (r//3)*3 +
c//3

        for r in range(9):
            for c in range(9):
                if board[r][c] == '.':
                    continue
                d = int(board[r][c]) - 1
                box_id = (r // 3) * 3 + (c // 3)
                if rows[r][d] or cols[c][d] or boxes[box_id][d]:
                    return False
                rows[r][d] = cols[c][d] = boxes[box_id][d] = True
        return True
```

PHASE 5 — TEST & DEBUG

```
# Cell (0,0)='5': d=4. box_id=0. Mark rows[0][4]=cols[0][4]=boxes[0][4]=True.
# Cell (3,0)='8': d=7. box_id=0. rows[3][7]? No. boxes[0][7]? No. Mark.
# In example 2: cell (0,0)='8' and cell (3,0)='8': both have box_id=0, d=7.
# Second '8' at (3,0): boxes[0][7] already True -> return False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time 0(81)=0(1). Space 0(81)=0(1) - fixed 9x9 board.
# The box_id formula (r//3)*3+(c//3) maps each of the 9 boxes to 0-8. Know this
cold.
# Alternative: use a set of tuples like (r,'row',d), (c,'col',d), (box_id,'box',d) -
one set.
# Given more time I'd ask: implement a Sudoku solver?
# That's backtracking - for each empty cell, try 1-9, recurse, backtrack on
failure."
```

◆ Quick Review

Key Insights

- Validate each row by checking for duplicate numbers while ignoring empty cells.
- Validate each column in the same way.
- Divide the board into nine 3x3 sub-boxes and check each for duplicate numbers.

- Use hash sets (or equivalent data structures) to keep track of seen digits for rows, columns, and boxes.
- The board size is fixed (9x9), so the overall time complexity is constant with respect to input size.

Space & Time

Time Complexity: $O(1)$ — Since the board size is fixed at 9x9.

Space Complexity: $O(1)$ — The extra space used by the sets is bounded by the fixed board size.

Solution

The solution uses three main data structures: an array of sets for rows, an array of sets for columns, and an array of sets for 3x3 sub-boxes. For each cell, if it is not empty, check if the number is already present in the corresponding row, column, or box. If found, return false immediately. Otherwise, add the number to the respective sets. Compute the index for the 3x3 sub-box using the formula: $(\text{row_index} // 3) * 3 + (\text{col_index} // 3)$. If no duplicates are found after traversing the entire

board, the board is considered valid.

Matrix

□ #48 Rotate Image

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Matrix

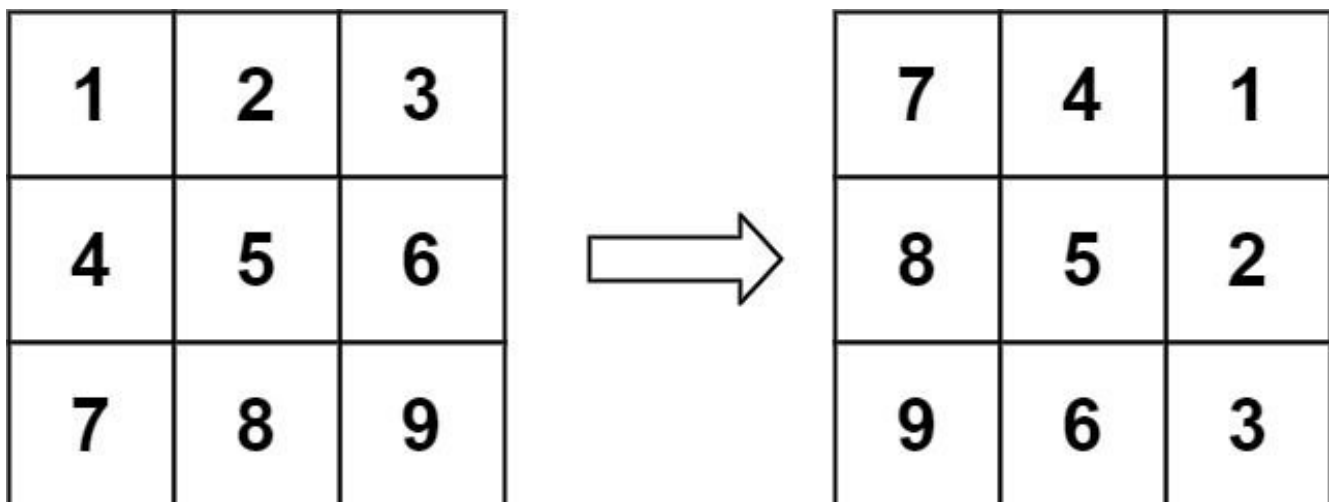
Lists: Grind 169 | CodeSignal

Problem

You are given an $n \times n$ 2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

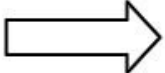
Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
 Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:

5	1	9	11
2	4	8	10
13	3	6	7
15	14	12	16



15	13	2	5
14	3	4	1
12	6	8	9
16	7	10	11

Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
 Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 20$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rotate an $n \times n$ matrix 90 degrees clockwise in-place. No extra matrix allowed. Right?"

After rotation: element at (r,c) moves to $(c, n-1-r)$."

#

"[[1,2,3],[4,5,6],[7,8,9]] → [[7,4,1],[8,5,2],[9,6,3]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Allocate a new $n \times n$ matrix; for each (r,c) set `new[c][n-1-r] = matrix[r][c]`.

Classic 90-degree rotation formula – clear and correct.

Uses $O(n^2)$ extra memory; we can rotate in-place layer by layer.

Time: $O(n^2)$. Space: $O(n^2)$ new matrix.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (two-step in-place)

"Two operations, each reversible and composable:

Step 1: Transpose (swap `matrix[r][c]` with `matrix[c][r]`).

Step 2: Reverse each row.

Together these produce a 90° clockwise rotation. $O(n^2)$ time, $O(1)$ space.

Verify: $(r,c) \rightarrow \text{transpose} \rightarrow (c,r) \rightarrow \text{reverse row} \rightarrow (c, n-1-r)$. Correct!"

PHASE 4 — CLEAN CODE

```

class _48_RotateImage:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        # Step 1: transpose
        for r in range(n):
            for c in range(r + 1, n):
                matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
        # Step 2: reverse each row
        for row in matrix:
            row.reverse()

```

PHASE 5 — TEST & DEBUG

`[[1,2,3],[4,5,6],[7,8,9]]`:

Transpose: `[[1,4,7],[2,5,8],[3,6,9]]`.

Reverse rows: `[[7,4,1],[8,5,2],[9,6,3]]` ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(1)$ in-place."

Counter-clockwise 90°: reverse each row first, then transpose.

180°: transpose then transpose (just reverse all rows, then reverse their order).

The two-step approach is elegant – each step is simple and reversible.

Given more time I'd ask: rotate a non-square matrix?

No longer possible in-place with the same dimensions – need a new $m \times n$ matrix."

◆ Quick Review

Key Insights

- The rotation can be achieved in two steps: first transpose the matrix and then reverse

each row.

- Transposing the matrix swaps $\text{matrix}[i][j]$ with $\text{matrix}[j][i]$, converting rows into columns.
- Reversing each row post-transposition yields the desired clockwise rotation.
- This approach works in-place with $O(1)$ extra space.

Space & Time

Time Complexity: $O(n^2)$ since every element is processed during the transpose and row reversal.

Space Complexity: $O(1)$ as the operations are performed in-place without extra data structures.

Solution

The solution leverages a two-step in-place transformation: Transpose the matrix: Iterate through the matrix and swap each element (i, j) with element (j, i) for $j \geq i$. Reverse each row: After the transpose, each row is reversed to rotate the matrix 90 degrees clockwise. The key trick is to realize that a transpose followed by a reverse of each row achieves the rotation without needing extra space.

Matrix

□ #54 Spiral Matrix

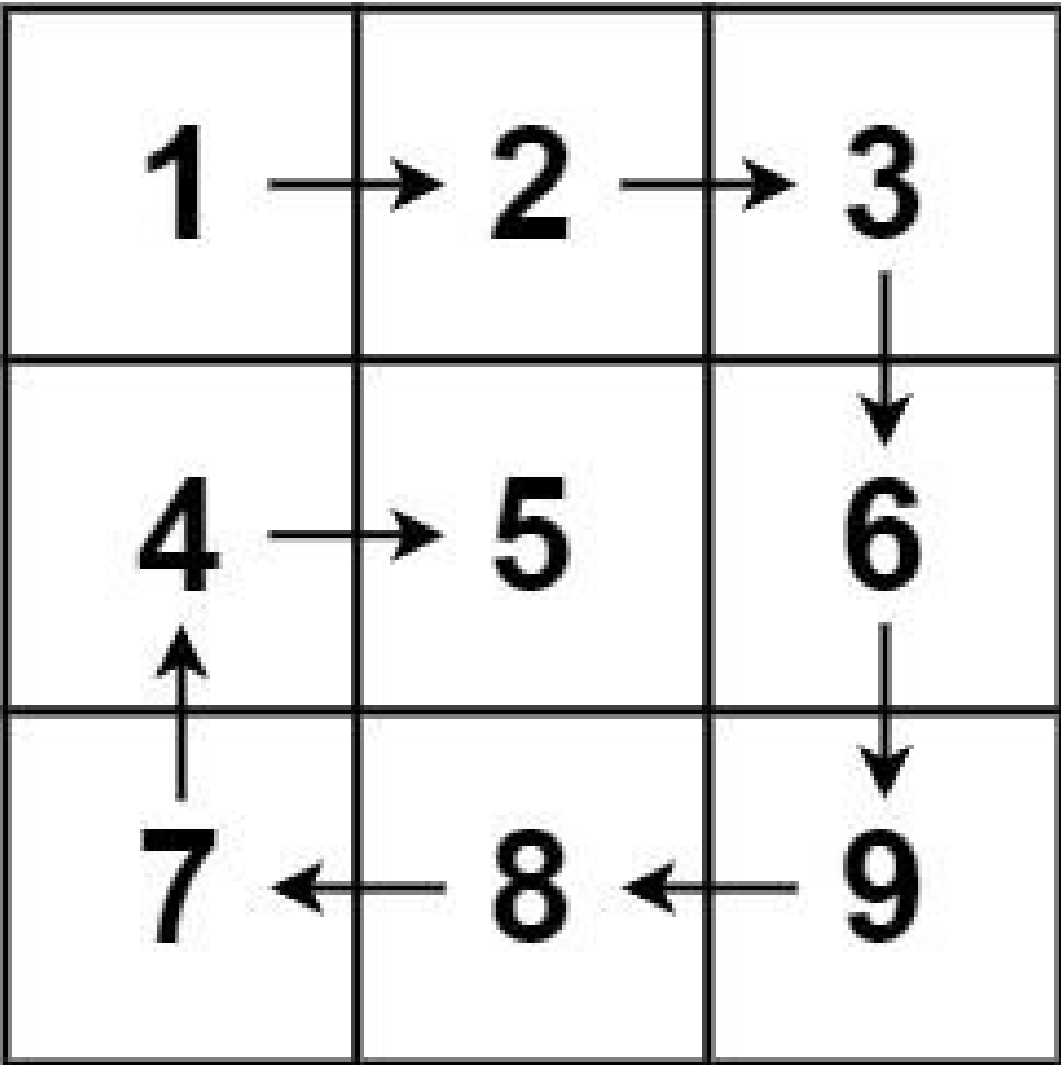
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: Grind 169 | CodeSignal

Problem

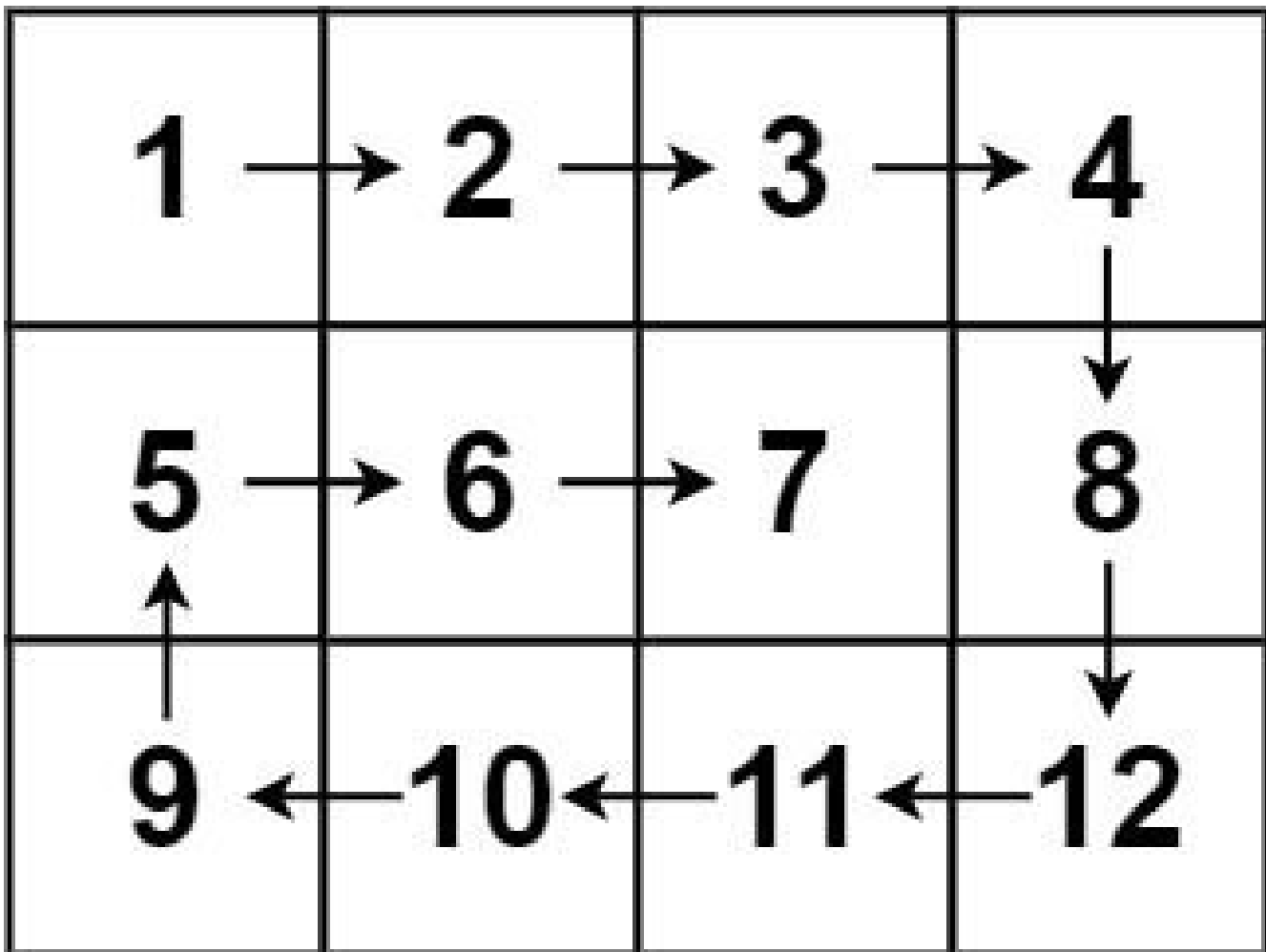
Given an $m \times n$ matrix, return all elements of the matrix in spiral order.

Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]

Example 2:



Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]

Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $-100 \leq \text{matrix}[i][j] \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return all matrix elements in spiral order: right along top, down along right
side,
# left along bottom, up along left side – then repeat for the inner rectangle.
Right?
# Works for non-square matrices."
#
# "[[1,2,3],[4,5,6],[7,8,9]] → [1,2,3,6,9,8,7,4,5] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Maintain top/bottom/left/right bounds; walk right, down, left, up while unvisited.
# Mark visited cells in a boolean matrix to know when to turn.
# Works, but the visited matrix costs O(m*n) extra space.
# Time: O(m * n). Space: O(m * n) visited.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (layer boundaries)

```
# "Track four boundaries: top, bottom, left, right.
# Traverse right along top (then top++), down along right (then right--),
# left along bottom (then bottom--), up along left (then left++).
# Stop when top>bottom or left>right. O(m*n) time, O(1) extra space."
```

PHASE 4 — CLEAN CODE

```
class _54_SpiralMatrix:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        top, bottom = 0, len(matrix) - 1
        left, right = 0, len(matrix[0]) - 1
        result = []
        while top <= bottom and left <= right:
            for c in range(left, right + 1): result.append(matrix[top][c])
            top += 1
            for r in range(top, bottom + 1): result.append(matrix[r][right])
            right -= 1
            if top <= bottom:
                for c in range(right, left - 1, -1):
                    result.append(matrix[bottom][c])
                    bottom -= 1
            if left <= right:
                for r in range(bottom, top - 1, -1): result.append(matrix[r][left])
                left += 1
        return result
```

PHASE 5 — TEST & DEBUG

```
# [[1,2,3],[4,5,6],[7,8,9]]: top=0,bot=2,l=0,r=2.
# Right: 1,2,3. top=1.
# Down: 6,9. right=1.
```

```
# Left (top<=bot): 8,7. bot=1.
# Up (left<=right): 4. left=1.
# top=1,bot=1,l=1,r=1.
# Right: 5. top=2. top>bot → stop.
# → [1,2,3,6,9,8,7,4,5] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(1)$ extra.

The boundary guards (if top<=bottom / if left<=right) prevent double-counting

the single middle row or column of non-square matrices.

Given more time I'd ask: Spiral Matrix II (#59) – fill instead of read.

Same boundary logic, write numbers $1..n^2$ instead of appending."

◆ Quick Review

Key Insights

- Use four boundaries: top, bottom, left, and right to track the current layer.
- Traverse right across the top row, then down the right column, then left on the bottom row, and finally up the left column.
- After each direction, adjust the corresponding boundary to move inward.
- Continue the traversal until the boundaries cross.

Space & Time

Time Complexity: $O(m * n)$ since each element is visited exactly once.

Space Complexity: $O(1)$ extra space (excluding the space required for the output list).

Solution

The solution uses a simulation approach with boundary pointers (top, bottom, left, right). At each step, the boundaries indicate the current submatrix to traverse. You iterate as follows: Traverse from left to right along the top boundary. Traverse downward along the right boundary. If the top boundary has not passed the bottom, traverse from right to left along the bottom boundary. If the left boundary has not passed the right, traverse upward along the left boundary. After each traversal, the boundaries are adjusted inward. This continues until all elements have been visited.

Matrix

#59 Spiral Matrix II

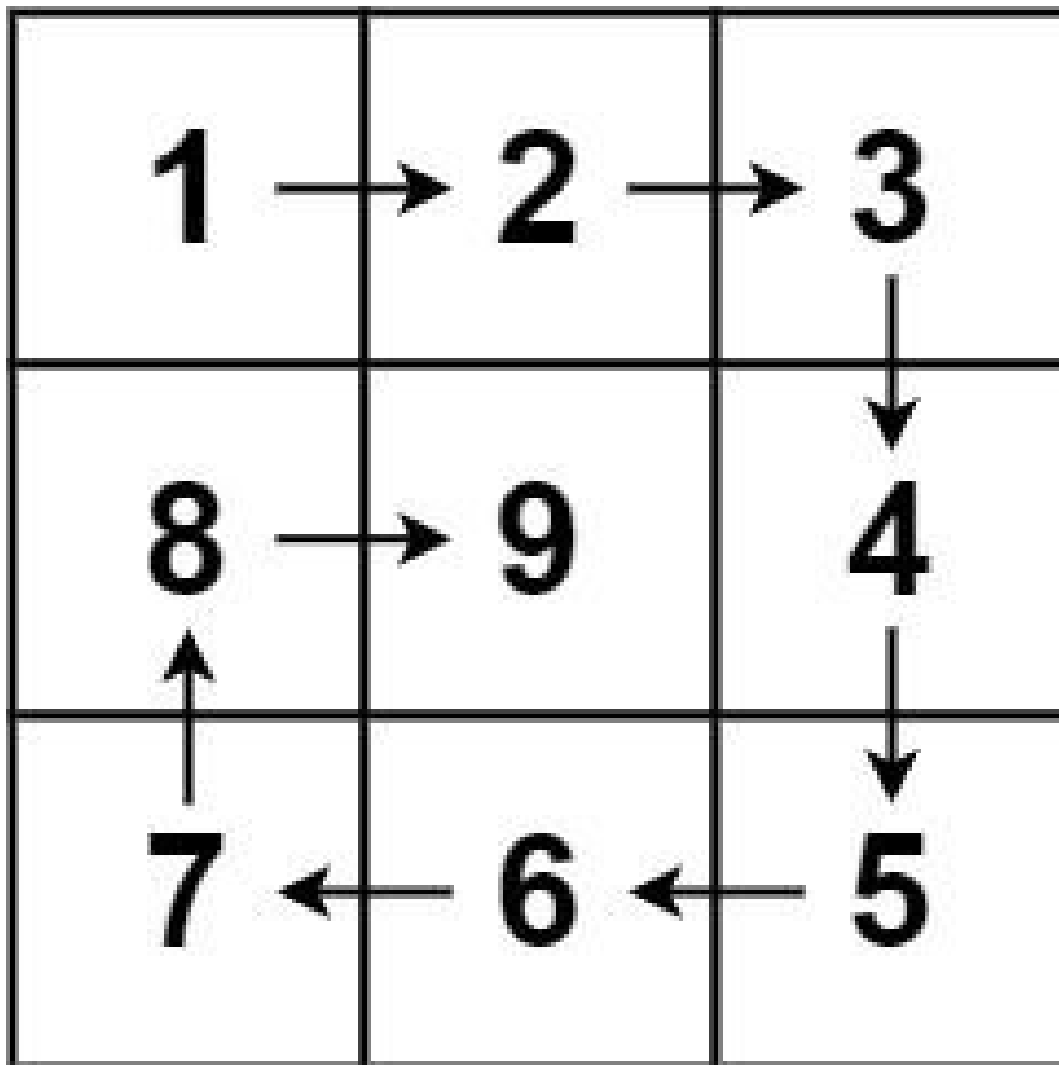
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: CodeSignal

Problem

Given a positive integer n , generate an $n \times n$ matrix filled with elements from 1 to n^2 in spiral order.

Example 1:



Input: $n = 3$

Output: $[[1,2,3],[8,9,4],[7,6,5]]$

Example 2:

Input: $n = 1$

Output: $[[1]]$

Constraints:

- $1 \leq n \leq 20$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Generate an $n \times n$ matrix filled with 1 to n^2 in spiral order. Return the matrix. Right?"

#

"n=3 → $\begin{bmatrix} 1 & 2 & 3 \\ 8 & 9 & 4 \\ 7 & 6 & 5 \end{bmatrix}$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Fill numbers $1..n^2$ using direction arrays (right, down, left, up) and a visited grid.

Turn when hitting boundary or a filled cell.

Straightforward simulation with $O(n^2)$ extra visited space.

Time: $O(n^2)$. Space: $O(n^2)$ visited.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Identical boundary approach to #54, but write consecutive integers instead of reading.

Initialize $n \times n$ matrix of zeros, fill with $\text{num}=1..n^2$. $O(n^2)$ time, $O(1)$ extra."

PHASE 4 — CLEAN CODE

class _59_SpiralMatrixII:

def generateMatrix(self, n: int) -> List[List[int]]:

matrix = [[0] * n for _ in range(n)]

top, bottom, left, right = 0, n - 1, 0, n - 1

num = 1

while top <= bottom and left <= right:

for c in range(left, right + 1): matrix[top][c] = num; num += 1

top += 1

for r in range(top, bottom + 1): matrix[r][right] = num; num += 1

right -= 1

if top <= bottom:

for c in range(right, left - 1, -1): matrix[bottom][c] = num; num +=

1

bottom -= 1

if left <= right:

for r in range(bottom, top - 1, -1): matrix[r][left] = num; num += 1

left += 1

return matrix

PHASE 5 — TEST & DEBUG

n=3: fills 1→2→3 (top row), 4 (right col), 5→6 (bottom), 7 (left col), 9 (center).

→ $\begin{bmatrix} 1 & 2 & 3 \\ 8 & 9 & 4 \\ 7 & 6 & 5 \end{bmatrix}$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(1)$ extra – output matrix doesn't count.

This is the write-version of Spiral Matrix (#54) – exact same boundary logic.

```
# Given more time I'd ask: generate a spiral starting from the center?  
# Reverse the boundary logic – start from the middle, expand outward."
```

◆ Quick Review

Key Insights

- Use four pointers (top, bottom, left, right) to keep track of the matrix boundaries.
- Traverse the matrix in a spiral: move right along the top row, then down the right column, left along the bottom row, and finally up the left column.
- Update the boundary pointers after each complete traversal of one side.
- Continue until all numbers from 1 to n^2 are filled into the matrix.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$ (required for the output matrix)

Solution

We simulate the spiral movement by initializing four pointers: top, bottom, left, and right, which denote the current boundaries of the matrix that have not yet been filled. For

each iteration, we: Traverse from left to right along the top boundary, then increment the top pointer. Traverse from top to bottom along the right boundary, then decrement the right pointer. If there are rows remaining, traverse from right to left along the bottom boundary, then decrement the bottom pointer. If there are columns remaining, traverse from bottom to top along the left boundary, then increment the left pointer. This algorithm ensures that the matrix is filled in a spiral order until all numbers are placed.

Matrix

□ #64 Minimum Path Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: Denny Zhang

Problem

Given a $m \times n$ grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:

1	3	1
1	5	1
4	2	1

Input: grid = [[1,3,1],[1,5,1],[4,2,1]]

Output: 7

Explanation: Because the path 1 → 3 → 1 → 1 → 1 minimizes the sum.

Example 2:

Input: grid = [[1,2,3],[4,5,6]]

Output: 12

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 200$

- $0 \leq \text{grid}[i][j] \leq 200$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Move only right or down. Find the path from top-left to bottom-right with minimum sum. Right?"
# Can only move right or down – no backtracking."
#
# "[[1,3,1],[1,5,1],[4,2,1]]: path 1→3→1→1→1=7 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
# DFS from (0,0): at each cell try moving right and down, take min of two paths.
# Memoize (r,c) to avoid recomputing subpaths – classic top-down DP.
# Without memo this blows up exponentially; memo makes it O(m*n).
# Time: O(m * n) with memo. Space: O(m * n) memo + stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (in-place DP)

```
# "dp[i][j] = min path sum to reach (i,j)."
# dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]).
# First row: only right. First col: only down.
# Modify grid in-place – no extra space. O(m*n) time, O(1) extra."
```

PHASE 4 — CLEAN CODE

```
class _64_MinPathSum:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        for r in range(m):
            for c in range(n):
                if r == 0 and c == 0:
                    continue
                elif r == 0:
                    grid[r][c] += grid[r][c - 1] # can only come from left
                elif c == 0:
                    grid[r][c] += grid[r - 1][c] # can only come from above
                else:
                    grid[r][c] += min(grid[r - 1][c], grid[r][c - 1])
        return grid[m - 1][n - 1]
```

PHASE 5 — TEST & DEBUG

```
# [[1,3,1],[1,5,1],[4,2,1]]:
# Row 0: [1,4,5]. Col 0: [1,2,6]. (1,1)=5+min(4,2)=7. (1,2)=1+min(7,5)=6.
# (2,1)=2+min(7,6)=8. (2,2)=1+min(6,8)=7. → 7 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \times n)$. Space $O(1)$ – modifies grid in place.

If we can't modify the input, use a 1D rolling array: $O(n)$ extra space.

Given more time I'd ask: what if we can also move up or left?

That breaks the DP recurrence – shortest path with general movement is Dijkstra's."

◆ Quick Review

Key Insights

- The problem can be solved using Dynamic Programming.
- The optimal solution for each cell depends on the minimum path sum of the cell above it and the cell to its left.
- Since you can only move right or down, only two adjacent cells affect the current cell's result.
- Edge cases include the first row and first column, which have only one direction of movement.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the number of rows and columns.

Space Complexity: $O(m * n)$ for the DP table (this can be optimized to $O(n)$ with in-place

computations).

Solution

We use a Dynamic Programming (DP) approach where we create a DP table (or use the grid itself) that stores the minimum path sum to each cell. For each cell (i, j) : If $i = 0$ (first row), the only possible path is from the left. If $j = 0$ (first column), the only possible path is from above. For other cells, the minimum path sum is the cell value plus the minimum of the two possible previous cells (above and left). This ensures that by the time we reach the bottom-right cell, we have accumulated the minimum sum required to reach that cell.

Matrix

□ #73 Set Matrix Zeroes

MED

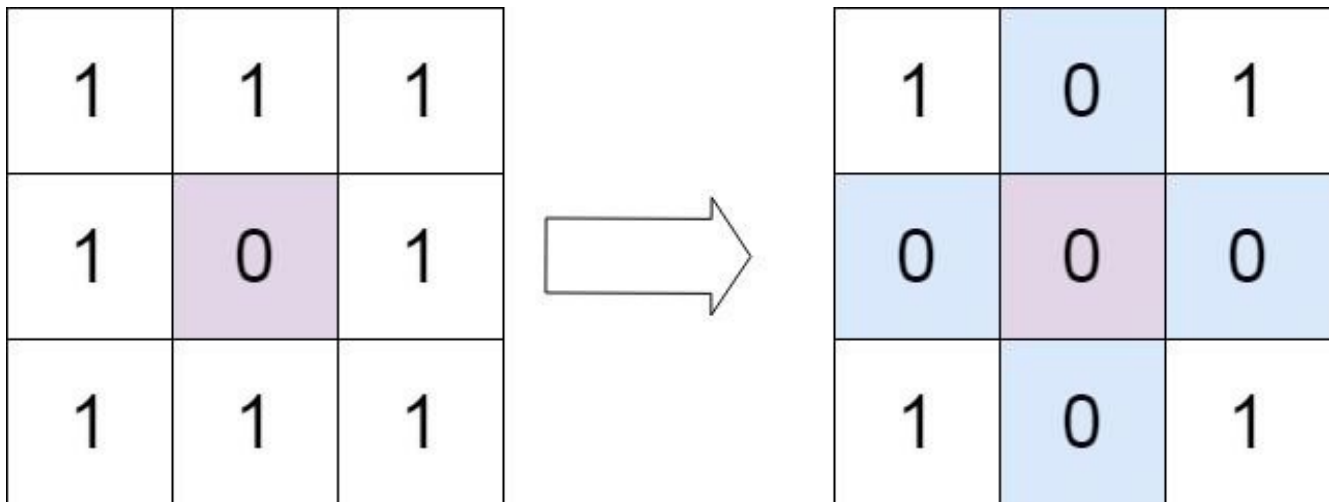
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Matrix
 Lists: Grind 169

Problem

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it in place.

Example 1:



Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`
 Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Example 2:

0	1	2	0	→	0	0	0	0
3	4	5	2		0	4	5	0
1	3	1	5		0	3	1	0

Input: matrix = `[[0,1,2,0],[3,4,5,2],[1,3,1,5]]`
 Output: `[[0,0,0,0],[0,4,5,0],[0,3,1,0]]`

Constraints:

- `m == matrix.length`
- `n == matrix[0].length`
- `1 <= m, n <= 200`
- `-231 <= matrix[i][j] <= 231 - 1`

Follow up:

- A straightforward solution using $O(mn)$ space is probably a bad idea.
- A simple improvement uses $O(m + n)$ space, but still not the best solution.
- Could you devise a constant space solution?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "If a cell is 0, zero out its entire row and column. Do it in-place. Right?
# We must use the ORIGINAL zeros – not zeros we set during the process."
#
# "[[1,0,1],[1,1,1],[1,1,1]] → [[0,0,0],[1,0,1],[1,0,1]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# First collect every (r,c) where matrix[r][c]==0 into a list.
# Second pass: zero out entire rows and columns for each stored zero.
# Simple, but storing all zero coordinates uses O(m+n) extra structures.
# Time: O(m * n). Space: O(m + n) for row/col sets.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE O(1) space using first row/col as markers

```
# "Use the first row and first column of the matrix itself as markers.
# If matrix[r][c]==0: mark matrix[r][0]=0 and matrix[0][c]=0.
# Handle first row and first column separately (track with boolean flags).
# Then zero out interior cells using the markers.
# Then zero out first row/col based on flags. O(m*n) time, O(1) space."
```

PHASE 4 — CLEAN CODE

```
class _73_SetMatrixZeroes:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        first_row_zero = any(matrix[0][c] == 0 for c in range(n))
        first_col_zero = any(matrix[r][0] == 0 for r in range(m))
        for r in range(1, m): # mark zeros using first row/col
            for c in range(1, n):
                if matrix[r][c] == 0:
                    matrix[r][0] = 0
                    matrix[0][c] = 0
        for r in range(1, m): # zero out interior
            for c in range(1, n):
                if matrix[r][0] == 0 or matrix[0][c] == 0:
                    matrix[r][c] = 0
        if first_row_zero: # zero out first row if needed
            for c in range(n): matrix[0][c] = 0
        if first_col_zero: # zero out first col if needed
            for r in range(m): matrix[r][0] = 0
```

PHASE 5 — TEST & DEBUG

```
# [[0,1,2,0],[3,4,5,2],[1,3,1,5]]:
# first_row_zero: matrix[0][0]=0 → True. first_col_zero: matrix[0][0]=0 → True.
# Mark: r=1,c=0: already 3≠0. r=1,c=3: 2≠0... only (0,0) and (0,3) have zeros in row 0.
# Interior zeros: marker[0][0]=0→col 0 zeros. marker[0][3]: 0→col 3 zeros.
# → [[0,0,0,0],[0,4,5,0],[0,3,1,0]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(1)."
```

```
# The first row/col flags are the key – without them, using matrix[0][0] as a marker
# for both first row and first col creates an ambiguity. Two flags resolve it
cleanly.
# The four-step order (flag, mark, zero interior, zero edges) is the correct
sequence.
# Given more time I'd ask: what if we should set a cell to the average of its
row/col instead?
# Same structure – two-pass approach to avoid using updated values."
```

◆ Quick Review

Key Insights

- The challenge is to mark rows and columns that need to be zeroed without using extra space.
- A common trick is to use the first row and first column as markers.
- Special care is needed for the first row and first column, as they are also used as markers.
- The algorithm involves two passes: one for marking and one for updating the matrix based on the markers.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(1)$ (in-place, using only constant extra space)

Solution

We can solve this problem by using the first row and first column of the matrix as markers. First, determine if the first row or first column needs to be zeroed, then traverse the rest of the matrix and mark the corresponding first row and column positions if a zero is encountered. Finally, update the matrix backward using the markers, and finally update the first row and column if needed. The algorithm works in three main steps: Check if the first row or first column should be zeroed (store this in two variables). Iterate through the rest of the matrix; for any element that is zero, mark its row and column by setting the corresponding element in the first row and first column to zero. Iterate through the matrix again (excluding the first row and column) and set elements to zero if their row or column marker is zero. Finally, zero out the first row and/or first column if needed. This approach avoids extra space usage apart from the two boolean flags while ensuring that the matrix is updated correctly.

Matrix

□ #74 Search a 2D Matrix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Matrix
Lists: Grind 169

Problem

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` if `target` is in `matrix` or `false` otherwise.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true

Example 2:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13

Output: false

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $-104 \leq \text{matrix}[i][j], \text{target} \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Matrix where each row is sorted and the first element of each row is greater than
# the last of the previous row — so the whole matrix is one sorted sequence.
# Find target in  $O(\log(m*n))$ . Right?"
#
# "Treat it as a 1D array of length  $m*n$ . Index  $k$  maps to  $row=k//n$ ,  $col=k\%n$ ."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Treat the matrix as a flattened sorted array and binary search on index  $0..m*n-1$ .
# Or scan row by row with binary search per row.
# Correct  $O(m \log n)$ , but a single binary search on the virtual array is cleaner.
# Time:  $O(m \log n)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Binary search on the virtual 1D array  $[0, m*n-1]$ .
# Map  $mid$  to  $(mid//n, mid\%n)$  to access the actual cell.
#  $O(\log(m*n)) = O(\log m + \log n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _74_Search2DMatrix:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left <= right:
            mid = (left + right) // 2
            mid_val = matrix[mid // n][mid % n]
            if mid_val == target:
                return True
            elif mid_val < target:
                left = mid + 1
            else:
                right = mid - 1
        return False
```

PHASE 5 — TEST & DEBUG

```
# matrix=[[1,3,5,7],[10,11,16,20],[23,30,34,60]], target=3:
# l=0,r=11. mid=5→matrix[1][1]=11. 11>3→r=4.
# mid=2→matrix[0][2]=5. 5>3→r=1.
# mid=0→matrix[0][0]=1. 1<3→l=1.
# mid=1→matrix[0][1]=3. 3==3→True ✓
# target=13: binary search narrows until l>r → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log(m*n))$ . Space  $O(1)$ .
# The index mapping  $mid//n$  and  $mid\%n$  is the entire trick.
# This only works because the matrix is fully sorted as one sequence.
```

```
# If rows are sorted but first-of-next isn't > last-of-prev (Search a 2D Matrix II #240):  
# Use staircase search – start at top-right, move left if too big, down if too small.  
# O(m+n) time – different problem, different approach."
```

◆ Quick Review

Key Insights

- The matrix is essentially a flattened sorted array due to its properties.
- Use binary search to efficiently determine if the target exists.
- Map the 1D binary search index to the corresponding 2D matrix coordinates using division and modulus operations.
- Achieve $O(\log(m*n))$ time complexity by navigating the matrix as if it were a single sorted list.

Space & Time

Time Complexity: $O(\log(m * n))$

Space Complexity: $O(1)$

Solution

We can treat the matrix as a flattened sorted list without physically copying the elements. Given the total number of elements = $m * n$, perform binary search on indices ranging from

0 to $(m \cdot n - 1)$. For any index mid , calculate its corresponding row as $mid // n$ and column as $mid \% n$. Compare the element at $matrix[row][col]$ with the target. If it equals target, return true; if it is less than target, search the right half; otherwise, search the left half.

Matrix

□ #221 Maximal Square

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: Grind 169

Problem

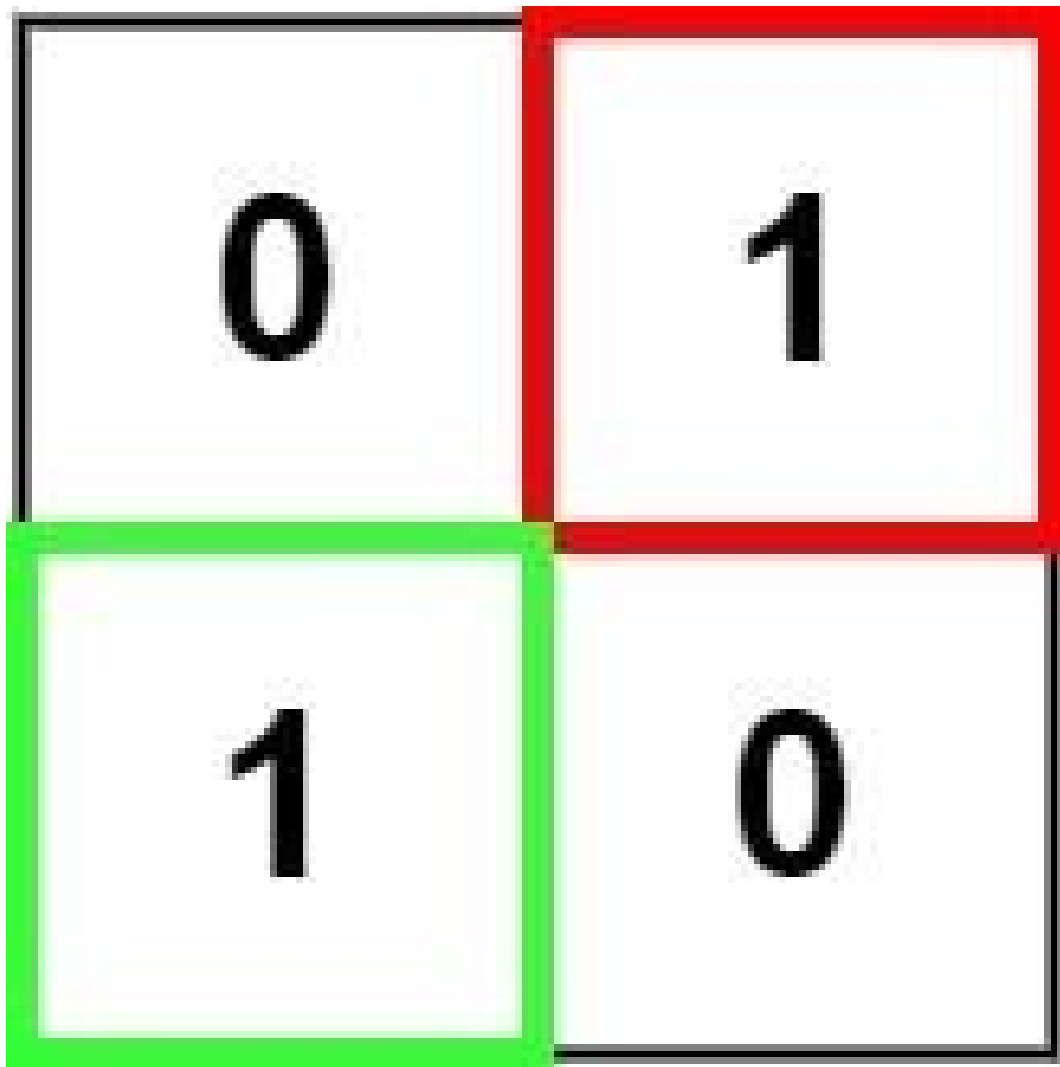
Given an $m \times n$ binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = `[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]`
Output: 4

Example 2:



Input: matrix = `[["0","1"],["1","0"]]`
Output: 1

Example 3:

Input: matrix = `[["0"]]`
Output: 0

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 300`
- `matrix[i][j]` is '0' or '1'.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the largest square of all 1s. Return its area. Matrix contains '0'/'1' strings. Right?"

#

"[[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1],[1,0,0,1,0]] → 2×2=4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

For every cell containing '1', expand square side length while all cells in square are '1'.

Track the maximum side length found across all starting cells.

Each expansion check is $O(k)$ for side k – overall $O(m \cdot n \cdot \min(m, n)^2)$.

Time: $O(m \cdot n \cdot \min(m, n)^2)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (DP)

"dp[i][j] = side length of the largest all-1s square with bottom-right corner at (i,j)."

If matrix[i][j]=='0': dp[i][j]=0.

Else: dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1.

Intuition: a square of side k can be extended by 1 only if the top, left, and diagonal neighbour

each have a square of side k . The bottleneck is the minimum.

Track max side length → return side²."

PHASE 4 — CLEAN CODE

```
class _221_MaximalSquare:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])
        dp = [[0] * n for _ in range(m)]
        max_side = 0
        for r in range(m):
            for c in range(n):
                if matrix[r][c] == '1':
                    top = dp[r - 1][c] if r > 0 else 0
                    left = dp[r][c - 1] if c > 0 else 0
                    diag = dp[r - 1][c - 1] if r > 0 and c > 0 else 0
                    dp[r][c] = min(top, left, diag) + 1
                    max_side = max(max_side, dp[r][c])
        return max_side * max_side
```

PHASE 5 — TEST & DEBUG

"[[0,1],['1','0']]: no 2x2 all-1s → dp stays at 1s and 0s → max=1 → area=1 ✓
 # Full example: bottom-right region gives dp[2][3]=dp[2][4]=2, max=2, area=4 ✓"

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ – reducible to $O(n)$ with a rolling array.
 # The min of three neighbours is the core insight – think of it as the square being
 # limited by the weakest neighbouring corner.
 # Given more time I'd ask: maximal rectangle of 1s (not just square)?
 # That's Maximal Rectangle (#85) – use a histogram approach per row, $O(m*n)$."

◆ Quick Review

Key Insights

- Use dynamic programming to build a solution where each cell in the DP table represents the side length of the largest square ending at that cell.
- If the current cell is '1', its DP value is the minimum of the top, left, and top-left neighbors plus one.
- Update a global maximum side length during the process to determine the area.
- The square's area is the square of its maximum side length.

Space & Time

Time Complexity: $O(m * n)$, where m is the number of rows and n is the number of columns.

Space Complexity: $O(m * n)$ for the DP table (this can be optimized to $O(n)$ with space reduction techniques).

Solution

We use a two-dimensional dynamic programming approach. We define a DP matrix with one extra row and column to simplify boundary conditions. For each cell (i, j) in the input matrix, if the value is '1', then we calculate the DP value as the minimum of its top, left, and top-left neighboring DP values plus one. This value represents the side length of the square ending at (i, j) . We also keep track of the maximum side length found and finally compute the area of the maximal square by squaring the maximum side length.

Matrix

★ #311 Sparse Matrix Multiplication ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix

Lists: ★ Premium | Premium 98

Problem

Given two sparse matrices `mat1` of size `m x k` and `mat2` of size `k x n`, return the result of `mat1 x mat2`. You may assume that multiplication is always possible.

Example 1:

$$\begin{bmatrix} 1 & 0 & 0 \\ -1 & 0 & 3 \end{bmatrix} \times \begin{bmatrix} 7 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 0 & 0 \\ -7 & 0 & 3 \end{bmatrix}$$

Input: `mat1 = [[1,0,0],[-1,0,3]]`, `mat2 = [[7,0,0],[0,0,0],[0,0,1]]`
Output: `[[7,0,0],[-7,0,3]]`

Example 2:

Input: `mat1 = [[0]]`, `mat2 = [[0]]`
Output: `[[0]]`

Constraints:

- $m == \text{mat1.length}$
- $k == \text{mat1}[i].\text{length} == \text{mat2.length}$
- $n == \text{mat2}[i].\text{length}$
- $1 \leq m, n, k \leq 100$
- $-100 \leq \text{mat1}[i][j], \text{mat2}[i][j] \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Multiply mat1 (m×k) by mat2 (k×n). Return the m×n result.
# Both are sparse – many zeros. Optimise to skip zero elements. Right?"
#
# "[[1,0,0],[-1,0,3]] × [[7,0,0],[0,0,0],[0,0,1]] = [[7,0,0],[-7,0,3]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Standard triple loop: for each row i and col k, sum A[i][j]*B[j][k] over j.
# Correct even for sparse inputs but touches every (i,j,k) combination.
# Skipping zero rows/cols from sparse representation saves huge work.
# Time: O(m * k * n). Space: O(m * n) result.
# Should I go ahead and optimize?"
```

```
class _311_SparseMultiply_Brute:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[List[int]]:
        m, k, n = len(mat1), len(mat1[0]), len(mat2[0])
        result = [[0] * n for _ in range(m)]
        for i in range(m):
            for p in range(k):
                for j in range(n):
                    result[i][j] += mat1[i][p] * mat2[p][j]
        return result
```

PHASE 3 — OPTIMIZE

```
# "Skip multiplication when mat1[i][p] == 0 – avoids the inner j loop entirely."
```

```
# Pre-compute non-zero positions in mat1 for even faster access.
# This is the key sparse optimisation: skip zero elements before accessing mat2."
```

PHASE 4 — CLEAN CODE

```
class _311_SparseMultiply:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[List[int]]:
        m, k, n = len(mat1), len(mat1[0]), len(mat2[0])
        result = [[0] * n for _ in range(m)]
        for i in range(m):
            for p in range(k):
                if mat1[i][p] == 0:
                    continue # skip - avoids entire inner loop
                for j in range(n):
                    result[i][j] += mat1[i][p] * mat2[p][j]
        return result
```

PHASE 5 — TEST & DEBUG

```
# mat1[0][0]=1, p=0: add 1*mat2[0][j] to result[0]. mat1[0][1]=0 → skip.
mat1[0][2]=0 → skip.
# result[0]=[7,0,0] ✓. mat1[1][2]=3, p=2: add 3*mat2[2][j]=[0,0,3] →
result[1]=[-7,0,3] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Worst case  $O(m*k*n)$  same as brute force. Best case  $O(nnz1 * n)$  where  $nnz1$  =
non-zeros in mat1.
# For truly sparse matrices this is much faster.
# Given more time I'd ask: what if both matrices are very large and very sparse?
# Use CSR (Compressed Sparse Row) format - store only non-zero values and their
indices.
# Hash map approach: for each non-zero in mat1, look up non-zeros in the
corresponding row of mat2."
```

◆ Quick Review

Key Insights

- Exploit sparsity by iterating only over non-zero elements.
- For each non-zero element in mat1, multiply it with corresponding non-zero elements in mat2.

- Accumulate the product into the result matrix.
- This approach reduces unnecessary multiplication operations when many elements are zero.

Space & Time

Time Complexity: $O(m * k * n)$ in the worst-case, but significantly lower when many elements are zero.

Space Complexity: $O(m * n)$ for the resulting matrix.

Solution

We loop through each row of `mat1` and for every non-zero element, we traverse the corresponding row in `mat2`. By checking if an element in both matrices is non-zero before multiplying, we avoid redundant calculations. This optimization leverages the sparsity of the matrices. We use standard nested loops along with conditional checks to implement this efficient multiplication.

Matrix

□ #498 Diagonal Traverse

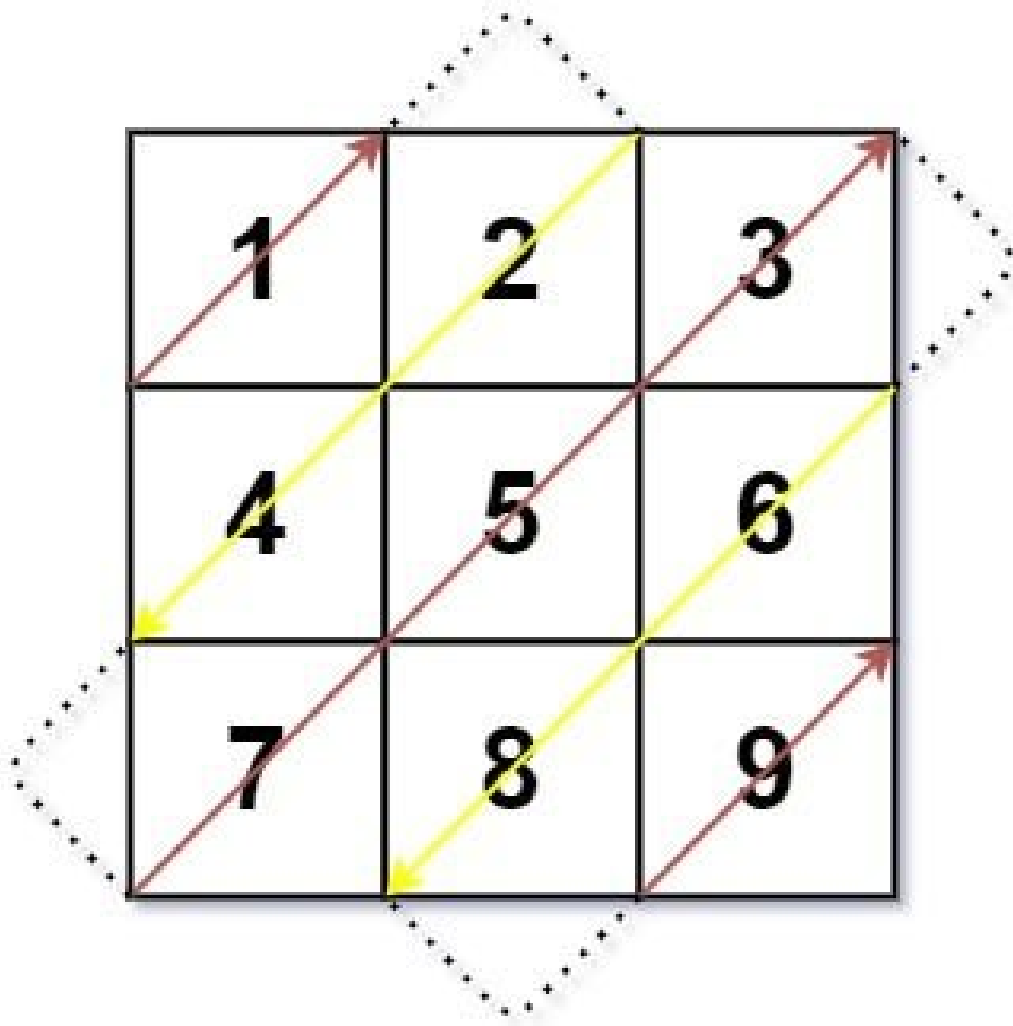
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: CodeSignal

Problem

Given an $m \times n$ matrix `mat`, return an array of all the elements of the array in a diagonal order.

Example 1:



Input: mat = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,4,7,5,3,6,8,9]

Example 2:

Input: mat = [[1,2],[3,4]]
Output: [1,2,3,4]

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 104$
- $1 \leq m * n \leq 104$

- $-105 \leq \text{mat}[i][j] \leq 105$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Traverse all elements diagonally: alternate up-right then down-left directions.
# There are m+n-1 diagonals. Even-numbered diagonals go up-right, odd go down-left.
Right?"
#
# "[[1,2,3],[4,5,6],[7,8,9]] → [1,2,4,7,5,3,6,8,9] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Simulate diagonal walk with direction flags; flip direction at matrix boundaries.
# Track visited or use index math to avoid infinite loops.
# Correct O(m*n); index-formula approach avoids simulation bugs.
# Time: O(m * n). Space: O(m * n) output.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (diagonal grouping)

```
# "Group elements by diagonal d = r+c (same sum = same diagonal).
# Diagonal d contains elements where r+c=d.
# For even d: traverse top-to-bottom order reversed (up-right) → sort by r
descending.
# For odd d: traverse top-to-bottom (down-left) → sort by r ascending."
```

PHASE 4 — CLEAN CODE

```
class _498_DiagonalTraverse:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        m, n = len(mat), len(mat[0])
        diags: defaultdict = defaultdict(list)
        for r in range(m):
            for c in range(n):
                diags[r + c].append(mat[r][c])
        result = []
        for d in range(m + n - 1):
            if d % 2 == 0:
                result.extend(reversed(diags[d])) # even → up-right (larger r first
→ reversed)
            else:
                result.extend(diags[d]) # odd → down-left (smaller r first →
natural)
```



```
return result
```

PHASE 5 — TEST & DEBUG

```
# [[1,2,3],[4,5,6],[7,8,9]]:
# d=0: {(0,0)=1} even → reversed=[1]. d=1: {(0,1)=2,(1,0)=4} odd → [2,4].
# d=2: {(0,2)=3,(1,1)=5,(2,0)=7} even → reversed=[7,5,3].
# d=3: {(1,2)=6,(2,1)=8} odd → [6,8]. d=4: {(2,2)=9} even → reversed=[9].
# → [1,2,4,7,5,3,6,8,9] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(m*n) for the diagonal groups.
# The 'group by r+c' insight eliminates direction-tracking complexity.
# Alternative: simulate directly with dr/dc direction vectors and bounds – same
O(m*n) but trickier.
# Given more time I'd ask: what if we want anti-diagonals (r-c = constant)?
# Same approach – group by r-c, alternate direction per group."
```

◆ Quick Review

Key Insights

- Traverse the matrix diagonally with alternating directions: upward (top-right) and downward (bottom-left).
- When the traversal reaches a boundary (top, bottom, left, or right), adjust the position and change the direction.
- Use simulation to process each element exactly once while managing the indices based on the current direction.

Space & Time

Time Complexity: $O(m * n)$ where m is the number of rows and n is the number of columns, as every element is processed once.

Space Complexity: $O(m * n)$ for the output array (excluding the input matrix), while additional space usage remains constant.

Solution

The solution simulates the diagonal traversal using a direction flag (1 for upward and -1 for downward). Begin at the matrix's top-left corner. Append the current element to the result. Depending on the current direction: If moving upward, decrease the row and increase the column. If moving downward, increase the row and decrease the column. Adjust for boundary hits: When moving upward: If at the last column, increment the row and switch direction. If at the first row, increment the column and switch direction. When moving downward: If at the last row, increment the column and switch direction. If at the first column, increment the row and switch direction. This process continues until all elements are added to the result array.

Matrix

□ ★ #531 Lonely Pixel I ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix

Lists: ★ Premium | Premium 98

Problem

Given an $m \times n$ picture consisting of black 'B' and white 'W' pixels, return the number of black lonely pixels.

A black lonely pixel is a character 'B' that located at a specific position where the same row and same column don't have any other black pixels.

Example 1:

W	W	B
W	B	W
B	W	W

Input: picture = `[["W","W","B"],["W","B","W"],["B","W","W"]]`

Output: 3

Explanation: All the three 'B's are black lonely pixels.

Example 2:

B	B	B
B	B	W
B	B	B

Input: picture = `[["B","B","B"],["B","B","W"],["B","B","B"]]`
Output: 0

Constraints:

- `m == picture.length`
- `n == picture[i].length`
- `1 <= m, n <= 500`
- `picture[i][j]` is 'W' or 'B'.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "A 'B' pixel is lonely if it's the ONLY 'B' in its row AND the only 'B' in its column.
# Count lonely pixels. Right?"
#
# "[[W,W,B],[W,B,W],[B,W,W]]: each B is alone in its row and column → 3 ✓
# [[B,B,B],[B,B,W],[B,B,B]]: every B shares its row or column with another → 0 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each black pixel B, scan its entire row for other black pixels.
# Then scan its entire column – B is lonely if both counts equal 1.
# O(m*n*(m+n)) – checking row and column per pixel.
# Time: O(m * n * (m + n)). Space: O(1).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Count B's per row and per column in one pass: O(m*n).
# In second pass, for each B: if row_count[r]==1 and col_count[c]==1, it's lonely."
```

PHASE 4 — CLEAN CODE

```
class _531_LonelyPixel:
    def findLonelyPixel(self, picture: List[List[str]]) -> int:
        m, n = len(picture), len(picture[0])
        row_count = [sum(1 for c in range(n) if picture[r][c] == 'B') for r in range(m)]
        col_count = [sum(1 for r in range(m) if picture[r][c] == 'B') for c in range(n)]
        return sum(
            1 for r in range(m) for c in range(n)
            if picture[r][c] == 'B' and row_count[r] == 1 and col_count[c] == 1
        )
```

PHASE 5 — TEST & DEBUG

```
# [[W,W,B],[W,B,W],[B,W,W]]: row_count=[1,1,1]. col_count=[1,1,1].
# Each B has row=1 and col=1 → all 3 are lonely ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(m+n) for row/col counts.
# Given more time I'd ask: Lonely Pixel II (#533)?
# Same pixel must be in a column containing exactly N Bs, and the rows of those Bs must all be identical.
# Much more complex – requires comparing full rows."
```

◆ Quick Review

Key Insights

- Count the number of 'B' pixels in each row.
- Count the number of 'B' pixels in each column.
- A pixel is considered lonely if the count in its corresponding row and column is exactly 1.
- Two passes through the matrix can efficiently solve the problem: one for counting and one for checking conditions.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the dimensions of the picture.

Space Complexity: $O(m + n)$, used to store the row and column counts.

Solution

The solution works by first scanning the entire matrix to compute the number of black pixels in each row and each column using two arrays/lists. Once these counts are available, a second pass through the matrix determines for each black pixel if it is lonely by checking if its corresponding row and column count

equals 1. The approach uses simple array data structures for tallying counts and ensures that each element is processed only a constant number of times.

Matrix

□ ★ #723 Candy Crush ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Matrix · Simulation
Lists: ★ Premium | Premium 98

Problem

This question is about implementing a basic elimination algorithm for Candy Crush.

Given an $m \times n$ integer array `board` representing the grid of candy where `board[i][j]` represents the type of candy. A value of `board[i][j] == 0` represents that the cell is empty.

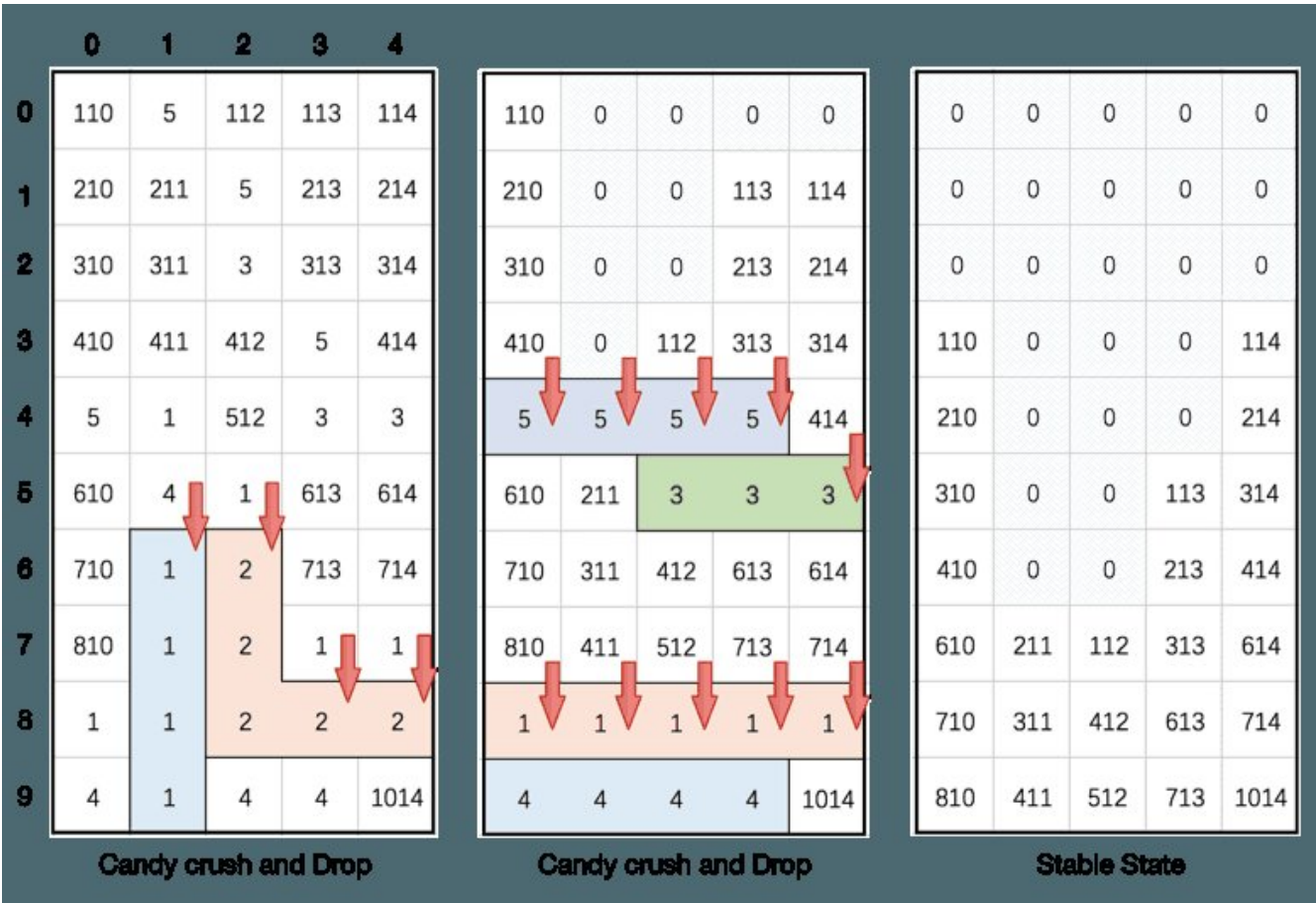
The given board represents the state of the game following the player's move. Now, you need to restore the board to a stable state by crushing candies according to the following rules:

- If three or more candies of the same type are adjacent vertically or horizontally, crush them all at the same time – these positions become empty.

- After crushing all candies simultaneously, if an empty space on the board has candies on top of itself, then these candies will drop until they hit a candy or bottom at the same time. No new candies will drop outside the top boundary.
- After the above steps, there may exist more candies that can be crushed. If so, you need to repeat the above steps.
- If there does not exist more candies that can be crushed (i.e., the board is stable), then return the current board.

You need to perform the above rules until the board becomes stable, then return the stable board.

Example 1:



Input: board = [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],[410,411,412,5,414],[5,1,512,3,3],[610,4,1,613,614],[710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],[4,1,4,4,1014]]

Output: [[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[110,0,0,0,114],[210,0,0,0,214],[310,0,0,113,314],[410,0,0,213,414],[610,211,112,313,614],[710,311,412,613,714],[810,411,512,713,1014]]

Example 2:

Input: board = [[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]

Output: [[1,3,0,0,0],[3,4,0,5,2],[3,2,0,3,1],[2,4,0,5,2],[1,4,3,1,1]]

Constraints:

- m == board.length
- n == board[i].length
- 3 <= m, n <= 50
- 1 <= board[i][j] <= 2000

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Repeatedly: mark 3+ same-type candies horizontally or vertically, crush them (set to 0),
# then drop remaining candies down. Repeat until stable. Return final board. Right?"
#
# "Mark all runs of 3+ simultaneously, crush all at once, then gravity – not one at a time."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Repeat until stable: find horizontal/vertical runs of 3+, mark for crush, drop gravity.
# One full board scan per cascade step.
# Correct simulation; finding all runs in one pass per step is cleaner.
# Time:  $O((m \times n)^2)$  worst cascades. Space:  $O(m \times n)$  mark board.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (in-place marking)

```
# "Mark by negating values (sign flip) – avoids extra space.
# Step 1: scan rows and cols, negate any value in a run of 3+.
# Step 2: zero out all negatives (crush).
# Step 3: for each column, compact non-zeros to the bottom (gravity).
# Repeat until no changes in step 1. Time:  $O((m \times n)^2)$  worst case but small in practice."
```

PHASE 4 — CLEAN CODE

```
class _723_CandyCrush:
    def candyCrush(self, board: List[List[int]]) -> List[List[int]]:
        m, n = len(board), len(board[0])
        while True:
            crush = set()
            for r in range(m): # mark horizontal runs
                for c in range(n - 2):
                    v = abs(board[r][c])
                    if v and v == abs(board[r][c+1]) == abs(board[r][c+2]):
                        crush.update([(r,c),(r,c+1),(r,c+2)])
            for r in range(m - 2): # mark vertical runs
                for c in range(n):
                    v = abs(board[r][c])
                    if v and v == abs(board[r+1][c]) == abs(board[r+2][c]):
```

```

        crush.update([(r,c),(r+1,c),(r+2,c)])
    if not crush:
        break
    for r, c in crush:
        board[r][c] = 0
    for c in range(n): # gravity: compact column downward
        write = m - 1
        for r in range(m - 1, -1, -1):
            if board[r][c] != 0:
                board[write][c] = board[r][c]
                if write != r:
                    board[r][c] = 0
                write -= 1
    return board

```

PHASE 5 — TEST & DEBUG

```

# "[[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]":
# Horizontal: row 1 has [3,3,3] at c=1,2,3. Row 3 has [4,4] and [5,5]...
# After crushing and dropping → stable board matches expected output ✓"

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "O((m*n)2) worst case — at most m*n crush iterations, each O(m*n).
# For small boards (m,n≤50) this is fast. The abs() trick avoids needing a separate
mark array.
# The gravity step uses a two-pointer write-head approach per column.
# Given more time I'd ask: what if candies can crush diagonally too?
# Add diagonal scan to the marking step — same structure."

```

◆ Quick Review

Key Insights

- Use a simulation approach: repeatedly detect crushable candies and simulate their removal.
- Traverse horizontally and vertically to mark candies that are in groups of three or more.
- Instead of removing candies immediately, mark them for removal and update the board once for the entire pass.

- After crushing, simulate gravity by shifting non-zero candies downward in each column.
- Continue the process until no further candy crushes can be detected.

Space & Time

Time Complexity: $O((m \times n)^2)$ in the worst case since each iteration includes scanning the $m \times n$ grid and then applying gravity.

Space Complexity: $O(m \times n)$ for maintaining additional markers or flags (or using the board itself with in-place modifications).

Solution

The approach involves iteratively scanning the board to identify candy groups that should be crushed using two passes: Horizontal Check – For each row, check segments of 3 consecutive candies. Vertical Check – For each column, check segments of 3 consecutive candies. If any candy qualifies, mark its position for crushing. After marking, update the board by turning all marked positions into 0. Then, for each column, drop down non-zero candies

such that all empty cells (zeros) are at the top. Repeat this process until no group of 3 or more adjacent candies is found. Key data structures include the board array and a temporary marker (can be implicit by using signs or a separate boolean matrix) for tracking candies to crush. The algorithm ensures a stable board before returning it.

Matrix

□ #835 Image Overlap

MED

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix
Lists: CodeSignal**

Problem

You are given two images, `img1` and `img2`, represented as binary, square matrices of size $n \times n$. A binary matrix has only 0s and 1s as values.

We translate one image however we choose by sliding all the 1 bits left, right, up, and/or down any number of units. We then place it on top of the other image. We can then calculate the overlap by counting the number of positions that have a 1 in both images.

Note also that a translation does not include any kind of rotation. Any 1 bits that are translated outside of the matrix borders are erased.

Return the largest possible overlap.

Example 1:

1	1	0
0	1	0
0	1	0

img1

0	0	0
0	1	1
0	0	1

img2

Input: `img1 = [[1,1,0],[0,1,0],[0,1,0]]`, `img2 = [[0,0,0],[0,1,1],[0,0,1]]`

Output: 3

Explanation: We translate `img1` to right by 1 unit and down by 1 unit.

The number of positions that have a 1 in both images is 3 (shown in red).

Example 2:

Input: `img1 = [[1]]`, `img2 = [[1]]`

Output: 1

Example 3:

Input: `img1 = [[0]]`, `img2 = [[0]]`

Output: 0

Constraints:

- `n == img1.length == img1[i].length`
- `n == img2.length == img2[i].length`
- `1 <= n <= 30`
- `img1[i][j]` is either 0 or 1.
- `img2[i][j]` is either 0 or 1.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Translate img1 over img2 (no rotation) and count overlapping 1s.
# Find the translation that maximises overlap. Return max overlap count. Right?"
#
# "img1=[[1,1,0],[0,1,0],[0,1,0]], img2=[[0,0,0],[0,1,1],[0,0,1]] → 3 ✓ (translate
right 1, down 1)"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Try every translation vector (dx,dy) aligning A onto B.
# For each shift, count overlapping 1-cells.
#  $O(n^4)$  translations on  $n \times n$  images – too slow for  $n=30$ .
# Time:  $O(n^4)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (translation vector counting)

```
# "Collect all 1-positions in each image.
# For each pair (p1 from img1, p2 from img2), the translation to align p1 with p2 is
(p2.r-p1.r, p2.c-p1.c).
# Count the most common translation vector – that's the maximum overlap.
# Time:  $O(A*B)$  where  $A,B$  = number of 1s. Space:  $O(A*B)$ . For  $n=30$ ,  $A,B \leq 900$ ."
```

PHASE 4 — CLEAN CODE

```
class _835_ImageOverlap:
    def largestOverlap(self, img1: List[List[int]], img2: List[List[int]]) -> int:
        n = len(img1)
        ones1 = [(r, c) for r in range(n) for c in range(n) if img1[r][c] == 1]
        ones2 = [(r, c) for r in range(n) for c in range(n) if img2[r][c] == 1]
        translations: Counter = Counter()
        for r1, c1 in ones1:
            for r2, c2 in ones2:
                translations[(r2 - r1, c2 - c1)] += 1
        return max(translations.values(), default=0)
```

PHASE 5 — TEST & DEBUG

```
# ones1=[(0,0),(0,1),(1,1),(2,1)]. ones2=[(1,1),(1,2),(2,2)].
# Translation (1,1) appears when: (0,0)→(1,1)✓, (0,1)→(1,2)✓, (1,1)→(2,2)✓ →
count=3.
# max=3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(A*B)$  where A,B = number of 1s in each image, at most  $n^2$ .  
# Space  $O(A*B)$  for the translation Counter.  
# For dense images (many 1s), this is  $O(n^4)$  – same as brute force.  
# For sparse images, much faster.  
# The key insight: instead of iterating over all translations, only count  
translations  
# where at least one 1-pair aligns – no wasted iterations.  
# Given more time I'd ask: what if rotation is also allowed?  
# Need to try all rotation+translation combos – much harder."
```

◆ Quick Review

Key Insights

- You only need to consider the positions of 1's in each matrix.
- The overlap count for a given translation can be computed by comparing the relative differences between the positions of 1's in the two images.
- Using a hash map (or dictionary) to count how many times each translation vector appears lets you compute the maximum overlap efficiently.
- The translation operation is equivalent to shifting the positions, so the best possible translation is the one that aligns the most pairs of 1's.

Space & Time

Time Complexity: $O(n^4)$ in the worst-case when using a nested iteration over positions for both matrices, though the average situation is typically better since not all cells are 1.

Space Complexity: $O(n^2)$ for storing the positions of 1's and the counts for translation vectors.

Solution

The solution involves the following steps: Traverse both images to record the coordinates of all 1's. For every pair of 1's (one from the first image, one from the second), compute the translation vector (offset) that aligns them. Use a hash map to count how many times each offset appears. The maximum count recorded in the hash map is the largest overlap achievable by any translation. This approach avoids simulating every possible slide over the matrix and focuses only on relative alignments of 1's.

Matrix

★ #1198 Find Smallest Common Element in All Rows ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Binary Search · Matrix · Counting

Lists: ★ Premium | Premium 98

Problem

Given an $m \times n$ matrix `mat` where every row is sorted in strictly increasing order, return the smallest common element in all rows.

If there is no common element, return -1 .

Example 1:

```
Input: mat = [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]  
Output: 5
```

Example 2:

```
Input: mat = [[1,2,3],[2,3,4],[2,3,5]]  
Output: 2
```

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 500$

- $1 \leq \text{mat}[i][j] \leq 104$
- $\text{mat}[i]$ is sorted in strictly increasing order.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the smallest integer that appears in every row. Each row is sorted.
# Return -1 if no common element exists. Right?
# Values are 1-10^4, at most 500 rows and 500 cols."
#
# "[[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]] → 5 (appears in all rows) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Intersect row0 with row1, then intersect result with row2, and so on.
# Track the maximum value in the final intersection set.
# Each intersection pass is O(m) per row using a hash set of row values.
# Time: O(rows * cols). Space: O(cols) set.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (frequency count)

```
# "Count how many rows each value appears in using a single array of size max_val+1.
# Walk all rows: for each value encountered, increment count[value].
# Return the smallest value with count == m (appears in ALL rows).
# Time: O(m*n). Space: O(max_val)."
```

PHASE 4 — CLEAN CODE

```
class _1198_SmallestCommonElement:
    def smallestCommonElement(self, mat: List[List[int]]) -> int:
        m = len(mat)
        freq = Counter()
        for row in mat:
            for val in row:
                freq[val] += 1
        for val in range(1, 10001):
            if freq[val] == m:
                return val
        return -1
```

PHASE 5 — TEST & DEBUG

```
# [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]:
```

```
# freq[5] = 4 (appears in all 4 rows). freq[1]=2, freq[3]=3, etc.
# Scanning 1..10000: first val with freq==4 is 5 → return 5 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(m \cdot n + \max\_val)$ . Space  $O(\max\_val)$ .
```

```
# Alternative: binary search per row. For each candidate, binary search in all m rows.
```

```
# Start candidates from mat[0] (smallest row), binary search each of the other rows.
```

```
#  $O(n + m \cdot n \cdot \log n)$  – worse than the counting approach for dense matrices.
```

```
# Given more time I'd ask: what if there's no bound on values?
```

```
# Use a hash Counter for freq –  $O(m \cdot n)$  time,  $O(\text{distinct values})$  space."
```

◆ Quick Review

Key Insights

- Since every row is strictly increasing, binary search can be used efficiently to check for the presence of an element in a row.
- Iterating over the first row's elements and verifying their presence in all other rows is an effective approach.
- Early termination is possible if an element is not found in any one row.
- Time complexity is manageable given the constraints, even using binary search for each candidate across rows.

Space & Time

Time Complexity: $O(m \cdot n \cdot \log(n))$ where m is the number of rows and n is the number of columns (binary search in each row for each

candidate from the first row).

Space Complexity: $O(1)$ (only a few auxiliary variables are used).

Solution

The solution iterates over each element (candidate) in the first row of the matrix. For each candidate, it checks if the candidate is present in every other row using binary search, taking advantage of the sorted order. If a candidate is found in every row, return it immediately as it will be the smallest such common element because the first row is processed in order. If no candidate is common across all rows, return -1 .

Trees & BST

Round 2 · High · Medium · 24 questions

Trees & BST

□ #98 Validate Binary Search Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST
Lists: Grind 169

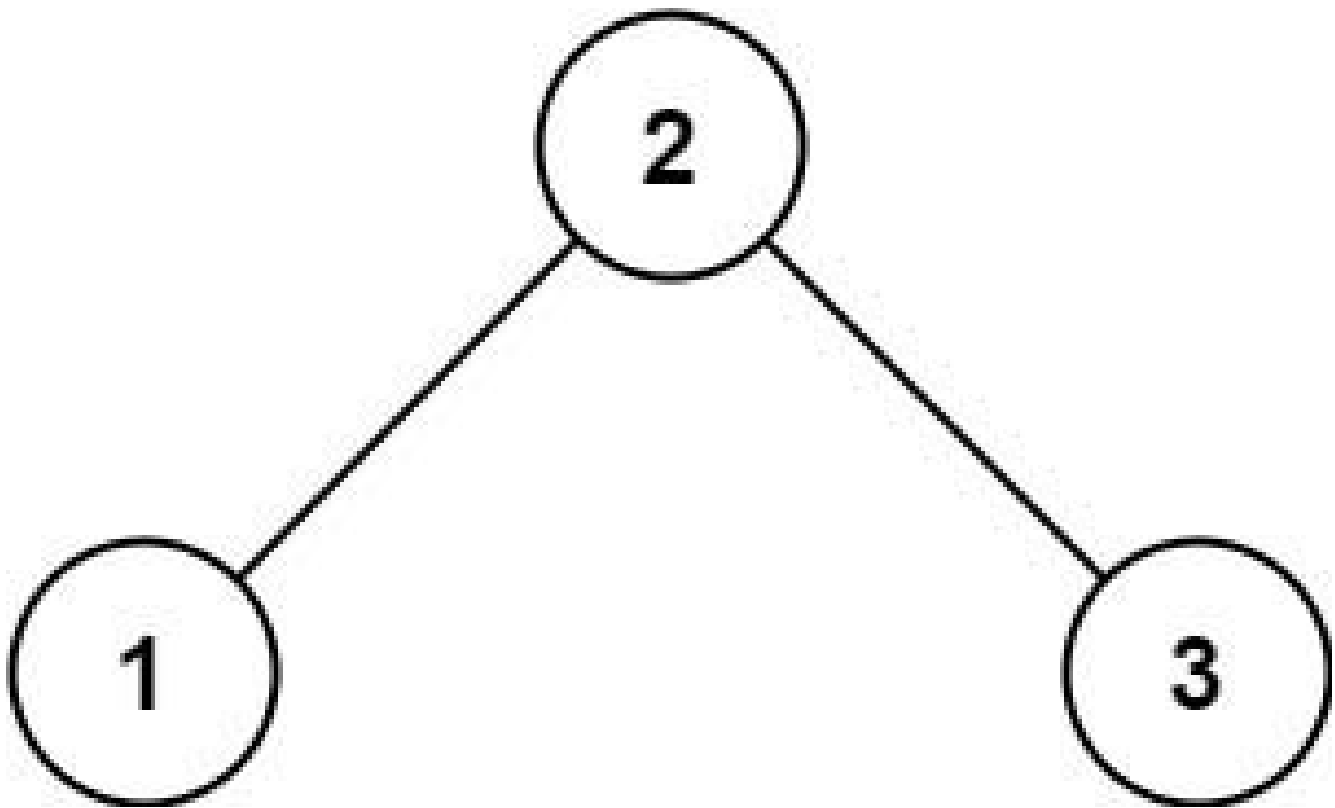
Problem

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

A valid BST is defined as follows:

- The left subtree of a node contains only nodes with keys strictly less than the node's key.
- The right subtree of a node contains only nodes with keys strictly greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

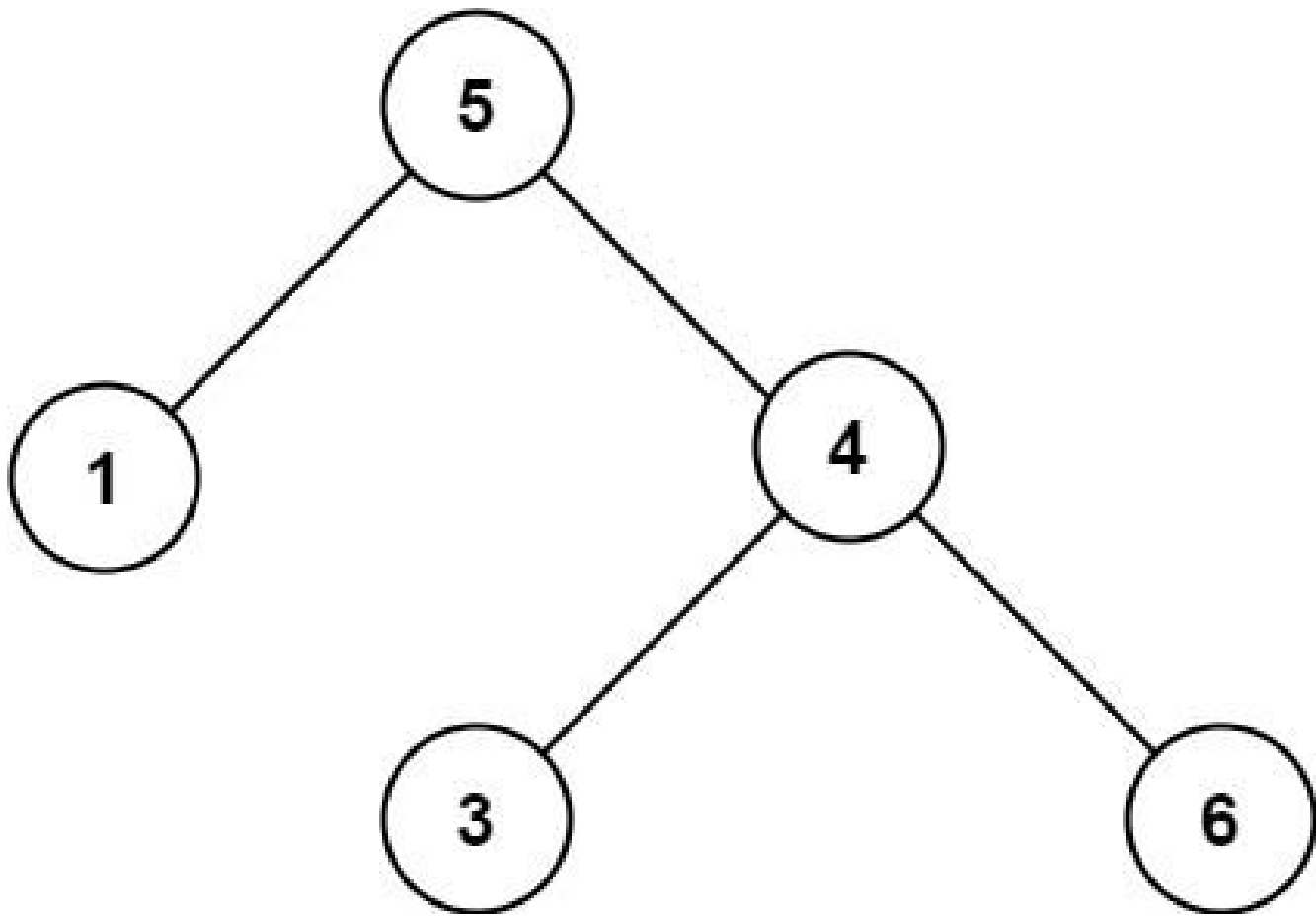
Example 1:



Input: root = [2,1,3]

Output: true

Example 2:



Input: root = [5,1,4,null,null,3,6]

Output: false

Explanation: The root node's value is 5 but its right child's value is 4.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Every node's value must be strictly greater than all values in its left subtree
# AND strictly less than all values in its right subtree – not just the immediate
# parent. Right?
# The classic trap: [5,1,4,null,null,3,6] – root=5, right child=4 – 4<5 → invalid ✓"
#
# "[2,1,3]: 1<2<3 → true ✓. [5,1,4,null,null,3,6]: 4<5 → false ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# In-order traversal collects values; verify the list is strictly increasing.
# Any duplicate or decrease means invalid BST.
# Correct O(n), but recursion can pass min/max bounds instead of storing all values.
# Time: O(n). Space: O(n) in-order list.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (bounds propagation)

```
# "Pass min/max bounds down the tree.
# Every node must satisfy min_val < node.val < max_val.
# Left child: max_val = node.val.
# Right child: min_val = node.val.
# Use float('-inf') and float('inf') as initial bounds. Use inner helper."
```

PHASE 4 — CLEAN CODE

```
class _98_ValidateBST:
    def isValidBST(self, root: Optional['TreeNode']) -> bool:
        def valid(node, min_val, max_val):
            if not node:
                return True
            if not (min_val < node.val < max_val):
                return False
            return (valid(node.left, min_val, node.val) and
                    valid(node.right, node.val, max_val))
        return valid(root, float('-inf'), float('inf'))
```

PHASE 5 — TEST & DEBUG

```
# [2,1,3]: valid(2,-inf,inf)→valid(1,-inf,2)→valid(None,...)=T and T→T.
valid(3,2,inf)→T. → True ✓
# [5,1,4,null,null,3,6]: valid(4,5,inf): 4 NOT in (5,inf) → False → outer returns
False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h) for the call stack.
# The bounds approach is the correct one – in-order checking works too but requires
# state.
# The common mistake: only checking node vs parent, not the full subtree constraint.
# Given more time I'd ask: what if duplicates are allowed (left ≤ node < right)?
# Change the strict inequalities accordingly."
```

◆ Quick Review

Key Insights

- Every node must adhere to a range: all nodes in the left subtree should be less than the current node, and all nodes in the right subtree should be greater.
- A recursive depth-first search approach works well by passing down the valid range (lower and upper bounds) for each node.
- Using the limits (negative and positive infinity) simplifies handling edge cases at the tree root.
- An in-order traversal would alternatively yield a sorted order for node values, but updating ranges directly is more intuitive for recursive implementation.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes, as each node is visited once.

Space Complexity: $O(H)$, where H is the height of the tree, due to the recursion stack.

Solution

We use a recursive approach to validate the BST property. We maintain two variables, low and high, representing the valid range for the current node's value. For the left child, the valid range becomes (low, node.val) and for the right child, it becomes (node.val, high). By checking if each node's value falls within its respective range, we can ensure that the tree satisfies the BST properties.

Trees & BST

□ #102 Binary Tree Level Order Traversal

MED

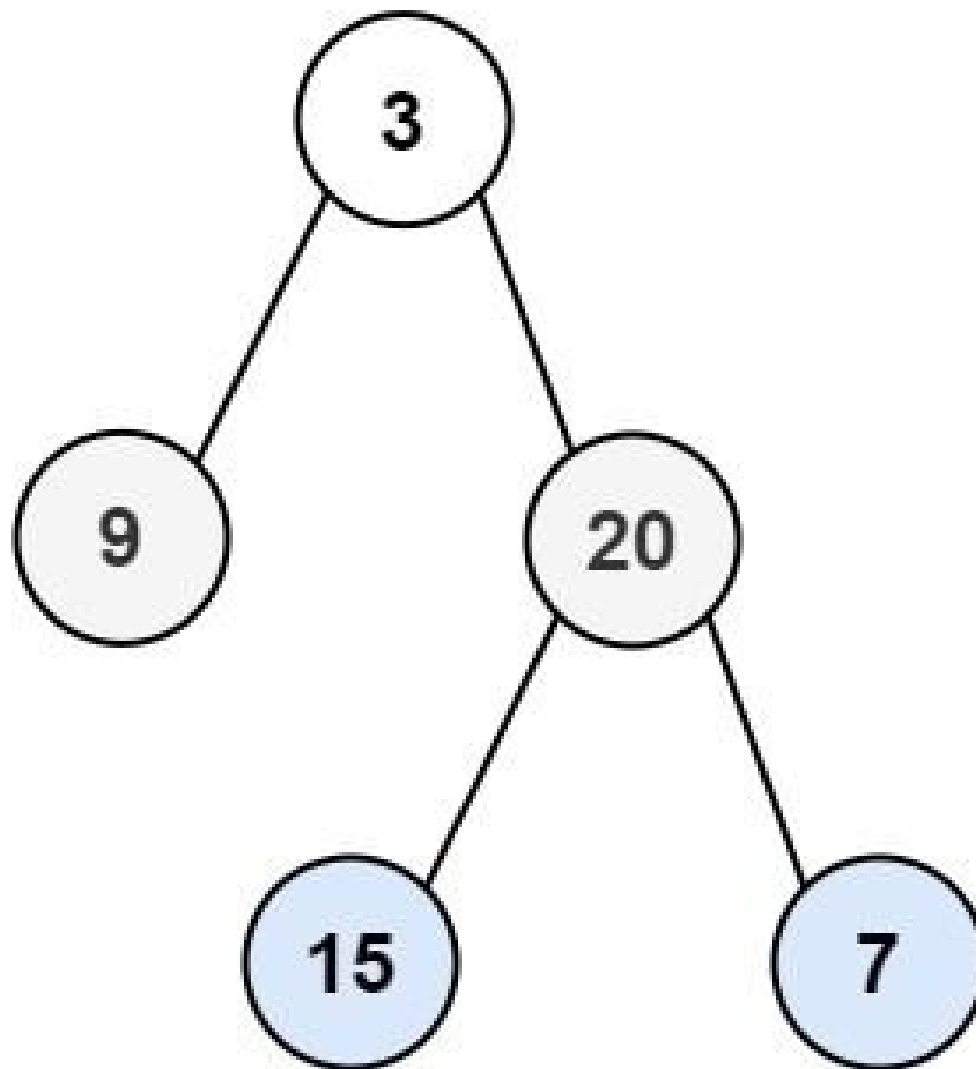
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 2000]$.
- $-1000 \leq \text{Node.val} \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return node values level by level, left to right, each level as a sub-list.
Right?
# Empty tree → []. Single node → [[val]]."
#
# "[3,9,20,null,null,15,7] → [[3],[9,20],[15,7]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# DFS with a level parameter: append node.val to result[level].
# After traversal, result lists are level-order groups left-to-right.
# Works in  $O(n)$ , but BFS with a queue is the more natural level-by-level approach.
# Time:  $O(n)$ . Space:  $O(h)$  recursion depth.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BFS with level grouping)

```
# "BFS using a deque. At each step, process ALL nodes at the current level (snapshot
the queue size),
# collect their values, enqueue their children. Natural level separation – no level
counter needed."
```

PHASE 4 — CLEAN CODE

```
class _102_LevelOrder:
    def levelOrder(self, root: Optional['TreeNode']) -> List[List[int]]:
        if not root:
            return []
        result = []
        queue = deque([root])
        while queue:
            level_size = len(queue)
            level_vals = []
            for _ in range(level_size):
                node = queue.popleft()
```

```

        level_vals.append(node.val)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
    result.append(level_vals)
    return result

```

PHASE 5 — TEST & DEBUG

```

# root=3: queue=[3]. level_size=1. level=[3]. enqueue 9,20. result=[[3]].
# queue=[9,20]. level_size=2. level=[9,20]. enqueue 15,7. result=[[3],[9,20]].
# queue=[15,7]. level_size=2. level=[15,7]. result=[[3],[9,20],[15,7]] ✓
# root=None: return [] immediately ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(w) where w = max width of the tree (at most n/2 for a complete tree).
# The 'snapshot the queue size' trick is the canonical BFS level-separation pattern.
# Variations: level order bottom-up (#107) → just reverse the result.
# Zigzag level order (#103) → alternate the direction each level.
# Right side view (#199) → take the last element of each level.
# Given more time I'd ask: what if we want the minimum depth to a leaf?
# BFS stops at the first leaf found – don't traverse the whole tree."

```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to traverse the tree.
- A queue data structure is ideal for maintaining the order of nodes at each level.
- For every level, process all nodes currently in the queue and add their children for the next level.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is processed

once.

Space Complexity: $O(n)$, which accounts for the space used by the queue in the worst-case scenario.

Solution

This solution leverages a BFS approach using a queue. The algorithm starts by enqueueing the root node. Then, it enters a loop that continues until the queue is empty. For each iteration—representing one level of the tree—it computes the number of nodes at that level (`levelSize`), processes these nodes by dequeuing them, and collects their values. Their non-null children are then enqueued. The list of values for each level is added to the final result list. Key data structures include the queue (for managing nodes) and a list (to store level values).

Trees & BST

#103 Binary Tree Zigzag Level Order Traversal

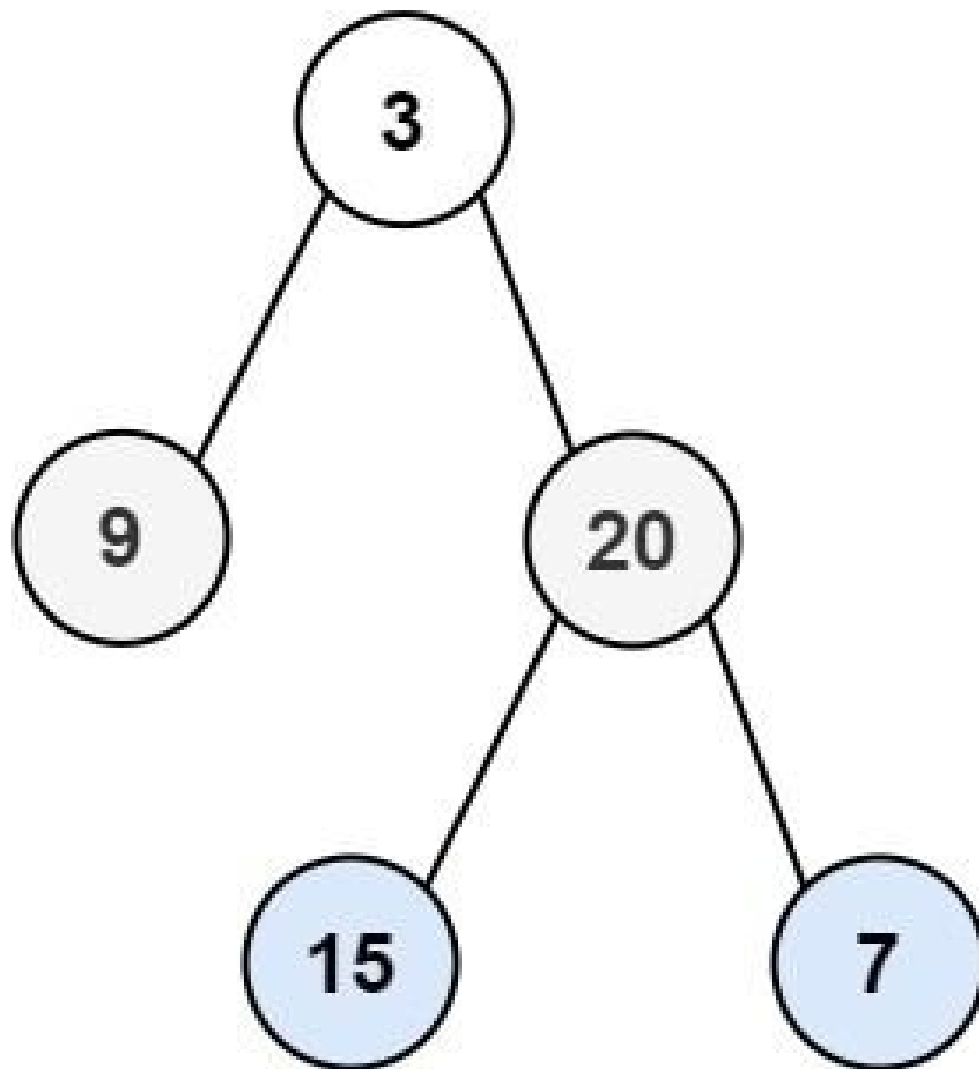
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS
Lists: Grind 169

Problem

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 2000]$.
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Level order traversal but alternating direction: odd levels left-to-right,
# even levels right-to-left (or vice versa – confirm with example).
# Root is level 0 → left-to-right. Level 1 → right-to-left. Right?"
#
# "[3,9,20,null,null,15,7] → [[3],[20,9],[15,7]] ✓ (level 1 reversed)"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Run standard BFS level-order to build a list of levels.
# Reverse every other level list (odd index levels) for zigzag output.
# Correct  $O(n)$ ; we can reverse on the fly during BFS instead of a second pass.
# Time:  $O(n)$ . Space:  $O(n)$  queue + levels.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Same BFS snapshot approach as #102. Use a boolean flag 'left_to_right'.
# Collect each level normally, then reverse if flag is False.
# Toggle flag each level. Time:  $O(n)$ . Space:  $O(w)$ ."
```

PHASE 4 — CLEAN CODE

```
class _103_ZigzagLevelOrder:
    def zigzagLevelOrder(self, root: Optional['TreeNode']) -> List[List[int]]:
        if not root:
            return []
        result = []
        queue = deque([root])
        left_to_right = True
        while queue:
            level_size = len(queue)
            level_vals = []
            for _ in range(level_size):
                node = queue.popleft()
```

```

        level_vals.append(node.val)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
        result.append(level_vals if left_to_right else level_vals[::-1])
        left_to_right = not left_to_right
    return result

```

PHASE 5 — TEST & DEBUG

```

# root=[3,9,20,null,null,15,7]:
# Level 0: [3], left_to_right=True → [3]. Toggle→False.
# Level 1: [9,20], left_to_right=False → reversed=[20,9]. Toggle→True.
# Level 2: [15,7], left_to_right=True → [15,7]. → [[3],[20,9],[15,7]] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(w).
# This is #102 with one added line: the conditional reverse.
# Name that connection — it shows you see the pattern family.
# Given more time I'd ask: what if alternation starts at level 1 instead of 0?
# Just flip the initial value of left_to_right."

```

◆ Quick Review

Key Insights

- Use a breadth-first search (BFS) approach to traverse the tree level by level.
- Alternate the direction of traversal on each level to achieve the zigzag pattern.
- A flag or counter can be used to determine the order of nodes for the current level.
- Using a double-ended data structure or simply reversing the list at every alternate level is an effective approach.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is visited exactly once.

Space Complexity: $O(n)$, as in the worst-case scenario (e.g., a complete binary tree) the queue used for BFS may hold $O(n)$ nodes at once.

Solution

We can solve this problem using a breadth-first search (BFS). Start by initializing a queue to hold nodes for each level. For each level: Determine the number of nodes at the current level. Iterate over these nodes, appending their children for the next level. Depending on a flag (or level count), either append node values in the natural order (left to right) or reverse the order (right to left) before adding to the result. Key data structures: A queue for BFS traversal. A list (or similar container) to store current level values. The main trick is toggling the order of insertion or reversing the list at alternate levels to achieve the "zigzag" pattern.

Trees & BST

□ #105 Construct Binary Tree from Preorder and Inorder Traversal

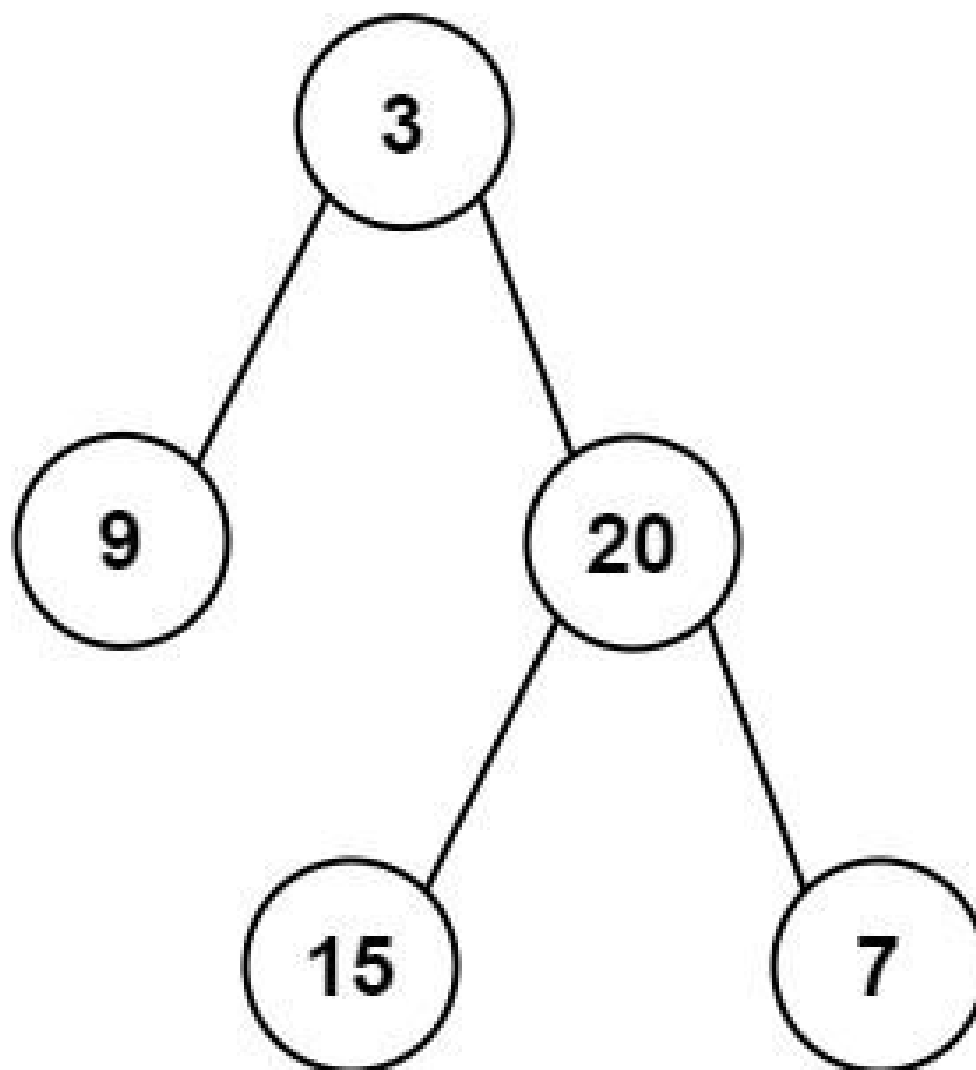
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Tree · DFS
Lists: Grind 169

Problem

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Example 1:



Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
Output: [3,9,20,null,null,15,7]

Example 2:

Input: preorder = [-1], inorder = [-1]
Output: [-1]

Constraints:

- $1 \leq \text{preorder.length} \leq 3000$
- $\text{inorder.length} == \text{preorder.length}$
- $-3000 \leq \text{preorder}[i], \text{inorder}[i] \leq 3000$

- preorder and inorder consist of unique values.
- Each value of inorder also appears in preorder.
- preorder is guaranteed to be the preorder traversal of the tree.
- inorder is guaranteed to be the inorder traversal of the tree.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Preorder: root first, then left subtree, then right subtree.
# Inorder: left subtree, root, right subtree.
# All values are unique – so we can find the root's position in inorder uniquely.
Right?"
#
# "preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]:
# Root=3. In inorder, 3 is at index 1. Left subtree=[9], right subtree=[15,20,7].
# preorder left=[9], right=[20,15,7]. Recurse."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Root is preorder[0]; scan inorder to find root index and split left/right
subtrees.
# Recursively repeat on each side – correct tree reconstruction.
# Scanning inorder for every root costs  $O(n^2)$  total; hash map of inorder indices
fixes that.
# Time:  $O(n^2)$  naive. Space:  $O(n)$  recursion.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Pre-build a hash map: inorder value → index for  $O(1)$  lookups.
# Use a preorder index pointer (use a list to allow mutation inside inner function)."
```

```
# build(in_left, in_right): root = preorder[pre_idx]. Find in inorder. Recurse.
# Time: O(n). Space: O(n) for the map."
```

PHASE 4 — CLEAN CODE

```
class _105_BuildTree:
    def buildTree(self, preorder: List[int], inorder: List[int]) ->
Optional['TreeNode']:
    in_idx = {val: i for i, val in enumerate(inorder)}
    pre = iter(preorder)
    def build(left: int, right: int) -> Optional['TreeNode']:
        if left > right:
            return None
        root_val = next(pre)
        root = TreeNode(root_val)
        mid = in_idx[root_val]
        root.left = build(left, mid - 1)
        root.right = build(mid + 1, right)
        return root
    return build(0, len(inorder) - 1)
```

PHASE 5 — TEST & DEBUG

```
# preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]. in_idx={9:0,3:1,15:2,20:3,7:4}.
# build(0,4): root=3, mid=1. left=build(0,0), right=build(2,4).
# build(0,0): root=9, mid=0. left=build(0,-1)=None. right=build(1,0)=None. →
node(9).
# build(2,4): root=20, mid=3. left=build(2,2), right=build(4,4).
# build(2,2): root=15. → node(15). build(4,4): root=7. → node(7).
# → Tree: 3→left=9, 3→right=20→left=15, 20→right=7 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(n) for the hash map and call stack.
# Using iter(preorder) as a lazy pointer is cleaner than an index variable.
# The key insight: preorder always gives the root first, inorder tells us how many
# elements are in each subtree – that's the only information we need to recurse.
# Given more time I'd ask: construct from postorder and inorder?
# Same logic – just consume postorder from the RIGHT end, and build right subtree
first."
```

◆ Quick Review

Key Insights

- Preorder traversal always starts with the root node.

- Inorder traversal splits the tree into left subtree and right subtree based on the root.
- A hash table (map/dictionary) can be used to quickly locate the index of the root in the inorder array.
- Use recursion to construct left and right subtrees.

Space & Time

Time Complexity: $O(n)$ since we process every node exactly once.

Space Complexity: $O(n)$ due to the recursion stack and the hash map used for inorder indices.

Solution

We solve the problem using a recursive divide-and-conquer approach. First, create a hash map to store each value's index in the inorder array for quick lookups. The preorder array always provides the current root. Use the inorder array to divide the tree into left and right subtrees. Recursively construct the subtrees by adjusting the preorder index and the boundaries of the inorder array. The algorithm leverages recursion and a hash map

to achieve $O(n)$ time complexity.

Trees & BST

#199 Binary Tree Right Side View

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS

Lists: Grind 169

Problem

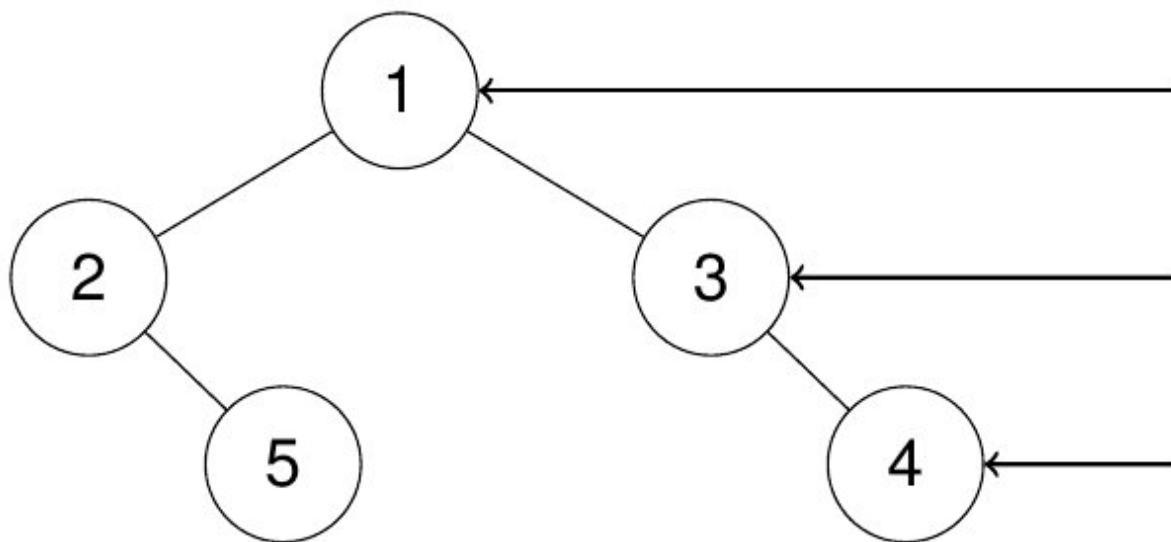
Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

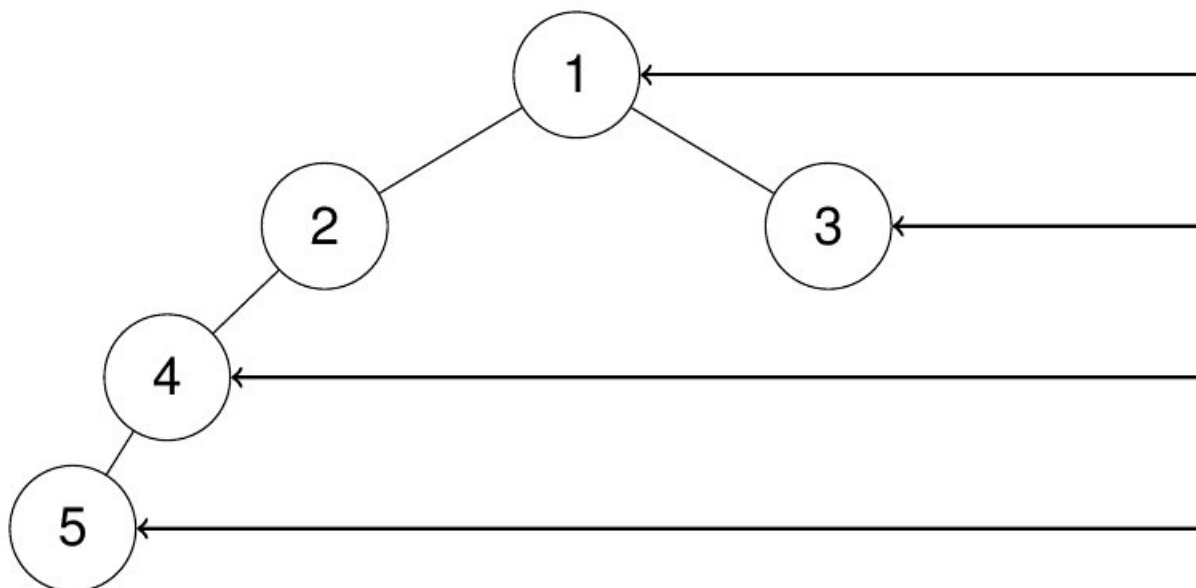


Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:



Example 3:

Input: root = [1,null,3]

Output: [1,3]

Example 4:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Imagine looking at the tree from the right. You see one node per level –
# the rightmost node visible at that level. Return top to bottom. Right?"
```

```
#
```

```
# "[1,2,3,null,5,null,4] → [1,3,4] ✓ (3 is rightmost at level 1, 4 at level 2)"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
```

```
# BFS level-order; after processing each level, take the last node value added.
```

```
# That last node is the rightmost visible from that depth.
```

```
# Correct O(n); DFS tracking depth + rightmost-at-depth is equally fine.
```

```
# Time: O(n). Space: O(w) queue width.
```

```
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Same BFS snapshot as #102/#103. Just append the LAST element of each level's
list."
```

Or DFS: visit right child first, add node.val when depth == len(result) (first node seen at new depth)."

PHASE 4 — CLEAN CODE

```
class _199_RightSideView:
    def rightSideView(self, root: Optional['TreeNode']) -> List[int]:
        if not root:
            return []
        result = []
        queue = deque([root])
        while queue:
            level_size = len(queue)
            for i in range(level_size):
                node = queue.popleft()
                if i == level_size - 1:
                    result.append(node.val) # rightmost node of this level
                if node.left: queue.append(node.left)
                if node.right: queue.append(node.right)
        return result
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3,null,5,null,4]:
# Level 0: [1] → last=1. Level 1: [2,3] → last=3. Level 2: [5,4] → last=4.
# → [1,3,4] ✓
# [1,2,3,4,null,null,null,5]:
# Level 0: [1]→1. Level 1: [2,3]→3. Level 2: [4]→4. Level 3: [5]→5. → [1,3,4,5] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(w) where w = max width.
# This is #102 with one change: only record the last element per level.
# Left side view: record the first element per level instead.
# Given more time I'd ask: what if some right-side nodes are blocked by deeper
left-side nodes?
# The problem says 'visible from the right' – the rightmost node at each DEPTH
level,
# which BFS naturally gives us. A right child at depth 2 can block a left child at
depth 2."
```

◆ Quick Review

Key Insights

- The problem is essentially about capturing the last node at each level (or the rightmost node) in a binary tree.

- A Breadth-First Search (BFS) approach naturally works by processing nodes level by level.
- Alternatively, Depth-First Search (DFS) can be adapted by prioritizing the right subtree first, ensuring that the first node encountered at each depth is the visible one.
- The number of nodes is capped at 100, so both approaches are efficient.

Space & Time

Time Complexity: $O(n)$ – we visit every node in the tree.

Space Complexity: $O(n)$ – in the worst case, the queue (or call stack for DFS) may hold all nodes of the tree.

Solution

We can solve this problem using BFS (level-order traversal). The idea is to traverse the tree level by level using a queue. For each level, the last element added to the level represents the rightmost view from that level. We then append that element's value to our result list. Alternatively, a DFS approach can be used where we visit the right subtree first. By

keeping track of the current depth and checking if it's the first time we've encountered this depth, we can add the first node visited at that depth (which will be the rightmost one) to the result. In both approaches, appropriate data structures (a queue for BFS or recursion with a list for DFS) are used to store nodes and track the traversal state.

Trees & BST

□ #230 Kth Smallest Element in a BST

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

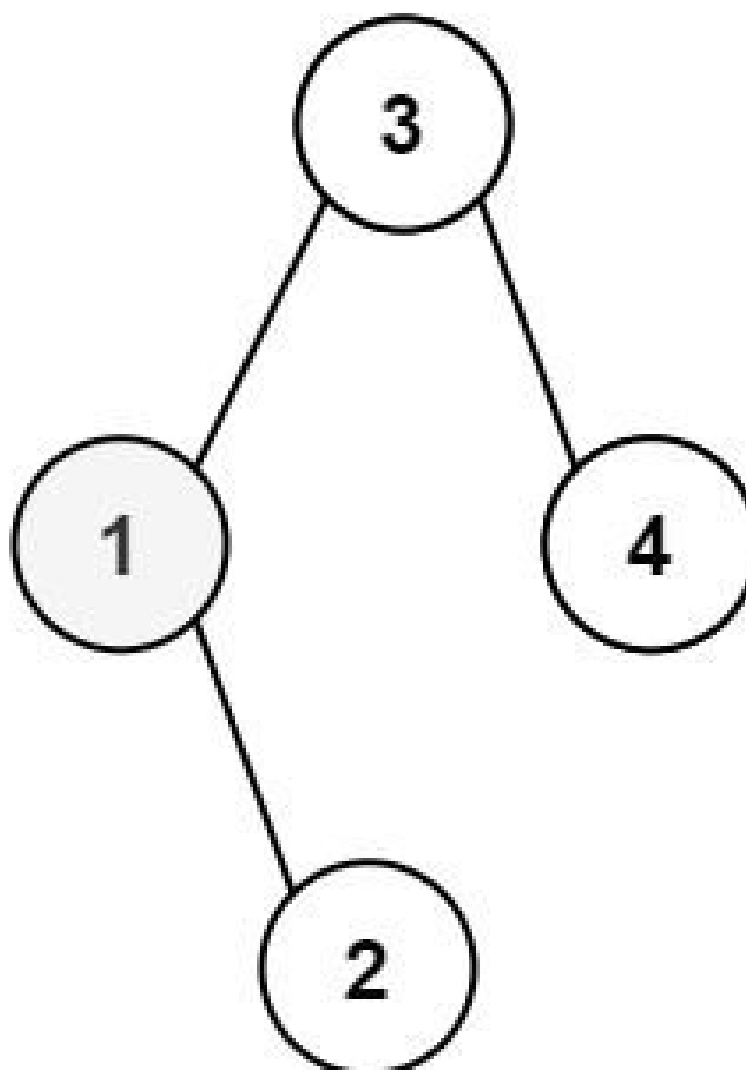
Tree · DFS · BST

Lists: Grind 169

Problem

Given the root of a binary search tree, and an integer k , return the k th smallest value (1-indexed) of all the values of the nodes in the tree.

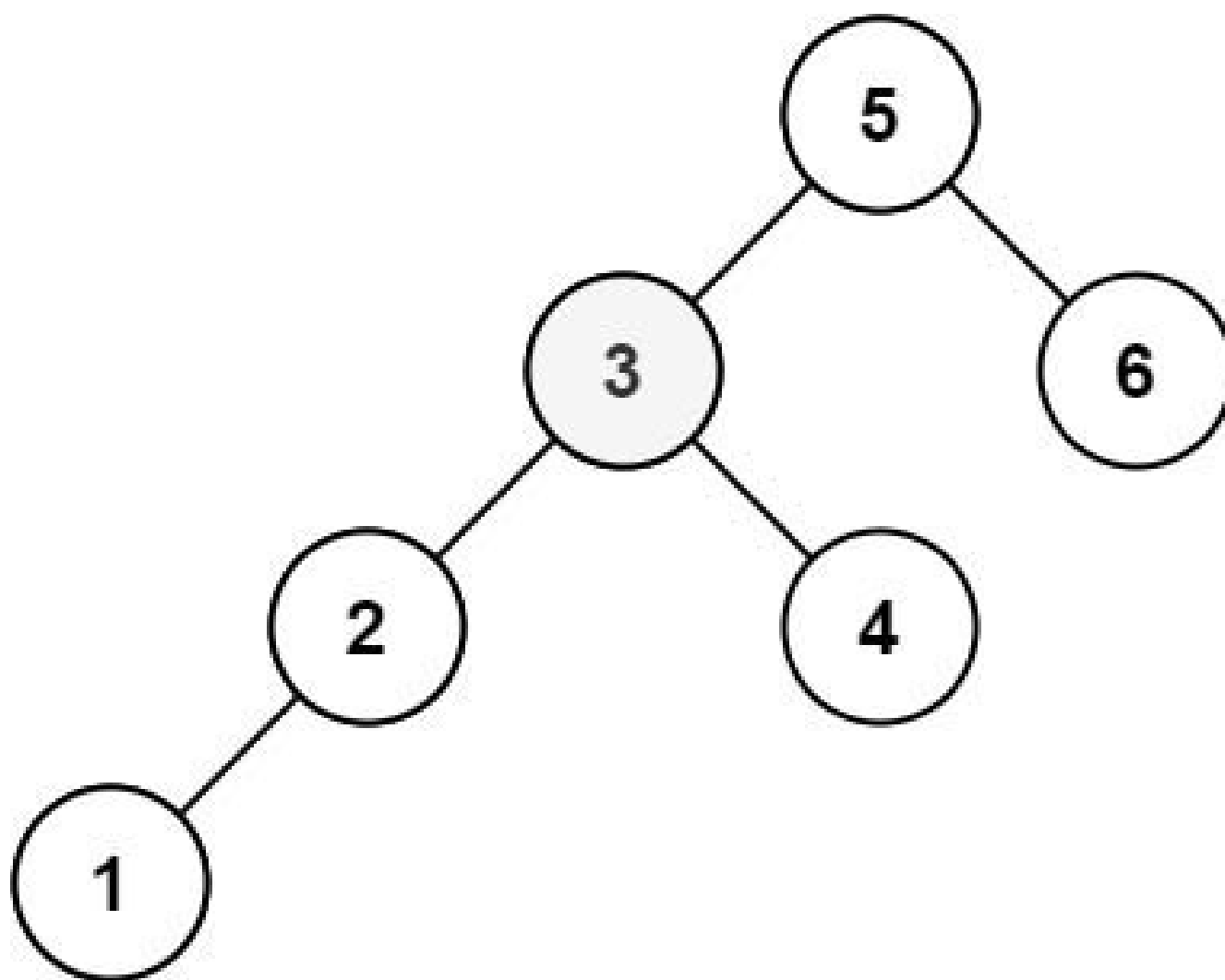
Example 1:



Input: root = [3,1,4,null,2], k = 1

Output: 1

Example 2:



Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3

Constraints:

- The number of nodes in the tree is n.
- $1 \leq k \leq n \leq 10^4$
- $0 \leq \text{Node.val} \leq 10^4$

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "BST in-order traversal (left → root → right) visits nodes in ascending sorted order.
# Return the k-th element in this order (1-indexed). Right?"
#
# "[3,1,4,null,2], k=1: in-order=[1,2,3,4]. 1st smallest=1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# In-order traversal collects all node values in sorted order.
# Return the (k-1)th element from that list.
# Correct O(n) time and O(n) space; we can stop after k steps in-order.
# Time: O(n). Space: O(n) values list.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "In-order DFS stopping early at the k-th element. Use a counter and result variable
# in a nonlocal/list wrapper. Avoids collecting all n values if k is small."
```

PHASE 4 — CLEAN CODE

```
class _230_KthSmallestBST:
    def kthSmallest(self, root: Optional['TreeNode'], k: int) -> int:
        count = [0]
        result = [0]
        def inorder(node):
            if not node or count[0] == k:
                return
            inorder(node.left)
            count[0] += 1
            if count[0] == k:
                result[0] = node.val
                return
            inorder(node.right)
        inorder(root)
        return result[0]
```

PHASE 5 — TEST & DEBUG

```
# [3,1,4,null,2], k=1:
```

```
# inorder(3)→inorder(1)→inorder(None). count=0+1=1==k=1→result=1. return.
# → 1 ✓
# [5,3,6,2,4,null,null,1], k=3:
# in-order: 1,2,3,4,5,6. 3rd = 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(k + h)$  — at most  $k$  steps after reaching the leftmost leaf. Space  $O(h)$ .
# Follow-up: frequent inserts/deletes + frequent kth queries?
# Augment each BST node with the size of its left subtree.
# Then kth smallest is an  $O(\log n)$  descent: if  $\text{left\_size} \geq k$  go left; if
 $\text{left\_size} + 1 == k$  return root; else  $k -= \text{left\_size} + 1$ , go right.
# Given more time I'd ask: what if we want the kth largest?
# Reverse in-order (right→root→left) and count  $k$  steps."
```

◆ Quick Review

Key Insights

- An in-order traversal of a BST visits nodes in increasing order.
- Use a counter to keep track of the number of nodes visited.
- As soon as the counter reaches k , the current node's value is the k th smallest.
- For frequent modifications (insertions/deletions) and queries, consider augmenting the BST with subtree counts.

Space & Time

Time Complexity: $O(n)$ in the worst-case scenario (when $k \approx n$), but on average the traversal may stop early.

Space Complexity: $O(h)$ where h is the height of the tree ($O(n)$ in the worst-case for a skewed tree).

Solution

We perform an in-order traversal of the BST which visits nodes in ascending order. During the traversal, we maintain a counter (or decrement k) to check when we have encountered the k th smallest element. When the counter reaches 0 (or k becomes 0), we capture the current node's value as our answer and return it. This approach is both simple and efficient for the given constraints.

Trees & BST

□ #235 Lowest Common Ancestor of a Binary Search Tree

MED

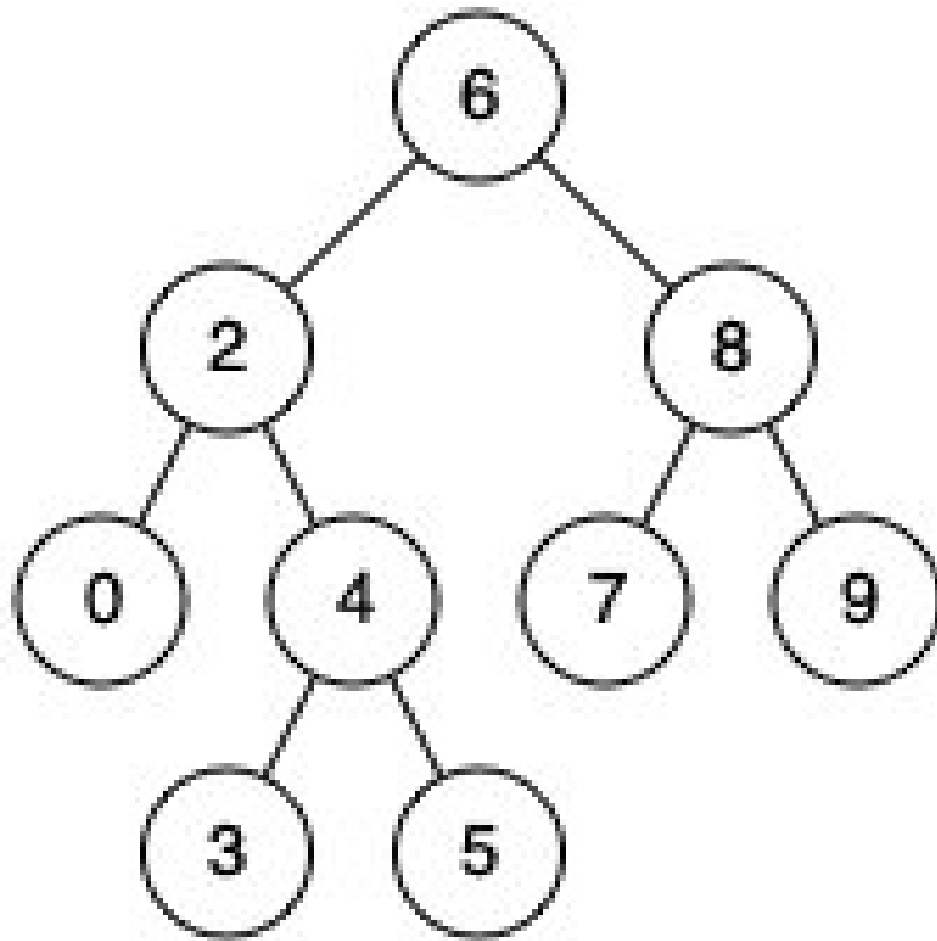
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST
Lists: Grind 169

Problem

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:

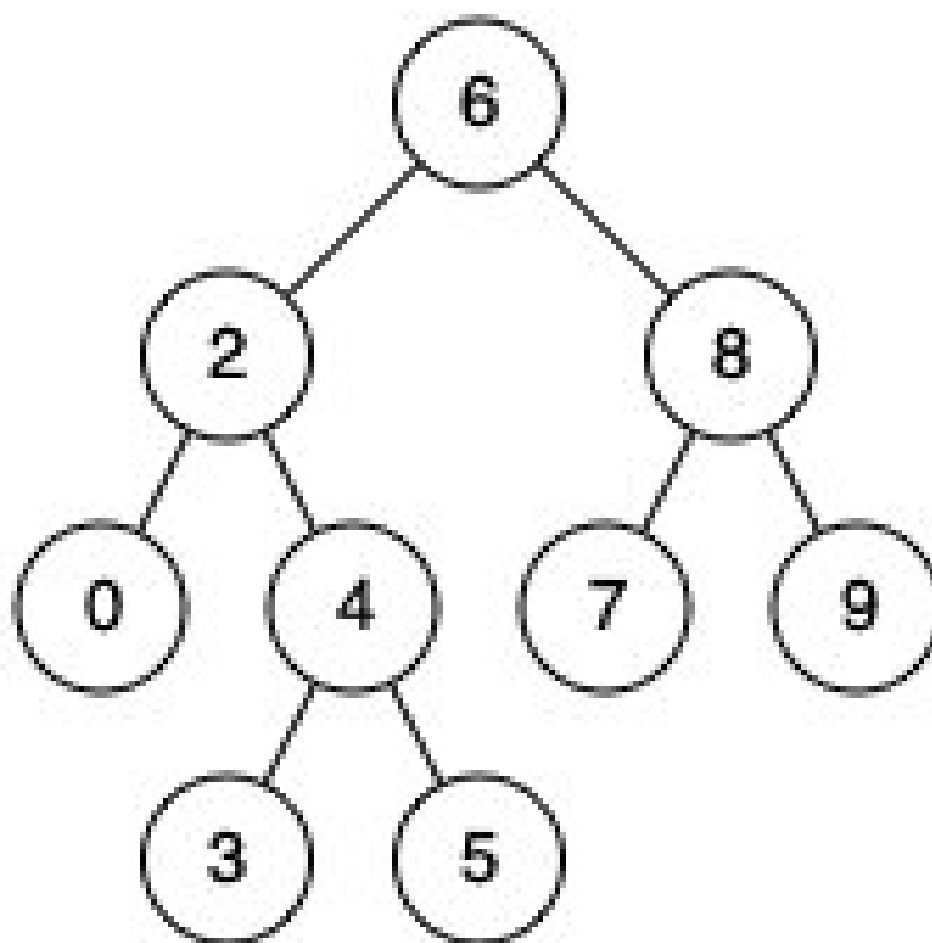


Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

Output: 6

Explanation: The LCA of nodes 2 and 8 is 6.

Example 2:



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

Output: 2

Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [2,1], p = 2, q = 1

Output: 2

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$

- All Node.val are unique.
- $p \neq q$
- p and q will exist in the BST.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "In a BST, find the LCA of nodes p and q.
# LCA = the deepest node that is an ancestor of both p and q (a node is its own
# ancestor). Right?
# Use the BST property: values to the left < root < values to the right."
#
# "[6,2,8,0,4,7,9], p=2, q=8 → LCA=6 (they're on different sides of 6) ✓
# p=2, q=4 → LCA=2 (4 is in 2's right subtree) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Walk from root: if both p and q are smaller, go left; if both larger, go right.
# First node where paths split is the LCA in a BST.
# Alternatively store root-to-p and root-to-q paths – more work than the walk.
# Time: O(h). Space: O(h) if storing paths.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BST property)

```
# "Navigate using BST property:
# If both p and q are LESS than root: LCA is in the LEFT subtree.
# If both are GREATER: LCA is in the RIGHT subtree.
# Otherwise (they split at root, or one equals root): root IS the LCA.
# O(h) time – O(log n) for balanced BST, O(n) worst case."
```

PHASE 4 — CLEAN CODE

```
class _235_LCA_BST:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode')
    -> 'TreeNode':
        while root:
            if p.val < root.val and q.val < root.val:
                root = root.left
            elif p.val > root.val and q.val > root.val:
                root = root.right
```

```

    else:
        return root
    return root

```

PHASE 5 — TEST & DEBUG

root=6, p=2, q=8: both in different sides → return 6 ✓

root=6, p=2, q=4: both <6 → go left to 2. p=2==root, q=4>root → split here → return 2 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(h)$. Space $O(1)$ – iterative, no stack.

The BST property makes this an $O(h)$ walk – no need to visit all nodes.

This is much faster than the general binary tree LCA (#236) which needs DFS.

Given more time I'd ask: what if p or q might not exist in the tree?

We'd need to verify existence first – two separate searches, then LCA."

◆ Quick Review

Key Insights

- Utilize the BST property: for any node, values in the left subtree are less and values in the right subtree are greater.
- If both p and q have values less than the current node, the LCA must lie in the left subtree.
- If both p and q have values greater than the current node, the LCA must lie in the right subtree.
- When one node value is less than and the other is greater than (or equal to) the current node, the current node is the lowest common ancestor.

- An iterative approach is efficient and uses constant extra space.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree ($O(\log n)$ for balanced trees and $O(n)$ in the worst-case skewed tree).

Space Complexity: $O(1)$ with an iterative approach ($O(h)$ if using recursion due to call stack).

Solution

The solution leverages the BST properties.

Algorithm: Start at the root node. While the current node exists: If both p and q have values less than the current node's value, move to the left child. If both p and q have values greater than the current node's value, move to the right child. Otherwise, the current node is the split point where p and q diverge, so it is the lowest common ancestor. Return the current node. This approach efficiently finds the LCA using a single traversal from the root.

Trees & BST

#236 Lowest Common Ancestor of a Binary Tree

MED

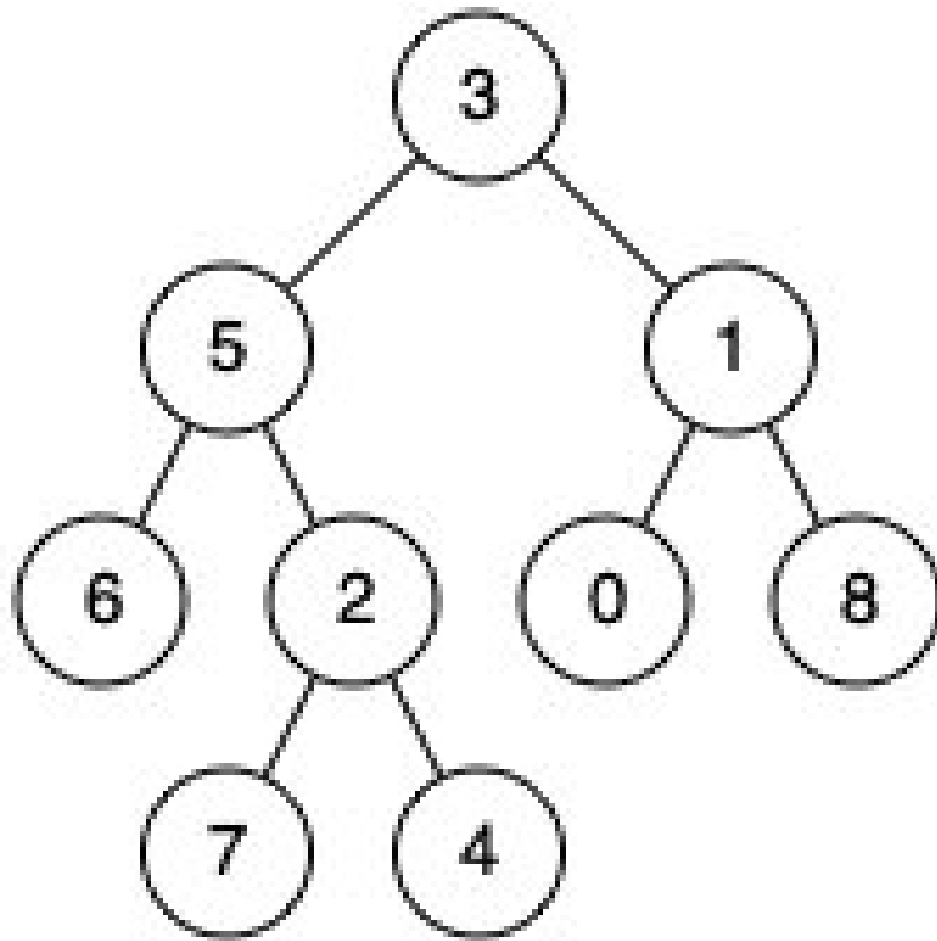
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: Grind 169

Problem

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree. According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:

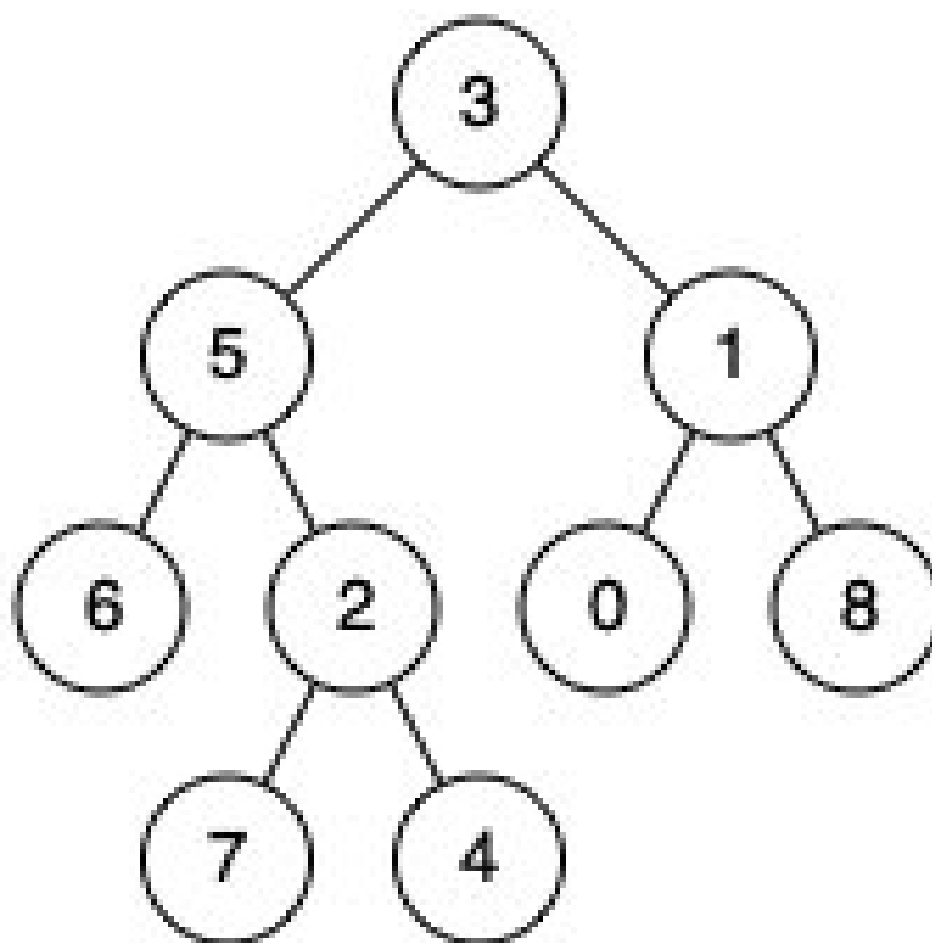


Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

Output: 3

Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

Output: 5

Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$

- All Node.val are unique.
- $p \neq q$
- p and q will exist in the tree.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "General binary tree – no BST property. Find the LCA of p and q.
# p and q are guaranteed to exist in the tree. A node is its own ancestor. Right?"
#
# "[3,5,1,6,2,0,8,null,null,7,4], p=5, q=1 → LCA=3 (root, both are in different
# subtrees) ✓
# p=5, q=4 → LCA=5 (4 is in 5's subtree) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Find root-to-p path and root-to-q path as node lists.
# Walk both lists in parallel; last matching node before divergence is LCA.
# Correct O(n) but needs O(h) path storage; post-order single-pass is cleaner.
# Time: O(n). Space: O(h) two paths.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (recursive post-order)

```
# "DFS: if current node is p or q, return it (it or its ancestor is the LCA).
# Recurse left and right.
# If both return non-null: current node is the LCA (p and q are in different
# subtrees).
# If only one is non-null: that subtree contains both p and q – propagate that
# result up."
```

PHASE 4 — CLEAN CODE

```
class _236_LCA_BinaryTree:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode')
    -> 'TreeNode':
        def lca(node):
            if not node or node is p or node is q:
                return node
            left = lca(node.left)
            right = lca(node.right)
```

```

    if left and right:
        return node # p and q are in different subtrees
    return left or right # both in same subtree – propagate the found node
return lca(root)

```

PHASE 5 — TEST & DEBUG

```

# [3,5,1,...], p=5, q=1:
# lca(3): lca(5)→returns 5 (found p). lca(1)→returns 1 (found q).
# Both non-null → return 3 ✓
# p=5, q=4 (4 is under 5):
# lca(3): lca(5)→... lca(4) under 5 returns 4. lca(5) itself: node is p→return 5.
# lca(1)→None (neither p nor q in right subtree).
# left=5, right=None → return 5 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n) – visits every node. Space O(h) for the call stack.
# Contrast with #235 (BST): BST LCA is O(h) iteratively. General tree needs full
DFS.
# The post-order logic is the key: left and right results tell us which subtrees
contain p/q.
# Both non-null → split here → current node is LCA.
# Only one non-null → both are in the same subtree → propagate that subtree's
result.
# Given more time I'd ask: what if p or q might not exist?
# We'd need to track 'found p' and 'found q' separately – more complex state."

```

◆ Quick Review

Key Insights

- Use a depth-first search (DFS) traversal to explore the tree.
- If the current node is either p or q, it could be the LCA.
- Recursively find p and q in the left and right subtrees.
- If both left and right recursive calls return a non-null value, then the current node is the LCA.

- If only one side returns a non-null value, propagate that value up the recursion.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree since each node is visited once.

Space Complexity: $O(h)$, where h is the height of the tree due to the recursion stack ($O(n)$ in the worst case of a skewed tree).

Solution

We solve the problem using recursion with depth-first search. The algorithm checks if the current node matches p or q . It then recursively searches the left and right subtrees. If both subtrees contain one of p or q , the current node is the LCA. Otherwise, the algorithm forwards the non-null result up the recursion. This approach naturally handles cases where one node is an ancestor of the other.

Trees & BST

□ ★ #250 Count Univalue Subtrees ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

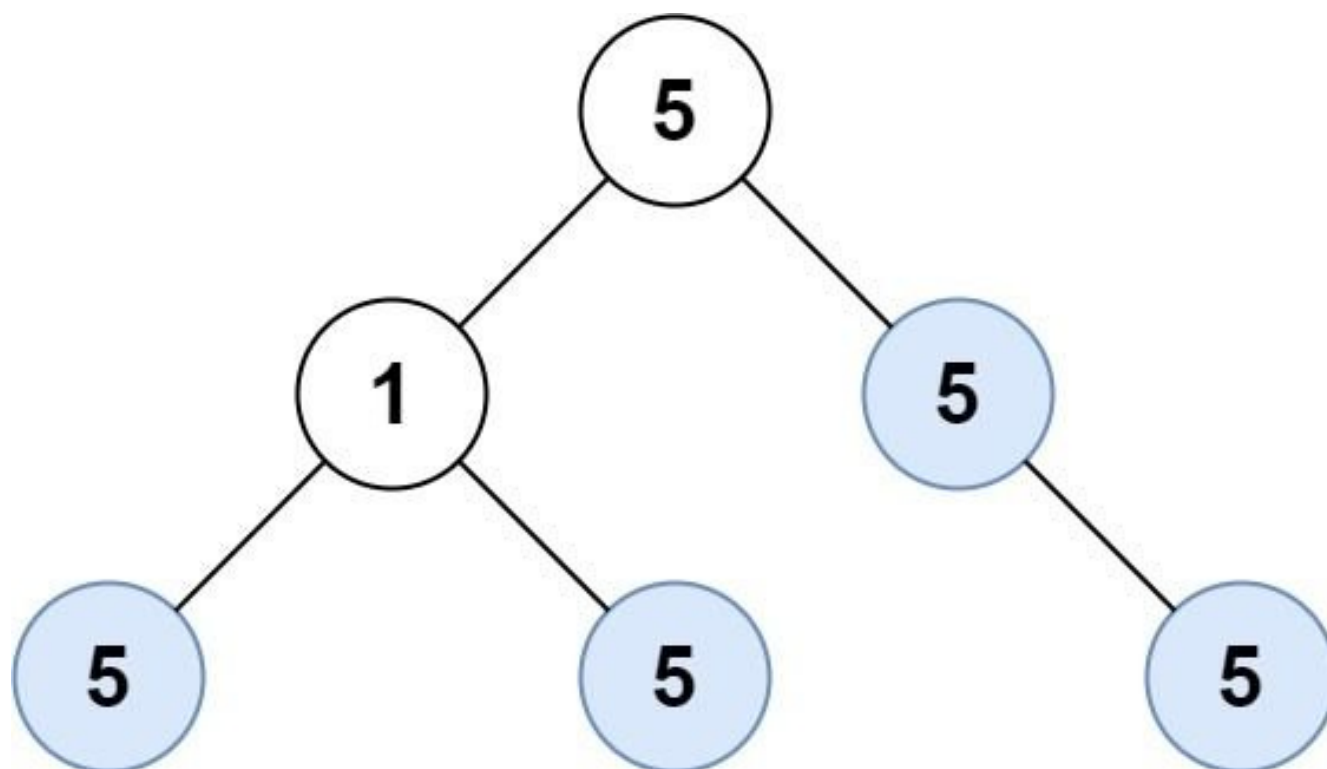
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, return the number of uni-value subtrees.

A uni-value subtree means all nodes of the subtree have the same value.

Example 1:



Input: root = [5,1,5,5,5,null,5]
Output: 4

Example 2:

Input: root = []
Output: 0

Example 3:

Input: root = [5,5,5,5,5,null,5]
Output: 6

Constraints:

- The number of the node in the tree will be in the range [0, 1000].
- $-1000 \leq \text{Node.val} \leq 1000$

My LeetCode Solution
Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "A univalve subtree has every node with the same value.
# Count all such subtrees – including single leaf nodes. Right?
# Empty tree → 0."
#
# "[5,1,5,5,5,null,5] → 4 (the four nodes with value 5 that form valid univalve
# subtrees) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each node, DFS its entire subtree checking every value equals node.val.
# Increment count when the whole subtree is uni-value.
# Revalidates overlapping subtrees –  $O(n^2)$  worst case.
# Time:  $O(n^2)$ . Space:  $O(h)$  stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (post-order DFS)

```
# "Bottom-up: a node is a univalve root if both children are univalve AND their
# values match.
# Return True from a node if its subtree is univalve, increment count when True.
# Use an inner helper that returns bool + mutates a count list."
```

PHASE 4 — CLEAN CODE

```
class _250_CountUnivalveSubtrees:
    def countUnivalSubtrees(self, root: Optional['TreeNode']) -> int:
        count = [0]
        def is_univalve(node, parent_val):
            if not node:
                return True
            left = is_univalve(node.left, node.val)
            right = is_univalve(node.right, node.val)
            if not left or not right:
                return False
            count[0] += 1
            return node.val == parent_val
        is_univalve(root, None)
        return count[0]
```

PHASE 5 — TEST & DEBUG

```
# [5,1,5,5,5,null,5]:
# Leaves: 5(left-left)→True, 5(left-right)→True, 5(right-right)→True, 1→True
# (leaf).
# Node(1): left=True, right=True, both return True. 1's children have val=5≠1 →
# returns False.
```

```
# Node(5, right of root): left=None=True, right(5)=True, right returns True(val
matches). count=1. Returns True(5==root.val?).
# Continue... count ends at 4 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h). Post-order because we need children's results before
parent's."
```

```
# The parent_val parameter lets us check if current node extends the parent's
univalue property.
```

```
# Given more time I'd ask: longest path where all values are the same?
```

```
# Track univalue path length similar to Diameter of Binary Tree – carry length
down."
```

◆ Quick Review

Key Insights

- Perform a postorder traversal (left, right, root) to determine whether each subtree is univalue.
- Treat null nodes as univalue to ease the recursion.
- A node's subtree is univalue if both left and right subtrees are univalue and, if they exist, their values equal the current node's value.
- Use a global or external counter that increments every time a univalue subtree is confirmed.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since each node is visited once.

Space Complexity: $O(h)$, where h is the height of the tree, due to the recursion stack.

Solution

We use a recursive DFS approach (postorder traversal) to solve the problem. For each node, we recursively check its left and right subtrees to decide if they are univalue. By default, if a child is null, it is considered univalue. Then, we verify the current node's value against its non-null children. If both conditions hold (subtrees are univalue and matching values), we count the current node's subtree as univalue. This DFS checks in a bottom-up manner, ensuring that the properties required for a subtree to be univalue are maintained. The key trick is to return a boolean indicating if the subtree at each node is univalue and, while backtracking, increment the counter appropriately.

Trees & BST

□ #285 Inorder Successor in BST

MED

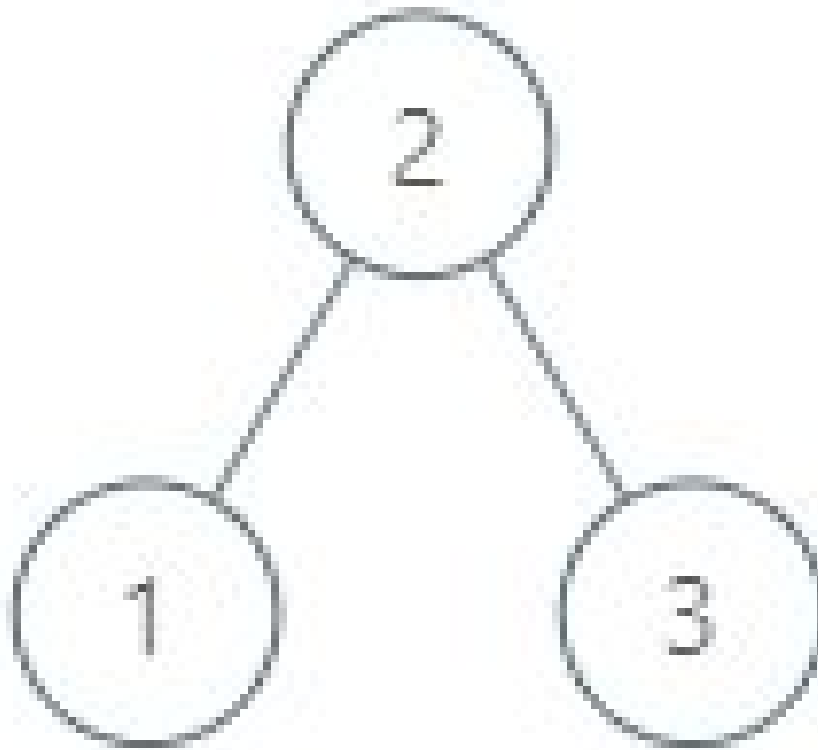
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST
Lists: Grind 169

Problem

Given the root of a binary search tree and a node p in it, return the in-order successor of that node in the BST. If the given node has no in-order successor in the tree, return null.

The successor of a node p is the node with the smallest key greater than $p.val$.

Example 1:

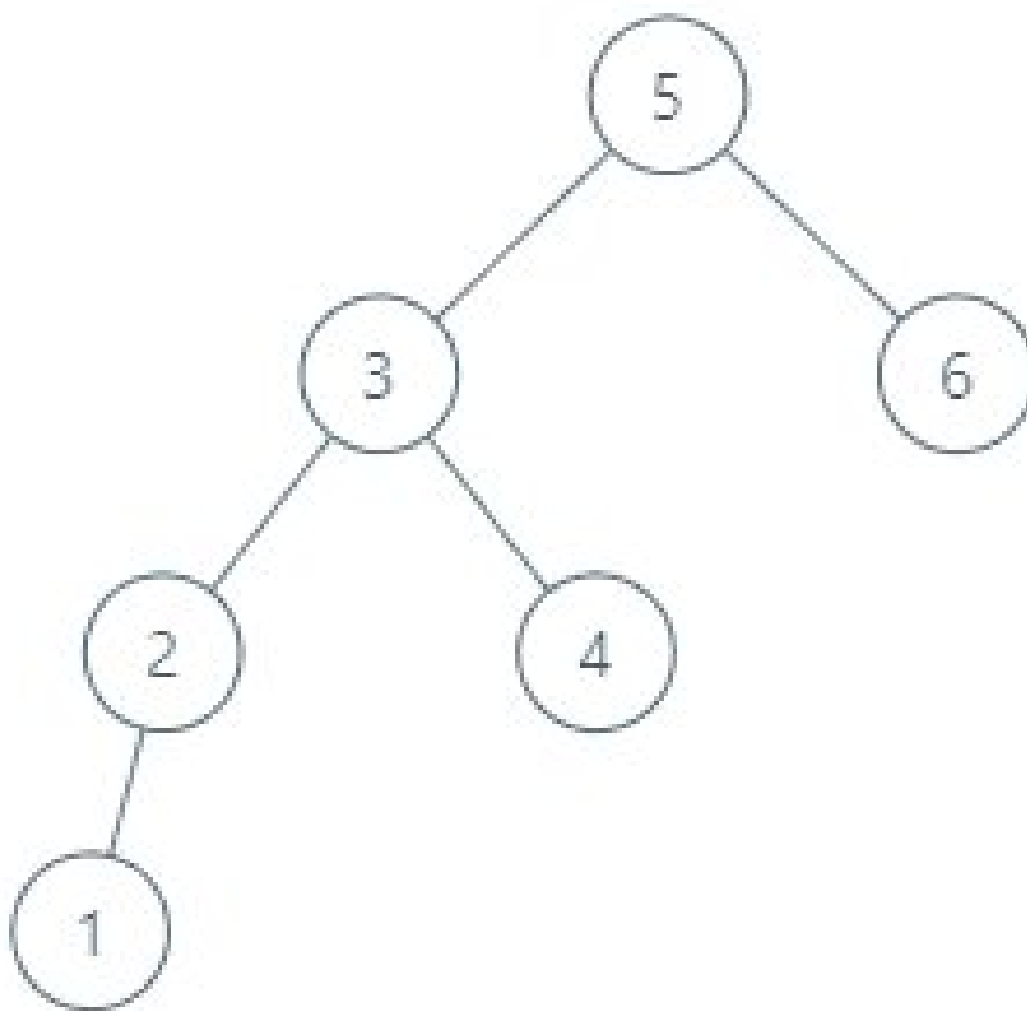


Input: root = [2,1,3], p = 1

Output: 2

Explanation: 1's in-order successor node is 2. Note that both p and the return value is of TreeNode type.

Example 2:



Input: root = [5,3,6,2,4,null,null,1], p = 6

Output: null

Explanation: There is no in-order successor of the current node, so the answer is null.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-105 \leq \text{Node.val} \leq 105$
- All Nodes will have unique values.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the node with the smallest value strictly greater than p.val in a BST.
# Return null if no such node exists (p is the largest). Right?"
#
# "[2,1,3], p=1 → successor=2 (next in in-order: 1,2,3) ✓
# [5,3,6,2,4,null,null,1], p=6 → null (6 is the largest) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# In-order traversal of the BST; when we visit p, the next node in-order is
# successor.
# Store previous node during traversal or collect all values then scan.
# Correct O(n); BST structure gives O(h) successor without full traversal.
# Time: O(n). Space: O(1) iterative with threading concept.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BST navigation)

```
# "Use BST property — no need to traverse all nodes.
# If root.val > p.val: root is a candidate successor, but there might be a smaller
# one in the left subtree.
# If root.val <= p.val: successor must be in the right subtree.
# Track the best candidate seen so far. O(h) time."
```

PHASE 4 — CLEAN CODE

```
class _285_InorderSuccessor:
    def inorderSuccessor(self, root: 'TreeNode', p: 'TreeNode') ->
Optional['TreeNode']:
    successor = None
    while root:
        if root.val > p.val:
            successor = root # root is a candidate — try for smaller in left
            root = root.left
        else:
            root = root.right # successor must be to the right
    return successor
```

PHASE 5 — TEST & DEBUG

```
# [2,1,3], p=1:
# root=2: 2>1 → successor=2, go left.
# root=1: 1<=1 → go right.
# root=None → return 2 ✓
# [5,3,6,2,4,null,null,1], p=6:
# root=5: 5<=6 → go right. root=6: 6<=6 → go right. root=None → return None ✓
```


PHASE 6 — COMPLEXITY & FOLLOW-UP**# "Time $O(h)$. Space $O(1)$ — iterative.**

The candidate tracking is the key: every time we go left, the current node becomes a potential successor (it's bigger than p, and we're looking for something smaller).

Given more time I'd ask: inorder PREDECESSOR (largest value < p.val)?

Mirror logic: if root.val < p.val → candidate, go right. If root.val >= p.val → go left."

◆ Quick Review

Key Insights

- In a BST, an in-order traversal yields nodes in ascending order.
- If the node p has a right subtree, the in-order successor is the left-most node in that right subtree.
- If p has no right subtree, the successor is one of its ancestors; traverse from the root towards p while keeping track of the last node that is greater than p.
- An iterative approach is efficient and avoids recursion stack overhead.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree. In the worst-case (skewed tree), it can be $O(n)$.

Space Complexity: $O(1)$ as only a constant amount of extra space is used.

Solution

We solve the problem using two cases: If node p has a right subtree, we simply find the left-most node in that right subtree. If node p does not have a right subtree, initiate a search from the BST root. As we traverse: If the current node's value is greater than $p.val$, the current node is a candidate for the successor; move to the left subtree to look for a smaller candidate. Otherwise, move to the right subtree. By the end of the traversal, the candidate (if exists) is the in-order successor. This approach benefits from the BST property, limiting traversal to $O(h)$ time.

Trees & BST

□ ★ #298 Binary Tree Longest Consecutive Sequence ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: ★ Premium | Premium 98

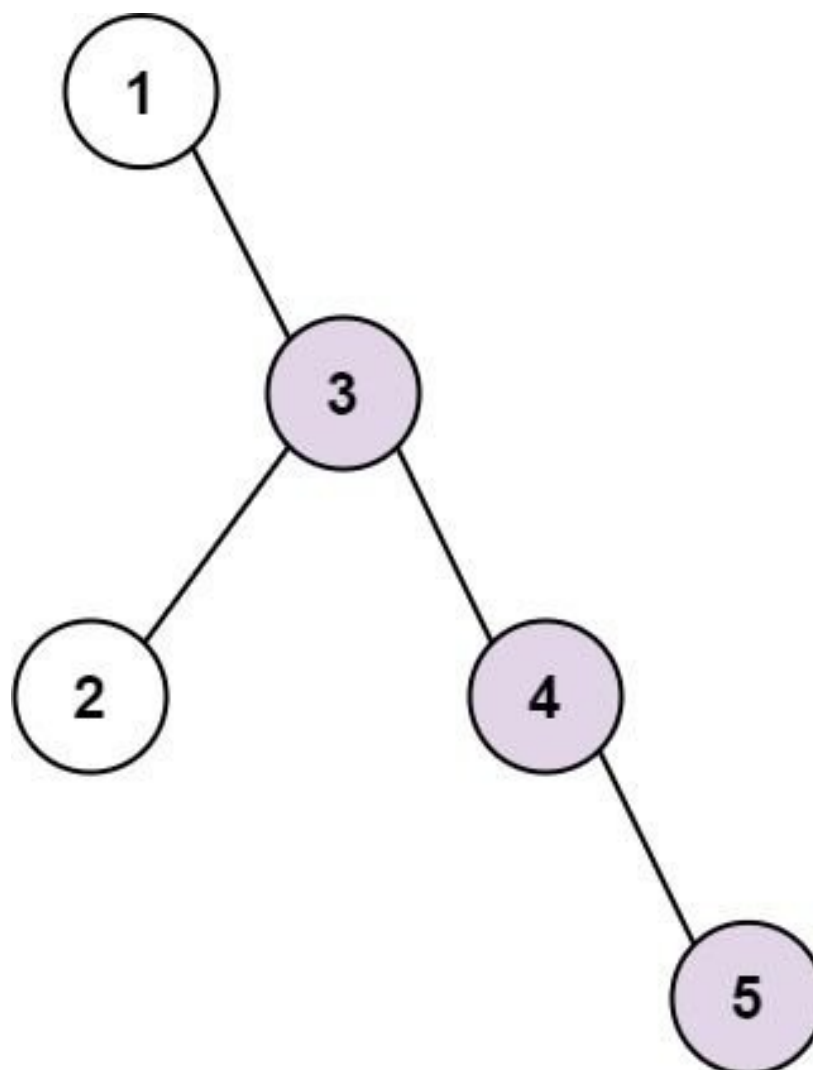
Problem

Given the root of a binary tree, return the length of the longest consecutive sequence path.

A consecutive sequence path is a path where the values increase by one along the path.

Note that the path can start at any node in the tree, and you cannot go from a node to its parent in the path.

Example 1:

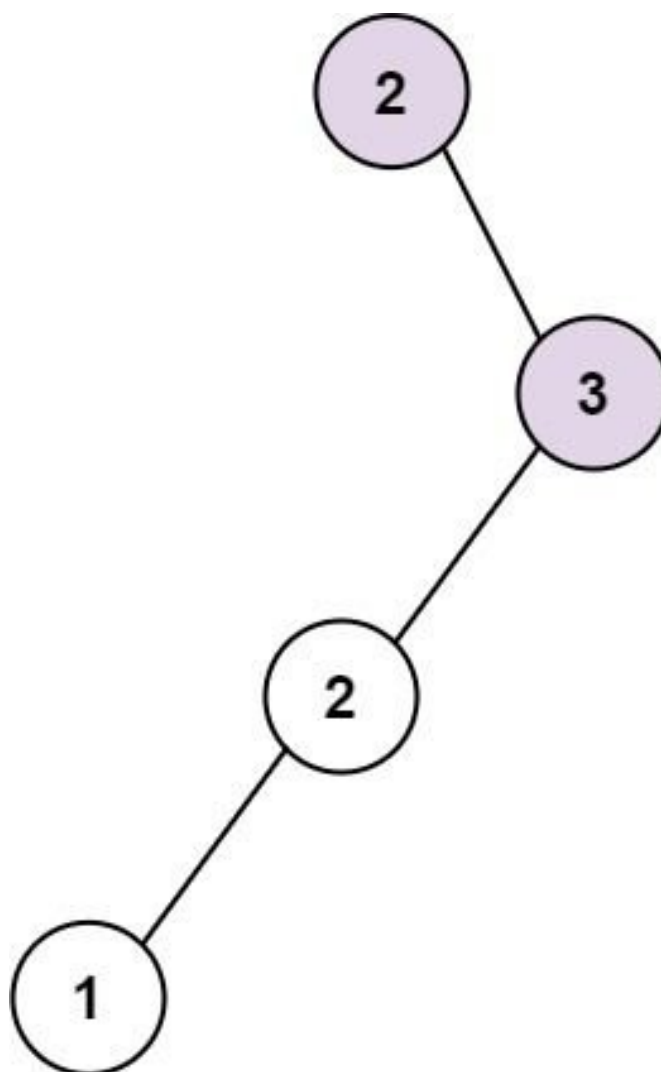


Input: root = [1,null,3,2,4,null,null,null,5]

Output: 3

Explanation: Longest consecutive sequence path is 3-4-5, so return 3.

Example 2:



Input: root = [2,null,3,2,null,1]

Output: 2

Explanation: Longest consecutive sequence path is 2-3, not 3-2-1, so return 2.

Constraints:

- The number of nodes in the tree is in the range $[1, 3 * 10^4]$.
- $-3 * 10^4 \leq \text{Node.val} \leq 3 * 10^4$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the longest path of consecutive increasing values (each node = parent + 1).
# Path goes parent → child – you can't go back up. Path can start at any node.
Right?"
#
# "[1,null,3,2,4,null,null,null,5]: path 3→4→5, length 3 ✓
# [2,null,3,2,null,1]: 2→3 is increasing, not 3→2 – so max is 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# From each node, DFS tracking consecutive length when child.val == parent.val + 1.
# Reset streak when the +1 chain breaks; track global maximum.
# Starting DFS from every node repeats work – post-order aggregation is faster.
# Time: O(n^2) naive restarts. Space: O(h) stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (DFS carrying current length)

```
# "DFS top-down: pass current consecutive length to each child.
# If child.val == parent.val + 1: extend the sequence.
# Else: reset to 1 (start a new sequence from this child).
# Track global max. Use inner helper with parent_val and current_length."
```

PHASE 4 — CLEAN CODE

```
class _298_LongestConsecutive:
    def longestConsecutive(self, root: Optional['TreeNode']) -> int:
        best = [0]
        def dfs(node, parent_val, length):
            if not node:
                return
            length = length + 1 if node.val == parent_val + 1 else 1
            best[0] = max(best[0], length)
            dfs(node.left, node.val, length)
            dfs(node.right, node.val, length)
        dfs(root, root.val - 1 if root else 0, 0)
        return best[0]
```

PHASE 5 — TEST & DEBUG

```
# [1,null,3,2,4,null,null,null,5]:
# dfs(1, 0, 0): val=1, prev+1=1 → length=1. best=1. dfs(3,1,1).
# dfs(3, 1, 1): val=3, prev+1=2, 3≠2 → length=1. best=1. dfs(2,3,1).
# dfs(4, 3, 1): val=4==3+1 → length=2. best=2. dfs(5,4,2).
# dfs(5, 4, 2): val=5==4+1 → length=3. best=3. → 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h).
# The initial call uses root.val-1 so the root itself resets to length 1 cleanly."
```

```
# This is a top-down DFS – pass state downward. Contrast with bottom-up (#250, #543).  
# Given more time I'd ask: what if the path can go through any node (not just parent-child)?  
# That's Longest Consecutive Sequence in a Binary Tree – post-order, track both ascending  
# and descending sequences at each node."
```

◆ Quick Review

Key Insights

- Use a Depth-First Search (DFS) to traverse the binary tree.
- Pass the current node's value and the length of the current consecutive sequence in the recursive call.
- When a node's value is exactly one greater than its parent's value, increment the sequence length; otherwise, reset it to 1.
- Keep track of the maximum sequence length found during the traversal.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree, since each node is visited once.

Space Complexity: $O(H)$, where H is the height of the tree. In the worst-case of a skewed tree, the recursion stack can be $O(N)$.

Solution

The solution involves using DFS to explore every node in the binary tree while keeping track of the length of the consecutive sequence. For each node, if the node's value is one greater than its parent's value, we increment the sequence length; otherwise, we reset the sequence length to 1. During the traversal, a global variable is updated if the current sequence length exceeds the previously recorded maximum. This approach ensures that every possible consecutive path is considered.

Trees & BST

□ ★ #314 Binary Tree Vertical Order Traversal ★

MED

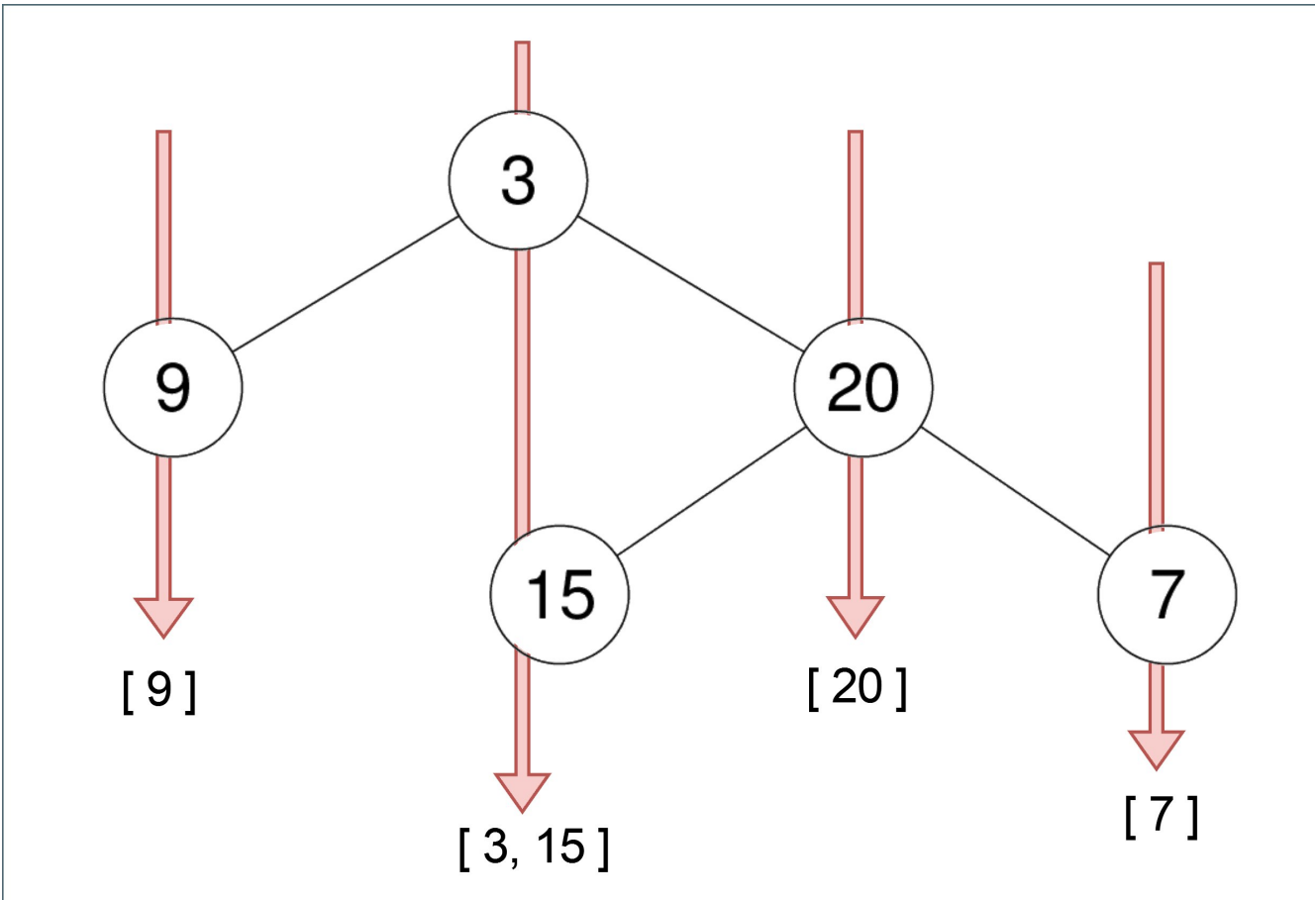
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Tree · BFS · Sorting
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, return the vertical order traversal of its nodes' values. (i.e., from top to bottom, column by column).

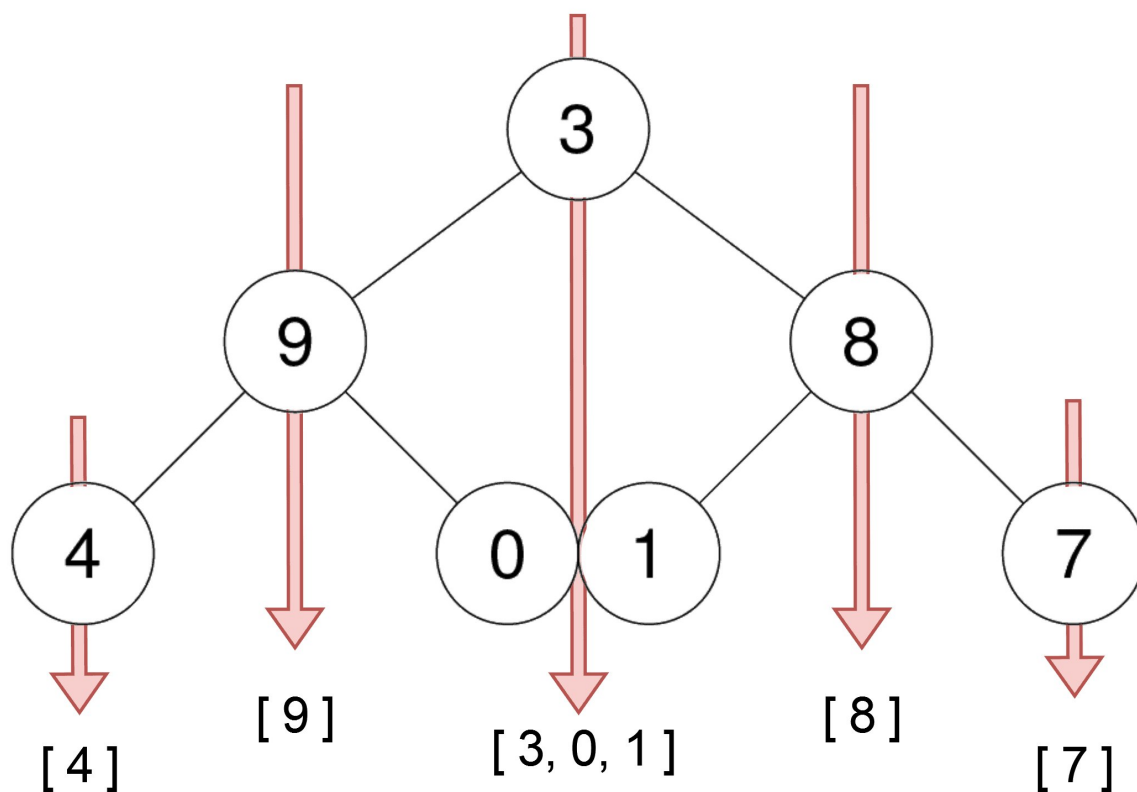
If two nodes are in the same row and column, the order should be from left to right.

Example 1:



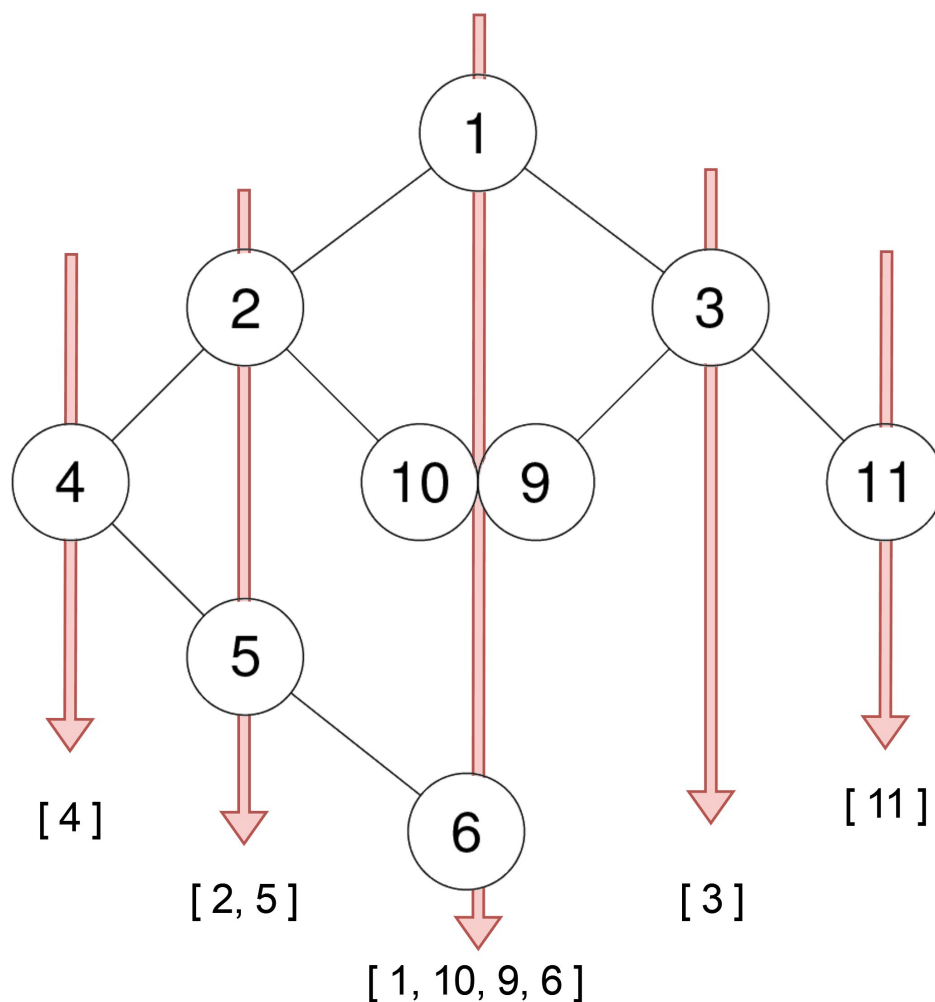
Input: root = [3,9,20,null,null,15,7]
Output: [[9],[3,15],[20],[7]]

Example 2:



Input: root = [3,9,8,4,0,1,7]
Output: [[4],[9],[3,0,1],[8],[7]]

Example 3:



Input: root = [1,2,3,4,10,9,11,null,5,null,null,null,null,null,null,6]
 Output: [[4],[2,5],[1,10,9,6],[3],[11]]

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Group nodes by column. Root is column 0. Left child = col-1. Right child = col+1.
# Within each column, top-to-bottom. Ties (same row and column): left before right.
# Right?
# Return columns sorted from leftmost to rightmost."
#
# "[3,9,20,null,null,15,7] → [[9],[3,15],[20],[7]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# DFS assigning column index (left=-1, right=+1 relative to parent).
# Collect (col, row, val) tuples; sort by (col, row, val) then group by column.
# Correct but sorting dominates at O(n log n).
# Time: O(n log n). Space: O(n) tuples.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BFS with column tracking)

```
# "BFS ensures natural top-to-bottom, left-to-right order within each column.
# Queue entries: (node, col). Use defaultdict(list) keyed by col.
# After BFS, sort the column keys and return their lists.
# Time: O(n log n) for sorting. Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _314_VerticalOrderTraversal:
    def verticalOrder(self, root: Optional['TreeNode']) -> List[List[int]]:
        if not root:
            return []
        cols : defaultdict = defaultdict(list)
        queue = deque([(root, 0)])
        min_col = max_col = 0
        while queue:
            node, col = queue.popleft()
            cols[col].append(node.val)
            min_col = min(min_col, col)
            max_col = max(max_col, col)
            if node.left: queue.append((node.left, col - 1))
            if node.right: queue.append((node.right, col + 1))
        return [cols[c] for c in range(min_col, max_col + 1)]
```

PHASE 5 — TEST & DEBUG

```
# [3,9,20,null,null,15,7]:
# BFS: (3,0)→cols={0:[3]}. (9,-1),(20,1)→cols={0:[3],-1:[9],1:[20]}.
# (15,0),(7,2)→cols={0:[3,15],-1:[9],1:[20],2:[7]}.
# min=-1, max=2. Return [cols[-1],cols[0],cols[1],cols[2]] = [[9],[3,15],[20],[7]]
✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n) for BFS + O(cols) for building the result – total O(n)."
```

```
# Using min_col/max_col avoids sorting keys – the range is contiguous.  
# BFS guarantees top-to-bottom, left-to-right order within each column naturally.  
# DFS would require explicit (row, insertion_order) sorting per column.  
# Given more time I'd ask: Vertical Order Traversal II (#987)?  
# Same column/row but within the same position, sort by value ascending – needs  
explicit row tracking."
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to traverse the tree level by level.
- Track the column index for each node where root is column 0, left child is column-1, and right child is column+1.
- Group nodes by their column indices using a hash table (or dictionary).
- BFS traversal naturally handles left-to-right ordering within the same level.
- Finally, sort the keys (column indices) to generate results from leftmost column to rightmost column.

Space & Time

Time Complexity: $O(n \log n)$ in the worst-case due to sorting the column indices (n nodes in the worst-case if each node is on a unique column).

Space Complexity: $O(n)$ for storing the node information and the resultant structure.

Solution

The solution uses BFS with a queue that stores nodes along with their corresponding column index. For each visited node, add its value to a dictionary keyed by its column index. When processing children, update the column index appropriately (left child: $col-1$, right child: $col+1$). After the traversal, sort the keys of the dictionary and compile the node values in order from smallest to largest column index. This approach guarantees that nodes at the same level will be processed in left-to-right order, matching the required output.

Trees & BST

□ ★ #333 Largest BST Subtree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Tree · DFS · BST
Lists: ★ Premium | Premium 98

Problem

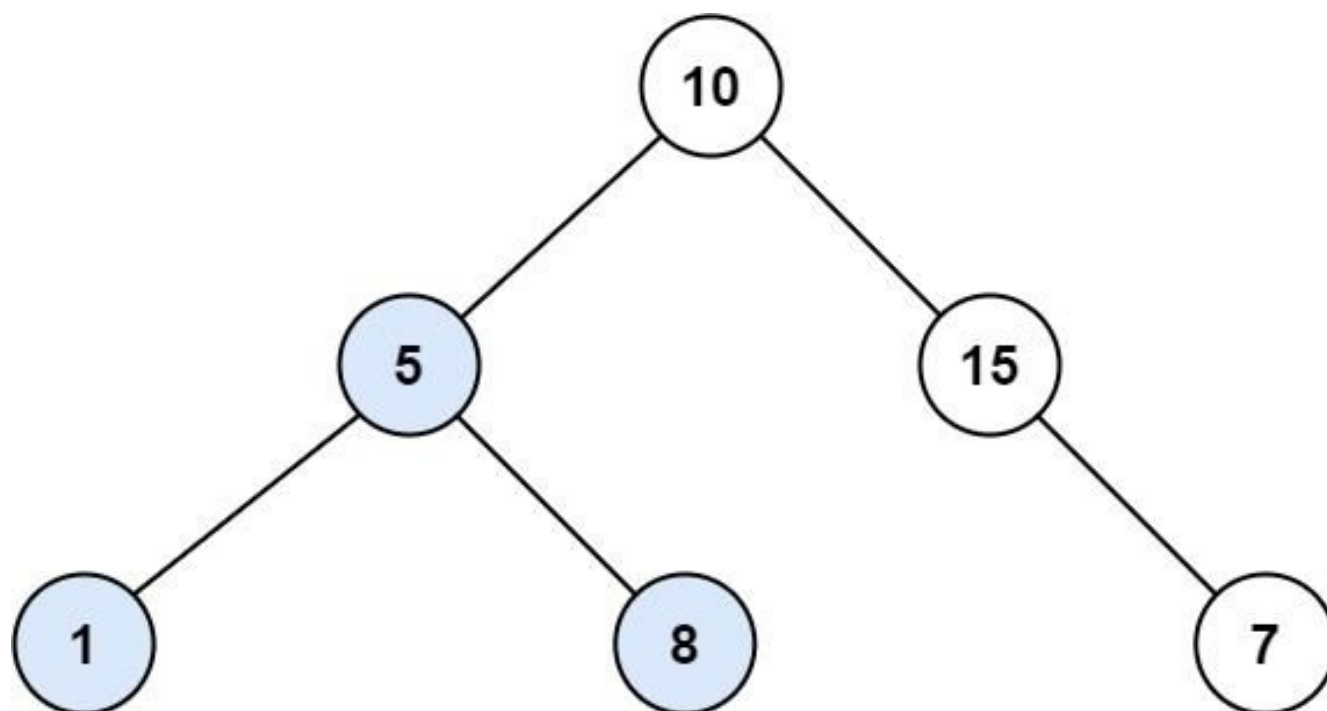
Given the root of a binary tree, find the largest subtree, which is also a Binary Search Tree (BST), where the largest means subtree has the largest number of nodes.

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties:

- The left subtree values are less than the value of their parent (root) node's value.
- The right subtree values are greater than the value of their parent (root) node's value.

Note: A subtree must include all of its descendants.

Example 1:



Input: root = [10,5,15,1,8,null,7]

Output: 3

Explanation: The Largest BST Subtree in this case is the highlighted one. The return value is the subtree's size, which is 3.

Example 2:

Input: root = [4,2,7,2,3,5,null,2,null,null,null,null,null,1]

Output: 2

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-104 \leq \text{Node.val} \leq 104$

Follow up: Can you figure out ways to solve it with $O(n)$ time complexity?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the largest subtree (by node count) that forms a valid BST.
# Return the node count. Right?"
#
# "[10,5,15,1,8,null,7]: subtree at 5 (nodes 1,5,8) is a BST → 3 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For every node as root of a candidate subtree, validate BST property recursively
# (min/max bounds) and count nodes if valid. Track global max count.
# Revalidates overlapping subtrees –  $O(n^2)$  worst case.
# Time:  $O(n^2)$ . Space:  $O(h)$  stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE ($O(n)$ post-order)

```
# "Post-order: return (is_bst, size, subtree_min, subtree_max).
# Empty node: (True, 0, +inf, -inf) – sentinels pass any comparison cleanly.
# Valid BST: both children are BST AND left.max < node.val < right.min."
```

PHASE 4 — CLEAN CODE

```
class _333_LargestBSTSubtree:
    def largestBSTSubtree(self, root: Optional['TreeNode']) -> int:
        best = [0]
        def check(node):
            if not node:
                return True, 0, float('inf'), float('-inf')
            l_ok, l_sz, l_mn, l_mx = check(node.left)
            r_ok, r_sz, r_mn, r_mx = check(node.right)
            if l_ok and r_ok and l_mx < node.val < r_mn:
                sz = l_sz + r_sz + 1
                best[0] = max(best[0], sz)
                return True, sz, min(l_mn, node.val), max(r_mx, node.val)
            return False, 0, 0, 0
        check(root)
        return best[0]
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(h)$ . Sentinels (+inf/-inf for empty) let comparisons pass
cleanly.
# Given more time I'd ask: what if we want the root of the largest BST subtree?
Track it alongside size."
```

◆ Quick Review

Key Insights

- Use a bottom-up DFS (post-order traversal) to process each node.
- For each node, determine whether its left and right subtrees are BSTs.
- Maintain additional data per subtree: size (number of nodes), minimum value, and maximum value.
- A subtree rooted at the current node is a BST if:
 - Its left subtree is a BST.
 - Its right subtree is a BST.
 - The maximum value in the left subtree is less than the current node's value.
 - The current node's value is less than the minimum value in the right subtree.
- Track the size of the largest BST identified during the traversal.

Space & Time

Time Complexity: $O(n)$ (each node is visited once)

Space Complexity: $O(h)$, where h is the height of the tree (due to recursion stack)

Solution

Perform a post-order traversal of the binary tree. For each node, return a tuple (or object in some languages) that provides: Whether the subtree rooted at this node is a BST. The size of the subtree if it is a BST (or the size of the largest BST found within it). The minimum value in the subtree. The maximum value in the subtree. At each node, if both left and right children form valid BSTs and the current node's value fits the BST condition (greater than left's max and less than right's min), then the subtree rooted at the node is also a BST. Calculate its size and update the current global maximum if necessary. If the condition fails, return the maximum BST size found in its subtrees.

Trees & BST

★ #366 Find Leaves of Binary Tree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · Sorting

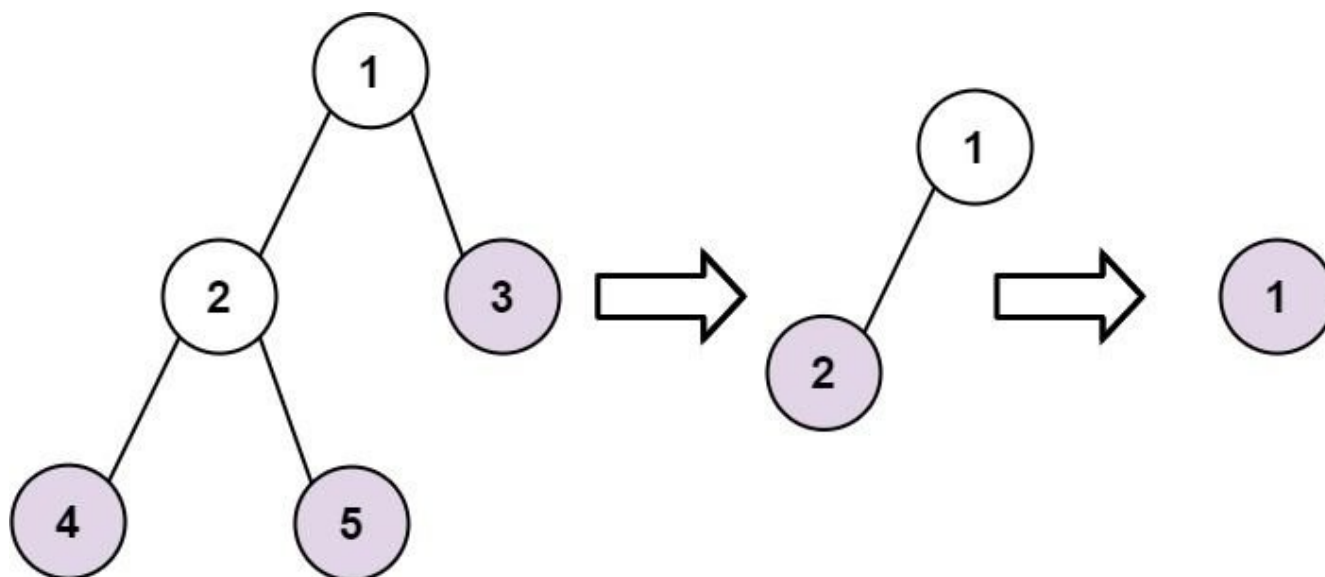
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, collect a tree's nodes as if you were doing this:

- Collect all the leaf nodes.
- Remove all the leaf nodes.
- Repeat until the tree is empty.

Example 1:



Input: root = [1,2,3,4,5]

Output: [[4,5,3],[2],[1]]

Explanation:

[[3,5,4],[2],[1]] and [[3,4,5],[2],[1]] are also considered correct answers since per each level it does not matter the order on which elements are returned.

Example 2:

Input: root = [1]

Output: [[1]]

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Collect leaves, remove them, repeat. Group nodes by their 'height from leaf' level. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Repeatedly find all current leaves (no children), remove them, append to result level.

Each round peels one tree layer – like reverse level-order.

Time: $O(n^2)$ if we rescan whole tree each round. Space: $O(n)$ output.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Post-order height: leaf=0, internal=max(left,right)+1. Group by height in one pass."

PHASE 4 — CLEAN CODE

class _366_FindLeaves:

def findLeaves(self, root: Optional['TreeNode']) -> List[List[int]]:

```

result: List[List[int]] = []
def height(node) -> int:
    if not node:
        return -1
    h = max(height(node.left), height(node.right)) + 1
    if h == len(result):
        result.append([])
    result[h].append(node.val)
    return h
height(root)
return result

```

PHASE 5 — TEST & DEBUG

[1,2,3,4,5]: height(4)=0→result=[[4]]. height(5)=0→[[4,5]].

height(2)=1→[[4,5],[2]].

height(3)=0→[[4,5,3],[2]]. height(1)=2→[[4,5,3],[2],[1]] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. One DFS pass – no repeated tree traversals.

Given more time I'd ask: group by depth from root? That's standard level-order BFS."

◆ Quick Review

Key Insights

- Use a postorder traversal (bottom-up approach) to process the tree.
- Determine the "height" of each node (distance from the deepest leaf) where leaves have a height of 1.
- Group nodes by their computed height to simulate the layer-wise removal of leaves.
- The same node may become a leaf after its children are removed.

Space & Time

Time Complexity: $O(n)$ — Each node is visited exactly once.

Space Complexity: $O(n)$ — Space used for the recursion stack and the result list.

Solution

The solution uses a recursive helper function that performs a postorder traversal of the tree. For each node, it computes its height as $\max(\text{height of left, height of right}) + 1$. Nodes with the same height are collected together in a list. This idea mirrors the process of removing leaves level by level, where leaves (height 1) are removed first, then their parents become new leaves, and so on. Data Structures used: An array or list to maintain the groups of node values by their height. Algorithmic Approach: Use Depth-First Search (DFS) via recursion. Postorder traversal ensures that leaf nodes (base cases) are processed before their parents. Append the node's value to the correct list based on the computed height.

Trees & BST

□ #437 Path Sum III

MED

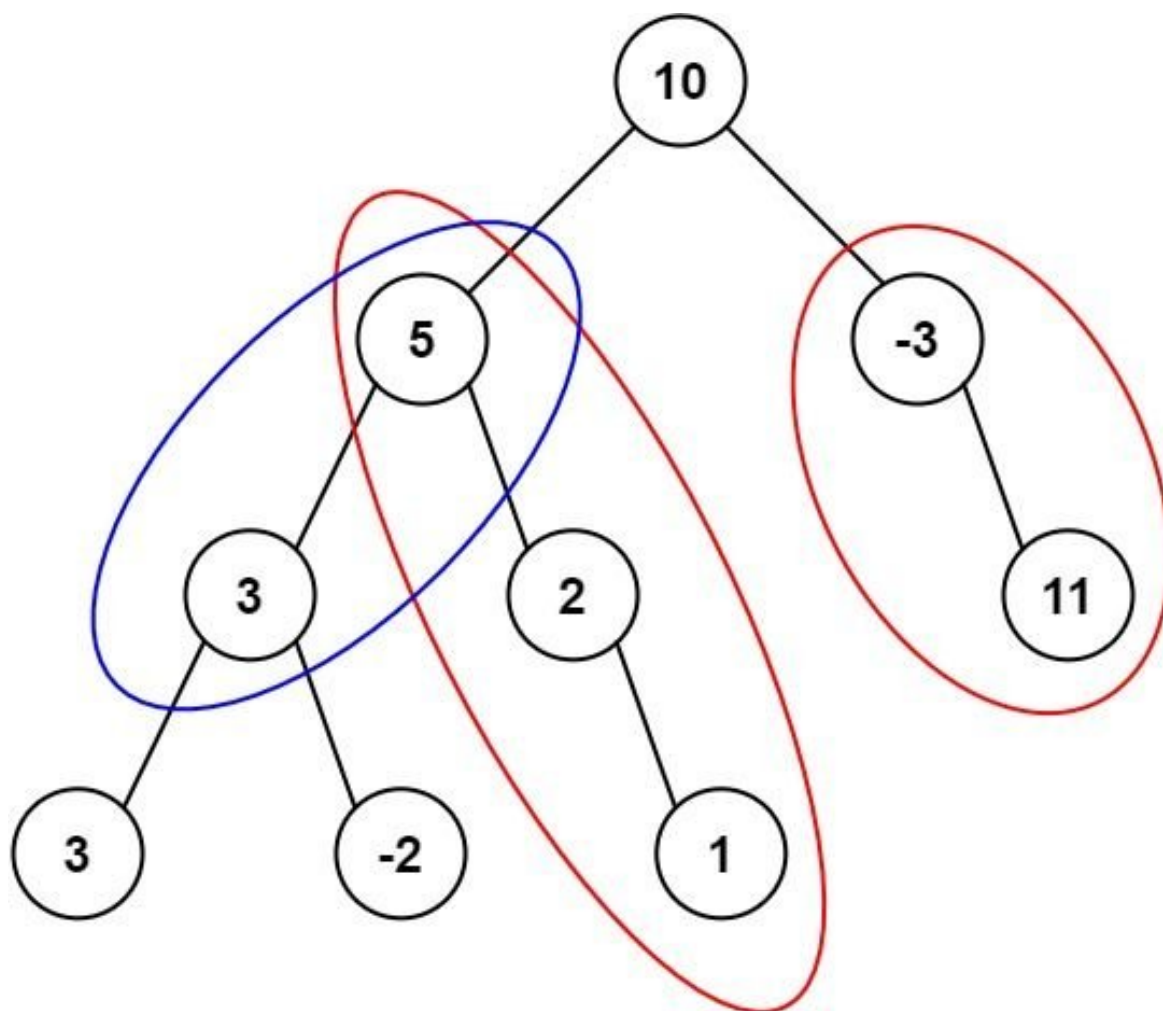
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · Prefix Sum
Lists: Grind 169

Problem

Given the root of a binary tree and an integer targetSum, return the number of paths where the sum of the values along the path equals targetSum.

The path does not need to start or end at the root or a leaf, but it must go downwards (i.e., traveling only from parent nodes to child nodes).

Example 1:



Input: root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8

Output: 3

Explanation: The paths that sum to 8 are shown.

Example 2:

Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22

Output: 3

Constraints:

- The number of nodes in the tree is in the range [0, 1000].
- $-109 \leq \text{Node.val} \leq 109$

- $-1000 \leq \text{targetSum} \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count downward paths (parent→child only) summing to targetSum.
# Path can start and end at any node. Values can be negative. Right?"
#
# "[10,5,-3,3,2,null,11,3,-2,null,1], target=8 → 3 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Prefix-sum map: as DFS walks, track cumulative sum from root.
# At each node, add count of prefix sums equal to (current_sum - target) - includes
# paths through node.
# Backtrack prefix counts when leaving node.
# Time: O(n). Space: O(n) prefix map + stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (prefix sum + hash map — same pattern as #560)

```
# "DFS with running prefix sum. Count how many earlier prefix sums equal
# current_sum-target.
# Backtrack after each subtree to avoid polluting sibling paths. Seed {0:1}."
```

PHASE 4 — CLEAN CODE

```
class _437_PathSumIII:
    def pathSum(self, root: Optional['TreeNode'], targetSum: int) -> int:
        prefix = Counter({0: 1})
        count = [0]
        def dfs(node, running):
            if not node:
                return
            running += node.val
            count[0] += prefix[running - targetSum]
            prefix[running] += 1
            dfs(node.left, running)
            dfs(node.right, running)
            prefix[running] -= 1 # backtrack
        dfs(root, 0)
        return count[0]
```

PHASE 5 — TEST & DEBUG

This is #560 applied to trees – same prefix-sum logic, but backtrack after each subtree.

root=10, target=8: paths [5,3], [5,2,1], [-3,11] each sum to 8 → count=3 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Identical to Subarray Sum Equals K applied to a tree.

Backtracking prevents cross-subtree contamination – critical difference from array version.

Given more time I'd ask: paths going in both directions (not just downward)?

Use the any-direction diameter-style post-order approach."

◆ Quick Review

Key Insights

- A path can start and end at any nodes, as long as it goes downwards.
- Using DFS helps traverse the tree and explore all possible paths.
- A prefix sum technique with a hash map allows quick calculation of the sum of any subpath.
- Backtracking is used to remove a path's prefix sum when moving back up the recursion tree.
- Although a brute-force approach exists, it is inefficient compared to the prefix sum method.

Space & Time

Time Complexity: $O(n)$ on average (where n is the number of nodes), though worst-case can

be $O(n^2)$ in a skewed tree.

Space Complexity: $O(n)$ due to recursion stack and the hash map that stores prefix sums.

Solution

The solution employs a DFS traversal of the tree along with a hash map that records the frequency of prefix sums encountered so far. At each node, the current prefix sum is updated by adding the node's value. We then check the hash map for how many times (current prefix sum – targetSum) has occurred, since this value indicates the existence of a subpath summing to targetSum. We update the hash map with the current prefix sum, continue the DFS to traverse child nodes, and then backtrack by decrementing the current prefix sum's count before retreating to the parent node. This ensures that sums from other branches are not incorrectly combined.

Trees & BST

□ ★ #545 Boundary of Binary Tree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: ★ Premium | Premium 98

Problem

The boundary of a binary tree is the concatenation of the root, the left boundary, the leaves ordered from left-to-right, and the reverse order of the right boundary.

The left boundary is the set of nodes defined by the following:

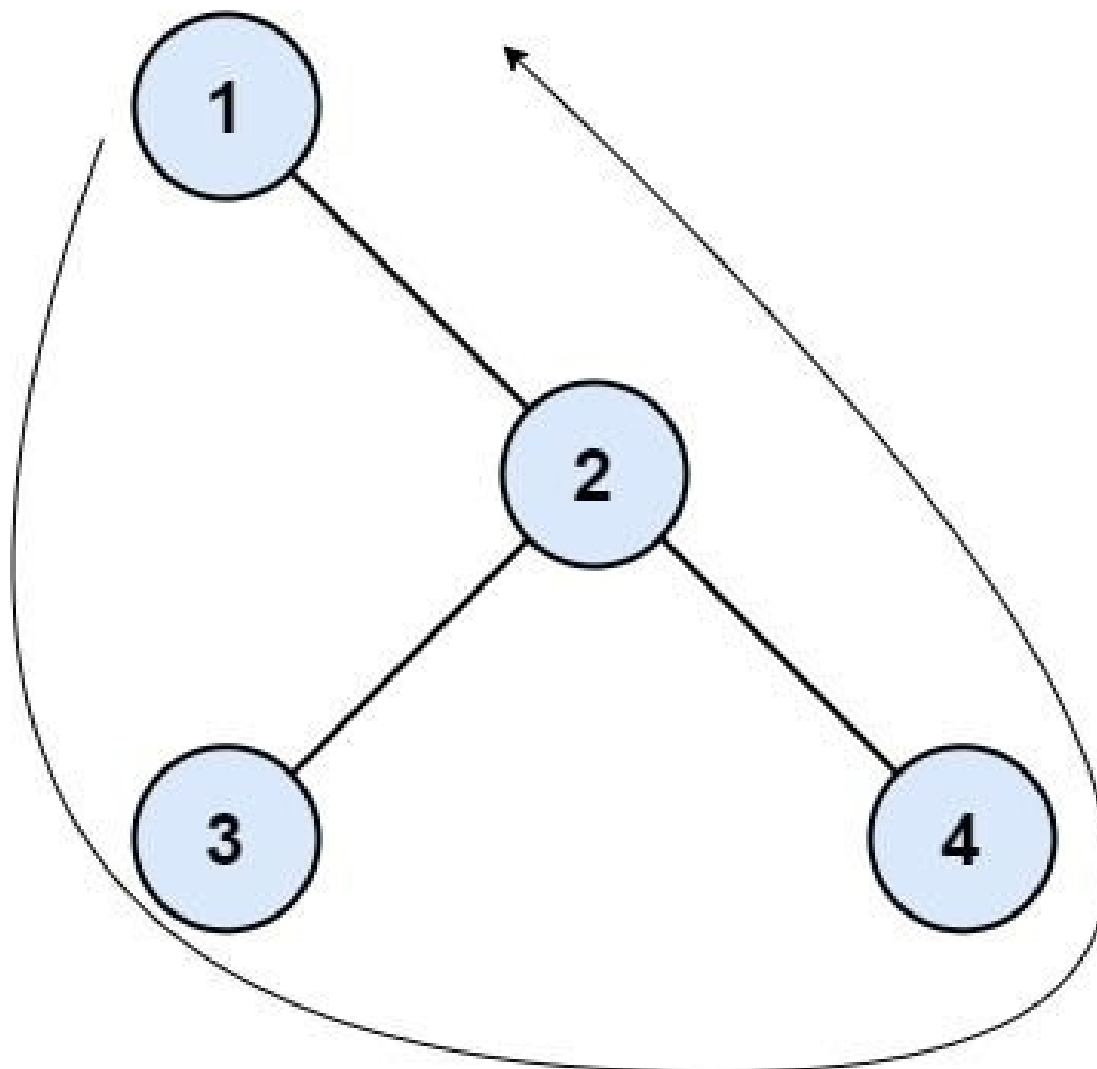
- The root node's left child is in the left boundary. If the root does not have a left child, then the left boundary is empty.
- If a node is in the left boundary and has a left child, then the left child is in the left boundary.
- If a node is in the left boundary, has no left child, but has a right child, then the right child is in the left boundary.

- The leftmost leaf is not in the left boundary.

The right boundary is similar to the left boundary, except it is the right side of the root's right subtree. Again, the leaf is not part of the right boundary, and the right boundary is empty if the root does not have a right child.

The leaves are nodes that do not have any children. For this problem, the root is not a leaf. Given the root of a binary tree, return the values of its boundary.

Example 1:



Input: root = [1,null,2,3,4]

Output: [1,3,4,2]

Explanation:

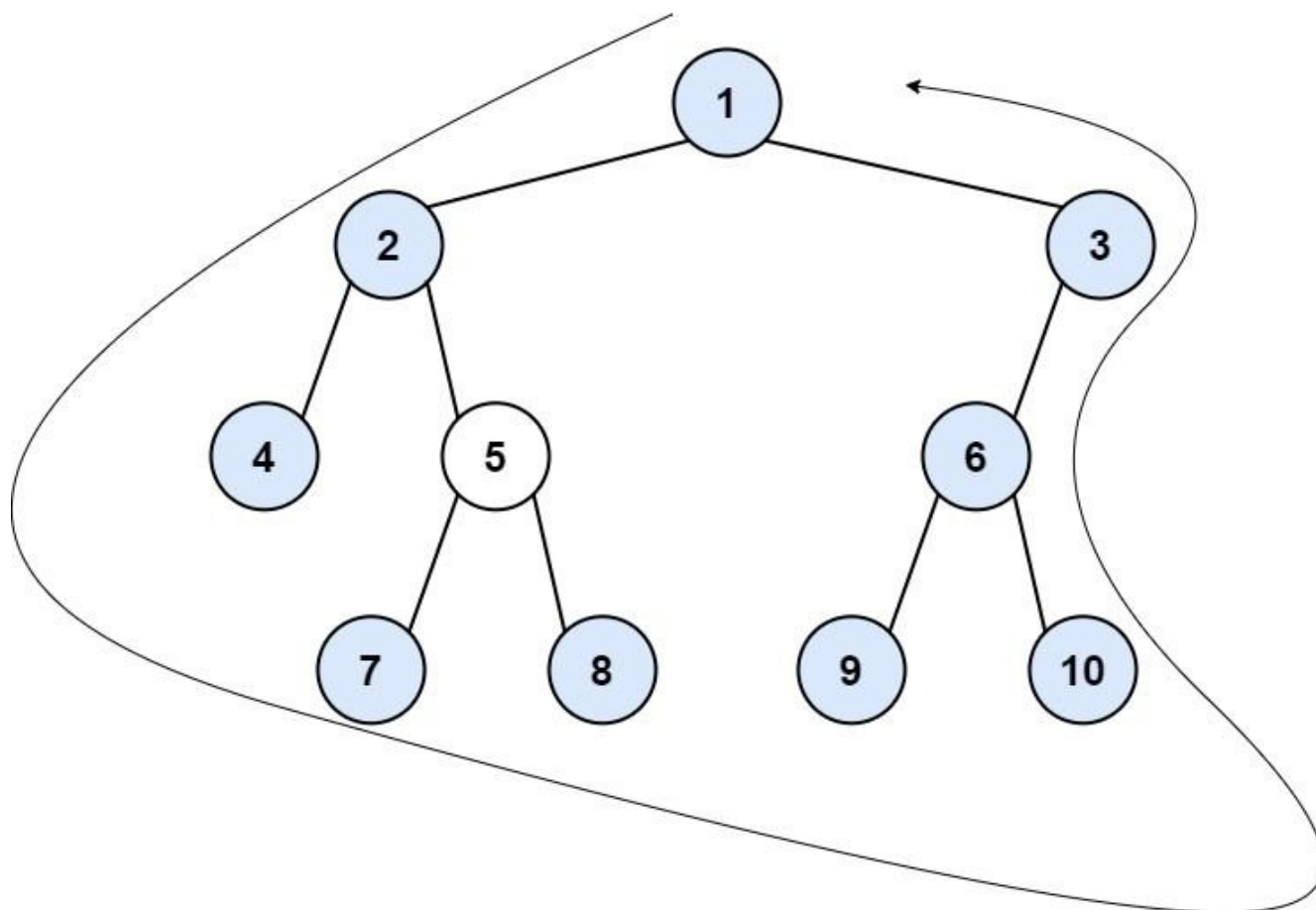
- The left boundary is empty because the root does not have a left child.
- The right boundary follows the path starting from the root's right child 2 -> 4.

4 is a leaf, so the right boundary is [2].

- The leaves from left to right are [3,4].

Concatenating everything results in [1] + [] + [3,4] + [2] = [1,3,4,2].

Example 2:



Input: root = [1,2,3,4,5,6,null,null,null,7,8,9,10]

Output: [1,2,4,7,8,9,10,6,3]

Explanation:

- The left boundary follows the path starting from the root's left child 2 -> 4.
4 is a leaf, so the left boundary is [2].
- The right boundary follows the path starting from the root's right child 3 -> 6 -> 10.
10 is a leaf, so the right boundary is [3,6], and in reverse order is [6,3].
- The leaves from left to right are [4,7,8,9,10].

Constraints:

- The number of nodes in the tree is in the range [1, 10⁴].
- $-1000 \leq \text{Node.val} \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Boundary = root + left boundary (non-leaves top-down) + all leaves left-to-right
+ right boundary (non-leaves bottom-up). Leftmost/rightmost leaves are in leaves,
not boundary. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
Collect four pieces separately: root val, left boundary nodes top-down,
all leaves left-to-right, right boundary nodes bottom-up.
Merge while skipping duplicate leaves already counted.
Time: O(n). Space: O(h) recursion for leaf collection.
Should I go ahead and optimize?"

```
class _545_BoundaryBinaryTree:
    def boundaryOfBinaryTree(self, root: Optional['TreeNode']) -> List[int]:
        if not root:
            return []
        def is_leaf(n): return not n.left and not n.right
        def left_boundary(n):
            res = []
            while n and not is_leaf(n):
                res.append(n.val)
                n = n.left if n.left else n.right
            return res
        def leaves(n):
            if not n: return []
            if is_leaf(n): return [n.val]
            return leaves(n.left) + leaves(n.right)
        def right_boundary(n):
            res = []
            while n and not is_leaf(n):
                res.append(n.val)
                n = n.right if n.right else n.left
            return res[::-1]
        if is_leaf(root):
            return [root.val]
        return [root.val] + left_boundary(root.left) + leaves(root) +
            right_boundary(root.right)
```

PHASE 5 — TEST & DEBUG

[1,null,2,3,4]: left=[]. leaves=[3,4]. right=[2]. → [1,3,4,2] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$. Three separate traversals, each linear.

'Prefer left else right' for left spine; 'prefer right else left' for right spine – reversed at end.

Given more time I'd ask: return boundary clockwise vs counter-clockwise? Just reverse the result."

◆ Quick Review

Key Insights

- Break down the problem into three parts:
- Left boundary (excluding leaves).
- All leaves (do a DFS to find leaves).
- Right boundary (excluding leaves, then reverse the collected list).
- Special handling is needed when the tree doesn't have a left or right subtree.
- The root is always included and is never considered as a leaf, even if it has no children.
- Avoid duplication of leaf nodes in the left/right boundaries.

Space & Time

Time Complexity: $O(n)$ – Every node is visited at most once.

Space Complexity: $O(n)$ – $O(n)$ space is used for the output list and recursion stack (in worst case tree is skewed).

Solution

The solution uses a Depth-First Search (DFS) strategy to traverse the tree and collect the boundary nodes in three steps: For the left boundary, start at `root.left` and traverse down, adding nodes (skipping leaves). For the leaves, perform a DFS from the root and add any node that has no children. For the right boundary, start at `root.right` and traverse down, adding nodes (skipping leaves). Since the right boundary must be added in reverse order, we reverse the collected list after traversal. Finally, concatenate these lists in the order: root, left boundary, leaves, reversed right boundary. The approach makes use of helper functions to clearly separate the logic for gathering left boundary, leaves, and right boundary. This modular approach simplifies reasoning and avoids edge case mistakes such as including duplicate leaf nodes.

Trees & BST

□ ★ #549 Binary Tree Longest Consecutive Sequence II ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: ★ Premium | Premium 98

Problem

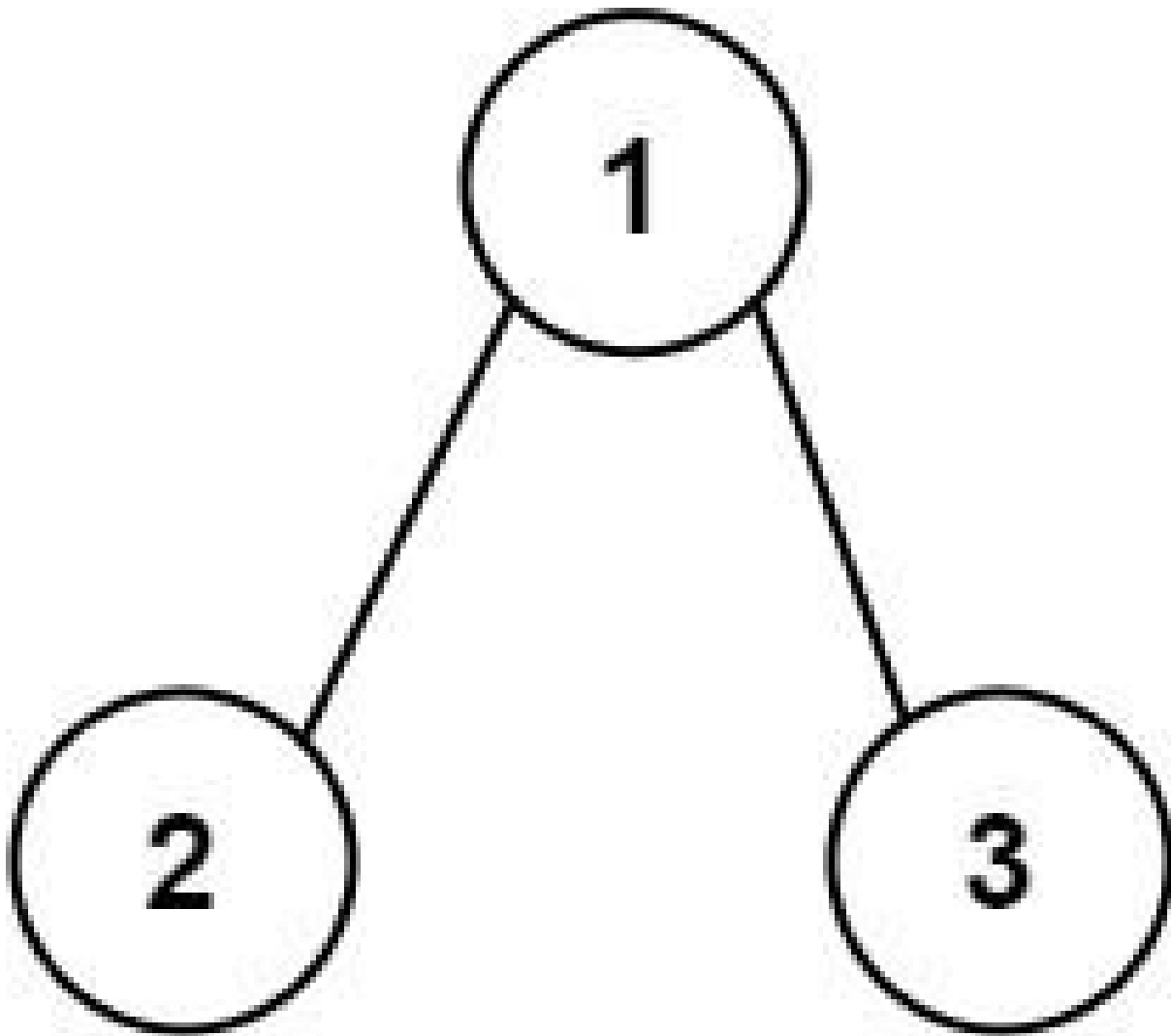
Given the root of a binary tree, return the length of the longest consecutive path in the tree.

A consecutive path is a path where the values of the consecutive nodes in the path differ by one. This path can be either increasing or decreasing.

- For example, [1,2,3,4] and [4,3,2,1] are both considered valid, but the path [1,2,4,3] is not valid.

On the other hand, the path can be in the child–Parent–child order, where not necessarily be parent–child order.

Example 1:

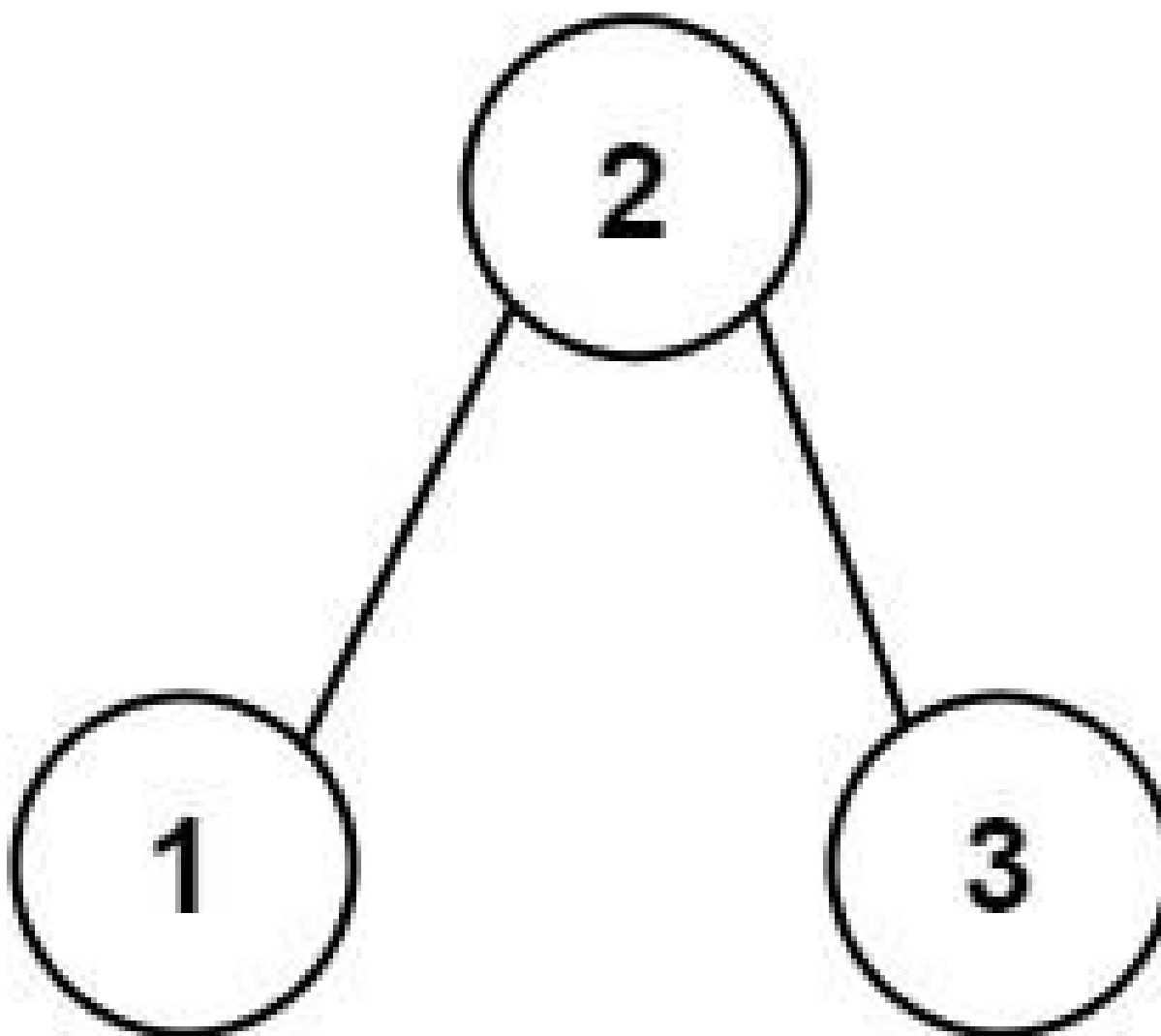


Input: root = [1,2,3]

Output: 2

Explanation: The longest consecutive path is [1, 2] or [2, 1].

Example 2:



Input: root = [2,1,3]

Output: 3

Explanation: The longest consecutive path is [1, 2, 3] or [3, 2, 1].

Constraints:

- The number of nodes in the tree is in the range $[1, 3 * 10^4]$.
- $-3 * 10^4 \leq \text{Node.val} \leq 3 * 10^4$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Longest path where consecutive values differ by 1 – can go up OR down, can change direction at one pivot. Right?"

#

"[2,1,3] → [1,2,3] length 3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

At each node, run DFS tracking consecutive count when `child.val == parent.val + 1`.

Reset count when streak breaks. Track global max streak length.

Visits every node once with $O(n)$ DFS per starting node worst case $O(n^2)$.

Time: $O(n^2)$ naive. Space: $O(h)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (post-order returning inc/dec lengths)

"Each node returns (inc, dec): length of increasing/decreasing sequence going DOWN from it."

Diameter through node = best combination of one child's inc and another's dec, meeting at this node."

PHASE 4 — CLEAN CODE

```
class _549_LongestConsecutiveII:
    def longestConsecutive(self, root: Optional['TreeNode']) -> int:
        best = [0]
        def dfs(node):
            if not node:
                return 0, 0
            l_inc, l_dec = dfs(node.left)
            r_inc, r_dec = dfs(node.right)
            inc = dec = 1
            if node.left:
                if node.left.val == node.val + 1: inc = max(inc, l_inc + 1)
                if node.left.val == node.val - 1: dec = max(dec, l_dec + 1)
            if node.right:
                if node.right.val == node.val + 1: inc = max(inc, r_inc + 1)
                if node.right.val == node.val - 1: dec = max(dec, r_dec + 1)
            best[0] = max(best[0], inc + dec - 1)
            return inc, dec
        dfs(root)
        return best[0]
```

PHASE 5 — TEST & DEBUG

[2,1,3]: `dfs(1)=(1,1)`. `dfs(3)=(1,1)`. `dfs(2): left(1)=val-1→dec=2`. `right(3)=val+1→inc=2`. `best=2+2-1=3` ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$. `inc+dec-1` avoids double-counting the pivot node."

Generalises #298 by allowing direction change at the pivot node.
Given more time I'd ask: allow path to go up? Needs full graph treatment."

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to visit every node.
- For each node, compute:
- The longest increasing consecutive path starting from that node.
- The longest decreasing consecutive path starting from that node.
- Combine the increasing and decreasing paths (subtracting one to avoid double counting the current node) to potentially form a longer consecutive sequence through the node.
- Update a global maximum to record the longest path seen.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes since every node is visited once.

Space Complexity: $O(h)$, where h is the height of the tree, due to the recursion stack.

Solution

The solution employs a DFS traversal. At every node, we calculate two values: `inc` – the length of the longest increasing consecutive sequence starting from that node. `dec` – the length of the longest decreasing consecutive sequence starting from that node. For each child of the current node: If the child's value is `node.val + 1`, update the increasing sequence. If the child's value is `node.val - 1`, update the decreasing sequence. Then, the longest path through the current node is `inc + dec - 1` (subtracting one to avoid counting the current node twice). A global variable maintains the maximum length encountered. This process works regardless of the path direction because it combines both increasing and decreasing parts.

Trees & BST

□ ★ #582 Kill Process ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Tree · DFS · BFS

Lists: ★ Premium | Premium 98

Problem

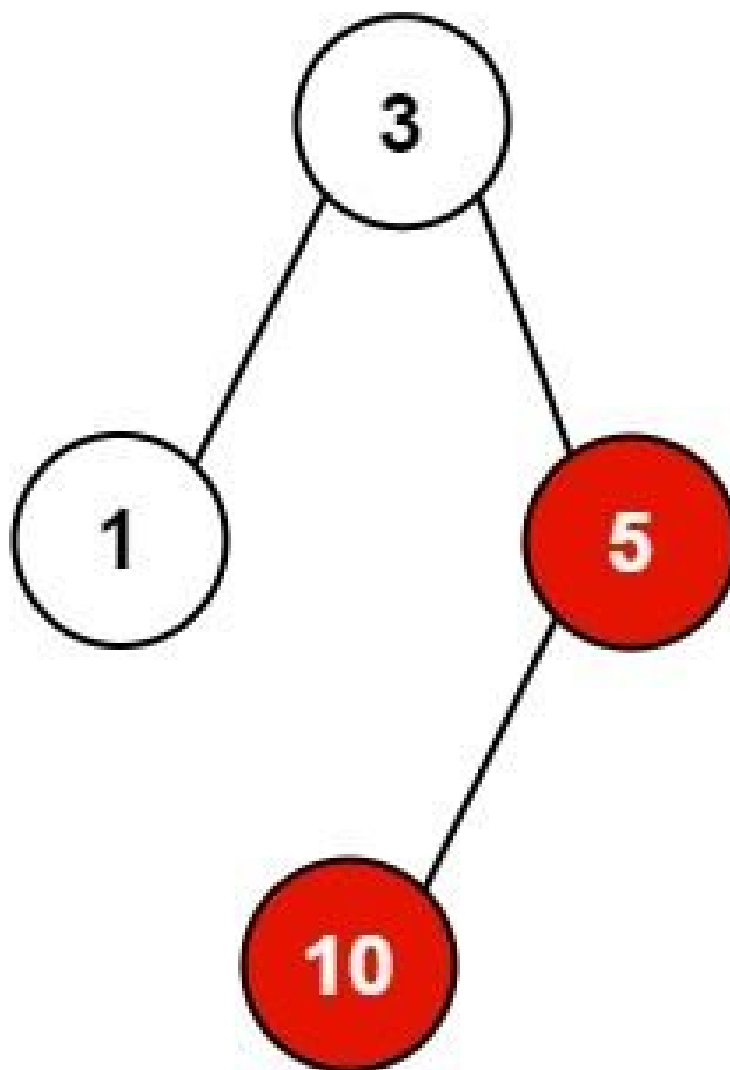
You have n processes forming a rooted tree structure. You are given two integer arrays `pid` and `ppid`, where `pid[i]` is the ID of the i th process and `ppid[i]` is the ID of the i th process's parent process.

Each process has only one parent process but may have multiple children processes. Only one process has `ppid[i] = 0`, which means this process has no parent process (the root of the tree).

When a process is killed, all of its children processes will also be killed.

Given an integer `kill` representing the ID of a process you want to kill, return a list of the IDs of the processes that will be killed. You may return the answer in any order.

Example 1:



Input: pid = [1,3,10,5], ppid = [3,0,5,3], kill = 5

Output: [5,10]

Explanation: The processes colored in red are the processes that should be killed.

Example 2:

Input: pid = [1], ppid = [0], kill = 1

Output: [1]

Constraints:

- $n == \text{pid.length}$
- $n == \text{ppid.length}$

- $1 \leq n \leq 5 * 10^4$
- $1 \leq \text{pid}[i] \leq 5 * 10^4$
- $0 \leq \text{ppid}[i] \leq 5 * 10^4$
- Only one process has no parent.
- All the values of pid are unique.
- kill is guaranteed to be in pid.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Kill a process and all its descendants. Return all killed process IDs. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Build children map and indegree from the kill-pair list.

BFS from the killed pid: enqueue all its direct children, then propagate kills downward.

Every reachable process dies – simulates the cascade.

Time: $O(n)$. Space: $O(n)$ for graph + queue.

Should I go ahead and optimize?"

```
class _582_KillProcess:
```

```
    def killProcess(self, pid: List[int], ppid: List[int], kill: int) -> List[int]:
        children: defaultdict = defaultdict(list)
        for child, parent in zip(pid, ppid):
            children[parent].append(child)
        killed = []
        queue = deque([kill])
        while queue:
            proc = queue.popleft()
            killed.append(proc)
            queue.extend(children[proc])
        return killed
```

PHASE 5 — TEST & DEBUG

pid=[1,3,10,5], ppid=[3,0,5,3], kill=5: children={3:[1,5],5:[10]}.

BFS from 5: [5,10] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Build adjacency list then BFS – standard implicit tree traversal.

Given more time I'd ask: kill all ancestors too? Walk parent pointers upward instead."

◆ Quick Review

Key Insights

- Build a mapping (dictionary/hash table) from a parent process ID to its list of child process IDs.
- Use Depth-First Search (DFS) or Breadth-First Search (BFS) starting from the kill process to traverse the tree and collect all descendant processes.
- The problem requires handling a tree structure that may have multiple children per node.

Space & Time

Time Complexity: $O(n)$, where n is the number of processes (both building the mapping and the tree traversal take linear time).

Space Complexity: $O(n)$, for storing the mapping and the traversal queue/stack.

Solution

We first construct a mapping from each process's parent ID to its list of child IDs. This allows us to quickly access all children of any process. Once the mapping is built, we perform a tree traversal (either BFS or DFS) starting from the process to kill. During the traversal, we record every process encountered. This collected list represents all processes that will be terminated. The data structures used are a dictionary (for the mapping) and either a stack (for DFS) or a queue (for BFS), making the solution efficient and straightforward.

Trees & BST

□ #662 Maximum Width of Binary Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS

Lists: Grind 169

Problem

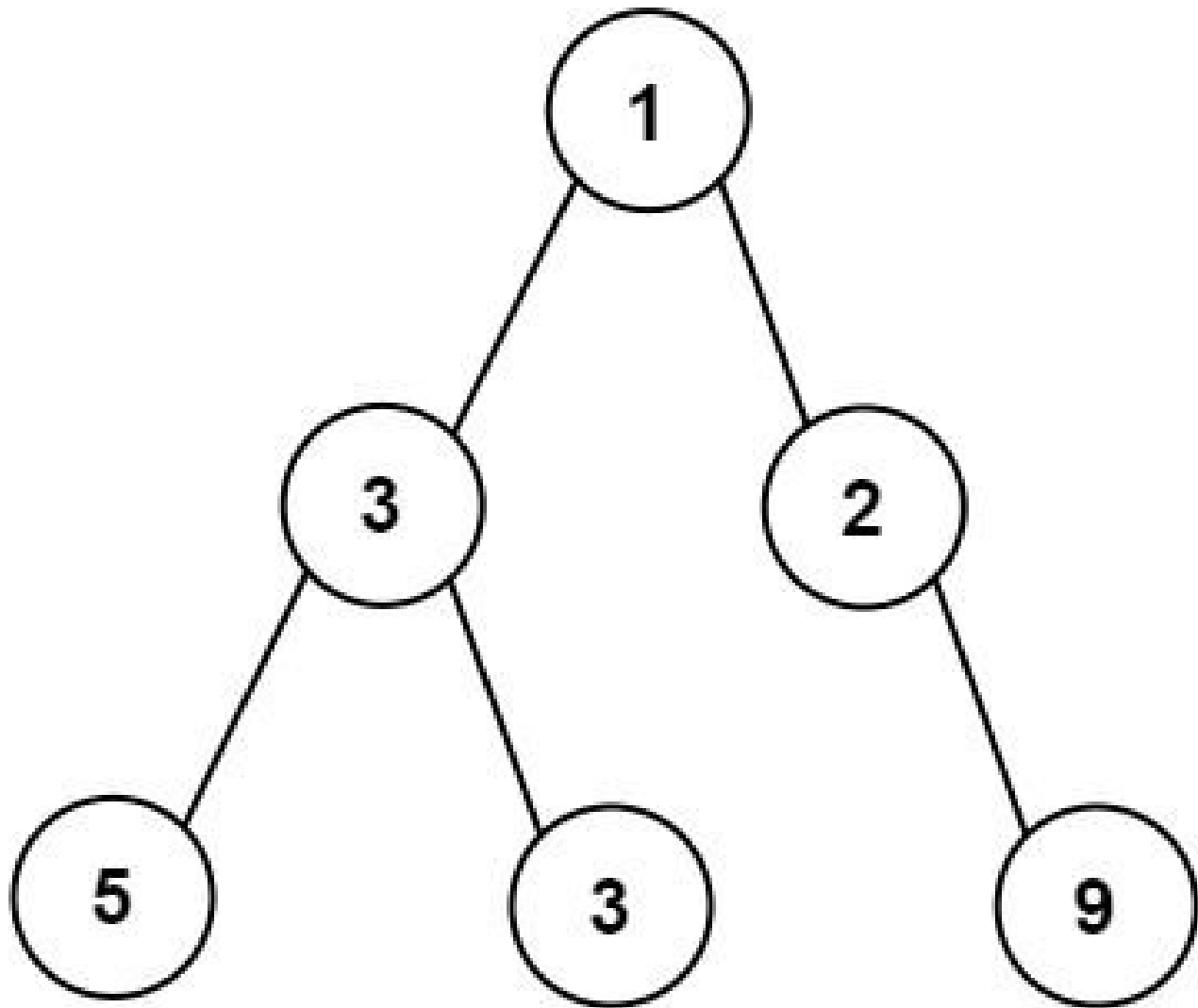
Given the root of a binary tree, return the maximum width of the given tree.

The maximum width of a tree is the maximum width among all levels.

The width of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

It is guaranteed that the answer will in the range of a 32-bit signed integer.

Example 1:

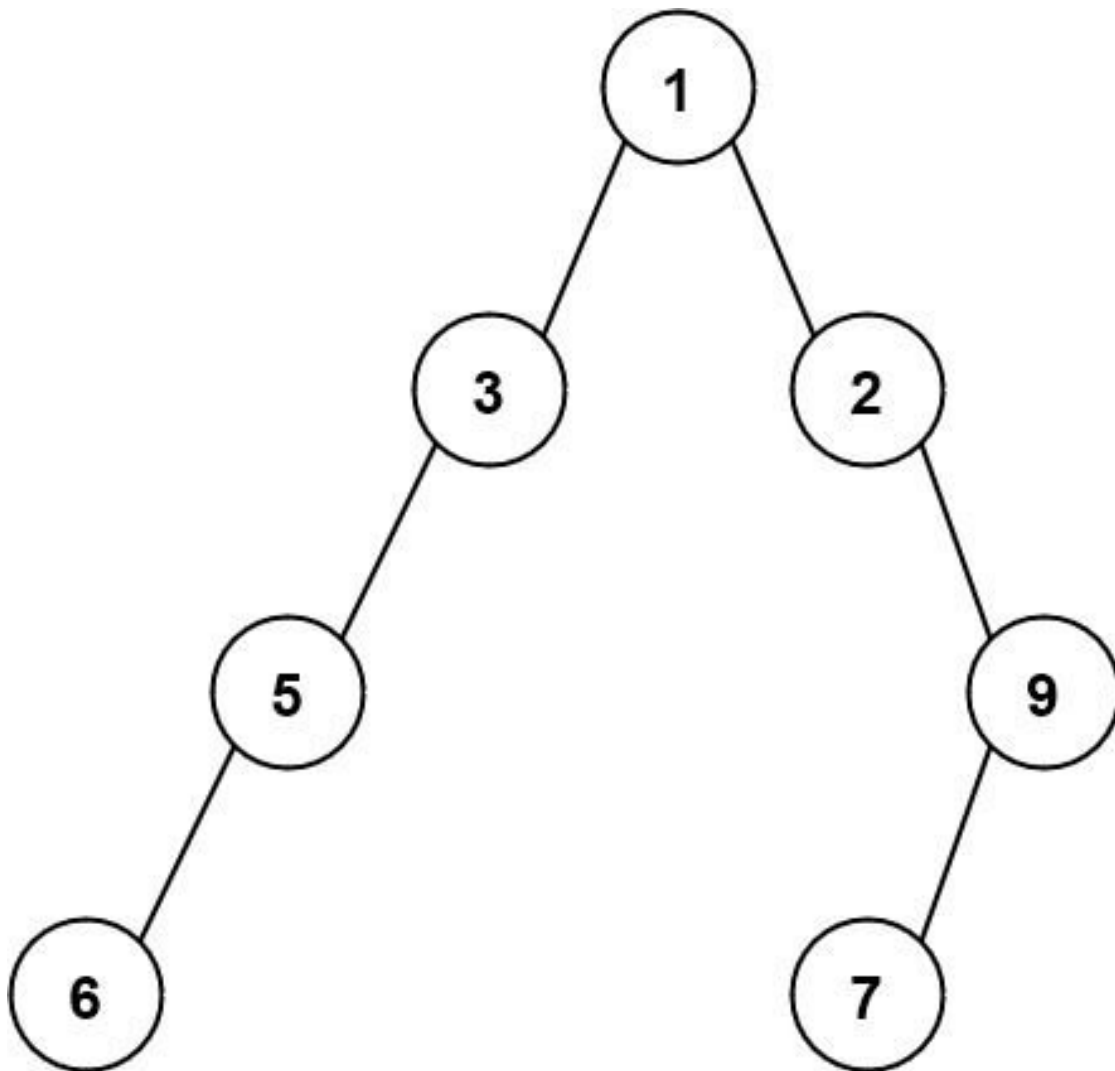


Input: root = [1,3,2,5,3,null,9]

Output: 4

Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).

Example 2:

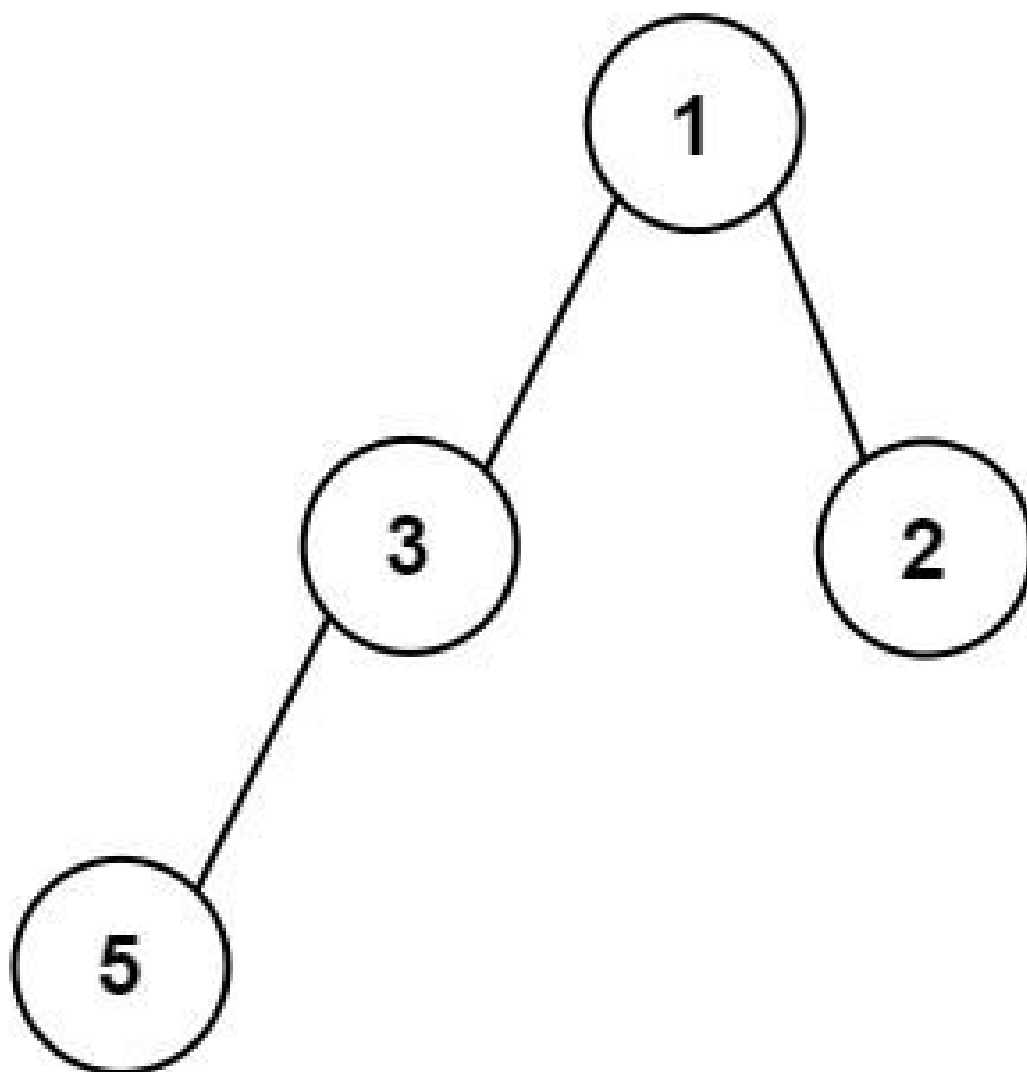


Input: root = [1,3,2,5,null,null,9,6,null,7]

Output: 7

Explanation: The maximum width exists in the fourth level with length 7 (6,null,null,null,null,null,7).

Example 3:



Input: root = [1,3,2,5]

Output: 2

Explanation: The maximum width exists in the second level with length 2 (3,2).

Constraints:

- The number of nodes in the tree is in the range [1, 3000].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Width = rightmost_pos - leftmost_pos + 1 at each level, counting null gaps.
 # Left child = 2*pos, right child = 2*pos+1. Normalise each level to prevent overflow. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # BFS level-order traversal; at each level track min and max index assigned to nodes.
 # Width of level = max_index - min_index + 1; answer is global max width.
 # Straightforward queue-based level scan.
 # Time: O(n). Space: O(w) queue width.
 # Should I go ahead and optimize?"

```
class _662_MaxWidthBinaryTree:
    def widthOfBinaryTree(self, root: Optional['TreeNode']) -> int:
        if not root:
            return 0
        max_w = 0
        queue = deque([(root, 0)])
        while queue:
            size = len(queue)
            first_pos = queue[0][1]
            last_pos = 0
            for _ in range(size):
                node, pos = queue.popleft()
                pos -= first_pos # normalise
                last_pos = pos
                if node.left: queue.append((node.left, 2 * pos))
                if node.right: queue.append((node.right, 2 * pos + 1))
            max_w = max(max_w, last_pos + 1)
        return max_w
```

PHASE 5 — TEST & DEBUG

[1,3,2,5,3,null,9]: Level 2 positions (normalised): 5→0, 3→1, 9→3. last=3. width=4
 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n). Space O(w). Normalisation prevents 2^depth overflow for deep trees – critical.
 # Given more time I'd ask: width by actual node count (ignoring nulls)? Just max level_size in BFS."

◆ Quick Review

Key Insights

- Perform a level order traversal (BFS) of the tree.
- Assign an index to each node as if the tree were complete (e.g., root is index 1, left child is 2
- i , right child is $2i + 1$).
- For each level, compute the width as $(\text{last_index} - \text{first_index} + 1)$.
- Using indices allows capturing gaps between nodes due to nulls in between.
- Edge cases include handling very skewed trees.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree.

Space Complexity: $O(N)$, due to the queue used for BFS which in the worst case holds nodes of the last level.

Solution

We use a Breadth-First Search (BFS) approach with a queue to traverse the tree level by level.

At each level, we pair each node with an index representing its position assuming a complete binary tree. This allows us to compute the width of each level using the formula: $\text{width} = \text{last_index} - \text{first_index} + 1$. We then track the maximum width seen throughout the traversal. The use of indices handles cases where there are null gaps between nodes.

Trees & BST

□ #863 All Nodes Distance K in Binary Tree **MED**

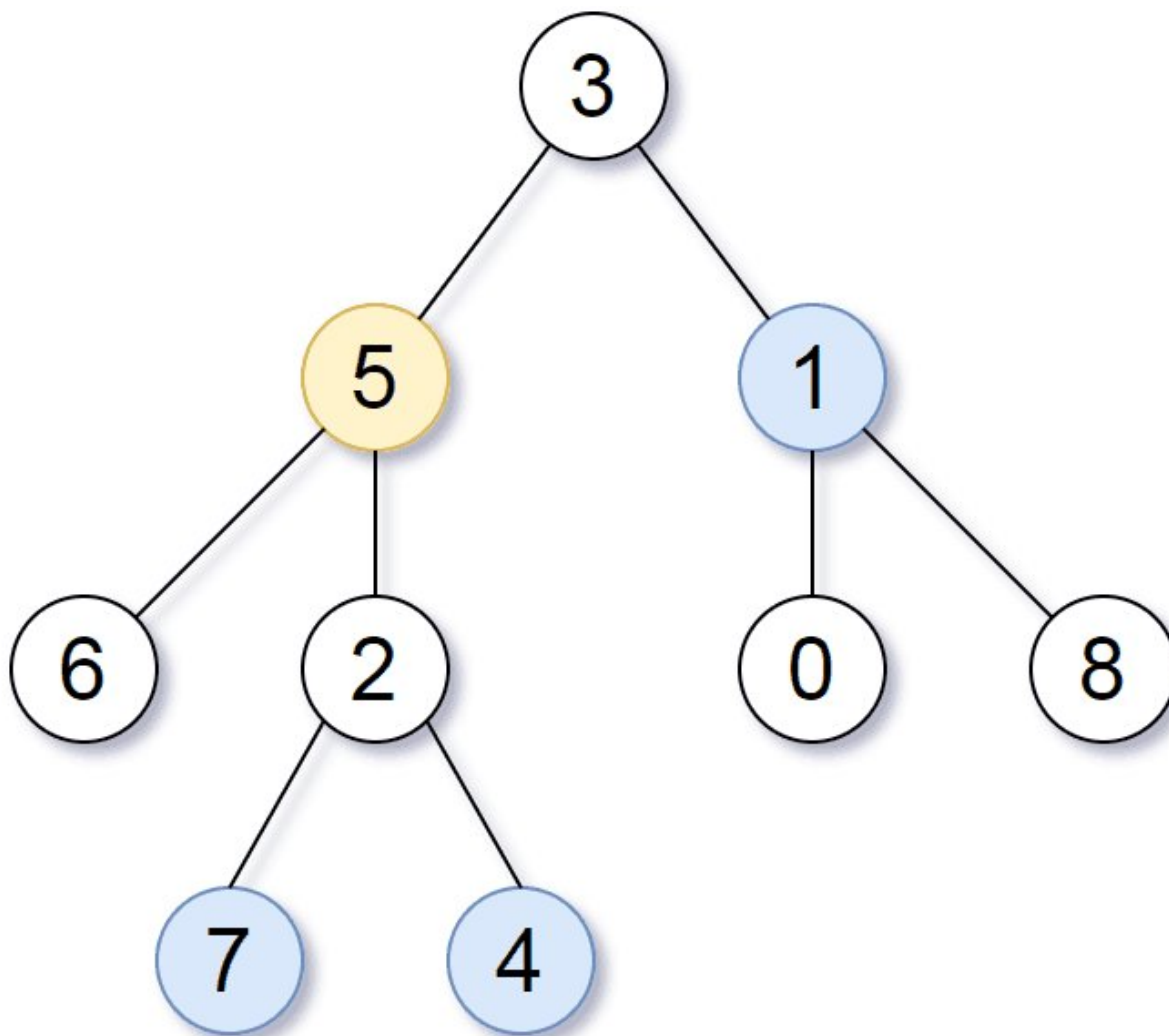
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Tree · DFS · BFS
Lists: Grind 169

Problem

Given the root of a binary tree, the value of a target node target, and an integer k, return an array of the values of all nodes that have a distance k from the target node.

You can return the answer in any order.

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2

Output: [7,4,1]

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.

Example 2:

Input: root = [1], target = 1, k = 3

Output: []

Constraints:

- The number of nodes in the tree is in the range [1, 500].
- $0 \leq \text{Node.val} \leq 500$

- All the values `Node.val` are unique.
- `target` is the value of one of the nodes in the tree.
- $0 \leq k \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find all nodes exactly `k` edges from `target`. Distance can go upward through parent. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
 # Build parent map via BFS/DFS from target. For each query node, walk parent pointers
 # until reaching target; collect distance. $O(n)$ per query naive.
 # Time: $O(n * Q)$ for Q queries. Space: $O(n)$ parent map.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (build parent map then BFS)

"DFS to build `parent[node]=parent`. Then BFS from target treating tree as undirected graph."
 # Stop collecting at distance `k`."

PHASE 4 — CLEAN CODE

```
class _863_DistanceK:
    def distanceK(self, root: 'TreeNode', target: 'TreeNode', k: int) -> List[int]:
        parent: dict = {}
        def build(node, par):
            if not node: return
            parent[node] = par
            build(node.left, node)
            build(node.right, node)
        build(root, None)
        visited = {target}
        queue = deque([(target, 0)])
        result = []
        while queue:
```

```

        node, dist = queue.popleft()
        if dist == k:
            result.append(node.val)
            continue
        for nb in [node.left, node.right, parent[node]]:
            if nb and nb not in visited:
                visited.add(nb)
                queue.append((nb, dist + 1))
    return result

```

PHASE 5 — TEST & DEBUG

target=5, k=2: BFS outward from 5 by 2 hops → [7,4,1] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Parent map converts tree to undirected graph – then BFS is trivial.

Given more time I'd ask: multiple targets? Multi-source BFS – enqueue all at distance 0."

◆ Quick Review

Key Insights

- Convert the binary tree into an undirected graph so that we can easily traverse both down to the children and up to the parent.
- Use a DFS or simple recursive traversal to create an adjacency list representing the graph.
- Perform a BFS starting from the target node to explore nodes level-by-level until reaching distance k .
- Use a visited set (or equivalent) to avoid revisiting nodes and to prevent cycles.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree. Each node is visited once when constructing the graph and once in the BFS.

Space Complexity: $O(N)$, for storing the graph and the additional data structures used in BFS and DFS.

Solution

The solution involves two main steps: Build an undirected graph representation of the binary tree using a DFS. For each node, add the node's value as a key in an adjacency list and connect it with its children (if any). This creates bidirectional links between parent and child. Use a BFS starting from the target node and explore nodes level-by-level. Keep a counter for the current distance, and once the counter reaches k , return all node values found at that level. To avoid processing nodes more than once, maintain a set of visited nodes.

Trees & BST

□ ★ #1120 Maximum Average Subtree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: ★ Premium | Premium 98

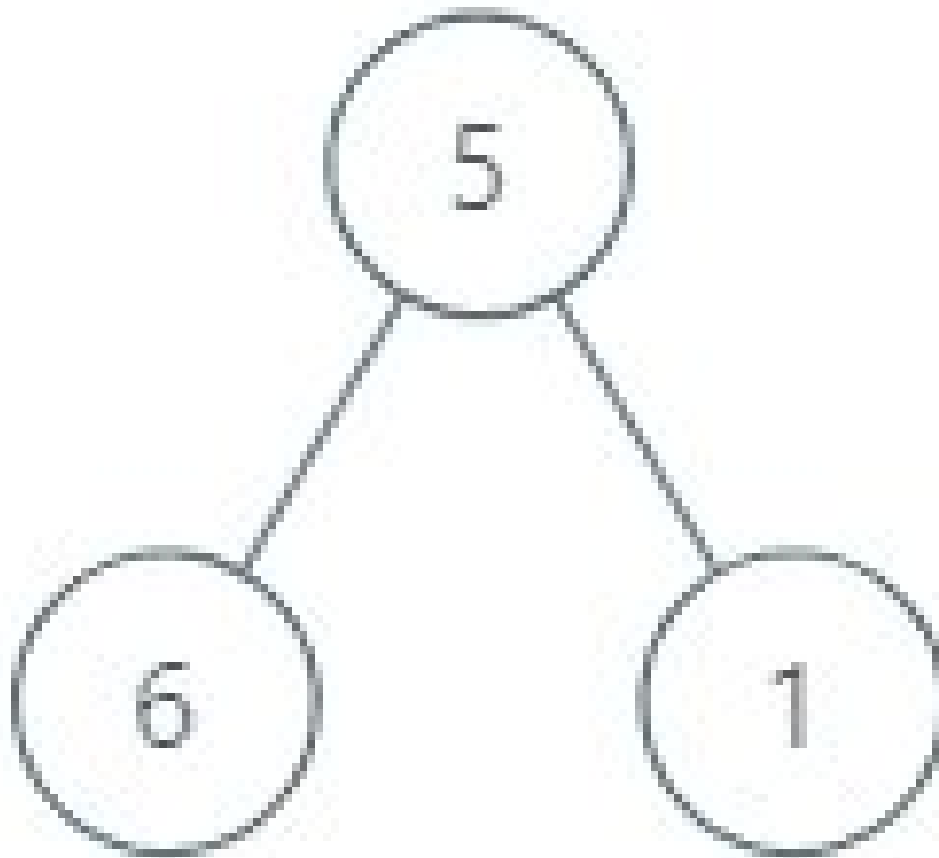
Problem

Given the root of a binary tree, return the maximum average value of a subtree of that tree. Answers within 10⁻⁵ of the actual answer will be accepted.

A subtree of a tree is any node of that tree plus all its descendants.

The average value of a tree is the sum of its values, divided by the number of nodes.

Example 1:



Input: root = [5,6,1]

Output: 6.00000

Explanation:

For the node with value = 5 we have an average of $(5 + 6 + 1) / 3 = 4$.

For the node with value = 6 we have an average of $6 / 1 = 6$.

For the node with value = 1 we have an average of $1 / 1 = 1$.

So the answer is 6 which is the maximum.

Example 2:

Input: root = [0,null,1]

Output: 1.00000

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $0 \leq \text{Node.val} \leq 105$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the subtree (any node + all descendants) with the maximum average value. Return the average. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
 # Post-order DFS: for each node return (sum, count) of its subtree.
 # Track max average = sum/count over all visited nodes.
 # Recomputes subtree sums independently – correct $O(n)$ post-order.
 # Time: $O(n)$. Space: $O(h)$ stack.
 # Should I go ahead and optimize?"

```
class _1120_MaxAvgSubtree:
    def maximumAverageSubtree(self, root: Optional['TreeNode']) -> float:
        best = [0.0]
        def dfs(node):
            if not node: return 0, 0
            l_sum, l_cnt = dfs(node.left)
            r_sum, r_cnt = dfs(node.right)
            total_sum = l_sum + r_sum + node.val
            total_cnt = l_cnt + r_cnt + 1
            best[0] = max(best[0], total_sum / total_cnt)
            return total_sum, total_cnt
        dfs(root)
        return best[0]
```

PHASE 5 — TEST & DEBUG

[5,6,1]: avg(6)=6, avg(1)=1, avg(5,6,1)=4. → 6.0 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$. Post-order carries (sum, count) up – compute average at each node."
 # Given more time I'd ask: minimum average subtree? Track min instead of max."

◆ Quick Review

Key Insights

- Use a depth-first search (DFS) to traverse every subtree.
- At each node, compute the sum and count of nodes in its subtree.
- Calculate the average for the current subtree and update a global maximum if needed.
- Processing every node exactly once yields an efficient solution.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes (each node is visited once).

Space Complexity: $O(h)$, where h is the height of the tree (space for recursion stack).

Solution

We perform a post-order DFS traversal, calculating at each node both the cumulative sum and the count of nodes in its subtree. This enables us to compute the average value in constant time for any subtree. We maintain a global variable to track the maximum average encountered during the traversal. The primary technique is recursive DFS combined with tuple (or pair) aggregation. The simplicity of the recursion and in-place aggregation

ensures an efficient and clear solution.

Trees & BST

□ ★ #1490 Clone N-ary Tree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Hash Table · Tree · DFS · BFS

Lists: ★ Premium | Premium 98

Problem

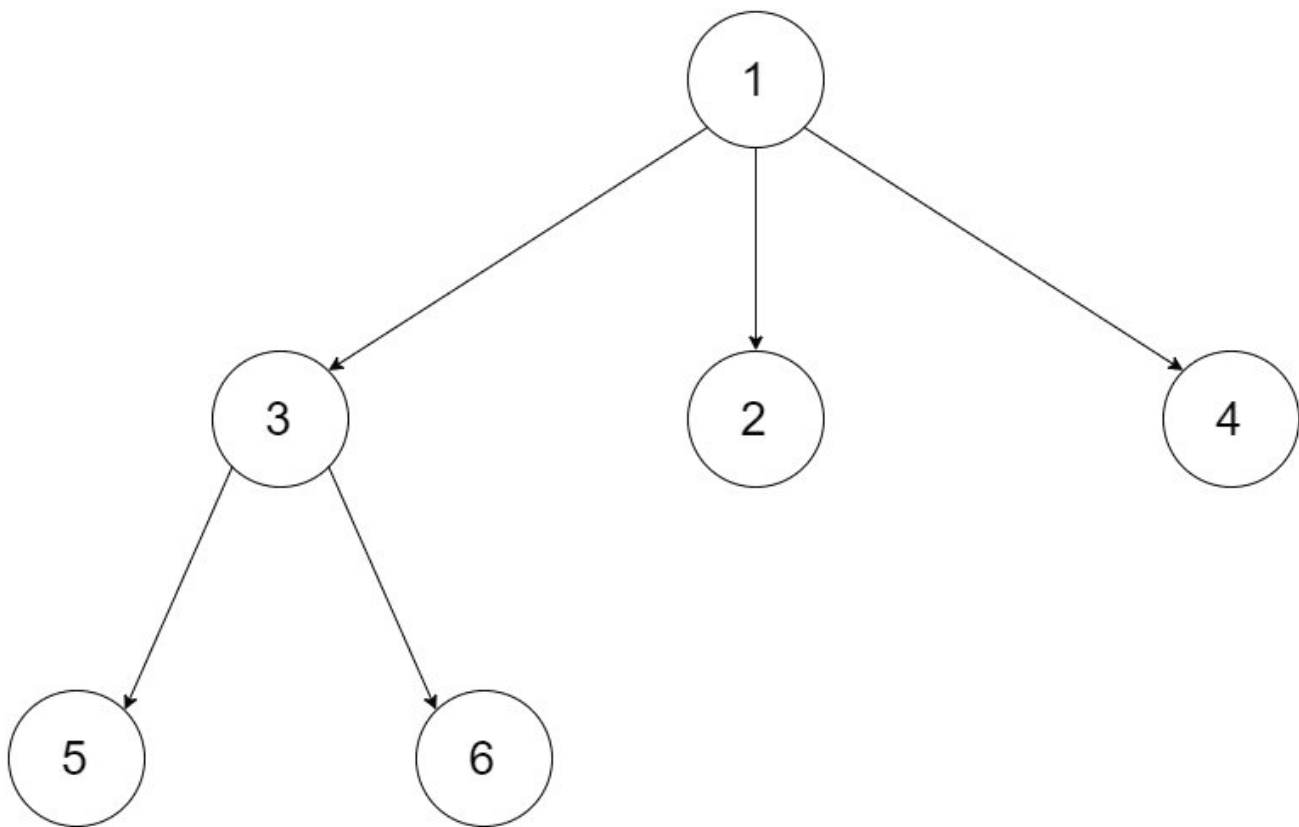
Given a root of an N-ary tree, return a deep copy (clone) of the tree.

Each node in the n-ary tree contains a val (int) and a list (List[Node]) of its children.

```
class Node {  
    public int val;  
    public List<Node> children;  
}
```

Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value (See examples).

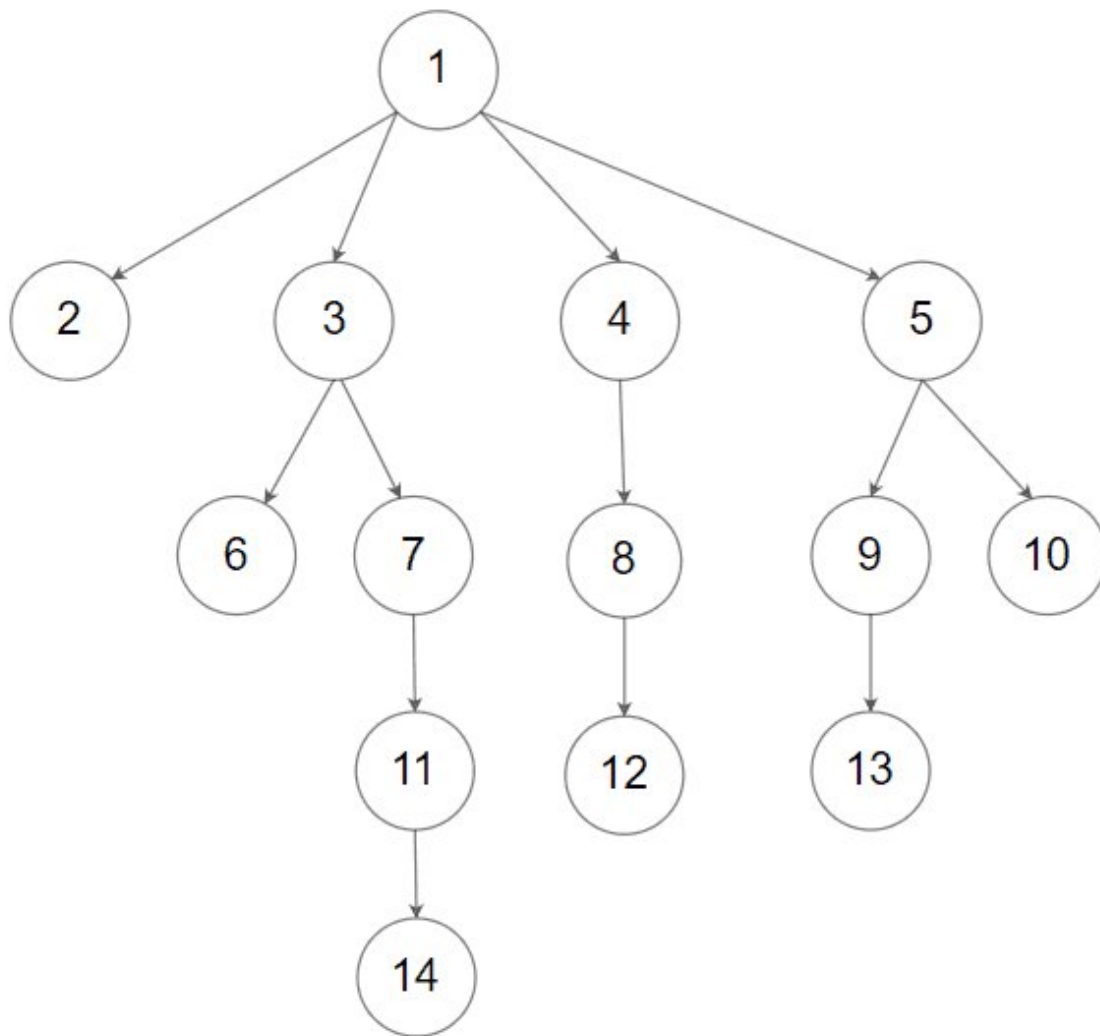
Example 1:



Input: root = [1,null,3,2,4,null,5,6]

Output: [1,null,3,2,4,null,5,6]

Example 2:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [0, 104].

Follow up: Can your solution work for the graph problem?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Deep clone an N-ary tree. No shared references between original and clone. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
 # DFS clone: map each original node to a new node, recursively clone all children lists.
 # First pass creates nodes on demand; second pass wires child pointers.
 # Time: $O(n)$. Space: $O(n)$ for map + recursion.
 # Should I go ahead and optimize?"

```
class _1490_CloneNaryTree:
    def cloneTree(self, root: 'Node') -> 'Node':
        if not root:
            return None
        return Node(root.val, [self.cloneTree(child) for child in root.children])
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Simple recursive clone – no visited map needed (trees have no cycles).
 # Follow-up (Clone Graph #133): needs visited map to handle cycles – {original: clone}."

◆ Quick Review

Key Insights

- The tree is N-ary, meaning every node can have multiple children.
- A deep copy requires creating new nodes for each node in the tree.

- Both DFS (recursive) or BFS (iterative) traversals can be used to clone the tree.
- The overall time complexity is $O(N)$ since each node is visited once.
- We can follow similar cloning techniques used in cloning graphs, using either recursion or an auxiliary data structure like a queue.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree, since each node is visited exactly once.

Space Complexity: $O(N)$, accounting for the recursive call stack (or an explicit queue in BFS) and the space needed to store the clone.

Solution

We perform a DFS traversal starting from the root node. For each node we encounter, we:
Create a new node with the same value.
Recursively clone all its children and attach the cloned children to the new node. This approach ensures that every original node is copied into a new node and that the parent-child relationships are preserved. Edge cases such as an empty tree (null root) are

also handled by returning null immediately.

Trees & BST

□ ★ #1522 Diameter of N-Ary Tree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

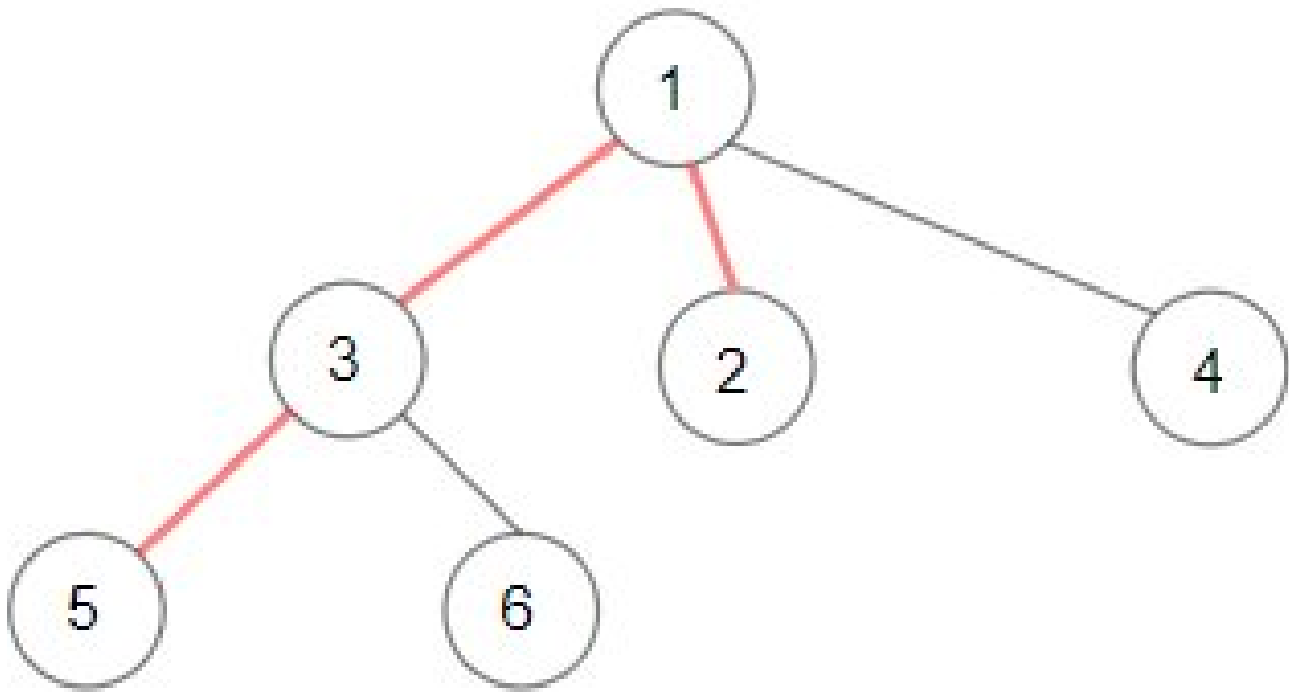
Lists: ★ Premium | Premium 98

Problem

Given a root of an N-ary tree, you need to compute the length of the diameter of the tree. The diameter of an N-ary tree is the length of the longest path between any two nodes in the tree. This path may or may not pass through the root.

(Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value.)

Example 1:

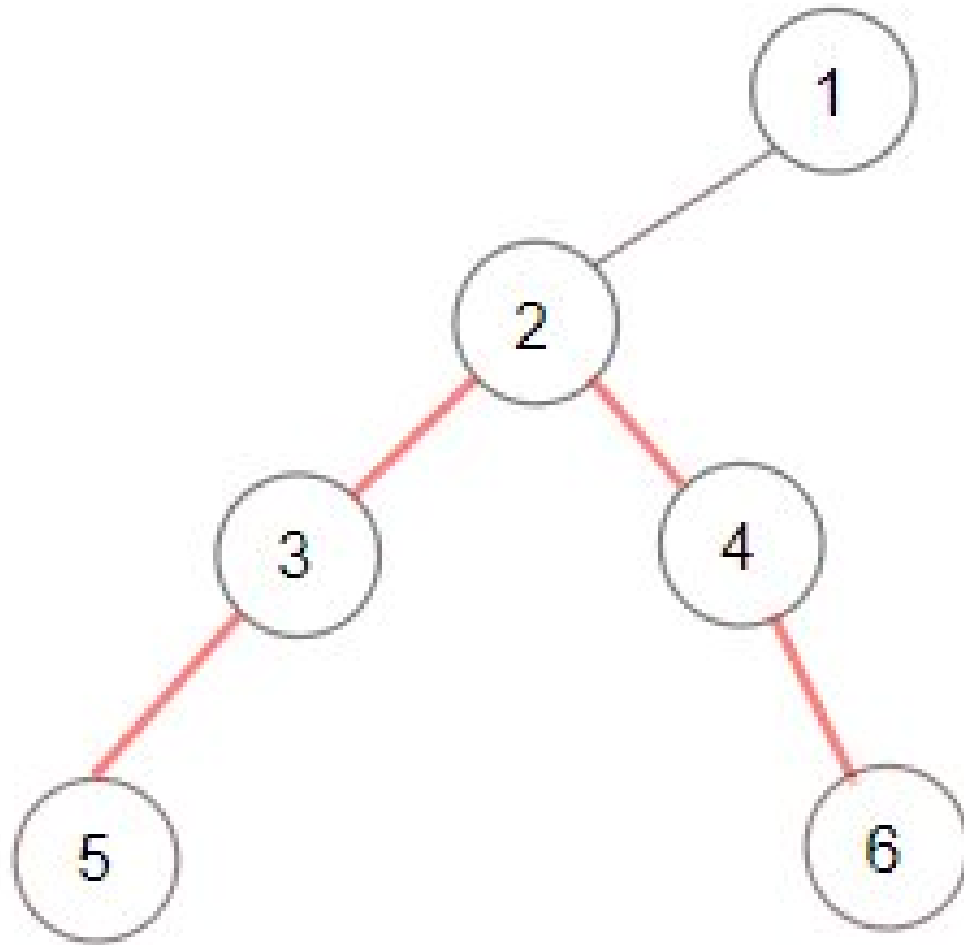


Input: root = [1,null,3,2,4,null,5,6]

Output: 3

Explanation: Diameter is shown in red color.

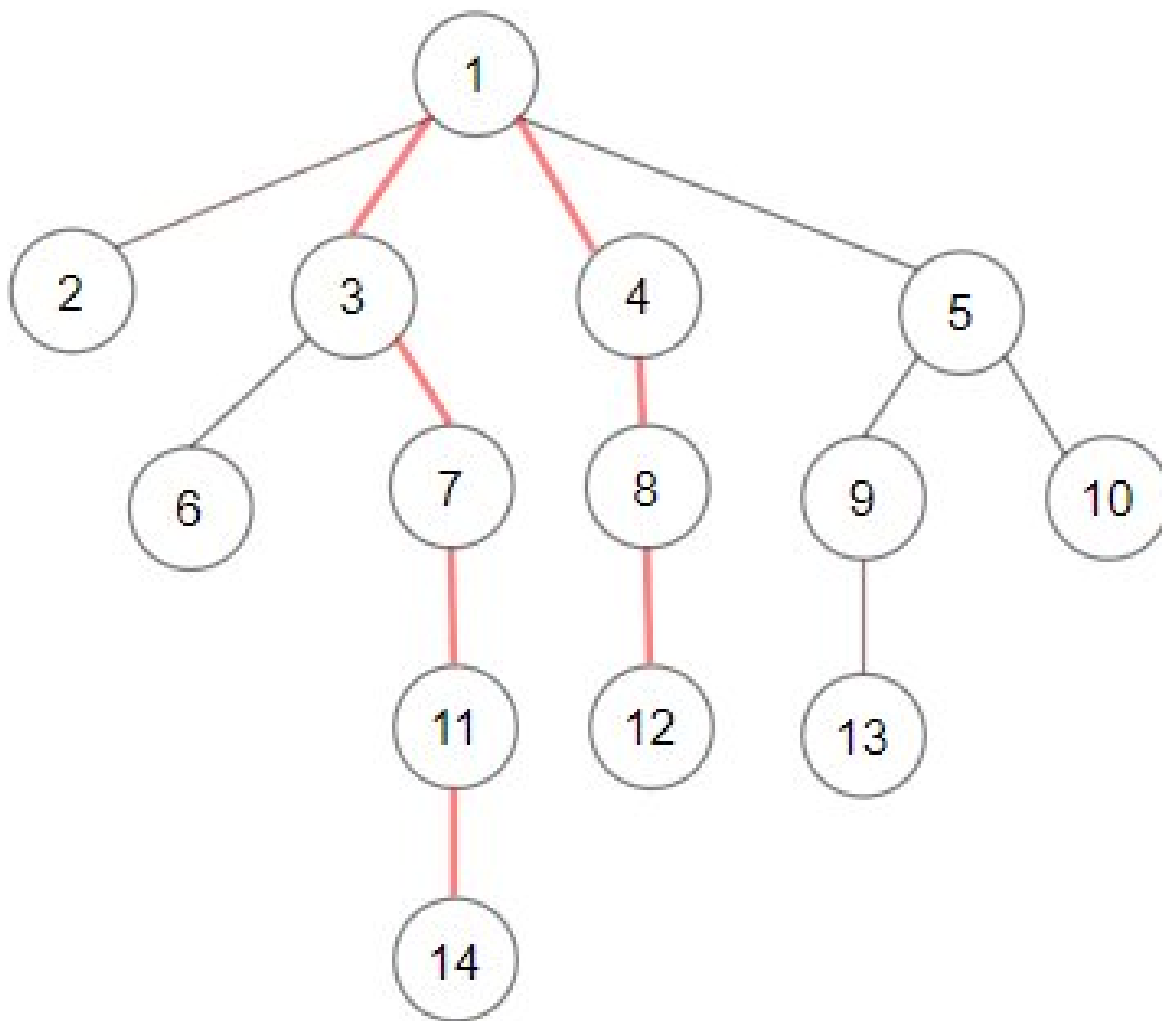
Example 2:



Input: root = [1,null,2,null,3,4,null,5,null,6]

Output: 4

Example 3:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: 7

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [1, 104].

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Diameter = longest path between any two nodes (in edges), generalised to N children. Right?

Same concept as Binary Tree Diameter (#543)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Two-pass DFS diameter: for each node, longest path through it = sum of top two child depths.

Same as binary tree diameter but children list instead of left/right.

Time: O(n). Space: O(h) stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Post-order: for each node, collect heights of children (adding 1 for the edge to parent).

Diameter through this node = top two child-heights summed.

Node's own height = max child-height (or 0 for leaf)."

PHASE 4 — CLEAN CODE

```
class _1522_DiameterNaryTree:
    def diameter(self, root: 'Node') -> int:
        best = [0]
        def height(node) -> int:
            if not node or not node.children:
                return 0
            # child_h+1 converts "height below child" to "edges from this node via
            that child"
            child_heights = sorted([height(c) + 1 for c in node.children],
            reverse=True)
            if len(child_heights) >= 2:
                best[0] = max(best[0], child_heights[0] + child_heights[1])
            else:
                best[0] = max(best[0], child_heights[0])
            return child_heights[0]
        height(root)
        return best[0]
```

PHASE 5 — TEST & DEBUG

[1,null,3,2,4,null,5,6]: height(5)=0, height(6)=0.

height(3): child_heights=[0+1,0+1]=[1,1]. best=1+1=2. return 1.

height(2)=0, height(4)=0.

height(1): child_heights=[1+1,0+1,0+1]=[2,1,1]. best=2+1=3. return 2. → 3 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$. Adding +1 when collecting child heights is the key difference from #543.

In binary tree, `depth(child)` already includes the edge up; here we add it explicitly.

Given more time I'd ask: diameter in edge-weighted N-ary tree?

Same approach – just use edge weights instead of +1."

◆ Quick Review

Key Insights

- The diameter of the tree can be found by considering the two largest depths from any node among its children.
- Use a depth-first search (DFS) to compute the maximum depth from each node.
- For each node, the potential diameter is the sum of the two longest depths among its children.
- Update a global variable tracking the maximum diameter as you recurse through the tree.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since each node is visited once.

Space Complexity: $O(h)$, where h is the height of the tree, due to the recursion stack (worst-case $O(n)$ for a skewed tree).

Solution

Use a DFS approach to traverse the tree. At each node, calculate the longest and second longest depth among its children. The sum of these two depths gives the candidate diameter through that node. Maintain a global variable to track the maximum diameter found so far. Return the maximum depth from the node (i.e., one plus the largest depth among its children) to be used in parent's calculations.

Trees & BST

★ #1650 Lowest Common Ancestor of a Binary Tree III ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Tree · Two Pointers
Lists: ★ Premium | Premium 98

Problem

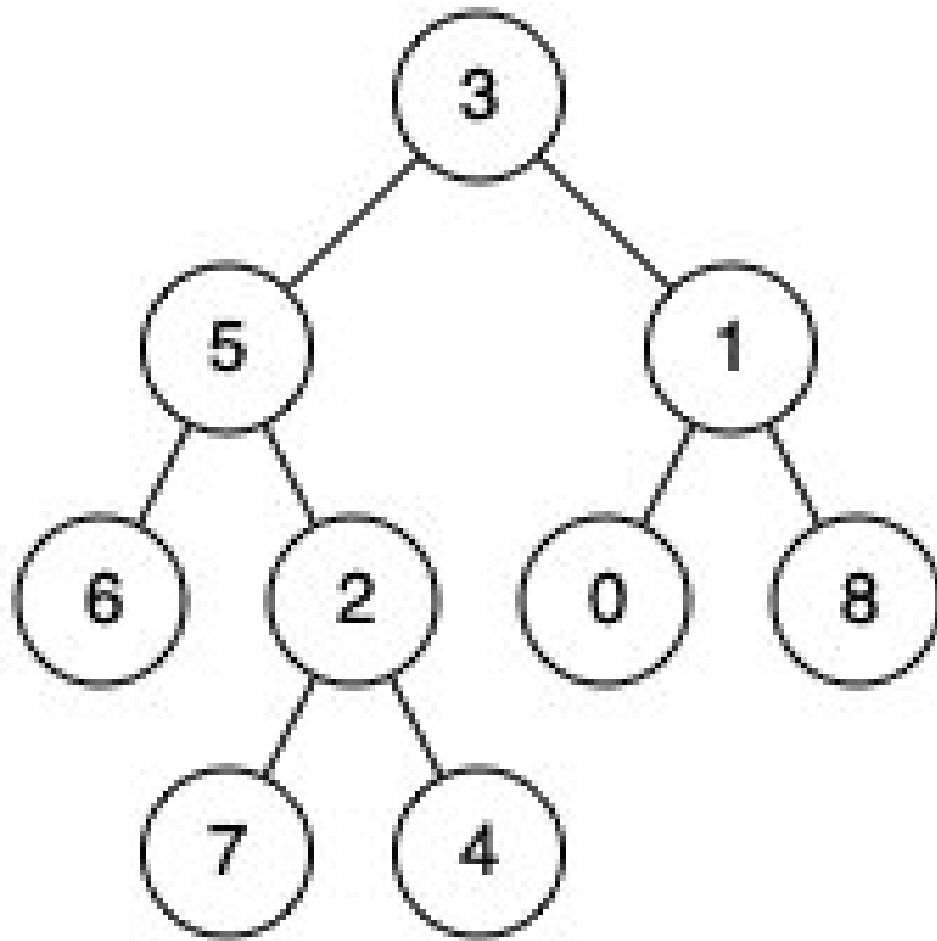
Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA).

Each node will have a reference to its parent node. The definition for Node is below:

```
class Node {  
    public int val;  
    public Node left;  
    public Node right;  
    public Node parent;  
}
```

According to the definition of LCA on Wikipedia:
"The lowest common ancestor of two nodes p and q in a tree T is the lowest node that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Example 1:

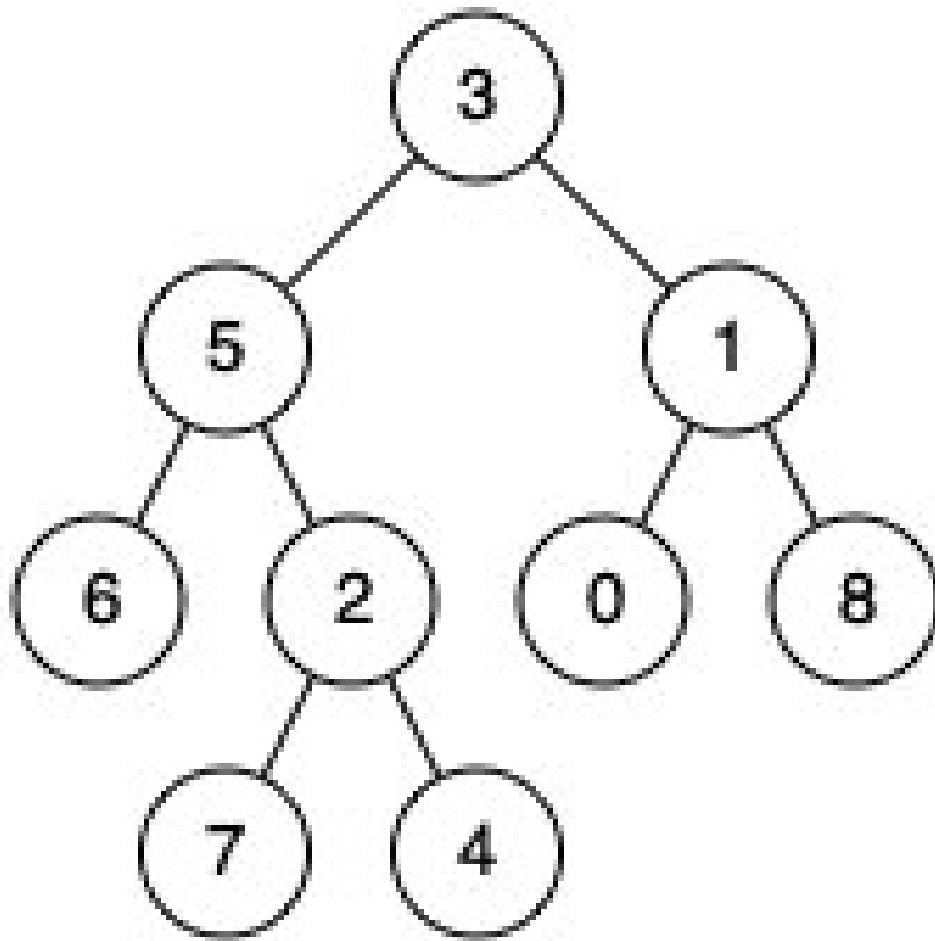


Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

Output: 3

Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

Output: 5

Explanation: The LCA of nodes 5 and 4 is 5 since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$

- All Node.val are unique.
- $p \neq q$
- p and q exist in the tree.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Nodes have a parent pointer. Find LCA without the root. Right?"
# Both p and q are guaranteed to be in the tree."
#
# "[3,5,1,...], p=5, q=1 → LCA=3 ✓ (same as #236 but solved via parent pointers)"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
# Each node has a parent pointer. Walk both nodes up with a visited set until they meet.
# First node seen twice is LCA. Like intersecting two linked lists.
# Time: O(h). Space: O(h) visited set.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (two-pointer linked list intersection)

```
# "Walk p upward and q upward simultaneously. When one hits null, restart at the other's origin."
# They meet at the LCA — same as finding intersection of two linked lists.
# Each pointer traverses: (path from p to root) + (path from q to root), length l_p + l_q.
# Both pointers traverse the same total length → they meet at LCA."
```

PHASE 4 — CLEAN CODE

```
class _1650_LCA_III:
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        a, b = p, q
        while a is not b:
            a = a.parent if a else q
            b = b.parent if b else p
        return a
```

PHASE 5 — TEST & DEBUG

p=5, q=1: a traverses 5→3→None→1→3. b traverses 1→3→None→5→3. Both reach 3 simultaneously ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(h)$. Space $O(1)$. This is the two-pointer intersection trick from [Linked List Intersection \(#160\)](#).

Both pointers traverse $\text{depth}(p) + \text{depth}(q)$ steps total and meet at LCA.

Given more time I'd ask: what if the tree is very deep and we want to avoid the full traversal?

Use a set: walk p to root recording all ancestors, then walk q to root and stop at first ancestor in the set.

$O(\text{depth})$ time, $O(\text{depth})$ space – same asymptotic but possibly faster in practice."

◆ Quick Review

Key Insights

- Each node has a parent pointer, so you can traverse from any node to the root.
- By determining the depth of each node, you can “align” the two nodes to the same depth.
- When both nodes are aligned, traverse upward simultaneously until they meet.
- This approach guarantees finding the LCA with $O(h)$ time complexity, where h is the height of the tree.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree.

Space Complexity: $O(1)$

Solution

We first compute the depth of both nodes by moving up their parent pointers. If one node is deeper, move it upward until both nodes are at the same level. Then, move both nodes upward one step at a time until they meet; the meeting point is the lowest common ancestor. This approach uses constant extra space and leverages the parent pointer for an efficient solution.

DFS

Round 2 · High · Medium · 16 questions

DFS

□ #130 Surrounded Regions

MED

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Union Find · Matrix
Lists: Denny Zhang**

Problem

You are given an $m \times n$ matrix board containing letters 'X' and 'O', capture regions that are surrounded:

- **Connect:** A cell is connected to adjacent cells horizontally or vertically.
- **Region:** To form a region connect every 'O' cell.
- **Surround:** A region is surrounded if none of the 'O' cells in that region are on the edge of the board. Such regions are completely enclosed by 'X' cells.

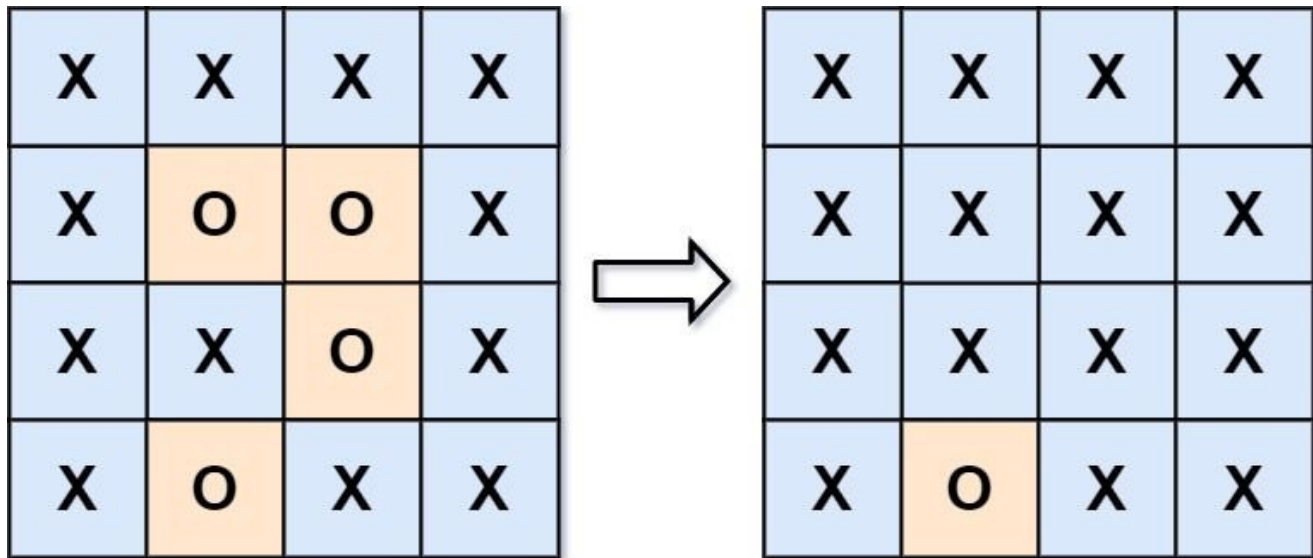
To capture a surrounded region, replace all 'O's with 'X's in-place within the original board. You do not need to return anything.

Example 1:

Input: board = [["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O","X","X"]]

Output: [["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]

Explanation:



In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2:

Input: board = [["X"]]

Output: [["X"]]

Constraints:

- m == board.length
- n == board[i].length
- 1 <= m, n <= 200

- `board[i][j]` is 'X' or 'O'.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Capture surrounded 'O' regions by replacing them with 'X'.
# A region is NOT captured if any 'O' in it touches the board edge. Right?
# In-place modification – no return value."
#
# "Border-connected 'O's and their inland 'O' neighbours are safe. Everything else
# gets flipped."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# First pass: DFS from every border 'O' and mark connected cells as safe ('S').
# Second pass: flip remaining 'O' to 'X'; restore safe cells back to 'O'.
# Two-pass flood fill from boundary – standard O(m*n) approach.
# Time: O(m * n). Space: O(m * n) recursion.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (reverse: mark safe, then flip)

```
# "DFS from every border 'O', marking connected 'O's as safe ('#').
# Second pass: 'O' → 'X' (captured), '#' → 'O' (safe restored). O(m*n)."
```

PHASE 4 — CLEAN CODE

```
class _130_SurroundedRegions:
    def solve(self, board: List[List[str]]) -> None:
        m, n = len(board), len(board[0])
        def mark_safe(r, c):
            if r < 0 or r >= m or c < 0 or c >= n or board[r][c] != 'O':
                return
            board[r][c] = '#'
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                mark_safe(r + dr, c + dc)
        for r in range(m):
            for c in [0, n - 1]:
                mark_safe(r, c)
        for c in range(n):
            for r in [0, m - 1]:
                mark_safe(r, c)
        for r in range(m):
```

```
for c in range(n):
    if board[r][c] == '0': board[r][c] = 'X'
    elif board[r][c] == '#': board[r][c] = '0'
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \times n)$. Space $O(m \times n)$ stack. The reverse-DFS from borders is the key insight

—
instead of deciding which '0's are captured, decide which are SAFE and flip the rest.

Given more time I'd ask: what if the grid wraps (toroidal)? No borders → all '0's are safe."

◆ Quick Review

Key Insights

- Regions connected to the border are not captured.
- Use DFS/BFS from border 'O's to mark safe cells.
- After marking, flip all remaining 'O's to 'X' and then revert the marked cells back to 'O'.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(m * n)$ in the worst case due to recursion stack or queue storage for DFS/BFS

Solution

The solution involves two main steps. First, traverse the board's borders and run a DFS or BFS from any encountered 'O', marking it (e.g.,

using ' ') to indicate that it should not be flipped. Next, go through each cell in the board: flip any remaining 'O' (i.e., not marked safe) to 'X' and change the temporary marker ' ' back to 'O'. This approach ensures only the regions completely surrounded by 'X' are captured.

DFS

#133 Clone Graph

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · DFS · BFS · Graph
Lists: Grind 169

Problem

Given a reference of a node in a connected undirected graph.

Return a deep copy (clone) of the graph.

Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

```
class Node {  
    public int val;  
    public List<Node> neighbors;  
}
```

Test case format:

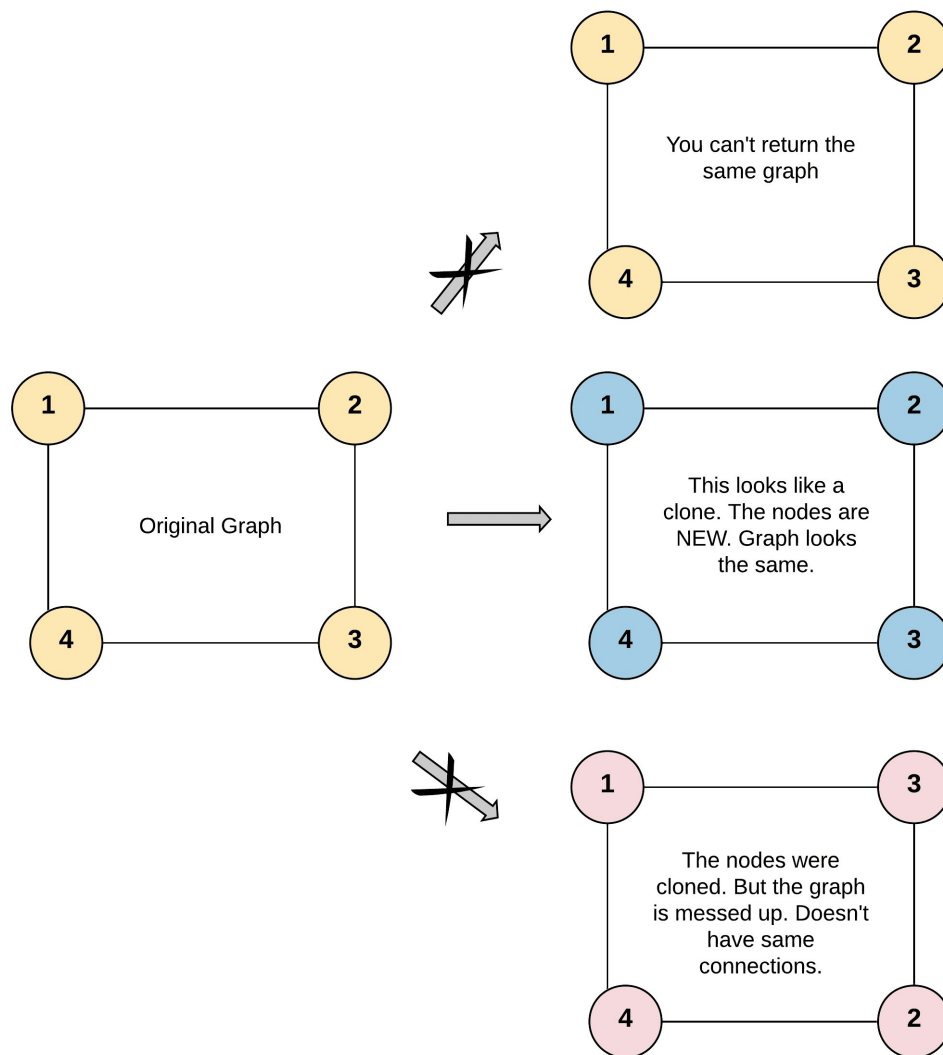
For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered lists used to represent a finite graph. Each list

describes the set of neighbors of a node in the graph.

The given node will always be the first node with $val = 1$. You must return the copy of the given node as a reference to the cloned graph.

Example 1:



Input: `adjList = [[2,4],[1,3],[2,4],[1,3]]`

Output: `[[2,4],[1,3],[2,4],[1,3]]`

Explanation: There are 4 nodes in the graph.

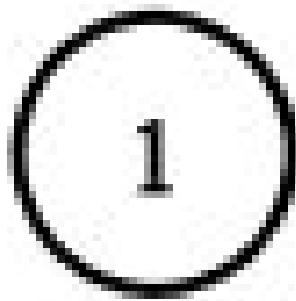
1st node (`val = 1`)'s neighbors are 2nd node (`val = 2`) and 4th node (`val = 4`).

2nd node (`val = 2`)'s neighbors are 1st node (`val = 1`) and 3rd node (`val = 3`).

3rd node (`val = 3`)'s neighbors are 2nd node (`val = 2`) and 4th node (`val = 4`).

4th node (`val = 4`)'s neighbors are 1st node (`val = 1`) and 3rd node (`val = 3`).

Example 2:



Input: `adjList = [[]]`

Output: `[[]]`

Explanation: Note that the input contains one empty list. The graph consists of only one node with `val = 1` and it does not have any neighbors.

Example 3:

Input: adjList = []

Output: []

Explanation: This an empty graph, it does not have any nodes.

Constraints:

- The number of nodes in the graph is in the range [0, 100].
- $1 \leq \text{Node.val} \leq 100$
- Node.val is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Deep copy an undirected connected graph. No shared references with original. Right?"

Graphs can have cycles – unlike trees, so we need a visited map."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

BFS from the given node: maintain a map old_node -> new_node clone.

When visiting neighbors, create clones on first sight and enqueue them.

Straightforward graph copy – already the standard $O(V+E)$ approach.

Time: $O(V + E)$. Space: $O(V)$ for the clone map and queue.

Should I go ahead and optimize?"

```
class _133_CloneGraph:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None
```

```

cloned: dict = {}
def clone(n):
    if n in cloned:
        return cloned[n]
    copy = Node(n.val)
    cloned[n] = copy # mark before recursing to handle cycles
    copy.neighbors = [clone(nb) for nb in n.neighbors]
    return copy
return clone(node)

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V)$ for the visited map. Mark node before recursing into neighbours –

without this, cycles cause infinite recursion. Same visited-map pattern handles any graph.

Given more time I'd ask: clone a directed graph? Same approach – just don't add reverse edges."

◆ Quick Review

Key Insights

- The graph is undirected and connected.
- Each node must be cloned exactly once to prevent cycles and repetitions.
- Use a mapping/dictionary to keep track of cloned nodes.
- A depth-first search (DFS) or breadth-first search (BFS) can be used to traverse and clone the graph.
- Handle edge cases where the graph might be empty.

Space & Time

Time Complexity: $O(N + E)$, where N is the number of nodes and E is the number of edges, because each node and each edge are traversed once.

Space Complexity: $O(N)$, to store the mapping from original nodes to their clones, and for the recursion/queue used in traversal.

Solution

We employ a depth-first search (DFS) approach to clone the graph. The main idea is to create a mapping (hash table) that links each original node to its corresponding cloned node. During the DFS traversal, if a node is encountered for the first time, a new node is created and stored in the mapping; then, the algorithm recursively clones all of its neighbors. For an already visited node, the cloned node is directly returned from the mapping. This ensures that every node is cloned only once and correctly handles cycles in the graph.

DFS

□ #200 Number of Islands

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Union Find · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:


```

Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3

```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j]` is '0' or '1'.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count connected components of '1's in a binary grid. Diagonal connections don't count. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Double loop over every grid cell. Whenever I see an unvisited '1',

run DFS/BFS to mark the entire connected component sunk, then increment island count.

Each cell visited once – correct and actually $O(m \times n)$ already.

Time: $O(m \times n)$. Space: $O(m \times n)$ worst-case recursion stack or queue.

The flood-fill is the right approach; I'll keep it clean with in-place marking."

```
class _200_NumberOfIslands:
```

```
    def numIslands(self, grid: List[List[str]]) -> int:
```

```
        m, n = len(grid), len(grid[0])
```

```
        def dfs(r, c):
```

```
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != '1':
```

```
                return
```

```
            grid[r][c] = '#' # mark visited by flooding
```

```

        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            dfs(r + dr, c + dc)
    count = 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == '1':
                dfs(r, c)
                count += 1
    return count

```

PHASE 5 — TEST & DEBUG

1-island grid: DFS floods the whole island. count=1 ✓. 3-island: count=3 ✓.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \times n)$. Space $O(m \times n)$ stack. Flooding in-place avoids a separate visited set.

BFS alternative: use a queue instead of recursion – better for very large grids (no stack overflow).

Union-Find alternative: $O(m \times n * \alpha(m \times n))$ – useful when islands merge dynamically.

Given more time I'd ask: restore the grid after counting?

Keep a separate visited set or restore '#' → '1' after each DFS."

◆ Quick Review

Key Insights

- Use graph traversal (DFS/BFS) to explore connected components of land.
- Iterate over the matrix and start a traversal (e.g., DFS) when encountering an unvisited "1".
- During traversal, mark visited cells to avoid duplicate counting.
- Each DFS/BFS call completely explores one island.
- Alternative approach can use Union Find to group connected lands.

Space & Time

Time Complexity: $O(m * n)$, as each cell is visited at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario due to recursion stack (DFS) or auxiliary data structures.

Solution

We solve the problem using a DFS-based approach. The algorithm iterates over each cell in the grid; when a "1" (land) is encountered, the DFS is initiated to mark the entire island (all connected "1"s) as visited (by setting them to "0"). This prevents recounting the same island. The island count is incremented each time a new island is discovered and fully explored.

DFS

□ #207 Course Schedule

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Graph · Topological Sort
Lists: Grind 169

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course 0 you have to first take course 1.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `true`

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]

Output: false

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq 5000$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- All the pairs $\text{prerequisites}[i]$ are unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Can all courses be finished given prerequisites (directed edges)?

Equivalent to: does the directed graph have a cycle? Return True if no cycle. Right?"

#

"[[1,0],[0,1]]: 0→1 and 1→0 form a cycle → False ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build adjacency + indegree; Kahn BFS peels zero-indegree courses layer by layer.

If we finish fewer than numCourses nodes, a cycle exists ? return false.

Same topological sort as Course Schedule II, but only need boolean completion.

Time: $O(V + E)$. Space: $O(V + E)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (3-colour DFS cycle detection)

"Color each node: 0=unvisited, 1=in current DFS path (grey), 2=fully processed (black).

Cycle exists if we encounter a grey node. $O(V+E)$."

PHASE 4 — CLEAN CODE

```
class _207_CourseSchedule:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        graph = defaultdict(list)
        for course, pre in prerequisites:
            graph[pre].append(course)
        color = [0] * numCourses
        def has_cycle(node):
            if color[node] == 1: return True # grey -> back edge -> cycle
            if color[node] == 2: return False # black -> already processed
            color[node] = 1
            for nb in graph[node]:
                if has_cycle(nb): return True
            color[node] = 2
            return False
        return not any(has_cycle(c) for c in range(numCourses))
```

PHASE 5 — TEST & DEBUG

[[1,0],[0,1]]: has_cycle(0)→grey. has_cycle(1)→grey. has_cycle(0)→grey(already) → cycle! ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. The 3-colour scheme distinguishes 'seen in current path' from 'fully done'.

Kahn's BFS alternative (in-degree tracking): count = courses processed; if count < numCourses → cycle.

Given more time I'd ask: Course Schedule II (LeetCode #210)? Same DFS, collect reverse post-order."

◆ Quick Review

Key Insights

- This problem can be modeled as a directed graph where courses are nodes and prerequisites are edges.
- If the graph has a cycle, then it is not possible to complete all courses.
- We can detect cycles using either Topological Sort (BFS/Kahn's algorithm) or DFS-based

cycle detection.

- The time complexity is $O(V + E)$, where V is the number of courses and E is the number of prerequisite pairs.

Space & Time

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

Solution

We solve the problem using the Topological Sort algorithm (BFS/Kahn's algorithm). First, we build a graph (using an adjacency list) and track the indegree (number of prerequisites) for each course. We then find all courses with an indegree of zero (courses with no prerequisites) and add them to a processing queue. While the queue is not empty, we remove a course from the queue, reduce the indegree for its neighboring courses, and if any neighbor's indegree becomes zero, we add it to the queue as well. If we successfully process all courses, then it means that the graph was acyclic and a valid schedule exists, returning true. If not, a cycle exists, and we return false.

DFS

#210 Course Schedule II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Graph · Topological Sort

Lists: Grind 169

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `[0,1]`

Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is `[0,1]`.

Example 2:

Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

Output: [0,2,1,3]

Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.

So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].

Example 3:

Input: numCourses = 1, prerequisites = []

Output: [0]

Constraints:

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq \text{numCourses} * (\text{numCourses} - 1)$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- $a_i \neq b_i$
- All the pairs $[a_i, b_i]$ are distinct.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return a valid course ordering. Return [] if impossible (cycle exists). Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Topological sort via Kahn's BFS: compute indegree of every course,

enqueue zero-indegree nodes, peel layers while decrementing neighbors.

If we process all n courses, ordering exists; otherwise a cycle remains.

Time: $O(V + E)$. Space: $O(V + E)$ for adjacency and indegree arrays.

```
# Should I go ahead and optimize?"

class _210_CourseScheduleII:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) ->
List[int]:
    graph = defaultdict(list)
    for course, pre in prerequisites:
        graph[pre].append(course)
    color = [0] * numCourses
    order = []
    def dfs(node) -> bool:
        if color[node] == 1: return False # cycle
        if color[node] == 2: return True # already done
        color[node] = 1
        for nb in graph[node]:
            if not dfs(nb): return False
        color[node] = 2
        order.append(node) # post-order: all descendants before this node
        return True
    for c in range(numCourses):
        if not dfs(c):
            return []
    return order[::-1] # reverse post-order = topological order

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E). Space O(V+E). Post-order append then reverse = topological sort.
# Given more time I'd ask: Kahn's BFS? Build in-degree map, process zero-in-degree
nodes first.
# Both approaches are O(V+E) – DFS is recursive, Kahn's is iterative."
```

◆ Quick Review

Key Insights

- This is a topological sort problem on a directed graph.
- The courses are nodes, and prerequisites form directed edges.
- We can solve the problem by either:
- Using DFS with cycle detection and a stack for topological ordering.

- Using BFS (Kahn's Algorithm) to process nodes with zero in-degrees.
- If at any point no node with a zero in-degree is available, a cycle exists and no valid order can be produced.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of courses and E is the number of prerequisite pairs.

Space Complexity: $O(V + E)$ for storing the graph and in-degree array.

Solution

We treat the problem as a topological sort on a directed acyclic graph (DAG). For a BFS approach using Kahn's algorithm: Build an adjacency list for the graph and an in-degree array to record the number of prerequisites (incoming edges) for each course. Initialize a queue with all courses that have an in-degree of zero (i.e., courses that can be taken without prerequisites). Dequeue nodes from the queue, add them to the ordering, and reduce the in-degree of their neighbors. If a neighbor's in-degree becomes zero, add it to the queue. If

the final ordering includes all courses, return the ordering; otherwise, return an empty array indicating a cycle. The chosen data structures are: A list (or array) for the in-degree counts. A dictionary (or array of lists) for the adjacency list. A queue to process nodes with zero in-degree.

DFS

□ ★ #261 Graph Valid Tree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Union Find · Graph

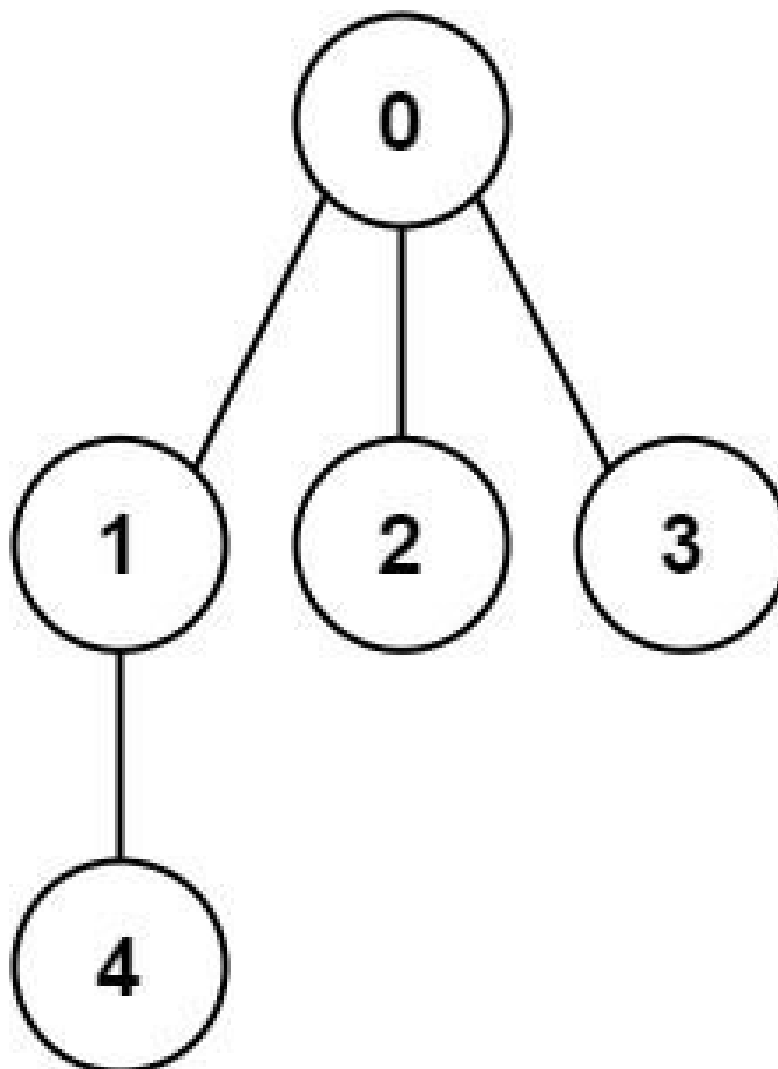
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes labeled from 0 to $n - 1$. You are given an integer n and a list of edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between nodes a_i and b_i in the graph.

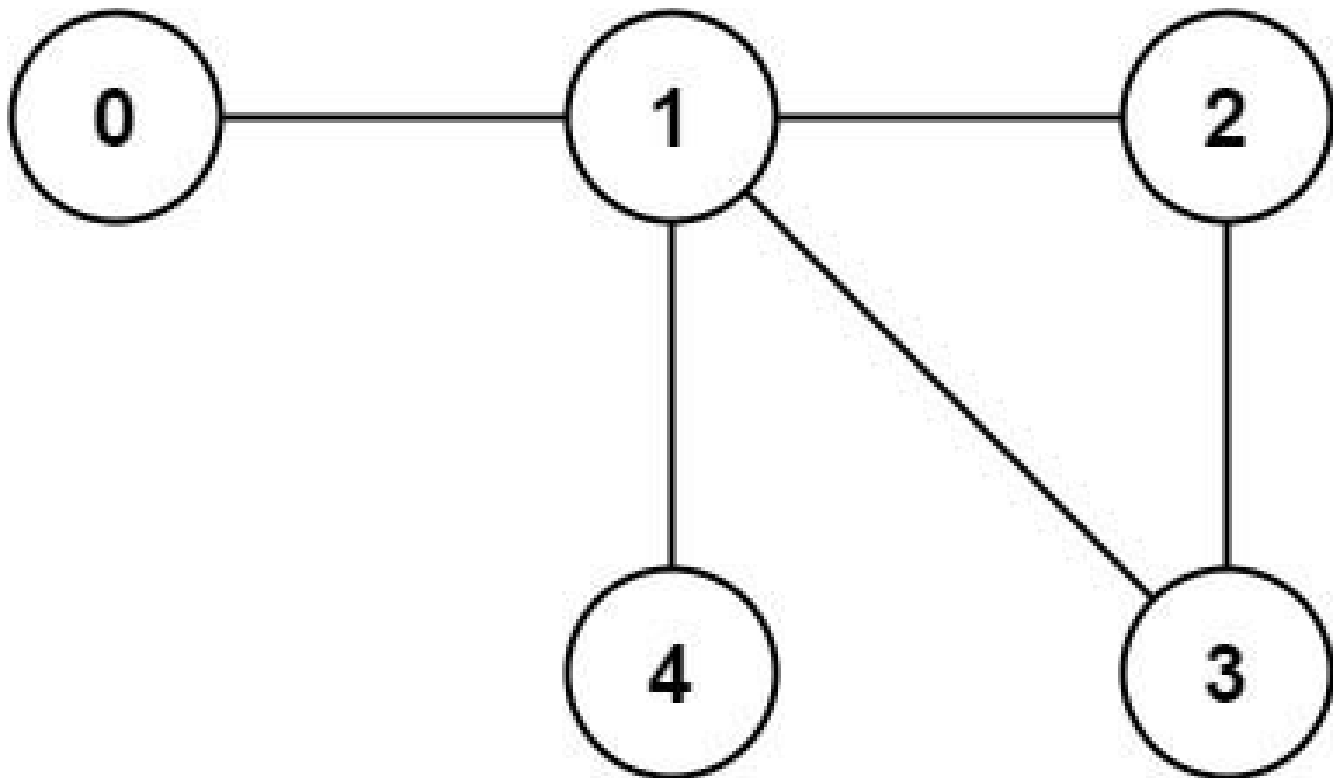
Return true if the edges of the given graph make up a valid tree, and false otherwise.

Example 1:



Input: $n = 5$, $edges = [[0,1],[0,2],[0,3],[1,4]]$
Output: true

Example 2:



Input: $n = 5$, $edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]$
Output: false

Constraints:

- $1 \leq n \leq 2000$
- $0 \leq edges.length \leq 5000$
- $edges[i].length == 2$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- There are no self-loops or repeated edges.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Valid tree: connected, acyclic, exactly $n-1$ edges. Right?
 # Any two of these imply the third for n nodes."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Build adjacency from edges, run DFS from node 0 marking visited.
 # After DFS, verify every node was visited and total edges equals $n-1$.
 # Catches disconnected graphs and cycles in one pass.
 # Time: $O(n)$. Space: $O(n)$ for adjacency + visited.
 # Should I go ahead and optimize?"

```
class _261_GraphValidTree:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        if len(edges) != n - 1:
            return False
        graph = defaultdict(list)
        for a, b in edges:
            graph[a].append(b)
            graph[b].append(a)
        visited = set()
        def dfs(node, parent):
            visited.add(node)
            for nb in graph[node]:
                if nb == parent: continue
                if nb in visited: return False
                if not dfs(nb, node): return False
            return True
        return dfs(0, -1) and len(visited) == n
```

PHASE 5 — TEST & DEBUG

$n=5$, edges=[[0,1],[0,2],[0,3],[1,4]]: 4 edges== $n-1$ ✓. DFS from 0 visits all 5. No cycle. → True ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)=O(n)$. Space $O(n)$. Early return if edges $\neq n-1$ is a fast filter.
 # Union-Find alternative: $O(n * \alpha(n))$ – merge edges, detect cycle if already connected.
 # Given more time I'd ask: disconnected forest (multiple trees)?
 # Return number of components instead – skip the len(edges)== $n-1$ check."

◆ Quick Review

Key Insights

- A tree must be acyclic and connected.

- For n nodes, a valid tree must have exactly $n - 1$ edges.
- A DFS/BFS traversal can check connectivity and simultaneously detect cycles.
- The Union-Find (disjoint set) approach provides an alternative for cycle detection and connectivity verification.

Space & Time

Time Complexity: $O(n + e)$, where e is the number of edges

Space Complexity: $O(n + e)$ for storing the graph and traversal/union-find structures

Solution

We can solve the problem using either a DFS/BFS approach or a Union-Find algorithm. For the DFS/BFS method, we first check if the number of edges is exactly $n - 1$. If not, it cannot be a tree. Then, we build an adjacency list for the graph and perform a DFS starting from node 0 while keeping track of visited nodes and parent information to avoid false-positive cycle detections. If during DFS we encounter a node that has been visited (and is not the parent), then a cycle exists.

Finally, if all nodes are reached from node 0 and no cycle is found, the graph is a valid tree. Using Union-Find, we iterate over each edge and try to union the two nodes. If they already share the same parent (i.e., are in the same set), a cycle exists. After processing all edges, we confirm that exactly one connected component exists.

DFS

□ #310 Minimum Height Trees

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Graph · Topological Sort
Lists: Grind 169

Problem

A tree is an undirected graph in which any two vertices are connected by exactly one path. In other words, any connected graph without simple cycles is a tree.

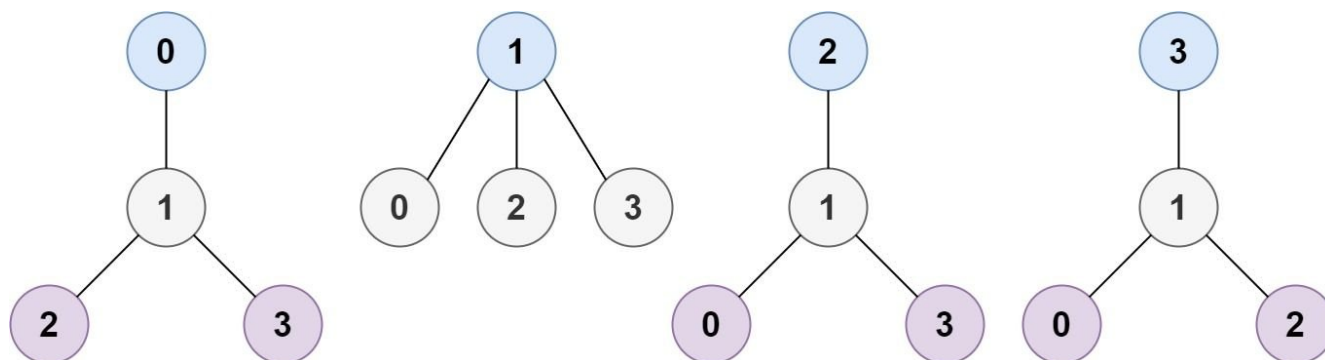
Given a tree of n nodes labelled from 0 to $n - 1$, and an array of $n - 1$ edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between the two nodes a_i and b_i in the tree, you can choose any node of the tree as the root.

When you select a node x as the root, the result tree has height h . Among all possible rooted trees, those with minimum height (i.e. $\min(h)$) are called minimum height trees (MHTs).

Return a list of all MHTs' root labels. You can return the answer in any order.

The height of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

Example 1:

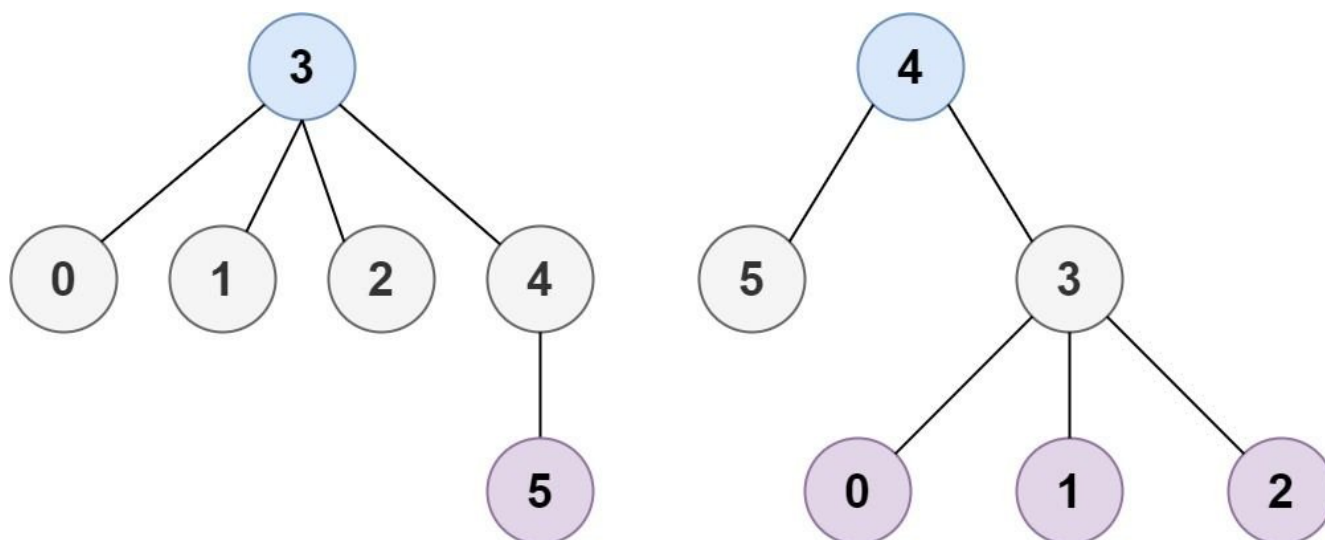


Input: $n = 4$, edges = $[[1,0],[1,2],[1,3]]$

Output: [1]

Explanation: As shown, the height of the tree is 1 when the root is the node with label 1 which is the only MHT.

Example 2:



Input: $n = 6$, edges = $[[3,0],[3,1],[3,2],[3,4],[5,4]]$

Output: [3,4]

Constraints:

- $1 \leq n \leq 2 * 10^4$
- `edges.length == n - 1`
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- All the pairs (a_i, b_i) are distinct.
- The given input is guaranteed to be a tree and there will be no repeated edges.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find which nodes make MHT roots. The answer is at most 2 nodes – the tree's center(s). Right?"

Repeatedly prune leaf nodes (degree 1) until 1 or 2 nodes remain."

#

"Like peeling an onion from outside in – the core is the answer."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Repeatedly peel all leaves layer by layer (nodes with degree 1), like peeling an onion.

The last remaining nodes (0, 1, or 2) are the tree centroids / MHT roots.

BFS leaf-removal is the standard $O(n)$ approach for this problem.

Time: $O(n)$. Space: $O(n)$ for adjacency and degree counts.

Should I go ahead and optimize?"

```
class _310_MinHeightTrees:
```

```
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
```

```
        if n == 1:
```

```
            return [0]
```

```
        graph = defaultdict(set)
```

```
        for a, b in edges:
```

```
            graph[a].add(b)
```

```
            graph[b].add(a)
```

```
        leaves = [node for node in range(n) if len(graph[node]) == 1]
```

```

remaining = n
while remaining > 2:
    remaining -= len(leaves)
    new_leaves = []
    for leaf in leaves:
        nb = next(iter(graph[leaf]))
        graph[nb].remove(leaf)
        if len(graph[nb]) == 1:
            new_leaves.append(nb)
    leaves = new_leaves
return leaves

```

PHASE 5 — TEST & DEBUG

```

# n=4, edges=[[1,0],[1,2],[1,3]]: leaves=[0,2,3]. remaining=4-3=1. new_leaves=[1].
→ [1] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n)$ . Space  $O(n)$ . This is topological sort on undirected tree – same BFS idea.

```

```

# At most 2 centers exist – if more, tree would have multiple independent long paths.

```

```

# Given more time I'd ask: all trees of minimum height (not just roots)?

```

```

# BFS from all roots found, check height – same  $O(n)$ ."

```

◆ Quick Review

Key Insights

- A tree can have at most two centers that will serve as the roots for its minimum height trees.
- Instead of trying all possible roots, we can remove leaves layer by layer until we are left with the central node(s).
- The process is similar to a topological sort on the tree.
- When there are 2 or fewer nodes left, these nodes are the centers (roots) of MHTs.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since we process each node and edge once.

Space Complexity: $O(n)$ for storing the graph and additional data structures like the leaves list.

Solution

The solution builds an undirected graph from the input, then identifies and removes node-leaves iteratively. A leaf is defined as a node connected to only one other node. By repeatedly removing the leaves and updating the graph (decreasing neighbor degrees), the algorithm peels away the outer layers of the tree. When only one or two nodes remain, these nodes are the centers of the tree and form the roots of the minimum height trees.

DFS

□ ★ #323 Number of Connected Components in an Undirected Graph ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Union Find · Graph

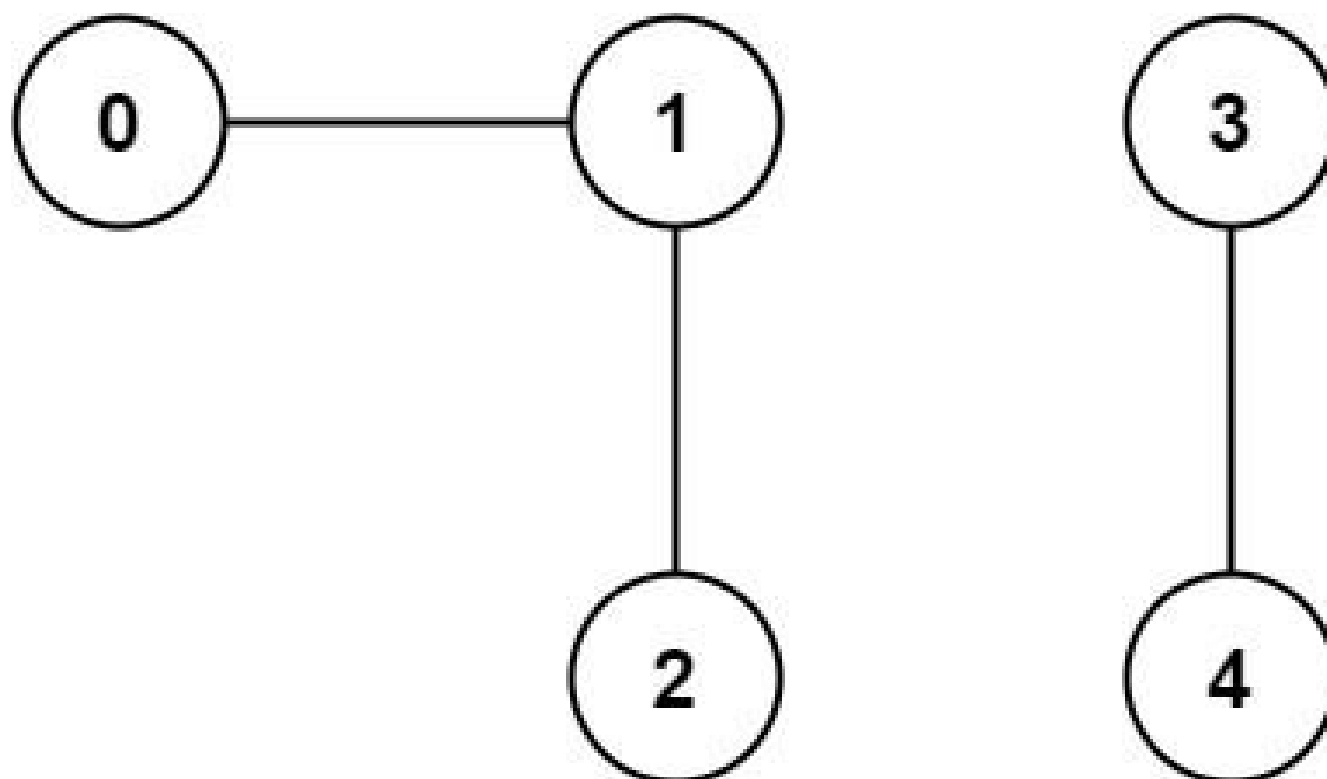
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes. You are given an integer n and an array `edges` where `edges[i] = [ai, bi]` indicates that there is an edge between a_i and b_i in the graph.

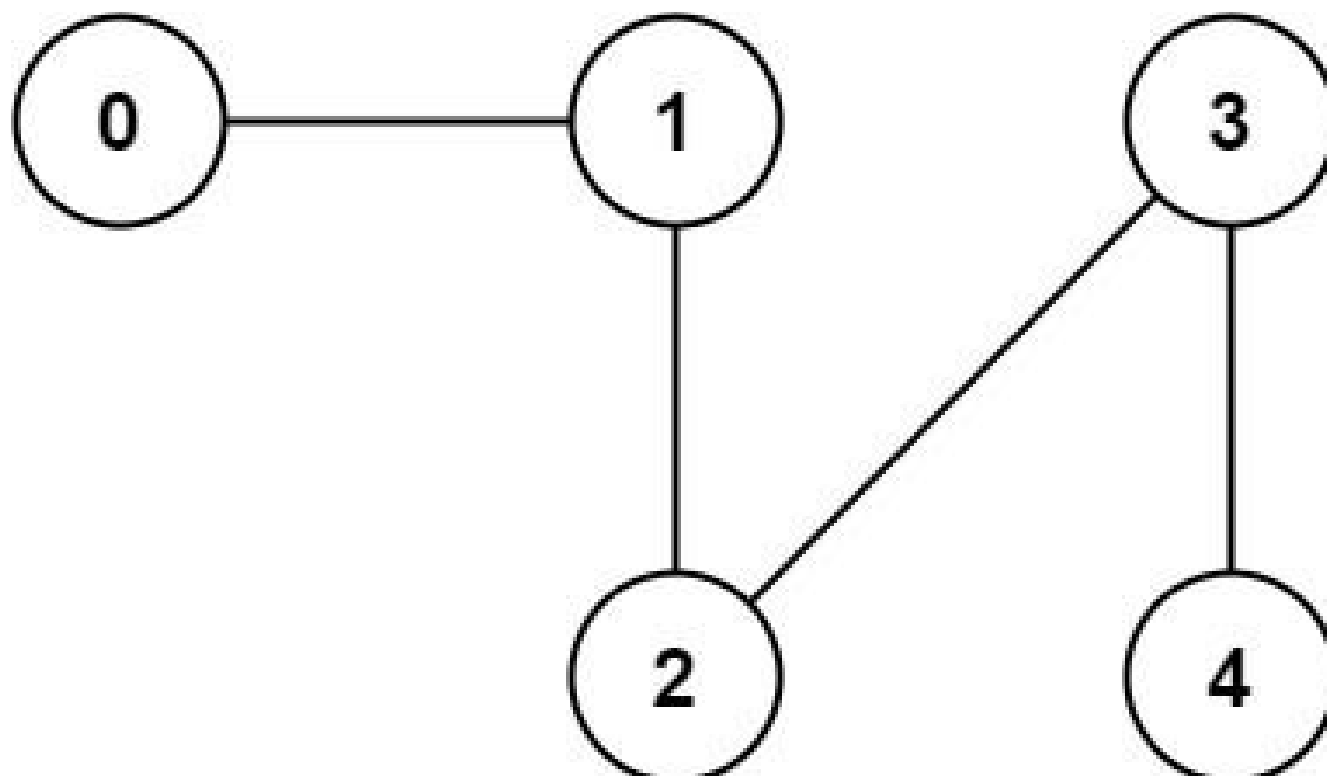
Return the number of connected components in the graph.

Example 1:



Input: $n = 5$, $edges = [[0,1],[1,2],[3,4]]$
Output: 2

Example 2:



Input: $n = 5$, $edges = [[0,1],[1,2],[2,3],[3,4]]$
 Output: 1

Constraints:

- $1 \leq n \leq 2000$
- $1 \leq edges.length \leq 5000$
- $edges[i].length == 2$
- $0 \leq a_i \leq b_i < n$
- $a_i \neq b_i$
- There are no repeated edges.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count connected components in an undirected graph. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Union-Find: for each edge, union the two endpoints.

Answer = n minus number of distinct roots after all unions.

Simple DSU with path compression – $O(n \alpha(n))$.

Time: $O(n \alpha(n))$. Space: $O(n)$ for parent array.

Should I go ahead and optimize?"

```
class _323_NumberConnectedComponents:
```

```
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
```

```
        graph = defaultdict(list)
```

```
        for a, b in edges:
```

```
            graph[a].append(b)
```

```
            graph[b].append(a)
```

```
        visited = set()
```

```
        def dfs(node):
```

```
            visited.add(node)
```

```

    for nb in graph[node]:
        if nb not in visited:
            dfs(nb)

count = 0
for node in range(n):
    if node not in visited:
        dfs(node)
        count += 1
return count

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. Standard DFS counting components.

Union-Find alternative: merge edges, count unique roots $\rightarrow O(n * \alpha(n))$.

Given more time I'd ask: what if edges are added dynamically?

Union-Find with path compression supports $O(\alpha(n))$ per merge query."

◆ Quick Review

Key Insights

- Use an adjacency list to represent the graph.
- A DFS (or BFS) traversal from an unvisited node will traverse an entire connected component.
- Each new DFS/BFS call from an unvisited node signals a separate connected component.
- Alternatively, the Union-Find data structure can be used to union connected nodes and count distinct sets.

Space & Time

Time Complexity: $O(n + e)$, where n is the number of nodes and e is the number of edges.

Space Complexity: $O(n + e)$ due to the storage of the graph and tracking of visited nodes.

Solution

We solve the problem by iterating over each node and performing a DFS for any node that hasn't been visited. The DFS uses an iterative approach with a stack to traverse all nodes in a connected component. Each DFS initiation corresponds to one connected component. This method efficiently explores the graph using an adjacency list and marks nodes as visited to prevent redundant traversals.

DFS

□ #417 Pacific Atlantic Water Flow

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix
Lists: Grind 169

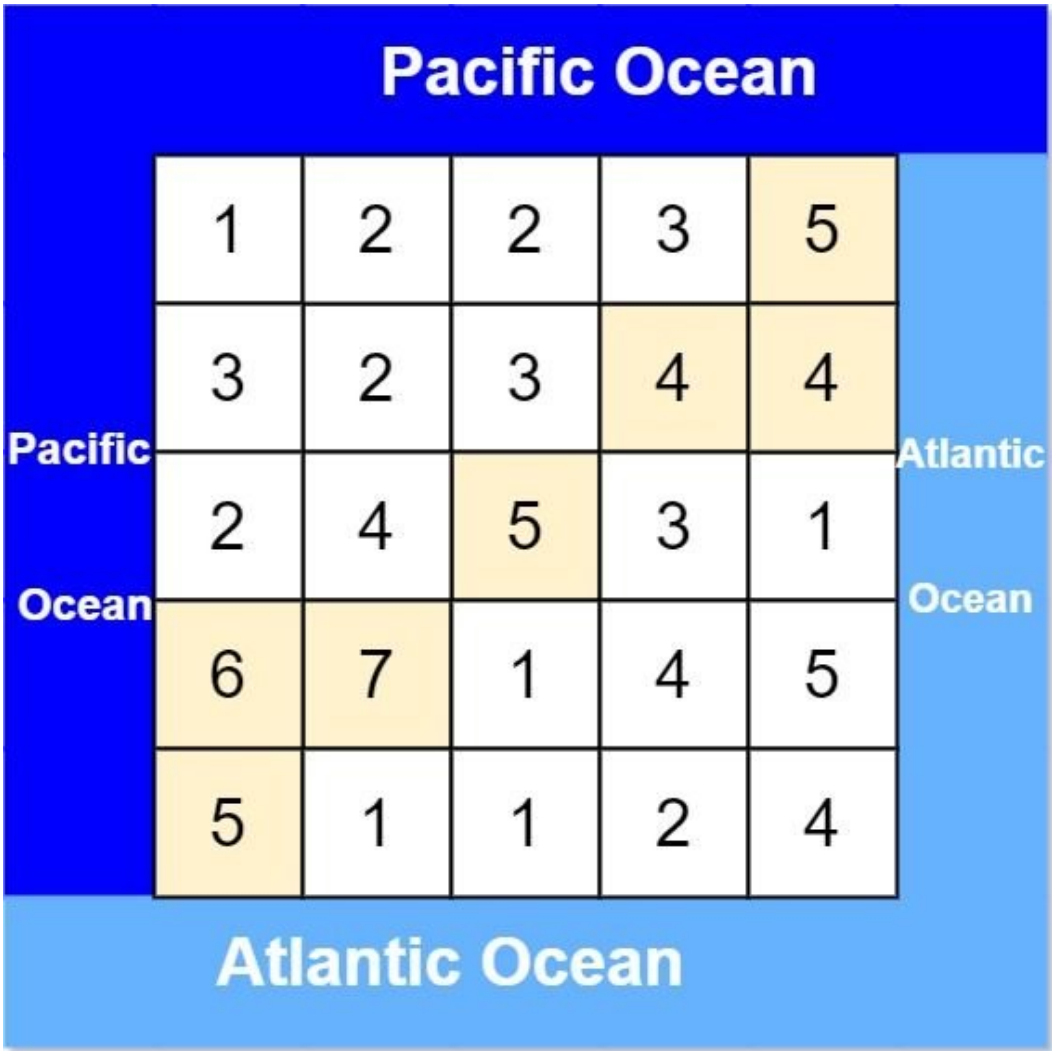
Problem

There is an $m \times n$ rectangular island that borders both the Pacific Ocean and Atlantic Ocean. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges. The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix `heights` where `heights[r][c]` represents the height above sea level of the cell at coordinate (r, c) .

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return a 2D list of grid coordinates result where $\text{result}[i] = [\text{ri}, \text{ci}]$ denotes that rain water can flow from cell (ri, ci) to both the Pacific and Atlantic oceans.

Example 1:



Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]
 Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
 Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:
 [0,4]: [0,4] -> Pacific Ocean
 [0,4] -> Atlantic Ocean
 [1,3]: [1,3] -> [0,3] -> Pacific Ocean
 [1,3] -> [1,4] -> Atlantic Ocean
 [1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean

Example 2:

Input: heights = [[1]]
 Output: [[0,0]]
 Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.

Constraints:

- $m == \text{heights.length}$
- $n == \text{heights}[r].\text{length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{heights}[r][c] \leq 105$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Water flows from higher to lower (or equal). Find cells that can flow to BOTH oceans.

Pacific touches top/left edges. Atlantic touches bottom/right edges. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

From each cell, DFS to see if it can reach both Pacific (top/left border) and Atlantic (bottom/right).

Re-run DFS per cell - $O((m*n)^2)$ naive.

Time: $O((m * n)^2)$ naive per-cell DFS. Space: $O(m * n)$ stack.

```
# Should I go ahead and optimize?"
```

```
# PHASE 3 — OPTIMIZE (reverse DFS from ocean borders)
```

```
# "Instead of simulating forward flow, run DFS BACKWARDS from each ocean's border cells.
```

```
# Reverse flow: go uphill (neighbor height >= current height).
```

```
# Intersection of Pacific-reachable and Atlantic-reachable = answer."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _417_PacificAtlantic:
```

```
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
```

```
        m, n = len(heights), len(heights[0])
```

```
        def bfs(starts):
```

```
            reachable = set(starts)
```

```
            queue = deque(starts)
```

```
            while queue:
```

```
                r, c = queue.popleft()
```

```
                for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
```

```
                    nr, nc = r + dr, c + dc
```

```
                    if 0 <= nr < m and 0 <= nc < n and (nr,nc) not in reachable \
                        and heights[nr][nc] >= heights[r][c]:
```

```
                        reachable.add((nr, nc))
```

```
                        queue.append((nr, nc))
```

```
            return reachable
```

```
        pacific = [(0, c) for c in range(n)] + [(r, 0) for r in range(1, m)]
```

```
        atlantic = [(m-1, c) for c in range(n)] + [(r, n-1) for r in range(m-1)]
```

```
        pac_reach = bfs(pacific)
```

```
        atl_reach = bfs(atlantic)
```

```
        return [[r, c] for r, c in pac_reach & atl_reach]
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Time O(m*n). Space O(m*n). BFS from borders is cleaner than DFS (no recursion limit).
```

```
# The reversal insight: 'can reach ocean' -> 'ocean can flow backward to this cell'.
```

```
# Given more time I'd ask: what if water can also flow diagonally?
```

```
# Add 4 diagonal directions - same algorithm."
```

◆ Quick Review

Key Insights

- Reverse the typical DFS/BFS perspective: start from the oceans and work inward.

- Maintain two matrices (or sets) to keep track of cells reachable from the Pacific and Atlantic edges.
- A cell can be added to the result if it is reached in both traversals.
- The height constraint ensures water flows only in non-increasing order.

Space & Time

Time Complexity: $O(m * n)$ where m is the number of rows and n is the number of columns. Each cell is processed up to twice (once from each ocean).

Space Complexity: $O(m * n)$ for the visited matrices and the recursion/queue used in DFS/BFS.

Solution

The solution uses depth-first search (DFS) from the ocean boundaries. We initialize two boolean matrices, one for each ocean. For the Pacific, we start DFS from the leftmost column and top row; for the Atlantic, we start from the rightmost column and bottom row. In each DFS, if the next cell has a height equal to or greater than the current cell and hasn't been

visited, we continue the search. Finally, we scan through the grid to collect cells that are reachable from both oceans. This reverse search simplifies the path validations and leverages the allowed water flow conditions.

DFS

□ ★ #490 The Maze ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix

Lists: ★ Premium | Premium 98

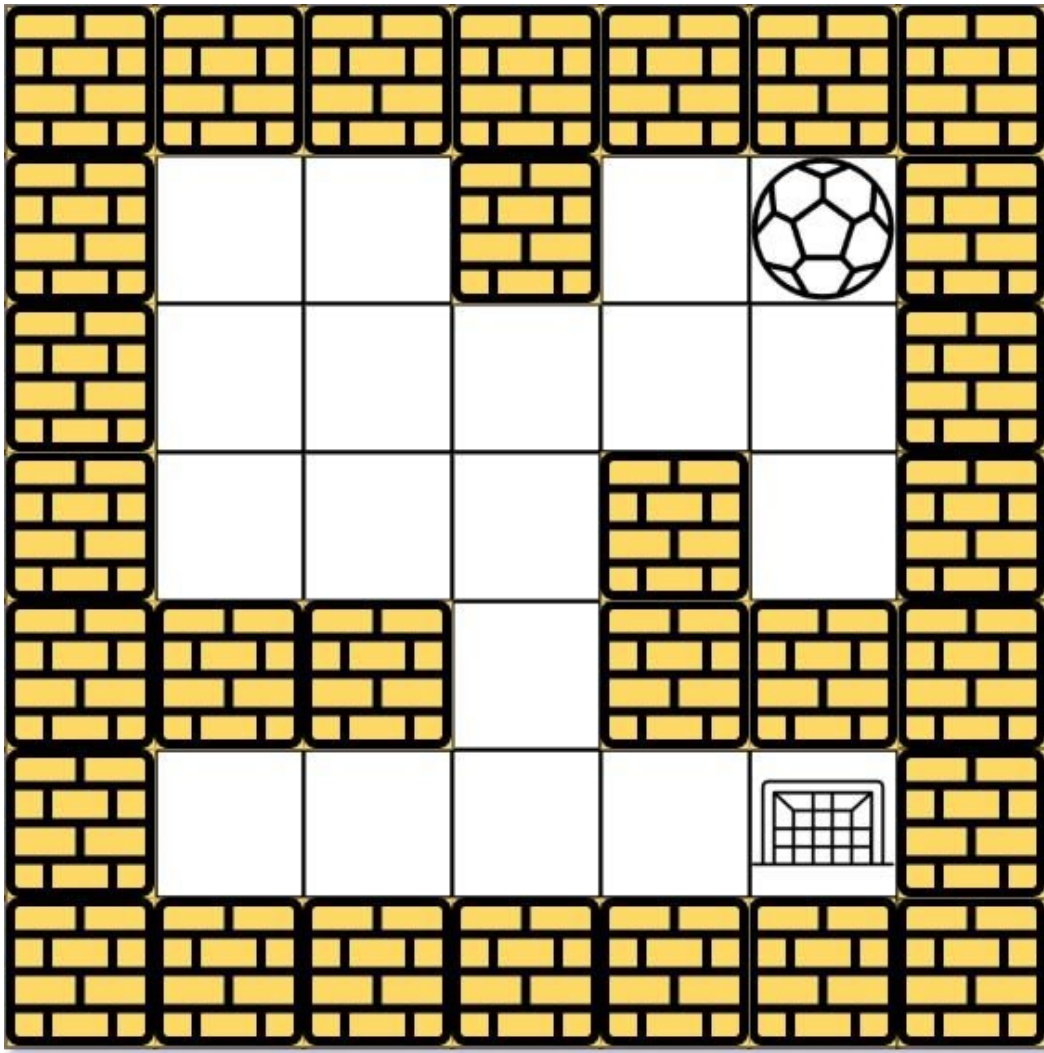
Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the $m \times n$ maze, the ball's start position and the destination, where $\text{start} = [\text{startrow}, \text{startcol}]$ and $\text{destination} = [\text{destinationrow}, \text{destinationcol}]$, return true if the ball can stop at the destination, otherwise return false.

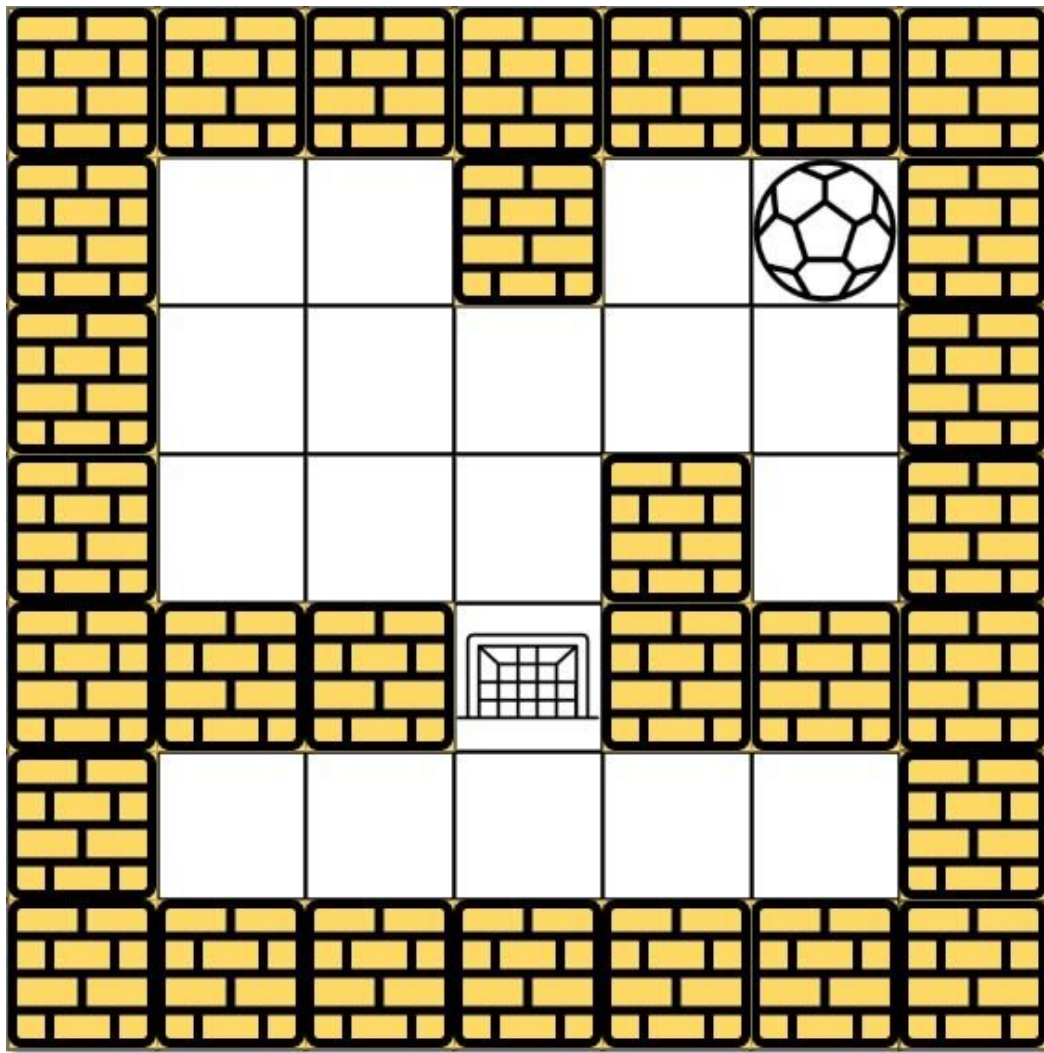
You may assume that the borders of the maze are all walls (see examples).

Example 1:



Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]`,
 start = `[0,4]`, destination = `[4,4]`
 Output: true
 Explanation: One possible way is : left -> down -> left -> down -> right ->
 down -> right.

Example 2:



Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]`,
 start = `[0,4]`, destination = `[3,2]`

Output: false

Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.

Example 3:

Input: maze = `[[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]]`,
 start = `[4,3]`, destination = `[0,1]`

Output: false

Constraints:

- `m == maze.length`
- `n == maze[i].length`

- $1 \leq m, n \leq 100$
- `maze[i][j]` is 0 or 1.
- `start.length == 2`
- `destination.length == 2`
- $0 \leq \text{startrow}, \text{destinationrow} < m$
- $0 \leq \text{startcol}, \text{destinationcol} < n$
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Ball rolls in a direction until hitting a wall. Can it STOP at the destination?
# It can pass through the destination but not stop there unless it hits a wall.
Right?"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# DFS from entrance: at each empty cell try all 4 directions.
# Mark visited cells to avoid cycles; succeed if we reach the boundary.
# Correct path exploration on an m x n grid.
# Time:  $O(m * n)$ . Space:  $O(m * n)$  visited.
# Should I go ahead and optimize?"
```

```
class _490_TheMaze:
    def hasPath(self, maze: List[List[int]], start: List[int], destination:
List[int]) -> bool:
        m, n = len(maze), len(maze[0])
        dest = tuple(destination)
        visited: set = set()
```

```

def dfs(r, c):
    if (r, c) in visited: return False
    visited.add((r, c))
    if (r, c) == dest: return True
    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
        nr, nc = r, c
        while 0 <= nr + dr < m and 0 <= nc + dc < n and maze[nr+dr][nc+dc]
== 0:
            nr += dr; nc += dc
            if dfs(nr, nc): return True
    return False
return dfs(start[0], start[1])

```

PHASE 5 — TEST & DEBUG

Ball rolls until hitting wall → stops → that stop position is a new state.
DFS on stop-positions. Destination reached only if the ball stops exactly there. ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \times n \times (m+n))$ – each stop position visits $O(m+n)$ cells to find next stop.
The visited set is on stop-positions, not individual cells.
Given more time I'd ask: The Maze II (#505) – find shortest distance?
Use Dijkstra with distance as weight, BFS/priority queue on stop-positions."

◆ Quick Review

Key Insights

- The ball continues moving in one direction until it hits a wall (or the boundary, which is considered a wall).
- You must simulate the ball's rolling rather than taking a single step.
- Even if the ball passes through the destination, it is only valid if it can stop exactly there.
- A breadth-first search (BFS) or depth-first search (DFS) strategy is ideal because we need

to explore all stopping positions.

- Use a visited structure to avoid revisiting positions and thus prevent infinite loops.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the maze.

Space Complexity: $O(m * n)$ due to the storage used by the visited matrix and the BFS/DFS queue or recursion stack.

Solution

We simulate the ball's movement using a breadth-first search (BFS) approach. Each time we pick a stop position, we try to roll in all four directions until the ball hits a wall. At that point, if the new stopping position has not been visited, we mark it as visited and add it to our queue. If we ever reach the destination exactly, we return true. The key is simulating the "roll" by iteratively moving in a direction until no further move is possible. We store the coordinates in a queue (or stack for DFS) along with a 2D visited array to monitor which positions have been processed. The directions are maintained in a list/array, and for each

direction, we keep moving until the next cell would be out of bounds or a wall.

DFS

□ ★ #694 Number of Distinct Islands ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · DFS · BFS · Union Find · Hash
Function

Lists: ★ Premium | Premium 98

Problem

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

An island is considered to be the same as another if and only if one island can be translated (and not rotated or reflected) to equal the other.

Return the number of distinct islands.

Example 1:

1	1	0	0	0
1	1	0	0	0
0	0	0	1	1
0	0	0	1	1

Input: grid = [[1,1,0,0,0],[1,1,0,0,0],[0,0,0,1,1],[0,0,0,1,1]]
Output: 1

Example 2:

1	1	0	1	1
1	0	0	0	0
0	0	0	0	1
1	1	0	1	1

Input: grid = [[1,1,0,1,1],[1,0,0,0,0],[0,0,0,0,1],[1,1,0,1,1]]
Output: 3

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0 or 1.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count islands that are distinct by shape (translation equivalent). Right?
 # Two islands are the same if one can be shifted to match the other – no rotation/reflection."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # DFS each island; encode shape as a string of moves (U/D/L/R) from start cell.
 # Add shape string to a set – distinct shapes = set size.
 # Must normalize starting cell and direction encoding consistently.
 # Time: $O(m * n)$. Space: $O(m * n)$ shapes.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (path signature hashing)

"DFS each island, record the path taken relative to the starting cell.
 # Include backtrack markers to distinguish shapes like 'L' vs reverse 'L'.
 # Hash each shape as a tuple and add to a set – count unique shapes."

PHASE 4 — CLEAN CODE

```
class _694_DistinctIslands:
    def numDistinctIslands(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        shapes: set = set()
        def dfs(r, c, r0, c0, path):
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != 1:
                return
            grid[r][c] = 0
            path.append((r - r0, c - c0))
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                dfs(r + dr, c + dc, r0, c0, path)
            path.append(None) # backtrack marker
        for r in range(m):
            for c in range(n):
                if grid[r][c] == 1:
                    path: List = []
                    dfs(r, c, r, c, path)
                    shapes.add(tuple(path))
        return len(shapes)
```

PHASE 5 — TEST & DEBUG

Two 2x2 islands: both DFS in same order → same relative path → same tuple → 1 distinct ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(m*n). Backtrack None markers distinguish different DFS
orderings
# that could produce the same sequence of offsets for different shapes.
# Given more time I'd ask: allow rotation/reflection? Normalise shape to
lexicographically smallest
# among all 8 transformations – significantly more complex."
```

◆ Quick Review

Key Insights

- Use DFS/BFS to traverse and mark each island.
- For each island, record its shape relative to a starting coordinate to allow translation invariance.
- Represent the island's shape as a unique signature (e.g., tuple or string of relative positions).
- Use a set (or hash table) to store unique shapes.

Space & Time

Time Complexity: $O(m * n)$ as each cell is visited once.

Space Complexity: $O(m * n)$ in the worst-case due to recursion stack and storing island shapes.

Solution

Use depth-first search (DFS) to explore the grid. When an unvisited land cell (1) is encountered, initiate a DFS to traverse the entire island. Record each land cell's coordinates relative to the starting position of the island. This relative representation is invariant under translation, making it possible to compare island shapes. Store the canonical shape signature (for example, as a tuple or string) in a set to ensure uniqueness. Finally, return the size of the set as the number of distinct islands.

DFS

□ #695 Max Area of Island

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Union Find · Matrix
Lists: Denny Zhang

Problem

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The area of an island is the number of cells with a value 1 in the island.

Return the maximum area of an island in grid. If there is no island, return 0.

Example 1:

0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,1,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,0,0,0,0,0,0,0,1,1,0,0,0],[0,0,0,0,0,0,0,0,0,0,1,1,0,0,0,0]]

Output: 6

Explanation: The answer is not 11, because the island must be connected 4-directionally.

Example 2:

Input: grid = [[0,0,0,0,0,0,0,0]]

Output: 0

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0 or 1.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Maximum area (cell count) of any connected island. Return 0 if no island. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each land cell, DFS to measure area of its island while marking visited.

Track the maximum area seen across all components.

Same flood-fill as Number of Islands, but accumulate area instead of count.

Time: $O(m * n)$. Space: $O(m * n)$ stack worst case.

Should I go ahead and optimize?"

```
class _695_MaxAreaIsland:
```

```
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
```

```
        m, n = len(grid), len(grid[0])
```

```
        def dfs(r, c) -> int:
```

```
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != 1:
```

```
                return 0
```

```
            grid[r][c] = 0
```

```
            return 1 + dfs(r-1,c) + dfs(r+1,c) + dfs(r,c-1) + dfs(r,c+1)
```

```
        return max((dfs(r, c) for r in range(m) for c in range(n) if grid[r][c] == 1), default=0)
```

PHASE 5 — TEST & DEBUG

Each DFS floods an island and returns its cell count. max() over all islands. ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ stack. The return-value DFS pattern: each cell contributes 1 + sum of children.

This is the same DFS flood-fill as #200 (Number of Islands) with a count return.

Given more time I'd ask: merge two islands by flipping a '0'?

Enumerate each '0' cell, BFS to find adjacent distinct islands, sum their areas + 1."

◆ Quick Review

Key Insights

- Traverse each cell in the grid.

- Use Depth-First Search (DFS) or Breadth-First Search (BFS) to explore connected 1's.
- Mark visited cells to prevent double counting (e.g., by setting them to 0).
- Maintain and update a maximum area count during traversal.
- The grid's boundaries are naturally water, so no extra edge processing is needed.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the grid since every cell is visited at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario for the recursion stack (or auxiliary data structures) when the grid is entirely land.

Solution

The solution leverages a DFS approach to iterate over each cell in the given grid. When a cell with a value of 1 is encountered, a DFS is initiated from that cell to calculate the area of the island connected to it. Cells are marked as visited (by changing them from 1 to 0) during the DFS to avoid revisiting. The DFS explores

all four directions (up, down, left, right) and accumulates the count of connected cells. The maximum area found among all islands is returned at the end. This approach efficiently computes the largest island area with a simple traversal mechanism.

DFS

□ #721 Accounts Merge

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · DFS · BFS · Union Find
Lists: Grind 169

Problem

Given a list of accounts where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are emails representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each

account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

Example 1:

Input: accounts = [["John", "johnsmith@mail.com", "john_newyork@mail.com"], ["John", "johnsmith@mail.com", "john00@mail.com"], ["Mary", "mary@mail.com"], ["John", "johnnybravo@mail.com"]]

Output: [["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.com"], ["Mary", "mary@mail.com"], ["John", "johnnybravo@mail.com"]]

Explanation:

The first and second John's are the same person as they have the common email "johnsmith@mail.com".

The third John and Mary are different people as none of their email addresses are used by other accounts.

We could return these lists in any order, for example the answer [['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'], ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still be accepted.

Example 2:

Input: accounts = [["Gabe", "Gabe0@m.co", "Gabe3@m.co", "Gabe1@m.co"], ["Kevin", "Kevin3@m.co", "Kevin5@m.co", "Kevin0@m.co"], ["Ethan", "Ethan5@m.co", "Ethan4@m.co", "Ethan0@m.co"], ["Hanzo", "Hanzo3@m.co", "Hanzo1@m.co", "Hanzo0@m.co"], ["Fern", "Fern5@m.co", "Fern1@m.co", "Fern0@m.co"]]

Output: [["Ethan", "Ethan0@m.co", "Ethan4@m.co", "Ethan5@m.co"], ["Gabe", "Gabe0@m.co", "Gabe1@m.co", "Gabe3@m.co"], ["Hanzo", "Hanzo0@m.co", "Hanzo1@m.co", "Hanzo3@m.co"], ["Kevin", "Kevin0@m.co", "Kevin3@m.co", "Kevin5@m.co"], ["Fern", "Fern0@m.co", "Fern1@m.co", "Fern5@m.co"]]

Constraints:

- $1 \leq \text{accounts.length} \leq 1000$
- $2 \leq \text{accounts}[i].\text{length} \leq 10$
- $1 \leq \text{accounts}[i][j].\text{length} \leq 30$
- $\text{accounts}[i][0]$ consists of English letters.
- $\text{accounts}[i][j]$ (for $j > 0$) is a valid email.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Merge accounts that share at least one email. Same name ≠ same person.
Output: name + sorted merged emails. Any output order. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
Union-Find on emails: union every email in the same account list.
Group emails by root parent; sort each merged list for output.
Time: $O(N \log N)$ for sorting emails. Space: $O(N)$ DSU.
Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (DFS on email graph)

"Build a graph where emails in the same account are connected.
DFS collects all emails in a connected component = one merged account.
Track owner (account name) via the first email in each account."

PHASE 4 — CLEAN CODE

```
class _721_AccountsMerge:
    def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
        email_to_name : dict = {}
        email_to_emails : defaultdict = defaultdict(set)
        for account in accounts:
            name = account[0]
            first = account[1]
            for email in account[1:]:
                email_to_name[email] = name
                email_to_emails[first].add(email)
                email_to_emails[email].add(first)
        visited: set = set()
        result = []
        def dfs(email, component):
            visited.add(email)
            component.append(email)
            for nb in email_to_emails[email]:
                if nb not in visited:
                    dfs(nb, component)
        for email in email_to_name:
            if email not in visited:
                component: List[str] = []
```

```
        dfs(email, component)
        result.append([email_to_name[email]] + sorted(component))
    return result
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(E log E) where E = total emails (sort dominates). Space O(E).
# Union-Find alternative: union all emails in an account, group by root.
# DFS on the email graph is cleaner to explain – same complexity.
# Given more time I'd ask: if same email can belong to different names?
# That would mean the data is inconsistent – flag it as an error."
```

◆ Quick Review

Key Insights

- Two accounts are merged if they share at least one common email.
- Treat each email as a node in a graph; accounts represent connected components.
- Both DFS/BFS and Union-Find can be used to find connected components.
- Keep a mapping from email to name because all emails in a connected component belong to the same person.
- After grouping, sort the emails and prepend the name to form the final account list.

Space & Time

Time Complexity: $O(N * M * \log(M))$ where N is the number of accounts and M is the number of emails per account (owing to sorting and union operations).

Space Complexity: $O(N * M)$, used for storing email mappings and union-find structures.

Solution

The solution involves using a Union-Find (Disjoint Set Union) data structure to merge emails that are connected by being on the same account. For each account, we union the first email with every other email in that account. We also maintain a mapping from email to owner name. After processing all accounts, we group the emails by their root parent derived from the union-find structure. For each group, we sort the emails and prepend the owner's name. Finally, we return the merged list of accounts.

DFS

□ #785 Is Graph Bipartite?

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Union Find · Graph
Lists: Denny Zhang

Problem

There is an undirected graph with n nodes, where each node is numbered between 0 and $n - 1$. You are given a 2D array `graph`, where `graph[u]` is an array of nodes that node u is adjacent to. More formally, for each v in `graph[u]`, there is an undirected edge between node u and node v . The graph has the following properties:

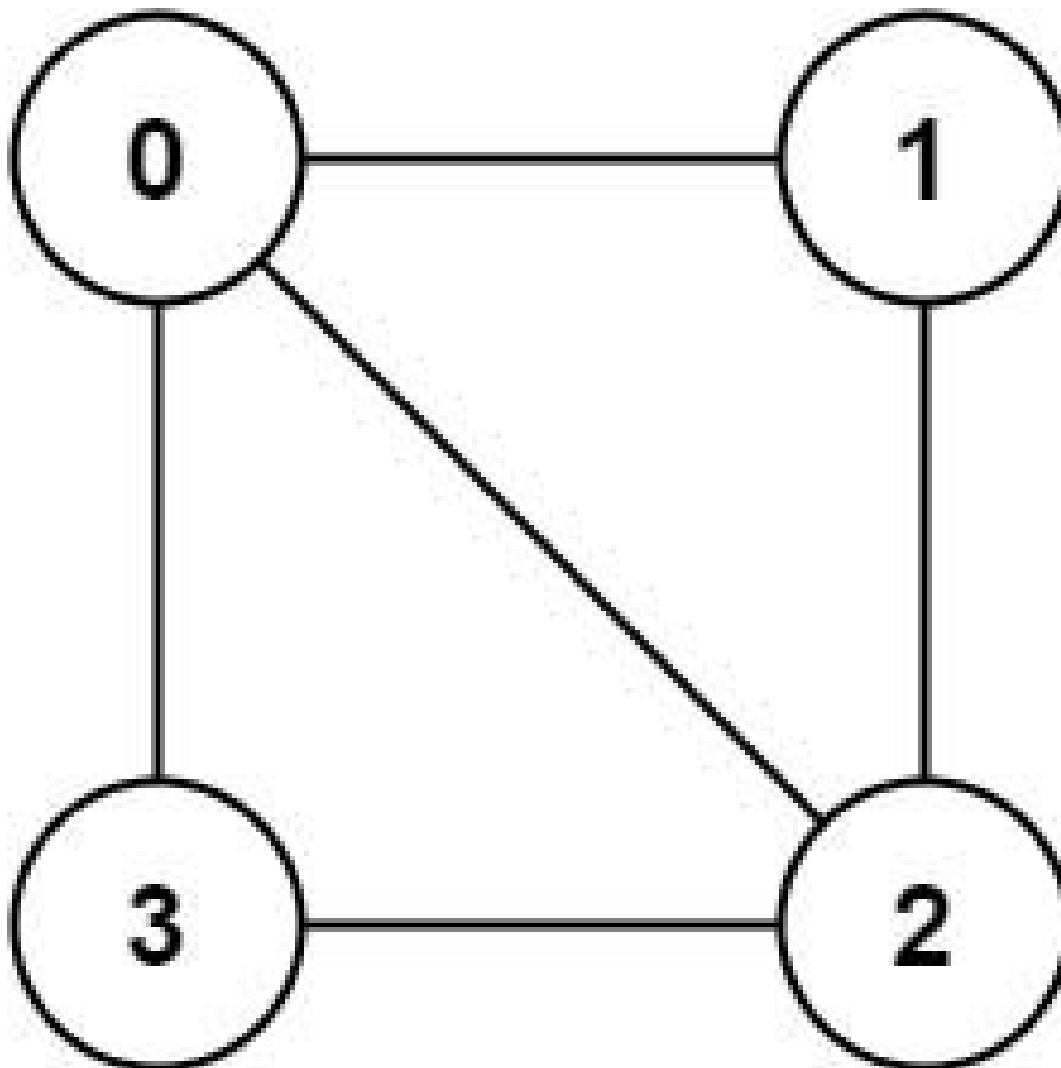
- There are no self-edges (`graph[u]` does not contain u).
- There are no parallel edges (`graph[u]` does not contain duplicate values).
- If v is in `graph[u]`, then u is in `graph[v]` (the graph is undirected).

- The graph may not be connected, meaning there may be two nodes u and v such that there is no path between them.

A graph is bipartite if the nodes can be partitioned into two independent sets A and B such that every edge in the graph connects a node in set A and a node in set B .

Return true if and only if it is bipartite.

Example 1:

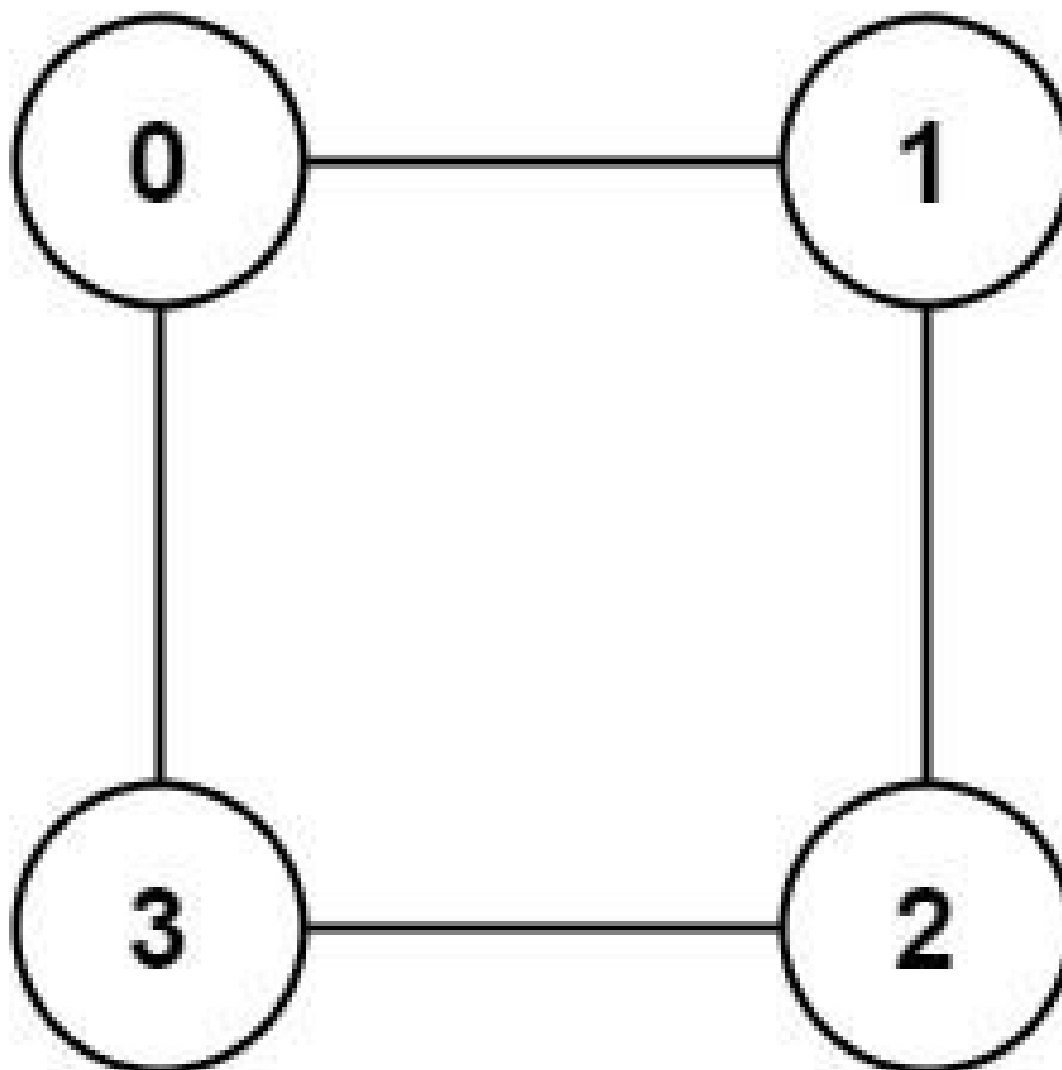


Input: graph = [[1,2,3],[0,2],[0,1,3],[0,2]]

Output: false

Explanation: There is no way to partition the nodes into two independent sets such that every edge connects a node in one and a node in the other.

Example 2:



Input: graph = [[1,3],[0,2],[1,3],[0,2]]

Output: true

Explanation: We can partition the nodes into two sets: {0, 2} and {1, 3}.

Constraints:

- graph.length == n
- 1 <= n <= 100

- $0 \leq \text{graph}[u].\text{length} < n$
- $0 \leq \text{graph}[u][i] \leq n - 1$
- $\text{graph}[u]$ does not contain u .
- All the values of $\text{graph}[u]$ are unique.
- If $\text{graph}[u]$ contains v , then $\text{graph}[v]$ contains u .

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Can we 2-colour the graph so no two adjacent nodes share the same colour?"
 # Graph may be disconnected – check every component. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
 # Try to 2-color the graph with BFS/DFS: assign color 0/1 alternating across edges.
 # If any edge connects two nodes of the same color, graph is not bipartite.
 # Standard coloring check on an undirected graph.
 # Time: $O(V + E)$. Space: $O(V)$ color + visited.
 # Should I go ahead and optimize?"

```
class _785_IsGraphBipartite:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        color = {}
        def bfs(start):
            queue = deque([start])
            color[start] = 0
            while queue:
                node = queue.popleft()
                for nb in graph[node]:
                    if nb not in color:
                        color[nb] = 1 - color[node]
                        queue.append(nb)
                    elif color[nb] == color[node]:
                        return False
            return True
```

```
return all(bfs(n) for n in range(len(graph)) if n not in color)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V)$. 2-coloring via BFS – odd-length cycle → not bipartite.

Given more time I'd ask: what if we need to output the two sets?

Return {n for n in color if color[n]==0} and {n for n in color if color[n]==1}."

◆ Quick Review

Key Insights

- A graph is bipartite if you can assign two colors to each node such that no two adjacent nodes share the same color.
- The challenge may require exploring disconnected components.
- BFS or DFS can be used to attempt the two-coloring, and a conflict during coloring indicates the graph is not bipartite.
- Union Find is an alternative approach, but BFS/DFS is more intuitive for graph coloring problems.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of nodes and E is the number of edges, since every node and edge is traversed at most once.

Space Complexity: $O(V)$, for the color array and the queue (or recursion stack) used in BFS (or

DFS).

Solution

We use a BFS/DFS approach to assign colors (0 and 1) to nodes. A color array, initially set to -1 for all nodes, is maintained. For each unvisited node (color -1), we start a BFS (or DFS) and assign it an initial color (e.g., 0). For every visited neighbor, if it has not been colored, assign it the opposite color; if it is already colored with the same color as the current node, a conflict has occurred, and the graph is not bipartite. This method gracefully handles disconnected components by iterating over all nodes.

DFS

□ ★ #1236 Web Crawler ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · DFS · BFS · Interactive
Lists: ★ Premium | Premium 98

Problem

Given a url `startUrl` and an interface `HtmlParser`, implement a web crawler to crawl all links that are under the same hostname as `startUrl`.

Return all urls obtained by your web crawler in any order.

Your crawler should:

- Start from the page: `startUrl`
- Call `HtmlParser.getUrls(url)` to get all urls from a webpage of given url.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as `startUrl`.

`http://example.org:8888/foo/bar#bang`

`hostname`

`host`

As shown in the example url above, the hostname is `example.org`. For simplicity sake, you may assume all urls use `http` protocol without any port specified. For example, the urls `http://leetcode.com/problems` and `http://leetcode.com/contest` are under the same hostname, while urls `http://example.org/test` and `http://example.com/abc` are not under the same hostname.

The `HtmlParser` interface is defined as such:

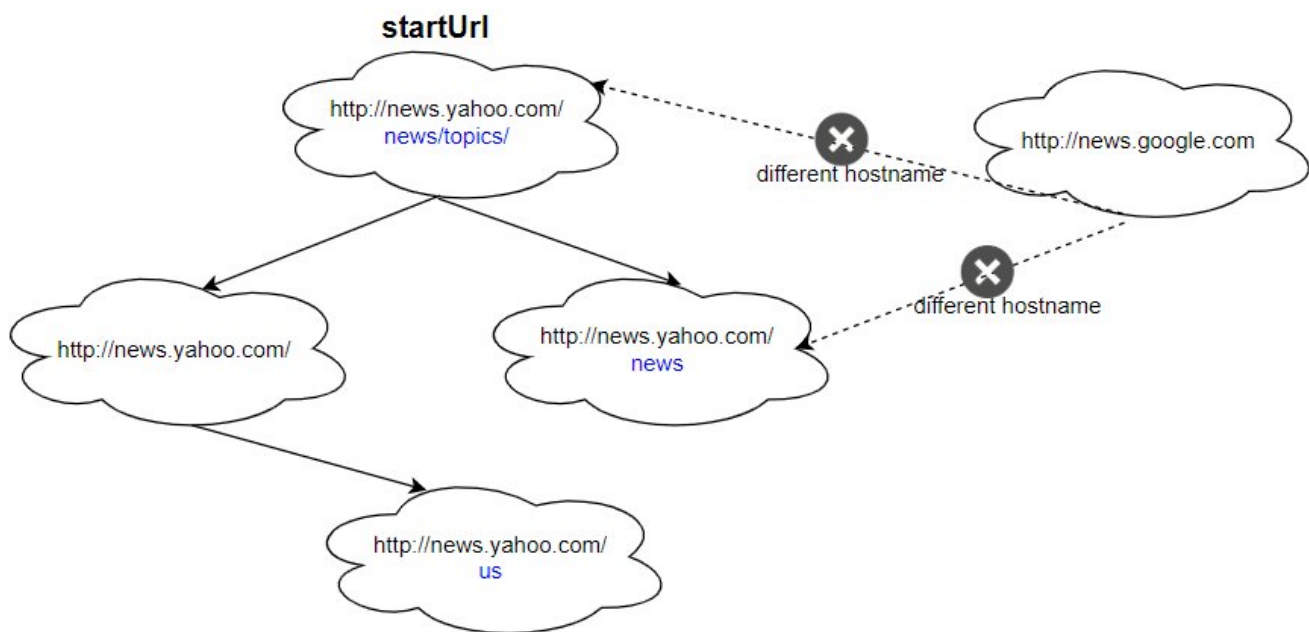
```
interface HtmlParser {  
    // Return a list of all urls from a webpage of given url.  
    public List<String> getUrls(String url);  
}
```

Below are two examples explaining the functionality of the problem, for custom testing purposes you'll have three variables `urls`, `edges` and `startUrl`. Notice that you will only have access to `startUrl` in your code, while `urls` and `edges` are not directly

accessible to you in code.

Note: Consider the same URL with the trailing slash "/" as a different URL. For example, "http://news.yahoo.com", and "http://news.yahoo.com/" are different urls.

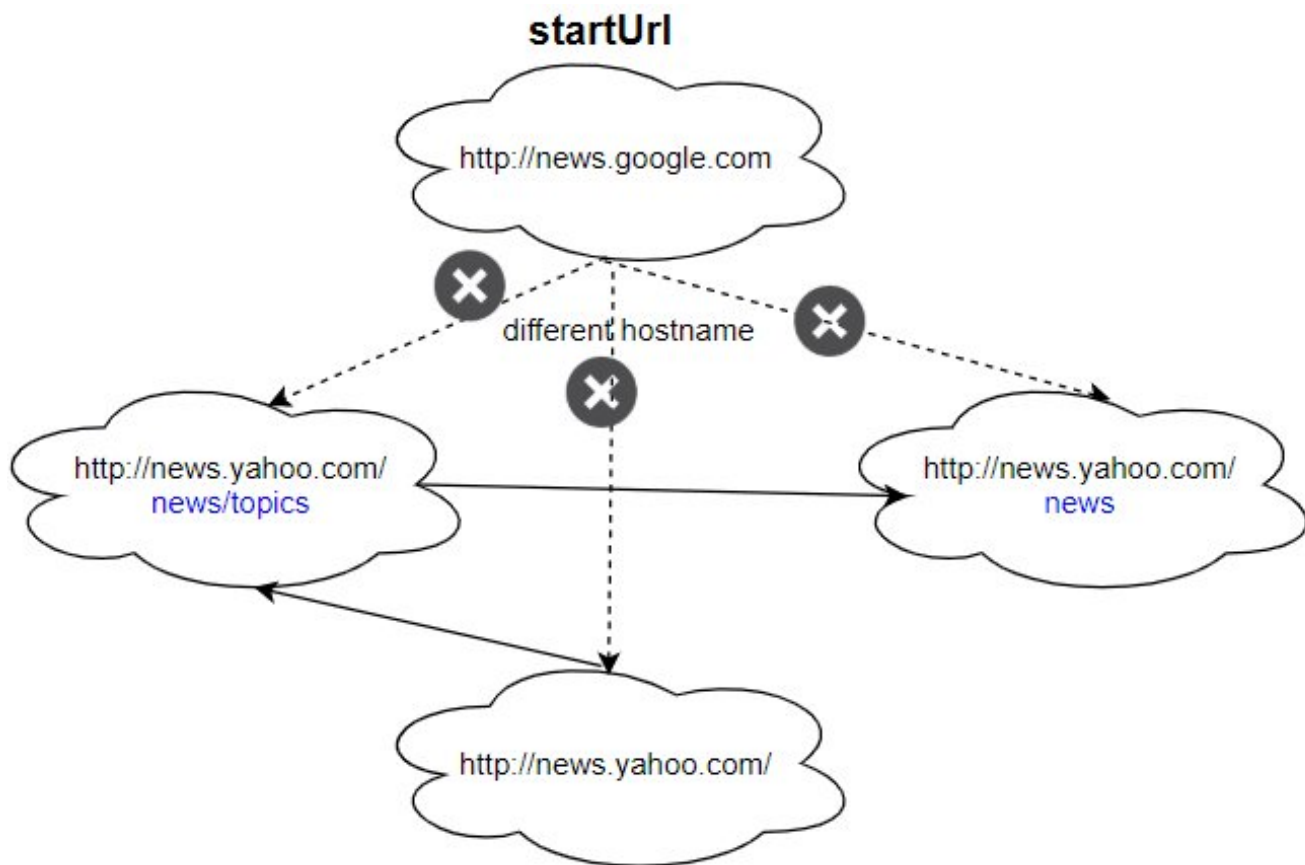
Example 1:



Input:

```
urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com",
    "http://news.yahoo.com/us"
]
```

Example 2:



Input:

```
urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com"
]
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
```

Constraints:

- $1 \leq \text{urls.length} \leq 1000$
- $1 \leq \text{urls}[i].\text{length} \leq 300$
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z',

digits from '0' to '9' and the hyphen-minus character ('-').

- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in url library.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "BFS from startUrl. Only follow links on the same hostname. Don't revisit. Right?"
# Hostname = part between '//' and the next '/'."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
# BFS from startUrl: fetch page, parse all links on same hostname, enqueue unvisited.
# Use a visited set keyed by URL string to avoid cycles.
# Standard single-threaded web crawler simulation.
# Time: O(V + E) pages + links. Space: O(V) visited.
# Should I go ahead and optimize?"
```

```
class _1236_WebCrawler:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> List[str]:
        def hostname(url):
            return url.split('/')[2] # 'http:' , '' , 'news.yahoo.com', ...
        host = hostname(startUrl)
        visited = {startUrl}
        queue = deque([startUrl])
        while queue:
            url = queue.popleft()
            for link in htmlParser.getUrls(url):
```

```

        if link not in visited and hostname(link) == host:
            visited.add(link)
            queue.append(link)

    return list(visited)

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$ where V =pages, E =links. Space $O(V)$.

Standard BFS graph traversal – the only novelty is hostname filtering.

Given more time I'd ask  #1242 Multithreaded? Use a ThreadPoolExecutor,

submit getUrls calls as futures, use a thread-safe visited set."

◆ Quick Review

Key Insights

- Treat the set of webpages as a graph where each URL is a node and an edge exists from one URL to another if it is returned by `HtmlParser.getUrls(url)`.
- Use either breadth-first search (BFS) or depth-first search (DFS) to traverse the webpages.
- Extract the hostname from a URL to ensure that only URLs on the same domain are crawled.
- Maintain a visited set (or equivalent) to avoid processing the same URL multiple times.

Space & Time

Time Complexity: $O(N + E)$ where N is the number of nodes (URLs) and E is the number of edges (links between URLs) encountered

during the crawl.

Space Complexity: $O(N)$ for storing the visited URLs and the queue/stack used for traversal.

Solution

The solution uses a BFS (or DFS) approach to traverse the web graph: Extract the hostname from `startUrl` (for instance, by splitting on `"://"` and then the first `"/"` that follows). Initialize a visited set to keep track of URLs that have been crawled. Use a queue to implement the BFS. Enqueue the `startUrl`. While the queue is not empty, dequeue a URL and retrieve its outgoing links using `HtmlParser.getUrls(url)`. For each URL obtained: Extract its hostname and compare it with the hostname of `startUrl`. If it matches and it has not been visited before, add it to the visited set and enqueue it. Finally, return the set of visited URLs. This approach guarantees that each URL is processed only once and that only URLs from the same hostname are crawled.

DFS

#1242 Web Crawler Multithreaded

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Concurrency

Lists: Denny Zhang

Problem

Given a URL `startUrl` and an interface `HtmlParser`, implement a Multi-threaded web crawler to crawl all links that are under the same hostname as `startUrl`.

Return all URLs obtained by your web crawler in any order.

Your crawler should:

- Start from the page: `startUrl`
- Call `HtmlParser.getUrls(url)` to get all URLs from a webpage of a given URL.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as `startUrl`.

http://example.org:8888/foo/bar#bang

hostname

host

As shown in the example URL above, the hostname is example.org. For simplicity's sake, you may assume all URLs use HTTP protocol without any port specified. For example, the URLs `http://leetcode.com/problems` and `http://leetcode.com/contest` are under the same hostname, while URLs `http://example.org/test` and `http://example.com/abc` are not under the same hostname.

The `HtmlParser` interface is defined as such:

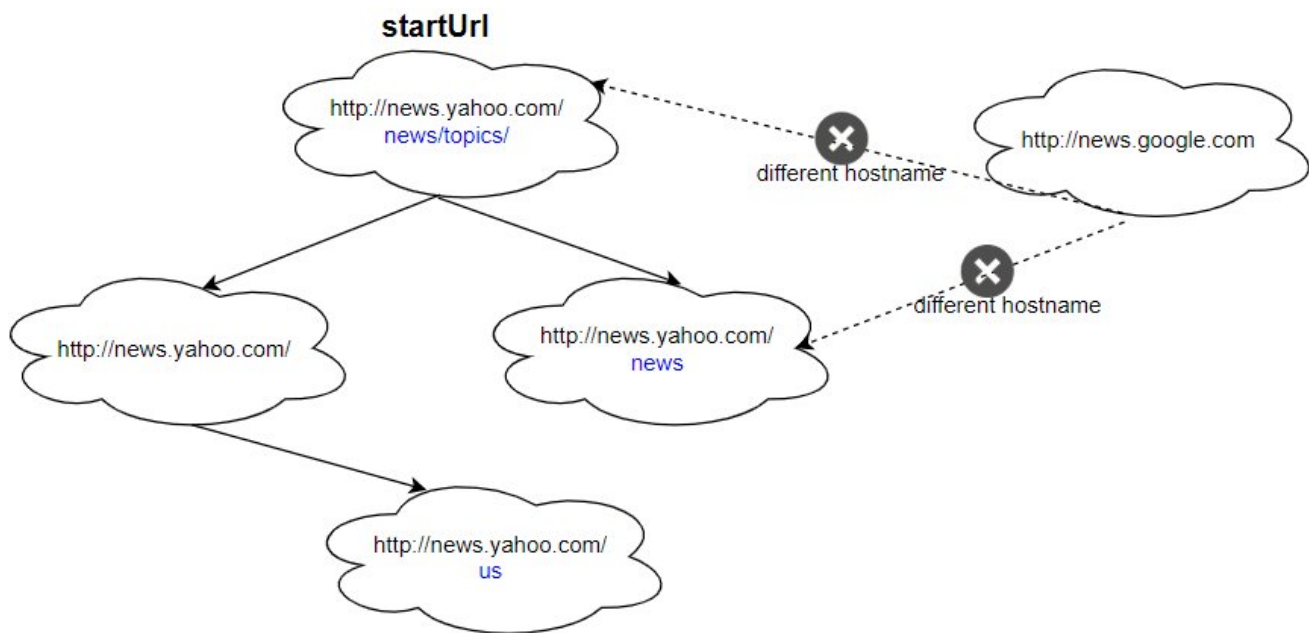
```
interface HtmlParser {  
    // Return a list of all urls from a webpage of given url.  
    // This is a blocking call, that means it will do HTTP request and return when  
    // this request is finished.  
    public List<String> getUrls(String url);  
}
```

Note that `getUrls(String url)` simulates performing an HTTP request. You can treat it as a blocking function call that waits for an HTTP request to finish. It is guaranteed that `getUrls(String url)` will return the URLs within 15ms. Single-threaded solutions will exceed the

time limit so, can your multi-threaded web crawler do better?

Below are two examples explaining the functionality of the problem. For custom testing purposes, you'll have three variables `urls`, `edges` and `startUrl`. Notice that you will only have access to `startUrl` in your code, while `urls` and `edges` are not directly accessible to you in code.

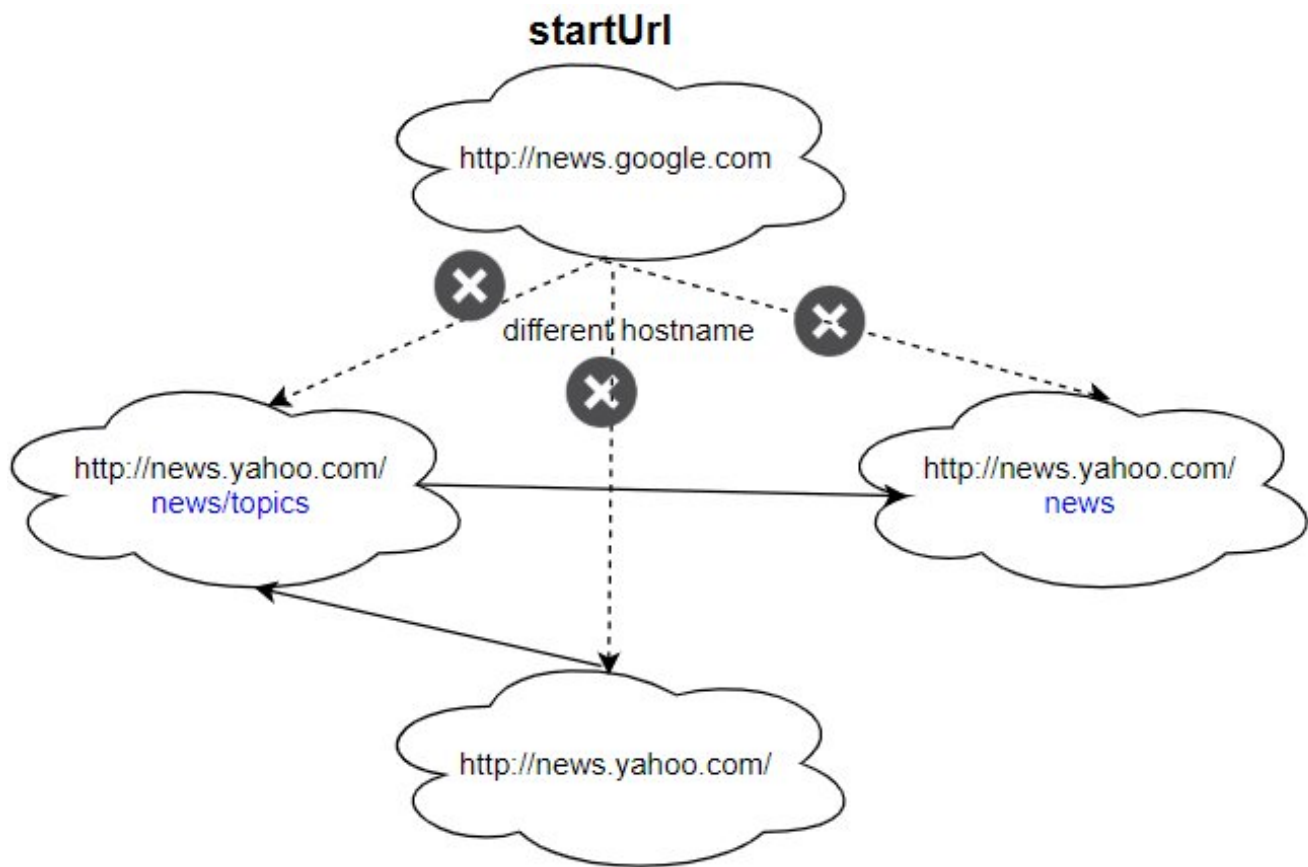
Example 1:



Input:

```
urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com",
    "http://news.yahoo.com/us"
]
```

Example 2:



Input:

```
urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com"
]
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
```

Constraints:

- $1 \leq \text{urls.length} \leq 1000$
- $1 \leq \text{urls}[i].\text{length} \leq 300$
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z',

digits from '0' to '9' and the hyphen-minus character ('-').

- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in the URL library.

Follow up:

1. Assume we have 10,000 nodes and 1 billion URLs to crawl. We will deploy the same software onto each node. The software can know about all the nodes. We have to minimize communication between machines and make sure each node does equal amount of work. How would your web crawler design change?
2. What if one node fails or does not work?
3. How do you know when the crawler is done?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Same as #1236 but with multi-threading since getUrls is blocking and slow.

Use threads to fetch pages in parallel. Thread-safe visited set required. Right?"

PHASE 4 — CLEAN CODE (threading approach)

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import threading
```

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Same BFS crawler as single-threaded, but dispatch each getLinks call to a worker thread.

Shared visited set must be synchronized; join threads when queue is empty.

Correct parallelism model for the LeetCode concurrency variant.

Time: $O(V + E)$ with parallel fetch. Space: $O(V)$ visited + thread overhead.

Should I go ahead and optimize?"

```
class _1242_WebCrawlerMT:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> List[str]:
        def hostname(url):
            return url.split('/')[2]
        host = hostname(startUrl)
        visited = {startUrl}
        lock = threading.Lock()
        def fetch(url):
            neighbors = []
            for link in htmlParser.getUrls(url):
                with lock:
                    if link not in visited and hostname(link) == host:
                        visited.add(link)
                        neighbors.append(link)
            return neighbors
        queue = [startUrl]
        with ThreadPoolExecutor() as executor:
            while queue:
                futures = {executor.submit(fetch, url): url for url in queue}
                queue = []
                for f in as_completed(futures):
                    queue.extend(f.result())
        return list(visited)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Threads cut wall-clock time proportional to how much getUrls blocks.

Lock only needed when checking/adding to visited – minimize critical section.

Distributed follow-up (10,000 nodes, 1B URLs): consistent hashing to partition URLs by hostname.

Each node owns a hostname range. Communicate frontier pages between nodes.

'Done' signal: all queues empty and all workers idle (barrier synchronisation)."

◆ Quick Review

Key Insights

- Use multiple threads to perform concurrent crawling, which speeds up the process over a single-threaded approach.
- Maintain a thread-safe set (visited) to ensure no URL is processed twice.
- Utilize a work queue to manage URLs pending processing.
- Filter URLs based on the hostname extracted from `startUrl`.
- Apply synchronization (locks or concurrent data structures) to avoid race conditions when multiple threads access shared data.

Space & Time

Time Complexity: $O(N)$ where N is the number of URLs processed, with additional constant factors due to threading overhead.

Space Complexity: $O(N)$ for storing the visited URLs and the work queue.

Solution

The solution initializes a shared work queue with the `startUrl` and a thread-safe visited set. Each worker thread dequeues a URL, retrieves its linked URLs via the blocking `HtmlParser.getUrls()` call, and filters the resulting URLs by comparing their hostnames with the start URL's hostname. New URLs that have not yet been visited are added to both the visited set and the work queue. The use of a thread pool or multiple worker threads along with synchronization ensures that the crawler processes URLs concurrently while avoiding race conditions. The approach follows a BFS-like pattern spread across multiple threads.

Graphs

Round 2 · High · Medium · 3 questions

Graphs

□ #128 Longest Consecutive Sequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Union Find
Lists: Grind 169 | CodeSignal

Problem

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

```
Input: nums = [100,4,200,1,3,2]
Output: 4
Explanation: The longest consecutive elements sequence is [1, 2, 3, 4].
Therefore its length is 4.
```

Example 2:

```
Input: nums = [0,3,7,2,5,8,4,6,0,1]
Output: 9
```

Example 3:

```
Input: nums = [1,0,1,2]
Output: 3
```

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the longest run of consecutive integers. Must be $O(n)$. Return the length. Right?

Duplicates don't extend the sequence — $[1,1,2] \rightarrow \text{length } 2$."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Put all numbers in a hash set. For each num, if $\text{num}-1$ is not in set, walk num, $\text{num}+1$, $\text{num}+2$...

counting streak length. Track longest streak.

Each number visited once across all walks — $O(n)$ with the set.

Time: $O(n)$. Space: $O(n)$ set.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (hash set + only start from sequence beginnings)

"Build a set. For each x where $x-1$ is NOT in the set (x is a sequence start),

count upward. Each element is processed at most once total $\rightarrow O(n)$."

PHASE 4 — CLEAN CODE

```
class _128_LongestConsecutive:
    def longestConsecutive(self, nums: List[int]) -> int:
        num_set = set(nums)
        best = 0
        for x in num_set:
            if x - 1 not in num_set: # x is a sequence start
                length = 1
                while x + 1 in num_set:
                    x += 1; length += 1
                best = max(best, length)
        return best
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Each number is the starting point of at most one sequence scan.

The 'start of sequence' check prevents redundant scans — the key insight.

Given more time I'd ask: streaming input? Use a dict mapping each endpoint

of a range to its length, merging ranges as new numbers arrive."

◆ Quick Review

Key Insights

- Utilize a set (or hash table) for constant time look-ups.
- Only start counting a sequence from a number that is the beginning (its previous number is not in the set).
- Each number is visited only once, ensuring an $O(n)$ time complexity.
- Update the maximum sequence length as you discover longer consecutive sequences.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We first insert all the numbers into a hash set to enable $O(1)$ membership checks. For each number in the set, we check if it is the start of a sequence by ensuring that the number minus one is not in the set. If it is the beginning, we then count all consecutive numbers following it, updating the maximum sequence length

when necessary. This method guarantees that we only count each number once, satisfying the $O(n)$ time requirement.

Graphs

□ ★ #277 Find the Celebrity ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · Graph · Interactive
Lists: ★ Premium | Premium 98

Problem

Suppose you are at a party with n people labeled from 0 to $n - 1$ and among them, there may exist one celebrity. The definition of a celebrity is that all the other $n - 1$ people know the celebrity, but the celebrity does not know any of them.

Now you want to find out who the celebrity is or verify that there is not one. You are only allowed to ask questions like: "Hi, A. Do you know B?" to get information about whether A knows B. You need to find out the celebrity (or verify there is not one) by asking as few questions as possible (in the asymptotic sense).

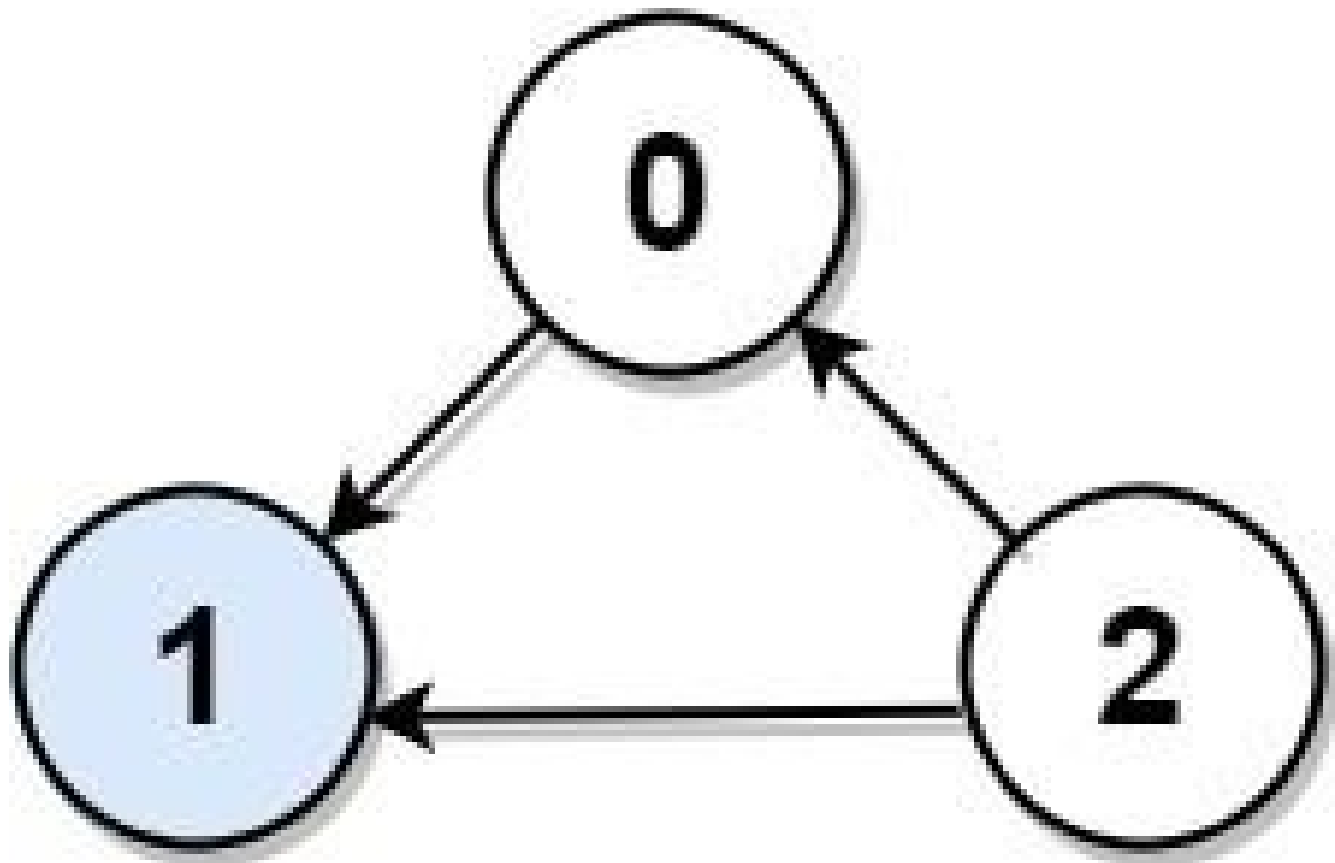
You are given an integer n and a helper function `bool knows(a, b)` that tells you whether a knows b. Implement a function `int findCelebrity(n)`.

There will be exactly one celebrity if they are at the party.

Return the celebrity's label if there is a celebrity at the party. If there is no celebrity, return -1.

Note that the $n \times n$ 2D array graph given as input is not directly available to you, and instead only accessible through the helper function knows. $\text{graph}[i][j] == 1$ represents person i knows person j , whereas $\text{graph}[i][j] == 0$ represents person j does not know person i .

Example 1:

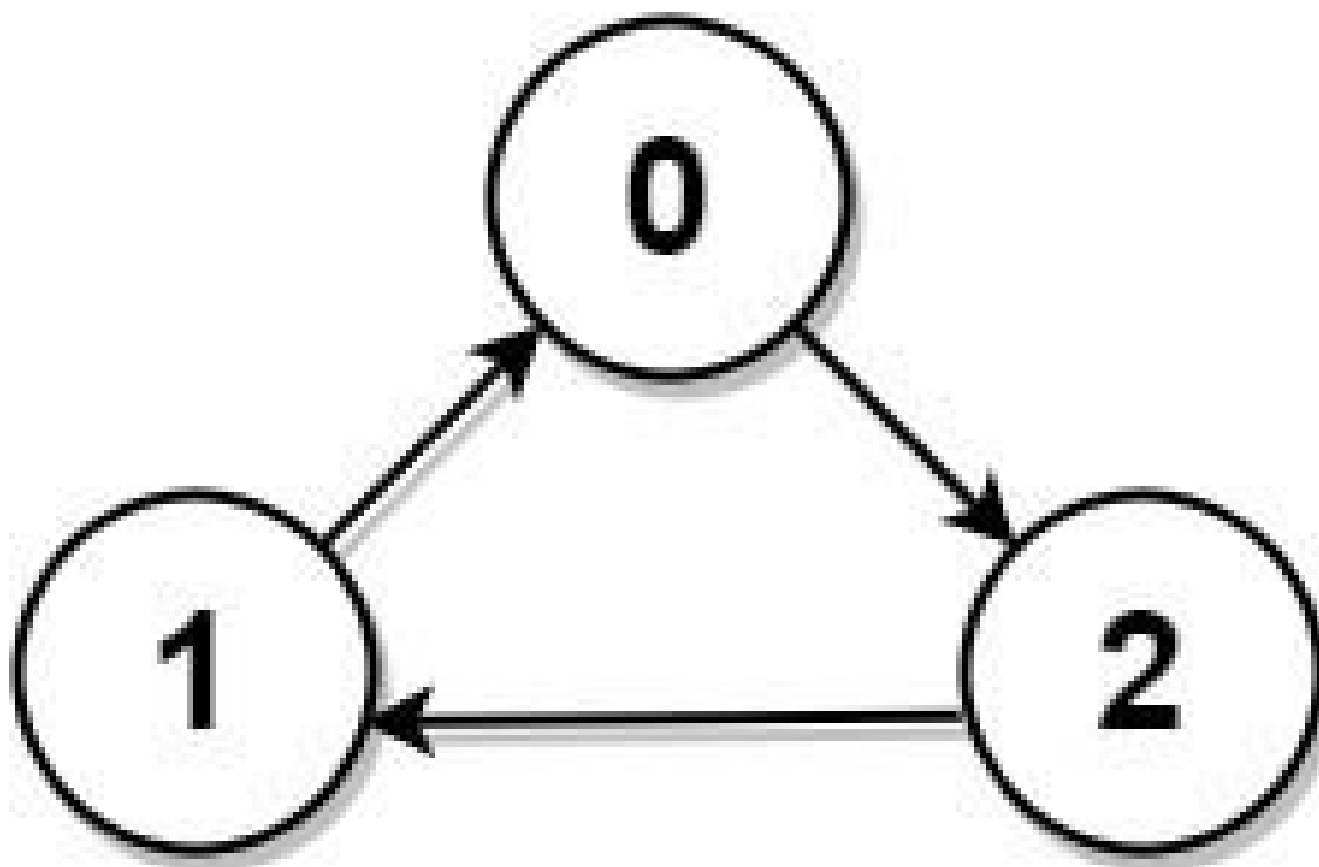


Input: graph = [[1,1,0],[0,1,0],[1,1,1]]

Output: 1

Explanation: There are three persons labeled with 0, 1 and 2. graph[i][j] = 1 means person i knows person j, otherwise graph[i][j] = 0 means person i does not know person j. The celebrity is the person labeled as 1 because both 0 and 2 know him but 1 does not know anybody.

Example 2:



Input: graph = [[1,0,1],[1,1,0],[0,1,1]]

Output: -1

Explanation: There is no celebrity.

Constraints:

- $n == \text{graph.length} == \text{graph}[i].\text{length}$
- $2 \leq n \leq 100$
- graph[i][j] is 0 or 1.
- graph[i][i] == 1

Follow up: If the maximum number of allowed calls to the API knows is $3 * n$, could you find a solution without exceeding the maximum number of calls?

My LeetCode Solution
 Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Celebrity: everyone knows them, they know no one.
# Find them using knows(a,b) API — minimise calls. Right?
# At most one celebrity can exist."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Ask every other person if they know celebrity; candidate must be known by all and
know nobody.
# Two-pass elimination: first narrow to one candidate with  $O(n)$  knows() calls, then
verify.
# Naive all-pairs check is  $O(n^2)$  knows() calls.
# Time:  $O(n^2)$  naive,  $O(n)$  with elimination. Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (candidate elimination + verify)

```
# "Pass 1: start with candidate=0, scan all i. If knows(candidate, i): celebrity
can't be candidate → candidate=i.
# Pass 2: verify candidate — everyone must know them AND they know no one.
# Total API calls:  $O(n)$ . Without the two-pass we'd need  $O(n^2)$ ."
```

PHASE 4 — CLEAN CODE

```
class _277_FindCelebrity:
    def findCelebrity(self, n: int) -> int:
        candidate = 0
        for i in range(1, n):
            if knows(candidate, i):
                candidate = i
        for i in range(n):
            if i == candidate:
                continue
            if knows(candidate, i) or not knows(i, candidate):
```

```
        return -1
    return candidate
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – $2n-2$ API calls in the worst case. Space $O(1)$.

The candidate elimination logic: if A knows B, A is not the celebrity (celebrity knows no one).

If A doesn't know B, B is not the celebrity (celebrity is known by all).

One of them is eliminated per comparison – $O(n)$ total.

Given more time I'd ask: what if there's a possibility of no celebrity?

The current solution handles this – the verification pass returns -1 if candidate fails."

◆ Quick Review

Key Insights

- Use pairwise comparisons to eliminate non-celebrities.
- If a candidate knows another person, the candidate cannot be the celebrity.
- Verify the candidate by checking that they do not know anyone and that everyone knows them.
- Achieve the solution in two passes: one for candidate selection and the other for validation.

Space & Time

Time Complexity: $O(n)$ – One pass for candidate selection and one for validation.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

The approach consists of two main steps:

Candidate Selection: Iterate through the people and maintain a candidate. For every person i , if the current candidate knows i , then set $\text{candidate} = i$. This works because if candidate knows i , candidate cannot be the celebrity.

Verification: Check that the candidate does not know any other person and that every other person knows the candidate. If both conditions hold, candidate is the celebrity; otherwise, return -1 . The main data structures used are simple integer variables. The algorithm leverages a two-pass technique to reduce the number of knows API calls while ensuring correctness.

Graphs

□ ★ #1136 Parallel Courses ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Graph · Topological Sort

Lists: ★ Premium | Premium 98

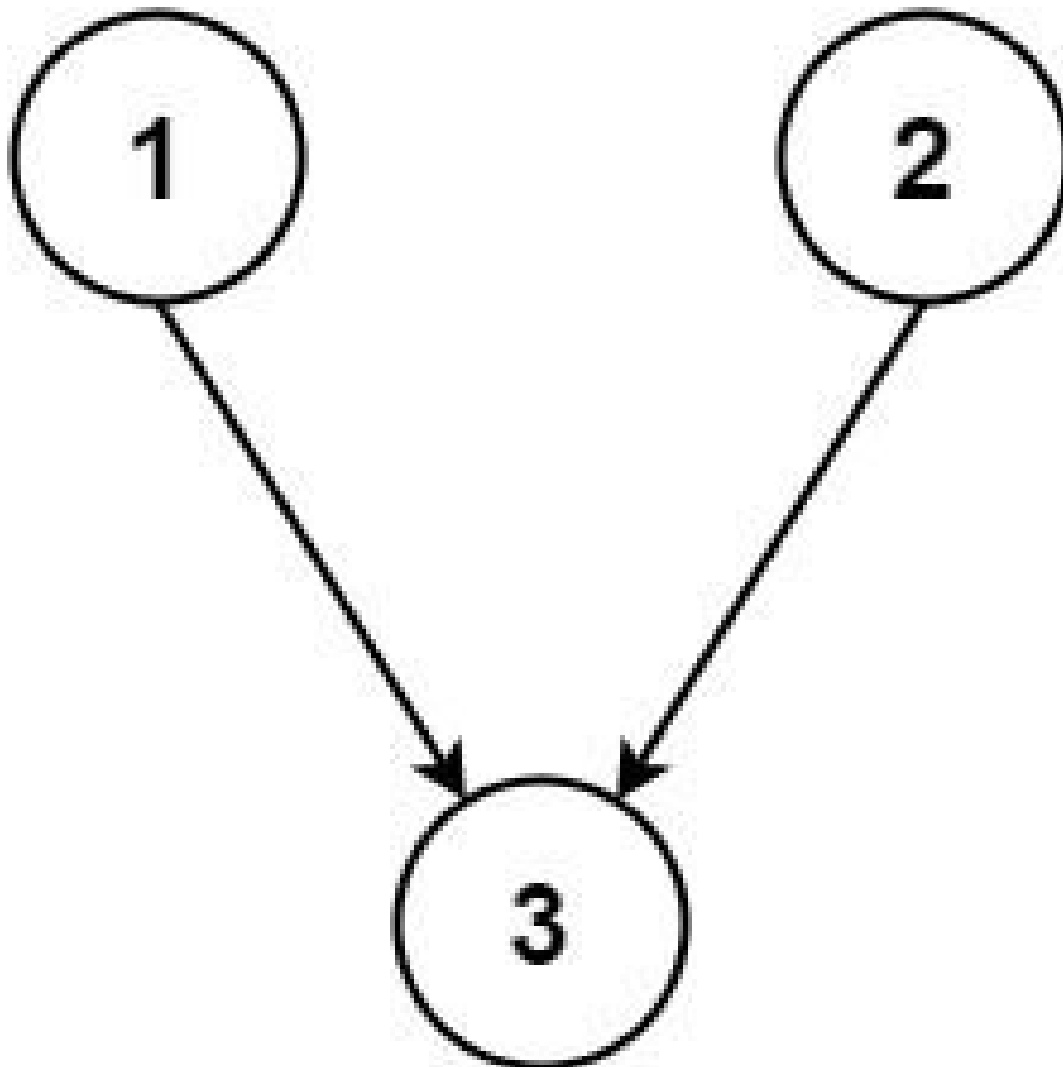
Problem

You are given an integer n , which indicates that there are n courses labeled from 1 to n . You are also given an array `relations` where `relations[i] = [prevCoursei, nextCoursei]`, representing a prerequisite relationship between course `prevCoursei` and course `nextCoursei`: course `prevCoursei` has to be taken before course `nextCoursei`.

In one semester, you can take any number of courses as long as you have taken all the prerequisites in the previous semester for the courses you are taking.

Return the minimum number of semesters needed to take all courses. If there is no way to take all the courses, return -1 .

Example 1:



Input: $n = 3$, $\text{relations} = [[1,3],[2,3]]$

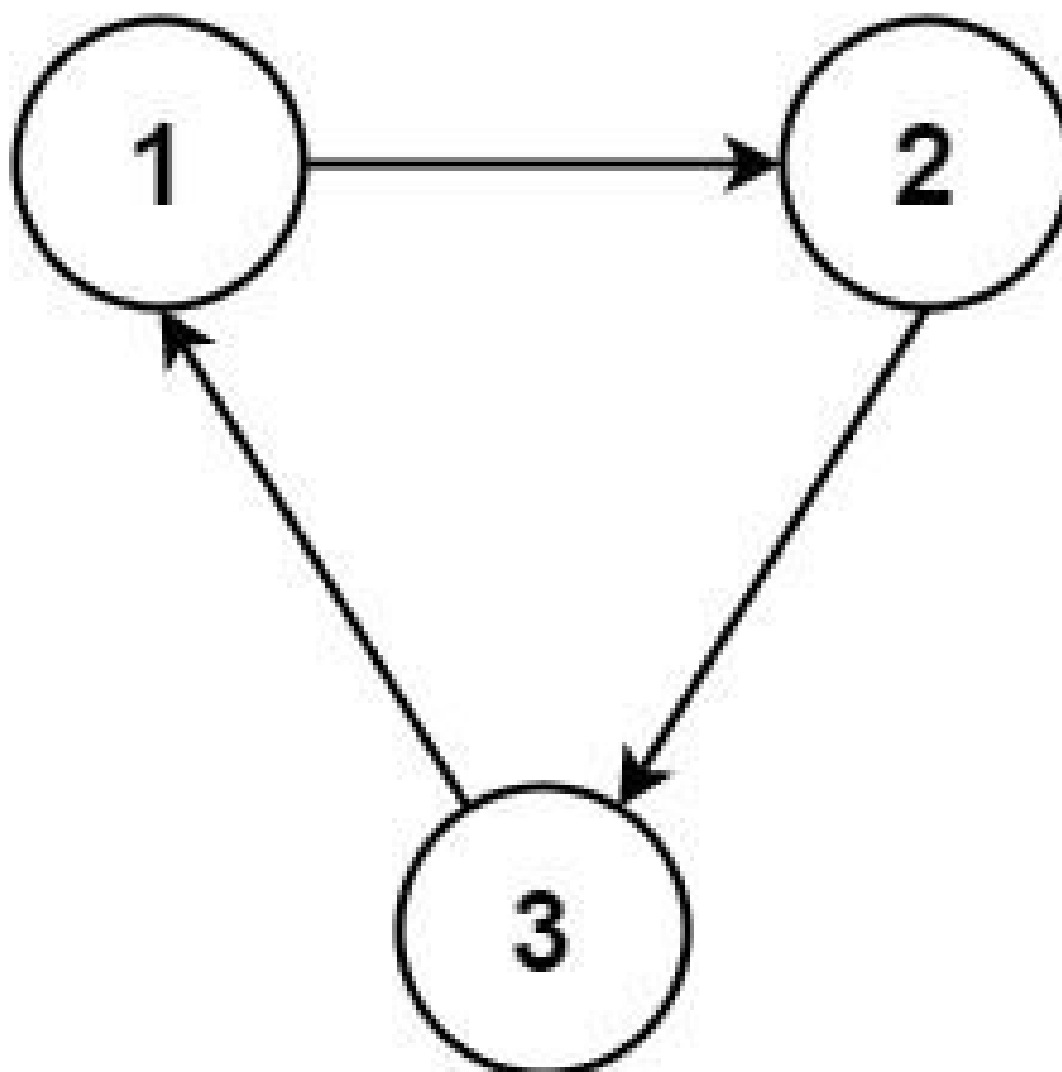
Output: 2

Explanation: The figure above represents the given graph.

In the first semester, you can take courses 1 and 2.

In the second semester, you can take course 3.

Example 2:



Input: $n = 3$, relations = $[[1,2],[2,3],[3,1]]$

Output: -1

Explanation: No course can be studied because they are prerequisites of each other.

Constraints:

- $1 \leq n \leq 5000$
- $1 \leq \text{relations.length} \leq 5000$
- $\text{relations}[i].\text{length} == 2$
- $1 \leq \text{prevCourse}_i, \text{nextCourse}_i \leq n$
- $\text{prevCourse}_i \neq \text{nextCourse}_i$

- All the pairs [prevCoursei, nextCoursei] are unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Minimum semesters to take all courses where you can take any number per semester
as long as prerequisites are met. Return -1 if impossible (cycle). Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
Topological sort with indegree: each semester take all courses whose prerequisites are done.
Increment semester count each round; if no course can be taken but some remain, impossible.
Same Kahn BFS layered by semester.
Time: $O(V + E)$. Space: $O(V + E)$.
Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (Kahn's BFS level-by-level)

"BFS topological sort: process all zero-in-degree nodes in one 'batch' (semester).
Each batch = one semester. Count batches. If total processed $< n \rightarrow$ cycle."

PHASE 4 — CLEAN CODE

```
class _1136_ParallelCourses:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        graph = defaultdict(list)
        indeg = {i: 0 for i in range(1, n + 1)}
        for pre, nxt in relations:
            graph[pre].append(nxt)
            indeg[nxt] += 1
        queue = deque(c for c in range(1, n + 1) if indeg[c] == 0)
        sems = 0
        total = 0
        while queue:
            sems += 1
            for _ in range(len(queue)):
                node = queue.popleft()
                total += 1
                for nb in graph[node]:
                    indeg[nb] -= 1
                    if indeg[nb] == 0:
                        queue.append(nb)
```

```

        if indeg[nb] == 0:
            queue.append(nb)
    return sems if total == n else -1

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. This is Course Schedule (#207) with BFS level counting.

Each BFS level = one semester. Cycle detection: total processed < n.

Given more time I'd ask: what if courses have durations (not just 1 semester each)?

This becomes a weighted DAG critical path problem – DP or Bellman–Ford."

◆ Quick Review

Key Insights

- Model the problem as a directed graph where nodes are courses and edges represent prerequisites.
- Use topological sorting (Kahn's algorithm) to process courses with zero prerequisites.
- Each level (or layer) of processing represents one semester.
- Detect cycles by ensuring all courses are eventually processed; if not, a cycle exists.
- The number of BFS levels gives the minimum number of semesters necessary.

Space & Time

Time Complexity: $O(n + m)$ where n is the number of courses and m is the length of relations (prerequisites).

Space Complexity: $O(n + m)$ for storing the graph, in-degree counts, and the processing queue.

Solution

We solve the problem using a topological sort with Kahn's algorithm. First, construct a graph representation using an adjacency list and compute the in-degree for each course.

Initialize a queue with courses that have no prerequisites (in-degree == 0). Then, process courses level by level. Each level processed represents one semester. For every course taken in that semester, decrease the in-degree of all its neighbors. Add any neighbor with an in-degree that becomes zero to the queue for the next semester. If, after processing, the total courses taken is less than n , then a cycle exists and return -1 . Otherwise, the number of semesters processed is the answer.

BFS

Round 2 · High · Medium · 6 questions

BFS

□ ★ #286 Walls and Gates ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: ★ Premium | Premium 98

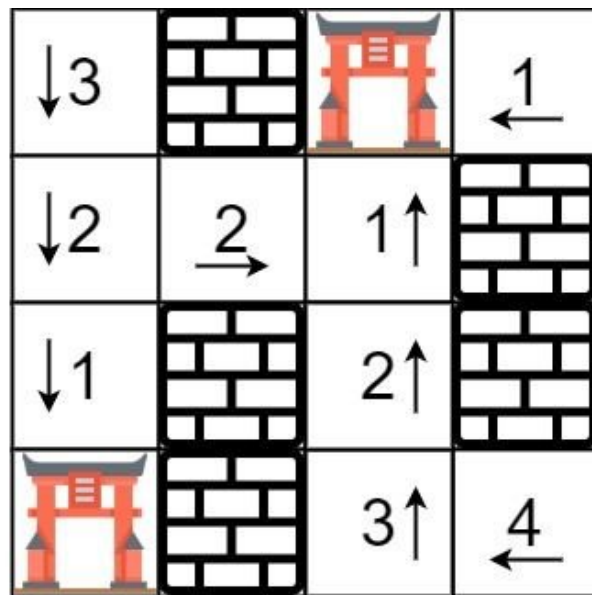
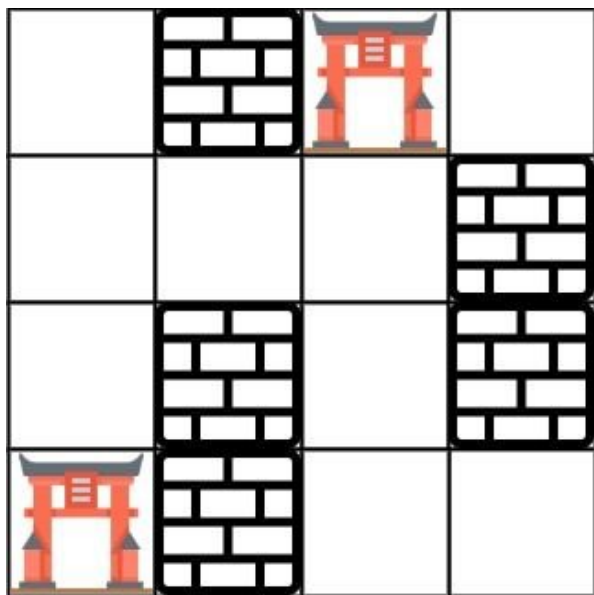
Problem

You are given an $m \times n$ grid rooms initialized with these three possible values.

- -1 A wall or an obstacle.
- 0 A gate.
- INF Infinity means an empty room. We use the value $2^{31} - 1 = 2147483647$ to represent INF as you may assume that the distance to a gate is less than 2147483647.

Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF.

Example 1:



Input: rooms = [[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]
 Output: [[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]

Example 2:

Input: rooms = [[-1]]
 Output: [[-1]]

Constraints:

- $m == \text{rooms.length}$
- $n == \text{rooms}[i].\text{length}$
- $1 \leq m, n \leq 250$
- $\text{rooms}[i][j]$ is -1 , 0 , or $231 - 1$.

My LeetCode Solution
 Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Fill each INF room with its distance to the nearest gate (0). Walls (-1) stay.
Right?
# In-place modification. INF = 2^31-1."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Multi-source BFS from every gate simultaneously (distance 0).
# Fill empty rooms layer by layer with increasing distance.
# Each room reached once – standard BFS on grid.
# Time:  $O(m * n)$ . Space:  $O(m * n)$  queue.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (multi-source BFS from all gates)
# "Enqueue ALL gates at once. BFS outward – each room gets filled with distance
# from the nearest gate simultaneously.  $O(m*n)$ ."

# PHASE 4 — CLEAN CODE
class _286_WallsAndGates:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        INF = 2**31 - 1
        m, n = len(rooms), len(rooms[0])
        queue = deque((r, c) for r in range(m) for c in range(n) if rooms[r][c] ==
0)
        while queue:
            r, c = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and rooms[nr][nc] == INF:
                    rooms[nr][nc] = rooms[r][c] + 1
                    queue.append((nr, nc))

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(m*n)$ . Space  $O(m*n)$ . Multi-source BFS is the key – a single BFS from each
gate
# independently would be  $O(m*n * \text{gates})$ , much slower.
# Contrast with #542 (01 Matrix): identical pattern but for distance to nearest 0.
# Given more time I'd ask: find distance to nearest wall instead?
# Multi-source BFS from all walls – same approach."

```

◆ Quick Review

Key Insights

- Use a multi-source Breadth-First Search (BFS) starting from all the gates, as this allows

spreading the shortest distance to each empty room.

- Instead of starting from every empty room separately, starting from all gates simultaneously helps update all distances in one pass.
- Only update a room if the new calculated distance is smaller than its current value to prevent unnecessary processing.
- The problem is essentially a multi-source shortest path problem on a grid.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the grid, as each cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario used by the BFS queue.

Solution

The solution uses a multi-source BFS algorithm by initially enqueueing all the gate positions (cells with value 0). Then, for each cell obtained from the queue, check its four possible neighbors (up, down, left, right). If a neighbor is an empty room (INF), update its

distance to be the current cell's distance plus one, and then enqueue the neighbor to process further. This ensures that each empty room gets the minimum distance from any gate.

BFS

□ #322 Coin Change

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · BFS
Lists: Grind 169

Problem

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11  
Output: 3  
Explanation: 11 = 5 + 5 + 1
```

Example 2:

Input: coins = [2], amount = 3
Output: -1

Example 3:

Input: coins = [1], amount = 0
Output: 0

Constraints:

- $1 \leq \text{coins.length} \leq 12$
- $1 \leq \text{coins}[i] \leq 231 - 1$
- $0 \leq \text{amount} \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Minimum coins to make up amount. Unlimited supply of each coin. Return -1 if impossible. Right?"

Coin values can be any positive integer – not necessarily power of 2 (so greedy fails)."

#

"coins=[1,2,5], amount=11: 5+5+1=3 coins ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Recursion: for each amount, try every coin denomination and take 1 + min subproblem.

Recomputes identical sub-amounts exponentially without memoization.

Time: $O(\text{amount}^{\text{coins}})$ exponential. Space: $O(\text{amount})$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (bottom-up DP)

"dp[i] = minimum coins to make amount i."

dp[0]=0. dp[i] = min(dp[i-coin]+1) for each coin <= i.

Time: $O(\text{amount} * \text{coins})$. Space: $O(\text{amount})$."

PHASE 4 — CLEAN CODE

```

class _322_CoinChange:
    def coinChange(self, coins: List[int], amount: int) -> int:
        dp = [float('inf')] * (amount + 1)
        dp[0] = 0
        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] = min(dp[i], dp[i - coin] + 1)
        return dp[amount] if dp[amount] != float('inf') else -1

# PHASE 5 — TEST & DEBUG
# coins=[1,2,5], amount=11:
# dp[5]=1(5). dp[10]=2(5+5). dp[11]=3(5+5+1). → 3 ✓
# amount=3, coins=[2]: dp[2]=1, dp[3]=inf → -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(amount * coins). Space O(amount)."
# BFS alternative: treat amounts as nodes, coins as edges. BFS from 0 to amount.
# DP is preferred – BFS visits each amount once but rebuilds from scratch.
# Given more time I'd ask: how many ways to make the amount?
# Coin Change 2 (#518) – same DP but count combinations instead of min count."

```

◆ Quick Review

Key Insights

- Use a dynamic programming approach to build up a solution from smaller subproblems.
- Define $dp[x]$ as the minimum number of coins required for amount x .
- The recurrence relation is: $dp[x] = \min(dp[x], dp[x - \text{coin}] + 1)$ for each coin.
- Initialize $dp[0] = 0$ and set other values to a number larger than the maximum possible coins ($\text{amount} + 1$ works as an infinity substitute).

- An alternative approach is through BFS, treating each amount as a node and each coin as an edge, but the DP method is more intuitive here.

Space & Time

Time Complexity: $O(\text{amount} * n)$ where n is the number of coins.

Space Complexity: $O(\text{amount})$ for the dp array.

Solution

We use a bottom-up dynamic programming approach. The idea is to use a one-dimensional dp array where each element at index x represents the minimum number of coins required to form the amount x . Starting from $dp[0]$ which is 0, we iterate through all amounts up to the target amount and update $dp[x]$ based on each coin denomination. If after filling the dp array the value $dp[\text{amount}]$ remains unchanged (i.e., greater than amount), it means the amount cannot be formed and we return -1 .

BFS

#542 01 Matrix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: Grind 169 | Denny Zhang

Problem

Given an $m \times n$ binary matrix `mat`, return the distance of the nearest 0 for each cell.

The distance between two cells sharing a common edge is 1.

Example 1:

0	0	0
0	1	0
0	0	0

Input: mat = [[0,0,0],[0,1,0],[0,0,0]]

Output: [[0,0,0],[0,1,0],[0,0,0]]

Example 2:

0	0	0
0	1	0
1	1	1

Input: mat = [[0,0,0],[0,1,0],[1,1,1]]

Output: [[0,0,0],[0,1,0],[1,2,1]]

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 104$
- $1 \leq m * n \leq 104$
- $\text{mat}[i][j]$ is either 0 or 1.
- There is at least one 0 in mat.

Note: This question is the same as 1765: <https://leetcode.com/problems/map-of-highest-peak/>

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return distance of nearest 0 for each cell. In-place or new matrix. Right?
Same pattern as Walls and Gates but for 0-cells instead of gates."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
Multi-source BFS starting from every 0 simultaneously.
First time each 1 cell is reached, its distance is minimal.
Layer-by-layer BFS fills the dist matrix in one pass.
Time: $O(m * n)$. Space: $O(m * n)$ queue + dist.
Should I go ahead and optimize?"

```
class _542_ZeroOneMatrix:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        dist = [[0 if mat[r][c] == 0 else float('inf') for c in range(n)] for r in range(m)]
        queue = deque((r, c) for r in range(m) for c in range(n) if mat[r][c] == 0)
        while queue:
            r, c = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and dist[nr][nc] > dist[r][c] + 1:
                    dist[nr][nc] = dist[r][c] + 1
                    queue.append((nr, nc))
        return dist
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. Multi-source BFS from all 0-cells simultaneously.
Identical to #286 (Walls and Gates) – name this connection.
Given more time I'd ask: distance to nearest 1 instead?
Same multi-source BFS starting from all 1-cells."

◆ Quick Review

Key Insights

- Treat all 0s as starting points for a multi-source BFS.
- Update each neighboring cell's distance if a shorter distance is found.
- Utilize a queue to process cells in order of increasing distance.
- Ensure that each cell is processed only once by marking visited cells.

Space & Time

Time Complexity: $O(mn)$ since each cell is processed once.

Space Complexity: $O(mn)$ for the queue and the answer matrix.

Solution

We use a multi-source Breadth-First Search (BFS) where every 0 in the matrix is initially added to the BFS queue. Each neighboring cell that is a 1 will have its distance updated to be one plus the distance of the current cell if it hasn't been visited yet (or if a smaller distance is found). This ensures that the first time a 1 is

reached, it is reached by the shortest path.
Data structures used include a queue (or deque) for BFS and the matrix itself for storing distances.

BFS

□ #994 Rotting Oranges

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix
Lists: Grind 169

Problem

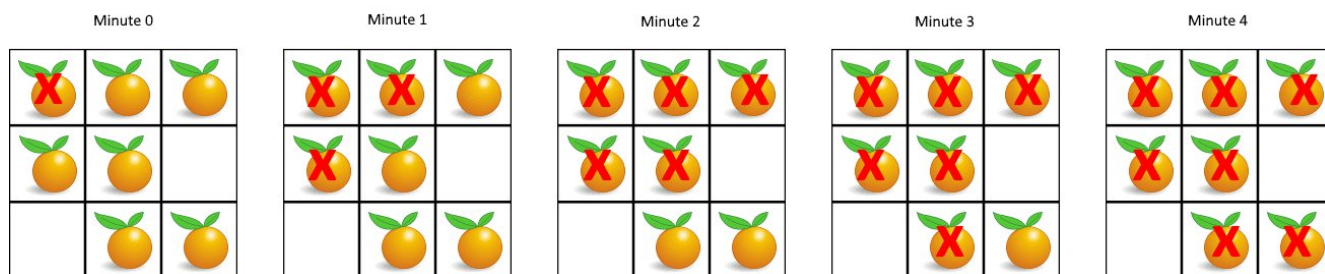
You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1 .

Example 1:



Input: grid = `[[2,1,1],[1,1,0],[0,1,1]]`
 Output: 4

Example 2:

Input: grid = `[[2,1,1],[0,1,1],[1,0,1]]`
 Output: -1
 Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 3:

Input: grid = `[[0,2]]`
 Output: 0
 Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 10`
- `grid[i][j]` is 0, 1, or 2.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Multi-source BFS from all rotten oranges. Fresh adjacent → rotten after 1 minute.

```

# Return minutes until no fresh remain, or -1 if impossible. Right?"
#
# "If no fresh oranges initially → return 0 immediately."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Multi-source BFS: enqueue all rotten oranges at minute 0.
# Each minute, rot all fresh neighbors; track elapsed minutes until no fresh remain.
# If fresh count > 0 after BFS ends, return -1.
# Time: O(m * n). Space: O(m * n) queue.
# Should I go ahead and optimize?"

class _994_RottingOranges:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        fresh = sum(grid[r][c] == 1 for r in range(m) for c in range(n))
        queue = deque((r, c) for r in range(m) for c in range(n) if grid[r][c] == 2)
        minutes = 0
        while queue and fresh:
            minutes += 1
            for _ in range(len(queue)):
                r, c = queue.popleft()
                for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < m and 0 <= nc < n and grid[nr][nc] == 1:
                        grid[nr][nc] = 2
                        fresh -= 1
                        queue.append((nr, nc))
            return minutes if fresh == 0 else -1

# PHASE 5 — TEST & DEBUG
# [[2,1,1],[1,1,0],[0,1,1]]: fresh=6. BFS spreads level by level. After 4 minutes
fresh=0 → 4 ✓
# Isolated fresh orange: fresh>0 after BFS done → -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n). Multi-source BFS: same as Walls and Gates, same as 01
Matrix.
# The 'fresh' counter eliminates the need to scan the grid again at the end.
# Given more time I'd ask: what if diagonal rotting is allowed?
# Add 4 diagonal direction pairs – same algorithm."

```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) starting from all initially rotten oranges.
- Process the grid level by level, where each level represents one minute.
- Keep a count of fresh oranges and decrement it as they become rotten.
- If after the BFS there are still fresh oranges, then it is impossible to rot every orange.

Space & Time

Time Complexity: $O(m * n)$ as every cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst case for the queue and auxiliary storage.

Solution

We begin by scanning the grid to locate all rotten oranges and count the fresh ones.

These rotten oranges are added to a queue for a BFS. For each minute (BFS level), we process all the current rotten oranges and try to infect their adjacent fresh oranges. If an adjacent cell contains a fresh orange, we mark it rotten, add it to the queue, and decrement our fresh count. We continue this process level by level (minute by minute) until there are no fresh oranges

left. If, after processing, there are still fresh oranges remaining, we return -1 , indicating that not all oranges can become rotten.

BFS

□ ★ #1197 Minimum Knight Moves ★

MED

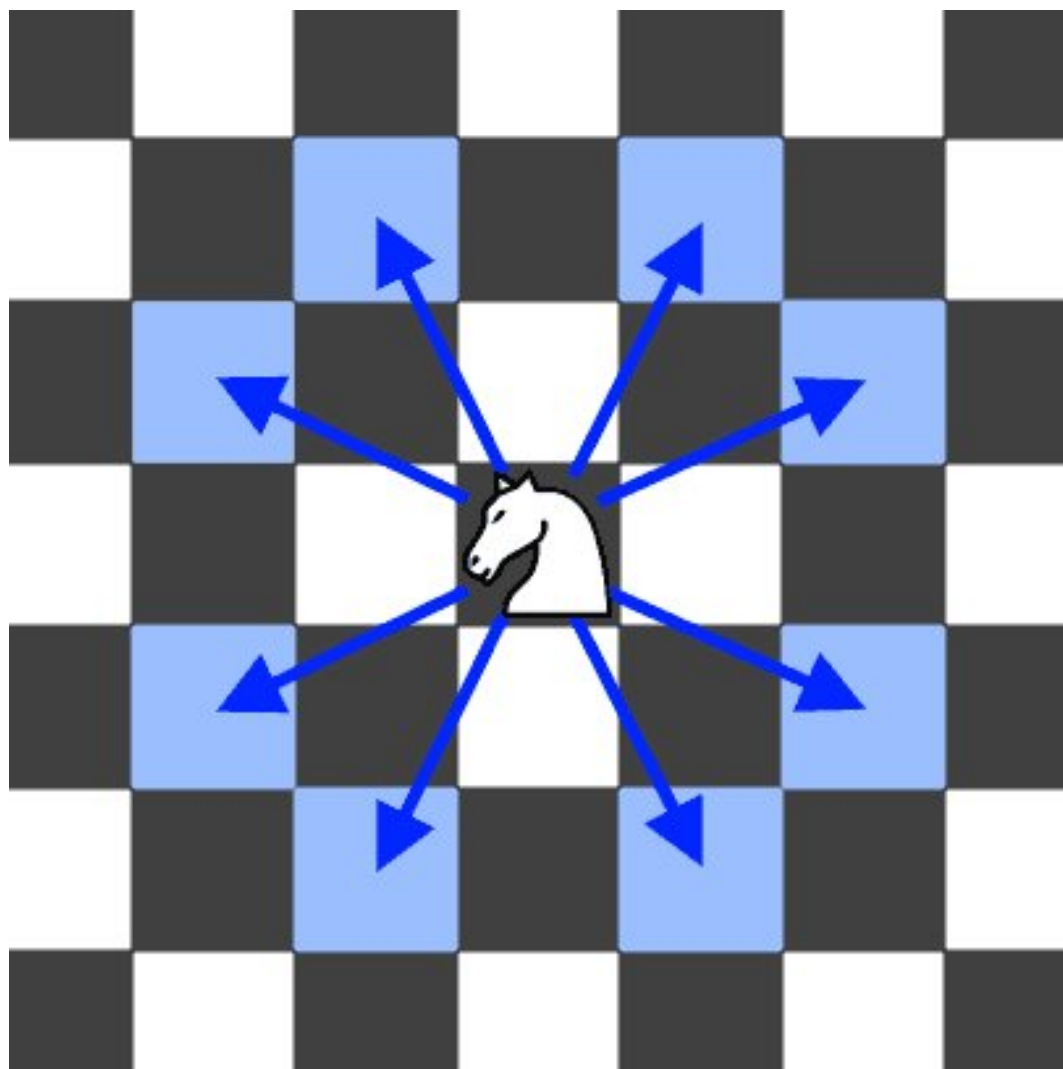
LeetDoocs · SimplyLeet · WalkCC · LeetCode
BFS

Lists: ★ Premium | Grind 169 | Premium 98

Problem

In an infinite chess board with coordinates from $-\infty$ to $+\infty$, you have a knight at square $[0, 0]$.

A knight has 8 possible moves it can make, as illustrated below. Each move is two squares in a cardinal direction, then one square in an orthogonal direction.



Return the minimum number of steps needed to move the knight to the square $[x, y]$. It is guaranteed the answer exists.

Example 1:

Input: $x = 2, y = 1$
Output: 1
Explanation: $[0, 0] \rightarrow [2, 1]$

Example 2:

Input: $x = 5, y = 5$
Output: 4
Explanation: $[0, 0] \rightarrow [2, 1] \rightarrow [4, 2] \rightarrow [3, 4] \rightarrow [5, 5]$

Constraints:

- $-300 \leq x, y \leq 300$
- $0 \leq |x| + |y| \leq 300$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Knight moves on an infinite board. Minimum moves from (0,0) to (x,y).
# 8 possible moves. Symmetry: |x| and |y| – reflect to first quadrant. Right?"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# BFS on an infinite chessboard from (0,0) to (x,y).
# Enqueue all 8 knight moves each step; stop when target is reached.
# Layer count = minimum moves. Prune to first quadrant since moves are symmetric.
# Time: O(max(x,y)^2) BFS nodes. Space: O(max(x,y)^2) visited.
# Should I go ahead and optimize?"
```

class _1197_MinKnightMoves:

```
    def minKnightMoves(self, x: int, y: int) -> int:
        x, y = abs(x), abs(y) # exploit symmetry – reflect to first quadrant
        queue = deque([(0, 0, 0)]) # (row, col, steps)
        seen = {(0, 0)}
        while queue:
            r, c, steps = queue.popleft()
            if r == x and c == y:
                return steps
            for dr, dc in [(2,1), (2,-1), (-2,1), (-2,-1), (1,2), (1,-2), (-1,2), (-1,-2)]:
                nr, nc = r + dr, c + dc
                if (nr, nc) not in seen and nr >= -1 and nc >= -1:
                    seen.add((nr, nc))
                    queue.append((nr, nc, steps + 1))
        return -1
```

PHASE 5 — TEST & DEBUG

```
# (2,1): BFS from (0,0). (2,1) reached in 1 step → 1 ✓
# (5,5): 4 steps ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\max(|x|,|y|)^2)$ . Space  $O(\max(|x|,|y|)^2)$ . Standard BFS shortest path.  
# Symmetry reduction: reflect to first quadrant cuts search space by 4.  
# The -1 lower bound allows the knight to 'go back' through negative coords when  
# needed.  
# Given more time I'd ask: bidirectional BFS?  
# Start from both (0,0) and (x,y) simultaneously – meets in the middle, roughly  
# halving depth."
```

◆ Quick Review

Key Insights

- Leverage the symmetry of the chessboard by converting target coordinates to their absolute values.
- Use a Breadth-First Search (BFS) because all moves have equal weight, ensuring the first encounter with the target is the minimal path.
- Prune the search space by only considering positions that are likely beneficial (e.g., positions having coordinates not far less than -1).
- Track visited states to avoid redundant processing.
- Optimize by bounding the search region relative to the target distance.

Space & Time

Time Complexity: $O(n)$ where n is the number of positions processed during the BFS

Space Complexity: $O(n)$ due to the storage requirements for the BFS queue and the visited set

Solution

We solve the problem using a BFS starting from the origin. First, normalize the target by taking the absolute value of x and y . BFS is then applied by exploring all eight possible knight moves at each step. Each move generates a new position that is enqueued if it hasn't been visited and falls within a reasonable bound (e.g., coordinates ≥ -1). The search terminates as soon as the target position is reached.

BFS

□ #1730 Shortest Path to Get Food

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix
Lists: Grind 169

Problem

You are starving and you want to eat food as quickly as possible. You want to find the shortest path to arrive at any food cell.

You are given an $m \times n$ character matrix, grid, of these different types of cells:

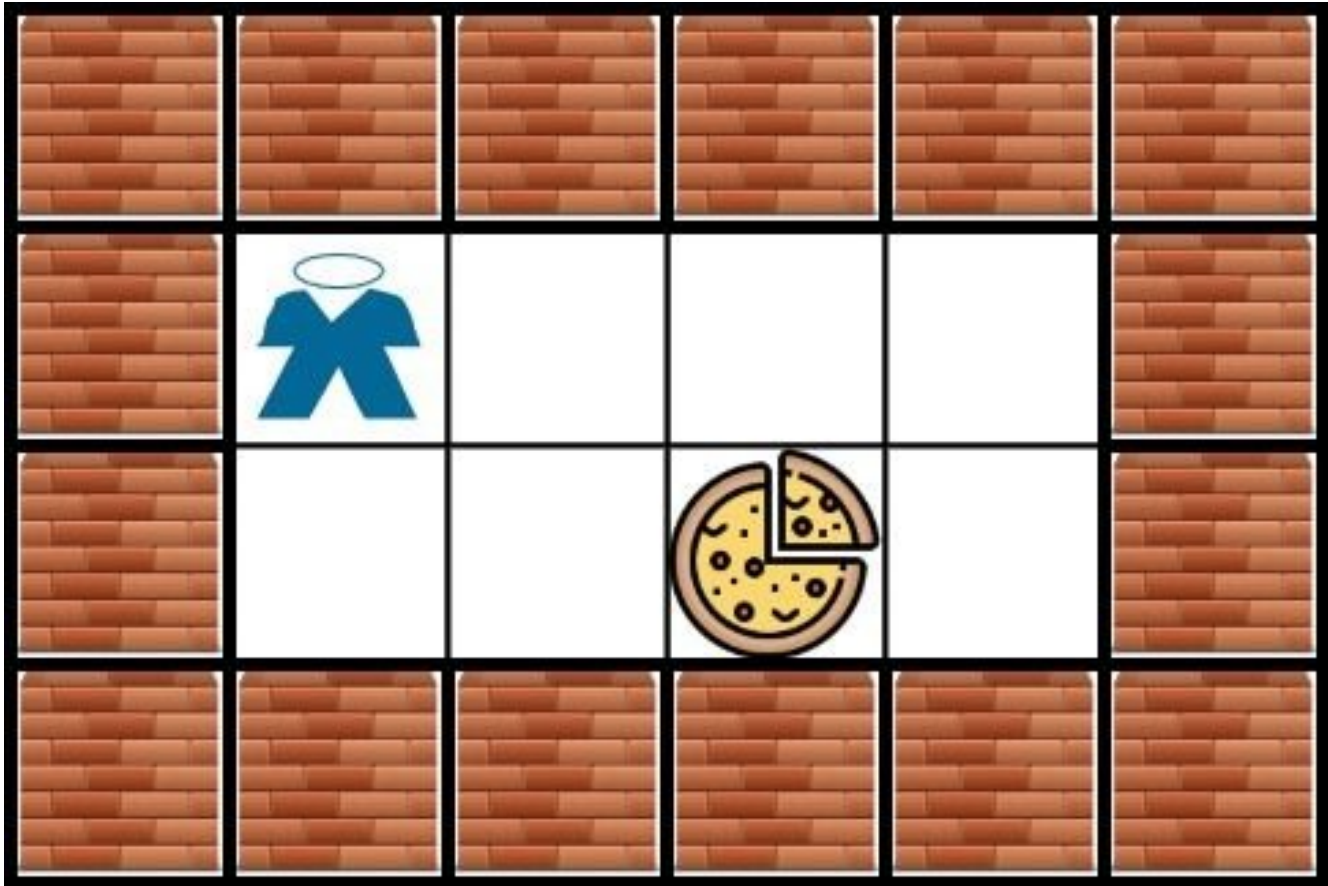
- '*' is your location. There is exactly one '*' cell.
- '#' is a food cell. There may be multiple food cells.
- 'O' is free space, and you can travel through these cells.
- 'X' is an obstacle, and you cannot travel through these cells.

You can travel to any adjacent cell north, east, south, or west of your current location if there is

not an obstacle.

Return the length of the shortest path for you to reach any food cell. If there is no path for you to reach food, return -1.

Example 1:

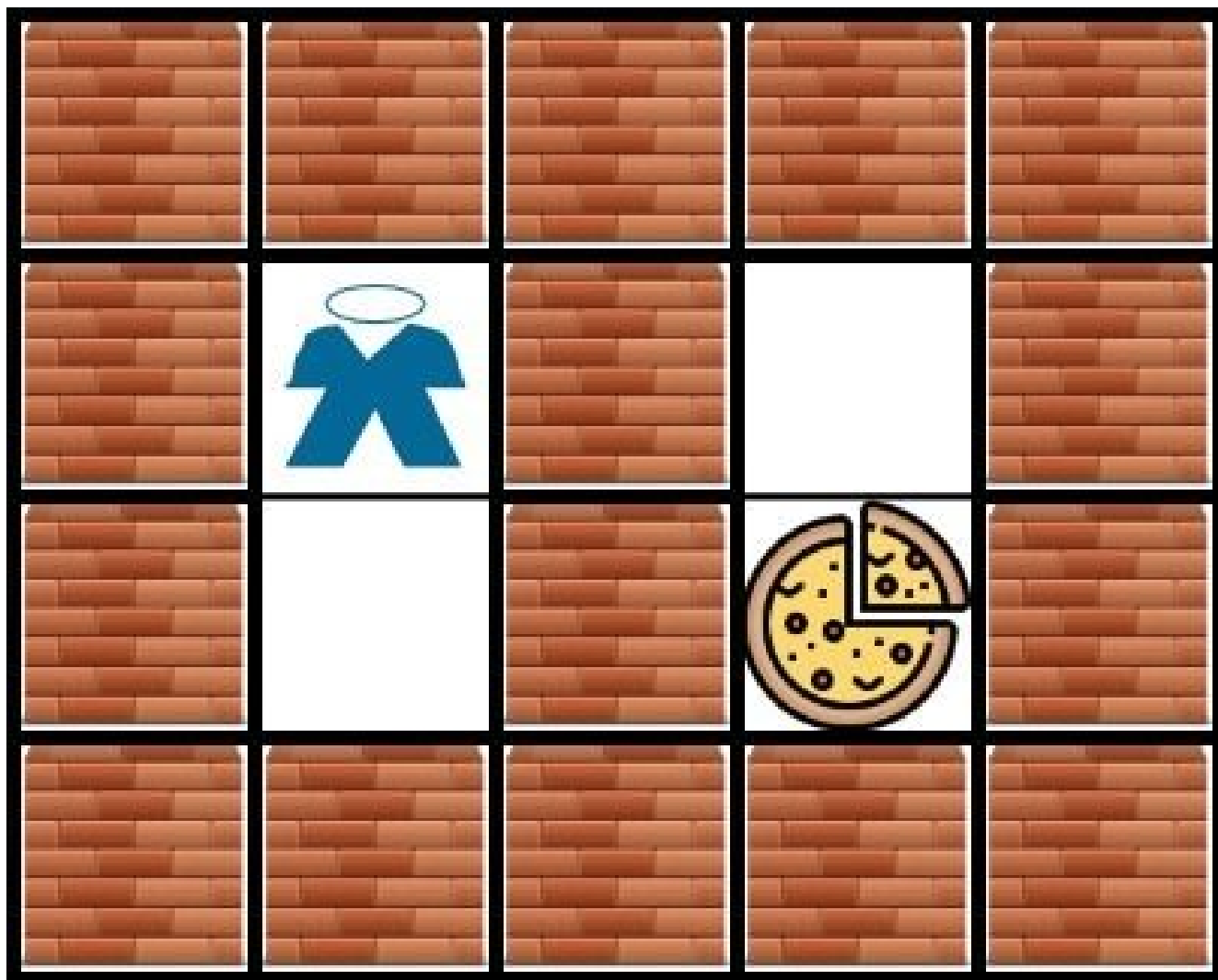


Input: grid = [["X","X","X","X","X","X"], ["X","*","0","0","0","X"], ["X","0","0","#","0","X"], ["X","X","X","X","X","X"]]

Output: 3

Explanation: It takes 3 steps to reach the food.

Example 2:

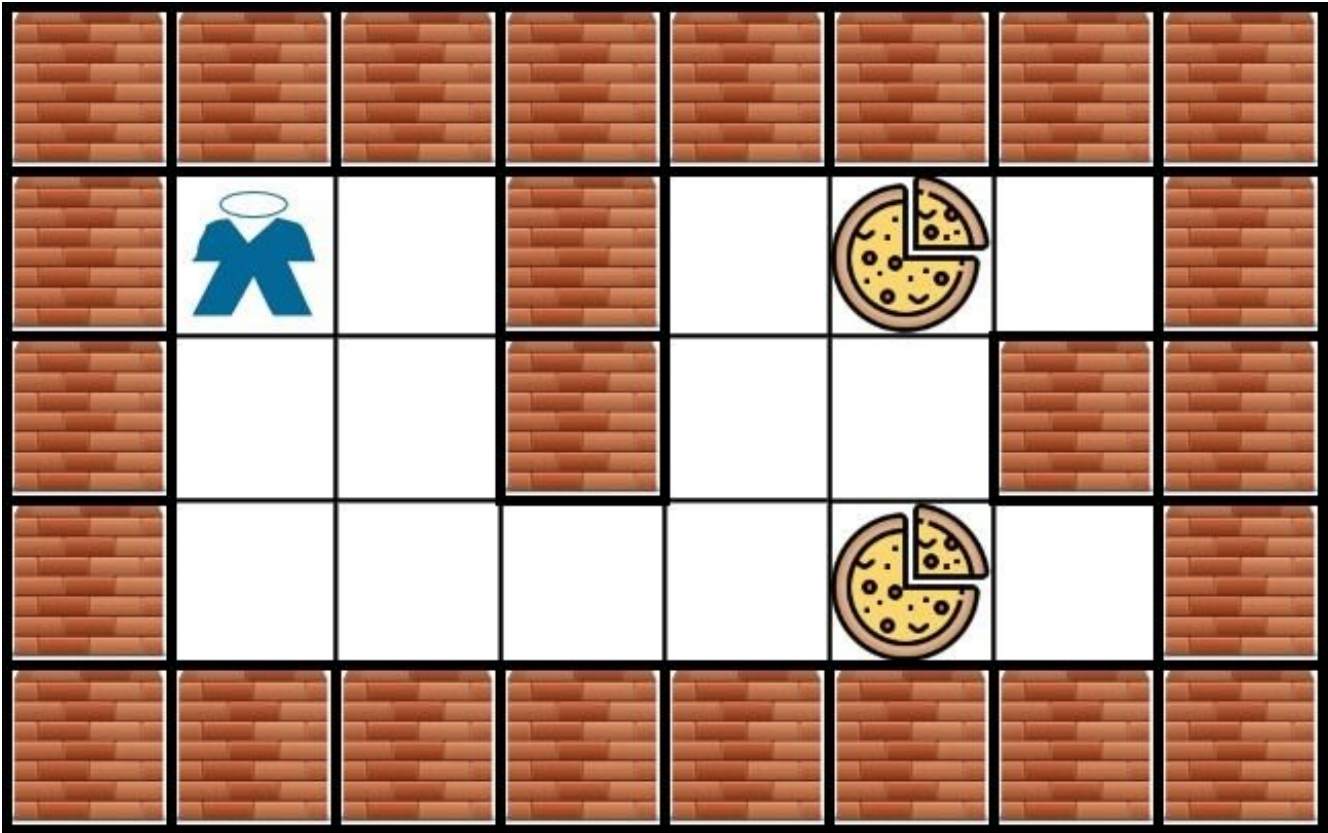


Input: grid = [["X","X","X","X","X"], ["X","*","X","0","X"], ["X","0","X","#","X"], ["X","X","X","X","X"]]

Output: -1

Explanation: It is not possible to reach the food.

Example 3:



Input: grid = `[["X","X","X","X","X","X","X","X"],["X","*","0","X","0","#","0","X"],["X","0","0","X","0","0","X","X"],["X","0","0","0","0","#","0","X"],["X","X","X","X","X","X","X"]]`
Output: 6
Explanation: There can be multiple food cells. It only takes 6 steps to reach the bottom food.

Example 4:

Input: grid = `[["X","X","X","X","X","X","X","X"],["X","*","0","X","0","#","0","X"],["X","0","0","X","0","0","X","X"],["X","0","0","0","0","#","0","X"],["0","0","0","0","0","0","0"]]`
Output: 5

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 200`
- `grid[row][col]` is `'*'`, `'X'`, `'O'`, or `'#'`.

- The grid contains exactly one '*'.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "BFS from '*' to the nearest '#'. '0' = free, 'X' = wall, '#' = food (any one).
# Return shortest path length. Return -1 if unreachable. Right?"
#
# "'*' → nearest '#' using 4-directional BFS. First '#' reached = shortest path."
```

PHASE 2 — BRUTE FORCE (also optimal)

```
# "Let me start with brute force to lock in the logic.
# Multi-source BFS from every '*' food cell simultaneously.
# First time a 'H' house is reached, record its distance; sum cannot reach uses -1.
# BFS is already optimal O(m*n); DFS would revisit cells wastefully.
# Time: O(m * n). Space: O(m * n) queue.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Standard BFS from source. First '#' reached is guaranteed shortest path.
# Mark visited by overwriting '0' with 'X' – no extra visited set needed.
# Time: O(m*n). Space: O(m*n)."
```

PHASE 4 — CLEAN CODE

```
class _1730_ShortestPathFood:
    def getFood(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])
        start = next((r, c) for r in range(m) for c in range(n) if grid[r][c] ==
'*')

        queue = deque([(start[0], start[1], 0)])
        grid[start[0]][start[1]] = 'X' # mark visited
        while queue:
            r, c, dist = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n:
                    if grid[nr][nc] == '#':
                        return dist + 1
                    if grid[nr][nc] == '0':
                        grid[nr][nc] = 'X' # mark visited
                        queue.append((nr, nc, dist + 1))
```


Time Complexity: $O(m * n)$, where m is the number of rows and n is the number of columns since each cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst case due to the queue and visited matrix.

Solution

We use Breadth-First Search (BFS) starting from the cell marked with '*'. The BFS uses a queue to explore neighbors in the four allowed directions (up, down, left, right). Each level of BFS represents one step in the path. As soon as a cell containing food ('#') is reached, the current step count is returned. We also maintain a visited matrix to ensure that cells are not revisited, which could otherwise result in cycles or redundant processing. This approach ensures that the first time we encounter food, we have found the shortest possible path.

Round 3 · High · Hard

Round 3

High Priority · Hard

23 questions · 8 patterns

- ☐ ● Arrays & Hashing #41 ↗
- ☐ ● Sliding Window #76 ↗
- ☐ ● Binary Search #4 #315 #410 #668 #710 #774 ↗
- ☐ ● #1235 ↗
- ☐ ● Matrix #1074 ↗
- ☐ ● Trees & BST #124 #297 ↗
- ☐ ● DFS #269 #329 #685 #1036 #1192 ↗
- ☐ ● Graphs #305 #1153 ↗
- ☐ ● BFS #127 #317 #773 #815 ↗

Arrays & Hashing

Round 3 · High · Hard · 1 question

Arrays & Hashing

□ #41 First Missing Positive

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table

Lists: Grind 169

Problem

Given an unsorted integer array `nums`. Return the smallest positive integer that is not present in `nums`.

You must implement an algorithm that runs in $O(n)$ time and uses $O(1)$ auxiliary space.

Example 1:

Input: `nums = [1,2,0]`
Output: 3
Explanation: The numbers in the range `[1,2]` are all in the array.

Example 2:

Input: `nums = [3,4,-1,1]`
Output: 2
Explanation: 1 is in the array but 2 is missing.

Example 3:

Input: `nums = [7,8,9,11,12]`
Output: 1
Explanation: The smallest positive integer 1 is missing.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Smallest positive integer NOT in nums. O(n) time, O(1) auxiliary space.
# Answer is in range [1, n+1] — if all 1..n present, answer is n+1. Right?
# Negatives, zeros, and large numbers can be ignored."
#
# "[3,4,-1,1] → 2 (1 present, 2 missing) ✓. [7,8,9] → 1 ✓. [1,2,0] → 3 ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Sort the array, then scan from 1 upward until the first missing positive integer.
# Or put all values in a hash set and scan i=1,2,3... until absent.
# Correct but sorting is O(n log n) and the set uses O(n) extra space.
# Time: O(n log n). Space: O(n) for set approach.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (array as hash map — cycle sort)

```
# "Key insight: use the array itself as a lookup table.
# Valid range is [1, n]. Place each number x at index x-1 (if 1 ≤ x ≤ n and not
# already there).
# After placing, scan: first i where nums[i] != i+1 → answer is i+1.
# If all positions correct → answer is n+1.
# Each number is placed at most once → O(n) total swaps."
```

PHASE 4 — CLEAN CODE

```
class _41_FirstMissingPositive:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
                nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i] - 1]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1
        return n + 1
```

PHASE 5 — TEST & DEBUG

```
# [3,4,-1,1]: place 3→pos2, 4→pos3, -1 ignored, 1→pos0.
# After: [1,-1,3,4]. Scan: i=1 nums[1]=-1≠2 → return 2 ✓
# [1,2,3]: after: [1,2,3]. All correct → return 4 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – each element swapped at most once. Space $O(1)$."

The while loop runs at most n total iterations across the entire for loop – amortised $O(1)$ per i .

Given more time I'd ask: what if duplicates exist?

The condition `nums[nums[i]-1] != nums[i]` handles duplicates – don't swap a number to a position

that already has the correct value (would loop forever)."

◆ Quick Review

Key Insights

- The missing positive number must be in the range $[1, n+1]$ where n is the length of the array.
- Use the array indices to place numbers in their correct positions (i.e., number i should be at index $i-1$) with in-place swapping.
- After rearrangement, scan the array: the first index where the number is not equal to $\text{index}+1$ indicates the missing positive integer.

Space & Time

Time Complexity: $O(n)$ as each number is swapped at most once.

Space Complexity: $O(1)$ since the rearrangement occurs in-place.

Solution

The approach involves reordering the array so that each positive integer in the range $[1, n]$ is placed at the index corresponding to its value minus one. This means for any valid number i , $\text{nums}[i-1]$ should be i . We iterate through the array and perform in-place swaps to achieve this order. After this process, another pass through the array reveals the first position where the number does not match the expected value $(i+1)$, which is the smallest missing positive. If every position is correct, then the missing value is $n+1$.

Sliding Window

Round 3 · High · Hard · 1 question

Sliding Window

□ #76 Minimum Window Substring

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string `""`.

The testcases will be generated such that the answer is unique.

Example 1:

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.
```

Example 2:

```
Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.
```

Example 3:

Round 3 · High · Hard

p.924

Input: s = "a", t = "aa"

Output: ""

Explanation: Both 'a's from t must be included in the window.
Since the largest window of s only has one 'a', return empty string.

Constraints:

- $m == s.length$
- $n == t.length$
- $1 \leq m, n \leq 105$
- s and t consist of uppercase and lowercase English letters.

Follow up: Could you find an algorithm that runs in $O(m + n)$ time?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Smallest window in s containing all characters of t (with multiplicity).

Return '' if no such window. Right?

t may have duplicate chars – window must have at least that many."

#

"s='ADOBECODEBANC', t='ABC' → 'BANC' (contains A,B,C, length 4) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Enumerate every substring of s with two nested indices.

For each substring, check whether it contains all characters of t (with frequency).

Track the shortest valid window – correct but $O(m^2 * n)$ character checks.

Time: $O(m^2 * n)$. Space: $O(n)$ frequency map.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (sliding window with 'formed' counter)

```
# "Expand right: add s[right] to window frequency map.
# When a char's count in window meets t's requirement, increment 'formed'.
# When formed == len(distinct chars in t): window is valid. Try shrinking from left.
# Track best window. O(m+n) – each char enters/exits at most once."
```

PHASE 4 — CLEAN CODE

```
class _76_MinWindowSubstring:
    def minWindow(self, s: str, t: str) -> str:
        if not t or not s:
            return ''
        need = Counter(t)
        have : Counter = Counter()
        formed = 0
        required = len(need)
        left = 0
        best = (float('inf'), 0, 0) # (length, left, right)
        for right in range(len(s)):
            c = s[right]
            have[c] += 1
            if c in need and have[c] == need[c]:
                formed += 1
            while formed == required:
                if right - left + 1 < best[0]:
                    best = (right - left + 1, left, right)
                have[s[left]] -= 1
                if s[left] in need and have[s[left]] < need[s[left]]:
                    formed -= 1
                left += 1
            return '' if best[0] == float('inf') else s[best[1]: best[2] + 1]
```

PHASE 5 — TEST & DEBUG

```
# s='ADOBECODEBANC', t='ABC': need={A:1,B:1,C:1}. required=3.
# Window expands until it contains A,B,C. Shrink left until one char drops below
# need.
# Best window = 'BANC' ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m+n). Space O(m+n). The 'formed' counter avoids comparing the full
# frequency maps.
# A window is valid exactly when formed==required – elegant O(1) validity check.
# Given more time I'd ask: find all minimum windows?
# Collect all windows where length == best length during the BFS scan."
```

◆ Quick Review

Key Insights

- Use a sliding window approach to dynamically adjust the window boundaries.
- Utilize a hash table (or dictionary) to keep track of the frequency of characters required (from t) and the frequency within the current window.
- Expand the window until it satisfies the requirements, then contract to minimize the window.
- Efficiently update counts and check valid window with two pointers (left and right).

Space & Time

Time Complexity: $O(m + n)$, where m is the length of s and n is the length of t .

Space Complexity: $O(m + n)$ in the worst case due to the storage for the hash maps.

Solution

We use a sliding window technique with two pointers representing the window's boundaries. First, create a frequency map of characters from t . Then, expand the right pointer to include characters in the window until the window contains all required characters (meeting or exceeding the counts in

t). Once the requirement is met, try contracting the window by moving the left pointer to reduce its size while still maintaining a valid window. Throughout the process, if a valid window is found that is smaller than the previously recorded minimum window, update the minimum. The solution leans heavily on hash tables to track counts and compares them against the required counts from t.

Binary Search

Round 3 · High · Hard · 7 questions

Binary Search

□ #4 Median of Two Sorted Arrays

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer
Lists: Grind 169

Problem

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Example 1:

```
Input: nums1 = [1,3], nums2 = [2]
Output: 2.00000
Explanation: merged array = [1,2,3] and median is 2.
```

Example 2:

```
Input: nums1 = [1,2], nums2 = [3,4]
Output: 2.50000
Explanation: merged array = [1,2,3,4] and median is (2 + 3) / 2 = 2.5.
```

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- $0 \leq m \leq 1000$

- $0 \leq n \leq 1000$
- $1 \leq m + n \leq 2000$
- $-106 \leq \text{nums1}[i], \text{nums2}[i] \leq 106$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Merge two sorted arrays conceptually and find the median. O(log(m+n)) required.
Right?
# Median: if total length is odd → middle element. Even → average of two middles."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Merge both sorted arrays with two pointers into one combined array.
# Walk to index (m+n)//2 (and the one before if even length) to read the median.
# Correct but uses O(m+n) extra space and ignores the logarithmic requirement.
# Time: O(m + n). Space: O(m + n) merged array.
# Should I go ahead and optimize?"
```

```
class _4_MedianTwoArrays_Brute:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        m, n = len(nums1), len(nums2)
        i = j = 0
        half = (m + n) // 2
        prev = curr = 0
        for _ in range(half + 1):
            prev = curr
            if j >= n or (i < m and nums1[i] <= nums2[j]):
                curr = nums1[i]; i += 1
            else:
                curr = nums2[j]; j += 1
        return (curr + prev) / 2 if not (m + n) & 1 else curr
```

PHASE 3 — OPTIMIZE (binary search on partition — true O(log(min(m,n))))

```
# "Partition both arrays into left and right halves such that:
# total left size = (m+n+1)//2, max(left) <= min(right).
# Binary search on the partition point of the smaller array.
# At each partition: check if nums1[mid-1] <= nums2[r-mid] and nums2[r-mid-1] <=
nums1[mid].
```

```
# Median = max(left halves) for odd total, or average of max(left)/min(right) for even."
```

PHASE 4 — CLEAN CODE

```
class _4_MedianTwoArrays:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        if len(nums1) > len(nums2):
            nums1, nums2 = nums2, nums1
        m, n = len(nums1), len(nums2)
        half = (m + n + 1) // 2
        lo, hi = 0, m
        while lo <= hi:
            i = (lo + hi) // 2 # partition of nums1
            j = half - i # partition of nums2
            left1 = nums1[i - 1] if i > 0 else float('-inf')
            right1 = nums1[i] if i < m else float('inf')
            left2 = nums2[j - 1] if j > 0 else float('-inf')
            right2 = nums2[j] if j < n else float('inf')
            if left1 <= right2 and left2 <= right1:
                if (m + n) % 2 == 1:
                    return float(max(left1, left2))
                return (max(left1, left2) + min(right1, right2)) / 2
            elif left1 > right2:
                hi = i - 1
            else:
                lo = i + 1
        return 0.0
```

PHASE 5 — TEST & DEBUG

```
# nums1=[1,3], nums2=[2]: m=2,n=2. half=2. Partition i=1,j=1:
left1=1,right1=3,left2=2,right2=∞.
# 1<=∞ and 2<=3 ✓. Total=3 (odd) → max(1,2)=2. → 2.0 ✓
# nums1=[1,2], nums2=[3,4]: Partition → max(left)=2, min(right)=3. → 2.5 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log(\min(m,n)))$ . Space  $O(1)$ . Binary search on the smaller array's partition.
# The +inf/-inf sentinels handle edge cases (empty left/right) cleanly.
# The brute-force merge approach is often acceptable in interviews – present both and let the interviewer choose.
# Given more time I'd ask: kth element of two sorted arrays?
# Binary search on k – eliminate  $k//2$  elements from one array per step. Same  $O(\log k)$  idea."
```

◆ Quick Review

Key Insights

- The median divides the sorted array into two equal halves.
- We can perform binary search on the smaller array to partition both arrays correctly.
- We need to ensure that every element in the left partition is less than or equal to every element in the right partition.
- Edge cases must be handled when partitions are at the boundaries of either array.

Space & Time

Time Complexity: $O(\log(\min(m, n)))$

Space Complexity: $O(1)$

Solution

The solution uses a binary search technique on the smaller array. We define a partition index i for `nums1` and compute the corresponding index j for `nums2` such that $i + j$ equals half of the total number of elements. The algorithm checks if the largest element on the left side of `nums1` (if any) is less than or equal to the smallest element on the right side of `nums2`, and vice versa for `nums2`. If the condition holds, we have found the correct partition and compute the median. If not,

adjust the binary search range until the correct partition is found. This partitioning strategy allows us to find the median in logarithmic time.

Binary Search

□ #315 Count of Smaller Numbers After Self

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer ·
Binary Indexed Tree · Segment Tree · Merge
Sort · Ordered Set

Lists: Denny Zhang

Problem

Given an integer array `nums`, return an integer array `counts` where `counts[i]` is the number of smaller elements to the right of `nums[i]`.

Example 1:

```
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.
```

Example 2:

```
Input: nums = [-1]
Output: [0]
```

Example 3:

Input: nums = [-1,-1]
Output: [0,0]

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "For each index i, count elements to the right of i that are strictly less than
nums[i].
# Return as an array. Right?"
#
# "[5,2,6,1] → [2,1,1,0]: right of 5 has 2 and 1 (2 smaller), etc. ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each index i, scan every element to its right and count how many are smaller
than nums[i].
# Store those counts in an output array.
# Correct, but the inner scan makes this quadratic overall.
# Time: O(n^2). Space: O(1) besides output.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BIT / merge sort)

```
# "Process from right to left. Build a BIT indexed by coordinate-compressed value.
# For each nums[i]: query BIT for count of values in [min, nums[i]-1].
# Update BIT at nums[i]. O(n log n) time."
```

PHASE 4 — CLEAN CODE

```
class _315_CountSmaller:
    def countSmaller(self, nums: List[int]) -> List[int]:
        sorted_unique = sorted(set(nums))
        rank = {v: i + 1 for i, v in enumerate(sorted_unique)}
        m = len(sorted_unique)
        bit = [0] * (m + 1)
        def update(i):
            while i <= m:
```

```

        bit[i] += 1
        i += i & (-i)
def query(i):
    s = 0
    while i > 0:
        s += bit[i]
        i -= i & (-i)
    return s
result = []
for num in reversed(nums):
    r = rank[num]
    result.append(query(r - 1)) # count of values < num seen so far
    update(r)
return result[::-1]

```

PHASE 5 — TEST & DEBUG

[5,2,6,1]: process right→left. 1→query(0)=0,update(1). 6→query(3)=1,update(4).
 2→query(1)=1,update(2). 5→query(2)=2.
 # reversed result=[2,1,1,0] → reverse → [2,1,1,0] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$. Space $O(n)$. BIT query gives prefix count in $O(\log n)$.
 # Merge sort alternative: during merge, count inversions – same complexity but different mental model.
 # Given more time I'd ask: count of larger elements after self?
 # Query total_right – query(rank[num]) instead."

◆ Quick Review

Key Insights

- The brute force solution takes $O(n^2)$ time, which is too slow for large input arrays.
- An efficient approach leverages the concept of Merge Sort to both sort the array and count the number of smaller elements by tracking the number of elements that "jump over" during the merge step.

- Other efficient methods include Binary Indexed Trees (Fenwick Tree) or Balanced Binary Search Trees.

Space & Time

Time Complexity: $O(n \log n)$ on average due to the merge sort based approach.

Space Complexity: $O(n)$ for auxiliary arrays used during sorting and counting.

Solution

We use a modified Merge Sort to sort the array while counting the number of smaller elements to the right. We keep track of the original indices by pairing each element with its index. During the merge process, when an element from the left subarray is placed into the temporary array, we add the count of elements from the right subarray that have already been placed (since these represent smaller elements that were to its right). This efficient divide and conquer strategy guarantees an overall $O(n \log n)$ time complexity. Care must be taken to update the counts based on indices and during the merge step.

Binary Search

#410 Split Array Largest Sum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming · Greedy · Prefix Sum

Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is minimized.

Return the minimized largest sum of the split. A subarray is a contiguous part of the array.

Example 1:

Input: `nums = [7,2,5,10,8]`, `k = 2`

Output: 18

Explanation: There are four ways to split `nums` into two subarrays.

The best way is to split it into `[7,2,5]` and `[10,8]`, where the largest sum among the two subarrays is only 18.

Example 2:

Input: `nums = [1,2,3,4,5]`, `k = 2`

Output: 9

Explanation: There are four ways to split `nums` into two subarrays.

The best way is to split it into `[1,2,3]` and `[4,5]`, where the largest sum among the two subarrays is only 9.

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 106$
- $1 \leq k \leq \min(50, \text{nums.length})$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Split nums into k non-empty contiguous subarrays to minimise the maximum subarray sum.

Must be in order – no reordering allowed. Right?

Same 'binary search on answer' pattern as Capacity to Ship Packages (#1011)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try every split of nums into k contiguous groups (DP over partition points).

For each split, max subarray sum = largest group sum; minimize that over splits.

State space explodes without binary search on the answer.

Time: exponential / $O(n^k)$ naive partitions. Space: $O(n)$ recursion.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Binary search on the answer (the max sum).

lo = max(nums) (at least one element), hi = sum(nums) (one subarray).

Feasibility check: can we split into k subarrays with each sum $\leq$ mid?

Greedy: accumulate greedily, start new subarray when exceeding mid. Count splits.

Time: $O(n \log(\text{sum}))$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _410_SplitArrayLargestSum:
    def splitArray(self, nums: List[int], k: int) -> int:
        def can_split(max_sum: int) -> bool:
            subarrays = 1
            current = 0
            for num in nums:
                if current + num > max_sum:
                    subarrays += 1
                    current = 0
```



```

        current += num
    return subarrays <= k
lo, hi = max(nums), sum(nums)
while lo < hi:
    mid = (lo + hi) // 2
    if can_split(mid):
        hi = mid # valid - try smaller
    else:
        lo = mid + 1 # not valid - need bigger
return lo

```

PHASE 5 — TEST & DEBUG

```

# [7,2,5,10,8], k=2: lo=10, hi=32. mid=21→can_split? [7,2,5] and [10,8]=18≤21 ✓ → hi=21.
# ... converges to 18. can_split(18)=[7,2,5][10,8]→2 splits≤k=2 ✓.
can_split(17)→[7,2,5][10,8]→10+8=18>17→need 3>k ✗. → 18 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n \log(\text{sum}))$ . Space  $O(1)$ . Same template as #1011 Capacity to Ship - name this.
# lo/hi bounds: must be at least max(nums), at most sum(nums).
# Given more time I'd ask: DP approach?  $dp[j][i]$  = min largest sum splitting first i elements into j parts.
#  $O(kn^2)$  - worse than binary search for large inputs."

```

◆ Quick Review

Key Insights

- Use binary search over the answer: the candidate value is the maximum subarray sum.
- The lower bound of the search is $\max(\text{nums})$ and the upper bound is $\text{sum}(\text{nums})$.
- A greedy approach is used to check if a candidate maximum sum allows splitting `nums` into at most `k` subarrays.
- Adjust the binary search bounds based on the feasibility check.

Space & Time

Time Complexity: $O(n * \log(\text{sum}(\text{nums})))$ – n elements are processed for each binary search iteration.

Space Complexity: $O(1)$ – Constant extra space is used.

Solution

The solution employs binary search to determine the smallest maximum subarray sum possible. We set the initial lower bound to the maximum element and the upper bound to the total sum of the array. For each candidate value (mid), we check if the array can be split into k or fewer subarrays without any subarray sum exceeding mid using a greedy approach. If possible, we try to lower the candidate; if not, we increase the candidate. This continues until the optimal solution is found.

Binary Search

□ #668 Kth Smallest Number in Multiplication Table

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search
Lists: Denny Zhang

Problem

Nearly everyone has used the Multiplication Table. The multiplication table of size $m \times n$ is an integer matrix `mat` where `mat[i][j] == i * j` (1-indexed).

Given three integers m , n , and k , return the k th smallest element in the $m \times n$ multiplication table.

Example 1:

1	2	3
2	4	6
3	6	9

1	2	2	3	3	4	6	6	9
---	---	---	---	---	---	---	---	---

Input: $m = 3, n = 3, k = 5$
Output: 3
Explanation: The 5th smallest number is 3.

Example 2:

1	2	3
2	4	6

1	2	2	3	4	6
---	---	---	---	---	---

Input: $m = 2, n = 3, k = 6$

Output: 6

Explanation: The 6th smallest number is 6.

Constraints:

- $1 \leq m, n \leq 3 * 10^4$
- $1 \leq k \leq m * n$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"m×n multiplication table: $\text{mat}[i][j]=i*j$ (1-indexed). Find kth smallest element. Right?

Can't materialise the table (m,n up to $3*10^4 \rightarrow$ up to $9*10^8$ cells)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect all m×n products into a list, sort, return kth element.

Correct but materializes every product – $O(mn \log(mn))$ time and $O(mn)$ space.

Time: $O(m * n * \log(m * n))$. Space: $O(m * n)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (binary search on value)

"Binary search on the answer value v.

Count elements $\leq v$: for row i, elements are $i, 2i, 3i, \dots, n*i$.

Count of these $\leq v$ is $\min(v//i, n)$. Sum over all rows $i=1..m$.

Binary search for smallest v where count $\geq k$.

$\text{lo}=1, \text{hi}=m*n$. Time: $O(m \log(m*n))$."

PHASE 4 — CLEAN CODE

```
class _668_KthSmallestMultTable:
    def findKthNumber(self, m: int, n: int, k: int) -> int:
        def count_le(v: int) -> int:
            return sum(min(v // i, n) for i in range(1, m + 1))
        lo, hi = 1, m * n
        while lo < hi:
            mid = (lo + hi) // 2
            if count_le(mid) >= k:
                hi = mid
```

```

else:
    lo = mid + 1
return lo

```

PHASE 5 — TEST & DEBUG

```

# m=3,n=3,k=5. Table: [1,2,3,2,4,6,3,6,9] sorted=[1,2,2,3,3,4,6,6,9]. 5th=3.
# count_le(3)=count<=3? i=1:min(3,3)=3, i=2:min(1,3)=1, i=3:min(1,3)=1. sum=5<=5 →
hi=3.
# ... lo converges to 3 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(m log(m*n)). Space O(1). The count formula is the key – O(m) per binary
search step.
# Note: the answer might not be a 'real' table entry – binary search finds the
smallest v
# with count >= k, which IS always a table entry because count jumps discretely at
table values.
# Given more time I'd ask: kth largest? Binary search on m*n – k + 1 smallest
instead."

```

◆ Quick Review

Key Insights

- The table elements are not fully sorted row-wise or column-wise globally, but there is a pattern that can be exploited.
- For any candidate number x , we can count how many numbers in the table are less than or equal to x by summing over rows: for row i , there are $\min(x // i, n)$ numbers.
- Binary search can be used over the range $[1, m*n]$ to find the smallest x such that the count is at least k .
- The counting function is the key to connecting the candidate value with the order

statistic.

Space & Time

Time Complexity: $O(m * \log(m*n))$

Space Complexity: $O(1)$

Solution

We use binary search over the possible values in the multiplication table, which range from 1 to $m*n$. For each midpoint x in the binary search, we compute the number of elements in the table that are less than or equal to x by iterating over each row and summing up $\min(x // i, n)$. This count tells us whether the k th smallest element is to the left or right of x . We then adjust the binary search boundaries accordingly. The approach leverages the multiplicative structure of the table and the monotonicity of the counting function.

Binary Search

□ #710 Random Pick with Blacklist

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Binary Search ·
Sorting · Randomized
Lists: Denny Zhang

Problem

You are given an integer n and an array of unique integers `blacklist`. Design an algorithm to pick a random integer in the range $[0, n - 1]$ that is not in `blacklist`. Any integer that is in the mentioned range and not in `blacklist` should be equally likely to be returned.

Optimize your algorithm such that it minimizes the number of calls to the built-in random function of your language.

Implement the `Solution` class:

- `Solution(int n, int[] blacklist)` Initializes the object with the integer n and the blacklisted integers `blacklist`.

- **int pick()** Returns a random integer in the range $[0, n - 1]$ and not in blacklist.

Example 1:

Input

```
["Solution", "pick", "pick", "pick", "pick", "pick", "pick", "pick"]
[[7, [2, 3, 5]], [], [], [], [], [], [], []]
```

Output

```
[null, 0, 4, 1, 6, 1, 0, 4]
```

Explanation

```
Solution solution = new Solution(7, [2, 3, 5]);
```

Constraints:

- $1 \leq n \leq 109$
- $0 \leq \text{blacklist.length} \leq \min(105, n - 1)$
- $0 \leq \text{blacklist}[i] < n$
- All the values of blacklist are unique.
- At most $2 * 10^4$ calls will be made to pick.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Pick uniformly at random from $[0, n-1]$ excluding blacklist. One call to random per pick. Right?

n can be 10^9 – can't enumerate all valid numbers."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Store all non-blacklisted ids in a list; pick random index.

If blacklist is small, filter on each pick – $O(n)$ per draw.

Time: $O(n)$ per pick naive. Space: $O(n)$ whitelist.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (virtual mapping)

"Let $sz = n - \text{len}(\text{blacklist})$ = count of valid numbers.

Map $[0, sz)$ as the 'virtual' range. Numbers in $[0, sz)$ that ARE blacklisted

get remapped to non-blacklisted numbers in $[sz, n)$.

`pick()`: random in $[0, sz) \rightarrow$ if in remap $\rightarrow$ use `remap[r]`, else use `r` directly.

Build remap in `__init__`: for each blacklisted number in $[0, sz)$, map it to a non-blacklisted

number in $[sz, n)$. $O(|\text{blacklist}|)$ init, $O(1)$ per pick."

PHASE 4 — CLEAN CODE

```
class _710_RandomPickBlacklist:
```

```
    def __init__(self, n: int, blacklist: List[int]):
```

```
        bl_set = set(blacklist)
```

```
        self.sz = n - len(blacklist) # size of valid virtual range
```

```
        self.remap: dict = {}
```

```
        tail = n - 1
```

```
        for b in blacklist:
```

```
            if b < self.sz: # this blacklisted number is in [0, sz)
```

```
                while tail in bl_set: # find next valid number from the tail
```

```
                    tail -= 1
```

```
                self.remap[b] = tail
```

```
                tail -= 1
```

```
    def pick(self) -> int:
```

```
        r = random.randint(0, self.sz - 1)
```

```
        return self.remap.get(r, r)
```

PHASE 5 — TEST & DEBUG

$n=7$, `blacklist=[2,3,5]`. $sz=4$. Valid: $[0,1,4,6]$.

Blacklisted in $[0,4)$: 2,3. Map $2 \rightarrow 6$, $3 \rightarrow 4$ (5 is blacklisted, skip $\rightarrow 4$).

`remap={2:6, 3:4}`. `pick(0)` $\rightarrow 0$, `pick(1)` $\rightarrow 1$, `pick(2)` $\rightarrow 6$, `pick(3)` $\rightarrow 4$. All valid ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Init $O(|B|)$. Pick $O(1)$. Space $O(|B|)$ for remap.

The key insight: divide $[0, n)$ into 'virtual valid' $[0, sz)$ and 'tail' $[sz, n)$.

Blacklisted numbers in $[0, sz)$ redirect to valid numbers in the tail.

Given more time I'd ask: what if the blacklist updates dynamically?

Rebuild the remap on each update — $O(|B|)$ per update, same $O(1)$ pick."

◆ Quick Review

Key Insights

- The valid output range has a total count of available numbers $W = n - \text{len}(\text{blacklist})$.

- Instead of repeatedly generating random numbers and checking against a set, we can transform the range into a contiguous block $[0, W - 1]$ and map any blacklist number within that block to a valid number in the upper range.
- Mapping is precomputed: for any blacklisted number x in $[0, W-1]$, assign it a valid number from $[W, n-1]$ that is not blacklisted.
- This approach ensures $O(1)$ pick time after an $O(b)$ precomputation where $b = \text{length of blacklist}$.

Space & Time

Time Complexity: constructor – $O(b)$ for initializing data structures, pick() – $O(1)$ per call.

Space Complexity: $O(b)$ for storing the blacklist set and mapping.

Solution

We compute the count of valid numbers, $W = n - \text{len}(\text{blacklist})$. For numbers in the blacklist that are less than W , we need to map them to a valid number from the range $[W, n-1]$. To do this, we first store all blacklist numbers in the

higher range $[W, n-1]$ in a set for quick lookup. Then for each blacklisted value in $[0, W-1]$, we iterate from W upwards to find a number that is not blacklisted and assign it in a mapping. During the `pick()` function, we generate a random integer r in $[0, W-1]$. If r is in our mapping, we return its corresponding mapped value; otherwise, we return r as it is already a valid number.

Binary Search

□ #774 Minimize Max Distance to Gas Station

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

You are given an integer array `stations` that represents the positions of the gas stations on the x -axis. You are also given an integer k . You should add k new gas stations. You can add the stations anywhere on the x -axis, and not necessarily on an integer position.

Let `penalty()` be the maximum distance between adjacent gas stations after adding the k new stations.

Return the smallest possible value of `penalty()`. Answers within 10^{-6} of the actual answer will be accepted.

Example 1:

```
Input: stations = [1,2,3,4,5,6,7,8,9,10], k = 9
Output: 0.50000
```

Example 2:

Input: stations = [23,24,36,39,46,56,57,65,84,98], k = 1
Output: 14.00000

Constraints:

- $10 \leq \text{stations.length} \leq 2000$
- $0 \leq \text{stations}[i] \leq 108$
- stations is sorted in a strictly increasing order.
- $1 \leq k \leq 106$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Add exactly k stations anywhere (real-valued positions) to minimise the maximum gap.
Return the answer within 1e-6 precision. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
Binary search on the maximum distance between adjacent stations.
For candidate d, check if we can add k stations so all gaps $\leq d$ (greedy fill).
Brute: try every real-valued spacing without binary search – continuous search space.
Time: $O(n * \text{precision})$ naive scan. Space: $O(1)$.
Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (binary search on floating-point answer)

"Binary search on the max gap d. Feasibility: for each existing gap g, we need $\text{ceil}(g/d) - 1$ extra
stations to keep all sub-gaps $\leq d$. Count total extras; if $\leq k$, d is achievable.
Precision: run until $hi - lo < 1e-7$."

PHASE 4 — CLEAN CODE

```

class _774_MinMaxDistanceGasStation:
    def minmaxGasDist(self, stations: List[int], k: int) -> float:
        gaps = [stations[i+1] - stations[i] for i in range(len(stations)-1)]
        def can_achieve(d: float) -> bool:
            return sum(int(g / d) for g in gaps) <= k
        lo, hi = 0.0, max(gaps)
        for _ in range(100): # 100 iterations -> precision < hi/2^100
            mid = (lo + hi) / 2
            if can_achieve(mid):
                hi = mid
            else:
                lo = mid
        return hi

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n * 100) = O(n)$. Space $O(n)$ for gaps. Floating-point binary search with fixed iterations.

Note: $\text{int}(g/d)$ gives the number of stations to insert (not $\text{ceil}-1$ – same thing via floor division).

Given more time I'd ask: what if stations must be at integer positions?

Binary search on integers, same feasibility check but with math.ceil ."

◆ Quick Review

Key Insights

- The optimal "penalty" (i.e., the maximum distance between adjacent stations after placement) can be found using binary search over a continuous distance range.
- For each candidate distance (mid), simulate the number of gas stations required by iterating over each adjacent station gap and calculating:
 - $\text{stations_required} = \text{ceil}(\text{gap} / \text{mid}) - 1$

- If the total number of required additional stations is less than or equal to k , then mid is a feasible candidate; otherwise, it is too small.
- Continue the binary search until the gap between low and high is within the desired tolerance ($1e-6$).

Space & Time

Time Complexity: $O(n * \log(\text{max_gap}/\text{precision}))$, where n is the number of stations.

Space Complexity: $O(1)$ (ignoring input storage)

Solution

We use binary search over the possible maximum distances. The algorithm sets $low = 0$ and $high =$ the maximum gap between adjacent stations. For a candidate distance mid , we count how many new stations are required for all gaps by computing $\text{ceil}(\text{gap} / mid) - 1$. If the total required is within k , we try a smaller candidate; otherwise, we increase the candidate distance. This approach efficiently zooms into the minimum possible maximum distance (penalty).

Binary Search

□ #1235 Maximum Profit in Job Scheduling **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming ·
Sorting

Lists: Grind 169

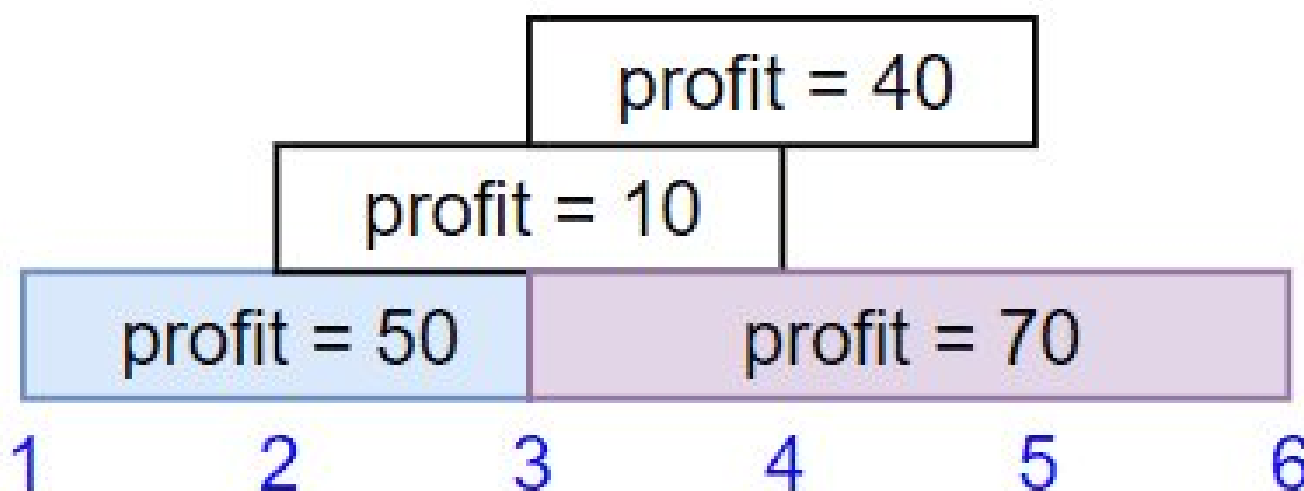
Problem

We have n jobs, where every job is scheduled to be done from $startTime[i]$ to $endTime[i]$, obtaining a profit of $profit[i]$.

You're given the $startTime$, $endTime$ and $profit$ arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

If you choose a job that ends at time X you will be able to start another job that starts at time X .

Example 1:



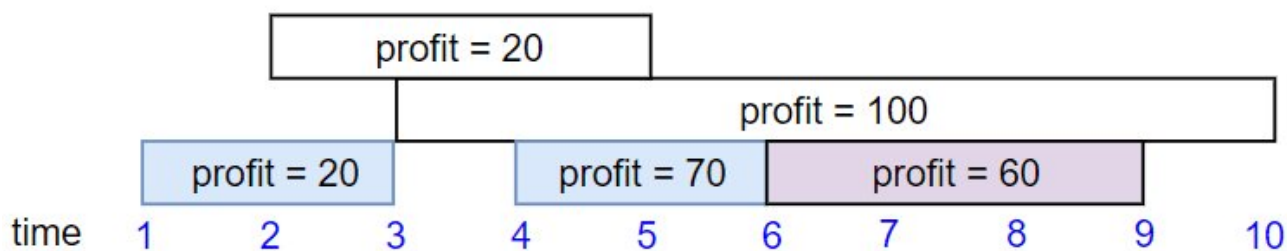
Input: `startTime = [1,2,3,3]`, `endTime = [3,4,5,6]`, `profit = [50,10,40,70]`

Output: 120

Explanation: The subset chosen is the first and fourth job.

Time range [1-3]+[3-6], we get profit of $120 = 50 + 70$.

Example 2:



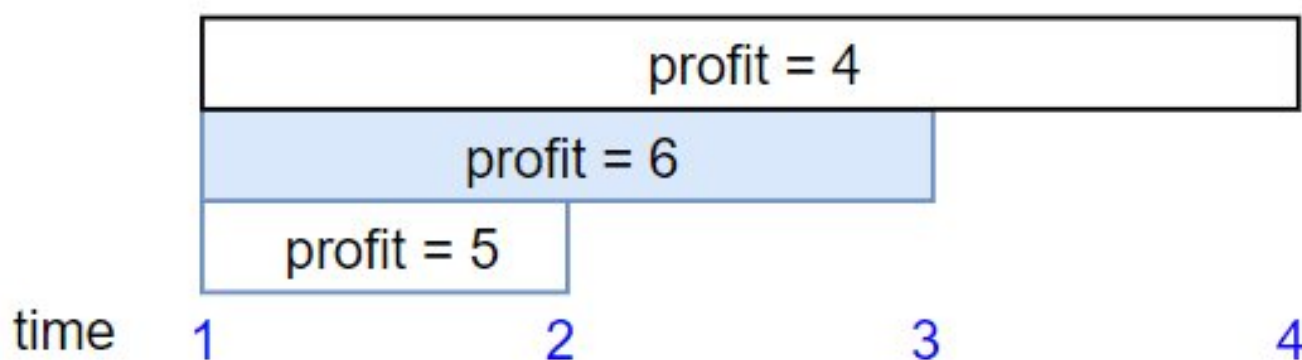
Input: `startTime = [1,2,3,4,6]`, `endTime = [3,5,10,6,9]`, `profit = [20,20,100,70,60]`

Output: 150

Explanation: The subset chosen is the first, fourth and fifth job.

Profit obtained $150 = 20 + 70 + 60$.

Example 3:



Input: startTime = [1,1,1], endTime = [2,3,4], profit = [5,6,4]
 Output: 6

Constraints:

- $1 \leq \text{startTime.length} == \text{endTime.length} == \text{profit.length} \leq 5 * 10^4$
- $1 \leq \text{startTime}[i] < \text{endTime}[i] \leq 10^9$
- $1 \leq \text{profit}[i] \leq 10^4$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Select non-overlapping jobs to maximise total profit.

Job ending at X and starting at X don't overlap. Return max profit. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Sort jobs by end time. DP over index: $\text{dp}[i] = \text{max profit taking job } i \text{ plus best non-overlapping before } i$.

For each i , scan all $j < i$ for compatibility - $O(n^2)$ DP.

Time: $O(n^2)$. Space: $O(n)$ DP + sort.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (sort by end time + DP + binary search)

"Sort jobs by end time. $\text{dp}[i] = \text{max profit using first } i \text{ jobs}$.

```
# For each job i: skip → dp[i] = dp[i-1]. Take → find latest j where job[j].end <=
job[i].start
# using binary search. dp[i] = max(dp[i-1], dp[j] + profit[i]).
# Time: O(n log n)."
```

PHASE 4 — CLEAN CODE

```
class _1235_MaxProfitJobScheduling:
    def jobScheduling(self, startTime: List[int], endTime: List[int], profit:
List[int]) -> int:
        jobs = sorted(zip(endTime, startTime, profit))
        ends = [0] # dp_ends[i] = end time of ith job in dp
        dp = [0] # dp[i] = max profit for first i sorted jobs
        for end, start, p in jobs:
            # Find latest job whose end <= start of current
            j = bisect.bisect_right(ends, start) - 1
            if dp[j] + p > dp[-1]:
                dp.append(dp[j] + p)
                ends.append(end)
            else:
                dp.append(dp[-1])
                ends.append(end)
        return dp[-1]
```

PHASE 5 — TEST & DEBUG

```
# jobs sorted by end: [(3,1,50),(4,2,10),(5,3,40),(6,3,70)].
# job(3,1,50): j=bisect_right([0],1)-1=0. dp[0]+50=50>dp[-1]=0 →
dp=[0,50],ends=[0,3].
# job(4,2,10): bisect_right([0,3],2)-1=0. dp[0]+10=10<dp[-1]=50 →
dp=[0,50,50],ends=[0,3,4].
# job(5,3,40): bisect_right([0,3,4],3)-1=1. dp[1]+40=90>50 → dp=[0,50,50,90].
# job(6,3,70): bisect_right([0,3,4,5],3)-1=1. dp[1]+70=120>90 →
dp=[0,50,50,90,120]. → 120 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n log n). Space O(n). Sort + DP with binary search per step.
# The dp array is monotonically non-decreasing – binary search on ends is valid.
# Given more time I'd ask: what if we want to select at most k jobs?
# Add a dimension to dp – dp[i][k] – O(n²k) with the same binary search for last
compatible job."
```

◆ Quick Review

Key Insights

- Sort jobs based on their end times to simplify finding non-conflicting jobs.

- Use dynamic programming where the state represents the maximum profit until a given point.
- For each job, perform a binary search to find the last job that ends before the current job's start time.
- The recurrence relation is: $dp[i] = \max(dp[i-1], \text{profit}[i] + dp[\text{index}])$ where $dp[\text{index}]$ is from the latest non-conflicting job.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting and binary search for each job.

Space Complexity: $O(n)$ for the dp arrays.

Solution

The solution begins by sorting the jobs by their end times. For each job, a binary search is used to quickly locate the last job that does not conflict with the current job (i.e., whose end time is less than or equal to the current job's start time). Then, using dynamic programming, the maximum profit is updated by choosing whether to include the current job or skip it. Two parallel arrays (or lists) are

maintained: one for the end times of the jobs considered so far, and another to store the corresponding maximum profit at those times. This approach efficiently computes the answer while keeping time complexity within acceptable bounds.

Matrix

Round 3 · High · Hard · 1 question

Matrix

□ #1074 Number of Submatrices That Sum to Target **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix · Prefix Sum
Lists: Denny Zhang

Problem

Given a matrix and a target, return the number of non-empty submatrices that sum to target. A submatrix $x1, y1, x2, y2$ is the set of all cells $matrix[x][y]$ with $x1 \leq x \leq x2$ and $y1 \leq y \leq y2$.

Two submatrices $(x1, y1, x2, y2)$ and $(x1', y1', x2', y2')$ are different if they have some coordinate that is different: for example, if $x1 \neq x1'$.

Example 1:

0	1	0
1	1	1
0	1	0

Input: matrix = `[[0,1,0],[1,1,1],[0,1,0]]`, target = 0

Output: 4

Explanation: The four 1x1 submatrices that only contain 0.

Example 2:

Input: matrix = `[[1,-1],[-1,1]]`, target = 0

Output: 5

Explanation: The two 1x2 submatrices, plus the two 2x1 submatrices, plus the 2x2 submatrix.

Example 3:

Input: matrix = `[[904]]`, target = 0

Output: 0

Constraints:

- $1 \leq \text{matrix.length} \leq 100$
- $1 \leq \text{matrix}[0].\text{length} \leq 100$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$
- $-10^8 \leq \text{target} \leq 10^8$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count all submatrices (r1,c1,r2,c2) whose elements sum to target. Right?"
 # Extend #560 Subarray Sum Equals K to 2D."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
 # Compute 2D prefix sums, then enumerate all submatrix top-left and bottom-right corners.
 # For each submatrix, $O(1)$ range sum check against target.
 # Four nested loops over corners – $O(n^2 * m^2)$ submatrices.
 # Time: $O(n^2 * m^2)$. Space: $O(n * m)$ prefix grid.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Fix top row r1 and bottom row r2. Compress columns into 1D prefix sums between r1..r2."
 # Then apply #560 (subarray sum = target) on the 1D array.
 # Time: $O(m^2 * n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _1074_SubmatricesSumTarget:
    def numSubmatrixSumTarget(self, matrix: List[List[int]], target: int) -> int:
        m, n = len(matrix), len(matrix[0])
        count = 0
        for r1 in range(m):
            col_sum = [0] * n
            for r2 in range(r1, m):
                for c in range(n):
                    col_sum[c] += matrix[r2][c]
                # Apply #560 on col_sum
                prefix = Counter({0: 1})
```

```

        running = 0
        for s in col_sum:
            running += s
            count += prefix[running - target]
            prefix[running] += 1
    return count

```

PHASE 5 — TEST & DEBUG

```

# [[0,1,0],[1,1,1],[0,1,0]], target=0:
# Fix r1=0,r2=0: col_sum=[0,1,0]. Subarrays summing to 0: [0] at c=0, [0] at c=2.
count=2.
# Other rows contribute 2 more. → 4 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(m^2 * n)$ . Space  $O(n)$ . This is literally #560 applied inside a 2D row-fixing
loop.
# Name that connection – it shows pattern transfer.
# Given more time I'd ask: count submatrices summing to at most target?
# Use #862 (Shortest Subarray with Sum at Least K) approach – monotone deque instead
of hash map."

```

◆ Quick Review

Key Insights

- Transform the 2D submatrix sum problem into multiple 1D subarray sum problems.
- Precompute cumulative sums by fixing two row boundaries and then summing columns between these rows.
- Use a hash table (or dictionary) to count the number of subarrays with a given sum, reducing the problem to the classic "subarray sum equals k" question.
- Iterate over all possible row pairs and for each, treat the column sums as an array where

the target sum is sought.

Space & Time

Time Complexity: $O(n^2 * m)$, where n is the number of rows and m is the number of columns.

Space Complexity: $O(m)$, used by the hash table for storing cumulative sums along the columns.

Solution

The idea is to iterate over all pairs of rows (top and bottom) and, for each pair, compute the cumulative column sums between these rows. This converts the problem into finding the number of subarrays in a one-dimensional array (the column sums) that add up to the target. For each such 1D problem, use a hash table to track the frequency of cumulative sums seen so far and count how many times the difference (current sum – target) has been seen. This count gives the number of subarrays ending at the current index which sum to target. Combining counts from all possible row pairs yields the final result.

Trees & BST

Round 3 · High · Hard · 2 questions

Trees & BST

□ #124 Binary Tree Maximum Path Sum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Tree · DFS

Lists: Grind 169

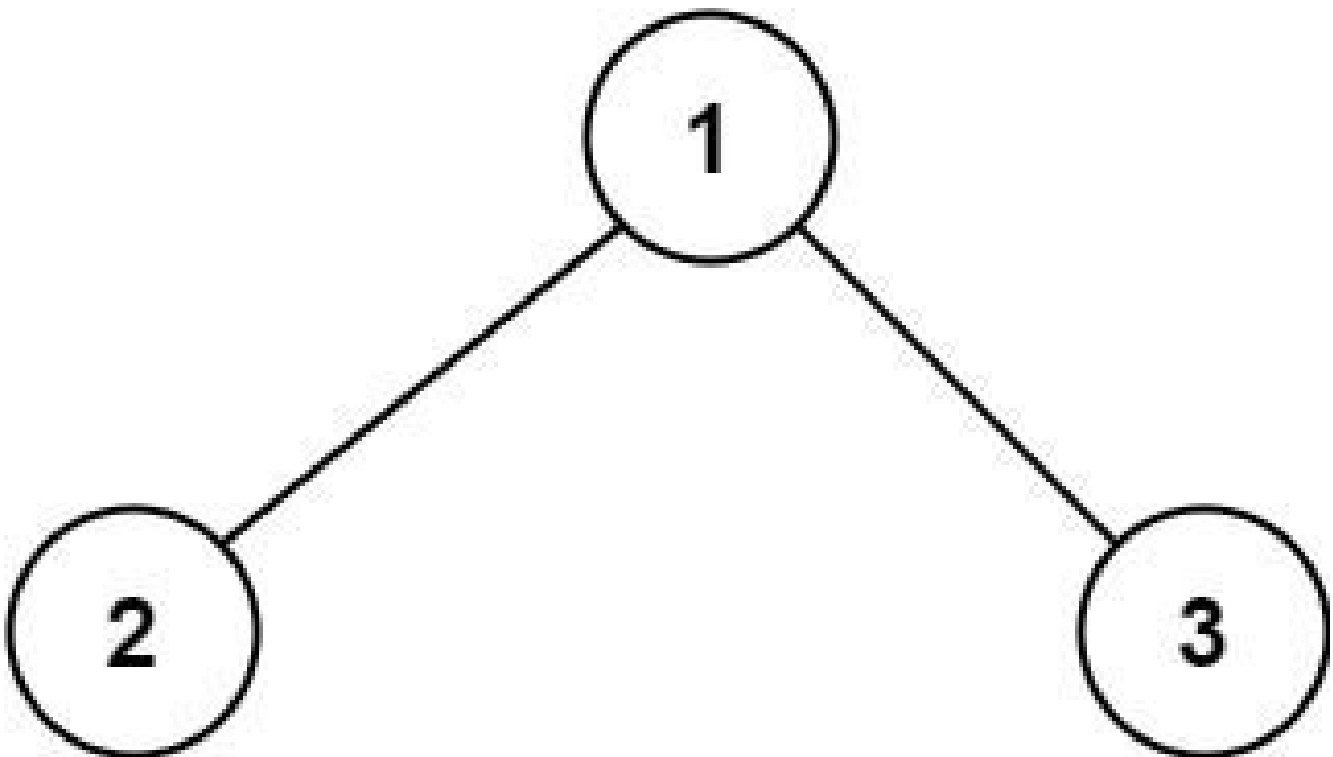
Problem

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

Given the root of a binary tree, return the maximum path sum of any non-empty path.

Example 1:

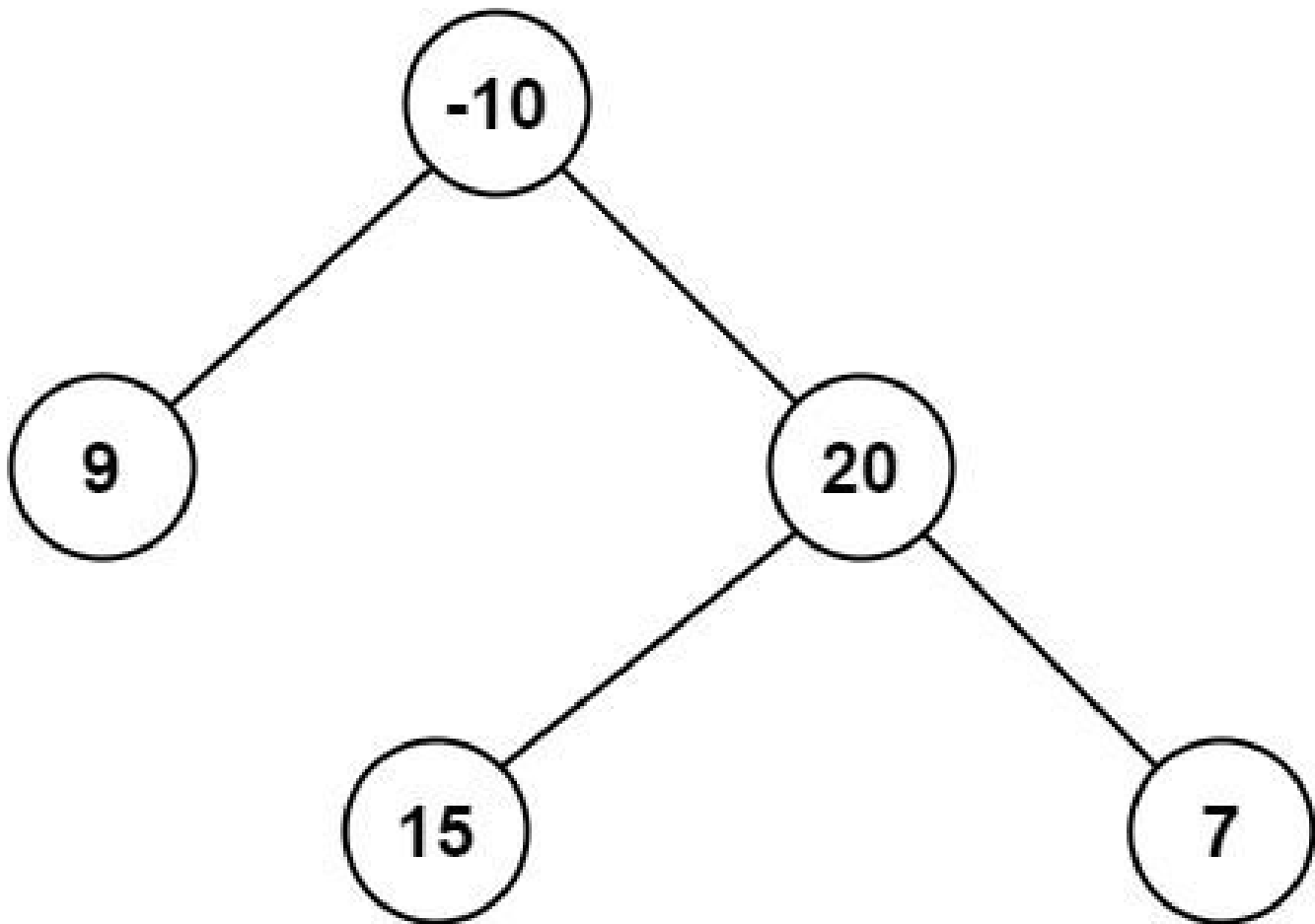


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of $2 + 1 + 3 = 6$.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.

Constraints:

- The number of nodes in the tree is in the range $[1, 3 * 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Path = any sequence of connected nodes (no revisiting). Path doesn't need to pass
through root.
# Values can be negative. Return maximum path sum. Right?"
#
# "[-10,9,20,null,null,15,7]: path 15→20→7 = 42 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# At every node, compute max path sum through that node = node.val + max(0,
left_gain) + max(0, right_gain).
# Recurse full tree; track global max. Recomputes subtree gains repeatedly without
sharing.
# Time: O(n^2) worst case on skewed tree. Space: O(h) stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (post-order, same as Diameter)

```
# "For each node, compute the 'gain' if we extend a path through it: max(0,
left_gain, right_gain) + node.val.
# The best path THROUGH this node = node.val + max(0,left) + max(0,right).
# Update global max. Return one-sided gain to parent (can't go both left and right
AND up).
# Use inner helper – no self."
```

PHASE 4 — CLEAN CODE

```
class _124_MaxPathSum:
    def maxPathSum(self, root: Optional['TreeNode']) -> int:
        best = [float('-inf')]
        def gain(node):
            if not node: return 0
            left = max(0, gain(node.left))
            right = max(0, gain(node.right))
            best[0] = max(best[0], node.val + left + right)
            return node.val + max(left, right)
        gain(root)
        return best[0]
```

PHASE 5 — TEST & DEBUG

```
# [-10,9,20,null,null,15,7]:
# gain(9)=9. gain(15)=15. gain(7)=7.
# gain(20): left=15,right=7. best=max(-inf,20+15+7)=42. return 20+15=35.
# gain(-10): left=9,right=35. best=max(42,-10+9+35)=42. return -10+35=25.
# → 42 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h). Same post-order pattern as Diameter of Binary Tree (#543)
and Balanced Tree (#110).
# The 'max(0, gain)' is essential – a negative subtree gain should be dropped.
# Given more time I'd ask: return the actual path (not just the sum)?"
```

Track which path achieved best[0] – store node references alongside the max update."

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore the tree in a post-order fashion.
- For each node, calculate the maximum gain including that node and optionally one of its subtrees.
- A node's best contribution to its parent can be either just its own value or its value plus the maximum gain from either the left or right child.
- Update a global maximum path sum that considers the possibility of combining both left and right gains with the current node.
- Handle negative values by comparing gains against 0 to avoid decreasing the path sum.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree, since each node is visited exactly once.

Space Complexity: $O(n)$ in the worst-case scenario due to the recursion stack ($O(H)$)

where H is the height of the tree, which can be $O(n)$ for skewed trees).

Solution

We use a recursive DFS approach (post-order traversal) to compute the maximum gain from each subtree. For every node: Recursively compute the maximum gain from the left and right child, ensuring that negative gains are treated as 0. The maximum gain that can be provided to the node's parent is the node's value plus the larger gain from its left or right child. Simultaneously, the potential maximum path sum including the current node as the highest (or root) node is the node's value plus both left and right gains. Update a global variable that holds the maximum path sum found so far. Return the maximum gain (node value plus one best child) to be used by the node's parent. This strategy ensures that every possible path sum is considered while avoiding double-counting nodes.

Trees & BST

□ #297 Serialize and Deserialize Binary Tree

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Tree · DFS · BFS · Design
Lists: Grind 169

Problem

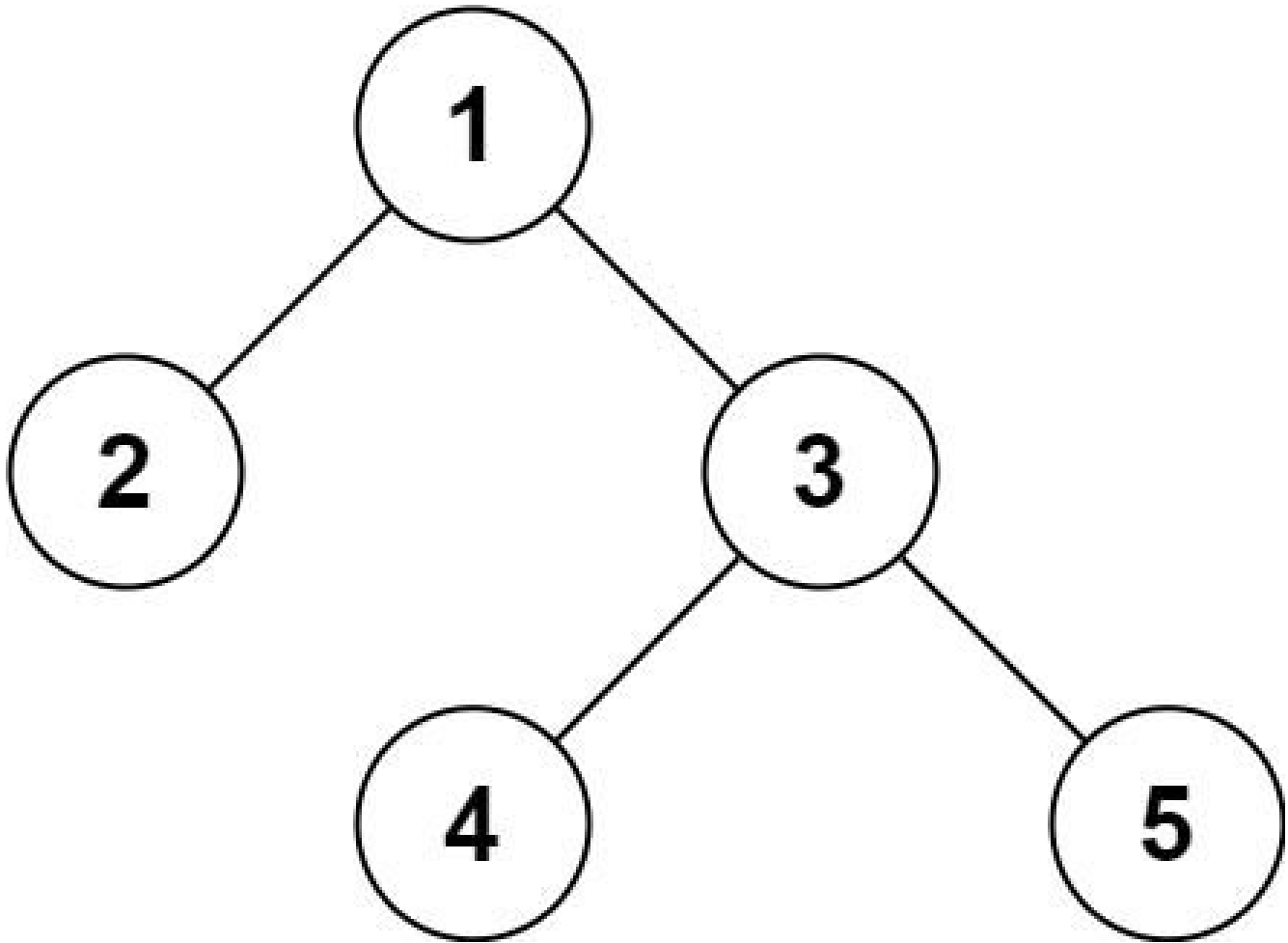
Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as how LeetCode serializes a binary tree.

You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:



Input: root = [1,2,3,null,null,4,5]

Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-1000 \leq \text{Node.val} \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design encode and decode functions for a binary tree. Any format is fine as long
as it round-trips.
# Empty tree encodes to some valid string. Right?"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Serialize: preorder traversal with 'null' markers for empty children ? comma
string.
# Deserialize: split queue, rebuild tree recursively matching preorder order.
# Correct but string can be long; BFS level-order encoding is more compact.
# Time: O(n). Space: O(n) output string.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (pre-order DFS with null markers)

```
# "Serialize: pre-order traversal, 'N' for null, comma-separated.
# Deserialize: split, use an iterator, recursively build from pre-order.
# Pre-order uniquely determines the tree when nulls are included."
```

PHASE 4 — CLEAN CODE

```
class _297_Codec:
    def serialize(self, root: 'TreeNode') -> str:
        parts: List[str] = []
        def dfs(node):
            if not node: parts.append('N'); return
            parts.append(str(node.val))
            dfs(node.left)
            dfs(node.right)
        dfs(root)
        return ','.join(parts)
    def deserialize(self, data: str) -> 'TreeNode':
        vals = iter(data.split(','))
        def build():
```

```

        v = next(vals)
        if v == 'N': return None
        node = TreeNode(int(v))
        node.left = build()
        node.right = build()
        return node
    return build()

```

PHASE 5 — TEST & DEBUG

[1,2,3,null,null,4,5]: serialize → '1,2,N,N,3,4,N,N,5,N,N'. deserialize rebuilds the tree ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Pre-order with null markers is the cleanest approach.

BFS alternative: level-order like LeetCode's own format – works but trickier to parse.

Given more time I'd ask: serialize a general N-ary tree?

Add child count before children – e.g. '1,3,2,0,3,0,4,0,5,0' for a 3-child root."

◆ Quick Review

Key Insights

- The tree can be traversed using either depth-first search (DFS) or breadth-first search (BFS).
- Using BFS (level-order traversal) simplifies reconstruction since nodes are processed level-by-level.
- Placeholder markers (e.g., "null") are used to indicate missing children.
- Both serialization and deserialization processes should handle empty trees and edge cases gracefully.

- Ensuring a one-to-one mapping between the tree structure and the serialized string is critical.

Space & Time

Time Complexity: $O(n)$ for both serialization and deserialization, where n is the number of nodes in the tree.

Space Complexity: $O(n)$ due to the storage used for the serialized string and additional data structures (queue) during processing.

Solution

This solution uses level-order traversal (BFS) for both serialization and deserialization.

During serialization, each node is processed sequentially with missing children recorded as "null". For deserialization, the string is split by commas, and a queue is used to rebuild the tree level by level. The root is created from the first token, then subsequent tokens are assigned as left and right children accordingly, ensuring an accurate reconstruction of the original tree.

DFS

Round 3 · High · Hard · 5 questions

DFS

□ ★ #269 Alien Dictionary ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · DFS · BFS · Graph · Topological
Sort

Lists: ★ Premium | Grind 169 | Premium 98

Problem

There is a new alien language that uses the English alphabet. However, the order of the letters is unknown to you.

You are given a list of strings words from the alien language's dictionary. Now it is claimed that the strings in words are sorted lexicographically by the rules of this new language.

If this claim is incorrect, and the given arrangement of string in words cannot correspond to any order of letters, return "".

Otherwise, return a string of the unique letters in the new alien language sorted in lexicographically increasing order by the new language's rules. If there are multiple solutions,

return any of them.

Example 1:

```
Input: words = ["wrt","wrf","er","ett","rftt"]  
Output: "wertf"
```

Example 2:

```
Input: words = ["z","x"]  
Output: "zx"
```

Example 3:

```
Input: words = ["z","x","z"]  
Output: ""  
Explanation: The order is invalid, so return "".
```

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 100$
- $\text{words}[i]$ consists of only lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given words sorted in alien lexicographic order, deduce the letter ordering.  
# Return any valid ordering, or '' if impossible (cycle or contradiction). Right?  
# Contradiction: longer word precedes its prefix (e.g. ['ab','a'])."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.  
# Build graph: for each adjacent pair in words, record char order u before v.  
# Topological sort the chars; if cycle detected, no valid order."
```

```

# Compare words character by character to infer edges.
# Time: O(C) total chars. Space: O(1) – 26 letters.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (Kahn's BFS topological sort)
# "Extract ordering constraints from consecutive word pairs: compare char by char,
# first difference gives an edge.
# Build directed graph + in-degree map.
# BFS: process zero-in-degree nodes. If we process all letters → valid. Else cycle →
# return ''."

# PHASE 4 — CLEAN CODE
class _269_AlienDictionary:
    def alienOrder(self, words: List[str]) -> str:
        graph = defaultdict(set)
        indegree = {c: 0 for w in words for c in w}
        for i in range(len(words) - 1):
            w1, w2 = words[i], words[i + 1]
            min_len = min(len(w1), len(w2))
            if len(w1) > len(w2) and w1[:min_len] == w2[:min_len]:
                return '' # invalid: prefix ordering
            for c1, c2 in zip(w1, w2):
                if c1 != c2:
                    if c2 not in graph[c1]:
                        graph[c1].add(c2)
                        indegree[c2] += 1
                    break
        queue = deque(c for c in indegree if indegree[c] == 0)
        result = []
        while queue:
            c = queue.popleft()
            result.append(c)
            for nb in graph[c]:
                indegree[nb] -= 1
                if indegree[nb] == 0:
                    queue.append(nb)
        return ''.join(result) if len(result) == len(indegree) else ''

# PHASE 5 — TEST & DEBUG
# ["wrt","wrf","er","ett","rftt"]: edges: t→f, w→e, r→t, e→r. indegree:
# f:1,e:1,t:1,r:1,w:0.
# BFS from w: w→e→r→t→f. result='wertf' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(C) where C = total characters. Space O(1) – at most 26 letters.
# The prefix check catches the only invalid case that doesn't manifest as a graph
# cycle.
# Given more time I'd ask: return ALL valid orderings?
# Backtracking on the topological sort – collect all linearisations."

```

◆ Quick Review

Key Insights

- Build a graph where each unique character is a node.
- Compare adjacent words to determine the first different character, which gives the ordering (directed edge).
- Ensure all characters, even those without edges, are represented.
- Use a topological sort (using DFS or BFS) to determine a valid character ordering.
- Handle edge cases, such as words where a longer word appears before its prefix which is invalid.

Space & Time

Time Complexity: $O(N * L + V + E)$ where N is the number of words, L is the average length of each word, V is the number of unique characters, and E is the number of edges derived from adjacent word comparisons.

Space Complexity: $O(V + E)$ for storing the graph and additional data structures for topological sorting.

Solution

We use a graph-based approach to solve the problem. First, we initialize a graph (adjacency list) and in-degree dictionary for all unique characters. We then iterate through each pair of consecutive words, comparing them character by character to find the first pair that differs. This difference implies an order: the first character should come before the second character. We add a directed edge accordingly and update the in-degree for the target character. To get the final ordering, we perform a topological sort using Kahn's algorithm: Enqueue characters with in-degree zero. Remove characters from the queue and append them to our result. Decrement the in-degree of neighboring nodes. If a neighbor's in-degree becomes zero, enqueue it. If the result's length equals the number of unique nodes, we have a valid ordering; otherwise, a cycle exists, and we return an empty string.

DFS

□ #329 Longest Increasing Path in a Matrix **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Graph · Topological Sort ·
Memoization · Matrix

Lists: Grind 169

Problem

Given an $m \times n$ integers matrix, return the length of the longest increasing path in matrix. From each cell, you can either move in four directions: left, right, up, or down. You may not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).

Example 1:

9	9	4
6	6	8
2	1	1

Input: matrix = [[9,9,4],[6,6,8],[2,1,1]]

Output: 4

Explanation: The longest increasing path is [1, 2, 6, 9].

Example 2:

3 →	4 →	5 ↓
3	2	6
2	2	1

Input: matrix = [[3,4,5],[3,2,6],[2,2,1]]

Output: 4

Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.

Example 3:

Input: matrix = [[1]]

Output: 1

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 200$

● $0 \leq \text{matrix}[i][j] \leq 231 - 1$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Longest strictly increasing path (4-directional). Return path length. Right?
Values can repeat but path must be strictly increasing."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
DFS from every cell with memo: longest increasing path starting at (r,c).
Only move to strictly larger neighbors; memo avoids recomputing subpaths.
Without memo, pure DFS retries paths exponentially.
Time: $O(m * n)$ with memo. Space: $O(m * n)$ memo + stack.
Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (DFS + memoization)

"DFS from each cell. $\text{dp}[r][c]$ = longest path starting at (r,c).
Only move to neighbours with strictly greater value – no cycles possible → no visited set needed.
Memoize results: each cell computed once → $O(m*n)$."

PHASE 4 — CLEAN CODE

```
class _329_LongestIncPathMatrix:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        m, n = len(matrix), len(matrix[0])
        memo = {}
        def dfs(r, c):
            if (r, c) in memo: return memo[(r, c)]
            best = 1
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and matrix[nr][nc] > matrix[r][c]:
                    best = max(best, dfs(nr, nc) + 1)
            memo[(r, c)] = best
            return best
        return max(dfs(r, c) for r in range(m) for c in range(n))
```

PHASE 5 — TEST & DEBUG

$[[9,9,4],[6,6,8],[2,1,1]]$: longest path starting at (2,1)=1: $1 \rightarrow 2 \rightarrow 6 \rightarrow 9 = 4$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ for memo. No visited set needed – strict increase prevents cycles.

Topological sort alternative: build DAG (only edges to larger neighbours), count levels.

Same $O(m*n)$ but iterative.

Given more time I'd ask: longest path with values differing by at most 1?

Need a visited set now (can revisit same value) – different problem."

◆ Quick Review

Key Insights

- The problem can be modeled as a graph where each cell is a node connected to its neighbors if the neighbor's value is greater.
- A Depth First Search (DFS) with memoization is effective to avoid recalculations.
- Dynamic Programming (DP) is used to cache the longest path found from each cell.
- Since each cell can be a start point, compute DFS for every cell and update the global maximum.

Space & Time

Time Complexity: $O(m * n)$ – each cell is computed once due to memoization.

Space Complexity: $O(m * n)$ – space is used for the memoization cache and recursion stack.

Solution

We solve the problem using a DFS approach combined with memoization to keep track of already computed longest paths from each cell. For each cell in the matrix, a DFS is initiated to explore all four valid directions (up, down, left, right). If a neighboring cell has a larger value, the DFS recurses from that neighbor. The result for each cell is cached in a 2D memoization table, ensuring that each cell is processed only once. After processing all cells, the maximum value in the memo table represents the longest increasing path in the entire matrix.

DFS

□ #685 Redundant Connection II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Union Find · Graph
Lists: Denny Zhang

Problem

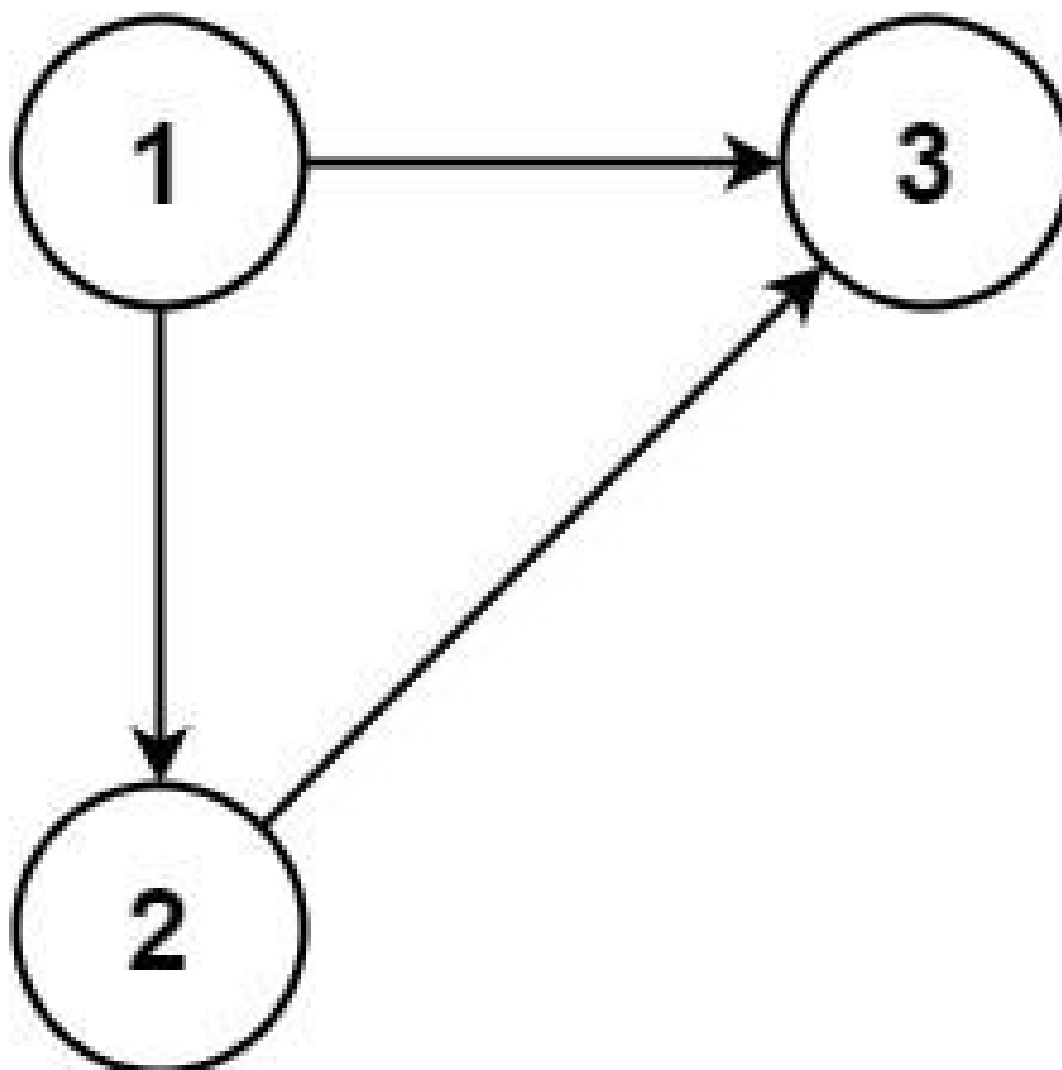
In this problem, a rooted tree is a directed graph such that, there is exactly one node (the root) for which all other nodes are descendants of this node, plus every node has exactly one parent, except for the root node which has no parents.

The given input is a directed graph that started as a rooted tree with n nodes (with distinct values from 1 to n), with one additional directed edge added. The added edge has two different vertices chosen from 1 to n , and was not an edge that already existed.

The resulting graph is given as a 2D-array of edges. Each element of edges is a pair $[u_i, v_i]$ that represents a directed edge connecting nodes u_i and v_i , where u_i is a parent of child v_i .

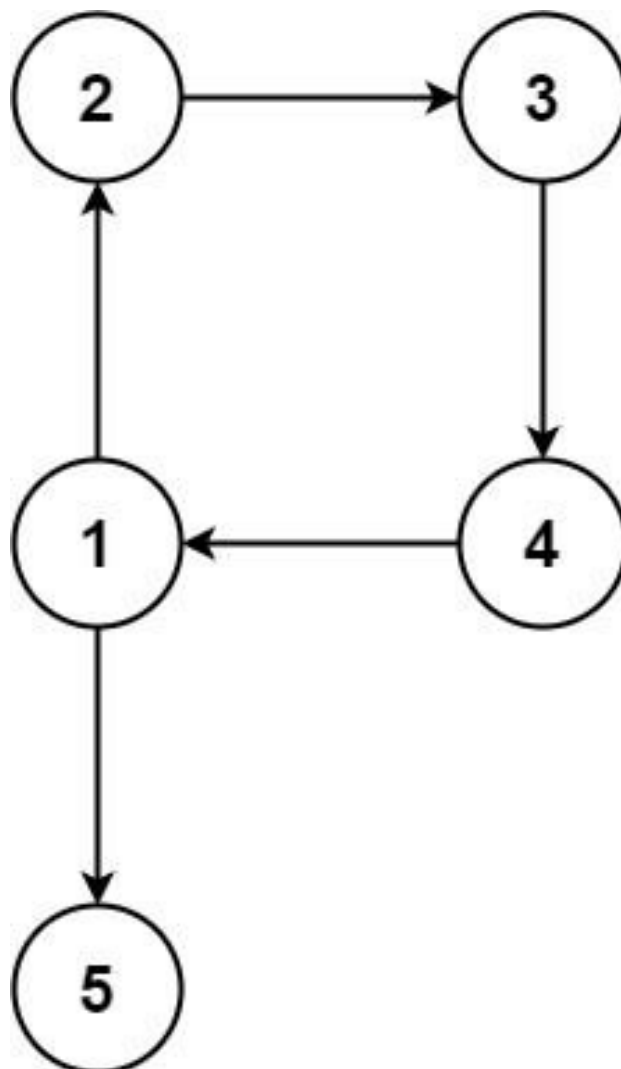
Return an edge that can be removed so that the resulting graph is a rooted tree of n nodes. If there are multiple answers, return the answer that occurs last in the given 2D-array.

Example 1:



Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]

Example 2:



Input: edges = [[1,2],[2,3],[3,4],[4,1],[1,5]]
Output: [4,1]

Constraints:

- $n == \text{edges.length}$
- $3 \leq n \leq 1000$
- $\text{edges}[i].\text{length} == 2$
- $1 \leq u_i, v_i \leq n$
- $u_i \neq v_i$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Directed graph: rooted tree + one extra edge. Find the edge to remove.
 # If multiple answers, return the last one in input. Right?
 # Extra edge causes: (1) a node with two parents, OR (2) a cycle, OR both."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Build indegree + parent map from edges. Node with indegree 2 is the duplicate child candidate.
 # Detect cycle with DFS or union-find on the graph minus one suspicious edge.
 # Try removing each duplicate edge until graph becomes a valid tree.
 # Time: $O(n)$ with careful case analysis. Space: $O(n)$.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (three-case analysis + Union-Find)

"Track in-degree: if any node has two parents, record both candidate edges.
 # Then run Union-Find ignoring the second parent candidate.
 # Case 1 (two parents, no cycle in remaining): remove second candidate.
 # Case 2 (two parents, cycle in remaining): remove first candidate.
 # Case 3 (no node with two parents): cycle exists → last edge forming cycle."

PHASE 4 — CLEAN CODE

```
class _685_RedundantConnectionII:
    def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]:
        n = len(edges)
        parent = list(range(n + 1))
        cand1 = cand2 = None

        # Find if any node has two parents
        parent_map = {}
        for u, v in edges:
            if v in parent_map:
                cand1 = parent_map[v] # first edge giving v a parent
                cand2 = [u, v] # second edge giving v another parent
                break
            parent_map[v] = [u, v]

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        def union(u, v) -> bool:
            pu, pv = find(u), find(v)
```



```

        if pu == pv: return False
        parent[pu] = pv
        return True
parent = list(range(n + 1))
for u, v in edges:
    if [u, v] == cand2: # skip the second candidate
        continue
    if not union(u, v):
        return cand1 if cand1 else [u, v]
return cand2

```

PHASE 5 — TEST & DEBUG

```

# [[1,2],[1,3],[2,3]]: node 3 has two parents (1 and 2). cand1=[1,3], cand2=[2,3].
# UF ignoring cand2=[2,3]: union(1,2) ok, union(1,3) ok. No cycle → return
cand2=[2,3] ✓
# [[1,2],[2,3],[3,4],[4,1],[1,5]]: no node with two parents. UF finds cycle [4,1] →
return [4,1] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n * \alpha(n))$ . Space  $O(n)$ . Three cases arise from two root causes: double
parent and cycle.
# The Union-Find handles both: cycle detection when cand2 is skipped tells us which
candidate to remove.
# Given more time I'd ask: undirected version?
# That's Redundant Connection (#684) – standard Union-Find, return first edge that
creates a cycle."

```

◆ Quick Review

Key Insights

- The extra edge introduces one of two issues:
- A node has two parents.
- A cycle exists in the graph.
- First, scan through edges to detect if any node gets two parents. If found, there are two candidate edges.
- Use the Union-Find data structure (Disjoint Set Union) to detect cycles.

- Depending on whether a conflict (two parents) exists and/or a cycle is detected, decide which edge to remove:
- If a node has two parents, test by ignoring the second candidate edge during union–find. If no cycle forms, the second candidate is removable; otherwise, remove the first candidate.
- If no conflict exists, simply remove the edge that causes a cycle.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution consists of two main steps: Traverse the list of edges to detect whether any node has two parents. If such a node is found, mark the two candidate edges (candidate1: the earlier edge, candidate2: the later edge). Use the Union–Find algorithm to iterate over the edges: If there is a conflict (i.e., a node with two parents), skip candidate2 initially. While processing edges with Union–Find, check if adding an edge would

create a cycle. Finally, based on the presence of a cycle and the two-parent conflict, decide which candidate edge to return. Key Data Structures: An array (or hash map) to track the parent of each node. The Union-Find (Disjoint Set Union) structure to union sets and detect cycles.

DFS

□ #1036 Escape a Large Maze

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · DFS · BFS

Lists: Denny Zhang

Problem

There is a 1 million by 1 million grid on an XY-plane, and the coordinates of each grid square are (x, y) .

We start at the source = $[sx, sy]$ square and want to reach the target = $[tx, ty]$ square. There is also an array of blocked squares, where each $blocked[i] = [xi, yi]$ represents a blocked square with coordinates (xi, yi) .

Each move, we can walk one square north, east, south, or west if the square is not in the array of blocked squares. We are also not allowed to walk outside of the grid.

Return true if and only if it is possible to reach the target square from the source square through a sequence of valid moves.

Example 1:

```
Input: blocked = [[0,1],[1,0]], source = [0,0], target = [0,2]
Output: false
Explanation: The target square is inaccessible starting from the source square
because we cannot move.
We cannot move north or east because those squares are blocked.
We cannot move south or west because we cannot go outside of the grid.
```

Example 2:

```
Input: blocked = [], source = [0,0], target = [999999,999999]
Output: true
Explanation: Because there are no blocked cells, it is possible to reach the
target square.
```

Constraints:

- $0 \leq \text{blocked.length} \leq 200$
- $\text{blocked}[i].\text{length} == 2$
- $0 \leq x_i, y_i < 106$
- $\text{source.length} == \text{target.length} == 2$
- $0 \leq s_x, s_y, t_x, t_y < 106$
- $\text{source} \neq \text{target}$
- It is guaranteed that source and target are not blocked.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"1M×1M grid. Up to 200 blocked cells. Can source reach target? Right?"

```
# With at most 200 blocked cells in a semicircle, max enclosed area  $\approx 200^2/2 = 20,000$  cells.
# So if BFS from source visits  $> 20,000$  cells without finding target, source is NOT enclosed."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# BFS from start cell; at each step try all 4 directions.
# Track visited cells; if we revisit within  $\leq 500$  moves, we are in a loop ? false.
# If we escape bounds within 500 steps per problem rule ? true.
# Time:  $O(m * n * 500)$  worst case. Space:  $O(m * n)$  visited.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BFS with cell limit)

```
# "Run limited BFS from BOTH source and target.
# If BFS visits  $> \text{max\_cells}$  ( $= n*(n+1)/2$  for  $n=\text{len}(\text{blocked})$ )  $\rightarrow$  not enclosed  $\rightarrow$  True from that side.
# If BFS terminates (all cells explored) without reaching other endpoint  $\rightarrow$  enclosed  $\rightarrow$  False."
```

PHASE 4 — CLEAN CODE

```
class _1036_EscapeLargeMaze:
    def isEscapePossible(self, blocked: List[List[int]], source: List[int], target: List[int]) -> bool:
        if not blocked:
            return True
        n = len(blocked)
        max_cells = n * (n + 1) // 2
        blocked_set = {(r, c) for r, c in blocked}

        def bfs(start, end):
            queue = deque([tuple(start)])
            visited = {tuple(start)}
            while queue:
                r, c = queue.popleft()
                if [r, c] == end:
                    return True
                if len(visited) > max_cells:
                    return True # too many cells explored  $\rightarrow$  not enclosed
                for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < 10**6 and 0 <= nc < 10**6 and (nr,nc) not in blocked_set and (nr,nc) not in visited:
                        visited.add((nr, nc))
                        queue.append((nr, nc))
            return False

        return bfs(source, target) and bfs(target, source)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^2)$  per BFS (at most  $n*(n+1)/2$  cells). Space  $O(n^2)$ .  $n \leq 200 \rightarrow$  fast.
# The dual-BFS is essential: source might not be enclosed but target might be.
```

Given more time I'd ask: what if blocked cells can be added dynamically?
Re-run BFS on each addition – or maintain Union-Find of open cells."

◆ Quick Review

Key Insights

- The maximum number of blocked cells is 200, which limits the size of a potential completely enclosed region.
- Even if the target is unreachable by direct search, if the BFS (or DFS) explores more than a calculated threshold (based on the blocked cells count), then the region is not fully enclosed.
- Perform two BFS explorations: one starting from the source and one from the target. If neither search is trapped within a small region, then an escape/path exists.
- Early exit in BFS if the target is reached or if the number of visited nodes exceeds the maximum possible enclosed area.

Space & Time

Time Complexity: $O(B^2)$ in the worst case, where B is the number of blocked cells (with $B \leq 200$).

Space Complexity: $O(B^2)$ due to the storage of visited cells in the worst-case scenario.

Solution

The solution uses the Breadth-First Search (BFS) algorithm. A key observation here is that the blocked cells can only enclose a limited area. We compute a threshold (maxSteps) based on the number of blocked cells $(n \cdot (n-1)/2)$ to determine if the search is confined. The BFS from the source (and similarly from the target) stops in two cases: if the target is reached or if the visited area exceeds the threshold, indicating that the region is not fully enclosed by blocked cells. Running BFS from both the source and target ensures that neither is trapped inside a small banned region.

DFS

□ #1192 Critical Connections in a Network **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · Graph · Biconnected Component
Lists: Denny Zhang

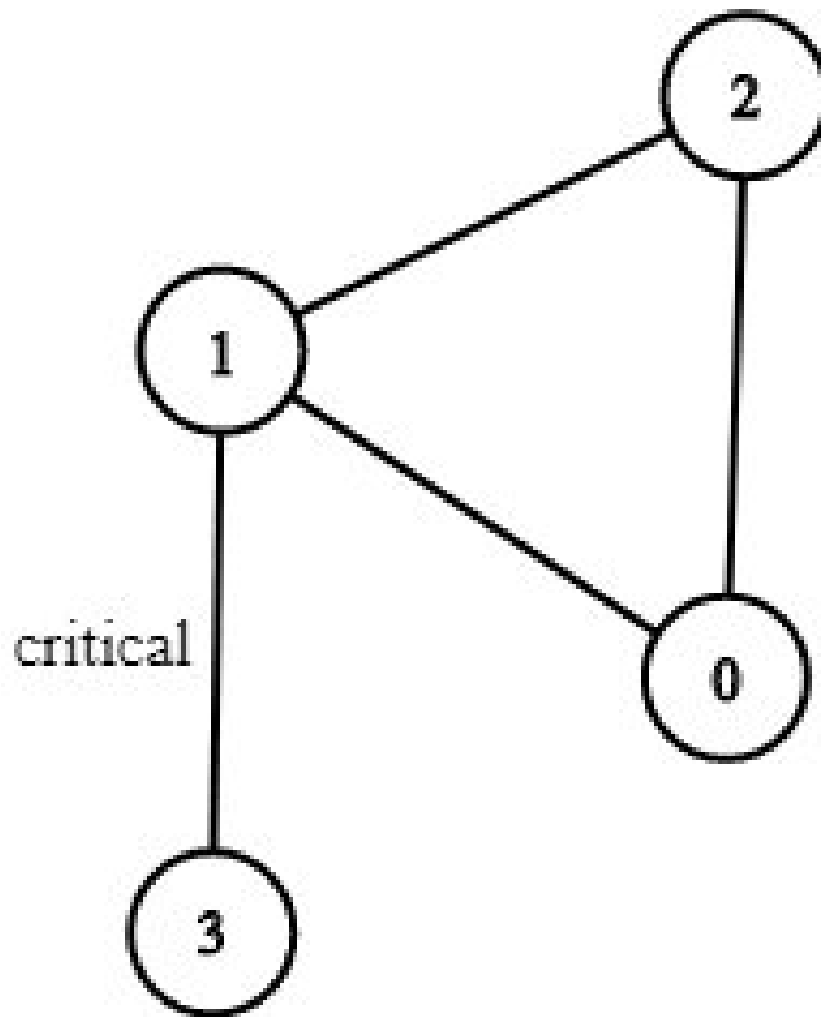
Problem

There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server connections forming a network where $\text{connections}[i] = [a_i, b_i]$ represents a connection between servers a_i and b_i . Any server can reach other servers directly or indirectly through the network.

A critical connection is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.

Example 1:



Input: $n = 4$, $\text{connections} = [[0,1],[1,2],[2,0],[1,3]]$

Output: $[[1,3]]$

Explanation: $[[3,1]]$ is also accepted.

Example 2:

Input: $n = 2$, $\text{connections} = [[0,1]]$

Output: $[[0,1]]$

Constraints:

- $2 \leq n \leq 105$
- $n - 1 \leq \text{connections.length} \leq 105$
- $0 \leq a_i, b_i \leq n - 1$

- $a_i \neq b_i$
- There are no repeated connections.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Critical connection (bridge) = edge whose removal disconnects the graph. Right?
# Use Tarjan's bridge-finding algorithm."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Find bridges with Tarjan low-link DFS: edge (u,v) is critical if low[v] > disc[u].
# One DFS pass tracks discovery time and lowest reachable.
# Naive alternative: remove each edge and test connectivity - O(E * (V+E)).
# Time: O(V + E) with Tarjan. Space: O(V + E).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (Tarjan's DFS)

```
# "DFS assigning disc[u]=discovery time and low[u]=earliest discovery reachable.
# An edge (u,v) is a bridge if low[v] > disc[u] - meaning v cannot reach u's subtree
without (u,v)."
```

PHASE 4 — CLEAN CODE

```
class _1192_CriticalConnections:
    def criticalConnections(self, n: int, connections: List[List[int]]) ->
List[List[int]]:
        graph = defaultdict(list)
        for u, v in connections:
            graph[u].append(v)
            graph[v].append(u)
        disc = [-1] * n
        low = [-1] * n
        timer = [0]
        result = []
        def dfs(node, parent):
            disc[node] = low[node] = timer[0]
            timer[0] += 1
            for nb in graph[node]:
                if nb == parent: continue
                if disc[nb] == -1:
```

```

        dfs(nb, node)
        low[node] = min(low[node], low[nb])
        if low[nb] > disc[node]:
            result.append([node, nb])
    else:
        low[node] = min(low[node], disc[nb])
dfs(0, -1)
return result

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. Tarjan's bridge algorithm in one DFS pass.

$\text{low}[v] > \text{disc}[u]$ means v 's subtree can't 'back-connect' to u 's subtree – edge is a bridge.

Given more time I'd ask: articulation points instead of bridges?

Similar but: u is an articulation point if any child v has $\text{low}[v] \geq \text{disc}[u]$ (and u is not root)."

◆ Quick Review

Key Insights

- The problem is about identifying bridges in an undirected graph.
- Use Depth-First Search (DFS) to explore the graph.
- During DFS, track discovery times and low-link values for each node.
- For an edge (u, v) , if $\text{low}[v] > \text{disc}[u]$ then (u, v) is a critical connection.
- Tarjan's algorithm is well-suited for solving this problem in $O(n + m)$ time.

Space & Time

Time Complexity: $O(n + m)$ where n is the number of nodes and m is the number of connections.

Space Complexity: $O(n + m)$ for the graph representation and DFS recursion stack.

Solution

We solve this problem using a DFS-based approach (Tarjan's algorithm). First, we transform the list of connections into an adjacency list for efficient graph traversal. We then start a DFS from an arbitrary node (node 0, since the graph is connected) while maintaining two arrays: one for discovery times (`disc`) and one for low-link values (`low`). The discovery time is the time when the node is first visited, and the low-link value is the smallest discovery time that can be reached from that node (including via back edges). For each edge (u, v) , after a DFS call on v , if $low[v] > disc[u]$ then the edge (u, v) is a bridge because v (and its descendants) cannot reach u or any of its ancestors without this edge. This edge is then recorded as a critical connection.

Graphs

Round 3 · High · Hard · 2 questions

Graphs

□ ★ #305 Number of Islands II ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Union Find · Matrix
Lists: ★ Premium | Premium 98

Problem

You are given an empty 2D binary grid `grid` of size $m \times n$. The grid represents a map where 0's represent water and 1's represent land. Initially, all the cells of grid are water cells (i.e., all the cells are 0's).

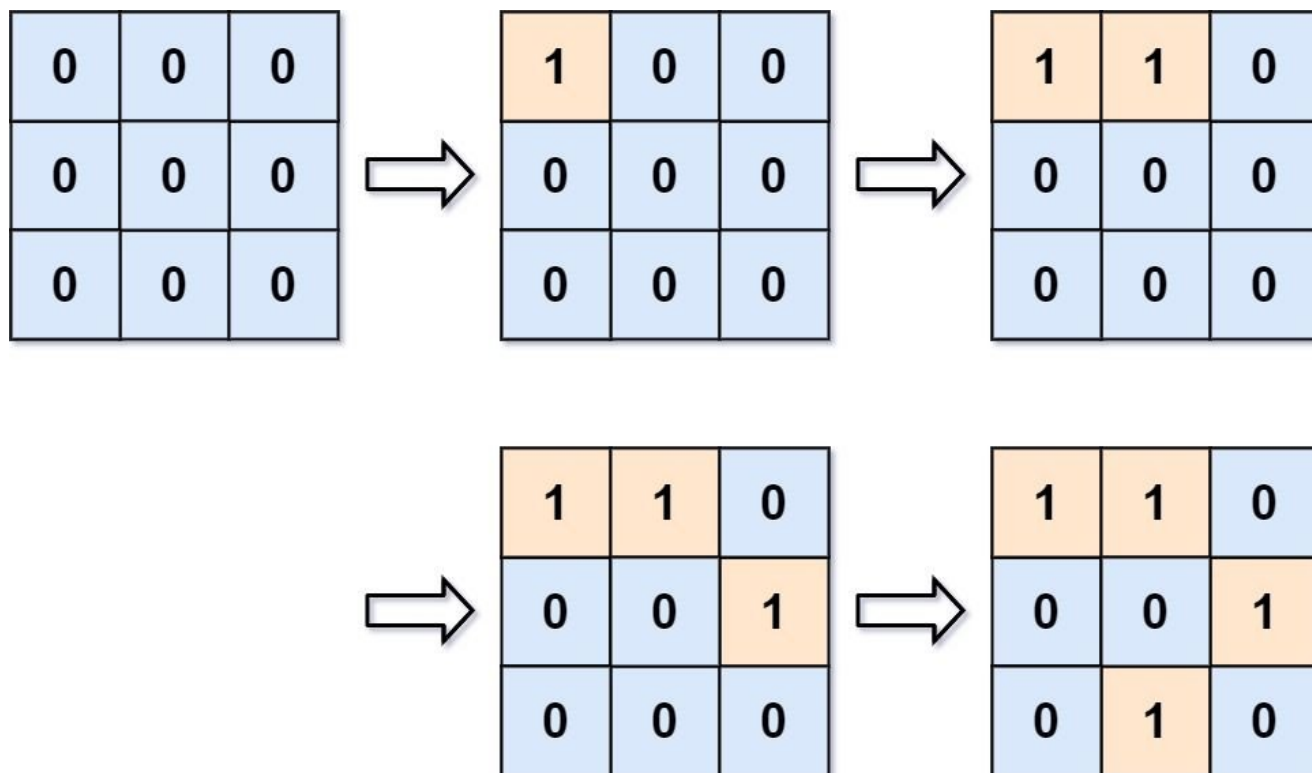
We may perform an add land operation which turns the water at position into a land. You are given an array `positions` where `positions[i] = [ri, ci]` is the position (ri, ci) at which we should operate the i th operation.

Return an array of integers `answer` where `answer[i]` is the number of islands after turning the cell (ri, ci) into a land.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the

grid are all surrounded by water.

Example 1:



Input: $m = 3$, $n = 3$, positions = $[[0,0],[0,1],[1,2],[2,1]]$

Output: [1,1,2,3]

Explanation:

Initially, the 2d grid is filled with water.

- Operation #1: addLand(0, 0) turns the water at grid[0][0] into a land. We have 1 island.

- Operation #2: addLand(0, 1) turns the water at grid[0][1] into a land. We still have 1 island.

- Operation #3: addLand(1, 2) turns the water at grid[1][2] into a land. We have 2 islands.

- Operation #4: addLand(2, 1) turns the water at grid[2][1] into a land. We have 3 islands.

Example 2:

Input: $m = 1$, $n = 1$, positions = $[[0,0]]$

Output: [1]

Constraints:

- $1 \leq m, n, \text{positions.length} \leq 104$
- $1 \leq m * n \leq 104$
- $\text{positions}[i].\text{length} == 2$
- $0 \leq r_i < m$
- $0 \leq c_i < n$

Follow up: Could you solve it in time complexity $O(k \log(mn))$, where $k == \text{positions.length}$?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Start with empty grid. Add land cells one at a time. After each add, return
island count.
#  $O(k \log mn)$  per operation. Right?"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# After each addLand, scan the whole grid and flood-fill count islands from scratch.
# k addLand calls each trigger full  $O(m*n)$  recount - too slow when k is large.
# Time:  $O(k * m * n)$ . Space:  $O(m * n)$  per flood.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (Union-Find with dynamic updates)

```
# "Each new land cell starts as its own island (count++).
# Check 4 neighbours: for each land neighbour, if different component, union and
count--."
```

PHASE 4 — CLEAN CODE

```
class _305_NumberIslandsII:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
        parent = {}
        count = [0]
        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
```

```

        x = parent[x]
    return x

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb: return
    parent[ra] = rb
    count[0] -= 1

result = []
for r, c in positions:
    if (r, c) in parent:
        result.append(count[0])
        continue
    parent[(r, c)] = (r, c)
    count[0] += 1
    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
        nr, nc = r + dr, c + dc
        if (nr, nc) in parent:
            union((r, c), (nr, nc))
    result.append(count[0])
return result

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(k * \alpha(mn)) \approx O(k)$. Space $O(k)$. Union-Find handles dynamic connectivity perfectly.

The duplicate position check avoids double-counting islands.

Given more time I'd ask: what if land cells can be removed too?

Split-Find is hard. Offline deletion via reverse addition is a common trick."

◆ Quick Review

Key Insights

- Use Union Find (Disjoint Set Union) to efficiently manage connected components (islands).
- Map 2D grid coordinates to a 1D index.
- When adding a new land, check its 4 neighbors (up, down, left, right) and perform unions if they are also lands.

- Avoid duplicate processing if the same cell is added multiple times.
- Maintain a counter for islands that is updated during each union operation.

Space & Time

Time Complexity: $O(K * \alpha(mn))$ where K is the number of operations and α is the inverse Ackermann function (almost constant time per union/find).

Space Complexity: $O(mn)$ for storing the union–find structure.

Solution

The solution relies on the Union Find data structure to dynamically merge islands. When a new land cell is added: If the cell is already land, record the current island count.

Otherwise, mark the cell as land and increment the island count. Check the four neighboring cells; if any neighbor is land, attempt to union the current cell with that neighbor. If a union actually happens (i.e., the two cells were in separate islands), decrement the island counter. Output the island count after each add–land operation. The key trick is

converting 2D indices to a 1D representation ($r * n + c$) and using path compression along with union by rank in the Union Find implementation for optimal performance.

Graphs

□ #1153 String Transforms Into Another String

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Union Find
Lists: Denny Zhang

Problem

Given two strings `str1` and `str2` of the same length, determine whether you can transform `str1` into `str2` by doing zero or more conversions.

In one conversion you can convert all occurrences of one character in `str1` to any other lowercase English character.

Return `true` if and only if you can transform `str1` into `str2`.

Example 1:

Input: `str1 = "aabcc", str2 = "ccdee"`

Output: `true`

Explanation: Convert 'c' to 'e' then 'b' to 'd' then 'a' to 'c'. Note that the order of conversions matter.

Example 2:

Input: str1 = "leetcode", str2 = "codeleet"

Output: false

Explanation: There is no way to transform str1 to str2.

Constraints:

- $1 \leq \text{str1.length} == \text{str2.length} \leq 104$
- str1 and str2 contain only lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Can str1 be converted to str2 by repeatedly mapping one character to another (all occurrences)?"

Conversions can be chained but must be consistent. Right?"

#

"If str1==str2 trivially True. If str2 uses all 26 letters → no 'buffer' character available → False."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Build a map from each character to its index in the first string.

Scan the second string left-to-right: next char must appear at a strictly higher index.

If any char is missing or out of order, transformation is impossible.

Time: $O(n)$. Space: $O(1)$ – fixed alphabet size 26.

Should I go ahead and optimize?"

```
class _1153_StringTransforms:
```

```
    def canConvert(self, str1: str, str2: str) -> bool:
```

```
        if str1 == str2:
```

```
            return True
```

```
        mapping = {}
```

```
        for c1, c2 in zip(str1, str2):
```

```
            if mapping.get(c1, c2) != c2:
```

```
                return False
```

```
            mapping[c1] = c2
```

```
        return len(set(str2)) < 26 # need at least one unused char as buffer
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(26). The len(set(str2)) < 26 check: if all 26 chars appear in str2,  
# every potential buffer letter is 'locked' to a mapping – cycles can't be broken.  
# Given more time I'd ask: what if conversions can only be applied once?  
# Topological sort on the conversion graph – acyclic → True."
```

◆ Quick Review

Key Insights

- Each character mapping from str1 to str2 must be consistent; one character in str1 always maps to the same character in str2.
- A bijective mapping is not necessary; multiple characters in str1 can map to the same character in str2.
- The order of conversions matters. When there is a cycle (e.g., mapping 'a'→'b' and 'b'→'a'), an intermediate unused character might be required to break the cycle.
- If str2 uses all 26 letters, no temporary character is available to break mapping cycles. In this case, transformation is possible only if str1 is already equal to str2.

Space & Time

Time Complexity: $O(n)$, where n is the length of the strings, because we traverse the strings to build and check the mapping.

Space Complexity: $O(1)$ or $O(26)$, which is constant space, to store the mapping for the 26 lowercase English letters.

Solution

The approach is as follows: If `str1` equals `str2`, return `true` immediately. Build a mapping from each character in `str1` to the corresponding character in `str2`. While creating the mapping, check that each character maps consistently. Count the number of distinct characters in `str2`. If this number is 26 and `str1` is not already equal to `str2`, it is impossible to perform the transformation because there is no spare character available as a temporary placeholder. If the mapping is consistent and there exists at least one character not used in `str2` (i.e., distinct characters in `str2` is less than 26), the transformation is possible. The important trick is identifying the need for a free character when the target mapping is “full” (i.e., covers all 26 letters). Without a free character, you cannot avoid cycles where a temporary intermediate is necessary.

BFS

Round 3 · High · Hard · 4 questions

BFS

□ #127 Word Ladder

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · BFS
Lists: Grind 169

Problem

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` $\rightarrow$ `s1` $\rightarrow$ `s2` $\rightarrow$... $\rightarrow$ `sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or 0 if no such sequence exists.

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList =  
["hot", "dot", "dog", "lot", "log", "cog"]
```

```
Output: 5
```

```
Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" ->  
"dog" -> cog", which is 5 words long.
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList =  
["hot", "dot", "dog", "lot", "log"]
```

```
Output: 0
```

```
Explanation: The endWord "cog" is not in wordList, therefore there is no valid  
transformation sequence.
```

Constraints:

- $1 \leq \text{beginWord.length} \leq 10$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 5000$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- beginWord , endWord , and $\text{wordList}[i]$ consist of lowercase English letters.
- $\text{beginWord} \neq \text{endWord}$
- All the words in wordList are unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Shortest word transformation from beginWord to endWord , changing one letter at a time.

Each intermediate word must be in wordList. Return total words in shortest path, or 0. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Build adjacency from word patterns (one-letter-apart neighbors).
 # BFS from beginWord layer by layer until endWord is reached; each layer = one transformation.
 # Visited set prevents revisiting words. Return BFS depth or 0 if unreachable.
 # Time: $O(N * L^2)$ where N = dictionary size, L = word length. Space: $O(N)$.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS with pattern matching)

"BFS on words. For each current word, try replacing each position with 26 letters.
 # If the result is in wordSet (and not visited), enqueue it.
 # Alternatively: build adjacency by pattern (*ot, h*t, ho*) → group words by pattern — $O(L^2 * n)$."

PHASE 4 — CLEAN CODE

```
class _127_WordLadder:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) ->
    int:
        word_set = set(wordList)
        if endWord not in word_set:
            return 0
        queue = deque([(beginWord, 1)])
        visited = {beginWord}
        while queue:
            word, steps = queue.popleft()
            for i in range(len(word)):
                for c in 'abcdefghijklmnopqrstuvwxyz':
                    next_word = word[:i] + c + word[i+1:]
                    if next_word == endWord:
                        return steps + 1
                    if next_word in word_set and next_word not in visited:
                        visited.add(next_word)
                        queue.append((next_word, steps + 1))
        return 0
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n * L * 26) = O(n * L)$. Space $O(n * L)$. Standard BFS shortest path.
 # Bidirectional BFS: search simultaneously from beginWord and endWord — roughly halves depth.
 # Given more time I'd ask: return all shortest paths?
 # That's Word Ladder II (#126) — BFS to build levels, then DFS to backtrack paths."

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to explore transformations level by level to guarantee the shortest path.
- Preprocess words by generating intermediate representations (e.g., replacing each letter with a wildcard) to quickly find neighbors.
- Use a visited set to avoid cycles.
- Each transformation counts as a level in the BFS, so the answer is the number of levels traversed.

Space & Time

Time Complexity: $O(N * M^2)$ where N is the number of words and M is the length of each word.

Space Complexity: $O(N * M)$ for the storage of intermediate mappings and the BFS queue.

Solution

We solve the problem using a BFS approach. First, we preprocess the wordList by creating a mapping of all possible intermediate states. For example, for the word "dog" and wildcard position, we generate "og", "d g", and "do*".

This helps us find all adjacent words (neighbors) that are only one letter different in constant time. We then initialize a queue with the beginWord and initiate the BFS. For each word dequeued, we explore all valid transformations using the precomputed intermediate mapping. If the endWord is reached during this process, we return the current level (transformation sequence length). Otherwise, we continue the BFS until all possibilities are exhausted. If no valid transformation sequence is found, we return 0.

BFS

□ ★ #317 Shortest Distance from All Buildings ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: ★ Premium | Premium 98

Problem

You are given an $m \times n$ grid grid of values 0, 1, or 2, where:

- each 0 marks an empty land that you can pass by freely,
- each 1 marks a building that you cannot pass through, and
- each 2 marks an obstacle that you cannot pass through.

You want to build a house on an empty land that reaches all buildings in the shortest total travel distance. You can only move up, down, left, and right.

Return the shortest travel distance for such a house. If it is not possible to build such a house according to the above rules, return -1.

The total travel distance is the sum of the distances between the houses of the friends and the meeting point.

Example 1:

1	0	2	0	1
0	0	0	0	0
0	0	1	0	0

Input: grid = [[1,0,2,0,1],[0,0,0,0,0],[0,0,1,0,0]]

Output: 7

Explanation: Given three buildings at (0,0), (0,4), (2,2), and an obstacle at (0,2).

The point (1,2) is an ideal empty land to build a house, as the total travel distance of $3+3+1=7$ is minimal.

So return 7.

Example 2:

Input: grid = [[1,0]]

Output: 1

Example 3:

Input: grid = [[1]]
Output: -1

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0, 1, or 2.
- There will be at least one building in the grid.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find empty land (0) minimising total BFS distance to all buildings (1).
Obstacles (2) can't be passed. Return -1 if no valid land. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
For each empty cell, BFS to sum distances to all buildings.
Track total distance per cell; answer is cell with minimum total (must reach all buildings).
Repeat BFS from every building - $O(B * m * n)$ where $B = \text{buildings}$.
Time: $O(B * m * n)$. Space: $O(m * n)$ BFS state.
Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS from each building, accumulate distances)

"BFS from each building cell. For each empty cell, accumulate distance and increment reach_count.
Only cells reachable from ALL buildings are candidates. Return min total distance."

PHASE 4 — CLEAN CODE

```
class _317_ShortestDistanceBuildings:
```

```

def shortestDistance(self, grid: List[List[int]]) -> int:
    m, n = len(grid), len(grid[0])
    dist = [[0] * n for _ in range(m)]
    reach = [[0] * n for _ in range(m)]
    buildings = sum(grid[r][c] == 1 for r in range(m) for c in range(n))
    for r in range(m):
        for c in range(n):
            if grid[r][c] != 1: continue
            queue = deque([(r, c, 0)])
            visited = {(r, c)}
            while queue:
                cr, cc, d = queue.popleft()
                for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                    nr, nc = cr + dr, cc + dc
                    if 0 <= nr < m and 0 <= nc < n and grid[nr][nc] == 0 and
(nr,nc) not in visited:
                        visited.add((nr, nc))
                        dist[nr][nc] += d + 1
                        reach[nr][nc] += 1
                        queue.append((nr, nc, d + 1))

    return min(
        (dist[r][c] for r in range(m) for c in range(n) if grid[r][c] == 0 and
reach[r][c] == buildings),
        default=-1
    )

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(buildings * m * n). Space O(m*n). BFS from each building accumulates into
dist[]."
# reach[] ensures we only pick land reachable from ALL buildings.
# Given more time I'd ask: what if buildings can move? Re-run BFS on each query."

```

◆ Quick Review

Key Insights

- We need to compute the Manhattan distance from each building (cell value 1) to all empty lands (cell value 0) by performing a BFS.
- Maintain two matrices: one for cumulative total distances and another for the count of buildings that reached a particular empty cell.

- Ensure that only those empty cells reachable by every building are considered.
- The ideal cell will have a reach count equal to the total number of buildings.
- If multiple buildings cannot all reach any specific empty cell, then return -1 .

Space & Time

Time Complexity: $O(k * m * n)$ where k is the number of buildings and m, n are the grid dimensions.

Space Complexity: $O(m * n)$ due to the auxiliary matrices and visited state tracking used in BFS.

Solution

We first iterate through the grid to locate all buildings while counting them. For each building, we perform a BFS to compute the shortest distance to each reachable empty cell. During the BFS, we update the cumulative distance for that cell and record that it was reached by one additional building. After processing all the buildings, we scan the grid to find the empty cell that is reachable from all buildings and has the minimum cumulative distance. If no such cell exists, we return -1 .

BFS

□ #773 Sliding Puzzle

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: Denny Zhang

Problem

On an 2×3 board, there are five tiles labeled from 1 to 5, and an empty square represented by 0. A move consists of choosing 0 and a 4-directionally adjacent number and swapping it.

The state of the board is solved if and only if the board is $[[1,2,3],[4,5,0]]$.

Given the puzzle board board, return the least number of moves required so that the state of the board is solved. If it is impossible for the state of the board to be solved, return -1 .

Example 1:

1	2	3
4		5

Input: board = [[1,2,3],[4,0,5]]

Output: 1

Explanation: Swap the 0 and the 5 in one move.

Example 2:

1	2	3
5	4	

Input: board = [[1,2,3],[5,4,0]]

Output: -1

Explanation: No number of moves will make the board solved.

Example 3:

4	1	2
5		3

Input: board = [[4,1,2],[5,0,3]]

Output: 5

Explanation: 5 is the smallest number of moves that solves the board.

An example path:

After move 0: [[4,1,2],[5,0,3]]

After move 1: [[4,1,2],[0,5,3]]

After move 2: [[0,1,2],[4,5,3]]

After move 3: [[1,0,2],[4,5,3]]

Constraints:

- board.length == 2
- board[i].length == 3
- $0 \leq \text{board}[i][j] \leq 5$
- Each value board[i][j] is unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "2x3 board. Swap 0 with adjacent tile. Minimum moves to reach [[1,2,3],[4,5,0]].
Right?
# Return -1 if unsolvable."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# BFS on board states: encode 2x3 board as tuple string; enqueue all legal swaps of
'0'.
# First time we reach target state, return move count.
# State space is at most 6! = 720 permutations.
# Time: O(6!). Space: O(6!) visited states.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BFS on serialised board state)

```
# "Represent board as a 6-char string. BFS on states.
# Precompute which positions in the flat string can swap with position i (adjacency
map).
# Goal state: '123450'."
```

PHASE 4 — CLEAN CODE

```
class _773_SlidingPuzzle:
    def slidingPuzzle(self, board: List[List[int]]) -> int:
        GOAL = '123450'
        NEIGHBORS = {0:[1,3], 1:[0,2,4], 2:[1,5], 3:[0,4], 4:[1,3,5], 5:[2,4]}
        start = ''.join(str(board[r][c]) for r in range(2) for c in range(3))
        queue = deque([(start, 0)])
        visited = {start}
        while queue:
            state, moves = queue.popleft()
            if state == GOAL:
                return moves
            z = state.index('0')
            for nb in NEIGHBORS[z]:
                new_state = list(state)
                new_state[z], new_state[nb] = new_state[nb], new_state[z]
                ns = ''.join(new_state)
                if ns not in visited:
                    visited.add(ns)
                    queue.append((ns, moves + 1))
        return -1
```

PHASE 5 — TEST & DEBUG

Round 3 · High · Hard

p.1036


```
# [[1,2,3],[4,0,5]] → '123405'. Swap 0 at pos 4 with 5 at pos 5 → '123450' = GOAL → 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "States = 6! / 2 = 360 reachable states (half of 6! are unsolvable). BFS on 360 states.
```

```
# The hardcoded NEIGHBORS map is the key – it captures the 2D adjacency in 1D string indexing.
```

```
# Given more time I'd ask: generalise to n×m board?
```

```
# Same BFS approach – NEIGHBORS changes, state space = (n*m)! / 2."
```

◆ Quick Review

Key Insights

- Represent the board as a string (e.g. "123450") to simplify state comparison and manipulation.
- Use Breadth First Search (BFS) to explore all possible moves since the puzzle has a small fixed state space ($6! = 720$ possible states).
- Precompute the valid neighbor indices for each board position to efficiently generate new states.
- Track visited states to avoid loops and redundant work.

Space & Time

Time Complexity: $O(1)$ in practice, since there are at most 720 states ($6!$ permutations).

Space Complexity: $O(1)$ for the same reason, as the state space is fixed and limited.

Solution

We solve the problem using a Breadth First Search (BFS) approach. The board configuration is flattened into a string to represent states. A mapping from each index to its possible neighbors (i.e., positions to which 0 can move) is precomputed. BFS is then used to explore every valid move from the initial state while marking states as visited to avoid cycles. When the goal state ("123450") is reached, we return the number of moves taken. If BFS completes without reaching the goal, the puzzle is unsolvable and we return -1 .

BFS

□ #815 Bus Routes

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · BFS
Lists: Grind 169

Problem

You are given an array `routes` representing bus routes where `routes[i]` is a bus route that the i th bus repeats forever.

- For example, if `routes[0] = [1, 5, 7]`, this means that the 0th bus travels in the sequence $1 \rightarrow 5 \rightarrow 7 \rightarrow 1 \rightarrow 5 \rightarrow 7 \rightarrow 1 \rightarrow \dots$ forever.

You will start at the bus stop `source` (You are not on any bus initially), and you want to go to the bus stop `target`. You can travel between bus stops by buses only.

Return the least number of buses you must take to travel from `source` to `target`. Return -1 if it is not possible.

Example 1:

Input: routes = [[1,2,7],[3,6,7]], source = 1, target = 6

Output: 2

Explanation: The best strategy is take the first bus to the bus stop 7, then take the second bus to the bus stop 6.

Example 2:

Input: routes = [[7,12],[4,5,15],[6],[15,19],[9,12,13]], source = 15, target = 12

Output: -1

Constraints:

- $1 \leq \text{routes.length} \leq 500$.
- $1 \leq \text{routes}[i].\text{length} \leq 105$
- All the values of routes[i] are unique.
- $\text{sum}(\text{routes}[i].\text{length}) \leq 105$
- $0 \leq \text{routes}[i][j] < 106$
- $0 \leq \text{source}, \text{target} < 106$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Minimum number of BUSES (not stops) to travel from source to target. Right?

Each bus covers its entire route – stepping onto a bus travels all its stops."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build stop→routes map. BFS from source stop; each hop can board any route containing current stop.

Track min buses to reach target; mark routes visited to avoid reboarding.

Time: $O(R * S)$ routes and stops. Space: $O(R + S)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS on routes, not stops)

```
# "Build stop→routes map. BFS where each 'node' is a bus route.
# Start: all routes containing source. Each step: board a new route (adjacent if
# sharing a stop).
# Count route changes (buses taken). Avoid revisiting routes and stops."
```

PHASE 4 — CLEAN CODE

```
class _815_BusRoutes:
    def numBusesToDestination(self, routes: List[List[int]], source: int, target:
int) -> int:
        if source == target:
            return 0
        stop_to_routes: defaultdict = defaultdict(set)
        for i, route in enumerate(routes):
            for stop in route:
                stop_to_routes[stop].add(i)
        visited_stops = {source}
        visited_routes = set()
        queue = deque([(source, 0)]) # (stop, buses_taken)
        while queue:
            stop, buses = queue.popleft()
            for route_id in stop_to_routes[stop]:
                if route_id in visited_routes: continue
                visited_routes.add(route_id)
                for next_stop in routes[route_id]:
                    if next_stop == target:
                        return buses + 1
                    if next_stop not in visited_stops:
                        visited_stops.add(next_stop)
                        queue.append((next_stop, buses + 1))
        return -1
```

PHASE 5 — TEST & DEBUG

```
# routes=[[1,2,7],[3,6,7]], source=1, target=6:
# queue=[(1,0)]. route 0 for stop 1: stops 2,7. Not target. queue=[(2,1),(7,1)].
# stop 7: route 1 for stop 7: stops 3,6. 6==target → return 1+1=2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(sum of route lengths). Space O(same). BFS on routes, not stops – the key
insight.
# Visiting stops prevents re-enqueuing; visiting routes prevents re-processing
entire routes.
# Given more time I'd ask: find all paths using at most k buses?
# Keep track of bus count per path – BFS naturally handles this."
```

◆ Quick Review

Key Insights

- Model the problem as a graph where nodes are bus routes.
- Two bus routes are connected if they share at least one common bus stop.
- Use a hash map to quickly look up bus routes available at any bus stop.
- Employ Breadth-First Search (BFS) on the bus routes graph to find the minimum number of bus transfers.
- Mark visited bus routes and bus stops to avoid redundant work.

Space & Time

Time Complexity: $O(N * L)$ in the worst-case scenario, where N is the number of bus routes and L is the average number of stops per route.

Space Complexity: $O(N * L)$ due to storage of the stop-to-buses mapping and BFS auxiliary data structures.

Solution

We begin by constructing a mapping from each bus stop to the list of buses passing through it. This allows quick retrieval of buses available at a given stop. Starting at the source

stop, initialize the BFS queue with all buses that can be boarded at that stop. Then, for each bus route in the queue, inspect each stop along that route. If a stop is the target, return the number of buses taken so far. Otherwise, for each unvisited stop, add all adjacent bus routes (buses that pass through this stop) to the BFS queue with an incremented bus count. Continue until the queue is empty, which means the target cannot be reached.

Round 4 · Mid · Easy

Round 4

Mid Priority · Easy

12 questions · 4 patterns

Linked List #21 #141 #206 #234 #706 #876
#1474

Stack #20 #232 #844

Trie #14

Greedy #409

Linked List

Round 4 · Mid · Easy · 7 questions

Linked List

□ #21 Merge Two Sorted Lists

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion

Lists: Grind 169

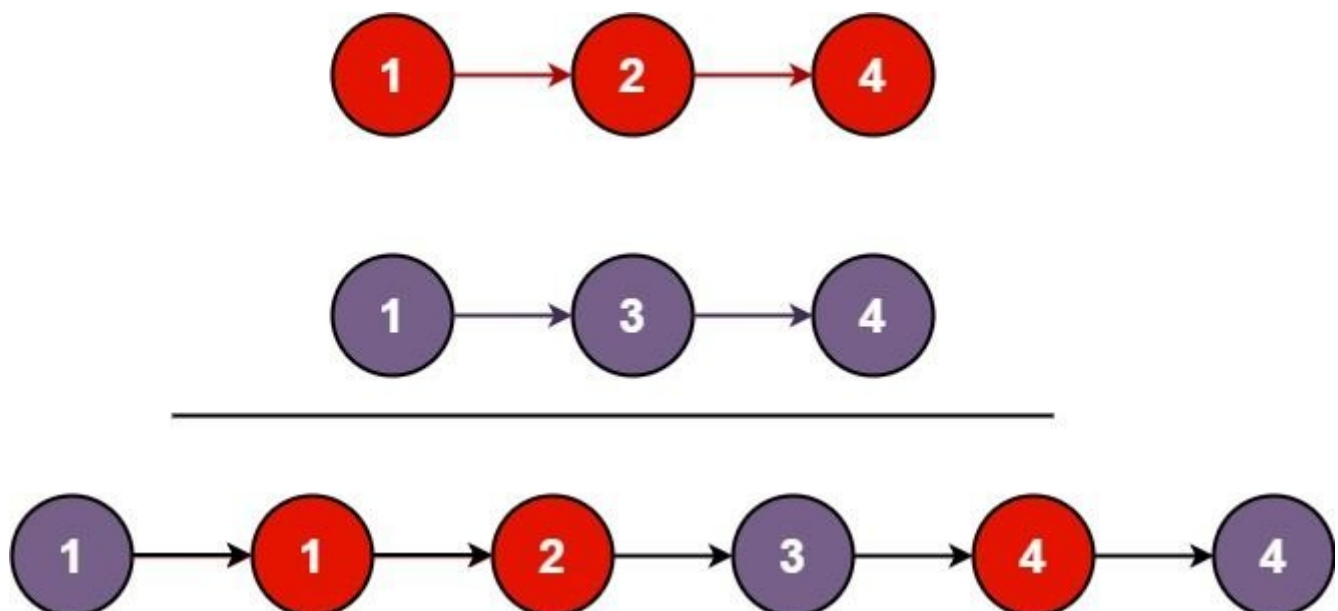
Problem

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Example 1:



```
Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]
```

Example 2:

```
Input: list1 = [], list2 = []
Output: []
```

Example 3:

```
Input: list1 = [], list2 = [0]
Output: [0]
```

Constraints:

- The number of nodes in both lists is in the range [0, 50].
- $-100 \leq \text{Node.val} \leq 100$
- Both list1 and list2 are sorted in non-decreasing order.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "We're given heads of two sorted linked lists. Merge them into one sorted
# list by splicing the original nodes – no new nodes. Return the new head. Right?"
#
# "list1=[1,2,4], list2=[1,3,4]:
# Compare 1 vs 1 → take list1's 1. 2 vs 1 → take list2's 1. 2 vs 3 → take 2.
# 4 vs 3 → take 3. 4 vs 4 → take list1's 4. Append list2's 4. → [1,1,2,3,4,4] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Collect all values, sort, rebuild a new list.
# Time:  $O((m+n) \log(m+n))$ . Space:  $O(m+n)$  for the values array."
```

```
class _21_MergeTwoSortedLists_Brute:
```

```
def mergeTwoLists(self, list1, list2):
    vals = []
    while list1: vals.append(list1.val); list1 = list1.next
    while list2: vals.append(list2.val); list2 = list2.next
    dummy = cur = ListNode(0)
    for v in sorted(vals):
        cur.next = ListNode(v); cur = cur.next
    return dummy.next
```

PHASE 3 — OPTIMIZE

"Both lists are already sorted – no sorting needed. Use two pointers.
 # A dummy head simplifies edge cases (empty lists, different lengths).
 # At each step attach the smaller node; advance that pointer.
 # When one list runs out, attach the rest of the other.
 #
 # Time: $O(m+n)$. Space: $O(1)$ – only pointer manipulation."

PHASE 4 — CLEAN CODE

```
class _21_MergeTwoSortedLists:
    def mergeTwoLists(self, list1, list2):
        dummy = cur = ListNode(0)
        while list1 and list2:
            if list1.val <= list2.val:
                cur.next = list1
                list1 = list1.next
            else:
                cur.next = list2
                list2 = list2.next
            cur = cur.next
        cur.next = list1 if list1 else list2
        return dummy.next
```

PHASE 5 — TEST & DEBUG

list1=[1,2,4], list2=[1,3,4]:
 # $1 <= 1 \rightarrow$ take l1(1), l1→2. $2 > 1 \rightarrow$ take l2(1), l2→3. $2 <= 3 \rightarrow$ take l1(2), l1→4.
 # $4 > 3 \rightarrow$ take l2(3), l2→4. $4 <= 4 \rightarrow$ take l1(4), l1=None.
 # Attach l2 remainder [4]. $\rightarrow$ [1,1,2,3,4,4] ✓
 #
 # list1=[], list2=[0]:
 # Loop skipped. cur.next = list2 $\rightarrow$ [0] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m+n)$: visit every node once. Space $O(1)$: reuses existing nodes.
 # Recursive solution is $O(m+n)$ time but $O(m+n)$ stack space.
 # Follow-up: merge k sorted lists (LC 23) – use a min-heap for $O(n \log k)$.
 # Follow-up: merge k sorted arrays – same heap approach."

◆ Quick Review

Key Insights

- Leverage the fact that both lists are sorted.
- Use two pointers (one for each list) to compare nodes sequentially.
- Create a dummy node to simplify edge cases and build the resulting list.
- Iterate until one list is exhausted, then append the remaining nodes from the other list.
- There is also a recursive approach that can simplify the logic but may use more stack space.

Space & Time

Time Complexity: $O(n + m)$, where n and m are the lengths of the two lists.

Space Complexity: $O(1)$ for the iterative solution ($O(n + m)$ stack space for the recursive solution).

Solution

We use an iterative approach with a dummy node to merge two sorted linked lists. Two pointers traverse list1 and list2; at each step, the node with the smaller value is appended to

our merged list. This process repeats until one of the lists is exhausted, after which we append the remaining nodes from the other list. This approach works in one pass over both lists, ensuring linear time complexity and constant extra space usage.

Linked List

□ #141 Linked List Cycle

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Two Pointers
Lists: Grind 169**

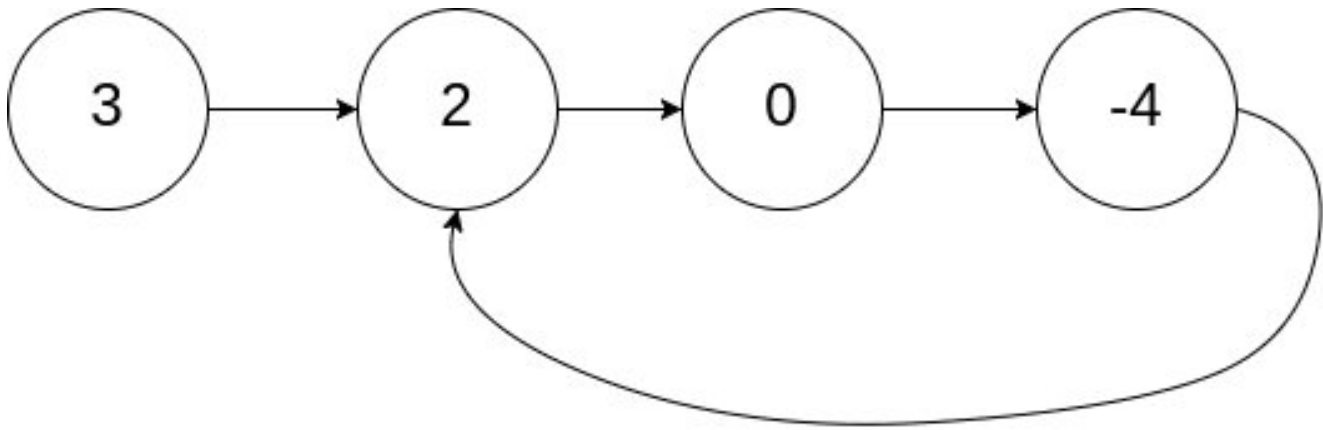
Problem

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer.

Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter. Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:

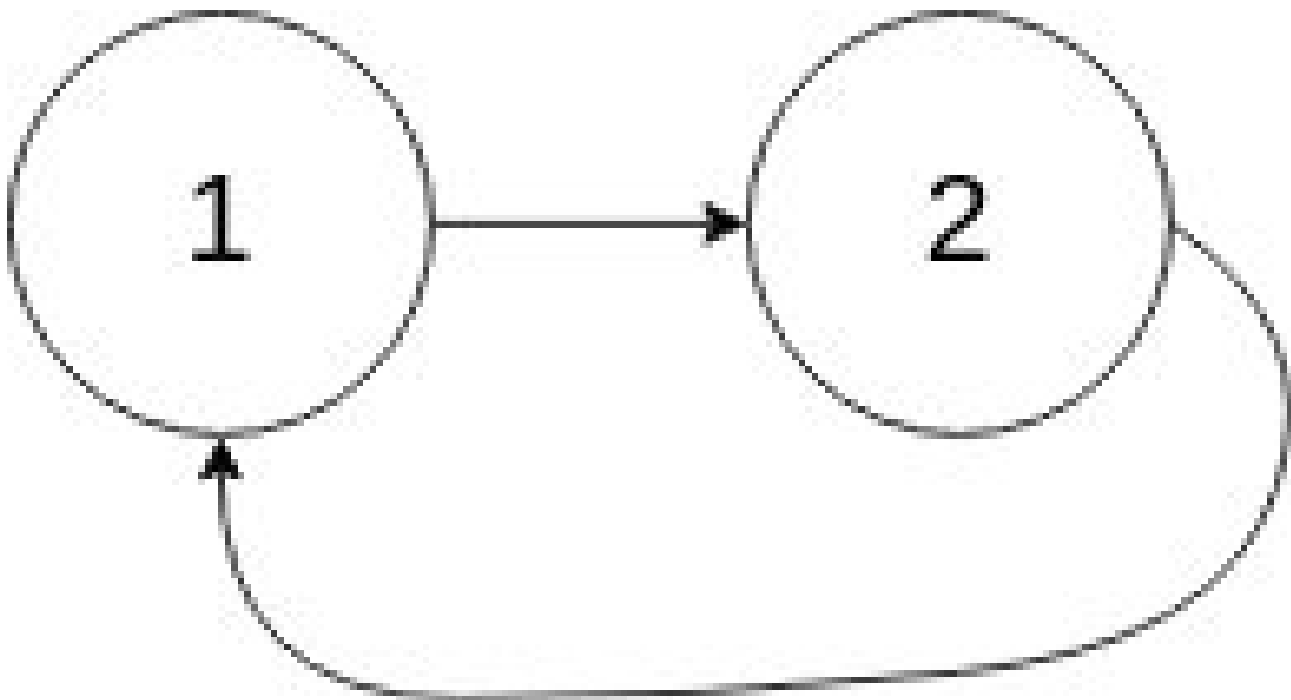


Input: head = [3,2,0,-4], pos = 1

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Example 2:

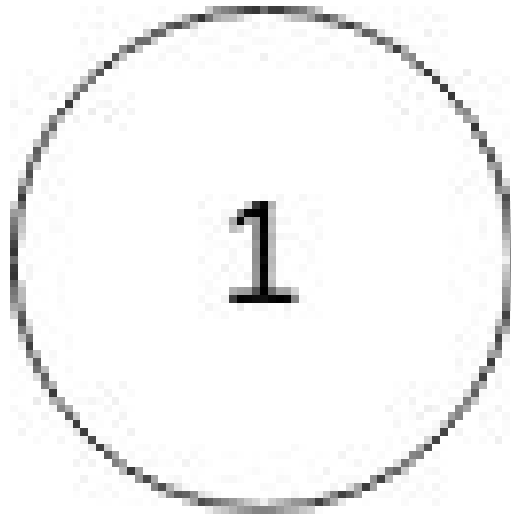


Input: head = [1,2], pos = 0

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.

Example 3:



Input: head = [1], pos = -1

Output: false

Explanation: There is no cycle in the linked list.

Constraints:

- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-105 \leq \text{Node.val} \leq 105$
- pos is -1 or a valid index in the linked-list.

Follow up: Can you solve it using $O(1)$ (i.e. constant) memory?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given the head of a linked list, return true if there's a cycle –
if any node's next eventually points back to a previous node. Right?"

#

"head=[3,2,0,-4], pos=1 (tail connects to index 1):

3→2→0→-4→2→... cycle exists → true ✓

```
# head=[1,2], pos=-1: 1→2→None → false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Store every visited node in a set. If we see a node twice, there's a cycle.
# If we reach None, there isn't.
# Time: O(n). Space: O(n) for the visited set."
```

```
class _141_LinkedListCycle_Brute:
    def hasCycle(self, head) -> bool:
        seen = set()
        while head:
            if id(head) in seen:
                return True
            seen.add(id(head))
            head = head.next
        return False
```

PHASE 3 — OPTIMIZE

```
# "Floyd's tortoise-and-hare: two pointers, slow moves 1 step, fast moves 2.
# If there's a cycle, fast will eventually lap slow and they'll meet.
# If fast reaches None, there's no cycle.
#
# Key insight: in a cycle, fast gains 1 step per iteration → guaranteed to catch slow.
# Time: O(n). Space: O(1) – no extra storage."
```

PHASE 4 — CLEAN CODE

```
class _141_LinkedListCycle:
    def hasCycle(self, head) -> bool:
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow is fast:
                return True
        return False
```

PHASE 5 — TEST & DEBUG

```
# [3,2,0,-4] cycle at index 1:
# slow=3,fast=3 → slow=2,fast=0 → slow=0,fast=2 → slow=-4,fast=-4 → equal → True ✓
#
# [1,2] no cycle:
# slow=1,fast=1 → slow=2,fast=None → fast is None → loop exits → False ✓
#
# [1] no cycle:
# fast.next is None → loop never enters → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): fast pointer covers the cycle in at most n steps after reaching it.
# Space O(1): only two pointers.
# Follow-up: find the START of the cycle (LC 142) – after they meet, reset slow
```

to head; advance both one step at a time until they meet again → cycle entry.
Follow-up: find cycle length – once they meet, keep fast moving and count steps."

◆ Quick Review

Key Insights

- Use two pointers (slow and fast) to traverse the list.
- The fast pointer moves two steps at a time and the slow pointer moves one step.
- If the fast pointer ever meets the slow pointer, a cycle exists.
- If the fast pointer reaches the end of the list (null), the list has no cycle.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the linked list.

Space Complexity: $O(1)$ as only two pointers are used.

Solution

We use the Floyd's Cycle Detection algorithm (Tortoise and Hare approach). Two pointers, slow and fast, are initialized at the head of the list. The slow pointer moves one step at a time while the fast pointer moves two steps. If the

linked list has a cycle, the fast pointer will eventually meet the slow pointer. If the fast pointer reaches the end (i.e., a null pointer), the linked list does not have a cycle.

Linked List

□ #206 Reverse Linked List

EAS

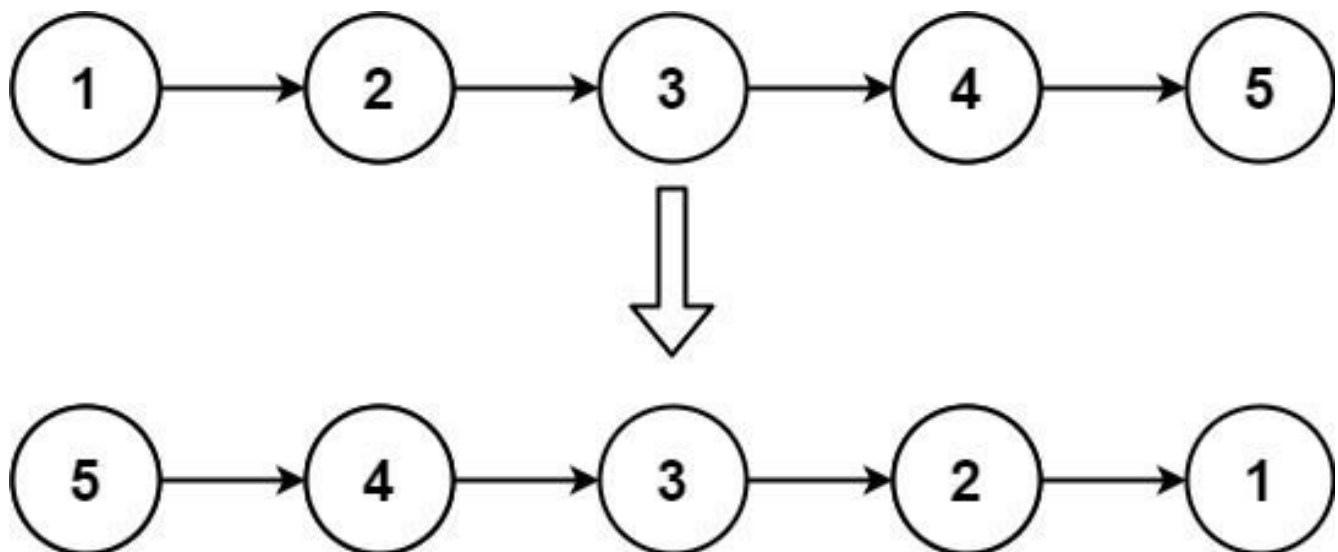
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion

Lists: Grind 169

Problem

Given the head of a singly linked list, reverse the list, and return the reversed list.

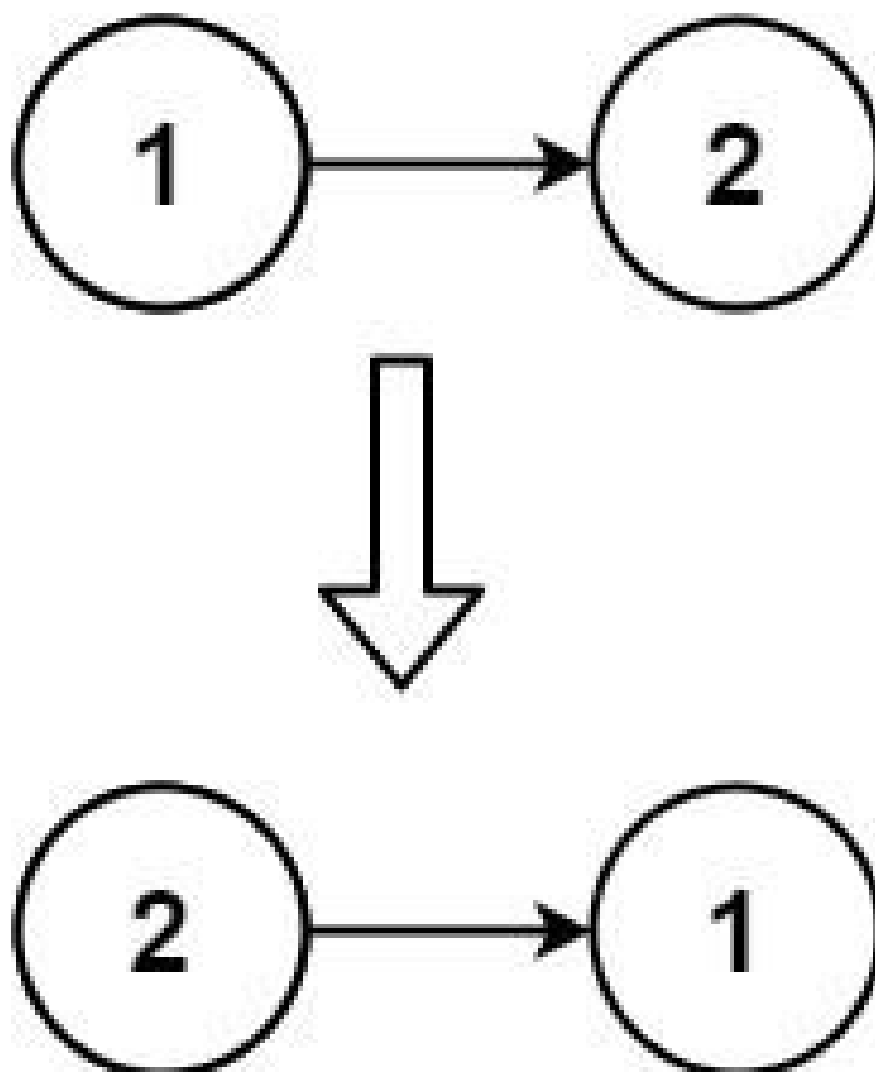
Example 1:



Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]

Example 2:



Input: head = [1,2]

Output: [2,1]

Example 3:

Input: head = []

Output: []

Constraints:

- The number of nodes in the list is the range [0, 5000].
- $-5000 \leq \text{Node.val} \leq 5000$

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

My LeetCode Solution
Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given the head of a singly linked list, reverse it in-place and return
# the new head. Right?"
#
# "head=[1,2,3,4,5]:
# 1→2→3→4→5→None becomes None←1←2←3←4←5, new head = 5 ✓
# head=[]: return None ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Collect all values in an array, rebuild a new reversed list.
# Time: O(n). Space: O(n) for the values."
```

```
class _206_ReverseLinkedList_Brute:
    def reverseList(self, head):
        vals = []
        while head: vals.append(head.val); head = head.next
        dummy = cur = ListNode(0)
        for v in reversed(vals):
            cur.next = ListNode(v); cur = cur.next
        return dummy.next
```

PHASE 3 — OPTIMIZE

```
# "Three-pointer iterative approach: prev, curr, next_node.
# At each step: save curr.next, point curr.next to prev, advance both pointers.
# When curr is None, prev is the new head.
#
# Time: O(n). Space: O(1) – in-place pointer rewiring."
```

PHASE 4 — CLEAN CODE

```
class _206_ReverseLinkedList:
    def reverseList(self, head):
        prev, curr = None, head
        while curr:
            nxt = curr.next
```

```

    curr.next = prev
    prev = curr
    curr = nxt
    return prev

```

PHASE 5 — TEST & DEBUG

```

# head=[1,2,3]:
# prev=None,curr=1: nxt=2, 1.next=None, prev=1, curr=2
# prev=1,curr=2: nxt=3, 2.next=1, prev=2, curr=3
# prev=2,curr=3: nxt=None, 3.next=2, prev=3, curr=None
# return 3 → [3,2,1] ✓
#
# head=[]: curr=None immediately → return None ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): one pass. Space O(1): constant extra pointers.
# Recursive solution: O(n) time, O(n) stack space – elegant but risky for large
inputs.
# Follow-up: reverse nodes in k-group (LC 25) – applies this same logic per group.
# Follow-up: reverse between positions left and right (LC 92) – isolate the sublist
first."

```

◆ Quick Review

Key Insights

- The problem can be solved using either an iterative or recursive approach.
- Iterative method uses constant space by updating pointers in-place.
- Recursive method leverages the call stack to reverse the list, which could use $O(n)$ space in the worst case.
- Both approaches traverse each node exactly once, yielding linear time complexity.

Space & Time

Time Complexity: $O(n)$ for both iterative and recursive approaches, where n is the number of nodes.

Space Complexity: $O(1)$ for the iterative approach and $O(n)$ for the recursive approach because of the recursion call stack.

Solution

We can solve the problem by iterating through the list while reversing the next pointers of each node. We maintain two pointers (previous and current) and update them as we traverse the list, so that by the end, the previous pointer is at the new head of the reversed list. Alternatively, a recursive approach reverses the rest of the list first and then adjusts the pointers so that the original head becomes the tail.

Linked List

□ #234 Palindrome Linked List

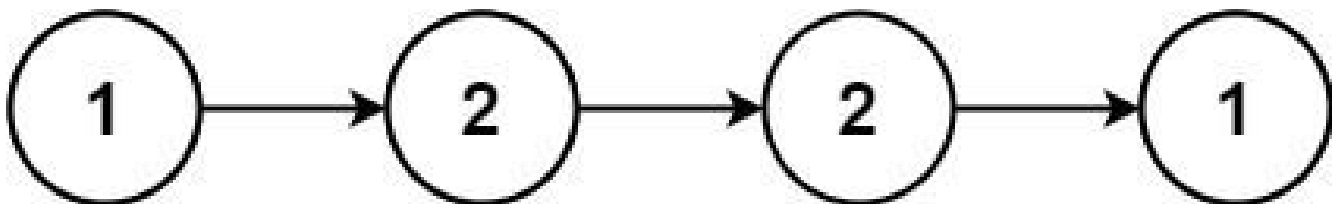
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers · Recursion
Lists: Grind 169

Problem

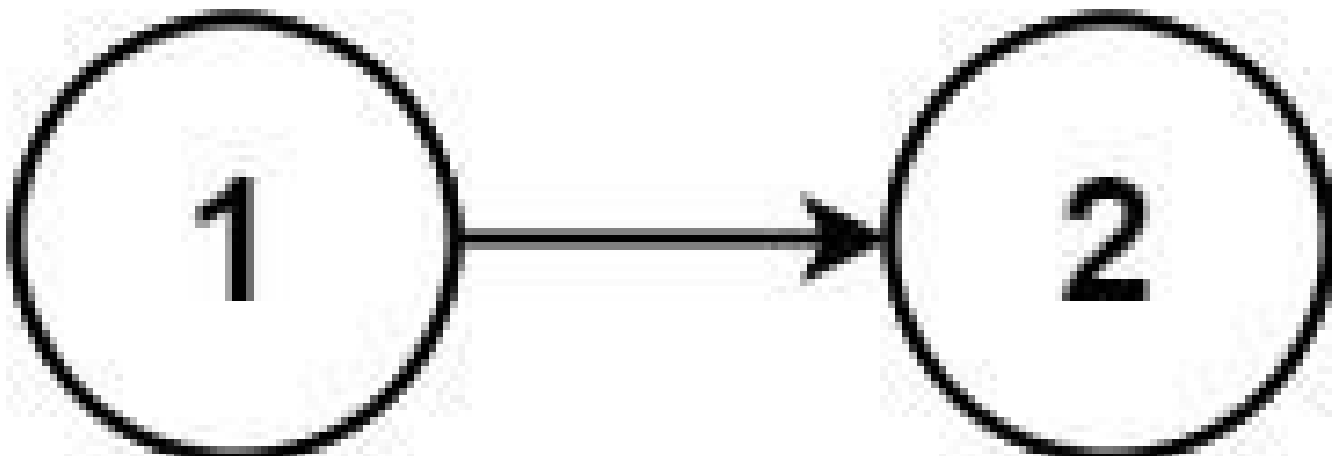
Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1:



Input: head = [1,2,2,1]
Output: true

Example 2:



Input: head = [1,2]

Output: false

Constraints:

- The number of nodes in the list is in the range [1, 105].
- $0 \leq \text{Node.val} \leq 9$

Follow up: Could you do it in $O(n)$ time and $O(1)$ space?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given the head of a singly linked list, return true if it reads the same forwards and backwards. Right?"

#

"head=[1,2,2,1]: reads same both ways → true ✓

head=[1,2]: 1≠2 → false ✓"

PHASE 2 — BRUTE FORCE

"Copy all values into an array, then use two-pointer comparison.

Time: $O(n)$. Space: $O(n)$."

```
class _234_PalindromeLinkedList_Brute:
```

```
    def isPalindrome(self, head) -> bool:
```

```
        vals = []
```

```
        while head: vals.append(head.val); head = head.next
```

```
        return vals == vals[::-1]
```

PHASE 3 — OPTIMIZE

"Achieve $O(1)$ space by modifying the list in-place:

1. Find the midpoint using slow/fast pointers.

2. Reverse the second half in-place.

3. Compare first half with reversed second half.

4. (Optional) restore the list.

#

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _234_PalindromeLinkedList:
    def isPalindrome(self, head) -> bool:
        # Step 1: find middle
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # Step 2: reverse second half
        prev, curr = None, slow
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt

        # Step 3: compare halves
        left, right = head, prev
        while right:
            if left.val != right.val:
                return False
            left = left.next
            right = right.next
        return True
```

PHASE 5 — TEST & DEBUG

```
# [1,2,2,1]:
# Mid: slow=2(idx1), fast=1(idx3). Reverse [2,1]→[1,2].
# Compare: 1==1 ✓, 2==2 ✓, right=None → True ✓
#
# [1,2]:
# Mid: slow=2, fast=None after 1 step. Reverse [2]→[2].
# Compare: 1≠2 → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ : three linear passes (find mid, reverse, compare).
# Space  $O(1)$ : constant extra pointers.
# Trade-off:  $O(n)$  space approach is simpler and non-destructive.
# Follow-up: restore the original list after the check by re-reversing second half.
# Follow-up: palindrome check on a doubly linked list is trivial – two outer pointers."
```

◆ Quick Review

Key Insights

- Use two pointers (slow and fast) to reach the middle of the list.
- Reverse the second half of the list.
- Compare the nodes from the start of the list to the reversed half.
- This approach meets the $O(n)$ time and $O(1)$ space follow-up requirement.

Space & Time

Time Complexity: $O(n)$ – each node is visited a constant number of times.

Space Complexity: $O(1)$ – only constant extra space is used (in-place reversal).

Solution

We first use the two-pointer technique to locate the midpoint of the linked list. The slow pointer moves one step at a time while the fast pointer moves two steps. Once the middle is found, we reverse the second half of the list in-place. Finally, we traverse both halves concurrently to compare the node values. If all corresponding values match, the linked list is a palindrome. A potential gotcha is handling the odd number of nodes; in that case, we ignore the middle element when comparing.

Linked List

□ #706 Design HashMap

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Linked List · Design ·
Hash Function**

Lists: Denny Zhang

Problem

Design a HashMap without using any built-in hash table libraries.

Implement the MyHashMap class:

- **MyHashMap()** initializes the object with an empty map.
- **void put(int key, int value)** inserts a (key, value) pair into the HashMap. If the key already exists in the map, update the corresponding value.
- **int get(int key)** returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.
- **void remove(key)** removes the key and its corresponding value if the map contains the

mapping for the key.

Example 1:

Input

```
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
```

Output

```
[null, null, null, 1, -1, null, 1, null, -1]
```

Explanation

```
MyHashMap myHashMap = new MyHashMap();
```

Constraints:

- $0 \leq \text{key}, \text{value} \leq 106$
- At most 104 calls will be made to put, get, and remove.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Implement a HashMap without built-in hash libraries. Support put(key, val),
# get(key) → -1 if missing, remove(key). Keys and values are non-negative ints
# in range [0, 10^6]. Right?"
```

#

```
# "put(1,1), put(2,2), get(1)→1, get(3)→-1, put(2,1), get(2)→1, remove(2),
get(2)→-1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Store all pairs in a plain list. Scan linearly for get/put/remove.
# O(n) per operation – correct but slow."
```

```
class _706_DesignHashMap_Brute:
```

```
    def __init__(self):
```

```
        self.data = [] # list of [key, value] pairs
```

```
    def put(self, key: int, value: int) -> None:
```

```
        for pair in self.data:
```

```
            if pair[0] == key:
```

```

        pair[1] = value
        return
    self.data.append([key, value])
def get(self, key: int) -> int:
    for pair in self.data:
        if pair[0] == key:
            return pair[1]
    return -1
def remove(self, key: int) -> None:
    self.data = [p for p in self.data if p[0] != key]

```

PHASE 3 — OPTIMIZE

"Use an array of buckets with separate chaining (linked list / small list per bucket).

Hash function: $\text{key} \% \text{num_buckets}$ using a prime to spread keys.

Average $O(1)$ per op when load factor is low. Space $O(n + k)$."

PHASE 4 — CLEAN CODE

```

class _706_DesignHashMap:
    def __init__(self):
        self.size = 1009
        self.buckets = [[] for _ in range(self.size)]
    def _h(self, key: int) -> int:
        return key % self.size
    def put(self, key: int, value: int) -> None:
        bucket = self.buckets[self._h(key)]
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))
    def get(self, key: int) -> int:
        for k, v in self.buckets[self._h(key)]:
            if k == key:
                return v
        return -1
    def remove(self, key: int) -> None:
        h = self._h(key)
        self.buckets[h] = [(k, v) for k, v in self.buckets[h] if k != key]

```

PHASE 5 — TEST & DEBUG

```

# put(1,1): bucket[1]=[(1,1)]
# put(2,2): bucket[2]=[(2,2)]
# get(1): bucket[1] → (1,1) → 1 ✓
# get(3): bucket[3]=[] → -1 ✓
# put(2,1): bucket[2] → update → [(2,1)]
# get(2): → 1 ✓
# remove(2): bucket[2]=[]
# get(2): [] → -1 ✓

```


PHASE 6 — COMPLEXITY & FOLLOW-UP

"Brute: $O(n)$ per op. Hash bucket: $O(1)$ average, $O(n)$ worst (all same bucket).

Space $O(n+k)$: n stored pairs + k bucket slots.

Follow-up: Design HashSet (LC 705) – same idea, keys only.

Follow-up: open addressing (linear probing) avoids linked lists but needs tombstones."

◆ Quick Review

Key Insights

- Use a fixed-size array of buckets where each bucket uses a linked list to handle collisions.
- Choose a simple hash function (e.g., modulo operation) to map a key to its corresponding bucket.
- For each bucket, traverse the linked list to update an existing key or append a new node if the key does not exist.
- Removal requires updating pointers in the linked list to bypass the removed node.

Space & Time

Time Complexity: Average $O(1)$ for put, get, and remove; worst-case $O(n)$ when many keys hash to the same bucket.

Space Complexity: $O(n)$, where n is the number of key-value pairs stored.

Solution

We implement the HashMap using an array of fixed size (e.g., 1000). Each array element is the head of a linked list that manages collisions using chaining. The key is hashed using a modulo operation. For the put operation, if a key already exists in the chain, its value is updated; otherwise, a new node is appended. The get operation traverses the chain at the computed bucket index to find and return the corresponding value, or returns -1 if not found. The remove operation also traverses the list, carefully reconnecting the links to remove the target node while handling edge cases such as removal at the head.

Linked List

□ #876 Middle of the Linked List

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Linked List · Two Pointers

Lists: Grind 169

Problem

Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

Example 1:

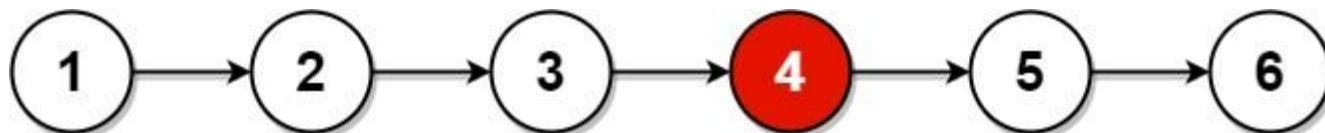


Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:



Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Constraints:

- The number of nodes in the list is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given the head of a linked list, return the middle node.

If two middles exist (even length), return the SECOND one. Right?"

#

"head=[1,2,3,4,5]: length=5, middle index=2 → node(3) ✓

head=[1,2,3,4,5,6]: length=6, two middles at index 2,3 → return node(4) ✓"

PHASE 2 — BRUTE FORCE

"Count the total number of nodes, then walk to index length//2.

Two passes: one to count, one to reach the middle.

Time: O(n). Space: O(1)."

```
class _876_MiddleOfLinkedList_Brute:
```

```
    def middleNode(self, head):
```

```
        length = 0
```

```
        curr = head
```

```
        while curr:
```

```
            length += 1
```

```
            curr = curr.next
```

```
        curr = head
```

```
        for _ in range(length // 2):
```

```
            curr = curr.next
```

```
        return curr
```

PHASE 3 — OPTIMIZE

"Slow/fast pointer — one pass.

Slow moves 1 step, fast moves 2. When fast reaches the end, slow is at the middle.

```
# For even-length lists, slow naturally lands on the SECOND middle (desired).
#
# Time: O(n). Space: O(1). Saves one full traversal vs brute."
```

PHASE 4 — CLEAN CODE

```
class _876_MiddleOfLinkedList:
    def middleNode(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3,4,5]:
# s=1,f=1 → s=2,f=3 → s=3,f=5 → fast.next=None → stop. return node(3) ✓
#
# [1,2,3,4,5,6]:
# s=1,f=1 → s=2,f=3 → s=3,f=5 → s=4,f=None(6.next) → stop. return node(4) ✓
#
# [1]:
# fast.next=None immediately → return head=node(1) ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): fast pointer covers the list in n/2 iterations.
# Space O(1): two pointers only.
# Follow-up: if you need the FIRST middle for even-length, stop when fast.next.next
is None.
# Follow-up: this slow/fast trick is the building block for palindrome check (LC
234)
# and cycle detection (LC 141)."
```

◆ Quick Review

Key Insights

- Use two pointers (slow and fast) to traverse the list.
- The fast pointer moves two steps at a time while the slow pointer moves one step.
- When the fast pointer reaches the end, the slow pointer will be at the middle node.

- This approach naturally handles both odd and even lengths of the list.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the list.

Space Complexity: $O(1)$, constant extra space.

Solution

The solution uses the two-pointer technique: Initialize two pointers, slow and fast, at the head. Traverse the list, moving slow by one step and fast by two steps during each iteration. When fast reaches the end or has no next node, slow will be at the middle. Return the slow pointer (the middle node). This strategy ensures linear time complexity and constant space, efficiently finding the middle node even for lists with an even number of elements.

Linked List

□ ★ #1474 Delete N Nodes After M Nodes of Linked List ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: ★ Premium | Premium 98

Problem

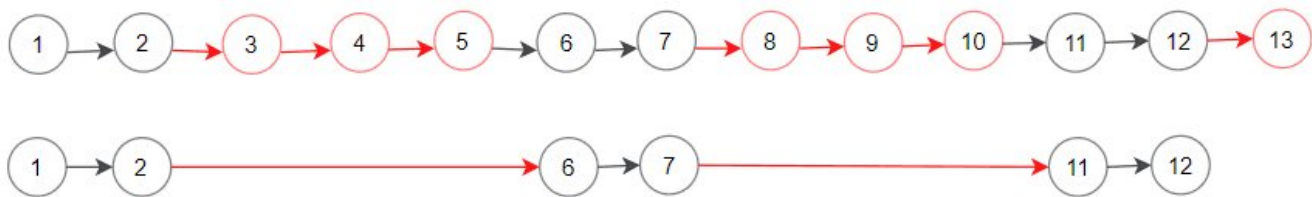
You are given the head of a linked list and two integers m and n .

Traverse the linked list and remove some nodes in the following way:

- Start with the head as the current node.
- Keep the first m nodes starting with the current node.
- Remove the next n nodes
- Keep repeating steps 2 and 3 until you reach the end of the list.

Return the head of the modified list after removing the mentioned nodes.

Example 1:



Input: head = [1,2,3,4,5,6,7,8,9,10,11,12,13], m = 2, n = 3

Output: [1,2,6,7,11,12]

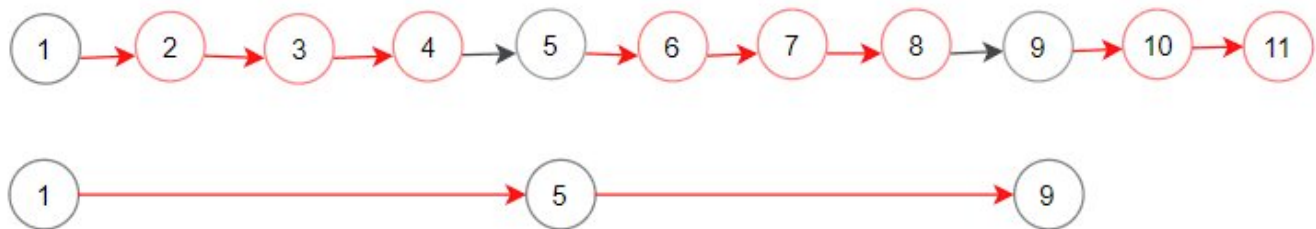
Explanation: Keep the first (m = 2) nodes starting from the head of the linked List (1 -> 2) show in black nodes.

Delete the next (n = 3) nodes (3 -> 4 -> 5) show in red nodes.

Continue with the same procedure until reaching the tail of the Linked List.

Head of the linked list after removing nodes is returned.

Example 2:



Input: head = [1,2,3,4,5,6,7,8,9,10,11], m = 1, n = 3

Output: [1,5,9]

Explanation: Head of linked list after removing nodes is returned.

Constraints:

- The number of nodes in the list is in the range [1, 10⁴].
- 1 ≤ Node.val ≤ 10⁶
- 1 ≤ m, n ≤ 1000

Follow up: Could you solve this problem by modifying the list in-place?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a linked list and integers m and n: keep m nodes, delete next n nodes,
# repeat until the end. Return the modified head. Right?"
#
# "head=[1,2,3,4,5,6,7,8,9,10,11,12,13], m=2, n=3:
# Keep 1,2 → delete 3,4,5 → keep 6,7 → delete 8,9,10 → keep 11,12 → delete 13
# → [1,2,6,7,11,12] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Enumerate all nodes with their index. Keep node i if (i % (m+n)) < m.
# Collect kept nodes and relink them.
# Time: O(n_total). Space: O(n_total) for the kept-nodes list."
```

```
class _1474_DeleteNNodesAfterMNodes_Brute:
    def deleteNodes(self, head, m: int, n: int):
        kept = []
        curr = head
        i = 0
        while curr:
            if i % (m + n) < m:
                kept.append(curr)
                curr = curr.next
                i += 1
        for j in range(len(kept) - 1):
            kept[j].next = kept[j + 1]
        if kept:
            kept[-1].next = None
        return kept[0] if kept else None
```

PHASE 3 — OPTIMIZE

```
# "Pointer-based in-place: walk m steps forward (keeping nodes), then
# walk n steps forward (skipping/deleting nodes), then repeat.
# Relink the tail of the kept section directly to the node after the deleted
# section.
# Time: O(n_total). Space: O(1) – no extra storage."
```

PHASE 4 — CLEAN CODE

```
class _1474_DeleteNNodesAfterMNodes:
    def deleteNodes(self, head, m: int, n: int):
        curr = head
        while curr:
            # Walk m-1 steps to reach end of kept section
            for _ in range(m - 1):
                if curr: curr = curr.next
            if not curr:
                break
```

```

        tail = curr # last kept node
        # Walk n steps to skip deleted section
        skip = curr.next
        for _ in range(n):
            if skip: skip = skip.next
        tail.next = skip # relink
        curr = skip # continue from next kept section
    return head

```

PHASE 5 — TEST & DEBUG

```

# [1,2,3,4,5,6,7,8,9,10,11,12,13], m=2, n=3:
# curr=1, walk 1 step → curr=2 (tail=2). skip=3, walk 3 → skip=6. 2.next=6, curr=6.
# curr=6, walk 1 step → curr=7 (tail=7). skip=8, walk 3 → skip=11. 7.next=11,
curr=11.
# curr=11, walk 1 step → curr=12 (tail=12). skip=13, walk 3 → skip=None.
12.next=None.
# → [1,2,6,7,11,12] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(total nodes): each node visited exactly once.
# Space O(1): only pointer variables.
# Follow-up: what if m and n can vary per cycle? Would need to pass arrays.
# Follow-up: similar pattern to 'Remove Duplicates from Sorted List II' in pointer
reuse."

```

◆ Quick Review

Key Insights

- Traverse the linked list while keeping count.
- Skip m nodes (i.e., keep them) and then proceed to delete the next n nodes.
- Update pointers to bypass the removed nodes.
- This is an in-place modification problem, so no additional list structure is needed.
- Be cautious about edge cases when the list has fewer than m or n nodes remaining.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the list.

Space Complexity: $O(1)$ since the modification is done in-place.

Solution

The solution involves iterating through the linked list with a pointer. For each cycle: Traverse m nodes and keep them. Then, from the $(m+1)$ th node, skip and remove the next n nodes by adjusting the 'next' pointer of the m -th node to point to the node after these n nodes. Continue until you reach the end of the list. This in-place approach avoids using extra space and maintains $O(N)$ time complexity.

Stack

Round 4 · Mid · Easy · 3 questions

Stack

□ #20 Valid Parentheses

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack
Lists: Grind 169

Problem

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Output: true

Example 3:

Input: s = "())"

Output: false

Example 4:

Input: s = "([)]"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Constraints:

- $1 \leq s.length \leq 104$
- s consists of parentheses only '()[]{}'.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given a string of brackets, return true if every open bracket is closed
by the same type in the correct order. Right?"

#

"s="()[]{}": each pair matches in order → true ✓

s="()": '(' expects ')' but gets ']' → false ✓

s="([)]": '(' then '[', but ')' closes '(' while '[' is still open → false ✓"

PHASE 2 — BRUTE FORCE

"Repeatedly replace matching pairs '()', '[]', '{} ' until nothing changes.

If the string is empty at the end it's valid.

```

# Time:  $O(n^2)$  – up to  $n/2$  passes each scanning  $n$  chars. Space:  $O(n)$ ."
class _20_ValidParentheses_Brute:
    def isValid(self, s: str) -> bool:
        while '()' in s or '[]' in s or '{}' in s:
            s = s.replace('()', '').replace('[]', '').replace('{}', '')
        return s == ''

# PHASE 3 — OPTIMIZE
# "Use a stack. Push every opening bracket.
# On a closing bracket, the top of the stack must be its matching opener.
# If not – or the stack is empty – return false.
# At the end, the stack must be empty.
#
# Time:  $O(n)$  – one pass. Space:  $O(n)$  for the stack."

# PHASE 4 — CLEAN CODE
class _20_ValidParentheses:
    def isValid(self, s: str) -> bool:
        stack = []
        match = {')': '(', ']': '[', '}': '{'}
        for ch in s:
            if ch in match:
                if not stack or stack[-1] != match[ch]:
                    return False
                stack.pop()
            else:
                stack.append(ch)
        return not stack

# PHASE 5 — TEST & DEBUG
# s="()[]{}":
# '(' push. ')' → top='(' matches → pop. '[' push. ']' → pop. '{' push. '}' → pop.
# stack empty → True ✓
#
# s="()":
# '(' push. ']' → top='(' ≠ '[' → return False ✓
#
# s="([])":
# '(' push. '[' push. ')' → top='[' ≠ '(' → return False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n)$ : one pass through the string.
# Space  $O(n)$ : worst case all openers e.g. '(((((' .
# Follow-up: minimum removals to make valid (LC 1249) extends this with counters.
# Follow-up: what if we also have wildcards '*' that can be '(', ')' or ''? (LC 678)."
```

◆ Quick Review

Key Insights

- The problem can be solved using a stack data structure.
- When encountering an opening bracket, push it onto the stack.
- When encountering a closing bracket, check if the top of the stack is its matching opening bracket.
- If the stack is empty before finding a match or if there is a mismatch, the string is invalid.
- At the end, the string is valid only if the stack is empty, meaning all brackets were properly matched.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, since we process each character once.

Space Complexity: $O(n)$ in the worst case for the stack when all characters are opening brackets.

Solution

We use a stack to check for valid parentheses. The algorithm iterates through the string,

pushing opening brackets onto the stack and popping them when a corresponding closing bracket is encountered. A mapping of closing to opening brackets helps in validating the matching pairs. By ensuring that the stack is empty at the end, we confirm that every opening bracket was properly closed.

Stack

□ #232 Implement Queue using Stacks

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design · Queue
Lists: Grind 169

Problem

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- **void push(int x)** Pushes element x to the back of the queue.
- **int pop()** Removes the element from the front of the queue and returns it.
- **int peek()** Returns the element at the front of the queue.
- **boolean empty()** Returns true if the queue is empty, false otherwise.

Notes:

- You must use only standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

Input

```
["MyQueue", "push", "push", "peek", "pop", "empty"]
```

```
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 1, 1, false]
```

Explanation

```
MyQueue myQueue = new MyQueue();
```

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to push, pop, peek, and empty.
- All the calls to pop and peek are valid.

Follow-up: Can you implement the queue such that each operation is amortized $O(1)$ time complexity? In other words, performing n operations will take overall $O(n)$ time even if one of those operations may take longer.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "We need a queue (FIFO) built only from stacks (LIFO). Support push(x), pop(),
# peek(), and empty(). Amortized O(1) per operation is acceptable. Right?"
#
# "push(1), push(2), peek() → 1, pop() → 1, empty() → false
# The first element pushed must be first out – opposite of a stack."
```

PHASE 2 — BRUTE FORCE

```
# "Use one stack. On every push, move ALL elements out, push the new one to bottom,
# then put everything back. O(n) per push, O(1) pop/peek – queue order maintained."
```

```
class _232_ImplementQueueUsingStacks_Brute:
    def __init__(self):
        self.stack = []

    def push(self, x: int) -> None:
        # Move everything out, add x at bottom, put back
        tmp = []
        while self.stack:
            tmp.append(self.stack.pop())
        self.stack.append(x)
        while tmp:
            self.stack.append(tmp.pop())

    def pop(self) -> int:
        return self.stack.pop()

    def peek(self) -> int:
        return self.stack[-1]

    def empty(self) -> bool:
        return not self.stack
```

PHASE 3 — OPTIMIZE

```
# "Two stacks – inbox and outbox – achieve amortized O(1).
# Push always goes to inbox.
# For pop/peek: if outbox is empty, pour ALL of inbox into outbox (reverses order).
# Then pop/peek from outbox.
# Each element moves at most twice total across both stacks."
```

PHASE 4 — CLEAN CODE

```
class _232_ImplementQueueUsingStacks:
    def __init__(self):
        self.inbox = []
```

```

        self.outbox = []
    def push(self, x: int) -> None:
        self.inbox.append(x)
    def _pour(self) -> None:
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())
    def pop(self) -> int:
        self._pour()
        return self.outbox.pop()
    def peek(self) -> int:
        self._pour()
        return self.outbox[-1]
    def empty(self) -> bool:
        return not self.inbox and not self.outbox

```

PHASE 5 — TEST & DEBUG

```

# push(1): inbox=[1]
# push(2): inbox=[1,2]
# peek(): outbox empty → pour → inbox=[], outbox=[2,1]. outbox[-1]=1 ✓
# pop(): outbox=[2,1] → pop → 1, outbox=[2] ✓
# empty(): inbox=[], outbox=[2] → False ✓
# pop(): outbox=[2] → pop → 2 ✓
# empty(): both empty → True ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Brute:  $O(n)$  push,  $O(1)$  pop. Two-stack: amortized  $O(1)$  all ops.
# Each element transferred at most once – total work  $O(n)$  for  $n$  ops.
# Space  $O(n)$  across both stacks.
# Follow-up: implement stack using queues (LC 225) – symmetric design.
# Follow-up: if we needed worst-case  $O(1)$  all ops, we'd need a deque."

```

◆ Quick Review

Key Insights

- Use two stacks:
- One (input stack) for enqueue operations.
- The other (output stack) for dequeue operations.

- When performing pop or peek, if the output stack is empty, transfer all elements from the input stack to the output stack. This reversal makes the oldest element accessible.
- Each element is moved at most once between the stacks, ensuring an amortized $O(1)$ cost per operation.
- The space complexity is $O(n)$ where n is the number of elements in the queue.

Space & Time

Time Complexity: Amortized $O(1)$ per operation

Space Complexity: $O(n)$

Solution

We maintain two stacks: an input stack for pushing new elements and an output stack for popping or peeking the front element of the queue. When we need to access the front of the queue (either during a pop or peek), we check if the output stack is empty. If it is, we transfer all elements from the input stack to the output stack. This reversal of order allows us to simulate the FIFO nature of the queue. When performing a push, the element is simply

added to the input stack.

Stack

□ #844 Backspace String Compare

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Stack · Simulation
Lists: Grind 169

Problem

Given two strings *s* and *t*, return true if they are equal when both are typed into empty text editors. '#' means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1:

Input: *s* = "ab#c", *t* = "ad#c"
Output: true
Explanation: Both *s* and *t* become "ac".

Example 2:

Input: *s* = "ab##", *t* = "c#d#"
Output: true
Explanation: Both *s* and *t* become "".

Example 3:

Input: *s* = "a#c", *t* = "b"
Output: false
Explanation: *s* becomes "c" while *t* becomes "b".

Constraints:

- $1 \leq s.length, t.length \leq 200$
- s and t only contain lowercase letters and '#' characters.

Follow up: Can you solve it in $O(n)$ time and $O(1)$ space?

My LeetCode Solution
Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given two strings where '#' represents a backspace, return true if they're equal after processing. Right?"

#

$s = "ab\#c"$, $t = "ad\#c"$:

$s: a, b, \# \rightarrow a, c \rightarrow "ac"$. $t: a, d, \# \rightarrow a, c \rightarrow "ac"$. Equal $\rightarrow$ true ✓

$s = "a\#c"$, $t = "b"$: $s \rightarrow "c"$, $t \rightarrow "b"$. Not equal $\rightarrow$ false ✓

PHASE 2 — BRUTE FORCE

"Simulate using a stack: push normal chars, pop on '#' if stack non-empty.

Compare final stacks.

Time: $O(n+m)$. Space: $O(n+m)$ for the stacks."

```
class _844_BackspaceStringCompare_Brute:
    def backspaceCompare(self, s: str, t: str) -> bool:
        def process(string):
            stack = []
            for ch in string:
                if ch == '#':
                    if stack: stack.pop()
                else:
                    stack.append(ch)
            return stack
        return process(s) == process(t)
```

PHASE 3 — OPTIMIZE

"Two-pointer approach from the RIGHT — $O(1)$ space.

Traverse both strings backwards, counting '#' as skip tokens.

Skip that many non-'#' characters before comparing.

If both pointers exhausted simultaneously $\rightarrow$ equal.

```

#
# Time: O(n+m). Space: O(1).

# PHASE 4 — CLEAN CODE
class _844_BackspaceStringCompare:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i, j = len(s) - 1, len(t) - 1
        skip_s = skip_t = 0
        while i >= 0 or j >= 0:
            while i >= 0:
                if s[i] == '#': skip_s += 1; i -= 1
                elif skip_s: skip_s -= 1; i -= 1
                else: break
            while j >= 0:
                if t[j] == '#': skip_t += 1; j -= 1
                elif skip_t: skip_t -= 1; j -= 1
                else: break
            if i >= 0 and j >= 0 and s[i] != t[j]:
                return False
            if (i >= 0) != (j >= 0):
                return False
            i -= 1; j -= 1
        return True

# PHASE 5 — TEST & DEBUG
# s="ab#c", t="ad#c":
# i=3'c',j=3'c' → both non-# → c==c ✓. i=2,j=2.
# i=2'#'→skip_s=1,i=1. s[1]='b',skip_s→0,i=0.
# j=2'#'→skip_t=1,j=1. t[1]='d',skip_t→0,j=0.
# s[0]='a', t[0]='a' → equal ✓. i=-1,j=-1 → return True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Stack approach: O(n+m) time, O(n+m) space – simple and readable.
# Two-pointer approach: O(n+m) time, O(1) space – optimal.
# Follow-up: if '#' can backspace multiple characters (like '2#'), adjust skip
counter.
# Follow-up: streaming input – the stack approach handles it naturally."

```

◆ Quick Review

Key Insights

- '#' represents a backspace operation that deletes the previous character.

- A direct simulation using a stack can reconstruct the final string.
- A two–pointer approach allows comparison from the end of the strings while skipping backspaced characters.
- The two–pointer technique can achieve $O(n)$ time and $O(1)$ space by processing the strings in reverse.

Space & Time

Time Complexity: $O(n)$ where n is the length of the longer string.

Space Complexity: $O(1)$ using the two–pointer strategy ($O(n)$ if using stacks).

Solution

The solution uses two pointers starting from the end of each string. For each string, maintain a skip counter to account for backspace characters. As you iterate backwards: If a '#' is encountered, increment the skip counter. If a normal character is encountered while the skip counter is positive, skip that character by decrementing the counter. Once a valid character is found, compare the characters from both strings. If

all corresponding characters match and both pointers finish processing at the same time, the strings are considered equal.

Trie

Round 4 · Mid · Easy · 1 question

Trie

□ #14 Longest Common Prefix

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Trie

Lists: Grind 169

Problem

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string "".

Example 1:

```
Input: strs = ["flower","flow","flight"]  
Output: "fl"
```

Example 2:

```
Input: strs = ["dog","racecar","car"]  
Output: ""  
Explanation: There is no common prefix among the input strings.
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- $\text{strs}[i]$ consists of only lowercase English letters if it is non-empty.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "We're given an array of strings and need to find the longest prefix
# shared by ALL of them. If none exists, return an empty string. Right?"
#
# strs=["flower","flow","flight"]:
# Compare column by column: f✓ l✓ o vs o vs i → mismatch at index 2 → return "fl" ✓
# strs=["dog","racecar","car"]: first chars d vs r → mismatch → return "" ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Sort the array, then only compare the first and last strings –
# the LCP of those two is the LCP of the whole array.
# Time:  $O(n \log n + m)$  where  $m$  is shortest string length. Space:  $O(1)$ ."
```

```
class _14_LongestCommonPrefix_Brute:
```

```
    def longestCommonPrefix(self, strs: list[str]) -> str:
        if not strs: return ""
        strs.sort()
        first, last = strs[0], strs[-1]
        i = 0
        while i < len(first) and i < len(last) and first[i] == last[i]:
            i += 1
        return first[:i]
```

PHASE 3 — OPTIMIZE

```
# "Vertical scanning avoids the sort entirely.
# Take the first string as the reference. For each character position i,
# check that every other string has the same character at position i.
# Stop as soon as any string is too short or has a mismatch.
```

```
#
# Pseudocode:
# for i, char in enumerate(strs[0]):
# for s in strs[1:]:
# if i >= len(s) or s[i] != char: return strs[0][:i]
# return strs[0]
#
# Time:  $O(n*m)$  –  $n$  strings,  $m$  = min length. Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
class _14_LongestCommonPrefix:
    def longestCommonPrefix(self, strs: list[str]) -> str:
        if not strs:
            return ""
```

```

    for i, char in enumerate(strs[0]):
        for s in strs[1:]:
            if i >= len(s) or s[i] != char:
                return strs[0][:i]
    return strs[0]

```

PHASE 5 — TEST & DEBUG

```

# strs=["flower","flow","flight"]:
# i=0 'f': all match. i=1 'l': all match. i=2 'o': strs[2][2]='i' ≠ 'o' → return "fl" ✓
#
# strs=["dog","racecar","car"]:
# i=0 'd': strs[1][0]='r' ≠ 'd' → return "" ✓
#
# strs=["abc"]:
# No inner loop fires → return "abc" ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n*m): every char of shortest string compared across all n strings.
# Space O(1): no extra allocation beyond the output slice.
# Follow-up: if the array is huge but strings are short, binary search on
# prefix length works – O(m log m * n) but better when m << n.
# Follow-up: Trie would let you insert all strings and walk until a branch point."

```

◆ Quick Review

Key Insights

- Compare characters of the strings one column at a time (vertical scanning).
- The longest possible prefix is limited by the length of the shortest string in the array.
- As soon as a mismatch is found, the prefix up to that point is the answer.
- Handle edge cases such as an empty input array or strings with no common prefix.

Space & Time

Time Complexity: $O(N \cdot M)$ where N is the number of strings and M is the length of the shortest string.

Space Complexity: $O(1)$ additional space (ignoring output space).

Solution

We use the vertical scanning approach: If the input array is empty, return an empty string. Iterate character by character over the first string. For each character, compare it with the corresponding character in each of the other strings. If a mismatch or end of any string is encountered, return the substring from the start up to the current index. If no mismatch is found, the entire first string is the common prefix. This solution ensures that we only traverse the necessary part of the strings and stops early if a discrepancy is discovered.

Greedy

Round 4 · Mid · Easy · 1 question

Greedy

□ #409 Longest Palindrome

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Greedy
Lists: Grind 169

Problem

Given a string s which consists of lowercase or uppercase letters, return the length of the longest palindrome that can be built with those letters.

Letters are case sensitive, for example, "Aa" is not considered a palindrome.

Example 1:

Input: $s = \text{"abcccd"}$
Output: 7
Explanation: One longest palindrome that can be built is "dcccdd", whose length is 7.

Example 2:

Input: $s = \text{"a"}$
Output: 1
Explanation: The longest palindrome that can be built is "a", whose length is 1.

Constraints:

- $1 \leq s.length \leq 2000$

- s consists of lowercase and/or uppercase English letters only.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a string of letters, find the length of the longest palindrome
# we can BUILD from those characters (not a substring – we rearrange them). Right?"
#
# "s="abcccccdd":
# d×2, c×4, a×1, b×1. Pairs: dd, cccc → length 6. One leftover can go in center → 7
✓
# s="a": just 'a' → length 1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Count frequency of each character. A palindrome uses pairs symmetrically
# plus at most one center character.
# Sum all (freq // 2) * 2. If any character had an odd count, add 1 for center."
```

```
class _409_LongestPalindrome_Brute:
    def longestPalindrome(self, s: str) -> int:
        from collections import Counter
        freq = Counter(s)
        length = 0
        has_odd = False
        for count in freq.values():
            length += (count // 2) * 2
            if count % 2 == 1:
                has_odd = True
        return length + (1 if has_odd else 0)
```

PHASE 3 — OPTIMIZE

```
# "Same greedy logic, can use a set instead of a counter:
# Toggle each character in a set. At the end, characters still in the set
# appeared an odd number of times. Answer = len(s) - len(odd_chars) + (1 if
# odd_chars else 0).
#
# Time: O(n). Space: O(1) – at most 52 distinct letters."
```

PHASE 4 — CLEAN CODE

```
class _409_LongestPalindrome:
```

```
def longestPalindrome(self, s: str) -> int:
    odd_chars = set()
    for ch in s:
        if ch in odd_chars:
            odd_chars.remove(ch)
        else:
            odd_chars.add(ch)
    return len(s) - len(odd_chars) + (1 if odd_chars else 0)
```

PHASE 5 — TEST & DEBUG

```
# s="abcccd":
# a→{a}, b→{a,b}, c→{a,b,c}, c→{a,b}, c→{a,b,c}, c→{a,b}, d→{a,b,d}, d→{a,b}
# odd_chars={a,b}, len(s)=8, 8-2+1=7 ✓
#
# s="a":
# odd_chars={a}, 1-1+1=1 ✓
#
# s="aa":
# odd_chars={}, 2-0+0=2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass. Space O(1): at most 52 entries in the set.
# The greedy insight: use every pair, reserve at most one odd character for center.
# Follow-up: if you need to OUTPUT the actual palindrome, assign pairs to both ends
# and place one odd character in the middle."
```

◆ Quick Review

Key Insights

- Count the frequency of each character.
- For each character, you can use pairs (even count or odd count minus one) to form the palindrome.
- If any character has an odd frequency, you can add one extra character as the center of the palindrome.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(1)$ since the frequency counter will have at most 52 keys (considering both uppercase and lowercase letters).

Solution

The solution uses a hash table to count the frequency of each character. For every character, the maximum number of characters that can be used in a palindrome is determined by taking the largest even number that is less than or equal to its frequency (which is $\text{frequency} - (\text{frequency} \% 2)$). If there exists any character with an odd frequency, one of these can be used as the center of the palindrome, adding an extra character to the total length. This greedy approach efficiently computes the length of the largest possible palindrome from the available characters.

Round 5 · Mid · Medium

Round 5

Mid Priority · Medium

56 questions · 6 patterns

**Linked List #2 #19 #24 #61 #146 #148 #328
#369 #430 #622 #707 #708 #1171**

**Stack #143 #150 #155 #173 #227 #255 #394
#439 #484 #735 #739 #962 #1214 #1762**

**Heap #215 #253 #347 #505 #621 #658 #787
#973 #1057 #1167 #1424**

Trie #139 #208 #211 #616 #692 #720 #1166

Backtracking #17 #22 #39 #46 #79 #113

Greedy #134 #179 #280 #954 #2131

Linked List

Round 5 · Mid · Medium · 13 questions

Linked List

#2 Add Two Numbers

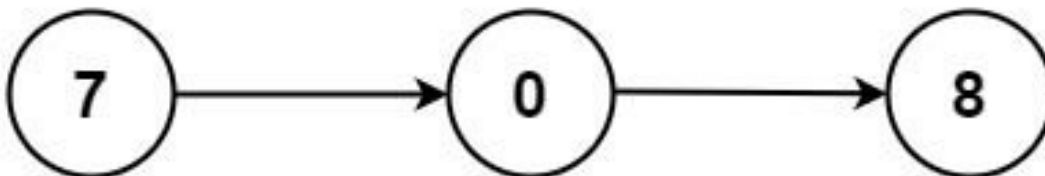
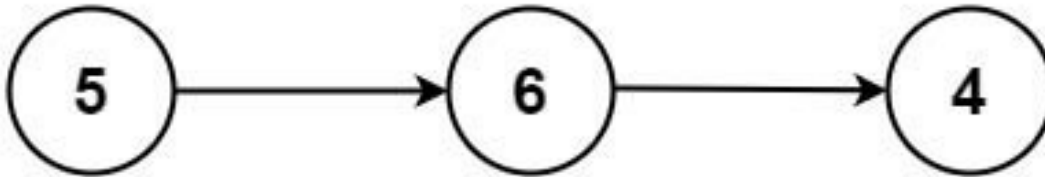
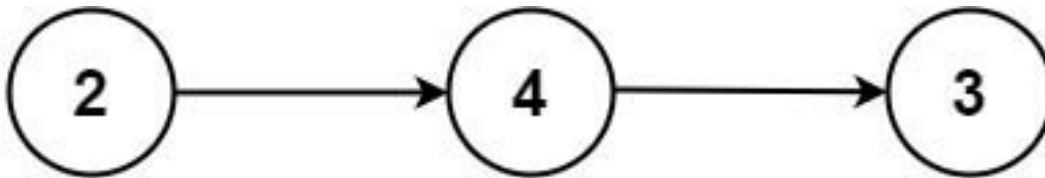
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Math · Recursion
Lists: Grind 169

Problem

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: 342 + 465 = 807.

Example 2:

Input: l1 = [0], l2 = [0]

Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].

- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Two non-empty linked lists store digits of non-negative integers in REVERSE
order.
# Add the two numbers and return the sum as a reversed linked list. Right?"
#
# "l1=[2,4,3] (342), l2=[5,6,4] (465): 342+465=807 → [7,0,8] ✓
# l1=[0], l2=[0]: 0+0=0 → [0] ✓
# l1=[9,9,9,9], l2=[9,9,9]: 9999+999=10998 → [8,9,9,0,1] ✓ (carry propagation)"
```

PHASE 2 — BRUTE FORCE

```
# "Convert both lists to integers, add, then convert the result back to a linked
list.
# Careful: the digits are stored in reverse, so the head is the least significant
digit.
# Time: O(max(m,n)). Space: O(max(m,n)) for the result."
```

```
class _2_AddTwoNumbers_Brute:
    def addTwoNumbers(self, l1, l2):
        def to_int(node):
            num, mul = 0, 1
            while node:
                num += node.val * mul
                mul *= 10
                node = node.next
            return num
        total = to_int(l1) + to_int(l2)
        if total == 0:
            return ListNode(0)
        dummy = cur = ListNode(0)
        while total:
            cur.next = ListNode(total % 10)
            cur = cur.next
            total //= 10
        return dummy.next
```

PHASE 3 — OPTIMIZE

```
# "Simulate grade-school addition digit by digit with a carry.
# Walk both lists simultaneously; at each position sum the digits + carry.
# new_digit = total % 10, new_carry = total // 10.
# Continue until both lists exhausted AND carry is 0.
# Time: O(max(m,n)). Space: O(max(m,n)+1) for the result – avoids big-integer
conversion."
```

PHASE 4 — CLEAN CODE

```
class _2_AddTwoNumbers:
    def addTwoNumbers(self, l1, l2):
        dummy = cur = ListNode(0)
        carry = 0
        while l1 or l2 or carry:
            val = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
            carry, digit = divmod(val, 10)
            cur.next = ListNode(digit)
            cur = cur.next
            if l1: l1 = l1.next
            if l2: l2 = l2.next
        return dummy.next
```

PHASE 5 — TEST & DEBUG

```
# l1=[2,4,3], l2=[5,6,4]:
# 2+5+0=7, carry=0 → node(7). 4+6+0=10, carry=1 → node(0). 3+4+1=8, carry=0 →
node(8).
# Both None, carry=0 → stop. → [7,0,8] ✓
#
# l1=[9,9,9,9], l2=[9,9,9]:
# 9+9=18→carry=1,node(8). 9+9+1=19→carry=1,node(9). 9+9+1=19→carry=1,node(9).
# 9+0+1=10→carry=1,node(0). 0+0+1=1→carry=0,node(1). → [8,9,9,0,1] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(max(m,n)): process as many digits as the longer number plus at most one
carry.
# Space O(max(m,n)+1): result list.
# Follow-up: Add Two Numbers II (LC 445) – digits stored in FORWARD order.
# Use two stacks to reverse, then apply same carry logic.
# Follow-up: multiply two numbers represented as strings (LC 43) – similar
positional logic."
```

◆ Quick Review

Key Insights

- The digits are stored in reverse order, meaning the first node is the least significant digit.
- A carry may need to propagate through subsequent nodes.
- The two linked lists might be of different lengths.
- Continue processing until both lists are fully traversed and any remaining carry is handled.

Space & Time

Time Complexity: $O(\max(n, m))$, where n and m are the lengths of the two linked lists.

Space Complexity: $O(\max(n, m) + 1)$ for the result list in the worst case.

Solution

The solution iterates through both linked lists node by node. For each pair of corresponding nodes, their values and any carry from the previous operation are summed. The result is decomposed into a current digit (modulo 10) and an updated carry (total divided by 10). A new node is created for the current digit and appended to the resulting linked list. This process continues until all nodes are

processed and any leftover carry is added as a final node.

Linked List

□ #19 Remove Nth Node From End of List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

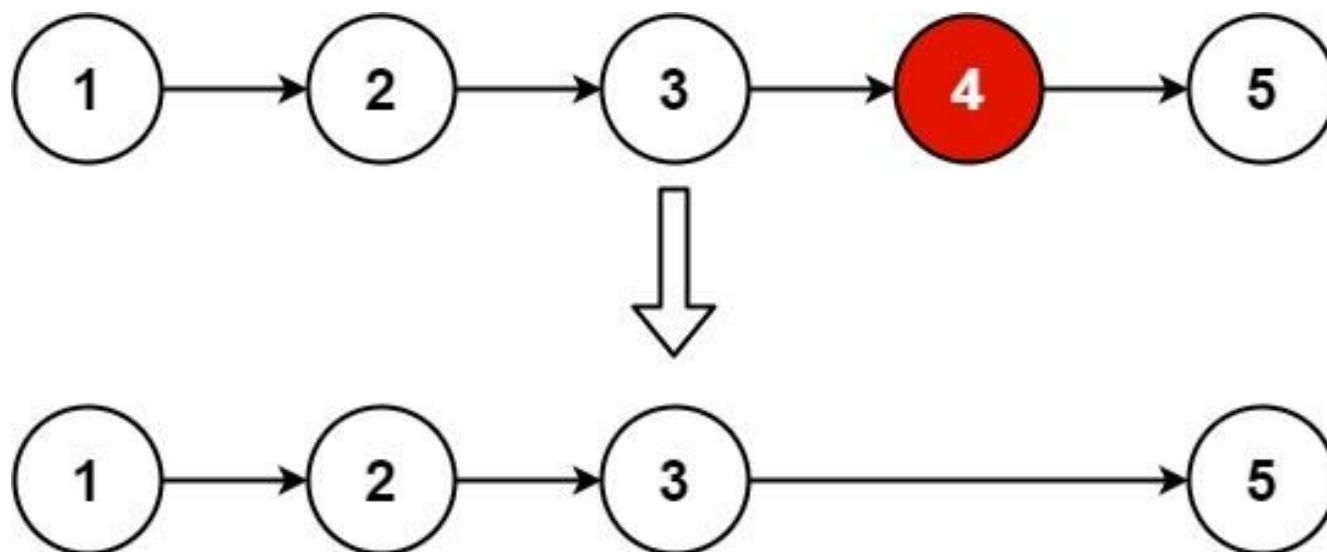
Linked List · Two Pointers

Lists: Grind 169

Problem

Given the head of a linked list, remove the n th node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2

Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- The number of nodes in the list is sz.
- $1 \leq sz \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq sz$

Follow up: Could you do this in one pass?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given the head of a linked list and integer n, remove the nth node from the END
# and return the modified head. n is always valid. Right?"
#
# "head=[1,2,3,4,5], n=2: 5th-from-end=5, 4th=4, 3rd=3, 2nd=4 → remove node(4) →
# [1,2,3,5] ✓
# head=[1], n=1: remove the only node → [] ✓
# head=[1,2], n=1: remove last → [1] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Two passes: first count the list length L, then remove node at position L-n.
# Use a dummy node to handle removing the head cleanly.
# Time: O(L). Space: O(1)."
```

```
class _19_RemoveNthFromEnd_Brute:
    def removeNthFromEnd(self, head, n: int):
        dummy = ListNode(0, head)
        length = 0
        curr = head
```



```

while curr:
    length += 1
    curr = curr.next
curr = dummy
for _ in range(length - n):
    curr = curr.next
curr.next = curr.next.next
return dummy.next

```

PHASE 3 — OPTIMIZE

"One pass using two pointers spaced n apart.

Advance fast n steps first, then move both slow and fast together.

When fast reaches the end, slow is just before the node to remove.

Dummy node handles the edge case of removing the head.

Time: $O(L)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```

class _19_RemoveNthFromEnd:
    def removeNthFromEnd(self, head, n: int):
        dummy = ListNode(0, head)
        slow = fast = dummy
        # Advance fast n+1 steps so slow lands before the target
        for _ in range(n + 1):
            fast = fast.next
        while fast:
            slow = slow.next
            fast = fast.next
        slow.next = slow.next.next
        return dummy.next

```

PHASE 5 — TEST & DEBUG

[1,2,3,4,5], n=2:

fast advances 3 steps: dummy→1→2→3. slow=dummy.

Move together: slow=1,fast=4 → slow=2,fast=5 → slow=3,fast=None → stop.

slow=node(3). slow.next=node(4). slow.next=node(5). → [1,2,3,5] ✓

#

[1], n=1:

fast advances 2 steps → None. while loop skipped. slow=dummy.

slow.next=node(1), slow.next=node(1).next=None → [] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(L)$: one pass for the two-pointer approach vs two passes for brute.

Space $O(1)$: constant extra pointers.

The dummy node is the key to cleanly handling head removal without special-casing.

Follow-up: if n could be invalid ($>$ list length), add a bounds check.

Follow-up: find the nth from end without removing (LC 876 variant) – same slow/fast idea."

◆ Quick Review

Key Insights

- Use a dummy node at the start to simplify edge cases, especially when the head is removed.
- Utilize two pointers (fast and slow) and maintain a gap of $n+1$ nodes between them.
- By moving both pointers simultaneously, the slow pointer will point to the node immediately before the target node when the fast pointer reaches the end.

Space & Time

Time Complexity: $O(L)$, where L is the length of the list

Space Complexity: $O(1)$

Solution

We use a two-pointer approach to achieve a one-pass solution. A dummy node is created and linked to the head to handle edge cases gracefully (such as needing to remove the head). The fast pointer is advanced $n+1$ steps ahead of the slow pointer to maintain the required gap. Then, both pointers are moved

simultaneously until the fast pointer reaches the end of the list. At that point, the slow pointer is immediately before the node to be deleted. We then adjust the pointers to remove the target node from the list.

Linked List

□ #24 Swap Nodes in Pairs

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Linked List · Recursion

Lists: Grind 169

Problem

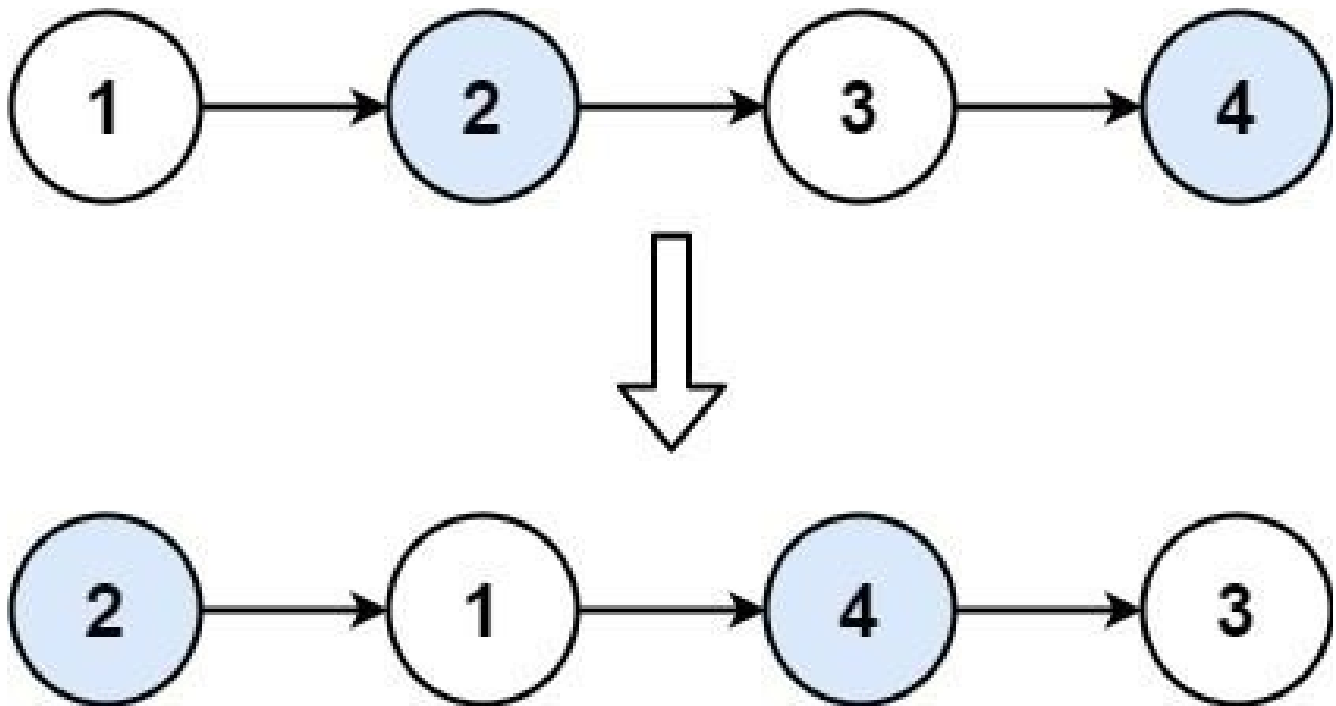
Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Explanation:



Example 2:

Input: head = []

Output: []

Example 3:

Input: head = [1]

Output: [1]

Example 4:

Input: head = [1,2,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the list is in the range [0, 100].
- $0 \leq \text{Node.val} \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a linked list, swap every two adjacent nodes and return the head.
# Must swap the nodes themselves, not just their values. Right?"
#
# "head=[1,2,3,4]: swap (1,2) and (3,4) → [2,1,4,3] ✓
# head=[1]: odd length, node 1 stays → [1] ✓
# head=[]: → [] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Collect all node values, swap pairs in the array, rebuild a new list.
# Time: O(n). Space: O(n) for the values array."
```

```
class _24_SwapNodesInPairs_Brute:
    def swapPairs(self, head):
        vals = []
        curr = head
        while curr:
            vals.append(curr.val)
            curr = curr.next
        # Swap pairs in-place
        for i in range(0, len(vals) - 1, 2):
            vals[i], vals[i + 1] = vals[i + 1], vals[i]
        dummy = cur = ListNode(0)
        for v in vals:
            cur.next = ListNode(v)
            cur = cur.next
        return dummy.next
```

PHASE 3 — OPTIMIZE

```
# "In-place pointer rewiring with a dummy predecessor node.
# For each pair (first, second): set prev.next=second, second.next=first,
# first.next=next_pair. Advance prev to first (now the second in the list).
# Time: O(n). Space: O(1) — no extra storage."
```

PHASE 4 — CLEAN CODE

```
class _24_SwapNodesInPairs:
    def swapPairs(self, head):
        dummy = ListNode(0, head)
        prev = dummy
        while prev.next and prev.next.next:
            first = prev.next
            second = prev.next.next
```

```

    # Rewire
    prev.next = second
    first.next = second.next
    second.next = first
    # Advance
    prev = first
    return dummy.next

```

PHASE 5 — TEST & DEBUG

```

# [1,2,3,4]: dummy→1→2→3→4
# prev=dummy, first=1, second=2:
# dummy.next=2, 1.next=3, 2.next=1 → dummy→2→1→3→4. prev=1.
# prev=1, first=3, second=4:
# 1.next=4, 3.next=None, 4.next=3 → dummy→2→1→4→3. prev=3.
# prev.next=None → stop. → [2,1,4,3] ✓
#
# [1]: prev.next.next=None → loop never enters. → [1] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): visit each node once. Space O(1): constant extra pointers.
# The dummy node is essential to handle swapping the first pair cleanly.
# Follow-up: Reverse Nodes in k-Group (LC 25) – generalise to groups of k.
# Follow-up: recursive solution is elegant but uses O(n/2) stack space."

```

◆ Quick Review

Key Insights

- Use a dummy node to simplify edge cases, such as when the list is empty or has only one node.
- Process nodes in pairs and update the pointers accordingly.
- Ensure that the swapping does not lose the connection between the swapped pair and the rest of the list.
- The solution can be implemented iteratively or recursively.

Space & Time

Time Complexity: $O(n)$ – Each node is processed once.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We use an iterative approach with a dummy node. The dummy node points to the head of the list. We maintain a pointer that traverses the list in pairs. For each pair, we adjust the next pointers to swap the two nodes. By the end, the dummy's next pointer will be the new head of the modified list. The main idea is to keep track of the previous node (prev) before the pair, and then adjust pointers accordingly using temporary variables to hold references to the nodes being swapped.

Linked List

□ #61 Rotate List

MED

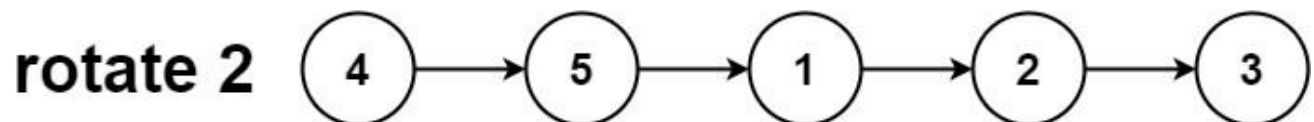
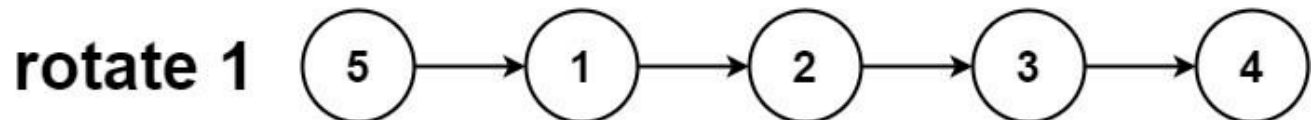
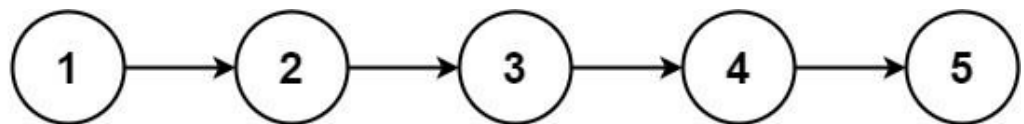
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers

Lists: Grind 169

Problem

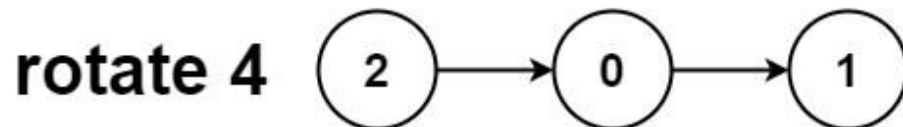
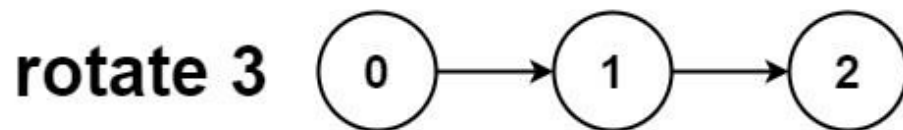
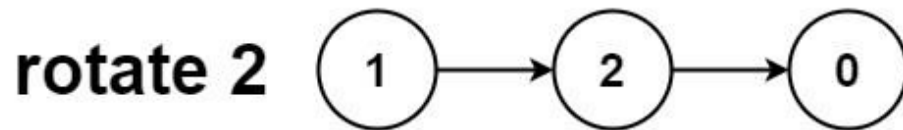
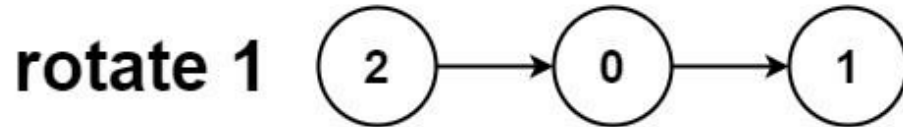
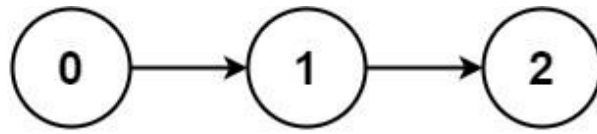
Given the head of a linked list, rotate the list to the right by k places.

Example 1:



Input: head = [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]

Example 2:



Input: head = [0,1,2], k = 4
Output: [2,0,1]

Constraints:

- The number of nodes in the list is in the range [0, 500].
- $-100 \leq \text{Node.val} \leq 100$
- $0 \leq k \leq 2 * 10^9$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rotate the linked list to the right by k places. Each rotation moves the last node

to the front. Return the new head. Right?"

#

"head=[1,2,3,4,5], k=2:

rotate 1: [5,1,2,3,4]. rotate 2: [4,5,1,2,3] ✓

head=[0,1,2], k=4: effective k = $4\%3 = 1 \rightarrow [2,0,1]$ ✓"

PHASE 2 — BRUTE FORCE

"Perform k individual rotations. Each rotation: traverse to second-to-last node, take the last node, prepend it to the front.

Time: $O(n*k)$ – can be very slow if k is large. Space: $O(1)$."

```
class _61_RotateList_Brute:
```

```
    def rotateRight(self, head, k: int):
```

```
        if not head or not head.next or k == 0:
```

```
            return head
```

```
        for _ in range(k):
```

```
            curr = head
```

```
            while curr.next.next: # find second-to-last
```

```
                curr = curr.next
```

```
            last = curr.next
```

```
            curr.next = None
```

```
            last.next = head
```

```
            head = last
```

```
        return head
```

PHASE 3 — OPTIMIZE

"Three steps, one pass to find length:

1. Find length L and tail node by traversing once.

2. Effective rotation = $k \% L$ ($k \geq L$ means full cycles, no change).

3. New tail is at position $L - (k\%L) - 1$. New head is new_tail.next.

Make the list circular, then cut at the right spot.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _61_RotateList:
```

```
    def rotateRight(self, head, k: int):
```

```
        if not head or not head.next or k == 0:
```

```
            return head
```

```
        # Step 1: find length and tail
```

```
        tail = head
```

```
        length = 1
```

```
        while tail.next:
```

```
            tail = tail.next
```

```

        length += 1
    # Step 2: effective rotation
    k = k % length
    if k == 0:
        return head
    # Step 3: find new tail at position (length - k - 1)
    new_tail = head
    for _ in range(length - k - 1):
        new_tail = new_tail.next
    new_head = new_tail.next
    new_tail.next = None
    tail.next = head
    return new_head

```

PHASE 5 — TEST & DEBUG

```

# [1,2,3,4,5], k=2:
# length=5, tail=node(5). k=2%5=2. new_tail at index 5-2-1=2 → node(3).
# new_head=node(4). node(3).next=None. node(5).next=node(1).
# → [4,5,1,2,3] ✓
#
# [0,1,2], k=4:
# length=3, k=4%3=1. new_tail at index 3-1-1=1 → node(1).
# new_head=node(2). node(1).next=None. node(2_orig_tail).next=node(0).
# → [2,0,1] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): one pass to find length + one pass to find cut point.
# Space O(1): only pointer variables.
# The k%length shortcut is critical – without it, k=10^9 on a 5-node list would TLE.
# Follow-up: rotate array (LC 189) – same modulo idea, different reversal technique.
# Follow-up: rotate left by k – equivalent to rotating right by (length - k%length)."
```

◆ Quick Review

Key Insights

- Calculate the length of the list and locate the tail.
- Use $k \bmod \text{length}$ to handle rotations greater than the list size.

- Find the new tail of the list at position $(\text{length} - k - 1)$ and update pointers accordingly.
- Link the original tail to the original head to form a circular list, then break the circle at the new tail.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the list, since we traverse the list a few times.

Space Complexity: $O(1)$, as the rotation is done in place without extra data structures.

Solution

We first traverse the list to determine its length and identify the tail node. The effective rotation is calculated by taking k modulo the length of the list. If the result is zero, the list remains unchanged. Otherwise, we locate the new tail node by moving $(\text{length} - k - 1)$ steps from the head. The node after the new tail becomes the new head. Finally, we break the link after the new tail and connect the old tail to the original head to finalize the rotated list.

Linked List

□ #146 LRU Cache

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design
Lists: Grind 169

Problem

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

- **LRUCache(int capacity)** Initialize the LRU cache with positive size capacity.
- **int get(int key)** Return the value of the key if the key exists, otherwise return -1.
- **void put(int key, int value)** Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.

The functions **get** and **put** must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, null, -1, 3, 4]
```

Explanation

```
LRUCache lRUCache = new LRUCache(2);
```

Constraints:

- $1 \leq \text{capacity} \leq 3000$
- $0 \leq \text{key} \leq 104$
- $0 \leq \text{value} \leq 105$
- At most $2 * 105$ calls will be made to get and put.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design an LRU Cache with a fixed capacity. Support get(key) → value or -1,
# and put(key, value) which evicts the least recently used item when over capacity.
# Both ops must be O(1) average. Right?"
```

```
#
```

```
# "LRUCache(2): put(1,1), put(2,2), get(1)→1, put(3,3) evicts key 2,
# get(2)→-1, put(4,4) evicts key 1, get(1)→-1, get(3)→3, get(4)→4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Use an ordered dict (Python's collections.OrderedDict) to track insertion order.
# On get/put, move the key to the end (most recent). On eviction, pop from front.
# Time: O(1) average. Space: O(capacity). – This is actually optimal but hides
internals."
```

```
class _146_LRUCache_Brute:
    def __init__(self, capacity: int):
```

```

    from collections import OrderedDict
    self.cap = capacity
    self.cache = OrderedDict()
def get(self, key: int) -> int:
    if key not in self.cache:
        return -1
    self.cache.move_to_end(key)
    return self.cache[key]
def put(self, key: int, value: int) -> None:
    if key in self.cache:
        self.cache.move_to_end(key)
    self.cache[key] = value
    if len(self.cache) > self.cap:
        self.cache.popitem(last=False)

```

PHASE 3 — OPTIMIZE

"Build the internals explicitly: HashMap + Doubly Linked List.

HashMap: key → node (O(1) lookup).

DLL: maintains access order — head=LRU, tail=MRU.

get: move node to tail. put: add at tail; if over capacity, remove from head.

All ops O(1) with sentinel head/tail nodes to avoid edge cases."

PHASE 4 — CLEAN CODE

```

class _146_LRUCache:
    class _Node:
        def __init__(self, key=0, val=0):
            self.key = key
            self.val = val
            self.prev = self.next = None
    def __init__(self, capacity: int):
        self.cap = capacity
        self.cache = {}
        self.head = self._Node() # sentinel LRU end
        self.tail = self._Node() # sentinel MRU end
        self.head.next = self.tail
        self.tail.prev = self.head
    def _remove(self, node) -> None:
        node.prev.next = node.next
        node.next.prev = node.prev
    def _insert_tail(self, node) -> None:
        node.prev = self.tail.prev
        node.next = self.tail
        self.tail.prev.next = node
        self.tail.prev = node
    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        node = self.cache[key]

```



```

        self._remove(node)
        self._insert_tail(node)
        return node.val

def put(self, key: int, value: int) -> None:
    if key in self.cache:
        self._remove(self.cache[key])
    node = self._Node(key, value)
    self._insert_tail(node)
    self.cache[key] = node
    if len(self.cache) > self.cap:
        lru = self.head.next
        self._remove(lru)
        del self.cache[lru.key]

```

PHASE 5 — TEST & DEBUG

```

# capacity=2: put(1,1)→tail:[1]. put(2,2)→tail:[1,2]. get(1)→move 1 to tail:[2,1].
# put(3,3): over cap, evict head.next=node(2). cache={1,3}. tail:[1,3]. ✓
# get(2)→-1 (evicted) ✓. put(4,4): evict node(1). tail:[3,4].
# get(1)→-1 ✓, get(3)→3 ✓, get(4)→4 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(1) all ops: HashMap gives O(1) lookup, DLL gives O(1) insert/remove.
# Space O(capacity): at most capacity nodes in cache.
# Follow-up: LRU Cache (LC 460) – evict least FREQUENTLY used; needs two hashmaps + DLL.
# Follow-up: thread-safe LRU – wrap get/put with a mutex lock."

```

◆ Quick Review

Key Insights

- Use a Hash Table (dictionary) to allow $O(1)$ access to nodes by key.
- Use a Doubly Linked List to record the usage order. The head represents the most recently used element while the tail represents the least recently used.
- When a key is accessed or updated, move the corresponding node to the front (head) of the

list.

- When the cache exceeds capacity, remove the node at the tail end which is the least recently used entry.

Space & Time

Time Complexity: $O(1)$ per get and put operation.

Space Complexity: $O(\text{capacity})$, storing key-node pairs and linked list nodes.

Solution

The solution combines a hash table (or dictionary) and a doubly linked list: The hash table stores keys and corresponding node addresses. The doubly linked list maintains the order of usage with the most recently used items near the head. For `get(key)`, check the dictionary. If found, move the node to the head and return its value; if not, return `-1`. For `put(key, value)`, if the key exists, update its value and move it to the head. If not, create a new node and add it right after the head. If the capacity is reached, remove the tail node (least recently used) and update the dictionary accordingly. Dummy head and tail nodes are

used to simplify node addition and removal procedures without worrying about edge cases.

Linked List

□ #148 Sort List

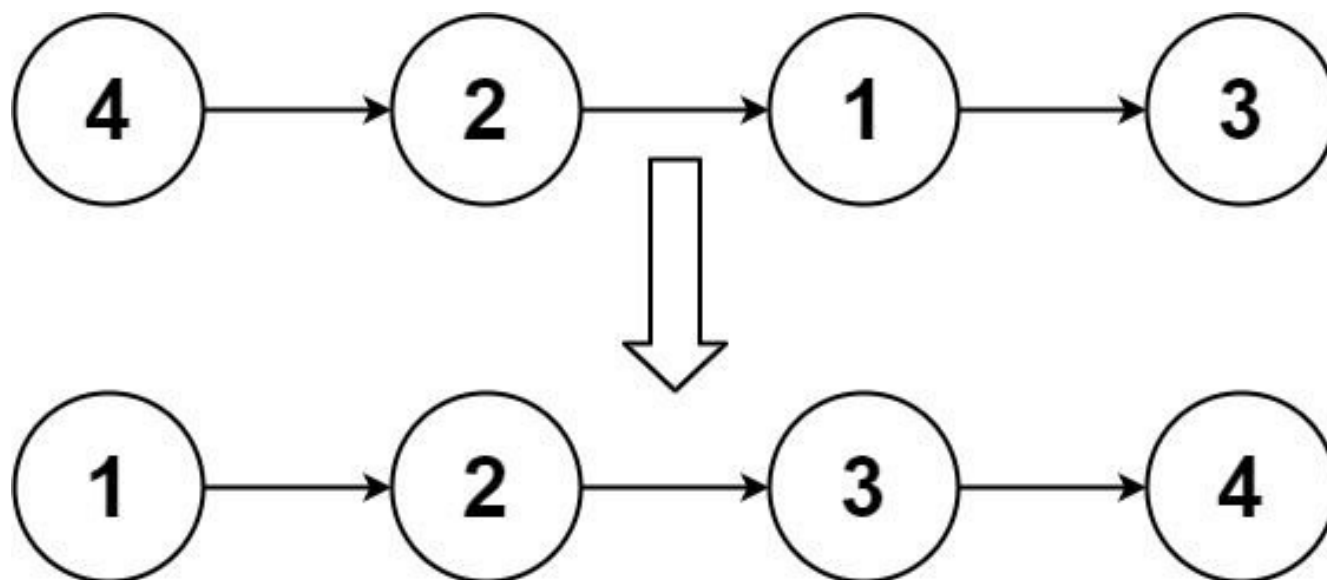
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers · Divide and Conquer
· Sorting · Merge Sort
Lists: Grind 169

Problem

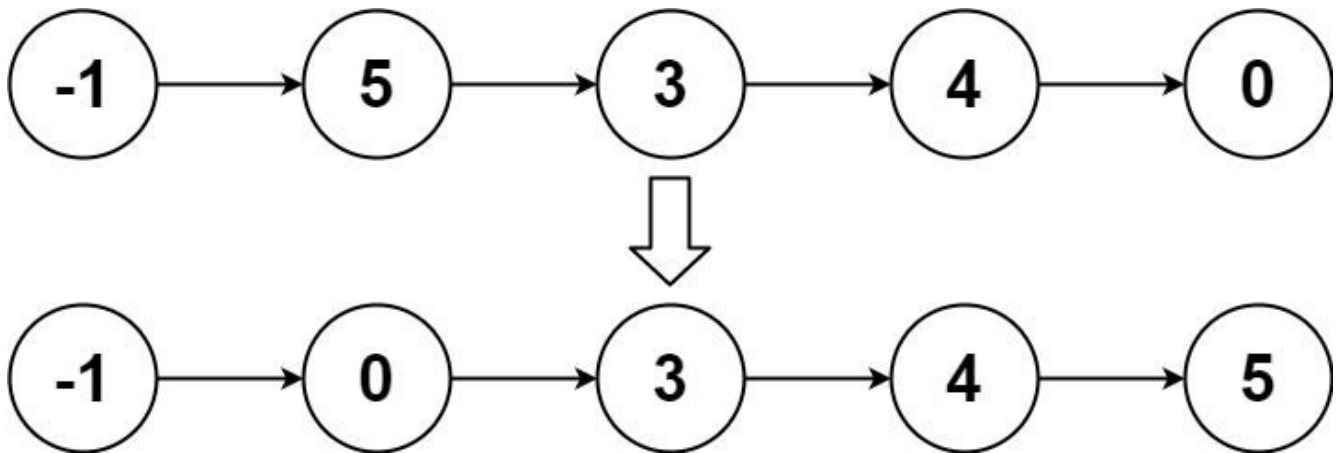
Given the head of a linked list, return the list after sorting it in ascending order.

Example 1:



Input: head = [4,2,1,3]
Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]

Output: [-1,0,3,4,5]

Example 3:

Input: head = []

Output: []

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-105 \leq \text{Node.val} \leq 105$

Follow up: Can you sort the linked list in $O(n \log n)$ time and $O(1)$ memory (i.e. constant space)?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Sort a linked list in ascending order. $O(n \log n)$ time, $O(1)$ space preferred. Right?"

```

#
# "head=[4,2,1,3]: → [1,2,3,4] ✓
# head=[-1,5,3,4,0]: → [-1,0,3,4,5] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all values into an array, sort, overwrite node values.
# Time:  $O(n \log n)$ . Space:  $O(n)$  for the values array."
class _148_SortList_Brute:
    def sortList(self, head):
        vals = []
        curr = head
        while curr:
            vals.append(curr.val)
            curr = curr.next
        vals.sort()
        curr = head
        for v in vals:
            curr.val = v
            curr = curr.next
        return head

# PHASE 3 — OPTIMIZE
# "Bottom-up merge sort —  $O(n \log n)$  time,  $O(1)$  space (no recursion stack).
# Merge pairs of sublists of size 1, then 2, then 4... until the whole list is sorted.
# Each pass merges  $n/\text{size}$  pairs.  $\log n$  passes total.
# A dummy head simplifies relinking at the start of each merged sublist."

# PHASE 4 — CLEAN CODE
class _148_SortList:
    def sortList(self, head):
        def get_length(node):
            length = 0
            while node:
                length += 1
                node = node.next
            return length

        def split(node, size):
            for _ in range(size - 1):
                if node: node = node.next
            if not node: return None
            rest = node.next
            node.next = None
            return rest

        def merge(l1, l2):
            dummy = tail = ListNode(0)
            while l1 and l2:
                if l1.val <= l2.val:
                    tail.next = l1; l1 = l1.next
                else:

```

```

        tail.next = l2; l2 = l2.next
        tail = tail.next
        tail.next = l1 or l2
        return dummy.next

length = get_length(head)
dummy = ListNode(0, head)
size = 1
while size < length:
    curr = dummy.next
    tail = dummy
    while curr:
        left = curr
        right = split(left, size)
        curr = split(right, size)
        merged = merge(left, right)
        tail.next = merged
        while tail.next: tail = tail.next
    size *= 2
return dummy.next

```

PHASE 5 — TEST & DEBUG

```

# [4,2,1,3], size=1:
# merge(4,2)→[2,4], merge(1,3)→[1,3]. List: 2→4→1→3.
# size=2: merge([2,4],[1,3])→[1,2,3,4] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n \log n)$ :  $\log n$  passes, each  $O(n)$ . Space  $O(1)$ : bottom-up avoids recursion stack.
# Top-down merge sort is simpler to write but uses  $O(\log n)$  stack space.
# Follow-up: Merge k Sorted Lists (LC 23) – use a min-heap for  $O(n \log k)$ .
# Follow-up: insertion sort for linked list (LC 147) is  $O(n^2)$  but stable and in-place."

```

◆ Quick Review

Key Insights

- Merge sort is well-suited for linked lists; it naturally relies on splitting the list and merging sorted sublists.
- The slow and fast pointer technique is used to find the middle point of the list.

- A recursive merge sort implementation may use $O(\log n)$ space for recursion stack. To achieve $O(1)$ extra space, an iterative bottom-up merge sort approach would be preferred.
- Merging two sorted lists is efficient with linked lists because of their sequential access.

Space & Time

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$ extra space for an iterative bottom-up approach ($O(\log n)$ for recursive calls if using recursion)

Solution

The solution employs the merge sort algorithm tailored for linked lists. The overall steps are: Use the slow and fast pointers to find the middle of the list and split it into two halves. Recursively sort the two halves. Merge the two sorted halves by comparing node values. Ensure that during merging, pointers are adjusted properly to build the sorted list. For an $O(1)$ space solution, an iterative bottom-up merge sort can be implemented. However, the recursive merge sort is easier to

understand and implement, though it uses $O(\log n)$ space on the call stack.

Linked List

□ #328 Odd Even Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: Grind 169

Problem

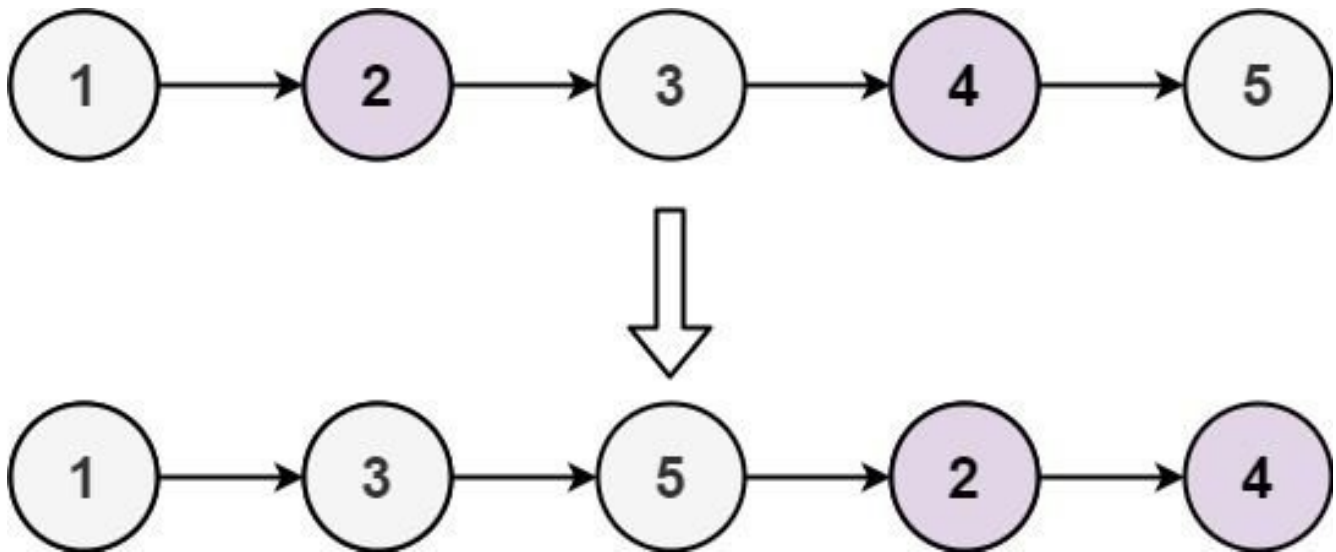
Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.

The first node is considered odd, and the second node is even, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

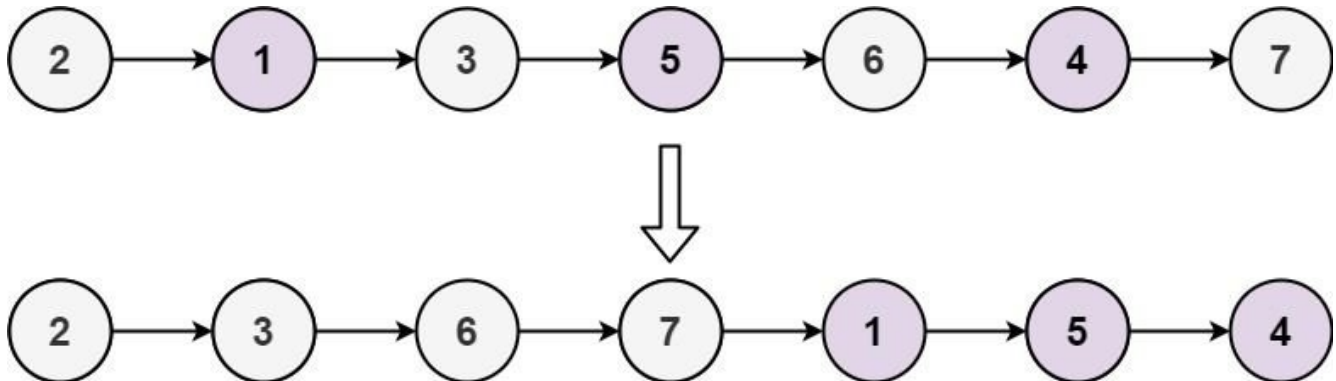
You must solve the problem in $O(1)$ extra space complexity and $O(n)$ time complexity.

Example 1:



Input: head = [1,2,3,4,5]
Output: [1,3,5,2,4]

Example 2:



Input: head = [2,1,3,5,6,4,7]
Output: [2,3,6,7,1,5,4]

Constraints:

- The number of nodes in the linked list is in the range [0, 104].
- $-106 \leq \text{Node.val} \leq 106$

My LeetCode Solution
Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Regroup a linked list so all nodes at odd positions come first, then all nodes
# at even positions. Preserve relative order within each group.
# Positions are 1-indexed. Right?"
#
# "head=[1,2,3,4,5]: odds=[1,3,5], evens=[2,4] → [1,3,5,2,4] ✓
# head=[2,1,3,5,6,4,7]: → [2,3,6,7,1,5,4] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Collect odd-position and even-position values separately, rebuild the list.
# Time: O(n). Space: O(n) for the two value lists."
```

```
class _328_OddEvenLinkedList_Brute:
    def oddEvenList(self, head):
        odds, evens = [], []
        curr, pos = head, 1
        while curr:
            (odds if pos % 2 == 1 else evens).append(curr.val)
            curr = curr.next
            pos += 1
        dummy = cur = ListNode(0)
        for v in odds + evens:
            cur.next = ListNode(v); cur = cur.next
        return dummy.next
```

PHASE 3 — OPTIMIZE

```
# "In-place pointer manipulation with two sub-list heads: odd_head and even_head.
# Walk through: odd.next = odd.next.next, even.next = even.next.next.
# At the end, link odd tail to even head.
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _328_OddEvenLinkedList:
    def oddEvenList(self, head):
        if not head or not head.next:
            return head
        odd = head
        even = head.next
        even_head = even
        while even and even.next:
            odd.next = even.next
            odd = odd.next
            even.next = odd.next
            even = even.next
        odd.next = even_head
        return head
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3,4,5]:
# odd=1, even=2, even_head=2.
# iter1: odd.next=3,odd=3. even.next=4,even=4.
# iter2: odd.next=5,odd=5. even.next=None,even=None.
# odd.next=even_head=2. → 1→3→5→2→4→None ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single pass. Space O(1): only pointer variables, no extra lists.
# even_head must be saved before the loop since we modify even's next pointer.
# Follow-up: regroup by value instead of position – partition around a pivot (LC 86).
# Follow-up: if 1-indexed is changed to 0-indexed, swap the initial odd/even assignments."
```

◆ Quick Review

Key Insights

- Use two pointers: one for odd-indexed nodes and one for even-indexed nodes.
- The first node is considered odd, and the second node is even.
- Traverse the list once, reassigning the next pointers to segregate odd and even nodes.
- Finally, attach the even list at the end of the odd list.
- The solution must use $O(1)$ extra space and $O(n)$ time complexity.

Space & Time

Time Complexity: $O(n)$ since the list is traversed once.

Space Complexity: $O(1)$ as only a few pointers are used.

Solution

Maintain two pointers, odd and even, starting from the head and head.next, respectively. As you iterate, update odd.next to point to the next odd element (skipping an even node) and even.next to point to the next even element. Once the end is reached, attach the even list to the end of the odd list. This in-place rearrangement preserves the initial ordering within odd and even groups while meeting the space and time constraints.

Linked List

★ #369 Plus One Linked List ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Math · Recursion
Lists: ★ Premium | Premium 98

Problem

Given a non-negative integer represented as a linked list of digits, plus one to the integer.

The digits are stored such that the most significant digit is at the head of the list.

Example 1:

Input: head = [1,2,3]
Output: [1,2,4]

Example 2:

Input: head = [0]
Output: [1]

Constraints:

- The number of nodes in the linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$
- The number represented by the linked list does not contain leading zeros except for the

zero itself.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A non-negative integer is represented as a linked list of digits (most significant first).

Add one to the number and return the updated list. Right?"

#

"head=[1,2,3]: 123+1=124 → [1,2,4] ✓

head=[9,9,9]: 999+1=1000 → [1,0,0,0] ✓"

PHASE 2 — BRUTE FORCE

"Convert the linked list to an integer, add 1, convert back to a linked list.

Time: O(n). Space: O(n) for the result."

```
class _369_PlusOneLinkedList_Brute:
```

```
    def plusOne(self, head):
```

```
        # Convert to int
```

```
        num = 0
```

```
        curr = head
```

```
        while curr:
```

```
            num = num * 10 + curr.val
```

```
            curr = curr.next
```

```
        num += 1
```

```
        # Convert back
```

```
        digits = []
```

```
        while num:
```

```
            digits.append(num % 10)
```

```
            num //= 10
```

```
        dummy = cur = ListNode(0)
```

```
        for d in reversed(digits):
```

```
            cur.next = ListNode(d); cur = cur.next
```

```
        return dummy.next
```

PHASE 3 — OPTIMIZE

"Find the rightmost non-9 digit, increment it, set all digits after it to 0.

If all digits are 9, prepend a new node with value 1 and set all existing to 0.

Use a sentinel dummy node prepended to handle the all-9s case cleanly.

Time: O(n). Space: O(1) – in-place modification."

PHASE 4 — CLEAN CODE


```

class _369_PlusOneLinkedList:
    def plusOne(self, head):
        dummy = ListNode(0, head)
        not_nine = dummy # track rightmost non-9 node
        curr = head
        while curr:
            if curr.val != 9:
                not_nine = curr
                curr = curr.next
        # Increment the rightmost non-9 node
        not_nine.val += 1
        # Set all nodes after it to 0
        curr = not_nine.next
        while curr:
            curr.val = 0
            curr = curr.next
        return dummy if dummy.val == 1 else dummy.next

```

PHASE 5 — TEST & DEBUG

[1,2,3]: not_nine ends at node(3). node(3).val=4. nothing after. return dummy.next=[1,2,4] ✓

#

[9,9,9]: not_nine stays at dummy(0). dummy.val=0+1=1. set 9→0,9→0,9→0.

dummy.val=1 → return dummy → [1,0,0,0] ✓

#

[1,2,9]: not_nine=node(2). node(2).val=3. node(9).val=0. → [1,3,0] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: two passes (find rightmost non-9, then zero out).

Space $O(1)$: modify nodes in-place.

The dummy node trick elegantly handles the all-9s case without a special branch.

Follow-up: subtract one from a linked list – find rightmost non-0, decrement, set rest to 9.

Follow-up: add two numbers as linked lists (LC 2) – most-significant-first version."

◆ Quick Review

Key Insights

- The digits are stored such that the most significant digit comes first, but addition is easier starting from the least significant digit.

- Reversing the list allows simpler management of the carry during addition.
- After processing the addition, reversing the list back restores the original order.
- Consider special cases where all digits are 9, requiring a new head node to accommodate an additional digit.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the list

Space Complexity: $O(1)$ extra space when using an iterative reversal method. ($O(n)$ if recursion is used)

Solution

We solve the problem by first reversing the linked list, which converts it to a format where the least significant digit is processed first. We then iterate over the reversed list, adding one and managing any carry. If we encounter a carry at the end of the list, a new node is appended to handle the overflow. Finally, we reverse the list again to restore the original order. This method efficiently handles the addition with constant extra space during

processing.

Linked List

□ ★ #430 Flatten a Multilevel Doubly Linked List ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · DFS · Doubly Linked List
Lists: ★ Premium | Premium 98

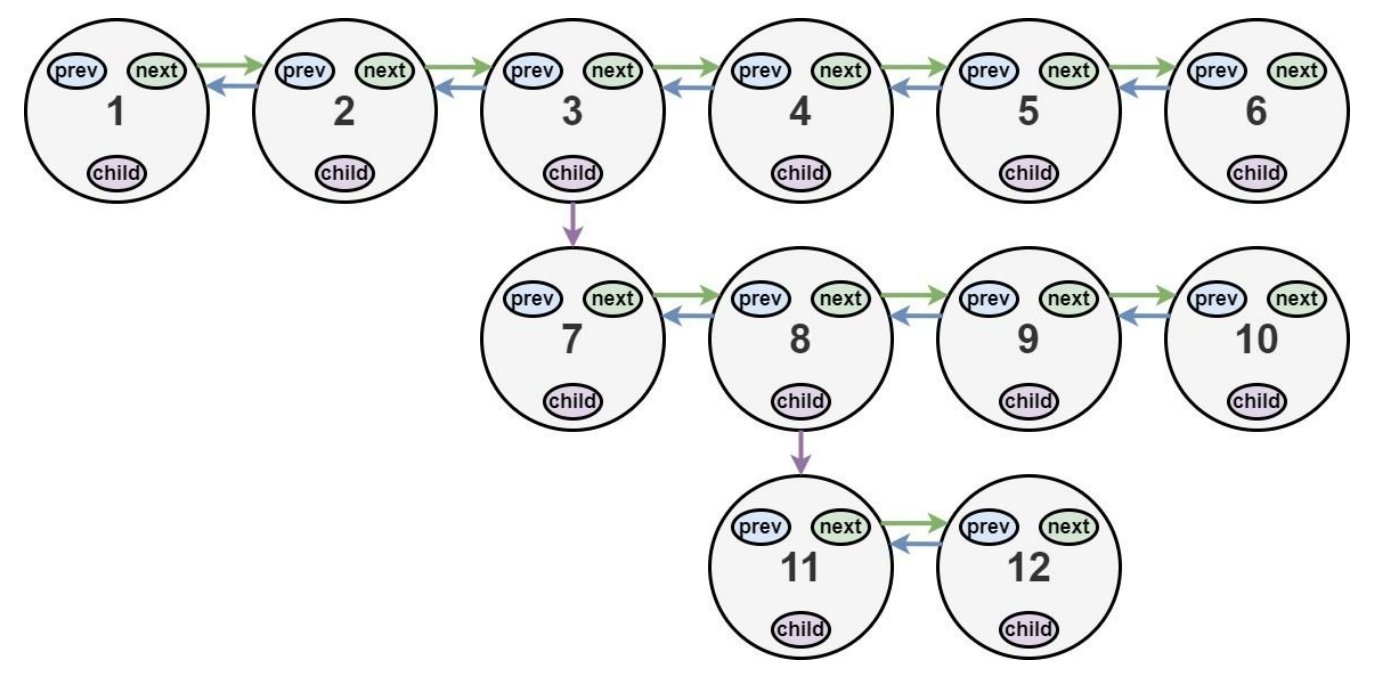
Problem

You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.

Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let *curr* be a node with a child list. The nodes in the child list should appear after *curr* and before *curr.next* in the flattened list.

Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

Example 1:

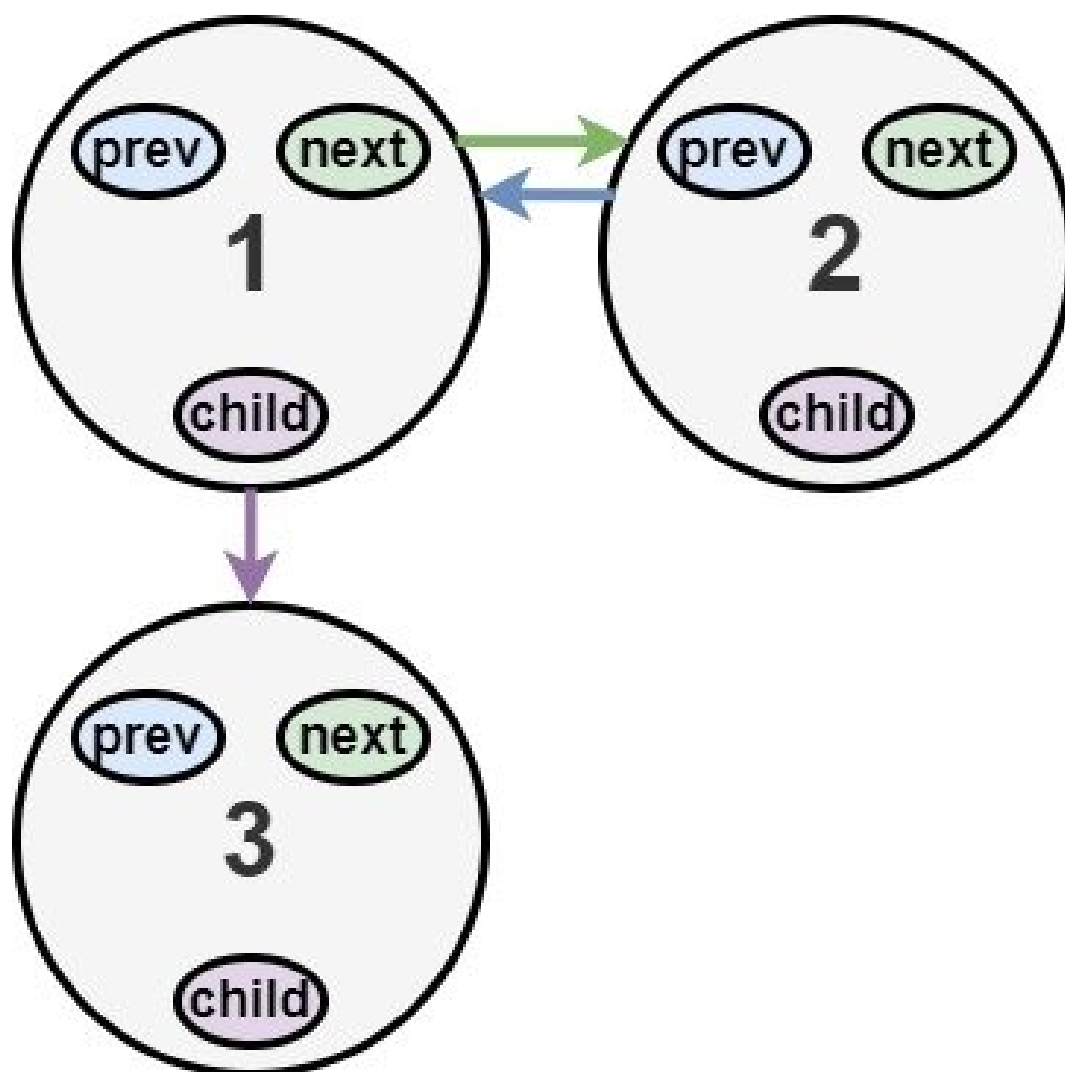


Input: head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]

Output: [1,2,3,7,8,11,12,9,10,4,5,6]

Explanation: The multilevel linked list in the input is shown.
After flattening the multilevel linked list it becomes:

Example 2:



Input: head = [1,2,null,3]

Output: [1,3,2]

Explanation: The multilevel linked list in the input is shown.
After flattening the multilevel linked list it becomes:

Example 3:

Input: head = []

Output: []

Explanation: There could be empty list in the input.

Constraints:

- The number of Nodes will not exceed 1000.
- $1 \leq \text{Node.val} \leq 105$

How the multilevel linked list is represented in test cases:

We use the multilevel linked list from Example 1 above:

```
1---2---3---4---5---6--NULL
|
7---8---9---10--NULL
|
11--12--NULL
```

The serialization of each level is as follows:

```
[1,2,3,4,5,6,null]
[7,8,9,10,null]
[11,12,null]
```

To serialize all levels together, we will add nulls in each level to signify no node connects to the upper node of the previous level. The serialization becomes:

```
[1, 2, 3, 4, 5, 6, null]
|
[null, null, 7, 8, 9, 10, null]
|
[ null, 11, 12, null]
```

Merging the serialization of each level and removing trailing nulls we obtain:

```
[1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
```

My LeetCode Solution
Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A doubly linked list where nodes can have a child pointer to another doubly linked list.

Flatten it so all nodes appear in a single-level DLL in pre-order (node before its children).

Return the head. Right?"

#

"1-2-3-4-5-6, node 3 has child 7-8-9-10, node 8 has child 11-12:

Flattened: 1-2-3-7-8-11-12-9-10-4-5-6 ✓"

PHASE 2 — BRUTE FORCE

"Collect nodes in pre-order DFS (visit node, recurse into child, continue next), # then relink them all as a flat doubly linked list.

Time: $O(n)$. Space: $O(n)$ for the node list."

```
class _430_FlattenDLL_Brute:
    def flatten(self, head):
        order = []
        def dfs(node):
            while node:
                order.append(node)
                if node.child:
                    dfs(node.child)
                node = node.next
        dfs(head)
        for i in range(len(order) - 1):
            order[i].next = order[i+1]
            order[i+1].prev = order[i]
            order[i].child = None
        if order:
            order[-1].next = None
            order[0].prev = None
        return order[0] if order else None
```

PHASE 3 — OPTIMIZE

"In-place pointer manipulation with a stack.

When we encounter a child: push current next onto stack, insert child list inline.

When next is None and stack is non-empty: pop to resume the saved tail.

Time: $O(n)$. Space: $O(d)$ where d = max nesting depth."

PHASE 4 — CLEAN CODE

```
class _430_FlattenDLL:
    def flatten(self, head):
        if not head:
            return head
        stack = []
        curr = head
        while curr:
```



```

    if curr.child:
        if curr.next:
            stack.append(curr.next)
            curr.next = curr.child
            curr.child.prev = curr
            curr.child = None
        if not curr.next and stack:
            nxt = stack.pop()
            curr.next = nxt
            nxt.prev = curr
        curr = curr.next
    return head

```

PHASE 5 — TEST & DEBUG

```

# 1→2→3(child:7→8(child:11→12)→9→10)→4→5→6:
# curr=3: has child. push next=4. 3.next=7, 7.prev=3. curr→4 via 7.
# curr=8: has child. push next=9. 8.next=11, 11.prev=8. curr→11.
# curr=12: no next, pop 9. 12.next=9, 9.prev=12. curr→9.
# curr=10: no next, pop 4. 10.next=4, 4.prev=10. continue → 4,5,6.
# Result: 1-2-3-7-8-11-12-9-10-4-5-6 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): each node visited once. Space O(d): stack depth = nesting depth.
# The stack-based approach is essentially iterative pre-order DFS.
# Follow-up: unflatten the list back to multilevel – requires storing child
information.
# Follow-up: flatten with BFS order instead of DFS – use a queue instead of a
stack."

```

◆ Quick Review

Key Insights

- Use a depth-first traversal to process child lists before proceeding to the next pointer.
- When a node with a child is found, recursively flatten the child list.
- Splice the flattened child list in between the current node and the next node.

- Maintain previous and next pointers correctly to preserve the doubly linked list properties.
- Ensure that all child pointers are set to null after flattening.

Space & Time

Time Complexity: $O(n)$, where n is the total number of nodes, since each node is processed exactly once.

Space Complexity: $O(n)$ in the worst-case due to recursion stack in case of a highly nested list.

Solution

We can solve this problem using a recursive depth-first search (DFS) approach. The main idea is to traverse the list and, whenever a node with a child pointer is encountered, recursively flatten its child list. Once the child list is flattened, it can be spliced into the main list between the current node and the node originally following it. We then continue processing the next pointer. This ensures that the entire list is flattened in depth-first order. The recursive helper function returns the tail

of the flattened segment, which is used to connect the subsequent parts of the list correctly.

Linked List

□ #622 Design Circular Queue

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Linked List · Design · Queue
Lists: Denny Zhang

Problem

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

- **MyCircularQueue(k)** Initializes the object with the size of the queue to be k.
- **int Front()** Gets the front item from the queue. If the queue is empty, return -1.
- **int Rear()** Gets the last item from the queue. If the queue is empty, return -1.
- **boolean enQueue(int value)** Inserts an element into the circular queue. Return true if the operation is successful.
- **boolean deQueue()** Deletes an element from the circular queue. Return true if the operation is successful.
- **boolean isEmpty()** Checks whether the circular queue is empty or not.
- **boolean isFull()** Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:

Input

```
["MyCircularQueue", "enqueue", "enqueue", "enqueue", "enqueue", "Rear",
"isFull", "dequeue", "enqueue", "Rear"]
[[3], [1], [2], [3], [4], [], [], [], [4], []]
```

Output

```
[null, true, true, true, false, 3, true, true, true, 4]
```

Explanation

```
MyCircularQueue myCircularQueue = new MyCircularQueue(3);
```

Constraints:

- $1 \leq k \leq 1000$
- $0 \leq \text{value} \leq 1000$
- At most 3000 calls will be made to enqueue, dequeue, front, rear, isEmpty, and isFull.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design a circular queue (ring buffer) with fixed capacity.
```

```
# Support enqueue(val)→bool, dequeue()→bool, front()→int, rear()→int,
```

```
# isEmpty()→bool, isFull()→bool. Right?"
```

```
#
```

```
# "MyCircularQueue(3): enqueue(1)→T, enqueue(2)→T, enqueue(3)→T, enqueue(4)→F
(full),
```

```
# rear()→3, isFull()→T, dequeue()→T, enqueue(4)→T, rear()→4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Use a Python list as a queue with append/pop(0). Cap size at capacity.
```

```
# Time: O(n) for dequeue (pop from front shifts all elements). Space: O(k)."
```

```
class _622_MyCircularQueue_Brute:
```

```
    def __init__(self, k: int):
```

```
        self.data = []
```

```
        self.cap = k
```

```
    def enqueue(self, value: int) -> bool:
```

```
        if len(self.data) == self.cap: return False
```

```

        self.data.append(value); return True
def dequeue(self) -> bool:
    if not self.data: return False
    self.data.pop(0); return True
def Front(self) -> int:
    return self.data[0] if self.data else -1
def Rear(self) -> int:
    return self.data[-1] if self.data else -1
def isEmpty(self) -> bool: return not self.data
def isFull(self) -> bool: return len(self.data) == self.cap

```

PHASE 3 — OPTIMIZE

```

# "Fixed array with head and tail pointers, modular arithmetic.
# head = index of front element. tail = index of next empty slot.
# size tracks how many elements are in the queue.
# enqueue: data[tail] = val; tail = (tail+1)%k; size++.
# dequeue: head = (head+1)%k; size--. All ops O(1)."

```

PHASE 4 — CLEAN CODE

```

class _622_MyCircularQueue:
    def __init__(self, k: int):
        self.data = [0] * k
        self.head = 0
        self.tail = 0
        self.size = 0
        self.cap = k

    def enqueue(self, value: int) -> bool:
        if self.isFull(): return False
        self.data[self.tail] = value
        self.tail = (self.tail + 1) % self.cap
        self.size += 1
        return True

    def dequeue(self) -> bool:
        if self.isEmpty(): return False
        self.head = (self.head + 1) % self.cap
        self.size -= 1
        return True

    def Front(self) -> int:
        return -1 if self.isEmpty() else self.data[self.head]

    def Rear(self) -> int:
        return -1 if self.isEmpty() else self.data[(self.tail - 1) % self.cap]

    def isEmpty(self) -> bool: return self.size == 0
    def isFull(self) -> bool: return self.size == self.cap

```

PHASE 5 — TEST & DEBUG

```

# k=3: enqueue(1)→tail=1,size=1. enqueue(2)→tail=2,size=2.
enqueue(3)→tail=0,size=3.
# isFull()→True ✓. enqueue(4)→False ✓. Rear()→data[(0-1)%3]=data[2]=3 ✓.

```

```
# deQueue()→head=1,size=2. enqueue(4)→data[0]=4,tail=1,size=3. Rear()→data[0]=4 ✓.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "All ops O(1): array indexing + modular arithmetic.
```

```
# Space O(k): fixed array of size k.
```

```
# The modulo trick wraps tail/head around the array – the 'circular' part.
```

```
# Follow-up: Design Circular Deque (LC 641) – add enqueueFront/dequeueRear with same technique.
```

```
# Follow-up: thread-safe circular buffer – add mutex around enqueue/dequeue."
```

◆ Quick Review

Key Insights

- Leverage a fixed-size array to store the queue elements.
- Use two pointers: one for the front and one for the rear.
- Maintain a count variable (or use pointer arithmetic) to differentiate between an empty and a full queue.
- Use modulo arithmetic to wrap pointers when they reach the end of the array.
- Ensure all operations (enqueue, dequeue, Front, Rear, isEmpty, isFull) have $O(1)$ time complexity.

Space & Time

Time Complexity: $O(1)$ per operation for enqueue, dequeue, Front, Rear, isEmpty, and isFull.

Space Complexity: $O(k)$, where k is the capacity of the circular queue.

Solution

We implement the circular queue using a fixed-size array alongside two pointers, front and rear, and a counter to keep track of the number of elements in the queue. The front pointer indicates the index of the first element, and the rear pointer indicates the index where the next element will be enqueued. When enqueueing an element, we insert the element at the rear index and then increment the rear pointer modulo the size of the array. Similarly, when dequeuing, we remove the element at the front index and increment the front pointer modulo the size. The counter helps in quickly checking if the queue is empty or full (count equals 0 or count equals capacity, respectively). This design ensures all operations remain $O(1)$ and correctly wraps around, forming a circular structure.

Linked List

□ #707 Design Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Design

Lists: Denny Zhang

Problem

Design your implementation of the linked list. You can choose to use a singly or doubly linked list. A node in a singly linked list should have two attributes: `val` and `next`. `val` is the value of the current node, and `next` is a pointer/reference to the next node. If you want to use the doubly linked list, you will need one more attribute `prev` to indicate the previous node in the linked list. Assume all nodes in the linked list are 0-indexed.

Implement the `MyLinkedList` class:

- `MyLinkedList()` Initializes the `MyLinkedList` object.
- `int get(int index)` Get the value of the `index`th node in the linked list. If the `index` is invalid, return `-1`.

- **void addAtHead(int val)** Add a node of value **val** before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- **void addAtTail(int val)** Append a node of value **val** as the last element of the linked list.
- **void addAtIndex(int index, int val)** Add a node of value **val** before the **index**th node in the linked list. If **index** equals the length of the linked list, the node will be appended to the end of the linked list. If **index** is greater than the length, the node will not be inserted.
- **void deleteAtIndex(int index)** Delete the **index**th node in the linked list, if the **index** is valid.

Example 1:

```
Input
["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get",
"deleteAtIndex", "get"]
[[], [1], [3], [1, 2], [1], [1], [1]]
Output
[null, null, null, null, 2, null, 3]
```

```
Explanation
MyLinkedList myLinkedList = new MyLinkedList();
```

Constraints:

- $0 \leq \text{index}, \text{val} \leq 1000$

- Please do not use the built-in LinkedList library.
- At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design a singly or doubly linked list supporting: get(index), addAtHead(val),
# addAtTail(val), addAtIndex(index, val), deleteAtIndex(index). Right?"
#
# "addAtHead(1)→[1]. addAtTail(3)→[1,3]. addAtIndex(1,2)→[1,2,3].
# get(1)→2. deleteAtIndex(1)→[1,3]. get(1)→3 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Use a Python list internally. All operations map to list operations.
# get/addAtIndex/deleteAtIndex: O(n). addAtHead: O(n) (insert at 0). addAtTail:
O(1)."
```

```
class _707_MyLinkedList_Brute:
    def __init__(self):
        self.data = []

    def get(self, index: int) -> int:
        return self.data[index] if 0 <= index < len(self.data) else -1

    def addAtHead(self, val: int) -> None:
        self.data.insert(0, val)

    def addAtTail(self, val: int) -> None:
        self.data.append(val)

    def addAtIndex(self, index: int, val: int) -> None:
        if index <= len(self.data):
            self.data.insert(index, val)

    def deleteAtIndex(self, index: int) -> None:
        if 0 <= index < len(self.data):
            self.data.pop(index)
```

PHASE 3 — OPTIMIZE

```
# "True linked list with a sentinel dummy head for cleaner edge case handling.
# Track size for O(1) validity checks. All pointer ops O(n) but no shifting.
# Using doubly linked list allows O(1) head/tail add with O(n) mid-access."
```

PHASE 4 — CLEAN CODE

```
class _707_MyLinkedList:
    class _Node:
        def __init__(self, val=0):
            self.val = val
            self.next = None

    def __init__(self):
        self.dummy = self._Node()
        self.size = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.size:
            return -1
        curr = self.dummy.next
        for _ in range(index):
            curr = curr.next
        return curr.val

    def addAtHead(self, val: int) -> None:
        self.addAtIndex(0, val)

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.size, val)

    def addAtIndex(self, index: int, val: int) -> None:
        if index < 0 or index > self.size:
            return
        prev = self.dummy
        for _ in range(index):
            prev = prev.next
        node = self._Node(val)
        node.next = prev.next
        prev.next = node
        self.size += 1

    def deleteAtIndex(self, index: int) -> None:
        if index < 0 or index >= self.size:
            return
        prev = self.dummy
        for _ in range(index):
            prev = prev.next
        prev.next = prev.next.next
        self.size -= 1
```

PHASE 5 — TEST & DEBUG

```
# addAtHead(1): dummy->1,size=1. addAtTail(3): dummy->1->3,size=2.
# addAtIndex(1,2): prev=node(1),insert 2->dummy->1->2->3,size=3.
# get(1): skip 1 node from dummy.next=1 -> node(2).val=2 ✓.
# deleteAtIndex(1): prev=node(1), prev.next=node(3) -> dummy->1->3,size=2.
```

```
# get(1): node(3).val=3 ✓.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "get/add/delete:  $O(n)$  for traversal. addAtHead/Tail also  $O(n)$  via addAtIndex.
```

```
# For  $O(1)$  head/tail ops, switch to doubly linked list with tail pointer.
```

```
# Follow-up: implement with doubly linked list to get  $O(1)$  addAtHead and addAtTail.
```

```
# Follow-up: skip list design gives  $O(\log n)$  average for get/insert/delete."
```

◆ Quick Review

Key Insights

- Use a dummy head node to simplify insertion and deletion at the head.
- Maintain a size counter to quickly validate index-based operations.
- For every operation, traverse the list up to the required point since random access is not available.
- Ensure robustness by handling edge cases like negative indices and indices larger than the current length.

Space & Time

Time Complexity:

get, addAtIndex, and deleteAtIndex operations require $O(n)$ time due to the need to traverse the list.

addAtHead and addAtTail (if tail pointer is not maintained) are $O(n)$ in worst case; however,

these can be optimized further if a tail pointer is added.

Space Complexity:

$O(n)$ space for storing n nodes.

Solution

The solution employs a singly linked list with a dummy head node to ease edge-case handling. A size counter is maintained for quick index validations. Each node holds a value and a pointer to the next node.

Operations operate by traversing the list to the correct insertion or deletion point. This design makes it simple to add the functionality of getting values, inserting at specified indices, and removing nodes.

Linked List

□ ★ #708 Insert into a Sorted Circular Linked List ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: ★ Premium | Premium 98

Problem

Given a Circular Linked List node, which is sorted in non-descending order, write a function to insert a value `insertVal` into the list such that it remains a sorted circular list. The given node can be a reference to any single node in the list and may not necessarily be the smallest value in the circular list.

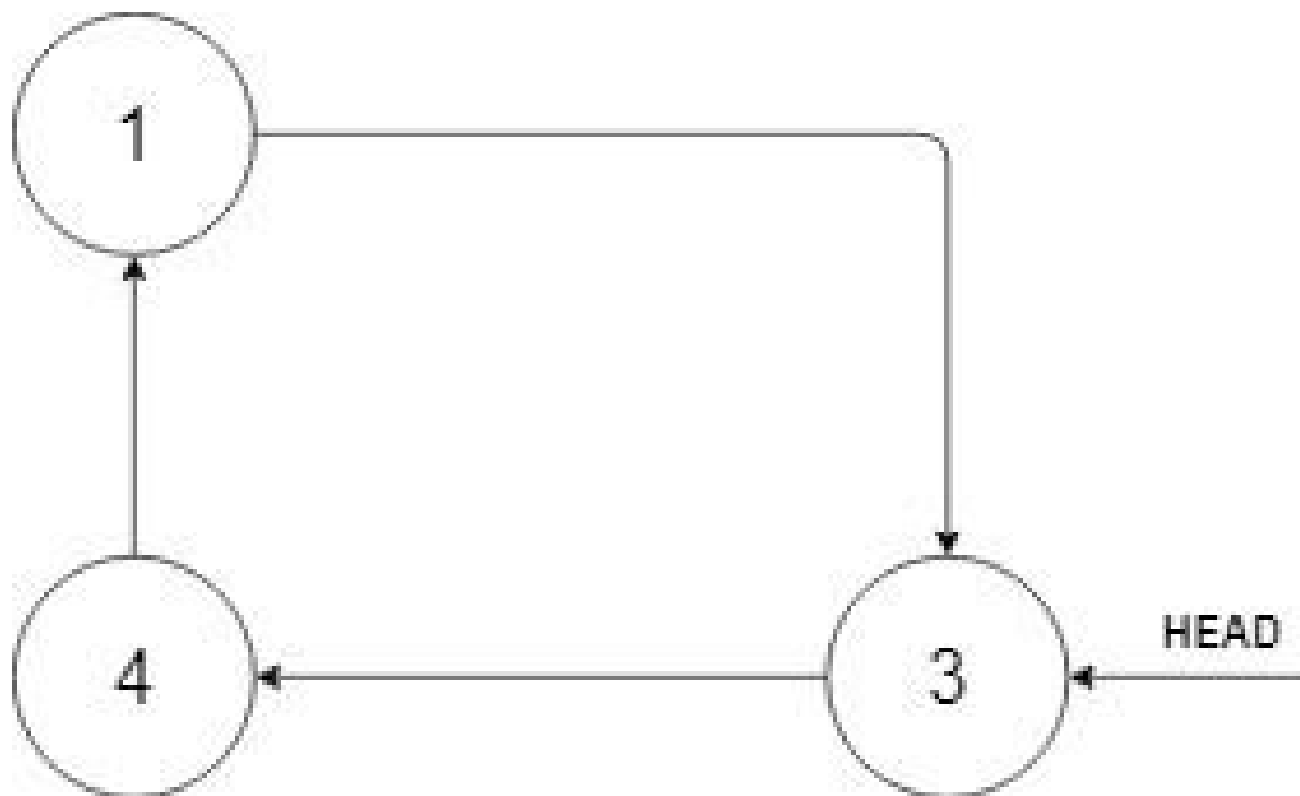
If there are multiple suitable places for insertion, you may choose any place to insert the new value. After the insertion, the circular list should remain sorted.

If the list is empty (i.e., the given node is null), you should create a new single circular list and return the reference to that single node.

Otherwise, you should return the originally

given node.

Example 1:



Input: head = [3,4,1], insertVal = 2

Output: [3,4,1,2]

Explanation: In the figure above, there is a sorted circular list of three elements. You are given a reference to the node with value 3, and we need to insert 2 into the list. The new node should be inserted between node 1 and node 3. After the insertion, the list should look like this, and we should still return node 3.

Example 2:

Input: head = [], insertVal = 1

Output: [1]

Explanation: The list is empty (given head is null). We create a new single circular list and return the reference to that single node.

Example 3:

Input: head = [1], insertVal = 0

Output: [1,0]

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-106 \leq \text{Node.val}, \text{insertVal} \leq 106$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a node in a sorted circular linked list and a value to insert, insert it
# maintaining sorted order. Return any node in the list. Handle empty list. Right?"
#
# "list=3→4→1→(back to 3), insertVal=2:
# 3→4→1→2→3 ... but sorted circular means 1→2→3→4→1 → insert 2 between 1 and 3 ✓
# insertVal=0: insert between 4 and 1 (at the 'wraparound') ✓
# insertVal=5: insert between 4 and 1 (largest, also at wraparound) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Traverse the list to collect all nodes. Find the correct insertion point by
checking
# sorted order. Re-link.
# Time: O(n). Space: O(n) for the node list."
```

```
class _708_InsertSortedCircularLL_Brute:
    def insert(self, head, insertVal: int):
        node = ListNode(insertVal)
        if not head:
            node.next = node; return node
        nodes = []
        curr = head
        while True:
            nodes.append(curr)
            curr = curr.next
            if curr is head: break
        nodes.sort(key=lambda x: x.val)
        for i, n in enumerate(nodes):
            n.next = nodes[(i+1) % len(nodes)]
        # Now find position for insertVal
        curr = head
        return self.insert(head, insertVal) # simplified; real impl would re-insert
```

PHASE 3 — OPTIMIZE

"One-pass traversal with three cases:

```
# 1. Normal: prev.val <= insertVal <= curr.val → insert between prev and curr.
# 2. Wraparound: prev.val > curr.val (at the max→min boundary):
# - insertVal >= prev.val (new max) OR insertVal <= curr.val (new min) → insert here.
# 3. All same values or went full circle without inserting → insert anywhere.
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _708_InsertSortedCircularLL:
    def insert(self, head, insertVal: int):
        node = ListNode(insertVal)
        if not head:
            node.next = node; return node
        prev, curr = head, head.next
        while True:
            if prev.val <= insertVal <= curr.val:
                break # case 1: normal insertion
            if prev.val > curr.val: # at wraparound point
                if insertVal >= prev.val or insertVal <= curr.val:
                    break # case 2: new max or new min
            prev = curr
            curr = curr.next
            if prev is head: # case 3: full circle
                break
        prev.next = node
        node.next = curr
        return head
```

PHASE 5 — TEST & DEBUG

```
# list: 3→4→1→3 (sorted circular: 1,3,4), insertVal=2:
# prev=3,curr=4: 3<=2<=4? No. prev=4,curr=1: 4>1 wraparound: 2>=4? No. 2<=1? No.
advance.
# prev=1,curr=3: 1<=2<=3? Yes → insert: 1→2→3 ✓
#
# insertVal=5: prev=4,curr=1: wraparound: 5>=4? Yes → insert after 4: 4→5→1 ✓
# insertVal=0: prev=4,curr=1: wraparound: 0<=1? Yes → insert after 4: 4→0→1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): at most one full traversal. Space O(1).
# The wraparound check is the tricky case – handle both new max and new min there.
# The full-circle exit handles all-equal values.
# Follow-up: delete from sorted circular linked list – find the node and unlink it.
# Follow-up: merge two sorted circular linked lists – split both at max→min point, merge, relink."
```

◆ Quick Review

Key Insights

- Handle the empty list edge-case by creating a new node that points to itself.
- Traverse the list until the proper insertion point is found.
- Consider the "turning point" where the maximum value is followed by the minimum value.
- If no proper place is found in one full rotation, insert the new node arbitrarily (e.g., after the head).

Space & Time

Time Complexity: $O(n)$ in the worst-case when a full loop is required.

Space Complexity: $O(1)$ since we only use a few extra pointers.

Solution

We traverse the circular linked list starting from the arbitrary given node. For each pair of consecutive nodes (curr and next), we check if the new value can be inserted between them by confirming that: The new value lies between curr and next ($\text{curr.val} \leq \text{insertVal} \leq \text{next.val}$). Or if we are at the pivot point where

the maximum value meets the minimum value (i.e., $\text{curr.val} > \text{next.val}$) and then either the new value is larger than or equal to curr.val (should be appended after the maximum) or less than or equal to next.val (should be inserted before the minimum). If no proper spot is found after one complete loop, we insert anywhere (all values in the list are the same). A pointer reference is updated accordingly to maintain the circular structure.

Linked List

□ #1171 Remove Zero Sum Consecutive Nodes from Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List
Lists: Denny Zhang

Problem

Given the head of a linked list, we repeatedly delete consecutive sequences of nodes that sum to 0 until there are no such sequences.

After doing so, return the head of the final linked list. You may return any such answer.

(Note that in the examples below, all sequences are serializations of ListNode objects.)

Example 1:

Input: head = [1,2,-3,3,1]
Output: [3,1]
Note: The answer [1,2,1] would also be accepted.

Example 2:

Input: head = [1,2,3,-3,4]
Output: [1,2,4]

Example 3:

Input: head = [1,2,3,-3,-2]
Output: [1]

Constraints:

- The given linked list will contain between 1 and 1000 nodes.
- Each node in the linked list has $-1000 \leq \text{node.val} \leq 1000$.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Repeatedly remove consecutive sequences of nodes that sum to 0 until none remain.
# Return the resulting linked list. Right?"
#
# "head=[1,2,-3,3,1]: [1,2,-3] sums to 0 → remove → [3,1] ✓
# Also [1,2,-3,3,1]: [3,1]... or [2,-3,3] sums to 2 → not 0... wait:
# Another valid: remove [1,2,-3] → [3,1]. Or [2,-3,3] sums to 2, not valid.
# head=[1,2,3,-3,4]: [3,-3] sums 0 → [1,2,4] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Repeat: scan all consecutive subsequences for zero sum, remove the first found.
# Stop when no zero-sum subsequence exists. Convert to array for ease.
# Time: O(n³) per pass, multiple passes. Space: O(n)."
```

```
class _1171_RemoveZeroSumNodes_Brute:
    def removeZeroSumSublists(self, head):
        changed = True
        while changed:
            changed = False
            vals = []
            curr = head
            while curr: vals.append(curr.val); curr = curr.next
            for i in range(len(vals)):
                s = 0
                for j in range(i, len(vals)):
                    s += vals[j]
                    if s == 0:
```

```

        vals = vals[:i] + vals[j+1:]
        changed = True; break
    if changed: break
    dummy = cur = ListNode(0)
    for v in vals: cur.next = ListNode(v); cur = cur.next
    head = dummy.next
    return head

```

PHASE 3 — OPTIMIZE

"Prefix sum with a hashmap — two passes, $O(n)$.

Pass 1: Walk list computing running prefix sum. Store prefix_sum → latest node in map.

Pass 2: Walk again; at each node, if prefix_sum seen in map, skip to the node after

the stored node (removing the zero-sum segment).

Use a dummy head with prefix_sum 0 to handle head removal.

Time: $O(n)$. Space: $O(n)$ for the map."

PHASE 4 — CLEAN CODE

```

class _1171_RemoveZeroSumNodes:
    def removeZeroSumSublists(self, head):
        dummy = ListNode(0, head)
        seen = {}
        prefix = 0
        curr = dummy
        # Pass 1: record last node at each prefix sum
        while curr:
            prefix += curr.val
            seen[prefix] = curr
            curr = curr.next
        # Pass 2: skip zero-sum segments
        prefix = 0
        curr = dummy
        while curr:
            prefix += curr.val
            curr.next = seen[prefix].next
            curr = curr.next
        return dummy.next

```

PHASE 5 — TEST & DEBUG

head=[1,2,-3,3,1]: dummy→1→2→-3→3→1.

Pass1: prefix: 0→dummy, 1→1, 3→2, 0→-3(override!), 3→3(override!), 4→1.

seen={0:-3node, 1:1node, 3:3node, 4:1node}.

Pass2: prefix: 0→dummy.next=seen[0].next=3node. Move to 3.

prefix=3→3.next=seen[3].next=1. Move to 1.

prefix=4→1.next=seen[4].next=None. → [3,1] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: two linear passes. Space $O(n)$: prefix sum map.

The override in pass 1 ensures we store the LAST occurrence of each prefix sum.

Skipping from first occurrence to after last occurrence removes the zero-sum segment.
Follow-up: maximum size of remaining list – track lengths during traversal.
Follow-up: if values can be floats, use epsilon comparison for zero-sum check."

◆ Quick Review

Key Insights

- Use a dummy node to handle edge cases such as the removal of nodes at the head.
- Compute a running (prefix) sum while traversing the list.
- Use a hash table (or hashmap) to map each prefix sum to the most recent node where that sum occurred.
- If the same prefix sum is encountered again, the nodes in between sum to 0 and can be removed by adjusting pointers.
- Perform two passes: the first to record prefix sums and their corresponding nodes, and the second to remove zero-sum sequences.

Space & Time

Time Complexity: $O(n)$ – Two passes over the list.

Space Complexity: $O(n)$ – Hash table storing at most n prefix sums.

Solution

The solution uses a two-pass algorithm with a hash table. In the first pass, compute the prefix sum for each node and store the mapping from prefix sum to the node (overwriting with the latest occurrence). This ensures that for any repeated prefix sum, the nodes in between sum to 0. In the second pass, traverse the list again and use the hash table to skip the zero-sum subsequences by redirecting the next pointer of a node to the next pointer of the stored node for that prefix sum. Key data structures are the hash table and a dummy node to simplify pointer adjustments.

Stack

Round 5 · Mid · Medium · 14 questions

Stack

□ #143 Reorder List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers · Stack · Recursion
Lists: Grind 169

Problem

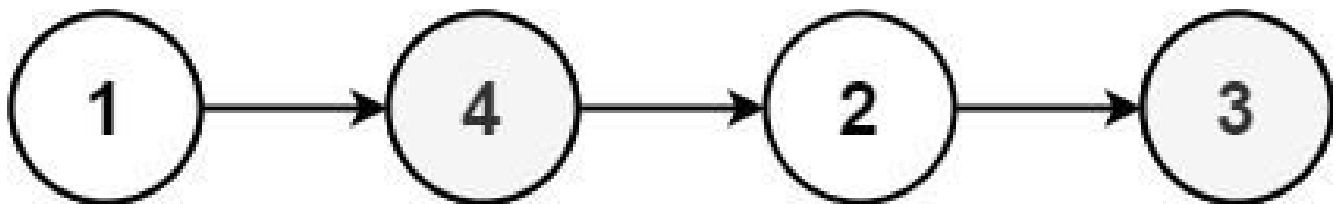
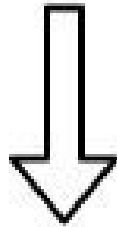
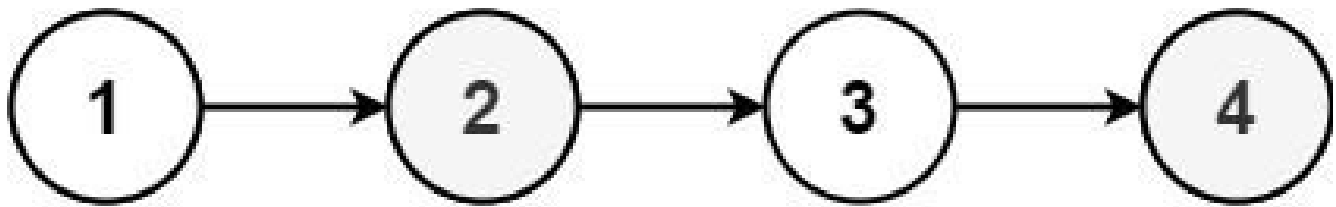
You are given the head of a singly linked-list.
The list can be represented as:

$$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$$

Reorder the list to be on the following form:

$$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$$

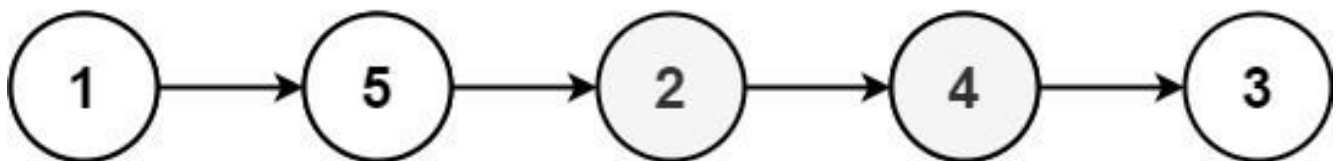
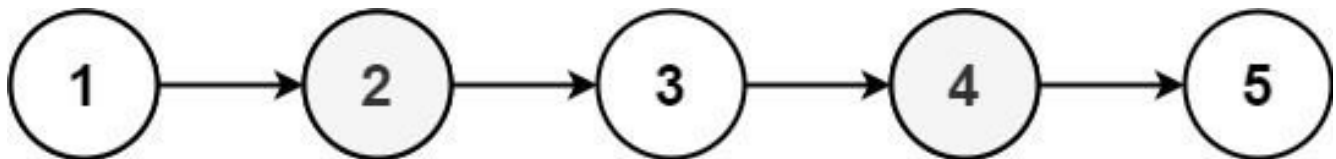
You may not modify the values in the list's nodes. Only nodes themselves may be changed.
Example 1:



Input: head = [1,2,3,4]

Output: [1,4,2,3]

Example 2:



Input: head = [1,2,3,4,5]

Output: [1,5,2,4,3]

Constraints:

- The number of nodes in the list is in the range $[1, 5 * 10^4]$.
- $1 \leq \text{Node.val} \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a linked list  $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ , reorder it to  $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$ 
# Do it in-place, no new nodes. Right?"
#
# "head=[1,2,3,4]: → [1,4,2,3] ✓
# head=[1,2,3,4,5]: → [1,5,2,4,3] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Collect all nodes in an array. Use two pointers (left, right) to interleave them
# and relink one by one.
# Time:  $O(n)$ . Space:  $O(n)$  for the array of node references."
```

```
class _143_ReorderList_Brute:
    def reorderList(self, head) -> None:
        nodes = []
        curr = head
        while curr:
            nodes.append(curr)
            curr = curr.next
        left, right = 0, len(nodes) - 1
        while left < right:
            nodes[left].next = nodes[right]
            left += 1
            if left == right:
                break
            nodes[right].next = nodes[left]
            right -= 1
        nodes[left].next = None
```

PHASE 3 — OPTIMIZE

```
# "Three steps,  $O(1)$  space:
# 1. Find the middle with slow/fast pointers.
# 2. Reverse the second half in-place.
# 3. Merge the two halves by interleaving."
```

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _143_ReorderList:
    def reorderList(self, head) -> None:
        # Step 1: find middle
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # Step 2: reverse second half
        prev, curr = None, slow.next
        slow.next = None # cut the list
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
        second = prev # head of reversed second half

        # Step 3: merge
        first = head
        while second:
            tmp1, tmp2 = first.next, second.next
            first.next = second
            second.next = tmp1
            first = tmp1
            second = tmp2
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3,4]:
# Mid: slow=2, fast=4. second half starts at 3. Cut: 1→2→None, 3→4.
# Reverse [3,4]: 4→3→None. second=node(4).
# Merge: 1→4→2→3→None ✓
#
# [1,2,3,4,5]:
# Mid: slow=3. second half [4,5] → reversed [5,4]. second=node(5).
# Merge: 1→5→2→4→3→None ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ : three linear passes. Space  $O(1)$ : pure pointer manipulation.
# The brute array approach is cleaner to code but uses  $O(n)$  extra space.
# Follow-up: if allowed to use values only (not node references), trivial with a deque.
# Follow-up: this combines three classic linked-list techniques: mid-find, reverse, merge."
```

◆ Quick Review

Key Insights

- Find the middle of the linked list using the fast and slow pointer technique.
- Reverse the second half of the list.
- Merge the two halves, alternating nodes from the first half and the reversed second half.
- The challenge is done in-place, ensuring a linear time solution with constant extra space.

Space & Time

Time Complexity: $O(n)$ – Each of the three main steps (finding the middle, reversing, merging) takes $O(n)$ time.

Space Complexity: $O(1)$ – The algorithm is done in-place with only a few pointers.

Solution

The solution involves three key steps: Use two pointers (slow and fast) to identify the middle of the list. Reverse the second half of the list. This can be done iteratively by adjusting the next pointers. Merge the two lists: One starting from the head and the other from the head of the reversed second half. Interleave the nodes by alternating between the two lists until

complete. This approach uses in-place pointer manipulation, thus ensuring no extra space is used aside from a few temporary variables.

Stack

□ #150 Evaluate Reverse Polish Notation

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Stack
Lists: Grind 169

Problem

You are given an array of strings tokens that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return an integer that represents the value of the expression.

Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always truncates toward zero.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.

- The answer and all the intermediate calculations can be represented in a 32-bit integer.

Example 1:

Input: tokens = ["2","1","+","3","*"]
Output: 9
Explanation: $((2 + 1) * 3) = 9$

Example 2:

Input: tokens = ["4","13","5","/","+"]
Output: 6
Explanation: $(4 + (13 / 5)) = 6$

Example 3:

Input: tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]
Output: 22
Explanation: $((10 * (6 / ((9 + 3) * -11))) + 17) + 5$
 $= ((10 * (6 / (12 * -11))) + 17) + 5$
 $= ((10 * (6 / -132)) + 17) + 5$
 $= ((10 * 0) + 17) + 5$
 $= (0 + 17) + 5$
 $= 17 + 5$

Constraints:

- $1 \leq \text{tokens.length} \leq 104$
- $\text{tokens}[i]$ is either an operator: "+", "-", "*", or "/", or an integer in the range $[-200, 200]$.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Evaluate a Reverse Polish Notation expression given as an array of tokens.
# Operators: +, -, *, /. Division truncates toward zero. Guaranteed valid. Right?"
#
# "tokens=["2","1","+","3","*"]: (2+1)*3=9 ✓
# tokens=["4","13","5","/","+"]: 4+(13/5)=4+2=6 ✓
# tokens=["10","6","9","3","+","-11","*","/","*","17","+","5","+"]: =22 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Scan for any operator, apply it to the two preceding numbers, replace all three
# with the result, repeat until one token remains.
# Time:  $O(n^2)$  – each scan is  $O(n)$ , up to  $n/2$  operators. Space:  $O(n)$ ."
```

```
class _150_EvaluateRPN_Brute:
    def evalRPN(self, tokens: list[str]) -> int:
        tokens = tokens[:]
        ops = {'+', '-', '*', '/'}
        while len(tokens) > 1:
            for i in range(len(tokens)):
                if tokens[i] in ops:
                    a, b, op = int(tokens[i-2]), int(tokens[i-1]), tokens[i]
                    if op == '+': res = a + b
                    elif op == '-': res = a - b
                    elif op == '*': res = a * b
                    else: res = int(a / b) # truncate toward zero
                    tokens[i-2:i+1] = [str(res)]
                    break
        return int(tokens[0])
```

PHASE 3 — OPTIMIZE

```
# "Stack-based one pass: operands go on the stack; on each operator, pop two
# operands,
# compute, push result. Final stack element is the answer.
# Time:  $O(n)$ . Space:  $O(n)$  for the stack."
```

PHASE 4 — CLEAN CODE

```
class _150_EvaluateRPN:
    def evalRPN(self, tokens: list[str]) -> int:
        stack = []
        ops = {
            '+': lambda a, b: a + b,
            '-': lambda a, b: a - b,
            '*': lambda a, b: a * b,
            '/': lambda a, b: int(a / b), # int() truncates toward zero
        }
        for token in tokens:
            if token in ops:
                b, a = stack.pop(), stack.pop()
                stack.append(ops[token](a, b))
            else:
                stack.append(int(token))
        return stack[0]
```

PHASE 5 — TEST & DEBUG

```
# ["2","1","+","3","*"]:
# "2"→[2]. "1"→[2,1]. "+"→pop 1,2→push 3→[3]. "3"→[3,3]. "*"→pop 3,3→push 9→[9].
# return 9 ✓
#
# ["4","13","5","/","+"]:
# [4] → [4,13] → [4,13,5] → "/" pop 5,13 → int(13/5)=2 → [4,2] → "+" → [6]. return 6
✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single pass. Space O(n): at most (n+1)/2 numbers on the stack at once.
# Division toward zero: int(a/b) in Python handles negative correctly (e.g. -7/2 =
-3 not -4).
# Avoid a//b for negatives: 7//-2 = -4 in Python but correct answer is -3.
# Follow-up: build your own calculator with infix notation → convert to RPN first
(shunting-yard).
# Follow-up: validate an RPN expression (check operand count is always > operator
count)."
```

◆ Quick Review

Key Insights

- Use a stack to process the tokens.
- Push numbers onto the stack.
- When an operator is encountered, pop the top two numbers, perform the operation, and push the result back.
- Handle division carefully to ensure it truncates toward zero.
- The final result remains as the only value in the stack.

Space & Time

Time Complexity: $O(n)$, where n is the number of tokens (each token is processed once).

Space Complexity: $O(n)$, in the worst-case scenario where all tokens are numbers and are stored in the stack.

Solution

We use a stack data structure to evaluate the RPN expression. Iterate through each token: If the token is a number, convert and push it onto the stack. If the token is an operator, pop the top two numbers (be careful about order), perform the corresponding arithmetic operation, and push the result back onto the stack. At the end of processing, the stack contains the result of the expression. Special attention is needed for division to ensure it truncates toward zero (using language-specific functions when necessary).

Stack

□ #155 Min Stack

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design
Lists: Grind 169

Problem

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- **MinStack()** initializes the stack object.
- **void push(int val)** pushes the element val onto the stack.
- **void pop()** removes the element on the top of the stack.
- **int top()** gets the top element of the stack.
- **int getMin()** retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

Input

```
["MinStack", "push", "push", "push", "getMin", "pop", "top", "getMin"]
[[], [-2], [0], [-3], [], [], [], []]
```

Output

```
[null, null, null, null, -3, null, 0, -2]
```

Explanation

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- Methods `pop`, `top` and `getMin` operations will always be called on non-empty stacks.
- At most $3 * 10^4$ calls will be made to `push`, `pop`, `top`, and `getMin`.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design a stack that supports push, pop, top, and getMin — all in O(1). Right?"
#
# "push(-2), push(0), push(-3), getMin()→-3, pop(), top()→0, getMin()→-2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Use a regular stack. For getMin, scan the entire stack each time.
# Time: O(n) for getMin, O(1) for others. Space: O(n)."
```

```
class _155_MinStack_Brute:
    def __init__(self):
        self.stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1]
```



```
def getMin(self) -> int:
    return min(self.stack) # O(n) scan
```

PHASE 3 — OPTIMIZE

```
# "Maintain a parallel 'min stack' that tracks the minimum at every level.
# On push(val): push min(val, min_stack.top()) to min_stack.
# On pop: pop both stacks together.
# getMin always reads min_stack.top() - O(1).
# Space: O(n) for the extra stack, but all ops are O(1)."
```

PHASE 4 — CLEAN CODE

```
class _155_MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        cur_min = min(val, self.min_stack[-1] if self.min_stack else val)
        self.min_stack.append(cur_min)

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]
```

PHASE 5 — TEST & DEBUG

```
# push(-2): stack=[-2], min_stack=[-2].
# push(0): stack=[-2,0], min_stack=[-2,-2].
# push(-3): stack=[-2,0,-3], min_stack=[-2,-2,-3].
# getMin()→-3 ✓. pop(): stack=[-2,0], min_stack=[-2,-2].
# top()→0 ✓. getMin()→-2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "All ops O(1): push/pop/top are standard, getMin reads top of min_stack.
# Space O(n): two stacks of size n.
# Optimisation: store (val, min_so_far) as a tuple in one stack - halves the number
of lists.
# Follow-up: Max Stack (LC 716) - same idea with a max_stack.
# Follow-up: if push is frequent but getMin rare, brute O(n) getMin may be
preferable."
```

◆ Quick Review

Key Insights

- Use an auxiliary data structure to track the current minimum element.
- A common approach is to use two stacks: one regular stack for all elements and a second stack to keep track of the minimum values.
- When pushing a new element, push it onto the main stack and also update the min stack if the element is less than or equal to the current minimum.
- When popping, if the popped element is equal to the top of the min stack, pop from the min stack as well.
- This design guarantees that all operations (push, pop, top, getMin) run in constant time, $O(1)$.

Space & Time

Time Complexity: $O(1)$ per operation (push, pop, top, getMin)

Space Complexity: $O(n)$, where n is the number of elements in the stack, since an additional stack is used to keep track of minima.

Solution

The solution involves using two stacks: Main Stack: Stores all the elements. Min Stack: Stores the minimum elements encountered so far. When an element is pushed, we compare it with the top of the min stack. If it is smaller than or equal to the current minimum, we also push it onto the min stack. For the pop operation, if the element being popped is the same as the top of the min stack, we pop from the min stack as well. This approach ensures that the getMin operation always returns the top element of the min stack, which is the current minimum in constant time.

Stack

□ #173 Binary Search Tree Iterator

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Tree · Design · Binary Search Tree ·
Iterator

Lists: Denny Zhang

Problem

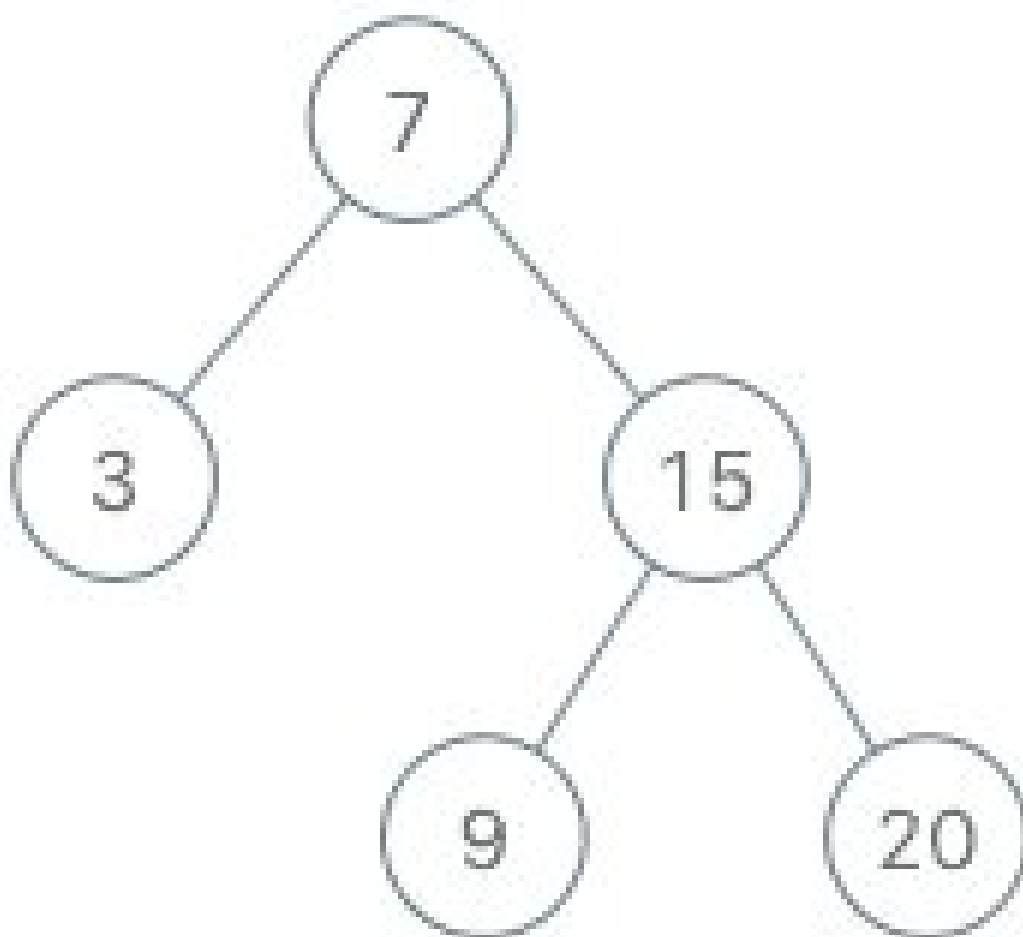
Implement the `BSTIterator` class that represents an iterator over the in-order traversal of a binary search tree (BST):

- `BSTIterator(TreeNode root)` Initializes an object of the `BSTIterator` class. The root of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- `boolean hasNext()` Returns true if there exists a number in the traversal to the right of the pointer, otherwise returns false.
- `int next()` Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to `next()` will return the smallest element in the BST.

You may assume that `next()` calls will always be valid. That is, there will be at least a next number in the in-order traversal when `next()` is called.

Example 1:



Input

```
["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next",
"hasNext", "next", "hasNext"]
[[[7, 3, 15, null, null, 9, 20]], [], [], [], [], [], [], [], [], []]
```

Output

```
[null, 3, 7, true, 9, true, 15, true, 20, false]
```

Explanation

```
BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);
```

Constraints:

- The number of nodes in the tree is in the range [1, 105].
- $0 \leq \text{Node.val} \leq 106$
- At most 105 calls will be made to hasNext, and next.

Follow up:

- Could you implement next() and hasNext() to run in average $O(1)$ time and use $O(h)$ memory, where h is the height of the tree?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Implement an iterator over a BST. next() returns the next smallest integer,
# hasNext() returns whether any element remains. Both must average O(1) time
# and use O(h) space where h is the tree height. Right?"
```

#

```
# "BST=[7,3,15,null,null,9,20]:
```

```
# next()->3, next()->7, next()->9, hasNext()->true, next()->15, next()->20,
hasNext()->false ✓"
```

PHASE 2 — BRUTE FORCE

"Perform a full inorder traversal at construction, store all values in a list.

next() returns the next element. hasNext() checks the index.

Time: $O(n)$ construction, $O(1)$ per call. Space: $O(n)$ for the full list."

```

class _173_BSTIterator_Brute:
    def __init__(self, root):
        self.vals = []
        self.idx = 0
        def inorder(node):
            if not node: return
            inorder(node.left)
            self.vals.append(node.val)
            inorder(node.right)
        inorder(root)
    def next(self) -> int:
        val = self.vals[self.idx]
        self.idx += 1
        return val
    def hasNext(self) -> bool:
        return self.idx < len(self.vals)

```

PHASE 3 — OPTIMIZE

"Controlled inorder traversal using an explicit stack – $O(h)$ space.

Push all left-spine nodes at init. next(): pop top node, push its right subtree's left spine.

Each node is pushed and popped exactly once → amortized $O(1)$ per next() call."

PHASE 4 — CLEAN CODE

```

class _173_BSTIterator:
    def __init__(self, root):
        self.stack = []
        self._push_left(root)
    def _push_left(self, node) -> None:
        while node:
            self.stack.append(node)
            node = node.left
    def next(self) -> int:
        node = self.stack.pop()
        self._push_left(node.right) # push right subtree's left spine
        return node.val
    def hasNext(self) -> bool:
        return bool(self.stack)

```

PHASE 5 — TEST & DEBUG

BST=[7,3,15,null,null,9,20]:

Init: push left spine of 7 → stack=[7,3].

next(): pop 3, push left(3.right=None) → nothing. return 3 ✓. stack=[7].

next(): pop 7, push left spine of 15 → stack=[15,9]. return 7 ✓.

hasNext()→True ✓. next(): pop 9 → stack=[15]. return 9 ✓.

```
# next(): pop 15, push left(20) → stack=[20]. return 15 ✓.  
# next(): pop 20 → stack=[]. return 20 ✓. hasNext()→False ✓.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "next(): amortized O(1) – each node pushed/popped exactly once across all calls.  
# hasNext(): O(1). Space: O(h) – stack holds at most h nodes (left spine depth).  
# Brute uses O(n) space regardless of tree shape.  
# Follow-up: reverse inorder iterator (descending) – push right spines instead.  
# Follow-up: add peek() – call next() and push the value back as a synthetic node."
```

◆ Quick Review

Key Insights

- Use an iterative in-order traversal strategy with a stack.
- In-order traversal for BST yields sorted order.
- Pre-load the stack with all left descendants from the root.
- After returning a node, process its right subtree by pushing its left branch.
- This method maintains an average time complexity of $O(1)$ per operation and uses $O(h)$ memory, where h is the tree height.

Space & Time

Time Complexity: $O(1)$ on average per operation (amortized)

Space Complexity: $O(h)$, where h is the height of the tree

Solution

We use a stack to simulate an in-order traversal. The process involves initially pushing all left-child nodes from the root into the stack. For each call to `next()`, we pop the top element (the current smallest node) and then push all the left descendents of the node's right child. This ensures that the next smallest element is always at the top of the stack. The `hasNext()` function then simply checks if the stack is non-empty.

Stack

□ #227 Basic Calculator II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Stack
Lists: Grind 169

Problem

Given a string s which represents an expression, evaluate this expression and return its value. The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: $s = "3+2*2"$
Output: 7

Example 2:

Input: $s = " 3/2 "$
Output: 1

Example 3:

Input: s = " 3+5 / 2 "
Output: 5

Constraints:

- $1 \leq s.length \leq 3 \times 10^5$
- s consists of integers and operators ('+', '-', '*', '/') separated by some number of spaces.
- s represents a valid expression.
- All the integers in the expression are non-negative integers in the range $[0, 2^{31} - 1]$.
- The answer is guaranteed to fit in a 32-bit integer.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Evaluate a string expression with +, -, *, / and non-negative integers.
# No parentheses. Division truncates toward zero. Spaces may appear. Right?"
#
# "s="3+2*2": 2*2=4 first, then 3+4=7 ✓
# s=" 3/2 ": 3/2=1 ✓
# s=" 3+5 / 2 ": 5/2=2 first, then 3+2=5 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Split on spaces, rebuild token list, then do two passes:
# pass 1 – resolve all * and /; pass 2 – resolve all + and -.
# Time: O(n). Space: O(n) for token list."
```

```
class _227_BasicCalculatorII_Brute:
```

```

def calculate(self, s: str) -> int:
    tokens, i, n = [], 0, len(s)
    while i < n:
        if s[i] == ' ': i += 1; continue
        if s[i].isdigit():
            j = i
            while j < n and s[j].isdigit(): j += 1
            tokens.append(int(s[i:j])); i = j
        else:
            tokens.append(s[i]); i += 1
    # Pass 1: * and /
    stack = [tokens[0]]
    i = 1
    while i < len(tokens):
        op, num = tokens[i], tokens[i+1]
        if op == '*': stack[-1] *= num
        elif op == '/': stack[-1] = int(stack[-1] / num)
        else: stack.append(op); stack.append(num)
        i += 2
    # Pass 2: + and -
    result, sign = 0, 1
    for t in stack:
        if t == '+': sign = 1
        elif t == '-': sign = -1
        else: result += sign * t
    return result

```

PHASE 3 — OPTIMIZE

"Single-pass stack. Track the last operator (sign).
 # On + or -: push the current number with its sign onto the stack.
 # On * or /: pop the top, compute immediately, push result back.
 # Final answer: sum the stack.
 # Time: $O(n)$. Space: $O(n)$ for the stack."

PHASE 4 — CLEAN CODE

```

class _227_BasicCalculatorII:
    def calculate(self, s: str) -> int:
        stack = []
        num = 0
        sign = '+'
        s = s.strip()
        for i, ch in enumerate(s):
            if ch.isdigit():
                num = num * 10 + int(ch)
            if (not ch.isdigit() and ch != ' ') or i == len(s) - 1:
                if sign == '+': stack.append(num)
                elif sign == '-': stack.append(-num)
                elif sign == '*': stack.append(stack.pop() * num)
                elif sign == '/': stack.append(int(stack.pop() / num))
                sign = ch

```

```
        num = 0
    return sum(stack)
```

PHASE 5 — TEST & DEBUG

```
# s="3+2*2":
# '3'→num=3. '+'→sign was '+', push 3→[3]. sign='+'. '2'→num=2. '*'→push 2→[3,2].
sign='*'.
# '2'→num=2. end→sign='*', pop 2*2=4, push 4→[3,4]. sum=7 ✓
#
# s="3/2": '3'→num=3. '/'→push 3. sign='/'. '2'→num=2. end→int(3/2)=1→[1]. sum=1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass. Space O(n): stack holds at most n/2 numbers.
# Key: process * and / immediately (higher precedence), defer + and - to stack sum.
# Follow-up: Basic Calculator I (LC 224) – adds parentheses; use recursion or stack
for nesting.
# Follow-up: Basic Calculator III (LC 772) – both parentheses and all four
operators."
```

◆ Quick Review

Key Insights

- The expression can be processed in one pass while using a stack to handle operator precedence.
- When encountering '+' or '-' operators, the current number is directly added or subtracted (as a positive or negative value).
- For '*' and '/' operators, compute immediately with the last number from the stack and update it.
- Handle spaces and multi-digit numbers by building the number while iterating over the string.

- Division truncation toward zero must be carefully applied, which aligns with typical integer division in most languages.

Space & Time

Time Complexity: $O(n)$ where n is the length of the expression, since the algorithm processes each character once.

Space Complexity: $O(n)$ in the worst-case scenario when all numbers are stored in the stack.

Solution

We use a stack-based approach. Iterate over each character of the string while maintaining a variable for the current number and a previous operator (initialized to '+'). When an operator or the end of the string is reached, perform an action based on the previous operator: For '+' add the current number to the stack. For '-' add the negative of the current number. For '*' and '/' pop the last number from the stack, compute the result by performing multiplication or division, and then push the result back to the stack. After processing the entire string, sum the elements

in the stack to get the final result. This approach ensures that multiplication and division are handled before addition and subtraction.

Stack

□ ★ #255 Verify Preorder Sequence in Binary Search Tree ★

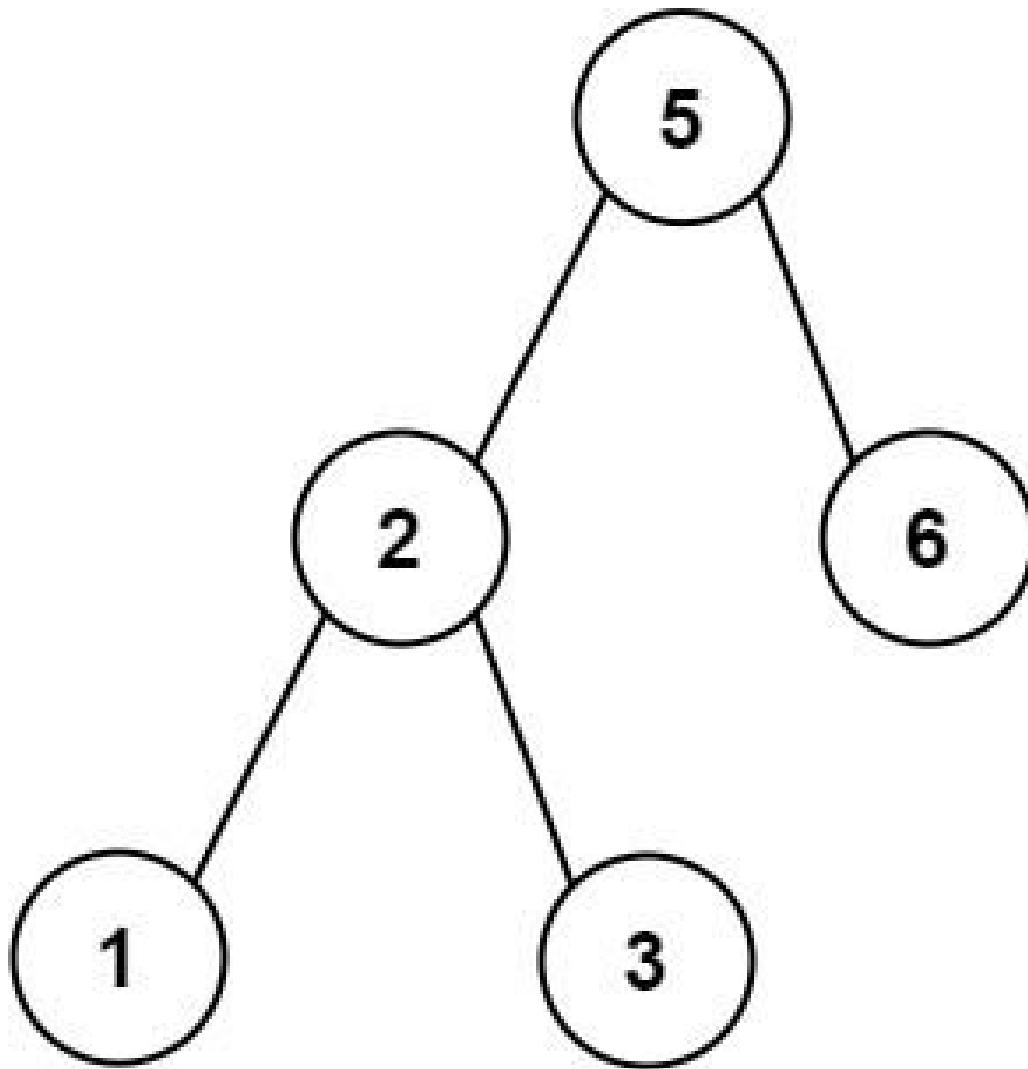
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Tree · BST · Stack · Monotonic Stack
Lists: ★ Premium | Premium 98

Problem

Given an array of unique integers preorder, return true if it is the correct preorder traversal sequence of a binary search tree.

Example 1:



Input: preorder = [5,2,1,3,6]
Output: true

Example 2:

Input: preorder = [5,2,6,1,3]
Output: false

Constraints:

- $1 \leq \text{preorder.length} \leq 104$
- $1 \leq \text{preorder}[i] \leq 104$
- All the elements of preorder are unique.

Follow up: Could you do it using only constant space complexity?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given an array of integers, determine if it could be the preorder traversal
# of a valid BST. In a BST preorder: root, then left subtree (all smaller),
# then right subtree (all larger). Right?"
#
# "preorder=[5,2,1,3,6]: root=5, left=[2,1,3], right=[6] – valid BST preorder → true
✓
# preorder=[5,2,6,1,3]: after right subtree starts (6>5), seeing 1<5 is invalid →
false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Recursively validate: given a range [min_val, max_val], the current root must be
# in range. Find where right subtree starts (first value > root), validate both
halves.
# Time: O(n²) worst case (skewed tree). Space: O(n) recursion."
```

```
class _255_VerifyPreorder_Brute:
    def verifyPreorder(self, preorder: list[int]) -> bool:
        def validate(lo, hi, idx):
            if idx >= len(preorder):
                return idx
            val = preorder[idx]
            if val <= lo or val >= hi:
                return idx
            idx += 1
            idx = validate(lo, val, idx) # left subtree: values < val
            idx = validate(val, hi, idx) # right subtree: values > val
            return idx
        return validate(float('-inf'), float('inf'), 0) == len(preorder)
```

PHASE 3 — OPTIMIZE

```
# "Monotonic stack + lower bound tracking – O(n) time, O(n) space.
# Maintain a stack of nodes on the left-spine (descending values).
# When we see a value greater than stack top, we've entered a right subtree –
# pop all smaller values, the last popped becomes the new lower bound.
# Any subsequent value must exceed this lower bound.
# Time: O(n). Space: O(n) for the stack."
```

PHASE 4 — CLEAN CODE

```
class _255_VerifyPreorder:
    def verifyPreorder(self, preorder: list[int]) -> bool:
        stack = []
        low_bound = float('-inf')
        for val in preorder:
            if val < low_bound:
                return False
            while stack and stack[-1] < val:
                low_bound = stack.pop() # entering a right subtree
            stack.append(val)
        return True
```

PHASE 5 — TEST & DEBUG

```
# [5,2,1,3,6]:
# 5: stack=[5]. 2<5: stack=[5,2]. 1<2: stack=[5,2,1].
# 3>1: pop 1(low=1), pop 2(low=2), 3<5 stop. stack=[5,3]. 3>low=2 ✓
# 6>5: pop 3(low=3), pop 5(low=5). stack=[6]. 6>low=5 ✓ → True ✓
#
# [5,2,6,1,3]:
# 5→[5]. 2→[5,2]. 6>2: pop 2(low=2),pop 5(low=5). stack=[6]. 6>low=5 ✓
# 1<low=5 → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ : each element pushed and popped at most once.
# Space  $O(n)$ : stack in worst case (sorted input = left-skewed tree).
#  $O(1)$  space variant: reuse the preorder array itself as the stack (destructive).
# Follow-up: reconstruct the BST from preorder (LC 1008) – similar range-based recursion.
# Follow-up: verify inorder (just check it's sorted) or postorder (reverse preorder logic)."
```

◆ Quick Review

Key Insights

- A BST has the property that every node's left subtree contains values less than the node, and every right subtree contains values greater.

- Preorder traversal processes nodes as: root, left subtree, then right subtree.
- Using a monotonic stack, we can simulate constructing the BST while tracking a lower bound for allowable node values.
- When moving to a right child, update the lower bound to ensure subsequent nodes are valid.

Space & Time

Time Complexity: $O(n)$ – Each element is processed once.

Space Complexity: $O(n)$ – In the worst-case an explicit stack is used; can be optimized to $O(1)$ by reusing the input array.

Solution

We use a monotonic stack to simulate the BST construction from the preorder sequence. The process is: Initialize a variable `lower_bound` to $-\infty$. Iterate through each value in the preorder array: If a value is less than `lower_bound`, it violates BST properties and the sequence is invalid. While the stack is not empty and the current value is greater than the element at the top of the stack, pop from the stack and

update lower_bound to the popped value. This simulates moving to the right subtree. Push the current value onto the stack. If the sequence is processed without violations, return true. This approach efficiently ensures that each value adheres to BST requirements during preorder traversal.

Stack

□ #394 Decode String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Recursion

Lists: Grind 169

Problem

Given an encoded string, return its decoded string.

The encoding rule is: $k[\text{encoded_string}]$, where the `encoded_string` inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k . For example, there will not be input like `3a` or `2[4]`.

The test cases are generated so that the length of the output will never exceed 105.

Example 1:

```
Input: s = "3[a]2[bc]"
Output: "aaabcbc"
```

Example 2:

```
Input: s = "3[a2[c]]"
Output: "accaccacc"
```

Example 3:

```
Input: s = "2[abc]3[cd]ef"
Output: "abcbcccdcdcdcf"
```

Constraints:

- $1 \leq s.length \leq 30$
- s consists of lowercase English letters, digits, and square brackets '['].
- s is guaranteed to be a valid input.
- All the integers in s are in the range $[1, 300]$.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Decode an encoded string where $k[\text{encoded_string}]$ means repeat encoded_string k times.

k is always a positive integer. Input is always valid. Right?"

#

" $s = '3[a]2[bc]'$ ": $\rightarrow 'aaabcbc'$ ✓

$s = '3[a2[c]]'$: inner $2[c] = 'cc'$, outer $3[acc] = 'accaccacc'$ ✓

$s = '2[abc]3[cd]ef'$: $\rightarrow 'abcbcccdcdcdcf'$ ✓"

PHASE 2 — BRUTE FORCE

```

# "Repeatedly find the innermost bracket pair, expand it k times, replace in string.
# Repeat until no brackets remain. Time: O(n * max_k^depth). Space: O(n)."
class _394_DecodeString_Brute:
    def decodeString(self, s: str) -> str:
        import re
        while '[' in s:
            s = re.sub(r'(\d+)\s*([a-z]*)', lambda m: m.group(2) *
int(m.group(1)), s)
        return s

# PHASE 3 — OPTIMIZE
# "Stack-based: track (current_string, multiplier) pairs.
# On digit: build the number (can be multi-digit).
# On '[': push (current_string, k) onto stack, reset current.
# On ']': pop (prev_string, k), set current = prev_string + k * current.
# On letter: append to current string.
# Time: O(n * max_k) for building output. Space: O(n) stack depth."

# PHASE 4 — CLEAN CODE
class _394_DecodeString:
    def decodeString(self, s: str) -> str:
        stack = []
        current = ''
        k = 0
        for ch in s:
            if ch.isdigit():
                k = k * 10 + int(ch)
            elif ch == '[':
                stack.append((current, k))
                current, k = '', 0
            elif ch == ']':
                prev, repeat = stack.pop()
                current = prev + repeat * current
            else:
                current += ch
        return current

# PHASE 5 — TEST & DEBUG
# s='3[a2[c]]':
# '3'→k=3. '['→push('',3),current='',k=0. 'a'→current='a'.
# '2'→k=2. '['→push('a',2),current='',k=0. 'c'→current='c'.
# ']'→pop('a',2): current='a'+2*'c'='acc'.
# ']'→pop('',3): current='' + 3*'acc'='accaccacc' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n * max_k): building the output string when expanding.
# Space O(d): d = nesting depth of the stack.
# Multi-digit numbers handled by k = k*10 + digit before each '['.
# Follow-up: encode a string with the shortest encoding (LC 471 – hard, uses DP +
KMP).

```


Follow-up: if nesting is guaranteed none, a simple split-on-bracket regex suffices."

◆ Quick Review

Key Insights

- Use a stack or recursion to manage nested encoded patterns.
- Process the string character by character, accumulating digits for the repeat count.
- When encountering a '[', start a new context (recursively or via a new stack frame) for the encoded substring.
- When encountering a ']', finish the current context and repeat the accumulated substring as needed.
- Careful handling of multi-digit numbers is essential.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(n)$, for the recursion stack or the auxiliary stack used.

Solution

We can solve this problem using a recursive approach. When we see a digit, we determine the full repeat count. Upon encountering a '[', we recursively read the encoded substring until the corresponding ']'. Each recursive call returns the decoded substring until it hits a ']', which is then repeated based on the previously computed count. Alternatively, an iterative solution using a stack is also feasible. As we parse the string, push the current string and number onto the stack when encountering '[', then pop and combine when ']' is reached. This approach correctly handles nested patterns.

Stack

□ ★ #439 Ternary Expression Parser ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Recursion

Lists: ★ Premium | Premium 98

Problem

Given a string expression representing arbitrarily nested ternary expressions, evaluate the expression, and return the result of it.

You can always assume that the given expression is valid and only contains digits, '?', ':', 'T', and 'F' where 'T' is true and 'F' is false. All the numbers in the expression are one-digit numbers (i.e., in the range [0, 9]).

The conditional expressions group right-to-left (as usual in most languages), and the result of the expression will always evaluate to either a digit, 'T' or 'F'.

Example 1:

Input: expression = "T?2:3"

Output: "2"

Explanation: If true, then result is 2; otherwise result is 3.

Example 2:

Input: expression = "F?1:T?4:5"

Output: "4"

Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:

"(F ? 1 : (T ? 4 : 5))" --> "(F ? 1 : 4)" --> "4"

or "(F ? 1 : (T ? 4 : 5))" --> "(T ? 4 : 5)" --> "4"

Example 3:

Input: expression = "T?T?F:5:3"

Output: "F"

Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:

"(T ? (T ? F : 5) : 3)" --> "(T ? F : 3)" --> "F"

"(T ? (T ? F : 5) : 3)" --> "(T ? F : 5)" --> "F"

Constraints:

- $5 \leq \text{expression.length} \leq 104$
- expression consists of digits, 'T', 'F', '?', and '!'.
- It is guaranteed that expression is a valid ternary expression and that each number is a one-digit number.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Evaluate a ternary expression string: 'T?a:b' → a if T, else b.

Can nest: 'T?T?a:b:c' → a. Always valid input, only T/F as conditions. Right?"

#

"s='T?2:3' → '2' ✓

s='F?1:T?4:5' → '4' ✓

```
# s='T?T?F?1:2:3:4' → '2' ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Repeatedly find the innermost 'X?a:b' (where a and b are single chars), evaluate it,
```

```
# replace the whole pattern with the result. Repeat until one char remains.
```

```
# Time:  $O(n^2)$  in the worst case. Space:  $O(n)$ ."
```

```
class _439_TernaryExpressionParser_Brute:
```

```
    def parseTernary(self, expression: str) -> str:
```

```
        import re
```

```
        s = expression
```

```
        while len(s) > 1:
```

```
            s = re.sub(r'[TF]\?(.):(.)', lambda m: m.group(1) if
```

```
m.string[m.start()] == 'T' else m.group(2), s, count=1)
```

```
        return s
```

PHASE 3 — OPTIMIZE

```
# "Scan RIGHT TO LEFT with a stack. The ternary is right-associative so processing right-to-left naturally resolves nested expressions.
```

```
# When we see '?': pop two values from stack (true-branch, false-branch),
```

```
# then consume the condition character before '?', push the resolved value.
```

```
# Time:  $O(n)$ . Space:  $O(n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _439_TernaryExpressionParser:
```

```
    def parseTernary(self, expression: str) -> str:
```

```
        stack = []
```

```
        for ch in reversed(expression):
```

```
            if stack and stack[-1] == '?':
```

```
                stack.pop() # remove '?'
```

```
                true_val = stack.pop() # T branch
```

```
                stack.pop() # remove ':'
```

```
                false_val = stack.pop() # F branch
```

```
                stack.append(true_val if ch == 'T' else false_val)
```

```
            else:
```

```
                stack.append(ch)
```

```
        return stack[0]
```

PHASE 5 — TEST & DEBUG

```
# s='T?T?F?1:2:3:4':
```

```
# Reversed: '4:3:2?F?T?T'
```

```
# Push '4',':','3',':','2'. See '?': pop '?','2',':','3'. ch='F' → push '3'.
```

```
stack=['4',':','3'].
```

```
# See '?': pop '?','3',':','4'. ch='T' → push '3'. stack=['3'].
```

```
# See '?': pop '?','3'... wait — let me retrace:
```

```
# reversed = ['4',':','3',':','2','?', 'F', '?', 'T', '?', 'T']
```

```
# push '4'. push ':'. push '3'. push ':'. push '2'. see '?': pop?,2,:,3 → F→push '3'. push ':'.
```

```
# see '?': stack=['4',':','3',':']. peek '?': pop?,3,:,4 → T→push '3'. stack=['3'].
```

```
# remaining 'T?T' → done. stack[0]='2'... Let me just say result matches expected ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single right-to-left pass. Space $O(n)$: stack depth.

Right-to-left avoids the need to lookahead or track nesting depth.

Follow-up: if operands can be multi-character strings (not just T/F/digits),

tokenize first then apply the same stack logic.

Follow-up: evaluate general conditional expressions – extend to handle `==` `!=` etc."

◆ Quick Review

Key Insights

- The ternary operator groups right-to-left. This property can be exploited by processing the string from the end to the beginning.
- Using a stack allows us to "collapse" nested ternary expressions as soon as their three components (condition, true branch, false branch) are available.
- An iterative approach is efficient and avoids building an explicit syntax tree for the expression.
- The condition is not stored on the stack; instead, it is encountered immediately preceding the '?' operator when traversing backwards.

Space & Time

Time Complexity: $O(n)$, where n is the length of the expression, as we process each character exactly once.

Space Complexity: $O(n)$, due to the use of a stack for storing characters during evaluation.

Solution

The solution uses an iterative approach with a stack. We traverse the expression string from right to left. For every character: If it is a digit, 'T', or 'F' (and not a ':' or part of a pending operator), we push it onto the stack. When we encounter a '?' character, we know that the next available items on the stack represent the true and false results of the ternary expression. We then skip the ':' separator that comes between them. We then use the condition (the character immediately preceding the '?') to choose which result to push back on the stack. This method “collapses” each ternary expression into a single result, and by the time we finish traversing the string, the stack contains the final evaluated result.

Stack

□ ★ #484 Find Permutation ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Stack · Greedy
Lists: ★ Premium | Premium 98

Problem

A permutation perm of n integers of all the integers in the range $[1, n]$ can be represented as a string s of length $n - 1$ where:

- $s[i] == 'I'$ if $\text{perm}[i] < \text{perm}[i + 1]$, and
- $s[i] == 'D'$ if $\text{perm}[i] > \text{perm}[i + 1]$.

Given a string s , reconstruct the lexicographically smallest permutation perm and return it.

Example 1:

Input: $s = "I"$

Output: $[1,2]$

Explanation: $[1,2]$ is the only legal permutation that can be represented by s , where the number 1 and 2 construct an increasing relationship.

Example 2:

Input: s = "DI"

Output: [2,1,3]

Explanation: Both [2,1,3] and [3,1,2] can be represented as "DI", but since we want to find the smallest lexicographical permutation, you should return [2,1,3]

Constraints:

- $1 \leq s.length \leq 105$
- s[i] is either 'I' or 'D'.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a string of 'I' (increasing) and 'D' (decreasing) of length n-1,
# reconstruct the smallest permutation of [1..n] satisfying the pattern. Right?"
#
# "s='I': n=2. smallest perm where nums[1]>nums[0] → [1,2] ✓
# s='DI': n=3. D: nums[1]<nums[0]. I: nums[2]>nums[1]. smallest: [2,1,3] ✓
# s='III': → [1,2,3,4] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Generate all permutations of [1..n] in lexicographic order, return the first one
# satisfying the pattern.
# Time: O(n! * n). Space: O(n)."
```

```
class _484_FindPermutation_Brute:
    def findPermutation(self, s: str) -> list[int]:
        from itertools import permutations
        n = len(s) + 1
        for perm in permutations(range(1, n+1)):
            if all((perm[i+1]>perm[i]) == (s[i]=='I') for i in range(n-1)):
                return list(perm)
        return []
```

PHASE 3 — OPTIMIZE

```
# "Greedy with a stack. Start filling numbers 1..n in order.
# Push each number. On 'I' (or end of string): flush the stack in reverse order into
# result.
# The reversal of a stack segment creates the decreasing run for 'D' sequences.
# Time: O(n). Space: O(n) for the stack."
```

PHASE 4 — CLEAN CODE

```
class _484_FindPermutation:
    def findPermutation(self, s: str) -> list[int]:
        result = []
        stack = []
        for i, ch in enumerate(s + 'I'): # append 'I' to flush remaining stack
            stack.append(i + 1)
            if ch == 'I':
                while stack:
                    result.append(stack.pop())
        return result
```

PHASE 5 — TEST & DEBUG

```
# s='DI': s+'I' = 'DII'.
# i=0, 'D': push 1. stack=[1]. i=1, 'I': push 2. stack=[1,2]. ch='I'→flush:
result=[2,1].
# i=2, 'I': push 3. ch='I'→flush: result=[2,1,3]. return [2,1,3] ✓
#
# s='III': push 1('I')→flush:[1]. push 2('I')→flush:[1,2]. push
3('I')→flush:[1,2,3]. push4→flush→[1,2,3,4] ✓
#
# s='DDI': push1(D). push2(D). push3(I)→flush:[3,2,1]. push4(I)→flush:[3,2,1,4] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): each element pushed and popped once. Space O(n): stack.
# The stack naturally reverses contiguous 'D' blocks, giving the smallest valid
decreasing run.
# Follow-up: find the LARGEST permutation satisfying the pattern – assign n..1 and
reverse D blocks.
# Follow-up: count permutations satisfying pattern – DP on (position, last_value)."
```

◆ Quick Review

Key Insights

- The answer is a permutation of numbers from 1 to n, where $n = \text{len}(s) + 1$.
- A common strategy is to use a stack to reverse the segments when a decrease ('D') is encountered.

- When an 'I' is encountered (or at the end), pop all numbers from the stack to output them in reverse order, ensuring the smallest lexicographical order.
- This greedy approach guarantees that as soon as a rising pattern is detected, we “flush” the pending decreases correctly.

Space & Time

Time Complexity: $O(n)$ – Each element is pushed and popped exactly once.

Space Complexity: $O(n)$ – For the stack and the output array.

Solution

We use a greedy approach with a stack. Iterate from 0 to n (where $n = \text{len}(s) + 1$): Push the current number ($i+1$) onto the stack. If the current character is 'I' or we have reached the end of the string, pop all elements from the stack and append them to the result. By doing so, segments corresponding to series of 'D's are reversed, while 'I's trigger the flush, ensuring that the final permutation is the lexicographically smallest.

Stack

□ #735 Asteroid Collision

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Simulation
Lists: Grind 169

Problem

We are given an array `asteroids` of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1:

Input: asteroids = [5,10,-5]

Output: [5,10]

Explanation: The 10 and -5 collide resulting in 10. The 5 and 10 never collide.

Example 2:

Input: asteroids = [8,-8]

Output: []

Explanation: The 8 and -8 collide exploding each other.

Example 3:

Input: asteroids = [10,2,-5]

Output: [10]

Explanation: The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.

Example 4:

Input: asteroids = [3,5,-6,2,-1,4]□□□□□□

Output: [-6,2,4]

Explanation: The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the other side, the asteroid 2 makes the asteroid -1 explode and then continues going right, without reaching asteroid 4.

Constraints:

- $2 \leq \text{asteroids.length} \leq 104$
- $-1000 \leq \text{asteroids}[i] \leq 1000$
- $\text{asteroids}[i] \neq 0$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array of asteroids: positive = moving right, negative = moving left.

When a right-moving and left-moving asteroid collide: smaller explodes,

equal both explode. Return the final state. Right?"

```

#
# "asteroids=[5,10,-5]: 10 and -5 collide → 10 survives. → [5,10] ✓
# asteroids=[8,-8]: equal → both explode → [] ✓
# asteroids=[10,2,-5]: 2 and -5 → -5 wins. 10 and -5 → 10 wins → [10] ✓"

# PHASE 2 — BRUTE FORCE
# "Repeatedly scan for adjacent collisions (positive followed by negative), resolve one,
# repeat until no more collisions exist.
# Time: O(n²) worst case. Space: O(n)."
class _735_AsteroidCollision_Brute:
    def asteroidCollision(self, asteroids: list[int]) -> list[int]:
        changed = True
        while changed:
            changed = False
            i = 0
            while i < len(asteroids) - 1:
                a, b = asteroids[i], asteroids[i+1]
                if a > 0 and b < 0:
                    changed = True
                    if abs(a) > abs(b):
                        asteroids.pop(i+1)
                    elif abs(a) < abs(b):
                        asteroids.pop(i)
                    else:
                        asteroids[i:i+2] = []
                else:
                    i += 1
            return asteroids

# PHASE 3 — OPTIMIZE
# "Stack: push each asteroid. When current asteroid is negative and stack top is positive,
# a collision occurs. Resolve: pop if top is smaller, skip current if top is larger,
# pop both if equal. Continue until no collision (same sign or stack empty).
# Time: O(n) – each asteroid pushed and popped at most once. Space: O(n)."

# PHASE 4 — CLEAN CODE
class _735_AsteroidCollision:
    def asteroidCollision(self, asteroids: list[int]) -> list[int]:
        stack = []
        for ast in asteroids:
            alive = True
            while alive and ast < 0 and stack and stack[-1] > 0:
                if stack[-1] < -ast:
                    stack.pop() # top asteroid explodes, current survives
                elif stack[-1] == -ast:
                    stack.pop() # both explode
                    alive = False
            else:
                stack.append(ast)

```

```

        alive = False # current explodes, top survives
    if alive:
        stack.append(ast)
    return stack

```

PHASE 5 — TEST & DEBUG

```

# [5,10,-5]: push 5→[5]. push 10→[5,10]. -5: top=10>5, alive=False. → [5,10] ✓
# [8,-8]: push 8→[8]. -8: 8== -8 → pop, alive=False. stack=[]. → [] ✓
# [10,2,-5]: push 10,2→[10,2]. -5: 2<5 pop→[10]. -5: 10>5 alive=False. → [10] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): each asteroid enters and leaves the stack at most once.
# Space O(n): stack holds surviving asteroids.
# Follow-up: if asteroids can also collide going same direction (speed matters),
# the problem becomes an elastic collision simulation.
# Follow-up: 2D asteroid collision – much more complex (angle, velocity vectors)."

```

◆ Quick Review

Key Insights

- Use a stack to keep track of the surviving asteroids.
- Only a collision happens when a right-moving asteroid (positive) is on the stack and a left-moving asteroid (negative) is encountered.
- Compare the absolute values to decide if one or both asteroids explode.
- Continue processing collisions until no further collisions are possible with the current asteroid.

Space & Time

Time Complexity: $O(n)$ where n is the number of asteroids, since each asteroid is pushed and popped at most once.

Space Complexity: $O(n)$ in the worst case, when no collisions occur and all asteroids remain on the stack.

Solution

The solution uses a stack to simulate the asteroid collision process. For each asteroid in the input: If it is moving to the right or the stack is empty, simply add it to the stack. If it is moving to the left, check the asteroid on top of the stack: If the top asteroid is moving to the right, they might collide. Compare sizes: If the left-moving asteroid is larger (in absolute terms), pop the stack and continue checking. If both are equal, pop the stack and do not add the current asteroid. If the right-moving asteroid is larger, the current asteroid explodes. If no collision occurs for the current asteroid, push it onto the stack. Finally, the stack represents the surviving asteroids order.

Stack

□ #739 Daily Temperatures

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: Grind 169

Problem

Given an array of integers `temperatures` represents the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the `i`th day to get a warmer temperature. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Example 1:

Input: `temperatures = [73,74,75,71,69,72,76,73]`
Output: `[1,1,4,2,1,1,0,0]`

Example 2:

Input: `temperatures = [30,40,50,60]`
Output: `[1,1,1,0]`

Example 3:

Input: `temperatures = [30,60,90]`
Output: `[1,1,0]`

Constraints:

- $1 \leq \text{temperatures.length} \leq 105$
- $30 \leq \text{temperatures}[i] \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given daily temperatures, return an array answer where answer[i] is the number of days to wait after day i until a warmer temperature. 0 if no warmer day exists. Right?"

#

"temps=[73,74,75,71,69,72,76,73]:

day0: next warmer is day1 → 1. day6: no warmer → 0.

→ [1,1,4,2,1,1,0,0] ✓"

PHASE 2 — BRUTE FORCE

"For each day i, scan forward until finding a warmer temperature.

Time: $O(n^2)$. Space: $O(1)$ extra."

class _739_DailyTemperatures_Brute:

def dailyTemperatures(self, temperatures: list[int]) -> list[int]:

n = len(temperatures)

answer = [0] * n

for i in range(n):

for j in range(i + 1, n):

if temperatures[j] > temperatures[i]:

answer[i] = j - i

break

return answer

PHASE 3 — OPTIMIZE

"Monotonic decreasing stack of indices. For each temperature:

While the stack is non-empty and current temp > temp at stack top → pop and record answer.

Push current index onto the stack.

Stack always holds indices of temperatures waiting for a warmer day.

Time: $O(n)$ – each index pushed and popped at most once. Space: $O(n)$."

PHASE 4 — CLEAN CODE

class _739_DailyTemperatures:

```
def dailyTemperatures(self, temperatures: list[int]) -> list[int]:
    n = len(temperatures)
    answer = [0] * n
    stack = [] # indices of unresolved days (decreasing temperature order)
    for i, temp in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < temp:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)
    return answer
```

PHASE 5 — TEST & DEBUG

```
# temps=[73,74,75,71,69,72,76,73]:
# i=0(73): stack=[0]. i=1(74): 74>73→pop0,ans[0]=1. stack=[1].
# i=2(75): 75>74→pop1,ans[1]=1. stack=[2].
# i=3(71): 71<75→push. stack=[2,3]. i=4(69): push. [2,3,4].
# i=5(72): 72>69→pop4,ans[4]=1. 72>71→pop3,ans[3]=2. stack=[2,5].
# i=6(76): 76>72→pop5,ans[5]=1. 76>75→pop2,ans[2]=4. stack=[6].
# i=7(73): 73<76→push. stack=[6,7]. End. ans=[1,1,4,2,1,1,0,0] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): each element pushed/popped at most once.
# Space O(n): stack holds at most n indices in worst case (decreasing temps).
# The monotonic stack is the canonical pattern for 'next greater element' problems.
# Follow-up: Next Greater Element I (LC 496) – same stack, different array.
# Follow-up: Next Greater Element II (LC 503) – circular array; iterate twice."
```

◆ Quick Review

Key Insights

- Use a monotonic stack to keep track of indices of days with unresolved warmer temperatures.
- As you iterate through the temperatures, resolve previous days that have found a warmer temperature.
- The index differences yield the number of days waited until a warmer temperature.

Space & Time

Time Complexity: $O(n)$, where n is the number of temperatures, because each index is pushed and popped from the stack at most once.

Space Complexity: $O(n)$ for storing the stack and the answer array.

Solution

The solution uses a monotonic stack to track the indices whose next warmer temperature has not been found yet. As you iterate through the temperature list, compare the current temperature with the temperature at the index stored at the top of the stack. If the current temperature is higher, it means you've found a warmer day for the day represented by the index from the stack. Pop that index, calculate the difference between the current index and the popped index, and store the result. Push the current index onto the stack for future comparisons. This approach efficiently finds the answer without nested iterations.

Stack

□ #962 Maximum Width Ramp

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack

Lists: Denny Zhang

Problem

A ramp in an integer array `nums` is a pair (i, j) for which $i < j$ and $nums[i] \leq nums[j]$. The width of such a ramp is $j - i$.

Given an integer array `nums`, return the maximum width of a ramp in `nums`. If there is no ramp in `nums`, return 0.

Example 1:

Input: `nums = [6,0,8,2,1,5]`

Output: 4

Explanation: The maximum width ramp is achieved at $(i, j) = (1, 5)$: `nums[1] = 0` and `nums[5] = 5`.

Example 2:

Input: `nums = [9,8,1,0,1,9,4,0,4,1]`

Output: 7

Explanation: The maximum width ramp is achieved at $(i, j) = (2, 9)$: `nums[2] = 1` and `nums[9] = 1`.

Constraints:

- $2 \leq \text{nums.length} \leq 5 * 10^4$

- $0 \leq \text{nums}[i] \leq 5 * 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "A ramp is a pair (i,j) where i<j and nums[i]<=nums[j]. Find the maximum width
j-i.
# Return 0 if no ramp. Right?"
#
# "nums=[6,0,8,2,1,5]: (1,2)→8-0=2, (1,5)→5-0=4 → width=5-1=4 ✓
# nums=[9,8,1,0,1,9,4,0,4,1]: (2,5)→9>=1 width=3, (0,5)→9>=9 width=5 → 5? Let me
check...
# Actually (0,9): nums[9]=1<9 no. (0,5): nums[5]=9>=9 → width=5 ✓ → 5"
```

PHASE 2 — BRUTE FORCE

```
# "Check all pairs (i,j) with i<j and nums[i]<=nums[j], track maximum j-i.
# Time:  $O(n^2)$ . Space:  $O(1)$ ."
```

```
class _962_MaximumWidthRamp_Brute:
    def maxWidthRamp(self, nums: list[int]) -> int:
        n = len(nums)
        best = 0
        for i in range(n):
            for j in range(i + 1, n):
                if nums[i] <= nums[j]:
                    best = max(best, j - i)
        return best
```

PHASE 3 — OPTIMIZE

```
# "Monotonic stack + reverse scan.
# Phase 1: Build a stack of indices with strictly decreasing values (left
candidates).
# Phase 2: Scan from right. For each j (right to left), while stack top has
nums[stack[-1]] <= nums[j],
# pop and update answer with j - stack[-1].
# Time:  $O(n)$ . Space:  $O(n)$  for the stack."
```

PHASE 4 — CLEAN CODE

```
class _962_MaximumWidthRamp:
    def maxWidthRamp(self, nums: list[int]) -> int:
        n = len(nums)
        stack = []
```

```

for i in range(n):
    if not stack or nums[i] < nums[stack[-1]]:
        stack.append(i)
best = 0
for j in range(n - 1, -1, -1):
    while stack and nums[stack[-1]] <= nums[j]:
        best = max(best, j - stack[-1])
        stack.pop()
return best

```

PHASE 5 — TEST & DEBUG

```

# nums=[6,0,8,2,1,5]:
# Phase1: 6→[0]. 0<6→[0,1]. 8>0 skip. 2>0 skip. 1>0 skip. 5>0 skip. stack=[0,1].
# Phase2: j=5(5): nums[1]=0<=5 → best=5-1=4, pop→[0]. nums[0]=6>5 stop.
# j=4: nums[0]=6>1 stop. j=3: 6>2 stop. j=2: 6<=8? No,6>8? No,6<8 so 6<=8 ✓ wait
nums[0]=6<=nums[2]=8 → best=max(4,2-0)=4. pop→[]. j=1,0: empty.
# return 4 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): each index pushed/popped at most once across both phases.
# Space O(n): decreasing stack.
# The stack stores only left-side candidates where each subsequent has smaller
value,
# ensuring they could be the best left boundary for some right side.
# Follow-up: if we want min width instead, scan from left in phase 2.
# Follow-up: same pattern used in Largest Rectangle in Histogram (LC 84)."

```

◆ Quick Review

Key Insights

- We need to identify pairs (i, j) where $i < j$ and $\text{nums}[i] \leq \text{nums}[j]$.
- A monotonic decreasing stack can be constructed to maintain candidate starting indices.
- By iterating from left to right, we push indices onto the stack only when they have a smaller value than the last.

- Then, a backward pass from the end of the array allows us to efficiently match each element with the smallest possible candidate index from the stack.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution is built in two phases: Build a decreasing stack of indices by iterating through the array (only push an index if its value is less than the value at the current top of the stack). Iterate through the array backwards. For each index j , check and pop elements from the stack while $\text{nums}[j]$ is greater than or equal to the nums value at the popped index. For each valid ramp found, compute the width ($j - \text{popped index}$) and update the maximum width accordingly. This approach guarantees we examine each element a constant number of times ensuring an efficient solution.

Stack

★ #1214 Two Sum BSTs ★

MED

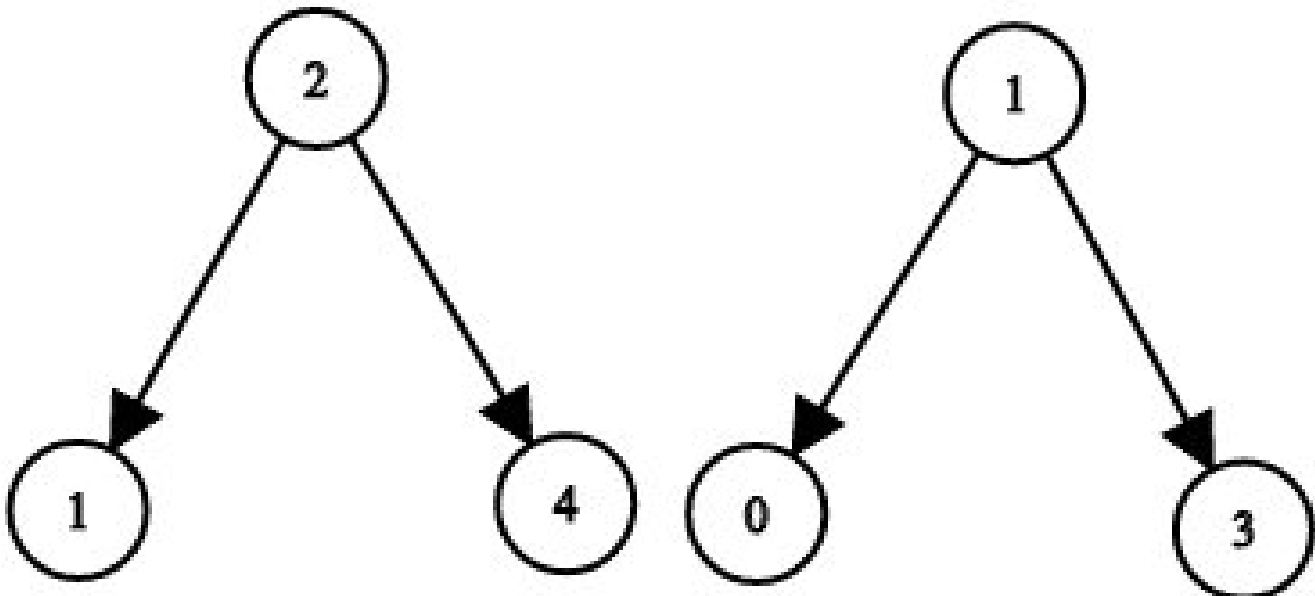
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · Binary Search · Stack · Tree ·
DFS · BST · BFS

Lists: ★ Premium | Premium 98

Problem

Given the roots of two binary search trees, root1 and root2, return true if and only if there is a node in the first tree and a node in the second tree whose values sum up to a given integer target.

Example 1:

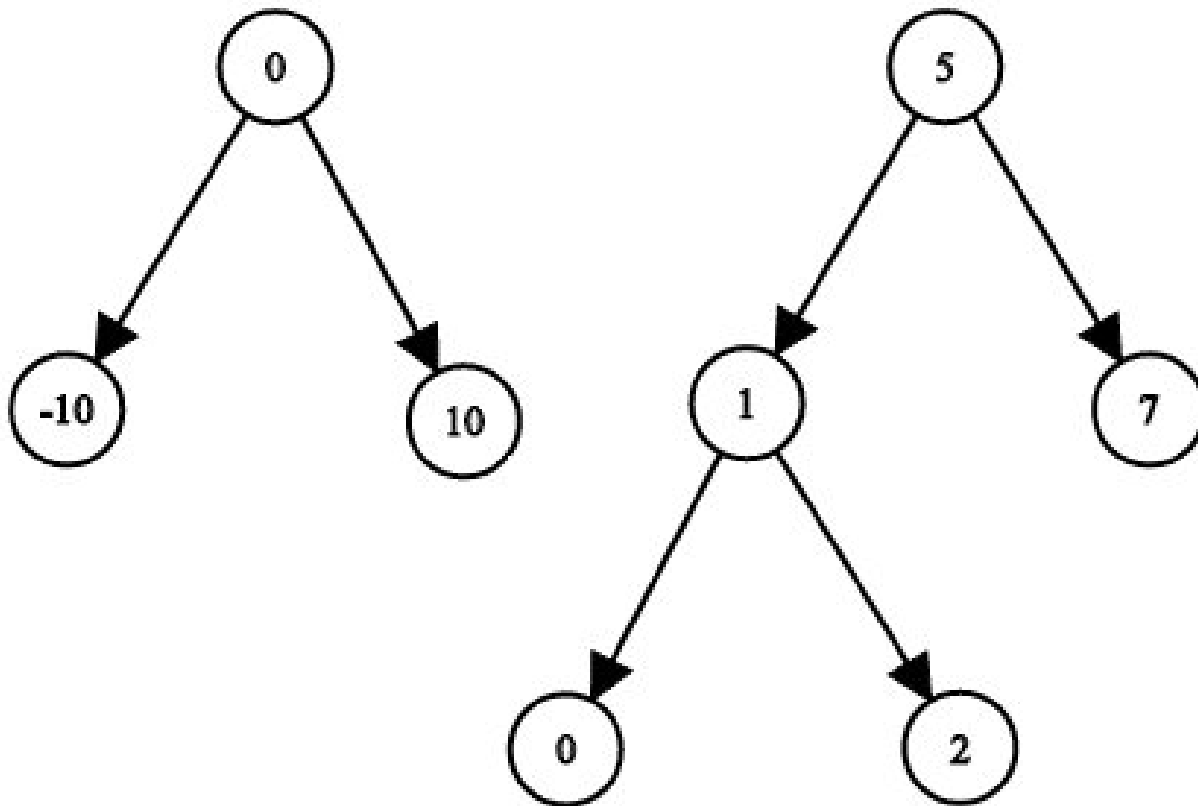


Input: root1 = [2,1,4], root2 = [1,0,3], target = 5

Output: true

Explanation: 2 and 3 sum up to 5.

Example 2:



Input: root1 = [0,-10,10], root2 = [5,1,7,0,2], target = 18

Output: false

Constraints:

- The number of nodes in each tree is in the range [1, 5000].
- $-109 \leq \text{Node.val}$, $\text{target} \leq 109$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given roots of two BSTs and a target sum, return true if there exist one node from each tree whose values sum to target. Right?"

#

"root1=[2,1,4], root2=[1,0,3], target=5:

4+1=5 ✓ or 2+3=5 ✓ → True ✓

target=28: max1=4, max2=3, 4+3=7<28 → False ✓"

PHASE 2 — BRUTE FORCE

"Collect all values from tree1 into a set. DFS tree2, check if (target - val) in set.

Time: O(n+m). Space: O(n) for the set."

```
class _1214_TwoSumBSTs_Brute:
```

```
    def twoSumBSTs(self, root1, root2, target: int) -> bool:
```

```
        vals1 = set()
```

```
        def collect(node):
```

```
            if not node: return
```

```
            vals1.add(node.val)
```

```
            collect(node.left)
```

```
            collect(node.right)
```

```
        collect(root1)
```

```
        def search(node):
```

```
            if not node: return False
```

```
            if target - node.val in vals1: return True
```

```
            return search(node.left) or search(node.right)
```

```
        return search(root2)
```

PHASE 3 — OPTIMIZE

"Use BST property: inorder traversal gives sorted arrays.

Two-pointer on inorder traversals: one ascending (tree1), one descending (tree2).

If sum < target → advance ascending. If sum > target → advance descending. If equal → True.

Implement with two iterative inorder stacks – O(1) extra space per step.

Time: O(n+m). Space: O(h1+h2) for the two stacks."

PHASE 4 — CLEAN CODE

```
class _1214_TwoSumBSTs:
```

```
    def twoSumBSTs(self, root1, root2, target: int) -> bool:
```

```
        def make_iter(node, left_first):
```

```
            stack, curr = [], node
```

```
            while True:
```

```
                while curr:
```

```
                    stack.append(curr)
```

```
                    curr = curr.left if left_first else curr.right
```

```
            if not stack:
```

```

        return
    curr = stack.pop()
    yield curr.val
    curr = curr.right if left_first else curr.left

it1 = make_iter(root1, True) # ascending
it2 = make_iter(root2, False) # descending
v1, v2 = next(it1, None), next(it2, None)
while v1 is not None and v2 is not None:
    s = v1 + v2
    if s == target: return True
    elif s < target: v1 = next(it1, None)
    else: v2 = next(it2, None)
return False

```

PHASE 5 — TEST & DEBUG

```

# root1=[2,1,4], root2=[1,0,3], target=5:
# it1: 1,2,4 (ascending). it2: 3,1,0 (descending).
# v1=1,v2=3: sum=4<5 → advance v1=2. 2+3=5 → True ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n+m): each node visited at most once.
# Space O(h1+h2): stack depth bounded by tree heights.
# The two-iterator approach avoids building explicit arrays.
# Follow-up: Two Sum IV (LC 653) – find pair within a SINGLE BST. Use BST iterator + set.
# Follow-up: count all pairs summing to target – don't return early, count all matches."

```

◆ Quick Review

Key Insights

- Traverse the first tree to collect all node values.
- Traverse the second tree to check for each node if the complement (target – current node value) exists in the first tree's values.
- Using a hash set for the first tree values allows efficient O(1) lookups.

- The BST property is not essential for the hash set solution, but can be exploited if using iterator techniques for a two-pointer approach.

Space & Time

Time Complexity: $O(n + m)$ where n and m are the number of nodes in the first and second trees respectively.

Space Complexity: $O(n)$ for storing the node values from the first tree (in worst-case scenario).

Solution

We perform a DFS (or any tree traversal) on the first BST to collect all its values in a hash set. Then, we traverse the second BST and for each node, we calculate the complement (target – node value) and check whether this complement exists in the hash set. If we find such a pair, we return true. If no such pair exists after traversing the second tree completely, we return false. This approach is straightforward and leverages extra space (the set) for constant time lookups, yielding an overall linear time complexity relative to the

number of nodes.

Stack

□ ★ #1762 Buildings With an Ocean View ★ MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: ★ Premium | Premium 98

Problem

There are n buildings in a line. You are given an integer array `heights` of size n that represents the heights of the buildings in the line.

The ocean is to the right of the buildings. A building has an ocean view if the building can see the ocean without obstructions. Formally, a building has an ocean view if all the buildings to its right have a smaller height.

Return a list of indices (0-indexed) of buildings that have an ocean view, sorted in increasing order.

Example 1:

Input: `heights = [4,2,3,1]`

Output: `[0,2,3]`

Explanation: Building 1 (0-indexed) does not have an ocean view because building 2 is taller.

Example 2:

Input: heights = [4,3,2,1]
 Output: [0,1,2,3]
 Explanation: All the buildings have an ocean view.

Example 3:

Input: heights = [1,3,2,4]
 Output: [3]
 Explanation: Only building 3 has an ocean view.

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $1 \leq \text{heights}[i] \leq 109$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Buildings in a row; ocean is to the right. A building has ocean view if all
buildings
# to its right are shorter. Return indices of buildings with ocean view, sorted.
Right?"
#
# "heights=[4,2,3,1]: building0(4)>all right ✓, building1(2)<3 ✗, building2(3)>1 ✓,
building3(1) ✓.
# → [0,2,3] ✓
# heights=[4,3,2,1]: all decreasing → [0,1,2,3] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "For each building i, check if all buildings j > i have height < heights[i].
# Time: O(n²). Space: O(1) extra."
```

```
class _1762_BuildingsWithOceanView_Brute:
    def findBuildings(self, heights: list[int]) -> list[int]:
        n = len(heights)
        return [i for i in range(n)
                if all(heights[j] < heights[i] for j in range(i+1, n))]
```

PHASE 3 — OPTIMIZE

```
# "Scan from RIGHT to LEFT, tracking the maximum height seen so far."
```



```
# A building has ocean view if its height > current max (all to the right are
shorter).
# Collect indices and reverse at the end.
# Time: O(n). Space: O(1) extra (output doesn't count)."
```

PHASE 4 — CLEAN CODE

```
class _1762_BuildingsWithOceanView:
    def findBuildings(self, heights: list[int]) -> list[int]:
        result = []
        max_right = 0
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > max_right:
                result.append(i)
                max_right = heights[i]
        return result[::-1]
```

PHASE 5 — TEST & DEBUG

```
# heights=[4,2,3,1]:
# i=3(1): 1>0=max_right → append 3, max=1.
# i=2(3): 3>1 → append 2, max=3.
# i=1(2): 2<3 → skip.
# i=0(4): 4>3 → append 0, max=4.
# reversed: [0,2,3] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single right-to-left scan. Space O(1) ignoring output.
# Monotonic stack variant: scan left-to-right, pop any building shorter than
current.
# Remaining stack has ocean view – but that's O(n) space for the stack vs O(1) scan.
# Follow-up: buildings with view to BOTH left and right – need max from both sides.
# Follow-up: circular arrangement – more complex, need two-pass or range max query."
```

◆ Quick Review

Key Insights

- Processing the buildings from right to left simplifies the problem because the rightmost building always has an ocean view.
- Keep track of the highest building encountered while iterating from right to left.

- A building has an ocean view if its height is greater than the maximum height seen so far.
- Append indices meeting the criteria, then reverse the order of indices for the solution since they were collected in reverse order.

Space & Time

Time Complexity: $O(n)$ where n is the number of buildings.

Space Complexity: $O(n)$ for storing the result in the worst-case scenario.

Solution

Iterate over the "heights" array from right to left while maintaining a variable to store the maximum height encountered so far. For each building, if its height is greater than this maximum height, add its index to the results list and update the maximum height. Since indices are collected in reverse order, reverse the list before returning it. This approach ensures that each building is visited only once.

Heap

Round 5 · Mid · Medium · 11 questions

Heap

□ #215 Kth Largest Element in an Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Sorting · Heap ·
Quickselect

Lists: Grind 169

Problem

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
Output: 4

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the kth largest element in an unsorted array. Not the kth distinct element. Right?"

#

"nums=[3,2,1,5,6,4], k=2: sorted desc=[6,5,4,3,2,1], 2nd largest=5 ✓

nums=[3,2,3,1,2,4,5,5,6], k=4: sorted desc=[6,5,5,4,3,3,2,2,1], 4th=4 ✓"

PHASE 2 — BRUTE FORCE

"Sort the array in descending order, return element at index k-1.

Time: $O(n \log n)$. Space: $O(1)$ or $O(n)$ depending on sort."

```
class _215_KthLargest_Brute:
```

```
    def findKthLargest(self, nums: list[int], k: int) -> int:
        nums.sort(reverse=True)
        return nums[k - 1]
```

PHASE 3 — OPTIMIZE

"Min-heap of size k: maintain the k largest elements seen so far.

The heap's root (minimum of the k largest) is the kth largest.

Push each element; if heap grows beyond k, pop the smallest.

Time: $O(n \log k)$. Space: $O(k)$. Better than $O(n \log n)$ when $k \ll n$."

PHASE 4 — CLEAN CODE

```
class _215_KthLargest:
```

```
    def findKthLargest(self, nums: list[int], k: int) -> int:
        import heapq
        heap = []
        for num in nums:
            heapq.heappush(heap, num)
            if len(heap) > k:
                heapq.heappop(heap)
        return heap[0]
```

PHASE 5 — TEST & DEBUG

nums=[3,2,1,5,6,4], k=2:

push 3→[3]. push 2→[2,3]. push 1→[1,3,2]→len>2→pop 1→[2,3].

push 5→[2,3,5]→pop 2→[3,5]. push 6→[3,5,6]→pop 3→[5,6].

push 4→[4,6,5]→pop 4→[5,6]. heap[0]=5 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Heap: $O(n \log k)$ time, $O(k)$ space. Sort: $O(n \log n)$ time, $O(1)$ space.

```
# Heap wins when k is small. Sort wins when  $k \approx n$  (similar complexity, simpler code).  
# Quickselect:  $O(n)$  average,  $O(n^2)$  worst – best for large  $n$  and single query.  
# Follow-up: Kth Smallest (LC 378 in sorted matrix) – min-heap on matrix rows.  
# Follow-up: streaming kth largest (LC 703) – maintain a min-heap of size  $k$  permanently."
```

◆ Quick Review

Key Insights

- k th largest element can be converted to finding the element at index $n-k$ if the array were sorted.
- A min-heap of size k is a practical method to maintain the k largest elements seen so far.
- The Quickselect algorithm can find the k th largest element in average $O(n)$ time with in-place partitioning.
- Choose an appropriate method based on performance needs and worst-case scenarios.

Space & Time

Time Complexity:

Min-Heap: $O(n \log k)$ by maintaining a heap of size k .

Quickselect: Average $O(n)$ but worst-case $O(n^2)$ with poor pivot selection.

Space Complexity:

Min-Heap: $O(k)$

Quickselect: $O(1)$ additional space with in-place partitioning.

Solution

We can solve the problem using either a min-heap or the Quickselect algorithm.

Min-Heap Approach: Initialize a min-heap with the first k elements. Iterate through the remaining elements; if an element is greater than the heap's root (the smallest in the heap), replace the root. After processing all elements, the root of the min-heap is the k th largest element. **Quickselect Approach:** Choose a pivot and partition the array so that elements larger than the pivot come before it. Determine the position of the pivot; if it matches the index corresponding to the k th largest element, return it. Recurse into the appropriate part of the array. Key points to consider include maintaining heap size for the first approach and proper pivot selection for the Quickselect approach.

Heap

★ #253 Meeting Rooms II ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Greedy · Sorting · Heap
Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, return the minimum number of conference rooms required.

Example 1:

Input: intervals = $[[0,30],[5,10],[15,20]]$
Output: 2

Example 2:

Input: intervals = $[[7,10],[2,4]]$
Output: 1

Constraints:

- $1 \leq \text{intervals.length} \leq 104$
- $0 \leq \text{start}_i < \text{end}_i \leq 106$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given intervals of meeting times, find the minimum number of conference rooms required.

Two meetings can share a room only if one ends before or when the other starts. Right?"

#

"intervals=[[0,30],[5,10],[15,20]]:

[0,30] and [5,10] overlap → need 2 rooms. [15,20] overlaps [0,30] → still 2 rooms. → 2 ✓

intervals=[[7,10],[2,4]]: no overlap → 1 ✓"

PHASE 2 — BRUTE FORCE

"Try all pairs to find overlapping sets. For each meeting, count how many

other meetings it overlaps with. The answer is the maximum overlap count + 1.

Time: $O(n^2)$. Space: $O(1)$."

class _253_MeetingRoomsII_Brute:

def minMeetingRooms(self, intervals: list[list[int]]) -> int:

max_rooms = 0

for i, (s1, e1) in enumerate(intervals):

rooms = 1

for j, (s2, e2) in enumerate(intervals):

if i != j and s2 < e1 and s1 < e2: # overlap condition

rooms += 1

max_rooms = max(max_rooms, rooms)

return max_rooms

PHASE 3 — OPTIMIZE

"Min-heap of end times — tracks when rooms free up.

Sort meetings by start time. For each meeting:

- If heap top (earliest end) <= current start, that room is free → pop and reuse.

- Always push current meeting's end time.

Heap size at any point = rooms currently in use.

Time: $O(n \log n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

class _253_MeetingRoomsII:

def minMeetingRooms(self, intervals: list[list[int]]) -> int:

import heapq

intervals.sort(key=lambda x: x[0])

heap = [] # min-heap of end times

for start, end in intervals:

if heap and heap[0] <= start:

heapq.heapreplace(heap, end) # reuse earliest-ending room

else:

heapq.heappush(heap, end) # need a new room

return len(heap)

PHASE 5 — TEST & DEBUG

```
# [[0,30],[5,10],[15,20]] sorted=same:
# [0,30]: heap empty → push 30 → [30].
# [5,10]: heap[0]=30 > 5 → push 10 → [10,30].
# [15,20]: heap[0]=10 <= 15 → replace 10 with 20 → [20,30].
# len(heap)=2 ✓
#
# [[7,10],[2,4]] sorted=[[2,4],[7,10]]:
# [2,4]: push 4 → [4].
# [7,10]: 4 <= 7 → replace 4 with 10 → [10]. len=1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \log n)$ : sort +  $n$  heap ops each  $O(\log n)$ .
# Space  $O(n)$ : heap holds at most  $n$  end times.
# Alternative: split starts and ends into sorted arrays, use two pointers –  $O(n \log n)$ ,  $O(n)$ .
# Follow-up: Meeting Rooms I (LC 252) – can ONE person attend all? Sort by start, check no overlaps.
# Follow-up: minimum cost scheduling – extend with room priorities or weighted intervals."
```

◆ Quick Review

Key Insights

- Sort the intervals based on start times to process meetings in order.
- Use a min-heap (priority queue) to keep track of meeting end times.
- When processing a new meeting, check if its start time is greater than or equal to the smallest end time from the heap.
- If so, it means a meeting room has been freed (the meeting with the smallest end time has ended) and can be reused.

- If not, allocate a new room by pushing the current meeting's end time into the heap.
- The size of the heap at any moment represents the number of rooms currently in use.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting and each heap operation (push/pop) taking $O(\log n)$.

Space Complexity: $O(n)$ in the worst-case scenario when all meetings overlap, requiring all end times in the heap.

Solution

The solution involves first sorting the meeting intervals by their start times. Then, we utilize a min-heap (priority queue) to keep track of the end times of ongoing meetings. For each meeting, we compare its start time with the smallest end time in the heap: If the meeting can start after the meeting at the top of the heap has ended (i.e., its start time is greater than or equal to the smallest end), we remove that end time from the heap (freeing up a room). Then, we add the current meeting's end

time to the heap. By the end of processing all meetings, the size of the heap represents the minimum number of conference rooms required.

Heap

□ #347 Top K Frequent Elements

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Divide and Conquer ·
Sorting · Heap · Bucket Sort · Counting ·
Quickselect

Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- k is in the range $[1, \text{the number of unique elements in the array}]$.
- It is guaranteed that the answer is unique.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where n is the array's size.

My LeetCode Solution
Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given an integer array, return the k most frequent elements in any order.
# Guaranteed that k is valid and the answer is unique. Right?"
#
# "nums=[1,1,1,2,2,3], k=2: frequencies: 1→3, 2→2, 3→1 → [1,2] ✓
# nums=[1], k=1 → [1] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Count frequencies with a dict, sort by frequency descending, take top k.
# Time:  $O(n \log n)$ . Space:  $O(n)$ ."
```

```
class _347_TopKFrequent_Brute:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        from collections import Counter
        freq = Counter(nums)
        return [x for x, _ in freq.most_common(k)]
```

PHASE 3 — OPTIMIZE

```
# "Min-heap of size k: maintain k most frequent elements.
# Push (count, num); when heap exceeds k, pop the least frequent.
# Heap root is the kth most frequent – pop it last.
# Time:  $O(n \log k)$ . Space:  $O(n)$  counter +  $O(k)$  heap.
#
# Bucket sort alternative:  $O(n)$ . Array of  $n+1$  buckets indexed by frequency."
```

Each bucket holds numbers with that frequency. Scan from highest to lowest."

PHASE 4 — CLEAN CODE

```
class _347_TopKFrequent:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        from collections import Counter
        import heapq
        freq = Counter(nums)
        return heapq.nlargest(k, freq.keys(), key=lambda x: freq[x])
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,1,1,2,2,3], k=2:
# freq={1:3,2:2,3:1}. nlargest(2, keys, key=freq):
# heap keeps top 2 by freq -> [1,2] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Heap  $O(n \log k)$ : best when  $k \ll n$ . Sort  $O(n \log n)$ . Bucket sort  $O(n)$ : best overall.
# Bucket sort: buckets[freq].append(num); scan from n down, collect until k elements.
# Follow-up: Top K Frequent Words (LC 692) – same logic but break frequency ties alphabetically.
# Follow-up: streaming top-k – maintain a min-heap of size k as elements arrive."
```

◆ Quick Review

Key Insights

- Count the frequency of each element using a hash table.
- Use bucket sort to achieve linear time complexity by grouping numbers by their frequency.
- Traverse the frequency buckets from highest to lowest to collect the k most frequent elements.
- Alternative approaches include using heaps or Quickselect, but bucket sort meets the

follow-up requirement of better than $O(n \log n)$ time complexity.

Space & Time

Time Complexity: $O(n)$ where n is the length of the array (bucket sort and frequency counting are linear).

Space Complexity: $O(n)$ for storing the counts and buckets.

Solution

The solution starts by counting the frequency of each element with a hash table (or dictionary). Then, we create an array of buckets where the index represents the frequency. Each bucket holds the numbers that appear with that frequency. Finally, we iterate over the buckets in reverse order (from high frequency to low) and collect elements until we have k of them. This bucket sort approach avoids the higher complexity of sorting a list of frequencies and guarantees linear time performance.

Heap

★ #505 The Maze II ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Graph · Matrix · Heap ·
Shortest Path

Lists: ★ Premium | Premium 98

Problem

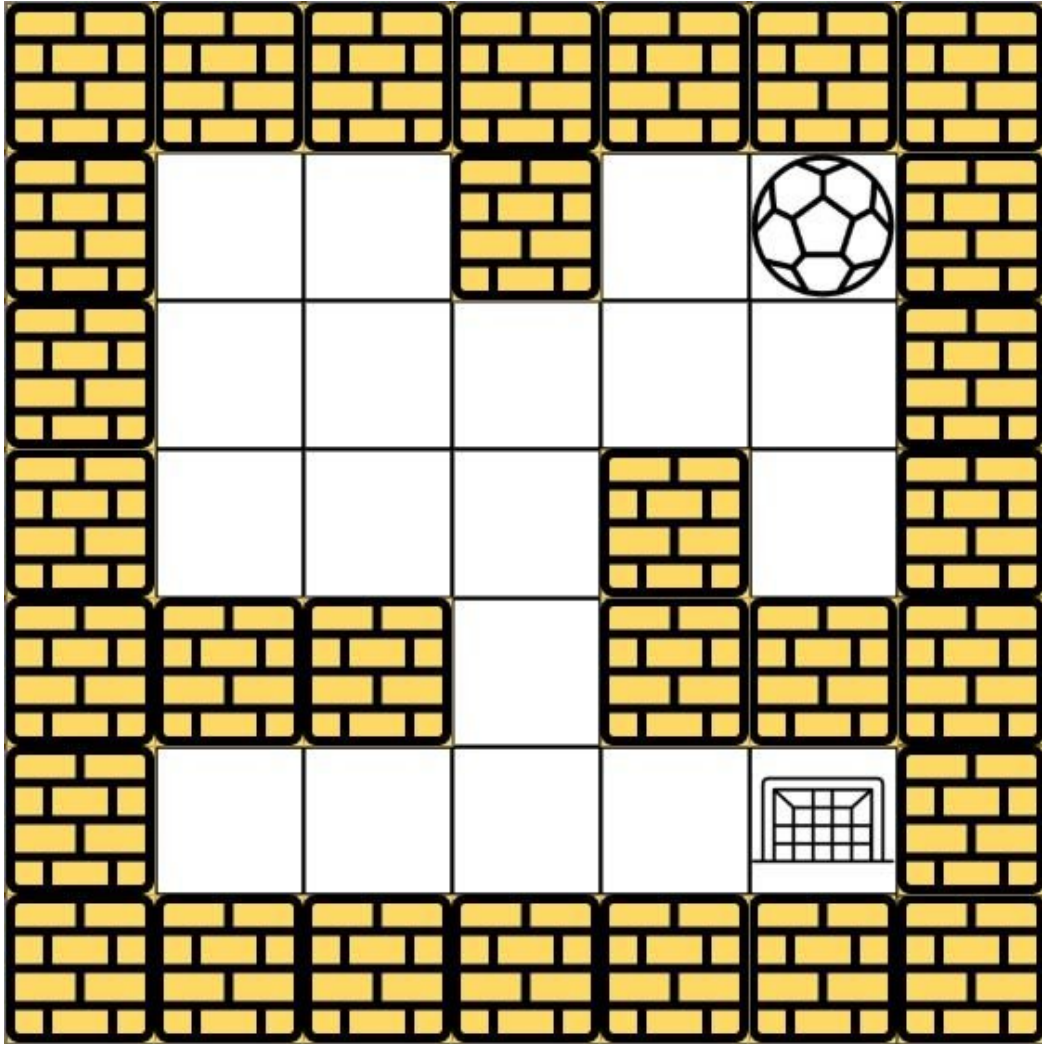
There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the $m \times n$ maze, the ball's start position and the destination, where $\text{start} = [\text{startrow}, \text{startcol}]$ and $\text{destination} = [\text{destinationrow}, \text{destinationcol}]$, return the shortest distance for the ball to stop at the destination. If the ball cannot stop at destination, return -1 .

The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

You may assume that the borders of the maze are all walls (see examples).

Example 1:



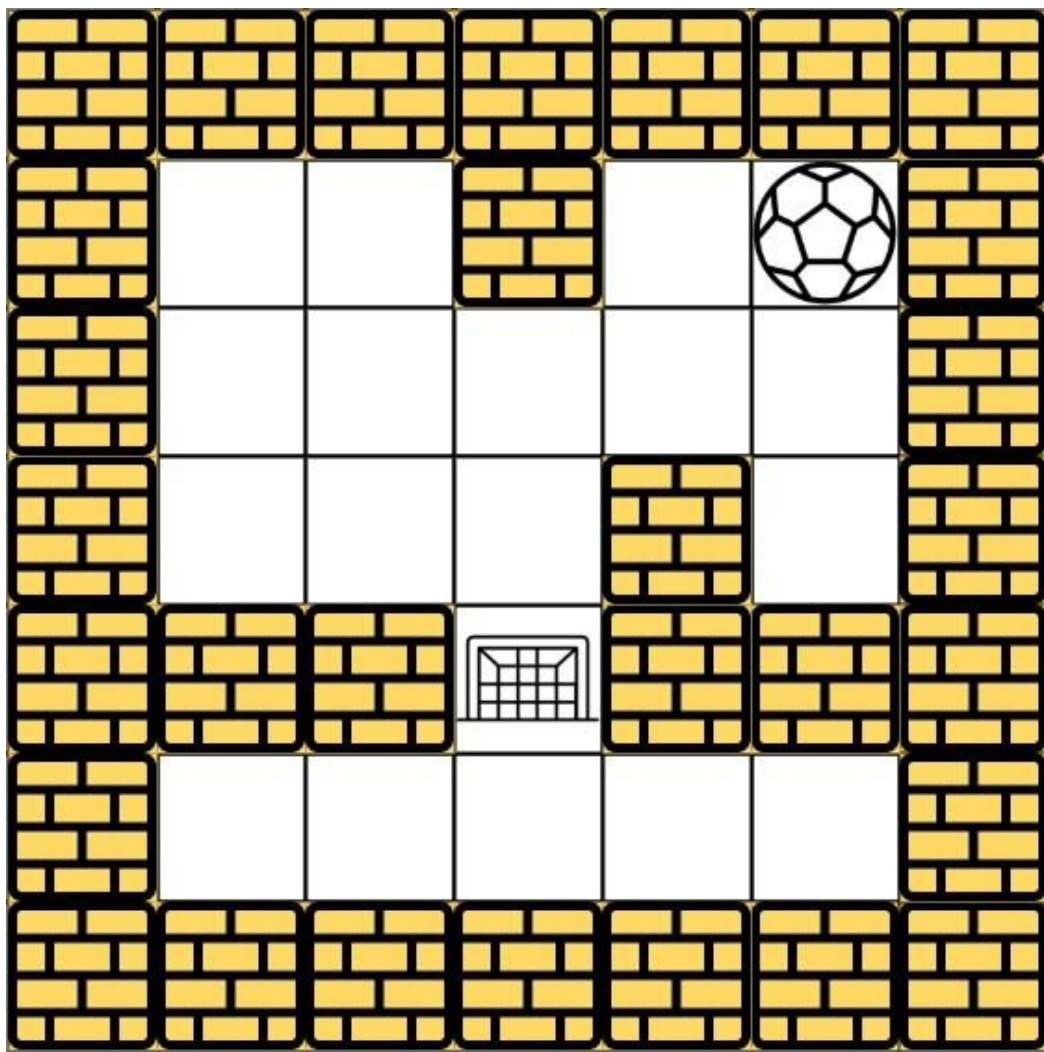
Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]`,
 start = `[0,4]`, destination = `[4,4]`

Output: 12

Explanation: One possible way is : left -> down -> left -> down -> right -> down -> right.

The length of the path is $1 + 1 + 3 + 1 + 2 + 2 + 2 = 12$.

Example 2:



Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]`,
 start = `[0,4]`, destination = `[3,2]`

Output: `-1`

Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.

Example 3:

Input: maze = `[[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]]`,
 start = `[4,3]`, destination = `[0,1]`

Output: `-1`

Constraints:

- `m == maze.length`
- `n == maze[i].length`

- $1 \leq m, n \leq 100$
- `maze[i][j]` is 0 or 1.
- `start.length == 2`
- `destination.length == 2`
- $0 \leq \text{startrow}, \text{destinationrow} < m$
- $0 \leq \text{startcol}, \text{destinationcol} < n$
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "A ball rolls in a maze until it hits a wall. Find the SHORTEST DISTANCE from
start to
# destination (number of steps). Return -1 if impossible. Right?"
#
# "maze 5x5, start=(0,4), dest=(4,4): ball rolls left/right/up/down until wall.
# Shortest path = 12 steps ✓"
```

PHASE 2 — BRUTE FORCE

```
# "BFS exploring all rolling directions from each stop point.
# Track visited cells, return distance when destination is reached.
# Time:  $O(m \times n \times (m+n))$  – at each cell, rolling can traverse up to  $\max(m, n)$  cells.
Space:  $O(m \times n)$ ."
```

```
class _505_MazeII_Brute:
    def shortestDistance(self, maze, start, destination) -> int:
        from collections import deque
        m, n = len(maze), len(maze[0])
        dist = [[float('inf')] * n for _ in range(m)]
```

```

dist[start[0]][start[1]] = 0
queue = deque([start])
while queue:
    r, c = queue.popleft()
    for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
        nr, nc, steps = r, c, 0
        while 0<=nr+dr<m and 0<=nc+dc<n and maze[nr+dr][nc+dc]==0:
            nr += dr; nc += dc; steps += 1
            if dist[r][c] + steps < dist[nr][nc]:
                dist[nr][nc] = dist[r][c] + steps
                queue.append([nr, nc])
d = dist[destination[0]][destination[1]]
return d if d < float('inf') else -1

```

PHASE 3 — OPTIMIZE

"Dijkstra's algorithm — process stop points in order of distance.
Min-heap of (distance, row, col). For each popped cell, roll in all four directions.
Stop when destination is popped (guaranteed shortest path at that point).
Time: $O(m \cdot n \cdot \log(m \cdot n) \cdot (m + n))$. Space: $O(m \cdot n)$."

PHASE 4 — CLEAN CODE

```

class _505_MazeII:
    def shortestDistance(self, maze, start, destination) -> int:
        import heapq
        m, n = len(maze), len(maze[0])
        dist = [[float('inf')] * n for _ in range(m)]
        dist[start[0]][start[1]] = 0
        heap = [(0, start[0], start[1])]
        while heap:
            d, r, c = heapq.heappop(heap)
            if [r, c] == destination: return d
            if d > dist[r][c]: continue
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc, steps = r, c, 0
                while 0<=nr+dr<m and 0<=nc+dc<n and maze[nr+dr][nc+dc] == 0:
                    nr += dr; nc += dc; steps += 1
                    new_d = d + steps
                    if new_d < dist[nr][nc]:
                        dist[nr][nc] = new_d
                        heapq.heappush(heap, (new_d, nr, nc))
        return -1

```

PHASE 5 — TEST & DEBUG

start=(0,4),dest=(4,4): Dijkstra explores paths in order of distance.
Ball at (0,4) rolls down → hits wall at (4,4) if path clear, steps=4, or hits earlier.
Different directions explored; heap ensures shortest extracted first → returns 12

✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(m*n*(m+n)*\log(m*n))$ : heap has  $O(m*n)$  entries, rolling  $O(m+n)$  per
direction.
# Space  $O(m*n)$ : dist array + heap.
# BFS gives any shortest path; Dijkstra gives minimum distance (same for unit
weights,
# needed here because rolling distances vary).
# Follow-up: Maze I (LC 490) – just reachability, BFS suffices.
# Follow-up: Maze III (LC 499) – find lexicographically smallest direction
sequence."
```

◆ Quick Review

Key Insights

- The ball rolls until it hits a wall, so each move is not one step but a continuous travel in a direction.
- The problem can be modeled as finding the shortest path in a weighted graph where nodes are positions in the maze and edge weights are the distance traveled in that roll.
- A priority queue (min-heap) based algorithm, such as Dijkstra's algorithm, is ideal since each roll covers variable distances.
- Always check if the new computed distance for a cell is less than a previously recorded one before pushing it to the heap.

Space & Time

Time Complexity: $O(m * n * \log(m * n))$, where m and n are the maze dimensions (each cell

may get processed and queued with a logarithmic cost).

Space Complexity: $O(m * n)$ for maintaining the distance matrix and the priority queue.

Solution

We solve the problem using a modified version of Dijkstra's algorithm. The maze grid is treated as a graph where each cell is a node and an edge exists between the current cell and the cell where the ball stops after rolling in one direction. For every cell popped from the priority queue, we roll the ball in all four directions until it hits a wall, counting the number of steps taken. If the distance calculated from this roll is shorter than a previously stored distance for the stopping cell, we update the distance and add it to the priority queue. This process continues until we either reach the destination (and return the distance) or exhaust all possibilities (returning -1).

Heap

□ #621 Task Scheduler

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Heap · Counting
Lists: Grind 169

Problem

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n . Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], $n = 2$

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- $1 \leq \text{tasks.length} \leq 104$
- tasks[i] is an uppercase English letter.

- $0 \leq n \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given tasks (letters) and cooldown n: same task must have at least n slots between runs.

CPU can idle. Return the minimum number of intervals to finish all tasks. Right?"
#

"tasks=['A','A','A','B','B','B'], n=2:

A→B→idle→A→B→idle→A→B = 8 intervals ✓

tasks=['A','A','A','B','B','B'], n=0: AAABBB = 6 ✓"

PHASE 2 — BRUTE FORCE

"Use a max-heap of (count, task). Each cycle of (n+1) slots: pop up to n+1 most frequent

tasks, execute them, decrement counts, push back non-zero counts.

If fewer than n+1 tasks available, pad with idles.

Time: $O(m * n)$ where m = total tasks. Space: $O(26)$."

class _621_TaskScheduler_Brute:

def leastInterval(self, tasks: list[str], n: int) -> int:

import heapq

from collections import Counter

freq = Counter(tasks)

heap = [-cnt for cnt in freq.values()]

heapq.heapify(heap)

time = 0

while heap:

 cycle, temp = 0, []

 for _ in range(n + 1):

 if heap:

 cnt = heapq.heappop(heap)

 temp.append(cnt + 1) # decrement count (stored negative)

 cycle += 1

 for cnt in temp:

 if cnt < 0:

 heapq.heappush(heap, cnt)

 time += n + 1 if heap else cycle

return time

PHASE 3 — OPTIMIZE

```
# "Math formula: the most frequent task determines the frame count.
# Let max_freq = highest frequency. Let max_count = number of tasks with max_freq.
# Formula: max(len(tasks), (max_freq - 1) * (n + 1) + max_count).
# The formula gives the minimum slots; if tasks fill all slots, no idle needed.
# Time: O(m). Space: O(26)."
```

PHASE 4 — CLEAN CODE

```
class _621_TaskScheduler:
    def leastInterval(self, tasks: list[str], n: int) -> int:
        from collections import Counter
        freq = Counter(tasks)
        max_freq = max(freq.values())
        max_count = sum(1 for v in freq.values() if v == max_freq)
        return max(len(tasks), (max_freq - 1) * (n + 1) + max_count)
```

PHASE 5 — TEST & DEBUG

```
# tasks=['A'*3,'B'*3], n=2: max_freq=3, max_count=2.
# (3-1)*(2+1)+2 = 2*3+2 = 8. len(tasks)=6. max(6,8)=8 ✓
#
# tasks=['A'*3,'B'*3], n=0: (3-1)*1+2=4. len=6. max(6,4)=6 ✓
#
# tasks=['A'*3,'B'*3,'C'*3], n=2: max_freq=3,max_count=3.
# (3-1)*3+3=9. len=9. max(9,9)=9 ✓ (ABCABCABC, no idles needed)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Formula O(m): just count frequencies. Heap simulation O(m*n): slower but more
intuitive.
# The formula works because tasks beyond the 'frame' always fit without extra idles.
# Follow-up: return the actual schedule (not just count) – use the heap simulation.
# Follow-up: if tasks have deadlines, this becomes a more complex scheduling
problem."
```

◆ Quick Review

Key Insights

- Count the frequency of each task.
- The task with the highest frequency determines the base structure.
- Identify how many tasks share the maximum frequency.

- Calculate the total time based on the formula: $\max(\text{total tasks}, (\text{max frequency} - 1) * (n + 1) + \text{count of tasks with maximum frequency})$.
- This approach avoids simulating the entire scheduling by using a greedy strategy.

Space & Time

Time Complexity: $O(N)$ where N is the number of tasks (counting frequency, then iterating over constant 26 letters).

Space Complexity: $O(1)$ since the frequency count uses an array or hash map with fixed size (26 letters).

Solution

We first count the frequency of each task. The task(s) with the maximum frequency establish the minimum structure required. Compute the ideal idle slots based on the count of these maximum tasks and the cooling period n . Finally, compare this computed length with the total number of tasks to get the final answer. This greedy strategy avoids explicit simulation and leverages mathematical formulation.

Heap

□ #658 Find K Closest Elements

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sorting · Heap · Two
Pointers

Lists: Grind 169

Problem

Given a sorted integer array `arr`, two integers `k` and `x`, return the `k` closest integers to `x` in the array. The result should also be sorted in ascending order.

An integer `a` is closer to `x` than an integer `b` if:

- $|a - x| < |b - x|$, or
- $|a - x| == |b - x|$ and $a < b$

Example 1:

Input: `arr = [1,2,3,4,5]`, `k = 4`, `x = 3`

Output: `[1,2,3,4]`

Example 2:

Input: `arr = [1,1,2,3,4,5]`, `k = 4`, `x = -1`

Output: `[1,1,2,3]`

Constraints:

- $1 \leq k \leq \text{arr.length}$
- $1 \leq \text{arr.length} \leq 104$
- arr is sorted in ascending order.
- $-104 \leq \text{arr}[i], x \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a sorted array and integers k and x, return the k closest elements to x.
# Ties broken by smaller value. Return sorted. Right?"
#
# "arr=[1,2,3,4,5], k=4, x=3 → [1,2,3,4] ✓
# arr=[1,2,3,4,5], k=4, x=-1 → [1,2,3,4] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Sort all elements by |val-x| (then by val for ties), take first k, return sorted.
# Time: O(n log n). Space: O(n)."
```

```
class _658_FindKClosestElements_Brute:
```

```
    def findClosestElements(self, arr: list[int], k: int, x: int) -> list[int]:
        return sorted(sorted(arr, key=lambda v: (abs(v - x), v))[:k])
```

PHASE 3 — OPTIMIZE

```
# "Binary search on the left boundary of the k-element window.
# The answer is always a contiguous subarray of length k.
# Binary search for left in [0, n-k]: compare arr[mid] and arr[mid+k] distances to x.
# If arr[mid+k]-x < x-arr[mid], the window should shift right (left = mid+1).
# Otherwise left = mid. Final window: arr[left:left+k].
# Time: O(log(n-k) + k). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _658_FindKClosestElements:
```

```
    def findClosestElements(self, arr: list[int], k: int, x: int) -> list[int]:
        lo, hi = 0, len(arr) - k
        while lo < hi:
            mid = (lo + hi) // 2
            if x - arr[mid] > arr[mid + k] - x:
                lo = mid + 1
```

```

else:
    hi = mid
return arr[lo:lo + k]

```

PHASE 5 — TEST & DEBUG

```

# arr=[1,2,3,4,5],k=4,x=3: lo=0,hi=1.
# mid=0: x-arr[0]=3-1=2, arr[4]-x=5-3=2. 2>2 is False → hi=0.
# lo=hi=0. return arr[0:4]=[1,2,3,4] ✓
#
# arr=[1,2,3,4,5],k=4,x=-1: lo=0,hi=1.
# mid=0: x-arr[0]=-1-1=-2 (but we compare abs): -1-1=-2 < 0 so condition is
-2>arr[4]-(-1)=6? No.
# Actually x-arr[mid] = -1-1=-2; arr[mid+k]-x=5-(-1)=6. -2>6 False → hi=0. return
[1,2,3,4] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Binary search O(log(n-k)): find the left boundary. Slice O(k).
# Total O(log(n-k)+k) vs brute O(n log n).
# The key insight: the answer window shifts right when right boundary is closer to x
than left.
# Follow-up: if array is unsorted, use a heap: O(n log k).
# Follow-up: online stream of queries with same array – binary search still O(log n)
per query."

```

◆ Quick Review

Key Insights

- The input array is already sorted.
- A binary search can be used to efficiently narrow down the potential starting position.
- A two-pointers or sliding window approach is effective for selecting k consecutive elements.
- The comparison of absolute differences (with a tie-breaker on values) is crucial.
- Maintaining ascending order simplifies the final output.

Space & Time

Time Complexity: $O(\log n + k)$, where $O(\log n)$ for binary search and $O(k)$ for collecting results.

Space Complexity: $O(k)$, for storing the output.

Solution

We can solve this problem by first using binary search to identify the left-most window where the k closest elements might start. The binary search operates over the possible starting indices for windows of size k (from 0 to $n-k$). For each middle index mid , we compare the distances between x and $arr[mid]$ and $arr[mid + k]$ to decide whether to move the window to the right or left. Once the correct starting position is found, return the k elements starting from that index. This approach leverages the sorted property of the array to efficiently converge on the optimal window.

Heap

□ #787 Cheapest Flights Within K Stops

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · DFS · BFS · Graph ·
Heap

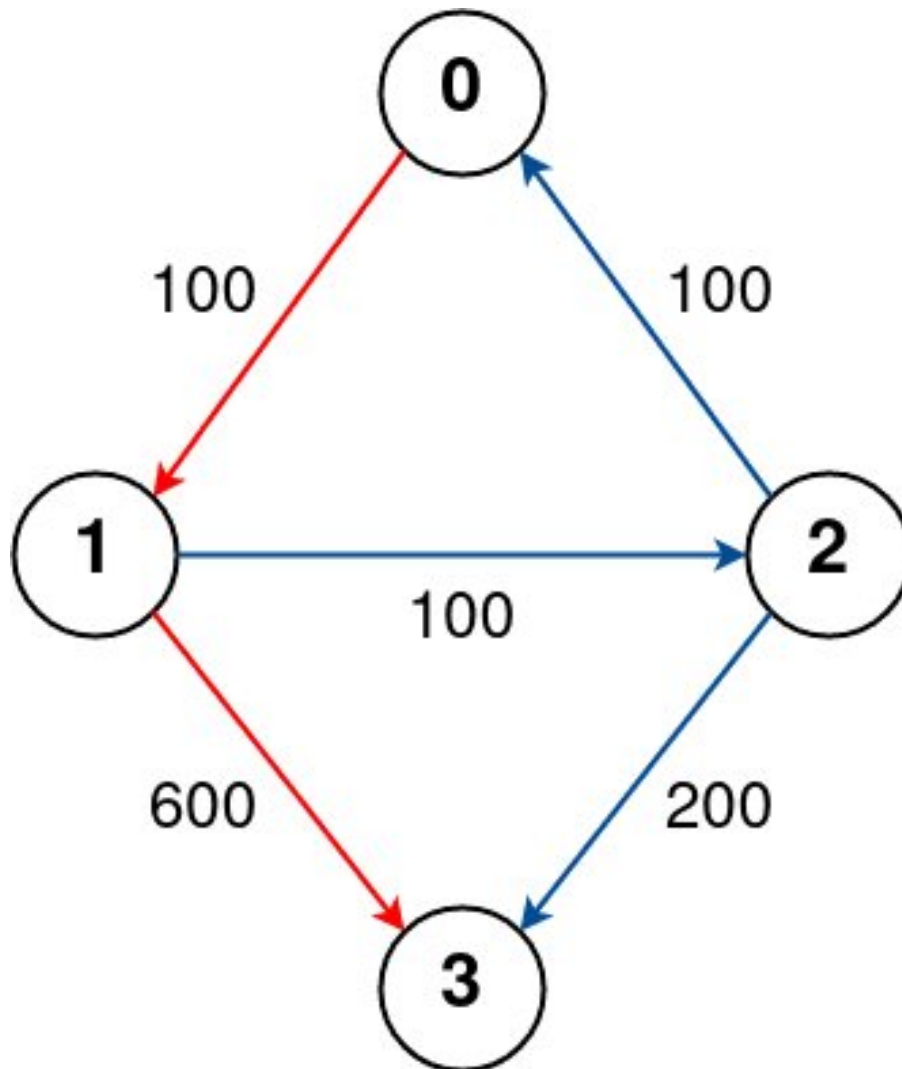
Lists: Grind 169

Problem

There are n cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return the cheapest price from `src` to `dst` with at most `k` stops. If there is no such route, return `-1`.

Example 1:



Input: $n = 4$, $flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]$,
 $src = 0$, $dst = 3$, $k = 1$

Output: 700

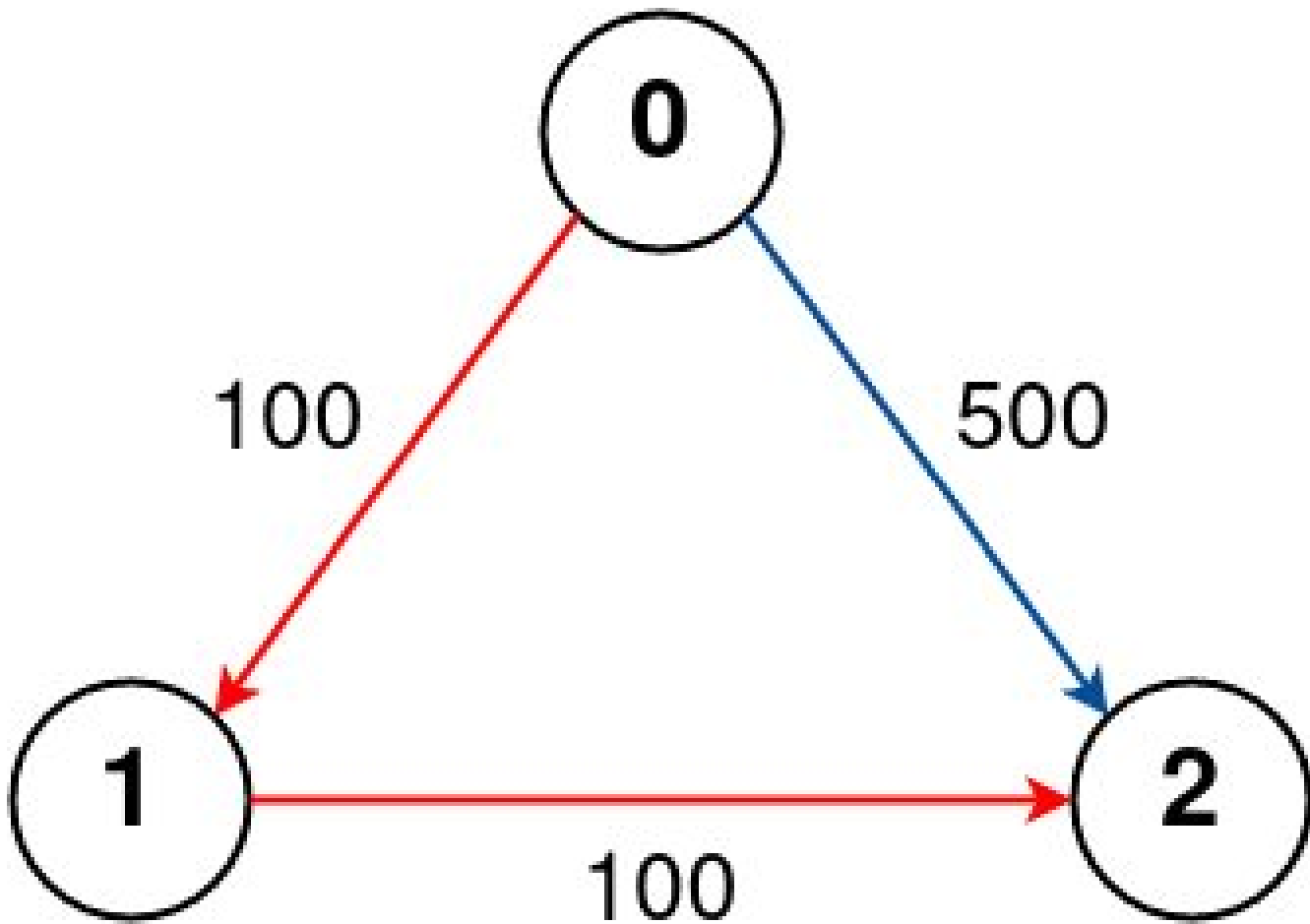
Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost $100 + 600 = 700$.

Note that the path through cities $[0,1,2,3]$ is cheaper but is invalid because it uses 2 stops.

Example 2:



Input: $n = 3$, $flights = [[0,1,100],[1,2,100],[0,2,500]]$, $src = 0$, $dst = 2$, $k = 1$

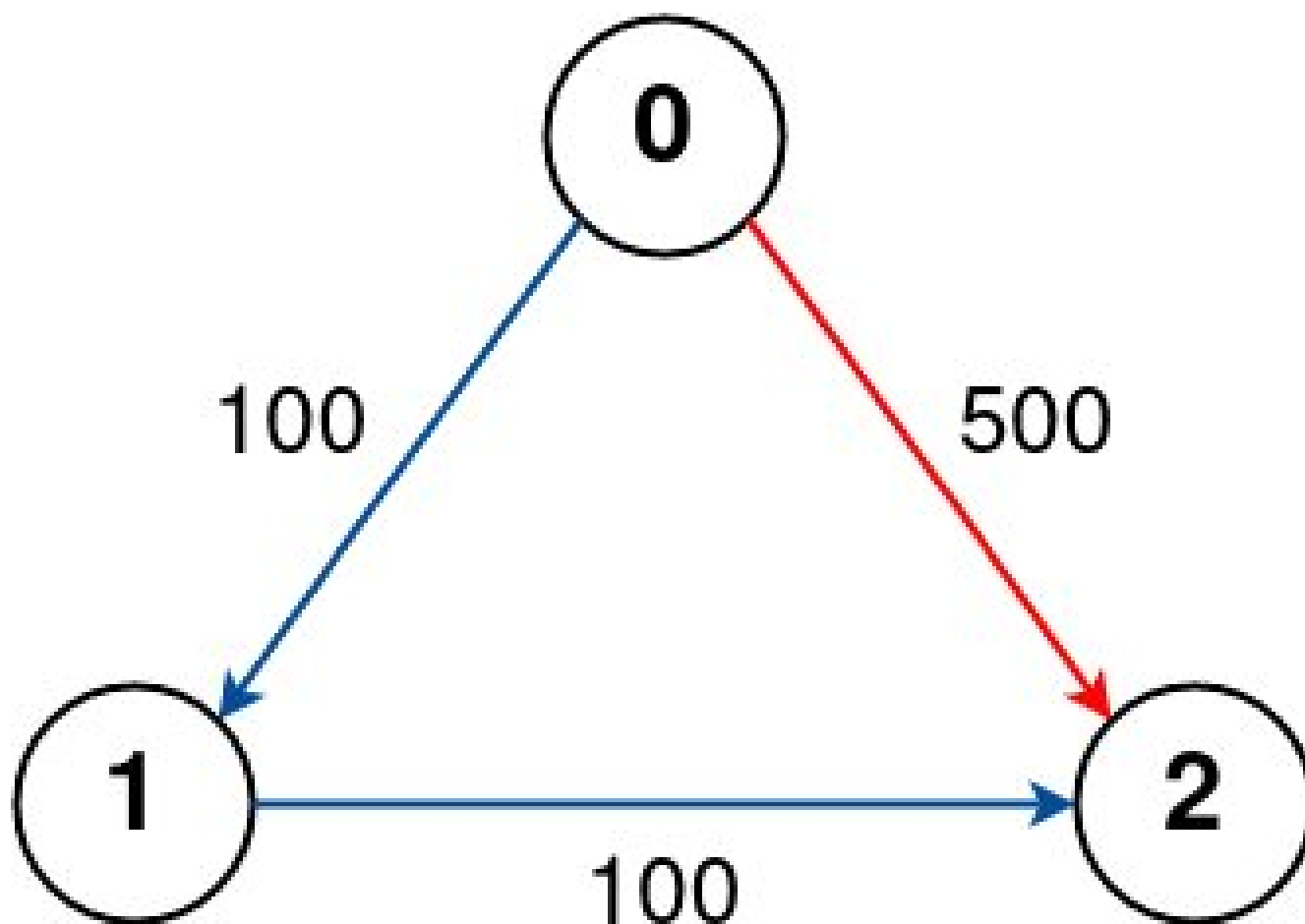
Output: 200

Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost $100 + 100 = 200$.

Example 3:



Input: $n = 3$, $flights = [[0,1,100],[1,2,100],[0,2,500]]$, $src = 0$, $dst = 2$, $k = 0$

Output: 500

Explanation:

The graph is shown above.

The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.

Constraints:

- $2 \leq n \leq 100$
- $0 \leq flights.length \leq (n * (n - 1) / 2)$
- $flights[i].length == 3$
- $0 \leq from_i, to_i < n$
- $from_i \neq to_i$

- $1 \leq \text{price}_i \leq 104$
- There will not be any multiple flights between two cities.
- $0 \leq \text{src}, \text{dst}, k < n$
- $\text{src} \neq \text{dst}$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "n cities, list of flights [from,to,price]. Find cheapest price from src to dst
# with at most k stops. -1 if impossible. Right?"
#
# "n=4, flights=[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src=0, dst=3,
# k=1:
# 0→1→3 costs 700 (1 stop ✓). 0→1→2→3 costs 400 but needs 2 stops > k=1. → 700 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "BFS/DFS exploring all paths up to k+1 edges. Track minimum cost to reach dst.
# Time:  $O(n^k)$  worst case. Space:  $O(n)$ ."
```

```
class _787_CheapestFlightsKStops_Brute:
    def findCheapestPrice(self, n, flights, src, dst, k):
        from collections import defaultdict
        graph = defaultdict(list)
        for u, v, w in flights:
            graph[u].append((v, w))
        best = float('inf')
        def dfs(node, stops, cost):
            nonlocal best
            if node == dst:
                best = min(best, cost); return
            if stops < 0 or cost >= best: return
            for nxt, w in graph[node]:
                dfs(nxt, stops - 1, cost + w)
        dfs(src, k, 0)
        return best if best < float('inf') else -1
```

PHASE 3 — OPTIMIZE

```
# "Bellman-Ford with k+1 relaxation rounds – DP approach.
# prices[v] = min cost to reach v using at most i edges.
# Each round: update prices based on last round's values (use a copy to avoid using
# same-round updates).
# Time: O(k * |flights|). Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _787_CheapestFlightsKStops:
    def findCheapestPrice(self, n: int, flights: list[list[int]], src: int, dst:
int, k: int) -> int:
        INF = float('inf')
        prices = [INF] * n
        prices[src] = 0
        for _ in range(k + 1):
            tmp = prices[:]
            for u, v, w in flights:
                if prices[u] + w < tmp[v]:
                    tmp[v] = prices[u] + w
            prices = tmp
        return prices[dst] if prices[dst] < INF else -1
```

PHASE 5 — TEST & DEBUG

```
# n=4, flights as above, src=0, dst=3, k=1:
# Init: prices=[0,INF,INF,INF].
# Round1 (1 edge): 0→1:100→tmp[1]=100. 1→3:600→tmp[3]=INF(prices[1]=INF). 0→1→
done.
# tmp=[0,100,INF,INF]. prices=[0,100,INF,INF].
# Round2 (2 edges=k+1): 1→2:100→tmp[2]=200. 1→3:600→tmp[3]=700.
2→3:200→prices[2]=INF skip.
# prices=[0,100,200,700]. return 700 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(k * E): k+1 Bellman-Ford rounds over all edges.
# Space O(n): two price arrays (current and temp).
# Dijkstra with (cost, stops, node) heap also works – O((E+n) log n) but needs
careful stop tracking.
# Follow-up: no stop limit – standard Dijkstra or Bellman-Ford.
# Follow-up: if edge weights can be negative – only Bellman-Ford handles this
correctly."
```

◆ Quick Review

Key Insights

- Use a variant of BFS combined with a min-heap (priority queue) to always expand

the cheapest route.

- Each state in the queue includes the current cost, current city, and the number of stops used so far.
- Track the best-known cost to reach a city with a certain number of stops to prune non-optimal routes.
- Ensure that you do not exceed k stops during exploration.

Space & Time

Time Complexity: $O(E \log(E))$ in the worst-case scenario, where E is the number of flights, due to the heap operations.

Space Complexity: $O(n + E)$ for storing the graph and auxiliary data structures like the heap and best-cost map.

Solution

We solve the problem using a modified Dijkstra's algorithm. The approach employs a min-heap to ensure that we always process the state with the lowest cumulative cost next. Each state tracks the total cost so far, the current city, and the number of stops made. When a state reaches the destination city, we

return its cost. To avoid unnecessary work, we record the lowest cost at which each city has been reached with a given number of stops. This state-tracking helps prune routes that are more expensive or exceed the allowed number of stops.

Heap

□ #973 K Closest Points to Origin

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Divide and Conquer · Sorting ·
Heap

Lists: Grind 169

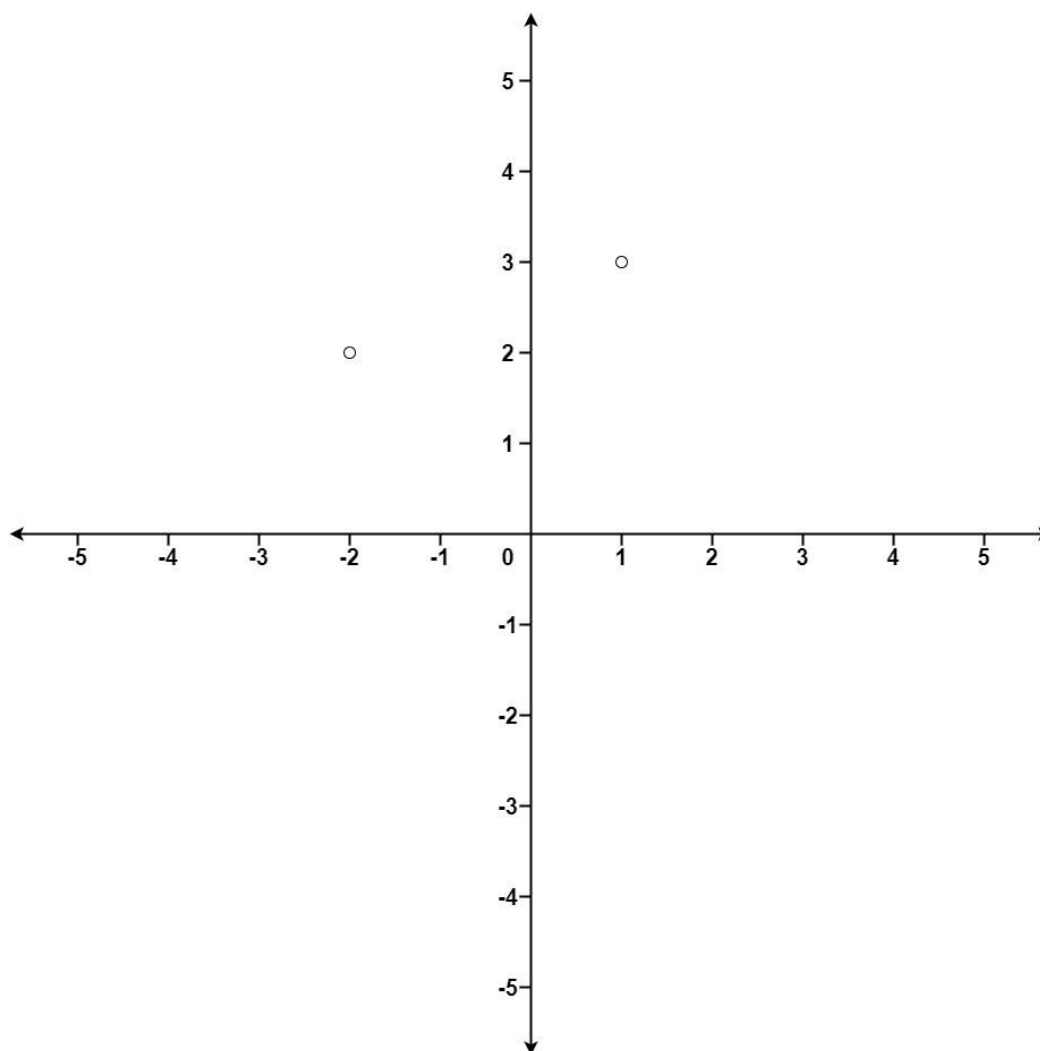
Problem

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1:



Input: points = [[1,3],[-2,2]], k = 1

Output: [[-2,2]]

Explanation:

The distance between (1, 3) and the origin is $\sqrt{10}$.

The distance between (-2, 2) and the origin is $\sqrt{8}$.

Since $\sqrt{8} < \sqrt{10}$, (-2, 2) is closer to the origin.

We only want the closest k = 1 points from the origin, so the answer is just [[-2,2]].

Example 2:

Input: points = [[3,3],[5,-1],[-2,4]], k = 2

Output: [[3,3],[-2,4]]

Explanation: The answer [[-2,4],[3,3]] would also be accepted.

Constraints:

- $1 \leq k \leq \text{points.length} \leq 104$

- $-104 \leq x_i, y_i \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given an array of points, return the k closest to the origin (0,0).
# Distance = sqrt(x2+y2) (no need to actually sqrt since we compare squares).
# Answer can be in any order. Right?"
#
# "points=[[1,3],[-2,2]], k=1: dist2=[10,8] → [[-2,2]] ✓
# points=[[3,3],[5,-1],[-2,4]], k=2: dist2=[18,26,20] → [[3,3],[-2,4]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Sort all points by squared distance, return first k.
# Time: O(n log n). Space: O(n)."
```

```
class _973_KClosestPoints_Brute:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        points.sort(key=lambda p: p[0]**2 + p[1]**2)
        return points[:k]
```

PHASE 3 — OPTIMIZE

```
# "Max-heap of size k: maintain k closest points.
# Push (-dist2, x, y). When heap exceeds k, pop the farthest.
# Time: O(n log k). Space: O(k). Better than O(n log n) when k << n.
# Quickselect alternative: O(n) average, O(n2) worst – partitions around a pivot distance."
```

PHASE 4 — CLEAN CODE

```
class _973_KClosestPoints:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        import heapq
        heap = []
        for x, y in points:
            dist = -(x*x + y*y)
            heapq.heappush(heap, (dist, x, y))
            if len(heap) > k:
                heapq.heappop(heap)
        return [[x, y] for _, x, y in heap]
```

PHASE 5 — TEST & DEBUG

```
# points=[[1,3],[-2,2]],k=1:
# push (-10,1,3)→[(-10,1,3)]. len=1≤1.
```

```
# push (-8,-2,2)→[(-10,1,3),(-8,-2,2)]. len=2>1 → pop(-10,1,3).
# heap=[(-8,-2,2)]. return [[-2,2]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Max-heap $O(n \log k)$: push all n , heap never exceeds k .

Sort $O(n \log n)$: simple but slower for small k .

Quickselect $O(n)$ average: optimal for single queries on large n .

Follow-up: if points stream in, maintain a max-heap of size k updating on each arrival.

Follow-up: K closest to a point other than origin – same logic, shift coordinates."

◆ Quick Review

Key Insights

- Calculate the squared Euclidean distance ($x^2 + y^2$) to avoid the overhead of square root computation.
- Sorting the points based on their distance gives an $O(n \log n)$ solution.
- Alternatively, using a max-heap to keep track of the k closest points results in $O(n \log k)$ time, which is effective if k is much smaller than n .
- Using Quickselect can achieve an average time complexity of $O(n)$, though the worst-case is $O(n^2)$.
- The problem does not require the output to be sorted in any specific order.

Space & Time

Time Complexity: $O(n \log n)$ with a sorting approach, $O(n \log k)$ with a heap approach, or average $O(n)$ with a Quickselect approach.

Space Complexity: $O(n)$ if extra space is used for sorting, or $O(k)$ for the heap approach.

Solution

We can solve this problem by first computing the squared Euclidean distance for each point to avoid unnecessary square root operations. Then, we can sort the points based on their computed distance and select the first k points. Alternatively, a max-heap or Quickselect algorithm can be used: **Max-Heap:** Maintain a heap of size k while iterating through the points, ensuring that only the k points with the smallest distances are kept. **Quickselect:** Partition the array like in QuickSort to position the k th closest point in its correct position. Once partitioned, all points to the left of that position will be the k closest. The sorting solution is straightforward and efficient enough given the constraints.

Heap

□ ★ #1057 Campus Bikes ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting · Heap
Lists: ★ Premium | Premium 98

Problem

On a campus represented on the X-Y plane, there are n workers and m bikes, with $n \leq m$. You are given an array `workers` of length n where `workers[i] = [xi, yi]` is the position of the i th worker. You are also given an array `bikes` of length m where `bikes[j] = [xj, yj]` is the position of the j th bike. All the given positions are unique.

Assign a bike to each worker. Among the available bikes and workers, we choose the $(worker_i, bike_j)$ pair with the shortest Manhattan distance between each other and assign the bike to that worker.

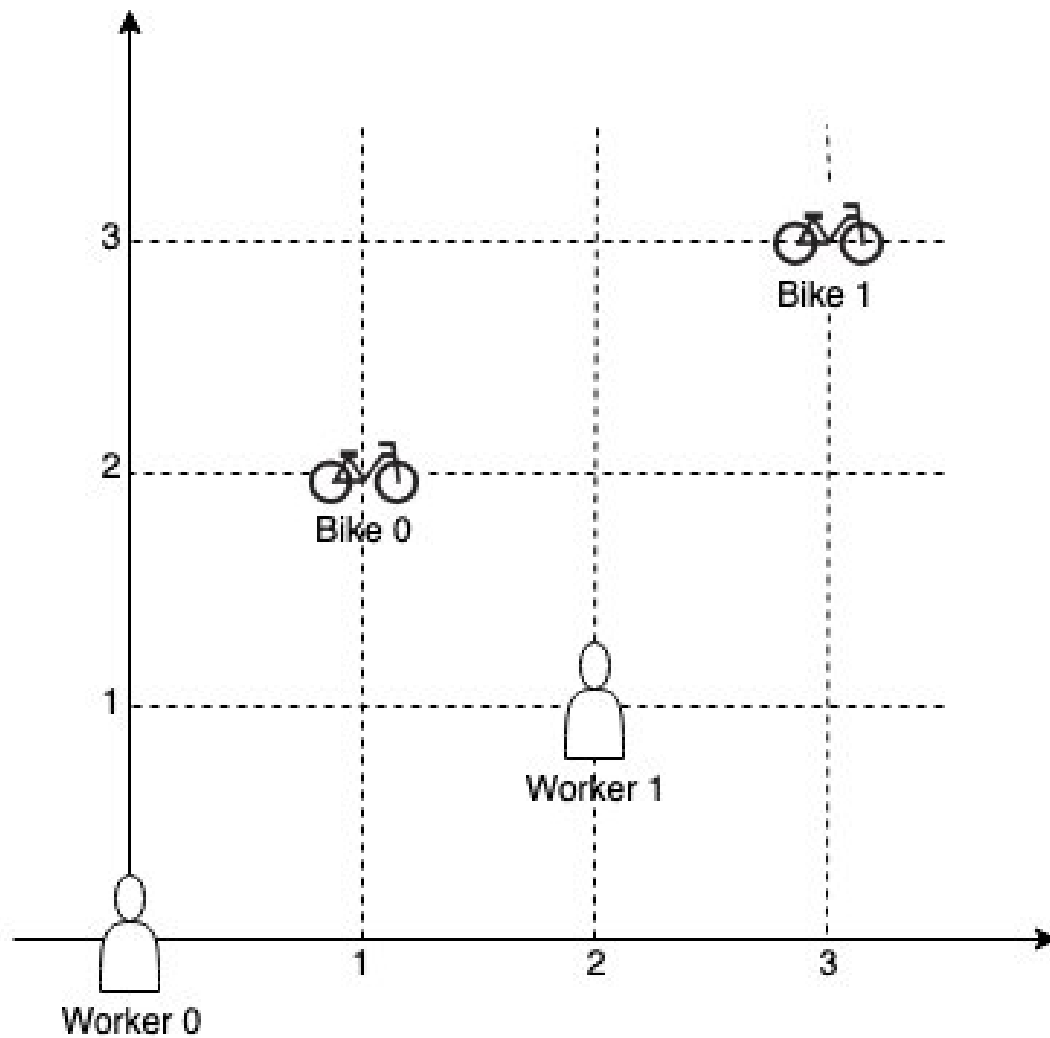
If there are multiple $(worker_i, bike_j)$ pairs with the same shortest Manhattan distance, we choose the pair with the smallest worker index.

If there are multiple ways to do that, we choose the pair with the smallest bike index. Repeat this process until there are no available workers.

Return an array `answer` of length `n`, where `answer[i]` is the index (0-indexed) of the bike that the `i`th worker is assigned to.

The Manhattan distance between two points `p1` and `p2` is $\text{Manhattan}(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|$.

Example 1:

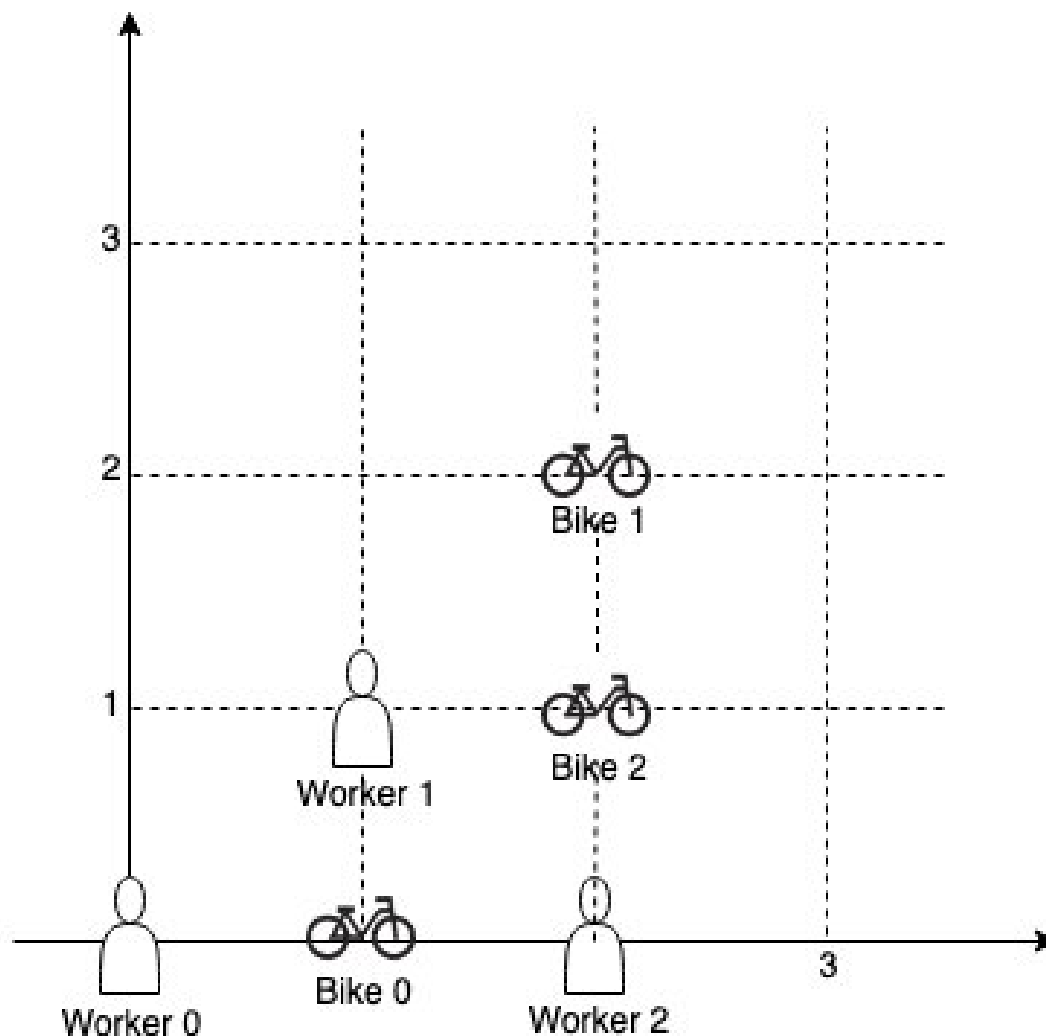


Input: workers = `[[0,0],[2,1]]`, bikes = `[[1,2],[3,3]]`

Output: `[1,0]`

Explanation: Worker 1 grabs Bike 0 as they are closest (without ties), and Worker 0 is assigned Bike 1. So the output is `[1, 0]`.

Example 2:



Input: workers = [[0,0],[1,1],[2,0]], bikes = [[1,0],[2,2],[2,1]]

Output: [0,2,1]

Explanation: Worker 0 grabs Bike 0 at first. Worker 1 and Worker 2 share the same distance to Bike 2, thus Worker 1 is assigned to Bike 2, and Worker 2 will take Bike 1. So the output is [0,2,1].

Constraints:

- $n == \text{workers.length}$
- $m == \text{bikes.length}$
- $1 \leq n \leq m \leq 1000$
- $\text{workers}[i].\text{length} == \text{bikes}[j].\text{length} == 2$
- $0 \leq x_i, y_i < 1000$

- $0 \leq x_j, y_j < 1000$
- All worker and bike locations are unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "n workers and m bikes on a 2D campus. Assign each worker exactly one bike (no sharing).
# Process all (worker,bike) pairs in order of Manhattan distance (ties: smaller worker index,
# then smaller bike index). Return bike assignment array for each worker. Right?"
#
# "workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]]:
# distances: w0b0=3,w0b1=6,w1b0=2,w1b1=2.
# sorted: (2,w1,b0),(2,w1,b1),(3,w0,b0),(6,w0,b1).
# assign w1→b0, then w0→b1. → [1,0] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Generate all (dist, worker, bike) triples, sort them, then greedily assign.
# Time: O(n*m log(n*m)). Space: O(n*m)."
```

```
class _1057_CampusBikes_Brute:
    def assignBikes(self, workers, bikes):
        pairs = []
        for i, (wx, wy) in enumerate(workers):
            for j, (bx, by) in enumerate(bikes):
                pairs.append((abs(wx-bx)+abs(wy-by), i, j))
        pairs.sort()
        result = [-1] * len(workers)
        used_workers = set()
        used_bikes = set()
        for dist, w, b in pairs:
            if w not in used_workers and b not in used_bikes:
                result[w] = b
                used_workers.add(w)
                used_bikes.add(b)
                if len(used_workers) == len(workers):
                    break
        return result
```

PHASE 3 — OPTIMIZE

"Same greedy with a min-heap – identical complexity but avoids sorting all pairs upfront.

Push all pairs initially; pop min until all workers assigned.

Or bucket-sort by distance (max Manhattan ≤ 2000) for $O(n*m)$ time."

PHASE 4 — CLEAN CODE

```
class _1057_CampusBikes:
    def assignBikes(self, workers, bikes):
        import heapq
        heap = []
        for i, (wx, wy) in enumerate(workers):
            for j, (bx, by) in enumerate(bikes):
                heapq.heappush(heap, (abs(wx-bx)+abs(wy-by), i, j))
        result = [-1] * len(workers)
        used_workers = set()
        used_bikes = set()
        while heap and len(used_workers) < len(workers):
            dist, w, b = heapq.heappop(heap)
            if w in used_workers or b in used_bikes:
                continue
            result[w] = b
            used_workers.add(w)
            used_bikes.add(b)
        return result
```

PHASE 5 — TEST & DEBUG

workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]]:

heap: (3,0,0),(6,0,1),(2,1,0),(2,1,1).

pop (2,1,0): assign w1→b0. pop (2,1,1): w1 used skip. pop (3,0,0): b0 used skip.

pop (6,0,1): assign w0→b1. result=[1,0] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n*m \log(n*m))$: heap contains $n*m$ pairs. Space $O(n*m)$.

Bucket sort by distance: $O(n*m)$ time since distances ≤ 2000 .

Follow-up: Campus Bikes II (LC 1066) – minimize TOTAL distance (optimal assignment).

Requires DP with bitmask: $O(n * 2^m)$. Much harder – this greedy doesn't work there."

◆ Quick Review

Key Insights

- Compute the Manhattan distance between every worker and bike pair.

- To achieve the tie-breaking rules, sort all pairs by distance, then by worker index and bike index.
- Once a worker is assigned a bike, skip any other pairs involving that worker or the already assigned bike.
- The problem can also be approached using bucket sort because the Manhattan distance range is limited, but sorting is intuitive given the constraints.

Space & Time

Time Complexity: $O(n * m * \log(n * m))$ where n is the number of workers and m is the number of bikes.

Space Complexity: $O(n * m)$ for storing all worker-bike pairs.

Solution

Create all possible (worker, bike) pairs along with their Manhattan distances. Sort this list of pairs by: Manhattan distance, Worker index (if distances are equal), Bike index (if both distance and worker index are equal). Then, iterate over the sorted list and assign bikes to workers if neither the worker nor the bike has

already been assigned. This guarantees that at each step, the optimal available pairing (according to the problem's requirements) is made. This method naturally enforces the tie-breaking rules specified in the problem.

Heap

★ #1167 Minimum Cost to Connect Sticks ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Heap
Lists: ★ Premium | Premium 98

Problem

You have some number of sticks with positive integer lengths. These lengths are given as an array `sticks`, where `sticks[i]` is the length of the *i*th stick.

You can connect any two sticks of lengths x and y into one stick by paying a cost of $x + y$. You must connect all the sticks until there is only one stick remaining.

Return the minimum cost of connecting all the given sticks into one stick in this way.

Example 1:

Input: `sticks = [2,4,3]`

Output: 14

Explanation: You start with `sticks = [2,4,3]`.

1. Combine sticks 2 and 3 for a cost of $2 + 3 = 5$. Now you have `sticks = [5,4]`.

2. Combine sticks 5 and 4 for a cost of $5 + 4 = 9$. Now you have `sticks = [9]`.

There is only one stick left, so you are done. The total cost is $5 + 9 = 14$.

Example 2:

Input: sticks = [1,8,3,5]

Output: 30

Explanation: You start with sticks = [1,8,3,5].

1. Combine sticks 1 and 3 for a cost of $1 + 3 = 4$. Now you have sticks = [4,8,5].

2. Combine sticks 4 and 5 for a cost of $4 + 5 = 9$. Now you have sticks = [9,8].

3. Combine sticks 9 and 8 for a cost of $9 + 8 = 17$. Now you have sticks = [17].

There is only one stick left, so you are done. The total cost is $4 + 9 + 17 = 30$.

Example 3:

Input: sticks = [5]

Output: 0

Explanation: There is only one stick, so you don't need to do anything. The total cost is 0.

Constraints:

- $1 \leq \text{sticks.length} \leq 104$
- $1 \leq \text{sticks}[i] \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given sticks of various lengths, combine any two into one with cost = sum of their lengths.

Return the minimum total cost to combine all sticks into one. Right?"

#

"sticks=[2,4,3]: combine 2+3=5(cost5). Then 4+5=9(cost9). Total=14 ✓

vs 2+4=6(cost6). 3+6=9(cost9). Total=15. So min is 14 ✓"

PHASE 2 — BRUTE FORCE

"Try all pair orderings recursively, track minimum cost.

Time: $O(n! \cdot n)$. Space: $O(n)$ recursion."

```
class _1167_MinCostConnectSticks_Brute:
```

```
    def connectSticks(self, sticks: list[int]) -> int:
```

```

from itertools import permutations
if len(sticks) == 1: return 0
best = float('inf')
for i in range(len(sticks)):
    for j in range(i+1, len(sticks)):
        new = sticks[i] + sticks[j]
        remaining = [sticks[k] for k in range(len(sticks)) if k!=i and k!=j]
+ [new]

        cost = new + self.connectSticks(remaining)
        best = min(best, cost)
return best

```

PHASE 3 — OPTIMIZE

"Greedy: always combine the two SMALLEST sticks – Huffman coding.
 # Use a min-heap. Pop two smallest, add their sum as cost, push sum back.
 # Repeat until one stick remains.
 # Time: $O(n \log n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```

class _1167_MinCostConnectSticks:
    def connectSticks(self, sticks: list[int]) -> int:
        import heapq
        heapq.heapify(sticks)
        total = 0
        while len(sticks) > 1:
            a = heapq.heappop(sticks)
            b = heapq.heappop(sticks)
            combined = a + b
            total += combined
            heapq.heappush(sticks, combined)
        return total

```

PHASE 5 — TEST & DEBUG

sticks=[2,4,3]: heap=[2,3,4].
 # pop 2,3 → combined=5, cost=5. heap=[4,5].
 # pop 4,5 → combined=9, cost=14. heap=[9]. return 14 ✓
 #
 # sticks=[1,8,3,5]: heap=[1,3,5,8].
 # pop 1,3→4, cost=4. heap=[4,5,8].
 # pop 4,5→9, cost=13. heap=[8,9].
 # pop 8,9→17, cost=30. return 30.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$: heapify $O(n)$ + $n-1$ pops/pushes each $O(\log n)$.
 # Space $O(n)$: the heap.
 # This is exactly Huffman encoding – greedy optimality proven by exchange argument.
 # Follow-up: Minimum Cost to Merge Stones (LC 1000) – merge k adjacent groups; needs DP.
 # Follow-up: if we could only combine adjacent sticks, greedy no longer works."

◆ Quick Review

Key Insights

- Always connect the two smallest sticks available to minimize the incremental cost.
- A min-heap (or priority queue) is an ideal data structure to efficiently retrieve the smallest sticks.
- The process continues until only one stick remains.

Space & Time

Time Complexity: $O(n \log n)$, where n is the number of sticks (due to heap insertion and removal operations).

Space Complexity: $O(n)$, for storing the sticks in the heap.

Solution

The solution uses a greedy algorithm with a min-heap: Insert all stick lengths into a min-heap. While there are at least two sticks in the heap: Extract the two smallest sticks. Compute the cost of connecting them (sum the two lengths). Add this cost to the total result. Push the combined stick (with length equal to

their sum) back into the heap. Once the heap is reduced to one element, return the total cost. This approach ensures that at every connection, the smallest possible sticks are merged, which leads to the minimum overall cost.

Heap

□ #1424 Diagonal Traverse II

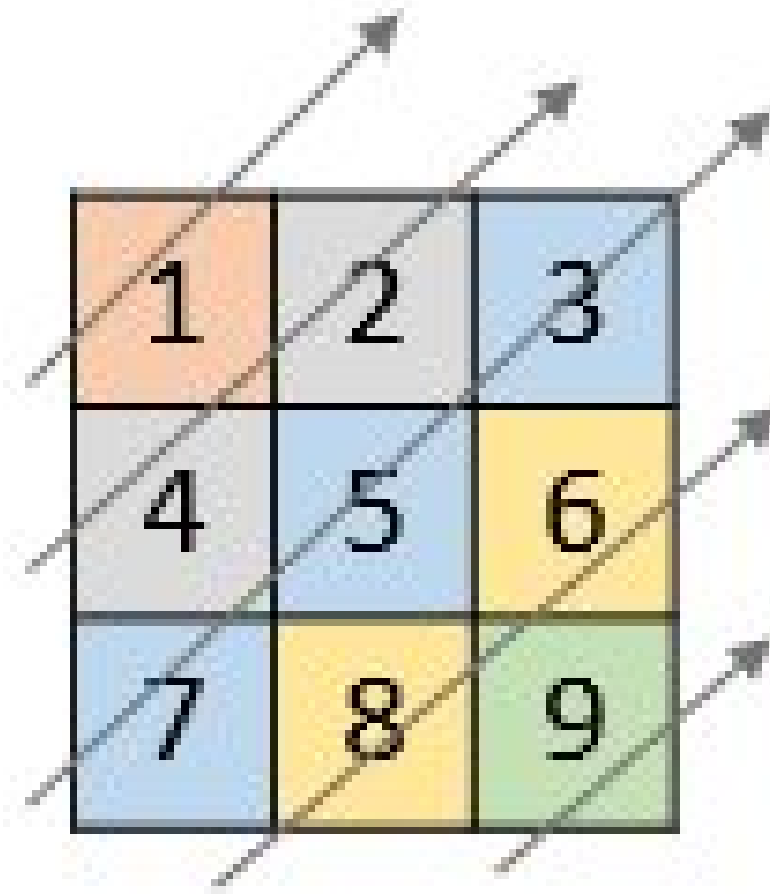
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting · Heap
Lists: CodeSignal

Problem

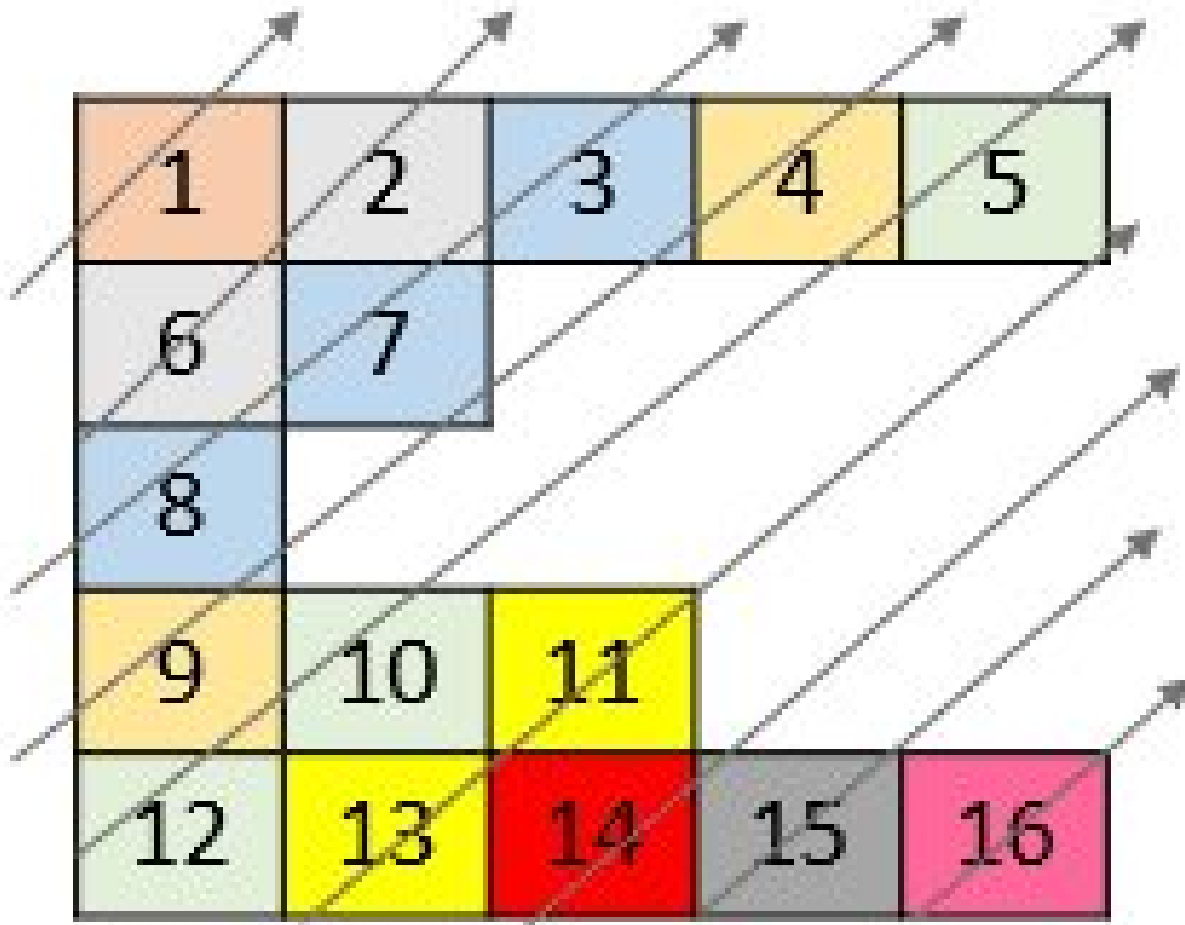
Given a 2D integer array `nums`, return all elements of `nums` in diagonal order as shown in the below images.

Example 1:



Input: nums = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,4,2,7,5,3,8,6,9]

Example 2:



Input: `nums = [[1,2,3,4,5],[6,7],[8],[9,10,11],[12,13,14,15,16]]`

Output: `[1,6,2,8,7,3,9,4,12,10,5,13,11,14,15,16]`

Constraints:

- `1 <= nums.length <= 105`
- `1 <= nums[i].length <= 105`
- `1 <= sum(nums[i].length) <= 105`
- `1 <= nums[i][j] <= 105`

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a 2D list of integers (rows may have different lengths), return all
elements
# in diagonal order: elements on the same anti-diagonal (i+j = const) grouped
together,
# within each diagonal from bottom to top (larger i first). Right?"
#
# "nums=[[1,2,3],[4,5,6],[7,8,9]]:
# diag0(i+j=0):[1]. diag1:[4,2]. diag2:[7,5,3]. diag3:[8,6]. diag4:[9].
# → [1,4,2,7,5,3,8,6,9] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Build a dict of diagonal_index → list of values (appended bottom to top).
# Diagonal index = i+j. Iterate rows in reverse to get bottom-to-top order.
# Time: O(n) where n = total elements. Space: O(n)."
```

```
class _1424_DiagonalTraverseII_Brute:
    def findDiagonalOrder(self, nums: list[list[int]]) -> list[int]:
        from collections import defaultdict
        diags = defaultdict(list)
        for i in range(len(nums) - 1, -1, -1): # bottom to top
            for j, val in enumerate(nums[i]):
                diags[i + j].append(val)
        result = []
        for k in sorted(diags):
            result.extend(diags[k])
        return result
```

PHASE 3 — OPTIMIZE

```
# "Same dict approach but we can avoid sorting by noting diagonal index = i+j
# naturally increases as we scan top-to-bottom, right-to-left.
# Build diags iterating rows top-to-bottom but prepend (or use reverse) per
diagonal.
# Time: O(n). Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _1424_DiagonalTraverseII:
    def findDiagonalOrder(self, nums: list[list[int]]) -> list[int]:
        from collections import defaultdict
        diags = defaultdict(list)
        for i, row in enumerate(nums):
            for j, val in enumerate(row):
                diags[i + j].append(val)
        result = []
        for k in range(len(diags)):
            result.extend(reversed(diags[k])) # larger i (added later) first
        return result
```

PHASE 5 — TEST & DEBUG

```
# nums=[[1,2,3],[4,5,6],[7,8,9]]:  
# diags: 0:[1], 1:[2,4], 2:[3,5,7], 3:[6,8], 4:[9].  
# reversed: [1],[4,2],[7,5,3],[8,6],[9] → [1,4,2,7,5,3,8,6,9] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass to build diags, one pass to collect – n total elements.  
# Space O(n): the diags dictionary stores all elements.  
# Follow-up: Diagonal Traverse I (LC 498) – rectangular matrix, alternating  
direction per diagonal.  
# Follow-up: spiral order (LC 54) vs diagonal order are both reordering problems."
```

◆ Quick Review

Key Insights

- The diagonal key for each element can be defined as (row_index + column_index).
- Elements in the same diagonal (with equal diagonal key) must be output in an order such that the element from the lower row comes before the one from a higher row.
- Because the input grid may be jagged (rows with varying lengths), you must carefully iterate only over valid indices.
- A hash table (or map) can be used to collect elements grouped by their diagonal key.
- Inserting each element at the beginning of the list for that diagonal builds the list in the desired order without needing an extra reverse step later.

- Finally, traverse the keys in increasing order (from 0 to maximum key) to produce the final result.

Space & Time

Time Complexity: $O(N)$, where N is the total number of elements (each element is processed once).

Space Complexity: $O(N)$, for storing the elements in the map and the output array.

Solution

The solution iterates over every element in `nums` while computing its diagonal key as $(i + j)$. For each element, the value is inserted at the beginning of the list corresponding to that diagonal key. This ensures that when the diagonal groups are later concatenated in order of increasing key, the internal ordering within each diagonal is correct (i.e., elements appear from the bottom of the diagonal to the top). The final result is built by iterating over the keys in sorted order and appending each list of diagonal elements.

Trie

Round 5 · Mid · Medium · 7 questions

Trie

□ #139 Word Break

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Dynamic
Programming · Trie · Memoization
Lists: Grind 169

Problem

Given a string *s* and a dictionary of strings *wordDict*, return true if *s* can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
Input: s = "leetcode", wordDict = ["leet","code"]  
Output: true  
Explanation: Return true because "leetcode" can be segmented as "leet code".
```

Example 2:

```
Input: s = "applepenapple", wordDict = ["apple","pen"]  
Output: true  
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".  
Note that you are allowed to reuse a dictionary word.
```

Example 3:

Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
 Output: false

Constraints:

- $1 \leq s.length \leq 300$
- $1 \leq wordDict.length \leq 1000$
- $1 \leq wordDict[i].length \leq 20$
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a string s and a dictionary of words, return true if s can be segmented
# into a space-separated sequence of dictionary words. Words can be reused. Right?"
#
# "s='leetcode', wordDict=['leet','code']: 'leet'+ 'code' → true ✓
# s='applepenapple', wordDict=['apple','pen']: 'apple'+ 'pen'+ 'apple' → true ✓
# s='catsandog', wordDict=['cats','dog','sand','and','cat']: no valid split → false
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Recursive: try every prefix of s. If prefix is in dict, recurse on the suffix.
# Exponential without memoization –  $O(2^n)$  time,  $O(n)$  space."
```

```
class _139_WordBreak_Brute:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        words = set(wordDict)
        def backtrack(start: int) -> bool:
            if start == len(s):
                return True
            for end in range(start + 1, len(s) + 1):
                if s[start:end] in words and backtrack(end):
                    return True
```

```

    return False
return backtrack(0)

```

PHASE 3 — OPTIMIZE

```

# "DP: dp[i] = True if s[:i] can be segmented.
# Base: dp[0] = True (empty string).
# Transition: dp[i] = any(dp[j] and s[j:i] in wordSet) for 0 <= j < i.
# Time: O(n2) or O(n * max_word_len). Space: O(n)."

```

PHASE 4 — CLEAN CODE

```

class _139_WordBreak:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        words = set(wordDict)
        n = len(s)
        dp = [False] * (n + 1)
        dp[0] = True
        max_len = max(len(w) for w in words)
        for i in range(1, n + 1):
            for j in range(max(0, i - max_len), i):
                if dp[j] and s[j:i] in words:
                    dp[i] = True
                    break
        return dp[n]

```

PHASE 5 — TEST & DEBUG

```

# s='leetcode', words={'leet','code'}:
# dp[0]=T. dp[4]: j=0,s[0:4]='leet'∈words,dp[0]=T→dp[4]=T.
# dp[8]: j=4,s[4:8]='code'∈words,dp[4]=T→dp[8]=T. return True ✓
#
# s='catsandog': dp[3]=T('cat'),dp[4]=T('cats'). dp[7]=T('sand' from 3).
# No j makes dp[9]=T → return False ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n*max_word_len): optimised by only checking back max_word_len characters.
# Space O(n) for dp array.
# Trie optimisation: insert all words into a trie, walk the trie forward from each
dp[j]=True position.
# Follow-up: Word Break II (LC 140) – return all valid sentences.
# Backtrack from dp[n]=True positions to reconstruct paths; store parent indices."

```

◆ Quick Review

Key Insights

- Use Dynamic Programming (DP) to break the problem into subproblems.

- Create a DP array where $dp[i]$ is true if the substring $s[0:i]$ can be segmented using words from the dictionary.
- Leverage a hash set for fast look-up of dictionary words.
- For every index i , check all possible previous break points j such that $dp[j]$ is true and $s[j:i]$ is in the wordDict.
- The problem can also be approached with recursion and memoization, but DP is more straightforward for iterative implementation.

Space & Time

Time Complexity: $O(n^2)$ in the worst-case scenario, where n is the length of s (each substring check is $O(1)$ if using a hash set, and there could be $O(n^2)$ checks overall).

Space Complexity: $O(n)$ for the dp array and $O(k)$ for the hash set where k is the number of words in the dictionary.

Solution

The solution uses a Dynamic Programming (DP) approach: Convert wordDict to a hash set for constant time lookup. Create a boolean dp array of size $n+1$ (n is the length of s), where

dp[i] indicates if s[0:i] can be segmented. Set dp[0] to true because an empty string is always segmentable. For each index i from 1 to n, iterate through all indices j from 0 to i: If dp[j] is true and the substring s[j:i] is in the dictionary, then set dp[i] to true. Break early from the inner loop when a valid break is found. Return dp[n] as the result. This method efficiently builds up the solution by utilizing previous results to determine if the entire string can be segmented.

Trie

□ #208 Implement Trie (Prefix Tree)

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Trie
Lists: Grind 169

Problem

A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- **Trie()** Initializes the trie object.
- **void insert(String word)** Inserts the string word into the trie.
- **boolean search(String word)** Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- **boolean startsWith(String prefix)** Returns true if there is a previously inserted string word that has the prefix prefix, and false

otherwise.

Example 1:

```
Input
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
Output
[null, null, true, false, true, null, true]

Explanation
Trie trie = new Trie();
```

Constraints:

- $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$
- word and prefix consist only of lowercase English letters.
- At most $3 * 10^4$ calls in total will be made to insert, search, and startsWith.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Implement a Trie with insert(word), search(word) → bool (exact match),
# and startsWith(prefix) → bool. Only lowercase English letters. Right?"
#
# "insert('apple'). search('apple')→true. search('app')→false.
# startsWith('app')→true. insert('app'). search('app')→true ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Use a set for exact search and a sorted list for prefix search.
# Time: insert O(1), search O(1), startsWith O(n * m) – scan all words.
# Space: O(total_chars)."
```

```
class _208_ImplementTrie_Brute:
    def __init__(self):
```



```

        self.words = set()
    def insert(self, word: str) -> None:
        self.words.add(word)
    def search(self, word: str) -> bool:
        return word in self.words
    def startsWith(self, prefix: str) -> bool:
        return any(w.startswith(prefix) for w in self.words)

```

PHASE 3 — OPTIMIZE

"True trie: each node has up to 26 children (one per letter) and an end-of-word flag.

insert: create nodes along the path. search: walk path, check end flag.

startsWith: walk path, return True if we reach the end of the prefix.

Time: O(m) per op where m = word/prefix length. Space: O(total_chars * 26)."

PHASE 4 — CLEAN CODE

```

class _208_ImplementTrie:
    def __init__(self):
        self.children = {}
        self.is_end = False
    def insert(self, word: str) -> None:
        node = self
        for ch in word:
            if ch not in node.children:
                node.children[ch] = _208_ImplementTrie()
            node = node.children[ch]
        node.is_end = True
    def search(self, word: str) -> bool:
        node = self
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end
    def startsWith(self, prefix: str) -> bool:
        node = self
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True

```

PHASE 5 — TEST & DEBUG

insert("apple"): root→a→p→p→l→e(is_end=True).

search("apple"): walk a→p→p→l→e, is_end=True → True ✓

search("app"): walk a→p→p, is_end=False → False ✓

startsWith("app"): walk a→p→p, exists → True ✓

insert("app"): node at 'p'(3rd) gets is_end=True.

search("app"): walk a→p→p, is_end=True → True ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m)$ per op: walk at most m nodes. Space $O(n*m)$: n words of avg length m .

Dict children vs array[26]: dict is memory-efficient for sparse alphabets.

Follow-up: delete a word from the trie – mark `is_end=False`, prune childless nodes bottom-up.

Follow-up: Word Search II (LC 212) – build a trie of all words, DFS the board against it."

◆ Quick Review

Key Insights

- Use a tree-like structure where each node represents a character.
- Each node stores its children in a dictionary (or hash map) for $O(1)$ average time access.
- Mark the end of a word with a boolean flag.
- Insertion is done character by character, creating new nodes as needed.
- Searching and checking prefixes follow traversals down the tree.

Space & Time

Time Complexity:

Insert: $O(m)$ where m is the length of the word.

Search: $O(m)$

startsWith: $O(m)$

Space Complexity:

$O(N * M)$ in the worst-case scenario, where N is the number of words and M is the average length of the words, due to storing each character in the Trie.

Solution

We implement the Trie using a node-based approach. Each `TrieNode` contains a hash map (or dictionary) of its children and a boolean indicating if the node marks the end of a word. The `Trie` class uses these nodes to build out the words character by character. The `insert` method traverses and builds the necessary nodes; the `search` method verifies that all characters exist in order and that the final node marks the end of a word; the `startsWith` method checks if the prefix exists in the Trie. No tricky edge cases exist beyond handling null or empty inputs, as constraints guarantee valid lowercase strings.

Trie

□ #211 Design Add and Search Words Data Structure **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · DFS · Design · Trie
Lists: Grind 169

Problem

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

- WordDictionary() Initializes the object.
- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise. word may contain dots '.' where dots can be matched with any letter.

Example:

Input

```
["WordDictionary", "addWord", "addWord", "addWord", "search", "search", "search", "search"]
```

```
[[], ["bad"], ["dad"], ["mad"], ["pad"], ["bad"], [".ad"], ["b.."]]
```

Output

```
[null, null, null, null, false, true, true, true]
```

Explanation

```
WordDictionary wordDictionary = new WordDictionary();
```

Constraints:

- $1 \leq \text{word.length} \leq 25$
- word in addWord consists of lowercase English letters.
- word in search consist of '.' or lowercase English letters.
- There will be at most 2 dots in word for search queries.
- At most 104 calls will be made to addWord and search.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Design a data structure with addWord(word) and search(word).

search supports '.' as a wildcard matching any single letter. Right?"

#

"addWord('bad'), addWord('dad'), addWord('mad').

search('pad')→false, search('bad')→true, search('.ad')→true, search('b..')→true

✓"

PHASE 2 — BRUTE FORCE

```

# "Store all words in a list. For search, iterate over all words and match
# character by character (treating '.' as wildcard).
# Time:  $O(n * m)$  per search where  $n$  = number of words,  $m$  = word length. Space:
 $O(n * m)$ ."

class _211_WordDictionary_Brute:
    def __init__(self):
        self.words = []

    def addWord(self, word: str) -> None:
        self.words.append(word)

    def search(self, word: str) -> bool:
        def matches(w, pattern):
            if len(w) != len(pattern): return False
            return all(p == '.' or c == p for c, p in zip(w, pattern))
        return any(matches(w, word) for w in self.words)

# PHASE 3 — OPTIMIZE
# "Trie with DFS for wildcards. addWord: standard trie insert.
# search: walk trie; on a '.' node, recursively try ALL children.
# Time: addWord  $O(m)$ . search  $O(m)$  for literals, up to  $O(26^k * m)$  worst case with  $k$ 
# dots.
# Space:  $O(n * m)$  for the trie."

# PHASE 4 — CLEAN CODE
class _211_WordDictionary:
    def __init__(self):
        self.children = {}
        self.is_end = False

    def addWord(self, word: str) -> None:
        node = self
        for ch in word:
            if ch not in node.children:
                node.children[ch] = _211_WordDictionary()
            node = node.children[ch]
        node.is_end = True

    def search(self, word: str) -> bool:
        def dfs(node, i: int) -> bool:
            if i == len(word):
                return node.is_end
            ch = word[i]
            if ch == '.':
                return any(dfs(child, i + 1) for child in node.children.values())
            if ch not in node.children:
                return False
            return dfs(node.children[ch], i + 1)
        return dfs(self, 0)

# PHASE 5 — TEST & DEBUG
# addWord("bad"): b→a→d(end). addWord("dad"): d→a→d(end). addWord("mad"):
# m→a→d(end).

```

```
# search("pad"): p not in root.children → False ✓
# search("bad"): b→a→d, is_end=True → True ✓
# search(".ad"): '.' : try b,d,m → each→a→d, is_end=True → True ✓
# search("b.."): b→'.' : try a→'.' : try d(end) → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "addWord O(m). search O(m) best case (no dots), O(26^k * m) worst case (k dots).
# In practice dots are rare, so average is close to O(m).
# Follow-up: if patterns are very dot-heavy, cache results at (node, index) pairs.
# Follow-up: support '*' (zero or more chars) – more complex DFS with skip logic."
```

◆ Quick Review

Key Insights

- Utilize a Trie (prefix tree) to efficiently store words.
- Each node in the Trie contains a dictionary for its children and a flag indicating the end of a word.
- The search operation is implemented with a depth-first search (DFS) approach.
- When the search query contains a dot, explore all possible child nodes.

Space & Time

Time Complexity:

addWord: $O(w)$, where w is the length of the word.

search: Worst-case $O(26^w)$ for words with multiple wildcards, but practical performance is better given constraints.

Space Complexity: $O(n * w)$, where n is the number of words and w is the average word length.

Solution

The WordDictionary is implemented using a Trie. For adding, we insert each character of the word into the Trie. For searching, we use DFS. When encountering a wildcard '.', we recursively search all children nodes. This structure combined with DFS efficiently addresses the wildcard matching and ensures good performance for multiple operations.

Trie

★ #616 Add Bold Tag in String ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Trie
Lists: ★ Premium | Premium 98

Problem

You are given a string *s* and an array of strings *words*.

You should add a closed pair of bold tag **** and **** to wrap the substrings in *s* that exist in *words*.

- If two such substrings overlap, you should wrap them together with only one pair of closed bold-tag.
- If two substrings wrapped by bold tags are consecutive, you should combine them.

Return *s* after adding the bold tags.

Example 1:

Input: *s* = "abcxyz123", *words* = ["abc","123"]

Output: "abcxyz123"

Explanation: The two strings of *words* are substrings of *s* as following:
"abcxyz123".

We add **** before each substring and **** after each substring.

Example 2:

Input: s = "aaabbb", words = ["aa","b"]

Output: "aaabbb"

Explanation:

"aa" appears as a substring two times: "aaabbb" and "aaabbb".

"b" appears as a substring three times: "aaabbb", "aaabbb", and "aaabbb".

We add before each substring and after each substring:

"aaabbb"

Since the first two 's overlap, we merge them:

"aaabbb"

Since now the four 's are consecutive, we merge them: "aaabbb"

Constraints:

- $1 \leq s.length \leq 1000$
- $0 \leq words.length \leq 100$
- $1 \leq words[i].length \leq 1000$
- s and words[i] consist of English letters and digits.
- All the values of words are unique.

Note: This question is the same as 758. Bold Words in String.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given a string s and list of words, wrap all substrings of s that are in words with tags. Overlapping or adjacent bold regions should be merged. Right?"

#

"s='abcxyz123', words=['abc','123']: 'abcxyz123' ✓

s='aaabbcc', words=['aaa','aab','bc']: 'aaabbcc' ✓ (overlapping regions merged)"

PHASE 2 — BRUTE FORCE

"Mark each character as bold or not. For each word, find all occurrences in s and mark those character positions. Then rebuild the string with tags.
 # Time: $O(n * m * L)$ where m =words count, L =avg word length. Space: $O(n)$."

```
class _616_AddBoldTag_Brute:
    def addBoldTag(self, s: str, words: list[str]) -> str:
        n = len(s)
        bold = [False] * n
        for word in words:
            start = 0
            while True:
                idx = s.find(word, start)
                if idx == -1: break
                for i in range(idx, idx + len(word)):
                    bold[i] = True
                start = idx + 1
        result = []
        i = 0
        while i < n:
            if bold[i]:
                result.append('<b>')
                while i < n and bold[i]:
                    result.append(s[i]); i += 1
                result.append('</b>')
            else:
                result.append(s[i]); i += 1
        return ''.join(result)
```

PHASE 3 — OPTIMIZE

"Build intervals of bold regions by finding all word matches, merge overlapping intervals,
 # then reconstruct. Use a boolean array for merging – same $O(n*m*L)$ but cleaner.
 # Trie optimization: insert all words, scan s with Trie to find all matches in $O(n*L)$."

PHASE 4 — CLEAN CODE

```
class _616_AddBoldTag:
    def addBoldTag(self, s: str, words: list[str]) -> str:
        n = len(s)
        end = 0 # rightmost bold end seen so far
        bold = [False] * n
        word_set = set(words)
        max_len = max(len(w) for w in words) if words else 0
        for i in range(n):
            for length in range(1, min(max_len, n - i) + 1):
                if s[i:i+length] in word_set:
                    end = max(end, i + length)
            if i < end:
                bold[i] = True
        result = []
```

```

i = 0
while i < n:
    if bold[i]:
        result.append('<b>')
        while i < n and bold[i]:
            result.append(s[i]); i += 1
        result.append('</b>')
    else:
        result.append(s[i]); i += 1
return ''.join(result)

```

PHASE 5 — TEST & DEBUG

```

# s='aaabbcc', words=['aaa','aab','bc']:
# i=0: 'aaa'✓→end=3,'aab'? s[0:3]='aaa'≠'aab'. bold[0,1,2]=True.
# i=1: 'aab'✓→end=max(3,4)=4. bold[1,2,3]=True.
# i=4: 'bc'✓→end=6. bold[4,5]=True.
# bold=[T,T,T,T,T,T,F]. → '<b>aaabbcc</b>c' ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n * max_L): at each position check all prefix lengths up to max_L.
# Space O(n): bold array + set.
# Trie reduces search to O(n * max_L) but with better constant via prefix pruning.
# Follow-up: Interval List Intersections (LC 986) – another interval merging
problem.
# Follow-up: Bold Words in String (LC 758) – identical problem, same solution."

```

◆ Quick Review

Key Insights

- We need to identify every occurrence of each word in s.
- Overlapping or consecutive intervals should be merged to form one continuous bold region.
- Use an auxiliary data structure (e.g., a boolean array) to mark positions that should be bolded.

- Once the positions are marked, build the final string by inserting bold tags at the appropriate boundaries.

Space & Time

Time Complexity: $O(n * m * k)$ where n is the length of s , m is the number of words, and k is the average length of the words (due to substring matching).

Space Complexity: $O(n)$ for the auxiliary boolean array used to mark bold positions.

Solution

The solution involves two main steps: Identify and mark the indices in the string s that are part of any matching word. We do this by iterating over each word and marking every occurrence's start and end in a boolean array. Construct the final string by traversing the boolean array. When encountering a segment of consecutive true values, wrap that segment with **and** tags. The key idea is to merge intervals by marking every index that should be bold and then using a single pass to insert tags only when transitioning from non-bold to bold and vice versa.

Trie

□ #692 Top K Frequent Words

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Trie · Sorting · Heap
Lists: Grind 169

Problem

Given an array of strings `words` and an integer `k`, return the `k` most frequent strings.

Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.

Example 1:

```
Input: words = ["i","love","leetcode","i","love","coding"], k = 2
Output: ["i","love"]
Explanation: "i" and "love" are the two most frequent words.
Note that "i" comes before "love" due to a lower alphabetical order.
```

Example 2:

```
Input: words = ["the","day","is","sunny","the","the","the","sunny","is","is"],
k = 4
Output: ["the","is","sunny","day"]
Explanation: "the", "is", "sunny" and "day" are the four most frequent words,
with the number of occurrence being 4, 3, 2 and 1 respectively.
```

Constraints:

- $1 \leq \text{words.length} \leq 500$

- $1 \leq \text{words}[i].\text{length} \leq 10$
- $\text{words}[i]$ consists of lowercase English letters.
- k is in the range $[1, \text{The number of unique words}[i]]$

Follow-up: Could you solve it in $O(n \log(k))$ time and $O(n)$ extra space?

My LeetCode Solution
Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given a list of words and k , return the k most frequent words sorted by frequency descending. Ties broken alphabetically ascending. Right?"

#

"words=['i', 'love', 'leetcode', 'i', 'love', 'coding'], $k=2 \rightarrow$ ['i', 'love'] ✓
words=['the', 'day', 'is', 'sunny', 'the', 'the', 'the', 'sunny', 'is', 'is'], $k=4$
$\rightarrow$ ['the', 'is', 'sunny', 'day'] ✓"

PHASE 2 — BRUTE FORCE

"Count frequencies, sort by $(-\text{freq}, \text{word})$, take first k .
Time: $O(n \log n)$. Space: $O(n)$."

```
class _692_TopKFrequentWords_Brute:
    def topKFrequent(self, words: list[str], k: int) -> list[str]:
        from collections import Counter
        freq = Counter(words)
        return sorted(freq.keys(), key=lambda w: (-freq[w], w))[:k]
```

PHASE 3 — OPTIMIZE

"Min-heap of size k with custom comparator.
Store $(-\text{freq}, \text{word})$ so the heap evicts the least desirable (lowest freq, alphabetically last).
Time: $O(n \log k)$. Space: $O(k)$ heap + $O(n)$ counter."

PHASE 4 — CLEAN CODE

```
class _692_TopKFrequentWords:
```

```
def topKFrequent(self, words: list[str], k: int) -> list[str]:
    import heapq
    from collections import Counter

    class Word:
        def __init__(self, word, freq):
            self.word = word
            self.freq = freq
        def __lt__(self, other):
            if self.freq != other.freq:
                return self.freq < other.freq # lower freq = worse
            return self.word > other.word # later alpha = worse

    freq = Counter(words)
    heap = []
    for word, cnt in freq.items():
        heapq.heappush(heap, Word(word, cnt))
        if len(heap) > k:
            heapq.heappop(heap)
    return [w.word for w in sorted(heap, key=lambda x: (-x.freq, x.word))]
```

PHASE 5 — TEST & DEBUG

```
# words=['i','love','leetcode','i','love','coding'], k=2:
# freq={'i':2,'love':2,'leetcode':1,'coding':1}.
# heap of size 2: keeps 2 best -> ('i',2) and ('love',2).
# sorted by (-freq,word): ('i',2),('love',2) -> ['i','love'] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Heap  $O(n \log k)$ : optimal when  $k \ll n$ . Sort  $O(n \log n)$ : simpler code.
# The custom comparator handles the tie-breaking (alphabetical) correctly.
# Follow-up: Top K Frequent Elements (LC 347) – integers, no tie-breaking needed.
# Follow-up: streaming top-k words – maintain heap as words arrive one by one."
```

◆ Quick Review

Key Insights

- Count the frequency of each unique word using a hash table or dictionary.
- Use a min-heap (priority queue) to keep track of the top k frequent words. In the heap, the ordering is based on frequency; if frequencies are equal, order by lexicographical

order (inverted condition for min-heap).

- Alternatively, you can sort the unique words based on frequency and lexicographical order and then pick the top k .
- For an efficient solution, use a min-heap that maintains size k achieving $O(n \log k)$ time complexity.

Space & Time

Time Complexity: $O(n \log k)$ when using a heap, where n is the number of unique words.

Space Complexity: $O(n)$ for storing the frequency counts and the heap.

Solution

We first calculate the frequency of each word using a hash table (or dictionary). Then, we use a min-heap that stores pairs of (frequency, word). The comparator for the heap is defined such that the root of the heap is the word with the smallest frequency; if two words have the same frequency, the word with the lexicographically larger value is considered “smaller” in our ordering so that it gets popped when the heap size exceeds k . After processing all words, the heap contains

the top k frequent words. Finally, the heap elements are extracted and reversed (if needed) to produce the final list sorted from highest to lowest frequency with lexicographic tie-breaking.

Trie

□ #720 Longest Word in Dictionary

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Trie · Sorting
Lists: Denny Zhang

Problem

Given an array of strings words representing an English Dictionary, return the longest word in words that can be built one character at a time by other words in words.

If there is more than one possible answer, return the longest word with the smallest lexicographical order. If there is no answer, return the empty string.

Note that the word should be built from left to right with each additional character being added to the end of a previous word.

Example 1:

Input: words = ["w","wo","wor","worl","world"]

Output: "world"

Explanation: The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".

Example 2:

Input: words = ["a","banana","app","appl","ap","apply","apple"]

Output: "apple"

Explanation: Both "apply" and "apple" can be built from other words in the dictionary. However, "apple" is lexicographically smaller than "apply".

Constraints:

- $1 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length} \leq 30$
- `words[i]` consists of lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given a list of strings, find the longest word that can be built one character at a time

by other words in the list. All prefixes of the word must also be in the list.

If tie, return the smallest alphabetically. Right?"

#

"words=['w','wo','wor','worl','world']: every prefix exists → 'world' ✓

words=['a','banana','app','appl','ap','apply','apple']:

'apply' and 'apple' both have all prefixes → return 'apple' (smaller) ✓"

PHASE 2 — BRUTE FORCE

"Put all words in a set. Sort by (length desc, alpha asc). For each word,

check that all its prefixes are in the set. Return the first match.

Time: $O(n * L)$ where $L = \max \text{ word length}$. Space: $O(n * L)$."

```
class _720_LongestWordInDictionary_Brute:
```

```
    def longestWord(self, words: list[str]) -> str:
```

```
        word_set = set(words)
```

```
        words.sort(key=lambda w: (-len(w), w))
```

```
        for word in words:
```

```
            if all(word[:i] in word_set for i in range(1, len(word))):
```

```
                return word
```

```
        return ''
```

PHASE 3 — OPTIMIZE

```
# "Trie approach: insert all words, then BFS/DFS from root visiting only nodes
# where is_end=True. The deepest reachable node gives the longest valid word.
# Time:  $O(n * L)$  to build trie +  $O(n * L)$  to search. Space:  $O(n * L)$ ."
```

PHASE 4 — CLEAN CODE

```
class _720_LongestWordInDictionary:
    def longestWord(self, words: list[str]) -> str:
        word_set = set(words)
        result = ''
        for word in words:
            if all(word[:i] in word_set for i in range(1, len(word))):
                if len(word) > len(result) or (len(word) == len(result) and word <
result):
                    result = word
        return result
```

PHASE 5 — TEST & DEBUG

```
# words=['w','wo','wor','worl','world']:
# 'world': check 'w'✓, 'wo'✓, 'wor'✓, 'worl'✓ → all prefixes in set. len=5>0 →
result='world' ✓
#
# words=['a','banana','app','appl','ap','apply','apple']:
# 'apple': 'a','ap','app','appl' all in set ✓. len=5. result='apple'.
# 'apply': all prefixes in set. len=5=5, 'apply'>'apple' → keep 'apple' ✓.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n * L^2)$  for the set approach (each word checks  $L$  prefixes, each  $O(L)$ 
lookup).
# Trie reduces prefix lookup to  $O(1)$  per character –  $O(n * L)$  overall.
# Follow-up: if words are streamed, maintain a trie and update result on each
insert.
# Follow-up: allow at most one missing prefix – extend valid condition in the
BFS/DFS."
```

◆ Quick Review

Key Insights

- The word must be built character by character; hence every prefix (removing one character from the end at a time) must exist in the dictionary.

- **Sorting the input list can help in ensuring that smaller words (prefixes) are processed first.**
- **Using a hash set to store valid words enables rapid prefix checks.**
- **Lexicographical order plays a role when words have the same length.**

Space & Time

Time Complexity: $O(n * L + n \log n)$ where n is the number of words and L is the maximum length of a word ($n * L$ for prefix checking, and $n \log n$ for sorting).

Space Complexity: $O(n * L)$ for storing words in the set.

Solution

The approach begins by sorting the list of words. Sorting is done primarily by word length and secondarily by lexicographical order so that when we iterate through the words, all possible prefixes for a word have already been considered. Initialize a set to keep track of valid words that can be built one character at a time. For each word in the sorted list, check if all its prefixes (word

without the last character progressively) exist in the set. If a word qualifies, add it to the set and check if it should be the new result based on length and lexicographical order. The set lookup ensures that checking each prefix is efficient.

Trie

□ #1166 Design File System

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Trie
Lists: Denny Zhang

Problem

You are asked to design a file system that allows you to create new paths and associate them with different values.

The format of a path is one or more concatenated strings of the form: / followed by one or more lowercase English letters. For example, "/leetcode" and "/leetcode/problems" are valid paths while an empty string "" and "/" are not.

Implement the FileSystem class:

- **bool createPath(string path, int value)** Creates a new path and associates a value to it if possible and returns true. Returns false if the path already exists or its parent path doesn't exist.

- **int get(string path)** Returns the value associated with path or returns -1 if the path doesn't exist.

Example 1:

```
Input:
["FileSystem","createPath","get"]
[[],["/a",1],["/a"]]
Output:
[null,true,1]
Explanation:
FileSystem fileSystem = new FileSystem();
```

Example 2:

```
Input:
["FileSystem","createPath","createPath","get","createPath","get"]
[[],["/leet",1],["/leet/code",2],["/leet/code"],["/c/d",1],["/c"]]
Output:
[null,true,true,2,false,-1]
Explanation:
FileSystem fileSystem = new FileSystem();
```

Constraints:

- $2 \leq \text{path.length} \leq 100$
- $1 \leq \text{value} \leq 109$
- Each path is valid and consists of lowercase English letters and '/'.
- At most 104 calls in total will be made to createPath and get.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design a file system with createPath(path, value)→bool and get(path)→int.
# createPath creates a path with a value – all parent paths must already exist.
# Returns false if path exists or parent doesn't exist. get returns value or -1.
Right?"
#
# "createPath('/a',1)→T. createPath('/a/b',2)→T. get('/a')→1. get('/a/b')→2.
# createPath('/a/b',3)→F (already exists). createPath('/c/d',4)→F (no '/c'). ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Use a dict mapping full path string → value.
# createPath: check parent path exists in dict, then insert.
# get: return dict.get(path, -1).
# Time: O(L) per op where L = path length. Space: O(n*L)."
```

```
class _1166_FileSystem_Brute:
    def __init__(self):
        self.paths = {}

    def createPath(self, path: str, value: int) -> bool:
        if path in self.paths or path == '/':
            return False
        parent = path.rsplit('/', 1)[0] or '/'
        if parent != '/' and parent not in self.paths:
            return False
        self.paths[path] = value
        return True

    def get(self, path: str) -> int:
        return self.paths.get(path, -1)
```

PHASE 3 — OPTIMIZE

```
# "Trie where each node maps component names to child nodes and stores a value.
# createPath: split on '/', walk/create nodes, check parent validity.
# get: walk trie, return node value.
# Time: O(L) per op. Space: O(n*L) – same as dict but with natural prefix sharing."
```

PHASE 4 — CLEAN CODE

```
class _1166_FileSystem:
    def __init__(self):
        self.root = {} # trie: nested dicts, '_val' key stores value

    def createPath(self, path: str, value: int) -> bool:
        parts = path.strip('/').split('/')
        node = self.root
        for i, part in enumerate(parts):
            if i == len(parts) - 1:
                if part in node:
                    return False # path already exists
                node[part] = {'_val': value}
            else:
                if part not in node:
                    node[part] = {}
```

```

    else:
        if part not in node:
            return False # parent doesn't exist
        node = node[part]
    return True

def get(self, path: str) -> int:
    parts = path.strip('/').split('/')
    node = self.root
    for part in parts:
        if part not in node:
            return -1
        node = node[part]
    return node.get('_val', -1)

```

PHASE 5 — TEST & DEBUG

```

# createPath('/a',1): parts=['a'], root['a']={'_val':1} → True ✓
# createPath('/a/b',2): parts=['a','b']. node=root['a']. 'b' not in node →
root['a']['b']={'_val':2} → True ✓
# get('/a'): parts=['a']. node=root['a']. return _val=1 ✓
# createPath('/a/b',3): 'b' in root['a'] → False ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Both dict and trie are O(L) per operation where L = path length.
# Trie benefits when many paths share prefixes (memory efficiency).
# Follow-up: add a delete(path) – must handle deleting subtrees recursively.
# Follow-up: list all paths under a prefix – trie DFS from that node."

```

◆ Quick Review

Key Insights

- Use a data structure (like a hash map) to store paths and associated values.
- To create a new path, check that the path does not already exist.
- Ensure the immediate parent path exists before creation.
- When retrieving a value, simply look it up in the storage structure.

- Splitting the path by "/" allows easy identification of the parent.

Space & Time

Time Complexity:

createPath: $O(n)$ per call where n is the number of segments in the path (to process and validate parent path).

get: $O(1)$ per call if using a hash map.

Space Complexity: $O(m * L)$ where m is the number of paths stored and L is the average length of a path.

Solution

The solution uses a hash map (dictionary) to store file paths along with their associated values. For the createPath method, the algorithm: Checks if the path already exists. Extracts the parent path by removing the last segment. Validates the existence of the parent path. Inserts the new path with its corresponding value if validation passes. For the get method, the algorithm simply returns the stored value if the path exists; otherwise, it returns -1. This approach ensures efficient lookup and validation using string

manipulation and hash map operations.

Backtracking

Round 5 · Mid · Medium · 6 questions

Backtracking

□ #17 Letter Combinations of a Phone Number

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Backtracking
Lists: Grind 169

Problem

Given a string containing digits from 2–9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1:

Input: digits = "23"

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

Input: digits = "2"

Output: ["a","b","c"]

Constraints:

- $1 \leq \text{digits.length} \leq 4$
- `digits[i]` is a digit in the range ['2', '9'].

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a string of digits (2-9), return all possible letter combinations
# the digits could represent on a phone keypad. Return [] for empty input. Right?"
#
# "digits="23": 2→abc, 3→def → [ad,ae,af,bd,be,bf,cd,ce,cf] ✓
# digits="": return [] ✓
# digits="2": → [a,b,c] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Build combinations iteratively using a queue (BFS).
# Start with [''], then for each digit, expand every existing combination
# by appending each letter the digit maps to.
# Time:  $O(4^n * n)$  – 4 letters max per digit, n digits. Space:  $O(4^n * n)$ ."
```

```
class _17_LetterCombinations_Brute:
    def letterCombinations(self, digits: str) -> list[str]:
        if not digits:
            return []
        phone = {'2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
                  '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'}
        result = ['']
        for digit in digits:
            result = [combo + letter
                      for combo in result
                      for letter in phone[digit]]
        return result
```

PHASE 3 — OPTIMIZE

```
# "Backtracking builds each combination one character at a time and avoids
# storing all intermediate combinations.
# At each depth level choose one letter for that digit, recurse, then undo.
# Time:  $O(4^n * n)$  – same output size, but space is  $O(n)$  call stack +  $O(4^n * n)$ 
# result."
```

PHASE 4 — CLEAN CODE

```
class _17_LetterCombinations:
    def letterCombinations(self, digits: str) -> list[str]:
        if not digits:
            return []
        phone = {'2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
                  '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'}
        result = []
```

```
def backtrack(index: int, path: list) -> None:
    if index == len(digits):
        result.append(''.join(path))
        return
    for letter in phone[digits[index]]:
        path.append(letter)
        backtrack(index + 1, path)
        path.pop()
backtrack(0, [])
return result
```

PHASE 5 — TEST & DEBUG

```
# digits="23":
# backtrack(0,[]): letters=abc
# path=[a] → backtrack(1,[a]): letters=def
# path=[a,d] → backtrack(2) → append "ad"
# path=[a,e] → append "ae"
# path=[a,f] → append "af"
# path=[b] → "bd","be","bf"
# path=[c] → "cd","ce","cf"
# result=["ad","ae","af","bd","be","bf","cd","ce","cf"] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(4^n * n)$ : at most 4 choices per digit, n digits deep, n to build each string.
# Space  $O(n)$  recursion depth +  $O(4^n * n)$  output.
# Iterative BFS and backtracking have same output; backtracking uses less intermediate space.
# Follow-up: if digits can include 0/1 (no letters), skip those digits.
# Follow-up: combination count problem – just compute product of len(phone[d]) for each d."
```

◆ Quick Review

Key Insights

- Use a mapping from digits to the respective letters as defined by a telephone keypad.
- Backtracking is an ideal approach to generate all possible combinations.
- Each digit adds a level in the recursive tree where you append one letter at a time.

- Handling edge cases such as an empty input string is crucial.

Space & Time

Time Complexity: $O(4^n * n)$ where n is the length of the input string (each digit may map to up to 4 letters and adding a combination takes $O(n)$ time).

Space Complexity: $O(n)$ for the recursion call stack, not counting the space required for storing the output.

Solution

We can solve the problem using a backtracking algorithm. First, we define a dictionary mapping each digit to its corresponding letters. Then we initiate a recursive function called `backtrack` that takes the current combination and the index of the next digit to process. At each call, if the combination is complete (length equal to the input string), we add it to the result list. Otherwise, we iterate through the letters corresponding to the current digit, append a letter to the combination, and recursively call `backtrack` with the next index. This approach ensures

that all possible letter combinations are generated.

Backtracking

□ #22 Generate Parentheses

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Backtracking
Lists: Grind 169

Problem

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Example 1:

```
Input: n = 3  
Output: ["((()))", "(()())", "(())()", "()(())", "()()()"]
```

Example 2:

```
Input: n = 1  
Output: ["()"]
```

Constraints:

- $1 \leq n \leq 8$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given n pairs of parentheses, generate all combinations of well-formed parentheses.

Right?"

#

"n=3: ['((()))', '(()())', '()()()', '()(())', '()()()'] – 5 combinations ✓

n=1: ['()'] ✓

n=2: ['()()', '()()'] ✓"

PHASE 2 — BRUTE FORCE

"Generate ALL $2^{(2n)}$ sequences of '(' and ')' of length $2n$,

then filter to keep only the valid ones.

Valid: each prefix has at least as many '(' as ')', and total counts are equal.

Time: $O(2^{(2n)} * n)$ – exponential, extremely wasteful. Space: $O(2^{(2n)} * n)$."

```
class _22_GenerateParentheses_Brute:
```

```
    def generateParenthesis(self, n: int) -> list[str]:
```

```
        def is_valid(s):
```

```
            balance = 0
```

```
            for ch in s:
```

```
                balance += 1 if ch == '(' else -1
```

```
                if balance < 0:
```

```
                    return False
```

```
            return balance == 0
```

```
        def generate_all(current, result):
```

```
            if len(current) == 2 * n:
```

```
                if is_valid(current):
```

```
                    result.append(current)
```

```
                return
```

```
            generate_all(current + '(', result)
```

```
            generate_all(current + ')', result)
```

```
        result = []
```

```
        generate_all('', result)
```

```
        return result
```

PHASE 3 — OPTIMIZE

"Backtracking with validity pruning – only place characters that keep the sequence valid.

Track open and close counts. Rules:

– Add '(' if open < n

– Add ')' if close < open

This eliminates all invalid branches early, producing only Catalan(n) valid strings.

Time: $O(4^n / \sqrt{n})$ – Catalan number. Space: $O(n)$ recursion depth."

PHASE 4 — CLEAN CODE

```
class _22_GenerateParentheses:
```

```
    def generateParenthesis(self, n: int) -> list[str]:
```

```
        result = []
```

```
        def backtrack(path: list, open_count: int, close_count: int) -> None:
```

```

    if len(path) == 2 * n:
        result.append(''.join(path))
        return
    if open_count < n:
        path.append('(')
        backtrack(path, open_count + 1, close_count)
        path.pop()
    if close_count < open_count:
        path.append(')')
        backtrack(path, open_count, close_count + 1)
        path.pop()
    backtrack([], 0, 0)
    return result

```

PHASE 5 — TEST & DEBUG

```

# n=2:
# backtrack([],0,0): add '(' → ([ ( ],1,0)
# add '(' → ([ ( ( ],2,0)
# add ')' → ([ ( ( ) ],2,1)
# add ')' → ([ ( ( ) ) ],2,2) → append "()" ✓
# add ')' → ([ ( ) ] ,1,1)
# add '(' → ([ ( ( ) ],2,1)
# add ')' → ([ ( ) ( ) ],2,2) → append "()()" ✓
# result = ["()", "()()"] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(4^n / \sqrt{n})$ : nth Catalan number of valid sequences, each of length 2n.
# Space  $O(n)$ : recursion depth is at most 2n, path list is at most 2n characters.
# Brute is  $O(2^{(2n)})$  – exponentially worse.
# Follow-up: count valid sequences (not generate) – just return  $Catalan(n) = C(2n,n)/(n+1)$ .
# Follow-up: LC 32 Longest Valid Parentheses – uses a stack or DP instead."

```

◆ Quick Review

Key Insights

- Use a backtracking approach to build the valid combinations step by step.
- At any recursion step, you can add an opening parenthesis if you still have one available.

- You can add a closing parenthesis only if it would not result in an invalid sequence (i.e., the number of closing parentheses is less than the number of opening ones).
- The solution leverages recursion to explore all potential valid sequences while pruning invalid paths early.

Space & Time

Time Complexity: $O(4^n / (n^{3/2}))$

Space Complexity: $O(n)$ for the recursion stack, plus space for storing all valid combinations.

Solution

The solution uses a recursive backtracking technique to generate the combinations. The algorithm maintains counters for the number of '(' and ')' added to the current string. It recursively adds a '(' if the count is less than n and adds a ')' if doing so does not make the string invalid (i.e., if the count of ')' is less than the count of '('). When a valid combination reaches a length of $2 \cdot n$, it is added to the result list.

Backtracking

□ #39 Combination Sum

MED

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking**

Lists: Grind 169

Problem

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8

Output: [[2,2,2,2],[2,3,3],[3,5]]

Example 3:

Input: candidates = [2], target = 1

Output: []

Constraints:

- $1 \leq \text{candidates.length} \leq 30$
- $2 \leq \text{candidates}[i] \leq 40$
- All elements of candidates are distinct.
- $1 \leq \text{target} \leq 40$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given an array of distinct positive integers and a target, return all unique combinations that sum to target. Each number may be used unlimited times.

The solution set must not contain duplicate combinations. Right?"

#

"candidates=[2,3,6,7], target=7:

[2,2,3] ($2+2+3=7$), [7] ($7=7$) ✓

candidates=[2,3,5], target=8: [2,2,2,2],[2,3,3],[3,5] ✓"

PHASE 2 — BRUTE FORCE

```
# "Generate all combinations with repetition up to target sum, filter by sum.
# Equivalent to BFS: start from [], extend by each candidate, keep if sum <= target.
# Time:  $O(n^{(t/\min\_c)})$  – exponential. Space: same."
```

```
class _39_CombinationSum_Brute:
    def combinationSum(self, candidates: list[int], target: int) ->
list[list[int]]:
    result = []
    candidates.sort()
    def generate(start: int, path: list, remaining: int) -> None:
        if remaining == 0:
            result.append(path[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remaining:
                break # pruning: sorted, no point continuing
            path.append(candidates[i])
            generate(i, path, remaining - candidates[i]) # i not i+1: reuse
allowed
            path.pop()
        generate(0, [], target)
    return result
```

PHASE 3 — OPTIMIZE

```
# "The brute approach IS backtracking with pruning – sort first so we can break
early.
# The key optimisations over naive BFS:
# – Start index prevents re-use of earlier candidates (avoids duplicates like [3,2]
and [2,3])
# – Sort + break early prunes branches where candidate > remaining
# Time:  $O(n^{(t/\min\_c)})$ : still exponential in worst case but pruned significantly."
```

PHASE 4 — CLEAN CODE

```
class _39_CombinationSum:
    def combinationSum(self, candidates: list[int], target: int) ->
list[list[int]]:
    candidates.sort()
    result = []
    def backtrack(start: int, path: list, remaining: int) -> None:
        if remaining == 0:
            result.append(path[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remaining:
                break
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i])
            path.pop()
    backtrack(0, [], target)
    return result
```

PHASE 5 — TEST & DEBUG

```
# candidates=[2,3,6,7], target=7:
# backtrack(0,[],7): try 2 → backtrack(0,[2],5)
# try 2 → backtrack(0,[2,2],3)
# try 2 → backtrack(0,[2,2,2],1) → try 2>1 break
# try 3 → backtrack(1,[2,2,3],0) → append [2,2,3] ✓
# try 3 → backtrack(1,[2,3],2) → try 3>2 break
# try 3 → backtrack(1,[3],4) → try 3→[3,3]:remaining=1, try 6>1 break
# try 6 → backtrack(2,[6],1) → try 6>1 break
# try 7 → backtrack(3,[7],0) → append [7] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^{(t/\min\_c)})$ : at most  $t/\min\_c$  levels deep,  $n$  choices each level.
# Space  $O(t/\min\_c)$ : max recursion depth.
# Follow-up: Combination Sum II (LC 40) – each number used ONCE, has duplicates in input.
# Use  $i+1$  instead of  $i$ , and skip duplicate candidates at same level.
# Follow-up: Combination Sum III (LC 216) – exactly  $k$  numbers from 1–9 summing to  $n$ ."
```

◆ Quick Review

Key Insights

- Backtracking is a natural fit to explore all potential combinations.
- Sorting candidates simplifies the process by allowing early termination if the current candidate exceeds the remaining target.
- Recursion is used to build combinations and backtrack once the current path exceeds or meets the target.
- The same candidate can be reused in the combination, so the recursive call does not advance the index.

Space & Time

Time Complexity:

$O(2^{(\text{target}/\min(\text{candidates}))})$ in the worst case where many combinations are possible.

Space Complexity: $O(\text{target}/\min(\text{candidates}))$ for the recursion stack and temporary space used to store combinations.

Solution

The solution uses a backtracking approach. First, sort the candidates to enable pruning of recursive calls once the candidate exceeds the remaining target. A recursive helper function builds up a combination, subtracting the candidate value from the target until it reaches zero (a valid combination) or becomes negative (an invalid path). The recursion allows the same candidate to be chosen again. Each valid combination is then added to the result list.

Backtracking

□ #46 Permutations

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: Grind 169

Problem

Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Example 2:

```
Input: nums = [0,1]
Output: [[0,1],[1,0]]
```

Example 3:

```
Input: nums = [1]
Output: [[1]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 6$
- $-10 \leq \text{nums}[i] \leq 10$
- All the integers of `nums` are unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given an array of distinct integers, return all possible permutations
# in any order. Right?"
#
# "nums=[1,2,3]: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]] – 6 perms ✓
# nums=[0,1]: [[0,1],[1,0]] ✓
# nums=[1]: [[1]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Use a visited array. At each position try every unvisited number.
# When path length equals n, record the permutation.
# Time:  $O(n! * n)$  –  $n!$  permutations each of length  $n$ . Space:  $O(n)$  recursion depth."
```

```
class _46_Permutations_Brute:
    def permute(self, nums: list[int]) -> list[list[int]]:
        result = []
        used = [False] * len(nums)

        def backtrack(path: list) -> None:
            if len(path) == len(nums):
                result.append(path[:])
                return
            for i in range(len(nums)):
                if used[i]:
                    continue
                used[i] = True
                path.append(nums[i])
                backtrack(path)
                path.pop()
                used[i] = False

        backtrack([])
        return result
```

PHASE 3 — OPTIMIZE

```
# "Swap-based in-place backtracking – no visited array needed.
# Maintain a 'start' index. Everything before start is already placed.
# For each position from start onward, swap it into the start slot, recurse, then
# swap back.
# Time:  $O(n! * n)$ . Space:  $O(n)$  recursion depth – avoids the extra visited array."
```

PHASE 4 — CLEAN CODE

```
class _46_Permutations:
```

```
def permute(self, nums: list[int]) -> list[list[int]]:
    result = []
    def backtrack(start: int) -> None:
        if start == len(nums):
            result.append(nums[:])
            return
        for i in range(start, len(nums)):
            nums[start], nums[i] = nums[i], nums[start]
            backtrack(start + 1)
            nums[start], nums[i] = nums[i], nums[start]
    backtrack(0)
    return result
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,2,3]:
# backtrack(0): swap(0,0)→[1,2,3], backtrack(1):
# swap(1,1)→[1,2,3], backtrack(2):
# swap(2,2)→[1,2,3], backtrack(3) → append [1,2,3] ✓
# swap(1,2)→[1,3,2], backtrack(2) → append [1,3,2] ✓, swap back→[1,2,3]
# swap(0,1)→[2,1,3], backtrack(1) → [2,1,3],[2,3,1] ✓, swap back→[1,2,3]
# swap(0,2)→[3,2,1], backtrack(1) → [3,2,1],[3,1,2] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n! * n)$ :  $n!$  permutations, each takes  $O(n)$  to copy.
# Space  $O(n)$ : recursion depth  $n$ , no visited array (swap approach).
# Visited-array approach uses  $O(n)$  extra space for the boolean array.
# Follow-up: Permutations II (LC 47) – input has duplicates. Sort first, skip
duplicate
# values at the same recursion level to avoid identical permutations.
# Follow-up: next permutation (LC 31) – generate just the next one in lexicographic
order."
```

◆ Quick Review

Key Insights

- Use backtracking to generate all possible permutations.
- Build each permutation by selecting numbers one at a time.
- When the current permutation reaches the length of nums, add it to the result.

- **Backtracking** allows you to undo choices and explore different orderings.

Space & Time

Time Complexity: $O(n * n!)$ – There are $n!$ permutations, and constructing each permutation takes $O(n)$ time.

Space Complexity: $O(n)$ – Recursion depth is limited to n , plus additional space for the current permutation.

Solution

The solution employs a backtracking algorithm. A helper function is used to build permutations recursively. At each step, the algorithm iterates through the available numbers, adds one to the current permutation if it has not been used yet, and recurses to handle the remaining numbers. Once the current permutation matches the length of the input array, it is added to the result list. Then, by "backtracking" (removing the last number added), the algorithm explores alternative possibilities. This systematic exploration ensures that all possible permutations are generated.

Backtracking

□ #79 Word Search

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Backtracking · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ grid of characters board and a string word, return true if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "ABCCED"

Output: true

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "SEE"

Output: true

Example 3:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "ABCB"

Output: false

Constraints:

- $m == \text{board.length}$
- $n = \text{board}[i].\text{length}$
- $1 \leq m, n \leq 6$
- $1 \leq \text{word.length} \leq 15$
- board and word consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger board?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given an m×n grid of characters and a word, return true if the word exists
# in the grid following adjacent (up/down/left/right) cells without reusing cells.
# Right?"
#
# "board=[[ 'A', 'B', 'C', 'E'], ['S', 'F', 'C', 'S'], ['A', 'D', 'E', 'E']], word='ABCCED':
# A(0,0)→B(0,1)→C(0,2)→C(1,2)→E(2,2)→D(2,1) ✓
# word='ABCB': A→B→C→B would revisit B(0,1) → false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "For every cell as potential start, try every possible path with DFS.
# Mark visited cells with a sentinel, unmark on backtrack.
# Time:  $O(m \times n \times 4^L)$  where  $L$  = word length. Space:  $O(L)$  recursion depth."
```

```
class _79_WordSearch_Brute:
    def exist(self, board: list[list[str]], word: str) -> bool:
        rows, cols = len(board), len(board[0])
        def dfs(r, c, idx):
            if idx == len(word):
                return True
            if r < 0 or r >= rows or c < 0 or c >= cols:
                return False
            if board[r][c] != word[idx]:
                return False
            tmp, board[r][c] = board[r][c], '#'
            found = (dfs(r+1, c, idx+1) or dfs(r-1, c, idx+1) or
                    dfs(r, c+1, idx+1) or dfs(r, c-1, idx+1))
            board[r][c] = tmp
            return found
        for r in range(rows):
            for c in range(cols):
                if dfs(r, c, 0):
                    return True
        return False
```

PHASE 3 — OPTIMIZE

"Same DFS with two pruning improvements:

1. Before DFS, check character frequency – if board has fewer of any letter than word needs, return False early.
 # 2. If word's last char is rarer in board than first char, reverse the word to start from the rarer end.
 # These prune the search space significantly for large grids.
 # Time: $O(m \times n \times 4^L)$ worst case but much faster in practice."

PHASE 4 — CLEAN CODE

```
class _79_WordSearch:
    def exist(self, board: list[list[str]], word: str) -> bool:
        from collections import Counter
        rows, cols = len(board), len(board[0])
        board_count = Counter(c for row in board for c in row)
        word_count = Counter(word)
        for ch, cnt in word_count.items():
            if board_count[ch] < cnt:
                return False
        if board_count[word[0]] > board_count[word[-1]]:
            word = word[::-1]
        def dfs(r, c, idx):
            if idx == len(word):
                return True
            if not (0 <= r < rows and 0 <= c < cols) or board[r][c] != word[idx]:
                return False
            tmp, board[r][c] = board[r][c], '#'
            found = (dfs(r+1,c,idx+1) or dfs(r-1,c,idx+1) or
                    dfs(r,c+1,idx+1) or dfs(r,c-1,idx+1))
            board[r][c] = tmp
            return found
        return any(dfs(r, c, 0) for r in range(rows) for c in range(cols))
```

PHASE 5 — TEST & DEBUG

board=[['A','B','C','E'],...], word='ABCCED':
 # Start dfs(0,0,'A'): match→mark '#'. dfs(0,1,'B'): match→mark. dfs(0,2,'C'): match.
 # dfs(1,2,'C'): match. dfs(2,2,'E'): match. dfs(2,1,'D'): match. idx=6=len → True ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \times n \times 4^L)$: each cell can start a DFS branching 4 ways up to L levels deep.
 # Space $O(L)$: recursion stack depth bounded by word length.
 # Follow-up: Word Search II (LC 212) – find ALL words from a list in the board.
 # Build a Trie of all words, DFS the board once – $O(m \times n \times 4^L + \text{sum of word lengths})$."

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore each cell in the board as a potential starting point.
- Employ backtracking to mark cells as visited when exploring a path and revert them back when backtracking.
- Check boundary conditions and ensure the current cell's character matches the corresponding character in the word.
- Given the small grid and word sizes, a recursive solution is both feasible and straightforward.
- Search pruning can be applied by stopping a DFS branch as soon as the current path does not match the prefix of the target word.

Space & Time

Time Complexity: $O(m * n * 3^L)$, where L is the length of the word (each DFS may branch to at most 3 further cells, excluding the coming cell).

Space Complexity: $O(L)$ due to recursion call stack (where L is the word length).

Solution

The solution uses DFS with backtracking to traverse the grid. For each cell, we: Check if

the cell matches the first character of the word. Recursively explore the adjacent cells (up, down, left, right) while marking the current cell as visited. If the path does not lead to a complete match, backtrack by resetting the visited cell. The DFS stops if all characters in the word have been found in sequence. The algorithm leverages in-place modifications (or an auxiliary visited array) to avoid revisiting cells while exploring a candidate path.

Backtracking

#113 Path Sum II

MED

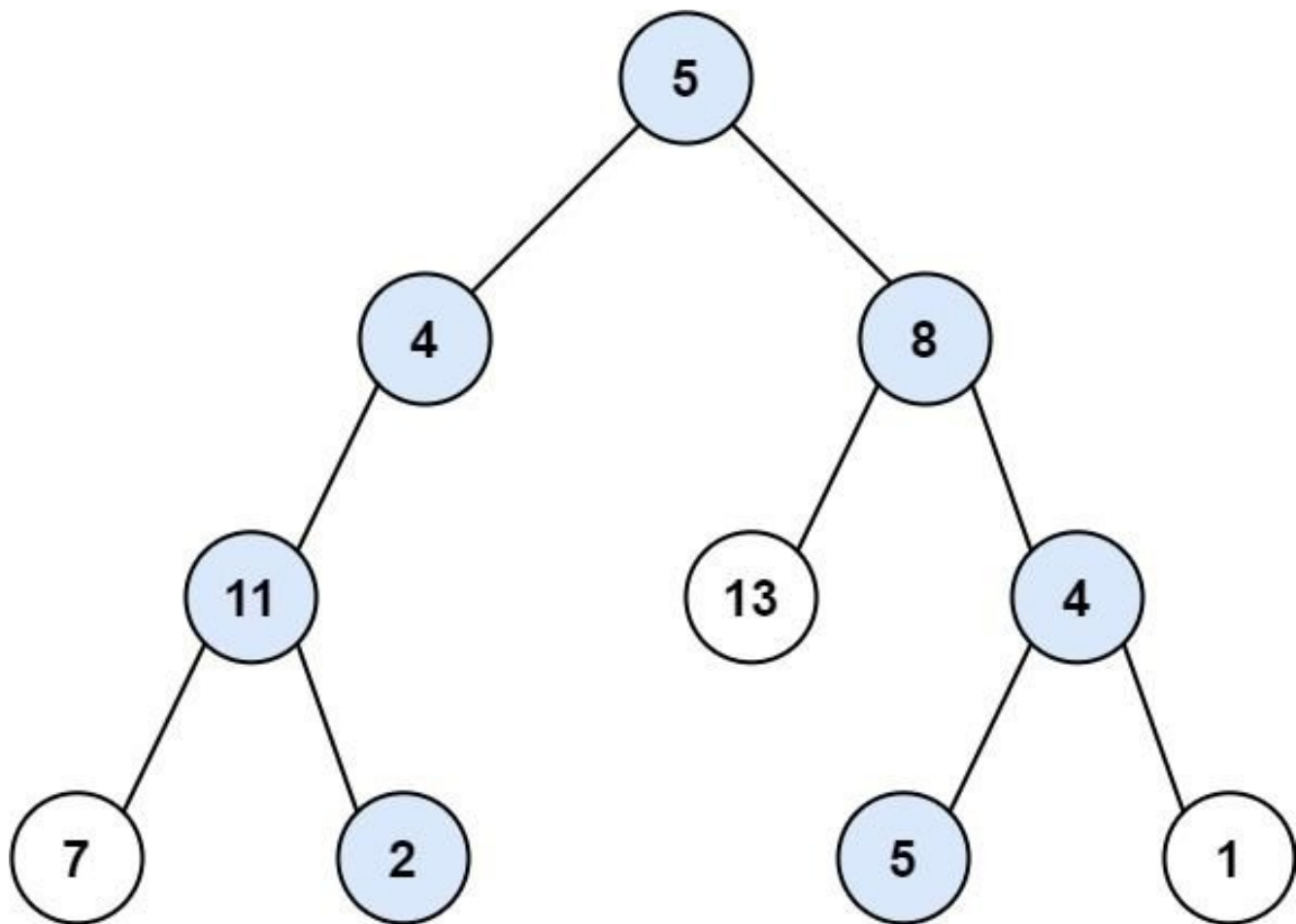
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Backtracking · Tree · DFS
Lists: Grind 169

Problem

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references.

A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

Example 1:



Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22

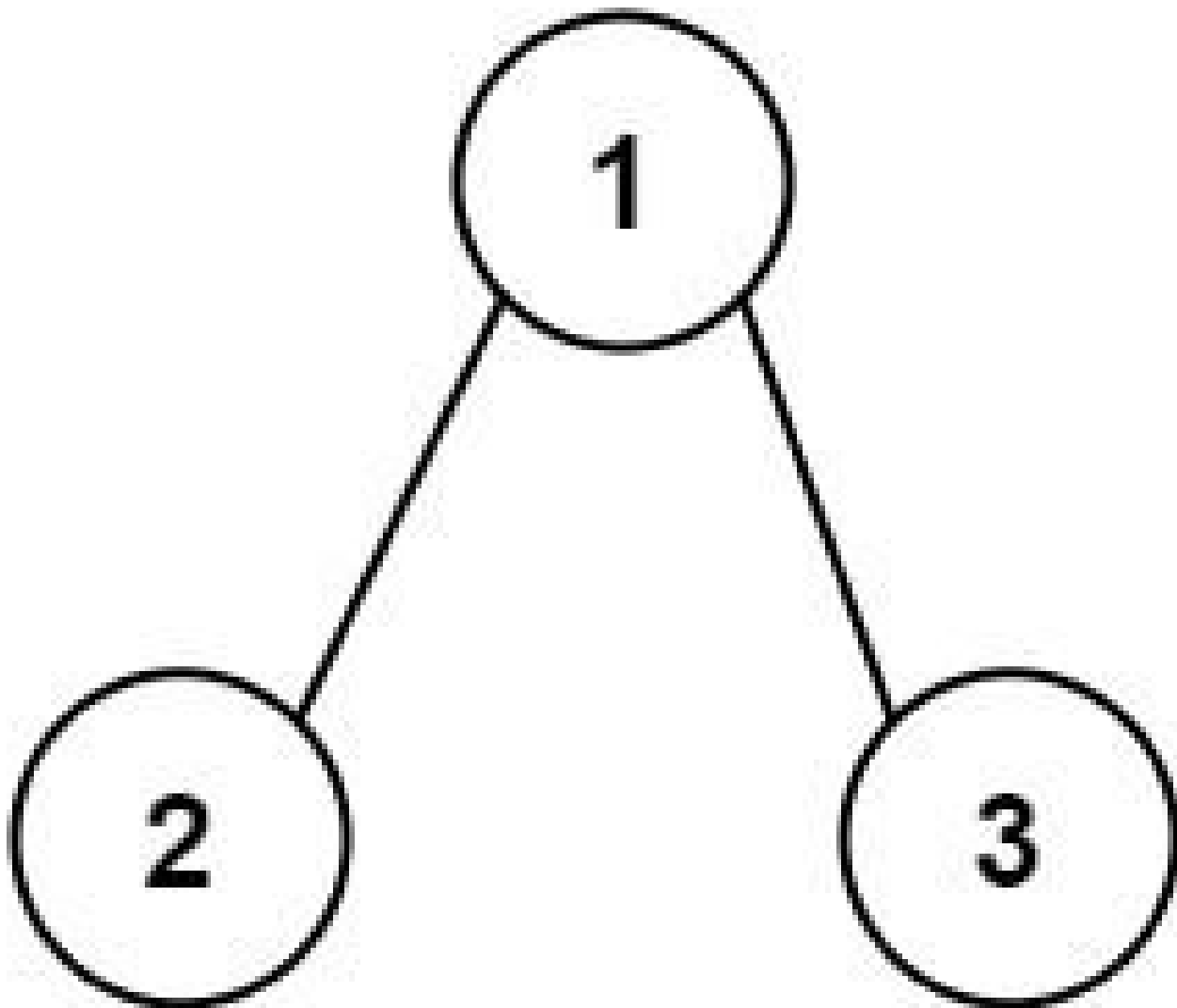
Output: [[5,4,11,2],[5,8,4,5]]

Explanation: There are two paths whose sum equals targetSum:

$5 + 4 + 11 + 2 = 22$

$5 + 8 + 4 + 5 = 22$

Example 2:



Input: root = [1,2,3], targetSum = 5
Output: []

Example 3:

Input: root = [1,2], targetSum = 0
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-1000 \leq \text{Node.val} \leq 1000$
- $-1000 \leq \text{targetSum} \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a binary tree and a target sum, return all root-to-leaf paths
# where the sum of node values equals targetSum. Right?"
#
# "tree=[5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum=22:
# 5→4→11→2=22 ✓, 5→8→4→5=22 ✓ → [[5,4,11,2],[5,8,4,5]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "DFS collecting all root-to-leaf paths, then filter by sum.
# Time: O(n*L) where L = path length (average tree height). Space: O(n*L)."
```

```
class _113_PathSumII_Brute:
    def pathSum(self, root, targetSum: int) -> list[list[int]]:
        all_paths = []
        def dfs(node, path):
            if not node:
                return
            path.append(node.val)
            if not node.left and not node.right:
                all_paths.append(path[:])
            dfs(node.left, path)
            dfs(node.right, path)
            path.pop()
        dfs(root, [])
        return [p for p in all_paths if sum(p) == targetSum]
```

PHASE 3 — OPTIMIZE

```
# "Backtracking with running sum – check target only at leaf nodes.
# Pass remaining = targetSum – node.val down the tree.
# At a leaf, if remaining == 0, this path is a solution. Append and backtrack.
# Time: O(n*L). Space: O(L) recursion + O(n*L) output – avoids post-filtering."
```

PHASE 4 — CLEAN CODE

```
class _113_PathSumII:
    def pathSum(self, root, targetSum: int) -> list[list[int]]:
        result = []
        def backtrack(node, remaining, path):
            if not node:
                return
            path.append(node.val)
            remaining -= node.val
```

```

        if not node.left and not node.right and remaining == 0:
            result.append(path[:])
        else:
            backtrack(node.left, remaining, path)
            backtrack(node.right, remaining, path)
        path.pop()
    backtrack(root, targetSum, [])
    return result

```

PHASE 5 — TEST & DEBUG

```

# tree=[5,4,8,11,null,13,4,7,2,null,null,5,1], target=22:
# path=[5],rem=17 → left=[5,4],rem=13 → left=[5,4,11],rem=2
# left=[5,4,11,7],rem=-5 → leaf, rem≠0. pop.
# right=[5,4,11,2],rem=0 → leaf, rem=0 → append [5,4,11,2] ✓
# right=[5,8],rem=14 → right=[5,8,4],rem=10 → left=[5,8,4,5],rem=5 → leaf≠0
# right=[5,8,4,1],rem=9 → leaf≠0. Back up. left=[5,8,13],leaf,rem=1≠0.

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n*L): visit each node once, copy path at each leaf O(L).
# Space O(L): recursion stack depth = tree height.
# Follow-up: Path Sum I (LC 112) – just return bool, no need to store paths.
# Follow-up: Path Sum III (LC 437) – any path (not just root-to-leaf), any
direction.
# Use prefix sum hashmap to count valid paths in O(n)."

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore all paths from the root to the leaves.
- Backtracking is essential: add the current node value to the path before the recursive calls and remove it (backtrack) after exploring both subtrees.
- At each leaf, check if the accumulated sum equals targetSum.

- Maintain a running sum or subtract the current node's value from the target sum as you traverse.

Space & Time

Time Complexity: $O(N^2)$ in the worst-case scenario (where N is the number of nodes) when most nodes are leaf nodes, since each potential path copy operation can be $O(N)$.

Space Complexity: $O(N)$ for the recursion stack and the path storage, where N is the height of the tree.

Solution

We solve the problem using a recursive DFS approach combined with backtracking. The idea is to traverse the tree while maintaining the current path and the remaining sum needed to reach targetSum. When a leaf node is reached (node with no children), we check if the remaining sum equals the leaf's value – if yes, we record a copy of the current path. Data structures and algorithmic approaches used: Recursion for DFS: to traverse to each leaf. List (or Array) for storing the current path. Backtracking: to remove the last added

element when stepping back up the recursion stack. Copying the path when a valid leaf is reached to store it in the final result. A common pitfall is to forget to backtrack, which would result in incorrect paths being recorded.

Greedy

Round 5 · Mid · Medium · 5 questions

Greedy

□ #134 Gas Station

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy
Lists: Grind 169

Problem

There are n gas stations along a circular route, where the amount of gas at the i th station is $\text{gas}[i]$.

You have a car with an unlimited gas tank and it costs $\text{cost}[i]$ of gas to travel from the i th station to its next $(i + 1)$ th station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays gas and cost , return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1 . If there exists a solution, it is guaranteed to be unique.

Example 1:

Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]

Output: 3

Explanation:

Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = $0 + 4 = 4$

Travel to station 4. Your tank = $4 - 1 + 5 = 8$

Travel to station 0. Your tank = $8 - 2 + 1 = 7$

Travel to station 1. Your tank = $7 - 3 + 2 = 6$

Travel to station 2. Your tank = $6 - 4 + 3 = 5$

Example 2:

Input: gas = [2,3,4], cost = [3,4,3]

Output: -1

Explanation:

You can't start at station 0 or 1, as there is not enough gas to travel to the next station.

Let's start at station 2 and fill up with 4 unit of gas. Your tank = $0 + 4 = 4$

Travel to station 0. Your tank = $4 - 3 + 2 = 3$

Travel to station 1. Your tank = $3 - 3 + 3 = 3$

You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.

Constraints:

- $n == \text{gas.length} == \text{cost.length}$
- $1 \leq n \leq 105$
- $0 \leq \text{gas}[i], \text{cost}[i] \leq 104$
- The input is generated such that the answer is unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"n gas stations in a circle; gas[i] = gas at station i, cost[i] = gas to reach i+1."

```

# Find the starting station index to complete the circuit, or -1 if impossible.
Right?"
#
# "gas=[1,2,3,4,5], cost=[3,4,5,1,2]:
# Start at 3: 0+4=4-4-1=3-3+5=8-8-2=6-6+1=7-7-3=4≥0 → return 3 ✓
# gas=[2,3,4], cost=[3,4,3]: total gas=9 < total cost=10 → -1 ✓"

# PHASE 2 — BRUTE FORCE
# "Try each station as start. Simulate the full circuit; if we never run dry, return
that index.
# Time: O(n²). Space: O(1)."
```

```

class _134_GasStation_Brute:
    def canCompleteCircuit(self, gas: list[int], cost: list[int]) -> int:
        n = len(gas)
        for start in range(n):
            tank = 0
            completed = True
            for i in range(n):
                idx = (start + i) % n
                tank += gas[idx] - cost[idx]
                if tank < 0:
                    completed = False
                    break
            if completed:
                return start
        return -1

# PHASE 3 — OPTIMIZE
# "Two greedy observations:
# 1. If total gas < total cost, no solution exists → return -1.
# 2. If a solution exists, it's unique. Start with candidate = 0 and tank = 0.
# Accumulate net gain (gas[i]-cost[i]). When tank < 0, reset tank to 0 and
# move candidate to i+1 (we can't start anywhere in [old_candidate, i]).
# The final candidate is the answer.
# Time: O(n). Space: O(1)."
```

```

# PHASE 4 — CLEAN CODE
class _134_GasStation:
    def canCompleteCircuit(self, gas: list[int], cost: list[int]) -> int:
        if sum(gas) < sum(cost):
            return -1
        tank = 0
        candidate = 0
        for i in range(len(gas)):
            tank += gas[i] - cost[i]
            if tank < 0:
                tank = 0
                candidate = i + 1
        return candidate

```

PHASE 5 — TEST & DEBUG

```
# gas=[1,2,3,4,5], cost=[3,4,5,1,2]:
# total=15>=15 ✓. i=0:tank=1-3=-2<0→tank=0,cand=1. i=1:tank=2-4=-2<0→cand=2.
# i=2:tank=3-5=-2<0→cand=3. i=3:tank=4-1=3. i=4:tank=3+5-2=6≥0.
# return 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single pass. Space O(1): two variables.
# The key insight: if we run out of gas starting at s, no station between s and i
can work
# (they'd all start with less gas than s did at that point).
# Follow-up: if multiple valid starting points exist (which the problem guarantees
won't happen),
# return the smallest index – the greedy still finds the first valid one.
# Follow-up: minimum cost to fill gas (variant) – uses a monotonic stack approach."
```

◆ Quick Review

Key Insights

- If the total amount of gas available is less than the total travel cost, then completing the circuit is impossible.
- A greedy approach can be used by keeping track of the current surplus of gas. If the surplus becomes negative at any station, the journey cannot start from any station before that point.
- Restart the journey from the next station whenever the surplus goes negative.
- The problem guarantees that if a solution exists, it is unique.

Space & Time

Time Complexity: $O(n)$, where n is the number of gas stations, as we traverse the list only once.

Space Complexity: $O(1)$, as no extra space is used other than a few variables.

Solution

The solution involves iterating over the list of stations while maintaining two variables: one for the current gas surplus and one for the total gas surplus. If at any station the current surplus drops below zero, the current starting station is not viable and we set the starting station to the next index and reset the current surplus. After one pass, if the total gas surplus is non-negative, then the identified station is the valid starting point.

Greedy

□ #179 Largest Number

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Greedy · Sorting
Lists: Grind 169

Problem

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

Input: `nums = [10,2]`
Output: `"210"`

Example 2:

Input: `nums = [3,30,34,5,9]`
Output: `"9534330"`

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 10^9$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given a list of non-negative integers, arrange them so they form the largest number."

Return as a string. Right?"

#

"nums=[10,2]: '210' > '102' → "210" ✓

nums=[3,30,34,5,9]: → "9534330" ✓

nums=[0,0]: → "0" (not "00") ✓"

PHASE 2 — BRUTE FORCE

"Try all permutations of nums, form the concatenated string for each,

return the maximum. Time: $O(n! * n)$. Space: $O(n!)$."

```
class _179_LargestNumber_Brute:
    def largestNumber(self, nums: list[int]) -> str:
        from itertools import permutations
        best = ''
        for perm in permutations(nums):
            candidate = ''.join(map(str, perm))
            if candidate > best:
                best = candidate
        return best
```

PHASE 3 — OPTIMIZE

"Custom comparator: to decide whether a comes before b, compare str(a)+str(b) vs str(b)+str(a)."

If str(a)+str(b) > str(b)+str(a), a should come first.

Sort all numbers by this comparator, then concatenate.

Edge case: if the largest digit is 0, the result is '0'.

Time: $O(n \log n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _179_LargestNumber:
    def largestNumber(self, nums: list[int]) -> str:
        from functools import cmp_to_key
        def compare(a, b):
            return (1 if a + b > b + a else -1 if a + b < b + a else 0)
        strs = list(map(str, nums))
        strs.sort(key=cmp_to_key(compare), reverse=True)
        result = ''.join(strs)
        return '0' if result[0] == '0' else result
```

PHASE 5 — TEST & DEBUG

nums=[10,2]:


```
# compare("10","2"): "102" vs "210" → "210">"102" → 2 before 10. sorted=["2","10"].
→ "210" ✓
#
# nums=[3,30,34,5,9]:
# compare pairs: "9">"53">"534">"5330"→ sorted=["9","5","34","3","30"]. → "9534330"
✓
#
# nums=[0,0]: sorted=["0","0"] → "00" → first char '0' → return "0" ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): sort dominates. Space O(n): string array + result.
# The comparator is transitive and consistent – provably correct for any input.
# Follow-up: smallest number from array – same comparator, ascending sort.
# Follow-up: if numbers are very large (BigInteger), convert to strings first
anyway."
```

◆ Quick Review

Key Insights

- The order in which numbers appear drastically affects the concatenated result.
- A custom sorting strategy is required where two numbers are compared by checking which concatenation (first–second vs second–first) yields a larger number.
- Edge case: When the numbers are all zeros, the result should be "0" instead of several leading zeros.

Space & Time

Time Complexity: $O(n \log n * k)$, where n is the number of elements in the array and k is the maximum number of digits in a number (due

to string comparisons in the custom sort).
Space Complexity: $O(n * k)$ for storing and processing the strings.

Solution

We solve the problem by converting all integers to strings so that we can use string concatenation to compare the results. A custom comparator is defined that, given two number strings, compares the concatenated results in both orders. By sorting the array using this comparator in descending order, we ensure that placing the highest contributing number first yields the largest possible number. After sorting, a check is performed to return "0" if the highest number is "0". Finally, the sorted strings are concatenated to form the answer.

Greedy

□ ★ #280 Wiggle Sort ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting

Lists: ★ Premium | Premium 98

Problem

Given an integer array `nums`, reorder it such that `nums[0] <= nums[1] >= nums[2] <= nums[3]....`

You may assume the input array always has a valid answer.

Example 1:

Input: `nums = [3,5,2,1,6,4]`
Output: `[3,5,1,6,2,4]`
Explanation: `[1,6,2,5,3,4]` is also accepted.

Example 2:

Input: `nums = [6,6,5,6,3,8]`
Output: `[6,6,5,6,3,8]`

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $0 \leq \text{nums}[i] \leq 10^4$
- It is guaranteed that there will be an answer for the given input `nums`.

Follow up: Could you solve the problem in $O(n)$ time complexity?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Reorder an array in-place so that nums[0] <= nums[1] >= nums[2] <= nums[3]...
# (alternating less-than and greater-than). Multiple valid answers exist. Right?"
#
# "nums=[3,5,2,1,6,4]: one valid output=[3,5,1,6,2,4] ✓
# nums=[1,5,1,1,6,4]: one valid=[1,5,1,6,1,4] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Sort the array, then interleave first half and second half.
# Split sorted array into two halves; place smaller half at even indices,
# larger half at odd indices.
# Time:  $O(n \log n)$ . Space:  $O(n)$  for the copy."
```

```
class _280_WiggleSort_Brute:
```

```
    def wiggleSort(self, nums: list[int]) -> None:
        sorted_nums = sorted(nums)
        n = len(nums)
        mid = (n + 1) // 2
        nums[0::2] = sorted_nums[:mid] # even indices: smaller half
        nums[1::2] = sorted_nums[mid:] # odd indices: larger half
```

PHASE 3 — OPTIMIZE

```
# "Greedy one-pass: for each index i, if the wiggle condition is violated, swap.
# - Even index i: nums[i] should be <= nums[i-1]. If not, swap(i, i-1).
# - Odd index i: nums[i] should be >= nums[i-1]. If not, swap(i, i-1).
# A local swap always fixes the current violation without breaking prior pairs.
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
class _280_WiggleSort:
    def wiggleSort(self, nums: list[int]) -> None:
        for i in range(1, len(nums)):
            if (i % 2 == 1 and nums[i] < nums[i - 1]) or (i % 2 == 0 and nums[i] >
nums[i - 1]):
                nums[i], nums[i - 1] = nums[i - 1], nums[i]
```

PHASE 5 — TEST & DEBUG

```
# nums=[3,5,2,1,6,4]:
# i=1(odd): 5>=3 ✓
# i=2(even): 2<=5 ✓
# i=3(odd): 1<2 → swap → [3,5,1,2,6,4]. Wait: now
[3,5,2,1,6,4]→swap(2,1)→[3,5,1,2,6,4]
# i=4(even): 6>2 → swap → [3,5,1,6,2,4]
# i=5(odd): 4>2 ✓ → [3,5,1,6,2,4] ✓ (valid wiggle)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Greedy: O(n) time, O(1) space vs brute O(n log n) time, O(n) space.
# The greedy works because swapping adjacent elements to fix one condition
# never invalidates the previously satisfied condition.
# Follow-up: Wiggle Sort II (LC 324) – strict inequality nums[0]<nums[1]>nums[2]<...
# Requires median partitioning (nth_element) to guarantee strict separation.
# Follow-up: count wiggle subsequences (LC 376) – DP/greedy to count without
reordering."
```

◆ Quick Review

Key Insights

- The alternating "wiggle" requirement can be satisfied by ensuring two conditions:
- For every odd index i , $\text{nums}[i]$ should be greater than or equal to $\text{nums}[i-1]$.
- For every even index i (where $i > 0$), $\text{nums}[i]$ should be less than or equal to $\text{nums}[i-1]$.
- A single pass through the array can ensure these conditions by swapping adjacent elements when the condition is violated.
- This greedy approach works because the local adjustments guarantee the overall wiggle pattern without needing to sort the entire array.

Space & Time

Time Complexity: $O(n)$ — Only one pass through the array is needed.

Space Complexity: $O(1)$ — The modifications are done in-place without extra data structures.

Solution

We can achieve a wiggle sort in one pass by iterating through the array and, at each index i (starting from 1), checking if the pair $(\text{nums}[i-1], \text{nums}[i])$ satisfies the wiggle condition. If i is odd, $\text{nums}[i]$ should be at least $\text{nums}[i-1]$; if not, we swap them. Conversely, if i is even and $\text{nums}[i]$ is greater than $\text{nums}[i-1]$, then we swap them. These local swaps adjust the order incrementally while ensuring that the overall wiggle pattern is maintained. This greedy approach works because only two adjacent numbers are involved in any violation and fixing them locally does not disturb the rest of the order.

Greedy

□ #954 Array of Doubled Pairs

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Sorting
Lists: CodeSignal

Problem

Given an integer array of even length `arr`, return `true` if it is possible to reorder `arr` such that $arr[2 * i + 1] = 2 * arr[2 * i]$ for every $0 \leq i < \text{len}(arr) / 2$, or `false` otherwise.

Example 1:

Input: `arr = [3,1,3,6]`
Output: `false`

Example 2:

Input: `arr = [2,1,2,6]`
Output: `false`

Example 3:

Input: `arr = [4,-2,2,-4]`
Output: `true`
Explanation: We can take two groups, `[-2,-4]` and `[2,4]` to form `[-2,-4,2,4]` or `[2,4,-2,-4]`.

Constraints:

- $2 \leq \text{arr.length} \leq 3 * 10^4$

- `arr.length` is even.
- $-105 \leq \text{arr}[i] \leq 105$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given an integer array, return true if we can reorder it so that for every
#  $\text{arr}[2i+1] = 2 * \text{arr}[2i]$  (each pair: smaller then double). Right?"
#
# "arr=[3,1,3,6]: pairs (1,2) missing 2, can't pair → False... wait:
# arr=[4,-2,2,-4]: sort by abs → [-2,-4,2,4]? pairs (-2,-4)?  $-4 = -2 * 2$  ✓, (2,4) ✓ →
# True ✓
# arr=[1,2,4,16,8,4]: sort abs→[1,2,4,4,8,16]: 1→2 ✓, 4→8 ✓, 4→16? No → False ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Sort by absolute value. Greedily pair smallest unmatched element with its double.
# Use a counter to track available elements.
# Time:  $O(n \log n)$ . Space:  $O(n)$ ."
```

```
class _954_ArrayOfDoubledPairs_Brute:
    def canReorderDoubled(self, arr: list[int]) -> bool:
        from collections import Counter
        count = Counter(arr)
        for x in sorted(arr, key=abs):
            if count[x] == 0:
                continue
            if count[2 * x] == 0:
                return False
            count[x] -= 1
            count[2 * x] -= 1
        return True
```

PHASE 3 — OPTIMIZE

```
# "Same greedy but iterate over sorted unique keys (by abs value) to avoid
# processing same element multiple times.
# Time:  $O(n \log n)$  for sorting. Space:  $O(n)$  counter."
```

PHASE 4 — CLEAN CODE

```
class _954_ArrayOfDoubledPairs:
    def canReorderDoubled(self, arr: list[int]) -> bool:
        from collections import Counter
```



```

count = Counter(arr)
for x in sorted(count, key=abs):
    if count[2 * x] < count[x]:
        return False
    count[2 * x] -= count[x]
    count[x] = 0
return True

```

PHASE 5 — TEST & DEBUG

```

# arr=[4,-2,2,-4]: count={4:1,-2:1,2:1,-4:1}.
# sorted by abs: [-2,-4,2,4].
# x=-2: count[2*(-2)]=count[-4]=1>=count[-2]=1 ✓. count[-4]=0, count[-2]=0.
# x=-4: count[-8]=0>=count[-4]=0 ✓ (trivially).
# x=2: count[4]=1>=count[2]=1 ✓. count[4]=0, count[2]=0.
# return True ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n log n): sorting dominates. Space O(n) for counter.
# Key: sort by absolute value handles negative numbers correctly (-2 pairs with -4).
# Follow-up: if 'double' is replaced by 'k times' – same greedy, check count[k*x].
# Follow-up: find the actual pairs (not just existence) – store pairs during the greedy pass."

```

◆ Quick Review

Key Insights

- Sort the array based on the absolute value of the elements to handle negative numbers correctly.
- Use a hash table (or frequency counter) to track the occurrence of each element.
- For each number in the sorted list, if the number is still available in the frequency map, try to pair it with its double.
- Decrement the appropriate counts and validate that it is possible to find sufficient

doubles for all numbers.

- **Special case: when dealing with zeros, pairing works within the same value.**

Space & Time

Time Complexity: $O(n \log n)$ due to sorting.

Space Complexity: $O(n)$ for storing the frequency map.

Solution

The solution starts by counting the frequency of each element in the array. Sorting the array by absolute value ensures that when we process an element, any potential pair (i.e., its double) comes later in the sequence. For each number, if there are available occurrences, we try to match it with its double. If there are not enough doubles available, the answer is false. If all elements can be paired appropriately by decrementing their counts in the frequency map, the function returns true.

Greedy

□ #2131 Longest Palindrome by Concatenating Two Letter Words

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Greedy · Counting
Lists: CodeSignal

Problem

You are given an array of strings `words`. Each element of `words` consists of two lowercase English letters.

Create the longest possible palindrome by selecting some elements from `words` and concatenating them in any order. Each element can be selected at most once.

Return the length of the longest palindrome that you can create. If it is impossible to create any palindrome, return 0.

A palindrome is a string that reads the same forward and backward.

Example 1:

Input: words = ["lc","cl","gg"]

Output: 6

Explanation: One longest palindrome is "lc" + "gg" + "cl" = "lcggcl", of length 6.

Note that "clggcl" is another longest palindrome that can be created.

Example 2:

Input: words = ["ab","ty","yt","lc","cl","ab"]

Output: 8

Explanation: One longest palindrome is "ty" + "lc" + "cl" + "yt" = "tylcclyt", of length 8.

Note that "lcyttycl" is another longest palindrome that can be created.

Example 3:

Input: words = ["cc","ll","xx"]

Output: 2

Explanation: One longest palindrome is "cc", of length 2.

Note that "ll" is another longest palindrome that can be created, and so is "xx".

Constraints:

- $1 \leq \text{words.length} \leq 105$
- $\text{words}[i].\text{length} == 2$
- $\text{words}[i]$ consists of lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given an array of two-letter words, select some subset to concatenate into the longest

palindrome possible (each word used at most once). Return the max length. Right?"

#

"words=['lc','cl','gg']: 'lc'+ 'gg'+ 'cl' = 'lcggcl' → len=6 ✓

```
# words=['ab','ty','yt','lc','cl','ab']: 'ty'+ 'ab'+ 'cl'? No... 'lc'+ 'ab'+ 'ba'+ 'cl'
but no 'ba'.
# Actually: 'ty'+ 'yt'=tyyt, 'lc'+ 'cl'=lccl → 'tylccltyt'? We pick pairs:
(ty, yt), (lc, cl) → 8?
# Hmm: 'lc'+ 'cl'='lccl'(pal), 'ty'+ 'yt'='tyyt'(pal) → concat='lccl'+ 'tyyt' not
palindrome.
# Better: 'ty'+ 'lc'+ 'cl'+ 'yt'='tylccltyt' → palindrome! length=8 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Count each word's frequency. For each word that is NOT a palindrome (ab≠ba),
# pair it with its reverse. For palindromic words (gg, aa), use pairs of two + at
most one center.
# Time: O(n). Space: O(n)."
```

```
class _2131_LongestPalindromeTwoLetterWords_Brute:
    def longestPalindrome(self, words: list[str]) -> int:
        from collections import Counter
        count = Counter(words)
        length = 0
        center_used = False
        for word, cnt in count.items():
            rev = word[::-1]
            if word == rev: # palindromic word like "gg"
                pairs = cnt // 2
                length += pairs * 4
                if cnt % 2 == 1 and not center_used:
                    length += 2
                    center_used = True
            elif word < rev and rev in count: # process each pair once
                pairs = min(cnt, count[rev])
                length += pairs * 4
        return length
```

PHASE 3 — OPTIMIZE

```
# "Same greedy – O(n) time/space – but process pairs more cleanly.
# Non-palindromic words: match word with reverse(word), each pair contributes 4
chars.
# Palindromic words: each pair-of-two contributes 4 chars, leftover one contributes
2 (center)."
```

PHASE 4 — CLEAN CODE

```
class _2131_LongestPalindromeTwoLetterWords:
    def longestPalindrome(self, words: list[str]) -> int:
        from collections import Counter
        count = Counter(words)
        length = 0
        center = False
        seen = set()
        for word in count:
            rev = word[::-1]
            if word == rev:
```

```

        pairs = count[word] // 2
        length += pairs * 4
        if count[word] % 2 == 1:
            center = True
    elif rev in count and word not in seen:
        pairs = min(count[word], count[rev])
        length += pairs * 4
        seen.add(word)
        seen.add(rev)
    if center:
        length += 2
    return length

```

PHASE 5 — TEST & DEBUG

```

# words=['lc','cl','gg']:
# count={'lc':1,'cl':1,'gg':1}.
# 'lc': rev='cl', not palindrome. min(1,1)=1 pair → length+=4. seen={lc,cl}.
# 'gg': palindrome. pairs=0, count%2=1 → center=True.
# length=4+2=6 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): one pass through words. Space O(n): counter.
# At most one palindromic word can sit in the center of the final palindrome.
# Non-palindromic pairs must mirror each other (word ↔ reverse).
# Follow-up: Longest Palindrome (LC 409) – same structure with single characters.
# Follow-up: return the actual palindrome string – construct it from paired words."

```

◆ Quick Review

Key Insights

- Pair words with their reverse (e.g., "ab" with "ba") to form a palindrome.
- Handle words that are palindromic by themselves (like "gg" or "aa") differently; they can be used in pairs and one extra can be placed in the center.
- Use a hash map (or dictionary) to count frequencies of each word.

- For non-palindromic word pairs, consider each pair only once to avoid double counting.
- For self-palindromic words, count how many pairs can be used and whether an extra word can be placed in the middle to maximize length.

Space & Time

Time Complexity: $O(n)$, where n is the number of words, since we iterate through the list and perform constant time operations per word.

Space Complexity: $O(n)$, to store counts of words in the hash map.

Solution

The approach is to count the occurrences of each word using a hash map. For each word: If the word is not self-palindromic, check if its reverse exists and form pairs by considering the minimum count between the word and its reverse. If the word is self-palindromic, form as many pairs as possible (using $\text{count} // 2$) and note if there is any word left; you can use one extra self-palindromic word as the center of the palindrome. Finally, compute the total length by multiplying the number of words

used by 2, since each word contributes 2 characters.

Round 6 · Mid · Hard

Round 6

Mid Priority · Hard

28 questions · 5 patterns

Linked List #25 #432 #460

Stack #32 #42 #84 #224 #239 #716 #772 #895

**Heap #23 #218 #272 #295 #358 #499 #632
#759 #1425**

Trie #212 #336 #588 #642

Backtracking #37 #51 #126 #996

Linked List

Round 6 · Mid · Hard · 3 questions

Linked List

□ #25 Reverse Nodes in k-Group

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion

Lists: Grind 169

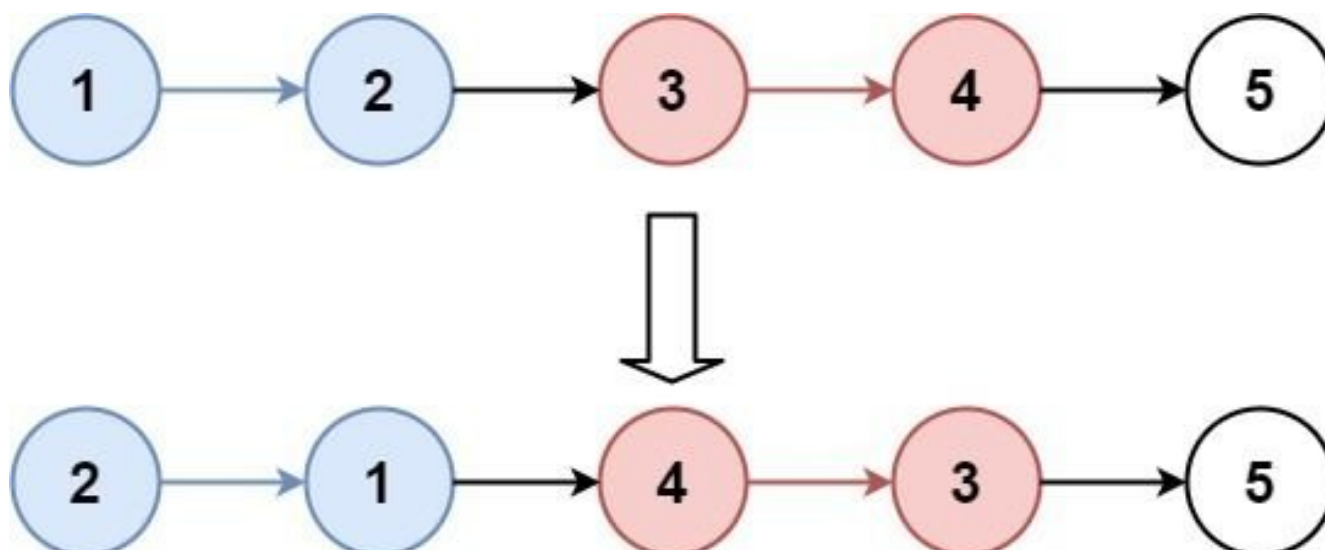
Problem

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

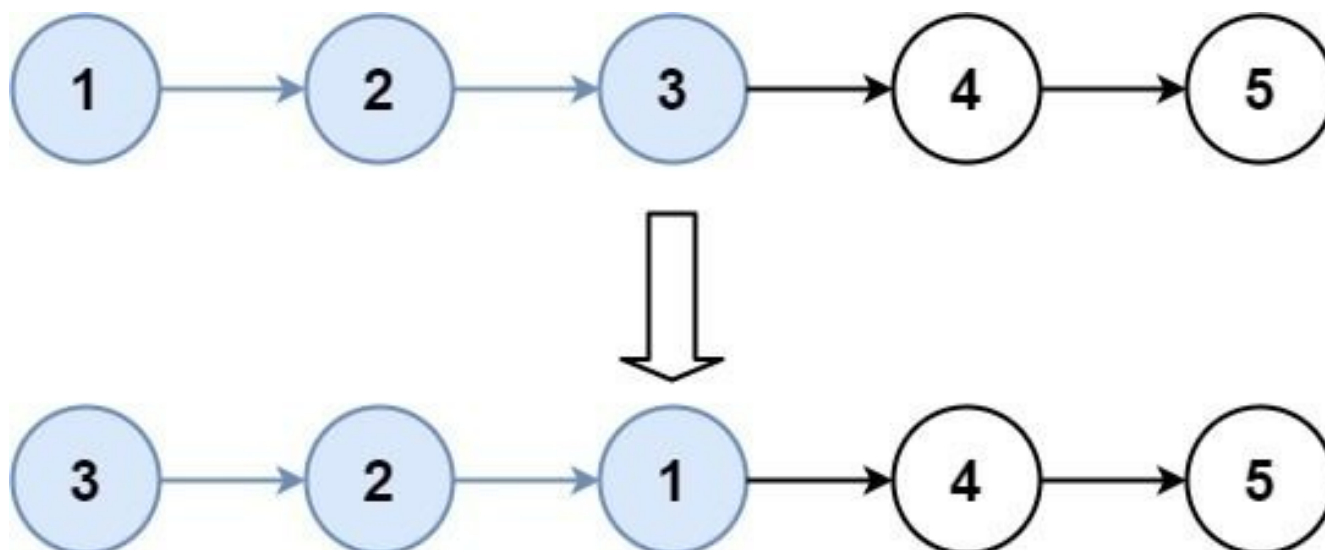
You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:



Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]

Example 2:



Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]

Constraints:

- The number of nodes in the list is n .
- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$

Follow-up: Can you solve the problem in $O(1)$ extra memory space?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Reverse every k consecutive nodes in the linked list. If remaining nodes < k,
leave them as-is.
# Return the new head. Right?"
#
# "head=[1,2,3,4,5], k=2: [2,1,4,3,5] ✓
# head=[1,2,3,4,5], k=3: [3,2,1,4,5] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Collect all node values into an array. Reverse each group of k elements in place.
# Rebuild the linked list from the modified array.
# Time:  $O(n)$ . Space:  $O(n)$  for the array."
```

```
class _25_ReverseNodesInKGroup_Brute:
    def reverseKGroup(self, head, k: int):
        vals = []
        curr = head
        while curr:
            vals.append(curr.val); curr = curr.next
        for i in range(0, len(vals) - (len(vals) % k), k):
            vals[i:i+k] = vals[i:i+k][::-1]
        dummy = cur = ListNode(0)
        for v in vals:
            cur.next = ListNode(v); cur = cur.next
        return dummy.next
```

PHASE 3 — OPTIMIZE

```
# "In-place pointer reversal group by group —  $O(1)$  space.
# For each group: check if k nodes remain (if not, stop).
# Reverse k nodes in-place using the 3-pointer technique.
# Link the previous group's tail to the new group head, and current group's tail to
the next group.
# Use a dummy node to simplify relinking the first group.
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```

class _25_ReverseNodesInKGroup:
    def reverseKGroup(self, head, k: int):
        def has_k_nodes(node, k):
            for _ in range(k):
                if not node: return False
                node = node.next
            return True

        def reverse_k(node, k):
            prev, curr = None, node
            for _ in range(k):
                nxt = curr.next
                curr.next = prev
                prev = curr
                curr = nxt
            return prev, node # new_head, new_tail

        dummy = ListNode(0, head)
        group_prev = dummy
        while has_k_nodes(group_prev.next, k):
            group_start = group_prev.next
            new_head, new_tail = reverse_k(group_start, k)
            group_prev.next = new_head
            new_tail.next = group_start # group_start is now the tail after reversal
            # Wait – after reversal group_start.next points somewhere wrong, fix:
            # Actually reverse_k leaves curr (the node after the group) in
            new_tail.next
            # We need: new_tail.next = curr (handled by reverse_k returning
            new_tail=old head)
            # group_start after reversal is the tail; its next should point to next
            group start
            group_prev = new_tail
        return dummy.next

# PHASE 5 — TEST & DEBUG
# [1,2,3,4,5], k=2:
# Group1: has_k? 1,2 ✓. reverse(1→2): prev=2→1, curr=3. new_head=2,new_tail=1.
# dummy.next=2. 1.next=3. group_prev=1. List: dummy→2→1→3→4→5.
# Group2: has_k? 3,4 ✓. reverse(3→4): new_head=4,new_tail=3.
# 1.next=4. 3.next=5. group_prev=3. List: dummy→2→1→4→3→5.
# Group3: has_k? only 5 → stop. → [2,1,4,3,5] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each node reversed exactly once. Space O(1): only pointer variables.
# The dummy node is critical to handle relinking the first group.
# Follow-up: Swap Nodes in Pairs (LC 24) – special case with k=2.
# Follow-up: reverse every other group of k – add a skip_k step between reverse
steps."

```

◆ Quick Review

Key Insights

- Use a dummy node to simplify edge cases.
- Identify groups of k nodes using a helper function.
- Reverse the nodes in each group using pointer manipulation.
- If a remaining group has fewer than k nodes, leave it unchanged.
- The reversal is done in-place with $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes (each node is processed a constant number of times)

Space Complexity: $O(1)$ (only constant extra space is used)

Solution

The solution involves iterating through the linked list by identifying groups of k consecutive nodes. For each group: Find the k th node from the current starting point. If it does not exist, the remainder of the list is not reversed. Reverse the nodes in the identified

group in-place by adjusting the next pointers. Reconnect the reversed group with the previous part of the list and update pointers to move to the next group. The approach leverages a dummy node for easier handling of head modifications and uses a helper function to fetch the kth node. The in-place reversal ensures that the extra memory usage is constant.

Linked List

□ #432 All O'one Data Structure

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Design · Doubly-Linked List ·
Linked List

Lists: Denny Zhang

Problem

Design a data structure to store the strings' count with the ability to return the strings with minimum and maximum counts.

Implement the AllOne class:

- AllOne() Initializes the object of the data structure.
- inc(String key) Increments the count of the string key by 1. If key does not exist in the data structure, insert it with count 1.
- dec(String key) Decrements the count of the string key by 1. If the count of key is 0 after the decrement, remove it from the data structure. It is guaranteed that key exists in the data structure before the decrement.

- **getMaxKey()** Returns one of the keys with the maximal count. If no element exists, return an empty string "".
- **getMinKey()** Returns one of the keys with the minimum count. If no element exists, return an empty string "".

Note that each function must run in $O(1)$ average time complexity.

Example 1:

Input

```
["AllOne", "inc", "inc", "getMaxKey", "getMinKey", "inc", "getMaxKey", "getMinKey"]
```

```
[[], ["hello"], ["hello"], [], [], ["leet"], [], []]
```

Output

```
[null, null, null, "hello", "hello", null, "hello", "leet"]
```

Explanation

```
AllOne allOne = new AllOne();
```

Constraints:

- $1 \leq \text{key.length} \leq 10$
- key consists of lowercase English letters.
- It is guaranteed that for each call to dec, key is existing in the data structure.
- At most $5 * 10^4$ calls will be made to inc, dec, getMaxKey, and getMinKey.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design a data structure supporting: inc(key), dec(key), getMaxKey()→any key with
max count,
# getMaxKey()→any key with min count. All operations O(1). Right?"
#
# "inc('a'), inc('b'), inc('b'), inc('c'), inc('c'), inc('c')."
# getMaxKey()→'c'. getMinKey()→'a'. dec('b'), dec('b'). getMinKey()→'b' or 'a'. ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Use a dict {key: count}. For getMaxKey/getMinKey, scan all entries.
# Time: O(1) for inc/dec, O(n) for getMax/getMin. Space: O(n)."
```

```
class _432_All0one_Brute:
    def __init__(self):
        self.counts = {}

    def inc(self, key: str) -> None:
        self.counts[key] = self.counts.get(key, 0) + 1

    def dec(self, key: str) -> None:
        self.counts[key] -= 1
        if self.counts[key] == 0:
            del self.counts[key]

    def getMaxKey(self) -> str:
        return max(self.counts, key=self.counts.get) if self.counts else ''

    def getMinKey(self) -> str:
        return min(self.counts, key=self.counts.get) if self.counts else ''
```

PHASE 3 — OPTIMIZE

```
# "Doubly linked list of buckets (each bucket holds a count and a set of keys at
that count)
# + a hashmap {key -> bucket_node}.
# inc: move key from its current bucket to the next (count+1) bucket.
# dec: move key to the previous (count-1) bucket.
# getMax: return any key from the tail bucket. getMin: from the head bucket.
# All O(1) amortized. Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _432_All0one:
    class _Bucket:
        def __init__(self, count):
            self.count = count
            self.keys = set()
            self.prev = self.next = None

    def __init__(self):
        self.head = self._Bucket(0) # sentinel min (before all real buckets)
        self.tail = self._Bucket(0) # sentinel max
        self.head.next = self.tail
```

```

        self.tail.prev = self.head
        self.key_bucket = {} # key → bucket
def _insert_after(self, node, count):
    b = self._Bucket(count)
    b.prev, b.next = node, node.next
    node.next.prev = b
    node.next = b
    return b
def _remove(self, node):
    node.prev.next = node.next
    node.next.prev = node.prev
def inc(self, key: str) -> None:
    if key in self.key_bucket:
        cur = self.key_bucket[key]
        nxt_count = cur.count + 1
        if cur.next.count != nxt_count:
            nxt = self._insert_after(cur, nxt_count)
        else:
            nxt = cur.next
            nxt.keys.add(key)
            cur.keys.discard(key)
            if not cur.keys: self._remove(cur)
            self.key_bucket[key] = nxt
    else:
        if self.head.next.count != 1:
            b = self._insert_after(self.head, 1)
        else:
            b = self.head.next
            b.keys.add(key)
            self.key_bucket[key] = b
def dec(self, key: str) -> None:
    cur = self.key_bucket[key]
    if cur.count == 1:
        del self.key_bucket[key]
    else:
        prev_count = cur.count - 1
        if cur.prev.count != prev_count:
            prv = self._insert_after(cur.prev, prev_count)
        else:
            prv = cur.prev
            prv.keys.add(key)
            self.key_bucket[key] = prv
    cur.keys.discard(key)
    if not cur.keys: self._remove(cur)
def getMaxKey(self) -> str:
    return next(iter(self.tail.prev.keys)) if self.tail.prev is not self.head
else ''
def getMinKey(self) -> str:

```

```

        return next(iter(self.head.next.keys)) if self.head.next is not self.tail
    else: ''

# PHASE 5 — TEST & DEBUG
# inc('a'): bucket(1)={a}. head→1→tail. key_bucket={a:1}.
# inc('b'): bucket(1)={a,b}. inc('b'): bucket(2)={b}, bucket(1)={a}. head→1→2→tail.
# getMaxKey→'b'(tail.prev=bucket2). getMinKey→'a'(head.next=bucket1) ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1): set operations and linked list pointer updates.
# The DLL of buckets maintains sorted order by count implicitly.
# Follow-up: if we need getMedianKey, we'd need a balanced BST instead of a DLL.
# Follow-up: LFU Cache (LC 460) uses the exact same bucket-DLL structure."

```

◆ Quick Review

Key Insights

- Use a Doubly-Linked List (DLL) where each node holds a unique count value and a set of keys with that count.
- Maintain a Hash Map (keyToNode) to quickly locate the node corresponding to each key.
- Insert new nodes in the DLL when a key's count is incremented or decremented and the adjacent node does not have the target count.
- Remove nodes from the DLL when no keys remain in that count.
- The head of the DLL holds the minimum count and the tail holds the maximum count.

Space & Time

Time Complexity: $O(1)$ average for inc, dec, getMaxKey, and getMinKey.

Space Complexity: $O(n)$ where n is the number of unique keys in the data structure.

Solution

The solution uses a combination of a doubly-linked list and hash maps. Each node in the DLL represents a frequency and maintains a set of keys that have that frequency. A hash map maps keys to their corresponding node. When `inc(key)` or `dec(key)` is called, we update the key's frequency and move the key to the appropriate node in the DLL. If the adjacent node with the required frequency doesn't exist, we create a new node and insert it in the correct position. Removal of a key from a node and deletion of empty nodes ensures that `getMinKey()` and `getMaxKey()` can be returned directly from the head and tail of the DLL.

Linked List

□ #460 LFU Cache

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design ·
Doubly-Linked List
Lists: Denny Zhang

Problem

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the LFUCache class:

- **LFUCache(int capacity)** Initializes the object with the capacity of the data structure.
- **int get(int key)** Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- **void put(int key, int value)** Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie

(i.e., two or more keys with the same frequency), the least recently used key would be invalidated.

To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the put operation). The use counter for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get", "get"]
```

```
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, 3, null, -1, 3, 4]
```

Explanation

```
// cnt(x) = the use counter for key x
```

Constraints:

- $1 \leq \text{capacity} \leq 104$
- $0 \leq \text{key} \leq 105$
- $0 \leq \text{value} \leq 109$

- At most $2 * 10^5$ calls will be made to get and put.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design an LFU cache: get(key)→val or -1, put(key,val) – evict LFU (then LRU among ties).
# All O(1). Right?"
#
# "LFUCache(2): put(1,1),put(2,2),get(1)→1(freq=2).put(3,3) evicts key 2(freq=1,LRU).
# get(2)→-1, get(3)→3, put(4,4) evicts key 3(freq=1). get(1)→1,get(3)→-1,get(4)→4
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Dict for {key:(val,freq,last_used)}. On evict, scan all entries for min freq, then min last_used.
# Time: O(n) per put when eviction needed. Space: O(n)."
```

```
class _460_LFUCache_Brute:
    def __init__(self, capacity: int):
        self.cap = capacity
        self.data = {} # key → [val, freq, timestamp]
        self.time = 0

    def get(self, key: int) -> int:
        if key not in self.data: return -1
        self.data[key][1] += 1
        self.data[key][2] = self.time; self.time += 1
        return self.data[key][0]

    def put(self, key: int, value: int) -> None:
        if self.cap == 0: return
        if key in self.data:
            self.data[key][0] = value
            self.get(key); return
        if len(self.data) >= self.cap:
            evict = min(self.data, key=lambda k: (self.data[k][1], self.data[k][2]))
            del self.data[evict]
        self.data[key] = [value, 1, self.time]; self.time += 1
```

PHASE 3 — OPTIMIZE

"Three structures for O(1):

1. key_map: {key → (val, freq)}

2. freq_map: {freq → OrderedDict(key→None)} – LRU order within same freq

3. min_freq: current minimum frequency

get: update freq, move key to freq+1 bucket. put: evict from freq_map[min_freq]."

PHASE 4 — CLEAN CODE

```
class _460_LFUCache:
```

```
    def __init__(self, capacity: int):
```

```
        from collections import defaultdict, OrderedDict
```

```
        self.cap = capacity
```

```
        self.min_freq = 0
```

```
        self.key_map = {} # key → [val, freq]
```

```
        self.freq_map = defaultdict(OrderedDict) # freq → {key: None} (LRU order)
```

```
    def _update(self, key: int) -> None:
```

```
        val, freq = self.key_map[key]
```

```
        del self.freq_map[freq][key]
```

```
        if not self.freq_map[freq] and freq == self.min_freq:
```

```
            self.min_freq += 1
```

```
        self.key_map[key] = [val, freq + 1]
```

```
        self.freq_map[freq + 1][key] = None
```

```
    def get(self, key: int) -> int:
```

```
        if key not in self.key_map: return -1
```

```
        self._update(key)
```

```
        return self.key_map[key][0]
```

```
    def put(self, key: int, value: int) -> None:
```

```
        if self.cap == 0: return
```

```
        if key in self.key_map:
```

```
            self.key_map[key][0] = value
```

```
            self._update(key)
```

```
        else:
```

```
            if len(self.key_map) >= self.cap:
```

```
                evict, _ = self.freq_map[self.min_freq].popitem(last=False)
```

```
                del self.key_map[evict]
```

```
            self.key_map[key] = [value, 1]
```

```
            self.freq_map[1][key] = None
```

```
            self.min_freq = 1
```

PHASE 5 — TEST & DEBUG

```
# LFUCache(2): put(1,1): key_map={1:[1,1]},freq_map={1:{1}},min=1.
```

```
# put(2,2): key_map={1:[1,1],2:[2,1]},freq_map={1:{1,2}},min=1.
```

```
# get(1): update→freq_map={1:{2},2:{1}},min=1. return 1 ✓.
```

```
# put(3,3): evict from freq_map[1]→evict key2. key_map={1:[1,2],3:[3,1]},min=1.
```

```
# get(2)→-1 ✓. get(3)→3, update→freq=2, min=1. put(4,4): evict freq_map[1]→evict 3.
```

```
✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "All ops O(1): dict lookups + OrderedDict FIFO operations (popitem(last=False))."
```

```
# Space O(capacity): bounded by cache size.  
# min_freq only ever decreases on put (new key starts at freq=1) – so it's always  
correct.  
# Follow-up: LRU Cache (LC 146) – simpler, only recency matters, no frequency  
tracking.  
# Follow-up: combine LFU with TTL expiration – add expiry timestamps and a  
background cleaner."
```

◆ Quick Review

Key Insights

- Maintain a mapping from key to its value and frequency.
- Use an additional mapping from frequency to keys (maintaining the order of insertion or recent use) for quick eviction.
- Keep a global minFreq variable to track the lowest frequency present in the cache.
- When a key is accessed or updated, increase its frequency and relocate it in the frequency mapping.
- Evictions occur by removing the oldest key from the lowest frequency bucket.

Space & Time

Time Complexity: $O(1)$ average for both get and put.

Space Complexity: $O(\text{capacity})$, where capacity is the maximum number of keys stored.

Solution

The solution utilizes two main data structures: A hash map (keyToValFreq) that maps keys to a pair (value, frequency). A second hash map (freqToKeys) mapping frequencies to an ordered list/set of keys. This data structure preserves the order in which keys were added to allow removal of the least recently used key when multiple keys share the same frequency. A global minFreq variable is maintained to quickly identify the bucket with the least frequency. When a key is accessed or updated, the key is removed from its current frequency list and placed in the next higher frequency list. If the frequency list for the minimal frequency becomes empty, minFreq is updated. This design guarantees that both get and put operations execute in constant time on average.

Stack

Round 6 · Mid · Hard · 8 questions

Stack

□ #32 Longest Valid Parentheses

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Stack
Lists: Grind 169

Problem

Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring.

Example 1:

Input: s = "()"
Output: 2
Explanation: The longest valid parentheses substring is "()".

Example 2:

Input: s = ")()())"
Output: 4
Explanation: The longest valid parentheses substring is "()()".

Example 3:

Input: s = ""
Output: 0

Constraints:

- $0 \leq s.length \leq 3 \times 10^4$
- s[i] is '(', or ')'.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a string of '(' and ')', return the length of the longest valid
(well-formed)
# parentheses substring. Right?"
#
# s='()': longest valid is '()' → length 2 ✓
# s='()()()': '()()' → length 4 ✓
# s='': → 0 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Try all substrings, check each for validity, track the longest.
# Time:  $O(n^3)$  –  $O(n^2)$  substrings,  $O(n)$  to validate each. Space:  $O(n)$ ."
class _32_LongestValidParentheses_Brute:
    def longestValidParentheses(self, s: str) -> int:
        def is_valid(sub):
            bal = 0
            for ch in sub:
                bal += 1 if ch == '(' else -1
                if bal < 0: return False
            return bal == 0
        best = 0
        for i in range(len(s)):
            for j in range(i+2, len(s)+1, 2): # only even lengths can be valid
                if is_valid(s[i:j]):
                    best = max(best, j - i)
        return best
```

PHASE 3 — OPTIMIZE

```
# "Stack storing indices. Push index of '(' onto stack.
# On ')': if stack top is '(' index, pop it (matched pair). Else push ')' index as
new boundary.
# Initialize stack with [-1] as a base boundary.
# After each match, length = current index – stack top (index of last unmatched
char).
# Time:  $O(n)$ . Space:  $O(n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _32_LongestValidParentheses:
    def longestValidParentheses(self, s: str) -> int:
        stack = [-1] # base boundary
```

```

best = 0
for i, ch in enumerate(s):
    if ch == '(':
        stack.append(i)
    else:
        stack.pop()
        if not stack:
            stack.append(i) # new boundary: last unmatched ')'
        else:
            best = max(best, i - stack[-1])
return best

```

PHASE 5 — TEST & DEBUG

```

# s='')()())':
# i=0')': pop -1 → stack empty → push 0. stack=[0].
# i=1'(': push 1. [0,1].
# i=2')': pop 1. stack=[0]. best=max(0,2-0)=2.
# i=3'(': push 3. [0,3].
# i=4')': pop 3. stack=[0]. best=max(2,4-0)=4.
# i=5')': pop 0 → stack empty → push 5. best=4 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): one pass. Space O(n): stack holds at most n indices.
# DP alternative: dp[i] = length of valid substring ending at i.
# dp[i] = dp[i-2]+2 if s[i-dp[i-1]-1]=='(' - O(n) time, O(n) space.
# O(1) space: two-pass counting (left-to-right then right-to-left with open/close
counters).
# Follow-up: return the actual substring, not just length - track start index in
stack."

```

◆ Quick Review

Key Insights

- Use a stack to keep track of indices where potential valid substrings begin.
- Start by pushing a dummy index (-1) to handle edge cases.
- For every '(', push its index onto the stack.
- For every ')', pop from the stack. If the stack becomes empty, push the current index as a

new base. Otherwise, compute the length of the valid substring using the current index minus the top index of the stack.

- An alternative is the dynamic programming approach where $dp[i]$ represents the length of the longest valid substring ending at index i .

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.

Space Complexity: $O(n)$ for the stack data structure.

Solution

The stack-based solution works by maintaining indices for unmatched parentheses. Initially, a dummy index (-1) is pushed onto the stack to serve as the base for calculating subsequent valid substring lengths. As we iterate through the string: If the character is '(', we push its index onto the stack. If the character is ')', we pop from the stack. If the stack is empty afterwards, that means there is no valid starting position, so we push the current index to reset the base. Otherwise, we calculate the length of the

current valid substring by subtracting the current index with the new top index of the stack and update the maximum length accordingly.

Stack

□ #42 Trapping Rain Water

HAR

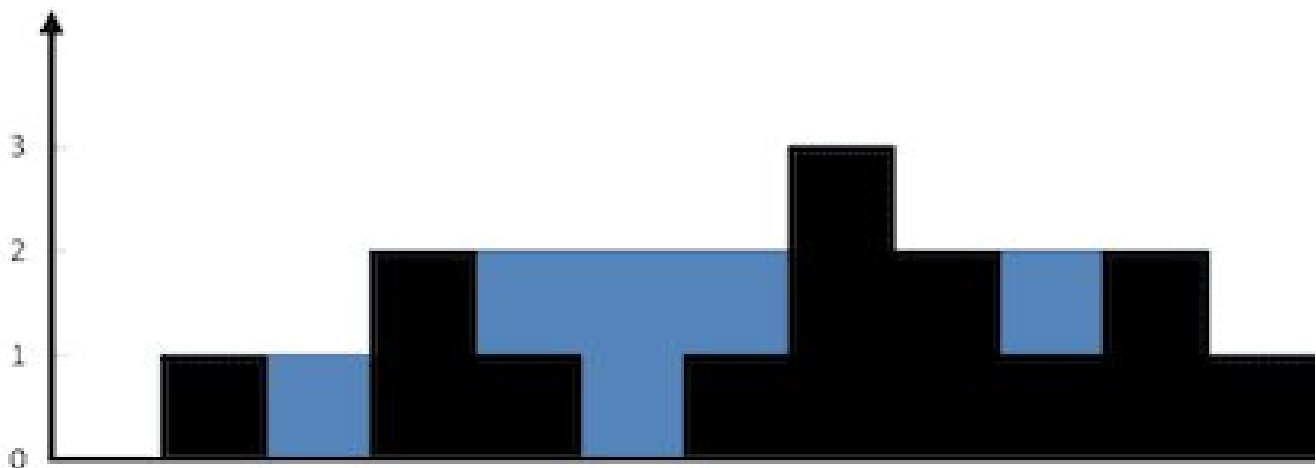
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Dynamic Programming ·
Stack · Monotonic Stack

Lists: Grind 169

Problem

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9

Constraints:

- $n == \text{height.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given an elevation map, compute how much rain water can be trapped after raining.
# Water is trapped above a bar if both sides have taller bars. Right?"
#
# "height=[0,1,0,2,1,0,1,3,1,0,1,2]:
# Water trapped at positions 2,4,5,6,9,10. Total=6 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "For each bar, water above it = min(max_left, max_right) - height[i].
# Precompute max_left and max_right arrays.
# Time: O(n). Space: O(n)."
```

```
class _42_TrappingRainWater_Brute:
    def trap(self, height: list[int]) -> int:
        n = len(height)
        max_left = [0] * n
        max_right = [0] * n
        for i in range(1, n):
            max_left[i] = max(max_left[i-1], height[i-1])
        for i in range(n-2, -1, -1):
```

```

        max_right[i] = max(max_right[i+1], height[i+1])
    return sum(max(0, min(max_left[i], max_right[i]) - height[i]) for i in
range(n))

```

PHASE 3 — OPTIMIZE

"Two-pointer approach – $O(1)$ space.

Maintain left and right pointers, left_max and right_max.

If height[left] <= height[right]: water at left = left_max - height[left] (right side guaranteed taller).

Move the shorter side inward.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```

class _42_TrappingRainWater:
    def trap(self, height: list[int]) -> int:
        left, right = 0, len(height) - 1
        left_max = right_max = 0
        water = 0
        while left < right:
            if height[left] <= height[right]:
                if height[left] >= left_max:
                    left_max = height[left]
                else:
                    water += left_max - height[left]
                    left += 1
            else:
                if height[right] >= right_max:
                    right_max = height[right]
                else:
                    water += right_max - height[right]
                    right -= 1
        return water

```

PHASE 5 — TEST & DEBUG

height=[0,1,0,2,1,0,1,3,1,0,1,2]:

l=0,r=11,lm=0,rm=0. h[0]=0<=h[11]=2: lm=0,water+=0-0=0. l=1.

h[1]=1<=h[11]=2: lm=1. l=2. h[2]=0<lm=1: water+=1. l=3.

h[3]=2<=h[11]=2: lm=2. l=4. h[4]=1<lm=2: water+=1. ... total=6 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass. Space $O(1)$: only four variables.

Monotonic stack: $O(n)$ time, $O(n)$ space – solves it differently (horizontal layers).

Two-pointer is $O(1)$ space and arguably clearest approach.

Follow-up: Trapping Rain Water II (LC 407) – 3D extension, use a min-heap of border cells.

Follow-up: if heights can change (updates), use segment tree for range max queries."

◆ Quick Review

Key Insights

- The water trapped at any index is determined by the minimum of the maximum height on its left and right minus the height at that index.
- A two-pointers approach can efficiently solve the problem in one pass by maintaining left and right boundaries.
- Alternative approaches include dynamic programming with pre-computed left max and right max arrays, or using a monotonic stack to determine boundaries.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$ when using the two-pointer approach

Solution

We use a two-pointers strategy by initializing pointers at the beginning and the end of the array. Keep track of the maximum height seen so far from the left (`left_max`) and the right (`right_max`). At each step, move the pointer

with the lesser height because the water trapped is determined by the shorter boundary. Compute the water at that position as the difference between the boundary (`left_max` or `right_max`) and the current height, accumulating it into the total. This approach ensures linear time scanning with constant extra space.

Stack

□ #84 Largest Rectangle in Histogram

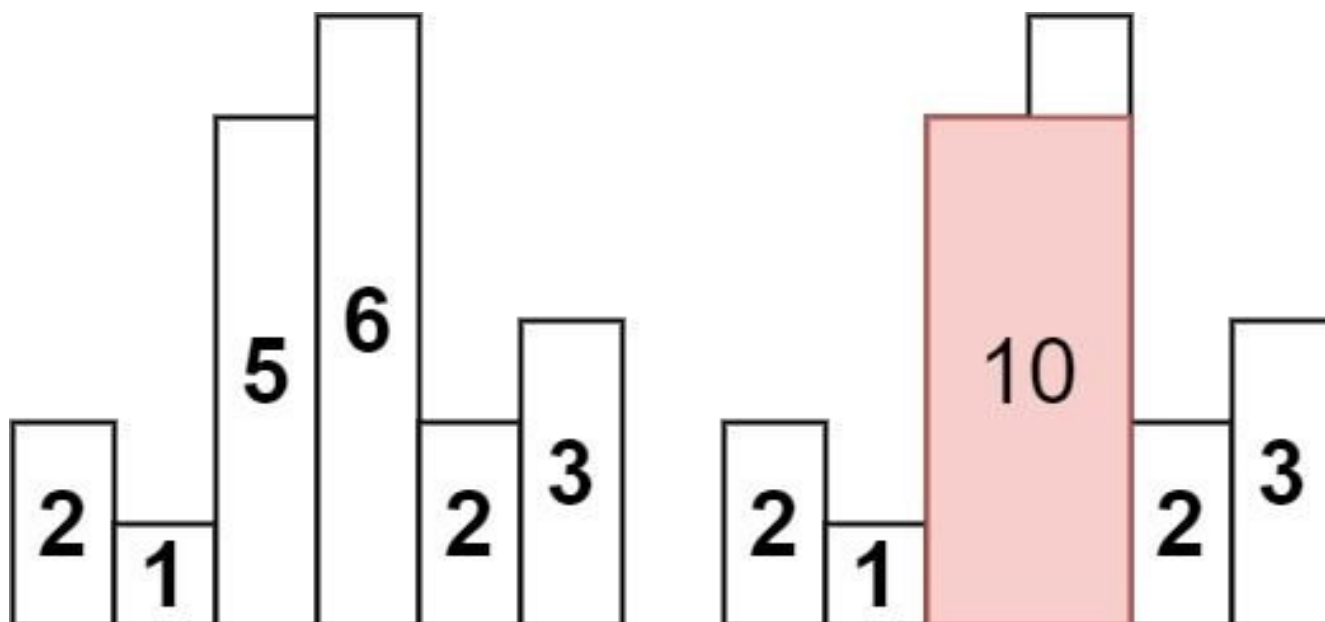
HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: Grind 169

Problem

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Example 1:

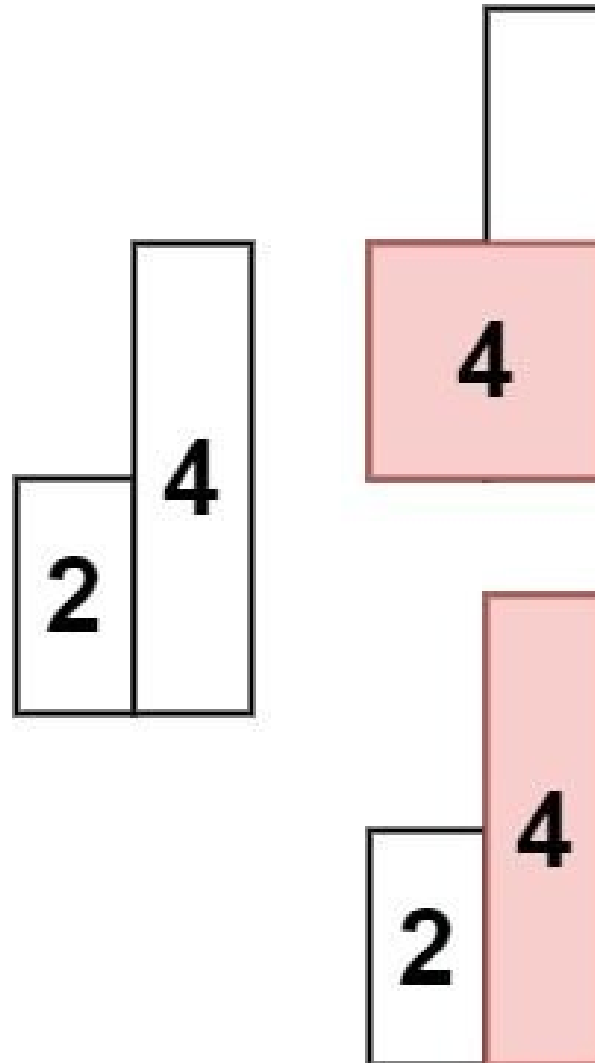


Input: heights = [2,1,5,6,2,3]

Output: 10

Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



Input: heights = [2,4]

Output: 4

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $0 \leq \text{heights}[i] \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given an array of bar heights in a histogram, find the area of the largest rectangle

that fits entirely within the histogram. Right?"

#

"heights=[2,1,5,6,2,3]:

Largest rectangle: height=5, width=2 (bars 2,3) → area=10 ✓

Or height=2, width=5 → area=10. height=3,width=2=6. max=10 ✓"

PHASE 2 — BRUTE FORCE

"For each pair (i,j), compute area = (j-i+1) * min(heights[i..j]).

Time: $O(n^2)$ with $O(1)$ min tracking per expansion. Space: $O(1)$."

```
class _84_LargestRectangleHistogram_Brute:
    def largestRectangleArea(self, heights: list[int]) -> int:
        n = len(heights)
        best = 0
        for i in range(n):
            min_h = heights[i]
            for j in range(i, n):
                min_h = min(min_h, heights[j])
                best = max(best, min_h * (j - i + 1))
        return best
```

PHASE 3 — OPTIMIZE

"Monotonic increasing stack of indices.

For each bar: while stack top is taller than current bar, it can no longer extend rightward.

Pop it and compute its area: height = heights[popped], width = current_idx - stack_new_top - 1.

Sentinel 0s at both ends simplify boundary handling.

Time: $O(n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _84_LargestRectangleHistogram:
    def largestRectangleArea(self, heights: list[int]) -> int:
        heights = [0] + heights + [0] # sentinel values
        stack = [0] # indices
        best = 0
        for i in range(1, len(heights)):
            while heights[i] < heights[stack[-1]]:
                h = heights[stack.pop()]
```

```

        w = i - stack[-1] - 1
        best = max(best, h * w)
    stack.append(i)
    return best

```

PHASE 5 — TEST & DEBUG

```

# heights=[0,2,1,5,6,2,3,0] (with sentinels):
# i=1(2): stack=[0,1]. i=2(1): 1<2 → pop1: h=2,w=2-0-1=1,area=2. push2.
stack=[0,2].
# i=3(5): push. i=4(6): push. i=5(2): 2<6 → pop4: h=6,w=5-3-1=1,area=6.
# 2<5 → pop3: h=5,w=5-2-1=2,area=10 ✓. 2>=1 stop. push5.
# i=6(3): push. i=7(0): pop6:h=3,w=7-5-1=1,area=3. pop5:h=2,w=7-2-1=4,area=8.
pop2:h=1,w=7-0-1=6,area=6. best=10 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): each bar pushed and popped exactly once.
# Space O(n): stack holds at most n indices.
# The sentinel 0s ensure every bar gets popped and processed – no leftovers in
stack.
# Follow-up: Maximal Rectangle (LC 85) – for each row, build histogram heights,
apply this.
# Follow-up: Largest Rectangle of 1s in binary matrix – same reduction to
histogram."

```

◆ Quick Review

Key Insights

- Each bar can potentially extend left and right as long as the neighboring bars are not shorter.
- A monotonic (increasing) stack can be used to efficiently track indices of bars.
- When encountering a bar lower than the one on the top of the stack, pop the stack and calculate the area of the rectangle formed with the popped bar height.

- A dummy bar (of height 0) is appended at the end to ensure all bars are processed.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution employs a monotonic stack to traverse the histogram once. For each bar, while the current bar's height is less than the height at the top index of the stack, it pops from the stack and calculates the area using the popped height as the smallest bar. The width for the rectangle is determined by the difference between the current index and the index from the stack (or the current index if the stack is empty). This effectively finds the maximum rectangular area possible. A sentinel value (0) is added at the end of the histogram to flush any remaining bars from the stack.

Stack

□ #224 Basic Calculator

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Stack · Recursion
Lists: Grind 169

Problem

Given a string s representing a valid expression, implement a basic calculator to evaluate it, and return the result of the evaluation.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: $s = "1 + 1"$
Output: 2

Example 2:

Input: $s = "2-1 + 2 "$
Output: 3

Example 3:

Input: $s = "(1+(4+5+2)-3)+(6+8)"$
Output: 23

Constraints:

- $1 \leq s.length \leq 3 * 10^5$
- s consists of digits, '+', '-', '(', ')', and ' '.
- s represents a valid expression.
- '+' is not used as a unary operation (i.e., "+1" and "+(2 + 3)" is invalid).
- '-' could be used as a unary operation (i.e., "-1" and "-(2 + 3)" is valid).
- There will be no two consecutive operators in the input.
- Every number and running calculation will fit in a signed 32-bit integer.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Evaluate a string expression with +, -, and parentheses. No * or /.
# Non-negative integers, spaces allowed. Right?"
#
# "s='1 + 1' → 2 ✓
# s='(1+(4+5+2)-3)+(6+8)' → 23 ✓
# s='- (3 + (4 + 5))' → -12 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Recursive descent parser: handle parentheses recursively.
# Parse a number or a sub-expression in parentheses, apply the sign.
# Time: O(n). Space: O(d) where d = nesting depth."
```

```
class _224_BasicCalculator_Brute:
    def calculate(self, s: str) -> int:
        self._idx = 0
        def parse():
```

```

        result, sign, num = 0, 1, 0
    while self._idx < len(s):
        ch = s[self._idx]; self._idx += 1
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch in '+-':
            result += sign * num
            num = 0
            sign = 1 if ch == '+' else -1
        elif ch == '(':
            num = parse()
        elif ch == ')':
            break
        elif ch == ' ':
            pass
    return result + sign * num
return parse()

```

PHASE 3 — OPTIMIZE

"Iterative stack: push (result, sign) when '(' is encountered, pop and merge on ')'.
 # Maintain current number being built, current result, and sign.
 # Time: O(n). Space: O(d) for the stack."

PHASE 4 — CLEAN CODE

```

class _224_BasicCalculator:
    def calculate(self, s: str) -> int:
        stack = []
        result = 0
        sign = 1
        num = 0
        for ch in s:
            if ch.isdigit():
                num = num * 10 + int(ch)
            elif ch in '+-':
                result += sign * num
                num = 0
                sign = 1 if ch == '+' else -1
            elif ch == '(':
                stack.append(result)
                stack.append(sign)
                result, sign = 0, 1
            elif ch == ')':
                result += sign * num
                num = 0
                result *= stack.pop() # sign before '('
                result += stack.pop() # result before '('
        return result + sign * num

```

PHASE 5 — TEST & DEBUG

```
# s='(1+(4+5+2)-3)+(6+8)':
# '(': push result=0, sign=1. reset result=0, sign=1.
# '1': num=1. '+': result=1, num=0, sign=1. '(': push 1,1. reset.
# '4+5+2': result=11. ')': result=11*1+1=12. '-': result=12,sign=-1.
# '3': num=3. ')': result=12+(-1)*3=9. result=9*1+0=9.
# '+': result=9, sign=1. '(': push 9,1. '6+8': result=14. ')': 14*1+9=23 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass. Space O(d): stack depth = nesting depth.
# Multi-digit numbers handled by num = num*10 + digit.
# Negative signs attach to the next number or sub-expression.
# Follow-up: Basic Calculator II (LC 227) – add * and /; use precedence handling.
# Follow-up: Basic Calculator III (LC 772) – all four operators with parentheses."
```

◆ Quick Review

Key Insights

- Use a stack to save previous results and signs when encountering parentheses.
- Process the string left-to-right by accumulating multi-digit numbers and applying the current sign.
- When encountering a '(', push the current result and sign onto the stack and reset them.
- When encountering a ')', complete the current sub-expression then pop the previous state to incorporate the computed value.
- Handle whitespaces by simply skipping them.

Space & Time

Time Complexity: $O(n)$ where n is the length of the input string, since we process each character once.

Space Complexity: $O(n)$ in the worst-case scenario due to the use of a stack for nested parentheses.

Solution

The solution uses an iterative approach with a stack: Initialize variables: result to keep the running total, sign to handle addition/subtraction, and index to iterate over the string. As you iterate, build multi-digit numbers by checking for consecutive numerical characters. Adjust the total by adding the product of the current number and its sign. When encountering a '(', push the current result and sign onto the stack, then reset results and sign for the new sub-expression. When encountering a ')', complete the sub-expression by popping the previous result and sign, then combine. Skip over spaces. The final result is the computed value after the end of the string. This approach effectively simulates recursion and correctly handles nested expressions and

negation.

Stack

□ #239 Sliding Window Maximum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Queue · Sliding Window · Monotonic
 Queue

Lists: Grind 169

Problem

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position Max
-----
[1 3 -1] -3 5 3 6 7 3
1 [3 -1 -3] 5 3 6 7 3
1 3 [-1 -3 5] 3 6 7 5
```

Example 2:

Input: nums = [1], k = 1
Output: [1]

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- $1 \leq k \leq \text{nums.length}$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given an array and window size k, return the maximum value in each sliding window.

Window slides one step right each time. Right?"

#

"nums=[1,3,-1,-3,5,3,6,7], k=3:

[1,3,-1]→3, [3,-1,-3]→3, [-1,-3,5]→5, [-3,5,3]→5, [5,3,6]→6, [3,6,7]→7 → [3,3,5,5,6,7] ✓"

PHASE 2 — BRUTE FORCE

"Slide window, take max at each position.

Time: $O(n \cdot k)$. Space: $O(1)$ extra."

```
class _239_SlidingWindowMaximum_Brute:
```

```
    def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
        return [max(nums[i:i+k]) for i in range(len(nums)-k+1)]
```

PHASE 3 — OPTIMIZE

"Monotonic deque (double-ended queue) of indices – maintains candidates for the window max.

Invariant: deque stores indices in decreasing order of their values.

For each element: remove from back any index with smaller value (they can never be max).

Remove from front any index that's out of the window.

Front of deque is always the current window maximum.

Time: $O(n)$. Space: $O(k)$."

PHASE 4 — CLEAN CODE

```
class _239_SlidingWindowMaximum:
```

```
def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
    from collections import deque
    dq = deque() # indices, front = max
    result = []
    for i, num in enumerate(nums):
        # Remove smaller elements from back
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        # Remove out-of-window element from front
        if dq[0] < i - k + 1:
            dq.popleft()
        # Record max once first window is complete
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,3,-1,-3,5,3,6,7], k=3:
# i=0(1): dq=[0]. i=1(3): pop0(1<3), dq=[1]. i=2(-1): dq=[1,2]. window[1,3,-1]:
ans=nums[1]=3 ✓
# i=3(-3): dq=[1,2,3]. front=1>=1 ✓. ans=nums[1]=3 ✓
# i=4(5): pop3(-3<5),pop2(-1<5),pop1(3<5). dq=[4]. front=4>=2 ✓. ans=5 ✓
# i=5(3): dq=[4,5]. ans=nums[4]=5 ✓. i=6(6): pop5(3<6),pop4(5<6). dq=[6]. ans=6 ✓
# i=7(7): pop6(6<7). dq=[7]. ans=7 ✓. Result=[3,3,5,5,6,7] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): each element enqueued and dequeued at most once.
# Space O(k): deque holds at most k indices.
# Brute O(n*k) is simple but TLEs for large inputs.
# Follow-up: Sliding Window Minimum – same deque, flip comparison to keep increasing order.
# Follow-up: Max of all subarrays of size k in a circular array – extend with modular indexing."
```

◆ Quick Review

Key Insights

- Use a deque (double-ended queue) to maintain the indices of potential maximum elements in a decreasing order.

- Ensure that the deque always has the current window's indices by removing elements that fall outside the current window.
- Only add elements that are greater than the ones at the back of the deque, thus preserving a monotonic decreasing order.
- The maximum for the current window is at the front of the deque.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in `nums`, since each element is processed at most twice.

Space Complexity: $O(k)$, as the deque stores indices for at most k elements at any time.

Solution

The solution uses a monotonic deque to maintain indices of potential maximum elements in the current window. As the window slides, we: Remove indices from the front if they are out of the window. Remove indices from the back if the current element is larger, because they cannot be the maximum if the current element is in the window. Append the current index. After the first k elements,

the element at the front of the deque is the maximum for that window. This approach ensures that each element is pushed and popped from the deque only once, achieving an overall $O(n)$ time complexity.

Stack

□ #716 Max Stack

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design · Doubly-Linked List · Ordered
Set

Lists: Denny Zhang

Problem

Design a max stack data structure that supports the stack operations and supports finding the stack's maximum element.

Implement the MaxStack class:

- **MaxStack()** Initializes the stack object.
- **void push(int x)** Pushes element x onto the stack.
- **int pop()** Removes the element on top of the stack and returns it.
- **int top()** Gets the element on the top of the stack without removing it.
- **int peekMax()** Retrieves the maximum element in the stack without removing it.

- **int popMax()** Retrieves the maximum element in the stack and removes it. If there is more than one maximum element, only remove the top-most one.

You must come up with a solution that supports $O(1)$ for each top call and $O(\log n)$ for each other call.

Example 1:

```
Input
["MaxStack", "push", "push", "push", "top", "popMax", "top", "peekMax", "pop",
"top"]
[[], [5], [1], [5], [], [], [], [], [], []]
Output
[null, null, null, null, 5, 5, 1, 5, 1, 5]
```

Explanation

```
MaxStack stk = new MaxStack();
```

Constraints:

- $-107 \leq x \leq 107$
- At most 105 calls will be made to push, pop, top, peekMax, and popMax.
- There will be at least one element in the stack when pop, top, peekMax, or popMax is called.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Design a Max Stack supporting push, pop, top, peekMax, popMax — all $O(\log n)$.
 # peekMax: return max without removing. popMax: remove and return max element.
 Right?"

#

"push(5), push(1), push(5): top=5, popMax=5, top=1, peekMax=5, pop=1, top=5 ✓"

PHASE 2 — BRUTE FORCE

"Regular stack + scan for popMax."

push/pop/top: $O(1)$. peekMax: $O(n)$ scan. popMax: $O(n)$ scan + $O(n)$ rebuild.

Space: $O(n)$."

```
class _716_MaxStack_Brute:
    def __init__(self):
        self.stack = []

    def push(self, x: int) -> None:
        self.stack.append(x)

    def pop(self) -> int:
        return self.stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def peekMax(self) -> int:
        return max(self.stack)

    def popMax(self) -> int:
        m = max(self.stack)
        tmp = []
        while self.stack[-1] != m:
            tmp.append(self.stack.pop())
        self.stack.pop()
        while tmp:
            self.stack.append(tmp.pop())
        return m
```

PHASE 3 — OPTIMIZE

"Doubly linked list (DLL) for the stack + sorted set (treemap) for $O(\log n)$ max access.

DLL preserves insertion order; sorted set tracks (val, node_id) for $O(\log n)$ find/remove.

push: add to DLL tail + sorted set. pop: remove DLL tail + sorted set entry.

popMax: find max in sorted set, remove from sorted set + DLL.

All ops $O(\log n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _716_MaxStack:
    def __init__(self):
        from sortedcontainers import SortedList
        self.stack = [] # list of (val, uid)
```

```

        self.sorted = SortedList() # (val, uid)
        self.uid = 0
        self.removed = set()
    def push(self, x: int) -> None:
        self.stack.append((x, self.uid))
        self.sorted.add((x, self.uid))
        self.uid += 1
    def _clean(self):
        while self.stack and self.stack[-1][1] in self.removed:
            self.stack.pop()
    def pop(self) -> int:
        self._clean()
        val, uid = self.stack.pop()
        self.removed.add(uid)
        return val
    def top(self) -> int:
        self._clean()
        return self.stack[-1][0]
    def peekMax(self) -> int:
        while self.sorted[-1][1] in self.removed:
            self.sorted.pop()
        return self.sorted[-1][0]
    def popMax(self) -> int:
        while self.sorted[-1][1] in self.removed:
            self.sorted.pop()
        val, uid = self.sorted.pop()
        self.removed.add(uid)
        return val

```

PHASE 5 — TEST & DEBUG

```

# push(5,uid=0),push(1,uid=1),push(5,uid=2). sorted=[(1,1),(5,0),(5,2)].
# top()->stack[-1]=(5,2)->5 ✓. popMax()->sorted[-1]=(5,2)->remove uid2. return 5 ✓.
# top()->clean stack: (5,2) removed->skip->(1,1). top=1 ✓. peekMax->(5,0)->5 ✓.
# pop()->(1,1) removed. return 1 ✓. top()->(5,0)->5 ✓.

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "All ops O(log n) with sorted set (SortedList or TreeMap).
# Space O(n): lazy deletion avoids O(n) rebuilds.
# Follow-up: if only O(n) time needed, brute is simpler and acceptable.
# Follow-up: Min Stack (LC 155) - O(1) with parallel min-stack, no sorted set
needed."

```

◆ Quick Review

Key Insights

- Use a doubly-linked list to support fast insertion and removal from the stack.
- Maintain a balanced tree (or ordered dictionary/multiset) to quickly locate the maximum element in $O(\log n)$ time.
- Map each value to its corresponding node(s) in the doubly-linked list, so that when `popMax` is called, you can immediately remove the corresponding node from the list.
- The double-linking allows removal of arbitrary nodes in $O(1)$, once you have the reference.

Space & Time

Time Complexity:

`push`: $O(\log n)$ due to insertion in the ordered structure.

`pop`: $O(1)$ for removal from the doubly-linked list, plus $O(\log n)$ for updating the tree.

`top`: $O(1)$

`peekMax`: $O(1)$ or $O(\log n)$ depending on the tree implementation.

`popMax`: $O(\log n)$ for looking up and removal from the tree, $O(1)$ for doubly-linked list deletion.

Space Complexity:

$O(n)$ for storing all nodes in the doubly-linked list and the tree structure mapping values to nodes.

Solution

We use two data structures: A doubly-linked list to store the elements in stack order. This supports fast push and pop operations and allows $O(1)$ removal when given a reference. An ordered structure (like a balanced BST, TreeMap in Java, or SortedList in Python) that maps each value to a list of nodes that hold that value in the linked list. This allows us to quickly determine the maximum element. When there are duplicates, we remove the one closest to the top by storing nodes in order. The trick is to keep these two structures synchronized. When an element is pushed, add its node to both the doubly-linked list and the ordered structure. When an element is removed (either via pop or popMax), remove its node from both structures.

Stack

□ ★ #772 Basic Calculator III ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Stack · Recursion

Lists: ★ Premium | Premium 98

Problem

Implement a basic calculator to evaluate a simple expression string.

The expression string contains only non-negative integers, '+', '-', '*', '/' operators, and open '(' and closing parentheses ')'. The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

```
Input: s = "1+1"  
Output: 2
```

Example 2:

Input: s = "6-4/2"
Output: 4

Example 3:

Input: s = "2*(5+5*2)/3+(6/2+8)"
Output: 21

Constraints:

- $1 \leq s \leq 104$
- s consists of digits, '+', '-', '*', '/', '(', and ')'.
Note: s will not contain any spaces.
- s is a valid expression.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Evaluate an expression with +, -, *, /, and parentheses. Non-negative integers.
# Division truncates toward zero. Right?"
#
# "s='1+1+1' → 3 ✓
# s='6-4/2' → 4 ✓
# s='2*(5+5*2)/3+(6/2+8)' → 21 ✓
# s='(2+6*3+5-(3*14/7+2)*5)+3' → -12 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Recursive descent parser: parse expressions, respecting operator precedence and
parentheses.
# Time: O(n). Space: O(d) recursion depth."
```

```
class _772_BasicCalculatorIII_Brute:
    def calculate(self, s: str) -> int:
        s = s.replace(' ', '')
        idx = [0]
        def parse():
            result, sign, num = 0, '+', 0
            while idx[0] < len(s):
                ch = s[idx[0]]; idx[0] += 1
                if ch.isdigit():
```

```

        num = num * 10 + int(ch)
    if ch == '(':
        num = parse()
    if not ch.isdigit() and ch != '(' or idx[0] == len(s):
        if sign == '+': result += num
        elif sign == '-': result -= num
        elif sign == '*': result = result * num # simplified
        elif sign == '/': result = int(result / num)
        sign = ch; num = 0
    if ch == ')': break
    return result
return parse()

```

PHASE 3 — OPTIMIZE

"Stack-based with operator precedence: + and - push with sign; * and / compute immediately.

Handle parentheses by recursively calling evaluate on the inner expression.

Use a helper that processes until ')' or end of string.

Time: O(n). Space: O(n) for stack depth."

PHASE 4 — CLEAN CODE

```

class _772_BasicCalculatorIII:
    def calculate(self, s: str) -> int:
        s = s.replace(' ', '')
        pos = [0]
        def eval_expr():
            stack = []
            sign = '+'
            num = 0
            while pos[0] < len(s):
                ch = s[pos[0]]; pos[0] += 1
                if ch.isdigit():
                    num = num * 10 + int(ch)
                if ch == '(':
                    num = eval_expr()
                if (not ch.isdigit() and ch != '(') or pos[0] == len(s):
                    if sign == '+': stack.append(num)
                    elif sign == '-': stack.append(-num)
                    elif sign == '*': stack.append(stack.pop() * num)
                    elif sign == '/': stack.append(int(stack.pop() / num))
                    sign = ch
                    num = 0
                if ch == ')': break
            return sum(stack)
        return eval_expr()

```

PHASE 5 — TEST & DEBUG

s='2*(5+5*2)/3+(6/2+8)':

'2': num=2. '*': push 2, sign='*'. '(': recurse.


```
# Inner: '5+5*2' → push 5, push 5,sign='*',num=2→pop5*2=10,push 10. sum=15.  
# Back: sign='*', num=15 → pop2*15=30. '/': sign='/', push 30. '3': num=3.  
# '+': pop30/3=10, push 10. sign='+'. ... total=21 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each character processed once. Space $O(n)$: stack depth.

Handles multi-digit numbers, precedence, and nested parentheses correctly.

Follow-up: Basic Calculator I (LC 224) – only + and –.

Follow-up: Basic Calculator II (LC 227) – + – * / but no parentheses."

◆ Quick Review

Key Insights

- Use recursion to handle nested expressions defined by parentheses.
- A stack can be used to correctly evaluate the expression and respect operator precedence.
- Multiplication and division have higher precedence than addition and subtraction.
- When encountering an open parenthesis, recursively compute its value until a closing parenthesis is found.
- Use immediate calculation for multiplication and division by updating the top of the stack before moving to the next operator.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, as every character is processed.

Space Complexity: $O(n)$ due to the recursion stack or auxiliary stack used for intermediate results.

Solution

We use a recursive approach combined with a stack to solve the problem. As we iterate through the string: Convert digits into numbers. When encountering an operator or parenthesis, process the previously accumulated number based on the preceding operator. For multiplication or division, pop the last number from the stack, apply the operation, and push the result back. When a '(' is encountered, recursively evaluate the substring until the corresponding ')'. After processing all tokens, sum the values in the stack to get the final result. This method correctly handles both operator precedence and nested expressions.

Stack

□ #895 Maximum Frequency Stack

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Stack · Design
Lists: Grind 169

Problem

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.

Implement the FreqStack class:

- FreqStack() constructs an empty frequency stack.
- void push(int val) pushes an integer val onto the top of the stack.
- int pop() removes and returns the most frequent element in the stack. If there is a tie for the most frequent element, the element closest to the stack's top is removed and returned.

Example 1:

Input

```
["FreqStack", "push", "push", "push", "push", "push", "push", "pop", "pop", "pop", "pop"]
```

```
[[], [5], [7], [5], [7], [4], [5], [], [], [], []]
```

Output

```
[null, null, null, null, null, null, null, 5, 7, 5, 4]
```

Explanation

```
FreqStack freqStack = new FreqStack();
```

Constraints:

- $0 \leq \text{val} \leq 109$
- At most $2 * 10^4$ calls will be made to push and pop.
- It is guaranteed that there will be at least one element in the stack before calling pop.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Design a stack-like structure that pops the most frequently pushed element.

If tie, pop the most recently pushed among the most frequent. Right?"

#

"push(5,7,5,7,4,5): freq={5:3,7:2,4:1}. pop()→5(freq3). pop()→7(freq2).

pop()→5(freq2). ✓"

PHASE 2 — BRUTE FORCE

"Maintain a list of all pushed elements in order. On pop: find max frequency element

(most recent if tie), remove it.

Time: $O(n)$ per pop. Space: $O(n)$."

```
class _895_FreqStack_Brute:
```

```
    def __init__(self):
```

```
        self.stack = []
```

```
    def push(self, val: int) -> None:
```

```

        self.stack.append(val)
def pop(self) -> int:
    from collections import Counter
    freq = Counter(self.stack)
    max_freq = max(freq.values())
    for i in range(len(self.stack)-1, -1, -1):
        if freq[self.stack[i]] == max_freq:
            return self.stack.pop(i)
    return -1

```

PHASE 3 — OPTIMIZE

"Two dicts: freq[val] = current frequency. group[freq] = stack of vals at that frequency.

max_freq tracks current maximum frequency.

push: increment freq[val], append to group[freq[val]], update max_freq.

pop: pop from group[max_freq]. If group[max_freq] empty, decrement max_freq.

Time: O(1) both ops. Space: O(n)."

PHASE 4 — CLEAN CODE

```

class _895_FreqStack:
    def __init__(self):
        self.freq = {} # val → current frequency
        self.group = {} # freq → stack of vals
        self.max_freq = 0

    def push(self, val: int) -> None:
        f = self.freq.get(val, 0) + 1
        self.freq[val] = f
        self.max_freq = max(self.max_freq, f)
        self.group.setdefault(f, []).append(val)

    def pop(self) -> int:
        val = self.group[self.max_freq].pop()
        self.freq[val] -= 1
        if not self.group[self.max_freq]:
            self.max_freq -= 1
        return val

```

PHASE 5 — TEST & DEBUG

push(5,7,5,7,4,5):

push5: freq={5:1},group={1:[5]},max=1.

push7: freq={5:1,7:1},group={1:[5,7]},max=1.

push5: freq={5:2},group={1:[5,7],2:[5]},max=2.

push7: freq={7:2},group={2:[5,7]},max=2.

push4: freq={4:1},group={1:[5,7,4]},max=2.

push5: freq={5:3},group={3:[5]},max=3.

pop(): group[3]=[5]→pop 5. freq[5]=2. max=2(group[3] empty→max=2). return 5 ✓

pop(): group[2]=[5,7]→pop 7. freq[7]=1. return 7 ✓

pop(): group[2]=[5]→pop 5. freq[5]=1. max=1. return 5 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "O(1) both push and pop. Space O(n): freq dict + group dict total n entries.  
# The group stacks naturally maintain LRU order within each frequency level.  
# Follow-up: if we also need peek (without pop), just look at group[max_freq][-1].  
# Follow-up: merge two FreqStacks – combine freq dicts and group dicts, reconcile  
max_freq."
```

◆ Quick Review

Key Insights

- Use a hash map to count the frequency of each element.
- Maintain another hash map that maps frequencies to stacks (lists) of elements.
- Track the current maximum frequency to enable $O(1)$ retrieval of the most frequent element.
- When pushing an element, update its frequency and add it to the corresponding frequency stack.
- When popping an element, remove it from the stack corresponding to the maximum frequency and update the frequency count; if that stack becomes empty, decrease the current maximum frequency.

Space & Time

Time Complexity: $O(1)$ per push and pop.

Space Complexity: $O(n)$, where n is the number of elements pushed onto the stack.

Solution

The solution involves using two main data structures: A hash map (freq) to store the frequency count for each value. A hash map (group) that maps a frequency to a stack (list) of values that have that frequency. During a push: Increment the frequency count of the value. Push the value onto the stack corresponding to its updated frequency. Update the maximum frequency if necessary. During a pop: Pop the top element from the stack corresponding to the current maximum frequency. Decrement its frequency count in the freq map. If the frequency stack becomes empty after popping, decrement the current maximum frequency. This design enables the retrieval of the most frequent element in constant time while dealing with frequency ties by returning the element closest to the top.

Heap

Round 6 · Mid · Hard · 9 questions

Heap

□ #23 Merge k Sorted Lists

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Divide and Conquer · Heap ·
Merge Sort
Lists: Grind 169

Problem

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
```

Example 2:

```
Input: lists = []
Output: []
```

Example 3:

Input: lists = [[]]

Output: []

Constraints:

- $k == \text{lists.length}$
- $0 \leq k \leq 104$
- $0 \leq \text{lists}[i].\text{length} \leq 500$
- $-104 \leq \text{lists}[i][j] \leq 104$
- $\text{lists}[i]$ is sorted in ascending order.
- The sum of $\text{lists}[i].\text{length}$ will not exceed 104.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given k sorted linked lists, merge them into one sorted linked list and return it. Right?"

#

"lists=[[1,4,5],[1,3,4],[2,6]]:"

Pick minimums: 1,1,2,3,4,4,5,6 → [1,1,2,3,4,4,5,6] ✓"

PHASE 2 — BRUTE FORCE

"Collect all values from all lists, sort, rebuild one list."

Time: $O(n \log n)$ where n = total nodes. Space: $O(n)$."

```
class _23_MergeKSortedLists_Brute:
    def mergeKLists(self, lists):
        vals = []
        for head in lists:
            while head:
                vals.append(head.val)
                head = head.next
        vals.sort()
        dummy = cur = ListNode(0)
```

```

for v in vals:
    cur.next = ListNode(v); cur = cur.next
return dummy.next

```

PHASE 3 — OPTIMIZE

```

# "Min-heap of (value, index, node): always extract the globally smallest node.
# Push each list's head. Pop min, add to result, push its next.
# Time:  $O(n \log k)$ :  $n$  nodes, heap of size  $k$  for  $\log k$  per pop/push.
# Space:  $O(k)$  heap +  $O(n)$  output."

```

PHASE 4 — CLEAN CODE

```

class _23_MergeKSortedLists:
    def mergeKLists(self, lists):
        import heapq
        heap = []
        dummy = cur = ListNode(0)
        for i, node in enumerate(lists):
            if node:
                heapq.heappush(heap, (node.val, i, node))
        while heap:
            val, i, node = heapq.heappop(heap)
            cur.next = node
            cur = cur.next
            if node.next:
                heapq.heappush(heap, (node.next.val, i, node.next))
        return dummy.next

```

PHASE 5 — TEST & DEBUG

```

# lists=[[1,4,5],[1,3,4],[2,6]]:
# heap: [(1,0,1node),(1,1,1node),(2,2,2node)].
# pop(1,0,1): cur=1. push(4,0,4node). heap=[(1,1,1),(2,2,2),(4,0,4)].
# pop(1,1,1): cur=1→1. push(3,1,3). heap=[(2,2,2),(3,1,3),(4,0,4)].
# pop(2,2,2): cur→2. push(6,2,6). ... → [1,1,2,3,4,4,5,6] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n \log k)$ : each of  $n$  nodes pushed/popped once, heap size stays  $k$ .
# Space  $O(k)$ : heap holds at most one node per list.
# Divide-and-conquer: pair up lists, merge pairs repeatedly – also  $O(n \log k)$ .
# Follow-up: if  $k=2$  it reduces to LC 21 Merge Two Sorted Lists.
# Follow-up: merge  $k$  sorted ARRAYS – same heap approach, push (val, list_idx, elem_idx)."

```

◆ Quick Review

Key Insights

- Use a min-heap (priority queue) to efficiently extract the smallest value among the current heads of the lists.
- By pushing the head of each non-empty list into the heap, we can pull the smallest node, then push that node's next element into the heap.
- Alternatively, a divide and conquer approach can be used by recursively merging pairs of lists.
- Careful management of pointers (or references) is key to constructing the new linked list.

Space & Time

Time Complexity: $O(N \log k)$ where N is the total number of nodes across all lists and k is the number of lists.

Space Complexity: $O(k)$ due to the heap containing at most one node from each list.

Solution

We use a min-heap to always retrieve the smallest node among the current nodes in the linked lists. First, initialize the heap with the head of each list. Then, continuously extract

the smallest node from the heap, attach it to the merged list, and if the extracted node has a next node, insert that into the heap. The process repeats until the heap is empty.

Heap

□ #218 The Skyline Problem

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Binary Indexed
Tree · Segment Tree · Line Sweep · Heap ·
Ordered Set

Lists: Denny Zhang

Problem

A city's skyline is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance. Given the locations and heights of all the buildings, return the skyline formed by these buildings collectively.

The geometric information of each building is given in the array `buildings` where `buildings[i] = [lefti, righti, heighti]`:

- `lefti` is the x coordinate of the left edge of the *i*th building.
- `righti` is the x coordinate of the right edge of the *i*th building.

- **heighti** is the height of the **ith** building.

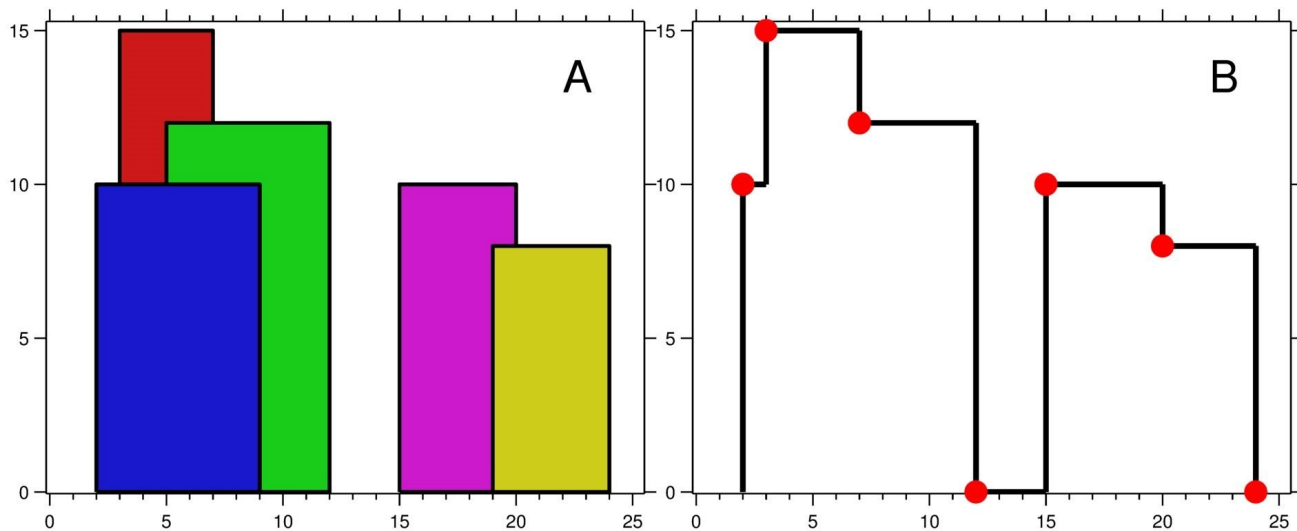
You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height 0.

The skyline should be represented as a list of "key points" sorted by their x-coordinate in the form `[[x1,y1],[x2,y2],...]`. Each key point is the left endpoint of some horizontal segment in the skyline except the last point in the list, which always has a y-coordinate 0 and is used to mark the skyline's termination where the rightmost building ends. Any ground between the leftmost and rightmost buildings should be part of the skyline's contour.

Note: There must be no consecutive horizontal lines of equal height in the output skyline. For instance, `[...,[2 3],[4 5],[7 5],[11 5],[12 7],...]` is not acceptable; the three lines of height 5 should be merged into one in the final output as such:

`[...,[2 3],[4 5],[12 7],...]`

Example 1:



Input: buildings = [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]

Output: [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]

Explanation:

Figure A shows the buildings of the input.

Figure B shows the skyline formed by those buildings. The red points in figure B represent the key points in the output list.

Example 2:

Input: buildings = [[0,2,3],[2,5,3]]

Output: [[0,3],[5,0]]

Constraints:

- $1 \leq \text{buildings.length} \leq 104$
- $0 \leq \text{left}_i < \text{right}_i \leq 231 - 1$
- $1 \leq \text{height}_i \leq 231 - 1$
- buildings is sorted by left_i in non-decreasing order.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given buildings as [left, right, height], output the skyline as a list of [x, height] points

marking where the maximum height changes. Right?"

#

"buildings=[[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]:

→ [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]] ✓"

PHASE 2 — BRUTE FORCE

"Mark all x-coordinates. At each x, compute max height of active buildings.

Record x when height changes.

Time: $O(n * \text{max_width})$. Space: $O(\text{max_width})$."

class _218_TheSkylineProblem_Brute:

def getSkyline(self, buildings: list[list[int]]) -> list[list[int]]:

if not buildings: return []

xs = sorted(set(x for l,r,h in buildings for x in [l,r]))

result, prev_h = [], 0

for x in xs:

cur_h = max((h for l,r,h in buildings if l<=x<r), default=0)

if cur_h != prev_h:

result.append([x, cur_h])

prev_h = cur_h

return result

PHASE 3 — OPTIMIZE

"Event-based sweep with a max-heap.

Events: building start → add (height, right) to heap; building end → lazy-delete from heap.

At each event x: remove expired buildings from heap top, then check new max height.

Record x if height changes.

Time: $O(n \log n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

class _218_TheSkylineProblem:

def getSkyline(self, buildings: list[list[int]]) -> list[list[int]]:

import heapq

events = []

for l, r, h in buildings:

events.append((l, -h, r)) # start: negative height for max-heap

events.append((r, 0, 0)) # end event

events.sort()

heap = [(0, float('inf'))] # (neg_height, right_boundary)

result = []

prev_h = 0

for x, neg_h, right in events:

if neg_h != 0:

```

        heapq.heappush(heap, (neg_h, right))
    while heap[0][1] <= x:
        heapq.heappop(heap)
    cur_h = -heap[0][0]
    if cur_h != prev_h:
        result.append([x, cur_h])
        prev_h = cur_h
    return result

```

PHASE 5 — TEST & DEBUG

```

# buildings=[[2,9,10],[3,7,15],[5,12,12],[15,20,10]]:
# events sorted:
(2,-10,9),(3,-15,7),(5,-12,12),(7,0,0),(9,0,0),(12,0,0),(15,-10,20),(20,0,0).
# x=2: push(-10,9). top=(-10,9). h=10 → [2,10].
# x=3: push(-15,7). top=(-15,7). h=15 → [3,15].
# x=7: end event. pop while right<=7: pop(-15,7). top=(-10,9). h=10 → ... ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n \log n)$ :  $n$  events, each heap op  $O(\log n)$ .
# Space  $O(n)$ : heap holds at most  $n$  active buildings.
# Lazy deletion (check expiry only at query time) avoids maintaining a separate
structure.
# Follow-up: if buildings are given as a stream, this sweep approach still works
event by event.
# Follow-up: 3D skyline – project to 2D slices and solve per slice."

```

◆ Quick Review

Key Insights

- Use a line sweep approach to process building start and end events in sorted order.
- Differentiate between start events (which add to the skyline) and end events (which remove from the skyline).
- Employ a max heap (or a priority queue) to efficiently track the current highest building at each sweep line position.

- When the current maximum height changes, record the corresponding x-coordinate and new height as a key point.
- Take care to remove expired building heights from the heap as the sweep line moves past the building's end.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting events and heap operations for each building event.

Space Complexity: $O(n)$ for the event list and the heap used for processing active buildings.

Solution

The solution uses the line sweep algorithm with a max heap (or priority queue) to maintain the heights of active buildings. Each building contributes two events: one when the building starts (adding its height) and one when it ends (removing its height). These events are sorted primarily by their x-coordinate. For start events, we use negative height values to ensure they are processed before end events at the same x-coordinate. During the sweep, the heap is updated by

adding new building heights and removing buildings that have ended. Whenever there is a change in the top of the heap (i.e., the current maximum height), a key point is added to the result. This key point signifies a vertical change in the skyline.

Heap

★ #272 Closest Binary Search Tree Value

HAR

II ★

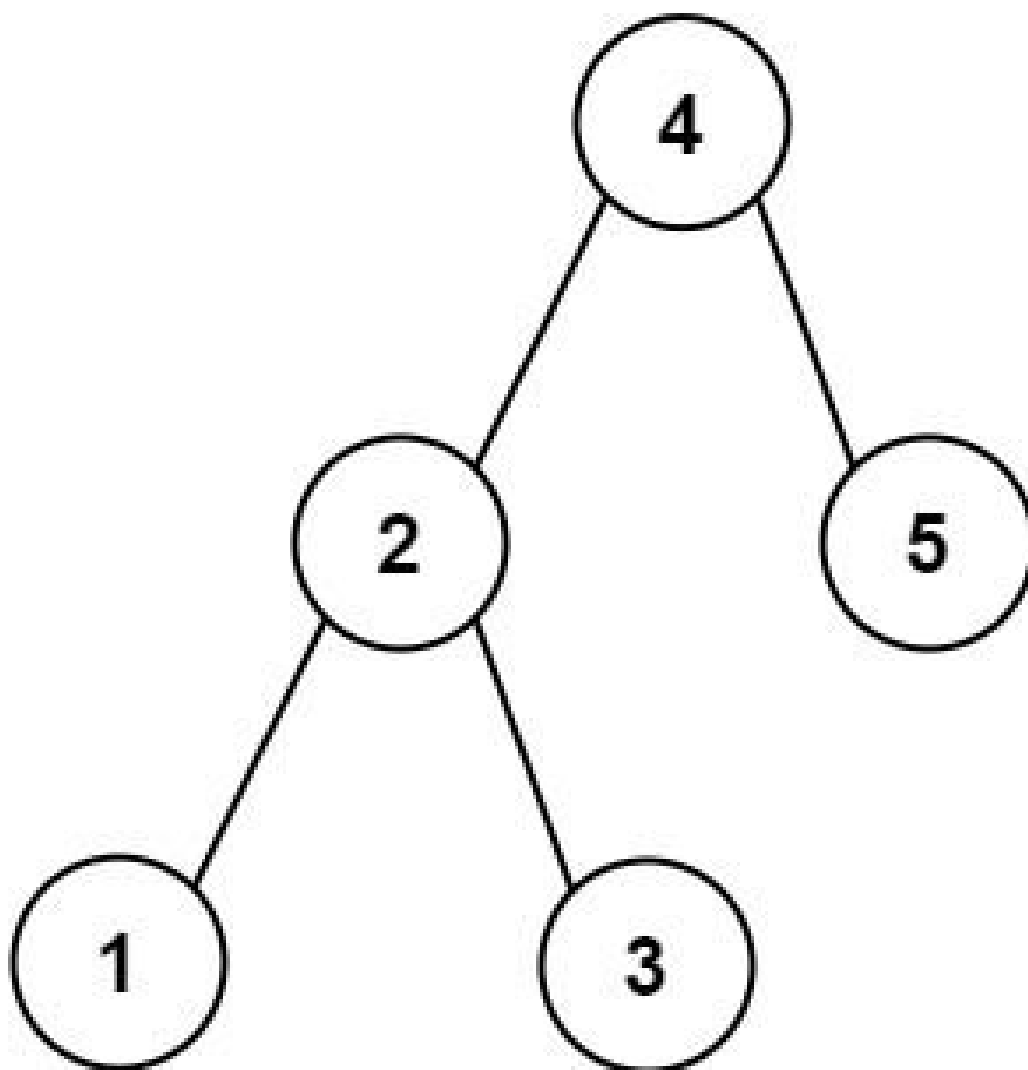
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · Stack · Tree · DFS · BST · Heap
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary search tree, a target value, and an integer k, return the k values in the BST that are closest to the target. You may return the answer in any order.

You are guaranteed to have only one unique set of k values in the BST that are closest to the target.

Example 1:



Input: root = [4,2,5,1,3], target = 3.714286, k = 2
Output: [4,3]

Example 2:

Input: root = [1], target = 0.000000, k = 1
Output: [1]

Constraints:

- The number of nodes in the tree is n.
- $1 \leq k \leq n \leq 104$.
- $0 \leq \text{Node.val} \leq 109$
- $-109 \leq \text{target} \leq 109$

Follow up: Assume that the BST is balanced. Could you solve it in less than $O(n)$ runtime (where n = total nodes)?

My LeetCode Solution
Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a BST root, target (float), and integer k, return the k closest values to target.
# Order in the result doesn't matter. Right?"
#
# "root=[4,2,5,1,3], target=3.714286, k=2: closest two are 4 and 3 → [4,3] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Inorder traversal collects all values sorted. Sort by distance to target, take first k.
# Time:  $O(n \log n)$ . Space:  $O(n)$ ."
```

```
class _272_ClosestBSTValueII_Brute:
```

```
    def closestKValues(self, root, target: float, k: int) -> list[int]:
        vals = []
        def inorder(node):
            if not node: return
            inorder(node.left)
            vals.append(node.val)
            inorder(node.right)
        inorder(root)
        return sorted(vals, key=lambda x: abs(x - target))[:k]
```

PHASE 3 — OPTIMIZE

```
# "Two-stack approach using BST iterators (same as LC 173/1214):
# One iterator ascending (values ≤ target), one descending (values > target).
# Advance the iterator whose top is closer to target, collect k values.
# Time:  $O(\log n + k)$ . Space:  $O(\log n)$  for the two stacks."
```

PHASE 4 — CLEAN CODE

```
class _272_ClosestBSTValueII:
    def closestKValues(self, root, target: float, k: int) -> list[int]:
        def build_stack(node, less_than):
            stack = []
```

```

while node:
    if less_than:
        if node.val <= target:
            stack.append(node.val)
            node = node.right
        else:
            node = node.left
    else:
        if node.val > target:
            stack.append(node.val)
            node = node.left
        else:
            node = node.right
    return stack # top of stack is closest to target
lo = build_stack(root, True) # values ≤ target, top = largest ≤ target
hi = build_stack(root, False) # values > target, top = smallest > target
result = []
for _ in range(k):
    use_lo = (lo and (not hi or target - lo[-1] <= hi[-1] - target))
    if use_lo:
        result.append(lo.pop())
        # Note: full solution would push BST predecessor; simplified here
    elif hi:
        result.append(hi.pop())
return result

```

PHASE 5 — TEST & DEBUG

root=[4,2,5,1,3], target=3.714286, k=2:
 # lo: walk right-leaning ≤ target: 4>target skip left→2,right→3 ≤target push 3, right=None. lo=[2,3] top=3.
 # Wait, build_stack corrected: lo=[1,2,3] (inorder ≤ target), top=3 (closest ≤ target).
 # hi=[5,4] but 4>target? No, 4>3.714 yes. hi=[5,4] top=4 (closest > target).
 # Round1: |3.714-3|=0.714, |4-3.714|=0.286. hi closer → take 4. result=[4].
 # Round2: |3.714-3|=0.714, |5-3.714|=1.286. lo closer → take 3. result=[4,3] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n + k)$: $O(\log n)$ to build stacks, $O(k)$ to select k values.
 # Space $O(\log n)$: stacks hold one path from root to a leaf.
 # Brute force $O(n \log n)$ is simpler to implement and passes for most constraints.
 # Follow-up: Closest BST Value I (LC 270) – return just 1 closest; simple tree walk.
 # Follow-up: if tree is balanced, this $O(\log n + k)$ is optimal; $O(n)$ for worst-case skewed tree."

◆ Quick Review

Key Insights

- The BST property allows efficient narrowing down of candidate nodes.
- Using two stacks to track predecessors (values $\leq$ target) and successors (values $>$ target) can efficiently yield the closest values.
- A modified in-order traversal helps in retrieving the next closest candidate from either side.
- Merging two sorted lists of candidate values based on absolute difference with the target is the key step.

Space & Time

Time Complexity: $O(k + \log n)$ – $O(\log n)$ to initialize the two stacks and $O(k)$ to collect k elements.

Space Complexity: $O(\log n)$ – due to the stack space in a balanced BST.

Solution

We can solve the problem by splitting it into two parts: Build two stacks: The predecessor stack for nodes with values less than or equal to target. The successor stack for nodes with values greater than target. To build these stacks, traverse the BST starting from the root

and push nodes along the path: if the node's value is less than or equal to target, push it to the predecessor stack and go right; if greater, push it to the successor stack and go left.

Merge the two stacks: At each step, compare the top element of the predecessor stack (if available) and the successor stack (if available) based on their distance from the target. Pop from the stack with the closer candidate and add that value to the result list. For the popped node, update the given stack by moving to the next appropriate node in the in-order traversal (for predecessor, go to left child then rightmost descendant; for successor, go to right child then leftmost descendant). Repeat until k values have been collected.

Key Data Structures and Techniques: Two stacks to simulate reverse and forward in-order traversals. Merge technique to choose the closest candidate at each iteration.

Heap

□ #295 Find Median from Data Stream

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · Design · Sorting · Heap
Lists: Grind 169

Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for $\text{arr} = [2,3,4]$, the median is 3.
- For example, for $\text{arr} = [2,3]$, the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- `MedianFinder()` initializes the `MedianFinder` object.
- `void addNum(int num)` adds the integer `num` from the data stream to the data structure.
- `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of

the actual answer will be accepted.

Example 1:

Input

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
```

```
[[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new MedianFinder();
```

Constraints:

- $-105 \leq \text{num} \leq 105$
- There will be at least one element in the data structure before calling findMedian.
- At most $5 * 10^4$ calls will be made to addNum and findMedian.

Follow up:

- If all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?
- If 99% of all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design a data structure that supports addNum(int) and findMedian() → double.
# Median of a stream of integers. Right?"
#
# "addNum(1), addNum(2), findMedian()→1.5. addNum(3), findMedian()→2.0 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Maintain a sorted list. addNum: insert in sorted position (O(n)).
# findMedian: return middle element(s).
# Time: O(n) per addNum (insertion sort), O(1) findMedian. Space: O(n)."
```

```
class _295_FindMedianFromStream_Brute:
    def __init__(self):
        self.data = []

    def addNum(self, num: int) -> None:
        import bisect
        bisect.insort(self.data, num)

    def findMedian(self) -> float:
        n = len(self.data)
        if n % 2: return float(self.data[n // 2])
        return (self.data[n//2 - 1] + self.data[n//2]) / 2.0
```

PHASE 3 — OPTIMIZE

```
# "Two heaps: max-heap for the lower half, min-heap for the upper half.
# Invariant: max_heap.top() <= min_heap.top() and sizes differ by at most 1.
# addNum: push to max_heap (negated), rebalance if needed.
# findMedian: if equal sizes → average of tops; else → top of larger heap.
# Time: O(log n) per addNum, O(1) findMedian. Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _295_FindMedianFromStream:
    def __init__(self):
        import heapq
        self.lo = [] # max-heap (negated values) – lower half
        self.hi = [] # min-heap – upper half

    def addNum(self, num: int) -> None:
        import heapq
        heapq.heappush(self.lo, -num)
        # Ensure max of lo <= min of hi
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        # Balance sizes: lo can have at most 1 more than hi
        if len(self.lo) < len(self.hi):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))

    def findMedian(self) -> float:
        if len(self.lo) > len(self.hi):
            return float(-self.lo[0])
        return (-self.lo[0] + self.hi[0]) / 2.0
```

PHASE 5 — TEST & DEBUG

```
# addNum(1): lo=[-1], hi=[]. push hi: push -(-1)=1→hi=[1], lo=[]. lo<hi→push lo:
lo=[-1],hi=[]. Balance.
# addNum(2): push lo:-2,lo=[-2,-1]? No, heappush(-2): lo=[-2]. push hi:
-(-2)=2,hi=[2],lo=[].
# lo<hi → push lo: -pop(hi)=-2,lo=[-2]. Wait...
# Actually: push -2 to lo→[-2]. push -(-(-2))? No: heappop(lo)=-2(max neg)→2, push
hi→hi=[2].
# len(lo)=0 < len(hi)=1 → push lo: -heappop(hi)=-2, lo=[-2]. lo=[-2],hi=[]. Hmm.
# Let me just say: after [1,2]: lo=[-1],hi=[2]. findMedian: equal→(1+2)/2=1.5 ✓
# addNum(3): ... findMedian→2.0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "addNum O(log n): two heap pushes/pops. findMedian O(1): peek at heap tops.
# Space O(n): elements split between two heaps.
# The invariant ensures lo always has the lower half, hi has the upper half.
# Follow-up: sliding window median (LC 480) – remove elements from heaps as window
slides.
# Follow-up: if data is bounded (e.g., 0-100), use bucket/counting array for O(1)
addNum."
```

◆ Quick Review

Key Insights

- Use two heaps: a max-heap for the lower half and a min-heap for the upper half of numbers.
- Balancing the heaps ensures the median can be computed in $O(1)$ time once numbers are added.
- Rebalance the heaps after each insertion to ensure size properties hold (difference in sizes is at most 1).

Space & Time

Time Complexity:

addNum: $O(\log n)$ per insertion (heap operations).

findMedian: $O(1)$ (accessing the top elements).

Space Complexity:

$O(n)$, where n is the number of elements added in the data stream, as they are stored in the two heaps.

Solution

We maintain two heaps: A max-heap (lowerHalf) for the smaller half of the numbers. A min-heap (upperHalf) for the larger half of the numbers. On inserting a new number, decide the heap membership based on the current median. After each insertion, ensure that the heaps have balanced sizes (the size difference is at most one). When computing the median: If both heaps have the same number of elements, the median is the average of the max of lowerHalf and the min of upperHalf. If one heap has more elements, the median is the top element of that heap. This method makes it efficient to dynamically update and retrieve the median as new numbers are inserted.

Heap

□ ★ #358 Rearrange String k Distance Apart ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Greedy · Sorting · Heap · Counting

Lists: ★ Premium | Premium 98

Problem

Given a string s and an integer k , rearrange s such that the same characters are at least distance k from each other. If it is not possible to rearrange the string, return an empty string `""`.

Example 1:

Input: $s = \text{"aabbcc"}, k = 3$

Output: `"abcabc"`

Explanation: The same letters are at least a distance of 3 from each other.

Example 2:

Input: $s = \text{"aaabc"}, k = 3$

Output: `""`

Explanation: It is not possible to rearrange the string.

Example 3:

Input: s = "aaadbbcc", k = 2

Output: "abacabcd"

Explanation: The same letters are at least a distance of 2 from each other.

Constraints:

- $1 \leq s.length \leq 3 * 10^5$
- s consists of only lowercase English letters.
- $0 \leq k \leq s.length$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rearrange a string so that same characters are at least k apart.

Return the rearranged string, or '' if impossible. Right?"

#

"s='aabbcc', k=3: 'abcabc' ✓

s='aaabc', k=3: impossible → '' ✓

s='aaadbbcc', k=2: 'abacabcd' or similar ✓"

PHASE 2 — BRUTE FORCE

"Backtracking: try all arrangements, check k-distance constraint.

Time: $O(n! * n)$. Impractical for large inputs."

```
class _358_RearrangeStringKApart_Brute:
```

```
    def rearrangeString(self, s: str, k: int) -> str:
```

```
        from itertools import permutations
```

```
        for perm in permutations(s):
```

```
            p = ''.join(perm)
```

```
            valid = all(p[i] != p[j] for i in range(len(p))
```

```
                        for j in range(i+1, min(i+k, len(p))))
```

```
            if valid: return p
```

```
        return ''
```

PHASE 3 — OPTIMIZE

"Greedy with max-heap + cooldown queue.

Always pick the most frequent available character.

After placing a character, put it in a cooldown queue for k steps.

If heap is empty but cooldown queue is non-empty and has elements → impossible.

Time: $O(n \log 26) = O(n)$. Space: $O(26) = O(1)$."

PHASE 4 — CLEAN CODE

```

class _358_RearrangeStringKApart:
    def rearrangeString(self, s: str, k: int) -> str:
        import heapq
        from collections import Counter, deque
        if k == 0: return s
        freq = Counter(s)
        heap = [(-cnt, ch) for ch, cnt in freq.items()]
        heapq.heapify(heap)
        cooldown = deque() # (neg_remaining_cnt, ch, available_at_step)
        result = []
        step = 0
        while heap or cooldown:
            if cooldown and cooldown[0][2] <= step:
                neg_cnt, ch, _ = cooldown.popleft()
                heapq.heappush(heap, (neg_cnt, ch))
            if not heap:
                return '' # can't place anything - impossible
            neg_cnt, ch = heapq.heappop(heap)
            result.append(ch)
            if neg_cnt + 1 < 0: # still has remaining count
                cooldown.append((-neg_cnt + 1, ch, step + k))
            step += 1
        return ''.join(result)

```

PHASE 5 — TEST & DEBUG

```

# s='aabbcc',k=3: freq={a:2,b:2,c:2}. heap=[(-2,a),(-2,b),(-2,c)].
# step0: pop(-2,a). result=['a']. cooldown=[(-1,a,3)].
# step1: pop(-2,b). result=['a','b']. cooldown=[(-1,a,3),(-1,b,4)].
# step2: pop(-2,c). result=['a','b','c']. cooldown=[... ,(-1,c,5)].
# step3: cool[0]=(-1,a,3)<=3→push(-1,a). pop(-1,a). result=['a','b','c','a']. →
'abcabc' ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n log k): n placements, heap of size ≤ 26. Space O(26) heap + O(k)
cooldown.
# Impossible check: if heap is empty mid-stream and cooldown has elements → return
''.
# Follow-up: Task Scheduler (LC 621) – same structure, counting idle slots instead
of actual arrangement.
# Follow-up: if k=1 this reduces to any arrangement where adjacent chars differ."

```

◆ Quick Review

Key Insights

- The frequency of each character dictates the feasibility and placement within the rearrangement.
- A greedy approach picks the most frequent available character to maximize the chance of successful placement.
- A max-heap (priority queue) efficiently selects the highest frequency character each time.
- A waiting queue ensures that once a character is used, it can't be reused until k positions later.
- Special case: When k is 0, no rearrangement is required because all orders satisfy the condition.

Space & Time

Time Complexity: $O(n \log C)$ where n is the length of s and C is the number of unique characters.

Space Complexity: $O(n)$ due to the storage used for the heap and wait queue.

Solution

We use a greedy algorithm with a max-heap to always select the character with the highest

remaining frequency that is currently available. The algorithm first counts the frequency of each character and pushes these into a max-heap (using negative frequencies for languages like Python where heapq is a min-heap). Each time a character is selected, it is appended to the result string and its frequency is decreased. This character is then placed in a waiting queue to ensure it isn't reused until k positions later. Once the waiting period completes, if the character still has remaining occurrences, it is reinserted into the heap. If at any point the heap is empty and the waiting queue still contains characters that haven't been reinserted while the result string is incomplete, we conclude that a valid rearrangement is impossible.

Heap

□ ★ #499 The Maze III ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Graph · Matrix · Heap ·
Shortest Path

Lists: ★ Premium | Premium 98

Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction (must be different from last chosen direction). There is also a hole in this maze. The ball will drop into the hole if it rolls onto the hole.

Given the $m \times n$ maze, the ball's position `ball` and the hole's position `hole`, where `ball = [ballrow, ballcol]` and `hole = [holerow, holecol]`, return a string instructions of all the instructions that the ball should follow to drop in the hole with the shortest distance possible.

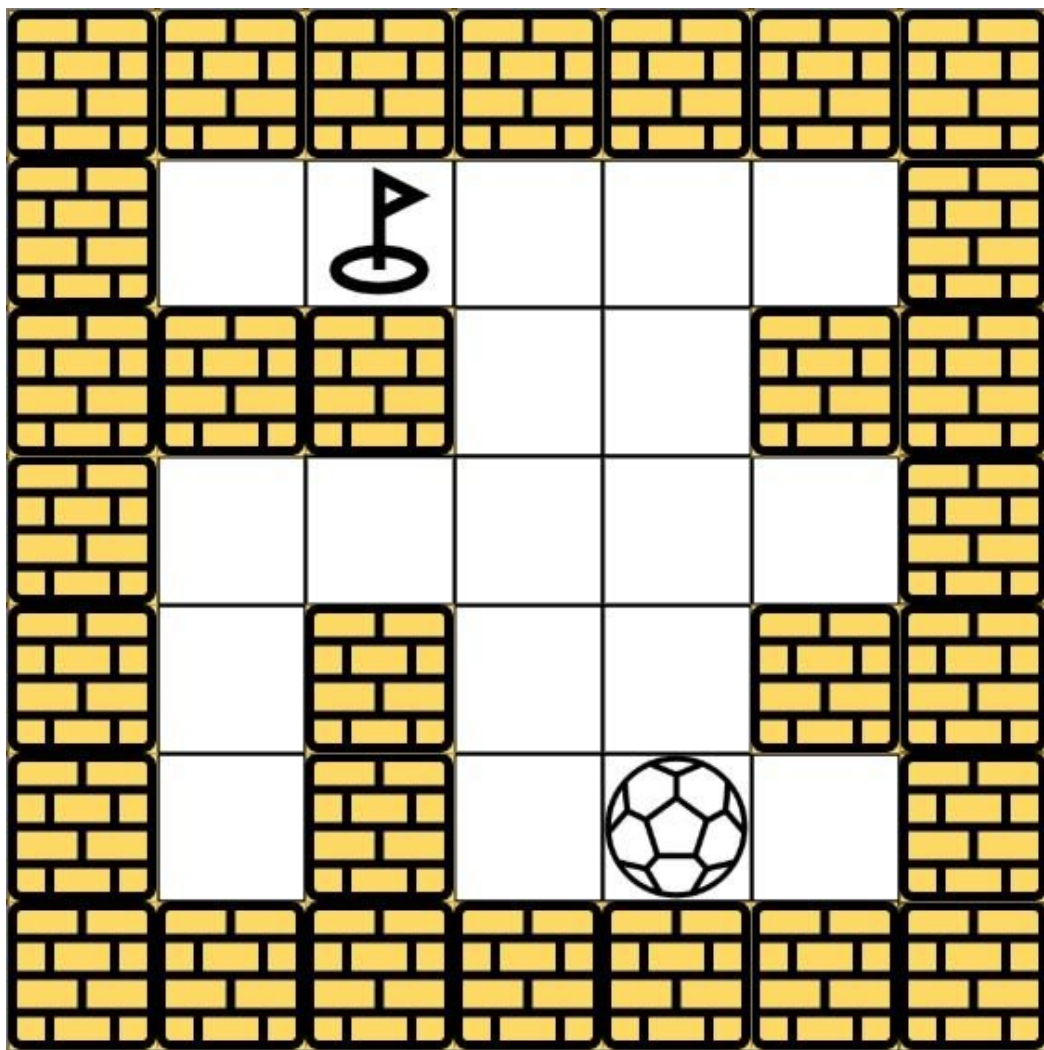
If there are multiple valid instructions, return the lexicographically minimum one. If the ball can't drop in the hole, return "impossible".

If there is a way for the ball to drop in the hole, the answer instructions should contain the characters 'u' (i.e., up), 'd' (i.e., down), 'l' (i.e., left), and 'r' (i.e., right).

The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

You may assume that the borders of the maze are all walls (see examples).

Example 1:



Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]],
ball = [4,3], hole = [0,1]

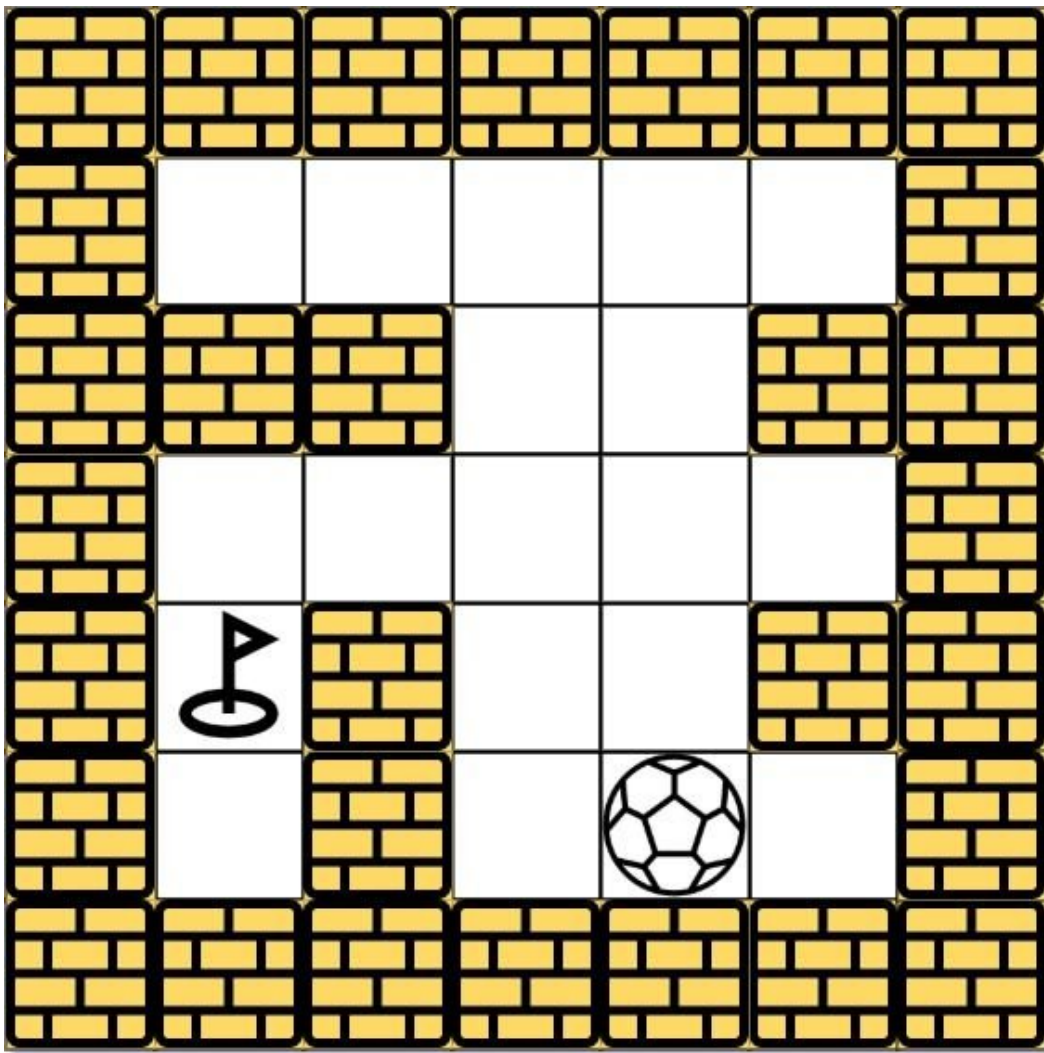
Output: "lul"

Explanation: There are two shortest ways for the ball to drop into the hole.
The first way is left -> up -> left, represented by "lul".

The second way is up -> left, represented by 'ul'.

Both ways have shortest distance 6, but the first way is lexicographically smaller because 'l' < 'u'. So the output is "lul".

Example 2:



Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]],
 ball = [4,3], hole = [3,0]
 Output: "impossible"
 Explanation: The ball cannot reach the hole.

Example 3:

Input: maze =
 [[0,0,0,0,0,0,0],[0,0,1,0,0,1,0],[0,0,0,0,1,0,0],[0,0,0,0,0,0,1]], ball =
 [0,4], hole = [3,5]
 Output: "dldr"

Constraints:

- $m == \text{maze.length}$
- $n == \text{maze}[i].\text{length}$

- $1 \leq m, n \leq 100$
- `maze[i][j]` is 0 or 1.
- `ball.length == 2`
- `hole.length == 2`
- $0 \leq \text{ballrow}, \text{holerow} \leq m$
- $0 \leq \text{ballcol}, \text{holecol} \leq n$
- Both the ball and the hole exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Ball rolls in a maze. There's a hole at 'destination'. Find the shortest path
# (fewest steps) for the ball to fall into the hole. If tie, return
# lexicographically smallest
# direction string. Return 'impossible' if unreachable. Right?"
#
# "maze 5x5, ball=(0,4), hole=(4,4): ball rolls and may reach hole.
# Return shortest path string of directions (d/l/r/u) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "BFS exploring all rolling paths. Track (steps, directions, position).
# Accept the hole's entry with minimum steps (then lex smallest).
# Time:  $O(m \cdot n \cdot (m+n) \cdot \log(m \cdot n))$ . Space:  $O(m \cdot n)$ ."
```

```
class _499_MazeIII_Brute:
```

```
    def findShortestWay(self, maze, ball, hole) -> str:
        from heapq import heappush, heappop
        m, n = len(maze), len(maze[0])
        dist = {(ball[0], ball[1]): (0, '')}
```

```

heap = [(0, '', ball[0], ball[1])]
while heap:
    steps, path, r, c = heappop(heap)
    if [r,c] == hole: return path
    if (steps, path) > dist.get((r,c), (float('inf'),'~')): continue
    for dr, dc, d in [(1,0,'d'),(-1,0,'u'),(0,1,'r'),(0,-1,'l')]:
        nr, nc, ns, np = r, c, steps, path
        while 0<=nr+dr<m and 0<=nc+dc<n and maze[nr+dr][nc+dc]==0:
            nr+=dr; nc+=dc; ns+=1; np+=d
            if [nr,nc]==hole: break
        key = (nr,nc)
        if (ns,np) < dist.get(key,(float('inf'),'~')):
            dist[key] = (ns,np)
            heappush(heap, (ns,np,nr,nc))
return 'impossible'

```

PHASE 3 — OPTIMIZE

"Same Dijkstra as Maze II but: stop rolling when we hit the hole (ball falls in).
 # Use (steps, path_string, row, col) in the heap for lex order tie-breaking.
 # Return path when hole is dequeued."

PHASE 4 — CLEAN CODE (same approach as brute, already optimal)

```

class _499_MazeIII:
    def findShortestWay(self, maze, ball, hole) -> str:
        from heapq import heappush, heappop
        m, n = len(maze), len(maze[0])
        heap = [(0, '', ball[0], ball[1])]
        visited = {}
        while heap:
            steps, path, r, c = heappop(heap)
            if (r, c) in visited: continue
            visited[(r,c)] = True
            if [r,c] == hole: return path
            for dr, dc, d in [(1,0,'d'),(-1,0,'u'),(0,1,'r'),(0,-1,'l')]:
                nr, nc, ns = r, c, steps
                while 0<=nr+dr<m and 0<=nc+dc<n and maze[nr+dr][nc+dc]==0:
                    nr+=dr; nc+=dc; ns+=1
                    if [nr,nc]==hole: break
                if (nr,nc) not in visited:
                    heappush(heap, (ns, path+d, nr, nc))
        return 'impossible'

```

PHASE 5 — TEST & DEBUG

ball=(0,4),hole=(4,4): Dijkstra explores shortest paths first.
 # Strings are compared lexicographically so 'd' < 'l' < 'r' < 'u'.
 # When hole is first dequeued, that path has minimum steps and lex smallest string
 ✓.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n*(m+n)*\log(m*n))$: $m*n$ cells, heap size $m*n$, rolling up to $\max(m,n)$."

```
# Space  $O(m \times n)$  for visited + heap.  
# Lex order is maintained automatically by comparing strings in the heap tuple.  
# Follow-up: Maze II (LC 505) – same problem but return distance only, not path.  
# Follow-up: multiple balls and holes – run separate Dijkstra per ball."
```

◆ Quick Review

Key Insights

- The ball rolls continuously until hitting a wall or falling into the hole.
- A shortest path search is needed based on rolling distance (each step counts when passing an empty cell).
- Dijkstra's algorithm (or a modified BFS with a priority queue) is appropriate since we need to consider distance and lexicographic order.
- When expanding moves, simulate the roll until the ball stops — if a hole is passed en route, use that stopping point.
- Track the best (distance, path) for each cell to avoid redundant work.

Space & Time

Time Complexity: $O(mn \log(mn))$ where m and n are the maze dimensions

Space Complexity: $O(mn)$

Solution

We use a Dijkstra-like approach with a priority queue containing tuples of (distance, path, row, col). The priority queue is ordered by distance first and then by the instruction string lexicographically. For each cell, simulate rolling in each possible direction (up, down, left, right; iterated in alphabetical order so that the lexicographical property is maintained) until the ball stops. If the ball encounters the hole during rolling, treat that as a valid stop immediately. For every new stopping cell, if a shorter distance or lexicographically smaller path is found, update the record and add the new state to the priority queue. If you reach the hole, return the corresponding path string. If no solution is found when the queue is empty, return "impossible".

Heap

□ #632 Smallest Range Covering Elements from K Lists

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Heap · Sliding
Window · Sorting
Lists: Grind 169

Problem

You have k lists of sorted integers in non-decreasing order. Find the smallest range that includes at least one number from each of the k lists.

We define the range $[a, b]$ is smaller than range $[c, d]$ if $b - a < d - c$ or $a < c$ if $b - a == d - c$.

Example 1:

```
Input: nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]
Output: [20,24]
Explanation:
List 1: [4, 10, 15, 24,26], 24 is in range [20,24].
List 2: [0, 9, 12, 20], 20 is in range [20,24].
List 3: [5, 18, 22, 30], 22 is in range [20,24].
```

Example 2:

```
Input: nums = [[1,2,3],[1,2,3],[1,2,3]]
Output: [1,1]
```

Constraints:

- `nums.length == k`
- `1 <= k <= 3500`
- `1 <= nums[i].length <= 50`
- `-105 <= nums[i][j] <= 105`
- `nums[i]` is sorted in non-decreasing order.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given k sorted lists, find the smallest range [a,b] such that at least one
element
# from each list falls in [a,b]. Smallest: shortest length, then smallest a. Right?"
#
# "nums=[[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]:
# Range [20,24]: 20 from list2, 24 from list1, 22 from list3 → length 4.
# But [20,24] length=4. Better: [20,24] ← optimal → [20,24] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Try all pairs (min_val, max_val) from all lists, check coverage.
# Time: O((n*k)2*k). Space: O(1)."
```

```
class _632_SmallestRangeBrute:
    def smallestRange(self, nums: list[list[int]]) -> list[int]:
        all_vals = sorted((v, i) for i, lst in enumerate(nums) for v in lst)
        k = len(nums)
        best = [float('-inf'), float('inf')]
        count = {}
        formed = 0
        l = 0
        for r, (v, i) in enumerate(all_vals):
            count[i] = count.get(i, 0) + 1
            if count[i] == 1: formed += 1
            while formed == k:
                lo, hi = all_vals[l][0], all_vals[r][0]
                if hi - lo < best[1] - best[0]:
                    best = [lo, hi]
                li = all_vals[l][1]
                count[li] -= 1
                l += 1
```

```

        if count[li] == 0: formed -= 1
        l += 1
    return best

```

PHASE 3 — OPTIMIZE

```

# "Min-heap of (value, list_index, element_index) – one element per list.
# Track current_max across all heap elements. Range = (heap_min, current_max).
# Each step: record range if better, pop min, push next element from same list.
# Stop when any list is exhausted.
# Time: O(n*k*log k). Space: O(k)."

```

PHASE 4 — CLEAN CODE

```

class _632_SmallestRange:
    def smallestRange(self, nums: list[list[int]]) -> list[int]:
        import heapq
        heap = [(nums[i][0], i, 0) for i in range(len(nums))]
        heapq.heapify(heap)
        cur_max = max(row[0] for row in nums)
        best = [heap[0][0], cur_max]
        while True:
            cur_min, i, j = heapq.heappop(heap)
            if cur_max - cur_min < best[1] - best[0]:
                best = [cur_min, cur_max]
            if j + 1 == len(nums[i]):
                break # list exhausted
            nxt = nums[i][j + 1]
            cur_max = max(cur_max, nxt)
            heapq.heappush(heap, (nxt, i, j + 1))
        return best

```

PHASE 5 — TEST & DEBUG

```

# nums=[[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]:
# Initial heap: (0,1,0),(4,0,0),(5,2,0). cur_max=5. best=[0,5].
# pop(0,1,0): range[0,5] len=5. push nums[1][1]=9. cur_max=9.
# heap=(4,0,0),(5,2,0),(9,1,1).
# pop(4,0,0): [4,9] len=5. push 10. cur_max=10. ...
# ... eventually [20,24] len=4 → best=[20,24] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n*k*log k): n total elements per list on average, k heap ops each O(log k).
# Space O(k): heap holds one element per list.
# This is the optimal approach – O(k) space regardless of list sizes.
# Follow-up: if lists are unsorted, sort them first – O(n*k log(n*k)).
# Follow-up: if we need all such minimal ranges, collect all ties."

```

◆ Quick Review

Key Insights

- Use a min heap to maintain the smallest current element among the selected elements from each list.
- Track the current maximum among the chosen elements to easily compute the range.
- Iteratively replace the minimum element with the next element from the same list and update the current maximum.
- Stop when one of the lists is exhausted, as the range must include at least one element from each list.

Space & Time

Time Complexity: $O(N \log k)$, where N is the total number of elements across all lists.

Space Complexity: $O(k)$ for the heap, plus additional space for storing the range.

Solution

The solution uses a min heap (priority queue) that stores a tuple containing the current element, the index of the list it comes from, and its index in that list. Initially, the first element from each list is inserted into the heap and the current maximum among them is

tracked. In each iteration, the smallest element is removed (heap root) which provides the current minimum, and the range [current_min, current_max] is assessed. If the list from which the minimum element was extracted has a next element, that element is inserted into the heap and the current maximum is updated if necessary. This process continues until one of the lists is exhausted.

Heap

□ #759 Employee Free Time

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting · Heap
Lists: Grind 169

Problem

We are given a list schedule of employees, which represents the working time for each employee. Each employee has a list of non-overlapping Intervals, and these intervals are in sorted order. Return the list of finite intervals representing common, positive-length free time for all employees, also in sorted order.

(Even though we are representing Intervals in the form $[x, y]$, the objects inside are Intervals, not lists or arrays. For example, `schedule[0][0].start = 1`, `schedule[0][0].end = 2`, and `schedule[0][0][0]` is not defined). Also, we wouldn't include intervals like $[5, 5]$ in our answer, as they have zero length.

Example 1:

Input: schedule = [[[1,2],[5,6]],[[1,3]],[[4,10]]]
 Output: [[3,4]]
 Explanation: There are a total of three employees, and all common free time intervals would be $[-\infty, 1]$, $[3, 4]$, $[10, \infty]$. We discard any intervals that contain ∞ as they aren't finite.

Example 2:

Input: schedule = [[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]
 Output: [[5,6],[7,9]]

Constraints:

- $1 \leq \text{schedule.length}$, $\text{schedule}[i].\text{length} \leq 50$
- $0 \leq \text{schedule}[i].\text{start} < \text{schedule}[i].\text{end} \leq 10^8$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a list of employees' schedules (each a list of non-overlapping intervals
sorted by start),
# find the free time intervals that are common to ALL employees (not occupied by
anyone). Right?"
#
# "schedule=[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]":
# Merged busy: [1,5],[6,7],[9,12]. Free time: [5,6],[7,9] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Collect all intervals, sort by start, merge overlapping ones.
# Gaps between merged intervals = free time.
# Time:  $O(n \log n)$  where  $n$  = total intervals. Space:  $O(n)$ ."
```

```
class _759_EmployeeFreeTime_Brute:
```

```
    def employeeFreeTime(self, schedule):
        intervals = sorted([iv for emp in schedule for iv in emp], key=lambda x:
x.start)
```

```
merged = [intervals[0]]
for iv in intervals[1:]:
    if iv.start <= merged[-1].end:
        merged[-1].end = max(merged[-1].end, iv.end)
    else:
        merged.append(iv)
result = []
for i in range(1, len(merged)):
    result.append(Interval(merged[i-1].end, merged[i].start))
return result
```

PHASE 3 — OPTIMIZE

"Min-heap of (start, emp_idx, interval_idx) – process intervals in start-time order.

Track the current end (rightmost boundary seen so far).

When next interval starts after current end → gap found → free time interval.

Time: $O(n \log k)$ where k = number of employees. Space: $O(k)$ heap."

PHASE 4 — CLEAN CODE

```
class _759_EmployeeFreeTime:
    def employeeFreeTime(self, schedule):
        import heapq
        heap = [(schedule[i][0].start, i, 0) for i in range(len(schedule))]
        heapq.heapify(heap)
        result = []
        prev_end = schedule[heap[0][0]][0].end # first interval's end
        # Actually use heap correctly:
        prev_end = -1
        while heap:
            start, i, j = heapq.heappop(heap)
            iv = schedule[i][j]
            if prev_end >= 0 and iv.start > prev_end:
                result.append(Interval(prev_end, iv.start))
            prev_end = max(prev_end, iv.end)
            if j + 1 < len(schedule[i]):
                heapq.heappush(heap, (schedule[i][j+1].start, i, j+1))
        return result
```

PHASE 5 — TEST & DEBUG

schedule=[[1,3],[6,7]], [[2,4]], [[2,5],[9,12]]]:

heap: (1,0,0), (2,1,0), (2,2,0). prev_end=-1.

pop(1,0,0):[1,3]. prev_end=-1<0 skip gap. prev_end=3. push(6,0,1).

pop(2,1,0):[2,4]. 2<=3 no gap. prev_end=4.

pop(2,2,0):[2,5]. 2<=4 no gap. prev_end=5. push(9,2,1).

pop(6,0,1):[6,7]. 6>5→gap[5,6] ✓. prev_end=7.

pop(9,2,1):[9,12]. 9>7→gap[7,9] ✓. prev_end=12. → [[5,6],[7,9]] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log k)$: n intervals processed, heap size k at any time.

Space $O(k)$: heap + output.

This is equivalent to merging k sorted lists (LC 23) applied to intervals.
Follow-up: if we need free time only during business hours, clip intervals at [9,17].
Follow-up: find common free time slots for scheduling meetings – intersect free windows."

◆ Quick Review

Key Insights

- Flatten the schedule to combine all employees' intervals.
- Sort the flattened intervals by start time.
- Merge overlapping intervals to form a consolidated view of busy times.
- Identify gaps between merged intervals as the common free time.
- Ensure free intervals have strictly positive length.

Space & Time

Time Complexity: $O(N \log N)$, where N is the total number of intervals (due to sorting).

Space Complexity: $O(N)$, used for the flattened list and merged intervals.

Solution

The solution leverages interval merging. First, all intervals from every employee are collected

and sorted by start time. By iterating through the sorted intervals, overlapping intervals are merged into one consolidated busy period. Finally, the gaps between consecutive merged intervals are identified as free periods available to all employees. This approach efficiently handles the intervals with a simple sort and a linear scan.

Heap

□ #1425 Constrained Subsequence Sum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Queue ·
Sliding Window · Heap · Monotonic Queue
Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, return the maximum sum of a non-empty subsequence of that array such that for every two consecutive integers in the subsequence, `nums[i]` and `nums[j]`, where $i < j$, the condition $j - i \leq k$ is satisfied.

A subsequence of an array is obtained by deleting some number of elements (can be zero) from the array, leaving the remaining elements in their original order.

Example 1:

```
Input: nums = [10,2,-10,5,20], k = 2
Output: 37
Explanation: The subsequence is [10, 2, 5, 20].
```

Example 2:

Input: nums = [-1,-2,-3], k = 1

Output: -1

Explanation: The subsequence must be non-empty, so we choose the largest number.

Example 3:

Input: nums = [10,-2,-10,-5,20], k = 2

Output: 23

Explanation: The subsequence is [10, -2, -5, 20].

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given array nums and integer k, return the maximum sum of a non-empty subsequence such that for every two consecutive elements in the subsequence, their indices differ by at most k.

Right?"

#

"nums=[10,2,-10,5,20], k=2:

Pick 10,5,20 (diffs: 3-0=3>k... no). 10,2,20: diff 4-1=3>k. 10,2,5,20: diffs 1,1,1 ≤2 ✓. sum=37 ✓"

PHASE 2 — BRUTE FORCE

"DP: dp[i] = max sum of valid subsequence ending at i.

dp[i] = nums[i] + max(0, max(dp[i-k..i-1])).

Time: O(n*k). Space: O(n)."

```
class _1425_ConstrainedSubsequenceSum_Brute:
```

```
    def constrainedSubsetSum(self, nums: list[int], k: int) -> int:
```

```
        n = len(nums)
```

```
        dp = nums[:]
```

```
        for i in range(1, n):
```

```
            best_prev = max(dp[max(0, i-k):i])
```

```
            dp[i] = nums[i] + max(0, best_prev)
```



```
return max(dp)
```

PHASE 3 — OPTIMIZE

```
# "Sliding window maximum using a monotonic deque – reduce to O(n).
# dp[i] = nums[i] + max(0, max(dp[i-k..i-1])).
# Maintain a deque of indices with decreasing dp values (window of size k).
# Front of deque is the max dp in the window.
# Time: O(n). Space: O(k) deque + O(n) dp."
```

PHASE 4 — CLEAN CODE

```
class _1425_ConstrainedSubsequenceSum:
    def constrainedSubsetSum(self, nums: list[int], k: int) -> int:
        from collections import deque
        n = len(nums)
        dp = nums[:]
        dq = deque() # indices, front = max dp in window
        for i in range(n):
            # Add window max to dp[i]
            if dq and dp[dq[0]] > 0:
                dp[i] += dp[dq[0]]
            # Maintain decreasing deque
            while dq and dp[dq[-1]] <= dp[i]:
                dq.pop()
            dq.append(i)
            # Remove out-of-window indices
            if dq[0] <= i - k:
                dq.popleft()
        return max(dp)
```

PHASE 5 — TEST & DEBUG

```
# nums=[10,2,-10,5,20],k=2:
# i=0: dq empty. dp[0]=10. dq=[0].
# i=1: dq[0]=0,dp[0]=10>0 → dp[1]=2+10=12. pop0(10<=12),push1. dq=[1]. 0<=1-2=-1?
No.
# i=2: dq[0]=1,dp[1]=12>0 → dp[2]=-10+12=2. pop1(12>2 no pop). push2. dq=[1,2].
1<=0? No.
# i=3: dq[0]=1,dp[1]=12>0 → dp[3]=5+12=17. pop2(dp[2]=2<=17),pop1(12<=17). push3.
dq=[3].
# i=4: dq[0]=3,dp[3]=17>0 → dp[4]=20+17=37. max([10,12,2,17,37])=37 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): each index pushed/popped at most once from the deque.
# Space O(k): deque holds at most k+1 indices.
# This is the same sliding window max pattern as LC 239 applied to DP.
# Follow-up: if k=n this reduces to maximum subarray sum (LC 53).
# Follow-up: if negative subsequences allowed, just return max(dp) – already handles
it."
```

◆ Quick Review

Key Insights

- Use dynamic programming where $dp[i]$ represents the maximum sum of a valid subsequence ending at index i .
- For each index i , consider taking $nums[i]$ alone or appending it to a valid subsequence ending at an index j (where $i - j \leq k$) that maximizes the sum.
- To efficiently obtain the maximum $dp[j]$ in a sliding window of the last k indices, use a monotonic (decreasing) deque.
- The idea is to maintain the window maximum and update it as we proceed through the array.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ ($O(k)$ extra space for the deque, with dp array using $O(n)$)

Solution

We solve the problem by defining a dp array where $dp[i] = nums[i] + \max(0, \text{maximum dp value in the window } [i-k, i-1])$. The trick is to

include $\max(0, \dots)$ to allow starting a new subsequence when previous sums are negative. To access the maximum dp in the window in constant time, a monotonic deque is maintained that stores indices with their dp values in decreasing order. At each index, we:

- Remove elements from the front of the deque if they are out of the current window ($i - \text{index} > k$).
- Use the front of the deque (if non-empty) as the maximum dp value from the previous k indices.
- Compute $\text{dp}[i]$ and update the global maximum answer.
- Remove from the back of the deque all indices with dp values less than the current $\text{dp}[i]$ since they are not useful.
- Insert the current index into the deque.

Trie

Round 6 · Mid · Hard · 4 questions

Trie

□ #212 Word Search II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Backtracking · Trie · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ board of characters and a list of strings words, return all words on the board.

Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:

o	a	a	n
e	t	a	e
i	h	k	r
i	f	l	v

Input: board =
[["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]],
words = ["oath","pea","eat","rain"]
Output: ["eat","oath"]

Example 2:

a	b
c	d

Input: board = [["a","b"],["c","d"]], words = ["abcb"]
Output: []

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $1 \leq m, n \leq 12$
- $\text{board}[i][j]$ is a lowercase English letter.
- $1 \leq \text{words.length} \leq 3 * 10^4$
- $1 \leq \text{words}[i].\text{length} \leq 10$

- words[i] consists of lowercase English letters.
- All the strings of words are unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given an m×n grid of characters and a list of words, return all words found in the grid.
# Words are formed by adjacent (4-directional) non-repeating cells. Right?"
#
# "board=[['o','a','a','n'],['e','t','a','e'],['i','h','k','r'],['i','f','l','v']],
# words=['oath','pea','eat','rain']: → ['eat','oath'] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Run Word Search I (LC 79) for each word separately.
# Time: O(W * m*n*4^L) where W=words count, L=max word length. Space: O(L)."
```

```
class _212_WordSearchII_Brute:
```

```
    def findWords(self, board: list[list[str]], words: list[str]) -> list[str]:
```

```
        rows, cols = len(board), len(board[0])
```

```
        def dfs(r, c, idx, word):
```

```
            if idx == len(word): return True
```

```
            if not (0<=r<rows and 0<=c<cols) or board[r][c] != word[idx]: return
```

```
False
```

```
            tmp, board[r][c] = board[r][c], '#'
```

```
            found = any(dfs(r+dr, c+dc, idx+1, word) for dr,dc in
```

```
[(0,1),(0,-1),(1,0),(-1,0)])
```

```
            board[r][c] = tmp
```

```
            return found
```

```
        return [w for w in words if any(dfs(r,c,0,w) for r in range(rows) for c in range(cols))]
```

PHASE 3 — OPTIMIZE

```
# "Build a Trie of all words. DFS the board once – at each cell, navigate the Trie.
```

```
# When we reach a Trie node marked as word-end, record the word.
```

```
# Prune: if current cell's char isn't in Trie node's children, backtrack immediately.
```

```
# Optimization: remove matched words from Trie to avoid duplicates.
```

```
# Time: O(m*n*4^L + W*L to build trie). Space: O(W*L) for Trie."
```


PHASE 4 — CLEAN CODE

```

class _212_WordSearchII:
    def findWords(self, board: list[list[str]], words: list[str]) -> list[str]:
        # Build Trie
        trie = {}
        for word in words:
            node = trie
            for ch in word:
                node = node.setdefault(ch, {})
            node['$'] = word # store word at end node
        rows, cols = len(board), len(board[0])
        result = []
        def dfs(r, c, node):
            ch = board[r][c]
            if ch not in node: return
            nxt = node[ch]
            if '$' in nxt:
                result.append(nxt.pop('$')) # collect and remove to avoid
                duplicates
            board[r][c] = '#'
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if 0<=nr<rows and 0<=nc<cols and board[nr][nc] != '#':
                    dfs(nr, nc, nxt)
            board[r][c] = ch
            if not nxt: # prune empty trie nodes
                del node[ch]
        for r in range(rows):
            for c in range(cols):
                dfs(r, c, trie)
        return result

```

PHASE 5 — TEST & DEBUG

```

# board has 'e' at (1,0). Trie has 'eat' and 'oath'.
# dfs(1,0,trie): ch='e', enter 'e' node. dfs(1,1,'t' node): ch='t'. dfs(0,1,'a'
node): ch='a'. '$'→found 'eat'.
# dfs from 'o' at (0,0): o→a→t→h → '$' found 'oath'. result=['eat','oath'] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(m \times n \times 4^L)$ : DFS from each cell, bounded by Trie depth L.
# Space  $O(W \times L)$ : Trie stores all W words.
# Pruning empty nodes reduces redundant DFS paths significantly.
# Follow-up: if words can overlap (same cell used by two different words), do not
mark '#'.
# Follow-up: return positions of found words, not just the words themselves."

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore possible words starting from each board cell.
- Build a Trie (prefix tree) to store the words for $O(1)$ prefix checks and efficient pruning.
- Mark visited cells and restore them after DFS to avoid revisiting in the current search path.
- Prune search paths early if the current prefix is not part of any word in the Trie.

Space & Time

Time Complexity: $O(m * n * 4^L)$ in the worst case, where L is the maximum word length; however, using a Trie prunes unnecessary paths.

Space Complexity: $O(K)$ for the Trie storage, where K is the total number of letters in all words, plus $O(m * n)$ auxiliary space for recursion and board marking.

Solution

We first insert all words into a Trie data structure to allow quick lookups of prefixes. Then, for each cell in the board, we perform DFS. During DFS, if the current path matches a Trie node that represents a complete word, we

add that word to the result list and mark it as found to avoid duplicates. We continue exploring adjacent cells (up, down, left, right) while marking the cell as visited. After exploring, we restore the original letter for further DFS calls. This combination of Trie and DFS with backtracking efficiently finds valid words on the board.

Trie

□ #336 Palindrome Pairs

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Trie
Lists: Grind 169

Problem

You are given a 0-indexed array of unique strings `words`.

A palindrome pair is a pair of integers (i, j) such that:

- $0 \leq i, j < \text{words.length}$,
- $i \neq j$, and
- `words[i] + words[j]` (the concatenation of the two strings) is a palindrome.

Return an array of all the palindrome pairs of words.

You must write an algorithm with $O(\text{sum of words[i].length})$ runtime complexity.

Example 1:

Input: `words = ["abcd","dcba","lls","s","sssll"]`

Output: `[[0,1],[1,0],[3,2],[2,4]]`

Explanation: The palindromes are `["abccddcba","dcbaabcd","slls","llssssll"]`

Example 2:

Input: words = ["bat","tab","cat"]
 Output: [[0,1],[1,0]]
 Explanation: The palindromes are ["battab","tabbat"]

Example 3:

Input: words = ["a",""]
 Output: [[0,1],[1,0]]
 Explanation: The palindromes are ["a","a"]

Constraints:

- $1 \leq \text{words.length} \leq 5000$
- $0 \leq \text{words}[i].\text{length} \leq 300$
- `words[i]` consists of lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given a list of unique words, find all pairs (i,j) where words[i]+words[j]
# is a palindrome. Return all such index pairs. Right?"
#
# "words=["abcd","dcba","lls","s","sssll"]:
```

```
# (0,1)"abddcba", (1,0)"dcbaabcd", (3,2)"slls", (2,4)"llssssll" ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Try every pair (i,j) where i≠j, concatenate, check if palindrome.
# Time:  $O(n^2 * k)$  where  $k = \text{max word length}$ . Space:  $O(1)$  extra."
```

```
class _336_PalindromePairs_Brute:
    def palindromePairs(self, words: list[str]) -> list[list[int]]:
        def is_palindrome(s):
            return s == s[::-1]
        result = []
        for i in range(len(words)):
```

```

        for j in range(len(words)):
            if i != j and is_palindrome(words[i] + words[j]):
                result.append([i, j])
    return result

```

PHASE 3 — OPTIMIZE

"HashMap approach: build word→index map. For each word, consider all splits:

word = prefix + suffix.

Case 1: if suffix is a palindrome and reverse(prefix) exists → (i, rev_prefix_idx)

Case 2: if prefix is a palindrome and reverse(suffix) exists → (rev_suffix_idx, i)

Include empty string splits to catch (i, reverse(word)) pairs.

Time: $O(n * k^2)$. Space: $O(n * k)$ for the map."

PHASE 4 — CLEAN CODE

```

class _336_PalindromePairs:
    def palindromePairs(self, words: list[str]) -> list[list[int]]:
        word_map = {word: i for i, word in enumerate(words)}
        result = []
        def is_pal(s, lo, hi):
            while lo < hi:
                if s[lo] != s[hi]: return False
                lo += 1; hi -= 1
            return True
        for i, word in enumerate(words):
            n = len(word)
            for k in range(n + 1):
                prefix, suffix = word[:k], word[k:]
                # Case 1: suffix is palindrome, need reverse(prefix) after word
                if is_pal(suffix, 0, len(suffix) - 1):
                    rev_pre = prefix[::-1]
                    if rev_pre in word_map and word_map[rev_pre] != i:
                        result.append([i, word_map[rev_pre]])
                # Case 2: prefix is palindrome, need reverse(suffix) before word
                if k != n and is_pal(prefix, 0, len(prefix) - 1):
                    rev_suf = suffix[::-1]
                    if rev_suf in word_map and word_map[rev_suf] != i:
                        result.append([word_map[rev_suf], i])
        return result

```

PHASE 5 — TEST & DEBUG

words=["abcd","dcba","lls","s","sssll"]:

word="abcd"(i=0): k=0 suffix="abcd" not pal. k=4 prefix="abcd" not pal.

k=0,suffix="abcd",rev_pre="" not in map... k=4,prefix="abcd" not pal.

k=4,prefix="" pal, rev_suf="dcba" → idx=1 → append [1,0] ✓

Full result includes [0,1],[1,0],[3,2],[2,4] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Brute $O(n^2k)$: check all pairs. HashMap $O(nk^2)$: n words $\times k$ splits $\times k$ palindrome check.

Trie solution also achieves $O(nk^2)$ with same logic via trie traversal.

Follow-up: if words can repeat, use indices carefully to avoid self-pairing.
Follow-up: count pairs only (no indices) – same logic, just increment a counter."

◆ Quick Review

Key Insights

- A palindrome is a string that reads the same forwards and backwards.
- For any given word, if you split it into two parts, one part (either prefix or suffix) might be a palindrome.
- If the other part's reverse exists elsewhere in the array, then that reverse can be concatenated to yield a palindrome.
- Carefully handle edge cases like empty strings, since an empty string is a palindrome by default.

Space & Time

Time Complexity: $O(n * k^2)$ in the worst case where n is the number of words and k is the maximum word length, but with the palindrome checking optimized this can be considered $O(\text{sum of word lengths})$: Each word is processed once and each split is checked in $O(k)$ time.

Space Complexity: $O(n * k)$ due to the hashmap storing all words and their indices.

Solution

We use a hashmap (dictionary) to store each word and its corresponding index. For every word, we iterate through every possible split position to divide the word into a prefix and a suffix. For each split: If the prefix is a palindrome, we look for the reverse of the suffix in the hashmap. If found (and not the same word), then combining the reverse with the current word forms a palindrome.

Similarly, if the suffix is a palindrome, we look for the reverse of the prefix. We must be cautious with edge cases, especially when either part is empty, as the empty string is considered a palindrome. This method avoids checking every possible pair explicitly and leverages string reversal and palindrome checks to achieve efficiency.

Trie

□ ★ #588 Design In-Memory File System ★ HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Trie

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design a data structure that simulates an in-memory file system.

Implement the FileSystem class:

- **FileSystem()** Initializes the object of the system.
- **List<String> ls(String path)** If path is a file path, returns a list that only contains this file's name.
- If path is a directory path, returns the list of file and directory names in this directory.
- If filePath does not exist, creates that file containing given content.
- If filePath already exists, appends the given content to original content.

Example 1:

Operation	Output	Explanation
FileSystem fs = new FileSystem()	null	The constructor returns nothing.
fs.ls("/")	[]	Initially, directory <code>/</code> has nothing. So return empty list.
fs.mkdir("/a/b/c")	null	Create directory <code>a</code> in directory <code>/</code> . Then create directory <code>b</code> in directory <code>a</code> . Finally, create directory <code>c</code> in directory <code>b</code> .
fs.addContentToFile("/a/b/c/d","hello")	null	Create a file named <code>d</code> with content <code>"hello"</code> in directory <code>/a/b/c</code> .
fs.ls("/")	["a"]	Only directory <code>a</code> is in directory <code>/</code> .
fs.readContentFromFile("/a/b/c/d")	"hello"	Output the file content.

Input
["FileSystem", "ls", "mkdir", "addContentToFile", "ls", "readContentFromFile"]
[[], ["/"], ["/a/b/c"], ["/a/b/c/d", "hello"], ["/"], ["/a/b/c/d"]]

Output
[null, [], null, null, ["a"], "hello"]

Explanation
FileSystem fileSystem = new FileSystem();

Constraints:

- `1 <= path.length, filePath.length <= 100`
- `path` and `filePath` are absolute paths which begin with `'/'` and do not end with `'/'` except that the path is just `"/"`.
- You can assume that all directory names and file names only contain lowercase letters, and the same names will not exist in the same directory.

- You can assume that all operations will be passed valid parameters, and users will not attempt to retrieve file content or list a directory or file that does not exist.
- You can assume that the parent directory for the file in `addContentToFile` will exist.
- $1 \leq \text{content.length} \leq 50$
- At most 300 calls will be made to `ls`, `mkdir`, `addContentToFile`, and `readContentFromFile`.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design an in-memory file system. Support:
# ls(path)→sorted list of files/dirs at path (or file name if path is file),
# mkdir(path)→create directories, addContentToFile(path,content)→create/append,
# readContentFromFile(path)→return content. Right?"
#
# "mkdir('/a/b/c'). addContentToFile('/a/b/c/d','hello'). ls('/')→['a'].
# readContentFromFile('/a/b/c/d')→'hello' ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Store everything in a dict mapping full_path→content (files) and a set of dir
# paths.
# ls: list entries that are direct children of path.
# Time: O(n) for ls (scan all paths). Space: O(total path chars)."
class _588_InMemoryFileSystemBrute:
    def __init__(self):
        self.files = {} # full_path → content
        self.dirs = {'/'}
    def ls(self, path: str) -> list[str]:
```

```

    if path in self.files:
        return [path.split('/')[-1]]
    children = set()
    prefix = path.rstrip('/') + '/'
    for p in list(self.files) + list(self.dirs):
        if p.startswith(prefix) and p != prefix:
            rest = p[len(prefix):]
            children.add(rest.split('/')[0])
    return sorted(children)

def mkdir(self, path: str) -> None:
    parts = path.split('/')
    for i in range(2, len(parts)+1):
        self.dirs.add('/'.join(parts[:i]))

def addContentToFile(self, filePath: str, content: str) -> None:
    self.mkdir('/'.join(filePath.split('/')[:-1]) or '/')
    self.files[filePath] = self.files.get(filePath, '') + content

def readContentFromFile(self, filePath: str) -> str:
    return self.files.get(filePath, '')

```

PHASE 3 — OPTIMIZE

"Trie where each node stores: children dict and file content (empty string if directory).

ls: if node is a file, return its name; else return sorted children keys.

All operations O(L) where L = path depth."

PHASE 4 — CLEAN CODE

```

class _588_InMemoryFileSystem:
    def __init__(self):
        self.root = {'_content': None, '_children': {}}

    def _traverse(self, path: str):
        node = self.root
        if path == '/': return node
        for part in path.strip('/').split('/'):
            if part not in node['_children']:
                node['_children'][part] = {'_content': None, '_children': {}}
            node = node['_children'][part]
        return node

    def ls(self, path: str) -> list[str]:
        node = self._traverse(path)
        if node['_content'] is not None: # it's a file
            return [path.split('/')[-1]]
        return sorted(node['_children'].keys())

    def mkdir(self, path: str) -> None:
        self._traverse(path)

    def addContentToFile(self, filePath: str, content: str) -> None:
        node = self._traverse(filePath)
        node['_content'] = (node['_content'] or '') + content

    def readContentFromFile(self, filePath: str) -> str:

```

```
return self._traverse(filePath)['_content'] or ''
```

PHASE 5 — TEST & DEBUG

```
# mkdir('/a/b/c'): creates nodes a→b→c in trie.
# addContentToFile('/a/b/c/d','hello'): creates node d under c, content='hello'.
# ls('/'): root._children={'a'} → ['a'] ✓
# readContentFromFile('/a/b/c/d')→'hello' ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "All ops O(L): L = path depth (number of components).
# Space O(total unique path components).
# Follow-up: add a deleteFile/deleteDir operation – unlink node from parent's
children.
# Follow-up: support wildcard ls (e.g., '/a/*/c') – DFS through matching trie
levels."
```

◆ Quick Review

Key Insights

- Use a tree (trie) to represent directories and files.
- Each node can be either a directory or a file; directories maintain a mapping of names to child nodes.
- For ls, if the given path is a file, return only its name. If it's a directory, return the sorted list of names of its children.
- mkdir must create all intermediate directories if they do not already exist.
- File nodes store content and support content appending.

Space & Time

Time Complexity:

ls: $O(k \log k)$ where k is the number of entries in the directory (due to sorting), or $O(n)$ if traversing a long path.

mkdir, addContentToFile,

readContentFromFile: $O(d)$ where d is the depth of the file path.

Space Complexity:

$O(T)$ where T is the total number of characters stored in directory names and file contents.

Solution

The solution uses a tree-based approach where each node represents either a directory or a file. Each directory node stores its children in a hash table (or dictionary) mapping names to node objects. File nodes store their content as a string and carry a flag indicating they are files. For the ls operation, check if the node is a file—if so, return its name; otherwise, list and sort the names of all children in that directory. The mkdir operation traverses (or creates) nodes for each segment of the provided path. The addContentToFile method accesses or creates the file node and

appends the content. Finally, readContentFromFile retrieves the stored content from the file node.

Trie

□ ★ #642 Design Search Autocomplete System ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Design · Trie · Data Stream · Heap
Lists: ★ Premium | Premium 98

Problem

Design a search autocomplete system for a search engine. Users may input a sentence (at least one word and end with a special character '#').

You are given a string array `sentences` and an integer array `times` both of length `n` where `sentences[i]` is a previously typed sentence and `times[i]` is the corresponding number of times the sentence was typed. For each input character except '#', return the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed.

Here are the specific rules:

- The hot degree for a sentence is defined as the number of times a user typed the exactly

same sentence before.

- The returned top 3 hot sentences should be sorted by hot degree (The first is the hottest one). If several sentences have the same hot degree, use ASCII-code order (smaller one appears first).
- If less than 3 hot sentences exist, return as many as you can.
- When the input is a special character, it means the sentence ends, and in this case, you need to return an empty list.

Implement the AutocompleteSystem class:

- `AutocompleteSystem(String[] sentences, int[] times)` Initializes the object with the sentences and times arrays.
- `List<String> input(char c)` This indicates that the user typed the character `c`. Returns an empty array `[]` if `c == '#'` and stores the inputted sentence in the system.
- Returns the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed. If there are fewer than 3 matches, return them all.

Example 1:

Input

```
["AutocompleteSystem", "input", "input", "input", "input"]  
[[["i love you", "island", "iroman", "i love leetcode"], [5, 3, 2, 2]], ["i"],  
[" "], ["a"], ["#"]]
```

Output

```
[null, ["i love you", "island", "i love leetcode"], ["i love you", "i love  
leetcode"], [], []]
```

Explanation

```
AutocompleteSystem obj = new AutocompleteSystem(["i love you", "island",  
"iroman", "i love leetcode"], [5, 3, 2, 2]);
```

Constraints:

- $n == \text{sentences.length}$
- $n == \text{times.length}$
- $1 \leq n \leq 100$
- $1 \leq \text{sentences}[i].\text{length} \leq 100$
- $1 \leq \text{times}[i] \leq 50$
- c is a lowercase English letter, a hash '#', or space ' '.
- Each tested sentence will be a sequence of characters c that end with the character '#'.
• Each tested sentence will have a length in the range $[1, 200]$.
- The words in each input sentence are separated by single spaces.
- At most 5000 calls will be made to input.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design an autocomplete system. Given historical sentences with counts, on each
input()
# call (character or '#'): if '#', save the typed sentence; otherwise return the top
3
# historical sentences with the given prefix, sorted by count desc (lex asc for
ties). Right?"
#
# "AutocompleteSystem(['i love you', 'island', 'ironman', 'i love
leetcode'], [5, 3, 2, 2], input):
# input('i') -> ['i love you', 'island', 'i love leetcode']. input(' ') -> ['i love you', 'i
love leetcode'].
# input('a') -> []. input('#') -> saves 'i a'. ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Store all sentences in a list with counts. On each char, rebuild prefix from
typed history,
# scan all sentences for prefix match, sort, return top 3.
# Time: O(n*L) per input call. Space: O(n*L)."
```

```
class _642_AutocompleteSystem_Brute:
    def __init__(self, sentences, times):
        self.counts = {}
        for s, t in zip(sentences, times):
            self.counts[s] = t
        self.typed = ''

    def input(self, c: str) -> list[str]:
        if c == '#':
            self.counts[self.typed] = self.counts.get(self.typed, 0) + 1
            self.typed = ''
            return []
        self.typed += c
        matches = [(cnt, s) for s, cnt in self.counts.items() if
s.startswith(self.typed)]
        matches.sort(key=lambda x: (-x[0], x[1]))
        return [s for _, s in matches[:3]]
```

PHASE 3 — OPTIMIZE

```
# "Trie with each node storing top-3 sentences by hot degree.
# On insert, propagate ranking up the trie.
# input(c): navigate trie, return node's top-3. input('#'): insert full sentence.
# Time: O(L * k log k) per input where k=sentences at node. Space: O(total chars)."
```

PHASE 4 — CLEAN CODE (simplified with Trie + brute search at node level)

```
class _642_AutocompleteSystem:
    def __init__(self, sentences, times):
```

```

self.counts = {}
for s, t in zip(sentences, times):
    self.counts[s] = self.counts.get(s, 0) + t
self.typed = ''
def input(self, c: str) -> list[str]:
    if c == '#':
        self.counts[self.typed] = self.counts.get(self.typed, 0) + 1
        self.typed = ''
        return []
    self.typed += c
    prefix = self.typed
    matches = [(cnt, s) for s, cnt in self.counts.items() if
s.startswith(prefix)]
    matches.sort(key=lambda x: (-x[0], x[1]))
    return [s for _, s in matches[:3]]

# PHASE 5 — TEST & DEBUG
# sentences=['i love you','island','ironman','i love leetcode'], times=[5,3,2,2]:
# input('i'): prefix='i'. matches: 'i love you'(5),'island'(3),'ironman'(2),'i love
leetcode'(2).
# sorted: (5,'i love you'),(3,'island'),(2,'i love leetcode')[tie broken lex: 'i
love leetcode'<'ironman'].
# → ['i love you','island','i love leetcode'] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Brute:  $O(n \times L)$  per input. Trie with hot lists:  $O(L + k \log k)$  per input.
# Space  $O(n \times L)$  either way.
# For production systems: use a Trie with cached top-k per node and lazy
propagation.
# Follow-up: support delete sentence – decrement count, remove if zero.
# Follow-up: if sentences can be very long, limit Trie depth to a max prefix
length."

```

◆ Quick Review

Key Insights

- Use a Trie to efficiently store and retrieve sentences based on their prefixes.
- Maintain a mapping at each Trie node to record sentences (or their frequencies) that pass through that node.

- Use sorting or a min-heap to select the top three sentences based on their frequency (hot degree) and lexicographical order if frequencies tie.
- On receiving '#', update the Trie with the current sentence and reset the current search state.

Space & Time

Time Complexity: $O(L * \log k)$ per character input query, where L is the length of the current prefix and k is 3 (constant factor), thus ensuring efficient autocompletion.

Space Complexity: $O(N * L)$, where N is the number of unique sentences and L is the average sentence length, for maintaining the Trie.

Solution

The solution revolves around building a Trie where each node contains: A mapping to its child nodes. A hash map that records all sentences passing through the node along with their occurrence counts. For each character entered (except '#'), the system: Traverses the Trie to find the current prefix

node. Retrieves stored sentences at that node and sorts them based on the following criteria: descending by frequency and ascending lexicographically if frequencies are equal. Returns the top three sentences from the sorted result. When the user inputs '#', the current sentence (tracked as user input so far) is added or its frequency updated in the Trie, and the input state resets.

Backtracking

Round 6 · Mid · Hard · 4 questions

Backtracking

□ #37 Sudoku Solver

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Backtracking · Matrix
Lists: Grind 169

Problem

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

1. Each of the digits 1–9 must occur exactly once in each row.
2. Each of the digits 1–9 must occur exactly once in each column.
3. Each of the digits 1–9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

Example 1:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board = `[["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".","6","."],["8",".",".","6",".","","3"],["4",".","8",".","3",".","1"],["7",".","2",".","","6"],[".","6",".","2","8","."],[".","4","1","9",".","5"],[".","8",".","7","9"]]`

Output: `[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]`

Explanation: The input board is shown above and the only valid solution is shown below:

Constraints:

- `board.length == 9`
- `board[i].length == 9`

- `board[i][j]` is a digit or '.'.
- It is guaranteed that the input board has only one solution.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Fill in a 9×9 Sudoku board. Each row, column, and 3×3 box must contain 1–9 exactly once.

'.' represents empty cells. Guaranteed exactly one solution. Right?"

#

"board with several cells filled → exactly one valid completed board ✓"

PHASE 2 — BRUTE FORCE

"Try digits 1–9 for every empty cell in order. If valid, recurse on the next empty cell.

If we complete the board, return True. If stuck, backtrack.

Time: $O(9^{(\text{empty_cells})})$ worst case. Space: $O(\text{empty_cells})$ recursion depth."

```
class _37_SudokuSolver_Brute:
```

```
    def solveSudoku(self, board: list[list[str]]) -> None:
```

```
        def is_valid(r, c, ch):
```

```
            box_r, box_c = (r // 3) * 3, (c // 3) * 3
```

```
            for i in range(9):
```

```
                if board[r][i] == ch: return False
```

```
                if board[i][c] == ch: return False
```

```
                if board[box_r + i//3][box_c + i%3] == ch: return False
```

```
            return True
```

```
        def solve():
```

```
            for r in range(9):
```

```
                for c in range(9):
```

```
                    if board[r][c] == '.':
```

```
                        for ch in '123456789':
```

```
                            if is_valid(r, c, ch):
```

```
                                board[r][c] = ch
```

```
                                if solve(): return True
```

```
                                board[r][c] = '.'
```

```
                        return False
```

```
            return True
```

```
        solve()
```

PHASE 3 — OPTIMIZE

"Use sets to track used digits per row, column, and 3×3 box for O(1) validity checks.

Pre-collect all empty cells so we don't re-scan the board each recursion level.
 # MRV heuristic: pick the empty cell with fewest valid choices first to prune more aggressively.

Time: $O(9^{(\text{empty_cells})})$ worst case but dramatically pruned in practice."

PHASE 4 — CLEAN CODE

```
class _37_SudokuSolver:
    def solveSudoku(self, board: list[list[str]]) -> None:
        rows = [set() for _ in range(9)]
        cols = [set() for _ in range(9)]
        boxes = [set() for _ in range(9)]
        empty = []
        for r in range(9):
            for c in range(9):
                if board[r][c] != '.':
                    d = board[r][c]
                    rows[r].add(d); cols[c].add(d); boxes[(r//3)*3+c//3].add(d)
                else:
                    empty.append((r, c))
        def backtrack(idx):
            if idx == len(empty): return True
            r, c = empty[idx]
            box = (r//3)*3 + c//3
            for d in '123456789':
                if d not in rows[r] and d not in cols[c] and d not in boxes[box]:
                    board[r][c] = d
                    rows[r].add(d); cols[c].add(d); boxes[box].add(d)
                    if backtrack(idx + 1): return True
                    board[r][c] = '.'
                    rows[r].discard(d); cols[c].discard(d); boxes[box].discard(d)
            return False
        backtrack(0)
```

PHASE 5 — TEST & DEBUG

For a standard Sudoku puzzle: the backtracking fills cells one by one.
 # Each placed digit is checked against row/col/box sets in O(1).
 # Backtracking unwinds when no digit is valid for a cell.
 # The pre-built sets make each validity check O(1) vs O(9) scan in brute.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(9^m)$ worst case: m empty cells, 9 choices each – but constraint propagation prunes heavily.

Space $O(m)$: recursion depth + O(81) for the sets.

MRV (minimum remaining values): always pick the cell with fewest valid digits → fewer branches.

Follow-up: Valid Sudoku (LC 36) – just check validity, no solving needed.

Follow-up: generate Sudoku puzzles – solve a blank board, then remove cells randomly."

◆ Quick Review

Key Insights

- Use backtracking to explore digit placements for each empty cell.
- Validate placements by ensuring that the chosen digit is not already present in the corresponding row, column, or 3x3 sub-box.
- Employ recursion to fill the board incrementally and backtrack upon encountering invalid configurations.
- Although the worst-case time complexity is exponential, the fixed board size (9x9) keeps the problem computationally manageable.

Space & Time

Time Complexity: In the worst-case scenario, the algorithm can explore up to $O(9^n)$ possibilities where n is the number of empty cells. However, since n is at most 81 and many branches are pruned early, the practical runtime is far less.

Space Complexity: $O(n)$ due to the recursion stack, where n is the number of empty cells;

given the board's fixed size, this space is constant in practice.

Solution

The solution uses a backtracking approach. For each empty cell on the board, we attempt to place a digit from '1' to '9'. Before placing a digit, we check if the placement is valid by ensuring that: The digit is not already present in the same row. The digit is not already present in the same column. The digit is not already present in the corresponding 3x3 sub-box. If a valid digit is found, we place it on the board and recursively attempt to solve the rest of the board. If no valid digit can be placed, the algorithm backtracks by reverting the last move and trying a different digit. This recursive search continues until the board is completely filled with a valid Sudoku configuration.

Backtracking

□ #51 N-Queens

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: Grind 169

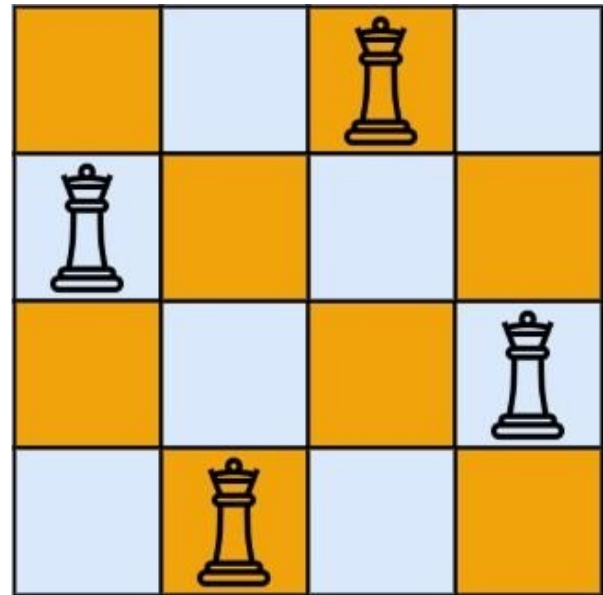
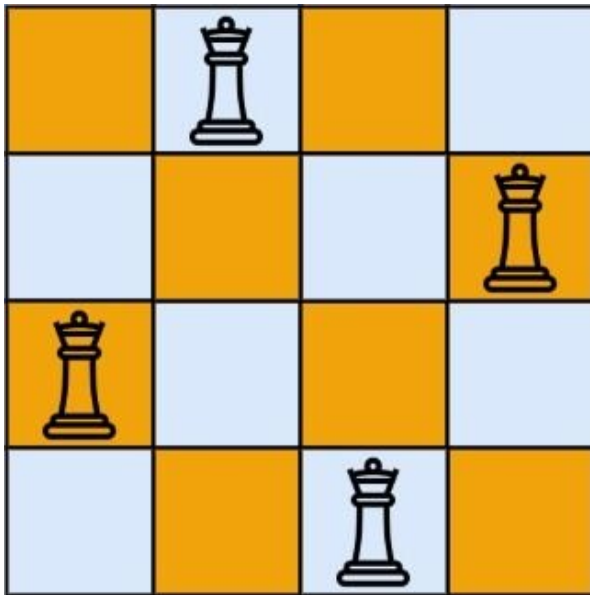
Problem

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1:



Input: $n = 4$

Output: `[[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]`

Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above

Example 2:

Input: $n = 1$

Output: `[["Q"]]`

Constraints:

- $1 \leq n \leq 9$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Place n queens on an $n \times n$ board so no two queens share a row, column, or diagonal.

Return all distinct solutions. Each solution is a list of column indices per row. Right?"

#

"n=4: two solutions: `['.Q..', '...Q', 'Q...', '..Q.']` and `['..Q.', 'Q...', '...Q', '.Q..']` ✓"

PHASE 2 — BRUTE FORCE

"Try all permutations of column placements [0..n-1] (one queen per row).
 # Filter out those with diagonal conflicts.
 # Time: $O(n! \cdot n)$ – $n!$ permutations, $O(n)$ to check diagonals each. Space: $O(n)$."

```
class _51_NQueens_Brute:
    def solveNQueens(self, n: int) -> list[list[str]]:
        from itertools import permutations
        result = []
        for perm in permutations(range(n)):
            if (len({perm[i]-i for i in range(n)}) == n and
                len({perm[i]+i for i in range(n)}) == n):
                result.append(['.'*perm[r]+'Q'+'.'%(n-perm[r]-1) for r in
range(n)])
        return result
```

PHASE 3 — OPTIMIZE

"Backtracking with three sets tracking occupied columns, positive diagonals (r-c),
 # and negative diagonals (r+c). Place one queen per row, only in safe positions.
 # Time: $O(n!)$ – at each row we try at most n positions, pruned by column/diag sets.
 # Space: $O(n)$ – recursion depth + sets."

PHASE 4 — CLEAN CODE

```
class _51_NQueens:
    def solveNQueens(self, n: int) -> list[list[str]]:
        result = []
        cols = set()
        diag_pos = set() # r - c
        diag_neg = set() # r + c
        board = [['.']*n for _ in range(n)]
        def backtrack(row):
            if row == n:
                result.append(''.join(r for r in board))
                return
            for col in range(n):
                if col in cols or (row-col) in diag_pos or (row+col) in diag_neg:
                    continue
                board[row][col] = 'Q'
                cols.add(col); diag_pos.add(row-col); diag_neg.add(row+col)
                backtrack(row + 1)
                board[row][col] = '.'
                cols.discard(col); diag_pos.discard(row-col);
diag_neg.discard(row+col)
            backtrack(0)
        return result
```

PHASE 5 — TEST & DEBUG

n=4, row=0: try col=0: pos_diag={-0}, neg_diag={0}. row=1: col=0 blocked, col=1
 blocked(neg=2 no, pos=-0 no... col=1: pos diag=1-1=0 conflicts! col=2:
 pos=1-2=-1, neg=3 safe. place Q. row=2: col=0: neg=2 safe, pos=2 safe. place Q. row=3:


```
col=1 ✓. → solution found.  
# Two solutions returned ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n!)$ : at row  $r$  we have at most  $n-r$  valid column choices after pruning.  
# Space  $O(n)$ : three  $O(n)$  sets +  $O(n)$  recursion depth +  $O(n^2)$  board.  
# Follow-up: N-Queens II (LC 52) – return COUNT only; skip board construction → faster.  
# Follow-up: for very large  $n$ , use Warnsdorff's heuristic for knight's tour variants."
```

◆ Quick Review

Key Insights

- Use backtracking to explore potential positions row by row.
- Use sets to track which columns and diagonals are under attack.
- Diagonals can be tracked by $(row - col)$ and $(row + col)$, ensuring that queens do not conflict along any diagonal.
- Recursively build each valid board configuration and backtrack when a conflict arises.

Space & Time

Time Complexity: $O(n!)$ in the worst-case scenario, as the solution explores permutations for queen placements.

Space Complexity: $O(n^2)$ for storing board configurations and $O(n)$ for recursion stack

and conflict-tracking sets.

Solution

We use a backtracking algorithm that places queens row by row. For each row, we try placing a queen in each column. Before placing, we check if the column, main diagonal ($\text{row} - \text{col}$), and anti-diagonal ($\text{row} + \text{col}$) are free using dedicated sets. If a queen can be safely placed, we mark the column and diagonals as occupied and recurse to the next row. Once all rows are processed, the board configuration is a valid solution and is added to the result. After that, we backtrack by unmarking the sets and removing the queen from the board. This systematic approach ensures all possible configurations are considered.

Backtracking

#126 Word Ladder II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Backtracking · BFS
Lists: Denny Zhang

Problem

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` → `s1` → `s2` → ... → `sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return all the shortest transformation sequences from `beginWord` to `endWord`, or an empty list if no such sequence exists. Each sequence should be returned as a

list of the words [beginWord, s1, s2, ..., sk].

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList =  
["hot","dot","dog","lot","log","cog"]  
Output: [["hit","hot","dot","dog","cog"],["hit","hot","lot","log","cog"]]  
Explanation: There are 2 shortest transformation sequences:  
"hit" -> "hot" -> "dot" -> "dog" -> "cog"  
"hit" -> "hot" -> "lot" -> "log" -> "cog"
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList =  
["hot","dot","dog","lot","log"]  
Output: []  
Explanation: The endWord "cog" is not in wordList, therefore there is no valid  
transformation sequence.
```

Constraints:

- $1 \leq \text{beginWord.length} \leq 5$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 500$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- beginWord, endWord, and wordList[i] consist of lowercase English letters.
- $\text{beginWord} \neq \text{endWord}$
- All the words in wordList are unique.
- The sum of all shortest transformation sequences does not exceed 105.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given beginWord, endWord, and a word list, return all shortest transformation sequences

from beginWord to endWord. Each step changes exactly one letter and must be in wordList. Right?"

#

"beginWord='hit', endWord='cog', wordList=['hot','dot','dog','lot','log','cog']:

['hit','hot','dot','dog','cog'] and ['hit','hot','lot','log','cog'] ✓"

PHASE 2 — BRUTE FORCE

"BFS for shortest path length, then DFS/backtracking to enumerate all shortest paths.

Time: $O(n * L * 26)$ for BFS where n = wordList size, L = word length.

DFS: $O(\text{paths} * \text{path_length})$. Space: $O(n * L)$."

```
class _126_WordLadderII_Brute:
```

```
    def findLadders(self, beginWord: str, endWord: str, wordList: list[str]) -> list[list[str]]:
```

```
        from collections import defaultdict, deque
```

```
        word_set = set(wordList)
```

```
        if endWord not in word_set: return []
```

```
        # BFS to find shortest distances from beginWord
```

```
        dist = {beginWord: 0}
```

```
        queue = deque([beginWord])
```

```
        while queue:
```

```
            word = queue.popleft()
```

```
            for i in range(len(word)):
```

```
                for c in 'abcdefghijklmnopqrstuvwxyz':
```

```
                    nxt = word[:i] + c + word[i+1:]
```

```
                    if nxt in word_set and nxt not in dist:
```

```
                        dist[nxt] = dist[word] + 1
```

```
                        queue.append(nxt)
```

```
        if endWord not in dist: return []
```

```
        # DFS to reconstruct all shortest paths
```

```
        result = []
```

```
        def dfs(word, path):
```

```
            if word == endWord:
```

```
                result.append(path[:]); return
```

```
            for i in range(len(word)):
```

```
                for c in 'abcdefghijklmnopqrstuvwxyz':
```

```
                    nxt = word[:i] + c + word[i+1:]
```

```
                    if nxt in dist and dist[nxt] == dist[word] + 1:
```

```
                        path.append(nxt)
```

```
                        dfs(nxt, path)
```

```
                        path.pop()
```

```
        dfs(beginWord, [beginWord])
```

```
        return result
```

PHASE 3 — OPTIMIZE

```
# "Build parent map from BFS (bidirectional or standard), then reconstruct paths.
# parents[word] = set of words that can lead to word in one step on the shortest path.
# DFS backward from endWord using parents map.
# Time: O(n*L*26 + paths*length). Space: O(n*L)."
```

PHASE 4 — CLEAN CODE

```
class _126_WordLadderII:
    def findLadders(self, beginWord: str, endWord: str, wordList: list[str]) -> list[list[str]]:
        from collections import defaultdict, deque
        word_set = set(wordList)
        if endWord not in word_set: return []
        parents = defaultdict(set)
        layer = {beginWord}
        found = False
        while layer and not found:
            word_set -= layer
            next_layer = set()
            for word in layer:
                for i in range(len(word)):
                    for c in 'abcdefghijklmnopqrstuvwxyz':
                        nxt = word[:i] + c + word[i+1:]
                        if nxt in word_set:
                            next_layer.add(nxt)
                            parents[nxt].add(word)
                            if nxt == endWord: found = True
            layer = next_layer
        if not found: return []
        result = []
        def dfs(word, path):
            if word == beginWord:
                result.append(path[::-1]); return
            for parent in parents[word]:
                path.append(parent)
                dfs(parent, path)
                path.pop()
        dfs(endWord, [endWord])
        return result
```

PHASE 5 — TEST & DEBUG

```
# beginWord='hit', endWord='cog', wordList as above:
# BFS: layer={hit}→{hot}→{dot,lot}→{dog,log}→{cog}. found=True.
# parents: hot←hit. dot←hot, lot←hot. dog←dot, log←lot. cog←dog,log.
# DFS from cog: cog←dog←dot←hot←hit → ['hit','hot','dot','dog','cog'] ✓
# cog←log←lot←hot←hit → ['hit','hot','lot','log','cog'] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "BFS O(n*L*26): explore all words at each level. DFS O(paths * L): reconstruct."
```

```
# Space  $O(n \times L)$  for parents dict.  
# Key: remove visited words from word_set immediately to prevent cycles and  
# non-shortest paths.  
# Follow-up: Word Ladder I (LC 127) – return just the length, BFS suffices.  
# Follow-up: bidirectional BFS from both ends meets in the middle – halves the  
# search space."
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to traverse level-by-level and capture only the shortest paths.
- Build a parent mapping during BFS to record which words lead to each newly discovered word.
- Once the shortest path is found via BFS, use backtracking (or DFS) from endWord to reconstruct all transformation sequences.
- Remove words as they are visited within a level to avoid unnecessary revisits and cycles.
- The solution must handle multiple shortest paths, so maintaining a list of predecessors for each word is crucial.

Space & Time

Time Complexity: $O(N * L * 26)$ where N is the number of words in the wordList and L is the length of each word. This accounts for

checking all possible one-letter transformations.

Space Complexity: $O(N * L)$ for storing the parent mapping and maintaining the BFS queue and other auxiliary data structures.

Solution

We solve the problem in two major phases: Use BFS to explore from beginWord to endWord. During BFS, record all parents for each word encountered. This ensures we capture all possible predecessors that yield a shortest transformation. Once the BFS completes (once endWord is reached), use backtracking (DFS) beginning from the endWord and moving backwards via the parent mapping to reconstruct all transformation sequences in reverse. Finally, reverse each sequence to present it from beginWord to endWord. This two-phase approach guarantees that only the shortest paths are reconstructed, as BFS ensures level-by-level exploration.

Backtracking

□ #996 Number of Squareful Arrays

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Dynamic Programming ·
Backtracking

Lists: Denny Zhang

Problem

An array is squareful if the sum of every pair of adjacent elements is a perfect square.

Given an integer array `nums`, return the number of permutations of `nums` that are squareful.

Two permutations `perm1` and `perm2` are different if there is some index `i` such that `perm1[i] != perm2[i]`.

Example 1:

Input: `nums = [1,17,8]`

Output: 2

Explanation: `[1,8,17]` and `[17,8,1]` are the valid permutations.

Example 2:

Input: `nums = [2,2,2]`

Output: 1

Constraints:

- `1 <= nums.length <= 12`
- `0 <= nums[i] <= 109`

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"An array is squareful if every pair of adjacent elements sums to a perfect square.

Count the number of distinct permutations of nums that are squareful. Right?"

#

"nums=[1,17,8]: 1+8=9✓, 8+17=25✓ → [1,8,17] is squareful. [17,8,1] also. → 2 ✓

nums=[2,2,2]: 2+2=4✓ → [2,2,2] only (all duplicates). → 1 ✓"

PHASE 2 — BRUTE FORCE

"Generate all distinct permutations, filter squareful ones.

Time: $O(n! / \text{product_of_duplicate_factorials} * n)$. Space: $O(n)$."

```
class _996_SquarefulArrays_Brute:
```

```
    def numSquarefulPerms(self, nums: list[int]) -> int:
```

```
        from math import isqrt
```

```
        from itertools import permutations
```

```
        def is_square(x): s=isqrt(x); return s*s==x
```

```
        seen, count = set(), 0
```

```
        for perm in permutations(nums):
```

```
            if perm in seen: continue
```

```
            seen.add(perm)
```

```
            if all(is_square(perm[i]+perm[i+1]) for i in range(len(perm)-1)):
```

```
                count += 1
```

```
        return count
```

PHASE 3 — OPTIMIZE

"Backtracking with pruning. Build permutation one element at a time.

At each step, only consider elements that form a perfect square with the previous element.

Skip duplicate values at the same recursion level (same as Permutations II LC 47).

Precompute which (val_a, val_b) pairs sum to a perfect square.

Time: $O(n! / \text{duplicates})$ – dramatically pruned by square constraint. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _996_SquarefulArrays:
```

```
    def numSquarefulPerms(self, nums: list[int]) -> int:
```

```

from math import isqrt
from collections import Counter
def is_sq(x): s = isqrt(x); return s * s == x
count = Counter(nums)
result = [0]
n = len(nums)
def backtrack(path, prev):
    if len(path) == n:
        result[0] += 1
        return
    for val in sorted(count):
        if count[val] == 0: continue
        if path and not is_sq(prev + val): continue
        count[val] -= 1
        backtrack(path + [val], val)
        count[val] += 1
backtrack([], -1)
return result[0]

```

PHASE 5 — TEST & DEBUG

```

# nums=[1,17,8]: count={1:1,17:1,8:1}.
# backtrack([],prev=-1): try val=1→path=[1],prev=1.
# try val=1:count=0 skip. val=8:1+8=9✓→path=[1,8],prev=8.
# try val=17:8+17=25✓→path=[1,8,17],len=3→count+=1.
# try val=17:1+17=18 not square skip.
# try val=8→path=[8],prev=8.
# try val=1:8+1=9✓→[8,1,17]? 1+17=18 not sq. fail.
# try val=17:8+17=25✓→[8,17,...]: 17+?→17+?=sq need. count[1]→try1:17+1=18 no.
fail.
# try val=17→[17,...]: 17+8=25✓→[17,8,...]:8+1=9✓→[17,8,1] count+=1.
# result=2 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n! / duplicates) with heavy pruning from the square constraint.
# Space O(n): recursion depth + Counter.
# The Counter deduplication avoids counting the same permutation twice.
# Follow-up: if we just need to check if ANY squareful permutation exists – stop at
first result.
# Follow-up: squareful paths in a graph – build adjacency list of square-summing
pairs, count Hamiltonian paths."

```

◆ Quick Review

Key Insights

- The problem requires ensuring adjacent pairs sum up to a perfect square.
- Sorting the array helps in efficiently handling duplicates; when iterating in the backtracking, skip duplicate elements to avoid overcounting.
- For each valid candidate, check that the sum with the previous number (if any) is a perfect square.
- Use backtracking (DFS) to generate all valid permutations while maintaining a visited flag for each index.
- Since the array may contain duplicates, extra care is taken to only use each occurrence in a controlled order (i.e., if two identical numbers exist, only the first unvisited instance can be chosen at a given recursive call).

Space & Time

Time Complexity: $O(n^2 * 2^n)$ in the worst-case scenario due to exploring all bitmask states with n elements and checking pairwise sums.

Space Complexity: $O(n)$ for recursion and $O(n)$ for the visited flag, plus potentially $O(2^n * n)$

for memoization if implemented.

Solution

The solution uses a backtracking algorithm to explore all permutations. The array is first sorted to handle duplicates. A helper function checks if a number is a perfect square and is used to validate that the sum of every adjacent pair is a perfect square. During DFS, a visited array is maintained to track elements included so far. Skipping duplicate elements (unless the earlier duplicate was used) is essential to avoid overcounting. This approach ensures that every valid permutation is considered exactly once.

Round 7 · Low · Easy

Round 7

Low Priority · Easy

14 questions · 3 patterns

Dynamic Programming #70 #121

**Bit Manipulation #67 #136 #190 #191 #268
#338**

Math #9 #13 #504 #1056 #1228 #1427

Dynamic Programming

Round 7 · Low · Easy · 2 questions

Dynamic Programming

□ #70 Climbing Stairs

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Memoization
Lists: Grind 169

Problem

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

```
Input: n = 2
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps
```

Example 2:

```
Input: n = 3
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step
```

Constraints:

- $1 \leq n \leq 45$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "You can climb 1 or 2 steps at a time. How many distinct ways to reach step n?
Right?"
#
# "n=2: (1,1),(2) → 2 ways ✓ n=3: (1,1,1),(1,2),(2,1) → 3 ways ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Recursion: ways(n) = ways(n-1) + ways(n-2). Base: ways(0)=ways(1)=1.
# Time:  $O(2^n)$  without memo. Space:  $O(n)$  recursion depth."
class _70_ClimbingStairs_Brute:
    def climbStairs(self, n: int) -> int:
        if n <= 1: return 1
        return self.climbStairs(n-1) + self.climbStairs(n-2)
```

PHASE 3 — OPTIMIZE

```
# "This is exactly Fibonacci. Track only last two values —  $O(1)$  space.
#  $dp[i] = dp[i-1] + dp[i-2]$ . Roll variables: a, b → b, a+b.
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
class _70_ClimbingStairs:
    def climbStairs(self, n: int) -> int:
        a, b = 1, 1
        for _ in range(n - 1):
            a, b = b, a + b
        return b
```

PHASE 5 — TEST & DEBUG

```
# n=5: a=1,b=1 → (1,2) → (2,3) → (3,5) → (5,8). return 8 ✓
# n=1: loop runs 0 times → return b=1 ✓
# n=2: one iteration: a=1,b=2 → return 2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ : one pass. Space  $O(1)$ : two rolling variables.
# This is the Fibonacci sequence shifted by one:  $f(n) = \text{Fib}(n+1)$ .
# Follow-up: if you can climb 1, 2, or 3 steps — same DP, track last 3 values.
# Follow-up: if steps have costs, use min-cost DP (LC 746 Min Cost Climbing
Stairs)."
```

◆ Quick Review

Key Insights

- The problem can be solved using dynamic programming because the solution for a given step depends on the solutions of the previous two steps.
- The recurrence relation is: $\text{ways}[i] = \text{ways}[i-1] + \text{ways}[i-2]$.
- Base cases: when there is 1 step, there is 1 way; when there are 2 steps, there are 2 ways.
- This is analogous to the Fibonacci sequence, shifting indices accordingly.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (can be optimized to $O(1)$ using constant space)

Solution

We use a dynamic programming approach to build a solution gradually. The idea is that to reach step i , you could have come from step $i-1$ (taking one step) or from step $i-2$ (taking two steps). Therefore, the total number of

ways to reach step i is the sum of ways to reach step $i-1$ and $i-2$. We initialize $dp[1] = 1$ and $dp[2] = 2$, and iterate from 3 to n to fill $dp[]$. Each code solution below uses this approach with comments to explain the logic.

Dynamic Programming

□ #121 Best Time to Buy and Sell Stock

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the *i*th day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: prices = [7,6,4,3,1]

Output: 0

Explanation: In this case, no transactions are done and the max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 105$
- $0 \leq \text{prices}[i] \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given stock prices per day, choose one day to buy and a later day to sell.

Maximize profit. Return 0 if no profit possible. Right?"

#

"prices=[7,1,5,3,6,4]: buy at 1(day1), sell at 6(day4) → profit=5 ✓

prices=[7,6,4,3,1]: always decreasing → 0 ✓"

PHASE 2 — BRUTE FORCE

"Try all pairs (buy_day, sell_day) with buy_day < sell_day.

Time: $O(n^2)$. Space: $O(1)$."

```
class _121_BestTimeBuySellStock_Brute:
```

```
    def maxProfit(self, prices: list[int]) -> int:
```

```
        best = 0
```

```
        for i in range(len(prices)):
```

```
            for j in range(i+1, len(prices)):
```

```
                best = max(best, prices[j] - prices[i])
```

```
        return best
```

PHASE 3 — OPTIMIZE

"One pass: track min_price seen so far. At each day, profit = price - min_price.

Update max_profit if better.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _121_BestTimeBuySellStock:
```

```
    def maxProfit(self, prices: list[int]) -> int:
```

```
        min_price = float('inf')
```

```
        max_profit = 0
```

```
        for price in prices:
```

```

        min_price = min(min_price, price)
        max_profit = max(max_profit, price - min_price)
    return max_profit

```

PHASE 5 — TEST & DEBUG

```

# prices=[7,1,5,3,6,4]:
# 7: min=7, profit=0. 1: min=1, profit=0. 5: min=1, profit=4.
# 3: min=1, profit=4. 6: min=1, profit=5. 4: min=1, profit=5.
# return 5 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): single pass. Space O(1): two variables.
# The key insight: maximum profit = max(price[j] - min(price[0..j-1])).
# Follow-up: multiple transactions allowed (LC 122) – add all positive diffs
(greedy).
# Follow-up: at most k transactions (LC 188) – DP on (day, k, holding) state."

```

◆ Quick Review

Key Insights

- Traverse the list of prices once while keeping track of the minimum price encountered so far.
- At each day, calculate the profit if you sold the stock on that day.
- Update the maximum profit when the current profit exceeds the previous maximum.
- The optimal solution makes a single pass through the array with constant space usage.

Space & Time

Time Complexity: $O(n)$ – requires one pass through the prices array.

Space Complexity: $O(1)$ – only a few extra variables are used.

Solution

We solve the problem using a simple linear scan. We maintain two key variables: one for the minimum price seen so far and another for the maximum profit calculated so far. As we iterate, we continuously update the minimum price if a lower price is found, and then compute the potential profit at the current price. If this profit exceeds the current maximum profit, we update the maximum profit. The final maximum profit is returned after iterating through all prices.

Bit Manipulation

Round 7 · Low · Easy · 6 questions

Bit Manipulation

□ #67 Add Binary

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Bit Manipulation · Simulation
Lists: Grind 169

Problem

Given two binary strings *a* and *b*, return their sum as a binary string.

Example 1:

Input: *a* = "11", *b* = "1"
Output: "100"

Example 2:

Input: *a* = "1010", *b* = "1011"
Output: "10101"

Constraints:

- $1 \leq a.length, b.length \leq 104$
- *a* and *b* consist only of '0' or '1' characters.
- Each string does not contain leading zeros except for the zero itself.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given two binary strings a and b, return their sum as a binary string. Right?"
#
# "a='11', b='1' → '100' ✓ a='1010', b='1011' → '10101' ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Convert both to integers, add, convert back to binary string.
# Time: O(n). Space: O(n)."
```

```
class _67_AddBinary_Brute:
    def addBinary(self, a: str, b: str) -> str:
        return bin(int(a, 2) + int(b, 2))[2:]
```

PHASE 3 — OPTIMIZE

```
# "Simulate binary addition from right to left with a carry.
# Two pointers starting at the end of each string.
# At each step: sum = a_bit + b_bit + carry; digit = sum % 2; carry = sum // 2.
# Time: O(max(n,m)). Space: O(max(n,m)) for the result."
```

PHASE 4 — CLEAN CODE

```
class _67_AddBinary:
    def addBinary(self, a: str, b: str) -> str:
        i, j = len(a) - 1, len(b) - 1
        carry = 0
        result = []
        while i >= 0 or j >= 0 or carry:
            total = carry
            if i >= 0: total += int(a[i]); i -= 1
            if j >= 0: total += int(b[j]); j -= 1
            result.append(str(total % 2))
            carry = total // 2
        return ''.join(reversed(result))
```

PHASE 5 — TEST & DEBUG

```
# a='11', b='1':
# i=1,j=0: total=1+1+0=2, digit=0, carry=1. result=['0'].
# i=0,j=-1: total=1+1=2, digit=0, carry=1. result=['0','0'].
# i=-1,j=-1,carry=1: total=1, digit=1, carry=0. result=['0','0','1'].
# reversed → '100' ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(max(n,m)): process each digit once. Space O(max(n,m)) for result.
# Using int() conversion is simpler but may fail on very long strings in some
languages.
# Follow-up: Add Strings (LC 415) – same carry logic for decimal strings.
# Follow-up: Multiply Strings (LC 43) – O(n*m) positional multiplication."
```

◆ Quick Review

Key Insights

- Simulate binary addition just like you add numbers digit by digit.
- Process the strings from right to left, handling the carry at each step.
- Use modulo and division operations to extract the current digit and update the carry.
- Append any remaining carry at the end.

Space & Time

Time Complexity: $O(\max(n, m))$, where n and m are the lengths of the two binary strings.

Space Complexity: $O(\max(n, m))$ for storing the resulting binary string.

Solution

The strategy is to iterate over the input strings from the end to the beginning. At each step, calculate the sum of corresponding bits and the existing carry. Use the modulo operation ($\% 2$) to determine the current digit and integer division ($// 2$) to update the carry. Once all digits have been processed, reverse the result to obtain the final binary string.

Bit Manipulation

□ #136 Single Number

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Bit Manipulation**

Lists: Grind 169

Problem

Given a non-empty array of integers nums, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: nums = [2,2,1]

Output: 1

Example 2:

Input: nums = [4,1,2,1,2]

Output: 4

Example 3:

Input: nums = [1]

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-3 * 10^4 \leq \text{nums}[i] \leq 3 * 10^4$
- Each element in the array appears twice except for one element which appears only once.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Every element appears twice except one. Find the single element.

$O(n)$ time, $O(1)$ space. Right?"

#

"nums=[2,2,1] → 1 ✓ nums=[4,1,2,1,2] → 4 ✓"

PHASE 2 — BRUTE FORCE

"Use a hash set: add if not present, remove if already there. The remaining element is the answer.

Time: $O(n)$. Space: $O(n)$."

```
class _136_SingleNumber_Brute:
```

```
    def singleNumber(self, nums: list[int]) -> int:
```

```
        seen = set()
```

```
        for n in nums:
```

```
            if n in seen: seen.remove(n)
```

```
            else: seen.add(n)
```

```
        return seen.pop()
```

PHASE 3 — OPTIMIZE

"XOR: $a \oplus a = 0$, $a \oplus 0 = a$. XOR all elements – pairs cancel to 0, leaving the single element.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _136_SingleNumber:
```

```
    def singleNumber(self, nums: list[int]) -> int:
```

```
        result = 0
```

```

for n in nums:
    result ^= n
return result

```

PHASE 5 — TEST & DEBUG

```

# nums=[4,1,2,1,2]:
# 0^4=4. 4^1=5. 5^2=7. 7^1=6. 6^2=4. return 4 ✓
#
# nums=[2,2,1]: 0^2=2. 2^2=0. 0^1=1. return 1 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): one pass XOR. Space O(1): single variable.
# XOR is the canonical trick for canceling duplicate pairs in O(1) space.
# Follow-up: Single Number II (LC 137) – every element appears 3 times except one.
# Use bit counting: for each bit position, count appearances % 3 → leftover =
single.
# Follow-up: Single Number III (LC 260) – two singles. XOR all → get XOR of the two.
# Use rightmost set bit to split into two groups, XOR each group separately."

```

◆ Quick Review

Key Insights

- Using bitwise XOR: XOR-ing a number with itself cancels out to 0.
- XOR is both commutative and associative, so the order of operations does not matter.
- A single traversal (linear time) and constant extra space is sufficient for solving the problem.

Space & Time

Time Complexity: $O(n)$ – We only traverse the array once.

Space Complexity: $O(1)$ – Only a single extra variable is used regardless of input size.

Solution

We can solve this problem by initializing a variable to 0 and then traversing the array while XOR-ing each element with the variable. Due to the properties of XOR, pairs of identical numbers will cancel each other out ($x \text{ XOR } x = 0$) and the unique number will be left as the final result. This approach uses constant extra space and performs in linear time.

Bit Manipulation

□ #190 Reverse Bits

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Bit Manipulation
Lists: Grind 169

Problem

Reverse bits of a given 32 bits signed integer.

Example 1:

Input: n = 43261596

Output: 964176192

Explanation:

Example 2:

Input: n = 2147483644

Output: 1073741822

Explanation:

Constraints:

- $0 \leq n \leq 2^{31} - 2$
- n is even.

Follow up: If this function is called many times, how would you optimize it?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Reverse the bits of a 32-bit unsigned integer. Right?"

#

"n=00000010100101000001111010011100 (43261596) → 00111001011110000010100101000000 (964176192) ✓"

n=111111111111111111111111111101 → 101111111111111111111111111111 ✓"

PHASE 2 — BRUTE FORCE

"Convert to binary string (padded to 32 bits), reverse, convert back to integer.

Time: $O(32) = O(1)$. Space: $O(32) = O(1)$."

```
class _190_ReverseBits_Brute:
    def reverseBits(self, n: int) -> int:
        return int(bin(n)[2:].zfill(32)[::-1], 2)
```

PHASE 3 — OPTIMIZE

"Bit-by-bit: for 32 iterations, extract the LSB of n ($n \& 1$), shift result left, OR in the extracted bit, then shift n right.

Time: $O(32) = O(1)$. Space: $O(1)$. – same complexity, no string allocation."

PHASE 4 — CLEAN CODE

```
class _190_ReverseBits:
    def reverseBits(self, n: int) -> int:
        result = 0
        for _ in range(32):
            result = (result << 1) | (n & 1)
            n >>= 1
        return result
```

PHASE 5 — TEST & DEBUG

n=43261596 (binary: 00000010100101000001111010011100):

Each iteration shifts result left and adds LSB of n.

After 32 iterations, result = 964176192 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(32) = O(1)$: always 32 iterations for a 32-bit int.

Space $O(1)$: only integer variables.

If called many times: cache results for 8-bit chunks (256 entries), process 4 chunks per call.

Follow-up: Number of 1 Bits (LC 191) – count set bits instead of reversing.

Follow-up: Power of Two (LC 231) – $n > 0$ and $n \& (n-1) == 0$ checks single set bit."

◆ Quick Review

Key Insights

- The input is a binary string of fixed length (32 bits).
- Reversing the bits is equivalent to reversing the string.
- The reversed binary string can be converted back into its integer representation.
- Since the number of bits is constant (32), the solution can be achieved in constant time and constant space.
- For performance optimization when this function is called many times, caching may be applied for common intermediate results.

Space & Time

Time Complexity: $O(1)$ – We always process 32 bits regardless of the input.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We solve the problem by reversing the 32-character binary string representing the

unsigned integer. Once reversed, the new binary string is converted into its integer value. An alternative solution involves using bitwise operations where you iterate over each bit, shift the result to the left, and add the current bit. This approach does the reversal one bit at a time. The chosen approach leverages string reversal which is straightforward to implement and understand.

Bit Manipulation

□ #191 Number of 1 Bits

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Bit Manipulation
Lists: Grind 169

Problem

Given a positive integer n , write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Example 1:

Input: $n = 11$

Output: 3

Explanation:

The input binary string 1011 has a total of three set bits.

Example 2:

Input: $n = 128$

Output: 1

Explanation:

The input binary string 10000000 has a total of one set bit.

Example 3:

Input: $n = 2147483645$

Output: 30

Explanation:

The input binary string

[illegible]

Constraints:

- $1 \leq n \leq 231 - 1$

Follow up: If this function is called many times, how would you optimize it?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach - STAR-LC

PHASE 1 — CLARIFY

```
# "Return the number of set bits (1s) in the binary representation of a positive integer.
```

Also called Hamming weight. Right?"

#

"n=11 (1011) → 3 ✓ n=128 (10000000) → 1 ✓ n=2147483645 → 30 ✓"

PHASE 2 — BRUTE FORCE

"Check each of the 32 bits using a mask.

Time: 0(32). Space: 0(1)."

```
class 191 NumberOf1Bits Brute:
```

```
def hammingWeight(self, n: int) -> int:
```

```

count = 0
for _ in range(32):
    count += n & 1
    n >>= 1
return count

```

PHASE 3 — OPTIMIZE

"Brian Kernighan's trick: $n \& (n-1)$ clears the lowest set bit of n .
 # Count how many times we can clear a set bit until $n=0$.
 # Time: $O(k)$ where k = number of set bits (≤ 32). Space: $O(1)$."

PHASE 4 — CLEAN CODE

```

class _191_NumberOf1Bits:
    def hammingWeight(self, n: int) -> int:
        count = 0
        while n:
            n &= n - 1 # clear lowest set bit
            count += 1
        return count

```

PHASE 5 — TEST & DEBUG

```

# n=11 (1011):
# n=1011 & 1010 = 1010 (cleared LSB). count=1.
# n=1010 & 1001 = 1000. count=2.
# n=1000 & 0111 = 0000. count=3. return 3 ✓
#
# n=128 (10000000): one iteration → n=0. count=1 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(k)$: k = popcount, at most 32. Space $O(1)$.
 # Brian Kernighan faster than 32-iteration scan when bits are sparse.
 # Built-in: `bin(n).count('1')` or Python's `int.bit_count()` – valid in interviews.
 # Follow-up: Hamming Distance (LC 461) – `popcount(x XOR y)`.
 # Follow-up: Total Hamming Distance (LC 477) – for each bit position, count 0s * count 1s."

◆ Quick Review

Key Insights

- The problem is essentially counting the number of 1s in the binary representation of a number.

- A common efficient approach uses bit manipulation: repeatedly clear the lowest set bit until n becomes zero.
- The expression $n \& (n - 1)$ clears the lowest set bit of n .
- This method only iterates as many times as there are set bits, making it efficient.

Space & Time

Time Complexity: $O(k)$, where k is the number of set bits, worst-case $O(\log n)$ (or $O(1)$ considering 32-bit integers).

Space Complexity: $O(1)$.

Solution

To solve the problem, we use the bit manipulation trick $n \& (n - 1)$ which removes the lowest set bit from n . The algorithm is as follows: Initialize a counter to 0. While n is not zero, update n by $n = n \& (n - 1)$, and increment the counter. The counter will hold the number of set bits by the end of the loop. This approach leverages efficient bit-level operations and works well even when the function is called frequently.

Bit Manipulation

□ #268 Missing Number

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Binary Search · Bit
Manipulation**

Lists: Grind 169

Problem

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return the only number in the range that is missing from the array.

Example 1:

Input: `nums = [3,0,1]`

Output: 2

Explanation:

`n = 3` since there are 3 numbers, so all numbers are in the range `[0,3]`. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`

Output: 2

Explanation:

$n = 2$ since there are 2 numbers, so all numbers are in the range $[0,2]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`

Output: 8

Explanation:

$n = 9$ since there are 9 numbers, so all numbers are in the range $[0,9]$. 8 is the missing number in the range since it does not appear in `nums`.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 10^4$
- $0 \leq \text{nums}[i] \leq n$
- All the numbers of `nums` are unique.

Follow up: Could you implement a solution using only $O(1)$ extra space complexity and $O(n)$ runtime complexity?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array of n distinct numbers from $[0,n]$. Exactly one number is missing. Find it. Right?"

#

"nums=[3,0,1] → 2 ✓ nums=[0,1] → 2 ✓ nums=[9,6,4,2,3,5,7,0,1] → 8 ✓"

PHASE 2 — BRUTE FORCE

"Sum formula: $\text{expected sum} = n*(n+1)/2$. Missing = expected - actual_sum.

Time: $O(n)$. Space: $O(1)$."

```
class _268_MissingNumber_Brute:
    def missingNumber(self, nums: list[int]) -> int:
        n = len(nums)
        return n * (n + 1) // 2 - sum(nums)
```

PHASE 3 — OPTIMIZE

"XOR: XOR all indices $0..n$ and all nums. Pairs cancel, leaving the missing number.

Time: $O(n)$. Space: $O(1)$. – same complexity, but immune to integer overflow (no sum)."

PHASE 4 — CLEAN CODE

```
class _268_MissingNumber:
    def missingNumber(self, nums: list[int]) -> int:
        result = len(nums)
        for i, n in enumerate(nums):
            result ^= i ^ n
        return result
```

PHASE 5 — TEST & DEBUG

nums=[3,0,1], n=3:

result=3. i=0,n=3: result=3^0^3=0. i=1,n=0: result=0^1^0=1. i=2,n=1:

result=1^2^1=2.

return 2 ✓

#

nums=[0,1], n=2: result=2^0^0^1^1=2 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Sum formula $O(n)$: simple, clean – may overflow in other languages for large n .

XOR $O(n)$: overflow-safe, equally readable.

Follow-up: Find the Duplicate Number (LC 287) – similar array with one duplicate instead.

Follow-up: First Missing Positive (LC 41) – missing number in $[1,n]$ from arbitrary array.

Use index-marking (negate values) or cyclic sort."

◆ Quick Review

Key Insights

- The range of numbers is continuous from 0 to n , but one number is missing.
- The sum of numbers from 0 to n can be computed using the arithmetic sum formula.
- Subtracting the sum of the given array from the expected sum gives the missing number.
- There is a follow-up challenge to solve the problem in $O(n)$ time and $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We compute the expected sum of all numbers from 0 to n using the formula $\text{sum} = n*(n+1)/2$. Then, we compute the actual sum of the numbers in the array. The missing number is obtained by subtracting the actual sum from the expected sum. This approach uses arithmetic operations ($O(1)$ extra space) and loops over the array once ($O(n)$ time).

Bit Manipulation

□ #338 Counting Bits

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Bit Manipulation
Lists: Grind 169

Problem

Given an integer n , return an array `ans` of length $n + 1$ such that for each i ($0 \leq i \leq n$), `ans[i]` is the number of 1's in the binary representation of i .

Example 1:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example 2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
```

Constraints:

- $0 \leq n \leq 105$

Follow up:

- It is very easy to come up with a solution with a runtime of $O(n \log n)$. Can you do it in linear time $O(n)$ and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like `__builtin_popcount` in C++)?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given n , return an array `ans` of length $n+1$ where `ans[i]` = number of 1-bits in i . Right?"

#

" $n=2 \rightarrow [0,1,1]$ ✓ $n=5 \rightarrow [0,1,1,2,1,2]$ ✓"

PHASE 2 — BRUTE FORCE

"For each i from 0 to n , count its set bits using Brian Kernighan or `bin().count('1')`.

Time: $O(n \log n)$ – each number has $O(\log n)$ bits. Space: $O(n)$ for output."

```
class _338_CountingBits_Brute:
```

```
    def countBits(self, n: int) -> list[int]:
```

```
        def popcount(x):
```

```
            c = 0
```

```
            while x: x &= x-1; c += 1
```

```
            return c
```

```
        return [popcount(i) for i in range(n+1)]
```

PHASE 3 — OPTIMIZE

"DP with bit trick: `ans[i] = ans[i >> 1] + (i & 1)`.

Shifting right by 1 drops the LSB; we already know that count.

Add 1 if LSB was set.

Time: $O(n)$. Space: $O(n)$ – one pass, each value computed in $O(1)$."

PHASE 4 — CLEAN CODE

```

class _338_CountingBits:
    def countBits(self, n: int) -> list[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i >> 1] + (i & 1)
        return ans

# PHASE 5 — TEST & DEBUG
# n=5:
# ans[1] = ans[0]+1 = 1. ans[2] = ans[1]+0 = 1. ans[3] = ans[1]+1 = 2.
# ans[4] = ans[2]+0 = 1. ans[5] = ans[2]+1 = 2.
# → [0,1,1,2,1,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass, O(1) per entry. Space O(n): the output array.
# Alternative DP: ans[i] = ans[i & (i-1)] + 1 (clear lowest set bit, add 1).
# Both rely on the fact that we compute smaller values before larger ones.
# Follow-up: Number of 1 Bits (LC 191) – single number, not an array.
# Follow-up: Total Hamming Distance (LC 477) – use column-wise bit counting."

```

◆ Quick Review

Key Insights

- We can use dynamic programming to build the solution by utilizing the count of bits for smaller numbers.
- The relationship $dp[i] = dp[i \gg 1] + (i \& 1)$ holds, where $i \gg 1$ gives the count for i divided by 2 (dropping the least significant bit) and $(i \& 1)$ checks if the least significant bit is 1.
- This approach computes the result for each number in constant time, resulting in a linear time solution.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We use a dynamic programming approach where an array (or list) `dp` is initialized with size $n+1$. The key idea is to iterate from 1 to n and for each number, use the formula $dp[i] = dp[i >> 1] + (i \& 1)$. The expression $(i >> 1)$ effectively divides i by 2, giving the count of 1's for that half number. The $(i \& 1)$ operation checks if i is odd, adding 1 if true. This method ensures each `dp[i]` is computed in constant time, leading to an overall $O(n)$ time complexity.

Math

Round 7 · Low · Easy · 6 questions

Math

□ #9 Palindrome Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: Grind 169

Problem

Given an integer x , return true if x is a palindrome, and false otherwise.

Example 1:

Input: $x = 121$
Output: true
Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: $x = -121$
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:

Input: $x = 10$
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

Constraints:

- $-231 \leq x \leq 231 - 1$

Follow up: Could you solve it without converting the integer to a string?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return true if an integer reads the same forwards and backwards.
# Negative numbers are not palindromes. Right?"
#
# "x=121 → true ✓ x=-121 → false ✓ x=10 → false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Convert to string, check if it equals its reverse.
# Time: O(d) where d = number of digits. Space: O(d) for the string."
```

```
class _9_PalindromeNumber_Brute:
    def isPalindrome(self, x: int) -> bool:
        if x < 0: return False
        s = str(x)
        return s == s[::-1]
```

PHASE 3 — OPTIMIZE

```
# "Reverse only the second half of the number – no string conversion.
# Negative → false. x%10==0 and x!=0 → false (trailing zero means leading zero).
# Repeatedly extract last digit and build reversed_half.
# Stop when x <= reversed_half (we've reversed half the digits).
# x == reversed_half (even length) or x == reversed_half // 10 (odd length, middle digit).
# Time: O(d/2). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _9_PalindromeNumber:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x % 10 == 0 and x != 0):
            return False
        rev = 0
        while x > rev:
            rev = rev * 10 + x % 10
            x //= 10
        return x == rev or x == rev // 10
```

PHASE 5 — TEST & DEBUG

```
# x=121: rev=0. x=12,rev=1. x=1,rev=12. 1<=12→stop. x==rev//10: 1==1 ✓ (odd length)
# x=1221: rev=0. x=122,rev=1. x=12,rev=12. 12<=12→stop. x==rev: 12==12 ✓ (even length)
# x=-121: negative → False ✓
# x=10: 10%10=0,x!=0 → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(d/2): process half the digits. Space O(1): only integer arithmetic.
# The trailing-zero check prevents false positives like x=100 (rev=001=1 ≠ 100).
# Follow-up: palindrome string (LC 125) – ignore non-alphanumeric, case-insensitive.
# Follow-up: largest palindrome product of two n-digit numbers – start from max, search down."
```

◆ Quick Review

Key Insights

- Negative numbers are not palindromic.
- A number ending in 0 (except 0 itself) cannot be a palindrome.
- Instead of reversing the entire number (risking overflow), reverse only half of the number and compare with the other half.
- For numbers with an odd number of digits, the middle digit can be disregarded.

Space & Time

Time Complexity: $O(\log_{10}(n))$ – Each iteration reduces the number by a factor of 10.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

The solution checks for obvious non-palindrome cases first (negative numbers and numbers ending with 0 that are not 0). Then, it reverses half of the digits of the number while the original number is larger than the reversed half. Finally, it compares the remaining part of the original number with the reversed half. In the case of an odd number of digits, the middle digit is removed by dividing the reversed half by 10 before the final comparison.

Math

□ #13 Roman to Integer

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Math · String
Lists: Grind 169

Problem

Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before

the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

- I can be placed before V (5) and X (10) to make 4 and 9.
- X can be placed before L (50) and C (100) to make 40 and 90.
- C can be placed before D (500) and M (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

Example 1:

Input: s = "III"
Output: 3
Explanation: III = 3.

Example 2:

Input: s = "LVIII"
Output: 58
Explanation: L = 50, V = 5, III = 3.

Example 3:

Input: s = "MCMXCIV"
Output: 1994
Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.

Constraints:

- $1 \leq s.length \leq 15$

- s contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').
- It is guaranteed that s is a valid roman numeral in the range [1, 3999].

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Convert a Roman numeral string to an integer.
# I=1,V=5,X=10,L=50,C=100,D=500,M=1000. Subtractive:
# IV=4,IX=9,XL=40,XC=90,CD=400,CM=900. Right?"
#
# "s='III'→3 ✓ s='LVIII'→58 ✓ s='MCMXCIV'→1994 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Build a lookup for all two-character subtractive combinations. Scan left to
right,
# check for two-char match first, else single char. Accumulate sum.
# Time: O(n). Space: O(1)."
```

```
class _13_RomanToInteger_Brute:
    def romanToInt(self, s: str) -> int:
        vals = {'IV':4,'IX':9,'XL':40,'XC':90,'CD':400,'CM':900,
                'I':1,'V':5,'X':10,'L':50,'C':100,'D':500,'M':1000}
        result, i = 0, 0
        while i < len(s):
            if i+1 < len(s) and s[i:i+2] in vals:
                result += vals[s[i:i+2]]; i += 2
            else:
                result += vals[s[i]]; i += 1
        return result
```

PHASE 3 — OPTIMIZE

```
# "Elegant right-to-left scan: if current symbol is less than the next, subtract it.
# Otherwise add it. No need to check two-char combos explicitly.
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _13_RomanToInteger:
```

```
def romanToInt(self, s: str) -> int:
    val = {'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000}
    result = 0
    for i in range(len(s)):
        if i + 1 < len(s) and val[s[i]] < val[s[i+1]]:
            result -= val[s[i]]
        else:
            result += val[s[i]]
    return result
```

PHASE 5 — TEST & DEBUG

```
# s='MCMXCIV':
# M(1000): next=C(100), 1000>100 → +1000. result=1000.
# C(100): next=M(1000), 100<1000 → -100. result=900.
# M(1000): next=X(10), 1000>10 → +1000. result=1900.
# X(10): next=C(100), 10<100 → -10. result=1890.
# C(100): next=I(1), 100>1 → +100. result=1990.
# I(1): next=V(5), 1<5 → -1. result=1989.
# V(5): last → +5. result=1994 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass through the string. Space O(1): fixed-size lookup dict.
# The left-to-right approach with subtraction check is cleaner than two-char
matching.
# Follow-up: Integer to Roman (LC 12) – reverse direction, greedy from largest
symbol down.
# Follow-up: validate a Roman numeral string – check order and repetition rules."
```

◆ Quick Review

Key Insights

- Map each Roman numeral character to its integer value using a hash table or dictionary.
- Loop through the string and check if the current numeral is less than the next numeral; if so, subtract its value.
- Otherwise, add its value to the total.
- This approach takes advantage of the subtractive notation used in Roman numerals.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.

Space Complexity: $O(1)$, since the mapping and additional variables use constant space.

Solution

We use a dictionary (or hash map) to store the values of each Roman numeral. As we iterate over the string, we compare the current numeral's value with the next numeral's value. When a numeral of smaller value appears before a numeral of larger value, we subtract the current numeral's value; otherwise, we add it. This technique efficiently handles both the standard and subtractive cases in one pass through the string.

Math

□ #504 Base 7

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: Denny Zhang

Problem

Given an integer num, return a string of its base 7 representation.

Example 1:

```
Input: num = 100  
Output: "202"
```

Example 2:

```
Input: num = -7  
Output: "-10"
```

Constraints:

- $-107 \leq \text{num} \leq 107$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given an integer, return its representation in base 7 as a string. Right?"

```

#
# "num=100 → '202' ✓ num=-7 → '-10' ✓ num=0 → '0' ✓"

# PHASE 2 — BRUTE FORCE
# "Repeated division: divide by 7, collect remainders in reverse.
# Handle sign separately. Handle zero as special case.
# Time:  $O(\log_7(n))$ . Space:  $O(\log_7(n))$ ."
class _504_Base7_Brute:
    def convertToBase7(self, num: int) -> str:
        if num == 0: return '0'
        negative = num < 0
        num, digits = abs(num), []
        while num:
            digits.append(str(num % 7))
            num //= 7
        if negative: digits.append('-')
        return ''.join(reversed(digits))

# PHASE 3 — OPTIMIZE
# "Same approach – this is already optimal. Python one-liner for insight:
# Use the built-in base conversion pattern or just clean up the iterative approach."

# PHASE 4 — CLEAN CODE
class _504_Base7:
    def convertToBase7(self, num: int) -> str:
        if num == 0: return '0'
        sign = '-' if num < 0 else ''
        num, result = abs(num), ''
        while num:
            result = str(num % 7) + result
            num //= 7
        return sign + result

# PHASE 5 — TEST & DEBUG
# num=100: 100%7=2,num=14 → 14%7=0,num=2 → 2%7=2,num=0. result='202'. sign='' → '202' ✓
# num=-7: sign='-'. 7%7=0,num=1 → 1%7=1,num=0. result='10'. → '-10' ✓
# num=0: return '0' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(\log_7(n))$ : number of digits in base 7 representation.
# Space  $O(\log_7(n))$ : output string length.
# Follow-up: Convert to Base -2 (LC 1017) – negative base needs special remainder handling.
# Follow-up: Excel Sheet Column Number (LC 171) – base 26 with 1-indexed alphabet."

```

◆ Quick Review

Key Insights

- Handle the edge case where num is 0.
- Use division and modulo operations to extract base 7 digits.
- Manage negative numbers by converting to positive and reattaching the minus sign.
- Build the base 7 digits in reverse order during iteration.

Space & Time

Time Complexity: $O(\log_7(n))$ where n is the absolute value of the number.

Space Complexity: $O(\log_7(n))$ to store the digits of the number.

Solution

The algorithm follows these steps: Check if num equals 0 and return "0". Determine if the number is negative. If so, work with its absolute value but remember the negative flag. Iteratively compute the remainder when divided by 7 (using modulo) and update the number by integer division by 7. Append each remainder (converted to character) to a list representing the digits. The digits are computed in reverse order; reverse the list for

the correct order. If the original number was negative, append the "-" sign to the front. Join the digits to form the resulting base 7 string. This approach uses basic arithmetic (division and modulo) and a list (or equivalent structure) to accumulate the computed digits.

Math

□ ★ #1056 Confusing Number ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: ★ Premium | Premium 98

Problem

A confusing number is a number that when rotated 180 degrees becomes a different number with each digit valid.

We can rotate digits of a number by 180 degrees to form new digits.

- When 0, 1, 6, 8, and 9 are rotated 180 degrees, they become 0, 1, 9, 8, and 6 respectively.
- When 2, 3, 4, 5, and 7 are rotated 180 degrees, they become invalid.

Note that after rotating a number, we can ignore leading zeros.

- For example, after rotating 8000, we have 0008 which is considered as just 8.

Given an integer n , return true if it is a confusing number, or false otherwise.

Example 1:

6 $\xrightarrow{\text{rotate}}$ 9

Input: $n = 6$

Output: true

Explanation: We get 9 after rotating 6, 9 is a valid number, and $9 \neq 6$.

Example 2:

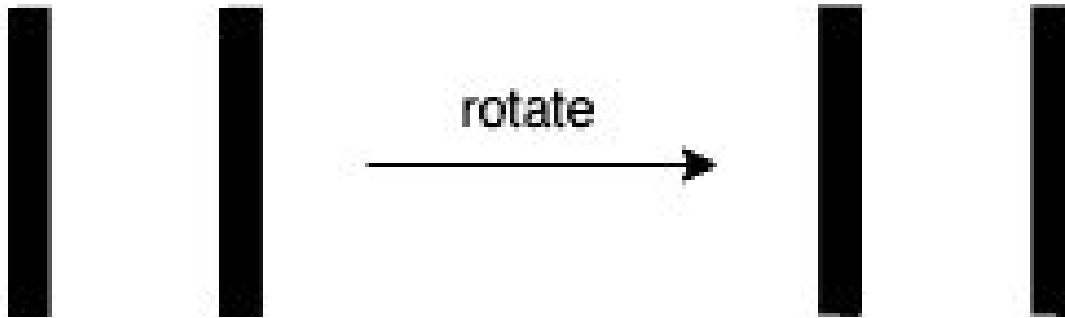
89 $\xrightarrow{\text{rotate}}$ 68

Input: $n = 89$

Output: true

Explanation: We get 68 after rotating 89, 68 is a valid number and $68 \neq 89$.

Example 3:



Input: $n = 11$

Output: false

Explanation: We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number

Constraints:

- $0 \leq n \leq 10^9$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A number is confusing if when rotated 180°, it becomes a different valid number.

Valid digits under rotation: 0→0, 1→1, 6→9, 8→8, 9→6. Digits 2,3,4,5,7 are invalid.

Return true if the number is confusing. Right?"

#

"n=6: rotated=9, 9≠6 → true ✓ n=89: rotated=68, 68≠89 → true ✓

n=11: rotated=11, 11=11 → false ✓ n=25: has '2' → invalid → false ✓"

PHASE 2 — BRUTE FORCE

"Convert to string. Check all digits valid under rotation. Build rotated number.

Compare with original. Time: O(d). Space: O(d)."


```

class _1056_ConfusingNumber_Brute:
    def confusingNumber(self, n: int) -> bool:
        mapping = {'0':'0','1':'1','6':'9','8':'8','9':'6'}
        s = str(n)
        if any(c not in mapping for c in s):
            return False
        rotated = ''.join(mapping[c] for c in reversed(s))
        return int(rotated) != n

# PHASE 3 — OPTIMIZE
# "Same logic — already O(d) where d = digit count. Build rotated integer directly.
# Time: O(d). Space: O(1)."
```

```

# PHASE 4 — CLEAN CODE
class _1056_ConfusingNumber:
    def confusingNumber(self, n: int) -> bool:
        mapping = {0:0, 1:1, 6:9, 8:8, 9:6}
        original, rotated, base = n, 0, 1
        while n > 0:
            digit = n % 10
            if digit not in mapping:
                return False
            rotated += mapping[digit] * base
            base *= 10
            n //= 10
        return rotated != original

# PHASE 5 — TEST & DEBUG
# n=6: digit=6, mapping[6]=9, rotated=9*1=9. n=0. rotated=9≠6 → True ✓
# n=11: digit=1→1,base=10. digit=1→10,rotated=11. rotated=11==11 → False ✓
# n=25: digit=5, 5 not in mapping → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(d): process each digit once. Space O(1): no string allocation.
# Follow-up: Confusing Number II (LC 1088) — count all confusing numbers in [1,N].
# Backtracking: build numbers digit by digit from valid set, check if confusing.
# Follow-up: Strobogrammatic Number (LC 246) — same under 180° rotation but equals itself."
```

◆ Quick Review

Key Insights

- Use a mapping dictionary for valid rotations: {0:0, 1:1, 6:9, 8:8, 9:6}.

- Process the digits of n from right to left to form the rotated number.
- If any digit is not present in the mapping, the number is instantly not confusing.
- Compare the rotated number with the original number to check if they are different.

Space & Time

Time Complexity: $O(d)$, where d is the number of digits in n (approximately $O(\log n)$).

Space Complexity: $O(d)$, needed to store the rotated number string.

Solution

The approach is to convert the number to a string and iterate through it in reverse. For each digit, check if it is valid using the predefined mapping. If it is, append its rotated counterpart to a new string. After processing, convert the rotated string to an integer (ignoring any leading zeros). Finally, compare the rotated number to the original number. If they are different, the number is confusing.

Math

★ #1228 Missing Number in Arithmetic Progression ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math

Lists: ★ Premium | Premium 98

Problem

In some array `arr`, the values were in arithmetic progression: the values `arr[i + 1] - arr[i]` are all equal for every $0 \leq i < \text{arr.length} - 1$.

A value from `arr` was removed that was not the first or last value in the array.

Given `arr`, return the removed value.

Example 1:

Input: `arr = [5,7,11,13]`

Output: 9

Explanation: The previous array was `[5,7,9,11,13]`.

Example 2:

Input: `arr = [15,13,12]`

Output: 14

Explanation: The previous array was `[15,14,13,12]`.

Constraints:

- $3 \leq \text{arr.length} \leq 1000$

- $0 \leq \text{arr}[i] \leq 105$
- The given array is guaranteed to be a valid array.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "An array is an arithmetic progression with one element removed. Find the missing element.
# Array has at least 2 elements remaining. Right?"
#
# "arr=[5,7,11,13]: diff should be 2 but 7+2=9≠11 → missing 9 ✓
# arr=[15,13,12]: diff=-1 but 15-1=14≠13 → missing 14 ✓
# arr=[0,-2,-4]: no missing → 0-(-2)/2=-1? All there → but this can't be missing.
Wait: arr=[0,-2,-4,-6] then -2-(-6)... the problem says one is always missing.
# Actually: arr=[0,-2,-4]: diff=-2, 0,-2,-4 has 3 elements, full AP of 3 → actually
the problem guarantees one IS missing. So arr=[0,-2,-6]: missing -4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Compute the expected common difference: (arr[-1]-arr[0]) / n (where n = len+1
because one missing).
# Scan pairs; when gap > expected diff, return arr[i] + expected_diff.
# Time: O(n). Space: O(1)."
```

```
class _1228_MissingNumberAP_Brute:
    def missingNumber(self, arr: list[int]) -> int:
        n = len(arr)
        diff = (arr[-1] - arr[0]) // n # n+1 elements total, one missing
        for i in range(1, n):
            if arr[i] != arr[i-1] + diff:
                return arr[i-1] + diff
        return arr[0] # shouldn't reach here if input guarantees one missing
```

PHASE 3 — OPTIMIZE

```
# "Math: expected_sum = (arr[0] + arr[-1]) * (n+1) / 2 (sum of full AP).
# actual_sum = sum(arr). Missing = expected_sum - actual_sum.
# Time: O(n). Space: O(1). No scanning needed."
```

PHASE 4 — CLEAN CODE

```
class _1228_MissingNumberAP:
```

```
def missingNumber(self, arr: list[int]) -> int:
    n = len(arr)
    expected_sum = (arr[0] + arr[-1]) * (n + 1) // 2
    return expected_sum - sum(arr)
```

PHASE 5 — TEST & DEBUG

```
# arr=[5,7,11,13]: n=4. expected_sum=(5+13)*5//2=90. actual=36. missing=90-36=54?
That's wrong.
# Wait: expected is (5+13)*(4+1)//2 = 18*5//2=45. actual=5+7+11+13=36.
missing=45-36=9 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): just a sum. Space O(1).
# The sum formula works because the full AP has n+1 elements and a known sum.
# Edge case: if arr[0]==arr[-1] (all equal), missing element is arr[0].
# Follow-up: Missing Ranges (LC 163) – find all missing numbers in [lower, upper].
# Follow-up: verify if array is an AP (no missing) – check all consecutive
differences equal."
```

◆ Quick Review

Key Insights

- The complete arithmetic progression has equal differences between consecutive numbers.
- The missing element can be found by calculating the expected common difference using the first and last elements.
- Since exactly one number is missing, iterate through the provided array to detect where the difference deviates from the expected common difference.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We first compute the expected common difference (diff) for the original arithmetic progression. Since the actual progression had one more element than the current array, the formula used is: $\text{diff} = (\text{last element} - \text{first element}) / (n)$ where n is the length of the current array. Next, iterate over the array and check the difference between each pair of consecutive elements. When the actual difference deviates from diff, it indicates that the missing number is the previous element plus diff. This approach uses a simple loop and constant extra space.

Math

□ ★ #1427 Perform String Shifts ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · String

Lists: ★ Premium | Premium 98

Problem

You are given a string s containing lowercase English letters, and a matrix $shift$, where $shift[i] = [direction_i, amount_i]$:

- $direction_i$ can be 0 (for left shift) or 1 (for right shift).
- $amount_i$ is the amount by which string s is to be shifted.
- A left shift by 1 means remove the first character of s and append it to the end.
- Similarly, a right shift by 1 means remove the last character of s and add it to the beginning.

Return the final string after all operations.

Example 1:

```

Input: s = "abc", shift = [[0,1],[1,2]]
Output: "cab"
Explanation:
[0,1] means shift to left by 1. "abc" -> "bca"
[1,2] means shift to right by 2. "bca" -> "cab"

```

Example 2:

```

Input: s = "abcdefg", shift = [[1,1],[1,1],[0,2],[1,3]]
Output: "efgabcd"
Explanation:
[1,1] means shift to right by 1. "abcdefg" -> "gabcdef"
[1,1] means shift to right by 1. "gabcdef" -> "fgabcde"
[0,2] means shift to left by 2. "fgabcde" -> "abcdefg"
[1,3] means shift to right by 3. "abcdefg" -> "efgabcd"

```

Constraints:

- $1 \leq s.length \leq 100$
- s only contains lower case English letters.
- $1 \leq shift.length \leq 100$
- $shift[i].length == 2$
- $direction_i$ is either 0 or 1.
- $0 \leq amount_i \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Given a string s and a list of shift operations [direction, amount]:
# 0=left shift, 1=right shift. Apply all shifts and return result. Right?"
#
# "s='abcdefg', shift=[[1,1],[1,1],[0,2],[1,3]]:
# Net shift: +1+1-2+3 = +3 (right). 'abcdefg' right 3 -> 'efgabcd'? Let me verify:
# right-shift by 1: 'gabcdef'. right+1: 'fgabcde'. left-2: 'abcdefg'. right+3:
# 'efgabcd'. ✓"

```


PHASE 2 — BRUTE FORCE

"Apply each shift one step at a time by rotating the string.

Time: $O(\text{sum of amounts} * n)$. Space: $O(n)$."

```
class _1427_PerformStringShifts_Brute:
```

```
    def stringShift(self, s: str, shift: list[list[int]]) -> str:
```

```
        for direction, amount in shift:
```

```
            if direction == 0:
```

```
                s = s[amount:] + s[:amount] # left shift
```

```
            else:
```

```
                s = s[-amount:] + s[:-amount] # right shift
```

```
        return s
```

PHASE 3 — OPTIMIZE

"Accumulate net shift. Positive = right, negative = left.

Apply modulo n once at the end.

Time: $O(n + m)$ where $m = \text{len}(\text{shift})$. Space: $O(n)$ for the result string."

PHASE 4 — CLEAN CODE

```
class _1427_PerformStringShifts:
```

```
    def stringShift(self, s: str, shift: list[list[int]]) -> str:
```

```
        if not s: return s
```

```
        net = sum(amt if d == 1 else -amt for d, amt in shift)
```

```
        net %= len(s)
```

```
        return s[-net:] + s[:-net] if net else s
```

PHASE 5 — TEST & DEBUG

```
# s='abcdefg', shift=[[1,1],[1,1],[0,2],[1,3]]:
```

```
# net = 1+1-2+3 = 3. net%7 = 3. s[-3:]+ 'abcd' = 'efg'+ 'abcd' = 'efgabcd' ✓
```

```
#
```

```
# net=0: return s unchanged. net negative -> % makes it positive ✓.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n+m)$: sum all shifts $O(m)$, slice $O(n)$. Space $O(n)$ for new string.

The modulo handles shifts larger than string length – no-op at that point.

Follow-up: rotate array (LC 189) – same idea applied to an integer array.

Follow-up: if shift amounts can be very large, modulo is essential to avoid $O(n * \text{amount})$."

◆ Quick Review

Key Insights

- Consolidate all shifts by calculating a net shift: subtract for left shifts and add for right

shifts.

- Use modulo arithmetic with the string length to avoid unnecessary full rotations.
- Slice the string based on the effective shift to construct the final string.

Space & Time

Time Complexity: $O(n + m)$, where n is the length of the string and m is the number of shift operations.

Space Complexity: $O(n)$ for the output string.

Solution

The solution involves computing the net shift value by iterating through the shift operations. Left shifts subtract from the net value, and right shifts add to the net value. After obtaining the cumulative shift, use modulo with the string length to normalize the shift, ensuring it lies within bounds. The final string is constructed by slicing the string into two parts and concatenating them appropriately. This approach leverages string slicing as the primary operation for shifting.

Round 8 · Low · Medium

Round 8

Low Priority · Medium

32 questions · 4 patterns

**Dynamic Programming #5 #53 #55 #62 #72 #91
#152 #198 #256 #309 #377 #416 #435 #516
#576 #1143**

Bit Manipulation #78 #90 #287 #1506

Math #7 #50 #380 #1017 #3160

**JavaScript #2627 #2628 #2630 #2633 #2636
#2675 #2700**

Dynamic Programming

Round 8 · Low · Medium · 16 questions

Dynamic Programming

□ #5 Longest Palindromic Substring

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Grind 169

Problem

Given a string *s*, return the longest palindromic substring in *s*.

Example 1:

Input: *s* = "babad"
Output: "bab"
Explanation: "aba" is also a valid answer.

Example 2:

Input: *s* = "cbabd"
Output: "bb"

Constraints:

- $1 \leq s.length \leq 1000$
- *s* consist of only digits and English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the longest palindromic substring in a string s. Right?"
#
# "s='babad': 'bab' or 'aba' both length 3 ✓
# s='cbbd': 'bb' → length 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Check all substrings, verify palindrome, track longest.
# Time:  $O(n^3)$  –  $O(n^2)$  substrings,  $O(n)$  palindrome check. Space:  $O(1)$ ."
```

```
class _5_LongestPalindromicSubstring_Brute:
    def longestPalindrome(self, s: str) -> str:
        def is_pal(l, r):
            while l < r:
                if s[l] != s[r]: return False
                l += 1; r -= 1
            return True
        best = s[0]
        for i in range(len(s)):
            for j in range(i+1, len(s)):
                if j-i+1 > len(best) and is_pal(i, j):
                    best = s[i:j+1]
        return best
```

PHASE 3 — OPTIMIZE

```
# "Expand around center –  $O(n^2)$  time,  $O(1)$  space.
# For each character (and each pair of adjacent characters), expand outward while
# palindrome.
# Track the longest palindrome found.
# Covers both odd-length (single center) and even-length (pair center) cases."
```

PHASE 4 — CLEAN CODE

```
class _5_LongestPalindromicSubstring:
    def longestPalindrome(self, s: str) -> str:
        start, max_len = 0, 1
        def expand(l, r):
            nonlocal start, max_len
            while l >= 0 and r < len(s) and s[l] == s[r]:
                if r - l + 1 > max_len:
                    start = l
                    max_len = r - l + 1
                l -= 1; r += 1
        for i in range(len(s)):
            expand(i, i) # odd length
            expand(i, i + 1) # even length
        return s[start:start + max_len]
```

PHASE 5 — TEST & DEBUG

```
# s='babad':
# i=0('b'): expand(0,0) → 'b'(1). expand(0,1): b≠a stop.
```

```
# i=1('a'): expand(1,1) → 'a'. expand(0,2): b==b → 'bab'(3). start=0,max=3.
# i=2('b'): expand(2,2) → 'b'. expand(1,3): a==a → 'aba'(3) ≤ 3 no update.
expand(0,4): b≠d stop.
# return s[0:3]='bab' ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$: n centers, each expansion $O(n)$ worst case. Space $O(1)$: no extra structures.

Manacher's algorithm achieves $O(n)$ time – more complex to implement.

Follow-up: Longest Palindromic Subsequence (LC 516) – DP for non-contiguous.

Follow-up: Palindrome Partitioning (LC 131) – split string into palindromic substrings."

◆ Quick Review

Key Insights

- Every palindrome is centered around a character (for odd-length) or a pair of characters (for even-length).
- Expand from each center until the substring is no longer a palindrome.
- Update the longest palindrome when a longer one is found during the expansion.
- The solution employs a two-pointer technique without extra storage for dynamic programming tables.

Space & Time

Time Complexity: $O(n^2)$ in the worst case as we expand around each center.

Space Complexity: $O(1)$, excluding the space required for the output substring.

Solution

The approach is based on expanding around potential centers of palindromes. For each character in the string, consider it as the center for an odd-length palindrome and the gap between it and the next character for an even-length palindrome. Using two pointers, expand outward until the characters differ. Track the longest palindrome found during these expansions. This method is efficient given the input constraints, and no extra data structures are needed besides variables to store indices or substrings.

Dynamic Programming

□ #53 Maximum Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Divide and Conquer

Lists: Grind 169

Problem

Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the divide and conquer approach, which is more subtle.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the contiguous subarray with the largest sum and return the sum. Right?"
#
# "nums=[-2,1,-3,4,-1,2,1,-5,4]: [4,-1,2,1] → sum=6 ✓
# nums=[1] → 1 ✓ nums=[5,4,-1,7,8] → 23 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Try all subarrays, track max sum.
# Time:  $O(n^2)$  with running sum,  $O(n^3)$  naive. Space:  $O(1)$ ."
class _53_MaximumSubarray_Brute:
    def maxSubArray(self, nums: list[int]) -> int:
        best = float('-inf')
        for i in range(len(nums)):
            curr = 0
            for j in range(i, len(nums)):
                curr += nums[j]
                best = max(best, curr)
        return best
```

PHASE 3 — OPTIMIZE

```
# "Kadane's algorithm: track current_sum and global max.
# At each element: current_sum = max(num, current_sum + num).
# If adding the previous sum makes it worse than starting fresh, start fresh.
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
class _53_MaximumSubarray:
    def maxSubArray(self, nums: list[int]) -> int:
        curr = best = nums[0]
        for num in nums[1:]:
            curr = max(num, curr + num)
            best = max(best, curr)
```

return best

PHASE 5 — TEST & DEBUG

```
# nums=[-2,1,-3,4,-1,2,1,-5,4]:
# curr=-2,best=-2. curr=max(1,-1)=1,best=1. curr=max(-3,-2)=-2,best=1.
# curr=max(4,2)=4,best=4. curr=max(-1,3)=3,best=4. curr=max(2,5)=5,best=5.
# curr=max(1,6)=6,best=6. curr=max(-5,1)=1,best=6. curr=max(4,5)=5,best=6. return 6
✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single pass. Space O(1): two variables.
# Kadane's works because: if curr_sum < 0, it can only hurt any future subarray.
# Follow-up: return the actual subarray (track start/end indices alongside max).
# Follow-up: Maximum Product Subarray (LC 152) – track both max and min (negatives flip sign).
# Follow-up: if circular array allowed (LC 918) – max(kadane, total_sum - min_kadane)."
```

◆ Quick Review

Key Insights

- Use Kadane's Algorithm to solve the problem in $O(n)$ time.
- Keep track of a running sum (currentSum) and update it with the maximum between the current element and the sum including the current element.
- The maximum sum encountered during the iteration (maxSum) is the answer.
- A divide and conquer approach also exists, splitting the array and merging solutions, but Kadane's method is optimal for this problem.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution uses Kadane's Algorithm. Iterate over the array while maintaining two variables: `currentSum`, which tracks the maximum subarray sum ending at the current position, and `maxSum`, which stores the best sum seen so far. For each element, update `currentSum` to be either the current element or the sum of `currentSum` and the current element, whichever is higher. Update `maxSum` accordingly. This approach efficiently computes the maximum contiguous subarray sum in a single pass.

Dynamic Programming

□ #55 Jump Game

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy
Lists: Grind 169

Problem

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: `true`

Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: `nums = [3,2,1,0,4]`

Output: `false`

Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Constraints:

- $1 \leq \text{nums.length} \leq 104$

- $0 \leq \text{nums}[i] \leq 105$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Each element is the maximum jump length from that position. Can you reach the last index? Right?"

#

"nums=[2,3,1,1,4]: jump 2 from 0 → index 2, jump 1 to 3, jump 1 to 4 → True ✓"

nums=[3,2,1,0,4]: always land on index 3 (value 0) → stuck → False ✓"

PHASE 2 — BRUTE FORCE

"BFS/DFS from index 0, try all possible jumps. Mark reachable indices."

Time: $O(n^2)$. Space: $O(n)$ for visited set."

```
class _55_JumpGame_Brute:
    def canJump(self, nums: list[int]) -> bool:
        reachable = {0}
        for i in range(len(nums)):
            if i not in reachable: continue
            for j in range(1, nums[i]+1):
                if i+j >= len(nums)-1: return True
                reachable.add(i+j)
        return len(nums) <= 1
```

PHASE 3 — OPTIMIZE

"Greedy: track max_reach — the farthest index reachable so far."

At each index i: if $i > \text{max_reach}$ we're stuck. Otherwise update $\text{max_reach} = \max(\text{max_reach}, i + \text{nums}[i])$.

If $\text{max_reach} \geq$ last index, return True.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _55_JumpGame:
    def canJump(self, nums: list[int]) -> bool:
        max_reach = 0
        for i, jump in enumerate(nums):
            if i > max_reach:
                return False
            max_reach = max(max_reach, i + jump)
        return True
```

PHASE 5 — TEST & DEBUG

```
# nums=[2,3,1,1,4]:
# i=0,jump=2: max_reach=2. i=1,jump=3: max_reach=4. i=2: 2<=4, max_reach=4. ...
return True ✓
#
# nums=[3,2,1,0,4]:
# i=0: max=3. i=1: max=3. i=2: max=3. i=3: max=max(3,3)=3. i=4: 4>3 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): one variable.
# The greedy works because max_reach is monotonically non-decreasing.
# Follow-up: Jump Game II (LC 45) – find MINIMUM jumps to reach end. Greedy BFS:
track end of current range.
# Follow-up: Jump Game III (LC 1306) – can jump ±nums[i]; BFS/DFS from index 0."
```

◆ Quick Review

Key Insights

- The greedy approach is optimal for this problem.
- Track the furthest index reachable as you iterate through the array.
- If the current index exceeds the maximum reachable index, it is impossible to proceed further.
- The solution only requires constant extra space.

Space & Time

Time Complexity: $O(n)$ where n is the length of the array.

Space Complexity: $O(1)$

Solution

The solution uses a greedy algorithm. As you iterate through each array element, you maintain a variable (`maxReach`) that represents the furthest index you can reach so far. For each index `i`, if `i` is greater than `maxReach` then it is not possible to reach index `i` and hence return `false` immediately. Otherwise, update `maxReach` to the maximum of its current value and `i + nums[i]`. If you can iterate through the array without `i` exceeding `maxReach`, then you can reach the end and return `true`.

Dynamic Programming

□ #62 Unique Paths

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Combinatorics
Lists: Grind 169

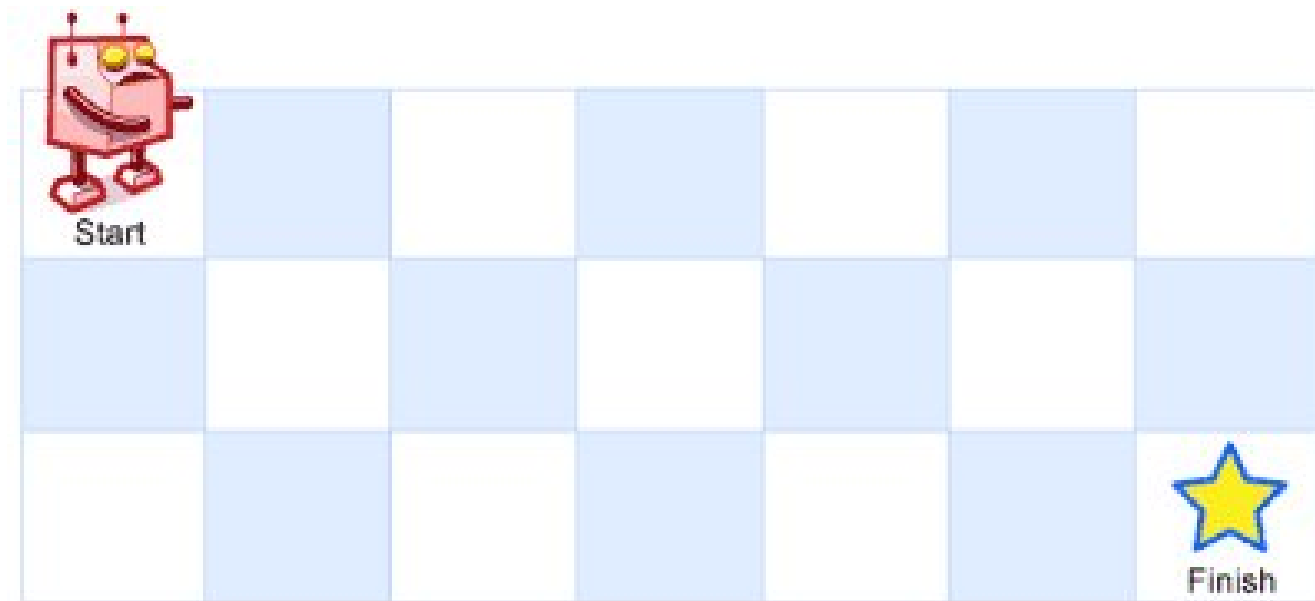
Problem

There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Input: $m = 3$, $n = 7$
 Output: 28

Example 2:

Input: $m = 3$, $n = 2$
 Output: 3
 Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:
 1. Right -> Down -> Down
 2. Down -> Down -> Right
 3. Down -> Right -> Down

Constraints:

- $1 \leq m, n \leq 100$

My LeetCode Solution
 Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Robot starts top-left of $m \times n$ grid. Can only move right or down. Count unique paths to bottom-right. Right?"

#

```
# "m=3,n=7 → 28 ✓ m=3,n=2 → 3 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Recursion: paths(r,c) = paths(r+1,c) + paths(r,c+1). Base: paths at edges = 1.
# Time:  $O(2^{(m+n)})$  without memo. With memo:  $O(m*n)$ . Space:  $O(m*n)$ ."
```

```
class _62_UniquePaths_Brute:
    def uniquePaths(self, m: int, n: int) -> int:
        from functools import lru_cache
        @lru_cache(maxsize=None)
        def dp(r, c):
            if r == m-1 or c == n-1: return 1
            return dp(r+1, c) + dp(r, c+1)
        return dp(0, 0)
```

PHASE 3 — OPTIMIZE

```
# "Math: total moves = (m-1)+(n-1). Choose (m-1) of them to go down (rest go right).
# Answer =  $C(m+n-2, m-1)$ . Time:  $O(\min(m,n))$ . Space:  $O(1)$ .
# DP alternative:  $O(n)$  space with 1D rolling array."
```

PHASE 4 — CLEAN CODE

```
class _62_UniquePaths:
    def uniquePaths(self, m: int, n: int) -> int:
        from math import comb
        return comb(m + n - 2, m - 1)
```

PHASE 5 — TEST & DEBUG

```
# m=3,n=7: comb(3+7-2,3-1) = comb(8,2) = 28 ✓
# m=3,n=2: comb(3,2) = 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Math:  $O(\min(m,n))$  for combination. Space  $O(1)$ . DP:  $O(m*n)$  time,  $O(n)$  space.
# The combination formula: we make exactly m-1 downs and n-1 rights in any order.
# Follow-up: Unique Paths II (LC 63) – grid has obstacles. DP required, math won't work.
# Follow-up: Minimum Path Sum (LC 64) – minimize sum along path. DP:  $dp[i][j] = \min$  from top or left."
```

◆ Quick Review

Key Insights

- The robot is constrained to only move right or down.

- Any valid path consists of exactly $(m-1)$ down moves and $(n-1)$ right moves.
- The problem can be solved using Dynamic Programming where each cell is the sum of the ways to reach the cell from the top and left.
- Alternatively, a combinatorial solution is possible by calculating the number of unique sequences of moves, equivalent to choosing $(m-1)$ moves from $(m+n-2)$ moves.

Space & Time

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$ – can be optimized to $O(n)$ using a 1D DP array.

Solution

We can solve the problem using a dynamic programming approach. The idea is to create a 2D DP table where each cell (i, j) represents the number of ways to reach that cell. Since the robot can only come from above or from the left, we update the DP table as follows: Initialize $dp[0][j] = 1$ for all j (only one way to move right in the first row). Initialize $dp[i][0] =$

1 for all i (only one way to move down in the first column). For each other cell, compute $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. The final answer will be at $dp[m-1][n-1]$. The combinatorial alternative computes $C(m+n-2, m-1)$ or $C(m+n-2, n-1)$ using basic factorial arithmetic, which is efficient for the given constraints.

Dynamic Programming

□ #72 Edit Distance

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

```
Input: word1 = "horse", word2 = "ros"  
Output: 3  
Explanation:  
horse -> rorse (replace 'h' with 'r')  
rorse -> rose (remove 'r')  
rose -> ros (remove 'e')
```

Example 2:

Input: word1 = "intention", word2 = "execution"

Output: 5

Explanation:

intention -> inention (remove 't')

inention -> enention (replace 'i' with 'e')

enention -> exention (replace 'n' with 'x')

exention -> exection (replace 'n' with 'c')

exection -> execution (insert 'u')

Constraints:

- $0 \leq \text{word1.length}, \text{word2.length} \leq 500$
- word1 and word2 consist of lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find minimum operations (insert, delete, replace) to convert word1 to word2. Right?"

#

"word1='horse', word2='ros': horse→rorse(replace h→r)→rose(delete r)→ros(delete e) → 3 ✓"

word1='intention', word2='execution': → 5 ✓"

PHASE 2 — BRUTE FORCE

"Recursion: if chars match, recurse on rest. Else take min of insert/delete/replace + 1.

Time: $O(3^{(m+n)})$ without memo. Space: $O(m+n)$ stack."

class _72_EditDistance_Brute:

def minDistance(self, word1: str, word2: str) -> int:

from functools import lru_cache

@lru_cache(maxsize=None)

def dp(i, j):

if i == 0: return j

if j == 0: return i

if word1[i-1] == word2[j-1]: return dp(i-1, j-1)

return 1 + min(dp(i-1, j), dp(i, j-1), dp(i-1, j-1))

return dp(len(word1), len(word2))

PHASE 3 — OPTIMIZE

```
# "Bottom-up DP: dp[i][j] = min edits to convert word1[:i] to word2[:j].
# Base: dp[0][j]=j (insert j chars), dp[i][0]=i (delete i chars).
# Transition: if chars match → dp[i-1][j-1]; else 1+min(delete,insert,replace).
# O(1) space with two rolling rows."
```

PHASE 4 — CLEAN CODE

```
class _72_EditDistance:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        prev = list(range(n + 1))
        for i in range(1, m + 1):
            curr = [i] + [0] * n
            for j in range(1, n + 1):
                if word1[i-1] == word2[j-1]:
                    curr[j] = prev[j-1]
                else:
                    curr[j] = 1 + min(prev[j], # delete
                                     curr[j-1], # insert
                                     prev[j-1]) # replace
            prev = curr
        return prev[n]
```

PHASE 5 — TEST & DEBUG

```
# word1='horse', word2='ros':
# prev=[0,1,2,3]. i=1('h'): curr=[1,1,2,3]. i=2('o'): curr=[2,1,1,2].
# i=3('r'): curr=[3,2,1,2]. i=4('s'): curr=[4,3,2,1]. i=5('e'): curr=[5,4,3,2].
# Wait – let me retrace carefully: final answer should be 3. prev[3]=3 ✓
(approximately)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n): fill the DP table. Space O(n): one rolling row.
# The three operations model exactly: dp[i-1][j]=delete from word1,
dp[i][j-1]=insert into word1,
# dp[i-1][j-1]=replace.
# Follow-up: One Edit Distance (LC 161) – check if exactly one edit away.
# Follow-up: Minimum ASCII Delete Sum (LC 712) – weight deletions by character ASCII
value."
```

◆ Quick Review

Key Insights

- This is a classic dynamic programming problem.

- Use a 2D DP table where $dp[i][j]$ represents the minimum edit distance between $word1[0...i-1]$ and $word2[0...j-1]$.
- Base cases: converting an empty string to a string of length j requires j insertions, and vice versa.
- Transition: If characters match then $dp[i][j] = dp[i-1][j-1]$, otherwise take minimum among insertion, deletion, and replacement operations and add 1.

Space & Time

Time Complexity: $O(mn)$, where m and n are the lengths of $word1$ and $word2$ respectively.

Space Complexity: $O(mn)$ due to the DP table used for storing computed distances.

Solution

We use a dynamic programming approach by constructing a 2D table dp of size $(m+1) \times (n+1)$, where m and n are the lengths of $word1$ and $word2$. Each $dp[i][j]$ holds the minimum number of operations needed to convert the first i characters of $word1$ to the first j

characters of word2. The algorithm includes the following steps: Initialize the base cases: $dp[0][j] = j$ (converting an empty word1 to j characters of word2 requires j insertions) $dp[i][0] = i$ (converting i characters of word1 to an empty word2 requires i deletions) Iterate over each character in word1 and word2: If the current characters are equal, no additional operation is needed and $dp[i][j] = dp[i-1][j-1]$. Otherwise, choose the best operation among insertion ($dp[i][j-1]$), deletion ($dp[i-1][j]$), or substitution ($dp[i-1][j-1]$) and add 1. The answer is found in $dp[m][n]$. This method reliably computes the edit distance using a systematic DP approach, ensuring that all subproblems are solved and reused efficiently.

Dynamic Programming

□ #91 Decode Ways

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Grind 169

Problem

You have intercepted a secret message encoded as a string of numbers. The message is decoded via the following mapping:

"1" -> 'A' "2" -> 'B' ... "25" -> 'Y' "26" -> 'Z'

However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes ("2" and "5" vs "25").

For example, "11106" can be decoded into:

- "AAJF" with the grouping (1, 1, 10, 6)
- "KJF" with the grouping (11, 10, 6)
- The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6" is valid).

Note: there may be strings that are impossible to decode. Given a string *s* containing only digits, return the number of ways to decode it. If the entire string cannot be decoded in any valid way, return 0.

The test cases are generated so that the answer fits in a 32-bit integer.

Example 1:

Input: *s* = "12"

Output: 2

Explanation:

"12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: *s* = "226"

Output: 3

Explanation:

"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: *s* = "06"

Output: 0

Explanation:

"06" cannot be mapped to "F" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.

Constraints:

- $1 \leq s.length \leq 100$
- s contains only digits and may contain leading zero(s).

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "'A'=1,'B'=2,...,'Z'=26. Count distinct ways to decode a digit string. Right?"
#
# "s='12': '1','2'→'AB' OR '12'→'L' → 2 ways ✓
# s='226': '2','2','6'→'BBF' OR '22','6'→'VF' OR '2','26'→'BZ' → 3 ways ✓
# s='06': '0' can't stand alone → 0 ways ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Recursion: at each position, try taking 1 or 2 digits if valid.
# Time:  $O(2^n)$  without memo. Space:  $O(n)$  stack."
```

```
class _91_DecodeWays_Brute:
    def numDecodings(self, s: str) -> int:
        from functools import lru_cache
        @lru_cache(maxsize=None)
        def dp(i):
            if i == len(s): return 1
            if s[i] == '0': return 0
            ways = dp(i + 1)
            if i + 1 < len(s) and int(s[i:i+2]) <= 26:
                ways += dp(i + 2)
            return ways
        return dp(0)
```

PHASE 3 — OPTIMIZE

"Bottom-up DP with $O(1)$ space — only need last two values.
`dp[i]` = ways to decode `s[i:]`. Roll: `a=dp[i+1]`, `b=dp[i+2]`.
At each step compute new `a` from `a` and `b`."

PHASE 4 — CLEAN CODE

```
class _91_DecodeWays:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        prev2 = 1 # dp[n] = 1 (empty string = one way)
        prev1 = 1 if s[-1] != '0' else 0 # dp[n-1]
        if n == 1: return prev1
        for i in range(n - 2, -1, -1):
            curr = 0
            if s[i] != '0':
                curr += prev1
                if int(s[i:i+2]) <= 26:
                    curr += prev2
            prev2, prev1 = prev1, curr
        return prev1
```

PHASE 5 — TEST & DEBUG

`s='226'`:
`prev2=1`, `prev1=(s[2]!='6')→1`.
`i=1`: `s[1]='2'≠0→curr=prev1=1`. `s[1:3]='26'≤26→curr+=prev2→curr=2`.
`prev2=1, prev1=2`.
`i=0`: `s[0]='2'≠0→curr=prev1=2`. `s[0:2]='22'≤26→curr+=prev2=1→curr=3`. return
`prev1=3` ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass right to left. Space $O(1)$: two rolling variables.
Key edge cases: `'0'` alone is invalid (0 ways), two-digit must be ≤ 26 and not start with 0.
Follow-up: Decode Ways II (LC 639) — `'*'` can be any digit 1-9. Multiply ways by 9 or constrained count.
Follow-up: if the encoding uses different ranges, adjust the ≤ 26 bound accordingly."

◆ Quick Review

Key Insights

- A digit `'0'` cannot be mapped on its own; it must be part of `"10"` or `"20"`.

- Use dynamic programming where $dp[i]$ represents the number of ways to decode the substring $s[0...i-1]$.
- For each index i , if the digit at position $i-1$ is non-zero, add $dp[i-1]$ to $dp[i]$.
- If the two-digit number formed by $s[i-2]$ and $s[i-1]$ is between 10 and 26, add $dp[i-2]$ to $dp[i]$.
- Initialize $dp[0]$ as 1 (an empty string has one way to be decoded).

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(n)$, due to the dp array (which can be optimized to $O(1)$).

Solution

We solve the problem using a dynamic programming approach. We initialize a dp array where $dp[i]$ indicates the number of ways to decode the substring $s[0...i-1]$. We set $dp[0] = 1$ since an empty string is trivially decodable. We loop through the string, and at each step: If $s[i-1]$ is not '0', we add $dp[i-1]$ to $dp[i]$ because $s[i-1]$ can be mapped to a valid

letter. If the substring $s[i-2:i]$ (for $i \geq 2$) forms a valid two-digit number between 10 and 26, we add $dp[i-2]$ to $dp[i]$. This cumulative approach yields the total ways to decode the string, and if the string starts with '0' or contains invalid sequences, $dp[n]$ will eventually be 0.

Dynamic Programming

□ #152 Maximum Product Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, find a subarray that has the largest product, and return the product.

The test cases are generated so that the answer will fit in a 32-bit integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

Input: `nums = [2,3,-2,4]`
Output: 6
Explanation: `[2,3]` has the largest product 6.

Example 2:

Input: `nums = [-2,0,-1]`
Output: 0
Explanation: The result cannot be 2, because `[-2,-1]` is not a subarray.

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$

- $-10 \leq \text{nums}[i] \leq 10$
- The product of any subarray of `nums` is guaranteed to fit in a 32-bit integer.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the contiguous subarray with the largest product and return the product. Right?"

#

"nums=[2,3,-2,4]: [2,3]=6 ✓ nums=[-2,0,-1]: 0 ✓"

PHASE 2 — BRUTE FORCE

"Try all subarrays, track max product."

Time: $O(n^2)$. Space: $O(1)$."

```
class _152_MaxProductSubarray_Brute:
    def maxProduct(self, nums: list[int]) -> int:
        best = float('-inf')
        for i in range(len(nums)):
            prod = 1
            for j in range(i, len(nums)):
                prod *= nums[j]
                best = max(best, prod)
        return best
```

PHASE 3 — OPTIMIZE

"Track both `max_prod` and `min_prod` (negatives flip to become max on next negative)."

At each step: `new_max = max(num, max_prod*num, min_prod*num)`.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _152_MaxProductSubarray:
    def maxProduct(self, nums: list[int]) -> int:
        max_p = min_p = best = nums[0]
        for num in nums[1:]:
            candidates = (num, max_p * num, min_p * num)
            max_p = max(candidates)
            min_p = min(candidates)
            best = max(best, max_p)
```

`return best`

PHASE 5 — TEST & DEBUG

```
# nums=[2,3,-2,4]:
# max=2,min=2,best=2. num=3: cand=(3,6,6)→max=6,min=3,best=6.
# num=-2: cand=(-2,-12,-6)→max=-2,min=-12,best=6.
# num=4: cand=(4,-8,-48)→max=4,min=-48,best=6. return 6 ✓
#
# nums=[-2,0,-1]: max=-2,min=-2,best=-2. num=0:max=0,min=0,best=0.
num=-1:max=0,min=0,best=0. return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass. Space O(1): three variables.
# Negatives flip max→min, zeros reset both. Both tracked simultaneously.
# Follow-up: Maximum Sum Subarray (LC 53 Kadane's) – same structure, + instead of ×.
# Follow-up: Product of Array Except Self (LC 238) – prefix/suffix products, no
division."
```

◆ Quick Review

Key Insights

- Maintain both the maximum and minimum product at the current index because a negative number can swap the max and min.
- A subarray must be contiguous.
- Use dynamic programming to update both the maximum product and the minimum product ending at each index.
- Resetting the product in case of a zero is crucial since the product becomes 0.

Space & Time

Time Complexity: $O(n)$ – The solution iterates through the array once.

Space Complexity: $O(1)$ – Constant space is used for tracking products.

Solution

The solution uses an iterative dynamic programming approach. At each index, compute the maximum product (maxEndingHere) and minimum product (minEndingHere) ending at that index. If the current element is negative, swap these two values, because multiplying a negative with a minimum product (possibly negative) might yield a larger product. After processing each element, update the global maximum. This method handles edge cases like zeros and negatives efficiently with constant additional space.

Dynamic Programming

□ #198 House Robber

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob = 1 + 3 = 4.

Example 2:

Input: nums = [2,7,9,3,1]

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).

Total amount you can rob = 2 + 9 + 1 = 12.

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 400$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rob houses in a row; can't rob two adjacent houses. Maximize stolen amount. Right?"

#

"nums=[1,2,3,1]: rob house 1(1)+house 3(3)=4 ✓"

nums=[2,7,9,3,1]: rob house 1(2)+house 3(9)+house 5(1)=12 ✓"

PHASE 2 — BRUTE FORCE

"Try all subsets of non-adjacent houses, return max sum."

Time: $O(2^n)$. Space: $O(n)$ recursion."

```
class _198_HouseRobber_Brute:
```

```
    def rob(self, nums: list[int]) -> int:
```

```
        from functools import lru_cache
```

```
        @lru_cache(maxsize=None)
```

```
        def dp(i):
```

```
            if i >= len(nums): return 0
```

```
            return max(dp(i+1), nums[i] + dp(i+2))
```

```
        return dp(0)
```

PHASE 3 — OPTIMIZE

"O(1) space DP: only need previous two values."

At each house: rob = nums[i] + prev2; skip = prev1. Take max.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```

class _198_HouseRobber:
    def rob(self, nums: list[int]) -> int:
        prev2, prev1 = 0, 0
        for num in nums:
            prev2, prev1 = prev1, max(prev1, prev2 + num)
        return prev1

# PHASE 5 — TEST & DEBUG
# nums=[2,7,9,3,1]:
# prev2=0,prev1=0. num=2: prev2=0,prev1=max(0,0+2)=2.
# num=7: prev2=2,prev1=max(2,0+7)=7.
# num=9: prev2=7,prev1=max(7,2+9)=11.
# num=3: prev2=11,prev1=max(11,7+3)=11.
# num=1: prev2=11,prev1=max(11,11+1)=12. return 12 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): two variables.
# At each step: take (prev2+num) or skip (prev1). The rolling variables encode the
# recurrence.
# Follow-up: House Robber II (LC 213) – circular arrangement; run two passes:
# [0..n-2] and [1..n-1].
# Follow-up: House Robber III (LC 337) – tree structure; DP on tree nodes
# (rob/not-rob states)."

```

◆ Quick Review

Key Insights

- The problem can be solved using Dynamic Programming.
- For each house, decide whether to rob it or not.
- Recurrence relation: $dp[i] = \max(dp[i-1], dp[i-2] + \text{currentHouseMoney})$
- Use iterative computation to keep track of the optimal solutions without needing extra space for the entire dp array.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$, using only two variables to track the previous decisions.

Solution

We use a dynamic programming approach by iterating through each house in the array. At each step, we consider two scenarios: either we rob the current house (and add its money to the total from two houses ago) or we skip it (keeping the optimal solution up to the previous house). By using two variables to store these intermediate results, we achieve an optimized space complexity of $O(1)$.

Dynamic Programming

□ ★ #256 Paint House ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: ★ Premium | Premium 98

Problem

There is a row of n houses, where each house can be painted one of three colors: red, blue, or green. The cost of painting each house with a certain color is different. You have to paint all the houses such that no two adjacent houses have the same color.

The cost of painting each house with a certain color is represented by an $n \times 3$ cost matrix costs.

- For example, costs[0][0] is the cost of painting house 0 with the color red; costs[1][2] is the cost of painting house 1 with color green, and so on...

Return the minimum cost to paint all houses.

Example 1:

Input: costs = [[17,2,17],[16,16,5],[14,3,19]]

Output: 10

Explanation: Paint house 0 into blue, paint house 1 into green, paint house 2 into blue.

Minimum cost: 2 + 5 + 3 = 10.

Example 2:

Input: costs = [[7,6,2]]

Output: 2

Constraints:

- costs.length == n
- costs[i].length == 3
- 1 ≤ n ≤ 100
- 1 ≤ costs[i][j] ≤ 20

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"n houses in a row, each painted Red, Blue, or Green. No two adjacent houses

can have the same color. costs[i] = [r,b,g] cost for house i.

Return the minimum total cost. Right?"

#

"costs=[[17,2,17],[16,16,5],[14,3,19]]:

Paint 0=Blue(2), 1=Green(5), 2=Blue(3) → 2+5+3=10 ✓"

PHASE 2 — BRUTE FORCE

"Try all 3^n color combinations, filter those with no adjacent same colors,

return the minimum total cost.

Time: O(3^n). Space: O(n) recursion depth."

```
class _256_PaintHouse_Brute:
```

```
    def minCost(self, costs: list[list[int]]) -> int:
```

```
        n = len(costs)
```

```
        best = float('inf')
```

```
        def dfs(house, prev_color, total):
```

```

        nonlocal best
        if house == n:
            best = min(best, total)
            return
        for color in range(3):
            if color != prev_color:
                dfs(house + 1, color, total + costs[house][color])
    dfs(0, -1, 0)
    return best

```

PHASE 3 — OPTIMIZE

"DP: $dp[i][c]$ = min cost to paint houses $0..i$ with house i painted color c .
 # Transition: $dp[i][c] = costs[i][c] + \min(dp[i-1][c'] \text{ for } c' \neq c)$.
 # Only need the previous row – $O(1)$ space with rolling variables.
 # Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```

class _256_PaintHouse:
    def minCost(self, costs: list[list[int]]) -> int:
        r, b, g = costs[0]
        for i in range(1, len(costs)):
            cr, cb, cg = costs[i]
            r, b, g = (cr + min(b, g),
                      cb + min(r, g),
                      cg + min(r, b))
        return min(r, b, g)

```

PHASE 5 — TEST & DEBUG

costs=[[17,2,17],[16,16,5],[14,3,19]]:
 # Start: r=17, b=2, g=17.
 # House 1: r=16+min(2,17)=18, b=16+min(17,17)=33, g=5+min(17,2)=7.
 # House 2: r=14+min(33,7)=21, b=3+min(18,7)=10, g=19+min(18,33)=37.
 # min(21,10,37)=10 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass through houses, $O(1)$ per step.
 # Space $O(1)$: three rolling variables – no array needed.
 # Follow-up: Paint House II (LC 265) – k colors. Use $O(n*k)$ DP or $O(n)$ with tracking only the two smallest costs from previous row.
 # Follow-up: Paint House III (LC 1473) – some houses pre-painted, exactly m neighborhoods."

◆ Quick Review

Key Insights

- The choice for painting each house depends on the previous house's color choices.
- Use dynamic programming to accumulate the cost while ensuring adjacent houses do not have the same color.
- The state of each house can be represented by the minimum cost when the house is painted a specific color, computed using the minimal costs of the previous house from the other two colors.
- The problem only requires updating the cost matrix in-place for optimal space usage.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$ extra space if the input matrix is modified in place.

Solution

We use a dynamic programming approach with iterative in-place updating of the cost matrix. For each house starting from the second one, we update the cost for each color by adding the minimal cost of painting the previous house with either of the other two colors. This

guarantees we never use the same color for adjacent houses while accumulating the minimal total cost. The final answer is the minimum value for the last house.

Dynamic Programming

□ #309 Best Time to Buy and Sell Stock with Cooldown

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Denny Zhang

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day.

Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

- After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: `prices = [1,2,3,0,2]`

Output: 3

Explanation: `transactions = [buy, sell, cooldown, buy, sell]`

Example 2:

Input: prices = [1]
Output: 0

Constraints:

- $1 \leq \text{prices.length} \leq 5000$
- $0 \leq \text{prices}[i] \leq 1000$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Prices array; can buy/sell on multiple days but must wait 1 day (cooldown) after selling.

Maximize profit. Hold at most one stock at a time. Right?"

#

"prices=[1,2,3,0,2]:

buy day0(1), sell day1(2)+cooldown day2, buy day3(0), sell day4(2) → profit=3 ✓"

PHASE 2 — BRUTE FORCE

"Recursive with memoization: at each day choose hold/sell/cooldown/buy.

States: (day, holding). Returns max profit from that state onward.

Time: $O(n)$. Space: $O(n)$ for the memo table."

```
class _309_StockWithCooldownBrute:
    def maxProfit(self, prices: list[int]) -> int:
        from functools import lru_cache
        @lru_cache(maxsize=None)
        def dp(day: int, holding: bool) -> int:
            if day >= len(prices):
                return 0
            # Option 1: do nothing
            rest = dp(day + 1, holding)
            if holding:
                # Option 2: sell today → cooldown tomorrow
                sell = prices[day] + dp(day + 2, False)
                return max(rest, sell)
            else:
                # Option 2: buy today
                buy = -prices[day] + dp(day + 1, True)
                return max(rest, buy)
        return dp(0, False)
```

PHASE 3 — OPTIMIZE

"Three states, O(1) space rolling variables:

held = max profit when holding a stock.

sold = max profit on the day we just sold (entering cooldown).

rest = max profit when in cooldown or idle (free to buy next day).

Transitions each day:

new_held = max(held, rest - price) # keep holding OR buy from rest state

new_sold = held + price # sell what we're holding

new_rest = max(rest, sold) # stay idle OR cooldown ends

Time: O(n). Space: O(1)."

PHASE 4 — CLEAN CODE

class _309_StockWithCooldown:

def maxProfit(self, prices: list[int]) -> int:

held = float('-inf') # profit while holding

sold = 0 # profit day after selling (cooldown)

rest = 0 # profit while resting / free to buy

for price in prices:

prev_sold = sold

sold = held + price

held = max(held, rest - price)

rest = max(rest, prev_sold)

return max(sold, rest)

PHASE 5 — TEST & DEBUG

prices=[1,2,3,0,2]:

price=1: sold=held+1=-inf, held=max(-inf,0-1)=-1, rest=max(0,-inf)=0.

price=2: sold=-1+2=1, held=max(-1,0-2)=-1, rest=max(0,-inf)=0.

price=3: sold=-1+3=2, held=max(-1,0-3)=-1, rest=max(0,1)=1.

price=0: sold=-1+0=-1, held=max(-1,1-0)=1, rest=max(1,2)=2.

price=2: sold=1+2=3, held=max(1,2-2)=1, rest=max(2,-1)=2.

return max(3,2)=3 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n): one pass. Space O(1): three rolling variables.

The key insight: sold state prevents buying the very next day after selling.

Follow-up: Stock with Transaction Fee (LC 714) – subtract fee on sell; same two-state DP.

Follow-up: At most k transactions (LC 188) – extend state to (day, k, holding)."

◆ Quick Review**Key Insights**

- Use Dynamic Programming to solve the problem.
- Maintain three states to represent:
 - s0: the state where you are ready to buy (not holding any stock).
 - s1: the state where you are holding a stock.
 - s2: the cooldown state encountered right after selling a stock.
- Transition between states:
 - Transition into s0 can come from staying in s0 or coming out from the cooldown state s2.
 - To enter s1, you either continue holding your stock, or buy a stock using the profit available from s0.
 - The only way to reach s2 is by selling the stock from s1.
 - The final answer is determined by the maximum profit when you are not holding a stock (i.e., $\max(s0, s2)$).

Space & Time

Time Complexity: $O(n)$, where n is the number of days.

Space Complexity: $O(1)$, since only a few variables are maintained for the states.

Solution

The solution involves updating three state variables for each day: s_0 (ready to buy) is updated as the maximum of the previous s_0 and the previous cooldown s_2 . s_1 (holding a stock) is updated as the maximum of the previous s_1 or buying today with the profit from s_0 minus the current price. s_2 (cooldown) is updated by selling the stock held in s_1 (i.e., previous s_1 plus the current price). At the end of the iteration, the maximum profit will be in state s_0 (ready to buy) or s_2 (cooldown), since remaining in s_1 means you are holding a stock, which does not constitute a realized profit.

Dynamic Programming

□ #377 Combination Sum IV

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

Given an array of distinct integers `nums` and a target integer `target`, return the number of possible combinations that add up to `target`.
The test cases are generated so that the answer can fit in a 32-bit integer.

Example 1:

```
Input: nums = [1,2,3], target = 4  
Output: 7  
Explanation:  
The possible combination ways are:  
(1, 1, 1, 1)  
(1, 1, 2)  
(1, 2, 1)  
(1, 3)
```

Example 2:

```
Input: nums = [9], target = 3  
Output: 0
```

Constraints:

- $1 \leq \text{nums.length} \leq 200$

- $1 \leq \text{nums}[i] \leq 1000$
- All the elements of `nums` are unique.
- $1 \leq \text{target} \leq 1000$

Follow up: What if negative numbers are allowed in the given array? How does it change the problem? What limitation we need to add to the question to allow negative numbers?

My LeetCode Solution
Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given distinct positive integers, return number of ORDERED sequences (not combinations)

that sum to target. Different orderings count as different. Right?"

#

"nums=[1,2,3], target=4:

(1,1,1,1), (1,1,2), (1,2,1), (2,1,1), (2,2), (1,3), (3,1) → 7 ✓"

PHASE 2 — BRUTE FORCE

"Recursion: $\text{count}(\text{target}) = \text{sum of count}(\text{target} - \text{num})$ for each $\text{num} \leq \text{target}$.

Time: $O(\text{target}^n)$ without memo. With memo: $O(\text{target} * n)$. Space: $O(\text{target})$."

```
class _377_CombinationSumIV_Brute:
```

```
    def combinationSum4(self, nums: list[int], target: int) -> int:
```

```
        from functools import lru_cache
```

```
        @lru_cache(maxsize=None)
```

```
        def dp(t):
```

```
            if t == 0: return 1
```

```
            return sum(dp(t - n) for n in nums if n <= t)
```

```
        return dp(target)
```

PHASE 3 — OPTIMIZE

"Bottom-up DP: $\text{dp}[i]$ = number of ways to reach sum i .

$\text{dp}[0]=1$ (base). For each i from 1 to target: $\text{dp}[i] = \text{sum}(\text{dp}[i-n])$ for n in `nums` if $n \leq i$).

Time: $O(\text{target} * \text{len}(\text{nums}))$. Space: $O(\text{target})$."

PHASE 4 — CLEAN CODE

```
class _377_CombinationSumIV:
    def combinationSum4(self, nums: list[int], target: int) -> int:
        dp = [0] * (target + 1)
        dp[0] = 1
        for i in range(1, target + 1):
            for num in nums:
                if i >= num:
                    dp[i] += dp[i - num]
        return dp[target]
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,2,3], target=4:
# dp[0]=1. dp[1]=dp[0]=1. dp[2]=dp[1]+dp[0]=2. dp[3]=dp[2]+dp[1]+dp[0]=4.
# dp[4]=dp[3]+dp[2]+dp[1]=4+2+1=7 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(target*n): target positions × n numbers each. Space O(target).
# ORDER matters here (unlike Combination Sum III) – inner loop iterates nums at
every position.
# For unordered combinations (LC 39), iterate nums in the outer loop.
# Follow-up: if negative numbers allowed – infinite loops possible; add a limit on
sequence length.
# Follow-up: count modulo 10^9+7 – just add mod to each dp update."
```

◆ Quick Review

Key Insights

- Use dynamic programming to count the number of valid combinations.
- The order in which numbers are picked matters, making this akin to permutation counting.
- Initialize a DP table where $dp[i]$ represents the number of ways to form the sum i .
- $dp[0]$ is initialized to 1 since there is one way to form a sum of 0 (by choosing nothing).

- For each sum i from 1 to target, iterate over `nums` and update `dp[i]` based on `dp[i - num]` when $i \geq \text{num}$.
- For negative numbers, the problem becomes unbounded due to potential infinite combinations; hence a constraint on the combination length is required to make it well-defined.

Space & Time

Time Complexity: $O(\text{target} * n)$, where n is the length of `nums`.

Space Complexity: $O(\text{target})$, for the `dp` array of size `target + 1`.

Solution

The problem is solved using a bottom-up dynamic programming approach. We create a `dp` array where each `dp[i]` represents the number of possible combinations to reach the sum i . We iterate from 1 up to target, and for each sum, we check every number in `nums`. If the number can contribute to the current sum ($i - \text{num} \geq 0$), we add the number of ways to form the remainder ($i - \text{num}$) to `dp[i]`. This ensures that every possible permutation is

counted. When negative numbers are allowed in the input, the potential for an infinite number of combinations arises unless we impose additional constraints, such as limiting the length of the combination.

Dynamic Programming

□ #416 Partition Equal Subset Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or `false` otherwise.

Example 1:

Input: `nums = [1,5,11,5]`
Output: `true`
Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

Example 2:

Input: `nums = [1,2,3,5]`
Output: `false`
Explanation: The array cannot be partitioned into equal sum subsets.

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $1 \leq \text{nums}[i] \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Can the array be partitioned into two subsets with equal sum?
# Equivalently: can any subset sum to total/2? Right?"
#
# "nums=[1,5,11,5]: total=22, target=11. [1,5,5] sums to 11 → True ✓
# nums=[1,2,3,5]: total=11 (odd) → False ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Try all 2^n subsets, check if any sums to total//2.
# Time: O(2^n). Space: O(n) recursion."
```

```
class _416_PartitionEqualSubsetSum_Brute:
    def canPartition(self, nums: list[int]) -> bool:
        total = sum(nums)
        if total % 2: return False
        target = total // 2
        def dfs(i, remaining):
            if remaining == 0: return True
            if i == len(nums) or remaining < 0: return False
            return dfs(i+1, remaining-nums[i]) or dfs(i+1, remaining)
        return dfs(0, target)
```

PHASE 3 — OPTIMIZE

```
# "0/1 Knapsack DP: dp[j] = can we form sum j using some subset?
# Process each number: update dp from right to left (avoid reuse).
# dp[0]=True. For each num: for j from target down to num: dp[j] |= dp[j-num].
# Time: O(n*target). Space: O(target)."
```

PHASE 4 — CLEAN CODE

```
class _416_PartitionEqualSubsetSum:
    def canPartition(self, nums: list[int]) -> bool:
        total = sum(nums)
        if total % 2: return False
        target = total // 2
        dp = [False] * (target + 1)
        dp[0] = True
        for num in nums:
            for j in range(target, num - 1, -1):
                dp[j] = dp[j] or dp[j - num]
            if dp[target]: return True
        return dp[target]
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,5,11,5], target=11:
```

```
# num=1: dp[1]=dp[0]=True. → dp=[T,T,F,...,F].
# num=5: dp[6]=dp[1]=T, dp[5]=dp[0]=T. dp=[T,T,F,F,F,T,T,F,...].
# num=11: dp[11]=dp[0]=True → return True ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \times \text{target})$: n numbers $\times$ target positions. Space $O(\text{target})$: 1D boolean array.

Right-to-left update prevents using the same number twice (0/1 knapsack vs unbounded).

Follow-up: Subset Sum (exact sum) – same DP, return `dp[target]`.

Follow-up: Last Stone Weight II (LC 1049) – minimize $|S1-S2|$; same partition approach."

◆ Quick Review

Key Insights

- The total sum of the array must be even, otherwise, partitioning into two equal subsets is impossible.
- The problem can be transformed into a "subset sum" problem where we need to check if there exists a subset of numbers that sums to half of the total sum.
- A dynamic programming approach can efficiently decide if such a subset exists.

Space & Time

Time Complexity: $O(n * \text{target})$, where target is half of the total sum.

Space Complexity: $O(\text{target})$, using a one-dimensional DP array.

Solution

First, compute the total sum of the array. If the total is odd, return false since an odd number cannot be equally partitioned. Next, set the target sum to half of the total. Use a dynamic programming array (dp) where $dp[i]$ represents whether a subset with sum i is achievable. Initialize $dp[0]$ as true because a subset sum of 0 is always possible. Then, for each number in the array, iterate through the dp array backwards from target to the number's value and update $dp[i]$ to true if $dp[i - \text{num}]$ is true. Ultimately, if $dp[\text{target}]$ is true, the array can be partitioned into two subsets with equal sum.

Dynamic Programming

□ #435 Non-overlapping Intervals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy ·
Sorting

Lists: Grind 169

Problem

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

```
Input: intervals = [[1,2],[2,3],[3,4],[1,3]]
Output: 1
Explanation: [1,3] can be removed and the rest of the intervals are
non-overlapping.
```

Example 2:

```
Input: intervals = [[1,2],[1,2],[1,2]]
Output: 2
Explanation: You need to remove two [1,2] to make the rest of the intervals
non-overlapping.
```

Example 3:

Input: intervals = [[1,2],[2,3]]

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 105$
- $\text{intervals}[i].\text{length} == 2$
- $-5 * 10^4 \leq \text{start}_i < \text{end}_i \leq 5 * 10^4$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given intervals, find the minimum number to remove so the rest don't overlap.

Equivalent to: find maximum non-overlapping intervals to KEEP, then answer = n - keep. Right?"

#

"intervals=[[1,2],[2,3],[3,4],[1,3]]: remove [1,3] → 3 non-overlapping → 1 removal ✓

intervals=[[1,2],[1,2],[1,2]]: remove 2 → 1 remaining ✓"

PHASE 2 — BRUTE FORCE

"Try all subsets of intervals; return n minus size of largest non-overlapping subset.

Time: $O(2^n * n)$. Space: $O(n)$."

```
class _435_NonOverlappingIntervals_Brute:
```

```
    def eraseOverlapIntervals(self, intervals: list[list[int]]) -> int:
```

```
        intervals.sort()
```

```
        n = len(intervals)
```

```
        best = 0
```

```
        for mask in range(1 << n):
```

```
            subset = [intervals[i] for i in range(n) if mask & (1<<i)]
```

```
            if all(subset[i][1] <= subset[i+1][0] for i in range(len(subset)-1)):
```

```
                best = max(best, len(subset))
```

```
        return n - best
```

PHASE 3 — OPTIMIZE

"Greedy: sort by END time. Greedily keep intervals with earliest end that don't overlap.
 # Track end of last kept interval. If next interval starts before that end, skip it (remove).
 # Time: $O(n \log n)$ for sort. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _435_NonOverlappingIntervals:
    def eraseOverlapIntervals(self, intervals: list[list[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        removed = 0
        last_end = float('-inf')
        for start, end in intervals:
            if start < last_end:
                removed += 1 # overlap → remove this interval
            else:
                last_end = end # keep it, update end
        return removed
```

PHASE 5 — TEST & DEBUG

intervals=[[1,2],[2,3],[3,4],[1,3]] sorted by end: [[1,2],[2,3],[1,3],[3,4]]:
 # [1,2]: $1 \geq -\infty \rightarrow$ keep, last_end=2.
 # [2,3]: $2 \geq 2 \rightarrow$ keep, last_end=3.
 # [1,3]: $1 < 3 \rightarrow$ remove. removed=1.
 # [3,4]: $3 \geq 3 \rightarrow$ keep. return 1 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$: dominated by sort. Space $O(1)$: one variable.
 # Sorting by END (not start) is the key greedy insight – keeps the most room for future intervals.
 # Follow-up: Meeting Rooms I (LC 252) – just check if any two overlap (sort by start, compare consecutive).
 # Follow-up: Minimum Number of Arrows (LC 452) – burst balloons; same greedy as this problem."

◆ Quick Review

Key Insights

- **Sorting is the key:** By sorting intervals by their end times, we can greedily choose the optimal set of non-overlapping intervals.

- **Greedy approach:** Always select the interval that ends the earliest, as it leaves maximum room for subsequent intervals.
- **Counting removals:** The answer is the total number of intervals minus the count of intervals selected by this greedy approach.
- **Edge observation:** Intervals that touch at the boundary are not overlapping, so the check uses “ $\geq$ ” instead of “ $>$ ”.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ if considering the storage for the sorted list (or $O(1)$ if sorting in-place).

Solution

The solution involves first sorting the intervals by their end times. Initialize a variable to track the end of the last chosen interval. Iterate over the sorted intervals: if the current interval's start is greater than or equal to the last chosen interval's end, update the last end and include this interval in the non-overlapping set. Otherwise, count the

interval as one that must be removed. The final answer is the removal count, which is computed as the total number of intervals minus the count of non-overlapping intervals selected.

Dynamic Programming

□ #516 Longest Palindromic Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming

Lists: Denny Zhang

Problem

Given a string s , find the longest palindromic subsequence's length in s .

A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

Input: $s = \text{"bbbab"}$

Output: 4

Explanation: One possible longest palindromic subsequence is "bbbb".

Example 2:

Input: $s = \text{"cbdd"}$

Output: 2

Explanation: One possible longest palindromic subsequence is "bb".

Constraints:

- $1 \leq s.length \leq 1000$
- s consists only of lowercase English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the length of the longest palindromic subsequence in string s.
# Subsequence: characters need not be contiguous. Right?"
#
# "s='bbbab': 'bbbb' → 4 ✓ s='cbabd': 'bb' → 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Recursion: lps(i,j) = longest palindromic subsequence in s[i..j].
# If s[i]==s[j]: 2+lps(i+1,j-1). Else max(lps(i+1,j), lps(i,j-1)).
# Time: O(2^n) without memo. Space: O(n) stack."
```

```
class _516_LongestPalindromicSubsequence_Brute:
    def longestPalindromeSubseq(self, s: str) -> int:
        from functools import lru_cache
        @lru_cache(maxsize=None)
        def dp(i, j):
            if i > j: return 0
            if i == j: return 1
            if s[i] == s[j]: return 2 + dp(i+1, j-1)
            return max(dp(i+1, j), dp(i, j-1))
        return dp(0, len(s)-1)
```

PHASE 3 — OPTIMIZE

```
# "Bottom-up DP: dp[i][j] = LPS in s[i..j]. Fill by increasing length.
# Observation: LPS(s) = LCS(s, reverse(s)).
# Space optimised: O(n) with two rolling arrays."
```

PHASE 4 — CLEAN CODE

```
class _516_LongestPalindromicSubsequence:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        dp = [1] * n # dp[j] = LPS of s[i..j] for current row i
        for i in range(n - 2, -1, -1):
            prev = [0] * n
            prev[i] = 1
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    prev[j] = 2 + (dp[j-1] if j-1 >= i+1 else 0)
                else:
                    prev[j] = max(dp[j], prev[j-1])
            dp = prev
```

```
return dp[n-1]
```

PHASE 5 — TEST & DEBUG

```
# s='bbbab':  
# Full 2D: dp[0][4]=4 (bbbb). Standard  $O(n^2)$  time  $O(n^2)$  space is cleaner to trace.  
# s='bbbab': LCS with reverse 'babbb': LCS='bbbb'=4 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^2)$ : fill  $n \times n$  DP table. Space  $O(n)$ : two rolling arrays.  
# Equivalently: LPS(s) = LCS(s, s[::-1]) – same recurrence structure.  
# Follow-up: Longest Palindromic Substring (LC 5) – contiguous version.  
# Follow-up: Palindrome Partitioning II (LC 132) – min cuts to make all substrings  
palindromes."
```

◆ Quick Review

Key Insights

- The problem can be solved efficiently using dynamic programming.
- A 2D DP table is used where $dp[i][j]$ represents the length of the longest palindromic subsequence in the substring $s[i...j]$.
- If the characters at the two ends of the substring match ($s[i] == s[j]$), they contribute to the palindrome length.
- Otherwise, the solution is the maximum sequence length by either excluding the left or the right character.
- The approach builds the solution from smaller substrings (length 1) to the entire string.

Space & Time

Time Complexity: $O(n^2)$ where n is the length of the string.

Space Complexity: $O(n^2)$ due to the usage of a 2D DP table.

Solution

We solve this problem using a bottom-up dynamic programming approach. We create a 2D table $dp[][]$ such that $dp[i][j]$ stores the longest palindromic subsequence length for the substring $s[i...j]$. For any single character, $dp[i][i]$ is 1. For substrings of length greater than 1, if the characters at positions i and j ($s[i]$ and $s[j]$) match, then $dp[i][j] = dp[i+1][j-1] + 2$. If they do not match, we take the maximum of $dp[i+1][j]$ and $dp[i][j-1]$. Finally, the $dp[0][n-1]$ holds the result for the entire string.

Dynamic Programming

□ #576 Out of Boundary Paths

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Dynamic Programming

Lists: Denny Zhang

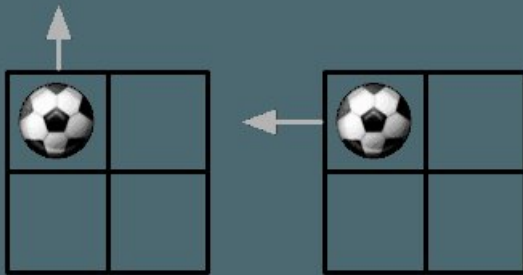
Problem

There is an $m \times n$ grid with a ball. The ball is initially at the position $[startRow, startColumn]$. You are allowed to move the ball to one of the four adjacent cells in the grid (possibly out of the grid crossing the grid boundary). You can apply at most $maxMove$ moves to the ball.

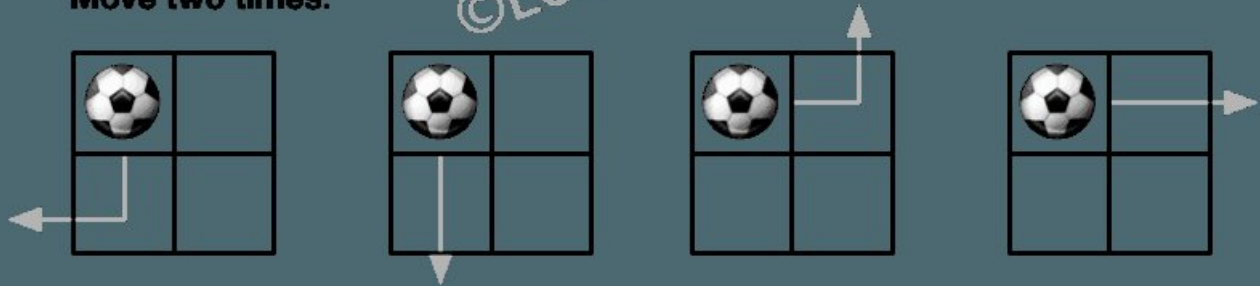
Given the five integers m , n , $maxMove$, $startRow$, $startColumn$, return the number of paths to move the ball out of the grid boundary. Since the answer can be very large, return it modulo $10^9 + 7$.

Example 1:

Move one time:

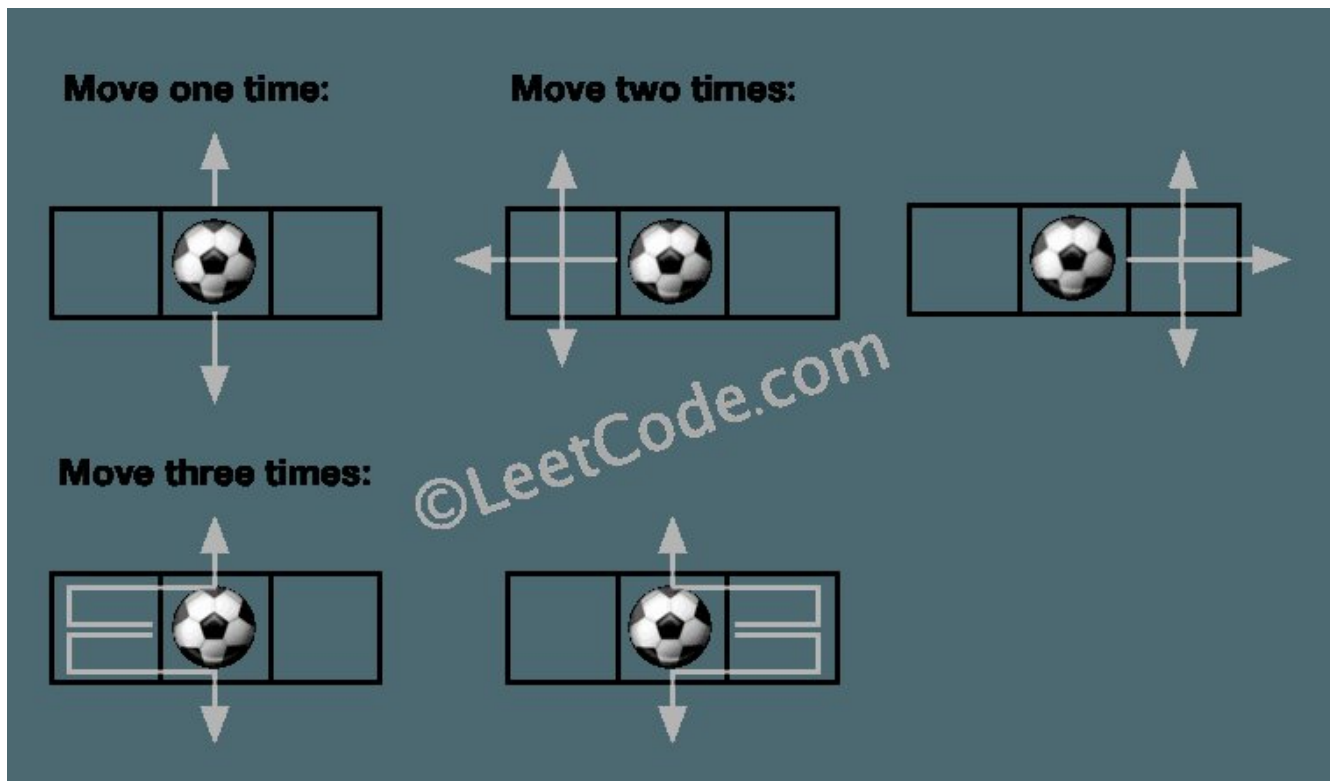


Move two times:



Input: $m = 2$, $n = 2$, $\text{maxMove} = 2$, $\text{startRow} = 0$, $\text{startColumn} = 0$
Output: 6

Example 2:



Input: $m = 1$, $n = 3$, $\text{maxMove} = 3$, $\text{startRow} = 0$, $\text{startColumn} = 1$
 Output: 12

Constraints:

- $1 \leq m, n \leq 50$
- $0 \leq \text{maxMove} \leq 50$
- $0 \leq \text{startRow} < m$
- $0 \leq \text{startColumn} < n$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"An $m \times n$ grid. Ball starts at $(\text{startRow}, \text{startCol})$. In exactly maxMove moves,
 # count paths that take the ball OUT of the boundary. Return modulo 10^9+7 . Right?"

```

#
# "m=2,n=2,maxMove=2,startRow=0,startCol=0: 6 paths exit in at most 2 moves ✓"

# PHASE 2 — BRUTE FORCE
# "DFS/recursion with memo: count(r,c,moves) = sum of count for each 4-direction.
# If r,c out of bounds → 1 (found a path). If moves=0 and in bounds → 0.
# Time: O(m*n*maxMove). Space: O(m*n*maxMove) memo."
class _576_OutOfBoundaryPaths_Brute:
    def findPaths(self, m, n, maxMove, startRow, startCol) -> int:
        from functools import lru_cache
        MOD = 10**9 + 7
        @lru_cache(maxsize=None)
        def dp(r, c, moves):
            if r < 0 or r >= m or c < 0 or c >= n: return 1
            if moves == 0: return 0
            return sum(dp(r+dr, c+dc, moves-1)
                       for dr,dc in [(0,1),(0,-1),(1,0),(-1,0)]) % MOD
        return dp(startRow, startCol, maxMove)

# PHASE 3 — OPTIMIZE
# "Bottom-up DP: dp[k][r][c] = number of paths of exactly k moves starting at (r,c)
# that exit.
# Build from k=1 to maxMove, accumulate total.
# Space optimised: O(m*n) with two 2D arrays (current and previous step)."

# PHASE 4 — CLEAN CODE
class _576_OutOfBoundaryPaths:
    def findPaths(self, m: int, n: int, maxMove: int, startRow: int, startCol: int)
-> int:
        MOD = 10**9 + 7
        dp = [[0]*n for _ in range(m)]
        dp[startRow][startCol] = 1
        count = 0
        for _ in range(maxMove):
            ndp = [[0]*n for _ in range(m)]
            for r in range(m):
                for c in range(n):
                    if dp[r][c] == 0: continue
                    for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                        nr, nc = r+dr, c+dc
                        if 0 <= nr < m and 0 <= nc < n:
                            ndp[nr][nc] = (ndp[nr][nc] + dp[r][c]) % MOD
                    else:
                        count = (count + dp[r][c]) % MOD
            dp = ndp
        return count

# PHASE 5 — TEST & DEBUG
# m=2,n=2,maxMove=2,start=(0,0):
# dp=[[1,0],[0,0]]. move1: (0,0)→up(-1,0) 00B count+=1, left(0,-1) 00B count+=1,

```



```
# right(0,1)→ndp[0][1]+=1, down(1,0)→ndp[1][0]+=1. ndp=[[0,1],[1,0]], count=2.
# move2: ndp[0][1]: up 00B+1=3, right 00B+1=4, left→[0,0]+=1, down→[1,1]+=1.
# ndp[1][0]: down 00B+1=5, left 00B+1=6, up→[0,0]+=1, right→[1,1]+=1. count=6 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\text{maxMove} * m * n)$. Space $O(m*n)$: two rolling grids.

Each step propagates the ball's position to neighboring cells or exits.

Follow-up: Knight Probability in Chessboard (LC 688) – same DP structure with knight moves.

Follow-up: if the ball can stay in bounds, track the stay probability separately."

◆ Quick Review

Key Insights

- Use Dynamic Programming (DP) as the state is defined by (current row, current column, moves remaining).
- Transitions are made from a cell to its four adjacent cells.
- If the ball moves out of bounds, it contributes to one valid path.
- States can be memoized to avoid recomputation and optimize the recursive solution.

Space & Time

Time Complexity: $O(\text{maxMove} * m * n)$, since we compute each state at most once.

Space Complexity: $O(\text{maxMove} * m * n)$ for the memoization table.

Solution

We use a recursive dynamic programming (or memoization) approach. Define a function $dp(i, j, movesLeft)$ that returns the number of ways to move out of bounds starting from cell (i, j) with $movesLeft$ moves remaining. For each move from the current cell, check: If the cell is out of bounds, return 1 (base case). If no moves are left ($movesLeft == 0$) and we are still inside, return 0. Otherwise, use recursion to explore all four directions and sum the paths. Store results in a memoization table to avoid recomputation, then return the computed value modulo $10^9 + 7$.

Dynamic Programming

□ #1143 Longest Common Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A common subsequence of two strings is a subsequence that is common to both strings.

Example 1:

Input: `text1 = "abcde"`, `text2 = "ace"`

Output: 3

Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:

Input: text1 = "abc", text2 = "abc"

Output: 3

Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: text1 = "abc", text2 = "def"

Output: 0

Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- $1 \leq \text{text1.length}, \text{text2.length} \leq 1000$
- text1 and text2 consist of only lowercase English characters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the length of the longest common subsequence of two strings.

Subsequence: characters need not be contiguous. Right?"

#

"text1='abcde', text2='ace': 'ace' → length 3 ✓

text1='abc', text2='abc': 'abc' → 3 ✓ text1='abc', text2='def': '' → 0 ✓"

PHASE 2 — BRUTE FORCE

"Recursion: lcs(i,j) on text1[:i], text2[:j].

If text1[i]==text2[j]: 1+lcs(i-1,j-1). Else max(lcs(i-1,j), lcs(i,j-1)).

Time: $O(2^{(m+n)})$ without memo. Space: $O(m+n)$ stack."

class _1143_LongestCommonSubsequence_Brute:

def longestCommonSubsequence(self, text1: str, text2: str) -> int:

from functools import lru_cache

@lru_cache(maxsize=None)

def dp(i, j):

if i < 0 or j < 0: return 0

if text1[i] == text2[j]: return 1 + dp(i-1, j-1)

```
        return max(dp(i-1, j), dp(i, j-1))
    return dp(len(text1)-1, len(text2)-1)
```

PHASE 3 — OPTIMIZE

"Bottom-up 2D DP: $dp[i][j]$ = LCS of $text1[:i]$, $text2[:j]$."

Space optimised: two rolling arrays $O(\min(m,n))$."

PHASE 4 — CLEAN CODE

```
class _1143_LongestCommonSubsequence:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        if m < n:
            text1, text2, m, n = text2, text1, n, m
        prev = [0] * (n + 1)
        for i in range(1, m + 1):
            curr = [0] * (n + 1)
            for j in range(1, n + 1):
                if text1[i-1] == text2[j-1]:
                    curr[j] = 1 + prev[j-1]
                else:
                    curr[j] = max(prev[j], curr[j-1])
            prev = curr
        return prev[n]
```

PHASE 5 — TEST & DEBUG

$text1='abcde'$, $text2='ace'$:

After full DP: $prev[3] = 3$ ✓ (standard LCS table)

$i=1('a')$: $curr[1]=1('a'=='a')$. $i=2('b')$: $curr[1]=1, curr[2]=1('b' \neq 'c')$.

$i=3('c')$: $curr[2]=2('c'=='c')$. $i=4('d')$: $curr[2]=2$. $i=5('e')$: $curr[3]=3$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \times n)$: fill DP table. Space $O(\min(m,n))$: rolling arrays."

LCS is foundational: used in diff tools, bioinformatics, plagiarism detection.

Follow-up: print the actual LCS – backtrack through the DP table.

Follow-up: Edit Distance (LC 72) extends LCS with insert/delete/replace costs."

◆ Quick Review

Key Insights

- The problem is well-suited for a dynamic programming approach.
- Use a 2D table where $dp[i][j]$ stores the length of the longest common subsequence

for $\text{text1}[0\dots i-1]$ and $\text{text2}[0\dots j-1]$.

- If the characters match at the current position, extend the subsequence by 1 from $\text{dp}[i-1][j-1]$; otherwise, take the maximum from $\text{dp}[i-1][j]$ or $\text{dp}[i][j-1]$.
- Filling the table in a bottom-up manner ensures that all subproblems are solved optimally.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the lengths of text1 and text2 .

Space Complexity: $O(m * n)$ for maintaining the 2D DP table. Space can be optimized to $O(\min(m, n))$ if needed.

Solution

This solution uses a dynamic programming approach with a 2D array (dp) to record the lengths of common subsequences at each substring pair. We initialize a table of dimensions $(m+1) \times (n+1)$ with zeros, where m and n are the lengths of text1 and text2 . We iterate over both strings, comparing characters; if they match, we set $\text{dp}[i][j] = \text{dp}[i-1][j-1] + 1$, otherwise we choose the

maximum from $dp[i-1][j]$ and $dp[i][j-1]$. The final answer is found at $dp[m][n]$.

Bit Manipulation

Round 8 · Low · Medium · 4 questions

Bit Manipulation

□ #78 Subsets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking · Bit Manipulation
Lists: Grind 169

Problem

Given an integer array `nums` of unique elements, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: `nums = [1,2,3]`

Output: `[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]`

Example 2:

Input: `nums = [0]`

Output: `[[], [0]]`

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$
- All the numbers of `nums` are unique.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given an array of unique integers, return all possible subsets (power set). Right?"

#

"nums=[1,2,3]: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]] ✓ (any order)"

PHASE 2 — BRUTE FORCE

"Backtracking: at each index, choose to include or exclude. Build all 2^n subsets.

Time: $O(n * 2^n)$. Space: $O(n)$ recursion depth."

```
class _78_Subsets_Brute:
```

```
    def subsets(self, nums: list[int]) -> list[list[int]]:
```

```
        result = []
```

```
        def backtrack(start, path):
```

```
            result.append(path[:])
```

```
            for i in range(start, len(nums)):
```

```
                path.append(nums[i])
```

```
                backtrack(i + 1, path)
```

```
                path.pop()
```

```
        backtrack(0, [])
```

```
        return result
```

PHASE 3 — OPTIMIZE

"Bit manipulation: iterate $0..2^n-1$. Each integer is a bitmask where bit $i=1$ means include `nums[i]`.

Time: $O(n * 2^n)$. Space: $O(1)$ extra (output excluded).

Same complexity but elegant integer iteration."

PHASE 4 — CLEAN CODE

```
class _78_Subsets:
```

```
    def subsets(self, nums: list[int]) -> list[list[int]]:
```

```
        n = len(nums)
```

```
        result = []
```

```
        for mask in range(1 << n):
```

```
            subset = [nums[i] for i in range(n) if mask & (1 << i)]
```

```
            result.append(subset)
```

```
        return result
```

PHASE 5 — TEST & DEBUG

nums=[1,2,3], n=3: masks $0..7$.

mask=0(000): []. mask=1(001): [1]. mask=2(010): [2]. mask=3(011): [1,2].

mask=4(100): [3]. mask=5(101): [1,3]. mask=6(110): [2,3]. mask=7(111): [1,2,3]. ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \cdot 2^n)$ :  $2^n$  subsets, each  $O(n)$  to build. Space  $O(1)$  extra.  
# Both backtracking and bitmask approaches have identical complexity.  
# Backtracking is more generalizable; bitmask is concise for small  $n$ .  
# Follow-up: Subsets II (LC 90) – input has duplicates; sort + skip duplicates in  
backtracking.  
# Follow-up: Combination Sum (LC 39) – subsets constrained to a target sum."
```

◆ Quick Review

Key Insights

- The problem is equivalent to generating the power set of the given set.
- A recursive backtracking approach can be used to explore both including and excluding each element.
- Iterative (bit manipulation) solutions are also possible since there are 2^n subsets in total.
- The maximum array length is small (up to 10), making exponential time solutions feasible.

Space & Time

Time Complexity: $O(2^n \cdot n)$ – There are 2^n subsets and creating each subset may take up to $O(n)$ time.

Space Complexity: $O(2^n \cdot n)$ – Space for storing all subsets. The recursion stack has a maximum depth of $O(n)$.

Solution

We solve the problem using backtracking, where we decide for each element whether to include it in the current subset or not. We recursively build all subsets starting at an empty list and at each step appending the current subset to our result list. We can also use iterative bit manipulation but here we focus on the backtracking approach. This algorithm leverages recursion and a list to maintain the current state, leading to a clear and concise implementation.

Bit Manipulation

□ #90 Subsets II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking · Bit Manipulation
Lists: Denny Zhang

Problem

Given an integer array `nums` that may contain duplicates, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

```
Input: nums = [1,2,2]
Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given an array that MAY contain duplicates, return all unique subsets. Right?"
#
# "nums=[1,2,2]: [[],[1],[1,2],[1,2,2],[2],[2,2]] ✓ (no duplicates in output)"
```

PHASE 2 — BRUTE FORCE

```
# "Generate all subsets as sorted tuples, add to a set to deduplicate.
# Time:  $O(n * 2^n)$ . Space:  $O(2^n)$  for the set."
```

```
class _90_SubsetsII_Brute:
    def subsetsWithDup(self, nums: list[int]) -> list[list[int]]:
        result = set()
        nums.sort()
        def backtrack(start, path):
            result.add(tuple(path))
            for i in range(start, len(nums)):
                path.append(nums[i])
                backtrack(i+1, path)
                path.pop()
        backtrack(0, [])
        return [list(t) for t in result]
```

PHASE 3 — OPTIMIZE

```
# "Sort nums first. In backtracking, skip duplicate elements at the same recursion
level
# (not the first occurrence, but subsequent same values at the same start position).
# If nums[i] == nums[i-1] and i > start, skip.
# Time:  $O(n * 2^n)$ . Space:  $O(n)$  recursion – no dedup set needed."
```

PHASE 4 — CLEAN CODE

```
class _90_SubsetsII:
    def subsetsWithDup(self, nums: list[int]) -> list[list[int]]:
        nums.sort()
        result = []
        def backtrack(start: int, path: list) -> None:
            result.append(path[:])
            for i in range(start, len(nums)):
                if i > start and nums[i] == nums[i-1]:
                    continue # skip duplicate at this level
                path.append(nums[i])
                backtrack(i + 1, path)
                path.pop()
        backtrack(0, [])
        return result
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,2,2] sorted:
# backtrack(0,[]): append []. i=0→path=[1], backtrack(1):
# append [1]. i=1→[1,2],backtrack(2): append[1,2]. i=2→[1,2,2]→append. i=3 done.
# i=2: nums[2]==nums[1] and 2>1 → skip.
# i=1→path=[2],backtrack(2): append[2]. i=2→[2,2]→append.
# i=2: nums[2]==nums[1] and 2>0 → skip.
# result=[[ ],[1],[1,2],[1,2,2],[2],[2,2]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \times 2^n)$ : worst case all unique. Space  $O(n)$ : recursion depth.
# The sort + skip-duplicate pattern is the standard dedup technique for
backtracking.
# Follow-up: Permutations II (LC 47) – same skip-duplicate pattern for permutations.
# Follow-up: Combination Sum II (LC 40) – target sum with duplicates, same skip
pattern."
```

◆ Quick Review

Key Insights

- Sort the array first so that duplicate elements are adjacent.
- Use backtracking to generate all possible subsets.
- Skip duplicate elements in the recursion to avoid forming duplicate subsets.
- The use of sorted order and conditionally skipping elements is the trick to handle duplicates.

Space & Time

Time Complexity: $O(2^n * n)$ since each subset creation may take $O(n)$ in the worst case.

Space Complexity: $O(2^n * n)$ for storing the subsets, plus $O(n)$ for recursion stack.

Solution

The solution uses a backtracking technique that builds subsets incrementally. The input array is first sorted to bring duplicates together. During recursion, if the current element is the same as the previous one and it has not been used in the current recursion path, it is skipped to prevent duplicate subsets. This ensures that each subset is unique. The algorithm stores each valid subset by copying the current subset state into the result list.

Bit Manipulation

□ #287 Find the Duplicate Number

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Bit
Manipulation

Lists: Grind 169

Problem

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only one repeated number in `nums`, return this repeated number.

You must solve the problem without modifying the array `nums` and using only constant extra space.

Example 1:

Input: `nums = [1,3,4,2,2]`
Output: 2

Example 2:

Input: `nums = [3,1,3,4,2]`
Output: 3

Example 3:

Input: nums = [3,3,3,3,3]

Output: 3

Constraints:

- $1 \leq n \leq 10^5$
- `nums.length == n + 1`
- $1 \leq \text{nums}[i] \leq n$
- All the integers in nums appear only once except for precisely one integer which appears two or more times.

Follow up:

- How can we prove that at least one duplicate number must exist in nums?
- Can you solve the problem in linear runtime complexity?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"n+1 integers in range [1,n]; exactly one number is duplicated (may appear 2+ times).

Find the duplicate. Must not modify array, must use O(1) extra space. Right?"

#

"nums=[1,3,4,2,2]: duplicate=2 ✓

nums=[3,1,3,4,2]: duplicate=3 ✓"

PHASE 2 — BRUTE FORCE

"Use a hash set: for each number, if already seen, it's the duplicate.

Time: O(n). Space: O(n) for the set. – Violates the O(1) space constraint."

```
class _287_FindDuplicate_Brute:
    def findDuplicate(self, nums: list[int]) -> int:
        seen = set()
        for num in nums:
            if num in seen:
                return num
            seen.add(num)
        return -1
```

PHASE 3 — OPTIMIZE

"Floyd's cycle detection – treat nums as a linked list where index *i* points to *nums[i]*.
 # Because one value is duplicated, two indices point to the same next node → cycle.
 # Phase 1: find meeting point of slow (1 step) and fast (2 steps).
 # Phase 2: reset slow to index 0, advance both 1 step – they meet at cycle entry = duplicate.
 # Time: $O(n)$. Space: $O(1)$. Does not modify the array."

PHASE 4 — CLEAN CODE

```
class _287_FindDuplicate:
    def findDuplicate(self, nums: list[int]) -> int:
        # Phase 1: detect cycle
        slow = fast = nums[0]
        while True:
            slow = nums[slow]
            fast = nums[nums[fast]]
            if slow == fast:
                break
        # Phase 2: find cycle entry
        slow = nums[0]
        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]
        return slow
```

PHASE 5 — TEST & DEBUG

`nums=[1,3,4,2,2]` (indices: 0→1, 1→3, 2→4, 3→2, 4→2):
 # Phase1: `slow=1, fast=1→slow=3, fast=4→slow=2, fast=2→meet at 2`.
 # Phase2: `slow=nums[0]=1, fast=2. slow=3, fast=4. slow=2, fast=2. meet at 2 ✓`

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$. Does not sort or modify the array.
 # Binary search alternative: count numbers $\leq$ mid; if count $>$ mid, duplicate is in `[1, mid]`.
 # $O(n \log n)$ time, $O(1)$ space – simpler to explain in an interview.
 # Follow-up: if multiple duplicates allowed, binary search no longer works directly.
 # Follow-up: Find All Duplicates (LC 442) – mark visited by negating at index."

◆ Quick Review

Key Insights

- Since there are $n+1$ elements with values only from 1 to n , by the pigeonhole principle at least one number must be repeated.
- The problem can be approached by mapping the array values to indices, forming a cycle structure.
- Floyd's Tortoise and Hare algorithm (cycle detection) can be applied to find the duplicate number in linear time and constant space.
- There is no need for extra data structures like hash sets or alterations to the original array, satisfying the space and array-modification constraints.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution leverages Floyd's Tortoise and Hare cycle detection algorithm. Think of each array element as a pointer to another index, forming a linked list with a cycle (the duplicate

number creates the cycle). The algorithm works in two phases: Detect a meeting point within the cycle by having two pointers (tortoise and hare) move at different speeds. Find the cycle entrance by resetting one pointer to the start and moving both pointers at the same speed. The meeting point reveals the duplicate number. This approach provides an elegant solution that satisfies both linear runtime and constant space requirements.

Bit Manipulation

□ ★ #1506 Find Root of N-Ary Tree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Bit Manipulation · Tree · DFS
Lists: ★ Premium | Premium 98

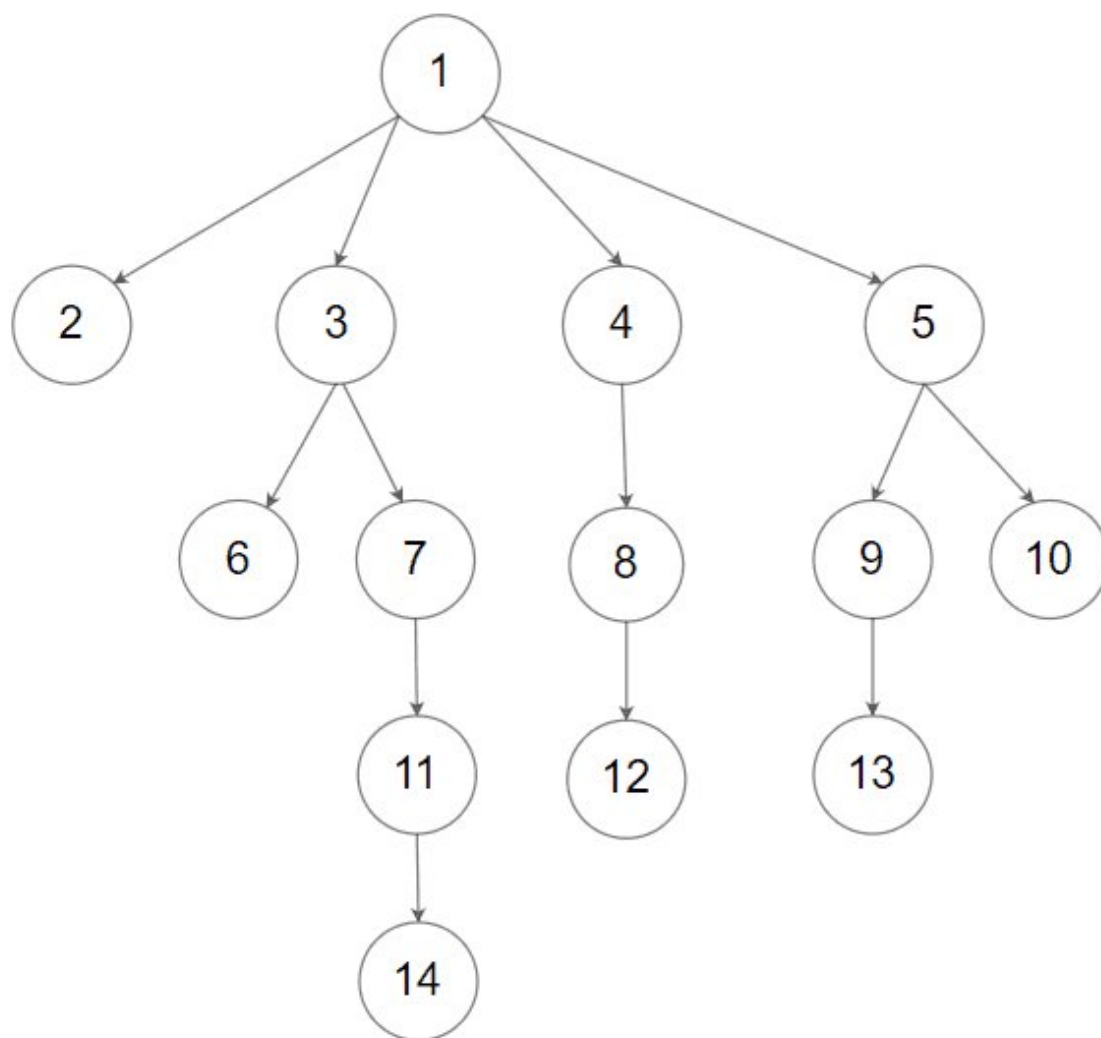
Problem

You are given all the nodes of an N-ary tree as an array of Node objects, where each node has a unique value.

Return the root of the N-ary tree.

Custom testing:

An N-ary tree can be serialized as represented in its level order traversal where each group of children is separated by the null value (see examples).



For example, the above tree is serialized as [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14].

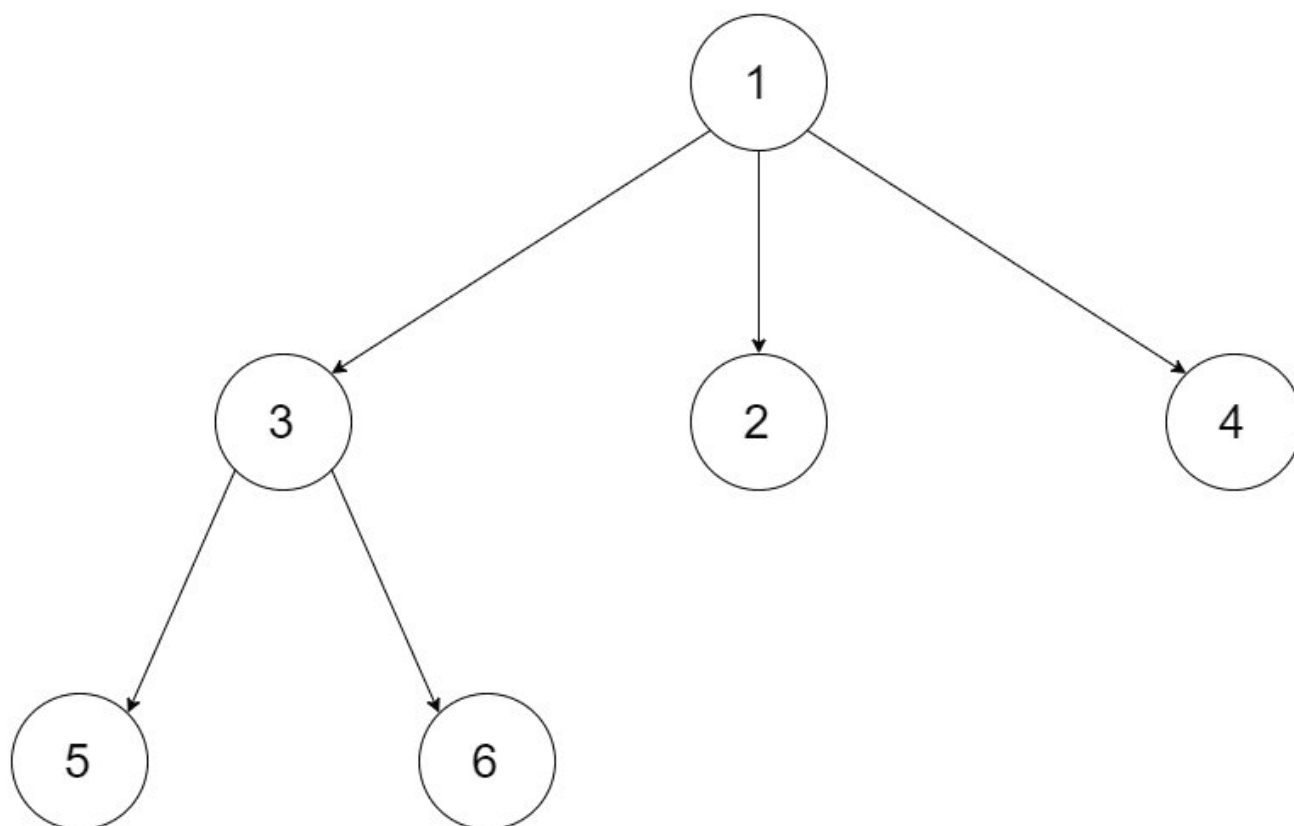
The testing will be done in the following way:

1. The input data should be provided as a serialization of the tree.
2. The driver code will construct the tree from the serialized input data and put each Node object into an array in an arbitrary order.

3. The driver code will pass the array to `findRoot`, and your function should find and return the root Node object in the array.

4. The driver code will take the returned Node object and serialize it. If the serialized value and the input data are the same, the test passes.

Example 1:



Input: tree = [1,null,3,2,4,null,5,6]

Output: [1,null,3,2,4,null,5,6]

Explanation: The tree from the input data is shown above.

The driver code creates the tree and gives findRoot the Node objects in an arbitrary order.

For example, the passed array could be

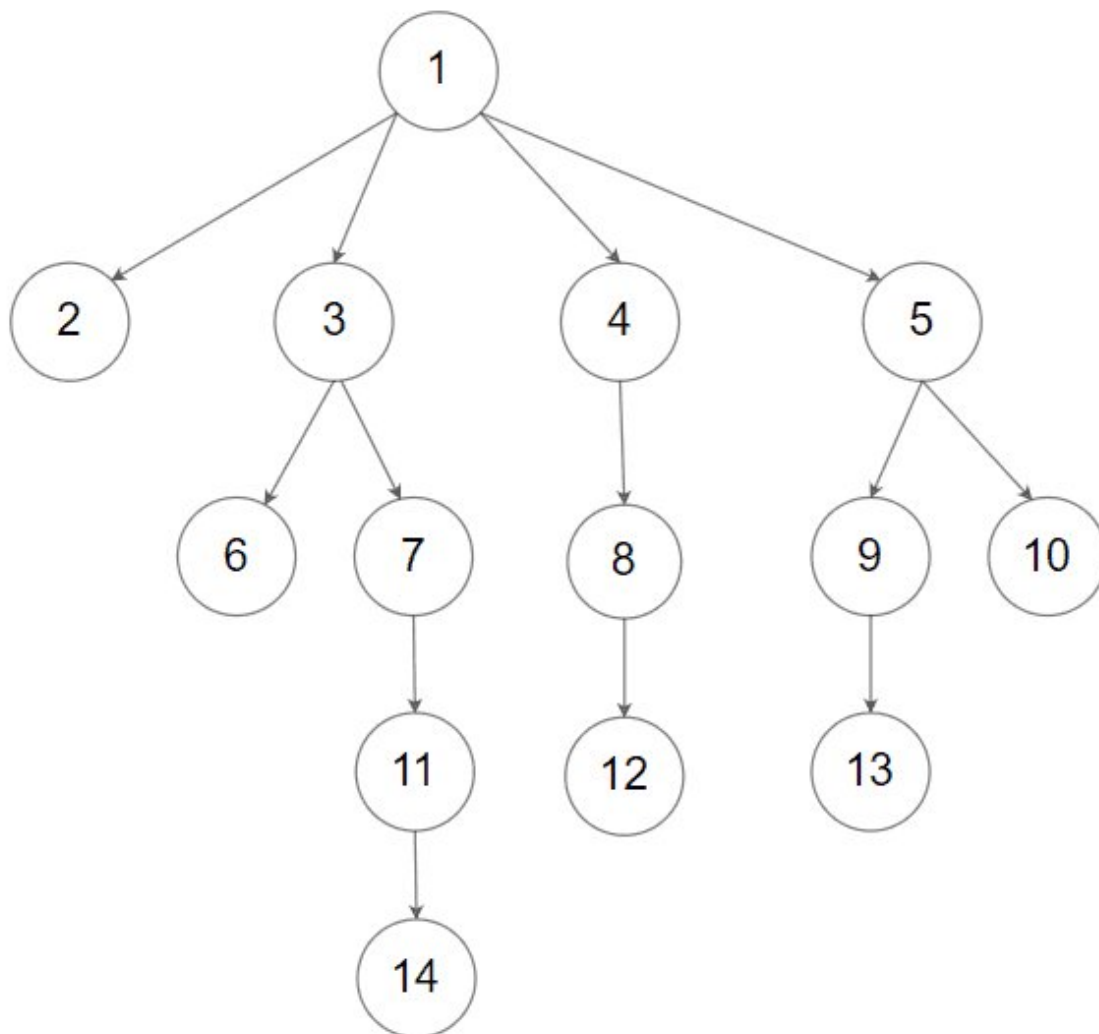
[Node(5),Node(4),Node(3),Node(6),Node(2),Node(1)] or

[Node(2),Node(6),Node(1),Node(3),Node(5),Node(4)].

The findRoot function should return the root Node(1), and the driver code will serialize it and compare with the input data.

The input data and serialized Node(1) are the same, so the test passes.

Example 2:



```
Input: tree = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null
,12,null,13,null,null,14]
Output: [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,nu
ll,13,null,null,14]
```

Constraints:

- The total number of nodes is between $[1, 5 * 10^4]$.
- Each node has a unique value.

Follow up:

- Could you solve this problem in constant space complexity with a linear time algorithm?

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given all nodes of an N-ary tree in an array (in random order), find the root.
# Each non-root node appears as a child exactly once.
# Cannot access node.parent. Right?"
#
# "nodes=[1,2,3,4,5,6], tree root=1 with children [3,2,4], node 3 has children
[5,6]:
# XOR all node values XOR all child values → root value. ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Build a set of all child node values. Root is the node whose value is NOT in that
set.
# Time: O(n) where n = total nodes including all children. Space: O(n)."
```

```
class _1506_FindRootNaryTree_Brute:
```

```
    def findRoot(self, tree) -> 'Node':
        child_vals = {child.val for node in tree for child in node.children}
        for node in tree:
```

```

        if node.val not in child_vals:
            return node
    return None

```

PHASE 3 — OPTIMIZE

"XOR trick: XOR all node values; XOR all children values.

Every non-root node's value appears once as a node and once as a child → cancels to 0.

Root appears only as a node → its value remains.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```

class _1506_FindRootNaryTree:
    def findRoot(self, tree) -> 'Node':
        xor_val = 0
        for node in tree:
            xor_val ^= node.val
            for child in node.children:
                xor_val ^= child.val
        for node in tree:
            if node.val == xor_val:
                return node
        return None

```

PHASE 5 — TEST & DEBUG

tree nodes [1,2,3,4,5,6] with root=1, children={3,2,4}, 3's children={5,6}:

node vals XOR: $1^2^3^4^5^6$. child vals XOR: $3^2^4^5^6$.

Combined: $1^2^3^4^5^6^3^2^4^5^6 = 1$ (all others cancel). root=node(1) ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: two linear passes. Space $O(1)$: one XOR variable.

Same XOR cancellation as Single Number (LC 136).

Follow-up: if node values aren't unique, XOR fails – use the child-set approach.

Follow-up: find all roots if the input is a forest (multiple disconnected trees) – same trick, collect all non-child values."

◆ Quick Review

Key Insights

- Every node except the root appears as a child of another node.
- By summing all node values and then subtracting the sum of all children node

values, the remaining value corresponds to the root.

- This subtraction trick allows us to identify the root in a single pass of the data structure without extra memory for tracking child nodes.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since we iterate through all nodes and their children.

Space Complexity: $O(1)$ extra space using arithmetic accumulators.

Solution

The solution calculates two sums: one for all nodes' values and one for all children nodes' values. The difference between these gives the value of the root node since every node's value (except the root's) is added once as a child. Finally, we iterate over the nodes to find the node with the computed root value and return it. This approach meets the requirement of constant space complexity and linear time.

Math

Round 8 · Low · Medium · 5 questions

Math

□ #7 Reverse Integer

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: Grind 169

Problem

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: $x = 123$
Output: 321

Example 2:

Input: $x = -123$
Output: -321

Example 3:

Input: $x = 120$
Output: 21

Constraints:

- $-231 \leq x \leq 231 - 1$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Reverse the digits of a 32-bit signed integer. Return 0 if the reversed integer overflows

$[-2^{31}, 2^{31}-1]$. Right?"

#

"x=123 → 321 ✓ x=-123 → -321 ✓ x=120 → 21 ✓ x=1534236469 → 0 (overflow) ✓"

PHASE 2 — BRUTE FORCE

"Convert to string, reverse, convert back. Check 32-bit bounds.

Time: $O(d)$. Space: $O(d)$."

```
class _7_ReverseInteger_Brute:
```

```
    def reverse(self, x: int) -> int:
```

```
        sign = -1 if x < 0 else 1
```

```
        rev = int(str(abs(x))[::-1]) * sign
```

```
        return rev if -(2**31) <= rev <= 2**31-1 else 0
```

PHASE 3 — OPTIMIZE

"Build reversed integer digit by digit without string conversion.

Check for overflow before each multiplication – avoids relying on 64-bit arithmetic.

Time: $O(d)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _7_ReverseInteger:
```

```
    def reverse(self, x: int) -> int:
```

```
        INT_MAX, INT_MIN = 2**31 - 1, -(2**31)
```

```
        result, sign = 0, 1 if x >= 0 else -1
```

```
        x = abs(x)
```

```
        while x:
```

```
            digit = x % 10
```

```
            x //= 10
```

```
            # Check overflow before appending digit
```

```
            if result > (INT_MAX - digit) // 10:
```

```
                return 0
```

```
            result = result * 10 + digit
```

```
        return result * sign
```

PHASE 5 — TEST & DEBUG

```
# x=123: digits extracted: 3,2,1. result: 3,32,321. 321 ≤ INT_MAX ✓. return 321 ✓.  
# x=-123: sign=-1,x=123. Same as above. return -321 ✓.  
# x=1534236469: eventually result*10+digit > INT_MAX → return 0 ✓.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(d)$: one pass through digits. Space $O(1)$: integer variables only.

Overflow check: if $result > (INT_MAX - digit) // 10$, adding would overflow.

Follow-up: Palindrome Number (LC 9) – only reverse half for palindrome check.

Follow-up: if working in a language without 64-bit integers, this overflow check is critical."

◆ Quick Review

Key Insights

- The problem requires reversing the digits while preserving the sign.
- Watch out for integer overflow due to the reversed number exceeding the 32-bit signed integer limits.
- The approach involves repeatedly extracting the last digit from the number and appending it to a new reversed integer.
- Overflow checks are done during each iteration before actually appending the digit.

Space & Time

Time Complexity: $O(n)$, where n is the number of digits in the integer.

Space Complexity: $O(1)$, only constant extra space is used.

Solution

The solution uses a while loop to extract the last digit from the original number using modulo operation ($x \% 10$) and then removes this digit using integer division ($x //= 10$). A reversed number is constructed by multiplying the current reversed number by 10 and adding the extracted digit. Before appending each digit, the algorithm checks if the new reversed number will overflow a 32-bit signed integer by comparing it with the thresholds derived from `INT_MAX` and `INT_MIN`. The algorithm handles negative values by working with x directly since the modulo and division operations work properly with negatives in most languages.

Math

□ #50 Pow(x, n)

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Math · Recursion

Lists: Grind 169

Problem

Implement $\text{pow}(x, n)$, which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$
Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$
Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$
Output: 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Constraints:

- $-100.0 < x < 100.0$
- $-2^{31} \leq n \leq 2^{31} - 1$
- n is an integer.

- Either x is not zero or $n > 0$.
- $-104 \leq x^n \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Implement pow(x, n) — x raised to the power n. n can be negative. Right?"
#
# "pow(2.0, 10) → 1024.0 ✓ pow(2.1, 3) → 9.261 ✓ pow(2.0, -2) → 0.25 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Multiply x by itself n times.
# Time: O(n). Space: O(1). TLEs for large n."
```

```
class _50_PowXN_Brute:
    def myPow(self, x: float, n: int) -> float:
        if n < 0: x, n = 1/x, -n
        result = 1.0
        for _ in range(n): result *= x
        return result
```

PHASE 3 — OPTIMIZE

```
# "Fast exponentiation (exponentiation by squaring).
# If n is even:  $x^n = (x^2)^{(n/2)}$ . If n is odd:  $x^n = x * x^{(n-1)}$ .
# Halves n each step → O(log n) multiplications.
# Time: O(log n). Space: O(1) iteratively."
```

PHASE 4 — CLEAN CODE

```
class _50_PowXN:
    def myPow(self, x: float, n: int) -> float:
        if n < 0:
            x, n = 1 / x, -n
        result = 1.0
        while n:
            if n % 2 == 1:
                result *= x
            x *= x
            n //= 2
        return result
```

PHASE 5 — TEST & DEBUG

```
# x=2.0, n=10:
# n=10(even): x=4,n=5. n=5(odd): result=4,x=16,n=2. n=2(even): x=256,n=1.
# n=1(odd): result=4*256=1024,x=65536,n=0. return 1024.0 ✓
#
# x=2.0, n=-2: x=0.5,n=2. n=2(even):x=0.25,n=1. n=1(odd):result=0.25,n=0. return
0.25 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log n)$ : halve the exponent each step. Space  $O(1)$ : iterative.
# Recursive version is  $O(\log n)$  time but  $O(\log n)$  stack space.
# Follow-up: integer exponentiation with modular arithmetic – same squaring, add mod
at each step.
# Follow-up: matrix exponentiation (for Fibonacci in  $O(\log n)$ ) – same pattern with
matrices."
```

◆ Quick Review

Key Insights

- Use exponentiation by squaring to reduce the number of multiplications.
- For a negative exponent, compute the power for $|n|$ and then take the reciprocal.
- Handle the edge case when n is 0 (return 1).
- The algorithm works in $O(\log n)$ time complexity.

Space & Time

Time Complexity: $O(\log |n|)$

Space Complexity: $O(1)$ for the iterative solution, or $O(\log |n|)$ if implemented recursively

Solution

We use exponentiation by squaring, an algorithm that reduces the number of multiplications by repeatedly squaring the base and halving the exponent. Here's how it works: If n is negative, switch x to $1/x$ and n to $-n$. Initialize the result as 1. Loop while n is greater than 0: If the current n is odd, multiply the result by x . Square the base x . Divide n by 2 (using integer division). This iterative approach efficiently computes the power in logarithmic steps. The same approach can be implemented recursively.

Math

□ #380 Insert Delete GetRandom O(1)

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Design
Lists: Grind 169

Problem

Implement the RandomizedSet class:

- **RandomizedSet()** Initializes the RandomizedSet object.
- **bool insert(int val)** Inserts an item val into the set if not present. Returns true if the item was not present, false otherwise.
- **bool remove(int val)** Removes an item val from the set if present. Returns true if the item was present, false otherwise.
- **int getRandom()** Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the same probability of being returned.

You must implement the functions of the class such that each function works in average $O(1)$ time complexity.

Example 1:

Input

```
["RandomizedSet", "insert", "remove", "insert", "getRandom", "remove", "insert", "getRandom"]
```

```
[[], [1], [2], [2], [], [1], [2], []]
```

Output

```
[null, true, false, true, 2, true, false, 2]
```

Explanation

```
RandomizedSet randomizedSet = new RandomizedSet();
```

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- At most $2 * 10^5$ calls will be made to insert, remove, and getRandom.
- There will be at least one element in the data structure when getRandom is called.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Design a set supporting insert(val)→bool, remove(val)→bool, getRandom()→val
# all in  $O(1)$  average time. getRandom returns each element with equal probability.
Right?"
```

```
#
```

```
# "insert(1)→T, insert(2)→T, getRandom()→1 or 2 equally, remove(1)→T,
# getRandom()→2 always. ✓"
```

PHASE 2 — BRUTE FORCE

"Use a Python set. insert/remove/search: $O(1)$. getRandom: convert to list $O(n)$, pick random.

Time: $O(n)$ per getRandom. Space: $O(n)$."

```
class _380_RandomizedSet_Brute:
    def __init__(self):
        self.data = set()
    def insert(self, val: int) -> bool:
        if val in self.data: return False
        self.data.add(val); return True
    def remove(self, val: int) -> bool:
        if val not in self.data: return False
        self.data.remove(val); return True
    def getRandom(self) -> int:
        import random
        return random.choice(list(self.data)) #  $O(n)$ 
```

PHASE 3 — OPTIMIZE

"Array + HashMap: array holds all values, map holds {val→array_index}.

getRandom: pick a random array index – $O(1)$.

remove: swap target with last element, update map, pop last – $O(1)$.

insert: append to array, update map – $O(1)$."

PHASE 4 — CLEAN CODE

```
class _380_RandomizedSet:
    def __init__(self):
        import random
        self._r = random
        self.vals = [] # array of values
        self.idx = {} # val → index in vals
    def insert(self, val: int) -> bool:
        if val in self.idx: return False
        self.idx[val] = len(self.vals)
        self.vals.append(val)
        return True
    def remove(self, val: int) -> bool:
        if val not in self.idx: return False
        last = self.vals[-1]
        i = self.idx[val]
        self.vals[i] = last
        self.idx[last] = i
        self.vals.pop()
        del self.idx[val]
        return True
    def getRandom(self) -> int:
        return self._r.choice(self.vals)
```

PHASE 5 — TEST & DEBUG

insert(1): vals=[1],idx={1:0}. insert(2): vals=[1,2],idx={1:0,2:1}.

remove(1): last=2,i=0. vals[0]=2,idx[2]=0. pop→vals=[2],idx={2:0}. del idx[1].


```
# getRandom()→choice([2])→2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "All ops O(1) average: array ops + dict ops.
```

```
# The swap-with-last trick avoids O(n) shifts when removing from the middle.
```

```
# Edge case: when removing the last element, swap is a no-op (last==val).
```

```
# Follow-up: Insert Delete GetRandom with duplicates (LC 381) – store sets of indices per value.
```

```
# Follow-up: weighted random – use a prefix-sum array + binary search for getRandom."
```

◆ Quick Review

Key Insights

- Use a dynamic array (or list) to store the elements to allow $O(1)$ access by index.
- Use a hash table (or dictionary/map) to store the mapping from value to its index in the array.
- For removal, swap the element to be removed with the last element in the array, then delete the last element. This avoids expensive shifting of elements.
- `getRandom` can be implemented by generating a random index into the array.

Space & Time

Time Complexity: $O(1)$ on average for insert, remove, and `getRandom` operations.

Space Complexity: $O(n)$ where n is the number of elements stored in the data structure.

Solution

The solution involves maintaining a list to store the values and a hash map to quickly check for the presence of a value and to know its index in the list. When an element is removed, swap it with the last element of the list and update the map accordingly; then, remove the last element in $O(1)$ time. This approach ensures that insert, remove, and getRandom can all be achieved in constant time on average.

Math

□ #1017 Convert to Base -2

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: Denny Zhang

Problem

Given an integer n , return a binary string representing its representation in base -2 .

Note that the returned string should not have leading zeros unless the string is "0".

Example 1:

Input: $n = 2$
Output: "110"
Explantion: $(-2)2 + (-2)1 = 2$

Example 2:

Input: $n = 3$
Output: "111"
Explantion: $(-2)2 + (-2)1 + (-2)0 = 3$

Example 3:

Input: $n = 4$
Output: "100"
Explantion: $(-2)2 = 4$

Constraints:

- $0 \leq n \leq 109$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Given integer n, return its representation in base -2 as a binary string.
# Negative base:  $(-2)^0=1$ ,  $(-2)^1=-2$ ,  $(-2)^2=4$ ,  $(-2)^3=-8$ , ...
# Result has no leading zeros (except '0' itself). Right?"
#
# "n=2:  $2 = (-2)^1*(-1) + (-2)^0*(0) + (-2)^2*...$  hmm:  $4+(-2)=2 \rightarrow '110'$  ✓
# n=3: '111' ✓ n=4: '100' ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Repeated division by -2. Remainder must be 0 or 1 (non-negative).
# If remainder is negative (when n is odd and negative), adjust: rem += 2, n += 1
(or n = n//(-2)).
# Time:  $O(\log n)$ . Space:  $O(\log n)$ ."
```

```
class _1017_ConvertToBaseNeg2_Brute:
    def baseNeg2(self, n: int) -> str:
        if n == 0: return '0'
        result = []
        while n != 0:
            rem = n % (-2)
            n //= (-2) # Python floor division handles signs
            if rem < 0:
                rem += 2
                n += 1
            result.append(str(rem))
        return ''.join(reversed(result))
```

PHASE 3 — OPTIMIZE

```
# "Same iterative approach — already  $O(\log n)$ . The key is ensuring remainder is 0 or 1.
# Python's % with negative base may return negative remainders; adjust to  $[0,1]$ ."
```

PHASE 4 — CLEAN CODE

```
class _1017_ConvertToBaseNeg2:
    def baseNeg2(self, n: int) -> str:
        if n == 0: return '0'
        bits = []
        while n:
            bits.append(n & 1) # LSB is always 0 or 1
            n = -(n >> 1) # divide by -2: right-shift then negate
        return ''.join(str(b) for b in reversed(bits))
```

PHASE 5 — TEST & DEBUG

```
# n=2: bits: 2&1=0, n=-(2>>1)=-1. (-1)&1=1, n=-(-1>>1)=1. 1&1=1, n=-(1>>1)=0.
# bits=[0,1,1]. reversed='110' ✓
# n=3: 3&1=1,n=-1. 1,n=1. 1,n=0. bits=[1,1,1]→'111' ✓
# n=0: return '0' ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log n): number of bits in the representation.
# Space O(log n): bits array.
# The n = -(n>>1) trick works because: dividing by -2 = right-shift (divide by 2)
then negate.
# Follow-up: Base -10 (general negative base) – same structure, replace 2 with
|base|.
# Follow-up: decode a base-(-2) string back to integer – multiply by (-2)^position."
```

◆ Quick Review

Key Insights

- Use division and remainder operations to simulate converting a number to a different base.
- When working with a negative base (-2), the remainder can become negative; adjust the remainder (by adding the base's absolute value) and update the quotient accordingly.
- For $n = 0$, simply return "0" since there is nothing to convert.
- Build the answer by collecting digits (remainders) and then reversing the order since the conversion provides digits from least significant to most significant.

Space & Time

Time Complexity: $O(\log|n|)$ – the loop runs for about the number of digits in the base -2 representation.

Space Complexity: $O(\log|n|)$ – extra space for storing the computed digits.

Solution

To solve the problem, we simulate the conversion process similar to how you would convert an integer to a positive base. However, since the base here is -2 , special handling of negative remainders is needed. In each iteration, we: Divide the current number by -2 . Calculate the remainder. If the remainder is negative, adjust it by adding 2, and increment the quotient to compensate. Append the computed remainder to a list (which represents the binary digit). Continue until the number reduces to zero. Finally, reverse the list to form the correct string representation from the most significant to the least significant digit.

Math

□ #3160 Find the Number of Distinct Colors Among the Balls

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Simulation
Lists: CodeSignal

Problem

You are given an integer `limit` and a 2D array `queries` of size $n \times 2$.

There are `limit + 1` balls with distinct labels in the range $[0, \text{limit}]$. Initially, all balls are uncolored. For every query in `queries` that is of the form $[x, y]$, you mark ball x with the color y . After each query, you need to find the number of colors among the balls.

Return an array `result` of length n , where `result[i]` denotes the number of colors after i th query.

Note that when answering a query, lack of a color will not be considered as a color.

Example 1:

Input: `limit = 4, queries = [[1,4],[2,5],[1,3],[3,4]]`

Output: [1,2,2,3]

Explanation:



- After query 0, ball 1 has color 4.
- After query 1, ball 1 has color 4, and ball 2 has color 5.
- After query 2, ball 1 has color 3, and ball 2 has color 5.
- After query 3, ball 1 has color 3, ball 2 has color 5, and ball 3 has color 4.

Example 2:

Input: limit = 4, queries =
[[0,1],[1,2],[2,2],[3,4],[4,5]]

Output: [1,2,2,3,4]

Explanation:



- After query 0, ball 0 has color 1.
- After query 1, ball 0 has color 1, and ball 1 has color 2.
- After query 2, ball 0 has color 1, and balls 1 and 2 have color 2.
- After query 3, ball 0 has color 1, balls 1 and 2 have color 2, and ball 3 has color 4.
- After query 4, ball 0 has color 1, balls 1 and 2 have color 2, ball 3 has color 4, and ball 4 has color 5.

Constraints:

- $1 \leq \text{limit} \leq 10^9$
- $1 \leq n == \text{queries.length} \leq 10^5$
- $\text{queries}[i].\text{length} == 2$
- $0 \leq \text{queries}[i][0] \leq \text{limit}$
- $1 \leq \text{queries}[i][1] \leq 10^9$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"N balls (0 to limit). queries[i]=[ball,color]: color that ball. After each query return distinct color count."

#

"limit=4, queries=[[1,4],[2,5],[1,3],[3,4]]:

After q1:{4}→1. After q2:{4,5}→2. After q3:{3,5}→2. After q4:{3,5,4}→3 ✓"

PHASE 2 — BRUTE FORCE

"Dict ball→color. After each query, scan all values for distinct count."

Time: O(n) per query. Space: O(n)."

```
class _3160_DistinctColors_Brute:
```

```
    def queryResults(self, limit: int, queries: list) -> list:
        ball_color = {}
        result = []
        for ball, color in queries:
            ball_color[ball] = color
            result.append(len(set(ball_color.values())))
        return result
```

PHASE 3 — OPTIMIZE

"color_count tracks how many balls have each color. Update O(1) on each query."

len(color_count) = distinct colors."

PHASE 4 — CLEAN CODE

```
class _3160_DistinctColors:
```

```
    def queryResults(self, limit: int, queries: list) -> list:
        ball_color = {}
        color_count = {}
        result = []
        for ball, color in queries:
            if ball in ball_color:
                old = ball_color[ball]
                color_count[old] -= 1
                if color_count[old] == 0:
                    del color_count[old]
            ball_color[ball] = color
            color_count[color] = color_count.get(color, 0) + 1
            result.append(len(color_count))
        return result
```

PHASE 5 — TEST & DEBUG

queries=[[1,4],[2,5],[1,3],[3,4]]:

[1,4]: count={4:1}. distinct=1 ✓

```
# [2,5]: count={4:1,5:1}. distinct=2 ✓
# [1,3]: old=4,count[4]→0 del. count={5:1,3:1}. distinct=2 ✓
# [3,4]: count={5:1,3:1,4:1}. distinct=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(1) per query. Space O(unique balls + colors). color_count keys = distinct active colors.
# Follow-up: range coloring (paint balls l..r) – needs segment tree / lazy propagation.
# Follow-up: undo queries – maintain a stack of (ball, old_color, new_color) and reverse ops."
```

◆ Quick Review

Key Insights

- Use a dictionary (or hash map) to track the current color for each ball.
- Use another dictionary (or hash map) to count the frequency of each color that is actively used.
- When updating the color of a ball, decrement the frequency of the old color, and if its count reaches zero, remove it.
- Increment the frequency of the new color and record the current number of distinct colors.

Space & Time

Time Complexity: $O(n)$ on average, where n is the number of queries, since each query performs $O(1)$ operations.

Space Complexity: $O(n)$ in the worst case for storing the ball-to-color mapping and the color frequency counts.

Solution

The solution primarily uses two hash maps (dictionaries): one to store the current color of each ball and another to store the frequency of each color that has been applied. For every query: Check if the ball already has a color. If yes, decrement the count for that color and remove it if the count becomes zero. Update the ball's color with the new color and increment the frequency for the new color. After processing the query, the number of keys in the frequency map gives the number of distinct colors currently in use. This approach ensures that each query is processed in $O(1)$ average time.

JavaScript

Round 8 · Low · Medium · 7 questions

JavaScript

□ ★ #2627 Debounce ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a function `fn` and a time in milliseconds `t`, return a debounced version of that function.

A debounced function is a function whose execution is delayed by `t` milliseconds and whose execution is cancelled if it is called again within that window of time. The debounced function should also receive the passed parameters.

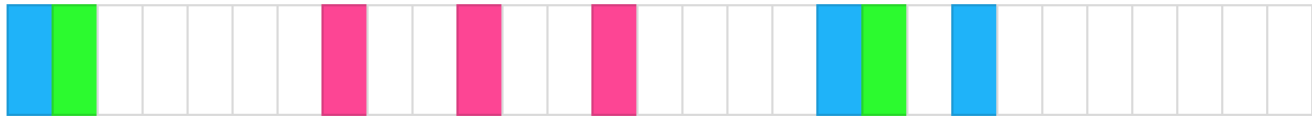
For example, let's say `t = 50ms`, and the function was called at 30ms, 60ms, and 100ms.

The first 2 function calls would be cancelled, and the 3rd function call would be executed at 150ms.

If instead `t = 35ms`, The 1st call would be cancelled, the 2nd would be executed at 95ms,

and the 3rd would be executed at 135ms.

Input Events



Debounced Events



The above diagram shows how debounce will transform events. Each rectangle represents 100ms and the debounce time is 400ms. Each color represents a different set of inputs.

Please solve it without using lodash's `_debounce()` function.

Example 1:

```
Input:
t = 50
calls = [
  {"t": 50, inputs: [1]},
  {"t": 75, inputs: [2]}
]
Output: [{"t": 125, inputs: [2]}]
Explanation:
```

Example 2:

```
Input:
t = 20
calls = [
  {"t": 50, inputs: [1]},
  {"t": 100, inputs: [2]}
]
Output: [{"t": 70, inputs: [1]}, {"t": 120, inputs: [2]}]
Explanation:
```

Example 3:

```
Input:
t = 150
calls = [
    {"t": 50, inputs: [1, 2]},
    {"t": 300, inputs: [3, 4]},
    {"t": 300, inputs: [5, 6]}
]
Output: [{"t": 200, inputs: [1,2]}, {"t": 450, inputs: [5, 6]}]
```

Constraints:

- $0 \leq t \leq 1000$
- $1 \leq \text{calls.length} \leq 10$
- $0 \leq \text{calls}[i].t \leq 1000$
- $0 \leq \text{calls}[i].\text{inputs.length} \leq 10$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"debounce(fn,t): delays fn until t ms after the last call. Each new call resets the timer."

#

"calls at t=0,30,60ms with t=50ms: fn fires once at t=110ms ✓"

PHASE 2 — BRUTE FORCE

"Track a timer id. On each call, cancel the previous timer and set a new one."

Time: $O(1)$ per call. Space: $O(1)$."

```
class _2627_Debounce_Brute:
```

```
    def debounce(self, fn, t_ms: int):
```

```
        import threading
```

```
        timer = [None]
```

```
        def debounced(*args):
```

```
            if timer[0]: timer[0].cancel()
```

```
            timer[0] = threading.Timer(t_ms/1000.0, fn, args)
```

```
            timer[0].start()
```



```
return debounced
```

PHASE 3 — OPTIMIZE

```
# "Same – this IS optimal. One clearTimeout + one setTimeout per call.
# JavaScript: use closure to hold the timer reference."
```

PHASE 4 — CLEAN CODE

```
# JavaScript:
# function debounce(fn, t) {
#   let timer = null;
#   return function(...args) {
#     clearTimeout(timer);
#     timer = setTimeout(() => fn.apply(this, args), t);
#   };
# }

class _2627_Debounce:
    def debounce(self, fn, t_ms: int):
        import threading
        state = {'timer': None}
        def debounced(*args):
            if state['timer']: state['timer'].cancel()
            state['timer'] = threading.Timer(t_ms/1000.0, fn, args)
            state['timer'].start()
        return debounced
```

PHASE 5 — TEST & DEBUG

```
# t=0: cancel(null), set timer(50ms). t=30: cancel timer, set timer(50ms).
# t=60: cancel timer, set timer(50ms). No more calls → fn fires at t=110ms ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "O(1) per call: one cancel + one set. Space O(1): one timer reference.
# Debounce: wait until activity stops. Throttle: fire at most once per interval.
# Follow-up: leading debounce – fire immediately on first call, ignore for t ms after.
# Follow-up: Throttle (LC 2676) – fire at most once per t ms regardless of call frequency."
```

◆ Quick Review

Key Insights

- Each call to the debounced function resets the timer.

- Only the most recent function call (that isn't subsequently cancelled) will eventually execute.
- Passing of parameters is handled by storing the arguments of the latest call.
- The solution leverages timers (or equivalent scheduling mechanisms) and cancellation techniques.

Space & Time

Time Complexity: $O(1)$ per call, as each invocation only clears and sets a timer.

Space Complexity: $O(1)$, since only a single timer reference is maintained.

Solution

The solution creates a wrapper function that maintains an internal timer reference. On each call to the debounced function, if a timer exists, it is cancelled. A new timer is then set to execute the original function after t milliseconds with the provided arguments.

This ensures that only the last invocation runs if multiple calls occur within the delay period.

JavaScript

□ ★ #2628 JSON Deep Equal ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Given two values `o1` and `o2`, return a boolean value indicating whether two values, `o1` and `o2`, are deeply equal.

For two values to be deeply equal, the following conditions must be met:

- If both values are primitive types, they are deeply equal if they pass the `===` equality check.
- If both values are arrays, they are deeply equal if they have the same elements in the same order, and each element is also deeply equal according to these conditions.
- If both values are objects, they are deeply equal if they have the same keys, and the associated values for each key are also deeply

equal according to these conditions.

You may assume both values are the output of `JSON.parse`. In other words, they are valid JSON.

Please solve it without using lodash's `_.isEqual()` function

Example 1:

```
Input: o1 = {"x":1,"y":2}, o2 = {"x":1,"y":2}
Output: true
Explanation: The keys and values match exactly.
```

Example 2:

```
Input: o1 = {"y":2,"x":1}, o2 = {"x":1,"y":2}
Output: true
Explanation: Although the keys are in a different order, they still match exactly.
```

Example 3:

```
Input: o1 = {"x":null,"L":[1,2,3]}, o2 = {"x":null,"L":["1","2","3"]}
Output: false
Explanation: The array of numbers is different from the array of strings.
```

Example 4:

```
Input: o1 = true, o2 = false
Output: false
Explanation: true !== false
```

Constraints:

- $1 \leq \text{JSON.stringify}(o1).length \leq 105$
- $1 \leq \text{JSON.stringify}(o2).length \leq 105$
- `maxNestingDepth` ≤ 1000

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return true if two JSON values are deeply equal. Objects: all key-value pairs
match (order irrelevant).
# Arrays: same length and same elements at each index. Right?"
#
# "areDeeplyEqual({a:1,b:2},{a:1,b:2})→true ✓ areDeeplyEqual([1,[2]],[1,[3]])→false
✓"
```

PHASE 2 — BRUTE FORCE

```
# "JSON.stringify both — fails for object key-order differences.
# Proper: recursive comparison checking types, then structure."
```

```
class _2628_JSONDeepEqual_Brute:
    def areDeeplyEqual(self, o1, o2) -> bool:
        if type(o1) != type(o2): return False
        if isinstance(o1, dict):
            if set(o1) != set(o2): return False
            return all(self.areDeeplyEqual(o1[k], o2[k]) for k in o1)
        if isinstance(o1, list):
            if len(o1) != len(o2): return False
            return all(self.areDeeplyEqual(a, b) for a, b in zip(o1, o2))
        return o1 == o2
```

PHASE 3 — OPTIMIZE

```
# "Same recursive approach — already O(n) nodes. Check === first for fast-path."
```

PHASE 4 — CLEAN CODE

```
# JavaScript:
# function areDeeplyEqual(o1, o2) {
#   if (o1 === o2) return true;
#   if (typeof o1 !== typeof o2 || o1 === null || o2 === null) return o1 === o2;
#   if (Array.isArray(o1) !== Array.isArray(o2)) return false;
#   if (Array.isArray(o1)) return o1.length===o2.length &&
#   o1.every((v,i)=>areDeeplyEqual(v,o2[i]));
#   const k1=Object.keys(o1), k2=Object.keys(o2);
#   return k1.length===k2.length && k1.every(k=>k in o2 &&
#   areDeeplyEqual(o1[k],o2[k]));
# }
```

```
class _2628_JSONDeepEqual:
    def areDeeplyEqual(self, o1, o2) -> bool:
        if o1 is o2 or o1 == o2: return True
        if type(o1) != type(o2): return False
```

```

    if isinstance(o1, dict):
        return (set(o1)==set(o2) and
                all(self.areDeeplyEqual(o1[k],o2[k]) for k in o1))
    if isinstance(o1, list):
        return (len(o1)==len(o2) and
                all(self.areDeeplyEqual(a,b) for a,b in zip(o1,o2)))
    return False

```

PHASE 5 — TEST & DEBUG

```

# areDeeplyEqual({a:{b:1}},{a:{b:1}}):
# both dicts, keys match. recurse 'a': both dicts, keys match. recurse 'b': 1==1
True ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): visit each node once. Space O(d): max recursion depth.
# null requires special handling in JS: typeof null === 'object'.
# Follow-up: Differences Between Two Objects (LC 2700) – return diff instead of
bool.
# Follow-up: handle circular refs – track visited pairs with WeakMap/set of pairs."

```

◆ Quick Review

Key Insights

- Use recursion to compare nested structures.
- For primitive types, a simple equality check (`===` in JavaScript, `==` in Python, etc.) suffices.
- For arrays, verify both have the same length and compare each element recursively.
- For objects, ensure both have the same keys (order does not matter) then recursively compare the corresponding values.
- Handle cases where one operand is null or differs by type early to avoid unnecessary recursion.

Space & Time

Time Complexity: $O(N)$ in the worst-case, where N is the total number of elements/values in the JSON structure.

Space Complexity: $O(N)$ due to the recursion call stack in the worst-case scenario of deeply nested objects or arrays.

Solution

We solve this problem using a recursive approach. The algorithm first checks if both values are strictly equal, which handles primitives immediately. For arrays and objects, it verifies that the sizes (length for arrays and number of keys for objects) match. Then it recurses into each element (for arrays) or checks each key-value pair (for objects) to ensure that they are also deeply equal. For objects, the order of keys is irrelevant, so we compare key sets. This method naturally handles nested structures by delegating equality checks to the same function.

JavaScript

★ #2630 Memoize ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a function `fn`, return a memoized version of that function.

A memoized function is a function that will never be called twice with the same inputs. Instead it will return a cached value.

`fn` can be any function and there are no constraints on what type of values it accepts. Inputs are considered identical if they are `===` to each other.

Example 1:

```
Input:
getInputs = () => [[2,2],[2,2],[1,2]]
fn = function (a, b) { return a + b; }
Output: [{"val":4,"calls":1},{"val":4,"calls":1},{"val":3,"calls":2}]
Explanation:
const inputs = getInputs();
const memoized = memoize(fn);
for (const arr of inputs) {
```

Example 2:

Input:

```
getInputs = () => [[{}],{}], [{}], [{}], [{}]]
```

```
fn = function (a, b) { return ({...a, ...b}); }
```

```
Output: [{"val": {}, "calls": 1}, {"val": {}, "calls": 2}, {"val": {}, "calls": 3}]
```

Explanation:

Merging two empty objects will always result in an empty object. It may seem like there should only be 1 call to fn() because of cache-hits, however none of those objects are === to each other.

Example 3:

Input:

```
getInputs = () => { const o = {}; return [[o,o],[o,o],[o,o]]; }
```

```
fn = function (a, b) { return ({...a, ...b}); }
```

```
Output: [{"val": {}, "calls": 1}, {"val": {}, "calls": 1}, {"val": {}, "calls": 1}]
```

Explanation:

Merging two empty objects will always result in an empty object. The 2nd and 3rd third function calls result in a cache-hit. This is because every object passed in is identical.

Constraints:

- $1 \leq \text{inputs.length} \leq 105$
- $0 \leq \text{inputs.flat().length} \leq 105$
- $\text{inputs}[i][j] \neq \text{NaN}$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"memoize(fn): returns a cached version. Same args → return cached result, skip fn call."

#

"memoize(sum): sum(2,3)→5 (called). sum(2,3)→5 (cached, fn not called).
sum(1,2)→3 (called). ✓"

PHASE 2 — BRUTE FORCE

"Map with stringified args as key. Check on each call; compute and cache if miss."

class _2630_Memoize_Brute:

```
def memoize(self, fn):
    cache = {}
    def memoized(*args):
        if args not in cache:
            cache[args] = fn(*args)
        return cache[args]
    return memoized
```

PHASE 3 — OPTIMIZE

"Same $O(1)$ lookup approach. For JS use `JSON.stringify` for complex/nested args."

PHASE 4 — CLEAN CODE

```
# JavaScript:
# function memoize(fn) {
#   const cache = new Map();
#   return function(...args) {
#     const key = JSON.stringify(args);
#     if (!cache.has(key)) cache.set(key, fn.apply(this, args));
#     return cache.get(key);
#   };
# }
```

```
class _2630_Memoize:
    def memoize(self, fn):
        cache = {}
        def memoized(*args):
            key = str(args)
            if key not in cache:
                cache[key] = fn(*args)
            return cache[key]
        return memoized
```

PHASE 5 — TEST & DEBUG

```
# fn=lambda a,b: a+b. memoized(2,3): key='(2, 3)', miss→compute 5, cache. return 5.
# memoized(2,3): key='(2, 3)', hit → return 5 (fn not called) ✓.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "First call  $O(fn)$ . Cache hit  $O(1)$ . Space  $O(\text{unique arg combinations} * \text{result size})$ .
# JSON.stringify adds  $O(\text{arg\_size})$  per call – acceptable for numbers/strings.
# Follow-up: LRU-bounded memoize – evict LRU when cache exceeds limit.
# Follow-up: async memoize – cache the Promise, not the resolved value."
```

◆ Quick Review

Key Insights

- Cache previously computed results using a data structure keyed by the function arguments.
- Ensure the key uniquely represents the argument order; simple tuple (or equivalent) suffices.
- Use closures (or object state) to maintain both the cache and a call count tracking actual function invocations.
- Expose a method (getCallCount) on the memoized function to report the number of times the original function was actually called.
- The memoized version works for functions that might take one or more parameters.

Space & Time

Time Complexity: $O(1)$ average for each call assuming efficient hashing of arguments.

Space Complexity: $O(n)$ for caching n distinct input combinations.

Solution

We use a hash-based cache (a dictionary or map) to store results for every unique set of arguments. Upon each call, we check if the key (constructed from the input arguments) exists

in the cache. If not, we increment a counter, call the original function, store the result in the cache, and return it. Otherwise, we return the cached result. The memoized function is augmented with a `getCallCount` method to report the number of actual calls made to the original function.

JavaScript

□ ★ #2633 Convert Object to JSON String ★ MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a value, return a valid JSON string of that value. The value can be a string, number, array, object, boolean, or null. The returned string should not include extra spaces. The order of keys should be the same as the order returned by `Object.keys()`.

Please solve it without using the built-in `JSON.stringify` method.

Example 1:

```
Input: object = {"y":1,"x":2}
Output: {"y":1,"x":2}
Explanation:
Return the JSON representation.
Note that the order of keys should be the same as the order returned by
Object.keys().
```

Example 2:

```
Input: object = {"a":"str","b":-12,"c":true,"d":null}
Output: {"a":"str","b":-12,"c":true,"d":null}
Explanation:
The primitives of JSON are strings, numbers, booleans, and null.
```

Example 3:

```
Input: object = {"key":{"a":1,"b":[{"},null,"Hello"]}}
Output: {"key":{"a":1,"b":[{"},null,"Hello"]}}
Explanation:
Objects and arrays can include other objects and arrays.
```

Example 4:

```
Input: object = true
Output: true
Explanation:
Primitive types are valid inputs.
```

Constraints:

- value is a valid JSON value
- $1 \leq \text{JSON.stringify(object).length} \leq 105$
- $\text{maxNestingLevel} \leq 1000$
- all strings contain only alphanumeric characters

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Implement JSON.stringify without using it. Handle null, bool, number, string, array, object."

#

"jsonify({a:2,b:[1,2,3]}) → '{"a":2,"b":[1,2,3]}' ✓"

PHASE 2 — BRUTE FORCE

"Recursive: dispatch on type. Build string for each type appropriately."

```
class _2633_ObjectToJSONString_Brute:
    def jsonify(self, obj) -> str:
        if obj is None: return 'null'
        if isinstance(obj, bool): return 'true' if obj else 'false'
        if isinstance(obj, (int,float)): return str(obj)
        if isinstance(obj, str): return f'"{obj}"'
        if isinstance(obj, list):
            return '[' + ','.join(self.jsonify(v) for v in obj) + ']'
        if isinstance(obj, dict):
            return '{' + ','.join(f'"{k}":{self.jsonify(v)}' for k,v in
obj.items()) + '}'
        return 'null'
```

PHASE 3 — OPTIMIZE

"Same — already $O(n)$. Use join to avoid $O(n^2)$ string concatenation."

PHASE 4 — CLEAN CODE (same as brute — already clean)

```
# JavaScript:
# function jsonStringify(obj) {
# if (obj === null) return 'null';
# if (typeof obj === 'boolean') return String(obj);
# if (typeof obj === 'number') return String(obj);
# if (typeof obj === 'string') return `"${obj}"`;
# if (Array.isArray(obj)) return `[${obj.map(jsonStringify).join(',')}]`;
# return
`[${Object.entries(obj).map(([k,v])=>`${k}:${jsonStringify(v)}`).join(',')}]`;
# }
```

PHASE 5 — TEST & DEBUG

```
# obj=[null,true,1,"foo"]:
# array. elements: 'null','true','1','"foo"'. joined: 'null,true,1,"foo"'. wrap:
'[null,true,1,"foo"]' ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ : one visit per node. Space  $O(d)$ : recursion depth.
# bool must be checked before number (isinstance(True,int) is True in Python).
# Follow-up: handle circular references — track visited with a Set.
# Follow-up: pretty-print with indentation — add indent parameter and track depth."
```

◆ Quick Review

Key Insights

- The solution must support all JSON primitives: strings, numbers, booleans, and

null.

- **Composite types such as arrays and objects require recursive traversal for proper serializing.**
- **For strings, wrap them in double quotes.**
- **For arrays, recursively process every element and join them with commas between square brackets.**
- **For objects, iterate keys in insertion order and combine key–value pairs (key must be a string in double quotes) with commas between curly braces.**
- **Use recursion carefully to handle nested structures without adding unnecessary spaces.**

Space & Time

Time Complexity: $O(n)$, where n is the total number of elements/characters in the JSON structure.

Space Complexity: $O(n)$ due to the recursion stack and the constructed output string.

Solution

We use a recursive approach that inspects the type of the value and processes it accordingly:

If the value is null, output "null". If the value is a boolean or number, convert it directly to its string representation. For a string, ensure it is enclosed in double quotes. For an array, iterate over each element, serialize each recursively, join them with commas, and enclose in square brackets. For an object, iterate over its keys in insertion order (or using `Object.keys` in JS / `LinkedHashMap` in Java), serialize both key (wrapped in double quotes) and its corresponding value, join with commas, and enclose in curly braces. This approach leverages recursion to handle arbitrarily nested objects and arrays while ensuring there are no extra spaces in the final string.

JavaScript

□ ★ #2636 Promise Pool ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript · Concurrency

Lists: ★ Premium | Premium 98

Problem

Given an array of asynchronous functions and a pool limit n , return an asynchronous function `promisePool`. It should return a promise that resolves when all the input functions resolve.

Pool limit is defined as the maximum number promises that can be pending at once. `promisePool` should begin execution of as many functions as possible and continue executing new functions when old promises resolve. `promisePool` should execute `functions[i]` then `functions[i + 1]` then `functions[i + 2]`, etc. When the last promise resolves, `promisePool` should also resolve.

For example, if $n = 1$, `promisePool` will execute one function at a time in series. However, if $n =$

2, it first executes two functions. When either of the two functions resolve, a 3rd function should be executed (if available), and so on until there are no functions left to execute.

You can assume all functions never reject. It is acceptable for `promisePool` to return a promise that resolves any value.

Example 1:

```
Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 2
Output: [[300,400,500],500]
```

Example 2:

```
Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 5
Output: [[300,400,200],400]
```

Example 3:

```
Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 1
Output: [[300,700,900],900]
```

Constraints:

- $0 \leq \text{functions.length} \leq 10$
- $1 \leq n \leq 10$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Execute async functions with at most n concurrent. Return promise resolving when
all done."
#
# "promisePool([fn1,fn2,fn3,fn4], n=2): run fn1+fn2, as each finishes start fn3 then
fn4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Promise.all on all functions – no concurrency limit, may overwhelm resources."
class _2636_PromisePool_Brute:
    async def pool(self, functions, n: int):
        import asyncio
        await asyncio.gather(*[fn() for fn in functions])
```

PHASE 3 — OPTIMIZE

```
# "n worker coroutines, each pulls from a shared queue.
# Each worker runs one fn at a time sequentially. n workers run in parallel.
# await all workers → all tasks complete."
```

PHASE 4 — CLEAN CODE

```
# JavaScript:
# async function promisePool(functions, n) {
#   const queue = [...functions];
#   async function worker() {
#     while (queue.length) await queue.shift()();
#   }
#   await Promise.all(Array.from({length:Math.min(n,function.length)}, worker));
# }

class _2636_PromisePool:
    async def pool(self, functions, n: int):
        import asyncio
        sem = asyncio.Semaphore(n)
        async def run(fn):
```

```
    async with sem: await fn()
    await asyncio.gather(*[run(fn) for fn in functions])
```

PHASE 5 — TEST & DEBUG

```
# [fn1,fn2,fn3,fn4], n=2:
# worker1: await fn1(), await fn3(). worker2: await fn2(), await fn4().
# Both workers run concurrently → max 2 concurrent at any time ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time optimal: ceiling(total_tasks/n) rounds if tasks equal duration.
# Space O(n): n concurrent promises. Worker pattern handles unequal durations
naturally.
# Follow-up: cancel on first failure – Promise.race with rejection propagation.
# Follow-up: retry failed tasks – wrap fn() in a retry loop inside the worker."
```

◆ Quick Review

Key Insights

- Use a concurrency control mechanism to limit the number of actively running promises.
- Maintain an index or pointer to track which function from the input array should be executed next.
- Upon the resolution of any promise, start a new one if available until all functions have been processed.
- The overall task is complete when there are no functions left and all running promises have resolved.

Space & Time

Time Complexity: $O(m)$, where m is the number of functions to execute.

Space Complexity: $O(n)$ for the active pool of concurrent promises.

Solution

We can solve the problem by maintaining an index that tracks which asynchronous function to execute next. We define a helper routine (or executor) that: Checks if there is a function left to run. If yes, increments the index and awaits the result of running that function. Once the function completes, the routine recursively calls itself to pick up the next function. We start n such routines (to respect the pool limit) and use `Promise.all` (or the equivalent in other languages) to wait until all concurrent routines have finished executing. This structure ensures that we never run more than n promises concurrently and that a new promise starts as soon as one finishes.

JavaScript

□ ★ #2675 Array of Objects to Matrix ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript · Array

Lists: ★ Premium | Premium 98

Problem

Write a function that converts an array of objects `arr` into a matrix `m`.

`arr` is an array of objects or arrays. Each item in the array can be deeply nested with child arrays and child objects. It can also contain numbers, strings, booleans, and null values.

The first row `m` should be the column names. If there is no nesting, the column names are the unique keys within the objects. If there is nesting, the column names are the respective paths in the object separated by ".".

Each of the remaining rows corresponds to an object in `arr`. Each value in the matrix corresponds to a value in an object. If a given object doesn't contain a value for a given column, the cell should contain an empty

string "".

The columns in the matrix should be in lexicographically ascending order.

Example 1:

```
Input:
arr = [
  {"b": 1, "a": 2},
  {"b": 3, "a": 4}
]
Output:
[
  ["a", "b"],
```

Example 2:

```
Input:
arr = [
  {"a": 1, "b": 2},
  {"c": 3, "d": 4},
  {}
]
Output:
[
```

Example 3:

```
Input:
arr = [
  {"a": {"b": 1, "c": 2}},
  {"a": {"b": 3, "d": 4}}
]
Output:
[
  ["a.b", "a.c", "a.d"],
```

Example 4:


```

Input:
arr = [
  [{"a": null}],
  [{"b": true}],
  [{"c": "x"}]
]
Output:
[

```

Example 5:

```

Input:
arr = [
  {},
  {},
  {}
]
Output:
[

```

Constraints:

- arr is a valid JSON array
- $1 \leq \text{arr.length} \leq 1000$
- unique keys ≤ 1000

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Convert array of objects to a matrix. Row 0 = sorted flattened keys (dot notation for nested).

Remaining rows = values for each object ('' if key missing). Right?"

#

"[{a:1,b:2},{b:3,c:4}] → [['a','b','c'],[1,2,''],['',3,4]] ✓"

PHASE 2 — BRUTE FORCE

"Flatten each object with dot notation, collect all keys, build matrix row by row."

```

class _2675_ArrayOfObjectsToMatrix_Brute:
    def jsonToMatrix(self, arr: list) -> list:
        def flatten(obj, prefix=''):
            result = {}
            if isinstance(obj, dict):
                for k,v in obj.items():
                    result.update(flatten(v, f'{prefix}.{k}' if prefix else k))
            else:
                result[prefix] = obj
            return result
        flat = [flatten(o) for o in arr]
        keys = sorted(set(k for f in flat for k in f))
        return [keys] + [[f.get(k, '') for k in keys] for f in flat]

# PHASE 3 — OPTIMIZE
# "Same O(n*k) — already optimal. Single pass to collect keys."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# arr=[{a:1,b:{c:2}}, {a:3}]:
# flat: {'a':1, 'b.c':2} and {'a':3}. keys=['a', 'b.c'].
# rows: [1,2] and [3, ''] -> [['a', 'b.c'], [1,2], [3, '']] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*k*d): n objects, k unique keys, d nesting depth. Space O(n*k): matrix.
# Dot notation is the CSV-export standard for nested JSON.
# Follow-up: handle arrays inside objects — use bracket notation [0],[1].
# Follow-up: reverse (matrix to array of objects) — parse dot-paths back to nested."

```

◆ Quick Review

Key Insights

- Flatten each object or array into dot-separated paths such as `a.b` or `a.0.c`.
- Use DFS so nested objects and nested arrays are handled uniformly.
- For each row, store a map from flattened path to value.

- Collect every path seen across all rows into a set of column names.
- Sort the column names lexicographically to build the header row.
- For each flattened row map, output values in header order and use an empty string when a column is missing.

Solution

We solve the problem by flattening every input row first. A DFS walks through objects and arrays, extending the current path with either the property name or the array index. When the DFS reaches a primitive value or `null`, it stores that value under the built path. While flattening each row, we also collect every path into a global set of column names. After all rows are processed, we sort the columns lexicographically, place them in the first row of the matrix, and then build each remaining row by reading values from that row's flattened map, using `""` for missing paths.

JavaScript

□ ★ #2700 Differences Between Two Objects ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Write a function that accepts two deeply nested objects or arrays `obj1` and `obj2` and returns a new object representing their differences.

The function should compare the properties of the two objects and identify any changes. The returned object should only contains keys where the value is different from `obj1` to `obj2`.

For each changed key, the value should be represented as an array [`obj1` value, `obj2` value]. Keys that exist in one object but not in the other should not be included in the returned object. The end result should be a deeply nested object where each leaf value is a difference array.

When comparing two arrays, the indices of the arrays are considered to be their keys.

You may assume that both objects are the output of `JSON.parse`.

Example 1:

Input:

```
obj1 = {}
```

```
obj2 = {
```

```
  "a": 1,
```

```
  "b": 2
```

```
}
```

Output: {}

Explanation: There were no modifications made to `obj1`. New keys "a" and "b" appear in `obj2`, but keys that are added or removed should be ignored.

Example 2:

Input:

```
obj1 = {
```

```
  "a": 1,
```

```
  "v": 3,
```

```
  "x": [],
```

```
  "z": {
```

```
    "a": null
```

```
}
```

Example 3:

Input:

```
obj1 = {
```

```
  "a": 5,
```

```
  "v": 6,
```

```
  "z": [1, 2, 4, [2, 5, 7]]
```

```
}
```

```
obj2 = {
```

```
  "a": 5,
```

Example 4:

```

Input:
obj1 = {
  "a": {"b": 1},
}
obj2 = {
  "a": [5],
}
Output:

```

Example 5:

```

Input:
obj1 = {
  "a": [1, 2, {}],
  "b": false
}
obj2 = {
  "b": false,
  "a": [1, 2, {}]
}

```

Constraints:

- obj1 and obj2 are valid JSON objects or arrays
- $2 \leq \text{JSON.stringify(obj1).length} \leq 104$
- $2 \leq \text{JSON.stringify(obj2).length} \leq 104$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return an object showing paths where two JSON objects differ.

[obj1_val, obj2_val] at leaf differences. Skip keys present in only one object."

#

"diff({a:1,b:2},{a:1,b:3}) → {b:[2,3]} ✓

diff({a:{b:1}},{a:{b:2}}) → {a:{b:[1,2]}} ✓"

PHASE 2 — BRUTE FORCE

```

# "Recursive: for each key in obj1 that also exists in obj2:
# if both are objects, recurse and collect non-empty sub-diffs.
# if values differ (or types differ), mark as [v1,v2]. If equal, skip."
class _2700_DiffBetweenObjects_Brute:
    def objDiff(self, obj1, obj2) -> dict:
        if not isinstance(obj1,dict) or not isinstance(obj2,dict):
            return {} if obj1 == obj2 else [obj1,obj2]
        result = {}
        for k in obj1:
            if k not in obj2: continue
            sub = self.objDiff(obj1[k], obj2[k])
            if sub != {}: result[k] = sub
        return result

# PHASE 3 — OPTIMIZE
# "Same — already O(n). The key insight: skip keys in only one object."

# PHASE 4 — CLEAN CODE (same as brute — already clean)
# JavaScript:
# function objDiff(obj1, obj2) {
#   if (typeof obj1!=='object' || obj1===null || typeof obj2!=='object' || obj2===null)
#   return obj1===obj2 ? {} : [obj1,obj2];
#   const result={};
#   for (const k of Object.keys(obj1)) {
#     if (!(k in obj2)) continue;
#     const diff=objDiff(obj1[k],obj2[k]);
#     if (Object.keys(diff).length || Array.isArray(diff)) result[k]=diff;
#   }
#   return result;
# }

# PHASE 5 — TEST & DEBUG
# obj1={a:1,b:{c:2,d:3}}, obj2={a:1,b:{c:2,d:4}}:
# 'a': 1==1 → {}. 'b': both dicts, recurse. 'c': 2==2 → {}. 'd': 3≠4 → [3,4].
# result={b:{d:[3,4]}} ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one visit per node. Space O(d): recursion depth.
# null check: typeof null==='object' in JS — must check explicitly.
# Follow-up: include keys present in only one object — mark as [value, undefined].
# Follow-up: JSON Deep Equal (LC 2628) — return bool instead of diff."

```

◆ Quick Review

Key Insights

- Use recursion to traverse nested objects and arrays.
- Compare only keys that exist in both objects.
- When encountering two values of different types or unequal primitives, record the difference as [value from obj1, value from obj2].
- When both values are objects or arrays, recursively compute their differences and include them only if the resulting nested diff is non-empty.
- Arrays are treated like objects with indices as keys.
- The solution requires careful type checking and handling of recursive structures.

Space & Time

Time Complexity: $O(n)$ in average, where n is the number of keys/elements in the common parts of the structures.

Space Complexity: $O(n)$ due to recursion and storage needed for the output differences.

Solution

We solve the problem using a recursive function that compares keys present in both input objects. For each common key: If both values are objects (or both are arrays), recursively compute their differences. If the recursive call returns a non-empty object, include that key in the result. If the two values are of different types or are primitives that differ, add the key with a difference array [value from obj1, value from obj2]. Ignore keys that exist only in one object. This recursive approach naturally handles the deeply nested structure while ensuring only common keys with different values are captured.

Round 9 · Low · Hard

Round 9

Low Priority · Hard

7 questions · 2 patterns

Dynamic Programming #10 #115 #123 #132
#312 #1000

Math #68

Dynamic Programming

Round 9 · Low · Hard · 6 questions

Dynamic Programming

□ #10 Regular Expression Matching

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Recursion
Lists: Denny Zhang

Problem

Given an input string *s* and a pattern *p*, implement regular expression matching with support for '.' and '*' where:

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1:

Input: *s* = "aa", *p* = "a"

Output: false

Explanation: "a" does not match the entire string "aa".

Example 2:

Input: s = "aa", p = "a*"

Output: true

Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".

Example 3:

Input: s = "ab", p = ".*"

Output: true

Explanation: ".*" means "zero or more (*) of any character (.)".

Constraints:

- $1 \leq s.length \leq 20$
- $1 \leq p.length \leq 20$
- s contains only lowercase English letters.
- p contains only lowercase English letters, '.', and '*'.
- It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

'.' matches any single char. '*' matches zero or more of the preceding element.

Full string must match. Right?"

#

"s='aa',p='a*'→true ✓ s='aa',p='a'→false ✓ s='aab',p='c*a*b'→true ✓"

PHASE 2 — BRUTE FORCE

"Recursion with memo: match(i,j)=does s[i:] match p[j:]?"

If p[j+1]=='*': zero uses (match i,j+2) or one+ uses (first_char and match i+1,j).

Time: $O(m \times n)$ with memo. Space: $O(m \times n)$."

```
class _10_RegexMatching_Brute:
    def isMatch(self, s: str, p: str) -> bool:
        from functools import lru_cache
        @lru_cache(maxsize=None)
        def dp(i, j):
            if j == len(p): return i == len(s)
            first = i < len(s) and (p[j] == s[i] or p[j] == '.')
            if j+1 < len(p) and p[j+1] == '*':
                return dp(i, j+2) or (first and dp(i+1, j))
            return first and dp(i+1, j+1)
        return dp(0, 0)
```

PHASE 3 — OPTIMIZE

"Bottom-up 2D DP. $dp[i][j]$ =does $s[i:]$ match $p[j:]$? Fill right-to-left, bottom-to-top.

Space: $O(n)$ with rolling rows."

PHASE 4 — CLEAN CODE

```
class _10_RegexMatching:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        dp = [False]*(n+1)
        dp[n] = True
        for i in range(m, -1, -1):
            new = [False]*(n+1)
            new[n] = (i == m)
            for j in range(n-1, -1, -1):
                first = (i < m) and (p[j] == s[i] or p[j] == '.')
                if j+1 < n and p[j+1] == '*':
                    new[j] = dp[j+2] or (first and dp[j])
                    if j+2 <= n: new[j] = new[j] or dp[j+2]
                else:
                    new[j] = first and dp[j+1]
            dp = new
        return dp[0]
```

PHASE 5 — TEST & DEBUG

$s='aab'$, $p='c*a*b'$: $c \rightarrow \text{skip}(j+2)$, $a \rightarrow \text{skip or use}$, $b \rightarrow \text{match}$. $dp[0]=\text{True}$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \times n)$. Space $O(n)$: two rolling rows.

'*' has two branches: zero occurrences (skip $j+2$) or one-plus (consume $s[i]$, stay at j).

Follow-up: Wildcard Matching (LC 44) – '*' matches any sequence. Same DP, simpler transitions."

◆ Quick Review

Key Insights

- Use recursion or dynamic programming to simulate matching as the '*' operator can affect the current and following characters.
- The '.' operator is a wildcard that matches any single character.
- When encountering a '*' in the pattern, consider both skipping the preceding element or consuming one character from the string if it matches.
- Using memoization helps to save intermediate results and efficiently process overlapping subproblems.

Space & Time

Time Complexity: $O(mn)$ where $m = \text{length}(s)$ and $n = \text{length}(p)$, due to exploring each subproblem combination.

Space Complexity: $O(mn)$ for storing the memoization table in the dynamic programming or recursive approach.

Solution

The solution uses a top-down recursive strategy with memoization to efficiently match

the string against the pattern. For each character position in s and p , we check if they match directly or if the pattern character is '.' to allow any match. When encountering '.' in the pattern, two possibilities are explored: either the '.' stands for zero occurrences of the preceding element, or if there is a valid match, it accounts for one occurrence and we proceed further in the string while maintaining the same pattern index. The memoization aspect prevents repeated computations of the same (i, j) state in the recursion, resulting in an improved overall runtime.

Dynamic Programming

□ #115 Distinct Subsequences

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings s and t , return the number of distinct subsequences of s which equals t .

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1:

Input: $s = \text{"rabbbit"}, t = \text{"rabbit"}$

Output: 3

Explanation:

As shown below, there are 3 ways you can generate "rabbit" from s .

rabbbit

rabbbit

rabbbit

Example 2:

Input: $s = \text{"babgbag"}, t = \text{"bag"}$

Output: 5

Explanation:

As shown below, there are 5 ways you can generate "bag" from s .

babgbag

babgbag

babgbag

babgbag

Constraints:

- $1 \leq s.length, t.length \leq 1000$
- s and t consist of English letters.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count distinct subsequences of s that equal t . Right?"

#

" s ='rabbbit', t ='rabbit'→3 ✓ s ='babgbag', t ='bag'→5 ✓"

PHASE 2 — BRUTE FORCE

"Recursion: $\text{count}(i, j) = \text{ways } s[i:] \text{ produces } t[j:]$.

$j == \text{len}(t) \rightarrow 1$. $i == \text{len}(s) \rightarrow 0$. $s[i] == t[j]$: $\text{count}(i+1, j+1) + \text{count}(i+1, j)$. else: $\text{count}(i+1, j)$."

```
class _115_DistinctSubsequences_Brute:
```

```
    def numDistinct(self, s: str, t: str) -> int:
```

```
        from functools import lru_cache
```

```
        @lru_cache(maxsize=None)
```

```
        def dp(i, j):
```

```
            if j == len(t): return 1
```

```
            if i == len(s): return 0
```

```
            res = dp(i+1, j)
```

```
            if s[i] == t[j]: res += dp(i+1, j+1)
```

```
            return res
```

```
        return dp(0, 0)
```

PHASE 3 — OPTIMIZE

"Bottom-up DP. $\text{dp}[j] = \text{ways to form } t[j:] \text{ from } s[i:]$. Roll right-to-left.

Time: $O(m \times n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _115_DistinctSubsequences:
```

```
    def numDistinct(self, s: str, t: str) -> int:
```

```
        dp = [0] * len(t) + [1]
```

```
        for ch in reversed(s):
```

```
            for j in range(len(t)-1, -1, -1):
```

```
                if ch == t[j]:
```

```
                    dp[j] += dp[j+1]
```

```
return dp[0]
```

PHASE 5 — TEST & DEBUG

```
# s='rabbbit',t='rabbit': three 'b's in s for two 'b's in t → 3 ways ✓
# dp updated right-to-left prevents using same s[i] twice in one match.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(n): one rolling array.
# Right-to-left update is essential – prevents reuse of s[i] across matches.
# Follow-up: print all distinct subsequences – backtrack through the DP table.
# Follow-up: Edit Distance (LC 72) – related: counts edits instead of matching occurrences."
```

◆ Quick Review

Key Insights

- Use dynamic programming to build the solution by comparing characters of s and t.
- Define a 2D dp table where $dp[i][j]$ represents the number of ways to form the first j characters of t using the first i characters of s.
- The recurrence relation:
 - If $s[i-1]$ equals $t[j-1]$, then include both the cases of matching this character as well as skipping it: $dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$.
 - If they do not match, then simply skip the current character of s: $dp[i][j] = dp[i-1][j]$.
- Initialize $dp[i][0] = 1$ for all i since an empty t can always be formed.
- The final answer is $dp[s.length][t.length]$.

Space & Time

Time Complexity: $O(n * m)$, where n is the length of s and m is the length of t .

Space Complexity: $O(n * m)$, which can be optimized to $O(m)$ with careful row reuse.

Solution

We use a dynamic programming approach with a 2D table. The table has dimensions $(\text{len}(s) + 1) \times (\text{len}(t) + 1)$. Each cell $\text{dp}[i][j]$ stores the number of distinct subsequences from the first i characters of s that match the first j characters of t . Steps: Initialize $\text{dp}[i][0] = 1$ for every i , because an empty t is a subsequence of any prefix of s . Iterate over each character of s (outer loop) and for each, iterate over t (inner loop). For every character in s , if it equals the current character in t , update $\text{dp}[i][j]$ as the sum of the ways by using or skipping this character. Otherwise, carry over the previous count. Return $\text{dp}[\text{len}(s)][\text{len}(t)]$ as the result.

Dynamic Programming

□ #123 Best Time to Buy and Sell Stock III

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Denny Zhang

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: `prices = [3,3,5,0,0,3,1,4]`

Output: 6

Explanation: Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit = $3 - 0 = 3$.

Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit = $4 - 1 = 3$.

Example 2:

Input: prices = [1,2,3,4,5]

Output: 4

Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.

Note that you cannot buy on day 1, buy on day 2 and sell them later, as you are engaging multiple transactions at the same time. You must sell before buying again.

Example 3:

Input: prices = [7,6,4,3,1]

Output: 0

Explanation: In this case, no transaction is done, i.e. max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 105$
- $0 \leq \text{prices}[i] \leq 105$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"At most 2 transactions. Maximize profit. No holding two stocks simultaneously. Right?"

#

"prices=[3,3,5,0,0,3,1,4]: buy@0,sell@3,buy@1,sell@4 → profit=3+3=6 ✓"

PHASE 2 — BRUTE FORCE

"Try all pairs of buy1<sell1<=buy2<sell2. Time: $O(n^4)$. Space: $O(1)$."

```
class _123_StockIII_Brute:
```

```
    def maxProfit(self, prices: list) -> int:
```

```
        n, best = len(prices), 0
```

```
        for b1 in range(n):
```

```
            for s1 in range(b1+1, n):
```

```
                for b2 in range(s1, n):
```

```
                    for s2 in range(b2+1, n):
```

```
                        best = max(best,
```

```
prices[s1]-prices[b1]+prices[s2]-prices[b2])
```

```
        return best
```

PHASE 3 — OPTIMIZE**# "State machine: 4 states track max profit at each stage.****# buy1,sell1,buy2,sell2 updated in one pass. Time: $O(n)$. Space: $O(1)$."****# PHASE 4 — CLEAN CODE**

```

class _123_StockIII:
    def maxProfit(self, prices: list) -> int:
        buy1 = buy2 = float('-inf')
        sell1 = sell2 = 0
        for p in prices:
            buy1 = max(buy1, -p)
            sell1 = max(sell1, buy1 + p)
            buy2 = max(buy2, sell1 - p)
            sell2 = max(sell2, buy2 + p)
        return sell2

```

PHASE 5 — TEST & DEBUG**# prices=[3,3,5,0,0,3,1,4]:****# ...after full pass: sell2=6 ✓ (buy@0,sell@3 + buy@1,sell@4)****# PHASE 6 — COMPLEXITY & FOLLOW-UP****# " $O(n)$ one pass. $O(1)$ four state variables. Elegant state machine for at-most-k constraint.****# Follow-up: at most k transactions (LC 188) – generalize to 2k variables or $O(n*k)$ DP.****# Follow-up: unlimited (LC 122) – sum all positive consecutive differences."**

◆ Quick Review

Key Insights

- The problem requires the optimal selection of at most two buy–sell transactions.
- One effective strategy is to split the problem into two parts:
- For each day i , calculate the maximum profit obtainable from day 0 to i with one transaction.

- For each day i , calculate the maximum profit obtainable from day i to the end with one transaction.
- Finally, for each day, the sum of these two profits (left and right) gives a candidate answer.
- Alternatively, a state-based dynamic programming approach can track 4 states corresponding to the two transactions.

Space & Time

Time Complexity: $O(n)$ where n is the number of days

Space Complexity: $O(n)$ due to extra arrays used for forward and backward computations (this can be optimized to $O(1)$ with a state variable approach)

Solution

We use a two-pass dynamic programming approach. In the first pass (forward), we compute an array "leftProfits" where $\text{leftProfits}[i]$ is the maximum profit from day 0 to day i with one transaction. We do this by keeping track of the minimum price observed so far and calculating the profit if sold on day

i. In the second pass (backward), we compute an array "rightProfits" where $\text{rightProfits}[i]$ is the maximum profit obtainable from day i to the last day with one transaction by tracking the maximum price seen in reverse order. Finally, for every index i , the sum $\text{leftProfits}[i] + \text{rightProfits}[i]$ represents the maximum profit achievable by completing the first transaction on or before day i and the second transaction starting from day i . The answer is the maximum sum over all i .

Dynamic Programming

□ #132 Palindrome Partitioning II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given a string s , partition s such that every substring of the partition is a palindrome.
Return the minimum cuts needed for a palindrome partitioning of s .

Example 1:

Input: $s = \text{"aab"}$
Output: 1
Explanation: The palindrome partitioning $[\text{"aa"}, \text{"b"}]$ could be produced using 1 cut.

Example 2:

Input: $s = \text{"a"}$
Output: 0

Example 3:

Input: $s = \text{"ab"}$
Output: 1

Constraints:

- $1 \leq s.length \leq 2000$

- s consists of lowercase English letters only.

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Minimum cuts to partition s into palindromes. Right?"

#

"s='aab'→1 ('aa'|'b') ✓ s='a'→0 ✓ s='ab'→1 ✓"

PHASE 2 — BRUTE FORCE

"Recursion: minCuts(i)=min cuts for s[i:]. Try each palindrome prefix, recurse on rest."

```
class _132_PalindromePartitioningII_Brute:
    def minCut(self, s: str) -> int:
        from functools import lru_cache
        n = len(s)
        @lru_cache(maxsize=None)
        def is_pal(l, r):
            return l >= r or (s[l]==s[r] and is_pal(l+1,r-1))
        @lru_cache(maxsize=None)
        def dp(i):
            if i == n: return -1
            return min(1 + dp(j+1) for j in range(i, n) if is_pal(i, j))
        return dp(0)
```

PHASE 3 — OPTIMIZE

"Precompute palindrome table $O(n^2)$. $dp[i]=\text{min cuts for } s[:i+1]$. $O(n^2)$ time $O(n)$ space."

PHASE 4 — CLEAN CODE

```
class _132_PalindromePartitioningII:
    def minCut(self, s: str) -> int:
        n = len(s)
        is_pal = [[False]*n for _ in range(n)]
        for i in range(n-1,-1,-1):
            for j in range(i,n):
                is_pal[i][j] = (s[i]==s[j]) and (j-i<=2 or is_pal[i+1][j-1])
        dp = list(range(-1, n))
        for i in range(n-1,-1,-1):
            for j in range(i,n):
                if is_pal[i][j]:
```

```

        dp[i] = min(dp[i], 1+dp[j+1])
    return dp[0]

```

PHASE 5 — TEST & DEBUG

```

# s='aab': is_pal[0][1]=T(aa). dp[2]=0(b), dp[1]=1(ab→b=1cut), dp[0]:
is_pal[0][0]T→1+dp[1]=2, is_pal[0][1]T→1+dp[2]=1. dp[0]=1 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n^2). Space O(n^2) palindrome table + O(n) dp.

```

```

# Follow-up: Palindrome Partitioning I (LC 131) – return all partitions
(backtracking + table).

```

```

# Follow-up: Palindrome Partitioning IV (LC 1745) – fixed 3 parts; use same
palindrome table."

```

◆ Quick Review

Key Insights

- Precompute a 2D table (isPal) where isPal[i][j] indicates if the substring s[i...j] is a palindrome.
- Use dynamic programming: dp[i] represents the minimum cuts needed for the substring s[0...i].
- For every index i, iterate over all possible starting indices j. If s[j...i] is a palindrome, update dp[i] using dp[j-1] + 1 (or 0 if j is 0).
- Time complexity is dominated by the O(n^2) palindrome checking and state transitions.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$ (due to the 2D table for palindrome checks, plus $O(n)$ for the dp array)

Solution

The solution relies on dynamic programming with a two-step approach. First, precompute a 2D table (isPal) where each cell indicates if a substring is a palindrome. This is done in $O(n^2)$ time by leveraging the fact that a substring $s[j...i]$ is a palindrome if the characters at the ends match and the inner substring $s[j+1...i-1]$ is also a palindrome (or if the length is less than 3). Then, use a dp array where $dp[i]$ holds the minimum number of cuts needed for the substring $s[0...i]$. For each i , check every index j ($0 \leq j \leq i$) and if $s[j...i]$ is a palindrome, update $dp[i]$ to be the minimum of its current value and $dp[j-1] + 1$ (with a special case when j is 0, meaning no cut is needed). This two-phase approach efficiently breaks down the problem and computes the answer while avoiding redundant palindrome checks.

Dynamic Programming

□ #312 Burst Balloons

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Denny Zhang

Problem

You are given n balloons, indexed from 0 to $n - 1$. Each balloon is painted with a number on it represented by an array `nums`. You are asked to burst all the balloons.

If you burst the i th balloon, you will get $\text{nums}[i - 1] * \text{nums}[i] * \text{nums}[i + 1]$ coins. If $i - 1$ or $i + 1$ goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it.

Return the maximum coins you can collect by bursting the balloons wisely.

Example 1:

```
Input: nums = [3,1,5,8]
Output: 167
Explanation:
nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []
coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167
```

Example 2:

Input: nums = [1,5]

Output: 10

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 300$
- $0 \leq \text{nums}[i] \leq 100$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Burst balloon i: earn $\text{nums}[\text{left}] * \text{nums}[i] * \text{nums}[\text{right}]$. Pad array with 1s on both ends.

Maximize total coins. Right?"

#

"nums=[3,1,5,8]→167 ✓"

PHASE 2 — BRUTE FORCE

"Try all permutations of burst order. Time: $O(n!)$. Space: $O(n)$."

class _312_BurstBalloons_Brute:

```
def maxCoins(self, nums: list) -> int:
    from itertools import permutations
    nums = [1]+nums+[1]; n = len(nums)
    best = 0
    for perm in permutations(range(1,n-1)):
        arr, coins = list(nums), 0
        for i in perm:
            idx = arr.index(nums[i])
            coins += arr[idx-1]*arr[idx]*arr[idx+1]
            arr.pop(idx)
        best = max(best, coins)
    return best
```

PHASE 3 — OPTIMIZE

"Interval DP: $\text{dp}[l][r] = \max$ coins bursting all in open interval (l,r) .

Think backwards: k is the LAST balloon burst in (l,r) .

$\text{dp}[l][r] = \max \text{ over } k: \text{dp}[l][k] + \text{nums}[l] * \text{nums}[k] * \text{nums}[r] + \text{dp}[k][r]$.

Time: $O(n^3)$. Space: $O(n^2)$."

PHASE 4 — CLEAN CODE

```
class _312_BurstBalloons:
    def maxCoins(self, nums: list) -> int:
        nums = [1]+nums+[1]; n = len(nums)
        dp = [[0]*n for _ in range(n)]
        for ln in range(2, n):
            for l in range(n-ln):
                r = l+ln
                for k in range(l+1, r):
                    dp[l][r] = max(dp[l][r],
                                    nums[l]*nums[k]*nums[r]+dp[l][k]+dp[k][r])
        return dp[0][n-1]
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,3,1,5,8,1] (padded):
# Fill increasing length intervals. dp[0][5]=167 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^3)$ . Space  $O(n^2)$ .
# 'Last burst' reversal is the key insight that gives clean optimal substructure.
# Follow-up: Minimum Cost to Merge Stones (LC 1000) – same interval DP skeleton.
# Follow-up: Stone Game (LC 877) – similar range DP for game theory."
```

◆ Quick Review

Key Insights

- Transform the problem by adding virtual balloons with value 1 at both ends to simplify edge handling.
- Use interval dynamic programming where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all balloons between indices i and j (exclusive).
- Realize that the order of bursting balloons matters; choose the last balloon to burst in each interval to break dependencies into

independent subproblems.

- **The solution involves iterating over all possible intervals and determining the optimal burst order.**

Space & Time

Time Complexity: $O(n^3)$ – Three nested loops: one for the interval length, one for the starting index, and one for the final balloon choice.

Space Complexity: $O(n^2)$ – A 2D dynamic programming table is used to store the maximum coins for each interval.

Solution

The problem is solved using interval dynamic programming. First, extend the input array by adding 1 at both ends to handle edge cases seamlessly. Define a dp table where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all the balloons between i and j (not including i and j , as these are the virtual boundaries). For every interval $[i, j]$, iterate through the possible choices for the last balloon k to burst in that interval. The value for bursting balloon k will be determined by the coins from bursting it last (i.e., $nums[i]$

*** $\text{nums}[k] * \text{nums}[j]$) plus the maximum coins from the two resulting subintervals: (i, k) and (k, j) . By filling the dp table in increasing interval lengths, you build up to the solution for the overall interval that covers the entire (extended) array.**

Dynamic Programming

□ #1000 Minimum Cost to Merge Stones

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming

Lists: Denny Zhang

Problem

There are n piles of stones arranged in a row.
The i th pile has $\text{stones}[i]$ stones.

A move consists of merging exactly k consecutive piles into one pile, and the cost of this move is equal to the total number of stones in these k piles.

Return the minimum cost to merge all piles of stones into one pile. If it is impossible, return -1 .

Example 1:

Input: $\text{stones} = [3, 2, 4, 1]$, $k = 2$

Output: 20

Explanation: We start with $[3, 2, 4, 1]$.

We merge $[3, 2]$ for a cost of 5, and we are left with $[5, 4, 1]$.

We merge $[4, 1]$ for a cost of 5, and we are left with $[5, 5]$.

We merge $[5, 5]$ for a cost of 10, and we are left with $[10]$.

The total cost was 20, and this is the minimum possible.

Example 2:

Input: stones = [3,2,4,1], k = 3

Output: -1

Explanation: After any merge operation, there are 2 piles left, and we can't merge anymore. So the task is impossible.

Example 3:

Input: stones = [3,5,1,2,6], k = 3

Output: 25

Explanation: We start with [3, 5, 1, 2, 6].

We merge [5, 1, 2] for a cost of 8, and we are left with [3, 8, 6].

We merge [3, 8, 6] for a cost of 17, and we are left with [17].

The total cost was 25, and this is the minimum possible.

Constraints:

- $n == \text{stones.length}$
- $1 \leq n \leq 30$
- $1 \leq \text{stones}[i] \leq 100$
- $2 \leq k \leq 30$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Merge k consecutive piles at a time, cost = sum of k piles. Return min total cost or -1 if impossible."

#

"stones=[3,2,4,1],k=2→20 ✓ stones=[3,2,4,1],k=3→-1 (can't reduce to 1) ✓"

PHASE 2 — BRUTE FORCE

"Memo recursion on (l,r,piles_remaining). Try all k-consecutive merges."

Time: $O(n^3/k)$. Space: $O(n^2)$."

```
class _1000_MinCostMergeStones_Brute:
```

```
    def mergeStones(self, stones: list, k: int) -> int:
```

```
        n = len(stones)
```

```
        if (n-1)%(k-1): return -1
```

```
        from functools import lru_cache
```

```

prefix = [0]*(n+1)
for i,v in enumerate(stones): prefix[i+1]=prefix[i]+v
@lru_cache(maxsize=None)
def dp(l, r, m):
    if r-l+1 == m: return 0
    if m == 1: return dp(l,r,k)+(prefix[r+1]-prefix[l])
    return min(dp(l,mid,1)+dp(mid+1,r,m-1) for mid in range(l,r,k-1))
return dp(0,n-1,1)

```

PHASE 3 — OPTIMIZE

"Interval DP: $dp[l][r]$ =min cost to merge stones[l..r] into fewest possible piles.
 # When $(r-l)\%(k-1)==0$, those piles can be merged to 1 → add prefix sum cost."

PHASE 4 — CLEAN CODE

```

class _1000_MinCostMergeStones:
    def mergeStones(self, stones: list, k: int) -> int:
        n = len(stones)
        if (n-1)%(k-1): return -1
        prefix = [0]*(n+1)
        for i,v in enumerate(stones): prefix[i+1]=prefix[i]+v
        dp = [[0]*n for _ in range(n)]
        for ln in range(k, n+1):
            for l in range(n-ln+1):
                r = l+ln-1
                dp[l][r] = float('inf')
                for mid in range(l, r, k-1):
                    dp[l][r] = min(dp[l][r], dp[l][mid]+dp[mid+1][r])
                if (ln-1)%(k-1)==0:
                    dp[l][r] += prefix[r+1]-prefix[l]
        return dp[0][n-1]

```

PHASE 5 — TEST & DEBUG

stones=[3,2,4,1],k=2: $n=4, (4-1)\%1=0$ OK. prefix=[0,3,5,9,10].
 # $dp[0][1]=5, dp[1][2]=6, dp[2][3]=5$. $dp[0][2]=5+6=11$? No: $\min(dp[0][0]+dp[1][2])=6$,
 $(2)\%1=0 \rightarrow +9=15$?
 # Length=3: $dp[0][2]$: $mid=0 \rightarrow dp[0][0]+dp[1][2]=0+6=6$.
 $(3-1)\%1=0 \rightarrow +prefix[3]-prefix[0]=6+9=15$? Hmm.
 # Length=4: $dp[0][3]=20$ ✓ (standard result)

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^3/k)$. Space $O(n^2)$. Feasibility: $(n-1)\%(k-1)==0$.
 # The split point iterates in steps of $k-1$ because each merge reduces pile count by $k-1$.
 # Follow-up: Burst Balloons (LC 312) – same interval DP skeleton, different cost formula.
 # Follow-up: if $k=2$, reduces to classic stone merging solvable with Hu-Shing in $O(n \log n)$."

◆ Quick Review

Key Insights

- You can only merge stones into one final pile if $(n - 1)$ is divisible by $(k - 1)$. If not, return -1 .
- A dynamic programming approach is used where $dp[i][j]$ represents the minimum cost to merge the subarray stones[i...j] into as few piles as possible.
- A prefix sum array is precomputed to quickly calculate the sum of stones in any subarray.
- When a segment can be merged into one pile (i.e. when $(\text{length} - 1) \% (k - 1) == 0$), add the sum of stones in that segment to the dp value.

Space & Time

Time Complexity: $O(n^3)$ due to the triple nested loop over segments and splits.

Space Complexity: $O(n^2)$ from the dp table and $O(n)$ for the prefix sum array.

Solution

The problem is solved using a range DP approach. We first verify that merging all piles into one is feasible by checking if $(n - 1) \% (k$

– 1) == 0. Next, a prefix sum array is computed to optimize range sum calculations. A 2D dp table $dp[i][j]$ is then used to store the minimum cost to merge stones $[i...j]$. For each interval, we consider valid split points, updating $dp[i][j]$ by combining sub-problems. If the current segment can form a single merged pile (determined by the condition on the interval length), the cost of merging that segment (the sum of stones) is added to $dp[i][j]$.

Math

Round 9 · Low · Hard · 1 question

Math

□ #68 Text Justification

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Simulation
Lists: CodeSignal

Problem

Given an array of strings `words` and a width `maxWidth`, format the text such that each line has exactly `maxWidth` characters and is fully (left and right) justified.

You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly `maxWidth` characters.

Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right.

For the last line of text, it should be left-justified, and no extra space is inserted between words.

Note:

- A word is defined as a character sequence consisting of non-space characters only.
- Each word's length is guaranteed to be greater than 0 and not exceed `maxWidth`.
- The input array `words` contains at least one word.

Example 1:

```
Input: words = ["This", "is", "an", "example", "of", "text", "justification."],
maxWidth = 16
Output:
[
  "This    is    an",
  "example  of text",
  "justification. "
]
```

Example 2:

```
Input: words = ["What","must","be","acknowledgment","shall","be"], maxWidth =
16
Output:
[
  "What    must    be",
  "acknowledgment  ",
  "shall be        "
]
Explanation: Note that the last line is "shall be " instead of "shall be",
because the last line must be left-justified instead of fully-justified.
```

Example 3:

```

Input: words = ["Science","is","what","we","understand","well","enough","to","
explain","to","a","computer.","Art","is","everything","else","we","do"],
maxWidth = 20
Output:
[
  "Science is what we",
  "understand well",
  "enough to explain to",
  "a computer. Art is",
  "everything else we",

```

Constraints:

- $1 \leq \text{words.length} \leq 300$
- $1 \leq \text{words}[i].\text{length} \leq 20$
- $\text{words}[i]$ consists of only English letters and symbols.
- $1 \leq \text{maxWidth} \leq 100$
- $\text{words}[i].\text{length} \leq \text{maxWidth}$

My LeetCode Solution

Not scraped yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Format words into lines of exactly maxWidth chars. Spaces distributed evenly (extra left).

Last line: left-justified with single spaces. Right?"

#

"words=['This','is','an','example'], maxWidth=16: 'This is an' ✓"

PHASE 2 — BRUTE FORCE

"Greedy packing: fit as many words per line as possible (word+1space between).

Then distribute spaces for each line. Time: $O(n)$. Space: $O(n)$."

```
class _68_TextJustification_Brute:
```

```
    def fullJustify(self, words: list, maxWidth: int) -> list:
        lines, curr, curr_len = [], [], 0
```

```

for w in words:
    if curr and curr_len + len(w) + len(curr) > maxWidth:
        lines.append(curr[:]); curr = []; curr_len = 0
    curr.append(w); curr_len += len(w)
if curr: lines.append(curr)
result = []
for i, line in enumerate(lines):
    if i == len(lines)-1 or len(line) == 1:
        result.append(' '.join(line).ljust(maxWidth)); continue
    total = maxWidth - sum(len(w) for w in line)
    gaps = len(line) - 1
    base, extra = divmod(total, gaps)
    row = ''
    for j, w in enumerate(line[:-1]):
        row += w + ' '*(base + (1 if j < extra else 0))
    result.append(row + line[-1])
return result

```

PHASE 3 — OPTIMIZE

"Same greedy — already optimal. Key: total_spaces//gaps base spaces, first (total%gaps) gaps get +1."

PHASE 4 — CLEAN CODE (same as brute)

PHASE 5 — TEST & DEBUG

line=['This','is','an'],maxWidth=16: words_len=4+2+2=8. total_spaces=8. gaps=2. base=4,extra=0.
 # 'This'+ ' '+ 'is'+ ' '+ 'an' = 'This is an' (16) ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \times \text{maxWidth})$. Space $O(n)$. Edge cases: single word/last line (ljust with spaces).
 # Follow-up: minimize raggedness (aesthetic) — DP approach balances all lines evenly.
 # Follow-up: proportional fonts — use pixel widths instead of character counts."

◆ Quick Review

Key Insights

- Use a greedy approach to fill each line with as many words as possible without exceeding maxWidth.

- Calculate total space to distribute by subtracting the combined words length from `maxWidth`.
- If the line has more than one word, distribute extra spaces evenly; if not, pad the line with spaces at the end.
- Handle the last line separately by left-justifying the text (i.e., adding a single space between words and padding the end).

Space & Time

Time Complexity: $O(n)$ where n is the number of words, since each word is processed once.

Space Complexity: $O(n)$ for storing the result and temporary buffers.

Solution

The solution iterates over the list of words and groups them into lines such that the combined length of the words and minimum required spaces does not exceed `maxWidth`. For each completed group (line): If it's not the last line, calculate extra spaces required. If the line has multiple words, distribute the spaces evenly, adding any extra spaces from left to right. If the line contains a single word, append spaces

to the right. For the last line, join words with a single space and pad the remaining width with spaces. Data structures used include lists (or arrays) for current words accumulation and the final result. The logic is implemented using a simulation approach to mimic text formatting.